Geely Automotive Research Institute (Ningbo)Co.Ltd	Document Release Status **RELEASED**	
	Release Date **2026-07-06**	Version **(10)**
	Document No **x04000000276AE537**	Revision **009**
	Volume No **001**	Page No 1(1521)

Document Type	Reference No
Note-SWRS	**/**

Document Name

Base Technology SWRS GEEA3.0&EEA4.0

All

All

xxx

Issued by	Approved by
Lin Ying	**Tan Tingyue**

Base Tech Software Requirement Specification GEEA3.0&EEA4.0

Contents

0 REQUIREMENT LOG 42

0.1 List of requirements 42

0.2 Requirements change log 83

0.2.1 New requirements in this issue 83

0.2.2 Updated requirements in this issue 85

0.2.3 Modifications in the updated requirements in this issue 86

0.2.4 Deleted requirements in this issue 86

1 LIST OF REQUIREMENTS WITH DEVIATIONS 87

2 INTRODUCTION 87

2.1 What is the purpose of this document? 87

2.2 Who should read this document? 87

2.3 How is this document organized? 87

2.4 How do you receive more information? 87

3 REVISION HISTORY 87

4 ECU SOFTWARE REQUIREMENTS 87

4.1 ECU Platform 91

4.1.1 General requirement 91

4.1.1.1 Generic HW requirements 91

4.1.1.1.1 Overview 91

4.1.1.1.1.1 Definitions & Abbreviations 91

4.1.1.1.1.2 Requisite documents 92

4.1.1.1.2 Requirements 92

4.1.1.1.2.1 Memory capacity 92

4.1.1.1.2.2 Non Volatile Memory (NVM) 92

4.1.1.1.2.3 Input and Output lines 95

4.1.1.2 Hardware Platform 95

4.1.1.2.1 Requirements 95

4.1.1.2.2 Avoid low voltage shut-down 96

4.1.1.3 Diagnostics and ECU Platform 97

4.1.1.3.1 Introduction 97

4.1.1.3.1.1 Definitions & Abbreviations 98

4.1.1.3.1.2 Requisite documents 99

4.1.1.3.1.3 Selection list 99

4.1.1.4 ECU Addresses 100

4.1.1.4.1 Requirements 100

4.1.1.5 Software Download 102

4.1.1.5.1 Requirements 103

4.1.1.6 Startup Time 106

4.1.1.6.1 Requirements 106

4.1.1.7 Diagnostic secure communication for Edge Node 107

4.1.1.7.1 Requirements 107

4.1.1.8 Zone node system requirement 107

4.1.1.8.1 Requirements 107

4.1.1.9 Car Configuration Master system requirement 108

4.1.1.9.1 Requirements 108

4.1.2 Guideline for update bit mechanism 108

4.1.2.1 Guideline for update bit mechanism 108

4.1.2.1.1 General 108

4.1.2.1.2 Update bit mechanism 111
4.1.2.1.2.1 The process of sender 111
4.1.2.1.2.2 Related parameter for sender 112
4.1.2.1.2.3 The process of receiver 113
4.1.2.1.2.4 Deadline monitoring for the receiver 115

4.1.2.1.3 RTE upate flag mechanism 117
4.1.2.1.3.1 Relationship between update flag and update bit 118
4.1.2.1.3.2 RTE update flag use case 118

4.1.2.2 Guideline for AliveTimeout setting 119

4.1.2.2.1 The rule to define Alive Timeout 119

4.1.3 AutoSAR Configuration 119

4.1.3.1 Autosar Configuration 119

4.1.3.1.1 Documents 119

4.1.3.1.2 Definitions and Abbreviations 119

4.1.3.1.3 Requirements 120

4.1.3.1.4 AUTOSAR CONFIGURATION REQUIREMENTS 121

4.1.3.2 Autosar Time Synchronization 124

4.1.3.2.1 TIME SYNCHRONIZATION CONCEPT 124
4.1.3.2.1.1 Time Synchronization Introduction 124
4.1.3.2.1.2 Time Synchronization Requirements 124
4.1.3.2.1.3 StbM-Synchronized Time Base Manager 125
4.1.3.2.1.4 CAN&CANFD Time Sync Requirements 127
4.1.3.2.1.5 EthTSyn-Time Synchronization over Ethernet 130

4.1.4 E2E 134

4.1.4.1 E2E Overview 134

4.1.4.1.1 E2E Overview 134

4.1.4.2 E2E Profile G 135

4.1.4.2.1 E2E PROTECTION CONCEPT 135
4.1.4.2.1.1 E2E Protection Introduction 135
4.1.4.2.1.2 E2E Protection Requirements 135
4.1.4.2.1.2.1 Profile G Requirements 136
4.1.4.2.1.2.1.1 Appendix Example 139
4.1.4.2.1.3 Profiles vs Transformers and Wrappers 140

4.1.4.3 E2E Profile4 and Profile11 140

4.1.4.3.1 E2E Profile4 and Profile11 140
4.1.4.3.1.1 Introduction 140
4.1.4.3.1.2 Requirements 140

4.1.4.4 E2E Protection for Service 147

4.1.4.4.1 E2E Protection for Service 147
4.1.4.4.1.1 Introduction 147
4.1.4.4.1.2 Requirements 147
4.1.4.4.1.2.1 Appendix Example for CRC Calculation 148
4.1.4.4.1.2.1.1 E2E Profile4 for service CRC calculation 148
4.1.4.4.1.2.1.2 E2E Profile11 for service CRC calculation 150

4.2 Car Configuration 152

4.2.1 Car Configuration 152

4.2.1.1 Car Configuration Slave 152

4.2.1.1.1 Requisite documents 152

4.2.1.1.2 Definitions and Abbreviations 152

4.2.1.1.3 Central Car Configuration Requirements 153
4.2.1.1.3.1 Car Configuration Parameter Data 153
4.2.1.1.3.1.1 Parameter Creation 153
4.2.1.1.3.1.2 Parameter Properties 154
4.2.1.1.3.1.3 Information in CCDB 154
4.2.1.1.3.1.4 Parameter Subscription 154
4.2.1.1.3.2 System Design 155
4.2.1.1.3.3 Car Configuration Subscriber Requirements 155
4.2.1.1.3.4 Signals provided by the system 164

4.3 Network Management 165

4.3.1 Network Management 165

4.3.1.1 Network Management - Generic reqs 165

4.3.1.1.1 Requirements 165

4.3.1.2 AutoSar NM - Generic reqs 168

4.3.1.2.1 Overview 168
4.3.1.2.1.1 Definitions and Abbreviations 169
4.3.1.2.1.2 Requisite documents 170

4.3.1.2.2 Introduction 170

4.3.1.2.3 General requirement 172

4.3.1.3 AutoSar NM - CAN reqs 173

4.3.1.3.1 Timing requirements 173
4.3.1.3.1.1 Microcontroller is powered in sleep 176
4.3.1.3.1.2 Microcontroller is unpowered in sleep 176

4.3.1.4 AutoSar NM - CAN cross networks reqs 177

4.3.1.4.1 Requirements 177
4.3.1.4.1.1 Node supports partial network clustering (PNC) 178

4.3.1.5 Ethernet TC10 180

4.3.1.5.1 Introduction 180
4.3.1.5.1.1 PHY Power Mode Definition 180

4.3.1.5.1.2 TC10 Wake-Up and Sleep Configuration 183
4.3.1.5.1.3 Cases Of TC10 Wake-up and Sleep 184

4.3.1.5.2 OpenAlliance - TC10 186

4.3.1.6 UDP Network Management 187

4.3.1.6.1 Requirements 187

4.4 Communication_LIN 189

4.4.1 Communication_LIN_Requirement_Specification 189

4.4.1.1 General 189

4.4.1.1.1 Definitions and Abbreviations 189

4.4.1.1.2 Requisite documents 189

4.4.1.1.3 Requirements 190

4.4.1.2 LIN Physical Layer -Generic requirements 191

4.4.1.2.1 LIN Physical Layer - Generic requirements 191
4.4.1.2.1.1 Operating Battery Power Voltage Range 192
4.4.1.2.1.2 Oscillator Tolerance Definition 193
4.4.1.2.1.3 General Design Requirements 193

4.4.1.3 LIN Physical Layer - HW Master requirements 194

4.4.1.3.1 Interface Implementation and Physical Layer Circuit 194

4.4.1.4 LIN Physical Layer - HW Slave requirements 195

4.4.1.4.1 Interface Implementation and Physical Layer Circuit 195

4.4.1.5 LIN Data Link Layer - Generic Requirements 196

4.4.1.5.1 LIN Data Link Layer - Generic Requirements 196
4.4.1.5.1.1 Wake-Up timing 199
4.4.1.5.1.2 Fault Tolerance Modes 200
4.4.1.5.1.3 Operating Battery Power Voltage Range 201

4.4.1.6 LIN Data Link Layer - Master Requirements 203

4.4.1.6.1 General LIN Master Requirements 203

4.4.1.6.2 Master Wake-Up Timing 203

4.4.1.7 LIN Data Link Layer - Slave Requirements 204
4.4.1.7.1 General LIN Slave Requirements 205
4.4.1.7.2 Slave Wake-Up Timing 205
4.4.1.7.3 LIN slave with E2E 206
4.5 Communication_CAN/CANFD 207
4.5.1 Communication_CAN&CANFD_Requirement_Specification 207
4.5.1.1 General 207
4.5.1.1.1 Definitions and Abbreviations 207
4.5.1.1.2 Requisite Documents 208
4.5.1.1.3 Requirements 208
4.5.1.2 CAN and CANFD Physical Layer 211
4.5.1.2.1 Requirements 211
4.5.1.2.1.1 CAN&CANFD controller 212
4.5.1.2.1.2 Bit Rate Tolerance Definition 212
4.5.1.2.1.3 Termination 213
4.5.1.2.1.4 Power Supply and Reset 213
4.5.1.2.1.5 Transceivers 214
4.5.1.2.1.6 Chokes 214
4.5.1.2.1.7 ESD Protection Components 214
4.5.1.2.1.8 ECU Delay 215
4.5.1.2.1.9 Interface implementation, physical layer and layout requirements215
4.5.1.2.1.9.1 General design requirements 215
4.5.1.2.1.10 Terminal ECUs on CAN&CANFD 218
4.5.1.2.2 Interface requirements 218
4.5.1.3 CAN and CANFD Data Link Layer 220
4.5.1.3.1 Requirements 220
4.5.1.3.1.1 ECU Level 221
4.5.1.3.1.1.1 Resynchronisation and bit sampling 223
4.5.1.3.1.1.2 Speed tolerance definition 224
4.5.1.3.1.1.3 Network Transmission Speed - 500kbps 224
4.5.1.3.1.1.4 CAN FD bit register settings for 2 Mbps 225
4.5.1.3.1.1.5 CAN FD bit register settings for 5 Mbps 226
4.5.1.3.1.2 Periodic Message Timing Parameter 226
4.6 Communication_Ethernet 227
4.6.1 Communication_Ethernet_Requirement_Specification 227

4.6.1.1 General 227
4.6.1.1.1 Definitions & Abbreviations 227
4.6.1.1.2 Requisite Documents 232
4.6.1.1.3 Requirements 233
4.6.1.2 Ethernet Physical Layer - 100BASE-TX 235
4.6.1.2.1 Introduction 235
4.6.1.2.2 Ethernet Physical Layer - 100BASE-TX - Generic requirements 235
4.6.1.2.3 Ethernet Physical Layer - 100BASE-TX - Edge Node requirements 254
4.6.1.3 Ethernet Physical Layer - 100BASE-T1 256
4.6.1.3.1 Introduction 256
4.6.1.3.2 Ethernet Physical Layer - 100BASE-T1 - Generic requirements 256
4.6.1.3.3 Ethernet Physical Layer - 100BASE-T1 - ECU requirements 259
4.6.1.4 Ethernet Physical Layer - 1000BASE-T1 266
4.6.1.4.1 Introduction 266
4.6.1.4.2 Ethernet Physical Layer - 1000BASE-T1 - Generic requirements 266
4.6.1.4.3 Ethernet Physical Layer - 1000BASE-T1 - ECU requirements 268
4.6.1.5 Ethernet Data Link Layer 275
4.6.1.5.1 Introduction 275
4.6.1.5.2 Generic Requirements 276
4.6.1.5.3 Switch Requirements 279
4.6.1.6 Internetworks General Specification 286
4.6.1.6.1 Introduction 286
4.6.1.6.1.1 External publications 287

4.6.1.6.2 Requirements 288

4.6.1.7 SOME IP 296

4.6.1.7.1 Introduction 296

4.6.1.7.2 Requirements 297
4.6.1.7.2.1 Configuration 297
4.6.1.7.2.2 SOME IP implementation 298
4.6.1.7.2.2.1 Serialization 303
4.6.1.7.2.3 SOME IP over TCP 309
4.6.1.7.2.4 SOME IP over UDP 311
4.6.1.7.2.5 Error Handling 312
4.6.1.7.2.6 Service Discovery 315
4.6.1.7.2.7 SOME IP TP 324
4.6.1.7.2.7.1 Sender specific behavior 326
4.6.1.7.2.7.2 Receiver specific behavior 327

4.6.1.8 RTP 328

4.6.1.8.1 Requirements 329
4.6.1.8.1.1 General RTP parameters 329
4.6.1.8.1.2 Media formats 331
4.6.1.8.1.3 Underlying layers 331

4.6.1.9 VLAN Configuration 332

4.6.1.9.1 Requirements 332
4.6.1.9.1.1 VLAN parameters 332

4.6.1.10 IP Routing and Firewalling 334

4.6.1.10.1 Introduction 334
4.6.1.10.1.1 Abbreviations & Definitions 335
4.6.1.10.1.2 External publications 337
4.6.1.10.1.3 Requisite documents 338

4.6.1.10.2 Requirements 338

4.6.1.11 Time Sensitive Network 347

4.6.1.11.1 Introduction 347
4.6.1.11.1.1 Definitions & Abbreviations 347
4.6.1.11.1.2 References 347

4.6.1.11.2 Requirements 348
4.6.1.11.2.1 802.1AS / gPTP 348
4.6.1.11.2.1.1 Introduction 348
4.6.1.11.2.1.2 General Requirement 348
4.6.1.11.2.1.3 Sync intervals 352

4.6.1.11.2.1.3.1 AVnu 802.1AS Recovered Clock Quality Testing 353
4.6.1.11.2.1.4 Pdelay_Req intervals 354
4.6.1.11.2.1.5 Storing measured gPTP values for next startup 355
4.6.1.11.2.1.6 Pdelay Requests from AED-B master ports 356
4.6.1.11.2.1.7 Time to wait for initial Sync message in AED-B 357
4.6.1.11.2.2 IEEE1722 / AVTP 357
4.6.1.11.2.2.1 AAF parameters 357
4.6.1.11.2.2.1.1 Supported audio sample rate 357
4.6.1.11.2.2.1.2 AED-A Talker Timestamp mode 358
4.6.1.11.2.2.1.3 Dynamic Channel Layout EventField 358
4.6.1.11.2.2.1.4 Number of channels 359
4.6.1.11.2.2.2 Video format 359
4.6.1.11.2.2.3 Clock Reference Format (CRF) 359
4.6.1.11.2.3 Traffic Class parametrization 360
4.6.1.11.2.3.1 SR classes 360
4.6.1.11.2.3.2 SR Class Template(s) 360
4.6.1.11.2.3.3 Mapping between PCP and SR Class 361
4.6.1.11.2.3.4 Maximum Transit Time (MTT) 361
4.6.1.11.2.4 Exception handling 362
4.6.1.11.2.4.1 Behaviour upon detection of lost or late data 362
4.6.1.11.2.4.1.1 Lost or late video data 362
4.6.1.11.2.4.1.2 Lost or late audio data 362
4.6.1.11.2.5 802.1CB 363
4.6.1.11.2.5.1 Introduction 363
4.6.1.11.2.5.2 Requirements 363
4.6.1.11.2.6 802.1Qav / Credit-Based Shaper(CBS) 365
4.6.1.11.2.6.1 Introduction 365
4.6.1.11.2.6.2 General Requirements 365
4.6.1.11.2.6.2.1 MAC Requirements 365
4.6.1.11.2.6.2.2 Bridge Requirements 366
4.6.1.11.2.6.2.3 Talker Requirements 367
4.6.1.11.2.6.2.4 Listener Requirements 368
4.6.1.11.2.6.3 Mapping Requirements 368
4.6.1.11.2.6.4 Traffic Shaping Implementation Requirements 369
4.6.1.11.2.7 802.1Qbv / Time-Aware Shaper(TAS) 371
4.6.1.11.2.7.1 Introduction 371
4.6.1.11.2.7.2 General Requirements 372
4.6.1.11.2.7.2.1 MAC Requirements 372
4.6.1.11.2.7.2.2 Bridge Requirements 373
4.6.1.11.2.7.2.3 Talker Requirements 374
4.6.1.11.2.7.2.4 Listener Requirements 375
4.6.1.11.2.7.3 Traffic Shaping Implementation Requirements 375

4.6.1.12 IP Link Manager requirement 380

4.6.1.12.1 Introduction 380
4.6.1.12.1.1 Abbreviations & Definitions 380
4.6.1.12.1.2 Requisite Documents 381

4.6.1.12.2 Requirements 381

4.6.1.13 SignalToUDP 395

4.6.1.13.1 Genral Requirement 395
4.6.1.13.1.1 Definitions and Abbreviations 395
4.6.1.13.1.2 Overview 395

4.6.1.13.1.3 Requisite documents 395

4.6.1.13.2 SignalToUDP Topology 395
4.6.1.13.2.1 Topology 395
4.6.1.13.2.2 Node Type 396

4.6.1.13.3 UDP message Layout 396
4.6.1.13.3.1 nPduUdpTxBuffer 396
4.6.1.13.3.2 I-PDU Header 397
4.6.1.13.3.3 I-PDU ID 397
4.6.1.13.3.4 UDP Unused bit rules 398

4.6.1.13.4 Routing Node Requirements 398
4.6.1.13.4.1 CAN/LIN transfer UDP requirements 398
4.6.1.13.4.1.1 UDP Message triggering mechanism 399
4.6.1.13.4.2 UDP Transfer CAN /LIN requirement 400

4.6.1.13.5 Endpoint Node Requirements 401
4.6.1.13.5.1 Endpoint Node Requirements 401

4.7 Communication_WLAN 403

4.7.1 Communication_Wlan_Requirement_Specification 403

4.7.1.1 WLAN General 403

4.7.1.1.1 Requirements 403

4.8 Diagnostics Transport & Network Layer 405

4.8.1 Transport & Network Layer 405

4.8.1.1 DoCAN 405

4.8.1.1.1 Introduction 405

4.8.1.1.2 Requisite documents 406

4.8.1.1.3 Definitions and Abbreviations 406

4.8.1.1.4 Transport layer protocol 407
4.8.1.1.4.1 Protocol control information specification 408
4.8.1.1.4.1.1 BlockSize (BS) parameter definition 408
4.8.1.1.4.1.2 Separation (STmin) parameter definition 410
4.8.1.1.4.1.3 Separation time between single frames 412
4.8.1.1.4.2 Maximum number of FC. Wait frame transmission (N_WFTmax)412
4.8.1.1.4.3 Network layer timing 413
4.8.1.1.4.3.1 Timing parameters 413
4.8.1.1.4.3.2 Unexpected arrival of N_PDU 420

4.8.1.1.5 Data link layer usage 421
4.8.1.1.5.1 Adressing formats 421

4.8.1.1.5.2 CAN frame data padding 421
4.8.1.1.5.3 Use data frame by frame 423
4.8.1.1.5.3.1 Example A 425
4.8.1.1.5.3.2 Example B 426

4.8.1.2 DoLIN 426

4.8.1.2.1 Introduction 426

4.8.1.2.2 Requisite documents 427

4.8.1.2.3 Definitions and Abbreviations 427

4.8.1.2.4 Transport Layer 427
4.8.1.2.4.1 LIN Master requirements 428
4.8.1.2.4.2 LIN Slave requirements 428

4.8.1.3 DoIP - Generic Info 430

4.8.1.3.1 Introduction 430

4.8.1.3.2 Requisite Documents 431

4.8.1.3.3 Definitions & Abbreviations 431

4.8.1.4 DoIP - Edge Node 433

4.8.1.4.1 Edge Node Requirements 433

4.8.1.5 DoIP - Client 444

4.8.1.5.1 Client Requirements 444

4.8.1.6 DoIP - InVehicle requirements 444

4.8.1.6.1 IP-ECU Requirements 444

4.8.1.6.2 IP-ECU Fast SWDL General Requirement 446

4.8.1.6.3 IP-ECU Fast SWDL Requirement for WiFi Diagnostics 447

4.8.1.7 DoIP - Edge Node - Transport Layer Secutiry 450

4.8.1.7.1 Diagnostic over Network 450

4.8.1.7.2 TLS secure diagnostic communication for DoIP 452
4.8.1.7.2.1 TLS for DoIP client side 454

4.8.1.7.2.2 TLS for DoIP server side 455

4.8.1.7.3 TLS DoIP configuration profile 456
4.8.1.7.3.1 Supported TLS versions for DoIP 456
4.8.1.7.3.2 Supported TLS cipher suites 456
4.8.1.7.3.3 Supported TLS extensions 457
4.8.1.7.3.4 Compression Method 458

4.8.1.7.4 Additional Requirements 459

4.9 Diagnostics Gateway 459

4.9.1 Diagnostic and Bootloader Gateway Specification 459

4.9.1.1 Gateway Generic 459

4.9.1.1.1 Overview 459
4.9.1.1.1.1 Definitions and Abbreviations 460
4.9.1.1.1.2 Requisite documents 463
4.9.1.1.1.2.1 Documents 463
4.9.1.1.1.2.2 Relation to other documents 464

4.9.1.1.2 Constraints and Assumptions 465

4.9.1.1.3 Gateway description for DIAG and SWDL 465

4.9.1.1.4 General In-vehicle gateway requirements 467

4.9.1.1.5 General allocation of buffers requirements 472
4.9.1.1.5.1 General allocation of buffers requirements, part 1 473
4.9.1.1.5.2 General allocation of buffers requirements, part 2 475

4.9.1.1.6 Parallel physical requests to different ECUs 475

4.9.1.1.7 Functional requests to all ECUs 477

4.9.1.1.8 Gateway sequence diagrams 480
4.9.1.1.8.1 Functional requests to all ECUs 480
4.9.1.1.8.2 Parallel physical requests to different ECUs 482

4.9.1.2 Gateway Generic DIAG 483

4.9.1.2.1 Routing table 483

4.9.1.3 Gateway Generic SWDL 484

4.9.1.3.1 Routing table 484

4.9.1.3.2 Queued physical requests to one ECU 485

4.9.1.3.3 Gateway sequence diagrams 487
4.9.1.3.3.1 Queued physical requests to same ECU 487

4.9.1.4 Gateway connected to External IP 488

4.9.1.4.1 IP diagnostic format on External IP 488
4.9.1.4.1.1 Diagnostic request on external IP network 489
4.9.1.4.1.2 Diagnostic response on external IP network 490

4.9.1.5 Gateway connected to Internal IP 491

4.9.1.5.1 IP diagnostic format on Internal IP 491
4.9.1.5.1.1 Diagnostic request on internal IP network 491
4.9.1.5.1.2 Diagnostic response on internal IP network 492

4.9.1.6 Gateway connected to CAN 493

4.9.1.6.1 CAN diagnostic frame format 493
4.9.1.6.1.1 Diagnostic Request frame on CAN network 493
4.9.1.6.1.2 Diagnostic Response frame on CAN network 494

4.9.1.7 Gateway connected to X-LIN 495

4.9.1.7.1 Conversion from TA to NAD to SA 495

4.9.1.7.2 Functional request 496

4.9.1.7.3 LIN master time out 497

4.9.1.7.4 Parallel requests 498

4.9.1.8 Vehicle Gateway - IP-IP 499

4.9.1.8.1 Requirements 499
4.9.1.8.1.1 Translation of diagnostic messages 499
4.9.1.8.1.1.1 Diagnostic request from external IP network to internal IP network 499
4.9.1.8.1.1.2 Diagnostic response from internal IP network to external IP network 500
4.9.1.8.1.1.3 Functional request External IP to Internal IP 501

4.9.1.9 Vehicle Gateway - IP-CAN(Network) 503

4.9.1.9.1 Translation of diagnostic messages 503
4.9.1.9.1.1 Diagnostic request from external IP network to CAN network . 503
4.9.1.9.1.2 Diagnostic response from CAN network to external IP network 504

4.9.1.9.2 Gateway allocation of communication channels 506

4.9.1.9.2.1 Network (CAN(FD)) - Client side 506

4.9.1.9.3 Gateway latency requirements 508
4.9.1.9.3.1 IP - CAN diagnostic messages 508
4.9.1.9.3.1.1 Diagnostic requests IP to CAN 509
4.9.1.9.3.1.2 Diagnostic responses CAN to IP 510

4.9.1.10 Vehicle Gateway DIAG 511

4.9.1.10.1 Gateway allocation of buffers for DIAG 511

4.9.1.11 Vehicle Gateway SWDL 515

4.9.1.11.1 Gateway allocation of buffers for SWDL 515

4.9.1.12 Vehicle Gateway J2534 521

4.9.1.12.1 J2534 CAN diagnostic frame format 521
4.9.1.12.1.1 Diagnostic Request frame on J2534 CAN network 521
4.9.1.12.1.2 Diagnostic Response frame on J2534 CAN network 521

4.9.1.12.2 J2534 Latency and Message based requirements 522

4.9.1.12.3 J2534 standard tester connected to Vehicle GW through CAN 524

4.9.1.13 InVehicle DoIP Gateway - IP - CAN 526

4.9.1.13.1 Translation of diagnostic messages 526
4.9.1.13.1.1 Diagnostic request from internal IP network to CAN network .. 526
4.9.1.13.1.2 Diagnostic response from CAN network to internal IP network 527

4.9.1.13.2 Gateway allocation of communication channels 529
4.9.1.13.2.1 Network (CAN) - Client side 529

4.9.1.13.3 Gateway latency requirements 531
4.9.1.13.3.1 IP - CAN diagnostic messages 531
4.9.1.13.3.1.1 Diagnostic requests IP to CAN 531
4.9.1.13.3.1.2 Diagnostic responses CAN to IP 533

4.9.1.14 InVehicle DoIP GW DIAG 534

4.9.1.14.1 Gateway allocation of buffers for DIAG 534

4.9.1.15 InVehicle DoIP GW SWDL 538

4.9.1.15.1 Gateway allocation of buffers for SWDL 538

4.9.1.16 Zone Network Gateway - CAN-CAN 543

4.9.1.16.1 Gateway allocation of communication channels 543
4.9.1.16.1.1 Backbone network - Server side 543
4.9.1.16.1.2 Zone Network - Client side 545

4.9.1.16.2 Gateway latency requirements 547
4.9.1.16.2.1 Diagnostic requests CAN to CAN 547
4.9.1.16.2.2 Diagnostic responses CAN to CAN 549
4.9.1.16.2.2.1 Diagnostic Response CAN to CANFD, when CAN L-SDU < CANFD L-SDU 549
4.9.1.16.2.2.2 Diagnostic Response CAN to CANFD, when CAN L-SDU = CANFD L-SDU 551
4.9.1.16.2.2.3 Periodic responses CAN to CANFD 552

4.9.1.17 Zone Network Gateway DIAG 553

4.9.1.17.1 Gateway allocation of buffers for DIAG 553

4.9.1.18 Zone Network Gateway SWDL 557

4.9.1.18.1 Gateway allocation of buffers for SWDL 557

4.9.1.19 Firewall CANX-CAN11 Generic 563

4.9.1.19.1 Buffer requirements 563

4.9.1.19.2 Client side filter 563

4.9.1.19.3 Timing and flow control parameters 564

4.9.1.19.4 Transport layer and latency 565

4.9.1.20 Firewall CAN11-CAN11 566

4.9.1.20.1 Possibilty to disable Firewall functionality 566

4.9.1.20.2 Service side filter 566

4.9.1.21 Gateway X-LIN DIAG 566

4.9.1.21.1 Gateway latency requirements 566

4.9.1.21.2 Gateway allocation of buffers for DIAG 567

4.9.1.22 Gateway X-LIN SWDL 568

4.9.1.22.1 Diagnostic schedule execution 568

4.9.1.22.2 Gateway allocation of buffers for SWDL 568

4.9.1.23 Domain Network Gateway - IP-IP/IP Routing 569

4.9.1.23.1 Requirements 569

4.9.1.23.2 Gateway allocation of buffers for DIAG 569

4.9.1.23.3 Gateway allocation of buffers for SWDL 570

4.9.1.24 InVehicle Network Gateway - IP-IP/DoIP GW 570

4.9.1.24.1 Translation of diagnostic messages 570
4.9.1.24.1.1 Diagnostic request from Internal IP network to Sub IP network 570
4.9.1.24.1.2 Diagnostic response from internal IP network to InVehicle IP network 571

4.9.1.25 Domain Network Gateway DIAG 572

4.9.1.25.1 Gateway allocation of buffers for DIAG 572

4.9.1.26 Domain Network Gateway SWDL 577

4.9.1.26.1 Gateway allocation of buffers for SWDL 577

4.9.1.27 Gateway Generic supporting periodic responses 584

4.9.1.27.1 Logical address format 584
4.9.1.27.1.1 Logical address format for periodic response 584
4.9.1.27.1.2 Example of adding time stamp in periodic response 584

4.9.1.28 Gateway connected to External IP supporting periodic responses 586

4.9.1.28.1 IP diagnostic format on External IP 586
4.9.1.28.1.1 Periodic response (SID 2A) on IP based network 586

4.9.1.29 Gateway connected to Internal IP supporting periodic responses 587

4.9.1.29.1 IP diagnostic format on Internal IP 587
4.9.1.29.1.1 Periodic response (SID 2A) on IP based network 587

4.9.1.30 Gateway connected to CAN supporting periodic responses 588

4.9.1.30.1 CAN diagnostic frame format 588
4.9.1.30.1.1 Periodic response (SID 2A) on CAN Network 588

4.9.1.31 Vehicle Gateway DIAG - IP-IP supporting periodic responses 589

4.9.1.31.1 Translation of diagnostic messages 589
4.9.1.31.1.1 Diagnostic periodic response from internal IP Network to external IP Network 589

4.9.1.32 Sub Network Gateway DIAG - CAN-CAN supporting perdiodic responses590

4.9.1.32.1 Gateway latency requirements 590
4.9.1.32.1.1 Periodic diagnostic responses CAN to CAN 590

4.10 Diagnostics 591

4.10.1 UDS Session 591

4.10.1.1 UDS Session generic reqs 591

4.10.1.1.1 Overview 591

4.10.1.1.2 Requisite documents 594

4.10.1.1.3 Definitions and Abbrevations 594

4.10.1.1.4 Message timing definition 594

4.10.1.2 UDS Session ECU reqs 595

4.10.1.2.1 Timing parameters 595
4.10.1.2.1.1 Message timing definition 595
4.10.1.2.1.1.1 P2Server 595
4.10.1.2.1.1.2 P2(star)Server 595

4.10.1.2.2 Timing handling during communication 596
4.10.1.2.2.1 Queued request messages to one server (Zone ECUs Support) . 596
4.10.1.2.2.2 P4 definition 598

4.10.1.2.3 Appendix 599
4.10.1.2.3.1 Example, queuing of requests in a server 599
4.10.1.2.3.2 Example, queuing of requests versus full duplex 601

4.10.1.3 UDS Session client reqs 603

4.10.1.3.1 Timing parameters 603
4.10.1.3.1.1 ISO 13400 DoIP message timing definition 603
4.10.1.3.1.1.1 (delta)P2 605
4.10.1.3.1.1.2 (delta)P2External_Response 606
4.10.1.3.1.1.3 (delta)P2Vehicle 606
4.10.1.3.1.1.4 P6Client 610
4.10.1.3.1.1.5 P6(star)Client 610
4.10.1.3.1.1.6 P3Client_Phys 611
4.10.1.3.1.1.7 P3Client_Func 612
4.10.1.3.1.1.8 P4 definition 612

4.10.1.3.2 Timing handling during communication 612
4.10.1.3.2.1 Queued request messages to one server 613

4.10.1.3.3 Appendix 617
4.10.1.3.3.1 Example, message length dependent parameter P6Client 617
4.10.1.3.3.1.1 Single frame request and single frame response, one server 617
4.10.1.3.3.1.2 Multi frame request and single frame response, one server 617
4.10.1.3.3.1.3 Single frame request and multi frame response, one server 619
4.10.1.3.3.1.4 Single frame request and multi frame response, multiple servers 620

4.10.2 UDS Services 620

4.10.2.1 UDS Services 620

4.10.2.1.1 Overview 620

4.10.2.1.2 Requisite Documents 620

4.10.2.1.3 Definitions and Abbreviations 621

4.10.2.1.4 Diagnostic message addressing 623
4.10.2.1.4.1 Logical address format for diagnostic request and response 624
4.10.2.1.4.2 Physical logical address range 624
4.10.2.1.4.3 Functional logical address range 626
4.10.2.1.4.4 Logical address range for periodic response messages 628
4.10.2.1.4.5 Logical address range for external diagnostic tester 628
4.10.2.1.4.6 ECU ability to receive requests 629

4.10.2.1.5 Diagnostic service requirements 631
4.10.2.1.5.1 Diagnostic services 634
4.10.2.1.5.1.1 Diagnostic and Communication Unit 635
4.10.2.1.5.1.2 Data Transmission Functional Unit 643
4.10.2.1.5.1.3 Stored Data Transmission Functional Unit 649
4.10.2.1.5.1.4 InputOutput Control Functional Unit 651
4.10.2.1.5.1.5 Routine Functional Unit 653
4.10.2.1.5.1.6 Upload download functional unit 655
4.10.2.1.5.1.7 Additional routine requirements 660
4.10.2.1.5.1.8 Additional session requirements 664
4.10.2.1.5.1.8.1 General 664
4.10.2.1.5.1.8.2 Response message data parameter sessionParameterRecord665
4.10.2.1.5.1.9 Additional ClearDiagnosticInformation requirements 666

4.10.2.1.6 Appendix 667
4.10.2.1.6.1 Appendix A- Suggestion code for transferResponseParameterRecord calculation 667
4.10.2.1.6.1.1 CRC16-CITT C-code example 1 (fast) 667
4.10.2.1.6.1.2 CRC16-CITT C-code example 2 (slow) 669
4.10.2.1.6.1.3 CRC32 C-code example 670

4.10.3 UDS Data 672

4.10.3.1 UDS Data - General Requirements 672
4.10.3.1.1 Overview 672
4.10.3.1.2 Requisite documents 672
4.10.3.1.3 Definitions and Abbreviations 673
4.10.3.1.4 DID General Requirement 674
4.10.3.1.5 RID General Requirement 680
4.10.3.1.6 DTC General Requirement 681
4.10.3.1.6.1 DTC Information 681
4.10.3.1.6.1.1 General 681
4.10.3.1.6.1.1.1 Calibration of DTCs 684
4.10.3.1.6.1.2 DTC status bits 685
4.10.3.1.6.1.2.1 Requirements from section DTC status bits 686
4.10.3.1.6.1.3 DTC snapshot data 689
4.10.3.1.6.1.3.1 DTC snapshot data identification 689
4.10.3.1.6.1.3.1.1 Requirements from section DTC snapshot data identification691
4.10.3.1.6.1.3.2 DTC snapshot sampling criteria 691
4.10.3.1.6.1.3.2.1 Requirements from section DTC snapshot sampling criteria 693
4.10.3.1.6.1.3.3 Global DTC snapshot data 695
4.10.3.1.6.1.3.3.1 Requirements from section Global DTC snapshot data 697
4.10.3.1.6.1.3.4 Local DTC snapshot data 701
4.10.3.1.6.1.3.4.1 Requirements from section Local DTC snapshot data 701
4.10.3.1.6.1.4 DTC extended data 702
4.10.3.1.6.1.4.1 DTC operation cycle counters 702
4.10.3.1.6.1.4.1.1 Requirements from section DTC operation cycle counters .. 707
4.10.3.1.6.1.4.2 DTC fault detection counters 712
4.10.3.1.6.1.4.2.1 Requirements from section DTC fault detection counters 713
4.10.3.1.6.1.4.3 DTC time stamp 715
4.10.3.1.6.1.4.3.1 Requirements from section DTC time stamp 716
4.10.3.1.6.1.4.4 DTC status indicators 718
4.10.3.1.6.1.4.4.1 Requirements from section DTC status indicators 721
4.10.3.1.6.1.5 Store DTC information 725
4.10.3.1.6.1.5.1 Requirements from section Store DTC information 727
4.10.3.1.6.1.5.2 Transfer of DTC information to long term memory 731
4.10.3.1.6.1.5.2.1 Requirements from section Transfer of DTC information to long term memory 731
4.10.3.1.6.1.6 Clear DTC information 732
4.10.3.1.6.1.6.1 Requirements from section Clear DTC information 734
4.10.3.1.6.2 Requirement Content 736
4.10.3.1.6.2.1 Requirements from section Record Data - General 736
4.10.3.1.6.2.2 Requirements from section DTC information - General 736
4.10.3.1.6.2.3 Requirements from section Calibration of DTCs 740
4.10.3.1.6.2.4 Requirements from section Detection of faults 743
4.10.3.1.6.2.5 Requirements for detection of faults 746
4.10.3.1.6.2.5.1 Detection of faults in external electrical circuits 746
4.10.3.1.6.2.5.1.1 Detection of faults for switch inputs 749
4.10.3.1.6.2.5.1.2 Detection of short circuit and open circuit faults in analog inputs 750

4.10.3.1.6.2.5.1.3 Detection of short circuit and open circuit faults in FM, PCM, PWM etc. ... 752
4.10.3.1.6.2.5.1.4 Detection of short circuit and open circuit faults in low side driver ou... 754
4.10.3.1.6.2.5.1.5 Detection of short circuit and open circuit faults in high side driver o... 757
4.10.3.1.6.2.5.1.6 Detection of other external electrical circuit faults 759
4.10.3.1.6.2.5.2 Detection of faults in internal electrical circuits 760
4.10.3.1.6.2.5.2.1 Detection of internal faults not on the main circuit board ... 762
4.10.3.1.6.2.5.2.2 Detection of data integrity fault in non volatile memory 763
4.10.3.1.6.2.5.2.3 Detection of data integrity fault in volatile memory 764
4.10.3.1.6.2.5.2.4 Detection of internal device communication error 765
4.10.3.1.6.2.5.2.5 Detection of programme execution fault 766
4.10.3.1.6.2.5.2.6 Detection of internally connected actuator fault 767
4.10.3.1.6.2.5.2.7 Detection of internally connected sensor fault 768
4.10.3.1.6.2.5.2.8 Detection of faults in other internal parts 769
4.10.3.1.6.2.5.3 Detection of faults inferred from system behaviour 770
4.10.3.1.6.2.5.3.1 Detection of failure to react after event 771
4.10.3.1.6.2.5.3.2 Detection of Comparison of system behaviour with predictive model 773
4.10.3.1.6.2.5.3.3 Detection of closed loop control system out of control 774
4.10.3.1.6.2.5.3.4 Detection of system adaptation out of range 776
4.10.3.1.6.2.5.3.5 Detection of other faults inferred from system behaviour ... 777
4.10.3.1.6.2.5.4 Detection of faults due to configuration errors 778
4.10.3.1.6.2.5.4.1 Detection of software incompatibility with another ECU 781
4.10.3.1.6.2.5.5 Detection of faults caused by received signals 783
4.10.3.1.6.2.5.5.1 Detection of other ECU not operating normally 784
4.10.3.1.6.2.5.5.2 Detection of other ECU sending data tagged as invalid 786
4.10.3.1.6.2.5.5.3 Detection of other ECU sending faulty data 787
4.10.3.1.6.2.5.5.4 Detection of corrupt data received from other ECU by rolling counter 789
4.10.3.1.6.2.5.5.5 Detection of corrupt data received from other ECU by end-to-end checksum 790
4.10.3.1.6.2.5.5.6 Detection of corrupt data received from other ECU by up-date-bit 791
4.10.3.1.6.2.5.6 Detection of faults caused by the ECU environment 792
4.10.3.1.6.2.5.6.1 Detection of events caused by other condition that is out of range for f... 794
4.10.3.1.6.2.5.6.2 Detection of faults caused by the ECU operating environment or customer action 795
4.10.3.1.6.2.5.6.3 Detection of events caused by ambient temperature 797
4.10.3.1.6.2.5.6.4 Detection of other faults inferred from system behaviour ... 798
4.10.3.1.6.2.5.7 Detection of events associated with driver indications 798
4.10.3.1.6.2.5.8 Detection of faults due to LIN related circuit faults for LIN Master 799
4.10.3.1.6.2.5.9 Detection of faults caused by too high quiescent current 800
4.10.3.1.6.2.5.10 DTC Data Verification DTCs 800
4.10.3.1.6.2.5.11 Detection of faults in private ECUs 801

4.10.3.2 UDS Data - Basetech Data Requirements 807

4.10.3.2.1 DID data 807
4.10.3.2.1.1 DID of Vehicle Identification 807
4.10.3.2.1.2 DID of Vehicle Status 841

4.10.3.2.2 RID data 876

4.10.3.2.3 DTC data 896
4.10.3.2.3.1 DTC of faults in external electrical circuits for CAN bus 907
4.10.3.2.3.2 DTC of LIN related circuit faults for LIN Master 911
4.10.3.2.3.3 DTC of Ethernet link failure 914
4.10.3.2.3.4 Electrical fault trouble code information 915

4.10.4 UDSonCAN 919

4.10.4.1 UDSonCAN - Generic 919

4.10.4.1.1 Introduction 919
4.10.4.1.1.1 Overview 920
4.10.4.1.1.2 Requisite documents 921
4.10.4.1.1.3 Definitions and Abbreviations 922

4.10.4.1.2 Application Layer 923

4.10.4.1.3 Session Layer 923
4.10.4.1.3.1 Addressing USDT frames 923

4.10.4.1.4 Transport_Network layer 924

4.10.4.1.5 Data Link Layer 925

4.10.4.1.6 Physical Layer 925

4.10.4.2 UDSonCAN - Public CAN ECU 925

4.10.4.2.1 Diagnostic Services 925

4.10.4.2.2 Diagnostic Data 926

4.10.4.3 UDSonCAN - Private Programmable CAN ECU 926

4.10.4.3.1 Diagnostic Data 926
4.10.4.3.1.1 RoutineIdentifier 928

4.10.4.3.2 Diagnostic Services 929

4.10.5 UDSonLIN 931

4.10.5.1 UDSonLIN - Generic 931

4.10.5.1.1 Introduction 931

4.10.5.1.2 Overview 932

4.10.5.1.3 Requisite documents 932

4.10.5.1.4 Definitions and Abbreviations 933

4.10.5.2 UDSonLIN - Generic LIN master 934

4.10.5.2.1 Diagnostic Data 934

4.10.5.3 UDSonLIN - LIN Slave 934

4.10.5.3.1 Unified diagnostic services implementation 934
4.10.5.3.1.1 Mandatory LIN services and data records 934
4.10.5.3.1.2 Defining NAD for LIN slaves 937

4.10.5.3.2 Requirements from section Detection of faults due to configuration errors 937

4.10.6 UDSonIP 938

4.10.6.1 UDSonIP - Generic reqs 938

4.10.6.1.1 Introduction 938

4.10.6.1.2 Overview 939

4.10.6.1.3 Requisite documents 941

4.10.6.1.4 Definitions and Abbreviations 942

4.10.6.1.5 Unified diagnostic service implementation 943
4.10.6.1.5.1 Diagnostic services 943
4.10.6.1.5.1.1 ReadDataByPeriodicIdentifier 943
4.10.6.1.5.2 Diagnostic data 943

4.10.6.1.6 Application Layer 943

4.10.6.2 UDSonIP - Edge node reqs 944

4.10.6.2.1 Unified Diagnostic Service Implementation Requirements 944
4.10.6.2.1.1 Diagnostic services 944
4.10.6.2.1.2 Diagnostic Data 944
4.10.6.2.1.3 Physical Connection 944

4.10.7 Data Security 948

4.10.7.1 Random number generation 948

4.10.7.1.1 Introduction 948

4.10.7.1.2 Requirements 949

4.10.7.2 Security Access algorithm 950

4.10.7.2.1 Introduction 950

4.10.7.2.2 Algorithm Description 950

4.10.7.2.3 Security Constant 951

4.10.7.2.4 Configuration of the Level of Protection 952

4.10.7.2.5 Random Number Generation 953

4.10.7.2.6 Security Levels (Areas) 954

4.10.8 Authentication (0x29) with PKI Certificate Exchange 955

4.10.8.1 Authentication General Requirements 955

4.10.8.1.1 Authentication function 955

4.10.8.1.2 Authentication state for UDS service 29 956

4.10.8.1.3 Appendix C- Authentication roles and White lists for UDS service 29958

4.10.8.2 Authentication with PKI Certificate Exchange 961

4.10.8.2.1 Introduction 961

4.10.8.2.2 Definitions and Abbreviations 961

4.10.8.2.3 Requisite documents 961

4.10.8.2.4 Authentication Process Description 962
4.10.8.2.4.1 Unidirectional authentication 962
4.10.8.2.4.2 Bidirectional authentication 964

4.10.8.2.5 Challenge Description 966

4.10.8.2.6 Certificate 967

4.10.8.2.7 Proof of Ownership 970

4.10.8.2.8 Secure diagnostic communication 972
4.10.8.2.8.1 Ephemeral Private/Public Key 972
4.10.8.2.8.2 Session Key Information 973

4.10.8.2.9 Signing Methods 974
4.10.8.2.9.1 Signing Method - Variability control 974
4.10.8.2.9.2 Signing Method - Algorithm Definitions 974

4.10.8.2.10 Key Exchange Methods 975
4.10.8.2.10.1 Key Exchange Method - Variability control 975
4.10.8.2.10.2 Key Exchange Method - Algorithm Definitions 975

4.10.8.2.11 Encryption Methods 976
4.10.8.2.11.1 Encryption Method - Variability control 976
4.10.8.2.11.2 Encryption Method - Algorithm Definitions 976
4.10.8.2.11.3 Encryption Method - Additional requirements 977

4.10.8.2.12 Configuration of the authentication of Protection 977

4.10.8.2.13 Examples 978
4.10.8.2.13.1 Key exchange example 978
4.10.8.2.13.2 Encrpytion of UDS Data example 979
4.10.8.2.13.3 Proof of ownership server in bdirectional authentication example979
4.10.8.2.13.4 Proof of ownership client in unidirectional authentication example979
4.10.8.2.13.5 OCSP Response Time Validatation Confirmation 980

4.10.9 Regulatory certification based on UDS 981

4.10.9.1 Scope 981

4.10.9.2 ZEVonUDS 981

4.10.9.2.1 Introduction 981

4.10.9.2.2 ZEVonUDS Diagnostic Link Definition 982

4.10.9.2.3 Genenral Requirments 985

4.10.9.2.4 ZEVonUDS Services Application 986

4.10.9.2.5 ZEVonUDS Data security 989

4.10.9.3 OBDonUDS 990

4.10.9.3.1 Introduction 990

4.10.9.3.2 OBDonUDS Diagnostic Link Definition 990

4.10.9.3.3 OBDonUDS Services Application 992

4.10.9.3.4 OBDonUDS Data security 996

4.10.9.4 ADAS_L3onUDS 996

4.10.9.4.1 Introduction 996

4.10.9.4.2 ADAS_L3onUDS Diagnostic Link Definition 997

4.10.9.4.3 Genenral Requirments 998

4.10.9.4.4 L3onUDS Services Application 998

4.10.9.4.5 L3onUDS Data security 1000

4.10.9.5 ePTIonUDS 1001

4.10.9.5.1 Introduction 1001

4.10.9.5.2 ePTIonUDS Diagnostic Link Definition 1001

4.10.9.5.3 Genenral Requirment 1004

4.10.9.5.4 ePTIonUDS Services Application 1004

4.10.9.5.5 ePTIonUDS Data security 1005

4.10.9.6 Data security 1006

4.10.9.6.1 Platform-based data protection solution based on Ethernet 1006

4.10.9.6.2 Platform-based data protection solution based on CAN 1007

4.10.10 Universal Calibration Protocol 1008

4.10.10.1 General Requirements for XCP 1008

4.10.10.1.1 XCP Introduction 1008

4.10.10.1.2 XCP requirements 1009

4.10.11 ECU Running Log Requirements 1013

4.10.11.1 Running Log Requirements 1013

4.10.11.1.1 Introduction 1013
4.10.11.1.1.1 Guidelines for logging 1013

4.10.11.1.2 Requirements 1014

4.11 Software download 1027

4.11.1 General Software Download 1027

4.11.1.1 Generic SWDL Overview 1027

4.11.1.1.1 Introduction 1027

4.11.1.1.2 What is the purpose of this document 1027

4.11.1.1.3 Overview 1027

4.11.1.1.4 Requisite documents 1027
4.11.1.1.4.1 Referenced document, structure and relations 1028

4.11.1.1.5 Definitions and Abbreviations 1030

4.11.1.2 General Bootlaoder requirements 1030

4.11.1.2.1 Target 1030

4.11.1.2.2 Power consumption 1032

4.11.1.2.3 Internal data transfer speed 1032

4.11.1.2.4 Handling of distribution relays 1033

4.11.1.2.5 High Voltage Control 1033

4.11.1.2.6 Calibration and customer data 1034

4.11.1.2.7 Two level bootloader 1034
4.11.1.2.7.1 Primary Bootloader (PBL) 1036
4.11.1.2.7.1.1 Primary bootloader diagnostic data 1037
4.11.1.2.7.1.2 Primary bootloader diagnostic services 1037
4.11.1.2.7.2 Secondary Bootloader (SBL) 1038
4.11.1.2.7.2.1 Secondary bootloader diagnostic data 1039
4.11.1.2.7.2.2 Secondary bootloader diagnostic services 1039

4.11.1.2.8 Additional bootloader requirements for secondary processors 1039

4.11.1.2.9 Additional bootloader requirements for multiple networks 1040

4.11.1.2.10 Additional bootloader requirements for private ECU 1041

4.11.1.2.11 Data compression and encryption 1042

4.11.1.2.12 Delta encoding 1043

4.11.1.2.13 ECU program mode 1043
4.11.1.2.13.1 Simplified bootloader state and flow chart diagram 1044

4.11.1.2.14 Enter the bootloader (PBL) from the application 1046

4.11.1.2.15 Time window for PROG detection 1048

4.11.1.2.16 File download order 1049

4.11.1.2.17 Separate memory sectors 1049

4.11.1.2.18 General Request Download 1050

4.11.1.2.19 Writing of received data to non-volatile memory 1053

4.11.1.2.20 Pre-calculated hash 1054

4.11.1.3 Bootoalder requirements for SWDL with check memory 1055

4.11.1.3.1 Software Authentication 1055
4.11.1.3.1.1 Software Structure 1056
4.11.1.3.1.1.1 ECU Software Structure (ESS) 1059
4.11.1.3.1.1.2 ECU Software Structure - Declaration 1062
4.11.1.3.1.1.3 Root ECU Software Structure 1063
4.11.1.3.1.2 Erase Status Information 1063
4.11.1.3.1.3 Verification Block Table 1064
4.11.1.3.1.4 Validation of ECU software(s) - Authenticity Verification 1066
4.11.1.3.1.4.1 Authenticity Verification - Overview 1066
4.11.1.3.1.4.2 Validation of Logical Blocks 1067
4.11.1.3.1.4.3 Validity Status Information 1069
4.11.1.3.1.4.4 Public Key Storage 1070
4.11.1.3.1.5 Additional requirements for ECUs having multiple processors 1070
4.11.1.3.1.5.1 Multiple processors - Secondary Bootloader 1071
4.11.1.3.1.6 Additional requirements for ECUs implementing delta encoding1073
4.11.1.3.1.7 Signing method 1075
4.11.1.3.1.8 Software Authentication - Generic security requirements 1075

4.11.1.3.2 Software Download 1077
4.11.1.3.2.1 Programming Sequence Overview 1077
4.11.1.3.2.2 Download and activate the SBL 1078
4.11.1.3.2.3 Memory Erasure 1080
4.11.1.3.2.4 Request Download 1080
4.11.1.3.2.5 Check Memory 1082

4.11.1.3.2.6 Complete and Compatible - Validation of ECU software(s) 1084
4.11.1.3.2.6.1 Complete and Compatible - Complete check 1087
4.11.1.3.2.6.2 Complete and Compatible - Compatible check 1088
4.11.1.3.2.6.3 CompleteCompatibleFunction() 1089
4.11.1.3.2.7 ECU Reset - Application start 1090
4.11.1.3.2.8 Request Upload 1091

4.11.1.3.3 Appendix 1091
4.11.1.3.3.1 Examples of different scenarios for the Complete & Compatible function 1091
4.11.1.3.3.2 Examples of different Compatible functions 1092
4.11.1.3.3.2.1 Bootloader Compatbile check 1092
4.11.1.3.3.2.2 Application Complete & Compatible block 1092

4.11.1.4 Bootloader requirements for SWDL with no check memory only for GEEA3.0 1095

4.11.1.4.1 Software Download 1095
4.11.1.4.1.1 Programming Sequence Overview 1095

4.11.1.4.2 Validation of ECU software(s) 1096
4.11.1.4.2.1 Application Complete & Compatible block 1096
4.11.1.4.2.1.1 StartCompleteCompatibleSignature 1097
4.11.1.4.2.1.2 ApplicationStartAddress 1098
4.11.1.4.2.1.3 CompleteCompatibleFunctionAddress 1098
4.11.1.4.2.1.4 EndCompleteCompatibleSignatureAddress 1099
4.11.1.4.2.1.5 CompleteCompatibleFunction() 1099
4.11.1.4.2.1.6 EndCompleteCompatibleSignature 1102
4.11.1.4.2.1.7 Complete & Compatible flow chart diagram 1102

4.11.1.4.3 Appendix 1104
4.11.1.4.3.1 Examples of different scenarios for the Complete & Compatible function 1104

4.11.1.5 General Bootloader requirements for file system 1105

4.11.1.5.1 File Download and Installation Sequence Overview 1105

4.11.1.5.2 Target 1106

4.11.1.5.3 Data compression and encryption 1107

4.11.1.5.4 Assignment push Control 1108

4.11.1.5.5 File transfer control 1110

4.11.1.5.6 Digital Envelope Transfer 1112

4.11.1.5.7 ECU program mode 1115

4.11.1.5.8 General file installation order 1115

4.11.1.5.9 Softeware authentication 1116
4.11.1.5.9.1 Software authentication for file system ECU- introduction 1116
4.11.1.5.9.2 Verification Block Table 1117
4.11.1.5.9.3 Validation of ECU software(s) - Authenticity Verification 1119
4.11.1.5.9.3.1 Validation of Logical Blocks 1119
4.11.1.5.9.3.2 Public Key Storage 1121
4.11.1.5.9.3.3 Softeware Authentication for file system ECU 1121

4.11.1.5.10 Validation of ECU software(s) - Complete and Compatible 1122

4.11.1.5.11 File installation 1125

4.11.1.5.12 Download file from local tester 1126

4.11.1.5.13 Appendix 1129
4.11.1.5.13.1 Examples of message flow 1129
4.11.1.5.13.2 Examples of different Compatible functions 1131
4.11.1.5.13.2.1 Bootloader Compatbile check 1131
4.11.1.5.13.2.2 Application Complete & Compatible block 1131

4.11.2 Software Authentication 1134

4.11.2.1 General Software Authentication 1134

4.11.2.1.1 Definitions and Abbreviations 1134

4.11.2.1.2 Requisite documents 1134

4.11.2.1.3 Introduction 1137

4.11.2.1.4 System Description 1142
4.11.2.1.4.1 Create Software Part 1143
4.11.2.1.4.1.1 Verification Block Table 1144
4.11.2.1.4.2 Sign Software Part 1150
4.11.2.1.4.2.1 Sign and verify unprocessed data 1155
4.11.2.1.4.2.2 Sign a Delta Software Part 1157
4.11.2.1.4.3 Download a (signed) software part 1157
4.11.2.1.4.4 Verify a signed software part 1158
4.11.2.1.4.5 Key Management for Test and production ECUs 1159

4.11.2.1.5 Signing Methods 1174
4.11.2.1.5.1 Signing Method - Variability control 1174
4.11.2.1.5.2 Signing Method - Algorithm Definitions 1176
4.11.2.1.5.3 Signing Method - Key balancing 1177
4.11.2.1.5.4 Software Settings File 1179

4.11.2.1.6 Appendix 1181

4.11.2.1.6.1 Example - Data format identifiers and signing (sw_part_based)1181
4.11.2.1.6.2 Example - Software Part evolution 1181
4.11.2.1.6.3 Example - Signing function when the processing method is unknown 1182
4.11.2.1.6.4 Example - PublicKeyCheckSum 1183

4.11.3 Data Compression and Encryption 1184

4.11.3.1 Data Compression and Encryption 1184

4.11.3.1.1 Overview 1184
4.11.3.1.1.1 Definitions and Abbreviations 1184
4.11.3.1.1.2 External Publications 1184

4.11.3.1.2 Software Compression 1184
4.11.3.1.2.1 Compression Method #1 1184
4.11.3.1.2.1.1 Data Compression 1184

4.11.3.1.3 Data Encryption 1185

4.11.3.1.4 Software Component Requirements 1185
4.11.3.1.4.1 Compression Method Selection 1185
4.11.3.1.4.1.1 Method Selection 1185
4.11.3.1.4.2 Compression Method #1 Requirements 1185
4.11.3.1.4.2.1 Data Compression 1185

4.11.3.1.5 Appendix 1187
4.11.3.1.5.1 Compression Method #1 Examples 1187
4.11.3.1.5.1.1 LZSS Compression C-code Example 1187
4.11.3.1.5.1.2 LZSS Decompression C-code Example 1197
4.11.3.1.5.1.3 Implementation Advice Regarding Decompression 1200

4.11.4 Delta Encoding for ECU 1202

4.11.4.1 Delta Encoding Introduction 1202

4.11.4.1.1 Definitions & Abbreviations 1202

4.11.4.1.2 Requisite Documents 1202

4.11.4.1.3 Logical System Components 1203

4.11.4.2 Delta generation 1203

4.11.4.2.1 Requirements 1203

4.11.4.3 Delta installation 1208

4.11.4.3.1 Requirements 1208
4.11.4.3.1.1 Error Handling 1212

4.11.4.4 Extended information 1215

4.11.4.4.1 Delta installation - Sequence and memory usage 1216

4.11.5 Delta Encoding for System 1216

4.11.5.1 Vehicle Software Configuration 1216

4.11.5.1.1 Requirements 1216

4.11.5.2 Workshop System 1217

4.11.5.2.1 Requirements 1217

4.11.5.3 Vehicle Service Delivery Platform 1218

4.11.5.3.1 Requirements 1218

4.11.6 Versatile Binary Format Specification 2.7 1218

4.11.6.1 Versatile Binary Format Generic reqs 1218

4.11.6.1.1 Software part format 1218

4.11.6.1.2 VBF Additional Requirements 1219

4.11.6.2 Versatile Binary Format Specification 1219

4.11.6.2.1 Overview 1220
4.11.6.2.1.1 Requisite document 1220
4.11.6.2.1.2 Definitions and Abbreviations 1220

4.11.6.2.2 Versatile Binary Format (VBF) 1220
4.11.6.2.2.1 VBF version 1220
4.11.6.2.2.2 Introduction 1220
4.11.6.2.2.3 File Naming 1221
4.11.6.2.2.4 VBF file generation 1222

4.11.6.2.3 Notation and lexical elements 1222
4.11.6.2.3.1 Notation 1222
4.11.6.2.3.2 General structure 1222
4.11.6.2.3.3 Identifiers, reserved words and expressions 1223
4.11.6.2.3.4 Integers 1224
4.11.6.2.3.5 Comments 1224
4.11.6.2.3.6 White Space Characters 1225
4.11.6.2.3.7 Non-printing Characters 1225

4.11.6.2.4 Overview of the VBF file 1225
4.11.6.2.4.1 Version Section 1225

4.11.6.2.4.2 Header Section 1226
4.11.6.2.4.2.1 description 1228
4.11.6.2.4.2.2 sw_part_number 1228
4.11.6.2.4.2.3 sw_current_part_number 1229
4.11.6.2.4.2.4 sw_version 1229
4.11.6.2.4.2.5 sw_current_version 1230
4.11.6.2.4.2.6 sw_part_type 1231
4.11.6.2.4.2.7 data_format_identifier 1232
4.11.6.2.4.2.8 address_and_length_format_identifier 1233
4.11.6.2.4.2.9 ecu_address 1233
4.11.6.2.4.2.10 erase 1234
4.11.6.2.4.2.11 call 1235
4.11.6.2.4.2.12 verification_block_start 1236
4.11.6.2.4.2.13 verification_block_length 1236
4.11.6.2.4.2.14 verification_block_root_hash 1237
4.11.6.2.4.2.15 sw_signature 1237
4.11.6.2.4.2.16 sw_signature_dev 1238
4.11.6.2.4.2.17 file_checksum 1238
4.11.6.2.4.2.18 parameter_settings 1239
4.11.6.2.4.2.19 Header section example #1 1240
4.11.6.2.4.2.20 Header section example #2 1241
4.11.6.2.4.2.21 Header section example #3 1242
4.11.6.2.4.2.22 Header section example #4 1244
4.11.6.2.4.3 Data Section 1245
4.11.6.2.4.3.1 Checksum calculation 1247

4.11.6.2.5 Appendix A 1247
4.11.6.2.5.1 CRC16-CITT C-code example 1247

4.11.6.2.6 Appendix B 1249
4.11.6.2.6.1 CRC32 C-code example 1249

4.11.7 Versatile Binary Format Specification 2.6 1251

4.11.7.1 Versatile Binary Format Generic reqs 1251

4.11.7.1.1 Software part format 1251

4.11.7.1.2 VBF Additional Requirements 1251

4.11.7.2 Versatile Binary Format Specification 1252

4.11.7.2.1 Overview 1252
4.11.7.2.1.1 Requisite documents 1252
4.11.7.2.1.2 Definitions and Abbreviations 1252

4.11.7.2.2 Versatile Binary Format (VBF) 1253
4.11.7.2.2.1 VBF version 1253
4.11.7.2.2.2 Introduction 1253
4.11.7.2.2.3 File Naming 1254
4.11.7.2.2.4 VBF file generation 1255

4.11.7.2.3 Notation and lexical elements 1255

4.11.7.2.3.1 Notation 1255
4.11.7.2.3.2 General structure 1255
4.11.7.2.3.3 Identifiers, reserved words and expressions 1256
4.11.7.2.3.4 Integers 1257
4.11.7.2.3.5 Comments 1257
4.11.7.2.3.6 White Space Characters 1258
4.11.7.2.3.7 Non-printing Characters 1258

4.11.7.2.4 Overview of the VBF file 1259
4.11.7.2.4.1 Version Section 1259
4.11.7.2.4.2 Header Section 1259
4.11.7.2.4.2.1 description 1261
4.11.7.2.4.2.2 sw_part_number 1262
4.11.7.2.4.2.3 sw_current_part_number 1263
4.11.7.2.4.2.4 sw_version 1263
4.11.7.2.4.2.5 sw_current_version 1264
4.11.7.2.4.2.6 sw_part_type 1265
4.11.7.2.4.2.7 data_format_identifier 1267
4.11.7.2.4.2.8 ecu_address 1268
4.11.7.2.4.2.9 erase 1268
4.11.7.2.4.2.10 call 1269
4.11.7.2.4.2.11 verification_block_start 1270
4.11.7.2.4.2.12 verification_block_length 1271
4.11.7.2.4.2.13 verification_block_root_hash 1271
4.11.7.2.4.2.14 sw_signature 1272
4.11.7.2.4.2.15 sw_signature_dev 1273
4.11.7.2.4.2.16 file_checksum 1273
4.11.7.2.4.2.17 parameter_settings 1274
4.11.7.2.4.2.18 Header section example #1 1276
4.11.7.2.4.2.19 Header section example #2 1277
4.11.7.2.4.2.20 Header section example #3 1278
4.11.7.2.4.2.21 Header section example #4 1280
4.11.7.2.4.3 Data Section 1280
4.11.7.2.4.3.1 Checksum calculation 1282

4.11.7.2.5 Appendix A 1282
4.11.7.2.5.1 CRC16-CITT C-code example 1282

4.11.7.2.6 Appendix B 1284
4.11.7.2.6.1 CRC32 C-code example 1284

4.11.8 Bootloader - UDSonCAN 1286

4.11.8.1 Bootloader - UDSonCAN 1286

4.11.8.1.1 Introduction 1286

4.11.8.1.2 Overview 1287

4.11.8.1.3 Requisite Documents 1288

4.11.8.1.4 Definitions and Abbreviations 1289

4.11.8.1.5 Unified Diagnostic Services Implementation 1289
4.11.8.1.5.1 Diagnostic Services 1289
4.11.8.1.5.1.1 RequestDownload (34 hex) Service 1289

4.11.8.1.6 ECU Program Mode 1290
4.11.8.1.6.1 Bootloader state diagram 1291
4.11.8.1.6.2 Bootloader flow chart diagram - Overview 1291
4.11.8.1.6.2.1 Bootloader flow chart diagram (Init) 1293
4.11.8.1.6.2.2 Bootloader flow chart diagram (ProgSession) 1298
4.11.8.1.6.2.3 Flow chart diagram (Application) 1300

4.11.8.1.7 Requirements 1301

4.11.9 Bootloader - UDSonLIN 1302

4.11.9.1 Bootloader - UDSonLIN - Generic 1302

4.11.9.1.1 Introduction 1302

4.11.9.1.2 Overview 1303

4.11.9.1.3 Requisite documents 1304

4.11.9.1.4 Definitions and Abbreviations 1305

4.11.9.2 Bootloader - UDSonLIN - LIN Slave 1305

4.11.9.2.1 Unified diagnostic services implementation requirements 1305
4.11.9.2.1.1 Diagnostic Services 1305
4.11.9.2.1.2 Diagnostic Data 1306

4.11.9.2.2 Session Layer requirements 1306
4.11.9.2.2.1 Queued requests 1306
4.11.9.2.2.2 P2Server 1307

4.11.9.2.3 ECU program mode 1307
4.11.9.2.3.1 Bootloader state diagram 1307
4.11.9.2.3.2 Bootloader flow chart diagram - Overview 1308
4.11.9.2.3.3 Bootloader flow chart diagram (Init) 1311
4.11.9.2.3.4 Bootloader flow chart diagram (ProgSession) 1315
4.11.9.2.3.5 Bootloader flow chart diagram (Application) 1318

4.11.10 Bootloader - UDSonIP 1319

4.11.10.1 BL - UDSonIP - Generic reqs 1319

4.11.10.1.1 Introduction 1319

4.11.10.1.2 Overview 1320

4.11.10.1.3 Requiste documents 1321

4.11.10.1.4 Definitions and Abbreviations 1322

4.11.10.1.5 Introduction 1322
4.11.10.1.5.1 Definitions and Abbreviations 1324
4.11.10.1.5.2 External publications 1325
4.11.10.1.5.3 Requiste documents 1325

4.11.10.1.6 Requirements 1326

4.11.10.2 BL - UDSonIP - IP ECU reqs 1333

4.11.10.2.1 Introduction 1333
4.11.10.2.1.1 IP ECU bootloader state diagram 1334
4.11.10.2.1.2 IP ECU bootloader flow chart - Overview 1334
4.11.10.2.1.2.1 IP ECU flow chart - Application 1336
4.11.10.2.1.2.2 IP ECU flow chart - Init 1336
4.11.10.2.1.2.3 IP ECU flow chart - ProgrammingSession 1337

4.11.10.2.2 Requirement 1338

4.11.10.3 BL - UDSonIP - Edge node reqs 1343

4.11.10.3.1 Introduction 1343
4.11.10.3.1.1 Edge Node's bootloader state diagram 1343
4.11.10.3.1.2 Edge Node's bootloader flow chart Overview 1344
4.11.10.3.1.2.1 Edge Node connection flow chart diagram 1346
4.11.10.3.1.2.2 The Edge Node's flow chart - Application 1347
4.11.10.3.1.2.3 The Edge Node's flow chart - Init 1347
4.11.10.3.1.2.4 The Edge Node's flow chart - ProgrammingSession and Undefined state 1348

4.11.10.3.2 Requirement 1349

4.11.11 Dual Bank Software Update 1358

4.11.11.1 General A/B Swap Requirements 1358

4.11.11.1.1 General A/B Swap Requirements 1358

4.11.11.2 A/B Swap flowchart 1360

4.11.11.2.1 overview of the boot loader flowchart 1360

4.11.11.2.2 INIT State 1361

4.11.11.2.3 Partition choice 1362

4.11.11.2.4 Rollback 1362

4.11.11.2.5 Switch A/B 1363

4.11.11.2.6 Default session 1363

4.11.11.2.7 Programming session 1365

4.11.11.3 A/B Swap Requirements 1366

4.11.11.3.1 Induction 1366

4.11.11.3.2 ECU reprogram in application mode 1367

4.11.11.3.3 session transition 1367

4.11.11.3.4 requirements for ECU wich support AB swap 1369

4.11.11.4 Dual Bank SWDL Based On MCU 1373

4.11.11.4.1 Introduction 1373

4.11.11.4.2 Scheme #1 1373
4.11.11.4.2.1 State diagram 1374
4.11.11.4.2.2 Flow chart diagram 1374

4.11.11.4.3 Scheme #2 1376
4.11.11.4.3.1 State diagram 1376
4.11.11.4.3.2 Flow chart diagram 1376

4.11.11.4.4 Requirement 1377

4.11.11.5 Dual Bank SWDL On POXIS System 1378

4.11.11.5.1 Introduction 1378

4.11.11.5.2 State diagram 1378

4.11.11.5.3 Flow chart diagram 1379

4.11.11.5.4 Requirement 1380

4.11.11.6 For Dual Bank ECU but Without Seamless Update 1381

4.11.11.6.1 Additional Requirements for Complete and Compatible 1381

4.11.11.7 External Flash A/B Requirements 1383

4.11.11.7.1 General Flash Requirements 1383

4.11.11.8 Diagnostic data for dual-bank ECU 1384

4.11.11.8.1 DID 1385

4.11.11.9 Non-sensible Programming For OTA 1385

4.11.11.9.1 Introduction 1386

4.11.11.9.2 State diagram 1386

4.11.11.9.3 General Requirements 1387

4.11.11.9.4 OTA SWDL Sequence 1390
4.11.11.9.4.1 Enter OTA Session 1391
4.11.11.9.4.2 Security Access 1393
4.11.11.9.4.3 Data Transfer 1393
4.11.11.9.4.3.1 Transfer Resume 1394
4.11.11.9.4.3.2 Erase 1396
4.11.11.9.4.4 AB Partition Switch 1396
4.11.11.9.4.5 Clear DTC 1397

4.11.11.9.5 CompleteAndCompatible Check 1398

4.11.11.9.6 Erase & write Inactive program partition 1399

4.11.11.9.7 Software download and process for distributed method 1399

4.11.11.9.8 Update process status 1399

4.11.11.10 Non-sensible Programming Requirement For Private ECU 1402

4.11.11.10.1 General Requirements 1402

4.11.11.11 Diagnostic data related to Non-sensible Programming 1404

4.11.11.11.1 DID 1404

4.11.11.11.2 RID 1407

4.12 Service-Oriented Vehicle Diagnostics (SOVD) 1410

4.12.1 Service-Oriented Vehicle Diagnostics (SOVD) 1410

4.12.1.1 General Requirements 1410

4.12.1.1.1 Overview 1410

4.12.1.1.2 Requisite documents 1411

4.12.1.1.3 Definitions and Abbreviations 1411

4.12.1.2 SOVD Architecture 1411

4.12.1.2.1 SOVD Architecture 1411

4.12.1.3 SOVD - Edge Node (Gateway) 1412

4.12.1.3.1 SVOD Vehicle Gateway 1412

4.12.1.3.2 SOVD Server Discovery 1414

4.12.1.3.3 Concurrency and Connection Management 1417
4.12.1.3.3.1 Client Connection Management 1417
4.12.1.3.3.2 Invehicle Connection Management 1418

4.12.1.4 SOVD API Methods 1421

4.12.1.4.1 SOVD API Methods 1421

4.12.1.4.2 Access of supported SOVD server versions 1422

4.12.1.4.3 Access to capability description content 1422

4.12.1.4.4 Discovery of entities and resources 1423

4.12.1.4.5 Tagged Entities and resources (Enhancement) 1424

4.12.1.4.6 Fault handling 1424

4.12.1.4.7 Data Resource read/write access 1428

4.12.1.4.8 Cyclic-subscriptions 1429

4.12.1.4.9 Triggers 1430

4.12.1.4.10 Configuration 1430

4.12.1.4.11 Clearing data 1431

4.12.1.4.12 Control of operations 1431

4.12.1.4.13 Script Execution 1432
4.12.1.4.13.1 Multi script compression upload 1434

4.12.1.4.14 Support of target modes 1435

4.12.1.4.15 Locking 1436

4.12.1.4.16 Restarting an Entity 1437

4.12.1.4.17 Handling of bulk-data 1437

4.12.1.4.18 Logging 1439

4.12.1.4.19 Communication-logs 1440

4.12.1.5 SOVD Authentication 1440

4.12.1.5.1 General Requirements 1440

4.12.1.5.2 Offline authentication and authorization 1442

4.12.1.6 SOVD Vehicle Update 1445

4.12.1.6.1 General Requirements 1445

4.12.1.6.2 Detailed Update Information Requirements 1448

4.12.1.6.3 Update Requirements 1450

4.12.1.6.4 Update Status Requirements 1457

4.12.1.6.5 Update Report Requirements 1459

4.12.1.7 SOVD Private Server 1459

4.12.1.7.1 Requirements 1459

4.12.1.8 HTTP Protocol Requirement 1460

4.12.1.8.1 Requirement 1460

4.12.1.8.1.1 HTTP/1.1 1460
4.12.1.8.1.2 HTTP/2.0 1460

4.12.1.9 SOVD - Classic Diagnostic Adapter 1461

4.12.1.9.1 Requirements 1461

4.13 Appendix for GEEA3.0&EEA4.0 1467

4.13.1 Gateway 1467

4.13.1.1 Zone Network Gateway - CAN - CAN without TP 1467

4.13.1.1.1 Gateway allocation of communication channels 1467
4.13.1.1.1.1 Backbone network 1467
4.13.1.1.1.2 Zone Network 1468

4.13.1.1.2 Gateway latency requirements 1469
4.13.1.1.2.1 Diagnostic requests/responses CAN to CAN 1469
5 DETAILED CHANGE INFO 1472

0 REQUIREMENT LOG

0.1 List of requirements

1362	LM purpose	1
1365	ECU wakeup detection	1
1366	Wakeup of internal IP Bus	1
1372	LM initialization	1
1373	LSC IP Bus release	1
1374	LSC IP bus request	1
1379	Number of Nodes in RG	1
1382	Resource Groups	1
1386	Port	1
1388	Local Configuration file	1
1427	Transport layer	1
1428	LIN master N_Cs performance in programming session	1
1429	LIN master usage of N_Cr	1
1430	Block size	1
1431	LIN slave proceed with current message if receiving an invalid frame	1
1432	LIN slave receiving a new message while processing a message	1
1433	Use data frame by frame	1
1641	Zone ID for IP networks	1
6286	Compatibility between versions	1
6287	Compression	1
6288	Data file type	1
6289	Delta file distribution	1
6290	Delta file name	1
6291	Delta file release information	1
6292	Delta file size	1
6293	Delta file structure	1
6294	Delta file version info	1
6295	Delta installation RAM buffer size	1
6296	Delta installation time	1
6297	New and obsolete data info	1
6298	Data decoding	1
6299	Data decompression	1
6300	Delta installation data	1
6301	Delta installation memory segment	1
6302	Delta installation RAM usage	1
6303	Delta memory segment size	1
6304	Delta SWDL in normal operational mode	1
6305	Delta SWDL in program mode	1
6306	Obsolete data removal	1
6307	Software consistency check	1
6308	Streamed installation	1
6309	Target sector erase check	1
6310	Consistency check failure	1
6311	Delta file corruption	1
6312	Error code mapping	1
6313	Full version install check	1
6314	Not enough memory	1
6315	Recover from interrupted Installation	1
6316	Subsequent reprogramming during recover	1
6317	Version mismatch	1
6318	Delta update requests	1

6319	Full version update requests	1
6320	Version handling	1
6321	Full version update requests	1
6322	Update request parameter - sw_version	1
6323	System availability notification	1
92884	initialLogSyncInterval and operLogSyncInterval	1
92885	Nominal Sync interval	1
92889	InitialLogPdelayReqInterval and operLogPdelayReqInterval	1
92891	Nominal Pdelay_Req interval	1
92893	NeighborPropDelay	1
92895	RateRatio and NeighborRateRatio	1
92897	Time to wait for initial Sync message	1
92899	Supported audio sample rate	1
92902	AED-A Talker Timestamp mode	1
92904	Dynamic Channel Layout EventField	1
92907	Number of channels	1
92918	SR classes	1
92919	SR Class Template(s)	1
92920	Mapping between PCP and SR Class	1
92922	Maximum Transit Time (MTT)	1
94967	Pdalay Request send	1
94968	Lost or late video data	1
94974	Testablity function	1
535473	TLS handshake process	1
535480	TLS authentication type	1
535492	TLS version selection	1
535493	Cipher suites selection for TLS 1.2	1
652157	Trusted network	1
652167	Ethernet point-to-point connection	1
652168	Untrusted network	1
652169	Safety TCP connection on untrusted network	1
652170	Safety TCP connection on trusted network	1
652173	Default safety communication configuration	1
652176	Initial safety communication for DoIP	1
652192	DoIP TLS TCP port	1
652213	Client TLS process	1
652215	Server TLS process	1
652225	Client TLS version	1
652228	Server TLS version	1
652229	Client TLS cipher suites	1
652231	Server TLS cipher suites	1
652246	Requirements to DoIP server	1
652247	Requirements to DoIP client	1
652293	Cipher suites selection for TLS 1.3	1
652304	Extension for TLS 1.2	1
652306	Extension for TLS 1.3	1
652424	TLS handshake establishment	1
660523	DID - External Signals Requirements	1
660524	DID - Internal Signals Requirements	1
660543	DID - Data types and Classifications	1
660568	Maximim allowed VBF File size	1
660570	Software part format	1
660791	CC Subscriber ECU configuration states	1
660792	CC Subscriber ECU in Bulk (unconfigured) state	1
660793	Routines for receiving VCP signal (CC Domain Master excluded)	1
660794	Invalid or unrecognized parameter values	1
660795	Configuration of ECU with updated CCP data	1
660796	Selection of CCPs in VCP signal	1

660797	Car Configuration data storage	1
660798	Car Configuration parameter definition	1
660799	Handling of parameter data	1
660800	ECU configuration after boot-up	1
660801	CC Subscriber ECU DID support	1
660803	Multiple CPU Design	1
660804	Zone Address	1
660805	ECU Addresses	1
660806	Component Selection - CAN Physical Interfaces	1
660807	Missing frame master - CAN	1
660808	No priority inversion	1
660809	Communication voltage range	1
660810	The development frame usage in the development	1
660811	LIN Master	1
660812	LIN Slave	1
660813	NAD addresses (Network ID) for private LIN slaves	1
660814	Diagnostic IP routing algorithm	1
660817	Link Speed & Duplex	1
660818	Eth Configuration parameters Update	1
660819	Mulitcast Addresses	1
660822	VLAN Membership	1
660824	Network management CAN frame identifiers	1
660826	NM frame shall be sending request of wakeup on CAN network	1
660827	Network management mechanism for handling abnormal communication	1
660837	Bootloader CAN busoff handling	1
660838	Bootloader routing - Spare networks	1
660839	Deployment of the J2534 requirements	1
660840	Separate software files	1
660841	In-Place Reconstruction of Delta Files	1
660842	SWDL Performance	1
660843	SWDL interrupt of ECU power removing	1
660844	Communication Gateway Processing	1
660845	Communication Stack Processing Period	1
660846	Communication Stack Processing Period Jitter	1
660847	AUTOSAR version	1
660848	Diagnostic Startup Time	1
660849	Standards for CAN interface	1
660850	The priority between standard documents and this document	1
660851	General requirement to match test case from standard	1
660852	Communication protocol	1
660853	Operating voltage range hysteresis	1
660854	CAN bus level during reset	1
660855	Bus fault reporting	1
660856	Resychronisation strategy	1
660857	Sample mode	1
660858	PLL bypass	1
660859	PLL locking	1
660860	Bit time set up for 500 kbps	1
660861	CAN FD bit register settings for 2 Mbps	1
660862	CAN FD bit register settings for 5 Mbps	1
660863	ECU Conformance	1
660864	Link Establishment Time	1
660865	Link speed and duplex	1
660866	MTU	1
660867	Multicast support	1
660868	Register read out using Diagnostics	1
660869	Register write using Diagnostics	1

660870	Internal MAC Addresses	1
660871	External MAC Addresses	1
660872	VLAN (IEEE 802.1Q)	1
660873	VLAN Identifier (VID)	1
660874	VLAN Priority	1
660875	Drop Eligible Indicator, DEI	1
660876	VLAN configuration	1
660877	Discard Untagged Frames	1
660889	Register Settings Switch	1
660890	Support of AVB/TSN	1
660891	Content of initial Configuration	1
660892	Format of local config content	1
660893	update Local Config File	1
660894	Independent switch operation	1
660895	Initial switch configuration	1
660896	Number of Queues per Port	1
660897	Limitation of switch configuration	1
660898	Return to Factory Port Settings when Link is not Established	1
660899	Throughput	1
660900	Wake Up Time	1
660902	Static VLAN	1
660903	VID Override	1
660904	BMCA, Best Master Clock Algorithm	1
660905	Clock Source Implementation	1
660906	Grandmaster Clock Selection	1
660907	Configuration Update	1
660909	Multiple time domains	1
660912	Timestamp Resolution	1
660913	The priority between standard documents and this specification	1
660914	Node conformance test	1
660915	Bus speed	1
660916	Application timing requirements	1
660917	LIN signal frames may appear arbitrarily	1
660918	Format of the part number data record	1
660919	Format of the serial number	1
660920	PLL used as clock	1
660921	Wakeup strategy	1
660922	Communication persistency	1
660923	ECU Power Loss	1
660924	Recovery from Bus short to battery	1
660925	Recovery from Bus short to ground	1
660926	Recovery from reset	1
660927	Normal battery voltage power operation	1
660928	Low Battery Voltage Operation	1
660929	Operating voltage range hysteresis	1
660930	LIN standard used in master node	1
660931	Header jitter	1
660933	Timeout and sleep commands	1
660934	Local event - master sends the first header	1
660935	Network event - wake up request to first header	1
660936	Power on - master sends the first header	1
660939	LIN standard used in slave node	1
660940	Transmission time	1
660941	Timeout and sleep commands	1
660942	Local event - slave sends the wake up request	1
660943	Network event - wake up request to slave ready to communicate	1
660944	Power on - slave sends the first response	1
660945	Transport and Network layer	1

660946	Precedence of requirements	1
660947	BlockSize parameter non-programming session server side	1
660948	CAN FD BlockSize parameter non-programming session server side	1
660949	BlockSize parameter non-programming session client side	1
660950	BlockSize parameter programming session server side	1
660951	BlockSize parameter programming session client side	1
660952	Separation time (STmin) non-programming session server side	1
660953	Separation time (STmin) non-programming session client side	1
660954	Separation time (STmin) programming session server side	1
660955	Separation time (STmin) programming session client side	1
660956	Separation time between single frames - programming session	1
660957	N_WFTmax value for server side	1
660958	N_WFTmax value for client side	1
660959	Flowcontrol Wait usage	1
660960	N_As timeout in non-programming session	1
660961	N_Ar timeout in non-programming session	1
660962	N_Bs timeout in non-programming session	1
660963	N_Br Performance requirement in non-programming session for FC.CTS and FC.OVFLW	1
660964	N_Br Performance requirement in non-programming session for FC.Wait	1
660965	N_Cs Performance requirement in non-programming session	1
660966	N_Cr timeout in non-programming session	1
660967	N_As timeout in programming session	1
660968	N_Ar timeout in programming session	1
660969	N_Bs timeout in programming session	1
660970	N_Br performance requirement in programming session for FC.CTS and FC.OVFLW	1
660971	N_Br Performance requirement in programming session for FC.Wait	1
660972	N_Cs Performance requirement in programming session	1
660973	N_Cr timeout in programming session	1
660974	Duplex communication	1
660975	Addressing format	1
660976	CAN frame identifier length	1
660977	Supporting functional requests	1
660978	Length of Classic CAN frames	1
660979	Length of CAN FD frames	1
660980	Handling of CAN requests in regards of DLC	1
660981	Unused data in the CAN frame	1
660982	Forward N_Data from each N_PDU to upper layer	1
660983	Concurrent DoIP Sessions	1
660984	Connected with configured AutoIP Address	1
660985	Disconnected with configured AutoIP Address	1
660986	DHCP Host name option with VIN	1
660987	DHCP Host name option with Ethernet MAC	1
660988	DHCP interval timing	1
660989	DHCP Retry	1
660990	Diagnostic Message ACK - Previous data	1
660991	Diagnostic Message NACK - Previous Data	1
660992	Diagnostic message negative acknowledge code 0x06	1
660993	DoIP ACK	1
660994	Asynchronous request-response handling	1
660995	Vehicle Announcement Message Interval	1
660996	Vehicle Announcement Message Number	1
660997	Vehicle Announcement when Connected	1
660998	Vehicle Announcement When No VIN Available	1

660999	Group ID Default Value	1
661000	ISO 13400	1
661001	Power-mode	1
661002	Accept TCP Connection requests for DoIP Messages	1
661003	Keep track of TCP connections	1
661004	Accept DoIP TCP Connection Requests From Already Connected IP ECUs	1
661005	Vehicle Identification Request Message with EID UHWID	1
661006	Specific DoIP payload types	1
661007	Manufacturer specific Further action required information	1
661008	Specific DoIP routing activation request activation types	1
661009	Specific DoIP routing activation response codes	1
661010	Client Behaviour During DoIP Session	1
661011	DoIP header	1
661012	DoIP Ports for Internal Communication	1
661013	Source Port for Internal Communication	1
661014	DoIP Support in Internal ECUs	1
661015	Lost TCP connection for DoIP Messages	1
661016	Open TCP Connection for DoIP Messages	1
661017	Reconnect With Same IP and Port Number	1
661018	Close TCP port on connection reset	1
661019	ARP Implementation	1
661020	ARP Cache entries	1
661021	ARP Announcements	1
661022	ARP Cache timeout	1
661023	Local Configurations	1
661024	ICMP support	1
661025	Internet Protocol Version	1
661026	IP fragmentation	1
661027	Reassembling fragmented packets	1
661029	Time-to-Live	1
661030	Internal IP Addresses	1
661032	IP Network Prioritization	1
661033	TCP Implementation	1
661034	TCP Keep-Alives	1
661035	Congestion control	1
661036	Congestion control strategies	1
661037	Graceful TCP Connection Shutdown on ECU Reset	1
661038	Initial TCP Window Size	1
661039	Internal TCP Connection Establishment Time	1
661040	UDP Implementation	1
661041	UDP Checksum	1
661042	Broadcasts	1
661139	Dual Stack	1
661140	Translation	1
661143	IP Broadcast Address Limitation	1
661144	IP NAT	1
661146	NAT support for SIP	1
661147	Port Forwarding	1
661148	DHCP - IP Reservation Table	1
661149	IP Implementation	1
661150	IP Multicast	1
661151	IP Multicasting Protocol	1
661152	IPv4 router implementation	1
661153	Routing Table Content	1
661154	Static Routing	1
661155	Throughput	1
661156	Fragmented ICMP	1

661159	IP Broadcast storms	1
661160	MAC Filtering	1
661167	Link Monitoring	1
661168	Link Monitoring - ECU reboot	1
661169	Link Monitoring - logging	1
661170	Link Monitoring Accessibility	1
661171	Link Monitoring Activation	1
661175	IP Fragmentation	1
661176	IP Option	1
661178	ICMP Unreachable	1
661186	Local configurations	1
661188	Alignment	1
661189	Deviations from AUTOSAR	1
661190	Endianess	1
661191	Message sequences in parallel	1
661194	SOME IP AUTOSAR Transformer Specification	1
661195	SOME IP Notification messaging	1
661196	SOME IP standard implementation	1
661197	SOME IP Header Validation	1
661198	Messages encapsulate	1
661199	Message Type	1
661200	Session Handling for Event	1
661201	Session handling for Sender-Receiver	1
661202	Event message missing	1
661203	The type of Datatype	1
661209	Serialization-Arrays (fixed length)	1
661210	Session handling for Client-Server	1
661212	TCP connection establishment	1
661213	TCP connection termination	1
661214	TCP connections	1
661215	TCP lost connection	1
661216	Magic cookie	1
661217	TCP Common requirements	1
661219	SOME IP over UDP max size	1
661220	UDP fragmentation	1
661221	UDP common requirements	1
661222	Return Codes in the Response Messages	1
661224	Explicit Error Messages	1
661225	Error handling	1
661229	Service Discovery protocol	1
661230	Service Discovery Message	1
661231	Service Discovery Messages byte order	1
661232	Entry and Option setting	1
661233	SOME/IP-SD Message Counter	1
661234	Client time parameters	1
661235	Server time parameters	1
661236	FindService sends in the Repetition phase	1
661237	Client wait for answer when not receiving OfferService	1
661238	Error handling for FindService	1
661239	FindService messages send to multicast MAC address	1
661240	FindService messages send to multicast IP address	1
661241	OfferService messages send to multicast MAC address	1
661242	OfferService messages send to multicast IP address	1
661243	Service sending and receiving when optimization implementation	1
661244	StopOfferService messages send to multicast MAC address	1
661245	StopOfferService messages send to multicast IP address	1
661246	StopOfferService message send condition	1

661247	Eventgroup messages send to MAC address	1
661248	Eventgroup messages send to IP address	1
661249	Always retrieve information from an eventgroup after power on	1
661250	Meet condition to retrieve the information of an eventgroup	1
661251	SubscribeEventgroup negative response error handling	1
661252	Initial event application condition	1
661253	the condition of StopSubscribeEventgroup messages sending	1
661254	SubscribeEventgroup message resend mechanism	1
661255	Restart behavior judgment	1
661256	Support of Mixers	1
661257	Support of Translators	1
661258	Media clock synchronization	1
661259	SSRC	1
661260	Support of RTCP	1
661261	Audio format	1
661262	Video format	1
661263	Audio/video container format	1
661264	Underlying transport protocol	1
661265	Port assignment	1
661266	Stream destination addresses	1
661267	Store-and-forward mode	1
661268	Jumbo frames	1
661269	Static ARL entries	1
661270	Shared VLAN learning	1
661271	Aging time for ARL table	1
661274	Mapping between PCP and Traffic Classes	1
661277	Double-tagging	1
661278	Mapping between use cases and VLAN ID	1
661279	Retagging and untagging rules for OBD TX port	1
661280	VLAN priority	1
661281	VLAN DEI	1
661282	Entropy for random numbers	1
661283	Random number generator algorithms	1
661284	Distribution of the Fixed Bytes	1
661285	Fixed Bytes default value	1
661286	Key Storage	1
661288	SecurityAccess (27) - time delay after false securityAccess attempts	1
661289	Seed - Random Number Generation	1
661291	Security Access Algorithm Description	1
661292	Default values - Delay timers and counters for false attempts	1
661293	Security Level usage	1
661295	Format of the Verification Block Table	1
661296	Verification Block Table Format Identifier - Definition	1
661297	Verification Block Table - Input order to Hash function per data block	1
661298	Verification Block Table Header Identifiers	1
661299	Verification Block Root Hash	1
661300	Verification Block Table on unprocessed data	1
661301	Verification Block Table per Software Part	1
661302	Verification Block Table in a padded data block	1
661303	Signing function - Integrity Check	1
661304	Sign Software Part - Private Key syntax	1
661305	Signing function - Generate signature	1
661306	Append signature to software part	1
661307	Sign and verify unprocessed data	1
661308	Sign a delta file	1
661309	Production Key programming	1

661310	Software Authentication Test and Development Key	1
661311	Software Authentication Test and Development Key for RSA3072	1
661312	Software AUthentication Test and Development Key for RSA4096	1
661313	Public Key Types	1
661314	Signing Method - Variability control	1
661315	Default Signing Method	1
661316	Signing Method #2	1
661317	Signing Method SHA256_RSA2048	1
661320	Signing Method SHA256_RSA3072	1
661323	Software authentication security classes	1
661324	Software Settings - Definition	1
661325	Software Settings File - valid part numbers	1
661335	Random Number Generation	1
661336	Client Challenge	1
661337	Server Challenge	1
661338	Storage of Client Certificate	1
661339	Parse and verify Client Certificate	1
661340	Check Client Certificate for revocation	1
661341	Providing Server Certificate	1
661342	Server Certification verification	1
661343	Proof of ownership server	1
661344	Proof of ownership client in unidirectional authentication	1
661345	Proof of ownership client in bidirectional authentication	1
661346	Proof of ownership client verification	1
661347	Proof of ownership server verification	1
661348	Secure diagnostic communciation	1
661349	Ephemeral Private Key	1
661350	Server Ephemeral Public Key	1
661351	Client Ephemeral Public Key	1
661352	Negotiated shared key	1
661353	Session Key Information	1
661354	Default Signing Method	1
661355	Signing Method SHA256_ECDSA - Hash function	1
661356	Signing Method SHA256_ECDSA - Elliptic Curve	1
661357	Signing Method SHA256_ECDSA - key size	1
661358	Default Key Exchange Method	1
661359	Key Exchange Method ECDHE - Elliptic Curve	1
661360	Key Exchange Method ECDHE - Negotiated shared key	1
661361	Default Encryption Method	1
661362	Encryption Method AES_GCM - Key and IV size	1
661363	Encryption Method AES_GCM - addtional authenticated data	1
661364	Encryption Method AES_GCM - Key and IV	1
661365	Encryption Method AES_GCM - Data arrangement	1
661366	Default values - Delay timers and counters for false attempts	1
661367	Flow control mechanism for diagnostic messages	1
661368	Gatewaying-on-the-fly	1
661369	Generic gateway latency requirement	1
661370	Multiple CPU design in the GW ECU	1
661371	No re-sending of diagnostic requests	1
661372	No re-sending of diagnostic responses	1
661373	Release buffers on communication errors	1
661374	Routing table	1
661375	Separate buffers for requests and responses	1
661376	Separate CCs for request and responses	1
661377	Session layer time-out handling	1
661378	Transfer all functional request before a reset of gateway	1

661379	Transport layer routing	1
661380	Allocation of buffers from buffer pools	1
661381	Static allocated buffers vs dynamic allocated buffers	1
661382	Keep chronologic order of parallel physical messages	1
661383	Parallel physical requests to different ECUs	1
661384	Wait for available communication channel	1
661385	Functional requests transmit (Tx) com. channel(s)	1
661386	Internal functional request handling	1
661387	Prioritize functional request	1
661388	Protection of routing table	1
661389	Static routing rules	1
661391	Static routing configuration	1
661392	Keep chronologial order for queued messages	1
661393	Queued physical request to one ECU	1
661394	Supported request format from external IP network	1
661395	Response format to external IP	1
661396	Diagnostic request format on internal IP network	1
661397	Diagnostic response format on internal IP	1
661398	Diagnostic request format on CAN	1
661399	Diagnostic response format on CAN	1
661400	External IP format to internal IP format in request	1
661401	Internal IP to external IP address in response	1
661402	Handling functional requests on IP	1
661404	CAN format to external IP format in response	1
661405	CAN response add IP address in external IP	1
661406	Transmit(Tx) CCs on Network (CAN(FD))	1
661407	Receive(Rx) CCs on Network (CAN(FD))	1
661408	Latency IP to CAN, GW_IP-CAN_L1	1
661409	Latency CAN to IP, GW_CAN-IP_L2	1
661410	Large buffers for request	1
661411	Large buffers for response	1
661413	Minimum number of buffers for responses	1
661414	Minimum RAM for request buffers	1
661415	Minimum RAM for response buffers	1
661416	Large buffers for request	1
661417	Large buffers for response	1
661418	Minimum number of buffers for requests	1
661419	Minimum RAM for request buffers	1
661420	Minimum RAM for response buffers	1
661421	Minmum number of buffers for responses	1
661422	J2534 CAN Identifier format request	1
661423	J2534 CAN Identifier format response	1
661424	Gateway timing	1
661425	Message based gateway	1
661426	Differentiate response to external network	1
661427	Gateway mode	1
661428	Generic gateway requirements	1
661429	Internal IP format to Sub IP format in request	1
661430	Large buffers for request	1
661431	Large buffers for response	1
661432	Minimum number of buffers for requests	1
661433	Minimum number of buffers for responses	1
661434	Minimum RAM for request buffers	1
661435	Minimum RAM for response buffers	1
661436	Large buffers for request	1
661437	Large buffers for response	1
661438	Minimum number of buffers for requests	1
661439	Minimum number of buffers for responses	1

661440	Minimum RAM for request buffers	1
661441	Minimum RAM for response buffers	1
661442	Support for queued request	1
661443	Receive(Rx) CCs on Backbone Network	1
661444	Transmit(Tx) CCs on Backbone Network	1
661445	Receive(Rx) CCs on Zone network	1
661446	Transmit(Tx) CCs on Zone network	1
661447	Latency CAN Backbone to CAN Zone, GW_Backbone_CAN-Zone_CAN_L	1
661448	Latency CAN Zone to Backbone CAN, GW_Backbone_CAN-Zone_CAN_L	1
661462	Firewall CANX-CAN11 Minimum buffer structure for requests	1
661463	Firewall CANX-CAN11 Minimum buffer structure for responses	1
661464	Firewall CANX-CAN11 Client side filter	1
661465	Firewall CANX-CAN11 Flow Control parameters	1
661466	Firewall CANX-CAN11 Timing parameter values	1
661467	Firewall CANX-CAN11 latency	1
661468	Firewall CANX-CAN11 Transport layer	1
661469	Firewall CAN11-CAN11 disable functionality	1
661470	Firewall CAN11-CAN11 Server side filter	1
661471	Conversion from TA to NAD to SA	1
661472	Physical request NAD validation	1
661473	LIN master routing functional requests	1
661474	LIN master time to poll time for functional addressed responses	1
661475	LIN master to prioritize routing functionally addressed requests	1
661476	LIN master time out not receiving first NRC 0x78	1
661477	LIN master time out not receiving consecutive 0x78 NRCs	1
661478	Parallel requests on LIN	1
661479	Concurrent LIN master and LIN slave diagnostic requests	1
661480	Time routing incoming Diagnostic request	1
661481	X-LIN DIAG Minimum RAM for request buffers	1
661482	X-LIN DIAG Minimum RAM for response buffers	1
661483	LIN scheduling method	1
661484	LIN scheduling interval	1
661485	X-LIN SWDL Minimum RAM for request buffers	1
661486	X-LIN SWDL Support for queued request	1
661487	X-LIN SWDL Minimum RAM for response buffers	1
661611	DTC Global Snapshot Data records defined in GMRDB	1
661613	Reporting of DTC information	1
661614	DTCs defined by GMRDB	1
661615	DTC Failure Type Byte	1
661616	Emissions related DTCs defined by SAEJ2012	1
661617	Non emissions related DTCs defined by SAEJ2012	1
661618	DTCs for secondary faults caused by data from other ECU indicated as invalid	1
661619	DTCs for secondary faults caused by data from other ECU not indicated as invalid	1
661620	DTCs for faults caused by the ECU operating environment or by customer	1
661621	DTCs that are needed only during the development of the ECU - implementer specified	1
661622	DTCs that are needed only during the development of the ECU	1
661623	ECU operation cycle	1
661624	Calibration of DTC test sample failed	1
661625	Calibration of DTC test sample passed	1
661626	Calibration of unconfirmedDTCLimit	1
661627	Calibration of DTCTestFailedLimit	1
661628	Calibration of DTCTestPassedLimit	1

661629	Calibration of confirmedDTCLimit	1
661630	Calibration of agedDTCLimit	1
661631	Test Run Criteria - Updating the DTC Fault Detection Counter	1
661632	Generic Test Run Criteria - enable criteria	1
661633	Generic Test Run Criteria - low power inhibit criteria	1
661634	Generic Test Run Criteria - usage mode transition inhibit criteria	1
661635	Generic Test Run Criteria - car modes inhibit criteria	1
661636	Generic Test Run Criteria - low voltage inhibit criteria	1
661637	Generic Test Run Criteria - car mode transition inhibit criteria	1
661638	DTC values	1
661639	Failure type value for DTCs	1
661640	Test Run Criteria for fault detection in external circuits	1
661641	Detection of faults for switch inputs	1
661642	Test period time for stuck at faults for switch inputs	1
661643	Test period time for other faults for switch inputs	1
661644	Base DTC value for switch inputs	1
661645	Failure type value for stuck at faults for switch inputs	1
661646	Detection of short circuit and open circuit faults in analog inputs	1
661647	Test period time for short circuit and open circuit faults in analog inputs	1
661648	Base DTC value for short circuit to battery or open circuit faults in an...	1
661649	Failure type value for short circuit to battery or open circuit faults i...	1
661650	Failure type value for short circuit to ground faults in analog inputs	1
661651	Detection of other faults for analog inputs	1
661652	Test period time for other faults for analog inputs	1
661653	Detection of short circuit and open circuit faults in FM, PCM, PWM etc. ...	1
661654	Test period time for short circuit and open circuit faults in FM, PCM, P...	1
661655	Base DTC value for short circuit and open circuit faults in FM, PCM, PWM...	1
661656	Failure type value for short circuit and open circuit faults in FM, PCM,...	1
661657	Detection of other faults for FM, PCM, PWM etc. inputs	1
661658	Test period time for other faults for FM, PCM, PWM etc. inputs	1
661659	Detection of short circuit and open circuit faults in low side driver ou...	1
661660	Test period time for short circuit and open circuit faults in low side d...	1
661661	Test Run Criteria for short circuit to battery faults in low side driver...	1
661662	Test Run Criteria for short circuit to ground or open circuit faults in ...	1
661663	Base DTC value for low side driver outputs	1
661664	Failure type value for short circuit to ground or open circuit faults in...	1
661665	Failure type value for short circuit to battery faults in low side drive...	1
661666	Detection of other faults for low side driver outputs	1
661667	Test period time for other faults for low side driver outputs	1
661668	Detection of short circuit and open circuit faults in high side driver o...	1
661669	Test period time for short circuit and open circuit faults in high side ...	1

661670	Test Run Criteria for short circuit to battery or open circuit faults in high side driver outputs	1
661671	Base DTC value for faults in high side driver outputs	1
661672	Failure type value for short circuit to battery or open circuit faults in high side driver outputs	1
661673	Failure type value for short circuit to ground faults in high side drive...	1
661674	Test Run Criteria for short circuit to ground faults in high side driver...	1
661675	Detection of other faults for high side driver outputs	1
661676	Test period time for other faults for high side driver outputs	1
661680	Detection of other external electrical circuit faults	1
661681	Test period time for detection of other external electrical circuit faults	1
661682	DTC value for other external electrical circuit faults	1
661692	Detection of internal faults not on the main circuit board	1
661693	Test Run Criteria for fault detection in internal circuits	1
661694	Detection of data integrity fault in non volatile memory	1
661695	Test Run Criteria and Test Period time for detection of data integrity f...	1
661696	Base DTC value for detection of data integrity fault in non volatile memory	1
661697	Failure type value for detection of data integrity fault in non volatile...	1
661698	Detection of data integrity fault in volatile memory	1
661699	Test Run Criteria and Test Period time for detection of data integrity f...	1
661700	Base DTC value for detection of data integrity fault in volatile memory	1
661701	Failure type value for detection of data integrity fault in volatile memory	1
661702	Detection of internal device communication error	1
661703	Test Run Criteria for detection of internal device communication error	1
661704	DTC value for detection of internal device communication error	1
661705	Failure type value for detection of internal device communication error	1
661706	Detection of programme execution fault	1
661707	Test Run Criteria and Test period time for detection of programme execution fault	1
661708	DTC value for detection of programme execution fault	1
661709	Failure type value for detection of programme execution fault	1
661710	Detection of internally connected actuator fault	1
661711	Test period time for detection of internally connected actuator fault	1
661712	Base DTC value for detection of internally connected actuator fault	1
661713	Failure type value for detection of internally connected actuator fault	1
661714	Detection of internally connected sensor fault	1
661715	Test period time for detection of internally connected sensor fault	1
661716	Base DTC value for detection of internally connected sensor fault	1
661717	Failure type value for detection of internally connected sensor fault	1
661718	Detection of faults in other internal parts	1
661719	Test period time for detection of faults in other internal parts	1

661720	Base DTC value for detection of faults in other internal parts	1
661721	Failure type value for detection of faults in other internal parts	1
661722	Test Run Criteria for detection of faults inferred from system behaviour	1
661723	Detection of failure to react after event	1
661724	Test Run Criteria for detection of failure to react after event	1
661725	Test period time for detection of failure to react after event	1
661726	Failure type value for detection of failure to react after event	1
661727	Base DTC value for detection of failure to react after event	1
661728	Comparison of system behaviour with predictive model	1
661729	Test Run Criteria for comparison of system behaviour with predictive model	1
661730	Test period time for comparison of system behaviour with predictive model	1
661731	Failure type value for comparison of system behaviour with predictive model	1
661732	Base DTC value for comparison of system behaviour with predictive model	1
661733	Detection of closed loop control system out of control	1
661734	Test Run Criteria for detection of closed loop control system out of con...	1
661735	Test period time for detection of closed loop control system out of control	1
661736	Failure type value for detection of closed loop control system out of co...	1
661737	Base DTC value for detection of closed loop control system out of control	1
661738	Detection of system adaptation out of range	1
661739	Test Run Criteria for detection of system adaptation out of range	1
661740	Failure type value for detection of system adaptation out of range	1
661741	Base DTC value for detection of system adaptation out of range	1
661742	Detection of other faults inferred from system behaviour	1
661743	Test period time for detection of other faults inferred from system beha...	1
661744	Base DTC value for detection of other faults inferred from system behaviour	1
661745	Failure type value for detection of other faults inferred from system be...	1
661753	Detection of software incompatibility with another ECU	1
661754	Base DTC value for detection of software incompatibility with another ECU	1
661755	Failure type value for detection of software incompatibility with anothe...	1
661756	Detection of suppler specific software installed in ECU	1
661757	Clearing of DTC for detection of suppler software installed in ECU.	1
661780	Detection of other ECU not operating normally	1
661781	Test Run Criteria for detection of other ECU not operating normally	1
661782	Base DTC value for detection of other ECU not operating normally	1
661783	Failure type value for detection of other ECU not operating normally	1
661784	Root DTC for ECU not operating normally	1
661785	Detection of other ECU sending data tagged as invalid	1
661786	Test Run Criteria for detection of other ECU sending data tagged as invalid	1

661787	Base DTC value for detection of other ECU sending data tagged as invalid	1
661788	Failure type value for detection of other ECU sending data tagged as inv...	1
661789	Root DTC for ECU sending data tagged as invalid	1
661790	Detection of other ECU sending faulty data	1
661791	Test Run Criteria for detection of other ECU sending faulty data	1
661792	Base DTC value for detection of other ECU sending faulty data	1
661793	Failure type value for detection of other ECU sending faulty data	1
661794	Detection of corrupt data received from other ECU by rolling counter	1
661795	Test Run Criteria for detection of corrupt data received from other ECU by rolling counter	1
661796	Base DTC value for detection of corrupt data received from other ECU by rolling counter	1
661797	Failure type value for detection of corrupt data received from other ECU by rolling counter	1
661798	Detection of corrupt data received from other ECU by end-to-end checksum	1
661799	Test Run Criteria for detection of corrupt data received from other ECU by end-to-end checksum	1
661800	Base DTC value for detection of corrupt data received from other ECU by end-to-end checksum	1
661801	Failure type value for detection of corrupt data received from other ECU by end-to-end checksum	1
661802	Detection of corrupt data received from other ECU by up-date-bit	1
661803	Test Run Criteria for detection of corrupt data received from other ECU by up-date bit	1
661804	Base DTC value for detection of corrupt data received from other ECU by up-date-bit	1
661805	Failure type value for detection of corrupt data received from other ECU by up-date bit	1
661806	Test Run Criteria for detection of faults caused by the ECU operating environment or customer action	1
661807	Test period time for detection of faults caused by ambient temperature	1
661808	Base DTC value for detection of faults caused by ambient temperature too ...	1
661809	Base DTC value for detection of faults caused by ambient temperature too low	1
661810	Failure type value for detection of faults caused by ambient temperature	1
661811	Detection of events caused by customer operating vehicle or vehicle equi...	1
661812	Test period time for detection of events caused by customer operating ve...	1
661813	Base DTC value for detection of events caused by customer operating vehi...	1
661814	Failure type value for detection of events caused by customer operating ...	1
661835	Detection of events caused by other condition that is out of range for f...	1
661836	Test period time for detection of events caused by other condition that is out of range for full ope	1
661837	Base DTC value for detection of events caused by other condition that is out of range for full opera	1
661838	Failure type value for detection of events caused by other	1

	condition tha...	
661839	Base DTC value for detection of other faults inferred from system behaviour	1
661840	Detection of events associated with driver indications	1
661843	Support for DTC status bit testFailed	1
661844	Support for DTC status bit testFailedThisOperationCycle	1
661845	Support for DTC status bit pendingDTC	1
661846	Support for DTC status bit confirmedDTC	1
661847	Support for DTC status bit testNotCompletedSinceLast-Clear	1
661848	Support for DTC status bit testFailedSinceLastClear	1
661849	Support for DTC status bit testNotCompletedThis-OperationCycle	1
661850	Support for DTC status bit warningIndicatorRequested	1
661851	Data records not included in local snapshot data	1
661852	Order of multple samples of data records in global and local snapshot data records	1
661853	Sampling criteria for DTCSnapshotRecordNumber 0x20, single sample	1
661854	Sampling criteria for DTCSnapshotRecordNumber 0x21, single sample	1
661855	Sampling criteria for DTCSnapshotRecordNumber 0x30 to 0x3F, multiple samples	1
661856	Sampling criteria for DTCSnapshotRecordNumber 0x40 to 0x4F, single sample	1
661857	Sampling criteria for DTCSnapshotRecordNumber 0x50 to 0x5F, other samples	1
661858	Supported data records in global DTC snapshot data	1
661859	Definition of global real time data record	1
661860	Maintaining global real time after power loss	1
661861	Definition of total distance data record	1
661862	Definition of vehicle battery voltage data record	1
661863	Distribution of global DTC snapshot data	1
661864	Storage of global DTC snapshot data when not available	1
661865	Support for local DTC snapshot data	1
661866	Storage of local DTC snapshot data when not available	1
661867	Support for Operation Cycle Counter #1	1
661868	Support for Operation Cycle Counter #2	1
661869	Support for Operation Cycle Counter #3	1
661870	Support for Operation Cycle Counter #4	1
661871	Support for Operation Cycle Counter #5	1
661872	Support for Operation Cycle Counter #6	1
661873	Support for Operation Cycle Counter #7	1
661874	Support for Fault Detection Counter #10	1
661875	Support for Fault Detection Counter #11	1
661876	Support for Fault Detection Counter #12	1
661877	Support for DTC time stamp #20	1
661878	Support for DTC time stamp #21	1
661879	Support for DTC status indicator #30	1
661880	Support for DTC status indicator #30, UnconfirmedDTC	1
661881	Support for DTC status indicator #30, UnconfirmedDTCThisOperationCyle	1
661882	Support for DTC status indicator #30, UnconfirmedDTCSinceLastClear	1
661883	Support for DTC status indicator #30, AgedDTC.	1
661884	Support for DTC status indicator #30, SymptomSinceLastClear	1
661885	Support for DTC status indicator #30, WarningIndicatorRequestedSinceLastClear	1
661886	Support for DTC status indicator #30, EmissionRelatedDTC	1

661887	Support for DTC status indicator #30, testFailedSinceLastClear/Aged	1
661888	Storage of DTC information in short term memory	1
661889	DTC storage time	1
661890	Storage of DTC status bits no 0, 2, 3, 4 and 5 in long term memory	1
661891	Storage of DTC snapshot data in long term memory	1
661892	Storage of DTC operation cycle counters 1, 2, 3, 4, 6 and 7 in long term memory	1
661893	Storage of DTC operation cycle counter 5 in long term memory	1
661894	Storage of DTC fault detection counters 12 in long term memory	1
661895	Storage of DTC time stamp in long term memory	1
661896	Storage of DTC status indicator bits no 0, 2-7 in long term memory	1
661897	Storage of DTC status bits no 1, 6 and 7 in short term memory	1
661898	Storage of DTC fault detection counters 10 and 11 in short term memory	1
661899	Storage of DTC status indicator bit no 1 in short term memory	1
661900	Transfer of DTC information to long term memory	1
661901	Clear DTC information by diagnostic service	1
661902	Clear DTC information when a DTC is aged	1
661903	Clear DTC information when an aged DTC reoccurs	1
661904	Clear DTC information when the long term memory overflows	1
662014	Detection of transmit error on private ECU network	1
662015	Test Run Criteria for detection of transmit error on private ECU network	1
662016	Failure type value for transmit error on private ECU network	1
662017	Detection of unexpected transmissions on private ECU network	1
662018	Test period time for detection of unexpected transmissions on private EC...	1
662019	Failure type value for unexpected transmissions on private ECU network	1
662020	Detection of CS- or CRC-error on private ECU network	1
662021	Test Run Criteria for CS- or CRC-error on private ECU network	1
662022	Failure type value for CS- or CRC-error on private ECU network	1
662023	Detection of incorrect timing and format on private ECU network	1
662024	Test Run Criteria for incorrect timing and format on private ECU network	1
662025	Failure type value for incorrect timing and format on private ECU network	1
662026	Detection of data not updated by transmitter on private ECU network	1
662027	Test Run Criteria for data not updated by transmitter on private ECU net...	1
662028	type value for data not updated by transmitter on private ECU network	1
662029	Detection of missed deadline for message on private ECU network	1
662030	Test Run Criteria for missed deadline for message on private ECU network	1
662031	Failure type value for missed deadline for message on private ECU network	1
662032	Detection of physical layer faults on private ECU network	1
662033	Test period time for physical layer faults on private ECU network	1
662034	Failure type value for physical layer faults on private ECU network	1

662035	Detection of other faults on private ECU network	1
662036	Test period time for detection of other faults on private ECU network	1
662082	Program and read of the public key	1
662496	P2Server_max - non programming session	1
662497	P2Server_max - ProgrammingSession	1
662498	P2server_min	1
662499	P2(star)Server_max	1
662500	P2(star)Server_min	1
662501	Server support of queued requests in programming session	1
662502	Start of P2Server for queued request in programming session	1
662503	Order of responses in programming session	1
662504	Server behavior for additional client requests in programming session	1
662505	P4Server_max definition	1
662506	P4Server_max equal to P2Server_max	1
662507	(delta)P2min	1
662508	(delta)P2max	1
662509	(delta)P2External_Response_min	1
662510	(delta)P2External_Response_max	1
662511	Dynamic selection of (delta)P2Vehicle based on message length	1
662512	(delta)P2Vehicle_singleframe_min	1
662513	(delta)P2Vehicle_singleframe_max in programming session	1
662514	(delta)P2Vehicle_singleframe_max in other than programming session	1
662515	(delta)P2Vehicle_multiframe_min	1
662516	(delta)P2Vehicle_multiframe_max in programming session	1
662517	(delta)P2Vehicle_multiframe_max in other than programming session	1
662518	(delta)P2Vehicle_multichannel_min	1
662519	(delta)P2Vehicle_multichannel_max in programming session	1
662520	(delta)P2Vehicle_multichannel_max in other than programming session	1
662521	P6Client_min	1
662522	P6Client_max	1
662523	P6(star)Client_min	1
662524	P6(star)Client_max	1
662525	P3Client_Phys_min	1
662526	P3Client_Phys_max	1
662527	P3Client_Func_min	1
662528	P3Client_Func_max	1
662529	Client shall wait for the DoIP Ack or Nack before queue any request	1
662530	Client sending additional requests to same server in programming session	1
662531	P6Client and P6(star)Client behavior during queued request	1
662532	Application layer	1
662533	Application bootloader requirement	1
662534	Diagnostic Session Layer	1
662535	Addressing USDT frames	1
662536	Transport_Network layer	1
662537	CAN data link layer	1
662538	CAN physical layer	1
662539	Diagnostic services	1
662540	Application diagnostic data	1
662674	ReadDataByIdentifier - Active diagnostic session	1
662675	ReadDataByIdentifier - Application Diagnostic Database Part Number	1

662676	ReadDataByIdentifier - Complete ECU Part Serial Number	1
662677	ReadDataByIdentifier - ECU Core Assembly Part Number	1
662678	ReadDataByIdentifier - ECU Delivery Assembly Part Number	1
662679	ReadDataByIdentifier - ECU Serial Number	1
662680	ReadDataByIdentifier - ECU Software Part Number(s)	1
662681	RoutineControl - Check Programming Pre-conditions	1
662682	DiagnosticSessionControl DiagnosticSessionType-programming session	1
662683	ReadDataByIdentifier	1
662684	RoutineControl	1
662685	Diagnostic gateway	1
662686	Diagnostic routing or switching	1
662687	ReadDataByPeriodicIdentifiers Response Message Type	1
662701	Application layer	1
662708	Edge Node - Check external Ethernet link	1
662709	Edge Node - Detection of WLAN Link	1
662710	Edge Node - Ethernet link is detected	1
662711	Edge Node - Ethernet link not detected	1
662712	Edge Node - External Ethernet cable rejects WLAN link	1
662713	Edge Node - IP address assignment	1
662714	Edge Node - Is DoIP session ended?	1
662715	Edge Node - Physical link connection and DoIP connection	1
662716	Edge Node - Vehicle announcement messages	1
662717	Edge Node - WLAN link does not exists	1
662718	Edge Node - WLAN link exists	1
662731	Base DTC value for LIN slave node failure	1
662746	DiagnosticSessionControl diagnosticSessionType programmingSession	1
662747	LIN slave ReadDataByIdentifier ECU serial number	1
662748	LIN slave ReadDataByIdentifier Application Diagnostic Database Part Number	1
662749	LIN slave ReadDataByIdentifier ECU Core Assembly Part Number	1
662750	LIN slave ReadDataByIdentifier ECU Delivery Assembly Part Number	1
662751	LIN slave ReadDataByIdentifier ECU Software Part Number(s)	1
662752	LIN slave ReadDataByIdentifier Complete ECU Part/Serial Number	1
662753	LIN slave ReadDataByIdentifier Active diagnostic session	1
662754	LIN slave supported NAD types	1
662755	Non-volatile memory for software storage	1
662756	Power Consumption	1
662757	Internal data transfer speed	1
662758	Handling of distribution relays	1
662759	Calibration and Customer data	1
662760	Two level bootloader	1
662761	PBL can only write data to volatile memory	1
662762	PBL in non-volatile memory	1
662763	Pre-condition state of the ECUs IO	1
662764	Protected boot sector	1
662765	Primary bootloader diagnostic data	1
662766	Primary bootloader diagnostic services	1
662767	Prevent the possibility to download the SBL when already activated	1
662768	SBL in volatile memory	1
662769	Support of erase and write data to non-volatile memory	1
662770	The SBL shall include all PBL functionality	1
662771	Secondary bootloader diagnostic data	1

662772	Secondary bootloader diagnostic services	1
662773	Interrupted communication to the secondary processor	1
662774	Programming sequence for secondary processors	1
662775	ECUs with multiple networks	1
662776	Gateway requirement	1
662777	Multiple networks communication request	1
662778	Interrupted communication to private ECU	1
662779	Programming sequence for private ECU	1
662780	Read information from a private ECU	1
662781	Support for (un)compressed data	1
662782	Delta encoding	1
662783	Disable monitoring and logging of DTCs	1
662784	Suspend non diagnostic communication	1
662785	ProgSignature - CLEAR	1
662786	ProgSignature - SET	1
662787	Time window (Timeout_Prog)	1
662788	Programming of Logical Blocks	1
662789	Software parts per logical block	1
662790	Number of logical blocks	1
662791	Logical Block - Properties	1
662792	Logical Block Declaration	1
662793	Storage of the ECU Software Structure (ESS)	1
662794	Reprogrammable ECU Software Structure (ESS)	1
662795	Immediately apply ESS update	1
662796	Restricted software download when no valid ESS stored	1
662797	Validation of the ECU Software Structure	1
662798	Erase memory when ECU Software Structure (ESS) is updated	1
662799	ECU Software Structure Declaration	1
662800	Root ECU Software Structure Declaration	1
662801	Erase Status Information	1
662802	Allocation of the Verification Block Table	1
662802	Allocation of the Verification Block Table	1
662803	Verification Block Table - Reserve space at application build process	1
662804	Conditions for a logical block to be invalid	1
662805	Condition for a logical block to be valid	1
662806	Validity Status Information - Properties	1
662807	Public Key storage	1
662808	Re-programmable initial development key	1
662809	Erase Identifier - restrictions	1
662810	Request Download - restrictions on number of addressed logical blocks	1
662811	Check Memory - Return values	1
662812	Check Memory - verification of programmed data	1
662813	ECU Reset - Application Start	1
662814	Secondary Bootloader - Condition for activation	1
662815	Secondary Bootloader Verification of unused data	1
662816	RequestUpload - Restricted use	1
662817	Multiple processors - verification principle	1
662818	Multiple processors - Main processor SBL verification	1
662819	Multiple processors - Secondary processor SBL verification	1
662820	Delta encoding - Verification	1
662821	Software authentication signing method and algorithms	1
662822	Allocation of the Authenticity Verification function	1
662823	Authenticity verification at software download (Program mode)	1
662824	Authenticity verification at every start-up	1
662825	Authenticity verification at run time (main application running)	1
662826	Debug tools and ports	1

662827	Validation of ECU Software(s)	1
662828	Complete and Compatible sequence diagram	1
662829	Complete and Compatible - Definition of being Complete	1
662830	Complete and Compatible - Complete function	1
662831	Complete&Compatible - Definition of being Compatible	1
662832	Complete and Compatible - Compatible function	1
662833	CompleteCompatibleFunction()	1
662834	CompleteCompatibleFunction() return value	1
662835	Maximum time to perform the Complete&Compatible validation	1
662836	CompleteCompatibleFunction - error handling	1
662837	File download order	1
662838	Separate memory sectors	1
662839	Writing of received data	1
662840	Pre-calculated hash	1
662844	Complete & Compatible block location	1
662845	Complete & Compatible block structure	1
662846	StartCompleteCompatibleSignature	1
662847	ApplicationStartAdress	1
662848	CompleteCompatibleFunctionAddress	1
662849	EndCompleteCompatibleSignatureAddress	1
662850	CompleteCompatibleFunction()	1
662851	CompleteCompatibleFunction() return value	1
662852	CompleteCompatibleFunction() sequence diagram	1
662853	Maximum time to execute the CompleteCompatibleFunction()	1
662854	EndCompleteCompatibleSignature	1
662855	Complete & Compatible flow chart diagram	1
662858	The storage partition of software file	1
662859	Correct target software installation	1
662860	Target data file parsing method	1
662861	The compression requirements for file system ECU	1
662862	After-sales assignment activation	1
662863	After-sales assignment Identifier	1
662864	After-sales assignment handling	1
662865	After-sales assignment push - routine accomplish	1
662866	Integrity and authentication for data files	1
662867	After-sales assignment lifetime	1
662868	The end of After-sales assignment	1
662869	File transfer	1
662870	File transfer - URL Information	1
662871	File transfer routine- Download accomplish	1
662872	File transfer routine - Download abort	1
662873	File transfer routine - Download progress	1
662874	File transfer control - Failure handling	1
662875	Digital Envelope Transfer	1
662876	Digital Envelope Transfer routine - Transfer accomplish	1
662877	Digital Envelope Transfer routine - Transfer abort	1
662878	Use of Digital Envelope	1
662879	Clearing of Digital Envelope	1
662880	Digital Envelope to FileRecordNumber	1
662881	Reliable installation capability	1
662882	General file installation order	1
662883	Condition for a logical block to be valid	1
662884	Conditions for a logical block to be invalid	1
662885	Number of attempts to verify file signatures	1
662886	File installation	1
662887	File installation routine- installation accomplish	1
662888	File installation-installation abort	1
662889	File installation-installation progress	1

662890	Data Compression Method Selection	1
662891	Data compression method 1	1
662892	Data compression Method 1 parameters	1
662893	Data compression Method 1 parameters	1
662894	Data compression Method 1 parameters	1
662900	RequestDownload Service	1
662907	Bootloader CAN busoff handling	1
662908	General bootloader requirements	1
662909	Bootloader flow chart	1
662910	Complete hardware initiation when starting the ECU	1
662911	Cold or warm start detected	1
662912	Cold start detected	1
662913	Warm start detected	1
662914	Check ProgSignature in RAM	1
662915	Clear ProgSignature in RAM	1
662916	ProgSignature is not SET	1
662917	ProgSignature is SET	1
662918	PROG received within Timeout_Prog	1
662919	No PROG received within Timeout_Prog	1
662920	Complete & Compatible function	1
662921	Non complete or compatible software(s)	1
662922	Complete and compatible software(s)	1
662923	Receive and process diagnostic programming session requests	1
662924	S3Server timer timeout in SBL	1
662925	S3Server timer timeout in PBL	1
662926	Reception of DiagnosticSessionControl(defaultSession)	1
662927	Reception of DiagSessionControl(programmingSession)	1
662928	Reception of the ECUReset service	1
662929	Enter bootloader from ECU application	1
662930	Write ProgSignature	1
662931	Trig a reset in ECU application mode	1
662932	Application layer	1
662933	TCP for internal communication	1
662934	TCP-IP layer	1
662935	Session layer	1
662936	Diagnostic gateway	1
662937	Diagnostic router	1
662938	RequestDownload Service	1
662939	TransferData Service	1
662940	Bootloader General SWDL requirements	1
662941	Check ProgSignature in RAM	1
662942	Clear ProgSignature	1
662943	Cold or warm start detected	1
662944	Cold start detected	1
662945	Complete & compatible function	1
662946	Complete and compatible software(s)	1
662947	Complete hardware initiation when starting the ECU	1
662948	Enter the bootloader from ECU application mode	1
662949	ProgSignature is not set	1
662950	ProgSignature is set	1
662951	Receive and process diagnostic programming session request	1
662952	The reception of DiagnosticSessionControl(defaultSession)	1
662953	The reception of DiagnosticSessionControl(programmingSession)	1
662954	The reception of ECU Reset service	1
662955	Trig a reset in ECU application mode	1
662956	Warm start detected	1
662957	Write ProgSignature	1

662958	IP ECU - DoIP Layer	1
662959	IP ECU - Non complete or compatible software(s)	1
662960	IP ECU - Bootloader startup sequence	1
662961	IP ECU - Define T_EstablishedTCPconnection timeout	1
662962	IP ECU - Establish TCP connection	1
662963	IP ECU - Incomplete or incompatible software	1
662964	IP ECU - No PROG received within Timeout_Prog	1
662965	IP ECU - PROG received within Timeout_Prog	1
662966	IP ECU - Restart T_EstablishedTCPconnection	1
662967	IP ECU - S3Server timer times out in the PBL	1
662968	IP ECU - S3Server timer times out in the SBL	1
662969	IP ECU - TCPconnection is established after cold start.	1
662970	IP ECU - TCPconnection is established after warm start.	1
662971	IP ECU - TCPconnection is not established after cold start.	1
662972	IP ECU - TCPconnection is not established after warm start.	1
662973	IP ECU - Transmission of Network Management PDU on CAN network	1
662974	Edge Node - DoIP layer	1
662975	Edge Node - DoIP messages used for internal communication	1
662980	Edge Node - Physical link connection and DoIP connection	1
662981	Edge Node - Bootloader flow chart	1
662982	Edge Node - Check external Ethernet link	1
662983	Edge Node - Check if IP address is set in RAM	1
662987	Edge Node - Edge Nodes PBL will contain SBL functionality	1
662988	Edge Node - Ethernet link connection	1
662990	Edge Node - Ethernet link exists	1
662991	Edge Node - Ethernet link is detected	1
662992	Edge Node - Ethernet link not detected	1
662993	Edge Node - Ethernet link timeout	1
662994	Edge Node - IP address assignment during startup sequence	1
662995	Edge Node - IP address is not set in RAM	1
662996	Edge Node - IP address is SET	1
662997	Edge Node - Non complete or compatible software(s)	1
662999	Edge Node - Receiving DoIP messages	1
663001	Edge Node - S3Server timer times out in the PBL	1
663002	Edge Node - Set IP address	1
663003	Edge Node - Set VIN number	1
663004	Edge Node - Start sending TesterPresent	1
663005	Edge Node - The periodicity of TesterPresent	1
663006	Edge Node - The sending of TesterPresent	1
663007	Edge Node - Vehicle announcement messages during startup sequence	1
663008	Edge Node - Activate Secondary Bootloader not supported	1
663011	Edge Node - Start S3Server Timer	1
663012	Edge Node - Transmission of Network Management PDU on CAN network	1
663013	LIN slave supported NAD types	1
663014	RequestDownload (34) maxNumberOfBlockLength	1
663015	No support of queued request in programming session	1
663016	General bootloader requirements	1
663017	Bootloader flow chart	1
663018	Complete hardware initiation when starting the ECU	1
663019	Check ProgSignature in RAM	1
663020	Clear ProgSignature in RAM	1
663021	Cold or warm start detected	1
663022	Cold start detected	1
663023	Complete & Compatible function	1
663024	Complete and compatible software(s)	1

663025	No PROG received within Timeout_Prog	1
663026	Non complete or compatible software(s)	1
663027	PROG received within Timeout_Prog	1
663028	ProgSignature is not set	1
663029	ProgSignature is set	1
663030	Timeout_Prog	1
663031	Warm start detected	1
663032	Receive and process diagnostic programming session requests	1
663033	Reception of DiagnosticSessionControl (programmingSession)	1
663034	Reception of DiagnosticSessionControl(defaultSession)	1
663035	Reception of the ECUReset service	1
663036	S3Server timer timeout in PBL	1
663037	S3Server timer timeout in SBL	1
663038	Enter bootloader from ECU application	1
663039	Trig a reset in ECU application mode	1
663040	Write ProgSignature	1
663041	File Naming	1
663042	VBF file generation	1
663043	Header expression termination	1
663044	Header Identifiers	1
663045	IdentifierValue format	1
663046	Integers	1
663047	Comments	1
663048	White space characters	1
663049	Non-printing characters	1
663050	Version section	1
663051	Header section	1
663052	Header section - Format	1
663053	description	1
663054	sw_part_number	1
663055	sw_current_part_number	1
663056	sw_version	1
663057	sw_current_version	1
663058	sw_part_type	1
663059	data_format_identifier	1
663060	ecu_address	1
663061	erase	1
663062	call	1
663063	verification_block_start	1
663064	verification_block_length	1
663065	verification_block_root_hash	1
663066	sw_signature	1
663067	sw_signature_dev	1
663068	file_checksum	1
663069	file_checksum - no data blocks	1
663070	parameter_settings	1
663071	Data section - start	1
663072	Software part version in the data section	1
663073	Data section - structure	1
663074	Data block checksum	1
663093	Autosar configuration - required configuration values	1
663094	Autosar configuration - recommended values	1
663095	Simultaneous use of Profiles and Transformers	1
663096	Use of Profiles and Transformers	1
663097	E2E CRC Calculation	1
663098	E2E Data-Type Size	1
663099	E2E Protection Mechanisms	1
663100	E2E Protection Wrapper	1

663101	Hardware time stamping support	1
663102	Use of AUTOSAR BSW modules	1
663107	General requirement to match test case from AutoSAR CANNM	1
663108	General requirement to match test case from AutoSAR COM	1
663109	Definition of TCANPowerWakeUpToApp	1
663110	Definition of TCANResumeCom	1
663111	Timing at power up or reset initialization CAN	1
663112	Timing at resumed communication CAN - relay powered/ignition sensing node	1
663125	Definition of TCANGwPncRequest	1
663126	Definition of TCANWakeAndGwPncRequest	1
663129	Timing at destination network wake up and gatewaying PNC request to CAN	1
663131	Timing at gatewaying PNC request to CAN	1
663132	Definition of TCANEventWakeUpToApp	1
663133	Timing at network and hardware wake up CAN - microcontroller powered in sleep	1
663134	Timing at network and hardware wake up CAN - microcontroller unpowered in sleep	1
663137	Spare memory	1
663139	NVM during power loss	1
663141	NVM bootsector	1
663142	NVM Error Code Correction	1
663143	NVM modifications	1
663144	NVM process	1
663145	Input and Output lines	1
663146	Error detection on Input and Output lines	1
663148	General requirement to match test case from standard	1
663149	Applicable CAN&CANFD standards	1
663150	CAN&CANFD conformance	1
663151	CAN&CANFD controller	1
663152	Short term bit rate tolerance	1
663153	PCB design for CAN&CANFD termination	1
663154	ECU internal voltage supply	1
663155	CAN&CANFD transceiver component selection	1
663156	Common mode chokes	1
663157	ESD protection components	1
663158	ECU internal delay	1
663159	Bus fault monitoring circuitry	1
663161	CAN&CANFD circuit traces on PCB	1
663162	CAN&CANFD wiring inside ECU	1
663163	Decoupling capacitor location	1
663164	Guard tracks on PCB	1
663165	PCB CAN trace symmetry	1
663166	Total interface capacitance	1
663167	Transceiver ground plane	1
663168	Transceiver grounding	1
663169	Transceiver location on PCB	1
663170	Terminating ECUs on CAN&CANFD	1
663171	Physical layer circuit for terminating node	1
663172	Physical layer circuit for non terminating node	1
663185	Document Precedence	1
663186	100BASE-T1 Network	1
663187	Automotive Environment Installation	1
663188	100BASE-T1 data pairs only in multipin connector	1
663189	Connectors	1
663190	Multipin connectors	1
663191	Pin Assignment in a Multipin Connector.	1

663192	Separate Connector	1
663193	Signal path through connector	1
663196	Automatic Polarity Detection	1
663197	ECU Conformance	1
663198	EMC Conformance	1
663199	Consult Transceiver Supplier Regarding Circuit Design	1
663200	Clock Signal Provision	1
663201	Common Mode Choke	1
663202	Matching CMC with Transceiver	1
663203	EMI Protection	1
663204	Termination	1
663205	MII and RMII Interface	1
663206	Signal Line Impedance	1
663207	Signal Line Routing on PCB	1
663208	Configure Force Mode	1
663209	ECU Behaviour Outside of Operating Voltage Range and Special Conditions	1
663210	Internal Power Distribution	1
663211	Operating Voltage Range	1
663212	Transceiver Oscillator and External Clock Tolerance	1
663213	Time Stamping	1
663225	Document Precedence	1
663226	Ethernet Network	1
663227	ISO 13400 - 3	1
663228	AEC Approved Components	1
663230	Clock Signal Provision	1
663231	ECU Ethernet Conformance	1
663233	ECU Connector Electrical Characteristics	1
663234	ECU Connector Shield Attached to Chassis Ground	1
663235	Shielding Strategy	1
663236	EMI Protection	1
663237	Consult Transceiver and Transformer Supplier Regarding Circuit Design	1
663238	Chassis Ground Plane	1
663239	Chassis Grounding of Components	1
663240	Component Positioning on the PCB relative the Transformer	1
663241	Signal Ground Plane	1
663242	Signal Grounding of Components	1
663243	Signal Line Routing on PCB	1
663244	Signals to Chassis Ground	1
663245	Transceiver Positioning on PCB	1
663246	Transformer Positioning on PCB	1
663247	MII/RMII Interface	1
663248	Connected Supply Voltage	1
663249	ECU Behavior Outside of Operating Voltage Range and Special Conditions	1
663250	Operating Voltage Range	1
663251	Remote Wake Up	1
663252	Termination	1
663253	Ethernet Transceiver Fast Ethernet Compliance	1
663254	Transceiver Qualification	1
663255	Transceiver EMC Conformance	1
663256	Transceiver Interoperability Testing	1
663257	Matching Transformer with Transceiver	1
663258	Transformer with Common Mode Choke	1
663259	Error Handling	1
663260	Link Auto Negotiation	1
663261	Nominal communication bit rate	1

663262	Cable Length between ECUs	1
663263	Connector Allocation Strategy	1
663264	Connector Type Requirements	1
663265	Jack Connectors	1
663266	Plug Connectors	1
663267	Shielded Connector - Internal Interfaces	1
663268	Ethernet Signal Crossover	1
663269	Network Sub Segment Signal Crossover Strategy	1
663270	Duplex	1
663271	Harness Compliance	1
663272	Harness Design	1
663273	MDI - All ports	1
663274	Ethernet Network Topology	1
663275	Cable Characteristics	1
663276	Cable Length to be used in Cable Characteristic Impedance Measurement	1
663277	Cable Temperature Span	1
663278	Method for Measurement of Cable Characteristic Impedance Measurement	1
663279	Shielded Cable - Internal Interfaces	1
663280	Twisted Pair Cable	1
663281	External Clock Source for the Transceiver	1
663282	Transceiver Oscillator and External Clock Tolerance	1
663283	Cable Impedance Measurement Procedure	1
663284	Diagnostic Edge Node	1
663285	No Voltage Transitions below V_inactive	1
663286	Link Auto Negotiation on DoIP Edge Node	1
663287	Cable Connecting Diagnostic Link Connector and DoIP Edge Node	1
663288	Diagnostic Link Connector	1
663289	Unshielded Connector - External Diagnostic Interface	1
663290	HP Auto MDI MDI-X on DoIP Edge Node	1
663291	Unshielded Cable - External Diagnostic Interface	1
663292	Requirement Exception Process	1
663293	The priority between standard documents and this specification	1
663294	Node conformance test	1
663296	Normal battery voltage power operation	1
663297	Transceiver mode uniformity	1
663298	Speed tolerance	1
663299	Circuit board tracks	1
663300	Ground plane	1
663301	Grounding of the LIN transceiver	1
663302	Location of the decoupling capacitor	1
663303	Location of the tranceiver	1
663304	Interface implementation and physical layer circuit master	1
663305	Interface implementation and physical layer circuit slave	1
663308	The priority between standard documents and this document	1
663312	Calculation of speed tolerance	1
663315	Physical layer circuit	1
663341	Document Precedence	1
663342	1000BASE-T1 Network	1
663343	Automotive Environment Installation	1
663344	1000BASE-T1 data pairs in multipin connector	1
663345	Connectors	1
663346	Signal path through connector	1
663349	Automatic Polarity Detection	1
663350	ECU Conformance	1
663351	EMC Conformance	1

663352	Consult Transceiver Supplier Regarding Circuit Design	1
663353	Clock Signal Provision	1
663354	Common Mode Choke	1
663355	Matching CMC with Transceiver	1
663356	EMI Protection	1
663357	Termination	1
663358	GMII and RGMII/SGMII Interface	1
663359	Signal Line Impedance	1
663360	Signal Line Routing on PCB	1
663361	Configure Force Mode	1
663362	ECU Behaviour Outside of Operating Voltage Range and Special Conditions	1
663363	Internal Power Distribution	1
663364	Operating Voltage Range	1
663365	Transceiver Oscillator and External Clock Tolerance	1
663366	Time Stamping	1
663781	DiagnosticSessionControl (0x10)	1
663824	DID - F106	1
663871	DID - F114	1
663872	DID - F1A0	1
663873	DID - F120	1
663978	ECUReset (0x11)	1
663979	SecurityAccess (0x27)	1
663980	CommunicationControl (0x28)	1
663982	Authentication (0x29)	1
663984	TesterPresent (0x3E)	1
663985	ControlDTCSetting (0x85)	1
663989	ReadDataByIdentifier (0x22)	1
663990	ReadMemoryByAddress (0x23)	1
663991	ReadDataByPeriodicIdentifier (0x2A)	1
663992	DynamicallyDefineDataIdentifier (0x2C)	1
663993	WriteDataByIdentifier (0x2E)	1
663994	WriteMemoryByAddress (0x3D)	1
663995	ClearDiagnosticInformation (0x14)	1
663996	ReadDTCInformation (0x19)	1
663998	InputOutputControlByIdentifier (0x2F)	1
663999	RoutineControl (0x31)	1
664001	RequestDownload (0x34)	1
664002	RequestUpload (0x35)	1
664003	TransferData (0x36)	1
664004	RequestTransferExit (0x37)	1
664005	RequestFileTransfer (0x38)	1
664181	DTC - F00362	1
664200	DID - F1A1	1
664201	DID - F121	1
664202	DID - F1A2	1
664203	DID - F122	1
664204	DID - F124	1
664209	DID - F1A5	1
664214	DID - F125	1
664226	DID - F126	1
664227	DID - F1AA	1
664228	DID - F12A	1
664229	DID - F1AB	1
664230	DID - F12B	1
664231	DID - F1AC	1
664232	DID - F12C	1
664233	DID - F1AE	1

664234	DID - F12E	1
664235	DID - F1AF	1
664236	DID - F12F	1
664237	DID - F1BB	1
664238	DID - F13F	1
664239	DID - F13B	1
664240	DID - F186	1
664241	DID - F18C	1
664242	DID - F190	1
664243	DID - ED20	1
664244	DID - EDA0	1
664683	DID - C011	1
664684	DID - D01C	1
664686	DID - D134	1
664687	DID - DED0	1
664688	DID - DED1	1
664689	DID - DD00	1
664690	DID - DD01	1
664691	DID - DD02	1
664692	DID - DD0A	1
664693	DID - DD0B	1
664694	DID - DD0C	1
664695	DID - E103	1
664701	DID - D0B5	1
664776	Updateable routing table	1
665099	External IP format to CAN format in request	1
665245	ECU Bulk State Flow Chart	1
665259	ECU Valid Configuration State Flow Chart	1
665261	VCP signal mapping	1
665263	Car Configuration DTC DID record for CC Subscriber ECUs	1
665392	RID - 0205	1
665393	RID - 0206	1
665394	RID - B000	1
665395	RID - B001	1
665397	RID - 0212	1
665398	RID - 0235	1
665399	RID - 0236	1
665400	RID - 0237	1
665401	RID - 0240	1
665402	RID - 0301	1
665403	RID - A000	1
665404	RID - A001	1
665405	RID - A002	1
665406	RID - A003	1
665407	RID - A004	1
665408	RID - A005	1
665409	RID - FF00	1
665436	General service requirements	1
665697	DID - Ethernet Error Message Counter	1
665699	DID - Ethernet Error Message Counter History	1
665701	DID - Ethernet Link Status	1
665702	DID - Extended Ethernet Link Status	1
665703	DID - Signal Quality Index	1
665704	DID - VLAN Membership Port #0 ~ #11	1
665710	DID - D0AF	1
665712	DID - C003	1
665713	DID - C106	1
665714	DID - C001	1

665715	DID - C000	1
665717	DID - C002	1
665731	Routine byte	1
665733	Routine result availability	1
665734	Routine Type 1- Short routine	1
665735	Routine Type 2- Long routine	1
665736	Routine Type 3 - Continuous routine	1
665737	RoutineStatus	1
665740	Reset active diagnostic functionality at session transitions	1
665741	Maintain active diagnostic functionality at session transitions	1
665742	SessionParameterRecord definition	1
665752	Session Layer	1
665955	Authenticated state	1
665957	Deauthenticated state	1
665959	Receiving de-authentication request	1
665960	Fallback to deauthenticated state on connection	1
665962	Authentication roles per client certificate	1
665964	Authentication role in Server	1
665965	Authentication roles for External Diagnostic Client	1
665966	Authentication role of Admin	1
665968	White lists in certificate	1
666002	Unique ECU address	1
666003	Allocating ECU addresses	1
666004	Assigning domain ID address	1
666005	Assigning network ID address	1
666006	Assigning ECU ID address	1
666012	Functional addressing specific networks	1
666013	Functional request shall be unsegmented on all networks	1
666023	Logical address for external diagnostic tester	1
666024	Logical address range for periodic response messages	1
666028	ECU ability to receive requests	1
666029	ECU start up time	1
666030	ECU operation cycle	1
666031	ECU ability to receive a complete message	1
666414	Response on the diagnostic service requests	1
666415	Response to accessing non-volatile memory	1
666416	Minimum number of buffers for requests	1
666440	Authentication protection for diagnostic services on Edge Node	1
666443	Authentication protection for diagnostic routing handling	1
666444	Authentication protection configuration	1
666478	DTC - Control module communication bus off	1
666487	GroupOfDTCSpecific DTC	1
666488	GroupOfDTC All DTCs in emission related ECU	1
666491	GroupOfDTC All Groups	1
666521	DTC - Common CAN electrical fault	1
666523	DTC - Permanent missing frame	1
666664	DTC - Faulty transmitted data	1
666683	DTC - Faulty LIN response	1
666686	DTC - Missing LIN response	1
666691	DTC - Faulty LIN physical bus	1
666761	DTC - E10156	1
666763	DTC - Detection of incompatible VIN	1
666776	DTC - D14B51	1
666781	DTC - D14C51	1
666797	DTC - EE0368	1
666798	DTC - EE0468	1
666800	Test Run Criteria for fault detection	1
666805	DTC - Other LIN network faults	1

666807	File transfer performance	1
666815	Application diagnostic data	1
666817	DID - DD07	1
666820	ECU Ethernet identification data	1
666821	Diagnostic services	1
666823	DTC - No Ethernet link to ECU for IP Router Switch ECU	1
667096	Definition of GNSS time	1
667098	Zone Master Node - Transmission of Network Management PDU on CAN network	1
667215	DID - C03D	1
667216	DID - C03C	1
667217	DID - C03B	1
667692	DTC - E30055	1
667698	DTC - E30056	1
667763	DTC - E30051	1
667780	DTC - E30057	1
668963	Maintain authenticated state functionality at ssion transitions	1
671173	No length field for basic datatypes	1
671190	Serialization-enumeration	1
671220	Serialization-Arrays (dynamic length)	1
671222	Serialization-Structs	1
671223	Serialization-Union	1
671305	Update bit sending for non-AutoSAR ECUs	1
671344	Serialization-Strings	1
673680	Application based communication Error Handling	1
673789	TCP client re-establishment	1
673803	TCP server initialization	1
674284	Update bit sending for gateway ECUs	1
675091	TLS compression for DoIP	1
675313	Signing Method #3	1
675314	Signing Method SHA256_RSA4096	1
675384	Return Code	1
675441	DID - E104	1
675447	Support for DTC time stamp #22	1
675449	Support for DTC time stamp #23	1
675938	Ethernet node linkup & wakeup strategy	1
676045	Diagnostic secure communication scheme	1
676168	DID - DD06	1
676288	Definition of vehicle speed	1
676865	DID - F18A	1
677100	DID - F109	1
677101	DID - F10B	1
679854	Definition of TPowerWakeUpToApp	1
680642	Ethernet Switch-support mirroring	1
680923	Transport Protocol	1
680924	LM IP_activity message structure	1
680925	Vehicle bus signal	1
680926	VFC IP wakeup	1
680927	IP Bus release	1
680928	IP Bus request	1
680929	LM initial discovery mechanism	1
680930	LSC notifying	1
680931	Software interface	1
680933	Active Request Session	1
680935	ARS timers	1
680936	Ongoing Request	1
680937	Request Monitoring Timeout	1
680999	Target ECUs	1

681000	LM broadcast	1
681106	TCP Retransmission timeout	1
681108	TCP SACK	1
681216	Periodic Message Timing Parameter definition	1
681666	Internal IP format to CAN format in request	1
681678	Reprogramming ability	1
681691	CAN format to internal IP format in response	1
681698	CAN response add IP address in internal IP	1
681716	Transmit(Tx) CCs on Network (CAN)	1
681717	Receive(Rx) CCs on Network (CAN)	1
681722	Minimum RAM for request buffers	1
681723	Minimum number of buffers for requests	1
681724	Large buffers for request	1
681725	Minimum RAM for response buffers	1
681740	Minimum number of buffers for responses	1
681741	Large buffers for response	1
681742	Minimum RAM for request buffers	1
681744	Minimum number of buffers for requests	1
681745	Large buffers for request	1
681746	Minimum RAM for response buffers	1
681747	Minmum number of buffers for responses	1
681748	Large buffers for response	1
681784	RID - A040	1
682017	Main Software Part(s) Update and Synchronization	1
682019	Common Software Part(s) Handle	1
682024	Expected Software Part(s) CompleteAndCompatible Check and Switch	1
682066	Normal Operation during Update Session	1
682067	Access and erase of inactive program partitions	1
682069	Software download and process for distributed method	1
682075	Buffer handling during Seamless Update	1
682079	Processing the data buffer	1
682085	Services - RequestDownload	1
682129	CompleteAndCompatible Check	1
682161	DTC - E30355	1
682163	DTC - E30141	1
682206	Usage of CRC	1
682209	byte order for time value signals	1
682210	DLC OF CANTSyn message	1
682211	message format	1
682360	Time Domain ID	1
682361	parameters	1
682545	RID - 0250 Installation Condition Check	1
682546	RID - 0255 Software Synchronization and Flag Activate	1
682668	Software process status	1
682680	Seamless Update process status	1
685689	The wakeup keep from diagnosis	1
686983	IP ECU - Start S3Server Timer	1
688989	DID - D105	1
692068	Deactivation the specific DoIP TCP connection	1
692129	Gateway Routing Capability	1
692237	Dual Bank Scheme	1
692287	DID - D088	1
692299	DID - D086	1
692300	DID - D084	1
692459	DID - D103	1
693274	Unused bit valuse	1
693454	Keep the delta couter contious in the source cluster	1

693571	Maximum allowed number of erase ranges in VBF header	1
693576	Maximum allowed number of data block	1
697110	Definition of SOME/IP TP	1
697111	Session Handling for SOME/IP TP	1
697113	Session ID about Session ID	1
697114	TP-Flag	1
697217	SOME/IP TP-Header structure	1
697233	SOME/IP-TP Header encode type	1
697247	Maximum length of a segment	1
697250	Configured segment	1
697252	Send order	1
697254	Segment Size	1
697255	not send overlapping or duplicated segments	1
697256	Segments reassemble Condition	1
697257	Session ID	1
697270	Reassemble check	1
697380	Received order	1
697381	When to cancle desegment	1
697382	Segmented message store buffer	1
697475	DID - D0B0	1
699068	Software Update on inactive partition	1
702544	Authentication method #1	1
702826	P2Server_max - ProgrammingSession	1
702988	Pseudo Primary Bootloader - Programming Session	1
703100	Diagnostic session for PseudoProgrammingSession	1
703288	Fixed Bytes Program Frequency Limit	1
703409	DID - D0A1	1
703410	RID - 0254 Check Flash Programming Pre-conditions	1
704491	Latency CAN to CAN, GW_Source_CAN-Target_CAN_L	1
704495	Receive(Rx) CCs on Zone network	1
704857	Diagnostic Routine Channels	1
705195	Receive(Rx) CCs on Backbone Network	1
705196	Transmit(Tx) CCs on Backbone Network	1
705197	Transmit(Tx) CCs on Zone network	1
706127	RID - 7027	1
711819	Signing Method SHA256_ECDSA - Signature format	1
714034	Hardware solution	1
714036	Software solution	1
714778	DID - D122	1
714886	DID - D123	1
714887	DID - D124	1
716317	Omitting Authentication of Internal Diagnostic Clients	1
716322	Authentication roles verification definition	1
716395	High Voltage Logic Function Control	1
716529	Vehicle Announcement Message/Identification Response Message UDP Port	1
717160	Initiation strategy for OS	1
717577	The definition of wireless TCP connection activation	1
717580	Disconnect DoIP TCP Connection with Edge Node	1
717582	Create Wireless DoIP TCP Connection in specific scenario	1
717598	The handle of wireless DoIP TCP Connection	1
838556	ZEVonUDS Data protection	1
841118	LIN slave with E2E	1
841273	Logical address for external legislated diagnostic tester	1
841703	Functional addressing in use case-specific standard	1
841909	Routing activation request activation types for	1
849660	mDNS parameter for SOVD	1
849661	mDNS Announcement/Response	1

849663	DHCP protocol support requirement	1
850391	Fixed Bytes Run	1
850700	Checking of received data	1
850799	Request Download process when download SBL	1
850800	Request Download process when download to non-volatile memory	1
852897	Stucked state handling	1
853765	General negative response code	1
855011	Usage of DoIP TLS	1
856222	Sector switch	1
857424	Memory Requirements	1
857534	UDP Length	1
857535	Buffer configuration	1
857536	I-PDU sequence mechanism	1
857537	nPdu and I-PDU timeout Triggering	1
857538	SoAdSocketnPduUdpTxBufferMin Triggering	1
857539	I-PDU with TRIGGER_ALWAYS Triggering	1
857540	VLAN Priorit y Setting Rules for nPDU	1
857543	Buffer Configuration	1
857544	I-PDU forwarding triggering mechanism	1
857545	UDP receiving process	1
857548	UDP Packaging mechanism	1
857549	ZEVonUDS base on Ethernet	1
857550	UDP Length	1
857551	UDP Message triggering mechanism	1
857553	VLAN Priority Setting Rules for nPDU	1
857554	UDP receiving process	1
857568	Diagnostic communication requirements after link confirmation	1
857676	Requirements for the hardware attributes of the OBD interface	1
857678	ZEVonUDS DoIP routing acivation type	1
857688	ZEV Propulsion ReadDTCInformation (0x19)	1
857858	ZEV Propulsion ClearDiagnosticInformation (0x14)	1
857861	ZEV Propulsion ReadDataByIdentifier (0x22)	1
857865	ZEV Propulsion RoutineControl (0x31)	1
857889	ZEV logical address assignment	1
870513	NM PDU Format	1
870516	NM Frame Format of CANFD Segment	1
871252	E2E Profiles	1
871254	E2E Supervision	1
871255	E2E Data Id	1
871256	E2E Transformer	1
871257	E2E Profile 11 configuration	1
871258	E2E Profile 11 header layout	1
871259	E2E Profile 11 CRC calculation	1
871261	E2E Profile 4 configuration	1
871262	E2E Profile 4 header layout	1
871263	E2E Profile 4 CRC calculation	1
871265	E2E Service Configuration for Profile11	1
871276	Physical Coding Sublayer	1
871278	Physical Media Attachment	1
871279	Interoperability	1
871711	TC10 Capability	1
871712	TC10 Sleep/Wake-up Enable	1
871713	Wake-Up Pattern	1
871714	Sleep Mode	1
871715	Sleep and Wake-up Forwarding	1
871716	WUP/WUR and LPS Command Transmitting Behavior	1
871740	Software Design Rules	1

871741	NM PDU Format	1
871742	NM Parameter Configuration	1
871754	MAC Security	1
871755	PHY integration	1
871766	Operations, Administration, and maintenance	1
871767	Management Data input/output Interface	1
871768	Advanced Diagnostic Features	1
871770	Physical Media Attachment	1
871771	Interoperability	1
871772	Ethernet PHY wake-up	1
871774	MAC Security	1
871787	PHY Integration	1
871789	TC8 Switch Conformance	1
871795	MAC Security	1
871800	Separate sessions for different media	1
871805	Synchronizing multimedia streams	1
871806	TC8 SOME/IP Conformance	1
871807	SOMEIP Port Assignment	1
871887	Hardware timestamping for Ethernet	1
871917	TC8 TCP/IP Conformance	1
871918	MSS	1
871919	MSL	1
871921	Timestamp Resolution	1
871922	gPTP Message Exchange	1
871923	Announce and Signaling Message	1
871924	Reference Implementation	1
871925	Time Interface for Upper Client	1
871926	Synchronization Accuracy	1
871927	Port Assignment for Transport Layer Protocol	1
871928	Standard Compatibility	1
871929	Support for Switch and End-point Chip	1
871930	FRER Functions	1
871931	Stream Identification Function	1
871932	Sequence Recovery Algorithm	1
871933	Latent Error Detection Support	1
871935	Redundancy Tag Support	1
872115	CANH circuit short to battery	1
872118	CANH circuit short to CANL circuit	1
872119	CANH circuit short to ground	1
872120	CANL circuit short to battery	1
872121	CANL circuit short to ground	1
908214	DID - D131	1
908320	mDNS paramenter for Web	1
908405	DID - D12C	1
908641	Hardware Clock Reference	1
908643	StbM Main Function Period	1
908645	Global Time Precision Measurement Support	1
908646	Synchronized Time Base	1
908648	Timeout for SyncLoss	1
908649	Offset Time Base	1
908651	Time Base Cloning	1
908652	Rate Correction	1
908654	Offset Correction by Rate Adaption	1
908655	Hardware Timestamping - Ethernet	1
908657	EthTSyn Main Function Period	1
908658	EthTSyn Time Domain	1
908659	VLAN Support	1
908661	Reference Synchronized Time Base	1

908663	Message Format Compliance	1
908666	Port Role	1
908668	CRC Support	1
908670	Master Slave Conflict Detection	1
908671	TX Debouncing Time	1
908673	Propagation Delay Measurement	1
908675	Propagation Delay Request Period	1
908677	Propagation Delay Threshold	1
908678	Propagation Delay Timeout	1
908679	Cyclic Transmission of Global Time - Ethernet	1
908680	FollowUp Timeout	1
908681	Sequence Counter Validation	1
908753	AutoIP protocol support requirement	1
908756	Concurrent SOVD External Client Connection	1
908758	Standards for CANFD interface	1
908760	External IP allocation	1
908805	MAC address and ethertype	1
908806	Untagged for PTP message	1
908810	Sync receipt timeout	1
908811	Domain number	1
908828	DID - D09D	1
908842	TCPIP stack startup time	1
908968	Hardware Timestamping - CAN	1
908970	Support for CAN	1
908971	Reference Synchronized Time Base	1
908972	TX Debounce Time	1
909056	Link Establishment Time	1
943093	The relationship between routine and action	1
975220	Separating type of electrical fault	1
975457	Detection of no production public key and Security Constant	1
975465	General XCP requirements	1
975466	Measurement specific XCP requirements	1
975467	Performance XCP requirements	1
975703	Bootloader startup	1
975704	Transistion to Partition choice	1
975705	Bootloader state variable	1
975706	Failure to boot NORMAL partition	1
975707	Rollback requested from application	1
975708	SWITCH requested from application	1
975709	Normal Partition selection	1
975710	Succesfull SWITCH performed	1
975711	Failure to perform SWITCH	1
975712	Rollback sucessful	1
975713	FAILURE state	1
975714	Rollback failed	1
975715	Retry to boot	1
975716	Maximum time to boot	1
975717	Session for reprogram in appliaction mode	1
975718	session transition description	1
975719	Partitioning	1
975720	Software is downloaded to the inactive partition	1
975721	Upgrade performed in the background of application context	1
975722	Additional requirements for software downloads	1
975723	Address remapping strategy	1
975724	Switch between active and inactive partitions	1
975725	AB partition additional requirements for ECUs having multiple processors	1
975726	Software activation requirements	1

975727	Automatic partition switching behavior must be avoided	1
975728	Support for write while read	1
975729	Additional requirement for switch partitions（reprogram via flash bootloader）	1
975730	Additional requirement for switch partitions（reprogram in OTA session）	1
975731	Bootloader addressing schema	1
975732	Bootloader timeout	1
975733	Routing of PDUs in programming session	1
975734	Software synchronization	1
975909	Monitoring and logging of DTCs	1
975910	Ensure non diagnostic communication	1
975911	Delta update and total update	1
975912	Requirement for delta update tool	1
975913	action retry	1
975916	Read AB Information	1
975917	Condition Check	1
975919	Security access in OTA programming mode	1
975920	Data Transfer	1
975922	Unintended Shutdown –Resume at the latest Checkpoint	1
975923	Intended Shutdown –Suspend & Resume of a Download	1
975924	Erase form breakpoint	1
975925	Check switch conditions	1
975926	Switch partition	1
975927	Automatic partition switching behavior	1
975928	Clear DTC	1
975929	Way of Update Private ECU	1
975930	Private ECU Support AB Swap	1
975931	Flash Way of Private ECU Not Support AB Swap	1
975932	Rollback of Private ECU	1
975933	Exception Handling After Failure	1
975934	Software Flash Requirements	1
975935	Rollback Requirements	1
975936	Software Synchronization Requirements	1
975937	Roles in the process of downloading file from tester	1
975938	Tester role requirement	1
975939	Edge node requirement	1
975940	Downloader requirement	1
976008	Large buffers for request	1
976009	Large buffers for response	1
976010	Minimum number of buffers for requests	1
976011	Minimum number of buffers for responses	1
976012	Minimum RAM for request buffers	1
976013	Minimum RAM for response buffers	1
976051	Periodic response format to external IP	1
976052	Supported periodic response format from internal IP	1
976053	Supported periodic response format from CAN	1
976054	Internal IP to external IP address in periodic response	1
976055	Large buffers for request	1
976056	Large buffers for response	1
976057	Minimum number of buffers for requests	1
976058	Minimum RAM for request buffers	1
976059	Minimum number of buffers for responses	1
976060	Minimum RAM for response buffers	1
976061	Support for queued request	1
976115	Chain of trust	1
976119	Action on revoked or unknown status of certificates	1
976129	Authentication Successful	1

976145	Default log level	1
976146	Supported programming languages	1
976147	Logging API	1
976148	Suppressing logging messages	1
976149	Supported data types	1
976150	Raw data	1
976151	Silence execution	1
976152	Destination of logging information	1
976153	Logging server temporary memory storage	1
976154	Application registration	1
976155	Providing logging severity	1
976156	Context initialization	1
976157	Timestamp	1
976158	Early logging	1
976159	Size of logging server temporary memory storage	1
976160	Application unregistration	1
976161	Grouping of logging messages	1
976162	Number of logging contexts	1
976163	Check current severity level	1
976164	Size of logging API temporary memory storage	1
976165	Run-time configuration of severity level	1
976166	Core dumps configuration	1
976167	Retrieve saved core dumps	1
976168	Number of core dumps	1
976169	Request for transferring core dump	1
976170	Stream log data to the Security Gateway	1
976171	Shaping network traffic	1
976172	Supported network protocols	1
976173	Local log storage	1
976174	Local log storage configuration	1
976175	Delete locally stored logs	1
976176	Retrieve locally stored logs	1
976177	Length of the logging message	1
976178	Size of core dump file	1
976179	Size of core dumps storage	1
976180	Message discarding notification	1
985754	OTA SWDL Sequence	1
985755	OTA Session	1
985756	Diagnostic services supported	1
985757	Confirm session	1
985758	Breakpoint record	1
985775	Obtain Certificate	1
987287	Edge Node - Transmission of TesterPresent	1
991947	Software part format	1
991948	Maximim allowed VBF File size	1
991949	Maximum allowed number of erase ranges in VBF header	1
991950	Maximum allowed number of data block	1
991959	File Naming	1
991960	VBF file generation	1
991972	Header expression termination	1
991974	Header Identifiers	1
991975	IdentifierValue format	1
991978	Integers	1
991979	Comments	1
991981	White Space Characters	1
991982	Non-printing Characters	1
991984	Version section	1
991985	Header section	1

991987	Header section - Format	1
991992	Data section - start	1
991993	description	1
991994	sw_part_number	1
992019	sw_current_part_number	1
992024	sw_version	1
992026	sw_current_version	1
992028	sw_part_type	1
992031	data_format_identifier	1
992032	ecu_address	1
992033	erase	1
992035	call	1
992040	verification_block_start	1
992045	verification_block_length	1
992049	verification_block_root_hash	1
992051	sw_signature	1
992054	sw_signature_dev	1
992061	file_checksum	1
992062	file_checksum - no data blocks	1
992063	parameter_settings	1
992074	Software part version in the data section	1
992075	Data section - structure	1
992076	Data block checksum	1
992348	address_and_length_format_identifier	1
1029358	OBDonUDS base on CAN bus	1
1029361	OBDonUDS address assignment	1
1029369	Regulation link identification process requirment	1
1029373	OBDonUDS Propulsion ReadDTCInformation (0x19)	1
1029379	E2E Service Configuration for Profile4	1
1029458	OBDonUDS Propulsion ClearDiagnosticInformation (0x14)	1
1029459	OBDonUDS Propulsion ReadDataByIdentifier (0x22)	1
1029460	OBDonUDS Propulsion RoutineControl (0x31)	1
1029552	OBDonUDS Data protection	1
1029553	L3onUDS Data protection	1
1029554	ePTIonUDS Data protection	1
1029718	ADAS_L3 base on Ethernet	1
1029736	L3 logical address assignment	1
1029761	Services related to regulations	1
1029812	DoIP Ports for Communication	1
1029813	L3onUDS DoIP routing acivation type	1
1029814	Vehicle gateway allows functional addressing of UDS services for routing	1
1029901	ePTI logical address assignment	1
1029903	ReadDTCInformation and ClearDiagnosticInformation	1
1030289	Services related to regulations	1
1030387	ClassicOBD Data protection	1
1030394	Regulation link identification process requirment-ECU	1
1030709	Diagnostic communication requirements after link confirmation	1
1030715	ePTIonUDS DoIP routing acivation type	1
1031152	Hostname definition	1
1032594	ECU diagnostic data handling	1
1032774	WLAN Performance Requirements	1
1032777	Roaming Support	1
1032778	Test Report Requirements	1
1032790	WLAN Parameter Monitor	1
1037501	DID - C098	1
1037608	ePTI base on Ethernet	1
1037652	Regarding the data protection requirements for DoIP links	1

1037654	Regarding the data protection requirements for DoCAN links	1
1038508	General requirement to match test case from Subsystem requirement	1
1038509	General requirement to match test case from Subsystem requirement	1
1039359	Vehicle authorization server	1
1039362	Resource Access of SOVD Server	1
1039363	Request with token	1
1039370	The ability of Token Verification	1
1039374	Authentication and authorization process	1
1039376	Request a token	1
1039393	mTLS connection	1
1039394	Client certificate identity verification	1
1039404	Access token	1
1040185	RID - 0294 Install Private Module	1
1040186	RID - 0296 Rollback External Flash Module	1
1040209	API Method - Register a vechicle update	1
1041300	API Method - Trigger a vehicle package download	1
1041301	API Method - Trigger a vehicle update installation	1
1041319	SOVD update Identifier	1
1041382	Definition of Vehicle update and configuration module	1
1041385	Data acquisition for vehicle update	1
1041388	Identification of the vehicle update	1
1041390	Cloud interaction restrictions during SOVD vehicle update	1
1041394	Vehicle update information	1
1041396	API Mehotd - Query for the update from vehicle	1
1041397	Update origins definition	1
1041466	API method - Retrieve details of a vehicle update	1
1041524	The handle of OTA update	1
1041625	The sovdResourceDuration definition and handle	1
1041634	Update origin request	1
1041637	The processing of registering a vehicle update	1
1041646	Update conflict handling	1
1041657	Software package download	1
1041658	Vehicle update installation	1
1041666	Vehicle update and resource handling	1
1041675	Read the status of the vehicle update	1
1041733	Processing of delayed installation	1
1041734	Data storage for vehicle update	1
1042798	Hostname Announcement	1
1042926	Debug interface	1
1042951	Performance requirement	1
1042955	Handling when internal connection resources are exhausted	1
1042962	General requirements	1
1042963	Internal request forwarding queue mechanism	1
1043035	HTTP/1.1 Communication	1
1043287	HTTP/2.0 Communication	1
1043289	HTTP/2.0 additional requirement	1
1043291	As the server of mTLS connection	1
1043323	Pre-established connection as the in-vehicle client	1
1043504	Retrieve online capability description	1
1043505	Specific responce Status Code 504 Supported	1
1043509	The lock mechanism after UDS service sent	1
1043510	Specific responce Status Code 502 Supported	1
1043511	Definition and requirements of faults	1
1043512	The map UDS-based content to SOVD	1
1043513	Definition of Scope	1
1043514	Definition of Status	1

1043515	Restart an UDS ECU	1
1043516	The environment data for classic ECU	1
1043529	The handling of parallel requests	1
1043531	The handling of multiple requests for same entity	1
1043535	Definition and Requirements of Data	1
1043538	General API Method Requirement	1
1043560	Data Resource APIs	1
1043604	Fault APIs	1
1043606	Tag-Related APIs	1
1043619	Entity-Related APIs	1
1043626	Capability Descripiton-Related APIs	1
1043658	Cyclic-Subscription APIs	1
1043663	Triggers APIs	1
1043684	Configuration APIs	1
1043685	Clearing Data-Releated APIs	1
1043686	Control of operation APIs	1
1043687	Script execution APIs	1
1043690	Target Mode APIs	1
1043691	Locking APIs	1
1043703	Restarting an Entity APIs	1
1043705	Bulk-data APIs	1
1043706	Logging APIs	1
1043708	Communication-Logs APIs	1
1043765	proximity proof required	1
1043842	Explicit control for security access	1
1043966	The parameter mode_expiration	1
1043982	Data Category and Logical Organization	1
1043983	Metadata management	1
1044054	Convert SOVD to/from UDS	1
1044319	As the in-vehicle server	1
1044347	API - Method - Delete vehicle update	1
1044365	In-vehicle SOVD communication vlan allocation	1
1045995	The number of connection for SOVD diagnsotic	1
1047368	The number of connection to SOVD Client	1
1047374	The number of connections in the connection pool	1
1047398	Specific responce Status Code 422 Supported	1
1047424	The handling of DoIP errors	1
1069504	Max MAC PDU size	1
1069505	Disable the operation of MAC control PAUSE	1
1069506	VLAN-aware Bridge	1
1069507	Support CBS as transmission selection algorithm for Bridge	1
1069508	Support CBS as transmission selection algorithm for Talker	1
1069509	Number of data streams for Talker	1
1069510	Number of data streams for Listener	1
1069511	Classify frames into traffic classes	1
1069512	Support a minimum of two traffic classes	1
1069514	Credit-based shaper(CBS) algorithm	1
1069516	CBS Configuration requirement	1
1069518	Disable SRP for bandwidth reservation	1
1069519	Support VLAN and queues	1
1069706	Support TAS as transmission selection algorithm for Bridge	1
1069709	gPTP time synchronization for Talker	1
1069711	Support TAS as transmission selection algorithm for Talker	1
1069712	TAS shall not be supported for Listener	1
1069732	GCL cycle and slot length	1
1069734	Time-Aware Shaper(TAS) algorithm	1
1069736	Disable dynamic GCL	1
1069739	TAS diagnostic logging requirement	1

1069744	TAS Configuration requirement	1
1070052	CBS and TAS are typically not co-deployed	1
1070713	CBS shall not be supported for Listener	1
1071359	Additional SWDL Performance for POXIS system	1
1071491	Major interface version and Minor interface Version	1
1071923	gPTP time synchronization for Bridge	1
1071924	CBS diagnostic logging requirement	1
1072236	Sub IP format to Internal IP format in response	1
1072314	Minimum RAM for buffers	1
1072320	Minimum RAM for buffers	1
1073566	The scrpit handling when upload	1
1073567	API - Multi script compression upload	1
1073591	Script compression method	1
1073594	Processing after script decompression	1
1073973	The overwrite of same bulk data	1
1074516	Scripts runtime	1
1074518	DTC - D63451	1
1074702	Connect Vehicle Implementation Rules	1
1074703	Vehicle connection management (Connect to UDS)	1
1075488	The bulk-data identifier	1
1075831	Watchdog management	1
1075871	Binding of request response	1
1075934	SOVD Client Connection Limit	1
1075998	Connection pool	1
1076320	Return and reuse mechanism for connections	1
1076475	The number of connection from Gateway	1
1076484	The usage of diagnostic database	1
1077149	ESS software solidification	1
1078814	DID - D1A7	1

0.2 Requirements change log

0.2.1 New requirements in this issue

1039359	Vehicle authorization server	1
1039362	Resource Access of SOVD Server	1
1039363	Request with token	1
1039370	The ability of Token Verification	1
1039374	Authentication and authorization process	1
1039376	Request a token	1
1039393	mTLS connection	1
1039394	Client certificate identity verification	1
1039404	Access token	1
1040209	API Method - Register a vechicle update	1
1041300	API Method - Trigger a vehicle package download	1
1041301	API Method - Trigger a vehicle update installation	1
1041319	SOVD update Identifier	1
1041382	Definition of Vehicle update and configuration module	1
1041385	Data acquisition for vehicle update	1
1041388	Identification of the vehicle update	1
1041390	Cloud interaction restrictions during SOVD vehicle update	1
1041394	Vehicle update information	1
1041396	API Mehotd - Query for the update from vehicle	1
1041397	Update origins definition	1
1041466	API method - Retrieve details of a vehicle update	1
1041524	The handle of OTA update	1
1041625	The sovdResourceDuration definition and handle	1
1041634	Update origin request	1

1041637	The processing of registering a vehicle update	1
1041646	Update conflict handling	1
1041657	Software package download	1
1041658	Vehicle update installation	1
1041666	Vehicle update and resource handling	1
1041675	Read the status of the vehicle update	1
1041733	Processing of delayed installation	1
1041734	Data storage for vehicle update	1
1043035	HTTP/1.1 Communication	1
1043287	HTTP/2.0 Communication	1
1043289	HTTP/2.0 additional requirement	1
1043504	Retrieve online capability description	1
1043505	Specific responce Status Code 504 Supported	1
1043509	The lock mechanism after UDS service sent	1
1043510	Specific responce Status Code 502 Supported	1
1043511	Definition and requirements of faults	1
1043512	The map UDS-based content to SOVD	1
1043513	Definition of Scope	1
1043514	Definition of Status	1
1043515	Restart an UDS ECU	1
1043516	The environment data for classic ECU	1
1043529	The handling of parallel requests	1
1043531	The handling of multiple requests for same entity	1
1043535	Definition and Requirements of Data	1
1043538	General API Method Requirement	1
1043560	Data Resource APIs	1
1043604	Fault APIs	1
1043606	Tag-Related APIs	1
1043619	Entity-Related APIs	1
1043626	Capability Descripiton-Related APIs	1
1043658	Cyclic-Subscription APIs	1
1043663	Triggers APIs	1
1043684	Configuration APIs	1
1043685	Clearing Data-Releated APIs	1
1043686	Control of operation APIs	1
1043687	Script execution APIs	1
1043690	Target Mode APIs	1
1043691	Locking APIs	1
1043703	Restarting an Entity APIs	1
1043705	Bulk-data APIs	1
1043706	Logging APIs	1
1043708	Communication-Logs APIs	1
1043765	proximity proof required	1
1043842	Explicit control for security access	1
1043966	The parameter mode_expiration	1
1043982	Data Category and Logical Organization	1
1043983	Metadata management	1
1044054	Convert SOVD to/from UDS	1
1044319	As the in-vehicle server	1
1044347	API - Method - Delete vehicle update	1
1044365	In-vehicle SOVD communication vlan allocation	1
1045995	The number of connection for SOVD diagnsotic	1
1047398	Specific responce Status Code 422 Supported	1
1047424	The handling of DoIP errors	1
1069504	Max MAC PDU size	1
1069505	Disable the operation of MAC control PAUSE	1
1069506	VLAN-aware Bridge	1
1069507	Support CBS as transmission selection algorithm for Bridge	1

1069508	Support CBS as transmission selection algorithm for Talker	1
1069509	Number of data streams for Talker	1
1069510	Number of data streams for Listener	1
1069511	Classify frames into traffic classes	1
1069512	Support a minimum of two traffic classes	1
1069514	Credit-based shaper(CBS) algorithm	1
1069516	CBS Configuration requirement	1
1069518	Disable SRP for bandwidth reservation	1
1069519	Support VLAN and queues	1
1069706	Support TAS as transmission selection algorithm for Bridge	1
1069709	gPTP time synchronization for Talker	1
1069711	Support TAS as transmission selection algorithm for Talker	1
1069712	TAS shall not be supported for Listener	1
1069732	GCL cycle and slot length	1
1069734	Time-Aware Shaper(TAS) algorithm	1
1069736	Disable dynamic GCL	1
1069739	TAS diagnostic logging requirement	1
1069744	TAS Configuration requirement	1
1070052	CBS and TAS are typically not co-deployed	1
1070713	CBS shall not be supported for Listener	1
1071359	Additional SWDL Performance for POXIS system	1
1071491	Major interface version and Minor interface Version	1
1071923	gPTP time synchronization for Bridge	1
1071924	CBS diagnostic logging requirement	1
1072236	Sub IP format to Internal IP format in response	1
1072314	Minimum RAM for buffers	1
1072320	Minimum RAM for buffers	1
1073566	The scrpit handling when upload	1
1073567	API - Multi script compression upload	1
1073591	Script compression method	1
1073594	Processing after script decompression	1
1073973	The overwrite of same bulk data	1
1074516	Scripts runtime	1
1074518	DTC - D63451	1
1074702	Connect Vehicle Implementation Rules	1
1074703	Vehicle connection management (Connect to UDS)	1
1075488	The bulk-data identifier	1
1075831	Watchdog management	1
1075871	Binding of request response	1
1075934	SOVD Client Connection Limit	1
1075998	Connection pool	1
1076320	Return and reuse mechanism for connections	1
1076475	The number of connection from Gateway	1
1076484	The usage of diagnostic database	1
1077149	ESS software solidification	1
1078814	DID - D1A7	1

0.2.2 Updated requirements in this issue

660805	ECU Addresses	1
660807	Missing frame master - CAN	1
660818	Eth Configuration parameters Update	1
660861	CAN FD bit register settings for 2 Mbps	1
660868	Register read out using Diagnostics	1
660869	Register write using Diagnostics	1
660907	Configuration Update	1
660953	Separation time (STmin) non-programming session client side	1
661023	Local Configurations	1
661024	ICMP support	1

661037	Graceful TCP Connection Shutdown on ECU Reset	1
661186	Local configurations	1
661197	SOME IP Header Validation	1
661222	Return Codes in the Response Messages	1
661274	Mapping between PCP and Traffic Classes	1
661293	Security Level usage	1
661429	Internal IP format to Sub IP format in request	1
661431	Large buffers for response	1
662756	Power Consumption	1
662802	Allocation of the Verification Block Table	1
662802	Allocation of the Verification Block Table	1
663093	Autosar configuration - required configuration values	1
663137	Spare memory	1
665395	RID - B001	1
665403	RID - A000	1
665404	RID - A001	1
665405	RID - A002	1
665406	RID - A003	1
665407	RID - A004	1
665408	RID - A005	1
666478	DTC - Control module communication bus off	1
675384	Return Code	1
849660	mDNS parameter for SOVD	1
849661	mDNS Announcement/Response	1
853765	General negative response code	1
857424	Memory Requirements	1
871742	NM Parameter Configuration	1
908756	Concurrent SOVD External Client Connection	1
992028	sw_part_type	1
1029361	OBDonUDS address assignment	1
1030394	Regulation link identification process requirment-ECU	1
1032778	Test Report Requirements	1
1032790	WLAN Parameter Monitor	1
1042955	Handling when internal connection resources are exhausted	1
1042963	Internal request forwarding queue mechanism	1
1043291	As the server of mTLS connection	1
1043323	Pre-established connection as the in-vehicle client	1
1047368	The number of connection to SOVD Client	1
1047374	The number of connections in the connection pool	1

0.2.3 Modifications in the updated requirements in this issue

For details, please look in the DETAILED CHANGE INFO

0.2.4 Deleted requirements in this issue

92898	isGM
1032793	AP Channel Select

1 LIST OF REQUIREMENTS WITH DEVIATIONS

2 INTRODUCTION

2.1 What is the purpose of this document?

This document is a Base Technology SWRS. It contains the software requirements for implementation of the Base Technology concepts needed for an ECU to be integrated into the vehicle electrical system. This specification may contain generic requirements that are not applicable to specific ECUs.

2.2 Who should read this document?

This document is intended for:

- Developers of ECU software and hardware at the car OEM, ECU suppliers (Tier 1:s) or sub suppliers/contractors (Tier 2:s).
- Test engineers of electrical systems, ECUs or ECU software and hardware.

2.3 How is this document organized?

This document contains one or several Base Technology concept specifications. Each concept is located in a separate section under "ECU Software Requirements" and can be seen as a stand alone document. Each concept typically has its own introduction chapters containing reference lists, definition and abbreviation lists and similar information.

2.4 How do you receive more information?

More information can be found in the reference document lists. Questions and support issues may also be sent to the team within Geely responsible for a specific ECU or to the central Base Technology team responsible for the specific project.

3 REVISION HISTORY

Revision	Date	Description	Author
01	2023-03-17		Tan Tingyue
06	2024-10-09	Based on V6 baseline	Tan Tingyue
07	2025.6.6	Based on V7 baseline	Tan Tingyue
08	2025-09-29	Based on V8 baseline	Tan Tingyue
09	2025-12-31	Based on V8a1 baseline	Tan Tingyue
	2026.7.6	Based on V9 baseline	Tan Tingyue

4 ECU SOFTWARE REQUIREMENTS

GEEA3.0基础技术需求V8基线的全部需求

ID	Allocated concept	Version	Status	Last Changed
x040000001DE130F9	A/B Swap flowchart	(5)	CS Released	2025-04-18
x040000001DE130F6	A/B Swap Requirements	(4)	CS Released	2025-04-18

ID	Allocated concept	Version	Status	Last Changed
x04000000204C9E86	ADAS_L3onUDS	(1)	CS Released	2025-09-19
x040000001EA78173	Authentication General Requirements	(1)	CS Released	2025-05-29
x0400000020F80FF0	Authentication with PKI Certificate Exchange	(6)	CS Released	2025-09-22
x0400000025FC893C	Autosar Configuration	(8)	CS Released	2026-05-22
x040000001E813AA4	AutoSar NM - CAN cross networks reqs	(2)	CS Released	2025-05-16
x040000001CF41B12	AutoSar NM - CAN reqs	(2)	CS Released	2025-03-27
x040000001E813A3E	AutoSar NM - Generic reqs	(2)	CS Released	2025-05-16
x0400000020BD0828	Autosar Time Synchronization	(3)	CS Released	2025-09-08
x040000001F10823E	BL - UDSonIP - Edge node reqs	(6)	CS Released	2025-06-12
x0400000013A4E88D	BL - UDSonIP - Generic reqs	(3)	CS Released	2023-07-27
x0400000014884B95	BL - UDSonIP - IP ECU reqs	(2)	CS Released	2023-09-21
x04000000195CA1C6	Bootloader - UDSonCAN	(4)	CS Released	2024-08-22
x040000000FF8512C	Bootloader - UDSonLIN - Generic	(1)	CS Released	2022-12-19
x0400000020FB92EA	Bootloader - UDSonLIN - LIN Slave	(3)	CS Released	2025-09-19
x0400000020FB8DCE	Bootloader requirements for SWDL with no check memory only for GEEA3.0	(3)	CS Released	2025-09-19
x0400000026379BF5	Bootoalder requirements for SWDL with check memory	(6)	CS Released	2026-06-03
x0400000002676D196	CAN and CANFD Data Link Layer	(6)	CS Released	2026-06-16
x04000000203FCCE2	CAN and CANFD Physical Layer	(5)	CS Released	2025-09-19
x0400000020FEF580	Car Configuration Master system requirement	(1)	CS Released	2025-09-22
x0400000020E1224E	Car Configuration Slave	(6)	CS Released	2025-09-12
x040000000FF74031	Data Compression and Encryption	(1)	CS Released	2022-12-16
x0400000020540974	Data security	(1)	CS Released	2025-09-03
x04000000025C5847	Delta Encoding Introduction	(2)	CS Released	2019-07-18
x04000000025C584C	Delta generation	(3)	CS Released	2020-10-08
x04000000025C5858	Delta installation	(4)	CS Released	2019-07-18
x0400000020A2D160	Diagnostic data for dual-bank ECU	(3)	CS Released	2025-09-04
x04000000215A1C0C	Diagnostic data related to Non-sensible Programming	(6)	CS Released	2025-11-18
x0400000012138629	Diagnostic secure communication for Edge Node	(1)	CS Released	2023-05-08
x0400000026061385	Diagnostics and ECU Platform	(3)	CS Released	2026-05-25
x0400000025E19908	DoCAN	(7)	CS Released	2026-05-19
x0400000020F5648C	DoIP - Client	(2)	CS Released	2025-09-18
x040000001EB9447F	DoIP - Edge Node	(6)	CS Released	2025-05-26
x040000001C5D0493	DoIP - Edge Node - Transport Layer Secutiry	(5)	CS Released	2025-03-14
x040000001005CCE4	DoIP - Generic Info	(1)	CS Released	2022-12-21
x0400000017B025BB	DoIP - InVehicle requirements	(4)	CS Released	2024-09-13
x040000001741B85F	DoLIN	(2)	CS Released	2024-04-08
x04000000243DF0F7	Domain Network Gateway - IP-IP/IP Routing	(2)	CS Released	2026-03-23
x040000001DE122BF	Domain Network Gateway DIAG	(2)	CS Released	2025-04-18
x040000001DE134F4	Domain Network Gateway SWDL	(1)	CS Released	2025-04-18
x04000000167A44BA	Dual Bank SWDL Based On MCU	(3)	CS Released	2024-02-15
x0400000020FBA98E	Dual Bank SWDL On POXIS System	(5)	CS Released	2025-09-19
x0400000020934758	E2E Overview	(1)	CS Released	2025-09-03
x040000001E126473	E2E Profile4 and Profile11	(1)	CS Released	2025-05-28
x040000002105CA6F	E2E Profile G	(2)	CS Released	2025-09-22
x0400000020526686	E2E Protection for Service	(2)	CS Released	2025-08-25
x04000000269292BB	ECU Addresses	(5)	CS Released	2026-06-16
x04000000204C349C	ePTIonUDS	(1)	CS Released	2025-09-19
x0400000026F70A89	Ethernet Data Link Layer	(7)	CS Released	2026-06-29
x0400000020FEF70B	Ethernet Physical Layer - 100BASE-T1	(5)	CS Released	2025-09-22
x040000001E95FE30	Ethernet Physical Layer - 100BASE-TX	(3)	CS Released	2025-05-19
x0400000020FEBD77	Ethernet Physical Layer - 1000BASE-T1	(5)	CS Released	2025-09-22

ID	Allocated concept	Version	Status	Last Changed
x040000001E6888B4	Ethernet TC10	(1)	CS Released	2025-05-06
x04000000025C585B	Extended information	(2)	CS Released	2019-07-18
x040000001DE1332D	External Flash A/B Requirements	(2)	CS Released	2026-02-27
x040000000FB82D0E	Firewall CAN11-CAN11	(1)	CS Released	2022-12-02
x040000001FFE6FA1	Firewall CANX-CAN11 Generic	(3)	CS Released	2025-07-29
x040000001628F5E1	For Dual Bank ECU but Without Seamless Update	(3)	CS Released	2024-02-16
x040000000FB827B9	Gateway connected to CAN	(1)	CS Released	2022-12-02
x040000001DE12992	Gateway connected to CAN supporting periodic responses	For3.0(3)	CS Released	2025-04-18
x040000000FB8274D	Gateway connected to External IP	(1)	CS Released	2022-12-02
x040000001DE12974	Gateway connected to External IP supporting periodic responses	For3.0(3)	CS Released	2025-04-18
x040000000FB82783	Gateway connected to Internal IP	(1)	CS Released	2022-12-02
x040000001DE12983	Gateway connected to Internal IP supporting periodic responses	For3.0(3)	CS Released	2025-04-18
x04000000139D694B	Gateway connected to X-LIN	(2)	CS Released	2023-07-25
x0400000020B6316E	Gateway Generic	(4)	CS Released	2025-09-08
x040000000FB826C8	Gateway Generic DIAG	(1)	CS Released	2022-12-02
x040000001DE1140F	Gateway Generic supporting periodic responses	(2)	CS Released	2025-04-18
x040000000FB82700	Gateway Generic SWDL	(1)	CS Released	2022-12-02
x040000000FB82DF0	Gateway X-LIN DIAG	(1)	CS Released	2022-12-02
x040000001E9A416F	Gateway X-LIN SWDL	(4)	CS Released	2025-05-20
x040000000FB8BDA7	General	(1)	CS Released	2022-12-15
x0400000026774064	General	(8)	CS Released	2026-06-18
x04000000266E3B6D	General	(6)	CS Released	2026-06-15
x040000001DE130FC	General A/B Swap Requirements	(5)	CS Released	2025-04-18
x0400000023EC1A75	General Bootlaoder requirements	(8)	CS Released	2026-06-23
x040000001F7A95DB	General Bootloader requirements for file system	(7)	CS Released	2025-07-11
x04000000212B534C	General Requirements	(2)	CS Released	2025-11-06
x040000001DE12BAB	General Requirements for XCP	(2)	CS Released	2025-04-18
x040000001FFD7AEF	General Software Authentication	(5)	CS Released	2025-07-29
x040000002730F83E	Generic HW requirements	(6)	CS Released	2026-06-30
x040000000FB857CF	Generic SWDL Overview	(1)	CS Released	2022-12-14
x040000000B8A8D36	Guideline for AliveTimeout setting	(1)	CS Released	2022-01-20
x04000000170CE864	Guideline for update bit mechanism	(2)	CS Released	2024-03-27
x04000000197CD585	Hardware Platform	(3)	CS Released	2024-08-30
x040000002216B897	HTTP Protocol Requirement	(1)	CS Released	2025-11-27
x0400000026FEA429	Internetworks General Specification	(7)	CS Released	2026-06-26
x04000000139D6938	InVehicle DoIP Gateway - IP - CAN	(1)	CS Released	2023-07-26
x0400000013A086F6	InVehicle DoIP GW DIAG	(1)	CS Released	2023-07-26
x0400000013A086FE	InVehicle DoIP GW SWDL	(1)	CS Released	2023-07-27
x04000000243DF370	InVehicle Network Gateway - IP-IP/DoIP GW	(2)	CS Released	2026-03-23
x0400000018D0C80B	IP Link Manager requirement	(4)	CS Released	2024-07-17
x0400000020F726BF	IP Routing and Firewalling	(2)	CS Released	2025-09-18
x0400000020F8E94C	LIN Data Link Layer - Generic Requirements	(5)	CS Released	2025-09-19
x040000001ED47308	LIN Data Link Layer - Master Requirements	(3)	CS Released	2025-05-29
x04000000197DC716	LIN Data Link Layer - Slave Requirements	(3)	CS Released	2024-08-30
x040000000FB86D40	LIN Physical Layer - HW Master requirements	(1)	CS Released	2022-12-19
x040000000FB86D4C	LIN Physical Layer - HW Slave requirements	(1)	CS Released	2022-12-19
x0400000020F8E7BB	LIN Physical Layer -Generic requirements	(3)	CS Released	2025-09-19
x040000001AF34343	Network Management - Generic reqs	(7)	CS Released	2025-01-06
x040000001E7E056B	Non-sensible Programming For OTA	For4.0(7)	CS Released	2025-05-29

ID	Allocated concept	Version	Status	Last Changed
x040000001DE13386	Non-sensible Programming Requirement For Private ECU	(1)	CS Released	2025-04-18
x040000002606069A	OBDonUDS	(2)	CS Released	2026-05-25
x0400000020F80FAB	Random number generation	(2)	CS Released	2025-09-19
x0400000020FEF383	RTP	(5)	CS Released	2025-09-22
x040000001DE124F0	Running Log Requirements	(1)	CS Released	2025-04-18
x040000002086D580	Scope	(1)	CS Released	2025-09-18
x04000000247F9C2C	Security Access algorithm	(8)	CS Released	2026-03-31
x0400000025FDFE6F	SignalToUDP	(3)	CS Released	2026-05-25
x04000000241767C2	Software Download	(8)	CS Released	2026-03-12
x040000002425095A	SOME IP	(7)	CS Released	2026-03-16
x04000000229708EE	SOVD - Classic Diagnostic Adapter	(2)	CS Released	2025-12-23
x0400000024150499	SOVD - Edge Node (Gateway)	(4)	CS Released	2026-05-14
x0400000024D49F3E	SOVD API Methods	(2)	CS Released	2026-06-08
x0400000026694E88	SOVD Architecture	(2)	CS Released	2026-06-15
x0400000025825965	SOVD Authentication	(2)	CS Released	2026-04-22
x0400000021AC1D00	SOVD Private Server	(1)	CS Released	2025-11-27
x0400000025BDEF69	SOVD Vehicle Update	(2)	CS Released	2026-06-08
x0400000013888566	Startup Time	(2)	CS Released	2023-07-18
x040000001DE10429	Sub Network Gateway DIAG - CAN-CAN supporting perdiodic responses	(1a2)	CS Released	2025-04-18
x0400000022ED1611	Time Sensitive Network	(5)	CS Released	2026-01-09
x0400000026067766	UDP Network Management	(2)	CS Released	2026-05-25
x040000002587394F	UDS Data - Basetech Data Requirements	(10)	CS Released	2026-06-25
x040000002625AB52	UDS Data - General Requirements	(10)	CS Released	2026-06-01
x04000000259FE125	UDS Services	(10)	CS Released	2026-05-01
x040000000FB84F52	UDS Session client reqs	(1)	CS Released	2022-12-02
x040000001E9A250D	UDS Session ECU reqs	(5)	CS Released	2025-08-18
x0400000020F5D254	UDS Session generic reqs	(2)	CS Released	2025-09-18
x04000000101685B3	UDSonCAN - Generic	(1)	CS Released	2023-01-04
x040000000FB854D0	UDSonCAN - Private Programmable CAN ECU	(1)	CS Released	2022-12-02
x040000001016A48E	UDSonCAN - Public CAN ECU	(1)	CS Released	2023-01-04
x0400000020F80F8F	UDSonIP - Edge node reqs	(3)	CS Released	2025-09-19
x04000000139AF56A	UDSonIP - Generic reqs	(3)	CS Released	2023-07-22
x040000000FB856E9	UDSonLIN - Generic	(1)	CS Released	2023-01-10
x04000000103AF47E	UDSonLIN - Generic LIN master	(1)	CS Released	2023-01-12
x040000002077B1E6	UDSonLIN - LIN Slave	(3)	CS Released	2025-08-28
x0400000016A044C0	Vehicle Gateway - IP-CAN(Network)	(4)	CS Released	2024-02-29
x040000000FB827F0	Vehicle Gateway - IP-IP	(1)	CS Released	2022-12-02
x04000000208E37AD	Vehicle Gateway DIAG	(5)	CS Released	2025-09-02
x040000001DE129A1	Vehicle Gateway DIAG - IP-IP supporting periodic responses	For3.0(3)	CS Released	2025-04-18
x0400000016B9C1D6	Vehicle Gateway J2534	(3)	CS Released	2024-03-06
x040000001B060EDE	Vehicle Gateway SWDL	(4)	CS Released	2025-01-09
x04000000025C5855	Vehicle Service Delivery Platform	(2)	CS Released	2019-07-18
x04000000025C584F	Vehicle Software Configuration	(2)	CS Released	2019-07-18
x040000001FFE5A75	Versatile Binary Format Generic reqs	(1)	CS Released	2025-07-29
x04000000153DE2A5	Versatile Binary Format Generic reqs	(2)	CS Released	2023-11-13
x04000000137E39B3	Versatile Binary Format Specification	(2)	CS Released	2023-07-13
x0400000022964EDB	Versatile Binary Format Specification	(2)	CS Released	2025-12-23
x0400000020FEBDF7	VLAN Configuration	(5)	CS Released	2025-09-22
x04000000224C1351	WLAN General	(2)	CS Released	2026-03-19
x04000000025C5852	Workshop System	(2)	CS Released	2019-07-18
x0400000026E6B757	ZEVonUDS	(3)	CS Released	2026-06-24
x0400000016B40073	Zone Network Gateway - CAN - CAN without	(1)	CS Released	2024-03-13

ID	Allocated concept	Version	Status	Last Changed
	TP			
x0400000016C42CAB	Zone Network Gateway - CAN-CAN	(3)	CS Released	2024-03-13
x04000000245CBC37	Zone Network Gateway DIAG	(5)	CS Released	2026-03-27
x04000000148E9FBB	Zone Network Gateway SWDL	(4)	CS Released	2023-09-22
x0400000020FEF3BB	Zone node system requirement	(1)	CS Released	2025-09-22

4.1 ECU Platform

4.1.1 General requirement

4.1.1.1 Generic HW requirements

•Overview

4.1.1.1.1 Overview

Who should read this document? The document is mainly intended for HW designers, HW suppliers and test engineers.

4.1.1.1.1.1 Definitions & Abbreviations

Abbreviation	Description
ASIL	Automotive Safety Integrity Level - ASIL specifies risk classes for safety relevant systems in automobiles
CAN	Controller Area Network - A standard for multiplex communication, called Controller Area Network. Described in ISO 11898 and in “CAN Specification” from Robert Bosch GmbH
DMA	Direct Memory Access
DSP	Digital Signal Processing
ECU	Electronic Control Unit - with a computer / microcontroller
EMC	Electromagnetic Compatibility
EMI	Electromagnetic Interference
ESD	Electrostatic Discharge
HW	Hardware
MCU	Micro Controller Unit
MOST	Media Oriented System Transport
PCB	Printed Circuit Board
PLL	Phase Locked Loop
ppm	Parts Per Million
PWM	Pulse Width Modulation
RFI	Radio Frequency Interference
RTOS	Real Time Operating System
SCI	Serial Communication Interface
SMD	Surface Mount Device
SW	Software
UI	Unit Interval

Definition	Description
Multiplex	To interleave or simultaneously transmit two or more messages on a single channel.

Network	A set of electronic and cabling devices facilitating the multidirectional exchange of information encoded in serial form. Each electronic device (in this case: ECU) is equipped with a specific, standardised electronic interface in order to guarantee compatibility between exchanged binary items of information.

4.1.1.1.1.2 Requisite documents

Reference ID	Document
[HW_1]	CAN Physical Layer - HW Generic requirements
[HW_2]	LIN Physical Layer - HW Generic requirements
[HW_4]	ISO 26262 Road vehicles -Functional safety

•Requirements

4.1.1.1.2 Requirements

4.1.1.1.2.1 Memory capacity

663137 v4 **Spare memory**

Version history:
(4): BT需求中无SOC 均值要求，但座舱和智驾这边都有使用SOC 均值来评估CPU load 性能；通过拉会对齐，增加SOC 均值要求。 详细信息见文档：https://gri.feishu.cn/wiki/TKg9wdPX8iySm1k4G9QcF6Z2nNc

Purpose: Handle future expansion of software functionality.
Source: 软件技术委员,管理办法见飞书：https://gri.feishu.cn/docx/TPp9dLs3eobQIkxYGUTcmcfLnUb
Verification Method: Analysis
Legacy ID: 53746v0

- **The CPU load assessment regulations for the MCU system are as follows:**

The average load of a single core is less than 70% of the total computing power of a single core, the peak load of a single core is less than 90% of the total computing power of a single core, and the load of all cores in any actual scenario is less than 85% of the total computing power of all cores.

- **The CPU load assessment regulations for the SOC system are as follows:**

The average load of a single core should be less than 80% of the total computing power of a single core, the peak load of a single core should be less than 90% of the total computing power of a single core, and the load of all cores in any actual scenario should be less than 80% of the total computing power of all cores.

4.1.1.1.2.2 Non Volatile Memory (NVM)

Non-volatile memory, in the most basic sense, is computer memory that can retain the stored information even when not powered.

NVM in this section address:
· Flash memory
· EEPROM

NVM memory technologies are subject to physical degradation that can eventually lead to device failure. Vendors use two end-of-life parameters to specify the performance of reprogrammable NVM memory. These two parameters are endurance and data retention. They are not mutually exclusive.

Endurance is the parameter that specifies the cumulative program/erase cycling capability of any individual sector within a device. Each sector-erase operation has the capability of introducing defects into the memory cell structure that may accumulate over time. At some point, these defects may prevent a cell from programming, erasing or

reading. When such a failure occurs in a cell that contains critical data, the device may cause system failure and the device is effectively end-of-life.

Data retention, is a cell-level specification that indicates how long data can be expected to be reliably retrieved following programming under specific conditions. The ability of the contents of any particular cell in an array to return correct programmed status over a period time is affected by numerous issues, including device design, manufacturing process variation, operating temperature and voltage, electrostatic environment, exposure to radiation, cumulative erase cycles, etc.

There is a real and measurable relationship between data retention and endurance (erase cycling), in all non-volatile flash devices. Defect accumulation resulting from repeated erasures impacts the ability of the programmed contents of a cell to be properly discerned by a read operation after a period of storage.

Executable code must necessarily have high data retention since the code must be correct for proper system operation. Since executable code is changed very infrequently, the sectors containing this code are not exposed to many erase cycles over the system life and the data within those sectors have very high retention capability.

Data that is captured periodically, such as calibration parameter data or event log data, is subject to frequent replacement. Since flash array storage capabilities are limited, frequent replacement equates to frequent sector erasures. For this reason, it is important to assess the impact of sector erase cycling on the data retention capability of the flash device and the data retention requirements of the system.

663142 v1 **NVM Error Code Correction**

Version history:
(1):

Purpose: Guarantee faultless NVM content.
Source:
Verification Method: Analysis
Legacy ID: 53752v0

NVM with built in ECC (Error Code Correction) has the ability to detect and correct single bit faults and detect double bit faults. A DTC with counters shall be set on single- and double bit errors to inform how often a fault occur.

663141 v1 **NVM bootsector**

Version history:
(1):

Purpose: Avoid accidental erase of the bootloader
Source:
Verification Method: Analysis
Legacy ID: 53751v0

The NVM shall contain a separate bootsector area, which is hardware write/erase protected.

663139 v1 **NVM during power loss**

Version history:
(1):

Purpose: Guarantee faultless NVM content.
Source:
Verification Method: Analysis
Legacy ID: 53749v0

NVM areas used for data storage shall be maintained during power loss and power failure. If an aborted NVM erase/write operation result in corrupted data, the data shall be restored to latest valid values.

663143 v1 NVM modifications

Version history:
(1):

Purpose: Guarantee faultless NVM content.
Source:
Verification Method: Analysis
Legacy ID: 53753v1

The microcontroller shall only modify NVM during normal ECU operation voltage.
The hardware/software design shall ensure successful completion of any NVM modification of a single NVM cell - even if the ECU loses power at the same time as the NVM modification starts.
The hardware/software design shall ensure data coherency - even when NVM modifications are interrupted due to a power loss.
When NVM is being modified, it is imperative that the microprocessor continue normal processing long enough to guarantee the write or erase operation completes correctly (especially when using on-chip EEPROM) before entering low-power mode. This can also apply to off-chip EEPROM where the microcontroller controls the power supply of the EEPROM.
The ECU shall not enter Sleep- or Low-Power mode while a NVM cell is being modified.
The ECU shall not perform a software controlled reset while a NVM cell is being modified.
After the sleep criteria have been satisfied, the algorithm must wait for NVM modification to complete before switching to low-power mode.

663144 v1 NVM process

Version history:
(1):

Purpose: Guarantee faultless NVM contents over vehicle lifetime.
Source:
Verification Method: Analysis
Legacy ID: 53755v0

The NVM process must be robust with an endurance of more than 1k write/erase cycles and data retention over 15 years with a fault drop out of less than 10 ppm of all devices, assuming application specific worst case conditions

856222 v1 Sector switch

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID:

When storing data in EEPROM or DFLASH, if the sector switch strategy is triggered, it needs to be completed within 5000ms.
数据存储EEPROM或DFLASH时，如果触发翻页策略，需在5s内完成。

4.1.1.1.2.3 Input and Output lines

663146 v1 **Error detection on Input and Output lines**

Version history:
(1):

Purpose: To allow efficient fault tracing.
Source: UDS Data - Diagnostics requirements on external input and output lines.
Verification Method: Analysis
Legacy ID:

All Input and output lines shall have error detection functionality to be able to detect:

- Open circuit
- Short circuit to ground
- Short circuit to battery

663145 v2 **Input and Output lines**

Version history:
(2):

Purpose: Improve EMC performance of the ECU.
Source:
Verification Method: Analysis
Legacy ID: 53759v1

Protection from transients shall be provided for all types of I/O lines.
The filtering shall be designed to match the specified bandwidth for each type of I/O interface.
The state of the I/O lines shall have a predefined value when powering up and down the ECU or when the ECU is in a reset state in order to not jeopardise the functionality
Further requirements are specified in the hardware specification for each ECU.

4.1.1.2 Hardware Platform

•Requirements

4.1.1.2.1 Requirements

660803 v1 **Multiple CPU Design**

Version history:
(1):

Purpose: To clarify that requirements are applicable for the complete ECU and always considered as one entity.
Source:
Verification Method: Analysis
Legacy ID:

An ECU is always considered as one entity and is not allowed to create bottlenecks due to connection dependencies between multiple CPUs.

***Note**: This requirement is valid for any ECU which uses multiple CPUs, e.g. one AUTOSAR based microcontroller for vehicle*

communication, and one microprocessor running an operating system such as Linux.

•Avoid low voltage shut-down

4.1.1.2.2 Avoid low voltage shut-down

714034 v1 **Hardware solution**

Version history:
(1):

Purpose: To avoid the low voltage shut-down.
Source:
Verification Method: Analysis
Legacy ID:

It is applicable for simple controllers that only wake up through CAN and LIN, such as panel buttons, motor actuators, etc.
1. It is recommended to prioritize the use of integrated chip solutions (MCU and CAN/LIN integrated), as software development is simple and can avoid systematic shut-down caused by software timing abnormalities or abnormal working voltage.
2. If using a separation scheme (MCU and CAN/LIN separation), it is recommended to choose a CAN/LIN transceiver with INH interrupt wake-up, and configure RX and INH wake-up sources to avoid working conditions where a single wake-up source fails and the controller cannot wake up; It is recommended to choose a chip with a watchdog function or a separate reset chip scheme for the power supply. The reset voltage point should be set at -10% of the MCU working voltage, which can reset the system in case of abnormal system shut-down or voltage drops, avoiding systemic shut-down.
3. Controllers with functional safety requirements must have an external watchdog design.

主要针对只通过CAN,LIN唤醒的简单控制器，如面板按键，电机执行器等。
1. 建议优先选用集成式芯片方案(MCU+CAN/LIN一体)，软件开发简单，可以避免软件时序异常或工作电压异常导致的系统性死机问题。
2. 如果选用分离方案(MCU+CAN/LIN分离)，建议选用带INH中断唤醒的CAN/LIN收发器，配置RX和INH两个唤醒源，避免单一唤醒源失效控制器唤醒不了的工况;电源方案建议选用带看门狗功能芯片，或分离的复位芯片方案，复位电压点设置在MCU工作电压的-10%，可以在系统异常死机或电压异常跌落时对系统进行复位，避免系统性死机问题。
3. 有功能安全要求等级的控制器，要求必须有外部看门狗设计。

714036 v1 **Software solution**

Version history:
(1):

Purpose: To avoid the low voltage shut-down.
Source:
Verification Method: Analysis
Legacy ID:

1. When designing software for controllers, it is necessary to configure the reset voltage range of the MCU. Generally, the reset voltage is set to -10% of the operating voltage of the MCU to avoid the probability of software logic errors and shut-down caused by low voltage.
2. After the controller receives the sleep command, it should close the CAN transceiver firstly, then deal with process events, and enter sleep mode after clearing the registers and checking their status for abnormalities.
3. When the controller wakes up through the bus, within the time window from receiving the sleep command to actual sleep, it must be awakened again and enter the power on/off cycle.

1. 控制器设计软件，必须配置MCU的复位电压区间，一般配置复位电压在MCU工作电压的-10%时进行复位，避免过低电压导致软件逻辑错误概率性死机问题。

2. 控制器接收休眠指令后，休眠逻辑为先关闭CAN收发器，再处理进程事件，在清寄存器并检查寄存器状态无异常后再进入休眠状态。
3. 控制器通过总线唤醒时，在接收休眠指令到真正休眠的时间窗口内，当接收到唤醒指令时必须重新唤醒再进入上下电循环。

4.1.1.3 Diagnostics and ECU Platform

•Introduction

4.1.1.3.1 Introduction

Statement of Work (SOW)
Diagnostics and ECU Platform Statement Of Work is the name of the SOW (Statement Of Work) from Diagnostics & ECU Platform and it is essential that it is part of the ESOW for each node that intend to use requirements from our area.

This SOW defines the additional required engineering tasks, responsibilities and deliverables for the design, engineering development, quality assurance and test of an Electronic Control Unit (ECU) / Subsystem that shall conform to the Vehicle Program requirements on Diagnostics and ECU Platform.

SW Platform
Issues with regards to the AUTOSAR platform and its development are described in the SOWs (mentioned above) with the Tier 1.

The main purpose of SWDL is:

- Reduced time for development
    - Avoided lead time for hardware changes
- Reduced number of hardware partnumbers
    - Variants are configured with software
    - Less delivery units
    - Less units in stock saves space (factory, aftersales and suppliers)
- The vehicle is after production possible to change
    - Updating of software “quality issues”
    - New functionality “replaces hardware changes”
    - Upgrade “increased/changed functionality”

A bootloader consists of a primary and secondary bootloader (xBL) that supports the software download protocol. The bootloader always gain program control after a reset, and passes the control over to the application when it has determined that no download is requested.

The software download concept is based on a primary bootloader that is fixed in the ECU and always executed at power on before the application is started and a secondary bootloader that is downloaded before downloading the applications and configuration datafiles.

Software download functionality is not a part of the AUTOSAR based ECU architecture. To support software download a bootloader shall be used (a stand alone application). Enter and exit of the bootloader is handled by a common API, known by the AUTOSAR based ECU architecture and the bootloader.

CarConfig

Car Configuration is used to organize car specific information and to minimize the software variants by sending out information about the car over the multiplexed network. This makes it possible to use the same software to perform functions that differ between the car models.

693274 v1 Unused bit valuse

Version history:
(1):

Purpose: Defines the execution scheme of the unused bit value.
Source:
Verification Method: Test
Legacy ID:

All unused bits and bytes shall be set to zeros by the sender except LIN cluster.

The unused bits and bytes of LIN master should be defined by SDB (zeros currently) . The LIN slave shall be set to ones.

The deviation shall be approved by Basetech team.

1075831 v1 Watchdog management

Version history:

Purpose: Avoiding abnormal resets during software operation
Source: 《软件架构设计指南》、《软件架构设计》
Verification Method: Analysis
Legacy ID:

The ECU shall avoid abnormal resets during software operation, especially for multi-task concurrency.
ECU在软件运行过程中应避免异常复位，特别是多任务并发过程中。

4.1.1.3.1.1 Definitions & Abbreviations

Abbr.	Stands for	Description
AUTOSAR	AUTomotive Open System ARchitecture	
CAN	Controller Area Network	
N/A	Not Applicable	
IP	Internet Protocol	The IP is the principal communications protocol used for relaying datagrams (packets) across an internetwork using the Internet Protocol Suite.
SW	SoftWare	
ECU	Electronic Control Unit	Electronic control unit with a computer/microcontroller.
TCP	Transmission Control Protocol	TCP is one of the core protocols of the Internet Protocol Suite. TCP is one of the two original components of the suite, complementing the IP, and therefore the entire suite is commonly referred to as TCP/IP.
SWC	SoftWare Component	
LIN	Local Interconnect Network	Low speed single wire network
MOST	Media Oriented Systems Transport	Optical Multimedia Network
RTE	RunTime Environment	
SRD	System Requirements	This document is an SRD.

	Description	
SWE	Swedish	Used to give a Swedish equivalence.
Edge node		An ECU (internal vehicle network node) which is the network node that is exposed towards the external network. The node implements a diagnostic over IP-protocol (DoIP - ISO13400) and provides diagnostic access to test equipment.
GMRDB	Global Master Reference DataBase	Database used for keeping track of all diagnostic entities in the vehicle.
S-feed	Switched power supply	Switched power supply– S-feed –is typically controlled by an external switch, usually a relay, known as "power-switched".
C-feed	Continuos power supply	Continuous power supply– C-feed –is typically (but not necessary) directly connected to battery, known as "30-feed" or "battery-powered".
TBD	To Be Defined	

4.1.1.3.1.2 Requisite documents

Ref.	Source	Description
[1]	Internal	Autosar Network Management - additional requirements
[2]	Internal	Components for Physical Interfaces
[3]	Internal	LIN Datalink Layer
[4]	Internal	Diagnostic and Bootloader Gateway Specification
	Internal	UDS Services
	Internal	UDSonCAN
	Internal	FlexRay Data Link Layer
	External	CAN 2.0B Standard
	External	MOST Specification 3VE2
	External	RFC 1918

4.1.1.3.1.3 Selection list

AUTOSAR BSW clusters

Regardless of the clustering level of the AUTOSAR BSW, the following Clusters shall be separable from the remaining BSW modules/clusters.

- Communication Cluster
- Diagnostic Cluster: DEM, DCM, FIM (optional), DET (optional)
- Mode management Cluster: ComM, EcuM, BswM

Suggestion of AutoSar protocol stack by GEELY:

No.	Tier2	Comments
1	Vector	
2	Mentor	
3	KPIT	
4	EB	
5	Hirain	Local（恒润）
6	isoft	Local（上海普华）
7	REACH	Local（东软睿驰）

If the tier1 protocol stack is not selected in the above list, GEELY will not pay any fees for subsequent changes.

We also have a few Tire1:s using their own AUTOSAR BSW, these are not openly available on the market:

- **Bosch**

- **Continental**

Other BSW developed by tier1 shall be handshaked with SWRT according to basetech SWRS.

4.1.1.4 ECU Addresses

•Requirements

4.1.1.4.1 Requirements

660805 v5 **ECU Addresses**

Version history:
(5): Update the IP netmask.

Purpose: To deploy valid ECU addresses to each ECU.
Source: -
Verification Method: Analysis
Legacy ID: 76545v32

CAN
ECUs with CAN interface
All CAN ECUs shall have a CAN ID according to ECU SWRS Requirements "ECU Addresses in GEEA3.0 - Superset". The CAN ID is a static address calculated from the ECU address and is used to address CAN ECUs for SWDL and Diagnostics. The CAN ID is included in the ECU extract.

LIN
LIN slave ECUs, using SWDL functionality (bootloader)
All LIN ECUs using a bootloader shall use a complete ECU address according to ECU SWRS Requirements "ECU Addresses in GEEA3.0 - Superset". The ECU address is a static address that is used to address LIN ECUs for SWDL and Diagnostics. The ECU address is included in the LDF file.

LIN slave ECUs without SWDL, but using LIN diagnostic frames
A LIN slave ECU (not using a bootloader) shall use a NAD address according to ECU SWRS Requirements "ECU Addresses in GEEA3.0 - Superset". Otherwise the LIN master will not be able to address diagnostic requests to the LIN slave. The NAD address is the last byte from the 2 byte ECU address and is used for diagnostics. The NAD address is included in the LDF file.

LIN slave ECUs without SWDL and without LIN diagnostic frames
If there is no need for a LIN master to address diagnostic requests to a LIN slave ECU, then a NAD address is NOT needed. In this case the LIN slave will be assigned a node address default value. The default value will be 0x7D. In case of multiple LIN Slaves on the same network the addresses will be set in decreasing order starting from 0x7D. The LIN address to be used by the LIN slave is included in the LDF file.

LIN masters
LIN masters shall use the NAD according to ECU SWRS "ECU Addresses in GEEA3.0 - Superset" in order to address diagnostic requests to LIN slaves.

Other Bus
Slave ECUs, i.e. with GMSL interface
All Slave ECUs shall use a complete ECU address according to ECU SWRS Requirements "ECU Addresses in GEEA3.0 - Superset". The ECU address is a static address that is used to address ECUs for SWDL and Diagnostics.

IP
ECUs with IP-based interfaces
The internal MAC addresses and internal IP addresses of IP-ECUs shall be assigned as according to ECU SWRS Requirements "ECU Addresses in GEEA3.0 - Superset". The IP address must be implemented into the IP ECU by supplier. The following IP address space has been chosen: 198.18.0.0/16 for the vehicle network. From this

address space, the vehicle IP network is using one subnet according to below:

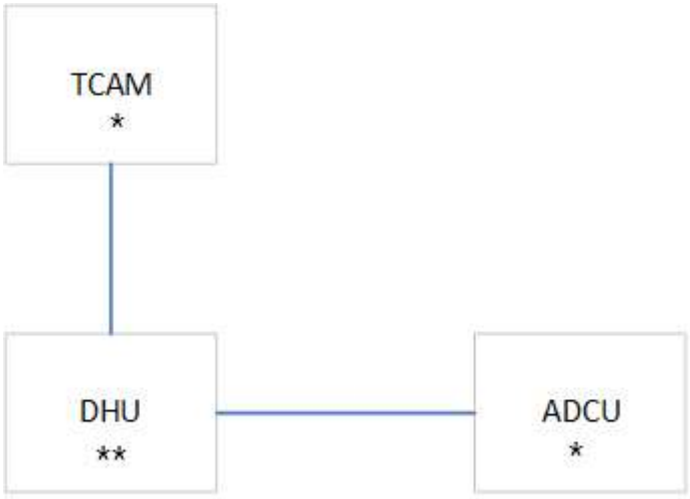


Figure:IP subnets of the GEEA 3.0 network

NOTE 1: In *Figure IP subnets of the vehicle network* and in the *table "ECU Addresses"*, some IP addresses are marked with a * or a **.

* == Internal ECU-IP address used for Diagnostic (ISO13400) TCP clients (Diagnostic Server) to connect to the Edge Node (DHU).

** == Internal ECU-IP address used by Edge Node (DHU) acting as TCP Server (Diagnostic Client) offering Diagnostic Service

Overview of IP-address space :

The vehicle IP network shall use one subnet according to table below.

Network Friendly Name:	**VEHICLE IP NETWORK**
Subnet ID:	198.18.0.0 / 16
Usable IP addresses:	198.18.0.1 – 198.18.255.254
Netmask:	255.255.255.0
~~**Broadcast address:**~~	~~198.18.255.255~~
Default GW address:	DHU – Ethernet interface according to, ***Table:*** *ECU addresses – Showing IP-based ECU's*

IP address space allocation depending on physical network

The IP address range of 198.18.0.0 / 16 has been divided into different address ranges to be able to identify which physical communication carrier is used for a specific ECU. The tables below are used in conjunction with *Req. ID* 660814.

Example: ECU's which are connected to Ethernet will have an IP address range of: 198.18.32.1-198.18.47.255.

Network Friendly Name:	**Ethernet**
Network ID:	198.18. **32**.0
IP addresses:	198.18.32.1 – 198.18.47.255

Relationship between ECU addresses and IP addresses
The ECU address consist of 16 bits and are divided into 4 nibbles as follows: [Address range].[Network id].[Zone

id].[Ecu id]

NOTE: The addressing format for diagnostic request and response, source address (SA) and target address (TA), using Zone, Network ID and ECU ID is defined in [*UDS Services*].

The IP address v4 consist of 32 bits and are divided into 4 bytes. It is required for diagnostic purposes, that it shall be possible to calculate the ECU address from the IP address of a specific ECU. The IP addresses have been divided as follows:

198.18.{ssssnnnn+VLAN ID}.{zzzzeeee}.

ssss = Consists of 4 bits, defines the start of the Network ID.
nnnn = Consists of 4 bits, defines the Network ID
zzzz = Consists of 4 bits, defines the Zone ID
eeee = Consists of 4 bits, defines the ECU ID

The ECU addresses are always represented in hexadecimal form and the IP addresses are written in decimal form. This means when calculating the ECU address from the IP address there is a need for conversion from hexadecimal to decimal.

Example:
ECU Address 0x1101, would mean IP address 198.18.44.17.

198.18.[S] [N].[Z][E]

[S] = 0x2 (Network Friendly Name = "**Ethernet**", Network ID: 198.18. **32**.0)
[N] = 0x0
[Z] = 0x1
[E] = 0x1

Table: IP Address calculation

IP address calculation				
	198	**18**	**{ssssnnnn+VLANID}**	**{zzzzeeee}**
Binary	1100 0110	0001 0010	0010 0000+0000 1100	0001 0001
Hex	0xC6	0x12	0x20+0xC	0x11
Decimal	198	18	44	17

660804 v2 **Zone Address**

Version history:
(2):

Purpose: Clarification of the addresses of the Zone node.
Source:
Verification Method: Analysis
Legacy ID: 342168v0

All Zone shall use the address 0x1Y01 where the two last nibbles, 01 states that the ECU is a Zone node.

4.1.1.5 Software Download

•Requirements

4.1.1.5.1 Requirements

1071359 v1 **Additional SWDL Performance for POXIS system**

Version history:

Purpose: For ECU with operating system, data transmission and installation are separated to adapt to actual development business.
Source:
Verification Method: Analysis AND Test
Legacy ID:

For *One ECU with an POXIS system*, its SWDL mainly includes two parts: file data transfer and system update (reassembly, decompression and installation). Therefore, this requirement constrains these two blocks separately.

Data transfer (SID 0x34 ~ SID 0x37):

- Ethernet 100Mbps:

ECU	Rates
DHU (Only architecture 3.0 or above)	≥ 110 Mbytes/min
Other ECU, e.g. ADCU, AI Box	≥ 225 Mbytes/min

- Ethernet 1000Mbps:

ECU	Rates
ECU, e.g. ADCU, AI Box	≥ 350 Mbytes/min

System update/installation time:

Main System Type	Update/Installation time	Example	Remark
Android	≤ 16 min	E.g. DHU, RDHU	When the size of ECU VBF is at least 9GB.
QNX	≤ 4 min	E.g. ASDM9	
Linux	≤ 9 min	E.g. ADCU, AI Box	When the size of ECU VBF is at least 9GB.

Note:
1. Suppliers shall provide information on which type of system ECU belongs to.
2. In order to manage the update time of ECU, it should not be more than 5 min for one single ECU update time. This deviation needs to be approved by Bastech Team.

660837 v3 **Bootloader CAN busoff handling**

Version history:
(3): Update the verification method from 'Analysis' to 'Analysis and Test'

Purpose: To increase availablility of the system. Avoid that an ECU gets stuck in bus-off state, or exits programming session due to network errors.
Source:
Verification Method: Analysis AND Test
Legacy ID: 392111v0

If hardware support automatic CAN busoff recovery, that recovery mechanism shall be enabled.
If hardware doesn't support automatic bus-off recovery the bootloader shall check all active CAN controllers for bus-off status at maximum 10 ms intervals and if bus-off is detected start the bus-off recovery sequence within 10ms.
This requirement applies to Primary Boot Loader (PBL) and Secondary Boot Loader (SBL).

Note1*: If a permanent busoff condition is detected that can not be recovered, and a complete and compatible application exist,*

the S3server timer shall still run and will finally elapse, causing and exit of the programming session.
Note2: *For an ECU connected to multiple networks, it will include the main network that receive diagnostic protocol and sub network. If only the sub network occurs Busoff while the main network received by the diagnostic protocol is in a normal state, the processing of DIAG and SWDL by the ECU should not be affected.*

660838 v2 Bootloader routing - Spare networks

Version history:
(2):

Purpose: To be able to handle future growth of networks.
Source: EESE strategical decision
Verification Method: Analysis
Legacy ID: 316251v2

The ECU shall prepare the bootloader for adding new networks according to the definitions made by the Base Technology Team.

```
1 : Z : N : E
|   |   |   |
|   |   |   --- ECU ID
|   |   --- Network ID
|   --- Zone ID
--- Always 1
```

For each Zone node:

- Network ID usable range shall be: 0x0..0xF.
- ECU ID usable range shall be: 0x0..0xE.(1)

Note 1: *ECU ID equal 0xF shall be reserved for functional addressing of ECUs.*
Note 2: *Number of Networks and ECUs shall be according to design.*
Note 3: *This requirement is applicable for DHU,ZCUDM and ADCU/ASDM.*

660839 v1 Deployment of the J2534 requirements

Version history:
(1):

Purpose: To define which ECU that shall implement the J2534 requirements.
Source: Diagnostic and Bootloader Gateway Specification
Verification Method: Analysis
Legacy ID: 76536v1

The ECU defined by The Base Technology Team shall implement the J2534 requirements according to *Diagnostic and Bootloader Gateway Specification* for reference see Diagnostics & ECU Platform - Requisite documents and also the Design Requirement - IDs according to J2534 standards

660841 v1 In-Place Reconstruction of Delta Files

Version history:
(1):

Purpose: To reduce the time spent on the programming phase and the time spent in transfer over data communication networks.
Source: General Bootloader requirements - Delta encoding

Verification Method: Analysis
Legacy ID:

The ECU shall support in-place reconstruction of software delta files according to the [*Delta Encoding*] specification.

This requirement is applicable for ECUs which software size exceeds 10MB in size or exceeds 10min in programming time.

Note: *The programming time is calculated from the first EraseMemory/RequestDownload request to the last positively confirmed RequestTransferExit/CheckMemory response (SBL is excluded from this time calculation).*

660842 v7 SWDL Performance

Version history:
(7): 1. SWDL rate shall not be under 10 % of the expected value defined by this requirement. 2. The expected rate of LIN ECU changes to 0.034MB/min according to LIN Boot schedule.

Purpose:
1. To optimize SWDL in factory and aftersales.
2. To meet the target vehicle reprogramming time (30 min).

Source:
Verification Method: Analysis AND Test
Legacy ID:

The figures below includes expected SWDL rate for the ECU to erase software, download software and verification. This requirement is valid on ECU level, assuming an ideal downloading tool. Supplier should do their best to make the SWDL time as short as possible and the measured SWDL rate shall not be under 10 % of the expected value defined by this requirement.

The SWDL performance shall follow the below rate for each ECU particular bus:

$$Calculated\ Rate\ [MB/s] = \left(\frac{SWDL\ compressed\ size\ [MB]}{Total\ measured\ time\ [s]}\right)$$

Total measure time is the time between the time stamps T1 and T2 described below:
T1= First Single 'Security Access' UDS service request
T2= 'Complete and Compatible Check' UDS service response

Expected rates:

LIN ECU: 0.034MB/Min

CAN ECU: 1MB/min

CANFD ECU: 3MB/min~3.5MB/min

Ethernet ECU(MCU): 10MB/min

Ethernet ECU(SOC): 100MB/min

NOTE 1: *Total Measured Time shall not overpass the total SWDL time in factory defined by EOL: 480s*
NOTE 2: *An ECU is only allowed to download files in Factory after agreement is made with the Base Technology Team.*
NOTE 3: *It should not be more than 5 min for one single ECU reprogramming time.*
NOTE 4: *One ECU with an POXIS system, viewed from an SOC perspective.*

660843 v2 SWDL interrupt of ECU power removing

Version history:
(2): Update the verification method from 'Test' to 'Analysis and Test'

Purpose: The SWDL interrupt of ECU power removing shall not prevent subsequent reprogramming
Source:
Verification Method: Analysis AND Test
Legacy ID:

The removing ECU power at any time shall not prevent the subsequent reprogramming of any normally programmable memory area. In addition, the complete or partial downloading of any non operational or partly operational software into the programmable memory areas of the ECU shall not prevent the subsequent reprogramming of any normally programmable memory area.

660840 v1 **Separate software files**

Version history:
(1):

Purpose: To reduce the amount of non-diffable content in software files and to enable the possibility of flexible ECU updates.
Source:
Verification Method: Analysis
Legacy ID:
Purpose:

In order to increase software flexibility and reduce the amount of non-diffable content in vbf-files, all graphics, fonts, and speech files used by an ECU must be delivered separately without any software dependencies.

Note: *This requirement is applicable for ECUs which stores vehicle HMI, fonts, or speech related files.*

4.1.1.6 Startup Time

•Requirements

4.1.1.6.1 Requirements

660848 v2 **Diagnostic Startup Time**

Version history:
(2):

Purpose: In order to conform with tools used during manufacturing, and reduce the time spent in workshops and over-the-air diagnostics/SWDL use cases.
Source:
Verification Method: Test
Legacy ID:
Purpose:

The ECU shall startup within 2.5 seconds and have full diagnostic and software download capabilities within this time. The time shall be measured from a state where the power to the ECU is completely turned off until it is capable of receiving diagnostic requests from both an internal tester in the vehicle, and an external tester connected through OBD.

Note*:*
1. *This requirement is valid for all ECUs with diagnostics and/or software download capabilities, and is applicable for ECUs with AUTOSAR based microcontrollers.*

2. *This timing value might be too short for those micro controllers/micro processors running on an operating system such as Linux. And for these specific cases, startup value other than specified should be used after an agreement with Base Technology team.*

4.1.1.7 Diagnostic secure communication for Edge Node

•Requirements

4.1.1.7.1 Requirements

676045 v1 **Diagnostic secure communication scheme**

Version history:
(1):

Purpose: Define the secure communication implementation.
Source:
Verification Method: Analysis
Legacy ID:

In order to protect the security of diagnostic communication, a secure communication scheme will be adopted between diagnostic tool and vehicle gateway in GEEA3.0 electrical and electronic architecture. When different diagnostic secure communication is configured, then:
1. Establish DoIP TLS between the diagnostic tool and the vehicle gateway, or
2. UDS secure diagnostic communication between the diagnostic tool and the entire vehicle, or
   - For diagnostic services supported by Vehicle GW but do not require protection, the Vehicle GW can perform diagnostics.
   - For diagnostic services supported by Vehicle GW and require protection, as well as the routing function of the Vehicle GW, unidirectional authentication of diagnostic service 0x29 is required.
3. Combination of the above.

Note:
The implementation of DoIP TLS, see charpter "DoIP - Edge Node - Transport Layer Secutiry" for details.
The implementation of diagnostic service 0x29, see charpter "UDS Services" and "Authentication with PKI Certificate Exchange" for details.

4.1.1.8 Zone node system requirement

•Requirements

4.1.1.8.1 Requirements

1038508 v1 **General requirement to match test case from Subsystem requirement**

Version history:
(1):

Purpose: To map the test case from subsystem requirement
Source: ECU Platform System (SRD ECU Platform)
Verification Method: Test
Legacy ID:

Reference number	Source	Title	Revision of document
1	Subsystem	ECU Platform System (SRD ECU Platform)	According to Baseline release

Note:
1. References are typically available from subsystem baseline release, it need to be added here to match some test case from Basetech Verification request.
2. Generally for BusAbnormalSleepAndWakeupRecord and so on.

4.1.1.9 Car Configuration Master system requirement

•Requirements

4.1.1.9.1 Requirements

1038509 v1 **General requirement to match test case from Subsystem requirement**

Version history:
(1):

Purpose: To map the test case from subsystem requirement
Source: Car Configuration Management subsystem
Verification Method: Test
Legacy ID:

Reference number	Source	Title	Revision of document
1	Subsystem	Car Configuration Management	According to Baseline release

Note:
1. References are typically available from subsystem baseline release, it need to be added here to match some test case from Basetech Verification request.
2. Generally for VIN port sending and some Car config master requirement.

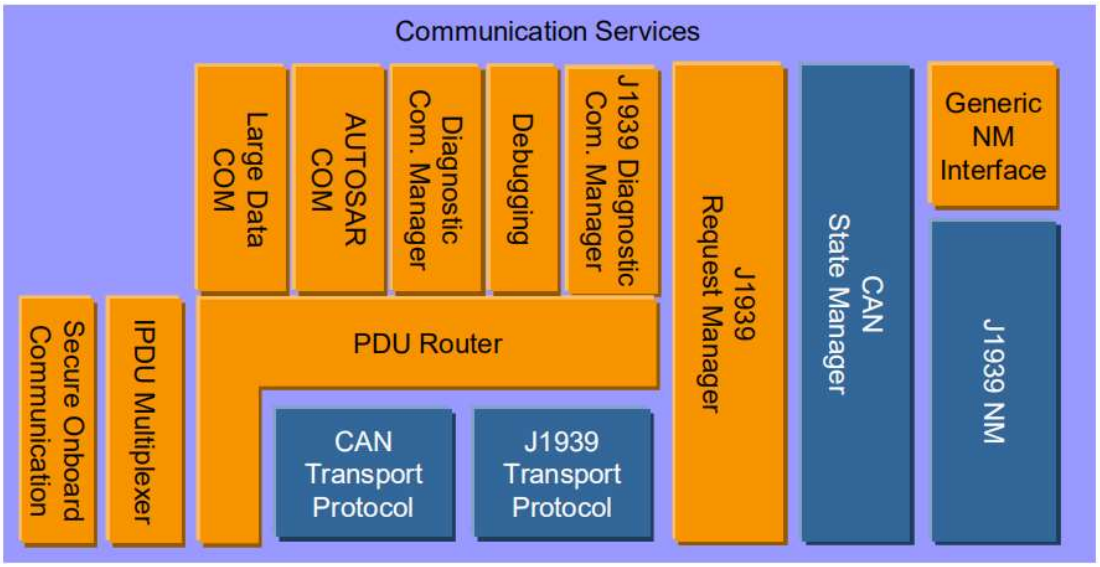
4.1.2 Guideline for update bit mechanism

4.1.2.1 Guideline for update bit mechanism

•General

4.1.2.1.1 General

Update bit concept is to provide the management of AutoSAR Signal state mechanism, used to represent a Signal/Signal group whether updated or not. The receiver can identify whether the sender sends the updated Signal/Signal group, then reducing the processing task from the receiver, and reducing CPU load.
Update bit mechanism is supported by AutoSAR COM module, and it is configured by the COM module of sender and receiver ECU, each Signal or Signal group respectively is added up to an Update bit. It's important to note that UB signals is not produced by software component,it is only used for internal processing, and it should not be as part of the signals or signal groups.



UB is introduce in the AutoSAR COM module

Update bit	
0	cleared/ data has not been updated
1	set/ data has been updated

674284 v1 **Update bit sending for gateway ECUs**

Version history:
(1):

Purpose: To define the usage of update bit on the signal gateway side.
Source:
Verification Method: Test
Legacy ID:

The gateway ECUs should always send the signal/Signal group in the target clusters according to the Signal Database.

Note:

1. The signals of source cluster is lost in the sending, the gateway ECUs should use the last value in the target cluster, setting UB as zero for this signals.

2. The signals is never received from the source cluster, the gateway ECUs should use the initial value in the target cluster, setting UB as zero for this signals.

671305 v1 **Update bit sending for non-AutoSAR ECUs**

Version history:
(1):

Purpose: To specify the update bit sending of signal/Signal group for non-AutoSAR ECUs.
Source:

Verification Method: Analysis

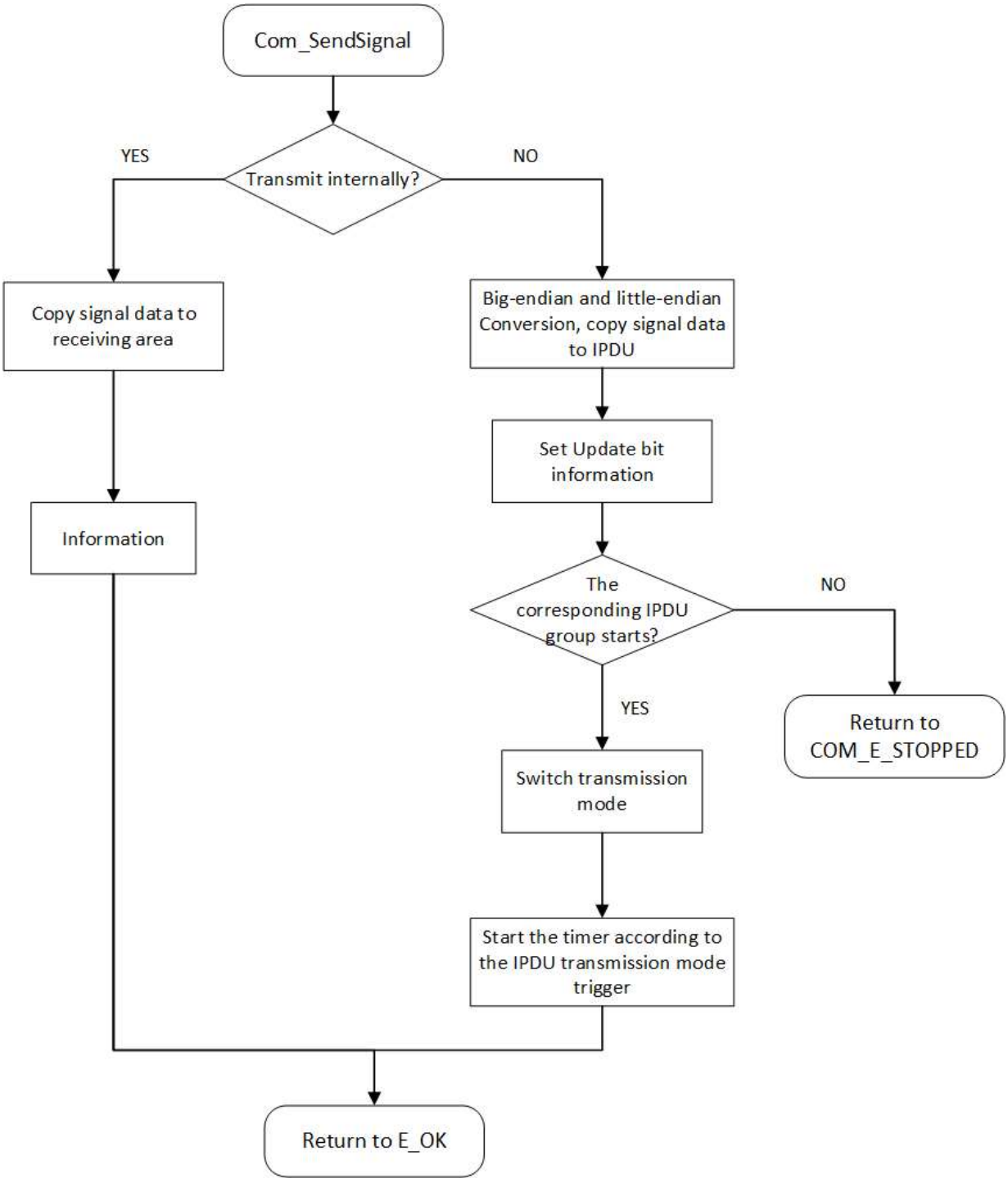
Legacy ID:

The ECU should always set the UB(update bit) as 1 when the signal/Signal group is triggered by the sender, this is applicable for the non-AutoSAR ECUs on CAN/CANFD.
It will not be set the UB if there is no UB of signal/signal group definition.

•Update bit mechanism

4.1.2.1.2 Update bit mechanism

4.1.2.1.2.1 The process of sender



The AUTOSAR COM module sets the Update Bit value of this signal as long as the RTE calls the Com_SendSignal / Com_SendSignalgroup function to update the signal value; in other cases, the sender's Update bit value is 0; as long as the RTE has Call the write operation of Com_SendSignal, even if the signal value is the same as the previous value, the Update bit needs to be 1; RTE calls the Com_SendSignal / Com_SendSignalgroup should

based on the generation type of this signal/signal group, which is defined in the ECU SWRS.

If the IPDU Tx mode is Periodic, the RTE does not call the Com_SendSignal or Com_SendSignalgroup function interface to update the signal value, and the UB value corresponding to the signal in the IPDU is 0 (meaning no update);

If the IPDU Tx mode is Direct/N-Times（NumberOfRepetitions≥1）, then the signal/signal group doesn't own the UB.

After the AUTOSAR COM module sends the IPDU to the PDUR, it will clear the Update bit value of all the included signals and signal groups to 0 to prepare for the next transmission.

4.1.2.1.2.2 Related parameter for sender

a. ComUpdateBitPosition

UpdateIndicationBit indicates to the receiver that the signal/signal group has been updated by the sender. The length is always one bit. The UpdateIndicationBitPosition property describes the location of the update bit in SignalIPdu. This property is not relevant for signals of ISignalGroup.

SWS Item	**ECUC_Com_00257 :**
Name	ComUpdateBitPosition
Description	Bit position of update-bit inside I-PDU. If this attribute is omitted then there is no update-bit. This setting must be consistently on sender and on receiver side. Range: 0..63 for CAN and LIN, 0..511 for CAN FD, 0..2031 for FlexRay, 0..4294967295 for TP.
Multiplicity	0..1
Type	EcucIntegerParamDef
Range	0 .. 4294967295
Default value	--
Post-Build Variant Multiplicity	true

Note: Because there is no AutoSAR request for LIN node, so there is no UB setting for this.

The update bit signal needs to be defined in the SDB. The BSW parameter needs to be associated with the attribute in ECU arxml and needs to be defined by the OEM, not from the Tier1. The following is the ECU arxml mapping relationship:

Communication::ISignalToIPduMapping.updateIndicationBitPosition

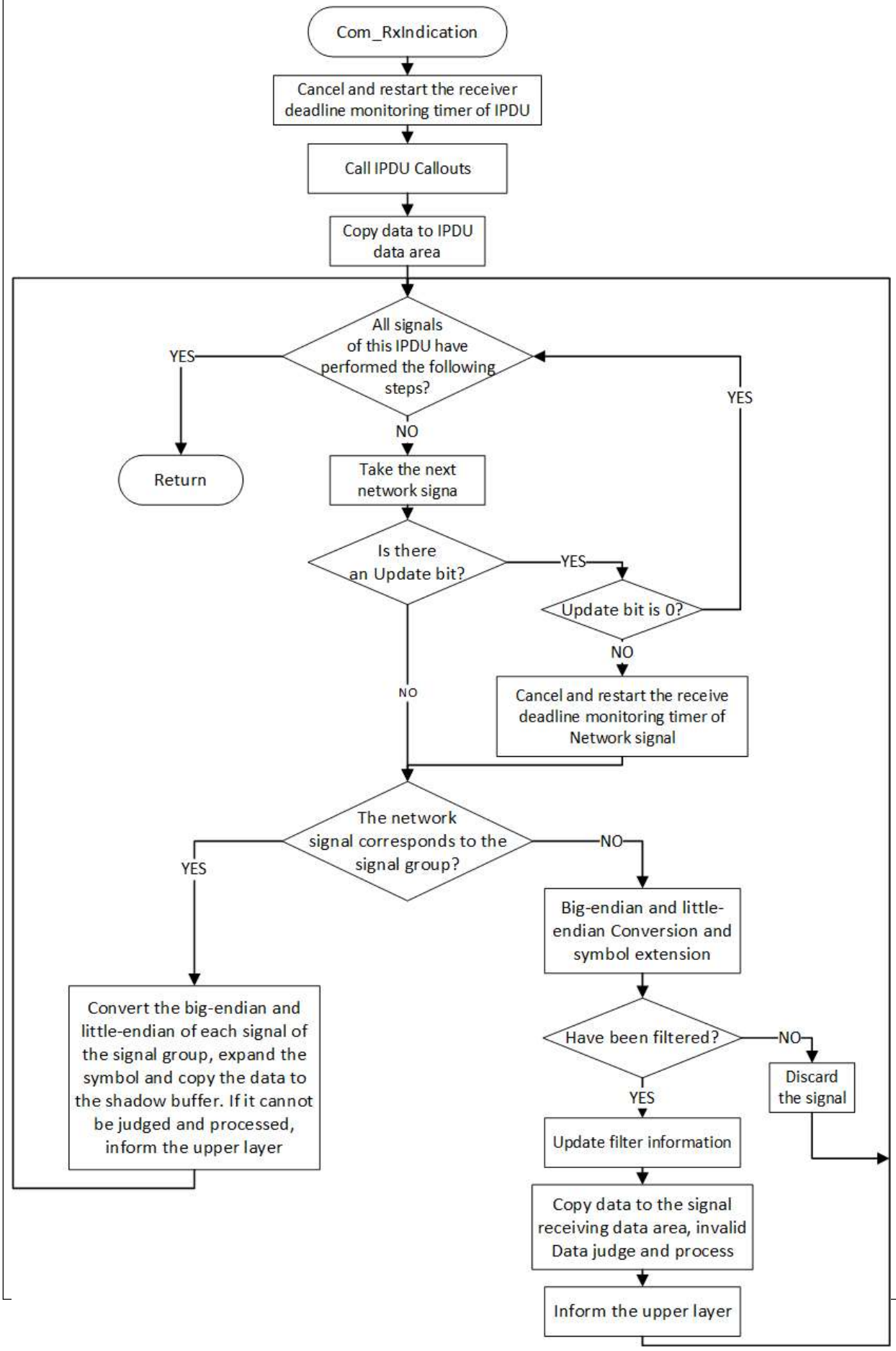
Example in SDB:

```
<I-SIGNAL-TO-I-PDU-MAPPING UUID="igA7BF49B6B8C4BE4A34C05692E061B771_mrx">
  <SHORT-NAME>igAsyDataWithCmpSafe_mrx</SHORT-NAME>
  <I-SIGNAL-GROUP-REF DEST="I-SIGNAL-GROUP">/Communication/ISignalGroup/igAsyDataWithCmpSafe</I-SIGNAL-GROUP-REF>
  <PACKING-BYTE-ORDER>MOST-SIGNIFICANT-BYTE-FIRST</PACKING-BYTE-ORDER>
  <START-POSITION>119</START-POSITION>
  <UPDATE-INDICATION-BIT-POSITION>168</UPDATE-INDICATION-BIT-POSITION>
</I-SIGNAL-TO-I-PDU-MAPPING>
```

b. ComTxIPduClearUpdateBit

BSW module	COM
BSW parameter	ComTxIPduClearUpdateBit
The rule	ComTxIPduClearUpdateBit defines three optional configuration parameters: Transmit, Confirmation and Triggertransmit. The specific parameter conditions are as follows: If the parameter ComTxIPduClearUpdateBit of I-PDU is configured as transmit, it will be displayed through PduR_ComTransmit sends the I-PDU and returns E_OK, the AUTOSAR COM module shall clear all contained signals and signal groups, and the update bit is 0. If the parameter ComTxIPduClearUpdateBit of I-PDU is configured as confirmation, it will be displayed through PduR_ComTransmit sends the i-pdu, pdur_ Comtransmit returns e_ OK and the bottom layer confirm that the I-PDU has been sent, and the AUTOSAR COM module shall clear all contained signals and signal groups, and the update bit is 0. If the parameter ComTxIPduClearUpdateBit of I-PDU is configured as triggertransmit, it is displayed in Com_TriggerTransmit successfully requests the contents of this I-PDU, AUTOSAR COM module shall clear all contained signals and signal groups, and the update bit is 0.
Description	Defines when to clear the update bit of the signal or signal group contained in the I-PDU.
Mapping with arxml	The ComTxIPduClearUpdateBit parameter belongs to local configuration, which requires the Tier1 to integrate BSW software configuration, and does not belong to SDB definition; Moreover, Basetech has no requirements for this parameter, which is configured by the Tier1.

4.1.2.1.2.3 The process of receiver


Com_RxIndication
Cancel and restart the receiver deadline monitoring timer of IPDU
Call IPDU Callouts
Copy data to IPDU data area
All signals of this IPDU have performed the following steps?
YES
NO
Return
Take the next network signa
Is there an Update bit?
YES
Update bit is 0?
YES
NO
Cancel and restart the receive deadline monitoring timer of Network signal
NO
The network signal corresponds to the signal group?
YES
NO
Convert the big-endian and little-endian of each signal of the signal group, expand the symbol and copy the data to the shadow buffer. If it cannot be judged and processed, inform the upper layer
Big-endian and little-endian Conversion and symbol extension
Have been filtered?
NO
Discard the signal
YES
Update filter information
Copy data to the signal receiving data area, invalid Data judge and process
Inform the upper layer

For the receiver, signal/signal group configure the update bit signal. The receiver first judges the value of the update bit signal. Only when the update bit value is 1, the AUTOSAR COM module will process the signal for subsequent filtering, notification, byte exchange and other tasks;
If the update bit value of the received signal or signal group is 0, the AUTOSAR COM module shall discard the signal, and the RTE will return the initial value or the last received value to the corresponding SWC module;
For the signal requiring gateway, the receiver receives the signal with update bit of 0 in the original network segment, and the COM module will not route the signal and directly discard the signal; (only applicable to signal gateway);
For the receiver, the number of signals processed at the same time can be reduced by configuring the signal and configuring the update bit. The signal with update bit 0 does not need to be processed and analyzed, which reduces the CPU processing task and reduces the CPU load;
For the signal configuration update bit, the receiver timeout check for the signal will be based on signal, not IPDU.

4.1.2.1.2.4 Deadline monitoring for the receiver

COM module provides two forms of deadline monitoring, one is for signal / signal group receiving as the object, and the other is for IPDU receiving as the object;
For deadline monitoring with signal as the object, update bit must be configured to realize;
If the signal is not configured with update bit, the COM module will monitor the deadline monitoring based on IPDU;
Because the platform forces signal to be configured with update bit, so deadline monitoring is based on signal / signal group;
This introduce the deadline monitoring mechanism based on signal.

For each signal with update bit, the com timeout value and the signal value used after timeout need to be configured for a separate signal;
For signal timeout judgment, if the update bit of the received signal is 0 within the timeout range, it will be recognized that the signal is lost, and the timeout notification will be sent to the corresponding software component, and the COM module will also write the valid data value internally and send it to the SWC through RTE;
Therefore, each signal needs to be configured with comtimeout and comrxdatatimeoutaction parameters. The following chapters mainly introduce the definition and application of these two parameters:

a.ComTimeout

BSW module	COM
BSW parameter	ComTimeout
The rule	Comtimeout configures the timer of signal timeout. If the specified timer does not receive corresponding data, it is necessary to notify the corresponding SWC before this time (through RTE); Value range: 0-3600s If the alivetimeout property is 0, it means that com does not perform timeout

	monitoring.
Description	Defines the length of time (in seconds) that the signal times out.
Mapping with arxml	The alive timeout (comtimeout) signal needs to be defined in the system design. The BSW parameter needs to be associated with the attribute in ECU arxml and needs to be defined by the OEM, not from the Tier1. The following is the ECU arxml mapping relationship: CoreCommunication::ISignalPort.timeout Example in SDB

b.ComRxDataTimeoutAction

BSW module	COM
BSW parameter	ComRxDataTimeoutAction
The rule	ComRxDataTimeoutAction defines two optional configuration parameters: replace or none, the specific parameters and conditions are as follows: If ComRxDataTimeoutAction is set to replace, when the received timeout timer of the signal expires, the AUTOSAR COM module will replace the value of the signal with its ComSignalInitValue. If ComRxDataTimeoutAction is set to none or not configured, when the receiving timeout timer of the signal expires, the AUTOSAR COM module cannot replace the signal or signal group value to ensure that the RTE can always access the last received value; The ComRxDataTimeoutAction parameter is set to none in the SDB to ensure that the RTE can always access the last received value to the SWC.
Description	Parameter defines the actions to be performed after the receiver timeout timer expires.
Mapping with arxml	The ComRxDataTimeoutAction needs to be defined by the OEM. At present, the parameter we define is none, and there is a mapping relationship with the handletimeouttype parameter in the arxml file. The following is the ECU arxml mapping relationship: Communication::NonqueuedReceiverComSpec.handleTimeoutType Example in SDB

•RTE upate flag mechanism

4.1.2.1.3 RTE upate flag mechanism

RTE update flag can let the software component know whether there is new update data to the corresponding interface since the last reading, and it can let the application layer know whether the signal is updated by com. The update flag is only applicable to RPortPrototype (receiving port). AUTOSAR defines the enableupdate parameter in the NonqueuedReceiverComSpec, which is used by the system design owner to select whether to use the RTE update flag mechanism. If the 'EnableUpdate' attribute of the NonqueuedReceiverComSpec is true, the update flag will be processed.

RTE Update Flag definition

Update flag return value definition	
True	The data element has been updated since the last read.
False	Settings / data not updated

Example in SDB:

```
<R-PORT-PROTOTYPE UUID="DCE:81f72be3-952f-66ce-a9ec-6de12ad79419">
  <SHORT-NAME>CarTiGlb</SHORT-NAME>
  <ADMIN-DATA>
  <REQUIRED-COM-SPECS>
    <NONQUEUED-RECEIVER-COM-SPEC>
      <DATA-ELEMENT-REF DEST="VARIABLE-DATA-PROTOTYPE">/PortInterfaces/CarTiGlb_5/CarTiGlb</DATA-ELEMENT-REF>
      <NETWORK-REPRESENTATION>
        <SW-DATA-DEF-PROPS-VARIANTS>
          <SW-DATA-DEF-PROPS-CONDITIONAL>
        </SW-DATA-DEF-PROPS-VARIANTS>
      </NETWORK-REPRESENTATION>
      <USES-END-TO-END-PROTECTION>false</USES-END-TO-END-PROTECTION>
      <ALIVE-TIMEOUT>0.0</ALIVE-TIMEOUT>
      <ENABLE-UPDATE>false</ENABLE-UPDATE>
      <HANDLE-NEVER-RECEIVED>false</HANDLE-NEVER-RECEIVED>
      <HANDLE-TIMEOUT-TYPE>NONE</HANDLE-TIMEOUT-TYPE>
      <INIT-VALUE>
    </NONQUEUED-RECEIVER-COM-SPEC>
  </REQUIRED-COM-SPECS>
  <REQUIRED-INTERFACE-TREF DEST="SENDER-RECEIVER-INTERFACE">/PortInterfaces/CarTiGlb_5</REQUIRED-INTERFACE-TREF>
</R-PORT-PROTOTYPE>
```

4.1.2.1.3.1 Relationship between update flag and update bit

Update bit is defined at the communication level of the COM module.

If the UB signal value is false. The COM module will throw away the signal value with UB false and will not notify the RTE.

If the signal value of COM UB is true, RTE will be notified and set the value of IsUpdated to 1. After receiving the signal that UB is false, IsUpdated will not be affected because there is no feedback from com to RTE. Therefore, for the software component, you can know whether the signal is updated through RTE update flag;

Note: The update flag of a specific DataElement in RPortPrototype should be cleared after reading the data element by Rte_Read or Rte_DRead.

4.1.2.1.3.2 RTE update flag use case

Based on AUTOSAR definition, UpdateFlag of DataElements can only be used for explicit communication.

If the 'enableUpdate' property of RPort configuration is true, the DataElement update flag should be set by receiving new data from com or local software components.

For the event trigger signal, the receiver can judge that the event is triggered through the update flag. The event trigger behavior needs to be described in SWRS, and the function scenario is required to be executed according to the trigger.

In addition, the receiver uses the update flag. The signal generation type of the sender's application layer is event triggered. Only the value triggered by the event (UB == 1) can be sent, and other values cannot send UB bits.

4.1.2.2 Guideline for AliveTimeout setting

•The rule to define Alive Timeout

4.1.2.2.1 The rule to define Alive Timeout

It is not the must to define Alive Timeout for each signal/signal group.

If some signal/signal group is received by LIN cluster, then the minimum alive timeout is set as 1,000 ms, else if max age *3 beyonds the 1,000 ms, then it should set as max age*3.

If some signal/signal group is receive by CAN cluster, then define the alive timeout as follows:

- max age less than 200 ms, the alive timeout set as max age * 5.
- max age greater than 200 ms, then alive timeout set as max age * 3.

Note:

1. The alive timeout should set as 50*x ms.

2. If the signal is PNC related, the alivetimeout should consider the signal within which PNCs.

3. The alivetimeout monitoring start-time shall beyond the startup time, 2.5 seconds or longer after the ECU wakeup.

4.1.3 AutoSAR Configuration

4.1.3.1 Autosar Configuration

•Documents

4.1.3.1.1 Documents

Reference	Source	Document
[AR_Conf_1]	Internal	Autosar Diagnostic Guideline
[AR_Conf_2]	Internal	Autosar Network Management - Additional requirements
[AR_Conf_3]	Internal	CAN Data Link Layer

Note:
Internal references is typically found as a separate chapter in ECU specific Base Technology SWRS.
Referenced AUTOSAR SWS Specification can be found at http://www.autosar.org/

•Definitions and Abbreviations

4.1.3.1.2 Definitions and Abbreviations

Abbreviation	Description
PC	PreCompile - A parameter that can't be changed without re-compiling the application
LT	LinkTime - A parameter that can't be changed without re-linking the application i.e. it is not necessary to re-compile but there will be a new version of the application
PB	PostBuild - A parameter that can be changed without updating the application

•Requirements

4.1.3.1.3 Requirements

660847 v2 **AUTOSAR version**

Version history:
(2):

Purpose: Specify AUTOSAR version.
Source:
Verification Method: Analysis
Legacy ID: 353876v0

AUTOSAR version 4.2.2 or higher shall be used.

Note: Other revisions such as 4.0.3 may be used after agreement with the base technology team.

660844 v1 **Communication Gateway Processing**

Version history:
(1):

Purpose: To guarantee message timing.
Source: Communication Gateway
Verification Method: Analysis
Legacy ID: 76543v1

The gateway functionality (which is a part of the communication layer) shall run every Communication SW processing period.

Note: This requirement is valid for all ECU having more than one bus interface.

660845 v1 **Communication Stack Processing Period**

Version history:
(1):

Purpose: To be able to optimize network performance.
Source: -
Verification Method: Analysis
Legacy ID: 76539v2

The Communication stack processing period shall be 5,0 ms.

Note: This requirement is valid for all ECUs with CAN, CANFD, or LIN Master bus interfaces.

660846 v2 **Communication Stack Processing Period Jitter**

Version history:
(2): Update the verification method from 'Analysis and Test' to 'Analysis'

Purpose: To be able to optimize network performance.
Source: -
Verification Method: Analysis
Legacy ID: 76542v2

The jitter of the Communication stack processing period shall be less than 1 ms.

Note: This requirement is valid for all ECUs with CAN, CANFD, or LIN Master bus interfaces.

•AUTOSAR CONFIGURATION REQUIREMENTS

4.1.3.1.4 AUTOSAR CONFIGURATION REQUIREMENTS

In this section all requirements are numbered in order to accomplish traceability during the development process. Text written outside the requirement tags in this section shall be regarded as complementing information and not as a requirement.

663094 v2 **Autosar configuration - recommended values**

Version history:
(2):

Purpose: Define recommended configuration values
Source:
Verification Method: Analysis
Legacy ID: 52547v7

These are recommended values that are suitable for most ECU:s but may be deviated from if needed (but any change of a parameter value must result in that requirements are fulfilled).

Parameter	Value
CanIfPrivateSoftwareFilterType	BINARY
CanIfPublicCancelTransmitSupport	FALSE
CanIfPublicReadRxPduDataApi	FALSE
CanIfPublicReadRxPduNotifyStatusApi	FALSE
CanIfPublicReadTxPduNotifyStatusApi	FALSE
CanIfPublicSetDynamicTxIdApi	FALSE
CanIfPrivateDlcCheck	FALSE
CanIfSupportTTCAN	FALSE
CanTimeoutDuration	0.002
CanTpSTmin	0.005
ComSignalInitValue	Different signal has different initial value, please refer to ECU arxml.
ComSignalType	Different signal has different signal type, please refer to ECU arxml.
ComTxIPduCancelTransmitSupport	FALSE
ComTxIPduUnusedAreasDefault	0
ComTxModeMode	Different signal has different Tx mode, please refer to ECU arxml.
ComTxModeNumberOfRepetitions	Different signal has different Tx mode, please refer to ECU arxml.
IpduMSelectorFieldLength	8
IpduMSelectorFieldPosition	0
LinChannelBaudRate	19200
LinIfCancelTransmitSupported	FALSE
LinIfResumePosition	Start_From_Beginning
SyncCounterInit	1 (Note 1)
MaxNoNewOrRepeatedData	14 (Note 2)
MaxDeltaCounterInit	14 (Note 3)

Note 1: *This is for duration of counter continuity check.*
Note 2: *This is for tolerance interval for maximum counter differences.*
Note 3: *This is for tolerance for initial counter difference. The default value of MaxDeltaCounterInit will be 14, it means there*

won't be cared about the counter continuity. Other value needs to be agreed by function safety team and update this in the subsystem design from subsystem owner.

663093 v5 **Autosar configuration - required configuration values**

Version history:
(5): Update the EEA from 3.5 to 4.0

Purpose: Enforce similar network behaviour for all modules in a vehicle
Source:
Verification Method: Analysis AND Test
Legacy ID: 52546v14

These are mandatory values for non-programming session. Using an Autosar version 4.0.3 or higher means that the communication stack must support Partial Network Cluster (PNC).

Parameter	Value
CanControllerFdBaudrateConfig	ECU with "Classic CAN": N/A ECU with CAN FD support: Configure according to requirements in the *[AR_Conf_4] CAN Data Link Layer* and/or the provided ECU Extract.
CanSetBaudrateApi (4.1.1-)	FALSE
CanIfRxPduReadData	FALSE
CanIfTxPduType	STATIC
CanIfSetBaudrateApi (4.1.1-)	FALSE
CanNmActiveWakeupBitEnabled (4.1.1-)	TRUE
CanNmAllNmMessagesKeepAwake (4.0.3-)	FALSE
CanNmBusLoadReductionActive	FALSE
CanNmBusLoadReductionEnabled	FALSE
CanNmBusSynchronizationEnabled	FALSE
CanNmCarWakeUpRxEnabled (4.0.3-)	FALSE
CanNmComUserDataSupport (4.0.2)	FALSE
CanNmComUserDataSupport (4.0.3-)	TRUE
CanNmImmediateNmCycleTime	0.02
CanNmImmediateNmTransmissions	20
CanNmImmediateRestartEnabled	FALSE
CanNmMainFunctionPeriod	0.005
CanNmMsgCycleOffset	0.1
CanNmMsgCycleTime	1.0
CanNmMsgTimeoutTime	0.5
CanNmNodeDetectionEnabled	TRUE
CanNmNodeIdEnabled (4.1.1-)	TRUE
CanNmPassiveModeEnabled	FALSE
CanNmPduCbvPosition	1
CanNmPduNidPosition	0
CanNmMsgReducedTime	NA
CanNmRepeatMessageTime	1.0
CanNmPnEiraCalcEnabled (4.0.3-)	TRUE
CanNmPnEnabled (4.0.3-)	TRUE
CanNmPnEraCalcEnabled (4.0.3-)	For a PNC gateway: TRUE. For a non-PNC-gateway: FALSE.
CanNmPnHandleMultipleNetworkRequests	TRUE

(4.0.3-)	
CanNmPnInfoLength (4.0.3-)	4
CanNmPnInfoOffset (4.0.3-)	2
CanNmPnResetTime (4.0.3-)	2.95
CanNmRetryFirstMessageRequest (4.2.1-)	TRUE
CanNmTimeoutTime	4.0
CanNmUserDataEnabled (4.0.3-)	TRUE
CanNmUserDataLength (4.0.3-4.1.3)	6
CanNmWaitBusSleepTime	4.0
CanSMBorCounterL1ToL2	For GEEA3.0: 32 For EEA4.0: 10
CanSMBorCounterL2Err (4.0.2)	For GEEA3.0: 31 For EEA4.0: 9
CanSMBorTimeL1	For GEEA3.0: 0.01 For EEA4.0: 0.1
CanSMBorTimeL2	For GEEA3.0: 60.0 For EEA4.0: 1.0
CanSMBorTimeTxEnsured	1.0
CanSMEnableBusOffDelay (4.1.3-)	FALSE
CanSMIcomSupport (4.2.1-)	FALSE
CanSMMainFunctionPeriod	0.005
CanSMPncSupport (4.1.2-)	TRUE
CanSMSetBaudrateApi (4.1.1-)	FALSE
CanTpChannelMode	CANTP_MODE_HALF_DUPLEX
CanTpFlexibleDataRateSupport (4.2.1-)	FALSE
CanTpMainFunctionPeriod	0.005
CanTpNar	1.0
CanTpNas	1.0
CanTpNbr	0.02
CanTpNbs	1.0
CanTpNcr	1.0
CanTpNcs	0.02
CanTpRxAddressingFormat	Standard
CanTpRxPaddingActivation	CANTP_ON
CanTpRxWftMax	255
ComFilterAlgorithm	ALWAYS
ComMMainFunctionPeriod	0.005
ComMNmPassiveModeEnable (4.1.1-)	FALSE
ComMNmVariant (4.0.3-)	FULL For managed channel :LIGHT
ComMPncNmRequest (4.0.3-)	TRUE
ComMPncEnabled (4.0.3-)	TRUE
ComMPncGatewayEnabled (4.0.3-)	Domain master ECU: TRUE Other than domain master ECU: FALSE
ComMPncPrepareSleepTimer (4.0.3-)	0.025
ComMPncSupport (4.0.3-)	TRUE
ComM0PncVectorAvoidance (4.3.1-)	FALSE Note: 1.This parameter is for PNC GW and ComMPncGatewayEnabled =TRUE. 2.Tier2 Vector 4.2.2 BSW already integrate this parameter.
NmBusSynchronizationEnabled	FALSE

NmComUserDataSupport (4.0.2)	FALSE
NmComUserDataSupport (4.0.3-)	TRUE
NmCoordinatorSupportEnabled	FALSE
NmCoordinatorSyncSupport (4.1.1-)	FALSE
NmNodeDetectionEnabled	TRUE
NmNodeIdEnabled	TRUE
NmPassiveModeEnabled	FALSE
NmUserDataEnabled (4.0.3-)	TRUE

Note:
When a configuration parameter value is mandatory for all Autosar versions, no Autosar version is denoted after the parameter name.
When different parameter values are required depending on Autosar version or the parameter exist only for one Autosar version, the applicable version is denoted, e.g. "(4.0.3)".
4.0.3- with a dash after means it is applicable for 4.0.3 and later versions.

4.1.3.2 Autosar Time Synchronization

•TIME SYNCHRONIZATION CONCEPT

4.1.3.2.1 TIME SYNCHRONIZATION CONCEPT

4.1.3.2.1.1 Time Synchronization Introduction

Time Synchronization is a method that enables distribution of time information over networks such as Ethernet, CAN and CANFD networks in order for ECUs to have a common understanding of time.

On Ethernet networks, the AUTOSAR standard has incorporated the mechanisms as specified in IEEE802.1AS-2011. For CAN and CANFD networks, a “new” Time Synchronization concept is developed by AUTOSAR.

Time Synchronization functionality shall realized with AUTOSAR BSW modules such as StbM (Synchronized Time Base Manager), EthTSyn (Time Syncronization on Ethernet), CanTSyn (Time Synchronization on CAN).

AUTOSAR standard defines different roles w.r.t. time synchronization and these roles are Global Time Master, Time Master, Time Slave and Time Gateway. All of these roles are "time aware" which is required by 802.1AS-2011.

The expectations w.r.t. time synchronization is that the difference in perception of time in any two ECUs in a vehicle network shall be less than 500us.

4.1.3.2.1.2 Time Synchronization Requirements

663101 v1 **Hardware time stamping support**

Version history:
(1):

Purpose: Hardware time stamping enables more accurate time information than software time stamping does.
A simplified explanation is that no internal latency is affecting the time synchronization mechanism in a negative way when using hardware time stamping.
Source:
Verification Method: Analysis
Legacy ID: 435545v0

Hardware time stamping support shall be enabled in the AUTOSAR BSW module EthTSyn.

663102 v1 **Use of AUTOSAR BSW modules**

Version history:
(1):

Purpose: Time Synchronization shall be realized using methods and modules as specified by AUTOSAR.
Source:
Verification Method: Analysis
Legacy ID: 435517v0

Time Synchronization functionality shall be realized with the following AUTOSAR modules:

- **StbM**, ref: "AUTOSAR Specification of Synchronized Time-Base Manager"
- **EthTSyn**, ref: “AUTOSAR Specification of Time Synchronization over Ethernet “
- **CanTSyn**, ref: “AUTOSAR Specification of Time Synchronization over Can”

All requirements, constraints and assumptions, dependencies to other modules etc. included in the AUTOSAR specifications above are applicable.

4.1.3.2.1.3 StbM-Synchronized Time Base Manager

908645 v1 **Global Time Precision Measurement Support**

Version history:
(1):

Purpose: The function is for testing and verification.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Time Slave and Gateway need to support time recording mechanism Global Time Precision Measurement to verify the precision of Local Time Base to the Global Time Base. Application could get record tables for Synchronized/Offset Time Base.
StbMIsSystemWideGlobalTimeMaster is set to FALSE.
StbMTimeRecordingSupport is set to TRUE.

908641 v1 **Hardware Clock Reference**

Version history:
(1):

Purpose: Chose a hardware counter instead of software to be the time source of StbM.
Source:
Verification Method: Analysis
Legacy ID:

For a better accuracy of Local Time Base, StbM local reference clock source ***StbMLocalTimeHardware*** needs to configure to the Ethernet Hardware counter.

908654 v1 **Offset Correction by Rate Adaption**

Version history:
(1):

Purpose: Parameters relative to Rate Adaption.
Source:
Verification Method: Analysis
Legacy ID:

StbM shall support to reduce offset between new received time and synclocal time with additional rate value to avoid a hard leap for Local Time Base, if the offset is less than 100us (experimental value).
StbMOffsetCorrectionJumpThreshold is set to 0.0001s.
StbMOffsetCorrectionAdaptionInterval is set to 0.125s.

908649 v1 Offset Time Base

Version history:
(1):

Purpose: Usage of Offset Time Base.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Offset Time Base combined with a specific Synchronized Time Base shall not be used.

908652 v1 Rate Correction

Version history:
(1):

Purpose: Define the specific Rate Correction behavior for each role of Time Sync.
Source:
Verification Method: Analysis
Legacy ID:

- For Time Master, remain the ability for adjusting Rate Correction value to received Time Base.
- For Global Time Master, require the ability for adjusting Rate Correction value to external time sources.

StbMAllowMasterRateCorrection is set to TRUE to be able to call *StbM_GetRateDeviation*.

- For Time Slave, StbM shall measure the rate deviation of each Time Base for Rate Correction i.e. rate correction measurement duration shall not be set to 0.

StbMRateCorrectionMeasurementDuration to set to 0.125s.

908643 v1 StbM Main Function Period

Version history:
(1):

Purpose: Define the frequency of Main Function to trigger relative functions.
Source:
Verification Method: Analysis
Legacy ID:

Require invocation period of *StbM_MainFunction* to be 5ms.
StbMMainFunctionPeriod is set to 0.005s.

908646 v1 **Synchronized Time Base**

Version history:
(1):

Purpose: Define the Time Base ID corresponding the Time Domain.
Source:
Verification Method: Analysis AND Test
Legacy ID:

There is one unified Synchronized Time Base ID to be used in each StbM for every existing Time Domain. ***StbMSynchronizedTimeBaseIdentifier*** (Time Base ID) of Synchronized Time Base (0 ~ 15) is set to **0** for Time Domain 0.

908651 v1 **Time Base Cloning**

Version history:
(1):

Purpose: Usage of Time Base Cloning.
Source:
Verification Method: Analysis
Legacy ID:

No need to support fallback or redundancy of Time Base by copying operation.

908648 v2 **Timeout for SyncLoss**

Version history:
(2): Change StbMSyncLossTimeout from 0.375s to 0.5s.

Purpose: Define the timeout value of receiving Sync.
Source:
Verification Method: Test
Legacy ID:

For Time Slave and Gateway, StbM shall monitor the timeout for Sync from the last call of StbM_BusSetGlobalTime().
StbMSyncLossTimeout is set to 0.5s.

4.1.3.2.1.4 CAN&CANFD Time Sync Requirements

682210 v1 **DLC OF CANTSyn message**

Version history:
(1):

Purpose: DLC of SYNC, FUP messages.
Source:
Verification Method: Test
Legacy ID:

The DLC of SYNC, FUP messages is 8 for classic CAN and CANFD.

908968 v1 Hardware Timestamping - CAN

Version history:
(1):

Purpose: Ensure the ability to capture time stamps through HW.
Source:
Verification Method: Analysis
Legacy ID:

CAN Egress/Ingress Timestamping functionality shall set enabled instead of software timestamping from StbM, to ensure a better accuracy for time base determining.
CanTSynHardwareTimestampSupport is set to TRUE.

908971 v1 Reference Synchronized Time Base

Version history:
(1):

Purpose: Each time domain used in CANTSyn shall be mapped to one synchronized time base.
Source:
Verification Method: Analysis
Legacy ID:

CanTSyn administrates multiple Time Domains. It is necessary to define the mapping relationship between each Time Domain of CanTSyn and corresponding Synchronized Time Base.
CanTSynSynchronizedTimeBaseRef configure to the interactive Synchronized Time Base.

908970 v1 Support for CAN

Version history:
(1):

Purpose: Define to use format of Classic CAN or CANFD.
Source:
Verification Method: Test
Legacy ID:

CanTSynUseExtendedMsgFormat is set to FALSE to active the support for Classic CAN.

908972 v1 TX Debounce Time

Version history:
(1):

Purpose: Avoid bursts and ensure the correct assignment of time stamp.
Source:
Verification Method: Analysis
Legacy ID:

CanTSyn need to do debouncing while transmitting Timesync messages. Debounce Time should be 5ms so the debounce counter to be 1 for sending the subsequent FUP message or Timesync messages for other Time Domain.

CanTSynGlobalTimeDebounceTime is set to 0.005s.

682360 v1 Time Domain ID

Version history:
(1):

Purpose: Time Domain ID and Sub_Domain ID.
Source:
Verification Method: Test
Legacy ID:

Time Domain ID is 0, and the sub_domain ID are defined in the SDB Arxmal.

682206 v1 Usage of CRC

Version history:
(1):

Purpose: Usage of CRC.
Source:
Verification Method: Test
Legacy ID:

CRC Calculation and Validation function for CanTSyn shall be turned ON.

682209 v1 byte order for time value signals

Version history:
(1):

Purpose: definite byte order for time value signals.
Source:
Verification Method: Test
Legacy ID:

The byte order for time value signals in Time Synchronization messages is "Big Endian".

682211 v2 message format

Version history:
(2): 1.Revise description.

Purpose: to definite message format.
Source:
Verification Method: Test
Legacy ID:

SYNC CRC and FUP CRC secured message format should send periodically follow AUTOSAR_SWS_TimeSyncOverCAN.

682361 v4 parameters

Version history:

(4): Remove Synlosstimeout to 908648 in the Stbm moudle.

Purpose: Define time parameters for Time Master and Time Slave
Source:
Verification Method: Test
Legacy ID:

Time parameters design is shown in **Table: time parameters for Time Master and Time Slave.**

Table: time parameters for Time Master and Time Slave

Name	Default Value
CanTSynGlobalTimeTxPeriod (ms)	50
CANTSyn_TXConfirmation (ms)	20
FollowUpoffset(ms)	20
CanTSynGlobalTimeFollowUpTimeout (ms)	100
CanTSynGlobalTimeSequenceCounterJumpWidth	1
CanTSynGlobalTimeSyncDataIDList	255
CanTSynGlobalTimeFupDataIDList	255

4.1.3.2.1.5 EthTSyn-Time Synchronization over Ethernet

908668 v1 CRC Support

Version history:
(1):

Purpose: Usage of CRC calculation and validation.
Source:
Verification Method: Analysis AND Test
Legacy ID:

CRC calculation for Ethernet Timesync message shall not to be supported.
EthTSynGlobalTimeTxCrcSecured is set to CRC_NOT_SUPPORTED.

CRC validation for Ethernet Timesync message shall not to be supported.
EthTSynRxCrcValidated is set to CRC_IGNORED.

908679 v1 Cyclic Transmission of Global Time - Ethernet

Version history:
(1):

Purpose: Define the static frequency to send Sync.
Source:
Verification Method: Analysis AND Test
Legacy ID:

For Time Master the transmission period of Sync message shall be 125ms.
EthTSynGlobalTimeTxPeriod is set to 0.125s.

Immediate transmission shall be disabled.
EthTSynImmediateTimeSync is set to FALSE.

908657 v1 EthTSyn Main Function Period

Version history:
(1):

Purpose: Define the frequency of Main Function to trigger relative functions.
Source:
Verification Method: Analysis
Legacy ID:

Require invocation period of *EthTSyn_MainFunction* to be 5ms.
EthTSynMainFunctionPeriod is set to 0.005s.

908658 v1 EthTSyn Time Domain

Version history:
(1):

Purpose: Define the Time Domain ID.
Source:
Verification Method: Test
Legacy ID:

Set Time Domain ID for EthTSyn of each Endpoint to be 0 if there is only one Time Domain.
EthTSynGlobalTimeDomainId is set to 0.

908680 v1 FollowUp Timeout

Version history:
(1):

Purpose: Define timeout value between Sync and Follow_Up.
Source:
Verification Method: Test
Legacy ID:

For Time Slave, observing the timeout between Sync and corresponding Follow_Up shall be 125ms.
EthTSynGlobalTimeFollowUpTimeout is set to 0.125s.

908655 v1 Hardware Timestamping - Ethernet

Version history:
(1):

Purpose: Ensure the ability to capture time stamps through HW.
Source:
Verification Method: Analysis
Legacy ID:

Ethernet Egress/Ingress Timestamping functionality shall set enabled instead of software timestamping from StbM, to ensure a better accuracy for time base determining.
EthTSynHardwareTimestampSupport is set to TRUE.

908670 v1 Master Slave Conflict Detection

Version history:
(1):

Purpose: Make sure receiving right Timesync messages from right port.
Source:
Verification Method: Analysis AND Test
Legacy ID:

If Time Master receives Sync message from another Time Master, or Time Slave receives Sync message with different sourcePortIdentity, EthTSyn shall report a runtime error with code ETHTSYN_E_TSCONFLICT then discard the Sync message.
EthTSynMasterSlaveConflictDetection is set to TRUE.

908663 v1 Message Format Compliance

Version history:
(1):

Purpose: Only IEEE 802.1AS message types shall be used.
Source:
Verification Method: Test
Legacy ID:

There are two format types of message format in AUTOSAR Time Sync.
EthTSynMessageCompliance is set to TRUE to support IEEE 802.1 AS format for Timesync Messages.

908666 v1 Port Role

Version history:
(1):

Purpose: This is for a port to be Time Master or Time Slave.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Specify the role of a port by configuring the container ***EthTSynGlobalTimeMaster*** *or* ***EthTSynGlobalTimeSlave*** based on the actual network topology for the specific project.

908673 v1 Propagation Delay Measurement

Version history:
(1):

Purpose: Enable dynamic calculation of Pdelay value.
Source:
Verification Method: Test
Legacy ID:

Enable cyclic propagation delay measurement rather than using a fixed delay value.
EthTSynGlobalTimePdelayRespEnable is set to TRUE.
No need to set the default value ***EthTSynGlobalTimePropagationDelay***.

908675 v1 Propagation Delay Request Period

Version history:
(1):

Purpose: Define the static frequency to send Pdelay request message.
Source:
Verification Method: Test
Legacy ID:

The cyclic Transmission period of Pdelay_Req message shall be 1s.
EthTSynGlobalTimeTxPdelayReqPeriod is set to 1s.

908677 v1 Propagation Delay Threshold

Version history:
(1):

Purpose: Avoid too much jitter to pass on the Global Time Tuple.
Source:
Verification Method: Test
Legacy ID:

Discard the Pdelay value if it exceeds the configurable upper limit.
EthTSynPdelayLatencyThreshold is set to 0.001s.

908678 v1 Propagation Delay Timeout

Version history:
(1):

Purpose: Define the timeout for Pdelay type messages.
Source:
Verification Method: Test
Legacy ID:

Timeout for Pdelay_Resp after Pdelay_Req and Pdelay_Resp_Follow_Up after Pdelay_Resp shall be 1000ms.
EthTSynPdelayRespAndRespFollowUpTimeout is set to 1s.

908661 v1 Reference Synchronized Time Base

Version history:
(1):

Purpose: Each time domain used in EthTSyn shall be mapped to one synchronized time base.
Source:
Verification Method: Analysis AND Test
Legacy ID:

EthTSyn administrates multiple Time Domains. It is mandatory to define the mapping relationship between each Time Domain of EthTSyn and corresponding Synchronized Time Base.
EthTSynSynchronizedTimeBaseRef configure to the interactive Synchronized Time Base.

908681 v1 **Sequence Counter Validation**

Version history:
(1):

Purpose: Sequence Counter validation on Time Slave side.
Source:
Verification Method: Test
Legacy ID:

Sequence Counter Validation shall be enabled for Time Slave.
EthTSynGlobalTimeSequenceCounterJumpWidth is set to 1, the jump of SC values between two consecutive Sync would be checked.

For the purpose of robustness, hysteresis mechanism should be used. If timeout occurs there would be at least three Sync-Follow_Up message pairs to be validated before valid time is forwarded.
EthTSynGlobalTimeSequenceCounterHysteresis is set to 3.

908671 v1 **TX Debouncing Time**

Version history:
(1):

Purpose: Avoid bursts and ensure the correct assignment of time stamp.
Source:
Verification Method: Analysis
Legacy ID:

EthTSyn need to do debouncing while transmitting Timesync messages. Debounce Time should be 5ms so the debounce counter to be 1 for sending the subsequent Timesync messages for this or the other Time Domains.
EthTSynGlobalTimeDebounceTime is set to 0.005s.

908659 v1 **VLAN Support**

Version history:
(1):

Purpose: Disable the usage of VLAN tag.
Source:
Verification Method: Test
Legacy ID:

Since VLAN priority and tag is not used for Timesync frames, ***EthTSynFramePrio*** shall not be configured in the Parent Container ***EthTSynGlobalTimeDomain.***

4.1.4 E2E

4.1.4.1 E2E Overview

•E2E Overview

4.1.4.1.1 E2E Overview

CAN、LIN及Ethernet信号通讯需要做E2E保护时，Profile选型为profileG；

Ethernet服务通讯需要做E2E保护时，数据长度小于或等于32字节Profile选型为profile11 VariantC，数据长度大于32字节Profile选型为profile4。

When E2E protection is required for CAN, LIN and Ethernet signal communication, profileG should be used for Profile selection.
When E2E protection is required for Ethernet service communication, for DataLength less than or equal to 32 bytes, the Profile is selected as profile11 VariantC; for DataLength greater than 32 bytes, the Profile is selected as profile4.

4.1.4.2 E2E Profile G

•E2E PROTECTION CONCEPT

4.1.4.2.1 E2E PROTECTION CONCEPT

693454 v1 **Keep the delta couter contious in the source cluster**

Version history:
(1):

Purpose: To keep the delta counter in the order from the sender
Source:
Verification Method: Test
Legacy ID:

The signal group with E2E from the sender should keep the counter continuous in the cluster with E2E needed according to the generation 'period' from the system definition when it is updated in the SWC.

4.1.4.2.1.1 E2E Protection Introduction

The E2E-protection mechanism within AUTOSAR is a methodology in which safety-related functions need to exchange safety-related information between SWCs. Such safety-related information most often consists of a set of signals, e.g. exchange of a sensor value together with its quality factor.

In E2E-protection there is a strict 1:n relationship, i.e. there is one sending SWC and one or more receiving SWCs. The receiving SWCs can be distributed across the network i.e. E2EProtected data need to be sent over the Network. The receiving SWC shall be able to choose whether to use or ignore the E2E-protection mechanism depending on its safety-classification.

To be able to detect errors in the communication, the safety-related signals are extended by additional signals, i.e. CRC checksum and a sequence counter. All data is grouped together within a signal group which is then transmitted over the network. To be able to “unpack” the safety-related data on a different ECU in a network, the description how to calculate the checksum is needed. These descriptions are called profiles in the AUTOSAR specifications.

The AUTOSAR standard defines three different ways to invoke the EndToEnd library, via ComCallout, Wrapper and Transformer. This specification allows two of these ways, i.e. Wrapper and Transformer.

4.1.4.2.1.2 E2E Protection Requirements

663095 v1 **Simultaneous use of Profiles and Transformers**

Version history:
(1):

Purpose: E2E Protection realization inside an ECU must not prevent simultaneous use of different E2E profiles and different ways to invoke the E2E library.
Source:
Verification Method: Analysis
Legacy ID: 435535v0

An ECU shall be able to handle multiple combinations of E2E Profiles, Serializing Transformers and E2E Wrappers simultaneously.

663096 v1 **Use of Profiles and Transformers**

Version history:
(1):

Purpose: E2E Protection shall be realized using methods and modules as specified by AUTOSAR.
Source:
Verification Method: Analysis
Legacy ID: 435534v0

E2E protection shall be realized using AUTOSAR modules:

Profile 1A
ComXf (ref: "AUTOSAR Specification of ComBasedTransformer") and **E2EXf** (ref: "AUTOSAR Specification of E2ETransformer").

Note: If the chosen AUTOSAR BSW version is not supporting ComXf and E2EXf, E2E protection shall be realized using an **E2E Wrapper** (ref: "AUTOSAR Specification of E2ELibrary").

Profile 4A
SomeIpXf (ref: "AUTOSAR Specification of SOMEIPTransformer") and **E2EXf** (ref: "AUTOSAR Specification of E2ETransformer").

Profile 5
ComXf (ref: "AUTOSAR Specification of ComBasedTransformer") and **E2EXf** (ref: "AUTOSAR Specification of E2ETransformer").

Profile G
E2E protection shall be realized using an **E2E Wrapper** (ref: "AUTOSAR Specification of E2ELibrary" and "Profile G requirements").

Note: *Profiles and transformers are defined per signal and communicated to supplier via the ECU Extract.*

4.1.4.2.1.2.1 Profile G Requirements

663097 v2 **E2E CRC Calculation**

Version history:
(2):

Purpose: Defines how to calculate CRC.
Source:

Verification Method: Analysis AND Test
Legacy ID: 181435v3

To use the E2E_P01Protect function, the data that should be computed shall be packed in an array of uint8s. To get the same result in both sender and receiver(s), the elements need to be packed in the same order in both sides.

The array sent to E2E_P01Protect shall have the following order:

1. Index 0: CRC (8-bit)
2. Index 1: 0x0 + 4-bit counter (4 bit counter extended to 8 bits using 0 as padding)
3. Index 2: data 1
4. Index n: data 2

The data (data 1, data 2) shall be ordered alphabetically by the name of the data-element variable.

Note:
1. The data-element variable is not referring to system signals, i.e. data shall not be sorted using the names of the system signal. Data-element variable is referring to VariableDataPrototype -> ApplicationRecordDataType -> ApplicationRecordElement. The data shall be sorted by the names of the ApplicationRecordElements.

2. All data element names should be unique, however if an extension to is used to make an unique naming e.g VehicleSpd -> VehicleSpd1, then following order should be used. (According to ASCII):

1. Alphanumeric characters
2. Alphabetic characters

Order of upper and lower case alphabetic characters should be disregarded. It should use lower case alphabetic to order this.

663098 v1 E2E Data-Type Size

Version history:
(1):

Purpose: Defines the size that shall be used when calculating checksums.
Source:
Verification Method: Analysis AND Test
Legacy ID: 181434v2

To be able to calculate the E2E checksum in the same way in all ECUs in the network, the same size of the E2E data-types shall be used. The size to be used is the size that is defined by the System Signal. In addition, the E2E_P01Protect function is expecting an array of uint8s and therefore all data shall be extended to the closest upper 8-bit boundary i.e. a two bit signal shall be represented with 8-bits during the CRC-calculations. Unused bits shall be set to 0 (zero).

If the data-type is larger than 8-bits, the little-endian method shall be used to pack the data into an 8-bit array.

Note: *The size of the E2E data type is defined by the System Signal. For signed signals, if any, the sign bit is included in the length of the System Signal, i.e. signed bit shall not be seen as an unused bit.*

663099 v1 E2E Protection Mechanisms

Version history:
(1):

Purpose: Defines the E2E Protection mechanisms.
Source:
Verification Method: Analysis AND Test
Legacy ID: 181433v1

The Profile G shall have the following mechanisms:

1. Counter - 4bit (explicitly sent) representing numbers from 0 to 14 incremented on every send request.
2. Timeout - Timeout is determined by E2E Library by means of evaluation of the Counter, by a non-blocking read at the receiver.
3. Data ID - 16 bit, unique number, included in the CRC calculation but not transmitted (implicit transmission).
4. CRC - CRC-8-SAE J1850 - 0x1D (x8 + x4 + x3 + x2 + 1) , but with different start and XOR values (both start value and XOR value are 0x00). This CRC is provided by CRC library. Starting with R4.0, the SAE8 CRC function of the CRC library uses 0xFF as start value and XOR value. To compensate a different behavior of the CRC library, the E2E Library applies additional XOR 0xFF operations starting with R4.0, to come up with 0x00 as start value and XOR value.

663100 v1 E2E Protection Wrapper

Version history:
(1):

Purpose: Defines the usage of Autosar E2EWrapper approach.
Source:
Verification Method: Analysis
Legacy ID: 181453v1

Within the SWC that is sending/receiving an E2E protected signal, the calculations shall be done using the Autosar E2EWrapper approach. This E2EWrapper shall be part of the sending/receiving SWC.

On the sending side, the wrapper shall do the following:

1. Pack the data into an array.
2. Call the E2E-library with the array as an argument (E2E_P01Protect).
3. On return from the E2E-library, put the data + CRC and counter into a signal group (record).
4. Send the signal group to the RTE.

On the receiving side, the wrapper shall do the following:

1. Receive the signal group from the RTE.
2. Get the data + CRC and counter from the signal group.
3. Pack everything in an array.
4. Send the array to the E2E-library (E2E_P01Check).
5. Evaluate the result from the E2E-library and take actions if the CRC or counter is not correct.

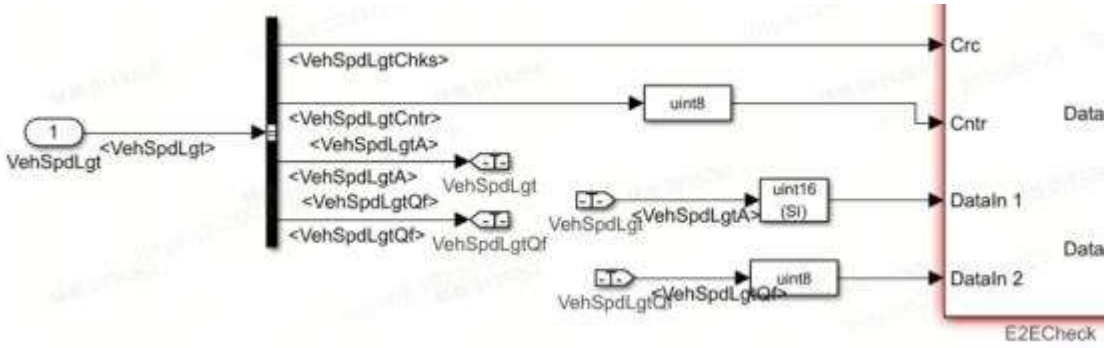
4.1.4.2.1.2.1.1 Appendix Example

1. 数据排序

例如：VehSpdLgt在吉利SDB定义中是1个安全信号组，定义如下图：

```
<APPLICATION-RECORD-DATA-TYPE UUID="f85776d5-bf56-48cd-bccf-63689bd8ff15_3">
  <SHORT-NAME>VehSpdLgtSafe1_3</SHORT-NAME>
  <CATEGORY>STRUCTURE</CATEGORY>
  <ADMIN-DATA>
  <SW-DATA-DEF-PROPS>
    <SW-DATA-DEF-PROPS-VARIANTS>
  </SW-DATA-DEF-PROPS>
  <ELEMENTS>
    <APPLICATION-RECORD-ELEMENT>
      <SHORT-NAME>VehSpdLgtA</SHORT-NAME>
      <CATEGORY>VALUE</CATEGORY>
      <ADMIN-DATA>
      <TYPE-TREF DEST="APPLICATION-PRIMITIVE-DATA-TYPE">/ApplicationDataTypes/Spd1_2</TYPE-TREF>
    </APPLICATION-RECORD-ELEMENT>
    <APPLICATION-RECORD-ELEMENT>
      <SHORT-NAME>VehSpdLgtQf</SHORT-NAME>
      <CATEGORY>VALUE</CATEGORY>
      <ADMIN-DATA>
      <TYPE-TREF DEST="APPLICATION-PRIMITIVE-DATA-TYPE">/ApplicationDataTypes/GenQf1_2</TYPE-TREF>
    </APPLICATION-RECORD-ELEMENT>
    <APPLICATION-RECORD-ELEMENT>
      <SHORT-NAME>VehSpdLgtCntr</SHORT-NAME>
      <CATEGORY>VALUE</CATEGORY>
      <ADMIN-DATA>
      <TYPE-TREF DEST="APPLICATION-PRIMITIVE-DATA-TYPE">/ApplicationDataTypes/UInt4_2</TYPE-TREF>
    </APPLICATION-RECORD-ELEMENT>
    <APPLICATION-RECORD-ELEMENT>
      <SHORT-NAME>VehSpdLgtChks</SHORT-NAME>
      <CATEGORY>VALUE</CATEGORY>
      <ADMIN-DATA>
      <TYPE-TREF DEST="APPLICATION-PRIMITIVE-DATA-TYPE">/ApplicationDataTypes/UInt8_2</TYPE-TREF>
    </APPLICATION-RECORD-ELEMENT>
  </ELEMENTS>
```

需要将除Cntr和Chks以外的信号（VehSpdLgtQf、VehSpdLgtA）按照字母顺序排序，排列好的顺序为VehSpdLgtChks，VehSpdLgtCntr，VehSpdLgtA，VehSpdLgtQf。



2. 数据取整与汇总

由于VehSpdLgtA 的数据类型是 Spd1，数据长度是 15bit，非整型，需要拆分成 2 个无符号 uint8 整型，最终数据填充可参考如下代码：

```
E2ECheck1_Data[0] = (((uint8_T)VehSpdLgt.VehSpdLgtChks) & 0xFFU);
E2ECheck1_Data[1] = (((uint8_T)VehSpdLgt.VehSpdLgtCntr) & 0x0FU);
E2ECheck1_Data[2] = (uint8_T)(VehSpdLgt.VehSpdLgtA & 0xFFU);
E2ECheck1_Data[3] = (uint8_T)(VehSpdLgt.VehSpdLgtA >> 8)& 0xFFU);
E2ECheck1_Data[4] = VehSpdLgt.VehSpdLgtQf;
```

3. CRC 计算

最终计算伪代码如下：

Get_Crc(DataID_Lsb + DataID_Msb + Counter + Sort(Signal1,Signal2,...));

4.1.4.2.1.3 Profiles vs Transformers and Wrappers

It is likely more than one profile needs to be supported in an ECU simultaneously. The type of network, size of the data, the need for gatewaying the data and AUTOSAR BSW version, are the main factors for deciding the profile and mechanism to use.

SOMEIPTransformer and E2ETransformer using Profile 4A is used on Ethernet for protection of data between 32byte up to 4kbyte.

ComBasedTransformer and E2ETransformer using Profile 1A is used on CAN and CANFD for protection of data up to 30byte. This combination can also be used on Ethernet when data is optimized for CAN, e.g. gatewaying from Ethernet to CAN.

ComBasedTransformer and E2ETransformer using Profile 5 can be used for protection of data between 18byte to 64byte when data is optimized for CAN FD, e.g. gatewaying from Ethernet to CAN FD.

E2E Wrapper using Profile 1A is used on CAN and CANFD for protection of data up to 30byte. E2E Wrapper is used when the AUTOSR BSW version does not support Transformers, i.e. prior to AUTOSAR version 4.2.1.

4.1.4.3 E2E Profile4 and Profile11

•E2E Profile4 and Profile11

4.1.4.3.1 E2E Profile4 and Profile11

4.1.4.3.1.1 Introduction

The E2E-protection mechanism within AUTOSAR is a methodology in which safety-related functions need to exchange safety-related information between SWCs. Such safety-related information most often consists of a set of signals, e.g. exchange of a sensor value together with its quality factor.

In E2E-protection there is a strict 1:n relationship, i.e. there is one sending SWC and one or more receiving SWCs. The receiving SWCs can be distributed across the network i.e. E2EProtected data need to be sent over the Network. The receiving SWC shall be able to choose whether to use or ignore the E2E-protection mechanism depending on its safety-classification.

To be able to detect errors in the communication, the safety-related signals are extended by additional signals, i.e. CRC checksum and a sequence counter. All data is grouped together within a signal group which is then transmitted over the network. To be able to “unpack” the safety-related data on a different ECU in a network, the description how to calculate the checksum is needed. These descriptions are called profiles in the AUTOSAR specifications.

4.1.4.3.1.2 Requirements

871255 v1 **E2E Data Id**

Version history:
(1): 1. 20250528 新增Profile4和Profile11的E2E需求；

Purpose: If ECU needs to support multiple profiles due to handling of both small and big data sizes. The Data ID may differ depending on which data size the profile supports.

Source:

Verification Method: Analysis

Legacy ID:

ECU shall be able to support at least unique Data IDs for each protected data element.

Note 1: Data ID will be provided in ECU Extract.

871259 v1 **E2E Profile 11 CRC calculation**

Version history:
(1): 1. 20250528 新增Profile4和Profile11的E2E需求；

Purpose: To increase error detection rate
Source:
Verification Method: Test
Legacy ID:

According to *[AUTOSAR_4 : E2E Protocol Specification]* and *[AUTOSAR_16 : Specification of CRC Routines]*;

E2E Profile	E2E CRC Method
Profile 11	Crc_CalculateCRC8 ()

For Profile 11:

E2E_P11DataIDMode = E2E_P11_DATAID_NIBBLE means that the lower nibble of the high byte of the DataID shall be transmitted explicitly at bit position DataIDNibbleOffset. CRC calculation shall be done by starting low byte[1] of the DataID which is implicitly transmitted, then a 0-byte[2] and then the user data[3]

As a generator polynomial 0x1D is used by Crc_CalculateCRC8 (). All calculation steps are clarified as following:

Example: Data ID = 0x123, Counter = 1, Data = 0x00 0x00 0x00 0x00 0x00 0x00

1) According to E2E Protocol Specification (R21-11), Figure 6.48, first calculate:

Crc_CalculateCRC8(Config->DataID, Crc_Length:1, Crc_StartValue8: 0xFF, Crc_IsFirstCall: FALSE)

Crc_CalculateCRC8 ()	
Polynomial	0x1D
CRC_StartValue8	0xFF
XOR Value	0xFF
Initial Value	0x00 {CRC_StartValue8 (0xFF) ^ PolynomialXORVal (0xFF)}
Input Data Reflected	No
Result Data Reflected	No
CRC Input Data	DataID Low Byte (Example : 0x23)
Computed CRC	According to the inputs, result is equal to 0x5F

2) According to E2E Protocol Specification (R21-11), Figure 6.48, then calculate:

Crc_CalculateCRC8(0, Crc_Length: 1, Crc_StartValue8: computedCRC, Crc_IsFirstCall: FALSE)

Crc_CalculateCRC8 ()	
Polynomial	0x1D
CRC_StartValue8	Computed CRC (0x5F)
XOR Value	0xFF
Initial Value	0xA0 {CRC_StartValue8 (0x5F) ^ PolynomialXORVal (0xFF)}
Input Data Reflected	No
Result Data Reflected	No

CRC Input Data	0x00
Computed CRC	According to the inputs, result is equal to 0x5E

3) According to E2E Protocol Specification (R21-11), Figure 6.48, then calculate:

Crc_CalculateCRC8(Crc_DataPtr:&Data[1], Crc_Length: Length-1, Crc_StartValue8: computedCRC, Crc_IsFirstCall: FALSE)

Crc_CalculateCRC8 ()	
Polynomial	0x1D
CRC_StartValue8	Computed CRC (0x5E)
XOR Value	0xFF
Initial Value	0xA1 {CRC_StartValue8 (0x5E) ^ PolynomialXORVal (0xFF)}
Input Data Reflected	No
Result Data Reflected	No
CRC Input Data	0x11 0x00 0x00 0x00 0x00 0x00 0x00
Computed CRC	According to the inputs, result is equal to 0x88

4) Lastly, computed CRC should be XORed with 0xFF. So, result is following:

CRC = computedCRC ^ 0xFF

CRC Result	
Computed CRC	0x88
XOR Value	0xFF
CRC (Result)	0x77

Note: According to Specification of CRC Routines (R21-11) section 8.3, if Crc_IsFirstCall is equal to FALSE, Crc_StartValue8 should be XORed with 'XOR Value' to find initial value. If Crc_IsFirstCall is equal to TRUE, then Crc_StartValue8 can be used as initial value.

871257 v1 **E2E Profile 11 configuration**

Version history:
(1): 1. 20250528 新增Profile4和Profile11的E2E需求；

Purpose: E2E Profile settings/configuration
Source:
Verification Method: Analysis
Legacy ID:

Required configuration parameters:

Table 1. Profile 11

Parameter	Value	Profile
DataId	Provided in ECU extract	11
DataLength	Provided in ECU extract	11
DataIdMode	E2E_P11DATAID_NIBBLE	11
CounterOffset	8	11
CrcOffset	0	11
DataIdNibbleOffset	12	11
MaxDeltaCounter	Consult Note 1	11

Note 1: MaxDeltaCounter will be maximum allowed gap between two counter values of two consecutively received valid data. Range for MaxDeltaCounter will be between 1 and 14.

Example: If the receiver gets Data with counter 1 and MaxDeltaCounter is 2, then at the next reception the receiver can accept counters with values 2 and 3 but not 4.

Note 2: DataIDNibbleOffset is the bit offset of the low nibble of the high byte of Data ID for Profile 11.

871258 v1 E2E Profile 11 header layout

Version history:
(1): 1. 20250528 新增Profile4和Profile11的E2E需求；

Purpose: E2E Profile 11 header layout
Source:
Verification Method: Analysis
Legacy ID:

For Profile 11, header shall be implemented as following:

Byte Order	**0**								**1**							
Bit Order	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8
	E2E CRC								DataID Nibble				E2E Counter			

Note 1: DataID Nibble is the low nibble of the high byte of Data ID.

871263 v1 E2E Profile 4 CRC calculation

Version history:
(1): 1. 20250528 新增Profile4和Profile11的E2E需求；

Purpose: To increase error detection rate
Source:
Verification Method: Test
Legacy ID:

According to *[AUTOSAR_4 : E2E Protocol Specification]* and *[AUTOSAR_16 : Specification of CRC Routines]:*

E2E Profile	**E2E CRC Method**
Profile 4	Crc_CalculateCRC32P4 ()

For Profile 4:

As a generator polynomial 0xF4ACFB13 is used by Crc_CalculateCRC32P4 (). First CRC is calculated for bytes before CRC. Then, CRC is calculated again for bytes after CRC.
To make the example shorter and easier, the Data length is not longer than 32 bytes, just to show how to calculate CRC32P4.
All calculation steps are clarified as following:

Example: *Length = 0x0010, Counter = 0x0000, Data ID = 0x0A0B0C0D,*
Data = 0x00 0x00 0x00 0x00

Let's find CRC.

Note: According to Specification of CRC Routines (R21-11) section 8.3, if Crc_IsFirstCall is equal to FALSE, Crc_StartValue32 should be XORed with 'XOR Value' to find initial value.
If Crc_IsFirstCall is equal to TRUE, then Crc_StartValue32 can be used as initial value.

1) According to E2E Protocol Specification (R21-11) Figure 6.145, first calculate:

Crc_CalculateCRC32(Crc_DataPtr:&Data[0], Crc_Length: Offset + P04CRC_POS,
Crc_StartValue32: 0xFFFFFFFF, Crc_IsFirstCall: TRUE)

Crc_CalculateCRC32P4 () with <First Call = TRUE>	
Polynomial	0xF4ACFB13
CRC_StartValue32	0xFFFFFFFF
XOR Value	0xFFFFFFFF
Initial Value	0xFFFFFFFF
Input Data Reflected	TRUE
Result Data Reflected	TRUE
CRC Input Data (Data before CRC)	$reflect_8$ (0x00 0x10 0x00 0x00 0x0A 0x0B 0x0C 0x0D) = 0x00 0x08 0x00 0x00 0x50 0xD0 0x30 0xB0
Computed CRC	According to the inputs, result is equal to 0x0B06A731
	$reflect_{32}$ (0x0B06A731) = 0x8CE560D0

Note: 'Input Data Reflected = TRUE' means that the bits of each input byte are reflected before being processed. 'Result Data Reflected = TRUE' means that the bits of CRC result are reflected. The result is reflected over 32-bit for CRC32P4.

2) According to E2E Protocol Specification (R21-11) Figure 6.145, then calculate:

Crc_CalculateCRC32(Crc_DataPtr:&Data[Offset + P04CRC_POS + P04CRC_LEN],
Crc_Length: Length - (Offset + P04CRC_POS + P04CRC_LEN),
Crc_StartValue32: ComputedCRC, Crc_IsFirstCall: FALSE)

Note: According to Specification of CRC Routines (R21-11) section 8.3, in the case of 'Crc_IsFirstCall = FALSE' && 'Result Data Reflected = TRUE', ComputedCRC should be XORed with 'XOR Value' and then should be reflected over CRC size.

Crc_CalculateCRC32P4 () with <First Call = FALSE>	
Polynomial	0xF4ACFB13
CRC_StartValue32	0x8CE560D0
XOR Value	0xFFFFFFFF
Initial Value	$reflect_{32}$ (0x8CE560D0 ⊕ 0xFFFFFFFF) = $reflect_{32}$ (0x731A9F2F) = 0xF4F958CE
Input Data Reflected	TRUE
Result Data Reflected	TRUE
CRC Input Data (Data before CRC)	$reflect_8$ (0x00 0x00 0x00 0x00) = 0x00 0x00 0x00 0x00
Computed CRC	According to the inputs, result is equal to 0x6AA0D461
	$reflect_{32}$ (0x6AA0D461) = 0x862B0556

As seen in E2E Protocol Specification (R21-11), CRC value should be equal to 0x862B0556 if the requirements are defined as in the beginning of the example.

871261 v1 E2E Profile 4 configuration

Version history:
(1): 1. 20250528 新增Profile4和Profile11的E2E需求；

Purpose: E2E Profile settings/configuration
Source:
Verification Method: Analysis
Legacy ID:

Required configuration parameters:

Table 1. Profile 4

Parameter	Value	Profile
DataId	Provided in ECU extract	4
Offset	0	4
MinDataLength	96	4
MaxDataLength	32768	4
MaxDeltaCounter	Consult Note 1	4

Note 1: MaxDeltaCounter will be maximum allowed gap between two counter values of two consecutively received valid data. Range for MaxDeltaCounter will be between 1 and 65535.

Example: If the receiver gets Data with counter 1 and MaxDeltaCounter is 2, then at the next reception the receiver can accept counters with values 2 and 3 but not 4.

Note 2: MinDataLength will have a default value of 96 which means data includes only E2E header.

Note 3: Data Length will not be stored in configuration parameters. However, the length shall be calculated after padding process during E2E header creation. The length will also be provided in ECU extract. Data Length shall include user data + E2E header (CRC + Counter + Length + DataID).

871262 v1 E2E Profile 4 header layout

Version history:
(1): 1. 20250528 新增Profile4和Profile11的E2E需求；

Purpose: E2E Profile 4 header layout
Source:
Verification Method: Analysis
Legacy ID:

For Profile 4, header shall be implemented as following:

Byte Order	0								1							
Bit Order	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8
	E2E Length (MSB)								E2E Length (LSB)							

Byte Order	2								3							
Bit Order	23	22	21	20	19	18	17	16	31	30	29	28	27	26	25	24
	E2E Counter (MSB)								E2E Counter (LSB)							

Byte Order	4								5							
Bit Order	39	38	37	36	35	34	33	32	47	46	45	44	43	42	41	40
	E2E DataID (MSB)								E2E DataID							

Byte Order	6								7							
Bit Order	55	54	53	52	51	50	49	48	63	62	61	60	59	58	57	56
	E2E DataID								E2E DataID (LSB)							

Byte Order	8								9							
Bit Order	71	70	69	68	67	66	65	64	79	78	77	76	75	74	73	72
	E2E CRC (MSB)								E2E CRC							

Byte Order	10								11							
Bit Order	87	86	85	84	83	82	81	80	95	94	93	92	91	90	89	88
	E2E CRC								E2E CRC (LSB)							

871252 v1 E2E Profiles

Version history:
(1): 1. 20250528 新增Profile4和Profile11的E2E需求；

Purpose: E2E communication protection assumes that safety-related data exchange shall be protected at runtime against the effects of faults on the communication link
Source:
Verification Method: Analysis
Legacy ID:

E2E Profiles shall be implemented according to *[AUTOSAR_4 : E2E Protocol Specification]*

Maximum Data length	E2E Profile
32B	Profile 11
4kB	Profile 4

Note 1: If the signal length <8 bits shall be allocated to one byte of a message. It shall not span over two bytes. If the signal length >= 8 bits shall start or finish at the byte limit of the message.

Normally, SDB Team will pack the E2E signals correctly in the ECU extract and suppliers don't need to do any other extra work for re-packing or re-padding.

Note 2: Data Length is the sum of the E2E header fields (CRC, DataID, Counter and DataLength(*only for Profile 4)) and signal(s) in the form of multiple of 8-bits.

871254 v1 E2E Supervision

Version history:
(1): 1. 20250528 新增Profile4和Profile11的E2E需求；

Purpose: For handling of E2E profiles
Source:
Verification Method: Analysis
Legacy ID:

E2E Supervision shall be able to support multiple E2E Profiles.

871256 v1 **E2E Transformer**

Version history:
(1): 1. 20250528 新增Profile4和Profile11的E2E需求；

Purpose: To define E2E supervision mechanism
Source:
Verification Method: Analysis
Legacy ID:

E2E Supervision shall use E2E Transformer *[AUTOSAR_6 : Specification of Module E2E Transformer]*.

4.1.4.4 E2E Protection for Service

•E2E Protection for Service

4.1.4.4.1 E2E Protection for Service

4.1.4.4.1.1 Introduction

The following requirements can be released to the ECUs that meet the following conditions:
- Service-based communication (in SOME/IP) is enabled.

Whether E2E will be applied to these individual services/service interfaces and the selection of Profiles will depend on the actual ARXML file.

4.1.4.4.1.2 Requirements

871265 v2 **E2E Service Configuration for Profile11**

Version history:
(2): 20250917更新描述，增加profile 11字样，验证方法从Test改为Analysis AND Test

Purpose: E2E Parameters for Service Communication from the AutoSAR based Endpoint
Source:
Verification Method: Analysis AND Test
Legacy ID:

E2E Service for Profile11 should be configured as the following table:

Attribute	Setting
Profile Name	PROFILE_11
Data ID Mode	E2E_P11DATAID_NIBBLE/ Lower 12 Bit
Upper Header Bits to Shift	64
Window Size for Valid	2
Window Size for Invalid	2
Window Size for Init	2
Offset	/
Sync Counter Init	0
Counter Offset	72
CRC Offset	64
Data Id Nibble Offset	76
Max Delta Counter	14
Max Delta Counter Init	14

Max Error State Init	1
Max Error State Valid	1
Max Error State Invalid	1
Max No New Or Repeated Data	/
Min Ok State Init	1
Min Ok State Valid	1
Min Ok State Invalid	1
"Version" of End to End Transformer	1.0.0

1029379 v1 **E2E Service Configuration for Profile4**

Version history:
(1): 20250917新增服务的Profile11 E2E需求

Purpose: E2E Parameters for Service Communication from the AutoSAR based Endpoint
Source:
Verification Method: Analysis AND Test
Legacy ID:

E2E Service for Profile4 should be configured as the following table:

Attribute	Setting
Profile Name	PROFILE_4
Data ID Mode	/
Upper Header Bits to Shift	64
Window Size for Valid	2
Window Size for Invalid	2
Window Size for Init	2
Offset	64
Sync Counter Init	0
Counter Offset	/
CRC Offset	/
Data Id Nibble Offset	/
Max Delta Counter	14
Max Delta Counter Init	14
Max Error State Init	1
Max Error State Valid	1
Max Error State Invalid	1
Max No New Or Repeated Data	/
Min Ok State Init	1
Min Ok State Valid	1
Min Ok State Invalid	1
"Version" of End to End Transformer	1.0.0
unusedBitPattern	0xFF

4.1.4.4.1.2.1 Appendix Example for CRC Calculation

4.1.4.4.1.2.1.1 E2E Profile4 for service CRC calculation

According to *[*AUTOSAR_FO_PRS_E2EProtocol*]*， *[*AUTOSAR_CP_SWS_E2ELibrary*]*
and *[*AUTOSAR_CP_SWS_CRCLibrary*]:*

E2E Profile	E2E CRC Method
Profile 4	Crc_CalculateCRC32P4 ()

For Profile 4:

As a generator polynomial 0xF4ACFB13 is used by Crc_CalculateCRC32P4 (). First CRC is calculated for bytes before CRC. Then, CRC is calculated again for bytes after CRC.
To make the example shorter and easier, the Data length is not longer than 32 bytes, just to show how to calculate CRC32P4.
All calculation steps are clarified as following:

Example: *Length = 0x0018, Counter = 0x0000, Data ID = 0x0A0B0C0D,*
Data = 0x00 0x00 0x00 0x00

Let's find CRC.

Note: According to Specification of CRC Library (R24-11) section 8.3, if Crc_IsFirstCall is equal to FALSE, Crc_StartValue32 should be XORed with 'XOR Value' to find initial value.
If Crc_IsFirstCall is equal to TRUE, then Crc_StartValue32 can be used as initial value.

1) According to E2E Protocol Specification (R24-11) Figure 6.8, first calculate:

Crc_CalculateCRC32P4(Crc_DataPtr:&Data[0], Crc_Length: Offset + P04CRC_POS,
Crc_StartValue32: 0xFFFFFFFF, Crc_IsFirstCall: TRUE)

Crc_CalculateCRC32P4 () with <First Call = TRUE>	
Polynomial	0xF4ACFB13
CRC_StartValue32	0xFFFFFFFF
XOR Value	0xFFFFFFFF
Initial Value	0xFFFFFFFF
Input Data Reflected	Yes
Result Data Reflected	Yes
CRC Input Data (Data before CRC)	reflect8 (0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x18 0x00 0x00 0x0A 0x0B 0x0C 0x0D) =0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x18 0x00 0x00 0x50 0xD0 0x30 0xB0
Computed CRC	According to the inputs, result is equal to 0xA8F8128D
	$reflect_{32}$ (0xA8F8128D) = 0xB1481F15

Note: 'Input Data Reflected = TRUE' means that the bits of each input byte are reflected before being processed. 'Result Data Reflected = TRUE' means that the bits of CRC result are reflected. The result is reflected over 32-bit for CRC32P4.

2) According to E2E Protocol Specification (R24-11) Figure 6.8, then calculate:

Crc_CalculateCRC32(Crc_DataPtr:&Data[Offset + P04CRC_POS + P04CRC_LEN],
Crc_Length: Length - (Offset + P04CRC_POS + P04CRC_LEN),
Crc_StartValue32: ComputedCRC, Crc_IsFirstCall: FALSE)

Note: According to Specification of CRC Library (R24-11) section 8.3, in the case of 'Crc_IsFirstCall = FALSE' && 'Result Data Reflected = TRUE', ComputedCRC should be XORed with 'XOR Value' and then should be reflected over CRC size.

Crc_CalculateCRC32P4 () with <First Call = FALSE>	
Polynomial	0xF4ACFB13
CRC_StartValue32	0xB1481F15

XOR Value	0xFFFFFFFF
Initial Value	$reflect_{32}$ (0xB1481F15 ^ 0xFFFFFFFF) = $reflect_{32}$ (0x4EB7E0EA) = 0x5707ED72
Input Data Reflected	Yes
Result Data Reflected	Yes
CRC Input Data (Data before CRC)	$reflect_{8}$ (0x00 0x00 0x00 0x00) = 0x00 0x00 0x00 0x00
Computed CRC	According to the inputs, result is equal to 0x740AEB96
	$reflect_{32}$ (0x740AEB96) = 0x69D7502E

As seen in E2E Protocol Specification (R24-11), CRC value should be equal to 0x69D7502E if the requirements are defined as in the beginning of the example.

4.1.4.4.1.2.1.2 E2E Profile11 for service CRC calculation

According to *[*AUTOSAR_FO_PRS_E2EProtocol*]*, *[*AUTOSAR_CP_SWS_E2ELibrary*]* and *[*AUTOSAR_CP_SWS_CRCLibrary*]:*

E2E Profile	E2E CRC Method
Profile 11	Crc_CalculateCRC8 ()

For Profile 11:

E2E_P11DataIDMode = E2E_P11_DATAID_NIBBLE means that the lower nibble of the high byte of the DataID shall be transmitted explicitly at bit position DataIDNibbleOffset. CRC calculation shall be done by starting low byte[1] of the DataID which is implicitly transmitted, then a 0-byte[2] and then the user data[3]

As a generator polynomial 0x1D is used by Crc_CalculateCRC8 (). All calculation steps are clarified as following:

Example: Data ID = 0x123, Counter = 0, Data = 0x00 0x00 0x00 0x00 0x00 0x00

1) According to E2E Protocol Specification (R24-11), Figure 6.97, first calculate:

Crc_CalculateCRC8(Config->DataID, Crc_Length:1, Crc_StartValue8: 0xFF, Crc_IsFirstCall: FALSE)

Crc_CalculateCRC8 ()	
Polynomial	0x1D
CRC_StartValue8	0xFF
XOR Value	0xFF
Initial Value	0x00 {CRC_StartValue8 (0xFF) ^ PolynomialXORVal (0xFF)}
Input Data Reflected	No
Result Data Reflected	No
CRC Input Data	DataID Low Byte (Example : 0x23)
Computed CRC	According to the inputs, result is equal to 0x5F

2) According to E2E Protocol Specification (R24-11), Figure 6.97, then calculate:

Crc_CalculateCRC8(0, Crc_Length: 1, Crc_StartValue8: computedCRC, Crc_IsFirstCall: FALSE)

Crc_CalculateCRC8 ()	
Polynomial	0x1D
CRC_StartValue8	Computed CRC (0x5F)
XOR Value	0xFF

Initial Value	0xA0 {CRC_StartValue8 (0x5F) ^ PolynomialXORVal (0xFF)}
Input Data Reflected	No
Result Data Reflected	No
CRC Input Data	0x00
Computed CRC	According to the inputs, result is equal to 0x5E

3) According to E2E Protocol Specification (R24-11), Figure 6.97, then calculate:

Crc_CalculateCRC8(Crc_DataPtr:&Data[0], Crc_Length: Length, Crc_StartValue8: computedCRC, Crc_IsFirstCall: FALSE)

Crc_CalculateCRC8 ()	
Polynomial	0x1D
CRC_StartValue8	Computed CRC (0x5E)
XOR Value	0xFF
Initial Value	0xA1 {CRC_StartValue8 (0x5E) ^ PolynomialXORVal (0xFF)}
Input Data Reflected	No
Result Data Reflected	No
CRC Input Data	0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
Computed CRC	According to the inputs, result is equal to 0x33

4) According to E2E Protocol Specification (R24-11), Figure 6.97, then calculate:

Crc_CalculateCRC8(Crc_DataPtr:&Data[Offset+1], Crc_Length: Length-Offset-1, Crc_StartValue8: ComputedCRC, Crc_IsFirstCall: FALSE)

Crc_CalculateCRC8 ()	
Polynomial	0x1D
CRC_StartValue8	Computed CRC (0x33)
XOR Value	0xFF
Initial Value	0xCC {CRC_StartValue8 (0x33) ^ PolynomialXORVal (0xFF)}
Input Data Reflected	No
Result Data Reflected	No
CRC Input Data	0x10 0x00 0x00 0x00 0x00 0x00 0x00
Computed CRC	According to the inputs, result is equal to 0x82

4) Lastly, computed CRC should be XORed with 0xFF. So, result is following:

CRC = computedCRC ^ 0xFF

CRC Result	
Computed CRC	0x82
XOR Value	0xFF
CRC (Result)	0x7D

Note: According to Specification of CRC Library (R24-11) section 8.3, if Crc_IsFirstCall is equal to FALSE, Crc_StartValue8 should be XORed with 'XOR Value' to find initial value. If Crc_IsFirstCall is equal to TRUE, then Crc_StartValue8 can be used as initial value.

4.2 Car Configuration

4.2.1 Car Configuration

4.2.1.1 Car Configuration Slave

•Requisite documents

4.2.1.1.1 Requisite documents

Reference Num	Title	Logical Component name
[CCF_1]	SRD Car Configuration Management System	Car Configuration Master
[CCF_2]	Unified Diagnostic Services (UDS) Data	UDS Data

•Definitions and Abbreviations

4.2.1.1.2 Definitions and Abbreviations

Definition	Description
CC Subscriber ECU	Car Configuration Subscriber: A Domain Master ECU or an ECU part of a Domain Master sub-bus network, utilizing Car Configuration parameters for SW configuration.
Component	An element that encapsulates behaviour. Could be either physical or logical.
System requirement	Input requirements to the system development process are referred to as System Requirements. These are requirements on a system as a whole and not divided onto components.
System design	A result from the part of the development process called system design. The main purpose of this process is to decide and describe how the system should be implemented in order to fulfil the system requirements (input requirements).
Function Area	A Function Area (FA) is an area of functional responsibility for a subset of the entire functionality of a car.

Abbreviation	Stands for	Description
CC	Car Configuration	Generally used as short for Central Car Configuration.
CCC	Central Car Configuration	A base technology for vehicle configuration.
CCDB	Car Configuration Database	A tool for handling all Central Car Configuration data and for generating all related logic files (CCL).
CCL	Car Configuration Logic	VBF-file containing all CCP logical conditions for generating the VCP data.
CCP	Car Configuration Parameter	Car Configuration parameters are considered global parameters broadcasted within the vehicle network.
CC Subscriber ECU	Car Configuration Subscriber	Car Configuration Subscriber: A Domain Master ECU or an ECU part of a Domain Master sub-bus network, utilizing Car Configuration parameters for SW configuration.
DID	Data ID	View *Ref[CCF_2]*.
DTC	Diagnostic Trouble Code	View *Ref[CCF_2]*.
ECU	Electronic Control Unit	
EoL	End-of-Line	Final manufacturing stage for customer cars, all software is downloaded via factory tester/VCATS etc.
NVM	Non-Volatile Memory	E.g. FLASH, EEPROM.
PSS	Product System Structure	
P/T	Powertrain	Vehicle propulsion unit; in this context consisting of an internal combustion engine, combined with clutch, transmission and/or electrical motor.
SRD	System Requirements and Description	
SW	Software	

SWDL	Software Download	
TBD	To Be Defined	
VBF	Versatile Binary Format	SWDL file format.
VCD	Vehicle Configuration Data	A factory system generated VBF file consisting of vehicle specific Information Data and P-Spec sections respectively.
VCP	Vehicle Configuration Parameters	The VCP is the actual generated parameter values prepared for broadcast within the ECU network architecture.
VM	Volatile Memory	E.g. RAM.

•Central Car Configuration Requirements

4.2.1.1.3 Central Car Configuration Requirements

Central Car Configuration is a function to perform multiple ECU software configurations within a multiplexed network. Pre-defined car/functional parameters are generated by a Domain Master and distributed periodically to all other Domain Masters and sub-bus network ECUs within the platform architecture.

Main purpose of the CCC function is to minimize the necessary amount of ECU hardware and software variants with diversified functional behaviour (e.g. dependent on customer vehicle fitted parts and feature setup), i.e. connected/subscribing ECUs subscribe to CCC data in order to adapt to its current environment.

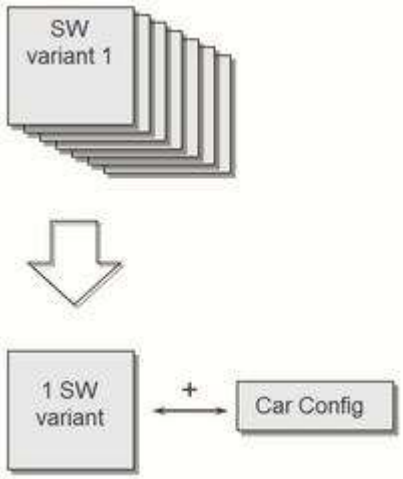


Figure: SW variant and Car Config

Car Configuration parameters are considered global parameters broadcasted within the vehicle network. CCPs are updated/modified through SWDL at EoL or aftersales. Customer settings/personalization, within an individual ECU SW, can interact with or override the global parameter setting for a specific function, but shall not alter the CCC data.

CCC ECU usage summary:

- Reduces the number of ECU software parts.
- Makes it possible to use the same software in an ECU - and configure the software “on-line”.
- Traditional software configuration can be combined with software configuration using Car Config.
- The software must be tested with all valid parameter settings - and not valid.
- Test work is not reduced.

4.2.1.1.3.1 Car Configuration Parameter Data

To support the working process some guidelines regarding administration and usage of CCPs are presented below. View figure *CCC Concept Overview* in chapter *Central Car Configuration* for concept overview and view chapter *Example of Car Config model* for usage example.

4.2.1.1.3.1.1 Parameter Creation

Function, subsystem or node developers request for parameter definitions and usage in order to configure specific

functionality for the realizing system solution.

All CCP change requests shall be initiated according to the process stated for each project, which the CC team will monitor. If the configuration need can be achieved using the VCD and CCL, e.g. within a specific vehicle platform, the definition of the parameter (and its connected logical conditions) is handled in CCDB, by the CC team.

4.2.1.1.3.1.2 Parameter Properties

Each CCP shall have a defined PSS ownership, presumably a function, subsystem or node owner.
Each CCP shall also have a lifetime limit, in reality meaning that the parameter can be removed in case it is no longer used for configuration in any ECU SW.
Changes and additions to parameter definition shall be approved by the parameter owner and the CC team.
Each CCP size is 1 Byte, 254 possible hex values (excluding 0x00 & 0xFF).
View the table below for CCP example and associated data.

Table 1. Example of a CCP definition.

CCP#	CCP Name	CCP Description	CCP Value (Hex)	CCP Value Descriptions	VCP Block ID# (Hex)	VCP Byte Pos
13	Propulsion type	Level of vehicle electrification	01	Combustion engine only	2	7
			02	Hybrid vehicle		
			03	Plugin hybrid vehicle		

Note in the table above:
'#'-sign is short for number in the above table. Also, every CCP shall have a defined serial number (CCP#), parameter description, value definitions and descriptions respectively. Each CCP# has a correspondent VCP signal position (BlockID# & Byte Position, view chapter "VCP signal mapping" for more information) for obtaining a set value when broadcasted within the network.

4.2.1.1.3.1.3 Information in CCDB

All information related to Car Configuration parameters shall exist in CCDB. This is information about parameter description & coding, generation logics, PSS ownership, which ECUs are allowed to subscribe to a parameter and which parameters a certain CC Subscriber ECU configures by.

4.2.1.1.3.1.4 Parameter Subscription

Each new ECU subscription to a CCP shall be requested to the CC team and approved by the parameter owner. It shall also be reported if a parameter is no longer used by a CC Subscriber ECU.

In addition, each new function/subsystem subscription to a CCP shall be approved by the parameter owner (see chapter *Parameter Properties*). I.e. even if a node is already using the parameter for one function a new application must be done if a new function in the node shall use the parameter.

4.2.1.1.3.2 System Design

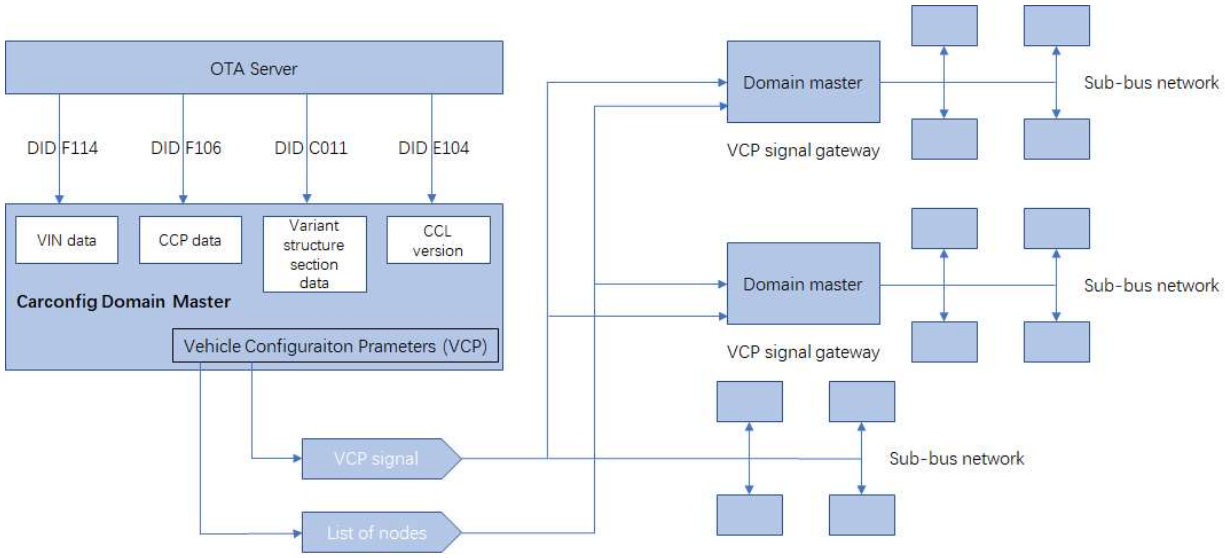


Figure: Car Configuration Management Sub-system design

The figure above gives a brief overview of the Car Configuration Management Sub-system design, details and requirements on actual implementation are given in *SRD Car Configuration Management System*, Requisite documents.

4.2.1.1.3.3 Car Configuration Subscriber Requirements

660801 v2 **CC Subscriber ECU DID support**

Version history:
(2):

Purpose: To identify how many, and which parameter number(s), that are faulty, up to 10 faulty parameters.
Source:
Verification Method: Test
Legacy ID: 65589v6
ASIL: QM - Not safety relevant

a) When receiving invalid or unrecognized parameter values (according to requirement *Invalid or unrecognized parameter values*), the CC Subscriber ECU shall store the faulty parameter and related value.

b) The CC Subscriber ECU shall support read out of *DID "Car Configuration - Faulty Parameters Received"* (0xE103 as specified in *[CCF_2] UDS Data*), format according to the table below. It shall be possible to read out up to 10 faulty CCPs identified by the CC Subscriber ECU, required only for parameter #/IDs that can be represented using 2 bytes hexadecimal code. If more then 10 faulty parameters are identified, 10 faulty parameters shall be able to read out but the number of invalid parameters shall represent the total amount of faulty parameters,If the number of invalid parameters roll over past 0xFF then the count value shall instead be maintained at 0xFF.

Table: DID Format, Invalid/Incompatible CCPs.

Byte Number	Content
(Byte 1)	Number of invalid parameters
Parameter 1 (Byte 2)	Faulty parameter (MSB)
Parameter 1 (Byte 3)	Faulty parameter (LSB)

Parameter 1 (Byte 4)	Faulty parameter value
Parameter 2 (Byte 5)	Faulty parameter (MSB)
Parameter 2 (Byte 6)	Faulty parameter (LSB)
Parameter 2 (Byte 7)	Faulty parameter value
…	…

660791 v3 **CC Subscriber ECU configuration states**

Version history:
(3): Update the verification method from 'Test' to 'Analysis and Test'

Purpose: To define the possible states (including necessary handling) of CC Subscriber ECU configuration using allowed CCP data.
Source:
Verification Method: Analysis AND Test
Legacy ID: 65555v6
ASIL: QM - Not safety relevant

a) CC legacy general principles:

The ECU/system shall determine what appropriate action(s) are to be taken for a specific function/feature when utilizing a default configuration or validated CCP data. The ECU/system failure mode handling/operating strategy might contain additional routines that need to be handled and documented.

CC Subscriber ECUs shall implement default values for CCPs based on any required default operating condition so that the appropriate functionality can be achieved and where necessary failure mode management actions shall be active.

The default operating conditions are dependent on the strategic requirements of the ECU. The operation of the functions within the ECU strategy shall be in line with FMEA rules affecting safety of the vehicle and occupants and customer satisfaction in the absence of configuration information. Wherever possible the vehicle should operate normally. If the vehicle could be made dangerous or perform unsatisfactory for the customer/occupant, then failure mode management actions need to be executed to prevent or change this vehicle behaviour. The failure mode management actions shall be in respect of the ECU FMEA recommended actions.

b) Two CC Subscriber ECU states are defined as follows:

- **Bulk State:** In this state to receive required Car Config parameters for the first time and keep state until all have been evaluated valid:
  - The below applies to every CCP that the subscriber ECU expects to receive and evaluate:
    - Whenever a valid CCP value is available/received it shall be used.
    - For a received but not valid CCP value, use the ECU default value for that parameter instead.

Note: The ECU default CCP value shall be defined before it is delivered, and it is up to the ECU owner confirm this with its supppliers. The general requirement of the default CCP value shall be safety and used in common vehicle.

- **Valid Configuration State:** In this state only to detect a possible change in car config parameter values:
  - The below applies to every CCP that the subscriber ECU expects to receive and evaluate:
    - For a received but not valid CCP value, consider using the previously stored valid value for that parameter instead.

ECU Bulk State Flow Chart and ECU Valid Configuration State Flow Chart provided for configuration using CCP data (distributed within the VCP signal) in the following requirements 665245 and 665259.

c) Evaluation of a CCP value implies that (compared to whatever configuration the ECU/system currently utilizes) the ECU/system must determine if a received CCP value is valid, e.g. recognized, within valid range for acceptance, possible to transition to if ECU already has a Valid Configuration State etc.

660792 v1 CC Subscriber ECU in Bulk (unconfigured) state

Version history:
(1):

Purpose: To identify an unconfigured/bulk state.
Source:
Verification Method: Test
Legacy ID: 65556v5
ASIL: QM - Not safety relevant

If the CC Subscriber ECU has not received or does not contain expected CCP data after 30 seconds from its application boot-up sequence while residing within ECU delivery default/Bulk State, the CC Subscriber ECU shall use *DTC "Car Configuration – Not Configured"* U2300-55, DTC test according to *[CCF_2] UDS Data*.

665263 v2 Car Configuration DTC DID record for CC Subscriber ECUs

Version history:
(2): Add note

Purpose: Table Car Configuration DTC DID record for CC Subscriber ECUs
Source: NA
Verification Method: Test
Legacy ID:

Table: Car Configuration DTC/DID record

Car Configuration DTC/DID record	UDS Data DTC/DID reference
Car Configuration – Not Configured	E300-55 (U2300-55)
Car Configuration – Invalid/Incompatible Configuration	E300-56 (U2300-56)
Car Configuration - Faulty Parameters Received	E103

Note:
1. DTC and DID will be cleared at the same time if there one pre-condition happen as follow:
1)The ECU receive the valid value of all the parameters they need.

2. The DID should report all 00(31bytes) if the ECU receive all valid value they need. The DID should report 0xFF(number of invalid parameters) if the ECU receive more than 255 invalid parameters.

3. The DID should only report the max 10 invalid values if ECU received above 10 incorrect parameters. But the number of invalid parameters should report the actual total number of invalid parameters.

660797 v3 Car Configuration data storage

Version history:
(3): Update the verification method from 'Test' to 'Analysis'

Purpose: The CC Subscriber ECU must be able to maintain parameter values between normal drive cycles, ECU resets, power cycles and low voltage due to engine cranking.
Source:
Verification Method: Analysis
Legacy ID: 65585v6
ASIL: QM - Not safety relevant

All CC Subscriber ECUs shall store their valid CCP data used for configuration in non-volatile memory (NVM). The format/interpretation of the parameter(s) to be stored shall be specified in the SWRS of each ECU subscribing to that parameter.

Note: The list of node shall be stored also if it is used by some ECUs.

660798 v2 Car Configuration parameter definition

Version history:
(2): Update the verification method from 'Analysis' to 'Analysis and Test'

Purpose: To assure that each Car Configuration parameter is used according to its original purpose.
Source:
Verification Method: Analysis AND Test
Legacy ID: 65586v2
ASIL: QM - Not safety relevant

A Car Configuration parameter shall have the same definition in the CC Subscriber ECU as in CCDB.

660795 v1 Configuration of ECU with updated CCP data

Version history:
(1):

Purpose: To enable automatic configuration/re-configuration of ECU SW using updated CCP data.
Source:
Verification Method: Test
Legacy ID: 65583v6
ASIL: QM - Not safety relevant

a) The CC Subscriber ECU shall have two separate states of configuration relating to Central Car Configuration parameters. The "Bulk State" is when the ECU is delivered/built-in without pre-programming, i.e. factory settings. The "Valid Configuration State" is when all expected/allowed CCP values are in use and have been received and evaluated valid at least once.

b) The CC Subscriber ECU is required to transition from "Bulk State" -> "Valid Configuration State" after a complete set of expected CCPs have been evaluated valid and used for SW configuration.

665245 v4 ECU Bulk State Flow Chart

Version history:
(4): Update the verification method from 'Test' to 'Analysis and Test'

Purpose: Guideline ECU Bulk State
Source: NA
Verification Method: Analysis AND Test
Legacy ID:

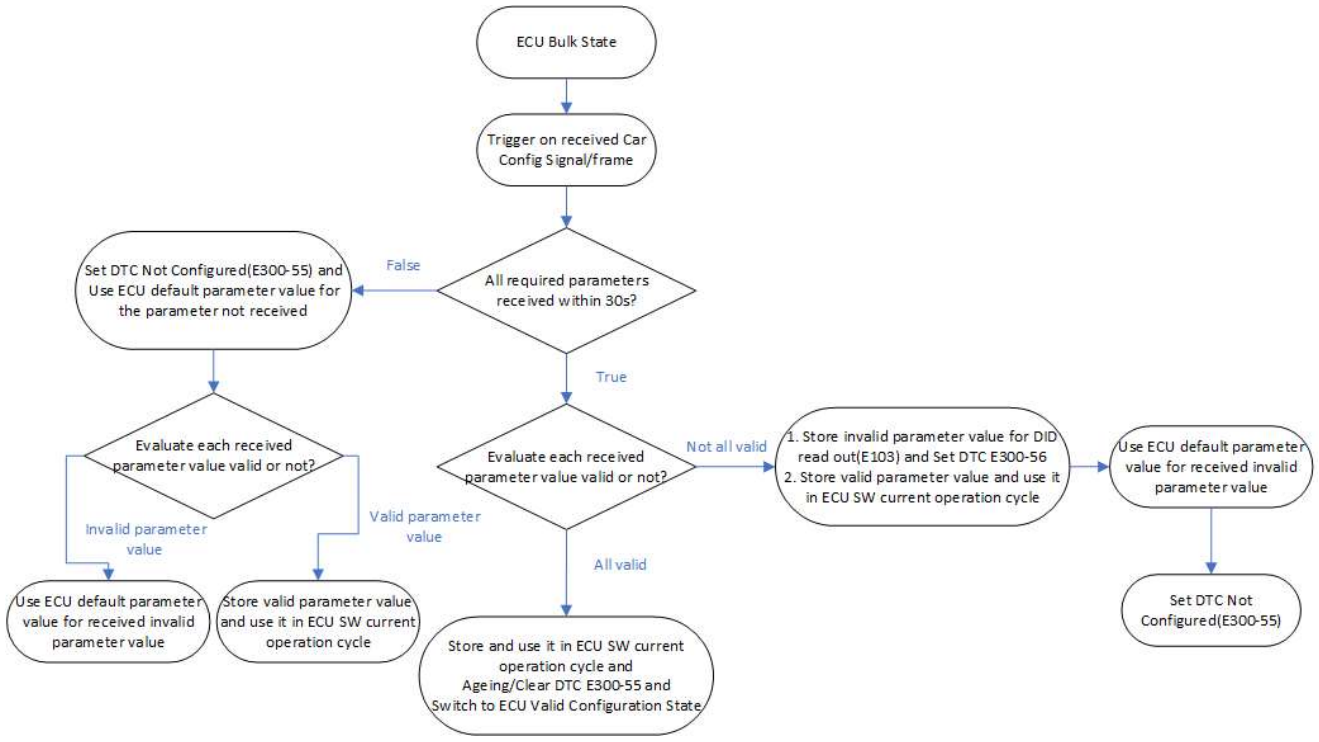


Figure: ECU Bulk State

Notes in the figure above:

1.Trigger on received Car Config Signal/frame: Useage mode changed from Abandon/Inactive/Convenience to Active/Driving.

2.Invalid value is the values are out of range in definition of ECU CCDB.

*3.*If the Car Config cannot be valid or be stored in ECU SW current operation cycle, such as requiring a restart or routine service to be valid or be stored, this deviation needs to be approved by GRI Bastech Team.

665259 v4 ECU Valid Configuration State Flow Chart

Version history:
(4): Update the verification method from 'Test' to 'Analysis and Test'

Purpose: Guideline ECU Valid Configuration State
Source: NA
Verification Method: Analysis AND Test
Legacy ID:

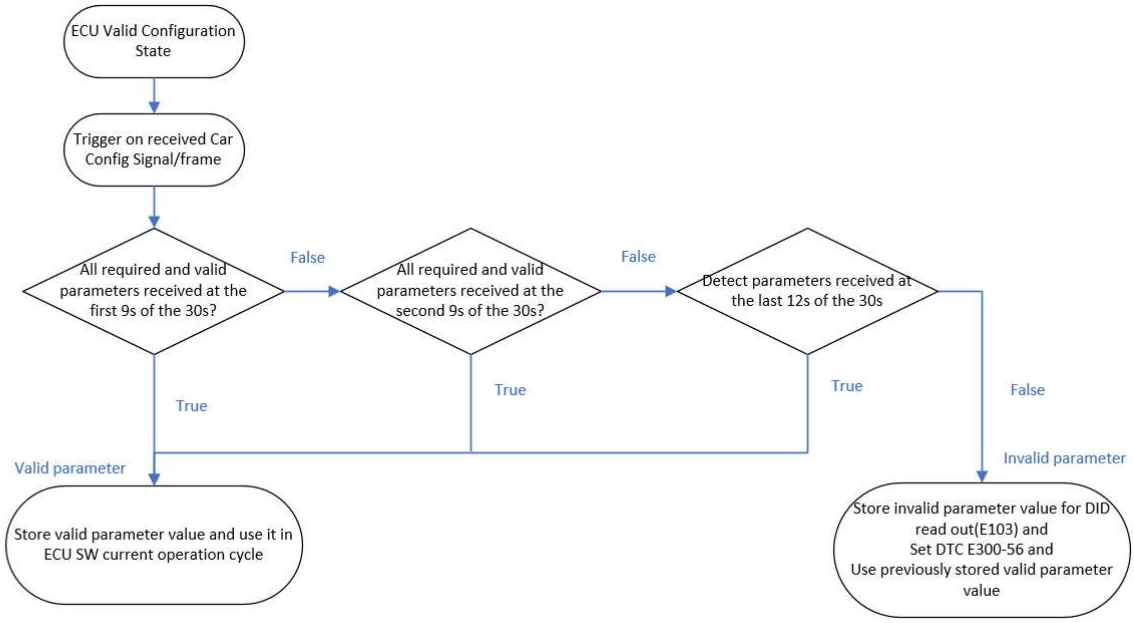


Figure: ECU Valid Configuration State

Notes in the figure above:
1.Trigger on received Car Config Signal/frame: Useage mode changed from Abandon/Inactive/Convenience to Active/Driving.
2.Invalid value is the values are out of range in definition of ECU CCDB.
3.For one parameter, the valid value received at second 9s or the last 12s should cover the invalid value received before, but the invalid value should not cover the valid value.
4.For one parameter, if the invalid value record in ECU was different from new invalid value in a new 30s,the ECU should report the lastest invalid value and old invalid value in ECU should be covered.
*5.*If the Car Config cannot be valid or be stored in ECU SW current operation cycle, such as requiring a restart or routine service to be valid or be stored, this deviation needs to be approved by GRI Bastech Team.

660800 v3 ECU configuration after boot-up

Version history:
(3): Update the verification method from 'Test' to 'Analysis'

Purpose: Enabling correct/updated CC Subscriber ECU configuration after ECU boot-up. Evaluation check of received parameters, after ECU boot-up, expected to be run only once per driving cycle.
Source:
Verification Method: Analysis
Legacy ID: 65588v5
ASIL: QM - Not safety relevant

After ECU application boot-up sequence the CC Subscriber ECU shall use the current set of stored CCP values (or default configuration if no previous data received/available).

Once, as soon as a new set of parameter values is received, the CC Subscriber ECU shall evaluate the new parameter values, dependent on Car Configure Database(CCDB) and ECU specification(ECU SWRS).

660799 v1 Handling of parameter data

Version history:
(1):

Purpose: General handling when reading CCPs within VCP signal and to avoid faulty ECU and vehicle behaviour respectively.
Source:
Verification Method: Test
Legacy ID: 65587v3
ASIL: QM - Not safety relevant

a) CCP value 00_h indicates that the CCP is not in use (not defined), e.g. not used within vehicle project/platform or spare/reserved CCP for future use.

b) CCP value FF_h indicates that the CCP has been generated erroneously.
c) CCP value 00_h and FF_h shall not be used for any ECU SW configuration.

660794 v2 Invalid or unrecognized parameter values

Version history:
(2): Update the verification method from 'Test' to 'Analysis and Test'

Purpose: To prevent any damage/security risk internally/externally in the CC Subscriber ECU associated with invalid or unrecognized parameter values.
Source:
Verification Method: Analysis AND Test
Legacy ID: 65582v6
ASIL: QM - Not safety relevant

If any CCP value(s) used for configuration (including values 00_H and FF_H) is received and evaluated invalid or unrecognized by the CC Subscriber ECU, the related function shall perform the following actions:

a) Enter a safe mode (depending on ECU strategy in line with FMEA rules) if there is an imminent risk of serious errors.

b) Use *DTC "Car Configuration –Invalid/Incompatible Configuration"* U2300-56, DTC test according to *[CCF_2] UDS Data*.

c) Minimize (avoid if possible) the impact on functions not related to parameter usage.

Note: For functions that are not safety nor security critical, whose behaviour/setting would be a similar or equal even if a received CCP value is not expected, a range of CCP values could be adopted as valid, e.g. CCP value range 0x01 to 0xFE is valid for configuration.

660793 v3 Routines for receiving VCP signal (CC Domain Master excluded)

Version history:
(3): Update the verification method from 'Test' to 'Analysis and Test'

Purpose: To assure flexibility of the CC Subscriber ECUs when receiving Car Configuration data.
Source:
Verification Method: Analysis AND Test
Legacy ID: 65561v7
ASIL: QM - Not safety relevant

a) The CC Subscriber ECU shall be able to read BlockID#, in the VCP signals, arriving in any order, in accordance with table *VCP signal mapping* in chapter *Record of tables.*

b) The CC Subscriber ECU shall be able to receive and process every arrival of a BlockID in the Vehicle configuration signals VehCfgPrm,VehCfgPrmExt and VehCfgPrmExt2.

660796 v2 **Selection of CCPs in VCP signal**

Version history:
(2): Update the verification method from 'Test' to 'Analysis and Test'

Purpose: To ensure that the correct CC Subscriber ECUs get the correct Car Configuration data, and to avoid misuse of CCP data resulting in unnecessary SW-updates when changes on CCP definitions occur.
Source:
Verification Method: Analysis AND Test
Legacy ID: 65584v3
ASIL: QM - Not safety relevant

The CC Subscriber ECU shall use information in a parameter list or equivalent (released/extracted from CCDB) containing the CCP numbers that the CC Subscriber ECU has requested use for or is required to configure by.

The BlockID# (in the VCP signal), and byte position, in which a specific CCP can be obtained, shall be according to table *VCP signal mapping* in chapter *Record of tables.*

665261 v2 **VCP signal mapping**

Version history:
(2):

Purpose: Table VCP signal mapping
Source: NA
Verification Method: Analysis
Legacy ID:

Tables 'VCP signal mapping' below gives a CCP number placement if broadcasted within a packet message of 64-bit data (8 bytes per VCP signal frame). When transmitted, a CCP# byte placement will in fact contain one vehicle specific generated hexadecimal value for that CCP#.

Table: VCP signal mapping - VehCfgPrm

Signal name: VehCfgPrm							
Byte_Pos_1 (Bits 7-0)	**Byte_Pos_2 (Bits 15-8)**	**Byte_Pos_3 (Bits 23-16)**	**Byte_Pos_4 (Bits 31-24)**	**Byte_Pos_5 (Bits 39-32)**	**Byte_Pos_6 (Bits 47-40)**	**Byte_Pos_7 (Bits 55-48)**	**Byte_Pos_8 (Bits 63-56)**
BlockID_#1 (MSB, Hex)	**<Car_Configuration_parameter_#>**						
01 $_H$	<1>	<2>	<3>	<4>	<5>	<6>	<7>
02 $_H$	<8>	<9>	<10>	<11>	<12>	<13>	<14>
03 $_H$	<15>	<16>	<17>	<18>	<19>	<20>	<21>
↓	...	...	...	...	...	...	...
48 $_H$	<498>	<499>	<500>	<501>	<502>	<503>	<504>

E.g. if CCP 10 has been assigned hex value 05, after a VCD/CCL à VCP conversion operation, subscribing ECUs can obtain the parameter 10 assigned value 0x05 (hex), when a 64-bit frame arrives with 1 st byte set to 02 $_h$, at byte position 4.

The BlockID number and the actual byte position (parameter byte offset within frame) that a specific CCP number

shall attain is calculated using the following formulas:

BlockID for CCPs 1-504:

$$BlockID\#_{10} = \text{int}\left(\frac{CCP\# + 6}{7}\right)$$

$$BytePos\# = CCP\# - (BlockID\#_{10} \times 7 - 6) + 2$$

Table: VCP signal mapping - VehCfgPrmExt

Signal name: VehCfgPrmExt							
Byte_Pos_1 (Bits 7-0)	Byte_Pos_2 (Bits 15-8)	Byte_Pos_3 (Bits 23-16)	Byte_Pos_4 (Bits 31-24)	Byte_Pos_5 (Bits 39-32)	Byte_Pos_6 (Bits 47-40)	Byte_Pos_7 (Bits 55-48)	Byte_Pos_8 (Bits 63-56)
BlockID_#1 (MSB, Hex)	<Car_Configuration_parameter_#>						
01_H	<505>	<506>	<507>	<508>	<509>	<510>	<511>
02_H	<512>	<513>	<514>	<515>	<516>	<517>	<518>
03_H	<519>	<520>	<521>	<522>	<523>	<524>	<525>
↓	...	...	...	...	...	...	...
47_H	<995>	<996>	<997>	<998>	<999>	NA	NA
48_H	NA	NA	NA	NA	NA	NA	NA

The BlockID number and the actual byte position (parameter byte offset within frame) that a specific CCP number shall attain is calculated using the following formulas:

BlockID for CCPs 505-999:

$$BlockID\#_{10} = \text{int}\left(\frac{CCP\# - 504 + 6}{7}\right)$$

$$BytePos\# = CCP\# - 504 - (BlockID\#_{10} \times 7 - 6) + 2$$

Table: VCP signal mapping - VehCfgPrmExt2

Signal name: VehCfgPrmExt2							
Byte_Pos_1 (Bits 7-0)	Byte_Pos_2 (Bits 15-8)	Byte_Pos_3 (Bits 23-16)	Byte_Pos_4 (Bits 31-24)	Byte_Pos_5 (Bits 39-32)	Byte_Pos_6 (Bits 47-40)	Byte_Pos_7 (Bits 55-48)	Byte_Pos_8 (Bits 63-56)
BlockID_#1 (MSB, Hex)	<Car_Configuration_parameter_#>						
01_H	<1256>	<1257>	<1258>	<1259>	<1260>	<1261>	<1262>
02_H	<1263>	<1264>	<1265>	<1266>	<1267>	<1268>	<1269>
03_H	<1270>	<1271>	<1272>	<1273>	<1274>	<1275>	<1276>
04_H	<1277>	<1278>	<1279>	<1280>	<1281>	<1282>	<1283>
05_H	<1284>	<1285>	<1286>	<1287>	<1288>	<1289>	<1290>
06_H	<1291>	<1292>	<1293>	<1294>	<1295>	<1296>	<1297>
07_H	<1298>	<1299>	<1300>	<1301>	<1302>	<1303>	<1304>

↓	...	...	...	...	...	...	...
46_H	<1739>	<1740>	<1741>	<1742>	<1743>	<NA>	<NA>
47_H	<NA>	<NA>	<NA>	<NA>	<NA>	<NA>	<NA>
48_H	<NA>	<NA>	<NA>	<NA>	<NA>	<NA>	<NA>

The BlockID number and the actual byte position (parameter byte offset within frame) that a specific CCP number shall attain is calculated using the following formulas:

BlockID for CCPs 1256-1743:

$$BlockID\#_{10} = \text{int}\left(\frac{CCP\# - 1255 + 6}{7}\right)$$

$$BytePos\# = CCP\# - 1255 - (BlockID\#_{10} \times 7 - 6) + 2$$

4.2.1.1.3.4 Signals provided by the system

Table: SDB signal VehCfgPrm **extract.**

Signal name VehCfgPrm					
Publishing component CC Domain Master	**Size [bits]** 64	**Signal type** Sporadic	**Write interval [ms]** 125	**Max read interval** 40ms	
Description:	Signal used for the distribution of Vehicle Configuration Parameters (VCP) #1 - #504. At least two consecutive cycles of blockID:s 0x01 - 0x48 (dec 1 - 72) are distributed within 30 seconds from transition of UsageMode to 'Active' or 'Driving'. CC Subscribers shall be able to handle a reduction of the write interval.				
Encoding type:	View table *VCP signal mapping* in chapter *Record of tables*.				

Table: SDB signal VehCfgPrmExt **extract.**

Signal name VehCfgPrmExt					
Publishing component CC Domain Master	**Size [bits]** 64	**Signal type** Sporadic	**Write interval [ms]** 125	**Max read interval** 40ms	
Description:	Signal used for the distribution of Vehicle Configuration Parameters (VCP) #505 - #999. At least two consecutive cycles of blockID:s 0x01 - 0x48 (dec 1 - 72) are distributed within 30 seconds from transition of UsageMode to 'Active' or 'Driving'. CC Subscribers shall be able to handle a reduction of the write interval.				
Encoding type:	View table *VCP signal mapping* in chapter *Record of tables*.				

Table: SDB signal VehCfgPrmExt2 **extract.**

Signal name

VehCfgPrmExt2					
Publishing component CC Domain Master	**Size [bits]** 64	**Signal type** Sporadic	**Write interval [ms]** 125	**Max read interval** 40ms	
Description:	Signal used for the distribution of Vehicle Configuration Parameters (VCP) #1256 - #1743. At least two consecutive cycles of blockID:s 0x01 - 0x48 (dec 1 - 72) are distributed within 30 seconds from transition of UsageMode to 'Active' or 'Driving'. CC Subscribers shall be able to handle a reduction of the write interval.				
Encoding type:	View table *VCP signal mapping* in chapter *Record of tables*.				

4.3 Network Management

4.3.1 Network Management

4.3.1.1 Network Management - Generic reqs

•Requirements

4.3.1.1.1 Requirements

675938 v2 **Ethernet node linkup & wakeup strategy**

Version history:
(2): Ethernet wake-up strategy adjustment, increase the IPLM mechanism, change the requirement description

Purpose: To define the ECU with Ethernet network management strategy.
Source:
Verification Method: Test
Legacy ID:

The Ethernet communication is woken up by the CAN communication, and the Ethernet communication is maintained by the IPLM. This means that if an Ethernet node wants to wake up the Ethernet communication of another node, it must first wake up the CAN communication of this node.

717160 v1 **Initiation strategy for OS**

Version history:
(1):

Purpose: Optimized the boot time of the operating system
Source:
Verification Method: Analysis
Legacy ID:

If the operating system in the controller is running on SOC, it is recommended to use methods such as STR (Suspend to RAM) to reduce the startup time of the operating system. It allows the operating system to save the current system state to RAM in a low-power state and restore the system state from RAM on the next wake-up, thereby achieving fast startup. If there are other quick start methods, you can also communicate with the Basetech Team.
Notes:
1. Any means of disabling or resetting STR must be communicated with the Basetech Team

2. If the controller enters STR mode, it must meet the requirements of VMM for static current

如果控制器中的操作系统运行在SOC上，为了减少操作系统的启动时间，建议采用如STR（Suspend to RAM）等手段，它允许操作系统在低功耗状态下将当前的系统状态保存到RAM中，并在下一次唤醒时从RAM中恢复系统状态，从而实现快速启动。如果有其他的快速启动方式，也可同Basetech Team沟通。

注意：

1、任何使STR失效或重置的手段均需同Basetech Team沟通

2、如果控制器进入STR模式，需满足VMM对静态电流的要求

870516 v1 **NM Frame Format of CANFD Segment**

Version history:
(1): 1. 20250528 新增CANFD通信协议中网络管理报文格式需求；

Purpose: To make partial networking wake-up compatible with regular frame wake-up
Source:
Verification Method: Test
Legacy ID: ZR 702450

Even in CANFD network segment, NM message in CAN frame format can only be sent, it means that the FDF Flag in NM Frame should be set to 0.

Note:TJA1145 Datasheet says that it can only be awaken by CAN Format frame.

870513 v1 **NM PDU Format**

Version history:
(1): 1. 20250528 新增NM报文格式需求；

Purpose: To define CAN NM PDU Format and default value for unused bits .
Source: Autosar NM ： Network Management PDU Structure
Verification Method: Test
Legacy ID: ZR 736260

The format of NM PDU is shown as follow:

	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
Byte0	Source Node Identifier							
Byte1	Control bit vector							
Byte2	PN bit vector							
Byte3	PN bit vector							
Byte4	PN bit vector							
Byte5	PN bit vector							
Byte6	User data 0							
Byte7	User data 1							

The figure below describes the format of the CBV byte:

	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
CBV	Reserved	Partial Network Information Bit	Reserved	Active Wakeup Bit	NM Coordinator Sleep Ready Bit	Reserved R3.2 NM Coordinator ID (High Bit)	Reserved R3.2 NM Coordinator ID (Low Bit)	Repeat Message Request

The Bit 4 & Bit 6 in CBV must be used and follow AUTOSAR requirement .
Note: Undefined or not applicable PN Bit must be set to 0 in PN Info by default.
Note：The Bit6 in CBV must be set 1.

660826 v2 **NM frame shall be sending request of wakeup on CAN network**

Version history:
(2):

Purpose: To secure that the first frame should be NM frame for active wakeup ECU on CAN network
Source:
Verification Method: Test
Legacy ID:

When the ECU needs to actively wake up the CAN/CANFD network, it always sends NM frames firstly, and can only send application frames after successful transmission.
The ECU usually sends application frame as soon as possible when it receive NM frame that requires ECU to activate.

Note1*: Passive mode is not allowed.*

Note2*:* A successful transmitted frame should have the complete frame structure, including the correct ACK, NAK error frame is not a successful transmitted frame.

660824 v2 **Network management CAN frame identifiers**

Version history:
(2):

Purpose: To ensure network management functionality.
Source: -
Verification Method: Test
Legacy ID: 130742v31

An ECU using AUTOSAR CAN/CANFD Network Management shall be able to receive network management frames with CAN/CANFD identifiers ranging from 0x500 to 0x53F.

All ECU which support network management shall only start transmission of application frames if an appropriate network management frame indicates the network should be active.

For CAN/CANFD Network Management the NodeId (CanNmNodeId) shall be equal to the 8 least significant bits of the CAN frame identifier in which the NM-PDU is transmitted.

Example:Platform CAN/CANFD frame identifier is 0x501, CanNmNodeId is 0x01.

The CAN/CANFD frame identifiers shall be defined in ECU Extract and DBC.

Note 1: *NM CAN/CANFD Frames uses IDs in range 0x500-0x53F. If NM CAN/CANFD Frames ID are not defined in this range by the ECU Extract and DBC they shall be considered spare IDs.*
Note 2: *This requirement applies to all CAN/CANFD ECUs where AUTOSAR Network Management is used.*

Note 3: *The ECU which connected OBD-II Diagnostic CAN directly can be waken up not only by NM frames but also by Diagnostic frames.*

660827 v1 **Network management mechanism for handling abnormal communication**

Version history:
(1):

Purpose: To ensure ECU can recover normal communication when abnormal awakening occurs
Source: VMM requirement ID:98057
Verification Method: Analysis
Legacy ID:

According to VMM requirement ID 98057 description, ECU shall ensure go to Off / Sleep state within 15 sec after the network is released (i.e. no communication). But sometime ECU may occurs software bug bring about ECU cannot go to sleep, ECU shall have a measure to ensure ECU can go to sleep after network is released.

Example of abnormal wakeup event handling is as follows:
ECU should monitor network states, when no ECU (including itself) request network communication, but ECU itself always sends NM frame without PNC setting on CAN/CANFD network , it may reset itself with 15 sec after it detect abnormal communication state, to ensure network communication can be restored.

This requirement is valid when there is no safety requirement that keeps the function/component active. For example, this requirement is not applicable for safety reasons while the vehicle is still moving.

Functions that need more than 15 sec must apply for a deviation. The deviation must be approved by the basetech team at Electrical Architecture.

4.3.1.2 AutoSar NM - Generic reqs

•Overview

4.3.1.2.1 Overview

Software Component Description
This document describes requirements that applies to an Autosar based node and are in addition to the behavior defined by Autosar network management BSW. It applies to CAN and CANFD networks.

Component context
This SWRS describes Network Management for Autosar based nodes. Network management within Autosar is limited compared to the desired network management functionality. The basic scope of Autosar network management is to coordinate the network requests of all or some nodes on a single network in order to obtain a controlled, simultaneous and synchronized shutdown of communication when those nodes are ready to go to sleep. With partial networking (PNC) the scope applies to multiple networks.

The scope of network management desired is more extensive and is described in this document. It includes:

- Start-up timing of communication on a node by network wake up, hardware wake up and power on (i.e. also relates to hardware performance).

The ECU specfic decision when to request a network to be active or not active is not defined in this document. However network management is a mechanism and protocol where NM-PDUs are sent between nodes for communicating the needs for network communication. Communication needs may be defined elsewhere and determined on logical level (in application SW-Cs) by Virtual Function Cluster (VFC) activation and deactivation criteras. If BSW supports partial networking, then VFC requests are translated in BSW to Partial Network Clusters (PNC) requests which are communicated among nodes in NM-PDUs. PNCs are related to physical networks being requested active or not active. Partial networking may improve energy efficiency by only activating currently needed networks and nodes. In this specification, timing requirements for a node are only applicable when network communication has been granted to be activated, the specific communication conditions is expected to be found in node specific requirements. Other generic requirements applicable when implementing logic for requesting

communication can be found in *Autosar - ECU Mode Management*.

Note
This document covers both the older CAN standard, referred to as classic CAN, and the updated standard CAN Flexible Data-Rate, referred to as CAN FD. Unless explicitly specifying the standard of CAN in this document, any reference to CAN refers to both the classic CAN and CAN FD standards.

4.3.1.2.1.1 Definitions and Abbreviations

Definition	Description
AR	Autosar
Bus	See network
Clamp 15	A power supply/control signal that activate nodes.
Clamp 30	A permanent power supply.
Coordination clustering	A discrete set of NM Channels on which shutdown is coordinated. The NM Coordinator will keep all presently awake NM Channels of a NM Coordination Cluster awake until it is possible to coordinate network sleep on all the awake channels.
ECU	Same as node in this document, see Node.
Full Autosar	A node where at least all RTE and all BSW modules are Autosar.
HW	Hardware
Network	An interconnection of nodes for the purpose of exchanging information.
Network type	Network types are LIN, CAN and CANFD networks.
NM	Network Management
NM User Data	A number of payload bytes in an NM-PDU in a frame that can be used for signalling arbitrary information. If PN is supported then PNC request bits are transmitted in NM User Data.
Node	An electronic control unit that execute software.
Node type	See *Node types, Software Component Requirements – Requirement Content*.
Partial network	System state in a CAN/CANFD system in which some nodes are sleeping while other nodes are communicating.
Selective wakeup transceiver	A CAN transceiver according to ISO11898-6 which allows a node that supports PN to be in sleep when no PNC requests are active for the specific node. The transceiver will wakeup the node when frames with defined CAN identifiers and PNC bits match a predefined configuration.
SW	Software

Abbreviation	Description
AR	Autosar
BSW	Basic Software
DID	Diagnostic Identifier
DTC	Diagnostic Trouble Code
FIFO	First in first out
HW	Hardware
IGN	Ignition
µC	Micro controller
N/A	Not applicable
NM	Network Management
NMPDU	Network Management Protocol Data Unit
PDU	Protocol Data Unit
PN	Partial Network
PNC	Partial Network Cluster. Partial Network Cluster is a group of system signals necessary to support one or more vehicle functions that are distributed across multiple ECUs in the vehicle network. This represents the system view of mapping a group of buses to one ore more VFCs.
POC	Protocol Operation Control

RTE	Real Time Environment
StbM	Synchronized Time Base Manager
SW-C	Software Component
VFC	Virtual Function Cluster. Virtual Function Cluster groups the communication on port level between SW- components that are required to realize one or more vehicle functions. This is the logical view and allows for a reusable bus/ECU independent design.

4.3.1.2.1.2 Requisite documents

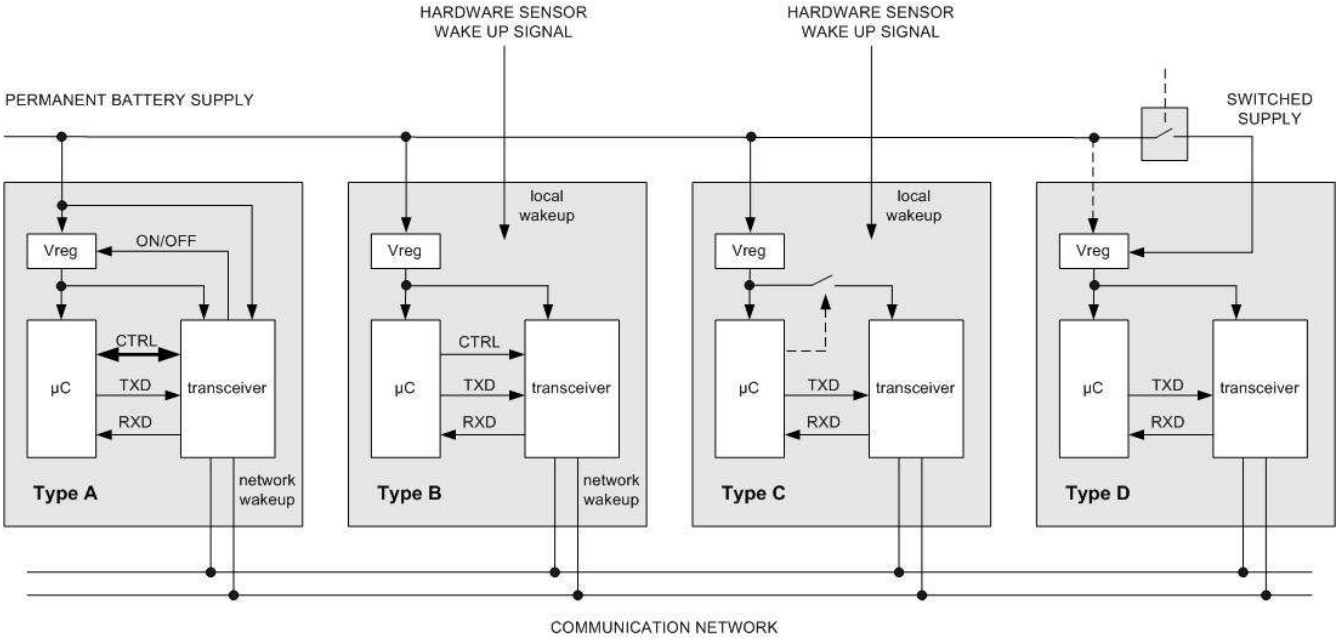
Reference	Source	Document
[AR_NM_1]	AUTOSAR	Autosar Specification of Generic Network Management Interface AUTOSAR_SWS_NetworkManagementInterface.pdf
[AR_NM_2]	AUTOSAR	Autosar Specification of LIN Network Management AUTOSAR_SWS_LINNetworkManagement.pdf
[AR_NM_3]	AUTOSAR	Autosar Specification of CAN Network Management AUTOSAR_SWS_CANNetworkManagement.pdf
[AR_NM_4]	AUTOSAR	Autosar Specification of CAN State Manager AUTOSAR_SWS_CANStateManager.pdf
[AR_NM_5]	AUTOSAR	Autosar Specification of Flash EEPROM Emulation AUTOSAR_SWS_FlashEEPROMEmulation.pdf
[AR_NM_6]	LIN Consortium	LIN Specification Package Revision 1.3
[AR_NM_7]	LIN Consortium	LIN Specification Package Revision 2.1
[AR_NM_8]	Internal	Autosar Communication Configuration
[AR_NM_9]	Internal	Autosar - ECU Mode Management
[AR_NM_10]	Internal	UDS Session
[AR_NM_11]	Internal	UDS Data

•Introduction

4.3.1.2.2 Introduction

Software Component Requirements
In this document all requirements are numbered in order to accomplish traceability during the development process. Text written outside the requirement tags in this section shall be regarded as complementing information and not as a requirement.

Node types
There are four basic node types; A, B ,C and D, defined based on how the node is started up, electrical current consumption performance, and resulting startup timing performance. The purpose of the node types shall be to identify the requirements based on whether network wake up exist, hardware sensor signal wake up exist, if supply voltage is permanent or switched. The selection shall only be based on how the internal power supply, microcontroller and transceiver that handle the network communication is activated by external signals, internal signals and power supply. Power supply to other parts of the node shall not influence the type selection. Note that node types A-D are the most common combinations. The timing requirements are grouped based on whether the microcontroller is actually powered or unpowered in a sleep mode, which a the major factor regarding possible startup timing performance. Autosar Network Management is expected to be used for all node types unless otherwise stated.



Node types and internal power supply and wake up.

Node Type A, permanent power supply and network wake up
Nodes which have to be functionally available all time, even when the vehicle is parked and ignition-key is off. These types of nodes are permanently supplied from a permanent battery supply line. These applications allow switching off the entire supply system of the node including the microcontroller supply while keeping the wake-up capability by the communication network. Node sleep current typically below 100 µA, while keeping the capability to support wake-up by the communication network and to restart the application.

Node Type B, permanent power supply, network and hardware wake up
Nodes which have to be available all time, even when the vehicle is parked and ignition-key is off. These types of nodes are permanently supplied from a permanent battery supply line, and using a continuously active Vcc supply to a microcontroller. In order to reduce the node power consumption, the transceiver needs to be set into a mode with reduced supply current while the Vcc stays active. This type of node support wake-up by the communication network and by an input signal from a hardware sensor.

Node Type C, permanent power supply and hardware wake up
Nodes which have to be available all time, even when the vehicle is parked and ignition-key is off. These types of nodes are permanently supplied from a permanent battery supply line, additionally come with a microcontroller controlled transceiver voltage supply. In contrast to node Type B, electrical sleep current consumption can be further reduced, because the transceiver becomes completely un-powered by microcontroller control. It supports wake up only by an input signal from hardware sensor.

Node Type D, switched power supply
Nodes which do not need to be available with ignition-key off are simply switched off and become totally un-powered during ignition-key off. They are supplied from a switched battery supply line, for example “Clamp-15”. This supply line is only switched on with ignition-key on or by a relay. Ignition-sensing nodes that have a permanent power supply and determine when to start communication based on e.g. a clamp-15 signal is also considered to be type D. Optionally, ignition sensing nodes may communicate after ignition off (after-run). For these nodes network management is also required since network communication is not expected automatically when a node has power supply. There must be an explicit need for network communication for the node to communicate since the node may be power supplied for other reasons, e.g. standalone operation in some situations. This enables decoupling the node application operation from communication.

Detailed functional requirements

The Autosar network management BSW modules are partly configured by the parameters in *Autosar Communication Configuration* to assure common fundamental aspects e.g. communication timing between nodes, while remaining configuration is expected bye the supplier. To define additional requirements on behavior the following sections add requirements that are common for network types CAN and CANFD and also specific for each network type.

•General requirement

4.3.1.2.3 General requirement

663107 v1 **General requirement to match test case from AutoSAR CANNM**

Version history:
(1):

Purpose: To match the test case from test request
Source: Match the test case from Standard
Verification Method: Test
Legacy ID:

Reference number	Source	Title	Revision of document
1	AutoSAR	AUTOSAR_SWS_CANNetworkManagement	4.2.2

Note:
References are typically available from AutoSAR standard, it need to be added here to match some test case from Verification request.

663108 v1 **General requirement to match test case from AutoSAR COM**

Version history:
(1):

Purpose: To match the test case from test request
Source: Match the test case from Standard
Verification Method: Test
Legacy ID:

Reference number	Source	Title	Revision of document
1	AutoSAR	AUTOSAR_SWS_COMManager	4.2.2
2	AutoSAR	AUTOSAR_SWS_COM	4.2.2

Note:
References are typically available from AutoSAR standard, it need to be added here to match some test case from Verification request.

685689 v4 **The wakeup keep from diagnosis**

Version history:
(4): 1. 验证方法从“测试”变更为”分析和测试“。
2. 诊断维持唤醒需求从网络管理通用需求迁移至Autosar网络管理通用需求中；

Purpose: To define network management behaviors during the diagnostic process.
Source: AUTOSAR_SWS_COMManager [SWS_ComM_00866] and AUTOSAR_SWS_DiagnosticCommunicationManager 'Interface to ComManager'
Verification Method: Analysis AND Test
Legacy ID:

In the default session, a diagnostic request triggers a network request to maintain network communication, releasing the network request when the diagnostic response is complete.
In a non-default session, the network should always be maintained without concern for diagnostic requests and responses.
Switching from the default session to a non-default session triggers a network request to maintain network communication, and releases the network request after switching back to the default session.

在默认会话中，诊断请求将触发网络请求维持网络通信，在诊断响应完成后释放上述网络请求。
在非默认会话中，应一直维持网络，不关心诊断请求和响应。
从默认会话切换为非默认会话将触发网络请求维持通信，在切换回默认会话后释放上述网络请求。

4.3.1.3 AutoSar NM - CAN reqs

•Timing requirements

4.3.1.3.1 Timing requirements

Startup timing requirements vs. Autosar
The wakeup timing requirements described, e.g. TCANPowerWakeUpToApp, TCANEventWakeUpToApp, do not have a direct configuration parameter in Autosar. The wakeup requirements are aimed towards the complete node consisting of hardware and software, and the interaction between them. Therefore attention must be paid on the hardware design as well even if this specification is called *Autosar Network Management - additional requirements*, and Autosar actually is a software architecture.
Communication startup timing for nodes with microcontroller powered and unpowered when prepared for wakeup.
For nodes that have possibility to wakeup and start communication due to hardware sensor signals and network activity will have the same timing requirements for these two wakeup sources. Different use cases result in different need for communication startup time, and also requirements for sleep current consumption affect the design choice. Therefore two implementations of the node internal hardware exist which enable shorter or longer time from the initiating event until frame reception and transmission is started. The key aspect is whether the microcontroller is kept powered or unpowered in sleep. Therefore two wakeup timing requirements exists based on whether the microcontroller has power supply when the node is in a mode prepared for wakeup. In a given situation only one of the requirements is valid. These timing requirements do not concern transmitted application signals being updated with valid data.

663132 v1 **Definition of TCANEventWakeUpToApp**

Version history:
(1):

Purpose: To define maximum time to start communication.
Source:
Verification Method: Test
Legacy ID: 68204v2

TCANEventWakeUpToApp shall be equal to 50 ms.
If another requirement define this time to be shorter that requirement shall override this requirement.

663109 v2 **Definition of TCANPowerWakeUpToApp**

Version history:

(2): 启动唤醒时间从150ms/250ms变更为350ms；

Purpose: To define maximum time to start communication.
Source:
Verification Method: Test
Legacy ID: 52306v4

TCANPowerWakeUpToApp shall be equal to 350 ms.
Note: There might be other functional requirements on application level which specifies shorter wakeup times, if so they have to be fulfilled.

663110 v1 Definition of TCANResumeCom

Version history:
(1): 1

Purpose: To define maximum time to start communication if communication is started some time after node is started.
Source:
Verification Method: Test
Legacy ID: 339294v0

TCANResumeCom shall be equal to 25 ms.

Note: There might be other functional requirements on application level which specifies shorter wakeup times, if so they have to be fulfilled.

663111 v2 Timing at power up or reset initialization CAN

Version history:
(2): 取消A/B/C类型节点的上电启动定义，由于上述类型节点上电不唤醒。

Purpose: To ensure sufficient response time on frame level.
Source: For definition of node types see Node Types, Autosar Network Management (additional requirements) - Overview
Verification Method: Test
Legacy ID: 52307v3

A node shall be ready to receive and transmit CAN frames maximum TCANPowerWakeUpToApp after power is applied, a hardware reset and a diagnostic ECUReset(hardReset).

For nodes **Type A**, **Type B** and **Type C** the following case applies:
- Diagnostic ECUReset(hardReset).

For nodes **Type D** the following cases applies:
- Power is supplied by a relay.
- Ignition On or Clamp-15 goes active.

The following sequence applies:

Wakeup sequence for a node power on or reset initialization.

Note: This requirement is not valid if the poweron was created by the battery connection was disconnected and then re-applied.

663112 v2 **Timing at resumed communication CAN - relay powered/ignition sensing node**

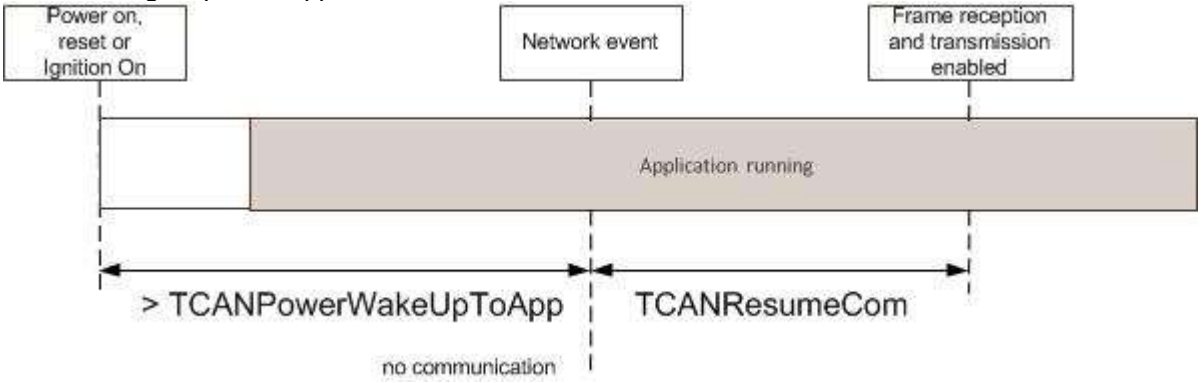
Version history:
(2): 增加A/B/C类型节点上电初始化完成后，触发唤醒条件唤醒的定义。

Purpose: To ensure sufficient response time on frame level.
Source: For definition of node types see Node Types, Autosar Network Management (additional requirements) - Overview
Verification Method: Test
Legacy ID: 339293v0

If more than TCANPowerWakeUpToApp has passed since power on, a hardware reset or a diagnostic ECUReset(hardReset), a node shall be ready to receive and transmit application signals maximum TCANResumeCom after communication is requested by another node. The requirement applies every time the node is requested to start communication and is currently not communicating. Communication shall not be enabled until all conditions are fulfilled.
For nodes **Type A**, **Type B** and **Type C** the following case applies:
- The permanent power supply is applied and communication stack is already initialized, communication is not currently active but becomes requested by another node.

For nodes Type D the following cases applies:
- Power is already supplied by a relay and communication stack is already initialized, communication is not currently active but becomes requested by another node.
- Ignition On or Clamp-15 is already active and communication stack is already initialized, communication is not currently active but becomes requested by another node.

The following sequence applies:



Sequence for activation of communication.

Note: For node Type D the software is expected to execute all time a relay or Clamp 15 supplies power. Since the communication stack is already initiated the node is expected to be able to start communication quicker than from

power on. Therefore the node is not considered woken up in this case, as the software is already executing. The external request to start communication occur when NM-PDUs are transmitted from another node. The requirement applies not just when communication resumes from previously being active but also if the first communication activation occure some time after the node is started.

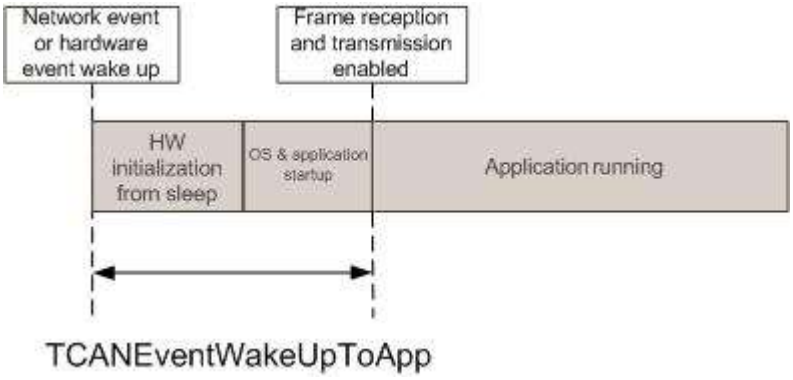
4.3.1.3.1.1 Microcontroller is powered in sleep

663133 v1 **Timing at network and hardware wake up CAN - microcontroller powered in sleep**

Version history:
(1):

Purpose: To ensure sufficient response time on frame level.
Source:
Verification Method: Test
Legacy ID: 59045v6

Network wakeup and hardware wakeup:
A node shall be ready to receive and transmit CAN frames maximum TCANEventWakeUpToApp after network activity or a hardware sensor signal change (e.g. a door switch is detecting a door being opened) is detected.
For node **Type B** the following case apply:
- Network activity is detected by a CAN transceiver.
For node types **Type B** and **Type C** the following case apply:
- Hardware sensor signal change is detected.
The following sequence applies:



Wakeup sequence for a node wake up by network activity.

Note: Some nodes may be equipped with "selective wake up transceivers". For such a node the requirement for network wakeup applies only when the CAN transceiver has been woken up by a PNC request that matches the partial networks it is part of. Nodes not equipped with such transceiver will always wake up on any network activity and therefore the requirement will always apply to those nodes.

4.3.1.3.1.2 Microcontroller is unpowered in sleep

663134 v2 **Timing at network and hardware wake up CAN - microcontroller unpowered in sleep**

Version history:
(2): 节点类型从A扩展至A/B/C，唤醒条件在网络激活的基础上增加硬线、本地激活；

Purpose: To ensure sufficient response time on frame level.

Source:
Verification Method: Test
Legacy ID: 58806v5

A node shall be ready to receive and transmit CAN frames maximum TCANPowerWakeUpToApp after network activity or a hardware sensor signal change (e.g. a door switch is detecting a door being opened) is detected.
For node type **Type A**, **Type B** and **Type C** the following case applies:
- Network activity or local activity is detected by CAN transceiver (SBC).

The following sequence applies:

Wakeup sequence for a node wake up by network activity or hardware sensor signal change.

Note: Some nodes may be equipped with "selective wake up transceivers". For such a node the requirement for network wakeup applies only when the CAN transceiver has been woken up by a PNC request that matches the partial networks it is part of. Nodes not equipped with such transceiver will always wake up on any network activity and therefore the requirement will always apply to those nodes.

4.3.1.4 AutoSar NM - CAN cross networks reqs

•Requirements

4.3.1.4.1 Requirements

General: These requirements apply when a node is connected to more than one CAN network. In this case it is possible to keep one, or only some of all networks active for the purpose of energy consumption reduction. The network from which a triggering condition to activate additional networks arise is called the source network, and the network that shall be woken up or change to be requested active by the node is called the destination network. The timing performance in the gateway node is described regarding maximum delay from an event on a source network until a destination network is ready for communication, or until a network request is transmitted as a NM-PDU in an already active network. The exact conditions that shall lead to an activation of a destination network is function and node specific and may be specified in node specific requirements.
Depending on the Autosar version, support for partial network clustering (PNC) is supported or not supported, which is a major difference regarding the mechanisms used for activating network communication. Therefore the following requirements are grouped in two groups based on this difference. For nodes that shall gateway application signals from a source network to a destination network, the gateway timing largely depend on if the destination network is already active or not. These requirements handle both cases, and there are also specific requirements depending on if the destination network is CAN.

Specific for Autosar 4.0.3 and later which support partial network clusters (PNC): The cluster request information, the PNC vector, placed in the NM User data part of NM PDUs are used as base for the requirements. If the source network is being woken up and a gateway also shall wake up a destination network, the timing chain shall be based on the timing requirements for the source network as a start. The maximum time where the gateway shall be able to transmit and receive frames on the source network shall be the starting point. From then on, a maximum delay TCANWakeAndGwPncRequest for CAN, is required until initiating a wakeup of the destination network.

4.3.1.4.1.1 Node supports partial network clustering (PNC)

663125 v1 **Definition of TCANGwPncRequest**

Version history:
(1):

Purpose: To define the maximum time for gatewaying the PNC request
Source:
Verification Method: Test
Legacy ID: 113172v1

TCANGwPncRequest shall be equal to 15ms.
If another requirement define this time to be shorter that requirement shall override this requirement.

663126 v1 **Definition of TCANWakeAndGwPncRequest**

Version history:
(1):

Purpose: To define the maximum time to start communication and gatewaying the PNC request
Source:
Verification Method: Test
Legacy ID: 113169v1

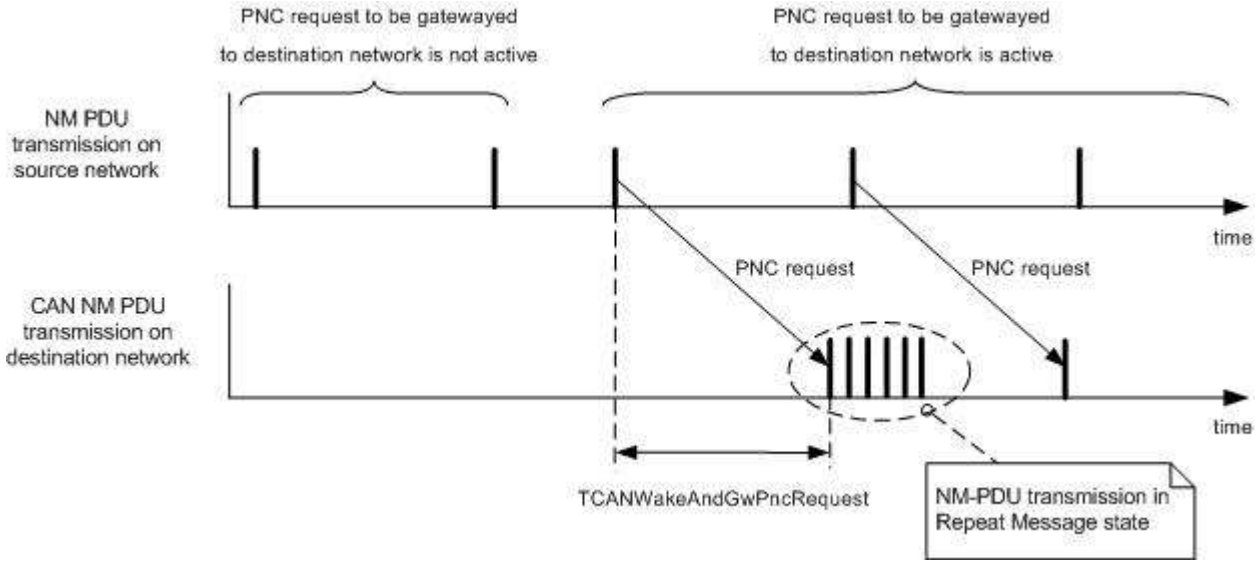
TCANWakeAndGwPncRequest shall be equal to 15 ms.
If another requirement define this time to be shorter that requirement shall override this requirement.

663129 v1 **Timing at destination network wake up and gatewaying PNC request to CAN**

Version history:
(1):

Purpose: The network to which the PNC request shall be gatewayed is not active. To ensure sufficiently short response time when waking up a network and activating a PNC.
Source:
Verification Method: Test
Legacy ID: 113122v2

If a PNC request becomes active on a source network and is to be gatewayed to a CAN destination network which is not active the node shall transmit the source PNC request in a NM-PDU on the CAN destination network and be ready to receive and transmit CAN frames maximum TCANWakeAndGwPncRequest after the source PNC request became active. The first NM-PDU transmitted on CAN destination network shall include the source PNC request that causes the destination network to become active.
The following sequence applies:



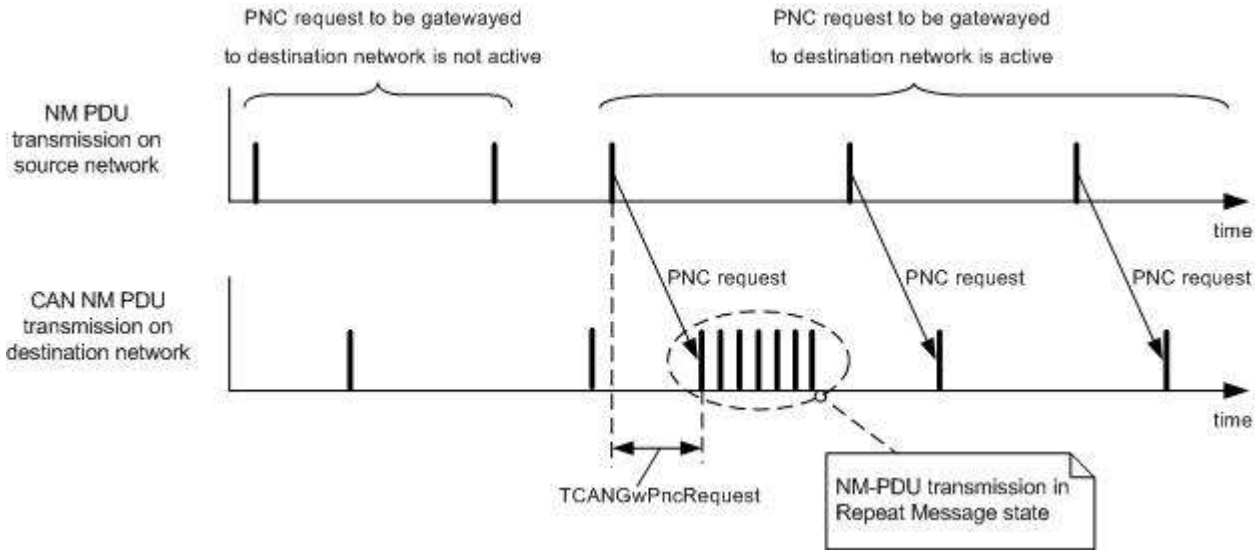
Wakeup sequence for a node that wake up a CAN network and gateway a PNC request to that network.
Note: Source network may be CAN or CANFD and destination network is a CAN network. The timing diagram is simplified; NM-PDU period of source and destination network may be different; especially if the source network is not CAN. Also, the source PNC request may reside in a NM-PDU from any node on the source network. The PNC request may be used for selective waking of some or all of the nodes equipped with "selective wake up transceivers". Nodes not equipped with such transceiver will always wake up on any network activity.

663131 v1 **Timing at gatewaying PNC request to CAN**

Version history:
(1):

Purpose: The network to which the PNC request shall be gatewayed is already active, due to at least one other PNC request. To ensure sufficiently short response time when activating a PNC.
Source:
Verification Method: Test
Legacy ID: 113146v2

If a PNC request becomes active on a source network and is to be gatewayed to a CAN destination network on which the gateway is already active the node shall transmit the source PNC request on the CAN destination network in a NM-PDU maximum TCANGwPncRequest after the source PNC request became active.
The following sequence applies:



Sequence for a node that gateway and activates a PNC on a CAN network.

Note: Source network may be CAN or CANFD and destination network is a CAN network. The timing diagram is simplified; NM-PDU period of source and destination network may be different especially if the source network is not CAN. Also, the source PNC request may reside in a NM-PDU from any node on the source network. The source PNC request shall be gatewayed with a maximum delay of TCANGwPncRequest and next CAN NM-PDU must be enforced to be transmitted as soon as possible via a dedicated call to CANNM. The PNC request on the destination network may also be used for selective waking of some or all nodes if they are equipped with "selective wake up transceivers". Nodes not equipped with such transceiver are assumed to be awake in this use case. A PNC request means a bit in NM User Data.

4.3.1.5 Ethernet TC10

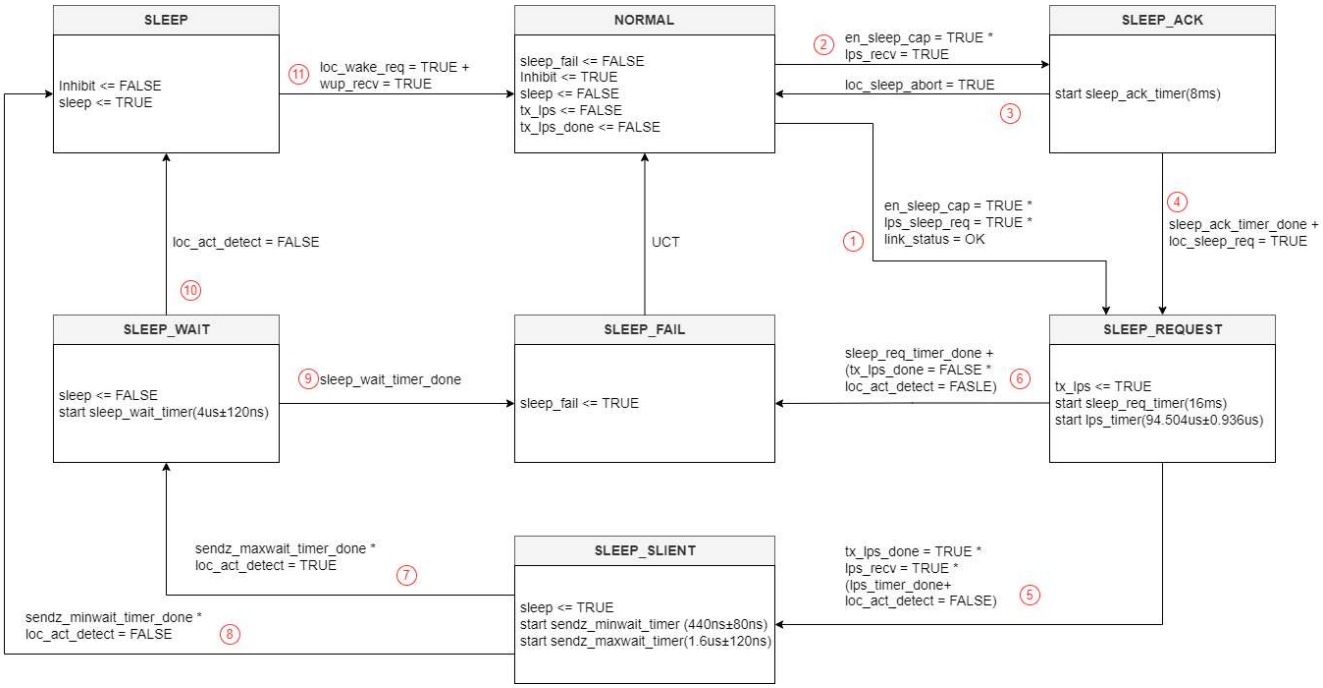
•Introduction

4.3.1.5.1 Introduction

4.3.1.5.1.1 PHY Power Mode Definition

- **State diagram of the PHY power mode :**

State diagram of the PHY power mode, which implements the two-way handshake protocol.



Note: In the flow chart,+ is the logical relationship of OR and * is the logical relationship of AND.

- Definition of power mode

PHY Power Mode	Description
NORMAL	Normal data transmission status
SLEEP_ACK	Sleep confirmation state, which PHY enters when it receives Low Power Sleep(LPS) commands
SLEEP_REQUEST	Sleep request status, PHY requests sleep itself or PHY enters after receiving the Low Power Sleep(LPS) commands for a certain time
SLEEP_SILENT	Sleep silent state, which is entered after the LPS command is sent and received. In this state, the energy detection circuit remains disabled to prevent spurious wake-up glitches and mutual wake-ups through LPS commands
SLEEP_WAIT	Sleep wait state, in the SLEEP_SLIENT state, PHY will jump to this state if the sendz_maxwait_timer expired and receives continuous non-zero symbols
SLEEP_FAIL	Sleep failure status, sleep is terminated, PHY will return to NORMAL status
SLEEP	In the sleep state, if both PHYs are in the silent state before the Timer timeout, in this state, some PHY functions are powered off, waiting for wake-up pulse or local wake-up

- Variable parameters

Parameter	Description
loc_act_detect	Local activity detection signal. The variable is set to FALSE if consecutive symbols of zeros were received, otherwise set to TRUE. The value of loc_act_detect shall be set to TRUE (FALSE) within 1 us
lps_recv	Set if a LPS command was entirely received
wur_recv	Set if a WUR command was entirely received
wup_recv	Set if WUP pulses were sensed
tx_lps	If set, LPS bits are transmitted
tx_lps_done	Set after entire OAM frame (12 OAM symbol) containing the LPS command has been transmitted

lps_timer	Timer used to ensure reliable transmission and decoding of LPS command. The timer shall expire 94.504us +/- 0.936us (approximately +/- 1%)
lps_timer_done	Set after lps_timer has expired
loc_sleep_req	Set if a sleep is requested by the local PHY
loc_wake_req	Set if a wakeup is requested by the local PHY
sleep_req_timer_done	Set when the sleep_req_timer has expired
sleep_ack_timer_done	Set when the sleep_ack_timer has expired.
loc_sleep_abort	Set if a remote sleep request is to be rejected while still in SLEEP_ACK phase
sleep_fail	Set if a sleep handshake was aborted by the link partner
inhibit	Set if the (external) power supply shutdown is inhibited
sleep	Set by the power state machine to notify PHY CTRL to disable transmission
en_sleep_cap	Indicates whether sleep capability is enabled
sleep_wait_timer_done	Set when sleep_wait_timer has expired

- Timing parameters

Timer	Description
wup_timer	A timer used to wait for reliable detection of WUP pulse. The timer shall expire 1ms +/- 0.3ms after being started
sleep_ack_timer	A timer used in SLEEP_ACK state to check whether NM decides to reject sleep flow on an incoming data message or not. The timer shall expires 8ms after being started
sleep_req_timer	A timer set up in SLEEP_REQ to check if the handshaking is properly done by both PHYs. If the PHY doesn't enter SLEEP state before timeout, it enters SLEEP_FAIL state and back to NORMAL. The timer shall expires 16ms after being started
sendz_minwait_timer	A timer to guarantee a minimum time SEND_Z is transmitted. The timer shall expire after 440 ns (+/- 80 ns)
sendz_maxwait_timer	A timer to limit the maximum number of SEND_Z transmissions to not trip the link partner's PMA watchdog. This timer shall expire after 1.6 us (+/- 120ns).
sleep_wait_timer	A timer to limit the time to stay in SLEEP_WAIT state before going to SLEEP_FAIL state, if local_act_detect is always true during SLEEP_WAIT state. The timer shall expire after 4 us (+/- 120 ns).

- PHY Power Mode transition relationship

Assume that at the beginning, both PHY A and PHY B on the link are in NORMAL mode. The meanings of the labels in the figure are as follows.

Label	Description
①	PHY A in NORMAL mode requests sleep and enters SLEEP_REQUEST mode, sends LPS to PHY B, starts sleep_req_timer and lps_timer.
②	PHY B in NORMAL mode receives LPS command from PHY A, enters SLEEP_ACK mode, and starts sleep_ack_timer.
③	If PHY B does not meet the sleep condition (for example, there is still data need to send), PHY B will return to NORMAL mode.
④	If PHY B does not refuse sleep (i.e. there is no data to be sent), it will also enter SLEEP_ REQUEST mode and send LPS command.
⑤	Due to receiving LPS first and sending LPS command, PHY B will directly enter SLEEP_SILENT mode; After receiving the LPS command sent by PHY B, PHY A will also enter SLEEP_ SILENT mode.
⑥	If PHY A does not receive LPS from PHY B when the timer sleep_req_timer times out, it will enter SLEEP_FAIL mode and then returns to NORMAL mode.
⑦	In SLEEP_SILENT mode, if A non-zero symbol (loc_act_detect = True) is received before the sleep_req_timer expires, PHY A/B enters SLEEP_FAIL mode and then returns to NORMAL mode.

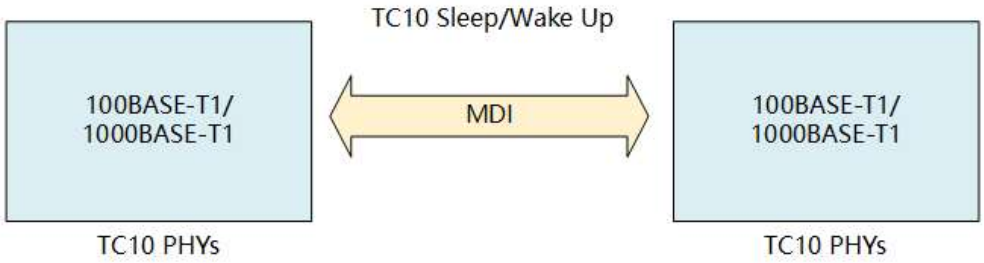
⑧	In SLEEP_SILENT mode, PHY A/B enters SLEEP mode if a continuous zero symbol (loc_act_detect = False) is received when the sleep_req_timer times out.
⑨	PHY A/B in SLEEP mode, upon receiving local or remote wake-up signals/commands, enters NORMAL mode and begins normal communication

4.3.1.5.1.2 TC10 Wake-Up and Sleep Configuration

TC10 focus on establishing support for sleep mode and a wake-up mechanism tailored for automotive use cases and defines not only how two devices go to sleep and remote wake up but also forwarding remote wake signal in intranet. This means switches can operate in a low power sleep mode with minimal standby current from the battery voltage with the main supply shut off. During this mode, the device can continuously monitor for an MDI wake up pulse or a pulse presented at the local WAKE pin and respond to this wake-up command as specified in OPEN Alliance Sleep/Wake Specification. The hardware obeys configurations of user to accept or reject sleep and wake event and notify upper layer if needed.

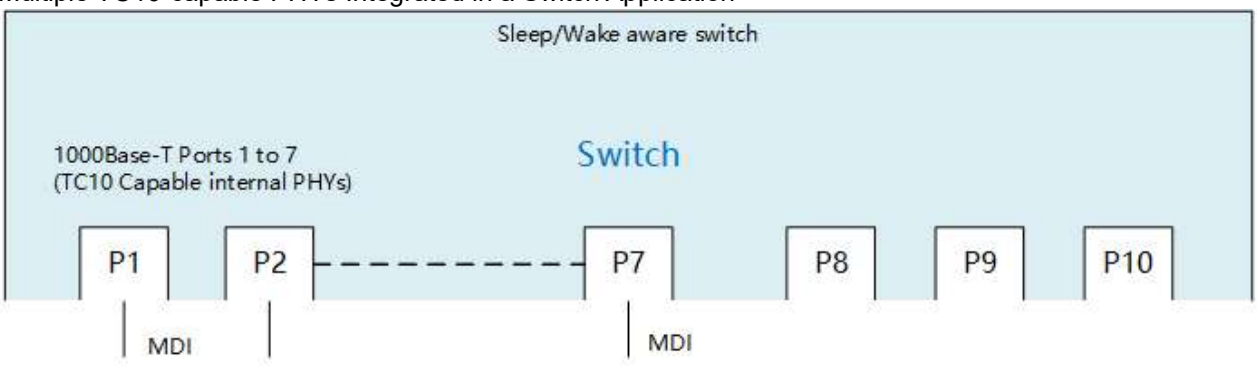
There exist different configuration possibilities depending on where TC10 capable PHY is placed as listed below.

- TC10 Standalone PHYs(connected to other standalone PHYs)



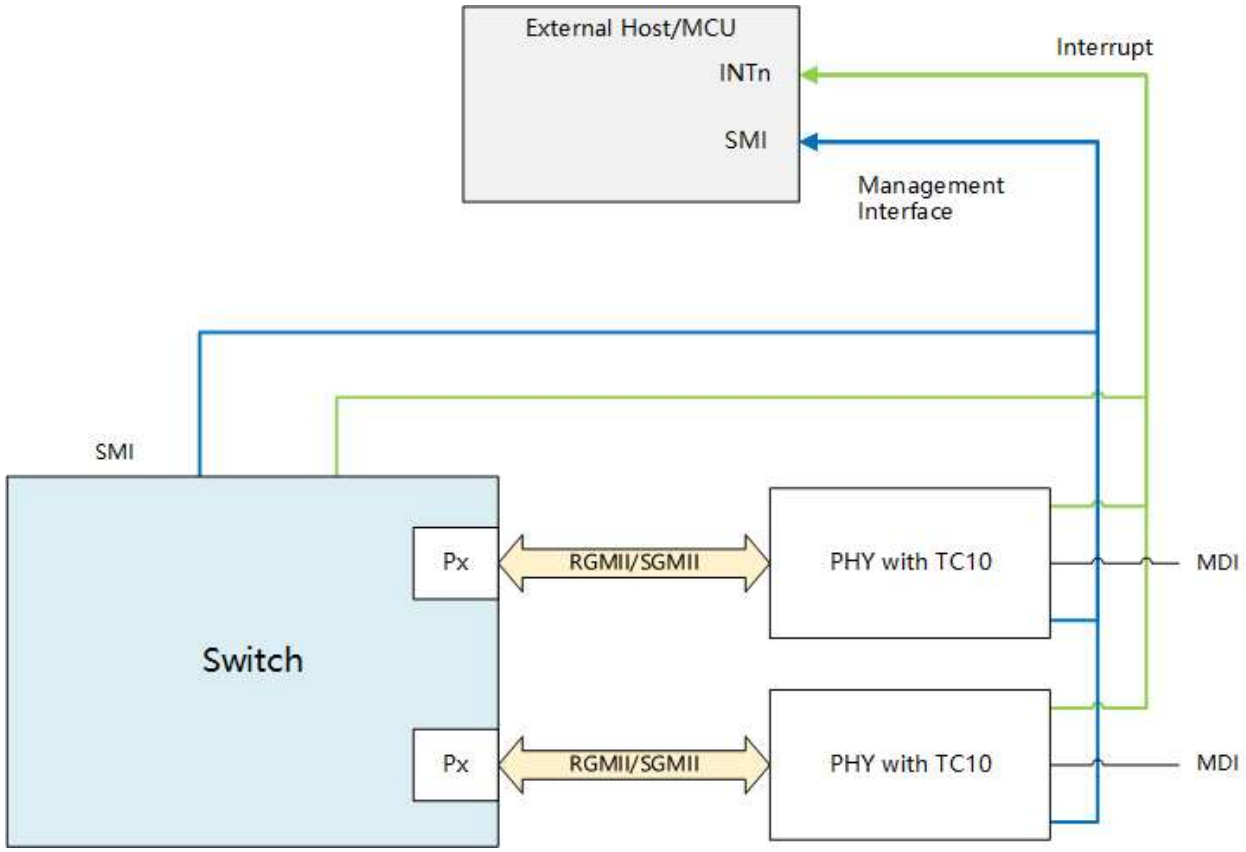
In this type of configuration, the two devices accomplish sleep/wakeup using MDI. Either of the device can initiate the sleep wake register by writing into register. The status of the register shall be available to higher layer.

- Multiple TC10-capable PHYs Integrated in a Switch Application



In the case of a switch with a TC10-capable internal application, although a port and its link partner may agree to enter TC10 deep sleep state (where power to the device is totally removed), the switch and remaining internal PHY ports may not be ready to enter the deep sleep state. So, the PHY port will enter a low power state while the remaining switch and others stay active. When all internal ports have agreed to enter the TC10 Sleep state with their link partner, the switch can then decide to enter the Deep Sleep state taking away power to the Switch.

- External TC10-capable PHY with MAC(switch)



For extended applications of PHYs connected with a switch, the switch level awareness of the Sleep/Wake events are required so that the switch can determine if the switch is ready to allow itself to enter sleep.

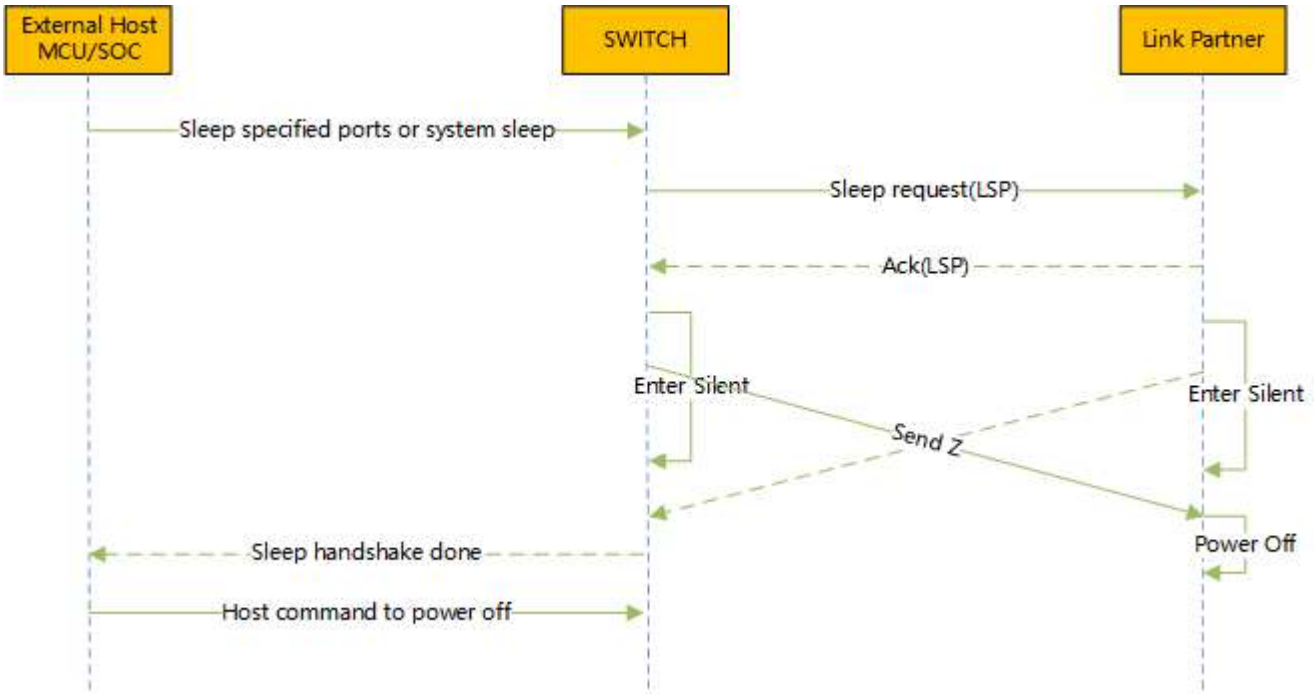
The external host can be used to manage the switch and forwarded sleep/wake-up request. This can be achieved with help of an external host as shown below:

When an external PHY agrees to enter the TC10 Sleep/Wake mode, either by receiving TC10 sleep/wake-up request from the link partner or sleep/wake-up request by the host, interrupt can be generated on one of PHY pins. All interrupts are grouped and connected to the external host. Upon receiving interrupt external host can poll the sleep/wake-up status of all externally connected PHY and based on software-defined sleep/wake forwarding table it can forward sleep/wake-up requests to other PHY in the system.

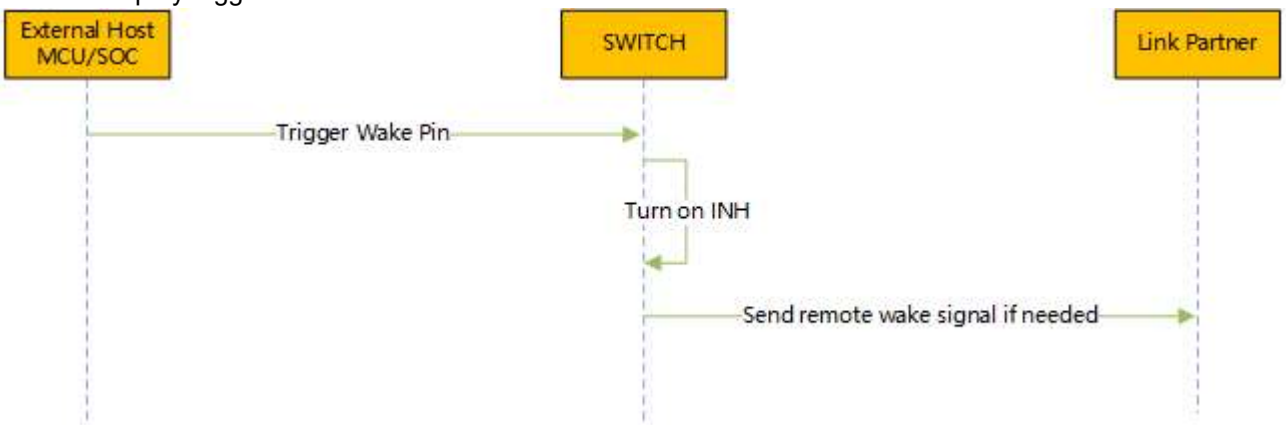
4.3.1.5.1.3 Cases Of TC10 Wake-up and Sleep

Take a Switch connected to other standalone PHYs as an example:

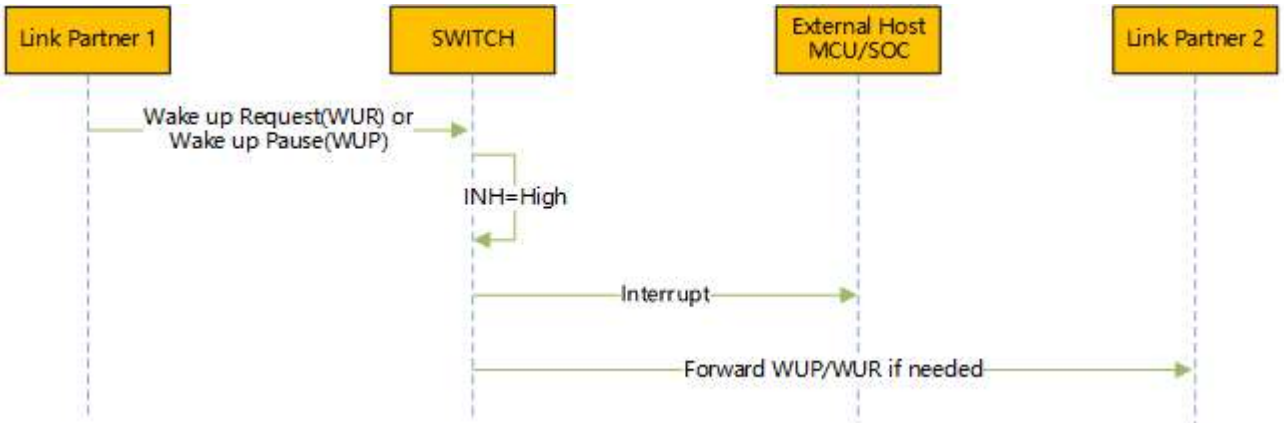
- Sleep handshake



- Wake up by trigged Wake Pin



- Wake up by Remote wake flow



•OpenAlliance - TC10

4.3.1.5.2 OpenAlliance - TC10

871714 v1 **Sleep Mode**

Version history:
(1):

Purpose: To specific the sleep mode for TC10 SWITCH
Source:
Verification Method: Analysis AND Test
Legacy ID:

For TC10 SWITCH, there must be two sleep modes:
a).TC10 single port sleep mode, where the device is still has power but in low power state, only some of the TC10 PHY port will enter a low power state similar to IEEE power down while the remaining SWITCH and others stay active.
b).Deep sleep mode, where the device has supply power completely removed.

871715 v1 **Sleep and Wake-up Forwarding**

Version history:
(1):

Purpose: To specific the Wakeup/Sleep forwarding feature
Source:
Verification Method: Test
Legacy ID:

The PHY/switch shall be support selective Wake-Up/Sleep request forwarding mechanism. if SWITCH detects a wake-up request in the form of WUP/WUR in the PHY of a port, then it must be able to forward the wake-up request to a specific one or more other PHYs, and the same goes for Sleep request.

Note: Wake-up and Sleep forwarding table shall be define by OEM.

871711 v1 **TC10 Capability**

Version history:
(1):

Purpose: To refer to the TC10 standard that should be the basis for implementations through out the vehicle.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The features of TC10 PHY and SWITCH: Wake-up/Sleep Interface Behavior, Timing Behavior, Service Primitives and Interfaces, Modification of the Physical Coding Sublayer of IEEE802.3bw/IEEE802.3bp, etc., must fully comply with *[OA_1 : OA TC10 Sleep/Wake-up Specification V2.1]* and *[OA_11 : OA TC10 802.3bp Sleep/Wake-up Specification V1.0]*

871712 v1 **TC10 Sleep/Wake-up Enable**

Version history:
(1):

Purpose: Description the method to enable TC10 Sleep/Wake-up feature
Source:
Verification Method: Analysis AND Test
Legacy ID:

The TC10 Sleep/Wake-up feature for the device shall be enabled via the configuration pin in the standalone PHYs. Depending on the type of device and functionality, a dedicated I/O pins shall be defined by PHY vendor.

871716 v1 **WUP/WUR and LPS Command Transmitting Behavior**

Version history:
(1):

Purpose: To ensure higher quality implementation of Partial Networking
Source:
Verification Method: Test
Legacy ID:

WUP, WUR, LPS commands can only be sent by the host MCU/SOC via API Settings or register Writings.

871713 v1 **Wake-Up Pattern**

Version history:
(1):

Purpose: Define the wake-up pattern for TC10 PHY and SWITCH
Source:
Verification Method: Analysis AND Test
Legacy ID:

TC10 PHY and SWITCH shall be woken up by a local wake-up I/O pin, a wake-up request command or a wake-up pulse from link partner.

4.3.1.6 UDP Network Management

•Requirements

4.3.1.6.1 Requirements

871741 v1 **NM PDU Format**

Version history:
(1):

Purpose: To define the NM PDU Format.
Source:
Verification Method: Test
Legacy ID:

The length of the NM PDU is 8 Bytes, and the format of the NM PDU is shown in the following figure:

	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
Byte0	Source Node Identifier							
Byte1	Control bit vector							
Byte2	PN bit vector							

Byte3	PN bit vector
Byte4	PN bit vector
Byte5	PN bit vector
Byte6	PN bit vector
Byte7	PN bit vector

Byte0 is configured as Source Node Identifier, Byte1 is configured as CBV, Byte2 to Byte7 are configured as PN bits

871742 v2 NM Parameter Configuration

Version history:
(2): Update the verificaton method from 'A&T' to 'T'

Purpose: To define the parameters required for network management.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Configuration Parameter	Value
UdpNmActiveWakeupBitEnabled	TRUE
UdpNmAllNmMessagesKeepAwake	FALSE
UdpNmBusSynchronizationEnabled	FALSE
UdpNmCarWakeUpRxEnabled	FALSE
UdpNmNodeDetectionEnabled	TRUE
UdpNmNodeIdEnabled	TRUE
UdpNmPassiveModeEnabled	FALSE
UdpNmPduCbvPosition	1
UdpNmPduNidPosition	0
UdpNmComUserDataSupport	TRUE
UdpNmImmediateNmCycleTime	0.02
UdpNmImmediateNmTransmissions	20
UdpNmImmediateRestartEnabled	FALSE
UdpNmMainFunctionPeriod	0.005
UdpNmMsgCycleOffset	0
UdpNmMsgCycleTime	1.0
UdpNmRepeatMessageTime	1.0
UdpNmRepeatMsgIndEnabled	FALSE
UdpNmRetryFirstMessageRequest	TRUE
UdpNmPnEnabled	TRUE
UdpNmPnEiraCalcEnabled	TRUE
UdpNmPnEraCalcEnabled	TRUE For PNC Gateway FALSE For Other
UdpNmPnHandleMultipleNetworkRequests	TRUE
UdpNmPnInfoLength	6
UdpNmPnInfoOffset	2
UdpNmPnResetTime	2.95
UdpNmTimeoutTime	3
UdpNmWaitBusSleepTime	4.0

871740 v1 Software Design Rules

Version history:
(1):

Purpose: To make sure that the software design rules must comply with regulations defined by relevant AUTOSAR specifications.

Source:

Verification Method: Analysis

Legacy ID:

Software implementations of ECUs shall follow the Autosar standard based on CP or AP platforms as described in *[AUTOSAR_9 : Specification of Network Management Adaptive Platform]* and *[AUTOSAR_10 : Specification of UDP Network Management Classic Platform]* and *[AUTOSAR_11 : Specification of the AUTOSAR Network Management Protocol Foundation].*

4.4 Communication_LIN

4.4.1 Communication_LIN_Requirement_Specification

4.4.1.1 General

•Definitions and Abbreviations

4.4.1.1.1 Definitions and Abbreviations

Abbreviation	Description
DPR	Design Prerequisites
ECU	Electronic Control Unit with a computer/microcontroller.
EMC	ElectroMagnetic Compatibility
IC	Integrated Circuit
ISO	International Organization for Standardization
LIN	Local Interconnect Network
SAE	Society of Automotive Engineers

For multiplex terminology, SAE J1213/1 JUN91 "Glossary of Vehicle Networks for Multiplexing and Data Communications" shall be used in applicable cases for documentation.

Definition	Description
Bus	The bus represents the physical wiring topology of an part of the communication network. A network may consist of several busses, electrically isolated from each other.
Dominant/recessive	The bus can have one of two complementary logical values: “dominant” or “recessive”. See *LIN Specification Package Revision 2.1,* LIN Physical Layer - Requisite documents, for the definition of dominant and recessive. During simultaneous transmission of “dominant” and “recessive” bits, the resulting bus value will be “dominant”.
Network	A set of electronic and cabling devices facilitating the multidirectional exchange of information encoded in serial form. Each electronic device (in this case: ECU) is equipped with a specific, standardized electronic interface in order to guarantee compatibility between exchanged binary items of information.
Normal mode	LIN node communication during operational voltages ranges and nonfaulted conditions.
Passive mode	the LIN node does not transmit (leaves bus output recessive), but the node does still receive

•Requisite documents

4.4.1.1.2 Requisite documents

These are requirements only if explicitly referred to as that.

Reference	Document
External Documents:	
[LIN_DL_1]	LIN Specification Revision 1.3
[LIN_DL_2]	LIN Specification Package Revision 2.1
[LIN_DL_3]	LIN 1.3 Conformance test
[LIN_DL_4]	LIN 2.1 Conformance test
[LIN_DL_5]	ISO 7498-1 Open Systems Interconnection - basic reference model
Internal documents:	
	Components for Physical Interfaces
	Design Verification Method for LIN
	EMC Component and Subsystem Requirements - project specific EMC specification as stated by the general component requirements.

•Requirements

4.4.1.1.3 Requirements

660811 v1 **LIN Master**

Version history:
(1):

Purpose: To ensure full compatibility with the network design tool (VNA).
Source: LIN Standard
Verification Method: Analysis
Legacy ID: 76526v1

All LIN Master nodes shall follow the LIN Specification Package version 2.1 or newer so they can be moved between LIN networks. Only frames specified by the LDF-file shall be implemented. No additional frames are allowed.

Note: This requirement is valid for all LIN Master ECU's.
LIN Master ECUs must also support LIN Slave ECUs that uses LIN standard 1.3, to be able to handle carry-over ECUs.
LIN Datalink Layer, for reference see Diagnostics & ECU Platform - Requisite documents, contains both LIN Specification Package 1.3 and LIN Specification Package 2.x.

660812 v1 **LIN Slave**

Version history:
(1):

Purpose: To ensure full compatibility with the network design tool (VSA).
Source: LIN Standard
Verification Method: Analysis
Legacy ID: 76528v1

All new LIN Slave node shall follow the LIN Specification Package 2.1 or higher so they can be moved between LIN networks. Only frames specified by the LDF-file shall be implemented. No additional frames are allowed.

Note: This requirement is valid for all LIN slaves.
LIN Datalink Layer, for reference see Diagnostics & ECU Platform - Requisite documents, contains both LIN Specification Package 1.3 and LIN Specification Package 2.x. LIN Frame_size/DLC shall be free length and NOT fixed in identifier.

660813 v1 **NAD addresses (Network ID) for private LIN slaves**

Version history:
(1):

Purpose: To prevent double allocations of the same ECU address for private LIN ECUs.
Source: LIN standard
Verification Method: Analysis
Legacy ID: 81843v3

Non programmable or diagnosable LIN slaves shall have a NAD address with a different Network ID address compared to programmable and diagnosable LIN slaves on the same LIN network.

4.4.1.2 LIN Physical Layer -Generic requirements

•LIN Physical Layer - Generic requirements

4.4.1.2.1 LIN Physical Layer - Generic requirements

663294 v2 **Node conformance test**

Version history:
(2): 验证方法从Analysis改为Analysis AND Test

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID: 53764v1

The node must demonstrate conformance to the applicable tests defined in one of the comformance tests in table below. Which one of the comformance tests is decided by the the LIN protocol the node has implemented.

Protocol	Comformace test
LIN 1.3	*LIN 1.3 Conformance test* , LIN Physical Layer - Requisite documents
LIN 2.1	*LIN 2.1 Conformance test* , LIN Physical Layer - Requisite documents

663292 v1 **Requirement Exception Process**

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 53725v1

aLIN ECUs that do not meet all the requirements of this document shall be evaluated and deviations approved on a case by case basis.

663293 v1 **The priority between standard documents and this specification**

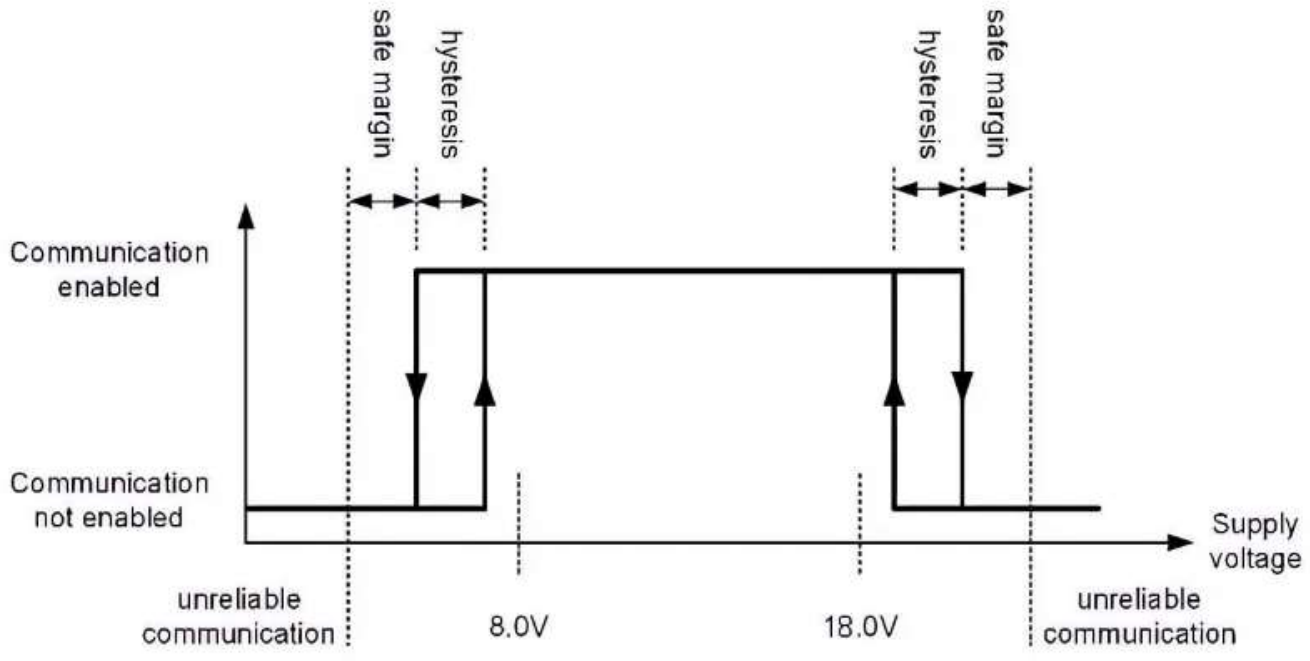
Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 53804v1

In case of discrepancy between the standard documents and this specification, this specification has precedence.

4.4.1.2.1.1 Operating Battery Power Voltage Range

The power supply voltage level at the node shall be measured continuously. HW and SW inaccuracy in the monitoring of the supply voltage level must be taken into account when the ECU is designed. The ECU must meet this operating voltage range requirement also in a worst case condition.
Network communication is permitted outside the specified voltage range, provided that the ECU can correctly read and write messages and not cause any disturbance on the network. There must be margins in both ends of the operating voltage range in regards to the voltage level where communication becomes unreliable or is no longer possible due hardware constraints. See figure below.



663296 v2 **Normal battery voltage power operation**

Version history:
(2): Change the voltage range to 8-18V

Purpose:
Source:
Verification Method: Test
Legacy ID: 53743v1

The node shall communicate on the network when the node voltage supply is 8,0 – 18,0 Volt. Network

communication is permitted outside the specified voltage range, provided that the ECU can correctly read and write messages and not cause any disturbance on the network.

663297 v1 **Transceiver mode uniformity**

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 53760v1

The transceiver shall not transmit a dominant state on the bus when the TxD is in a recessive state.

4.4.1.2.1.2 Oscillator Tolerance Definition

663298 v1 **Speed tolerance**

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 53728v1

The total network transmission speed tolerance shall include all causes . For crystals and resonators this shall include manufacturing tolerance, ageing and temperature. For PLL's, the tolerance shall be determined as specified by the semiconductor manufacturer, and shall include the effects of manufacturing, ageing, temperature and transmission speed.
Note: particular care must be taken to ensure the appropriate tolerance is met when a resonator is being used with a PLL.

4.4.1.2.1.3 General Design Requirements

663299 v1 **Circuit board tracks**

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 53774v1

The LIN circuits tracks between connector and transceiver should be as short as possible.

663300 v1 **Ground plane**

Version history:
(1):

Purpose:

Source:
Verification Method: Analysis
Legacy ID: 53772v1

A ground plane is required under the transceiver chip on the same layer as the transceiver.

663301 v1 **Grounding of the LIN transceiver**

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 53771v1

All grounding of the LIN transceiver and the filter capacitors shall be made to ECU signal ground.

663302 v1 **Location of the decoupling capacitor**

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 53776v1

The decoupling capacitor C2 shall be placed as close as possible to the transceiver.

663303 v1 **Location of the tranceiver**

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 53773v1

The transceiver should be located as close to the connector as possible. Other IC’s are not permitted between connector and the transceiver.

4.4.1.3 LIN Physical Layer - HW Master requirements

•Interface Implementation and Physical Layer Circuit

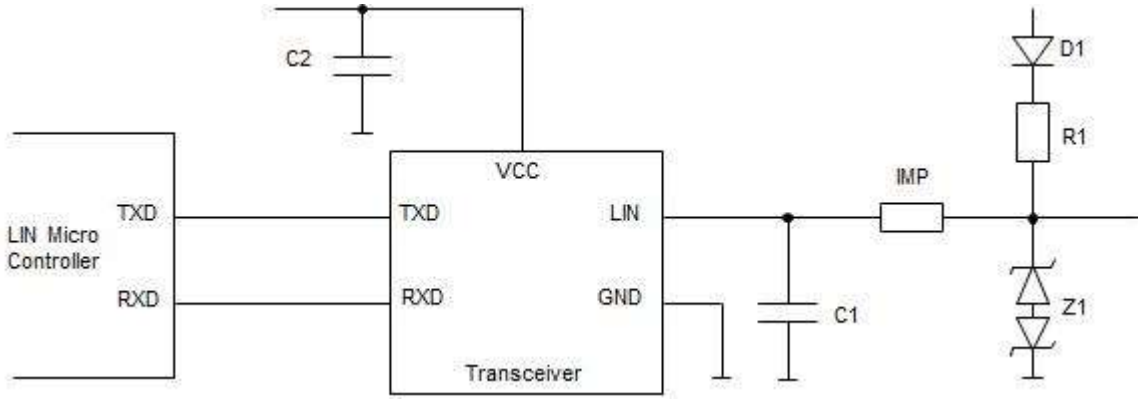
4.4.1.3.1 Interface Implementation and Physical Layer Circuit

663304 v1 **Interface implementation and physical layer circuit master**

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 53784v4

The circuits defined in the figure below shall be used for all designs.



Component ID	Description	Value
C1	Monolithic ceramic capacitor	680 pF ±10% ³ 100V rating
C2	Monolithic ceramic capacitor	³ 90 nF ³ 40V rating
D1	Diode [1]	³ 100V ³ 100mA
R1	Resistor	1 kOhm
IMP	Impedance	see Note 2
Z1	Dual Monolithic Avalanche Diode for ESD and over voltage protection.	27 V breakdown voltage

1) If the pull-up resistor R1 is permanently connected to battery voltage ($V_{batt\ ECU}$) the diode may be omitted.

2) The impedance Imp shall be selected such that:
- the impedance is ≤30Ω at ≤30kHz and ≥10Ω at ≤100kHz.
and
- EMC Component / Subsystem Requirements will be met, in particular the ESD requirements.

4.4.1.4 LIN Physical Layer - HW Slave requirements

•Interface Implementation and Physical Layer Circuit

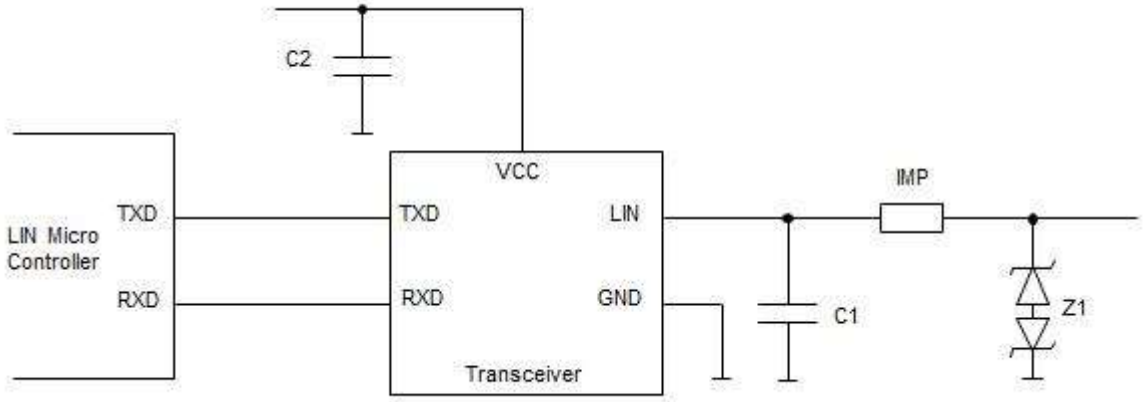
4.4.1.4.1 Interface Implementation and Physical Layer Circuit

663305 v1	Interface implementation and physical layer circuit slave

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 53787v4

The circuit defined in the figure below shall be used for all designs.



Component ID	Description	Value
C1	Monolithic ceramic capacitor	220 pF±10% 100V rating
C2	Monolithic ceramic capacitor	³ 90 nF ³ 40V rating
IMP	Impedance	see Note 1
Z1	Dual Monolithic Avalanche Diode for ESD and over voltage protection.	27 V breakdown voltage

1) The impedance Imp shall be selected such that:
- the impedance is ≤30Ω at ≤30kHz and ≥10Ω at ≤100kHz.
and
- EMC Component / Subsystem Requirements will be met, in particular the ESD requirements.

4.4.1.5 LIN Data Link Layer - Generic Requirements

•LIN Data Link Layer - Generic Requirements

4.4.1.5.1 LIN Data Link Layer - Generic Requirements

660916 v1 **Application timing requirements**

Version history:
(1):

Purpose:
Source:

Verification Method: Analysis
Legacy ID: 53542v1

The node application must be designed according to functional timing requirements, not according to the timing properties of a specific LIN bus implementation (i.e. a specific .ldf-file or similar).

660915 v1 Bus speed

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 53415v1

All modules shall be capable of operating at the nominal bus speeds of 9.6kbps or 19.2kbps. The operating speed shall be defined on a bus by bus basis.

660918 v1 Format of the part number data record

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 53564v3

When a part number for a LIN slave is to be transmitted from the slave to the master node, the format of the data shall be as follows:

Two part number formats shall be supported 8 digit and 10 digit format.
8 digit part number format shall have 8 digits for part number + 3 characters (version suffix).
10 digit part number format shall have 10 digits for part number + 3 characters (version suffix).

The part number digits shall be coded in BCD and the 3 characters (version suffix) shall be coded in ASCII, right justified, with any unused digit filled with 0H. In total 7 bytes(8 digit format) or in total 8 bytes (10 digit format).
If version suffix contains one character, there shall be two space between part number and version suffix.
If version suffix contains two characters, there shall be one space between part number and version suffix.

Example:
1234567<space><space>A -> (hex) 01 23 45 67 20 20 41
12345678<space><space>A -> (hex) 12 34 56 78 20 20 41

1234567<space>AB -> (hex) 01 23 45 67 20 41 42
12345678<space>AB -> (hex) 12 34 56 78 20 41 42

1234567ABC -> (hex) 01 23 45 67 41 42 43
12345678ABC -> (hex) 12 34 56 78 41 42 43

660919 v1 Format of the serial number

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 53565v1

When a serial number for a LIN slave is to be transmitted from the slave to the master node, the format of the data shall be as follows (see *[LIN_DL_2] LIN Specification Package Revision 2.1*):

8 digits for serial number. The Serial Number shall be coded in Hex, right justified with any unused digit filled with 0H in total 4 bytes.

Example:
12345678 -> (hex) 00 BC 61 4E
123456 -> (hex) 00 01 E2 40

660917 v2 LIN signal frames may appear arbitrarily

Version history:
(2): 验证方法从Analysis改为Test

Purpose:
Source:
Verification Method: Test
Legacy ID: 53562v1

The node must not assume that headers in the range 0x00 to 0x3B appear in a specific relative order.

660914 v1 Node conformance test

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID: 53536v1

The node must demonstrate conformance to the applicable tests defined in one of the comformance tests in table below. Which one of the comformance tests is decided by the LIN protocol the node has implemented.

Protocol	Comformace test
LIN 1.3	*[LIN_DL_3] LIN 1.3 Conformance test*
LIN 2.1	*[LIN_DL_4] LIN 2.1 Conformance test*

660920 v1 PLL used as clock

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 53413v1

When a PLL is being used to clock the LIN interface, transmission and reception are only allowed when the PLL is

locked.

660913 v1 **The priority between standard documents and this specification**

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 53411v1

In case of discrepancy between the standard documents and this specification, this specification has precedence.

4.4.1.5.1.1 Wake-Up timing

660921 v3 **Wakeup strategy**

Version history:
(3):

Purpose: 明确LIN主、从节点唤醒过程行为
Source: 《LIN Specification Package Revision 2.1》章节2.6
Verification Method: Test
Legacy ID: 53421v1

All nodes shall follow the strategy specified in *[LIN_DL_2] LIN Specification Package Revision 2.1*.
Note：LIN slaves designed according to LIN 1.3 Standard may follow the wake up strategy specified in *[LIN_DL_1] LIN Specification Revision 1.3*.

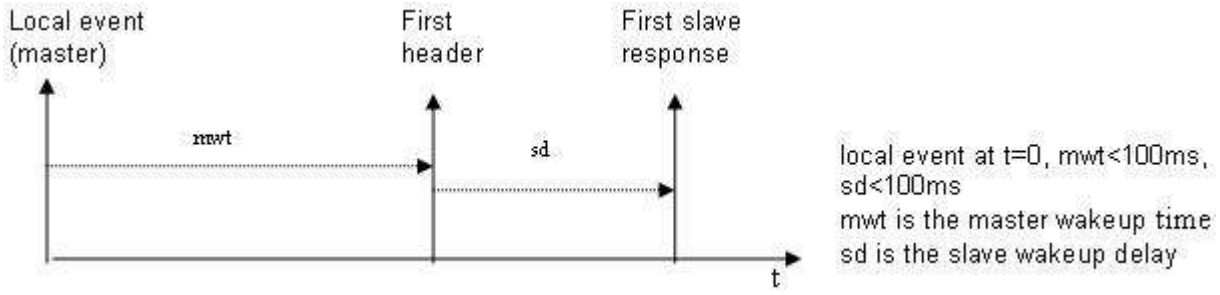


Figure: Master node sends wake up request.

The figure above shows the scenario when the master node sends a wake up request (first header) due to a local event. The debouncing time is not included in the figure. The first header is sent within a specific time(mwt). After beeing woken up the slave node is ready to transmit valid data within a specific time(sd).

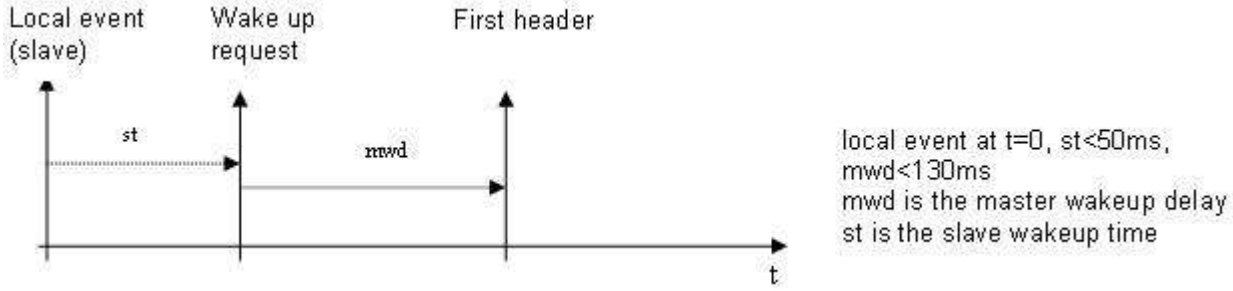


Figure: A slave node sends one wakeup request.

The figure above shows the scenario when a slave node wakes up the network due to a local event. The master node sends the first header mwd time after the first wake up request.

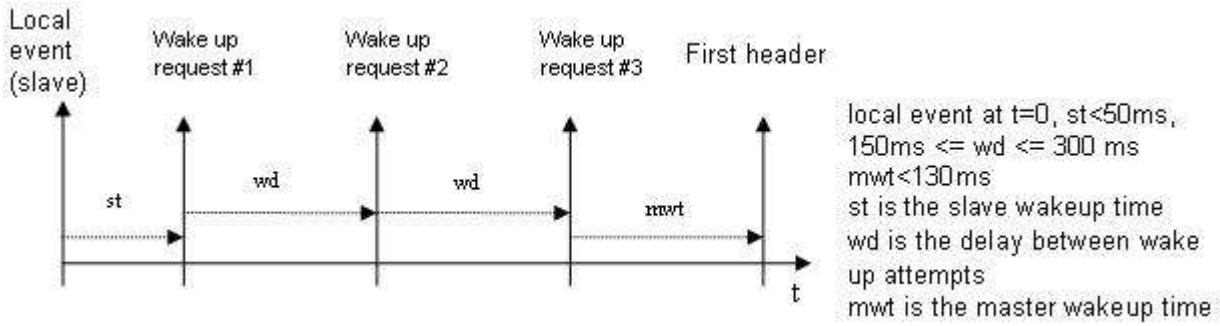


Figure: A slave node sends three wakeup requests.

The figure above shows the scenario when a slave node wakes up the network due a local event. When the slave node does not receive a header, it can send up to three wakeup requests.

4.4.1.5.1.2 Fault Tolerance Modes

660922 v2 **Communication persistency**

Version history:
(2): 验证方法从Analysis改为Analysis AND Test

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID: 53505v1

A short or open in any single wiring circuit of an ECU, except for power, ground, or serial data, shall not preclude the ability to communicate with that ECU for diagnostic purposes.

660923 v1 **ECU Power Loss**

Version history:
(1):

Purpose:

Source:
Verification Method: Test
Legacy ID: 53474v1

ECUs shall not interfere with normal communication among the remaining bus ECUs during a loss of power (or low voltage) condition. Upon return of power, normal operation shall resume without any operator intervention within a time determined by the vehicle manufacturer.

660924 v1 **Recovery from Bus short to battery**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 53483v1

Upon removal of a bus short to battery voltage, normal operation shall resume without any operator intervention within 1 second.

660925 v1 **Recovery from Bus short to ground**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 53478v1

Upon removal of a bus short to ground, normal operation shall resume without any operator intervention within 1 second.

660926 v1 **Recovery from reset**

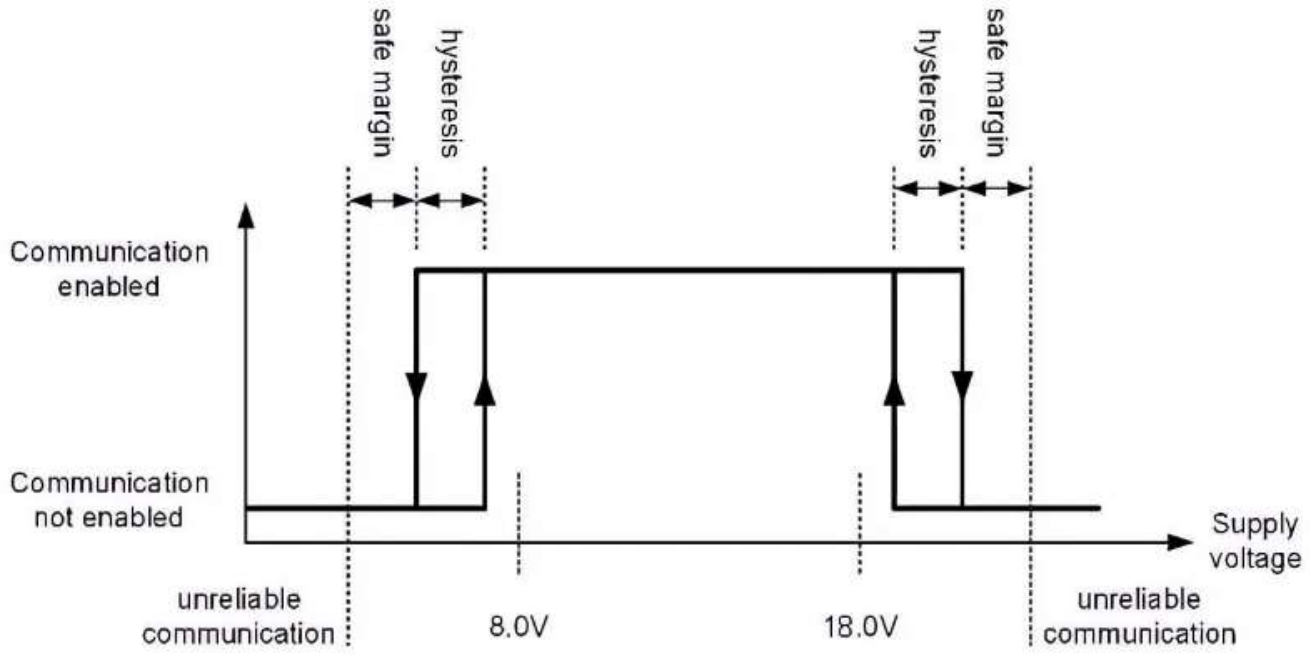
Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 53476v1

Following a reset the node must enter into a predefined state, defined in a ECU specific design document.

4.4.1.5.1.3 Operating Battery Power Voltage Range

The power supply voltage level at the node shall be measured continuously. HW and SW inaccuracy in the monitoring of the supply voltage level must be taken into account when the ECU is designed. The ECU must meet this operating voltage range requirement also in a worst case condition.
Network communication is permitted outside the specified voltage range, provided that the ECU can correctly read and write messages and not cause any disturbance on the network. There must be margins in both ends of the operating voltage range in regards to the voltage level where communication becomes unreliable or is no longer possible due hardware constraints. See figure below.



660928 v1 **Low Battery Voltage Operation**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 53535v1

For $0 < V_{batt\ ECU} < 8.0$ volts the bus may operate in either the normal or the passive mode. In the Passive Mode the bus shall be recessive (not be pulled or driven to ground) and TxD shall be in the high state. Fault codes shall be suppressed below 10 volts .

660927 v2 **Normal battery voltage power operation**

Version history:
(2): Change the voltage range to 8-18V

Purpose:
Source:
Verification Method: Test
Legacy ID: 53523v1

The node shall communicate on the network when the node voltage supply is 8,0 – 18,0 Volt. Network communication is permitted outside the specified voltage range, provided that the ECU can correctly read and write messages and not cause any disturbance on the network.

660929 v2 **Operating voltage range hysteresis**

Version history:
(2):

Purpose: Ensure stability at high and low supply voltage.
Source: N.A.
Verification Method: Test
Legacy ID: 53524v1

使用迟滞来确保在低电压和高电压条件下的稳定行为。迟滞电压至少为100mV。
注：此需求仅针对总线通信行为，非ECU的工作电压要求。

Hysteresis shall be used to ensure stable behaviour under low and high supply voltage conditions,see figure in section "Operating Battery Power Voltage Range".The hysteresis shall be at least 100 mV.
Note: This requirement addresses bus communication only and is not aligned with the operating voltage range requirement for the specific ECU.

4.4.1.6 LIN Data Link Layer - Master Requirements

•General LIN Master Requirements

4.4.1.6.1 General LIN Master Requirements

660931 v1 **Header jitter**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 53566v2

The transmission of headers must follow the time schedule given by the schedule table. There will normally be a jitter in the transmission of headers *[LIN_DL_2] LIN Specification Package Revision 2.1*, caused by internal processes inside the master ECU. The worst case jitter and the $T_{Header_Maximum}$ must be calculated by the supplier of the master ECU and reported to the OEM design team responsible for the ECU. The worst case jitter must not exceed 1 ms.

660930 v1 **LIN standard used in master node**

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID: 53641v1

The master for each network shall support the required LIN standards for each slave connected to the network in accordance with *[LIN_DL_1] LIN Specification Revision 1.3* or *[LIN_DL_2] LIN Specification Package Revision 2.1*.

•Master Wake-Up Timing

4.4.1.6.2 Master Wake-Up Timing

660934 v1 **Local event - master sends the first header**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 53582v1

A master node shall transmit the first header within 100 ms after a local event.

660935 v1 **Network event - wake up request to first header**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 53583v1

The LIN master shall detect and begin transmission of the first header of the appropriate schedule table no longer than 130ms after the wake up request. Specific subsystems that require a faster wakeup response time can be defined in the subsystem application requirements.

660936 v1 **Power on - master sends the first header**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 53584v1

A master node shall be able to transmit the first header within 250 ms after power is applied or after a hardware reset.

660933 v1 **Timeout and sleep commands**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 53581v1

The master shall always issue a sleep command when power down of the bus is required, *[LIN_DL_2] LIN Specification Package Revision 2.1.*

4.4.1.7 LIN Data Link Layer - Slave Requirements

•General LIN Slave Requirements

4.4.1.7.1 General LIN Slave Requirements

660939 v2 **LIN standard used in slave node**

Version history:
(2):

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID: 53701v1

All slave modules must be compliant with one of the following Local Interconnect Network (LIN) standards: *[LIN_DL_1] LIN Specification Revision 1.3* or *[LIN_DL_2] LIN Specification Package Revision 2.1*. The chosen standard must be agreed with the GRI Basetech Team.

660940 v1 **Transmission time**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 53702v1

The true $T_{Response_Maximum}$ value, in which a node can complete its response, shall be stated in the free text section of the Node Capability File. In the event a value is not provided, the communication schedule (defined in the .ldf-file or equivalent) shall presume a value of $T_{Response_Maximum}$ as defined in *[LIN_DL_2] LIN Specification Package Revision 2.1*.

•Slave Wake-Up Timing

4.4.1.7.2 Slave Wake-Up Timing

660942 v1 **Local event - slave sends the wake up request**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 53706v1

A slave node shall transmit a wake up request within 50ms after a local event.

660943 v1 **Network event - wake up request to slave ready to communicate**

Version history:
(1):

Purpose:
Source:
Verification Method: Test

Legacy ID: 53708v1

The LIN slave node shall detect the wake up request and be able to transmit valid data within 100ms. Slaves shall not respond until their data is valid. Diagnostic supervision in the master shall take this delay into account. Specific subsystems that require a faster wakeup response time can be defined in the subsystem application requirements.

660944 v1 **Power on - slave sends the first response**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 53709v1

A slave node shall be able to transmit valid data within 100 ms after power is applied or after a hardware reset. Slaves shall not respond until their data is valid. Diagnostic supervision in the master shall take this delay into account. Specific subsystems that require a faster wakeup response time can be defined in the subsystem application requirements.

660941 v1 **Timeout and sleep commands**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 53704v1

All slaves must implement both the timeout and sleep command according to *[LIN_DL_2] LIN Specification Package Revision 2.1*.

•LIN slave with E2E

4.4.1.7.3 LIN slave with E2E

841118 v1 **LIN slave with E2E**

Version history:
(1):

Purpose: LIN slave with E2E
Source:
Verification Method: Test
Legacy ID:

To use the E2E_P01Protect function, the data that should be computed shall be packed in an array of uint8s. To get the same result in both sender and receiver(s), the elements need to be packed in the same order in both sides.

The array sent to E2E_P01Protect shall have the following order:

1. Index 0: CRC (8-bit)
2. Index 1: 0x0 + 4-bit counter (4 bit counter extended to 8 bits using 0 as padding)

3. Index 2: data 1

4. Index n: data 2

The data (data 1, data 2) shall be ordered alphabetically by the name of the data-element variable.

Note:
1. The data-element variable is not referring to system signals, i.e. data shall not be sorted using the names of the system signal. Data-element variable is referring to VariableDataPrototype -> ApplicationRecordDataType -> ApplicationRecordElement. The data shall be sorted by the names of the ApplicationRecordElements.

2. All data element names should be unique, however if an extension to is used to make an unique naming e.g VehicleSpd -> VehicleSpd1, then following order should be used. (According to ASCII):

1. Alphanumeric characters
2. Alphabetic characters

Order of upper and lower case alphabetic characters should be disregarded. If there is upper and lower case alphabetic to be compared for the same alphabetic characters, it should be use lower case alphabetic.

3. The signal group with E2E from the sender should keep the counter continuous on every sending in the schedule.

4.5 Communication_CAN/CANFD

4.5.1 Communication_CAN&CANFD_Requirement_Specification

4.5.1.1 General

•Definitions and Abbreviations

4.5.1.1.1 Definitions and Abbreviations

Definition	Explanation
CAN H	One of the two wires connecting the CAN interfaces in the network. When the CAN H is switched from recessive state to dominant state, the voltage on the wire is increased.
CAN L	One of the two wires connecting the CAN interfaces in the network. When the CAN L is switched from recessive state to dominant state, the voltage on the wire is decreased.
Classic CAN	The CAN protocol in its original format, where a frame has 0 – 8 byte of user data and a fixed bitrate of up to 1Mbit/s throughout the frame. If the value of the FDF bit in the frame is set to dominant the frame is a Classic CAN frame.
CAN FD	An upgraded CAN protocol format, where a frame has 0 – 64 byte of user data and a bitrate that may be set to a higher value in the user data field compared to the rest of the frame. If the value of the FDF bit in the frame is set to recessive the frame is a CAN FD frame.
FDF bit FD	Format indicator bit to indicate whether a CAN frame is a Classic CAN frame or a CAN FD frame. The exact function of the FDF bit is defined in the ISO 11898-1:2015 standard.

Abbreviation	Description
CAN	Controller Area Network
CAN FD	Controller Area Network with Flexible Data-Rate
ECU	Electronic Control Unit
EMC	Electromagnetic Compatibility
ISO	International Standards Organisation

PCB	Printed Circuit Board
PLL	Phase Locked Loop
SAE	Society of Automotive Engineers

•Requisite Documents

4.5.1.1.2 Requisite Documents

These references are requirements only if explicitly referred to as that.

Reference	Document
[Ext CAN PL 1]	ISO 7498-1 Open Systems Interconnection - basic reference model
[Ext CAN PL 2]	CAN specification V2.0 part B (BOSCH)
[Ext CAN PL 3]	ISO 11898-1:2015(E) Road Vehicles - Controller Area Network (CAN) – Part 1: Data Link Layer and Physical Signalling
[Ext CAN PL 4]	ISO 11898-2:2016(E) Road Vehicles - Controller Area Network (CAN) – Part 2: High-Speed Medium Access Unit
[Ext CAN PL 5]	SAE J2284 – 3 High Speed CAN for Vehicle Applications at 500 KBPS, issued 2010-03
[CAN PL 6]	UDSonCAN (Internal)
[Ext CAN PL 7]	ISO 16845 Road Vehicles - Controller Area Network (CAN) - Conformance Test Plan
[Ext CAN PL 8]	CAN High-Speed Transceiver Conformance Test, version 1.0 (GIFT/ICT)
[CAN PL 9]	~~Components for Physical Interfaces (Internal)~~
[CAN PL 10]	EMC System and Components Requirements - project specific EMC specification as stated by the general component requirements.
[Ext CAN PL 11]	SAE J2284 – 4 High Speed CAN for Vehicle Applications at 500 Kbps with CAN FD Data at 2Mbps
[Ext CAN DL 1]	ISO 7498-1: Open Systems Interconnection - basic reference model
[Ext CAN DL 2]	CAN specification V2.0 part B (BOSCH)
[Ext CAN DL 3]	ISO 11898-1:2015(E): Road Vehicles - Controller Area Network (CAN) – Part 1: Data Link Layer and Physical Signalling
[Ext CAN DL 4]	ISO 11898-2:2016(E): Road Vehicles - Controller Area Network (CAN) – Part 2: High-Speed Medium Access Unit
[Ext CAN DL 5]	SAE J2284 – 4: High Speed CAN for Vehicle Applications at 500 KBPS with CAN FD Data at 2 Mbps, issued 2016-06
[CAN DL 6]	UDSonCAN

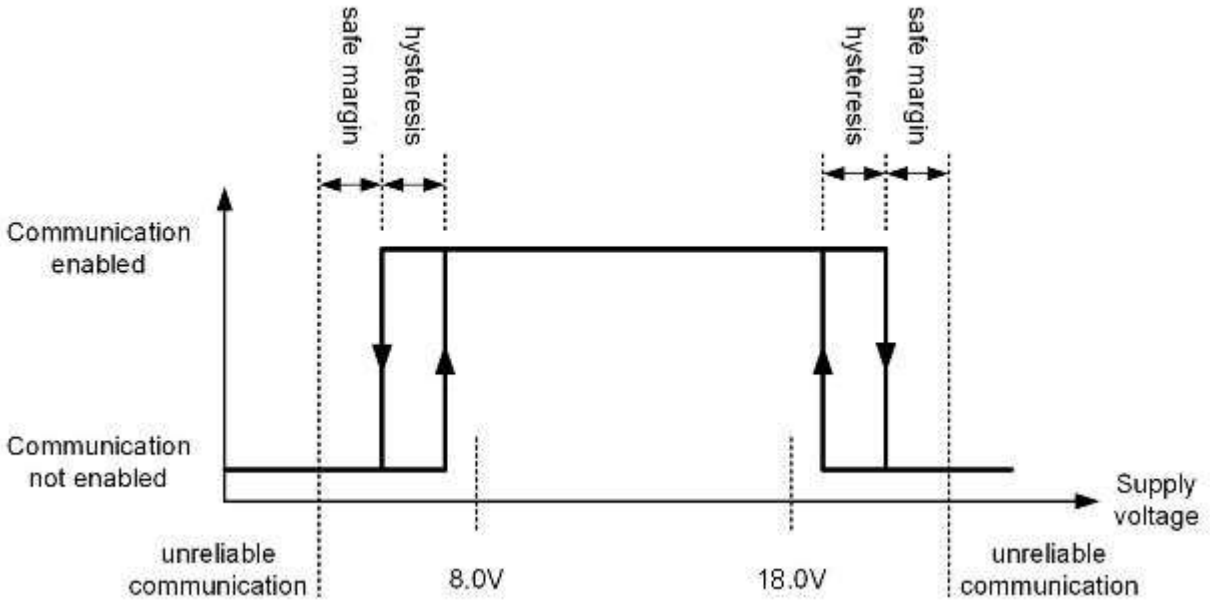
•Requirements

4.5.1.1.3 Requirements

660809 v2	Communication voltage range

Version history:
(2):

Purpose: To ensure that the network fulfills operating voltage range requirements on vehicle level.
Source: N.A.
Verification Method: Test
Legacy ID: 53914v3



The node shall communicate on the network when the node voltage supply is 8,0 – 18,0 Volt. Network communication is permitted outside the specified voltage range, provided that the ECU can correctly read and write messages and not cause any disturbance on the network.

Note: This requirement addresses bus communication only and is not aligned with the operating voltage range requirement for the specific ECU.

660806 v2 Component Selection - CAN Physical Interfaces

Version history:
(2):

Purpose: This is to prevent extra DC load on the network.
Source: This is necessary for maintaining the CAN signal voltage level specified in partially powered network
Verification Method: Analysis
Legacy ID: 76513v1

For power-switched ECUs an “ideal passive” transceiver shall be used. Transceivers shall not leak more than 5µA from an active CAN bus, in power-off mode.

For battery-powered ECUs a transceiver with sleep functionality must be used.

Note: *This requirement is valid for all CAN ECU's.*

660807 v3 Missing frame master - CAN

Version history:

(3): 增加XCU/XCM/XCUP/PCMU/ZCUT节点

Purpose: The missing frame concept are used for pinpointing network errors on the bus.
Source: UDSonCAN
Verification Method: Test
Legacy ID: 76550v2

The ECU routing diagnostic requests to and from a CAN network shall be missing frame master for that network. The missing frame master ECU's are DHU/IHU,ZCUD,ZCUP,ADCU/ASDM,XCU/XCM/XCUP/PCMU/ZCUT, and the monitoring slave ECU will be defined in the subsystem design from Basetech.

660808 v1 No priority inversion

Version history:
(1):

Purpose: The concept of guaranteed worst case latency offers several advantages to the system designer as well as to the node designer and node supplier. The SDB team will use guaranteed worst case latency to design a set of frames which will fulfil the signalling requirements given in the specifications defining vehicle functions. It will also be possible to guarantee that no signals will be overwritten, even when they have been transferred over gateways. Guaranteed worst case latency also permits busload to be significantly higher than the 30% often seen as the practical limit for reliable communication on CAN buses.

The communication driver shall be designed to be free of priority inversion when frames are transmitted on the CAN bus by AUTOSAR based ECU's. If all CAN stacks are designed according to the priority inversion avoidance (PIA) concept it becomes possible to analyse the bus behaviour a priori. This analysis makes it possible to calculate and guarantee the worst case latency of all frames.
Source: EESE Strategic Decision
Verification Method: Analysis
Legacy ID: 77275v1

The communication driver shall be designed to be free of priority inversion when frames are transmitted on the CAN bus.

660810 v3 The development frame usage in the development

Version history:
(3):

Purpose: To specific the usage of development frame.
Source:
Verification Method: Test
Legacy ID:

If one ECU needs to use the development frame, it should be approved by the basetech team.

and the application development frame should be removed in the final software before Job1 in the specific program.

To be important, all the development frames should be controlled by diagnostic service 0x2E with DID D12C to close or open it.

The ID range of development frame is 0x580~0x5FF.

4.5.1.2 CAN and CANFD Physical Layer

•Requirements

4.5.1.2.1 Requirements

663149 v2 **Applicable CAN&CANFD standards**

Version history:
(2): 将需求Name、Purpose和Description中的CAN替换为CAN&CANFD；Description中增加CIA 601-4规范；Description中删除各规范的年份，默认为最新版。

Purpose: Specification of CAN&CANFD protocol used
Source: N.A:
Verification Method: Analysis
Legacy ID: 53902v3

General specification of bus shall apply following documents. References can be found in CAN&CANFD Physical Layer - Requisite documents.

- *ISO 11898-1*
- *ISO 11898-2*
- *SAE J2284-4*
- *CIA 601-4*

663150 v2 **CAN&CANFD conformance**

Version history:
(2): 将需求Name、Purpose和Description中的CAN替换为CAN&CANFD

Purpose: To ensure compatibility of all modules communicating on the CAN&CANFD bus
Source: N.A.
Verification Method: Analysis
Legacy ID: 53903v3

All CAN&CANFD controllers shall be fully ISO 16845 compliant [Ext CAN PL 7]. All module suppliers shall obtain the ISO 16845 compliance certificate from their chosen semiconductor supplier.

663148 v1 **General requirement to match test case from standard**

Version history:
(1):

Purpose: To match the test case from test request
Source: Match the test case from Standard
Verification Method: Test
Legacy ID:

Reference number	Source	Title	Revision number
1	ISO	ISO11898-2	2016
2	SAE	SAE J2284-3	

Note:
References are typically available from ISO and SAE standard, it need to be added here to match some test case

from Verification request.

663308 v2 **The priority between standard documents and this document**

Version history:
(2): 将需求Purpose中的CANFD替换为CAN&CANFD

Purpose: To optimise the CAN&CANFD system solution
Source:
Verification Method: Analysis
Legacy ID:

In case of discrepancy between the standard documents and this specification, this specification has precedence.

4.5.1.2.1.1 CAN&CANFD controller

663151 v2 **CAN&CANFD controller**

Version history:
(2): 将需求Name和Description中的CAN替换为CAN&CANFD

Purpose: To fulfil communication concepts.
Source:
Verification Method: Analysis
Legacy ID: 54002v2

The CAN&CANFD controller must support predictable message priority and arbitration.

When the bus is free, the message with the highest priority, of those ready to be transmitted, shall in all cases be the one arbitrating for bus access.

4.5.1.2.1.2 Bit Rate Tolerance Definition

663312 v1 **Calculation of speed tolerance**

Version history:
(1):

Purpose: To ensure that the calculation of data speed inaccuracy includes all causes of inaccuracy
Source:
Verification Method: Analysis
Legacy ID:

The total network transmission speed tolerance shall include all causes. For crystals and resonators this shall include manufacturing tolerance, ageing and temperature. For PLL's, the tolerance shall be determined as specified by the semiconductor manufacturer, and shall include the effects of manufacturing, ageing, temperature and transmission speed.
Note: particular care must be taken to ensure the appropriate tolerance is met when a resonator is being used with a PLL.
Note: For crystal, tolerance<=0.1%; For ceramic oscillator, tolerance <=0.2%.

663152 v1 **Short term bit rate tolerance**

Version history:
(1):

Purpose: To ensure sufficient bit rate accuracy throughout the arbitration phase
Source: N.A.
Verification Method: Analysis
Legacy ID: 53905v3

Bit rate tolerance shall apply over all periods between 2 and 13 bit times.

4.5.1.2.1.3 Termination

663153 v2 **PCB design for CAN&CANFD termination**

Version history:
(2): 将需求Name中的CAN替换为CAN&CANFD

Purpose: To ensure that a ECU is, without PCB redesign, able to assume the role of terminating node in a network.
Source: N.A.
Verification Method: Analysis
Legacy ID: 53908v2

Capability to support termination shall be provided on the PCB by each ECU. Each vehicle program will specify which ECU are to be terminating ECU.

4.5.1.2.1.4 Power Supply and Reset

The power supply voltage level at the node shall be measured continuously. HW and SW inaccuracy in the monitoring of the supply voltage level must be taken into account when the ECU is designed. The ECU must meet this operating voltage range requirement also in a worst case condition.

Network communication outside the mandatory voltage range is encouraged on the condition that the ECU can correctly read and write messages and cause no disturbance to the network. There must be margins in both ends of the operating voltage range in regards to the voltage level where communication becomes unreliable or is no longer possible due hardware constraints.

663154 v2 **ECU internal voltage supply**

Version history:
(2): 将Description中的CAN-circuitry修改为CAN&CANFD-circuitry

Purpose: In order to protect components on the PCB from short circuits in the wiring.
Source: N.A.
Verification Method: Analysis
Legacy ID: 53917v2

The voltage supply used by the network interface must not be directly connected to any external part. That means that the regulated power used by the CAN&CANFD-circuitry on the printed circuit board shall be used only inside the cover of the ECU.

4.5.1.2.1.5 Transceivers

663155 v3 CAN&CANFD transceiver component selection

Version history:
(3): 将需求Name中的CAN替换为CAN&CANFD;Description中增加Note，要求供应商提供组件信息和物理拓扑仿真信息。

Purpose: Preventing HW issues to be found in late car program phases.
Source: N.A.
Verification Method: Analysis
Legacy ID: 53920v4

It is the responsibility of the ECU supplier to select components which are appropriate for the intended use, with respect to, but not limited to, bus bit rate, operating voltage range, diagnostic capabilities etc.And Components recommended for use in ECU's connected to CAN networks should be informed to GRI Base Technology team.

Note: The supplier shall also provide Geely Network Physical Topology Simulation Information in accordance with GEELY's requirements.

4.5.1.2.1.6 Chokes

663156 v2 Common mode chokes

Version history:
(2): 需求Description中对共模电感值的要求改为100uH or less.

Purpose: To prevent EMC issues
Source: N.A.
Verification Method: Analysis
Legacy ID: 53922v4

If common mode chokes are used they must meet the following requirements. These requirements must be confirmed by the datasheet or test results.

1. Inductance is 100μH or less
2. Choke type is common mode choke with bifilar winding
3. Voltage rating of 40 Volt or higher

4.5.1.2.1.7 ESD Protection Components

663157 v2 ESD protection components

Version history:
(2):

Purpose: Protect interface components from high static discharge voltage levels.
Source: N.A.
Verification Method: Analysis
Legacy ID: 53924v4

The ESD protection components shall be Bidirectional Zener diodes.

4.5.1.2.1.8 ECU Delay

663158 v2 **ECU internal delay**

Version history:
(2): 验证方法修改为Analysis;Description中CAN替换为CAN&CANFD，增加Note。

Purpose: To ensure proper function of the network.
Source: N.A.
Verification Method: Analysis
Legacy ID: 53925v4

The interface implementation must meet the round trip delay requirement in ISO11898-2 [Ext CAN PL 4]. The Round Trip Delay in the CAN&CANFD interface must be not longer than 255ns.

Note: Optional AND gate connecting a second controller to the transceiver must be included in the round trip delay calculation, see item Physical Layer Circuit.

4.5.1.2.1.9 Interface implementation, physical layer and layout requirements

663315 v2 **Physical layer circuit**

Version history:
(2): 将需求Purpose中的CAN FD替换为CAN&CANFD

Purpose: To standardise the implementation of the CAN&CANFD interface in all ECUs on the bus
Source:
Verification Method: Analysis
Legacy ID:

The following circuit diagram shall be implemented in order to protect the circuit from damage caused by electrostatic discharge and optimise the EMC performance. I t remains the responsibility of the ECU supplier to ensure that the ECU fully complies with all functional requirements on the ECU, such as support for partial networking, bus failure reporting and EMC *[CAN PL 10]* .

4.5.1.2.1.9.1 General design requirements

663159 v1 **Bus fault monitoring circuitry**

Version history:
(1):

Purpose: To maintain good CMRR when adding components to the standard interface circuit.
Source: N.A.
Verification Method: Analysis
Legacy ID: 435859v1

In order to monitor bus electrical status it may be necessary to add components to the CAN interface. Any such circuitry must be symmetrical to CAN H and CAN L with respect to geometry and impedance.

663161 v2 **CAN&CANFD circuit traces on PCB**

Version history:
(2): 将需求Name中的CAN替换为CAN&CANFD，修改Description中描述Guard tracks are required for all CAN H

and CAN L, TXD and RXD circuits on the same PCB layer.

Purpose: To improve EMC performance
Source: N.A.
Verification Method: Analysis
Legacy ID: 53946v3

CAN H/CAN L traces between the ECU connector and transceiver shall be as short as possible and routed side-by-side. Guard tracks are required for all CAN H and CAN L, TXD and RXD circuits on the same PCB layer. Guard tracks should not be placed between CAN H and CAN L.

663162 v3 CAN&CANFD wiring inside ECU

Version history:
(3): 将需求Name的CAN替换为CAN&CANFD；将Description中的Basetech team替换为GRI Base Technology team

Purpose: The ensure that the bus wiring length calculation includes significant intra module CAN wiring length.
Source: N.A.
Verification Method: Analysis
Legacy ID: 435874v0

In case the vehicle wire harness is not connected directly to the CAN interface PCB, the added distance between the CAN interface and the vehicle harness must be reported to GRI Base Technology team. The additional wire length must be included in the permitted wire length budget for the network.
NOTE: This requirement applies for example when a larger module is designed in such a way that the vehicle wiring connector is mounted some distance away from the PCB where the CAN interface is located.

663163 v1 Decoupling capacitor location

Version history:
(1):

Purpose: To prevent switching noise from the transceiver to propagate into the PCB circuitry
Source: N.A.
Verification Method: Analysis
Legacy ID: 53948v3

The decoupling capacitor C_pwr shall be placed as close as possible to the transceiver.

663164 v1 Guard tracks on PCB

Version history:
(1):

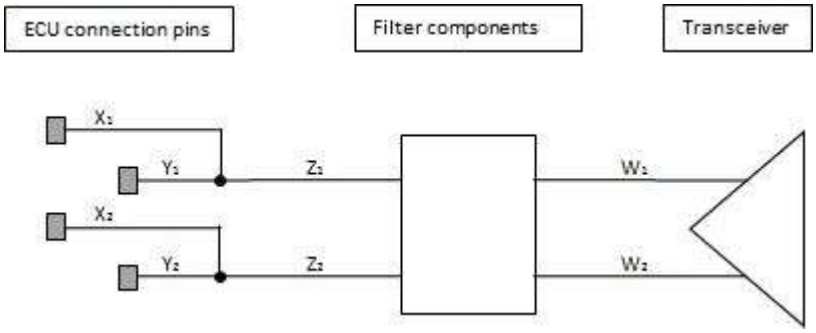
Purpose: To improve EMC performance.
Source: N.A.
Verification Method: Analysis
Legacy ID: 53947v2

All guard tracks shall be at least 0.5 mm wide and grounded at least every 10 mm. Guard tracks shall be symmetrical and the stray capacitance caused by the guard tracks shall be included in the calculation of the total interface capacitance.

663165 v1 PCB CAN trace symmetry

Version history:
(1):

Purpose: Interface symmetry is a prerequisite for good CMRR.
Source: N.A.
Verification Method: Analysis
Legacy ID: 435857v2



The CAN L and CAN H tracks, including PCB traces and connector lead frames, must be of equal length to all connection and junction points. As shown in the illustration the lengts of X, Y, Z and W respectively must be equal within 1mm.

663166 v1 Total interface capacitance

Version history:
(1):

Purpose: To ensure full interface impedance symmetry in order to maintain good CMRR.
Source: N.A.
Verification Method: Analysis
Legacy ID: 435855v1

The capacitance from CAN L resp. CAN H to ground shall be ≤ 47pF and the difference in capacitance shall be ≤ 10 %. The calculation shall include the capacitance, including tolerances, and the parasitic capacitance of all components. It shall also include the parasitic capacitance of the PCB layout.

663167 v1 Transceiver ground plane

Version history:
(1):

Purpose: To improve EMC performance
Source: N.A.
Verification Method: Analysis
Legacy ID: 53944v2

A ground plane is required under the transceiver chip on the component layer of the PCB.

663168 v1 **Transceiver grounding**

Version history:
(1):

Purpose: To minimise the impact of a noisy power ground.
Source: N.A.
Verification Method: Analysis
Legacy ID: 53941v2

All grounding of the CAN transceiver and capacitors shall be made to ECU signal ground.

663169 v1 **Transceiver location on PCB**

Version history:
(1):

Purpose: To reduce the Common Mode interference coupling beween PCB and bus wiring.
Source: N.A.
Verification Method: Analysis
Legacy ID: 53945v2

Transceiver shall be located as close to the ECU connector as possible. Other IC's are not permitted between the ECU connector and the transceiver.

4.5.1.2.1.10 Terminal ECUs on CAN&CANFD

663170 v3 **Terminating ECUs on CAN&CANFD**

Version history:
(3): 将需求Name中的CAN替换为CAN&CANFD

Purpose:
Source: ECU topology
Verification Method: Analysis AND Test
Legacy ID:

The terminal node should be referred to the latest network topology.
It should care about the terminal information in the basetech handshake, whether the ECU is terminal node or not.

•Interface requirements

4.5.1.2.2 Interface requirements

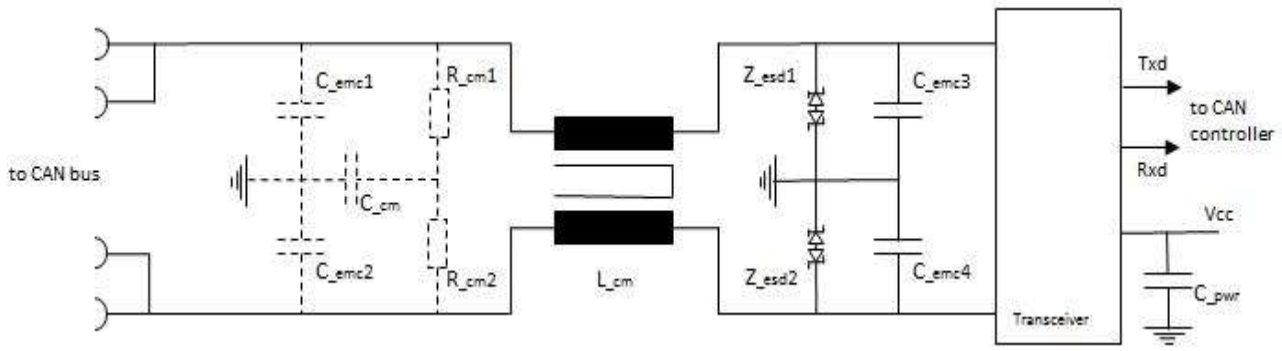
663172 v1 **Physical layer circuit for non terminating node**

Version history:
(1):

Purpose: To standardise the implementation of the CAN interface in all ECUs on the bus
Source:
Verification Method: Analysis

Legacy ID: 467135v0

The following circuit diagram shall be implemented in order to protect the circuit from damage caused by electrostatic discharge and optimise the EMC performance. It remains the responsibility of the ECU supplier to ensure that the ECU fully complies with all functional requirements on the ECU.



Component ID	Description	Value
C_pwr	Monolithic ceramic capacitor	≥ 90 nF, ≥ 16V rating
C_emc1, C_emc2	Monolithic ceramic capacitor	Optional. ≤ 15 pF, ≥ 100V rating
C_cm	Monolithic ceramic capacitor	4.7nF ±10%, ≥ 50V rating
C_emc3, C_emc4	Monolithic ceramic capacitor	15 pF ±10%, ≥ 50V rating
R_cm1, R_cm2	Resistor	PCB preparation(Note 1)
Z_esd1, Z_esd2	Bidirectional Zener diode	27 V breakdown voltage
L_cm	Common Mode Choke	100μH, Bifilar winding

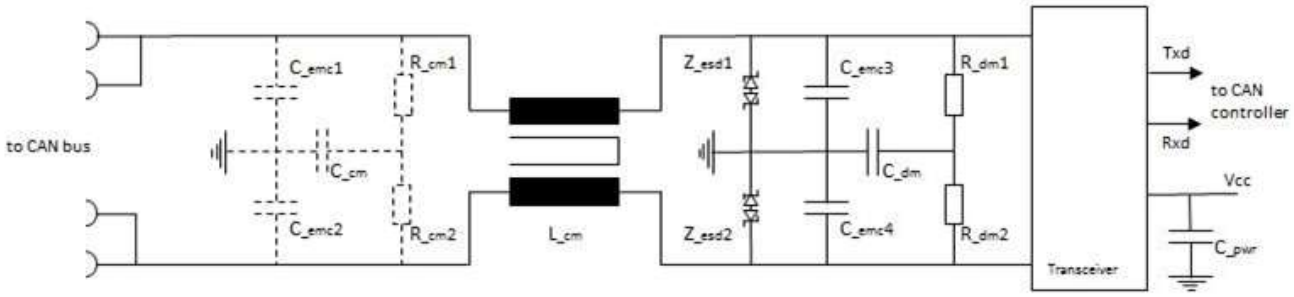
Note1: This resistor should follow the latest standard<SAEJ-2284>.

663171 v1 **Physical layer circuit for terminating node**

Version history:
(1):

Purpose: To standardise the implementation of the CAN interface in all ECUs on the bus
Source: N.A.
Verification Method: Analysis
Legacy ID: 53928v4

The following circuit diagram shall be implemented in order to protect the circuit from damage caused by electrostatic discharge and optimise the EMC performance. It remains the responsibility of the ECU supplier to ensure that the ECU fully complies with all functional requirements on the ECU.



Component ID	Description	Value
C_pwr	Monolithic ceramic capacitor	≥ 90 nF, ≥ 16V rating
C_emc1, C_emc2	Monolithic ceramic capacitor	Optional. ≤ 15 pF, ≥ 100V rating
C_cm, C_dm	Monolithic ceramic capacitor	4.7nF ±10%, ≥ 50V rating
C_emc3, C_emc4	Monolithic ceramic capacitor	15 pF ±10%, ≥ 50V rating
R_cm1, R_cm2	Resistor	PCB preparation
R_dm1, R_dm2	Resistor	R_dm1+R_dm2 = 118–132Ω (including tolerance), R_dm1/R_dm2 must be matched to ±1%, 250 mW
Z_esd1, Z_esd2	Bidirectional Zener diode	27 V breakdown voltage
L_cm	Common Mode Choke	100μH, Bifilar winding

4.5.1.3 CAN and CANFD Data Link Layer

•Requirements

4.5.1.3.1 Requirements

660851 v1 **General requirement to match test case from standard**

Version history:
(1):

Purpose: To match the test case from test request
Source: Match the test case from Standard
Verification Method: Test
Legacy ID:

Reference number	Source	Title	Revision number
1	ISO	ISO11898-1	2015

Note:
References are typically available from ISO standard, it need to be added here to match some test case from Verification request.

660849 v1 **Standards for CAN interface**

Version history:
(1):

Purpose: Specification of CAN protocol
Source: N.A.
Verification Method: Analysis AND Test
Legacy ID: 53881v2

The interface shall be specified according to CAN standard using bit rate specified in section Network Transmission Speed 500 kbps.

908758 v1 **Standards for CANFD interface**

Version history:
(1): 新增CANFD协议的传输速率需求

Purpose: Specification of CANFD protocol
Source:
Verification Method: Analysis AND Test
Legacy ID:

The interface shall be specified according to CANFD standard using bit rate specified in section Network Transmission Speed arbitraion part 500 kbps and data part 2000 kbps.

660850 v1 **The priority between standard documents and this document**

Version history:
(1):

Purpose: To optimise the CAN system solution
Source: N.A.
Verification Method: Analysis
Legacy ID: 53850v2

In case of discrepancy between the standard documents and this specification, this specification has precedence.

4.5.1.3.1.1 ECU Level

660855 v1 **Bus fault reporting**

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 53885v2

At least one module on each bus must be able to detect and report faults on the physical bus. The faults and their

respective fault codes are specified in [CAN DL 6] . The SRD Diagnostics and ECU Platform for the specific vehicle program specifies which module should perform this fault monitoring.

660854 v1 CAN bus level during reset

Version history:
(1):

Purpose: Prevent the ECU from disturbing the network during a local reset.
Source: N.A.
Verification Method: Test
Legacy ID: 53884v2

Modules shall not drive the CAN bus dominant during module reset.

660852 v1 Communication protocol

Version history:
(1):

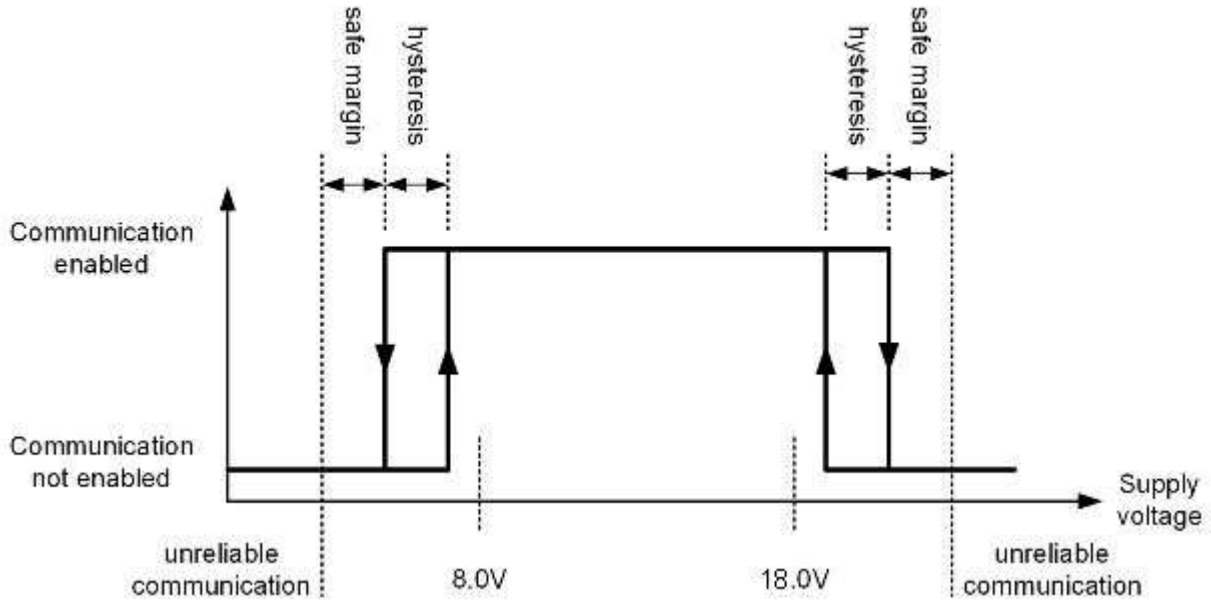
Purpose: Specification of CAN protocol used
Source: N.A.
Verification Method: Analysis
Legacy ID: 53848v2

Controller Area Network (CAN) in accordance with CAN V2.0 [Ext CAN DL 2] .

660853 v3 Operating voltage range hysteresis

Version history:
(3):

Purpose: Ensure stability at high and low supply voltage
Source: N.A.
Verification Method: Test
Legacy ID: 53883v2



使用迟滞来确保在低电压和高电压条件下的稳定行为。迟滞电压至少为100mV。
注：此需求仅针对总线通信行为，非ECU的工作电压要求。

Hysteresis shall be used to ensure stable behaviour under low and high supply voltage conditions. The hysteresis shall be at least 100 mV.
Note: This requirement addresses bus communication only and is not aligned with the operating voltage range requirement for the specific ECU.

4.5.1.3.1.1.1 Resynchronisation and bit sampling

660856 v1 **Resychronisation strategy**

Version history:
(1):

Purpose: Increase communication robustnes
Source: N.A.
Verification Method: Analysis
Legacy ID: 53862v2

Resynchronisation on recessive to dominant transitions only.

660857 v2 **Sample mode**

Version history:
(2): 增加采样模式中的SSP，适用于2Mbps的CANFD通信。

Purpose: Sample mode at 500 kbit/s and 2Mbit/s.
Source: N.A.
Verification Method: Analysis
Legacy ID: 53863v2

For 500 kbps, only single sample mode shall be used.
For 2 Mbps or higher, SSP mode shall be used.

4.5.1.3.1.1.2 Speed tolerance definition

660858 v1 **PLL bypass**

Version history:
(1):

Purpose: Minimise communication jitter
Source: N.A.
Verification Method: Analysis
Legacy ID: 53853v2

If a PLL is being used by, and if supported by the chosen microcontroller, the CAN physical layer shall be clocked directly from the base oscillator, i.e. PLL bypass should be used.
Note: there are various ways to achieve this, e.g. control register in the CAN controller, microcontroller central clock module, etc.

660859 v1 **PLL locking**

Version history:
(1):

Purpose: Minimise the number of communication error events
Source: N.A.
Verification Method: Analysis
Legacy ID: 53861v2

When a PLL is being used to clock the CAN physical layer, transmission and reception are only allowed when the PLL is locked.

4.5.1.3.1.1.3 Network Transmission Speed - 500kbps

660860 v1 **Bit time set up for 500 kbps**

Version history:
(1):

Purpose: Specify accepted sampling point range
Source: N.A.
Verification Method: Test + Analysis
Legacy ID: 53864v2

Bit timing set up for nodes with maximum transmission speed tolerance ± 0.4% .
For a maximum transmission speed tolerance of ± 0.4%, the bit time set up for nodes shall be:

BIT NtQ	tSJWmin Num tQ	tSEG2min Num tQ	tSEG2max Num tQ	Sample point %	SJWmin %
22	3	3	5	77 – 86	13.6
21	3	3	5	76 – 86	14.3
20	3	3	5	75 – 85	15.0
19	3	3	4	79 – 84	15.8

18	2	2	4	78 – 89	11.1
17	2	2	4	76 – 88	11.8
16	2	2	3	81 – 88	12.5
15	2	2	3	80 – 87	13.3
14	2	2	3	79 – 86	14.3
13	2	2	2	85	15.4
12	2	2	2	83	16.7
11	2	2	2	82	18.2
10	2	2	2	80	20.0

Note 1: Calculation of the Tseg1 value
NTq = Sync_Seg + Prop_Seg + Phase_Seg1 + Phase_Seg2, see [Ext CAN DL 3]
Sync_Seg = 1
Tseg1 = Prop_Seg + Phase_Seg1
Tseg2 = Phase_Seg2

Note 2: to achieve 10 or more tQ at 500kbps, the input clock to the CAN controller must have a minimum frequency of 5MHz.

4.5.1.3.1.1.4 CAN FD bit register settings for 2 Mbps

660861 v3 **CAN FD bit register settings for 2 Mbps**

Version history:
(3): Add the note.

Purpose: Support CAN FD with 2 Mbps data rate
Source:
Verification Method: Analysis AND Test
Legacy ID:

CANFD sample point setting

Parameter	Min	Nominal	Max
Sample point(Arbitration part)	-	80%	-
Sample point(Data part)	-	70%	-
Sample frequency	-	1	-

CANFD sample point setting table

Bit per second	Clock	Nq	Tq	Prop_Seg	Phase_Seg1	Phase_Seg2	SJW	SP	SSP offset	TDC enable
Arbitraion part(500kbps)	40MHz	80	25ns	47	16	16	16	80%	/	/
Data part(2Mbps)	40MHz	20	25ns	7	6	6	6	70%	65%	Yes(Auto TDC)
Arbitraion part(500kbps)	80MHz	160	12.5ns	95	32	32	32	80%	/	/
Data part(2Mbps)	80MHz	40	12.5ns	15	12	12	12	70%	67.5%	Yes(Auto TDC)

Note 1:Other settings may only be used after agreement with the Geely Base Technology responsible.以上图示仅为示例，如果有不同的配置，需要和Geely Basetech团队确认后使用。

Note 2:The unit of SSP offset is the minimum time quantum (mTq) before BRP((Baud Rate Prescaler)) frequency division. It is recommended to configure it one mTq ahead of SP, and BRP should be configured as 1 or 2.

Meanwhile, it is recommended that the same Tq length is used in the nominal bit time and the in the data bit time。SSP offset的单位是未经BRP(Baud Rate Prescaler)分频前的最小时间量子（mTq），推荐配置为比SP提前一个mTq，且BRP应配置为1或2。同时，建议数据场和仲裁场的Tq length相同。

4.5.1.3.1.1.5 CAN FD bit register settings for 5 Mbps

660862 v1 **CAN FD bit register settings for 5 Mbps**

Version history:
(1):

Purpose: Support CAN FD with 5 Mbps data rate
Source:
Verification Method: Analysis AND Test
Legacy ID:

The following standardized bit rate settings shall be used.

CAN clock	SP (DATA)	SSP (DATA)	BRP	Tq Per bit (DATA)	Sync_Seg [tq]	Prop_SegD [tq]	ps1D [tq]	ps2D [tq]	SjwD [tq]	SSP [offset tq]
80MHz	62.5%	62.5%	1	16	1	4	4+1	5+1	5+1	200ns*62.5% =125ns 125ns = 12.5ns*10 10(OCT)=A(HEX)

Other settings may only be used after agreement with the OEM project Base Technology responsible.

4.5.1.3.1.2 Periodic Message Timing Parameter

681216 v2 **Periodic Message Timing Parameter definition**

Version history:
(2):

Purpose: To define the message period
Source:
Verification Method: Test
Legacy ID:

The periodic message is transmitted on a periodic basis only at the periodic time interval T, and it is applicable for the component.

Name	Unit	Min	Nom	Max	Comments
T	ms	0.97×T	T	1.03×T	T ≥ 100ms
		0.9×T	T	1.1×T	10ms ≤ T < 100ms

Note: The deviation should be approved by the bastech team.

The periodic message is transmitted on the real cluster of the vechicle, the allowed period interval can be a more T.

4.6 Communication_Ethernet

4.6.1 Communication_Ethernet_Requirement_Specification

4.6.1.1 General

•Definitions & Abbreviations

4.6.1.1.1 Definitions & Abbreviations

Abbreviation	Stands for	Description
DLC	Diagnostic Link Connector	-
DoIP	Diagnostic communication over Internet Protocol	-
DPR	Design Prerequisites	-
EMC	Electromagnetic Compatibility	-
EMI	Electromagnetic Immunity	-
ESR	Equivalent Series Resistance	-
IOL	InterOperability Laboratory	-
PCB	Printed Circuit Board	-
PLL	Phase Locked Loop	-
PPM	Parts Per Million	-
SAE	Society of Automotive Engineers	-
SFTP	Shielded Foil Twisted Pair	-
UTP	Unshielded Twisted Pair	-
CIDM	Characteristic Impedance Differential mode	-
CMC	Common Mode Choke	-
EDS	Electrical Distribution System	-
STP	Shieled Twisted Pair	-
AVB	Audio/Video Bridged Networks	-
CRC	Cyclic Redundancy Check	An error-detecting code commonly used in digital networks and storage devices to detect accidental changes to raw data
DEI	Drop Eligible Indicator	-
DID	Data Identifier	-
ECU	Electric Control Unit	-
EEPROM	Electronically Erasable Programmable Read Only Memory	-
gPTP	generalized Precision Time Protocol	Is part of the IEEE Audio Video Bridging (AVB) group of standards. It specifies a profile for use of IEEE 1588-2008 for time synchronization over a virtual bridged local area network (as defined by IEEE 802.1Q).
IEEE	Institute of Electrical and Electronics Engineers	-

IP	Internet Protocol	IP is a protocol used for communicating data across a packet-switched internetwork using the Internet Protocol Suite
ISO	International Standards Organization	-
LAN	Local Area Network	-
MAC	Media Access Control	-
MTU	Maximum Transmission Unit	-
OPEN	One Pair Ethernet	-
OSI	Open Systems Interconnect	-
OUI	Organizationally Unique Identifier	-
PCP	Priority Code Point	-
PTP	Precision Time Protocol	A protocol used to synchronize clocks throughout a computer network
QoS	Quality of service	A resource reservation control mechanism to ensure a guaranteed performance of a data flow
SQI	Signal Quality Indicator	-
SWDL	Software Download	-
TCAM	Ternary Content-addressable memory	-
TCP	Transmission Control Protocol	TCP is one of the core protocols of the Internet Protocol Suite
UDP	User Datagram Protocol	UDP is one of the core members of the Internet Protocol Suite
VLAN	Virtual Local Area Network	VLAN is a group of hosts with a common set of requirements that communicate as if they were attached to the same broadcast domain, regardless of their physical location
VID	VLAN Identifier	-
ACK	Acknowledgment	A packet message used in TCP to acknowledge receipt of a packet.
ARP	Address Resolution Protocol	ARP is a computer networking protocol for determining a hardware address when only its IP address is known.
DA	Destination Address	The destination physical address is the Link Layer Address to which the packet was sent.
IETF	Internet Engineering Task Force	The IETF develops and promotes Internet standards.
MAC address	Media Access Control address	MAC address is a identifier assigned to network interfaces for communications on the physical network segment.

RFC	Request For Comments	In computer network engineering, a RFC is a memorandum published by the IETF describing methods, behaviours, research, or innovations applicable to the working of the Internet and Internet-connected systems.
SA	Source Address	
SWS	Silly Window Syndrome	Silly window syndrome is a problem in computer networking caused by poorly-implemented TCP flow control.
TTL	Time-To-Live	A counter that indicates how long a packet can live in the network. It decreases for each hop until reached 0 at which point it is dropped.
WAN	Wide Area Network	The global internet
API	Application Programing interface	-
RPC	Remote Procedure Call	-
SOME/IP	Scalable service-Oriented MiddlewarE over IP	-
SWFA	SOME/IP Wait For Acknowledgment	-
SWFR	SOME/IP Wait For Response	-
AVC	Advanced Video Coding	-
FQTSS	Forwarding and Queueing for Time-Sensitive Streams	-
IHU	Infotainment Head Unit	-
RTCP	RTP Control Protocol	-
RTP	Real-time Transport Protocol	-
SR	Stream Reservation	-
SSRC	Synchronization Source	-
ARL	Address Resolution Logic	-
CPU	Central Processing Unit	-
DHCP	Dynamic Host Configuration Protocol	-
ICMP	Internet Control Message Protocol	-
NM	Network Management	-
OBD	On-board diagnostics	-
SVL	Shared VLAN Learning	-
SRP	Stream Reservation Protocol	-
TPID	Tag Protocol Indentifier	-
UDS	Unified Diagnostic Services	-

Definition	Description
Brownout	Temporary voltage drop. Compare with a ‘blackout’, which refers to the total loss of power.
Cat 5e	Category 5e cable. Fast Ethernet is specified to use this cable type. The cable consists of 4 twisted pairs specified for usage up to 100 MHz.
Connector	In this specification ‘connector’ means the complete connection of the ECU and

	cable.
ECU	Electronic Control Unit. An electronic component, normally containing one or more micro controllers, used to measure and control sensors and actuators. Sensors and actuators can be of different types. Sensors may be, but not limited to, switches, knobs and cameras. Actuators may be, but not limited to, electric engines, relays and monitors. Same as node.
Edge Node	An ECU, where the Ethernet activation line as defined in ISO 13400-3 is terminated.
External Diagnostic Interface	Interface that is part of the diagnostic link connecting the Edge Node with the Diagnostic Link Connector.
Fast Ethernet	Ethernet according to 100BASE-TX in Ethernet, Ethernet Physical Layer - External publications.
HP Auto – MDI/MDI-X	Technology invented and standardized by Hewlett Packard that automatically detects whether a straight or crossover cable is connected and compensates accordingly.
Internal Interface	Interface that is not part of the diagnostic link connecting the Edge Node with the Diagnostic Link Connector.
Jack	Female part of the connector.
MAC	Media Access Control, OSI layer 2, implemented either together with the PHY in an Ethernet controller or as a component in a micro controller. In case of an implementation separated from the PHY it typically communicates with the transceiver over a MII interface.
MII	Media Independent Interface, standardized interface that enables the ease of changing physical layer, OSI layer 1, without affecting the implementation of the upper layers. The MII interface is the first interface being developed for Fast Ethernet. The MII transfers 4 bit nibbles at 25MHz when the Ethernet traffic is at 100 MBit/s.
MLT-3	Multi Level Transmit coding in three levels. A coding technique developed for FDDI on copper. The FDDI spec ANSI X3.263: 1995 (TP-PMD), is referenced in Ethernet, Ethernet Physical Layer - External publications.
Multiplex	To interleave or simultaneously transmit two or more messages on a single channel.
Network	A set of electronic and cabling devices facilitating the multidirectional exchange of information encoded in serial form. Each electronic device (in this case: ECU) is equipped with a specific, standardized electronic interface in order to guarantee compatibility between exchanged binary items of information.
Network segment	A network segment consists of all components necessary to connect two ECUs to each other i.e. cable and connectors. A network segment may consist of several sub segments.
Network sub segment	If a network segment need to be divided into several parts due to assembly procedures one of these parts is defined as a network sub segment. A network sub segment consists of a cable with mounted connectors.
NEXT	Near End Cross Talk.
PHY	Physical Ethernet interface, OSI layer 1, implemented together with the MAC in an Ethernet controller or separately in a transceiver. In case of a separate transceiver implementation it communicates with the MAC typically using a MII interface. The PHY is typically enabling either electrical or optical interfaces.
Plug	Male part of the connector.
RMII	Reduced Media Independent Interface, the standard is developed from the MII standard and features a lower pin count at the cost of a higher communication rate. The RMII transfers 2 bit at 50 MHz when the Ethernet traffic is at 100 MBit/s.
RX+	Positive part of the differential signaling on the reception port.
RX-	Negative part of the differential signaling on the reception port.
Twisted Pair	Cable type that consists of two separately insulated wires that are twisted together to become balanced. This balanced cable is intended for differential voltage signaling and has good EMI properties which make it suitable for serial communication. By adjusting the twist rate for different pairs further beneficial properties, like reduced crosstalk, are gained.

TX+	Positive part of the differential signaling on the transmit port.
TX-	Negative part of the differential signaling on the transmit port.
AVA	Audio Video Bridging for Automotive, a subset of the AVB specification which refers to synchronization and transfer of multimedia streams only.
BroadR-Reach / 100BASE-T1	Ethernet physical layer designed for use in automotive applications.
Ethernet	A family of computer networking technologies for local area networks (LANs).
Factory Port Settings	Setting where all registers that have been changed by diagnostics and configuration loaded from the EEPROM are restored to the values prior to these changes.
Frame	A complete unit of data transmission as transmitted on the network, including media access control (MAC) layer addressing and cyclic redundancy check (CRC).
IEEE 802.1	Higher Layer LAN Protocols Working Group within IEEE.
IEEE 802.3	Ethernet Working Group within IEEE.
Multicast	Delivery of messages or information to a group of IP devices in a single transmission.
Packet	A formatted unit of data. Packet consists of two kinds of data: control information and user data (payload). Refers to the IEC 61883 definition. Packet is used to refer to specific features of 61883 that use this term, such as source packet header (SPH) or common isochronous packet (CIP).
Power on event	Wake up pattern received on LIN, CAN, FlexRay, etc, with signals to the ECU to power up.
Switch	A device that connects network segments, which operates at the Data link Layer of the OSI model.
Edge device	A host that is located at the outer edge of a network, normally connected via one interface.
Egress	IP network traffic originating from the internal vehicle network and destined for host on the external network.
External network	Networks that are physically located outside the vehicle that the vehicle can connect to or vice versa.
Host	A device that is connected to a network, generally edge devices. In a car a host is any ECU
Internal ECU	Internal vehicle network ECU that only communicate within the vehicle.
IP-ECU	An ECU that is capable of communicating over IP based protocols
Inner tag	Customer tag (C-TAG) according to IEEE 802.1Q.
Ingress	IP network traffic originating from external networks and destined for the IP-ECU's within the vehicle network.
Internal network	The network within the car.
Network Interface	Network interface, or LAN adapter, is a computer hardware component designed to allow computers to communicate over a computer network.
Point-to-point	A connection between two hosts
Port	Physical interface
Subnet	Subnet, or sub network, is a logically visible, distinctly addressed part of a single Internet Protocol network. The process of subnetting is the division of a computer network into groups of computers that have a common, designated IP address routing prefix.
TCP/IP stack	A protocol stack that supports TCP, IP and UDP.
Tester	A system that controls functions such as test, inspection, monitoring, or diagnosis of an on-vehicle electronic control unit and may be dedicated to a specific type of operators (e.g. a scan tool dedicated to garage mechanics or a test tool dedicated to assembly plant agents). The tester is also referenced as the client.
Outer tag	Service tag (S-TAG) according to IEEE 802.1Q.
VLAN tag	Refers to the IEEE 802.1Q header which extends the Ethernet frame with 4 bytes.
Interface definition	Where all messages and their contents are specified.

Non-AUTOSAR ECU	An ECU running an OS other than AUTOSAR. Could be Linux, Windows or other

•Requisite Documents

4.6.1.1.2 Requisite Documents

Reference ID	Doc. Number	Title
[ETH REF 1]	ISO 7498-1	Open Systems Interconnection - basic reference model
[ETH REF 2]	SAE J1213 - 1	Glossary of vehicle networks for multiplexing and data communications
[ETH REF 3]	OA - EMC Definions for Communication Channel 2.0_141128. Version 2.0	BroadR-Reach® Definitions for Communication Channel
[ETH REF 4]	OA - Automotive_Ethernet_ECU_TestSpecification_v1.0	OPEN Alliance Automotive Ethernet ECU Test Specification
[ETH REF 5]	TC9-Gbps-channel-and-components-2.0	Channel and Components Requirements for 1000 BASE-T1 Link Segment Type A
[ETH REF 6]	IEEE 802.3bw-2015	IEEE Standard for Ethernet Amendment 1: Physical Layer Specifications and Management Parameters for 100 Mb/s Operation over a Single Balanced Twisted Pair Cable (100BASE-T1)
[ETH REF 7]	IEEE 802.3bp–2016	Physical Layer Specifications and Management Parameters for 1 Gb/s Operation over a Single Twisted-Pair Copper Cable
[ETH REF 8]	IEEE 802.1Q-2014	Media Access Control (MAC) Bridges and Virtual Bridge Local Area Networks
[ETH REF 9]	IEEE 802.1D-2004	IEEE Standard for Local and Metropolitan Area Networks—Media access control (MAC) Bridges
[ETH REF 10]	IEEE 802.1AS-2011	Timing and Synchronization for Time-Sensitive Applications in Bridged Local Area Networks.
[ETH REF 11]	IEEE 1588-2008	Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems
[ETH REF 12]	IETF RFC3550	IEEE Standard for Local and metropolitan area networks—Bridges and Bridged Networks
[ETH REF 13]	-	AUTOSAR_PRS_SOMEIPProtocol (AUTOSAR FO R19-11)
[ETH REF 14]	-	AUTOSAR_PRS_SOMEIPServiceDiscoveryProtocol (AUTOSAR FO R19-11)
[ETH REF 15]	-	SOME/IP Static Interface Description
[ETH REF 16]	-	Ethernet Physical Layer
[ETH REF 17]	-	Ethernet Data Link Layer
[ETH REF 18]	-	Internetworks General Specification
[ETH REF 19]	-	SRD Diagnostics and ECU Platform
[ETH REF 20]	-	Switch Configuration
[ETH REF 21]	-	VLAN Configuration
[ETH REF 22]	31835908	UDSonIP, Unified diagnostic services (UDS) – Part 5: Unified diagnostic services on IP implementation
[ETH REF 23]	31835909	Bootloader UDSonIP, Bootloader — Unified diagnostic services (UDS) — Part 5: Unified diagnostic services on IP implementation
[ETH REF 24]		LC:ECU Platform System (SRD ECU Platform)

[ETH REF 25]	-	Project specific EMC Component and Subsystem Requirements
[ETH REF 26]	-	Project specific SRD Diagnostics & ECU Platform
[ETH REF 27]		OA TC11- Switch Semiconductor Test Specification
[ETH REF 28]		OA TC1 - 100BASE-T1 Physical Coding Sublayer Test Suite
[ETH REF 29]		OA TC1 - 100BASE-T1 Physical Media Attachment Test Suite
[ETH REF 30]		OA TC1 - 100BASE-T1 PHY Control Test Suite
[ETH REF 31]		OA TC1 - 100BASE-T1 Interoperability Test Suite]
[ETH REF 32]		OA TC10: Sleep/Wake-up Specification
[ETH REF 33]		OA TC12 - 1000BASE-T1 System Implementation Specification
[ETH REF 34]		OA TC1 - 100BASE-T1 System Implementation Specification
[ETH REF 35]		OA TC8 - Automotive Ethernet ECU Test Specification
[ETH REF 36]		Specification of SOME/IP Transformer
[ETH REF 37]		IEEE802.1AE
[ETH REF 38]		OA TC12 - 1000BASE-T1 Physical Media Attachment Test Suite
[ETH REF 39]		OA TC12 - 1000BASE-T1 EMC Test Specification for Common Mode Chokes
[ETH REF 40]		OA TC12 - Advanced diagnostic features for 1000BASE-T1 automotive Ethernet PHYs
[ETH REF 41]		UDS Data - Basetech Data Requirements (internal)
[ETH REF 42]		LC: CommunicationLogSystem

•Requirements

4.6.1.1.3 Requirements

660814 v3 **Diagnostic IP routing algorithm**

Version history:
(3): Revise description.

Purpose: Clarification of how the diagnostic IP routing algorithm shall work.
Source:
Verification Method: Analysis
Legacy ID: 342169v0

A general rule for the Edge Node to be able to decide what kind of diagnostic message which shall be send over the IP-links or other vehicle network (CAN, LIN) is to investigate the IP address and search for the Zone ID.

If the three lowest nibbles of the ECU address matches the three lowest nibbles in an IP address, use this IP address when route/gateway diagnostic messages. Otherwise use the IP address of the Central Control Zone for diagnostic messages. (i.e. IP address that match ECU address [Zone ID].0.1).

Example ECU: DHU (Edge node) has active DoIP TCP connections from [198.18.44.2] & [198.18.44.49]

When a request for 0x1002 is received the following happens: About 0x1002 the first 0 means Zone 0 => IP / Eth interface, Network ID 0 means IP ECU, accroding to REQID 660805, Zone ID 0 and ECU ID 2 match the two lowest nibbles in 198.18.44.2 => use the connection with IP 198.18.44.2.

When a request for 0x1301 is received the following happens: About 0x1301 the first number 3 means Zone 3 => IP / Eth interface, Network ID 0 means IP ECU, accroding to REQID 660805, Zone ID 3 and ECU ID 1 match the two lowest nibbles in 198.18.44.49=> use the connection with IP 198.18.44.49

When a request for 0x1332 is received the following happens: About 0x1332 the first number 3 means Zone 3 => IP / Eth interface, Network ID 3 means the ECU is a subnode connected Zone GW => Try with the Central Control Zone address 0x1301 instead => Zone ID 3 and ECU ID 1 match the two lowest nibbles in 198.18.44.49=> use the connection with IP 198.18.44.49.

When a request for 0x1241 is received, Zone 2 is still IP/Eth, Network ID 4 means the ECU is a subnode connected GW => Try with the Central Control Zone address 0x1201 instead => 0x1201 does not match any active connection => Send DoIP Nack(Unknown Target Address) to tester.

660818 v3 Eth Configuration parameters Update

Version history:
(3):

Purpose: Description on Update Scope and Update Method of Configuration Parameters for Each Layer of ECU's Ethernet Communication.
Source:
Verification Method: Analysis
Legacy ID: 436145v0

IP ECUs shall support updating Ethernet configuration parameters via software download (OTA, UDS, USB flash disk and other approaches). Typical configuration examples for each layer are listed below:

1.Physical Layer: PHY master/slave attribute and related parameters.
2.Data Link Layer: VLAN configuration and related parameters.
3.Network Layer: IP address and related parameters.
4.Transport Layer: UDP port number and related parameters.
5.Application Layer: SOME/IP parameters, DoIP data update configurations, etc.

Refer to BT documents and Ethernet Subsystem Requirements Specification-[ETH REF 24] for complete and detailed configuration parameters.

660817 v2 Link Speed & Duplex

Version history:
(2):

Purpose: To have the link speed pre-configured will decrease the start up time. This requirement will state link speed and duplex for each link.
Source: N/A
Verification Method: Test
Legacy ID: 442629v0

For in-vehicle ECU, the link speed and duplex shall be 100 Mbit/s or 1000 Mbit/s full duplex for all Ethernet links. For out-vehicle ECU connected to OBD, the link speed and duplex shall be 100 Mbit/s full duplex for all Ethernet links.

660819 v1 Mulitcast Addresses

Version history:
(1):

Purpose: To define which multicast addresses that are used in this project.
Source:
Verification Method: Analysis
Legacy ID: 442631v0

The Local Network Control Block (224.0.0.0/24) shall be used for multicast addressing, according to the table below:

Multicast address	Description
224.0.0.1	All ECUs on this subnet
224.0.0.2	All domain masters on this subnet
224.0.0.117-224.0.0.250	Available for project specific multicast use cases. Multicast addresses from this range shall be appointed after discussions the base technology team.

660822 v1 **VLAN Membership**

Version history:
(1):

Purpose: To define which interfaces that are part of each VLAN.
Source:
Verification Method: Analysis
Legacy ID: 442635v0

IP-ECU's with VLAN will be appointed a VID after discussion with the base technology team.

1641 v6 **Zone ID for IP networks**

Version history:
(6): Update the verification from 'analysis' to 'Analysis & Test', to map the subsystem requirement.

Purpose: Clarification of which Domain ID's are dedicated for IP networks.
Source:
Verification Method: Analysis AND Test
Legacy ID: 342167v1

Zone ID 0,1,2 , 3 and D states that this belongs to IP-based communication links.

Note: This requirement is valid for all ECU communicating with IP ECU's.

4.6.1.2 Ethernet Physical Layer - 100BASE-TX

•Introduction

4.6.1.2.1 Introduction

This document is a technical regulation for a switched network and defines the 100BASE-TX physical layer for such a network.
• Ethernet Physical Layer - 100BASE-TX - Generic requirements

4.6.1.2.2 Ethernet Physical Layer - 100BASE-TX - Generic requirements

The figure below shows a principal Ethernet transceiver circuit diagram from the MII/RMII interface to the physical medium. The circuitry requires the transformers with common mode chokes which isolates the chassis ground from the signal ground.

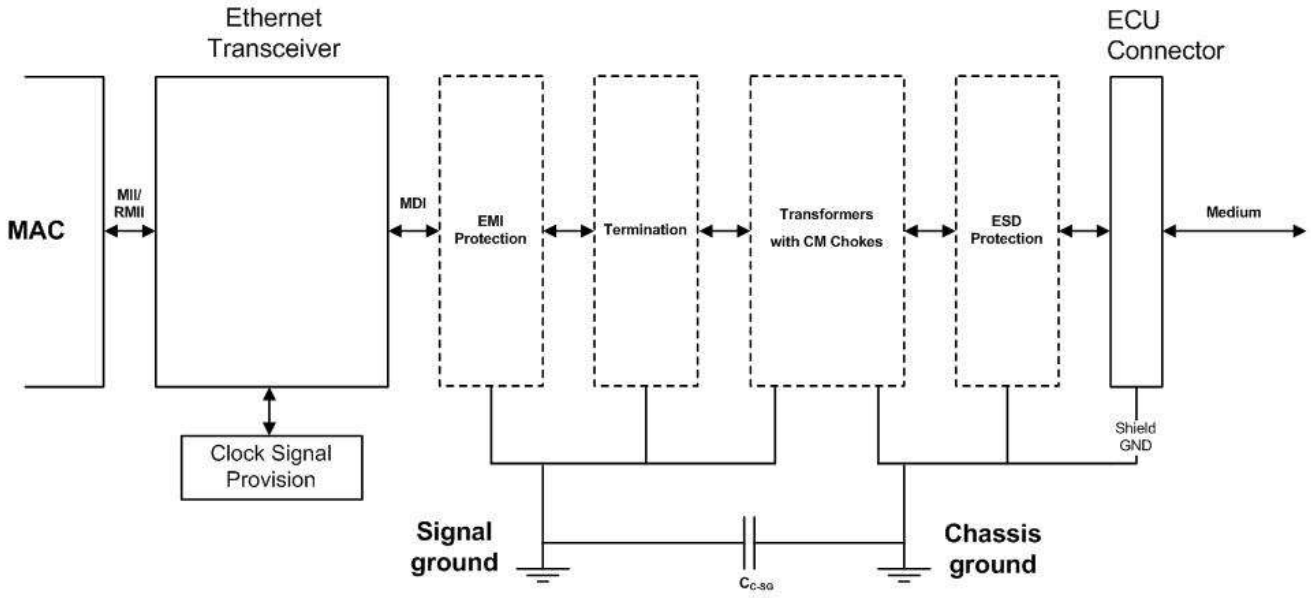


Figure: Principal Ethernet transceiver circuitry.

663228 v1 AEC Approved Components

Version history:
(1):

Purpose: To require that all components are fulfilling the AEC standards.
Source: See description in this requirement
Verification Method: Analysis
Legacy ID: 52525v1

All components used in the Ethernet circuitry shall be qualified according to applicable standard out of documents in *External publications:*

- *Failure Mechanism Based Stress Test Qualification for Integrated Circuits*
- *Stress test qualification for automotive grade discrete semiconductors*
- *Stress test qualification for passive components*

663275 v1 Cable Characteristics

Version history:
(1):

Purpose: To achieve cable electrical characteristics according to the referenced Fast Ethernet standard.
Source: Ethernet, Ethernet Physical Layer - External publications section 25.4.7.1
Verification Method: Analysis
Legacy ID: 52440v1

Cable characteristics shall comply with the electrical parts of the cable specification Category 5e parameters in External Publications, *Ethernet*, Ethernet Physical Layer - External publications

663283 v1 Cable Impedance Measurement Procedure

Version history:

(1):

Purpose: It is important to have a cable that is matched with termination; the method described is stable and easily reproduced.
Source: Not available
Verification Method: Analysis
Legacy ID: 52571v1

The characteristic impedance, propagation velocity, and loss of a cable shall be measured with a vector network analyzer (VNA). Since the twisted pair cable is balanced it is important to connect a VNA balun to the coaxial connector. A balun can be constructed from a 180-degree hybrid. The VNA shall be calibrated at the point where the cable is connected, i.e., the balun is included in calibration The measurement shall be executed as following:

1. Take a cable with a length of approximately 1/8 wavelength at the highest frequency you want to measure. Remember that the propagation velocity is usually around 60-70% of that in free-space. If the cable is a quarter wavelengths long the measurement will not work. The cable can also be shorter than 1/8 wavelength. A 25 cm long cable works well for Ethernet which is measured at 100 MHz.
2. Short circuit the termination end of the cable and measure the complex valued input reflection coefficient
2. $S_{11,sc}$
2. . The complex valued short-circuit impedance is then
$$Z_{sc} = Z_{balun}\frac{S_{11,sc}+1}{S_{11,sc}-1}$$
2.
2. , where
2. Z_{balun}
2. is the reference impedance of the output port of the balun. Use a balun with
2. $Z_{balun} = 100\,\Omega$
3. Open the termination circuit and measure the complex valued input reflection coefficient
3. $S_{11,oc}$
3. . The open-circuit impedance is then
$$Z_{oc} = Z_{balun}\frac{S_{11,oc}+1}{S_{11,oc}-1}$$
3.
3. .
4. The characteristic impedance of the cable we measure is given by
4. $$Z_{cable} = \mathrm{Re}\left[\sqrt{Z_{sc}Z_{oc}}\right]$$
4. , where the Re[·]-function represents the real part.
5. Calculate the complex propagation constant
5. $$\gamma = \frac{1}{L}\tanh^{-1}\sqrt{\frac{Z_{sc}}{Z_{oc}}}$$
5. , where *L* is the length of the cable
6. The attenuation constant is
6. $$\alpha = \mathrm{Re}[\gamma]$$
6. and the phase constant is
6. $$\beta = \mathrm{Im}[\gamma]$$

6. .
7. The wave velocity in the cable is given by
7. $c = \frac{2\pi f}{\beta}$
7. , where *f* is the frequency.
8. The attenuation shall be presented in decibel per kilometer,
8. $A_{dB/km} = 8686 \times \alpha$
8. , and the cable delay as
8. $T = 1/c$

663262 v1 **Cable Length between ECUs**

Version history:
(1):

Purpose: To ensure correct communication maximum cable length between nodes. According to Fast Ethernet may one Ethernet link be maximum 100m.
Source: Ethernet, Ethernet Physical Layer - External publications section 24.1.2
Verification Method: Analysis
Legacy ID: 52449v1

There shall be a furthest wiring length between two nodes according to the table below.
Table: Network speed vs. harness length

Bit Rate of the network	**Network distance between ECUs**
100 Mbit/s	Max 100 m

663276 v1 **Cable Length to be used in Cable Characteristic Impedance Measurement**

Version history:
(1):

Purpose: To define the cable length to be used when measuring the cable characteristic impedance. Measurements shall be made on a cable length which is not close to a ¼ of the wavelengths for frequencies used in the measurements. Cable t0 = 5.5 ns/m and fmax = 100 MHz results in a ¼ wavelength of about 0.5 meters. For Ethernet use a 25 cm cable for determining cable characteristics.
Source: *REQPROD 52571*
Verification Method: Analysis
Legacy ID: 52445v2

Measurements shall be made on a cable length that is 25 cm +/- 2 cm.

663277 v1 **Cable Temperature Span**

Version history:
(1):

Purpose: To ensure that the cable do not prevent proper communication by exposing undesired properties within automotive ambient temperature span.
Source: Ethernet, Ethernet Physical Layer - External publications section 25.4.7.1
Verification Method: Analysis
Legacy ID: 52443v1

Cable characteristics shall comply with cable specification Category 5e parameters in *Ethernet,* Ethernet Physical Layer - External publications for the temperature span. -40 to +125 ºC.

663238 v1 Chassis Ground Plane

Version history:
(1):

Purpose: To require signal ground plane under components.
Source: Not available
Verification Method: Analysis
Legacy ID: 52561v2

An ECU chassis ground plane is required under the chassis grounding components directly beneath the component layer of the PCB. This applies for the following components: ESD protection, capacitors and resistors on the connector side of the transformers with Common Mode Choke and the connector. This requirement may be superseded by REQPROD 52555 .

663239 v1 Chassis Grounding of Components

Version history:
(1):

Purpose: To require component grounding to chassis ground.
Source: Not available
Verification Method: Analysis
Legacy ID: 52559v2

All grounding of the ESD protection, capacitors the connector side of the transformers with Common Mode Choke and the connector shield shall be made to ECU chassis ground. This requirement may be superseded by REQPROD 52555 .

663230 v1 Clock Signal Provision

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 52544v0

The clock signal provision circuit shall be as specified by the transceiver supplier.

663240 v1 Component Positioning on the PCB relative the Transformer

Version history:
(1):

Purpose: The transformer may produce a magnetic field that could affect other components. Carefulness has to be observed when positioning components close to the transformer.
Source: Not available
Verification Method: Analysis
Legacy ID: 52564v2

Components shall not be placed within 25 mm from the transformer without observing possible effects caused by magnetic fields from the transformer. This requirement may be superseded by REQPROD 52555 .

663248 v1 Connected Supply Voltage

Version history:
(1):

Purpose: The regulated power used by the Ethernet circuitry on the PCB shall be used only inside the cover of the ECU and not be connected to any connector pins or similar parts exposing interface to the ECU internal supply voltage without protective circuitry.
Source: Not available
Verification Method: Analysis
Legacy ID: 52549v1

Internally generated supply voltage connected to the transceiver pins shall not be directly connected to any externally available interface.

663263 v1 Connector Allocation Strategy

Version history:
(1):

Purpose: To require the ECU and cable harness to provide connectors necessary for the Ethernet link.
Source: Not available
Verification Method: Analysis
Legacy ID: 52450v1

ECUs shall provide one separate connector for each Ethernet network link that it connects to. Harnesses shall provide one connector at each end point for each network link.

663264 v1 Connector Type Requirements

Version history:
(1):

Purpose: The telecom industry standard connectors of type 8P8C shall not be used, this to make it more difficult to attach foreign Ethernet equipment to the network. The 8P8C contact is commonly known as a RJ45 contact and is specified in Telecommunications - Telephone Terminal Equipment - Technical Requirements for Connection of Terminal Equipment to the Telephone Network, Ethernet Physical Layer - External publications.
Source: Not available
Verification Method: Analysis
Legacy ID: 52452v1

The plugs and jacks of 8P8C type shall not be used.

663237 v2 Consult Transceiver and Transformer Supplier Regarding Circuit Design

Version history:
(2):

Purpose: To require that the supplier of the component is asked for advice regarding circuit design and layout. The advices from the supplier shall be followed as long as requirements regarding functionality and EMI (radiation, susceptibility and immunity) can be guaranteed.

Source: Not available
Verification Method: Analysis
Legacy ID: 52555v3

The supplier of the transceiver and transformer shall be consulted regarding circuit components and layout. The advice by the suppliers supersedes the general design requirements in this specification and shall be implemented, after agreement with GRI Elactrical Architecture, with regards to functionality according to the requirements set by documents:

Project specific EMC Component and Subsystem Requirements, Ethernet Physical Layer - Requisite documents

Ethernet IEEE 802.3-2015 , Ethernet Physical Layer - External publications.

Road vehicles -- Diagnostic communication over Internet Protocol (DoIP) -- Part 3: Wired vehicle interface based on IEEE 802.3 , Ethernet Physical Layer - External publications.

663225 v2 Document Precedence

Version history:
(2):

Purpose: To require precedence for this specification in case of discrepancies between this specification and referenced standards.
Source: See description in this requirement for source
Verification Method: Analysis
Legacy ID: 52388v1

This specification has precedence in case of discrepancy between this specification and the standard documents in *External Publications:*

- *Ethernet, IEEE 802.3-2015*
- *Generic Telecommunications Cabling for Customer Premises*
- *Balanced Twisted-Pair Telecommunications Cabling and Components Standard*
- *Telecommunications – Telephone Terminal Equipment – Technical Requirements for Connection of Terminal Equipment to the Telephone Network*
- *ANSI X3.263: 1995 (TP-PMD)*
- *Road vehicles -- Diagnostic communication between test equipment and vehicle over internet protocol -- Part 3: IEEE802.3 based wired vehicle interface*

663270 v1 Duplex

Version history:
(1):

Purpose: To require the use of components capable of communicating with full duplex throughout the system. Half duplex components will reduce the bandwidth to half of full duplex, which is not acceptable.
Source: Not available
Verification Method: Test
Legacy ID: 52519v1

The ECUs of the Fast Ethernet network shall be able to provide full duplex as communication mode. Half duplex shall not be used.

663249 v2 ECU Behavior Outside of Operating Voltage Range and Special Conditions

Version history:
(2): 1.Revise voltage to 18V.

Purpose: To require the ECU to not disturb the Ethernet network outside of the operating range or during special operating conditions. It is recommended that ECUs continue to communicate outside the voltage operating range as long as they have a correct behavior.
Source: Not available
Verification Method: Test
Legacy ID: 52552v4

ECUs shall not send corrupt messages, or disrupt the transmission of the other ECU outside of its specified voltage operating range, by partial or complete ECU startup, shutdown, reset, brownout or comparable events. There must be margins in both ends of the operating voltage range in regards to the voltage level where communication becomes unreliable or is no longer possible due to hardware constraints. The hysteresis shall be at least 100 mV. See Figure Operating voltage range.

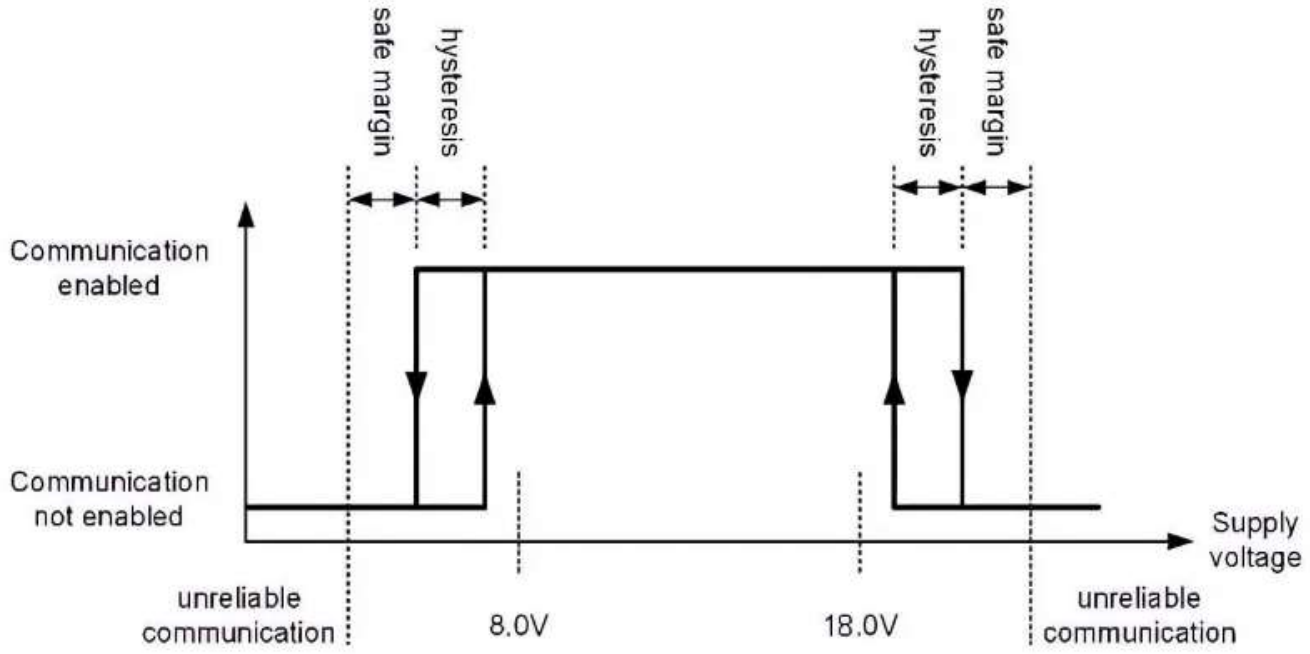


Figure - Operating voltage range.

663233 v1 ECU Connector Electrical Characteristics

Version history:
(1):

Purpose: The ECU connector shall comply with the standard requirements in Ethernet, Ethernet Physical Layer - External publications and the requirements in "Generic Telecommunications Cabling for Customer Premises" and "Balanced Twisted-Pair Telecommunications Cabling and Components Standard", which are referenced by Ethernet, Ethernet Physical Layer - External publications
Source: Ethernet, Ethernet Physical Layer - External publications, section 25.4.7.1
Verification Method: Analysis
Legacy ID: 52529v1

The ECU connector shall comply with the electrical requirements in *Ethernet*, Ethernet Physical Layer - External publications.

663234 v1 ECU Connector Shield Attached to Chassis Ground

Version history:
(1):

Purpose: To support the fulfilment of EMC requirements.
Source: Figure Physical Layer Circuit found in section Physical Layer Circuit
Verification Method: Analysis
Legacy ID: 52527v6

The cable shield shall via the ECU connector be connected to chassis ground on the PCB. The impedance between the shield and the chassis shall be less than 0.1 ohm for frequencies up to 1 GHz.

663231 v2 ECU Ethernet Conformance

Version history:
(2):

Purpose: To require that the ECU is conformant to the Ethernet 100BASE-TX standard. The ECU supplier will have to provide test results of the applicable test cases. It is not required that new tests are performed for test cases that the ECU supplier can derive from the conformance tests made by the transceiver supplier.
Source: Ethernet, Ethernet Physical Layer - External publications
Verification Method: Analysis
Legacy ID: 52566v2

The ECU shall conform to applicable parts for Fast Ethernet of *Ethernet*, Ethernet Physical Layer - External publications. To prove conformance, the ECU shall pass the applicable test suites according to the table below performed by IOL.
All test results must be provided to GIR and will be kept confidential.

Table : IOL Test Suits

Test Suite	Type of ECU
Clause 4 Media Access Control (MAC) Test Suite	All
Clause 25 100BASE-TX PMD Test Suite	All
Clause 24 100BASE-X PCS Test Suite	All
Clause 31 Flow Control Test Suite	All
Clause 27 100BASE-X Repeater Test Suite	None
Clause 40 Auto-Crossover Test Suite	Edge Node
Clause 28 Auto-Negotiation State Machine Test Suite	Edge Node
Clause 28 Auto-Negotiation Next Page Test Suite	Edge Node
Clause 28 Auto-Negotiation Management System Test Suite	Edge Node
Clause 28 Auto-Negotiation Management Register Test Suite	Edge Node
Clause 25 100BASE-TX PMD-EEE Test Suite	All

663236 v1 EMI Protection

Version history:
(1):

Purpose:
Source:

Verification Method: Analysis
Legacy ID: 52543v0

The EMI protection circuit shall be as specified by the transceiver supplier.

663259 v1 Error Handling

Version history:
(1):

Purpose: To require standard Ethernet error handling. There are transceivers that contain functionality beyond what is specified in the standard; if possible this functionality shall be used to diagnose the errors.
Source: Ethernet, Ethernet Physical Layer - External publications
Verification Method: Analysis
Legacy ID: 52569v1

The transceiver shall be able to support the requirements in *Ethernet*, Ethernet Physical Layer - External publications . If the transceiver contains further diagnostic capabilities the possibility to utilize this functionality shall be investigated. A decision whether to use the extra functionality or not shall be made together with the department issuing this document.

663226 v2 Ethernet Network

Version history:
(2):

Purpose: To reference specification applicable for the network. The applicable parts of the specification for 100BASE-TX are located, according to the introduction in section1, section2 and section5.
Source: Ethernet, Ethernet Physical Layer - External publications
Verification Method: Analysis
Legacy ID: 52387v1

The Fast Ethernet Network 100BASE-TX shall be implemented in accordance with *Ethernet,* Ethernet Physical Layer - External publications.

663274 v1 Ethernet Network Topology

Version history:
(1):

Purpose: To require repeater less network topologies according to Ethernet, Ethernet Physical Layer - External publications. Repeaters, also known as hubs, shall not be used on the network segment. If several point to point network segments shall be connected to each other they shall connect through a switch. A hub will reduce the communication speed to half-duplex. The switch will add some latency to the network.
Source: Not available
Verification Method: Analysis
Legacy ID: 52516v1

The Fast Ethernet network segments shall be point to point only. Network segments end points may be connected to a switch to form the Fast Ethernet network.

663268 v1 Ethernet Signal Crossover

Version history:

(1):

Purpose: To ensure correct communication cross over cabling between ECUs connecting TX+/TX- to RX+/RX- and vice versa.
Source: Not available
Verification Method: Analysis
Legacy ID: 52447v1

For each network segment the signal crossover shall be according to the table below. This crossover shall be implemented in the harness and not in the ECUs.

Table: Signal crossover conducted by the cable

Wires	ECU1	ECU2
Pair 1+	TX+	RX+
Pair 1-	TX-	RX-
Pair 2+	RX+	TX+
Pair 2-	RX-	TX-

663253 v1 Ethernet Transceiver Fast Ethernet Compliance

Version history:
(1):

Purpose: To require transceivers to be approved.
Source: Ethernet, Ethernet Physical Layer - External publications
Verification Method: Analysis
Legacy ID: 52536v2

All Ethernet transceivers shall be fully Ethernet compliant according to applicable parts of *Ethernet*, Ethernet Physical Layer - External publications in External Publications. All ECU suppliers shall obtain the Ethernet compliance certificate from their chosen semiconductor supplier, and provide this upon request.

663281 v1 External Clock Source for the Transceiver

Version history:
(1):

Purpose: To guarantee Ethernet operation frequency when the transceiver frequency is controlled by an external source.
Source: Not available
Verification Method: Analysis
Legacy ID: 52430v2

When the Ethernet transceiver is clocked by an external clock source, transmission and reception shall only be possible when the external clock source is providing a clock signal within the tolerances defined in REQPROD 52427 .

663271 v1 Harness Compliance

Version history:
(1):

Purpose: To show that the harness is conformant with the requirements necessary to realize a Fast Ethernet network.
Source: Ethernet, Ethernet Physical Layer - External publications section 25.4.7.1

Verification Method: Analysis
Legacy ID: 52438v1

The harness shall, for each network segment, be verified to be compliant with *Ethernet*, Ethernet Physical Layer - External publications.

663272 v1 Harness Design

Version history:
(1):

Purpose: To require that the design of the harness is regarding the 100BASE-TX requirements on a system level so that the harness suppliers can implement the harnesses according to the requirements.
Source: Ethernet, Ethernet Physical Layer - External publications section 24.1
Verification Method: Analysis
Legacy ID: 52435v1

The harness design shall comply with the system Fast Ethernet requirements, *Ethernet*, Ethernet Physical Layer - External publications

663227 v1 ISO 13400 - 3

Version history:
(1):

Purpose: The ISO 13400 standard is a new vehicle standard which enables diagnostic communication with the vehicle over the internet protocol. This is more informally called DoIP, Diagnostic over Internet Protocol. ISO 13400 - 3 covers the Physical Layer and the Data Link Layer requirements of the standard.
Source: Road vehicles -- Diagnostic communication between test equipment and vehicle over internet protocol -- Part 3: IEEE802.3 based wired vehicle interface. Date: 2010-09-13, Ethernet Physical Layer - External publications.
Verification Method: Analysis AND Test
Legacy ID: 52425v3

ISO 13400 – 3 shall be implemented according to *Road vehicles -- Diagnostic communication over Internet Protocol (DoIP) -- Part 3: Wired vehicle interface based on IEEE 802.3 ,* Ethernet Physical Layer - External publications. If ISO 13400 -3 is updated after the release of this specification, the changes in ISO 13400 -3 shall be implemented after agreement with the department issuing this document.

663265 v1 Jack Connectors

Version history:
(1):

Purpose: The specification prohibits the use of telecom industry standard 8P8C connectors. This requirement ensures compliance with Fast Ethernet, from electrical point of view, by specifying the female part of the connector attached to the wire harness. It is desirable to minimize the un-twist length as much as possible, maximum 13 mm, for improved return loss and NEXT performance. The guidelines for the termination on a jack (female) connector is included in TIA 568-C.0 (Generic Telecommunications Cabling for Customer Premises, Ethernet Physical Layer - External publications).
Source: Ethernet, Ethernet Physical Layer - External publications section 25.4.7.1
Verification Method: Analysis
Legacy ID: 52454v1

A jack connector terminating a network sub segment shall follow all electrical requirements defined in *Ethernet,* Ethernet Physical Layer - External publications

663260 v1 Link Auto Negotiation

Version history:
(1):

Purpose: It is not possible to compensate for permanent or transient link issues by reducing the bit rate to 10 Mbit/s and/or entering half duplex mode. Because of that there is no reason to allow or use the Link Auto Negotiation in a vehicle.
Source: Not available
Verification Method: Test
Legacy ID: 52520v2

The ECUs of the Fast Ethernet network shall not use auto negotiation with regards to bit rate and duplex.

663273 v1 MDI - All ports

Version history:
(1):

Purpose: To set MDI on all ports and removing the need of Auto HP MDI/MDI-X negotiation. Auto HP MDI/MDI-X negotiation increases the time to set up the link. Cross over is done in the harness
Source: Not available
Verification Method: Test
Legacy ID: 52521v3

The ECUs of the Fast Ethernet network shall use MDI on transceivers connected to vehicle internal network segments. This is valid for single port PHYs as well as ports integrated in a switch.

663247 v2 MII/RMII Interface

Version history:
(2): Revise a typo in the title.

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 52548v1

The MII/RMII interface circuit shall be as specified by the transceiver supplier.

663257 v1 Matching Transformer with Transceiver

Version history:
(1):

Purpose: Make sure that the transceiver and transformer is well matched.
Source: Not available
Verification Method: Analysis
Legacy ID: 52535v1

The transceiver supplier shall be asked for advice regarding selection of transformer component. The transceiver

suppliers advise shall be implemented with the requirement that the transformer shall be specified for automotive use and fulfill *Ethernet*, Ethernet Physical Layer - External publications

663278 v1 Method for Measurement of Cable Characteristic Impedance Measurement

Version history:
(1):

Purpose: To require a verified stable and reproducible measurement method.
Source: Not available
Verification Method: Analysis
Legacy ID: 52444v2

Cable characteristic measurements shall be done with a balanced measurement method according to REQPROD 52571 .

663269 v1 Network Sub Segment Signal Crossover Strategy

Version history:
(1):

Purpose: To ensure correct communication cross over cabling between ECUs connecting TX+/TX- to RX+/RX- and vice versa.
Source: Not available
Verification Method: Analysis
Legacy ID: 52448v2

If network segments consist of several network sub segments must an odd number of network sub segments be crossover cables.

663261 v1 Nominal communication bit rate

Version history:
(1):

Purpose: To require the network to communicate at 100 Mbit/s
Source: Ethernet, Ethernet Physical Layer - External publications section 24.1
Verification Method: Test
Legacy ID: 52432v1

The nominal bit rate of the communication link shall be 100 Mbit/s.

663250 v2 Operating Voltage Range

Version history:
(2): 1.Revise voltage to 18V.

Purpose: To require functionality as a function of ECU external supply voltage. It is recommended that all ECUs on the all Ethernet network segment links have the same bus availability requirement. ECU functional requirements may require a larger operating range.
Source: Not available
Verification Method: Test
Legacy ID: 52551v2

The Ethernet network shall be fully functional over an operating range of 8 to 18 volts.

663266 v1 Plug Connectors

Version history:
(1):

Purpose: The specification prohibits the use of telecom industry standard 8P8C connectors. This requirement ensures compliance with Fast Ethernet, from electrical point of view, by specifying the male part of the connector attached to the wire harness. The orientation of the wires in a plug connector must be tightly controlled and varies according to the plug design. Plugs should be terminated in a controlled repeatable environment and tested for performance according to the requirements in TIA 568-C.2 (see Balanced Twisted-Pair Telecommunications Cabling and Components Standard in Ethernet Physical Layer - External publications). These tests are to guarantee the NEXT performance of mated connections.
Source: Ethernet, Ethernet Physical Layer - External publications section 25.4.7.1
Verification Method: Analysis
Legacy ID: 52512v1

A plug connector terminating a network sub segment shall follow all electrical requirements defined in *Ethernet,* Ethernet Physical Layer - External publications.

663251 v1 Remote Wake Up

Version history:
(1):

Purpose: Remote Wake up, also called Wake-on-LAN has some disadvantages. The MAC address must be known for the device that shall be woken up and the processing of the Ethernet frame at the receiver side increases the current consumption.
Source: Not available
Verification Method: Test
Legacy ID: 52553v1

Remote wake up shall not be implemented.

663279 v1 Shielded Cable - Internal Interfaces

Version history:
(1):

Purpose: To require a dense shield for cables that connects internal interfaces to prevent EMI disturbances. The requirement is a braid and foil screened twisted pair cable normally referred to as SFTP cable. Tests have shown that the visual coverage given below is sufficient to prevent EMI disturbances.
Source: Not available
Verification Method: Analysis
Legacy ID: 52442v2

The cable used to connect internal interfaces shall be shielded by a foil screen with a guaranteed visual coverage of 100% and a braid with a nominal visual coverage of 80% .

663267 v1 Shielded Connector - Internal Interfaces

Version history:
(1):

Purpose: To ensure an uninterrupted shield throughout the full network segment. The shielded connectors are needed for all network segments that connect two ECUs or an ECU with a switch.
Source: Not available
Verification Method: Analysis
Legacy ID: 52453v2

Both the ECU connectors and cable harness connectors for internal interfaces shall be shielded.

663235 v1 Shielding Strategy

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 355316v0

The shield of the connector normally needs to be connected directly to ground in both ends of the link to get a robust connection with fault free data transfer and fulfilment of stipulated EMC requirements. A shielded connection needs to be analyzed to ensure that a loss of ECU ground connection or internal ground loss will not cause thermal damages due to excessive and unintended direct current flow. Other strategies as multiple ground path, robustification of existing ground path, current sense cut of(Over Current Protection Device), that could minimize risk of harmful effects should be evaluated. A ground loss in either end of the shield must not cause uncontrolled or unpredictable functional behaviour.

How final solution shall be implemented is dependent of the construction of the ECUs in both ends of the connection and needs to be handled in a System FMEA where this link is included in the analysis.

663241 v1 Signal Ground Plane

Version history:
(1):

Purpose: To require signal ground plane under components.
Source: Not available
Verification Method: Analysis
Legacy ID: 52560v2

An ECU signal ground plane is required under the components grounded to signal ground directly beneath the component layer of the PCB. This applies for the following components: Ethernet transceiver, capacitors and resistors of the termination, the transceiver side of the transformers with Common Mode Choke and the optional EMI suppression capacitors. This requirement may be superseded by REQPROD 52555 .

663242 v1 Signal Grounding of Components

Version history:
(1):

Purpose: To require component grounding to signal ground.
Source: Not available
Verification Method: Analysis
Legacy ID: 52558v1

All grounding of the Ethernet transceiver, capacitors of the termination, the transceiver side of the transformers with Common Mode Choke and the optional EMI suppression shall be made to ECU signal ground. This requirement may be superseded by *REQPROD 52555.*

663243 v1 Signal Line Routing on PCB

Version history:
(1):

Purpose: To require how signal line routing is done on the PCB.
Source: Not available
Verification Method: Analysis
Legacy ID: 52565v2

TX+ /TX- and RX+/RX- are two signal line pairs. The signal lines of a pair shall be routed side-by-side and each signal line of a pair shall equal in length. This requirement may be superseded by REQPROD 52555 .

663244 v1 Signals to Chassis Ground

Version history:
(1):

Purpose: ESD protection between signal and chassis ground.
Source: Not available
Verification Method: Analysis
Legacy ID: 52911v2

A filtering capacitor, $\mathbf{C}_{\mathbf{S\text{-}CGnd}}$, shall separate the signal and chassis grounds. This requirement may be superseded by REQPROD 52555 .

663252 v1 Termination

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 52542v0

The termination and filtering circuit shall be as specified by the transceiver supplier .

663255 v1 Transceiver EMC Conformance

Version history:
(1):

Purpose: To require conformance to EMC specification
Source: Project specific EMC Component and Subsystem Requirements, Ethernet Physical Layer - Requisite documents
Verification Method: Analysis
Legacy ID: 52539v2

The transceiver shall be conformant to EMC according to the *Project specific EMC Component and Subsystem Requirements,* Ethernet Physical Layer - Requisite documents for the project where the transceiver will be used.

663256 v2 Transceiver Interoperability Testing

Version history:
(2):

Purpose: To require that the transceiver has passed the Ethernet interoperability tests.
Source: Not available
Verification Method: Analysis
Legacy ID: 52538v1

The transceiver supplier shall contract with IOL at the University of New Hampshire to perform the interoperability testing. Contact information for IOL is as follows:

InterOperability Laboratory http://www.iol.unh.edu/

All test results must be provided to GRI and will be kept confidential.

663282 v1 Transceiver Oscillator and External Clock Tolerance

Version history:
(1):

Purpose: To guarantee Ethernet operation frequency when the transceiver frequency is controlled by an oscillator or an external clock. The total deviation for the transmitter clock frequency shall be within ± 100 ppm, however, it is recommended, by transceiver suppliers to set the deviation requirement for the external clock source to be within ± 50 ppm. This is recommended since the transceiver might be relatively sensitive for the external clock. The supplier of the transceiver shall be asked for advice with regards to required precision.
Source: Not available
Verification Method: Analysis
Legacy ID: 52427v1

Tolerance for the oscillator or external clock used with the transceiver shall be within ± 50 ppm (i.e. 0.005%). The tolerance for oscillators shall include contributions from manufacturing, ageing and temperature.
The transceiver supplier shall be consulted with regards to the selected timing device and the transceiver supplier recommendations shall be implemented.

663245 v1 Transceiver Positioning on PCB

Version history:
(1):

Purpose: To add requirement on transceiver positioning on the PCB. The transformer may produce a magnetic field that could affect the transceiver so precaution has to be taken with regards to that.
Source: Not available
Verification Method: Analysis
Legacy ID: 52563v2

The transceiver shall be located at least 25 mm from the transformer.
This requirement may be superseded by REQPROD 52555 .

663254 v1 Transceiver Qualification

Version history:

(1):

Purpose: To require that the transceiver has passed the Ethernet qualification tests.
Source: Not available
Verification Method: Analysis
Legacy ID: 52537v1

All approved transceivers need to pass the following qualification requirements. Once the test documentation has been presented to and accepted by the authors of this document, the transceiver will become a Qualified Transceiver and will be placed on the approved component list, *Components for Physical Interfaces,* Ethernet Physical Layer - Requisite documents . All transceiver revisions must be reported to and reviewed by the department issuing this document. Information will be kept confidential.

663246 v1 Transformer Positioning on PCB

Version history:
(1):

Purpose: To add requirement on transformer positioning on the PCB.
Source: Not available
Verification Method: Analysis
Legacy ID: 52562v2

The transformer shall be located as close to the ECU connector as possible but not further away than 25 mm. This requirement may be superseded by REQPROD 52555 .

663258 v1 Transformer with Common Mode Choke

Version history:
(1):

Purpose: To require the use of a transformer with common mode choke
Source: Not available
Verification Method: Analysis
Legacy ID: 52533v1

Transformer with Common Mode Choke shall be used in all nodes.

663280 v1 Twisted Pair Cable

Version history:
(1):

Purpose: To require a cable with capability to transfer Fast Ethernet.
Source: Ethernet, Ethernet Physical Layer - External publications section 25.4.7.1
Verification Method: Analysis
Legacy ID: 52441v1

The actual multiplex network cable type shall consist of two twisted pairs.

- **Ethernet Physical Layer - 100BASE-TX - Edge Node requirements**

4.6.1.2.3 Ethernet Physical Layer - 100BASE-TX - Edge Node requirements

663287 v1 **Cable Connecting Diagnostic Link Connector and DoIP Edge Node**

Version history:
(1):

Purpose: To ensure that the total link length is not passing the 100 m total link length.
The DoIP specification requires the attached link segment between the diagnostic test equipment and the diagnostic connector to be no more than 50 meters. It is not likely that the in-vehicle cable in a car will be close to the 50 meters that would be necessary to surpass the maximum length required by the Fast Ethernet standard.
Source: Road vehicles -- Diagnostic communication over Internet Protocol (DoIP) -- Part 3: Wired vehicle interface based on IEEE 802.3, Ethernet Physical Layer - External publications
Verification Method: Analysis
Legacy ID: 52659v2

The cables connecting the DLC and the DoIP edge node for Ethernet functionality shall be no longer than 50 meters.

663284 v1 **Diagnostic Edge Node**

Version history:
(1):

Purpose: For the diagnostic edge node shall the connection to the DLC be according to Annex A in *ISO 13400-3.*
Source: Road vehicles - Diagnostic communication over Internet Protocol (DoIP) -- Part 3: Wired vehicle interface based on IEEE 802.3, Ethernet Physical Layer - External publications
Verification Method: Analysis
Legacy ID: 52671v3

The ECU's Ethernet interface towards the DLC shall be implemented according to Annex A in DoIP standard, *Road vehicles -- Diagnostic communication over Internet Protocol (DoIP) -- Part 3: Wired vehicle interface based on IEEE 802.3 ,* Ethernet Physical Layer - External publications.

663288 v1 **Diagnostic Link Connector**

Version history:
(1):

Purpose: A diagnostic connector for DoIP use is not yet specified by ISO, but in an informative chapter of ISO 13400 - 3 there is a proposal on how to use the ISO 15031-3 / SAE J1962 connector for DoIP in existing vehicle network architectures. Until the new diagnostic connector is specified the proposal shall be followed.
Source: Road vehicles -- Diagnostic communication over Internet Protocol (DoIP) -- Part 3: Wired vehicle interface based on IEEE 802.3, Ethernet Physical Layer - External publications
Verification Method: Analysis
Legacy ID: 52514v2

A Diagnostic Link Connector shall be implemented according to ISO 13400-3 Annex A in *Road vehicles -- Diagnostic communication over Internet Protocol (DoIP) -- Part 3: Wired vehicle interface based on IEEE 802.3 ,* Ethernet Physical Layer - External publications.

663290 v1 **HP Auto MDI MDI-X on DoIP Edge Node**

Version history:
(1):

Purpose: To allow the use of Auto HP MDI/MDI-X negotiation configuration when communicating with the external tester. Link set up time is not critical. By connecting a tester to the car, complete control over cable and remote end point with regards to cross over and HP Auto MDI/MDI-X is not ensured. Allowing HP Auto MDI/MDI-X will increase the likelihood of setting up a successful connection.
Source: Road vehicles -- Diagnostic communication over Internet Protocol (DoIP) -- Part 3: Wired vehicle interface based on IEEE 802.3, Ethernet Physical Layer - External publications
Verification Method: Test
Legacy ID: 52668v3

The ECU implementing the DoIP edge node interface shall use Auto MDI/MDI-X negotiation according to *Road vehicles -- Diagnostic communication over Internet Protocol (DoIP) -- Part 3: Wired vehicle interface based on IEEE 802.3 ,* Ethernet Physical Layer - External publications on the edge node interface.

663286 v1 Link Auto Negotiation on DoIP Edge Node

Version history:
(1):

Purpose: According to the DoIP standard is the DoIP edge node required to have auto negotiation with regards to link speed and duplex enabled.
Source: Road vehicles -- Diagnostic communication over Internet Protocol (DoIP) -- Part 3: Wired vehicle interface based on IEEE 802.3, Ethernet Physical Layer - External publications
Verification Method: Test
Legacy ID: 52662v2

The ECUs implementing the DoIP edge node interface shall use auto negotiation with regards to bit rate and duplex according to *Road vehicles -- Diagnostic communication over Internet Protocol (DoIP) -- Part 3: Wired vehicle interface based on IEEE 802.3 ,* Ethernet Physical Layer - External publications, on the edge node interface.

663285 v1 No Voltage Transitions below V_inactive

Version history:
(1):

Purpose: To avoid EMC disturbances on the cable between the DLC and the edge node when there is no tester connected to the vehicle.
Source: Road vehicles - Diagnostic communication over Internet Protocol (DoIP) -- Part 3: Wired vehicle interface based on IEEE 802.3, Ethernet Physical Layer - External publications
Verification Method: Test
Legacy ID: 70021v2

No voltage transitions shall occur on the cable between the DLC and the edge node when the voltage level on the activation line is below V_inactive, defined in *Road vehicles -- Diagnostic communication over Internet Protocol (DoIP) -- Part 3: Wired vehicle interface based on IEEE 802.3* , Ethernet Physical Layer - External publications.

663291 v1 Unshielded Cable - External Diagnostic Interface

Version history:
(1):

Purpose: Unshielded cables and connectors need to be used in cases where it isn't possible to connect a shielded cable to the diagnostic link connector. This is the case when the conductors for the shielded cables are too thin to be connected to the receptacle terminals on the diagnostic link

connector.
Source: Not available
Verification Method: Analysis
Legacy ID: 180273v1

The cable used to connect the Edge Node to the diagnostic link connector shall be unshielded.

663289 v1 **Unshielded Connector - External Diagnostic Interface**

Version history:
(1):

Purpose: Unshielded cables and connectors need to be used in cases where it isn't possible to connect a shielded cable to the diagnostic link connector. This is the case when the conductors for the shielded cables are too thin to be connected to the receptacle terminals on the diagnostic link connector.
Source: Not available
Verification Method: Analysis
Legacy ID: 180473v0

Both the ECU connectors and cable harness connectors for the external diagnostic interfaces shall be unshielded.

4.6.1.3 Ethernet Physical Layer - 100BASE-T1

•Introduction

4.6.1.3.1 Introduction

This document is a technical regulation for a switched network and defines the 100BASE-T1 physical layer for such a network.
•Ethernet Physical Layer - 100BASE-T1 - Generic requirements

4.6.1.3.2 Ethernet Physical Layer - 100BASE-T1 - Generic requirements

663186 v1 **100BASE-T1 Network**

Version history:
(1):

Purpose: To reference specification applicable for the 100BASE-T1 network.
Source: *[ETH REF 6]*
Verification Method: Analysis
Legacy ID: 337037v3

The 100BASE-T1 network shall be implemented in accordance with *[ETH REF 6].*

663188 v1 **100BASE-T1 data pairs only in multipin connector**

Version history:
(1):

Purpose: To enhance RF immunity and decrease EMC emissions.
Source: N/A
Verification Method: Analysis
Legacy ID: 435321v1

If there is more than one 100BASE-T1 data pair in a multipin connector, no other signals (power, ground, other differential signals, etc.) shall be connected to that multipin connector.

663187 v1 Automotive Environment Installation

Version history:
(1):

Purpose: To support the "Automotive environment installation" feature in 802.3bw-2015, which is optional in the standard.
Source: *[ETH REF 6]*
Verification Method: Analysis
Legacy ID: 435356v1

The "Automotive environment installation" feature and all its conditional features in *[ETH REF 6]* shall be supported.

663189 v1 Connectors

Version history:
(1):

Purpose: This requirement ensures compliance with100BASE-T1 technology from electrical point of view, by specifying electrical parameters for the connector attached to the wire harness.
Source:
Verification Method: Analysis
Legacy ID: 349897v3

The connectors used in a 100BASE-T1 network shall fulfill the requirements for connectors in *[ETH REF 3].*

663185 v1 Document Precedence

Version history:
(1):

Purpose: To require precedence for this specification in case of discrepancies between this specification and referenced standards.
Source: *[ETH REF 6], [ETH REF 3]*
Verification Method: Analysis
Legacy ID: 337038v3

This specification has precedence in case of discrepancy between this specification and the documents *[ETH REF 6],* and *[ETH REF 3].*

871279 v2 Interoperability

Version history:
(2):

Purpose: To fulfill requirements of IOP.
Source:
Verification Method: Analysis
Legacy ID:

The interaction and interoperability of two PHYs with the same type from the manufactory shall be fulfilled the requirements of [[ETH REF 31]OA TC1 - 100BASE-T1 Interoperability Test Suite].

663190 v1 Multipin connectors

Version history:
(1):

Purpose: To allow multipin connectors as long as they fulfil the electrical requirements as specified for 100BASE-T1.
Source:
Verification Method: Analysis
Legacy ID: 349898v5

A multipin connector used in a 100BASE-T1 network shall fulfill all requirements applicable for a multipin connector in *[ETH REF 3].*

871276 v1 Physical Coding Sublayer

Version history:
(1):

Purpose: To fulfill requirements of PCS.
Source:
Verification Method: Analysis
Legacy ID:

The funtionality of PCS shall be fulfulled the requirements according to [[ETH REF 28]OA TC1 - 100BASE-T1 Physical Coding Sublayer Test Suite].

871278 v1 Physical Media Attachment

Version history:
(1):

Purpose: To fulfill requirements of PMA.
Source:
Verification Method: Analysis
Legacy ID:

The funtionality of PMA shall be fulfulled the requirements according to [[ETH REF 29] OA TC1 - 100BASE-T1 Physical Media Attachment Test Suite]and PHY control operation requirement of [[ETH REF 30]OA TC1 - 100BASE-T1 PHY Control Test Suite].

663191 v1 Pin Assignment in a Multipin Connector.

Version history:
(1):

Purpose: When a multipin connector is used it is important that adjacent pins are populated in the connector in order to minimize the length of the untwist of the cable and thus minimize radiated emissions and increase the robustness for immunity, etc.
Source: N/A
Verification Method: Analysis
Legacy ID: 350456v4

In a multipin connector adjacent pins must be used for each pair of BI_DA+ and BI_DA-.

663192 v1 **Separate Connector**

Version history:
(1):

Purpose: To enhance RF immunity and decrease EMC emissions.
Source: N/A
Verification Method: Analysis
Legacy ID: 435322v0

The connector used for 100BASE-T1 shall not be shared with other differential signals.

663193 v1 **Signal path through connector**

Version history:
(1):

Purpose: To make sure that the length of the signal path through the connector is equal in length.
Source: N/A
Verification Method: Analysis
Legacy ID: 435320v0

The signal path through the connector shall equal in length for BI_DA+ and BI_DA-.

•Ethernet Physical Layer - 100BASE-T1 - ECU requirements

4.6.1.3.3 Ethernet Physical Layer - 100BASE-T1 - ECU requirements

This is an example of an implementation of the physical layer circuit. For the detailed physical interface, recommendations from chosen suppliers of transceiver and CMC circuits needs to be followed. This example may be superseded by REQPROD 350480.

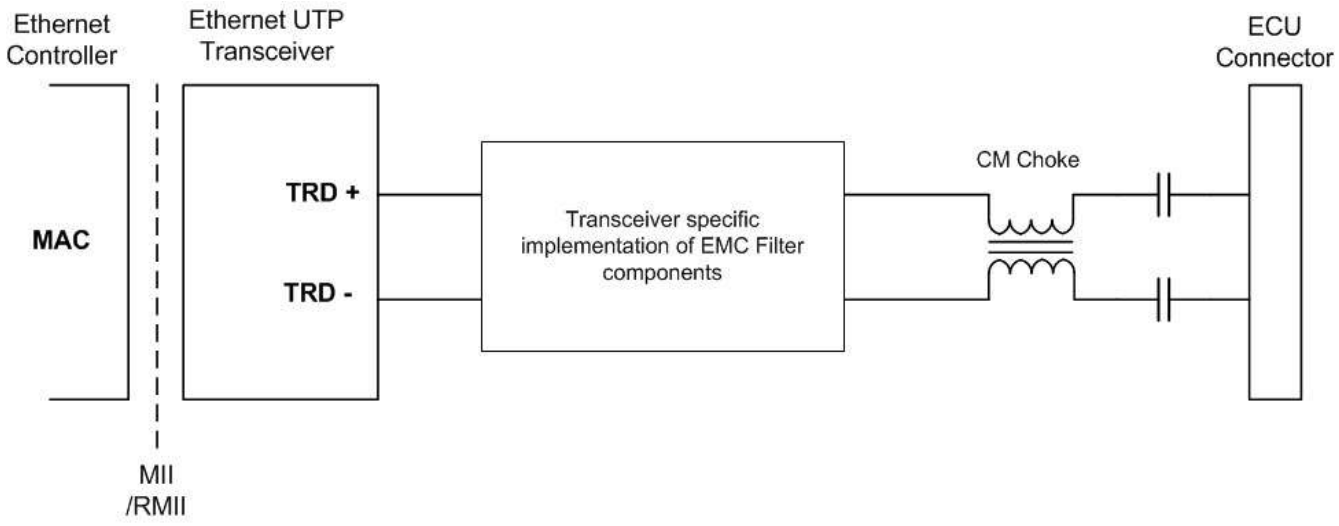


Figure: Physical Layer Circuit

663196 v1 **Automatic Polarity Detection**

Version history:

(1):

Purpose: To minimize the time to set up a link. The function is optional and not needed in a static network.
Source: *[ETH REF 6]*
Verification Method: Test
Legacy ID: 350459v2

The ECUs of the 100BASE-T1 network, shall not use Automatic polarity detection on transceivers connected to vehicle internal network segments.

663200 v1 Clock Signal Provision

Version history:
(1):

Purpose: To require a circuit design and layout that is aligned with the guidelines from the component supplier
Source: N/A
Verification Method: Analysis
Legacy ID: 360501v1

The clock signal provision circuit shall be as specified by the transceiver supplier.

663201 v1 Common Mode Choke

Version history:
(1):

Purpose: To require the use of a common mode choke.
Source: N/A
Verification Method: Analysis
Legacy ID: 350464v2

A Common Mode Choke shall be used for the 100BASE-T1 interface.

663208 v1 Configure Force Mode

Version history:
(1):

Purpose: To minimize the time for the initialization of the link. With "force mode" it is set if the PHY is slave or master.
Source: *[ETH REF 26], [ETH REF 6]*
Verification Method: Analysis
Legacy ID: 357399v6

Each PHY shall be configured to "force mode" as defined in *[ETH REF 6].* If a specific PHY shall be master or slave is specified in *[ETH REF 26].*

663199 v1 Consult Transceiver Supplier Regarding Circuit Design

Version history:
(1):

Purpose: To require a circuit design and layout that is aligned with the guidelines from the component supplier and preserves the functionality and robustness of the link.
Source: N/A
Verification Method: Analysis
Legacy ID: 350480v5

The Tier 1 shall review circuit components and PCB layout together with transceiver manufacturer and the supplier shall provide the documentation of the review. If a deviation is approved as a result of the review, the recommendations by the transceiver supplier will supersede the requirements in the Interface Implementation section of this specification. If a deviation is approved by, the recommendations shall be implemented with the prerequisite that the performance of the link, EMI robustness and EMC emissions for the ECU are preserved or improved.

663209 v2 ECU Behaviour Outside of Operating Voltage Range and Special Conditions

Version history:
(2): 1.Revise voltage to 18V.

Purpose: To require the ECU to not disturb the100BASE-T1 network outside of the operating range or during special operating conditions. It is recommended that ECUs continue to communicate outside the voltage operating range as long as they have a correct behaviour.
Source: N/A
Verification Method: Test
Legacy ID: 350478v3

ECUs shall not send corrupt messages, or disrupt the transmission of the other ECU outside of its specified voltage operating range, by partial or complete ECU start up, shutdown, reset, brownout or comparable events. There must be margins in both ends of the operating voltage range in regards to the voltage level where communication becomes unreliable or is no longer possible due to hardware constraints. The hysteresis shall be at least 100 mV. See Figure Operating voltage range.

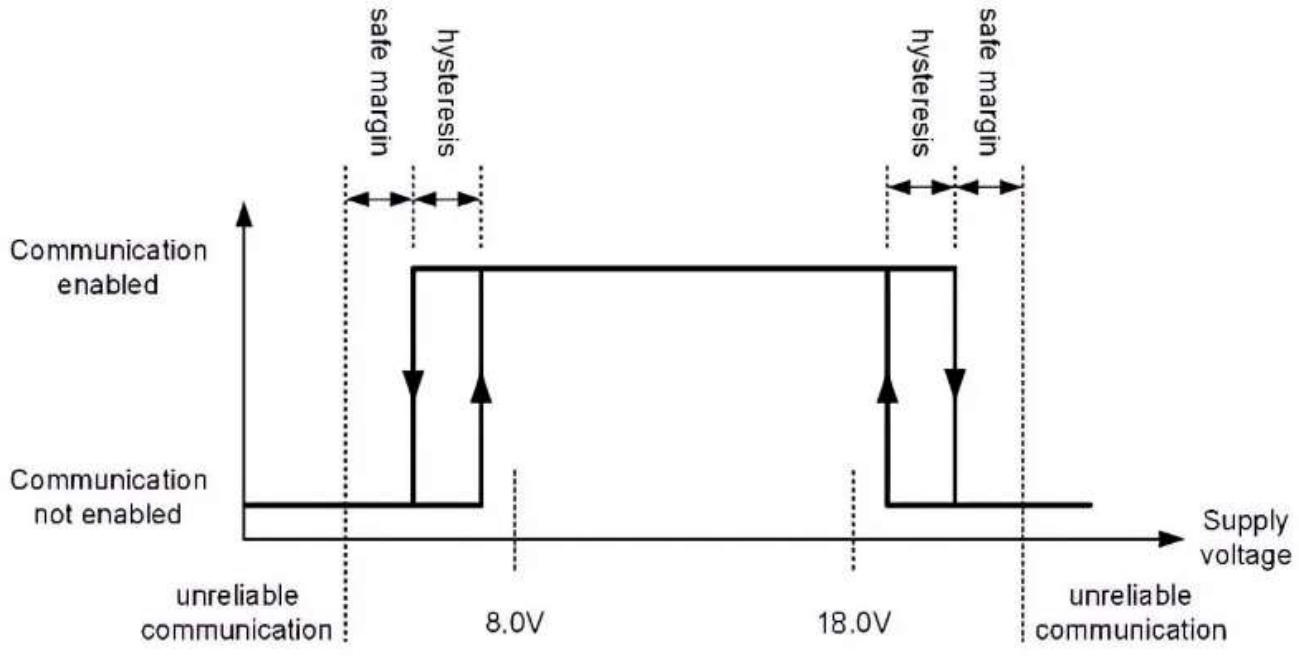


Figure - Operating voltage range.

663197 v1 ECU Conformance

Version history:
(1):

Purpose: To ensure that the ECU is conformant to the 100BASE-T1 standard. The ECU supplier will have to provide test results of the applicable test cases.
Source: *[ETH REF 4]*
Verification Method: Analysis
Legacy ID: 350487v5

The ECU shall prove conformance to 100BASE-T1 by passing all layer 1 test cases in [ETH REF 4]. The test suites shall be performed by an approved external test house. All test results must be provided and will be kept confidential.

663198 v1 EMC Conformance

Version history:
(1):

Purpose: To fulfill EMC requirements.
Source:
Verification Method: Analysis
Legacy ID: 355816v5

The ECU shall be fulfill EMC requirements according to *[ETH REF 25]*.

663203 v1 EMI Protection

Version history:
(1):

Purpose: To require a circuit design and layout that is aligned with the guidelines from the component supplier
Source: N/A
Verification Method: Analysis
Legacy ID: 360499v1

The EMI protection circuit shall be as specified by the transceiver supplier.

871772 v1 Ethernet PHY wake-up

Version history:
(1):

Purpose: To enable Autosar Network Management on Ethernet.
Source:
Verification Method: Analysis AND Test
Legacy ID:

PHY must be compliant with TC10 [[ETH REF 32]: OA TC10: Sleep/Wake-up Specification].

663210 v2 Internal Power Distribution

Version history:
(2): 1.delete FR.

Purpose: To support a short time to start the network.
Source: N/A
Verification Method: Analysis
Legacy ID: 434740v2

All circuitry needed to set up a 100BASE-T1 link shall be powered within 5 ms. The time starts when at least one of the following occurs:

- The ECU is powered.
- A wake up signal that implies that the link is needed is received by the ECU. A wake up signal can be, but is not limited to a wake up signal received via other networks like CAN, a button that is pressed, wake up via a cellular network or wake up via a WiFi Network.

909056 v1 Link Establishment Time

Version history:
(1):

Purpose: To set the time allowed for link establishment.
Source:
Verification Method: Analysis
Legacy ID:

The time from the PHY's powering on to link up shall less than 100ms.

871774 v1 MAC Security

Version history:
(1):

Purpose: To increase the security in the system and to be able to use MACsec we need MACsec support in the HW.
Source:
Verification Method: Analysis
Legacy ID:

The PHY shall have HW support for MAC Security (MACsec) according to [[ETH REF 37] : IEEE802.1AE].

663205 v1 MII and RMII Interface

Version history:
(1):

Purpose: To require a circuit design and layout that is aligned with the guidelines from the component supplier.
Source: N/A
Verification Method: Analysis
Legacy ID: 360498v1

The MII/RMII interface circuit shall be as specified by the transceiver supplier.

663202 v1 Matching CMC with Transceiver

Version history:

(1):

Purpose: Make sure that the transceiver and CMC are well matched.
Source: *[ETH REF 6]*
Verification Method: Analysis
Legacy ID: 350465v5

The Tier 1 shall ask the transceiver supplier for advice regarding selection of CMC component. The transceiver suppliers advise shall be implemented with the requirement that the CMC shall be specified for automotive use and fulfil the requirements in *[ETH REF 6].*

663211 v2 Operating Voltage Range

Version history:
(2):

Purpose: To require functionality as a function of ECU external supply voltage. It is recommended that both ECUs on a 100BASE-T1 network link have the same bus availability requirement. ECU functional requirements may require a larger operating range.
Source: N/A
Verification Method: Test
Legacy ID: 350477v2

The 100BASE-T1 network shall be fully functional over an operating range of 8 to 18 volts.

871787 v1 PHY Integration

Version history:
(1):

Purpose: To ensure that the integration of the PHY is following the Open Alliance standard.
Source:
Verification Method: Analysis
Legacy ID:

The PHY shall be integrated on the PCB according to the reference "system implementation specification" in [[ETH REF 34] OA TC1 - 100BASE-T1 System Implementation Specification].

663206 v1 Signal Line Impedance

Version history:
(1):

Purpose: To ensure impedance matching on the PCB.
Source: N/A
Verification Method: Analysis
Legacy ID: 435348v1

The differential impedance of BI_DA+ and BI_DA- traces shall be 100 ohm +/- 10% at frequencies from 10 MHz up to 100 MHz.

663207 v1 Signal Line Routing on PCB

Version history:
(1):

Purpose: To require how signal line routing is done on the PCB.
Source: N/A
Verification Method: Analysis
Legacy ID: 350486v3

The pair (BI_DA+ and BI_DA-) of signal lines shall be routed side-by-side and each signal line of a pair be shall equal in length. The difference in length shall be < 1 mm. This requirement may be superseded by *REQPROD 350480* .

663204 v1 Termination

Version history:
(1):

Purpose: To require a circuit design and layout that is aligned with the guidelines from the component supplier
Source: N/A
Verification Method: Analysis
Legacy ID: 360500v1

The termination and filtering circuit shall be as specified by the transceiver supplier.

663213 v1 Time Stamping

Version history:
(1):

Purpose: To ensure enough precision for synchronization, this can be achieved either in PHY or in MAC.
Source: *[ETH REF 10]*
Verification Method: Analysis
Legacy ID: 361896v5

It shall be possible to time stamp all frames according to *[ETH REF 10]* with hardware.

663212 v1 Transceiver Oscillator and External Clock Tolerance

Version history:
(1):

Purpose: To guarantee operation frequency when the transceiver frequency is controlled by an oscillator or an external clock.
Source: N/A
Verification Method: Analysis
Legacy ID: 337039v1

The Tolerance of the oscillator or external clock used with the transceiver shall be within ± 100 ppm (i.e. 0.01%).
The tolerance shall include contributions from manufacturing, ageing and temperature.
The transceiver supplier shall be consulted with regards to the selected timing device and the transceiver supplier recommendations shall be implemented.

4.6.1.4 Ethernet Physical Layer - 1000BASE-T1

•Introduction

4.6.1.4.1 Introduction

This document is a technical regulation for a switched network and defines the 1000BASE-T1 physical layer for such a network.

•Ethernet Physical Layer - 1000BASE-T1 - Generic requirements

4.6.1.4.2 Ethernet Physical Layer - 1000BASE-T1 - Generic requirements

663342 v1 **1000BASE-T1 Network**

Version history:
(1):

Purpose: To reference specification applicable for the1000BASE-T1 network.
Source: *[ETH REF 6]*
Verification Method: Analysis
Legacy ID:

The 1000BASE-T1 network shall be implemented in accordance with *[ETH REF 7]*.

663344 v1 **1000BASE-T1 data pairs in multipin connector**

Version history:
(1):

Purpose: To enhance RF immunity and decrease EMC emissions.
Source: N/A
Verification Method: Analysis
Legacy ID:

Multipin connecter shall not be used.

871768 v2 **Advanced Diagnostic Features**

Version history:
(2): Change test to Analysis AND Test.

Purpose: To have access to data from the PHY for fault tracing potential issues with link quality.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The 1000BASE-T1 automotive Ethernet PHY shall offer the following parameters according to Advanced diagnostic features for 1000BASE-T1 automotive Ethernet PHYs in [[ETH REF 40] : OA TC12 - Advanced diagnostic features for 1000BASE-T1 automotive Ethernet PHYs].

- **SQI** - Signal Quality Index - A classification of the signal quality in 8 stages (3 bit) shall be carried out.
- **OS** - OPEN/SHORT detection - Cable harness errors (short circuit or open line) shall be detected.
- **TDR** - SHORT/OPEN/NOISE/DISTANCE detection - Cable harness errors detected by TDR like test approach.

- **LTT** - LinkUp Total Time - The time of the last link training shall be stored.
- **LL** - Link Losses - number of link losses since the last power cycle shall be stored.
- **COM** - communication ready - Optimized Link Status information. Signals when communication is possible.
- **FECC** - FEC Counter.

663343 v1 Automotive Environment Installation

Version history:
(1):

Purpose: To support the "Automotive environment installation" feature in 802.3bp-2016.
Source: *[ETH REF 6]*
Verification Method: Analysis
Legacy ID:

The "Automotive environment installation" feature and all its conditional features in[ETH REF 7] shall be supported.

663345 v1 Connectors

Version history:
(1):

Purpose: This requirement ensures compliance with1000BASE-T1 technology from electrical point of view, by specifying electrical parameters for the connector attached to the wire harness.
Source: N/A
Verification Method: Analysis
Legacy ID:

The connectors used in a 1000BASE-T1 network shall fulfill the requirements for connectors in *[ETH REF 5].*

663341 v1 Document Precedence

Version history:
(1):

Purpose: To require precedence for this specification in case of discrepancies between this specification and referenced standards.
Source: *[ETH REF 7], [ETH REF 5]*
Verification Method: Analysis
Legacy ID:

This specification has precedence in case of discrepancy between this specification and the documents *[ETH REF 7],* and *[ETH REF 5].*

871771 v1 Interoperability

Version history:
(1):

Purpose: To fulfill requirements of IOP.
Source:

Verification Method: Analysis
Legacy ID:

The interaction and interoperability of two PHYs with the same type from the manufactory shall be fulfilled the requirements of [ETH REF 33] : OA TC12 - 1000BASE-T1 Interoperability Test Suite].

871770 v1 **Physical Media Attachment**

Version history:
(1):

Purpose: To fulfill requirements of PMA.
Source:
Verification Method: Analysis
Legacy ID:

The funtionality of PMA shall be fulfulled the requirements according to [[ETH REF 38] OA TC12 - 1000BASE-T1 Physical Media Attachment Test Suite].

663346 v1 **Signal path through connector**

Version history:
(1):

Purpose: To make sure that the length of the signal path through the connector is equal in length.
Source: N/A
Verification Method: Analysis
Legacy ID:

The signal path through the connector shall equal in length for BI_DA+ and BI_DA-.

•Ethernet Physical Layer - 1000BASE-T1 - ECU requirements

4.6.1.4.3 Ethernet Physical Layer - 1000BASE-T1 - ECU requirements

This is an example of an implementation of the physical layer circuit. For the detailed physical interface, recommendations from chosen suppliers of transceiver and CMC circuits needs to be followed. This example may be superseded by REQPROD *209387.*

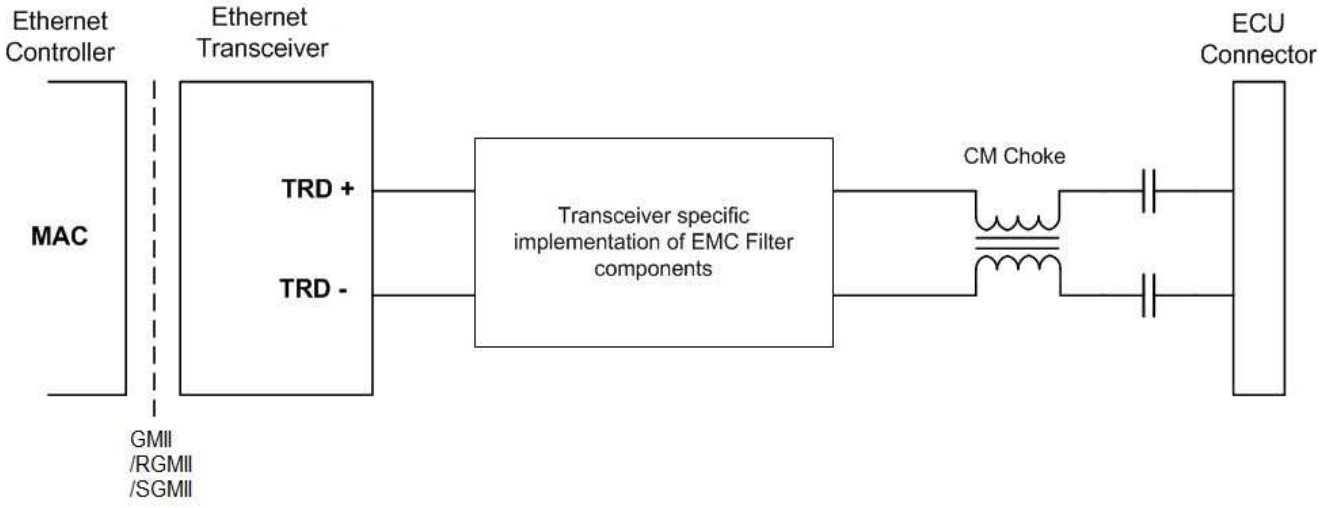


Figure: Physical Layer Circuit

663349 v2 Automatic Polarity Detection

Version history:
(2): Change to should use.

Purpose: It is mandatory to use Auto Polarity Detection and Polarity Correction feature in 802.3bp according to OA - TC12.
Source: *[ETH REF 7]*
Verification Method: Test
Legacy ID:

The ECUs of the 1000BASE-T1 network, shall use Automatic polarity detection on transceivers connected to vehicle internal network segments.

663353 v1 Clock Signal Provision

Version history:
(1):

Purpose: To require a circuit design and layout that is aligned with the guidelines from the component supplier.
Source: N/A
Verification Method: Analysis
Legacy ID:

The clock signal provision circuit shall be as specified by the transceiver supplier.

663354 v2 Common Mode Choke

Version history:
(2): Add Source Spec.

Purpose: To require the use of a common mode choke.
Source: N/A
Verification Method: Analysis

Legacy ID:

A Common Mode Choke shall be used for the 1000BASE-T1 interface and related requirements need to be fulfilled according to [[ETH REF 39]: OA TC12 - 1000BASE-T1 EMC Test Specification for Common Mode Chokes].

663361 v1 Configure Force Mode

Version history:
(1):

Purpose: To minimize the time for the initialization of the link. With "force mode" it is set if the PHY is slave or master.
Source: *[ETH REF 26], [ETH REF 7]*
Verification Method: Analysis
Legacy ID:

Each PHY shall be configured to "force mode" as defined in *[ETH REF 7].* If a specific PHY shall be master or slave is specified in *[ETH REF 26*].

663352 v2 Consult Transceiver Supplier Regarding Circuit Design

Version history:
(2): 1.Add Legacy ID 209387.

Purpose: To require a circuit design and layout that is aligned with the guidelines from the component supplier and preserves the functionality and robustness of the link.
Source: N/A
Verification Method: Analysis
Legacy ID: 209387

The Tier 1 shall review circuit components and PCB layout together with transceiver manufacturer and the supplier shall provide the documentation of the review. If a deviation is approved as a result of the review, the recommendations by the transceiver supplier will supersede the requirements in the Interface Implementation section of this specification. If a deviation is approved by, the recommendations shall be implemented with the prerequisite that the performance of the link, EMI robustness and EMC emissions for the ECU are preserved or improved.

663362 v2 ECU Behaviour Outside of Operating Voltage Range and Special Conditions

Version history:
(2): 1.Revise voltage from 16V to 18v.

Purpose: To require the ECU to not disturb the1000BASE-T1 network outside of the operating range or during special operating conditions. It is recommended that ECUs continue to communicate outside the voltage operating range as long as they have a correct behaviour.
Source: N/A
Verification Method: Test
Legacy ID:

ECUs shall not send corrupt messages, or disrupt the transmission of the other ECU outside of its specified voltage operating range, by partial or complete ECU start up, shutdown, reset, brownout or comparable events. There must be margins in both ends of the operating voltage range in regards to the voltage level where communication becomes unreliable or is no longer possible due to hardware constraints. The hysteresis shall be at least 100 mV. See Figure Operating voltage range.

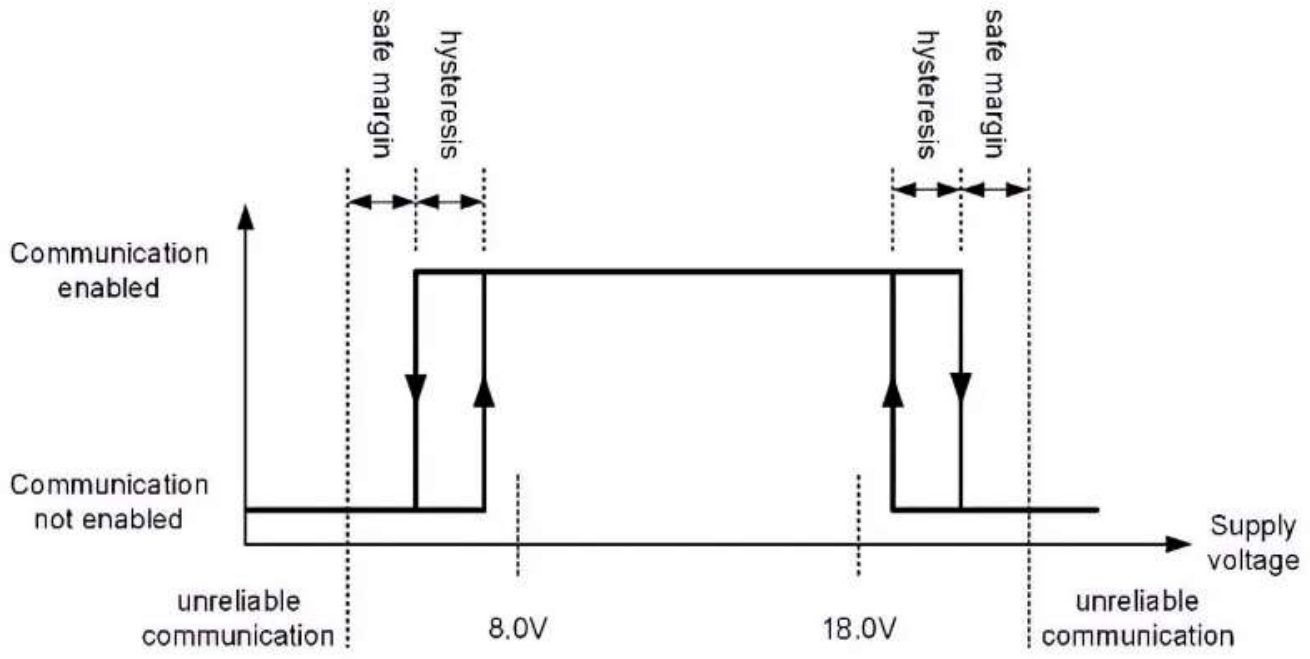


Figure - Operating voltage range.

663350 v1 **ECU Conformance**

Version history:
(1):

Purpose: To ensure that the ECU is conformant to the 1000BASE-T1 standard. The ECU supplier will have to provide test results of the applicable test cases.
Source: ETH 1000BASE-T1 14
Verification Method: Analysis
Legacy ID:

The ECU shall prove conformance to 1000BASE-T1 by passing all layer 1 test cases in *[ETH 1000BASE-T1 14]*. The test suites shall be performed by an approved external test house. All test results must be provided and will be kept confidential.

663351 v1 **EMC Conformance**

Version history:
(1):

Purpose: To fulfill EMC requirements.
Source: *[ETH REF 25]*
Verification Method: Analysis
Legacy ID:

The ECU shall be fulfill EMC requirements according to *[ETH REF 25]*.

663356 v1 EMI Protection

Version history:
(1):

Purpose: To require a circuit design and layout that is aligned with the guidelines from the component supplier.
Source: N/A
Verification Method: Analysis
Legacy ID:

The EMI protection circuit shall be as specified by the transceiver supplier.

663358 v2 GMII and RGMII/SGMII Interface

Version history:
(2): Change SRGMII to SGMII.

Purpose: To require a circuit design and layout that is aligned with the guidelines from the component supplier.
Source: N/A
Verification Method: Analysis
Legacy ID:

The GMII/RGMII/SGMII interface circuit shall be as specified by the transceiver supplier.

663363 v1 Internal Power Distribution

Version history:
(1):

Purpose: To support a short time to start the network.
Source: N/A
Verification Method: Analysis
Legacy ID:

All circuitry needed to set up a 1000BASE-T1 link shall be powered within 5 ms. The time starts when at least one of the following occurs:

- The ECU is powered.
- A wake up signal that implies that the link is needed is received by the ECU. A wake up signal can be, but is not limited to a wake up signal received via other networks like CAN and FlexRay, a button that is pressed, wake up via a cellular network or wake up via a WiFi Network.

660864 v2 Link Establishment Time

Version history:
(2): Revise PHY link up condition.

Purpose: To set the time allowed for link establishment.
Source: Not available
Verification Method: Analysis
Legacy ID: 353337v4

The time from the PHY's powering on to link up shall less than 100ms.

871754 v1 **MAC Security**

Version history:
(1):

Purpose: To increase the security in the system and to be able to use MACsec we need MACsec support in the HW.
Source:
Verification Method: Analysis
Legacy ID:

The PHY shall have HW support for MAC Security (MACsec) according [ETH REF 37:IEEE802.1AE].

871767 v1 **Management Data input/output Interface**

Version history:
(1):

Purpose: To ensure availably of the PHY info for testability and fault tracing.
Source:
Verification Method: Test
Legacy ID:

The PHY shall support MDIO according to [ETH REF 7:IEEE 802.3bp].

663355 v1 **Matching CMC with Transceiver**

Version history:
(1):

Purpose: Make sure that the transceiver and CMC are well matched.
Source: [ETH REF 7]
Verification Method: Analysis
Legacy ID:

The Tier 1 shall ask the transceiver supplier for advice regarding selection of CMC component. The transceiver suppliers advise shall be implemented with the requirement that the CMC shall be specified for automotive use and fulfil the requirements in *[ETH REF 7].*

663364 v2 **Operating Voltage Range**

Version history:
(2):

Purpose: To require functionality as a function of ECU external supply voltage. It is recommended that both ECUs on a 1000BASE-T1 network link have the same bus availability requirement. ECU functional requirements may require a larger operating range.
Source: N/A
Verification Method: Test
Legacy ID:

The 1000BASE-T1 network shall be fully functional over an operating range of 8 to 18 volts.

871766 v1 **Operations, Administration, and maintenance**

Version history:
(1):

Purpose: To ensure availably of the PHY health info for testability and fault tracing.
Source:
Verification Method: Test
Legacy ID:

The PHY shall support OAM according to [ETH REF 7:*IEEE 802.3bp*].

871755 v1 **PHY integration**

Version history:
(1):

Purpose: To ensure that the integration of the PHY is following the open alliance standards.
Source:
Verification Method: Analysis
Legacy ID:

The PHY shall be integrated on the PCB according to the reference "system implementation specification" in [ETH REF 33]OA TC12 - 1000BASE-T1 System Implementation Specification.

663359 v1 **Signal Line Impedance**

Version history:
(1):

Purpose: To ensure impedance matching on the PCB.
Source: N/A
Verification Method: Analysis
Legacy ID:

The differential impedance of BI_DA+ and BI_DA- traces shall be 100 ohm +/- 5% at frequencies from 1 MHz up to 600 MHz.

663360 v2 **Signal Line Routing on PCB**

Version history:
(2):

Purpose: To require how signal line routing is done on the PCB.
Source: N/A
Verification Method: Analysis
Legacy ID:

The pair (BI_DA+ and BI_DA-) of signal lines shall be routed side-by-side and each signal line of a pair be shall equal in length. The difference in length shall be < 1 mm. This requirement may be superseded by REQPROD *209387*.

663357 v1 **Termination**

Version history:
(1):

Purpose: To require a circuit design and layout that is aligned with the guidelines from the component supplier.
Source: N/A
Verification Method: Analysis
Legacy ID:

The termination and filtering circuit shall be as specified by the transceiver supplier.

663366 v1 **Time Stamping**

Version history:
(1):

Purpose: To ensure enough precision for synchronization, this can be achieved either in PHY or in MAC.
Source: *[ETH REF 10]*
Verification Method: Analysis
Legacy ID:

It shall be possible to time stamp all frames according to *[ETH REF 10]* with hardware.

663365 v1 **Transceiver Oscillator and External Clock Tolerance**

Version history:
(1):

Purpose: To guarantee operation frequency when the transceiver frequency is controlled by an oscillator or an external clock.
Source: N/A
Verification Method: Analysis
Legacy ID:

The Tolerance of the oscillator or external clock used with the transceiver shall be within ± 100 ppm (i.e. 0.01%).
The tolerance shall include contributions from manufacturing, ageing and temperature.
The transceiver supplier shall be consulted with regards to the selected timing device and the transceiver supplier recommendations shall be implemented.

4.6.1.5 Ethernet Data Link Layer

•Introduction

4.6.1.5.1 Introduction

This document is a technical regulation for a switched network and defines the data link layer for such a network.

•Generic Requirements

4.6.1.5.2 Generic Requirements

660875 v1 **Drop Eligible Indicator, DEI**

Version history:
(1):

Purpose: To ensure support for the 1-bit Drop Eligible Indicator field in the VLAN tag.
Source: *[ETH REF 9]*
Verification Method: Test
Legacy ID: 435735v0

The Drop Eligible Indicator, DEI-flag shall be supported and configurable per VID.

660863 v2 **ECU Conformance**

Version history:
(2): 1.Add 1000baseT1

Purpose: To ensure that the ECU is conformant to the 100BASE-T1 or 1000BASE-T1 standard. The ECU supplier will have to provide test results of the applicable test cases.
Source: *[ETH REF 4]*
Verification Method: Analysis AND Test
Legacy ID: 435344v0

The ECU shall prove conformance to 100BASE-T1 or 1000BASE-T1 by passing all layer 2 test cases in *[ETH REF 4]*.
The test suites shall be performed by an external test house approved by the base technology team (PSS333).
All test results must be provided and will be kept confidential.

660871 v1 **External MAC Addresses**

Version history:
(1):

Purpose: To ensure proper assignment of MAC addresses to interfaces exposed to external networks.
Source: N/A
Verification Method: Analysis
Legacy ID: 40841v4

MAC addresses of interfaces reachable from external networks shall be universally administered. The supplier is responsible for acquiring OUIs and address ranges for such interfaces.

660870 v1 **Internal MAC Addresses**

Version history:
(1):

Purpose: To avoid purchasing one or more OUIs for vehicle-internal MAC addresses.
Source: *[ETH REF 24]*
Verification Method: Test
Legacy ID: 30001v10

The internal MAC address shall be configurable.

Note: MAC addresses of internal ECU interfaces are allocated in *[*ETH REF 24*, REQPROD 76545]*.

660865 v1 Link speed and duplex

Version history:
(1):

Purpose: In order to optimize the startup time Auto negotiation shall not be used. Instead the link speed and duplex of each port connected to a PHY will be set statically.
Source: *[ETH REF 24]*
Verification Method: Test
Legacy ID: 347357v2

The link speed and duplex for each port that is connected to an internal or external PHY shall be set according to *[ETH REF 24].*

660866 v1 MTU

Version history:
(1):

Purpose: Define the default Maximum Transmission Unit, MTU size of Ethernet frames, to achieve consistency throughout the system.
Source:
Verification Method: Test
Legacy ID: 412631v1

The default MTU size of Ethernet packets shall be set to 1500 bytes.
MTU size shall be configurable.

660867 v1 Multicast support

Version history:
(1):

Purpose: To make it possible to use multicasts on data link layer level in the network.
Source: *[ETH REF 24]*
Verification Method: Analysis
Legacy ID: 357356v2

Multicast addresses shall be supported on the data link layer. The multicast addresses that shall be supported are found in *[ETH REF 24].*

660868 v2 Register read out using Diagnostics

Version history:
(2): 1. Re-description of requirements: Define the method for reading registers and the reference requirements for the read content.

Purpose: To be able to read registers using diagnostics. The details are not described here since these are dependent of the chip supplier.
Source: *[ETH REF 22]*

Verification Method: Test
Legacy ID: 347562v2

The ECU shall be possible to read some registers of the PHY and Switch through UDS diagnostic and local log fun ction. The contents of the registers read by the UDS diagnostic must follow *[ETH REF 41]*, and the contents of the registers read by the local log function must follow *[ETH REF 42].*

660869 v2 Register write using Diagnostics

Version history:
(2): Re-description of requirements: Define the method for writing registers and the reference requirements for the write content.

Purpose: To have the possibility to change the register settings using diagnostics. The details are not described here since these are dependent of the chip supplier.
Source: *[ETH REF 22]*
Verification Method: Test
Legacy ID: 347561v2

The ECU shall be possible to write some registers of the PHY and Switch through UDS diagnostic. The contents of the registers write by the UDS diagnostic must follow *[ETH REF 41].*

660872 v2 VLAN (IEEE 802.1Q)

Version history:
(2): Update the document version.

Purpose: To ensure support for Virtual Local Area Networks, VLAN.
Source: *[ETH REF 8]*
Verification Method: Analysis
Legacy ID: 54952v4

Virtual Local Area Network, VLAN shall be implemented according to *[IEEE 802.1Q-2014]*.

660873 v1 VLAN Identifier (VID)

Version history:
(1):

Purpose: To define which of the VIDs that can be configured.
Source: *[ETH REF 8]*
Verification Method: Test
Legacy ID: 359419v2

ECUs shall support configuration with the full range of possible VLAN identifiers, except for the reserved VIDs stated in *[Table 9-2— Reserved VID values,* [ETH REF 8]*].*

660874 v3 VLAN Priority

Version history:
(3): Add the reference document.

Purpose: To ensure support for prioritizing traffic differently depending on their importance, using the VLAN priority bits PCP.

Source: *[ETH REF 9], [ETH REF 8], [ETH REF 19], [ETH REF 20]*
Verification Method: Test
Legacy ID: 54977v6

Several use cases require strict guaranteed maximum delays. In non-deterministic Ethernet networks with unpredictable traffic load from different network participants, the network delay cannot be guaranteed with absolute certainty, and even frame loss can occur in scenarios of high traffic load that temporarily oversubscribes single network paths. However, for a small set of use cases the traffic can be prioritized with the QoS mechanisms in [ETH REF 20] such that the delay for this traffic is kept at a low level even in situations when the overall traffic load on the network is high. These mechanisms are based on the VLAN priority carried in the PCP field of VLAN tagged frames. Nevertheless, in situations with (temporarily) high traffic load (e.g. traffic bursts) congestion in the egress queues still can lead to increased delays or even frame loss.
The VLAN priority bits (PCP) in the VLAN tag *[ETH REF 8]*, shall be set according to traffic types and prioritization in *[ETH REF 9]*. The priority bits shall be configurable.
For the specific PCP value of each VLAN, refers to *[ETH REF 19]*.

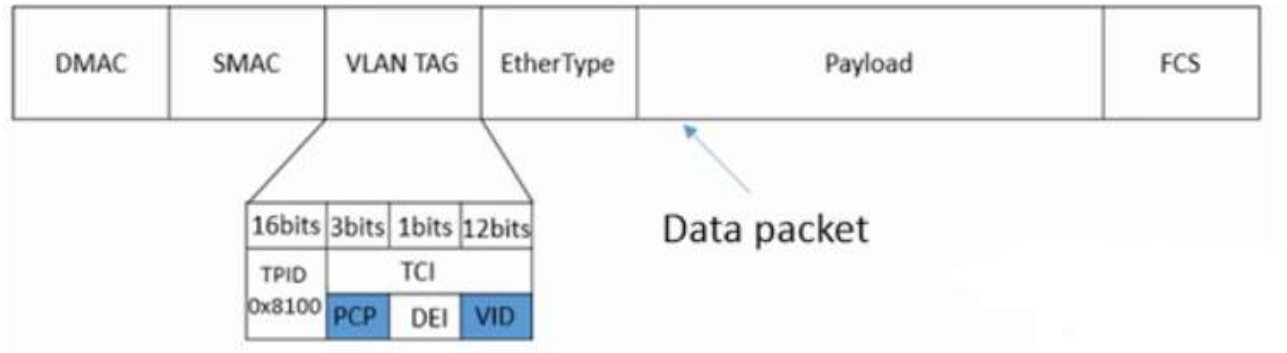
660876 v2 **VLAN configuration**

Version history:
(2): Add 802.1Q format.

Purpose: To ensure flexibility when using VLANs.
Source: LC: Diagnostics and ECU Platform *REQPROD 359541, REQPROD 54977, REQPROD 359419*
Verification Method: Test
Legacy ID: 359617v3

For VLANs, the following parameters shall be configurable per VLAN interface:

- VLAN ID, VID
- Priority, PCP
- Drop Eligible Indicator, DEI



•Switch Requirements

4.6.1.5.3 Switch Requirements

661271 v1 **Aging time for ARL table**

Version history:
(1):

Purpose: To defined the ageing time for dynamic MAC addresses.
Source:
Verification Method: Analysis
Legacy ID:

The Ageing Time for dynamically learnt MAC addresses shall be the set to the default value of 300 seconds.

660891 v1 Content of initial Configuration

Version history:
(1):

Purpose: To define what content needs to be configured at start up of the switch and where to find the details of this content.
Source: *[ETH REF 24], [ETH REF 9]*
Verification Method: Analysis
Legacy ID: 347362v2

The following content that shall be configured during the startup of the switch.

- Register settings as specified in *[ETH REF 24].*
- Prioritization based on *[ETH REF 9]* as specified in *[ETH REF 24].*
- VLAN memberships as specified in *[ETH REF 24].*

680642 v3 Ethernet Switch-support mirroring

Version history:
(3): Delete decription about filter stragy and IG.

Purpose: Defining the Ethernet Switch mirroring function requires support.
Source: Requirements for Ethernet Switch Semiconductors V1.0 Sep-201616 (TC11)

Verification Method: Analysis
Legacy ID:

The Ethernet Switch shall support mirroring for each port. The port mirroring should be enabled/disabled by diagnostic.

660892 v1 Format of local config content

Version history:
(1):

Purpose: How to handle prioritization settings, VLAN memberships and register settings, may vary depending on the switch supplier. Due to that the Tier 1 needs to define how these are contained in the local config file.
Source: N/A
Verification Method: Analysis
Legacy ID: 347559v3

The Tier 1 shall define how the content shall be contained in the local config file.

660894 v1 Independent switch operation

Version history:
(1):

Purpose: To make sure that the switch will function without involvement of a micro controller.
Source: *REQPROD 347362*
Verification Method: Analysis
Legacy ID: 434764v1

The switch shall be configured with the content described in *REQPROD 347362* independent from a microcontroller.

660895 v1 Initial switch configuration

Version history:
(1):

Purpose: To make sure that the switch is correctly configured before communication starts on any link connected to the switch.
Source: *REQPROD 347362*
Verification Method: Analysis
Legacy ID: 347358v2

The switch shall be configured with the content described in *REQPROD 347362* before the link is up on any port.

661268 v1 Jumbo frames

Version history:
(1):

Purpose: To clarify that jumbo frames are not supported by switch.
Source:
Verification Method: Test
Legacy ID:

Support of jumbo frames shall be deactivated.

660897 v1 Limitation of switch configuration

Version history:
(1):

Purpose: To ensure stable links after initial configuration.
Source: Not available
Verification Method: Test
Legacy ID: 435015v1

The microcontroller is not allowed to make a reconfiguration of the switch after the initial configuration, if the reconfiguration implies that a link will drop.

871795 v1 MAC Security

Version history:

(1):

Purpose: To enable MACsec in the vehicle network the switch needs to support this feature.
Source:
Verification Method: Analysis
Legacy ID:

The Switch port with integrated PHY shall support MACsec [[ETH REF 37] : IEEE 802.1AE].

661274 v3 **Mapping between PCP and Traffic Classes**

Version history:
V9(3): Change PCP4 shall be mapped to traffic class 4. to"PCP4 shall be mapped to traffic class 0".
Add comment.

Purpose: To define the mapping table between PCP and TC.
Source:
Verification Method: Test
Legacy ID:

The mapping of PCP values to traffic classes and vice versa shall correspond to the recommendation in Table 34-1 of *[ETH REF 8]*:

PCP0 shall be mapped to traffic class 1.

PCP1 shall be mapped to traffic class 0.

PCP2 shall be mapped to traffic class 2.

PCP3 shall be mapped to traffic class 3.

PCP4 shall be mapped to traffic class 0.

PCP5 shall be mapped to traffic class 5.

PCP6 shall be mapped to traffic class 6.

PCP7 shall be mapped to traffic class 7.

The value of the PCP field shall be assigned to the use cases according to the assignment stated in *[ETH REF 24]*.

If another requirement that defines PCP mapping differs from this one, it shall override this requirement.

660896 v2 **Number of Queues per Port**

Version history:
(2): Revise to eight queues per port from four.

Purpose: To support prioritization of traffic. To make priorities of the traffic flows there needs to be different queues to handle the different priorities.
Source: Not available
Verification Method: Analysis
Legacy ID: 356958v0

The switch shall have at least eight queues per port to match the traffic class of 0-7.

660889 v2 Register Settings Switch

Version history:
(2): Update the requirement description.

Purpose: To define register settings for switches
Source:
Verification Method: Analysis
Legacy ID: 442632v0

It shall be possible to configure the switch through the register settings.
Configurable parameters shall include how the switch boots and options while booting.
The supplier shall provide a complete description of the available configuration registers.

660898 v2 Return to Factory Port Settings when Link is not Established

Version history:
(2): Optimize the timeout time and increase the number of resets.

Purpose: To make each port where a link is not established go back to initial settings. This is needed if a port is configured in a way where it cannot communicate with the link partner (if wrong link speed, etc is set).
Source: Not available
Verification Method: Test
Legacy ID: 358820v0

If a link is not established on a port for 15s after power up/ wake up/ reset/ during runtime, the PHY or switch port shall execute soft reset and re-initialized to the initial configuration.
The reset operation shall retry 3 times.

661270 v2 Shared VLAN learning

Version history:
(2): 1.Revise name to Shared VLAN learning.

Purpose: To clarify that SVL is used in switch.
Source:
Verification Method: Analysis
Legacy ID:

Shared VLAN Learning (SVL) according to *[ETH REF 8]* shall be used.

661269 v1 Static ARL entries

Version history:
(1):

Purpose: To clarify that switch shall be able to support static ARL table and contain the MAC addresses defined for ECUs
Source:
Verification Method: Analysis
Legacy ID:

The ARL tables in switch shall contain static entries with the pre-defined MAC addresses of all ECUs.

660902 v1 Static VLAN

Version history:
(1):

Purpose: To ensure support for static VLAN membership configurations per physical port.
Source: 802.1Q-2014
Verification Method: Analysis
Legacy ID: 54971v6

ECUs with multiple ports, such as switches, shall support configuration of static VLAN membership per physical interface.

661267 v1 Store-and-forward mode

Version history:
(1):

Purpose: To define the store and forward mode is used in switching frames, which allow the switch is able to check the FCS.
Source:
Verification Method: Test
Legacy ID:

The switch shall be configured to work in store-and-forward mode.

660890 v2 Support of AVB/TSN

Version history:
(2): Add TSN protocal.

Purpose: To support AVB/TSN networks. In an AVB/TSN network all equipment between end nodes need to support AVB/TSN. This means that for ECUs containing a switch, the switch needs to support AVB/TSN even if the ECU itself does not support AVB/TSN.
Source: *[ETH REF 8], [ETH REF 10]*
Verification Method: Analysis
Legacy ID: 356959v1

The switch hardware shall support the following AVB/TSN standards:
IEEE 802.1AS
IEEE 802.1Qat (included in 802.1Q)
IEEE 802.1Qav (included in 802.1Q)
IEEE 802.1Qbv
IEEE 802.1Qbu
IEEE 802.1CB
IEEE 802.1Qci

871789 v1 TC8 Switch Conformance

Version history:
(1):

Purpose: To ensure that the switch in the ECU is conformant to the Open alliance standard. The ECU supplier will have to provide test results of the applicable

test cases.

Source:

Verification Method: Analysis AND Test

Legacy ID:

The ECU shall implement layer 2 according to [IEEE_7 : IEEE 802.1Q] and shall prove conformance to Open Alliance by passing all layer 2 test cases in [[ETH REF 35] : OA TC8 - Automotive Ethernet ECU Test Specification].

All test results must be provided and will be kept confidential.

660899 v1 Throughput

Version history:
(1):

Purpose: To make sure that the Switch can handle full through put on all links simultaneously.
Source: N/A
Verification Method: Test
Legacy ID: 435619v0

The switch shall have the capacity to switch a link load of 100% on all ports full duplex simultaneously.

660903 v1 VID Override

Version history:
(1):

Purpose: To define the possibility to overwrite a tagged Ethernet frames VID.
Source: N/A
Verification Method: Analysis
Legacy ID: 359620v2

An ECU with multiple ports shall support overriding the VID of tagged incoming frames with the VID for the outgoing port.

660900 v3 Wake Up Time

Version history:
(3): Optimize the measurement start description and modify the time.

Purpose: To get the switch to boot up and be configured within an acceptable time from a power on or wake up event.
Source: *REQPROD 347362*
Verification Method: Analysis AND Test
Legacy ID: 347360v3

Ethernet switch shall boot up and be configured within 140 ms. The time starts at the point of switch circuit to be powered up.

660893 v2 update Local Config File

Version history:
(2): Add process mode with development phase.

Purpose: In order to have a way to modify the configuration used at start up of the switch.
Source: *[ETH REF 23]*
Verification Method: Test
Legacy ID: 347363v2

It shall be possible to download a local config file, which contains the configuration used at start up of the switch.
Remark:
During engineering development phase, the switch firmware and config file shall be able to be updated by ways like JTAG, Ethernet or others.

4.6.1.6 Internetworks General Specification

•Introduction

4.6.1.6.1 Introduction

The Internet Protocol Suite, or more commonly, TCP/IP, is a mature set of standards and may be considered the working horse of the Internet. It is a packet oriented system with mechanisms for flow control, data integrity and many other features. Practically every modern customer-grade PC contains software for communicating over TCP/IP which makes it ideal for interfacing between a very open-ended user machine and vehicle. UDP is the sister protocol to TCP and is necessary for smooth integration with an Internet-like environment. One may have TCP without UDP or even IP, or any combination but for communication with PC:s they are all needed and should be implemented together in a bundle. The bundle of the Internet Protocol Suite and all other protocols and algorithms for communicating over the Internet is sometimes referred to as "Internetworks", hence the title of this document.

The Internet Protocol suite will most likely be implemented in-vehicle as a result of a series of various factors, such as: law requirements regarding vehicle diagnosis, demands for higher bandwidth due to growth in ECU software and not least connectivity with customer applications.

There are many TCP/IP stacks available with various licenses and openness ranging from the very permissive to closed. In short, there is an abundance of mature available implementations that can, and should, be used.

This document will sometimes mention the underlying physical layers, but most of the requirements in this document should be considered generally valid for most high-speed physical layers with some modifications. Requirements for bandwidth / throughput are NOT specified by this document, but will rather be defined by either the physical layer specification and / or the application layer specification.

The Open System Interconnection Reference Model (OSI Reference Model or OSI Model) is an abstract description for layered communications and computer network protocol design. The Internet Protocol Suite defines only the Network (IP) and Transport Layer (TCP/UDP).

OSI Model	Specification Coverage
Application	
Presentation	
Session	
Transport (TCP, UDP)	Internetworks General Specification
Network (IP, ARP, ICMP, IGMP)	
Data Link Layer (802.1Q)	Ethernet Datalink Layer
Physical Layer (802.3)	Ethernet Physical Layer

Relation between the OSI model and the specification *coverage*.

TCP, UDP and IP may be considered very mature protocols and have been deployed in a wide range of systems.

However, they are quite complex and there is an abundance of extensions and protocols working on the different levels in the OSI model in order to maximize performance and reliability of the network. Many of these, such as ARP will be loosely specified in this document with references to their respective specification. The requirements in this document apply to all ECUs that use TCP/IP/UDP in the vehicle network. This specification is to be regarded as a minimum that all TCP/IP/UDP ECUs must implement.

4.6.1.6.1.1 External publications

Document number	Source	Title	Version or Date
RFC-791	IETF	RFC-791 : The Internet Protocol	September 1981
RFC-2460	IETF	RFC-2460: The Internet Protocol Version 6	December 1998
RFC-1191	IETF	RFC-1191: Path MTU Discovery	November 1990
RFC-793	IETF	RFC-793: Transmission Control Protocol	September 1981
RFC-1122	IETF	RFC-1122: Requirements for Internet Hosts -- Communication Layers	October 1989
RFC-792	IETF	RFC-792: Internet Control Message Protocol	September 1981
RFC-815	IETF	RFC-815: IP Datagram Reassembly Algorithms	July 1982
RFC-5681	IETF	RFC-5681: TCP Congestion Control	September 2009
RFC-2018	IETF	RFC-2018: TCP Selective Acknowledgment Options	October 1996
RFC-896	IETF	RFC-896: Congestion Control in IP/TCP Internetworks	January 1984
RFC-826	IETF	RFC-826: An Ethernet Address Resolution Protocol	November 1982
RFC-2002	IETF	RFC-2002: IP Mobility Support	October 1996
RFC-1981	IETF	RFC-1981: Path MTU Discovery for IP version 6	August 1996
RFC-919	IETF	RFC-919: Broadcasting Internet Datagrams	October 1984
RFC-1918	IETF	RFC-1918: Address Allocation for Private Internets	February 1996
RFC-5735	IETF	RFC-5735: Special Use IPv4 Addresses	January 2010
RFC-3390	IETF	RFC-3390: Increasing TCP's Initial Window	October 2002
RFC-4632	IETF	Classless Inter-domain Routing (CIDR): The Internet Address Assignment and Aggregation Plan	August 2006
RFC-768	IETF	RFC-768: User Datagram Protocol	August 1980
IEEE 802.3	IEEE	IEEE 802.3—Carrier sense multiple access with collision detection (CSMA/CD) access method and physical layer specifications	26 December 2008
RFC-813	IETF	RFC-813: Window and Acknowledgement Strategy in TCP	July 1982
RFC-922	IETF	RFC-922: Broadcasting Internet Datagrams in the Presence of Subnets	October 1984
RFC-1469	IETF	RFC-1469: IP Multicast over Token-Ring Local Area Networks	June 1993
RFC-2474	IEFT	RFC-2474: Differentiated Services Field (DS Field) in the IPv4 and IPv6 headers	1998
IEEE 802.1Q	IEEE	IEEE 802.1Q - Media Access Control (MAC) Bridges and Virtual Bridge Local Area Networks	2014
IEEE 802.1p	IEEE	IEEE 802.1p	

Note 1: Information obtaining ISO documents may be found on the internet: www.iso.org

Note 2: Information obtaining RFC documents may be found on the internet:
http://www.ietf.org/rfc.html
Note 3: Information obtaining IEEE documents may be found on the internet:
http://standards.ieee.org
•Requirements

4.6.1.6.2 Requirements

661021 v2 **ARP Announcements**

Version history:
(2): 1.Revie the ARP announcement behivior.

Purpose: In a static in-vehicle network, this packet is not needed.
Source:
Verification Method: Analysis AND Test
Legacy ID: 30042v3

Announcement/gratuitous ARP is not used for in-vehicle network.

661020 v3 **ARP Cache entries**

Version history:
(3): 1.Revise the number to "at least 30".

Purpose: To ensure a dynamic ARP cache is operating for every logical interface.
Source:
Verification Method: Test
Legacy ID: 30041v4

IP-ECUs shall implement one dynamic ARP cache for each logical interface and the size shall be at least 30 entries.

661022 v4 **ARP Cache timeout**

Version history:
(4): Change to at least 30 minutes.

Purpose: To ensure old entries are removed from the ARP cache, making room for new ones.
Source:
Verification Method: Test
Legacy ID: 438171v0

ARP entries about the dynamic ARP shall be removed after a certain amount of time if not used. The timeout shall be set to at least 30 minutes.
For static ARP entry, it will not be aging out.

661019 v2 **ARP Implementation**

Version history:
(2): Add comment about Static Arp.

Purpose: The Address Resolution Protocol, ARP is required to resolve *IPv4 addresses into data-link layer addresses.*

Source: *RFC 826*
Verification Method: Analysis
Legacy ID: 30040v3

The Address Resolution Protocol, ARP shall be implemented as specified by *[RFC 826]*.The static ARP is mandatory.

661042 v1 Broadcasts

Version history:
(1):

Purpose: To be able to send datagrams to multiple hosts, broadcasting is an alternative.
Source: *RFC 919, RFC 922*
Verification Method: Analysis
Legacy ID: 29983v4

IP-ECUs shall support broadcasting as specified in *[RFC 919]* and *[RFC 922]*.

661035 v2 Congestion control

Version history:
(2): Add In-vehicle.

Purpose: To ensure the presence of a congestion control mechanism
Source: *RFC 896*
Verification Method: Analysis
Legacy ID: 438184v0

In-vehicle IP-ECUs shall implement Nagle Algorithm as a mechanism for congestion control as defined in *[RFC 896]*. It shall be configurable and be disabled as default.

661036 v2 Congestion control strategies

Version history:
(2): Add In vehicle.

Purpose: To ensure support for congestion control mechanisms slow-start, congestion avoidance, fast retransmit and fast recovery.
Source: *RFC 5681*
Verification Method: Analysis
Legacy ID: 438185v0

In-vehicle IP-ECUs shall implemented congestion control mechanisms slow-start, congestion avoidance, fast retransmit and fast recovery as defined in *[RFC 5681]*. It shall be possible to enable and disable the features through configuration and the default state shall be disabled.

661037 v2 Graceful TCP Connection Shutdown on ECU Reset

Version history:
(2): 部分ECU反馈：ECU复位要求中止所有TCP连接，导致复位时长过长； 所以增加描述，为了避免复位时TCP连接超时，可以适用RST 报文快速关闭TCP连接，缩短复位耗时。

Purpose: To define the behavior of IP-ECUs during a reset with regard to established TCP connections.

Source:
Verification Method: Test
Legacy ID: 54108v2

During a reset procedure, an IP-ECU shall close all open sockets and ensure that TCP's connection termination procedure runs to completion before doing the actual reset.

To shorten TCP re-connection time, it is OK to use RST message to quickly close TCP connection.

661024 v3 ICMP support

Version history:
(3): 1. 部分AUTOSAR CP ECU 反馈不支持 message type 为 Time Exceeded的ICMP。同时经过讨论，这种类型ICMP是用在网关/路由器上的。所以需求增加描述说，内部不包含3层路由器功能的ECU，不需要支持此类型的ICMP。

Purpose: Ensure support for specific ICMP messages.
Source: RFC 792, RFC 1122
Verification Method: Analysis
Legacy ID: 30037v3

ICMP shall be implemented as specified by *[RFC 792]* and *[RFC 1122]*. The implementation shall include support for ICMP messages of the type stated in the table below.

Message Type	Description
Echo	Used for testing and troubleshooting
Destination Unreachable	Destination host is not reachable
Time Exceeded	The TTL field has reached zero

For ECUs that don't use three-layer routers, it is not a must to support ICMP Message where Message Type is Time Exceed.

661032 v1 IP Network Prioritization

Version history:
(1):

Purpose: To define how to use the IP network prioritization using the Differentiated Service Code Point (DSCP) in the IPv4 header.
Source: *RFC 2474, IEEE 802.1p*
Verification Method: Analysis
Legacy ID: 360156v2

IP-ECUs shall be able to use the prioritize tag (DSCP - DiffServ) of IP packets.
It shall be possible to configure prioritization for different traffic classes.

661026 v2 IP fragmentation

Version history:
(2): Add description about application's deal with.

Purpose: to support IP fragmentation.
Source: *RFC 791*
Verification Method: Analysis

Legacy ID: 438181v0

Fragmentation on IP layer shall be enable but it shall be avoided through upper layer behavior i.e. TCP segmented transmission with limited MSS or UDP splited packets sent by application. Ensure the size of passing down packets at IP layer is no more than MTU 1500 Bytes.

661038 v1 Initial TCP Window Size

Version history:
(1):

Purpose: It is desirable that the increase of TCP's initial window size is implemented according to [RFC 3390], for performance reasons.
Source: RFC 793, RFC 3390
Verification Method: Test
Legacy ID: 30027v4

The initial size of TCP's window shall be set to a value no lesser than the upper bound specified in *RFC 3390*---around 4K bytes---, and implemented in conformance with *[RFC 793]*.

661030 v1 Internal IP Addresses

Version history:
(1):

Purpose: To define that the IP address shall be configurable via a SWDL process.
Source: *RFC 1918, RFC 5735*, LC: Diagnostics and ECU Platform, *REQPROD 436145*
Verification Method: Test
Legacy ID: 30000v7

Internal IP addresses shall be configurable.

Note: Default IP addresses for internal ECUs are allocated in *[REQPROD 76545]*.

661039 v2 Internal TCP Connection Establishment Time

Version history:
(2): 1.Revise to Internal TCP Connection Establishment Time.

Purpose: To set an upper bound for the time it should takes to establish an internal TCP connection.
Source:
Verification Method: Test
Legacy ID: 40844v2

From the time a client initiates a TCP connection until the connection is established shall take maximum 2 seconds.

***Note**: This requirement has the precondition that both ends of the connection are ready to communicate over TCP*.

661025 v2 Internet Protocol Version

Version history:
(2): Add not support IPv6.

Purpose: To ensure support for IPv4.
Source: *RFC 791*
Verification Method: Analysis
Legacy ID: 29963v4

In-Vehicle IP-ECUs shall implement IPv4 as specified by *[RFC 791]*. IPv6 shall not be used.

661023 v2 Local Configurations

Version history:
(2): Change to [REQID 660818].

Purpose: To state the locally configurable parameters.
Source: LC: Diagnostics and ECU Platform, *[REQID 660818].*
Verification Method: Analysis
Legacy ID: 436221v1

In the requirements within this LC, there are parameters required to be "configurable". Configurability of these parameters shall be implemented according to *[REQID 660818].*

871919 v1 MSL

Version history:
(1):

Purpose: Define the default MSL value.
Source:
Verification Method: Test
Legacy ID:

The maximum segment lifetime (MSL) shall be configured to 30 seconds as default.

871918 v1 MSS

Version history:
(1):

Purpose: Define the TCP MSS.
Source:
Verification Method: Test
Legacy ID:

The maximum segment size should be configured to 1460 as default.

871927 v1 Port Assignment for Transport Layer Protocol

Version history:
(1):

Purpose: to define use rules of Port.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Unless specified particularly, the port number for client side is allocated by system (e.g. Linux, Autosar AP) at random.

For the special case if either server or client is Autosar CP classic system, the port number shall be pre-defined by OEM otherwise by Tier1.

661027 v1 Reassembling fragmented packets

Version history:
(1):

Purpose: To ensure support for reassembling datagrams that are fragmented
Source: RFC 815
Verification Method: Analysis
Legacy ID: 438182v0

IP-ECUs shall support reassembling of incoming datagrams that are fragmented according to *[RFC 815]*.

871917 v1 TC8 TCP/IP Conformance

Version history:
(1):

Purpose: To specify TCP/IP stack.
Source:
Verification Method: Analysis
Legacy ID:

The ECU with TCP/IP stack shall prove conformance to Open Alliance by passing related test cases in [[ETH REF 35] : OA TC8 - Automotive Ethernet ECU Test Specification].

All test results must be provided and will be kept confidential.

661034 v1 TCP Keep-Alives

Version history:
(1):

Purpose: To enable TCP Keep-Alive for in-car communication and specify related parameters.
Source: *RFC 1122*
Verification Method: Test
Legacy ID: 40881v3

IP-ECUs shall implement and enable TCP Keep-Alive according to *[RFC 1122*].
The implementation shall include the parameters *KeepaliveTimer, KeepaliveRetries* and *KeepaliveFrequency*.

KeepaliveTimer is the time an established connection may be idle before sending the first Keep-Alive probe.
KeepaliveRetries defines the number of times an unresponsive connection shall be probed before declared dead, and the *KeepaliveRetryFrequency* defines the interval between the retries.

When a connection is declared dead by the Keep-Alive mechanism, it shall be removed including all data associated with it, after which the host must be ready to setup new connections. All parameters shall be configurable and have the following default values:

KeepaliveTimer = 5 seconds

KeepaliveRetries = 2 times
KeepaliveRetryFrequency = *5* seconds

661033 v1 **TCP Implementation**

Version history:
(1):

Purpose: The Transmission Control Protocol is one of the core protocols of the Internet protocol suite.
Source: *RFC 793*
Verification Method: Analysis
Legacy ID: 30022v3

Transmission Control Protocol, TCP shall be implemented as specified by *[RFC 793]*.

681106 v2 **TCP Retransmission timeout**

Version history:
(2):

Purpose: Define to TCP retransmission timeout parameters.
Source:
Verification Method: Test
Legacy ID:

TCPIP stack shall support retransmission timeout mechanism when the acknowledge packet of the SYN packet or data packet is not received according to *[IETF_26 : RFC-793]* and *[IETF_15 : RFC-6298]*.

As default, the initial retranmission timeout(RTO) shall be configured to **200 ms** and a **exponential back off** mechanism shall be supported for successive retransmission. Retry at most **4** times. Retry timeout at most **3 s**.

For AutoSAR CP stack, refer to following configuration.

MaxRtx = 4

RetransmissionTimeout = 0.2 s

RetransmissionTimeoutMax = 3000 ms

For Unix like system(e.g. Linux), a dynamic round trip time(RTT) should be used to calculate the RTO according to RFC6298. Other parameters should be configured as following.

tcp_syn_retries = 4

tcp_synack_retries = 4

tcp_retries2 = 4

681108 v2 **TCP SACK**

Version history:
(2): Revise to enable TCP SACK.

Purpose: Enable SACK for TCP.
Source:
Verification Method: Test
Legacy ID:

TCP option SACK(Selective ACK) turns on for in-vehicle nodes communication.

673803 v1 TCP server initialization

Version history:
(1):

Purpose: To define the server behavior upon startup (power-on) when TCP is used.
Source:
Verification Method: Test
Legacy ID:

The server shall at startup initialize, open server port and listen on connection attempts.

908842 v1 TCPIP stack startup time

Version history:
(1):

Purpose: To make sure all ECUs can communicate with each other over Ethernet within the required time.
Source:
Verification Method: Test
Legacy ID:

TCP/IP protocol stack startup time start from wakeup message on CAN bus, and end when TCP/IP protocol has initialized and can communicate with other nodes on all related Eth channels (for example send out a UDP or SOME/IP SD or TCP syn message, response to a ICMP ping request or ARP request or TCP syn).

ALL time-critical Ethernet node (MCU) TCP/IP protocol stack start time shall be within 350ms, unless otherwise specified.

ALL non time-critical Ethernet node (SOC) TCP/IP protocol stack start time shall be as soon as possible.

661029 v3 Time-to-Live

Version history:
(3): Add Multicast and CP config Note.

Purpose: To avoid excessive packet lifetimes within the vehicle network.
Source: http://www.iana.org/assignments/ip-parameters.
Verification Method: Test
Legacy ID: 29980v3

The Time-to-live(TTL) value of Unicast IP package shall be configured to 64.
The Time-to-live(TTL) value of Multicast IP package shall be configured to 1.
Note：Autosar CP Tcpip shall be configured to 64.

661041 v1 **UDP Checksum**

Version history:
(1):

Purpose: Checksums in UDP are optional but needed for IP-ECUs.
Source: *RFC 768*
Verification Method: Analysis
Legacy ID: 30020v3

UDP checksums shall be generated and validated as specified by *[RFC 768]*.

661040 v1 **UDP Implementation**

Version history:
(1):

Purpose: UDP is required for several services and protocols, DNS and DHCP being two examples.
Source: RFC 768
Verification Method: Analysis
Legacy ID: 30002v3

The User Datagram Protocol shall be implemented as specified by *[RFC 768]*.

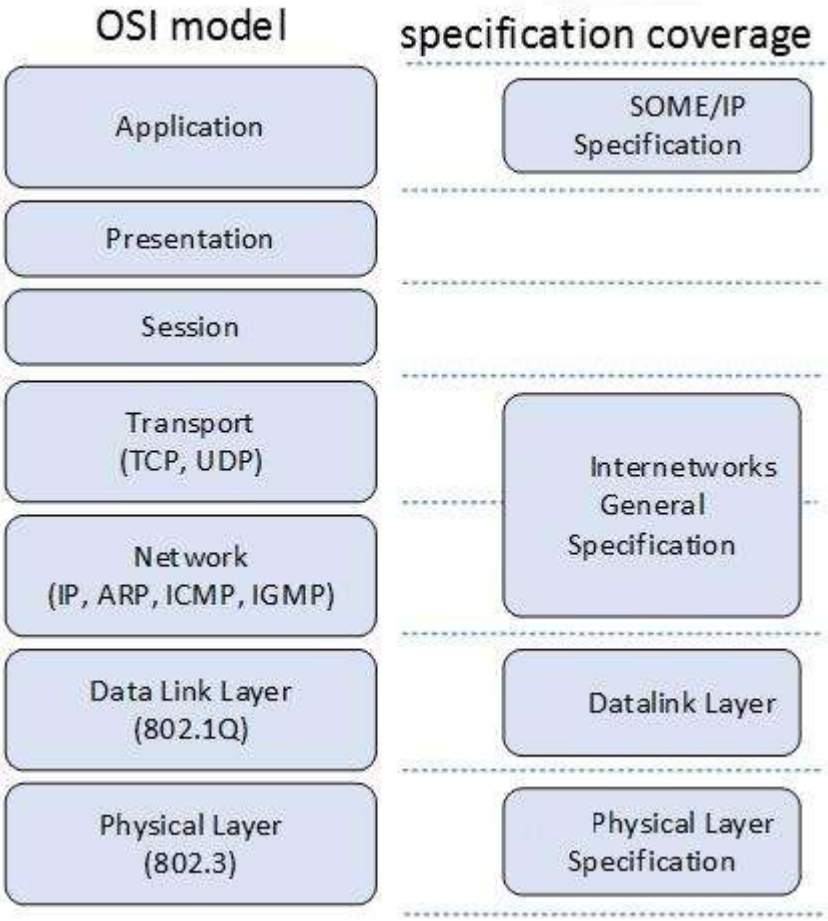
4.6.1.7 SOME IP

•Introduction

4.6.1.7.1 Introduction

SOME/IP, Scalable service-Oriented MiddlewarE over IP is an RPC protocol within AUTOSAR since version 4.2.1. It is a protocol purposed for in-car control communication within TCP/IP enabled networks. The protocol is designed with the conventional in-car communication busses in mind to avoid creating limitations for gateways interconnecting different bus types.
SOME/IP includes support for different types of messaging such as server/client messaging, notification messaging, event-based messaging and send/receive. This is convenient since different applications often behave differently thus need the different messaging patterns to match the behavior.
SOME/IP is designed to make use of transport protocols TCP and UDP, which are assumed available at the target ECU. More information about TCP and UDP is available in [ETH REF 18] .

There is another aspect of SOME/IP that need attention, namely the *interface definition* where the communications content is specified.



•**Requirements**

4.6.1.7.2 Requirements

4.6.1.7.2.1 Configuration

Configuration techniques differ depending on type of platform. AUTOSAR based systems can advantage the AUTOSAR standard post-build configuration while non-AUTOSAR platforms may have to implement support for a local configuration file to enable configuration outside development processes.

661186 v3 **Local configurations**

Version history:
(3): Change ID to [REQID 660818].

Purpose: To define the locally configurable parameters for this specification
Source: *[ETH REF 19]*
Verification Method: Analysis
Legacy ID: 435986v0

Configurations are done according to *[REQID 660818].*

4.6.1.7.2.2 SOME IP implementation

661189 v1 **Deviations from AUTOSAR**

Version history:
(1):

Purpose: To specify AUTOSAR requirements that are not necessary for a non-AUTOSAR ECU.
Source: *[ETH REF 13]*
Verification Method: Analysis
Legacy ID: 435971v0

Below is a list of references to AUTOSAR requirements specified in *[ETH REF 13]* that are not required for a non-AUTOSAR ECU.

Requirement/Reference	Name/Description
[SWS_SomeIpXf_00136]	Implementation structure specifics
[SWS_SomeIPxf_00184]	Development errors
[SWS_SomeIPxf_00185]	Tool-based configuration
[SWS_SomeIpXf_00150]	Function definition, execution order
[API Specification]	AUTOSAR specific API requirements

661190 v1 **Endianess**

Version history:
(1):

Purpose: To specify the encoding rule for the SOME/IP protocol
Source:
Verification Method: Test
Legacy ID: 435962v0

The Byte Order of the transport parameter of SOME/IP message shall be BigEndian, which means the ByteOrder should be configured as MostSignificantByteFirst.

661202 v2 **Event message missing**

Version history:
(2):

Purpose: description the method to determine whether there is a missing event message
Source:
Verification Method: Analysis
Legacy ID:

The subscriber can determine whether there is a missing event message by comparing the Session ID of two receiving consecutive SOME/IP message, which can be passed to upper-layer application for error handling. It shall be confirmed with GEELY in system design.

1071491 v1 Major interface version and Minor interface Version

Version history:

Purpose: the development and testing basis of the Major and Minor version.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The development and testing of Major and Minor versions must strictly adhere to the definitions in the ARXML at each stage.

661199 v3 Message Type

Version history:
(3): Revise TP response and TP Error.

Purpose: to describe SOME/IP message type
Source:
Verification Method: Test
Legacy ID:

The Message Type field is used to differentiate different types of messages and shall support the types in following Table.

Table :Message Type

Number	Value
0x00	REQUEST
0x01	REQUEST_NO_RETURN
0x02	NOTIFICATION
0x80	RESPONSE
0x81	ERROR
0x20	TP_REQUEST
0x21	TP_REQUEST_NO_RETURN
0x22	TP_NOTIFICATION
0xa0	TP_RESPONSE
0xa1	TP_ERROR

661191 v1 Message sequences in parallel

Version history:
(1):

Purpose: To ensure multiple request can be handled in parallel to avoid the risk of unnecessary communication delays.
Source: *[ETH REF 13]*
Verification Method: Test
Legacy ID: 435970v0

Message sequences are specified with different behaviors and may cause system unresponsiveness if handled one

at a time, especially when introducing message-level delays. Thus, the SOME/IP protocol shall be able to handle up to 50 message sequences in parallel.

661198 v1 **Messages encapsulate**

Version history:
(1):

Purpose: to encapsulate several SOME/IP messages in one Ethernet message
Source:
Verification Method: Test
Legacy ID:

The sending side can encapsulate several SOME/IP messages in one Ethernet message correctly, and the receiving side should be able to parse such SOME/IP messages correctly.

661194 v2 **SOME IP AUTOSAR Transformer Specification**

Version history:
(2): change API to Transformer.

Purpose: To clarify that the Transformer Specification for AUTOSAR and non-AUTOSAR ECUs.
Source:
Verification Method: Analysis
Legacy ID: 435969v0

IP-ECUs shall support SOME/IP according to AUTOSAR specification [[ETH REF 36] Specification of SOME/IP Transformer] if not stated otherwise, regardless of target ECU being AUTOSAR based or not.

661197 v3 **SOME IP Header Validation**

Version history:
(3): SomelP TP/XF spec在 ASR21-11 中已经删除了service/method error，； 所以不再将 return code 0x02 和 0x03 作为强制要求。

Purpose: To specify an order in which the SOME/IP header should be validated based on a table of ordered return code
Source: *[ETH REF 13]*
Verification Method: Test
Legacy ID: 435967v0

The SOME/IP Header shall be validated in the order indicated by the return codes in the following table:

Return code	Code Name	Description
0x07	SOMEIPXF_E_WRONG_PROTOCOL_VERSION	Version of SOMEIP protocol not supported
~~0x02~~	~~SOMEIPXF_E_UNKNOWN_SERVICE~~	~~Requested Service ID is unknown~~
0x08	SOMEIPXF_E_WRONG_INTERFACE_VERSION	The interface version of the requested service mismatch
~~0x03~~	~~SOMEIPXF_E_UNKNOWN_METHOD~~	~~The requested Method ID is unknown~~

0x0A	SOMEIPXF_E_WRONG_MESSAGE_TYPE	An unexpected message type was received (e.g. REQUEST_NO_RETURN for a method defined as REQUEST)
0x04	SOMEIPXF_E_NOT_READY	The application providing the method is not running
0x09	SOMEIPXF_E_MALFORMED_MESSAGE	Deserialization error so that payload cannot be deserialized
0x01	SOMEIPXF_E_NOT_OK	An unspecified error occurred

661195 v1 SOME IP Notification messaging

Version history:
(1):

Purpose: To specify the usage of message type Notification.
Source: *[ETH REF 13]*
Verification Method: Test
Legacy ID: 435968v0

SOME/IP defines notification messaging which shall be implemented to support both event based and cyclic. An event-based notification as indicated by its name is of the type sent only in the occurrence of events. A cyclic event is the type of messaging sent periodically with a fixed interval specific for each method of the type.
The interface definition specifies what type of notification to use for each notification message and its behavior, such as the periodicity of a cyclic notification.

661196 v1 SOME IP standard implementation

Version history:
(1):

Purpose: To refer to the SOME/IP standard that should be the basis for all different implementations thorough out the vehicle.
Source: *[ETH REF 13]*
Verification Method: Analysis
Legacy ID: 435959v0

IP-ECUs shall support SOME/IP according to AUTOSAR specification *[ETH REF 13]* if not stated otherwise, regardless of target ECU being AUTOSAR based or not.

871807 v2 SOMEIP Port Assignment

Version history:
(2): Revise descriptions about TCP,UDP Ports.

Purpose: To ensure a controlled usage of ports.
Source:
Verification Method: Analysis AND Test
Legacy ID:

For SOME/IP configuration, the port number for client side is allocated by system (e.g. Linux, Autosar AP) at random.
For the special case if either server or client is Autosar CP classic system, the port number shall be pre-defined by OEM via service interface design.

The TCP/UDP ports of SOME/IP service interface are defined in the Arxml by OEM.

661200 v1 Session Handling for Event

Version history:
(1):

Purpose: to define session handling for Event
Source:
Verification Method: Test
Legacy ID:

Event (message type=0x02) in each Service shall has independent Session Handling management. Session ID starts from 0x0001 and increments by 1 each time, wraps around and thus starts with 1 again.

661210 v2 Session handling for Client-Server

Version history:
(2): Due to the fact that the current requirements for SOA have not been defined,change test to Analysis,

Purpose: to define Session handling for Client-Server.
Source:
Verification Method: Analysis
Legacy ID:

The SOME/IP Transformer shall use session handling for Client-Server Communication.

661201 v3 Session handling for Sender-Receiver

Version history:
(3): Due to the fact that the current requirements for SOA have not been defined,change test to Analysis,

Purpose: to define Session handling usage condition
Source:
Verification Method: Analysis
Legacy ID:

The SOME/IP Transformer shall use session handling for Sender-Receiver Communication, which means *SessionHandlingSR* should be set to Active.

871806 v1 TC8 SOME/IP Conformance

Version history:
(1):

Purpose: To ensure that the ECU with SOME/IP stack is conformant to TC8.
Source:
Verification Method: Analysis
Legacy ID:

The ECU with SOME/IP stack shall prove conformance to Open Alliance by passing related test cases in [ETH REF 35] : OA TC8 - Automotive Ethernet ECU Test Specification].

All test results must be provided and will be kept confidential.

4.6.1.7.2.2.1 Serialization

661188 v1 **Alignment**

Version history:
(1):

Purpose: To specify the alignment used when the serializer arranges the parameters into a byte stream.
Source: *[ETH REF 13]*
Verification Method: Test
Legacy ID: 435960v0

The SOME/IP serializer shall implement an alignment of 1 byte.

671173 v1 **No length field for basic datatypes**

Version history:
(1):

Purpose: Clarifying the basic datatype serialization does not need to be defined length field.
Source:
Verification Method: Test
Legacy ID:

The basic datatype serialization does not need to be defined length field.

671220 v1 **Serialization-Arrays (dynamic length)**

Version history:
(1):

Purpose: Define the Arrays (dynamic length) type serialization requirements
Source:
Verification Method: Test
Legacy ID:

Following requirements are common for dynamic length array.
A length field at the beginning of an array should be used to specify the length of the array in Bytes.
The length field shall have a length of 32 Bits.
The length does not include the size of the length field.
The layout of dynamic arrays is shown in *Figure : One-dimensional array (dynamic length)* and *Figure: Multidimensional array (dynamic length)*.

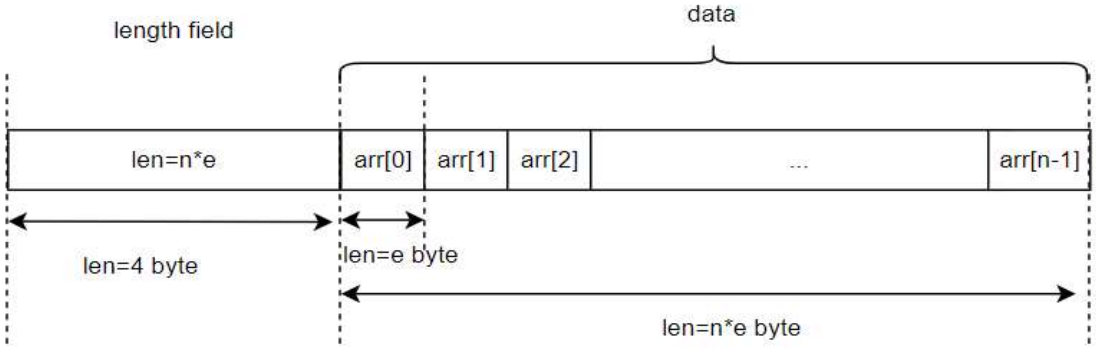


Figure : One-dimensional array (dynamic length)

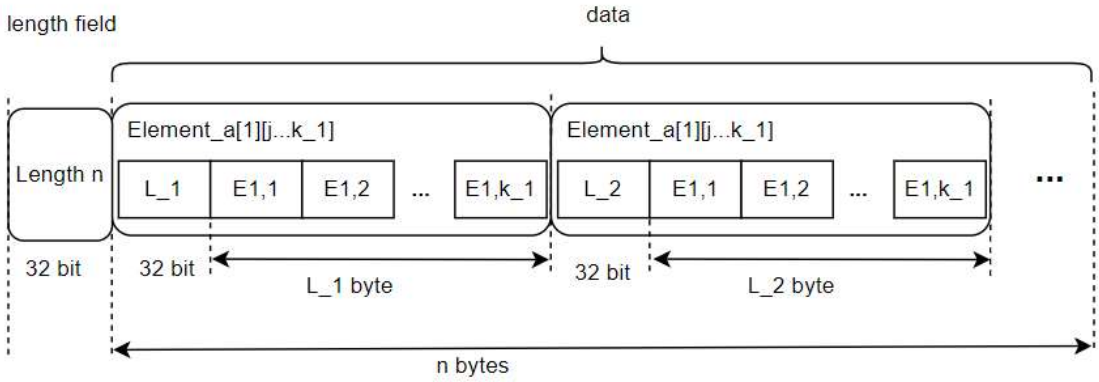


Figure : Multidimensional array (dynamic length)

In multidimensional arrays every sub array of different dimensions shall have its own length field.

661209 v2 **Serialization-Arrays (fixed length)**

Version history:
(2):

Purpose: Define the Arrays (fixed length) type serialization requirements
Source:
Verification Method: Test
Legacy ID:

The size of all length fields (in Bytes) of fixed-size arrays in the SOME/IP message shall not be set, which means *SizeOfArrayLengthFields* should be set to 0.

The serialization of multidimensional arrays follows the inmemory layout of multidimensional arrays in the programming language (row-major order).
The layout of dynamic arrays is shown in *Figure : One-dimensional array (* fixed *length)* and *Figure: Multidimensional array (* fixed *length)*.

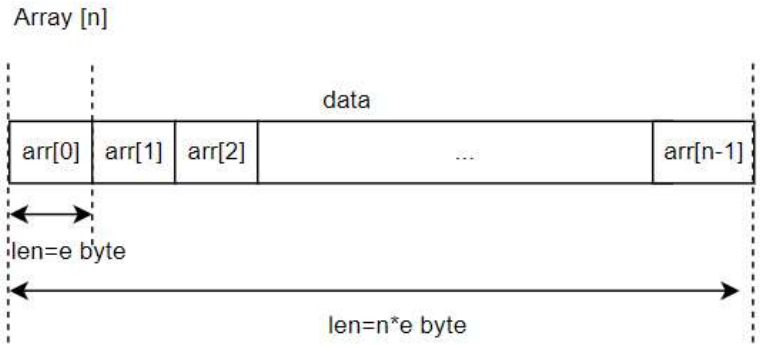


Figure : One-dimensional array (fixed length)

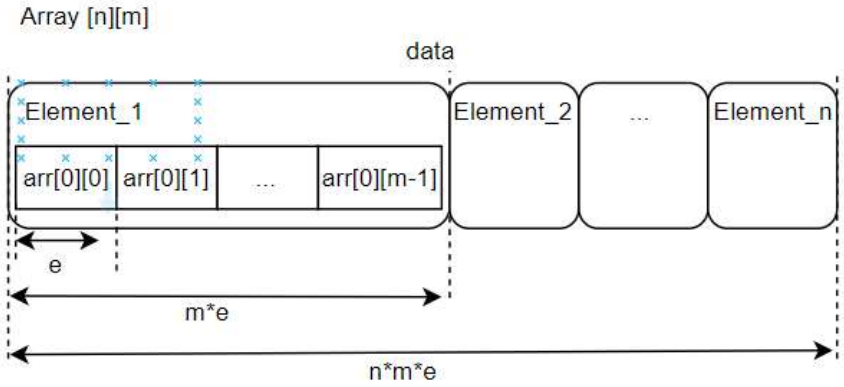


Figure: Multidimensional array (fixed length).

671344 v1 **Serialization-Strings**

Version history:
(1):

Purpose: Define the strings type serialization requirements.
Source:
Verification Method: Test
Legacy ID:

Following requirements are common for both fixed length and dynamic length strings.
Strings in the SOME/IP message shall be serialized according to the SOME/IP specification for Strings，which means implementsLegacyStringSerialization should be configured to be false.
All strings shall always start with a Byte Order Mark (BOM) in the first three (UTF-8) of the to be serialized array containing the string. The BOM shall be included in fixed-length-strings as well as dynamic-length strings. BOM allows the possibility to detect the used encoding.
Unicode encoding shall be supported UTF-8.
UTF-8 strings shall be zero terminated with a "\0" character.This means they shall end with a 0x00 Byte.
Strings shall start with a length field. The length is measured in Bytes.
The length field is placed before the BOM, and the BOM is included in the length.
The length field shall have a length of 32 Bits.

The serialization result of the string "12345" is as follows *Figure: Example of Serialization of String.*

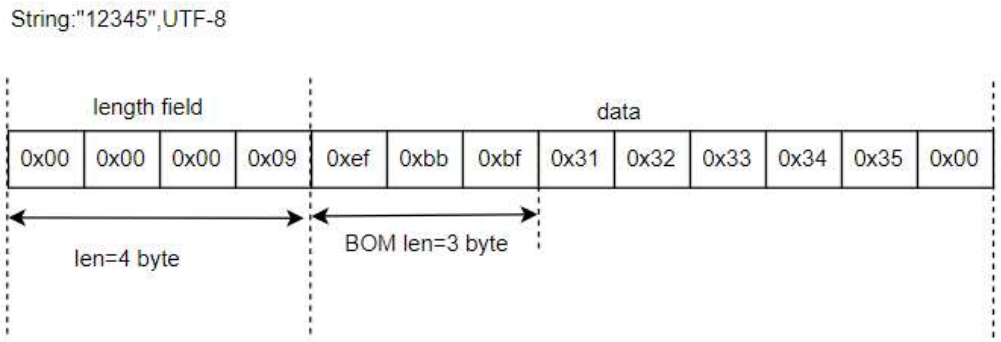


Figure: Example of Serialization of String

671222 v1 **Serialization-Structs**

Version history:
(1):

Purpose: Define the struct type serialization requirements
Source:
Verification Method: Test
Legacy ID:

Following requirements are common for structs.
A length field of 32 Bit may be inserted in front of the Struct .
The length field of the struct shall describe the number of bytes this struct occupies for SOME/IP transport.
The serialization of structs shall follow the depth-firsttraversal of the structured data type.
For example, *Figure:Serialization of Structs*

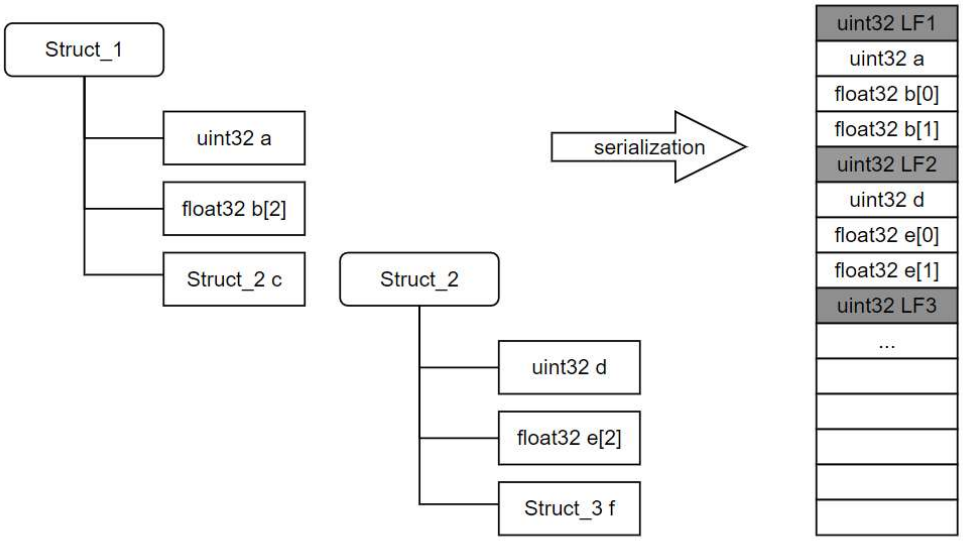


Figure:Serialization of Structs

Note:The gray marks in the figure represent the LF(length field) of the structure.

671223 v2 **Serialization-Union**

Version history:
(2): Change the error in Figure: Example of Serialization of Union.

Purpose: Define the Union type serialization requirements
Source:
Verification Method: Test
Legacy ID:

Following requirements are common for unions.
A union shall consist of a length field, type field and the payload.
The length field shall define the size of the data and does not include the size of the length field and type field.
Type field is the sequence number of the serialized union element.
Elements in the union are numbered from the first element, with type 1 for the first element, type 2 for the second element, and so on.
The length field shall have a length of 32 Bits.
The type field shall have a length of 32 Bits.

The following Figure: *Example of Serialization of Union* shows an example of a Union serialization

```
Union union_1 {
 Uint8_t a;
 Uint32_t b;
 Uint16_t c;
};
```

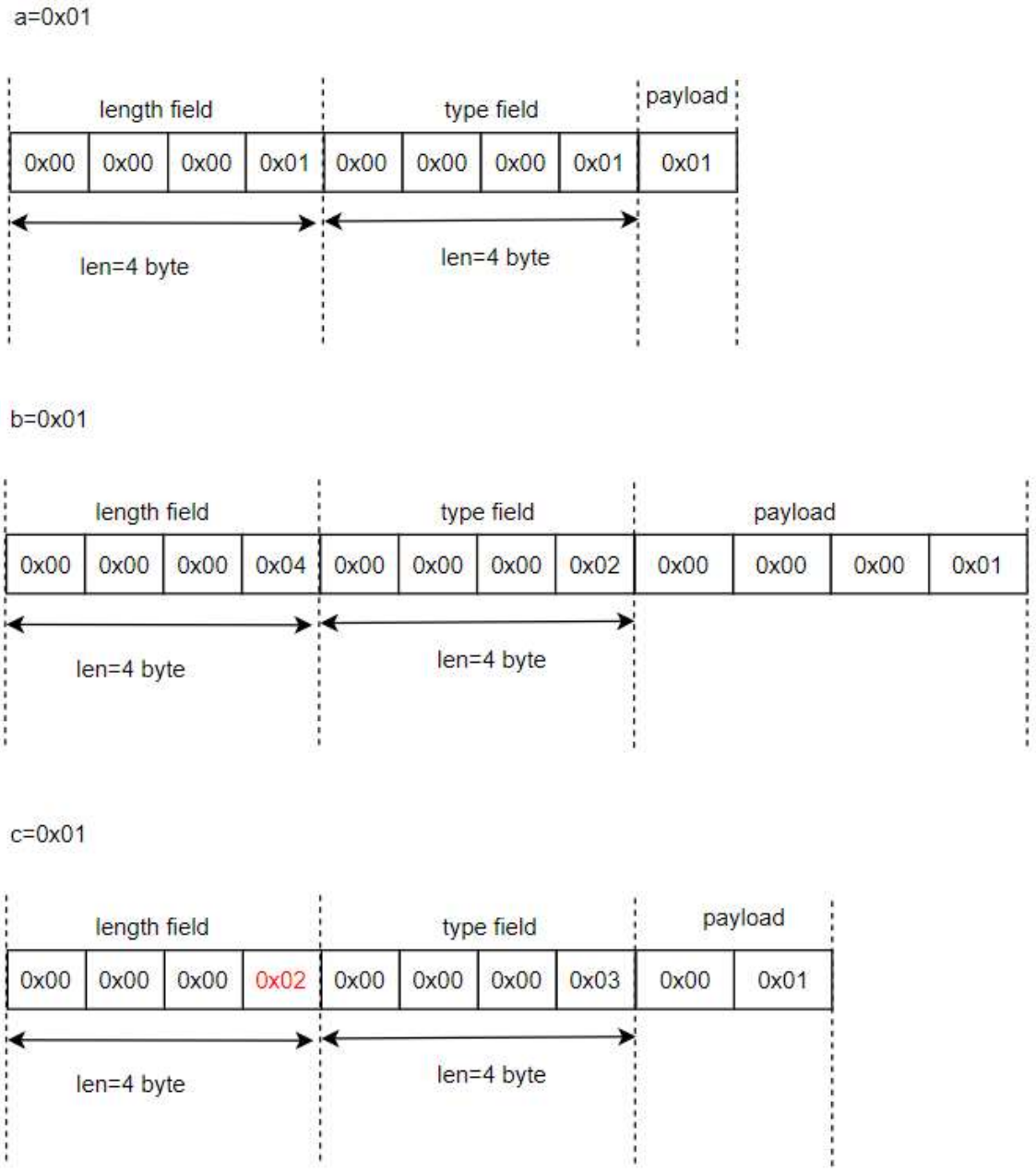


Figure: Example of Serialization of Union

671190 v1 **Serialization-enumeration**

Version history:
(1):

Purpose: Define the enumeration type serialization requirements
Source:
Verification Method: Test
Legacy ID:

The enumeration type has the same length as the base data type and does not have a length field.

661203 v1 **The type of Datatype**

Version history:
(1):

Purpose: to define the type of Datatype
Source:
Verification Method: Test
Legacy ID:

The type of Datatype shall be able to support the following basic datatypes and complex datatypes, as shown in following Table.

Table:Datatypes

Category	Datatype
Basic Datatype	uint8
	uint16
	uint32
	uint64
	sint8
	sint16
	sint32
	sint64
	float32
	float64
Complex Datatype	Structure Datatypes (structs)
	Strings (fixed and dynamic)
	Arrays (fixed length)
	Dynamic length arrays / Variable Size Arrays
	Union / Variant

4.6.1.7.2.3 SOME IP over TCP

661216 v1 **Magic cookie**

Version history:
(1):

Purpose: to define Magic cookie usage condition
Source:
Verification Method: Analysis
Legacy ID:

Magic cookie shall not be used if no specific requirement.

661217 v2 TCP Common requirements

Version history:
(2):

Purpose: to describe TCP common requirements document
Source:
Verification Method: Analysis
Legacy ID:

Common requirements for TCP shall be implemented in accordance with [ETH REF 18].

673789 v2 TCP client re-establishment

Version history:
(2): Change 5ms to 10ms, to reduce CPU load.

Purpose: To define the server/client communication reestablishment mechanism upon startup (power-on), when TCP is used and connection attempts fails.
Source:
Verification Method: Test
Legacy ID:

The client TCP connections re-establish attempts shall be performed every 10 ms after a connection attempt has been refused.

661212 v3 TCP connection establishment

Version history:
(3): Update description.

Purpose: To explain when the TCP connection should be initiated
Source:
Verification Method: Test
Legacy ID: 435974v0

After ECU is power on, TCP connection shall be established if the Service or/and Notification requested.

661213 v1 TCP connection termination

Version history:
(1):

Purpose: To ensure connections are properly closed.
Source:
Verification Method: Test
Legacy ID: 435977v0

Only the client is allowed to initiate closing of a connection. A server shall not close any connections unless initiated by the client.

661214 v2 **TCP connections**

Version history:
(2):

Purpose: To ensure reasonable number of TCP connections are set up, to avoid unnecessary usage of bandwidth.
Source:
Verification Method: Test
Legacy ID: 435973v0

The client and server shall use a single TCP connection for all methods, events, and notifications of a service instance. When having more than one instance of a service a TCP connection per services instance is needed.

661215 v2 **TCP lost connection**

Version history:
(2):

Purpose: To handle outstanding requests in case of lost TCP connection.
Source:
Verification Method: Analysis
Legacy ID: 435975v0

In the occurrence of a lost TCP connection, all outstanding requests shall be handled as timeouts.
Stand on the server side, the subsequent data processing depends on the functional requirements of the application layer.It shall be confirmed with GEELY in system design.

4.6.1.7.2.4 SOME IP over UDP

661219 v1 **SOME IP over UDP max size**

Version history:
(1):

Purpose: To specify the max size of SOME/IP packets sent over UDP.
Source:
Verification Method: Test
Legacy ID: 435979v0

When using UDP the maximum size of a SOME/IP payload over UDP shall be 1400 bytes.

661221 v2 **UDP common requirements**

Version history:
(2):

Purpose: to describe UDP common requirements document
Source:
Verification Method: Analysis

Legacy ID:

Common requirements for UDP shall be implemented in accordance with [ETH REF 18].

661220 v1 **UDP fragmentation**

Version history:
(1):

Purpose: To specify SOME/IP behaviour over UDP
Source:
Verification Method: Test
Legacy ID: 435978v0

UDP fragmentation shall not be used for SOME/IP packets.

4.6.1.7.2.5 Error Handling

Error handling can be done in the application or the communication layer below. Therefore SOME/IP supports two different mechanisms:
- Return Codes in the Response Messages of methods
- Explicit Error Messages

673680 v1 **Application based communication Error Handling**

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID:

When considering the transport of RPC messages different reliability semantics exist:
- Maybe — the message might reach the communication partner
- At least once — the message reaches the communication partner at least once
- Exactly once — the message reaches the communication partner exactly once

When using the above terms, in regard to Request/Response the term applies to both messages (i.e. request and response or error).
While different implementations may implement different approaches, SOME/IP currently achieves "maybe" reliability when using the UDP binding and "exactly once" reliability when using the TCP binding. Further error handling is left to the application.
For "maybe" reliability, only a single timeout is needed, when using request/response communication in combination of UDP as transport protocol. Figure 1 shows the state machines for "maybe" reliability. The client's SOME/IP implementation has to wait for the response for a specified timeout. If the timeout occurs SOME/IP shall signal E_TIMEOUT to the client application.

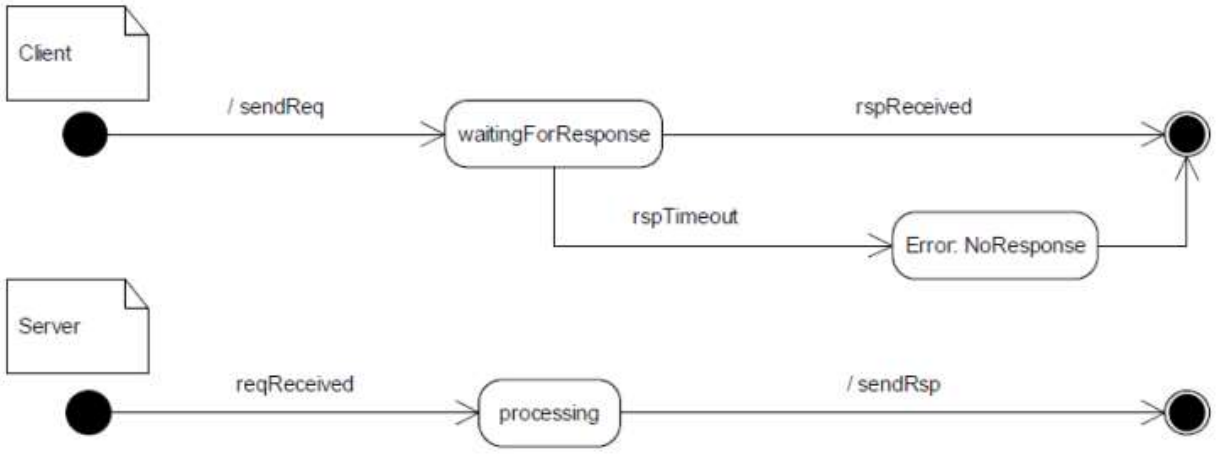


Figure 1 : State Machines for Reliability "Maybe"

For "exactly once" reliability the TCP binding may be used, since TCP was defined to allow for reliable communication.
The application can easily implement "at least once" reliability by using idempotent operations (i.e. operation that can be executed multiple times without side effects) and using a simple timeout mechanism. Figure 2 shows the state machines for "at least once" reliability using implicit acknowledgements. When the client sends out the request it starts a timer with the timeout specified for the specific method. If no response is received before the timer expires (round transition at the top), the client will retry the operation. A Typical number of retries would be 2, so that 3 requests are sent.
The number of retries, the timeout values, and the timeout behavior (constant or exponential back off) are outside of the SOME/IP specification.

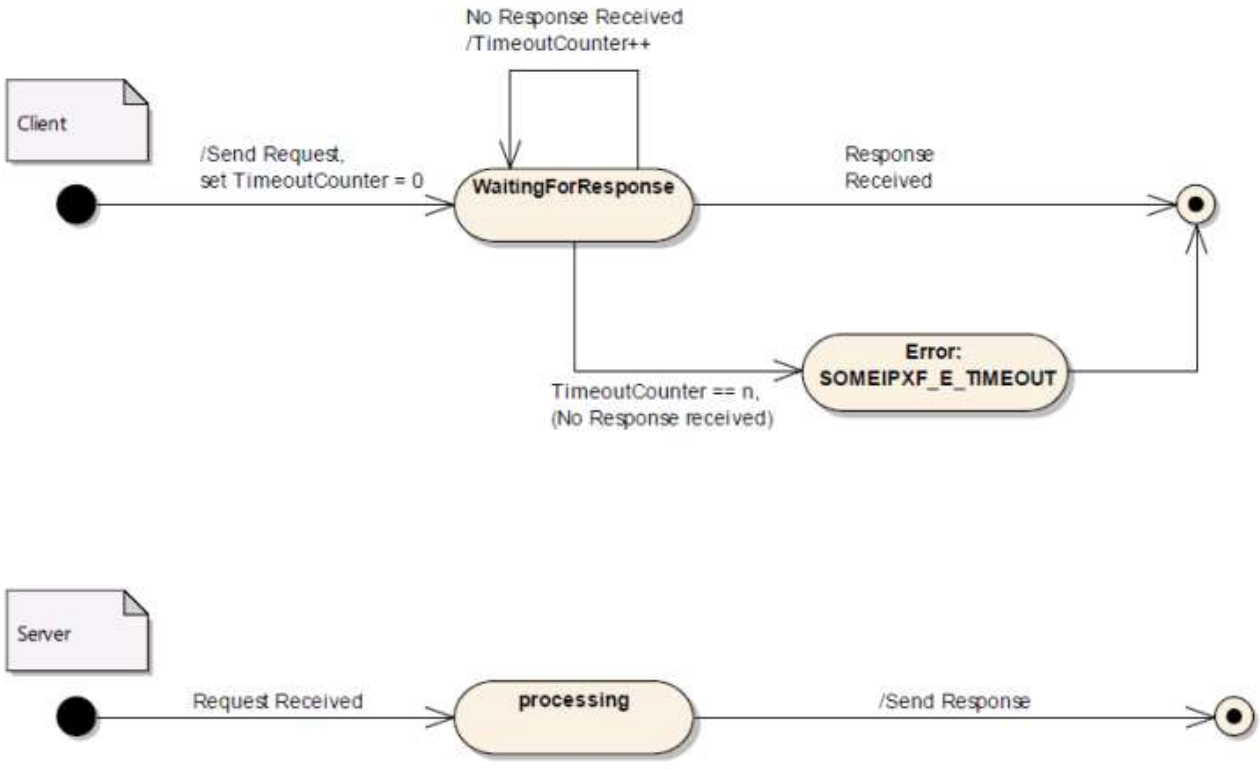


Figure 2 : State Machines for Reliability "At least once" (idempotent operations)

661225 v2 Error handling

Version history:
(2): 1.Change name to handling.

Purpose: to specify error handling implement
Source:
Verification Method: Analysis
Legacy ID:

When the caller of a method receives information of SOME/IP generic error or application error from the provider, the corresponding error handling should be left to application.

661224 v4 Explicit Error Messages

Version history:
(4):

Purpose: to describe detailed Error Message send condition
Source:
Verification Method: Analysis AND Test
Legacy ID:

Error Message (Message Type=0x81) shall be used to transport application errors and the response data or generic SOME/IP errors from the provider to the caller of a method.

If more detailed error information need to be transmitted, the payload of the Error Message (Message Type=0x81) shall be filled with error specific data, e.g. an exception string. In this case, the return code 0x20-0x5E shall be used.

675384 v3 Return Code

Version history:
(3): SomeIP TP/XF spec在 ASR21-11 中已经删除了service/method error； 所以不再将 return code 0x02 和 0x03 作为强制要求。

Purpose: to describe Return Code
Source:
Verification Method: Analysis
Legacy ID:

Currently defined Return Codes are shown in below table:

ID	CodeName	Description
0x00	E_OK	No error occurred.
0x01	E_NOT_OK	An unspecified error occurred.
~~0x02~~	~~E_UNKNOWN_SERVICE~~	~~The requested Service ID is unknown.~~
~~0x03~~	~~E_UNKNOWN_METHOD~~	~~The requested Method ID is unknown. Service ID is known.~~
0x04	E_NOT_READY	Service ID and Method ID ARE known. Application not running.
0x05	E_NOT_REACHABLE	System running the service is not reachable (internal error code only).

0x06	E_TIMEOUT	A timeout occurred (internal error code only).
0x07	E_WRONG_PROTOCOL_VERSION	Version of SOME/IP protocol not supported.
0x08	E_WRONG_INTERFACE_VERSION	The interface version of the requested service mismatch.
0x09	E_MALFORMED_MESSAGE	Deserialization error so that payload cannot be deserialized.
0x0a	E_WRONG_MESSAGE_TYPE	An unexpected message type was received (e.g. REQUEST_NO_RETURN for a method defined as REQUEST).
0x0b	E_E2E_REPEATED	Repeated E2E calculation error.
0x0c	E_E2E_WRONG_SEQUENCE	Wrong E2E sequence error.
0x0d	E_E2E	Not further specified E2E error.
0x0e	E_E2E_NOT_AVAILABLE	E2E not available.
0x0f	E_E2E_NO_NEW_DATA	No new data for E2E calculation present.
0x10 - 0x1f	RESERVED	Reserved for generic SOME/IP errors. These errors will be specified in future versions of this document.
0x20 - x5e	RESERVED	Reserved for specific errors of services and methods. These errors are specified by the interface specification.

661222 v5 **Return Codes in the Response Messages**

Version history:
(5): AUTOSAR SOMEIP 定义中， application error 可以使用message type 0x81, 也可以使用 message type 0x80。实际项目调查中，有的协议栈厂商如vector 只支持使用 message type 0x81 来传递application error。为了避免不兼容和模糊，定义所有的error message，包括application error 和 geneiric error 都使用 message type 0x81 来传递。

Purpose: to describe Response Message send condition
Source:
Verification Method: Analysis AND Test
Legacy ID:

Return Codes in the Response Messages of methods shall be used to transport ~~application errors and~~ the response data of a method from the provider to the caller of a method. Response Message (Message Type=0x80) shall be used.

Application error shall be transport according to *[REQID 661224]*.

4.6.1.7.2.6 Service Discovery

661249 v1 **Always retrieve information from an eventgroup after power on**

Version history:
(1):

Purpose: to ensure retrieve information from an eventgroup
Source:
Verification Method: Test
Legacy ID:

If the client will always need to retrieve information from an eventgroup after power on, ***SdConsumedEventgroupAutoRequire*** shall be configured as TRUE, which means Client will automatically sends SubscribeEventgroup after receiving OfferService. It shall be confirmed with GEELY in system design phase and should be included in *[ETH REF 15]*.

661234 v2 Client time parameters

Version history:
(2): 1.Change the format.

Purpose: to define client time parameters
Source:
Verification Method: Test
Legacy ID:

Client time parameters design during the SD phase is shown in **Table: SD time parameters for client.**

Table: SD time parameters for client.

Name	Default Value
INITIAL_DELAY_MIN (ms)	10
INITIAL_DELAY_MAX (ms)	100
REPETITIONS_MAX (times)	0
TTL (s)	3
REQUEST_RESPONSE_DELAY_MIN (ms)	0
REQUEST_RESPONSE_DELAY_MAX (ms)	0

The allowable error range of parameter CYCLIC_OFFER_DELAY is ±10%.

661237 v1 Client wait for answer when not receiving OfferService

Version history:
(1):

Purpose: to ensure OfferService shall be recived by Client
Source:
Verification Method: Test
Legacy ID:

If there is no answer to FindService (means not receiving OfferService) the Client has to wait until Offer Service has been sent by the Server.

679854 v2 Definition of TPowerWakeUpToApp

Version history:
(2):

Purpose: To define maximum time to start SOME/IP communication.
Source:
Verification Method: Test
Legacy ID:

The ECU should start and send the first SD message within 500 ms. The time should be measured from the state where the ECU is in sleep until it can receive wake-up requests from itself or other ECUs.

Note:
This requirement applies to all ECUs that apply the SOME/IP protocol, as well as ECUs with AUTOSAR based

microcontrollers.
For microcontrollers/microprocessors running on operating systems such as Linux, this timing value may be too short. For these specific situations, after reaching an agreement with the basetech team, start-up values other than those specified should be used.

661232 v2 Entry and Option setting

Version history:
(2):

Purpose: to ensure the minimum sets of Entry and Option can be implemented
Source:
Verification Method: Analysis
Legacy ID:

The minimum sets of Entry and Option that needed to be supported shall follow the Chapter 4.1 of *[ETH REF 14]*.

661238 v1 Error handling for FindService

Version history:
(1):

Purpose: to specify error handling when no answer
Source:
Verification Method: Analysis
Legacy ID:

If there is no answer to FindService (means not receiving OfferService), and the Client needs further error handling, it shall be left to the application.

661248 v1 Eventgroup messages send to IP address

Version history:
(1):

Purpose: to specify address that Eventgroup messages need to send
Source:
Verification Method: Test
Legacy ID:

Eventgroup messages shall be sent to/from the requested IP address, which follows the definition of ECU Address.

661247 v1 Eventgroup messages send to MAC address

Version history:

(1):

Purpose: to specify address that Eventgroup messages need to send
Source:
Verification Method: Test
Legacy ID:

Eventgroup messages shall be sent to/from the requested MAC address, which follows the definition of ECU Address.

661240 v1 **FindService messages send to multicast IP address**

Version history:
(1):

Purpose: to defind FindService messages sending method
Source:
Verification Method: Test
Legacy ID:

FindService messages shall always be sent to the defined multicast IP address.

661239 v1 **FindService messages send to multicast MAC address**

Version history:
(1):

Purpose: to defind FindService messages sending method
Source:
Verification Method: Test
Legacy ID:

FindService messages shall always be sent to the defined multicast MAC address.

661236 v2 **FindService sends in the Repetition phase**

Version history:
(2): Delete the reference.

Purpose: to specify how to send FindService in the Repetition phase
Source:
Verification Method: Test
Legacy ID:

The Client sends FindService only once in the Repetition Phase, and then goes directly into the Main Phase.

661252 v1 **Initial event application condition**

Version history:
(1):

Purpose: to define initial event application condition
Source:
Verification Method: Analysis
Legacy ID:

After the Client receives SubscribeEventgroupACK, the scenario that Client receives initial event should follow the Chapter 4.1.6 of *[ETH REF 14]*. If there is a need to define other scenarios that Client receives initial event. It shall be confirmed with GEELY.

661250 v2 **Meet condition to retrieve the information of an eventgroup**

Version history:
(2): 1.Revise the reference number.

Purpose: to ensure retrieve information from an eventgroup
Source:
Verification Method: Test
Legacy ID:

If the client is powered on, and there are certain conditions that need to be met to retrieve the information of an eventgroup, ***SdConsumedEventgroupAutoRequire*** shall be configured as FALSE, which means Client will sends SuscribeEventgroup after receiving subscribe request from application and receiving OfferService. It shall be confirmed with GEELY and should be included in *[ETH REF 15]*.

If ***SdConsumedEventgroupAutoRequire*** is configured as TURE, and the recommended practice is not applying the optimization implementation defined in *[REQ_ID_*661243*]*, and there will be no delay from Server sending OfferService to Client sending SubscribeEventgroup.

If ***SdConsumedEventgroupAutoRequire*** is configured as FALSE, and the recommended practice is not apply the optimization implementation defined in *[REQ_ID_*661243*]*, and there will be at most a delay equal to CYCLE_OFFER_DELAY from completing the application layer setting to completing eventgroup subscription. If the Client wants to retrieve information of the eventgroup before this delay, it can be done by designing a getter for the information in system design phase.

661242 v1 **OfferService messages send to multicast IP address**

Version history:
(1):

Purpose: to defind OfferService messages sending method
Source:
Verification Method: Test

Legacy ID:

OfferService messages, based on multicast, shall always be sent to the defined multicast IP address unless there is need for unicast reply to FindService message.

661241 v1 OfferService messages send to multicast MAC address

Version history:
(1):

Purpose: to defind OfferService messages sending method
Source:
Verification Method: Test
Legacy ID:

OfferService messages shall always be sent to the defined multicast MAC address unless there is need for unicast reply to FindService message.

661255 v1 Restart behavior judgment

Version history:
(1):

Purpose: to define the restart behavior judgment method
Source:
Verification Method: Test
Legacy ID:

The subscriber can determine whether the opposite side has restarted by comparing the Session ID and Reboot Flag of two receiving consecutive SOME/IP-SD messages.

661233 v2 SOME/IP-SD Message Counter

Version history:
(2):

Purpose: to specify how to implement SOME/IP-SD Message counter
Source:
Verification Method: Test
Legacy ID:

The Counter field in SOME/IP-SD Message shall not be used, which means Counter field shall be configured as 0.

661235 v2 Server time parameters

Version history:
(2): Revise CYCLIC_OFFER_DELAY from 2s to 1s.

Purpose: to define server time parameters
Source:
Verification Method: Test
Legacy ID:

Server time parameters design during the SD phase is shown in **Table: SD time parameters for server**.

Table: SD time parameters for server

Name	Default Value
INITIAL_DELAY_MIN (ms)	10
INITIAL_DELAY_MAX (ms)	100
REPETITIONS_BASE_DELAY (ms)	300
REPETITIONS_MAX (times)	3
CYCLIC_OFFER_DELAY (ms)	1000
TTL (s)	3
REQUEST_RESPONSE_DELAY_MIN (ms)	0
REQUEST_RESPONSE_DELAY_MAX (ms)	0

The allowable error range of parameter CYCLIC_OFFER_DELAY is ±10%.

661230 v1 Service Discovery Message

Version history:
(1):

Purpose: to describe how to set Undefined bits within Flag
Source:
Verification Method: Test
Legacy ID:

In the Service Discovery Message, Undefined bits within Flag field and the 24-bit reserved field shall be set to ‘0’ when sending and ignored on receiving.

661231 v1 Service Discovery Messages byte order

Version history:
(1):

Purpose: to define service discovery byte order
Source:
Verification Method: Test
Legacy ID:

All Service Discovery Messages shall be in Big Endian Byte Order.

661229 v1 **Service Discovery protocol**

Version history:
(1):

Purpose: to specify the protocol implemented by service discovery
Source:
Verification Method: Test
Legacy ID:

The Service Discovery protocol shall be implemented using IPv4.

661243 v1 **Service sending and receiving when optimization implementation**

Version history:
(1):

Purpose: to define service sending and receiving when optimization implementation
Source:
Verification Method: Test
Legacy ID:

If the optimization implementation in [PRS_SOMEIPSD_00423] of [ETH REF 14] is used, when FindService messages received, shall use the next multicast OfferService as the RESPONSE if the last OfferService was sent 1/2 CYCLIC_OFFER_DELAY or longer ago.

661246 v1 **StopOfferService message send condition**

Version history:
(1):

Purpose: to specify the StopOfferService message send condition
Source:
Verification Method: Test
Legacy ID:

StopOfferService message should be sent when the Server is no longer providing the corresponding service (e.g. sleep condition is met, the service is not available, etc.).

661245 v1 **StopOfferService messages send to multicast IP address**

Version history:
(1):

Purpose: to specify address that StopOfferService need to send
Source:
Verification Method: Test
Legacy ID:

StopOfferService messages shall always be sent to the defined multicast IP address.

661244 v1 **StopOfferService messages send to multicast MAC address**

Version history:
(1):

Purpose: to specify address that StopOfferService need to send
Source:
Verification Method: Test
Legacy ID:

StopOfferService messages shall always be sent to the defined multicast MAC address.

661254 v1 **SubscribeEventgroup message resend mechanism**

Version history:
(1):

Purpose: to define SubscribeEventgroup message resend mechanism condition
Source:
Verification Method: Test
Legacy ID:

The SubscribeEventgroup message resend mechanism shouldn't be used in SOME/IP-SD communication behavior, which means **SUBSCRIBE_RETRY_MAX** and **SUBSCRIBE_RETRY_DELAY** should be set to 0.

661251 v3 **SubscribeEventgroup negative response error handling**

Version history:
(3):

Purpose: to describe SubscribeEventgroup negative response error handling
Source:
Verification Method: Analysis
Legacy ID:

If Client sends out SubscribeEventgroup and receives negative response (it means receiving

SubscribeEventgroupNACK), the corresponding error handling shall be left to the application.

661253 v1 **the condition of StopSubscribeEventgroup messages sending**

Version history:
(1):

Purpose: to define the condition of StopSubscribeEventgroup message seding
Source:
Verification Method: Test
Legacy ID:

The StopSubscribeEventgroup message should be sent if the client no longer needs the information provided by the Events and Fields of the according Eventgroup.

4.6.1.7.2.7 SOME IP TP

The UDP binding of SOME/IP can only transport SOME/IP messages that fit directly into an IP packet. If larger SOME/IP messages need to be transported over UDP (e.g. of 32 KB) the SOME/IP Transport Protocol (SOME/IP-TP) shall be used. The SOME/IP message too big to be transported directly with the UDP binding shall be called "original" SOME/IP message. The "pieces" of the original SOME/IP message payload transported in SOME/IP-TP messages shall be called "segments".

697110 v2 **Definition of SOME/IP TP**

Version history:
(2): Revise description.

Purpose: To define whether to implement SOME/IP-TP.
Source:
Verification Method: Analysis
Legacy ID:

There is no new application requirement for SOME/IP-TP since 25R1 stage, but the protocol stack needs to be supported.

697247 v1 **Maximum length of a segment**

Version history:
(1):

Purpose: Maximum length of a segment.
Source:
Verification Method: Test
Legacy ID:

Since UDP-based SOME/IP messages are limited to 1400 bytes payload, the maximum length of a segment that is correctly aligned is 1392 bytes.

697217 v1 **SOME/IP TP-Header structure**

Version history:

(1):

Purpose: Definition of SOME/IP TP Header structure.
Source:
Verification Method: Test
Legacy ID:

SOME/IP TP-Header structure is according to [ETH REF 13].

697233 v1 SOME/IP-TP Header encode type

Version history:
(1):

Purpose: Definition of SOME/IP-TP Header encode type.
Source:
Verification Method: Test
Legacy ID:

SOME/IP-TP Header shall be encoded in network byte order (big endian).

697111 v1 Session Handling for SOME/IP TP

Version history:
(1):

Purpose: to define Session handling for SOME/IP TP.
Source:
Verification Method: Test
Legacy ID:

SOME/IP messages using SOME/IP-TP shall activate Session Handling (Session ID must be unique for the original message.

697113 v1 Session ID about Session ID

Version history:
(1):

Purpose: Session ID about Session ID
Source:
Verification Method: Test
Legacy ID:

All SOME/IP-TP segments shall carry the Session ID of the original message; thus, they have all the same Session-ID.

697114 v1 TP-Flag

Version history:
(1):

Purpose: TP-Flag
Source:

Verification Method: Test
Legacy ID:

SOME/IP-TP segments shall have the TP-Flag of the Message Type set to 1.

4.6.1.7.2.7.1 Sender specific behavior

697250 v1 **Configured segment**

Version history:
(1):

Purpose: configured Segment.
Source:
Verification Method: Analysis
Legacy ID:

The sender shall segment only messages that were configured to be segmented.

697254 v1 **Segment Size**

Version history:
(1):

Purpose: Segment Size shall have the same size.
Source:
Verification Method: Test
Legacy ID:

The sender shall segment in a way that all segments with the More Segment Flag set to 1 are of the same size about 1392 Bytes.

697252 v1 **Send order**

Version history:
(1):

Purpose: Send order about segment.
Source:
Verification Method: Test
Legacy ID:

The sender shall send segments in ascending order.

697255 v1 **not send overlapping or duplicated segments**

Version history:
(1):

Purpose: Overlapping or duplicated segments
Source:
Verification Method: Test
Legacy ID:

The sender shall not send overlapping or duplicated segments.

4.6.1.7.2.7.2 Receiver specific behavior

697270 v1 **Reassemble check**

Version history:
(1):

Purpose: Reassemble check.
Source:
Verification Method: Analysis
Legacy ID:

The receiver should only reassemble up to its configured buffer size and skip the rest of the message.And Only correctly reassembled message of up to the configured size shall be passed to an application.
Note: This means that the implementation must make sure that all bytes of the message must be bytes that were received and reassembled correctly. Counting non-overlapping, non-duplicated bytes and comparing this to the length could be a valid check.

697380 v1 **Received order**

Version history:
(1):

Purpose: Received order.
Source:
Verification Method: Test
Legacy ID:

The receiver shall support reassembly of segments that are received in ascending and descending order.

697382 v1 **Segmented message store buffer**

Version history:
(1):

Purpose: Segmented message store buffer.
Source:
Verification Method: Analysis
Legacy ID:

Interleaving of different segmented messages using the same buffer (e.g. only the Session-ID and payload are different) is not supported.
Note: This prohibits that equal events (same Message-ID, IP-Addresses, ports numbers, and transport protocol) arrive in the wrong order, when some of their segments get reordered.
Reordering of segments of completely different original messages (e.g. Message ID is different) is not of concern since those segments go to different buffers.
The receiver shall correctly reassemble overlapping and duplicated segments by overwriting based on the last received segment.
The receiver may cancel reassembly, if overlapping or duplicated segments change already written bytes in the buffer, if this feature can be turned off by configuration.
The receiver shall be able to detect and handle obvious errors gracefully. E.g. cancel reassembly if segment length of a segment with MS=1 is not a multiple of 16.
Note: This means that buffer overflows or other malfunction shall must be prevented by the receiving code.

697256 v1 **Segments reassemble Condition**

Version history:
(1):

Purpose: Segments reassemble Condition.
Source:
Verification Method: Test
Legacy ID:

The receiver shall match segments for reassembly based on the configured values of Message-ID, Protocol-Version, Interface-Version and Message-Type (w/o TP Flag).
It shall be supported to reassemble multiple messages with the same Message ID but sent from different clients (difference in Sender IP, Sender Port, or Client ID) in parallel. This should be controlled by configuration and determines the amount of "reassembly buffers".

697257 v1 **Session ID**

Version history:
(1):

Purpose: Session ID.
Source:
Verification Method: Test
Legacy ID:

The Session ID shall be used to detect the next original message to be reassembled.
The receiver shall start a new reassembly (and may be throw away old segments that were not successfully reassembled), if a new segment with a different Session-ID is received.

697381 v2 **When to cancle desegment**

Version history:
(2):

Purpose: When to cancle desegment.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The receiver should use limited resources to support reassembly of reordered segments of a single original message. At least a reorder distance of 3 shall be supported (i.e. segments are allowed up to 3 positions away from their correct place in sequence).
Note: This could mean for example that the receiver can only desegment if segments are in order but it stores the last 4 segments and sorts them before trying to deserialize. Or it could mean that all segments are written into a buffer and 4 meta-data structures (e.g. start and length) to store which data of the buffer is valid are present.
The receiver shall cancel desegmentation for a single original message, if missing segments of this original message are detected and resources do not permit to further wait on the segment (e.g. because the next message already starts or reorder distance is higher than expected).
Note: This means that reordering inside a single original message is allowed, if resources permit this.

4.6.1.8 RTP

•Requirements

4.6.1.8.1 Requirements

This document describes a concept for audio and video streaming over RTP as a complement to FQTSS according to the switch concept in *[ETH REF 20]*.

871805 v1 **Synchronizing multimedia streams**

Version history:
(1):

Purpose: A reference clock(e.g. NTP) among all the senders could be used to synchronize multiple streams.
Source:
Verification Method: Analysis AND Test
Legacy ID:

For use case which has multiple media streams to be transmitted, a reference time (wallclock time) shall be used among all the senders to synchronize these streams in the recipient side. According to [IETF_17 : RFC-3550], a NTP clock shall be implemented and distributed through RTCP in this use case.
Note: If the streams could be synchronized in the sources by other methods, a NTP reference clock shall not be implemented.

4.6.1.8.1.1 General RTP parameters

This chapter contains information about general parameters used for the media data transmission over RTP according to *[ETH REF 12]*.

661258 v1 **Media clock synchronization**

Version history:
(1):

Purpose: There is no need to implement the synchronization of the talker and listener media clocks
Source:
Verification Method: Analysis
Legacy ID:

Unless future use cases make it necessary, synchronization of the talker and listener media clocks need not be implemented for communication inside the car.

Disclaiming media clock synchronization will lead to rare drop-outs; a method to avoid such drop-outs would require hardware timestamping. Other methods of mitigating the effect of media clock drift/skew/jitter may exist in the application context, but these are outside of the scope of this document.

661259 v1 **SSRC**

Version history:
(1):

Purpose: To avoid SSRC conflicts by using Static SSRC
Source:
Verification Method: Analysis
Legacy ID:

The SSRC should be defined according to use cases. Instead of RTCP, for these use cases static SSRC identifiers shall be used according to 'SSRC' to avoid SSRC conflicts, and the stream quality shall not be changed dynamically.

871800 v2 Separate sessions for different media

Version history:
(2):

Purpose: Audio and video stream shall be transmitted through different RTP session.
Source:
Verification Method: Analysis AND Test
Legacy ID:

For use cases that has multiple media streams to be transmitted(e.g. audio/video), saparate RTP sessions shall be used for each media.

661256 v1 Support of Mixers

Version history:
(1):

Purpose: There is no need to use Mixers in the car unless further notice.
Source:
Verification Method: Analysis
Legacy ID:

Unless future use cases or use case scenarios make it necessary, Mixers according to *[ETH REF 12]* need not be implemented at Ethernet switch devices inside the car.

661260 v3 Support of RTCP

Version history:
(3):

Purpose: Choose whether to use RTCP based on the actual situation.
Source:
Verification Method: Test
Legacy ID:

RTCP according to *[ETH REF 12]* need to be implemented for media streams whose media source and sink both reside inside the car.

Unless future use cases make it necessary, RTCP need to be implemented. RTCP frame should be on the IP bus.

661257 v1 Support of Translators

Version history:
(1):

Purpose: There is no need to use translators unless further notice.

Source:
Verification Method: Analysis
Legacy ID:

Unless future use cases or use case scenarios make it necessary, Translators according to *[ETH REF 12]* need not be implemented at Ethernet switch devices inside the car.

4.6.1.8.1.2 Media formats

661261 v1 **Audio format**

Version history:
(1):

Purpose: Audio format used in usecases should be well-defined.
Source:
Verification Method: Analysis
Legacy ID:

Audio format should be defined in every use cases.

661263 v1 **Audio/video container format**

Version history:
(1):

Purpose: The video/audio format should be specified, the formats which do not use inside the car need not be implemented.
Source:
Verification Method: Test
Legacy ID:

Unless future use cases make it necessary, audio/video container formats like MPEG2-TS need not be implemented for communication inside the car.

661262 v1 **Video format**

Version history:
(1):

Purpose: Video format used in usecases should be well-defined.
Source:
Verification Method: Analysis
Legacy ID:

Video format should be defined in every use cases.

4.6.1.8.1.3 Underlying layers

661265 v1 **Port assignment**

Version history:
(1):

Purpose: A specified transport layer port is needed for RTP/RTCP stream.
Source:
Verification Method: Test
Legacy ID:

The port assignment shall obey the recommendation in [ETH REF 12] to use an even number as the transport layer port number for RTP and the next higher (odd) number for RTCP. Specification of the exact UDP port numbers to be assigned to RTP/RTCP is defined in RTP stream definition.

661266 v1 **Stream destination addresses**

Version history:
(1):

Purpose: All unicast RTP stream should have unicast destination IP and MAC addresses.
Source:
Verification Method: Test
Legacy ID:

Unless future use cases make it necessary to send media streams to more than only one receiving ECU (i.e. multicast streams), all destination addresses shall be unicast IP and MAC addresses according to the receiving ECUs' configuration.

661264 v1 **Underlying transport protocol**

Version history:
(1):

Purpose: As the underlying transport protocol for RTP, UDP is used to provide low latencies.
Source:
Verification Method: Test
Legacy ID:

In order to provide low latencies for all media stream use cases, UDP over IPv4 shall be used as the underlying transport protocol for RTP.

4.6.1.9 VLAN Configuration

•Requirements

4.6.1.9.1 Requirements

4.6.1.9.1.1 VLAN parameters

660877 v2 **Discard Untagged Frames**

Version history:
(2): 1.Revise the description about untag frame.

Purpose: To define how to handle untagged ingress traffic per interface.
Source: N/A

Verification Method: Test
Legacy ID: 359456v2

The ECU shall support per-interface discarding of Ethernet frames except the special cases which are untagged defined in the VLAN allocation table of [ETH REF 24] .
Note: Rules shall be configurable.

661277 v1 Double-tagging

Version history:
(1):

Purpose: To define the dropping rules for outer VLAN.
Source:
Verification Method: Test
Legacy ID:

All ECUs (including the switches) shall drop frames with more than one outer VLAN Tag with TPID 0x8100.

All ECUs (including the switches) shall drop frames with an outer VLAN Tag with TPID 0x9100 or 0x88a8.

661278 v2 Mapping between use cases and VLAN ID

Version history:
(2): Delete the default VLAN,

Purpose: Provide further explanation for VLAN concept.
Source:
Verification Method: Test
Legacy ID:

The VLAN IDs shall be assigned to the use cases according to VLAN allocation table.

Every frame that is exclusively used for the particular use case shall be tagged by the sending ECU with the VLAN ID mentioned in the VLAN ID column of the table.

For the use cases with ‘ – ‘ in the VLAN ID column, the sending ECU shall not attach a VLAN tag.

661279 v5 Retagging and untagging rules for OBD TX port

Version history:
(5): Change test to Analysis AND Test.

Purpose: To define the rules when switch processing untagged frames in the project.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Untagged frames shall be tagged with the VID 10 and PCP 0 at the ingress of the switch OBD TX port.

Frames that are tagged with the VID 10 shall be untagged at the egress the switch OBD TX port.

661281 v1 **VLAN DEI**

Version history:
(1):

Purpose: The frames are not allowed to be dropped intentionally.
Source:
Verification Method: Test
Legacy ID:

All DEI bit in VLAN relevant Messages shall be configured as 0.

661280 v2 **VLAN priority**

Version history:
(2): To clarity that VLAN priority 's configuration requirements source.

Purpose: To clarity that VLAN priority 's configuration requirements definition.
Source:
Verification Method: Test
Legacy ID:

The VLAN priority should be defined according to the use cases in [ETH REF 24].

4.6.1.10 IP Routing and Firewalling

•Introduction

4.6.1.10.1 Introduction

The subject of routing focuses on the interpretation process of IP packets, which enables forwarding of IP packets with the shortest path or the most optimal path in a network. To be able to make a good forwarding decision, various forwarding algorithms and techniques can be used. Routers operate at Layer 3 of the OSI model and solves the problem of sending packets across one or more networks. An important function of the router is the separation of private networks from the public networks. By using this separation, it is easier to control traffic flow and implement security. Network devices with multiple interfaces need routing functionality to be able to communicate with its neighbors.

Routing plays a central part when multiple IP networks needs to communicate. Routing protocols are used for specifying how routers communicate with each other to provide information about the networks. The maturity level and benefits of existing routing protocols varies and the usage of a specific routing protocol depends on the use-case. However, their routing goals are the same, which are to provide an efficient and robust way to forward IP packets.

In the figure below, a relation between the different documents is described.

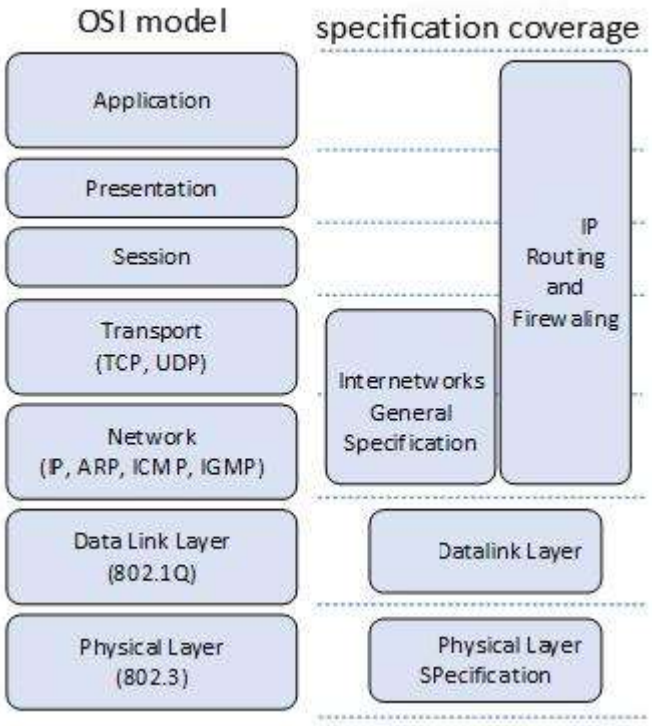


Figure: OSI model vs. Specification coverage

4.6.1.10.1.1 Abbreviations & Definitions

Abbr.	Stands for	Description
ARP	Address Resolution Protocol	A method for finding a host's Ethernet address from its Internet address.
ATP	Apple Talk Protocol	A set of protocols developed by Apple Inc.
CIDR	Classless Inter Domain Routing	CIDR is a methodology of allocating IP addresses and routing Internet Protocol packets.
DA	Destination Address	The destination physical address is the Link Layer Address to which the packet was sent.
DoS	Denial of Service	An attack against a system, to attempt to make the network resources unavailable for its intended users.
FCS	Frame Check Sequence	FCS refers to the extra checksum characters added to a frame in a communication protocol for error detection and correction.
ICMP	Internet Control Message Protocol	A core protocol of the Internet Protocol suite. Are typically used for generating errors in IP datagram's, diagnostics or for routing purposes.
IGMP	Internet Group Management Protocol	Protocol used to handle group membership requests. Routers use this protocol to query for active group members in a subnet.
IP	Internet Protocol	IP is a protocol used for communicating data

		across a packet-switched internetwork using the Internet Protocol Suite.
IPv6	Internet Protocol Version 6	128 bit addresses
IPv4	Internet Protocol Version 4	32 bit addresses
ISP	Internet Service Provider	ISP is a company that offers its customer's access to the Internet.
LAN	Local Area Network	Covers a small area to provide high bandwidth with low delays.
LQM	Link Quality Monitoring	A mechanism which monitors the quality of a specific link (interface).
MAC	Media Access Control	The physical address of a network interface.
NAT	Network Address Translation	Is responsible for modifying network address information in IP packets when moving one IP packet from one IP address space to another.
RFC	Request For Comments	An Internet Engineering Task Force (IETF) memorandum on Internet systems and standards.
PIM	Protoocol-Independent Multicast	Multicasting routing protocol according to RFC 2117.
SA	Source Address	The source physical address is the Link Layer Address of the host from which the IP packet was received.
SSID	Service Set ID	A name of a wireless local area network (WLAN).
TCP	Transmission Control Protocol	TCP is one of the core protocols of the Internet Protocol Suite.
UDP	User Datagram Protocol	UDP is one of the core members of the Internet Protocol Suite.
VLAN	Virtual LAN	VLAN is a group of hosts with a common set of requirements that communicate as if they were attached to the same broadcast domain, regardless of their physical location.
VPN	Virtual Private Network	VPN is a network that uses a public telecommunication infrastructure, such as the Internet, to provide remote offices or individual users with secure access to their organization's network.
QoS	Quality of Service	A resource reservation control mechanism to ensure a guaranteed performance of a data flow.
WAN	Wide Area Network	Interconnected LANs.

Definition	Description
Broadcast	Distribution of data to multiple recipients.
Dual stack	A device which supports both IPv6 and IPv4.
ECU	Electronic Control Unit. Device with a computer/microcontroller.
Egress	IP network traffic originating from the internal vehicle network and

	destined for host on the external network.
Gateway	Gateway is the old definition of today's IP router. Gateway is a network node equipped for interfacing with another networks.
Ingress	IP network traffic originating from external networks and destined for the ECU's within the vehicle network.
Interface	The connection between a router and one of its attached networks. An interface can also be referred as a link.
In-car network	The vehicle internal network composed of the In-Vehicle ECUs.
In-Vehicle ECUs	Internal vehicle network device which are only communicating within the vehicle.
Internet	A collection of IP based networks, which are connected via routers.
IP ECU	An Electronic Control Unit which communicates over an IP based network.
IP Link Manager	The IP Link Manager, implements a mechanism with the purpose to inform Local Software Components (within an ECU) or software components running on other ECU's that services are available
IPSec	IP Security is a protocol which creates a border between protected and unprotected parts of the network.
IP Network	IP network or IP subnet. It's possible for one physical IP network to be assigned multiple IP network/subnet numbers.
IP Routing	IP routing includes the mechanism to forward IP traffic.
Local Analysis Tool	A tool which provides software access during online analysis of a router ECU.
Local Configuration Data File	This software part contains parameters and/or data used locally to configure the executable software application. This software is used for handling a smaller part than the executable software application. A reason for this may be limiting the time for software download during car assembly in factory and in workshop.
Local Configuration Tool	A tool which provides software access to the ECU router. Could be used for online and offline analysis.
Multicast	Delivery of messages or information to a group of IP devices in a single transmission.
Network Interface	Network interface, or LAN adapter, is a computer hardware component designed to allow computers to communicate over a computer network.
Nomadic Device	A handheld wireless device such as a PDA or smartphone. External equipment which may connect to the vehicle network in a controlled way.
Packet	A formatted unit of data. Packet consists of two kinds of data: control information and user data (payload).
Port	The unique endpoints of communication on a host.
Router ECU	A IP ECU in the vehicle network that has two or more network interfaces and selectively interchanges data packets. Often called a level three Internet Protocol packet switch.
Silently Discard	The implementation discards (drops) the packet without further processing.
Switch	A network switch that connects network segments, which operates at the data-link layer of the OSI model.

4.6.1.10.1.2 External publications

Reference Number	Source	Title	Version or Date	Document Number
[IP Routing 2]	IEEE	IEEE 802.3 – CSMA/CD Access Method and Physical Layer Specification	2008	
[IP Routing 3]		Classless Inter-Domain Routing (CIDR)		RFC-1519
[IP Routing 4]		Requirements for IP Version 4 Routers		RFC-1812
[IP Routing 5]		IP Network Translator (NAT)		RFC-2663
[IP Routing 6]		A Framework for IP Based Virtual Private Networks		RFC-2764
[IP Routing 7]		Network Ingress Filtering		RFC-2827
[IP Routing 8]		Transition Mechanism for IPv6 Hosts and Routers		RFC-2893
[IP Routing 9]	RFC	Traiditional IP Network Address Translator		RFC-3022
[IP Routing 10]		An IPv6-to-IPv4 Transport Relay Translator		RFC-3142
[IP Routing 11]		Security Architecture for the Internet Protocol		RFC-4301
[IP Routing 12]		IP Authentication Header		RFC-4302
[IP Routing 13]		IP Encapsulating Security Payload (ESP)		RFC-4303
[IP Routing 14]	RFC	Deprecation of ICMP Source Quench Messages		RFC-6633
[IP Routing 15]	RFC	Changing the Default for Directed Broadcasts in Routers		RFC-2644
[IP Routing 16]	RFC	Session Initiation Protocol		RFC 3261

Note: If a later version is available, then this shall be consulted with document owner before used.

4.6.1.10.1.3 Requisite documents

Reference Number	Source	Title	LC
[IP Routing 1]		Internetworks General Specification	*LC : Internetworks General Specification*

•Requirements

4.6.1.10.2 Requirements

661148 v2 **DHCP - IP Reservation Table**

Version history:
(2): Verification Method: Test -> Analysis.

Purpose: To ensure that a specific host (with a fixed MAC address) gets the same IP address every time, if a DHCP server is used.

Source: REQPROD 264356

Verification Method: Analysis
Legacy ID: 55069v6

A DHCP server running on any router ECU shall support IP addresses reservation for Nomadic Devices based on their MAC addresses. It shall be possible to configure the IP Address Reservation table.

661139 v1 Dual Stack

Version history:
(1):

Purpose: To ensure that communication with both IPv4 and IPv6 devices is handled by a router ECU.
Source: RFC 2893
Verification Method: Test
Legacy ID: 55042v4

Dual stack support shall be implemented on the router ECU that may receive an IPv4 and/or IPv6 address from e.g. external Internet Service Providers (ISP).

661156 v1 Fragmented ICMP

Version history:
(1):

Purpose: While they can occur, fragmented ICMP packets are usually not seen, except in conditions which could be labeled as "malicious".
Source:
Verification Method: Test
Legacy ID: 55062v3

Fragmented ICMP packets shall be blocked without further analysis.

661178 v2 ICMP Unreachable

Version history:
(2): Verification Method: Test -> Analysis.

Purpose: To define the firewalling rule for ICMP unreachable messages.
Source:
Verification Method: Analysis
Legacy ID:

Firewalls should be configured to elicit or suppress ICMP unreachable messages. ICMP unreachable messages should be filtered to allow only known sources, for example those from known subnets. The following example illustrates filtering ICMP unreachable messages to permit only messages to known sources:

!
icmp permit host 10.0.0.1 unreachable mgmt

!

To alleviate CPU utilization, ICMP unreachable messages should be limited to one packet every second by default.

Here is the example for configuration in linux system. ICMP unreachable message generation can be disabled using the global configuration command **icmp deny any unreachable <interface>**. ICMP unreachable rate limiting can be changed from the default using the **icmp unreachable rate-limit** *rate* **burst-size***size* global configuration command.

661143 v2 **IP Broadcast Address Limitation**

Version history:
(2): Verification Method: Test -> Analysis.

Purpose: To block obsolete forms of broadcast addresses.
Source: RFC 1812
Verification Method: Analysis
Legacy ID: 54907v3

Broadcast addresses of the form (<Network-prefix>, 0) including 0.0.0.0 are obsolete, thus shall be discarded.

661159 v2 **IP Broadcast storms**

Version history:
(2): Verification Method: Test -> Analysis.

Purpose: To avoid having storm of broadcasts or multicasts from overwhelming the in-car network.
Source:
Verification Method: Analysis
Legacy ID: 55051v4

The router ECU shall be able to identify a broadcast storm, disable the subjected interface and re-enable the interface again after 1000 ms. When the interface is re-enabled and if a broadcast storm is still ongoing, the interface shall be disabled and once again enabled after a time configured as a back-off timer. It should be possible to limit the rate of the allowed broadcasts and the timers via parameters by means of local configuration.

661175 v1 **IP Fragmentation**

Version history:
(1):

Purpose: The IP Fragmentation is not allowed.
Source:
Verification Method: Test
Legacy ID:

IP Fragmentation shall be forbidden.

661149 v1 **IP Implementation**

Version history:
(1):

Purpose: To make sure that all IP-ECUs in the network have the same feature base.
Source: LC: Internetworks General Specification
Verification Method: Analysis
Legacy ID: 54894v4

IP routers shall implement the protocol stack according to [LC: Internetworks General Specification].

661150 v2 **IP Multicast**

Version history:
(2): Verification Method: Test -> Analysis.

Purpose: To ensure that router ECUs does not block IP multicast packets.
Source: RFC 3376
Verification Method: Analysis
Legacy ID: 352479v3

The router ECU shall support forwarding of IP Multicast messages.

661151 v2 **IP Multicasting Protocol**

Version history:
(2): Verification Method: Analysis AND Test -> Analysis.

Purpose: To be able to establish multicast group memberships.
Source: RFC 3376, RFC 4606
Verification Method: Analysis
Legacy ID: 352716v2

The router ECU shall support Internet Group Management Protocol, IGMP version 3, to maintain host group membership on a local network.

661144 v2 **IP NAT**

Version history:
(2): Verification Method: Test -> Analysis.

Purpose: To ensure that in-car IP-ECUs can access the public internet and in some cases intentionally allow external traffic to enter the in-car network.
Source: RFC 3022
Verification Method: Analysis
Legacy ID: 54909v5

Router ECUs with a WAN connection shall implement a Traditional IP Network Address Translator, Traditional NAT according to [RFC 3022].

Note: The NAT shall be able to handle IPv6 from external networks.

661176 v1 **IP Option**

Version history:
(1):

Purpose: To define the IP option is not used.
Source:
Verification Method: Test
Legacy ID:

IP Option shall be forbidden.

661152 v1 **IPv4 router implementation**

Version history:
(1):

Purpose: To ensure compatibility with other IP networks and to make sure of a robust vehicle forwarding mechanism.
Source: RFC 1812, RFC 6633, RFC 2644
Verification Method: Analysis
Legacy ID: 54895v4

The design and implementation of a router ECU shall follow the requirements according to [RFC 1812], [RFC 2644] and [RFC 6633]. This routing specification supersedes theses RFCs where there is a conflict.

661167 v2 **Link Monitoring**

Version history:
(2): Verification Method: Test -> Analysis.

Purpose: To possess the link quality information for interfaces.
Source: REQPROD 352558
Verification Method: Analysis
Legacy ID: 352480v2

The router shall implement a feature to monitor the quality of each interface.
The Link Monitoring shall set a number of counters defined in the table below:

Parameter	Description	Example of trigger conditions
Tx Packets	Number of transmitted	The router sends/forwards a

	packets	packet.
Tx Bytes	Transmitted number of bytes	The router sends/forwards a packet of a known size.
Rx Packets	Number of received packets	The router receives a packet
Rx Bytes	Received number of bytes	The router receives a packet of a known size.
Tx Packets Drop	Number of transmitted packets which is dropped	Router sends a packet but due to errors, collisions or router resource limitations fails.
Rx Packets Drop	Number of received packets which is dropped	Router receives a packet but due to errors, collisions or router resource limitations fails.
Errors	Number of transmission errors.	If used error detection mechanism detects a fault, e.g. checksum fails. This counter value should be increased. Additionally, on a wireless network, a noisy radio-frequency environment can cause high error rate. Also these errors should be covered by the counter

661168 v2 **Link Monitoring - ECU reboot**

Version history:
(2): Verification Method: Test -> Analysis.

Purpose: To define the reset behavior of the LQM counters.
Source: REQPROD 352480
Verification Method: Analysis
Legacy ID: 352482v3

The link-monitoring counters described in [REQPROD 352480] shall be reset when the router ECU is rebooting.

661169 v2 **Link Monitoring - logging**

Version history:
(2): Verification Method: Test -> Analysis.

Purpose: To ensure a strategy for logging the link monitoring data.
Source: REQPROD 352480, REQPROD 352558

Verification Method: Analysis
Legacy ID: 352481v3

When the link-monitoring feature is enabled, it shall be possible to store all link-monitoring parameters per interface into a log-file. The log-file shall be of the type ringbuf with a max size of 10MB. The size shall be configurable. Two triggers for writing the information to the ringbuf shall be implemented. One being a configuration parameter that triggers a write action every 5 seconds and the second being a manual action included in the Analysis Tool.
<<Interface Name>> shall be replaced with the physical interface name at the router ECU.

The log-file content shall in the beginning of the file contain the following:

o Date: <<Year – Month – Data – Time>>
  << *Iteration starts here* >>
o Interface: << Interface Name >>
o ECU Name: << ECU Name >>
o ECU Hardware ID: << ECU Hardware ID >>
o ECU Software ID: << ECU Software ID >>
o RoutingTable: << Routing Table information >>
o … followed by information according to REQPROD 352480.
<< *Further iterations…*>>

661170 v2 Link Monitoring Accessibility

Version history:
(2): Verification Method: Test -> Analysis.

Purpose: To define how to access the Link Monitoring parameters.
Source: REQPROD 352480
Verification Method: Analysis
Legacy ID: 352558v3

Link Monitoring parameters shall be accessible for analysis via an Analysis Tool, which the supplier shall deliver.

661171 v2 Link Monitoring Activation

Version history:
(2): Verification Method: Test -> Analysis.

Purpose: To ensure it will be possible to enable and disable link quality monitoring.
Source: REQPROD 352480, REQPROD 352558
Verification Method: Analysis
Legacy ID: 352484v3

Two triggers for writing the link-monitoring information of [REQPROD 352480] shall be implemented. One being a configuration parameter that triggers a write action every 5 seconds and the second being a manual action included in the Analysis Tool. The default state shall be disable.

661160 v2 MAC Filtering

Version history:
(2): Verification Method: Test -> Analysis.

Purpose: To control the set of hosts which should be allowed within the in-car network and to block specific hosts if needed.
Source:
Verification Method: Analysis
Legacy ID: 55070v3

Router ECU shall support MAC filtering based on both source and destination MAC addresses.
The MAC filtering rules shall be configurable.

661146 v2 NAT support for SIP

Version history:
(2): Verification Method: Test -> Analysis.

Purpose: To allow SIP traffic into the in-car network when needed.
Source: RFC 3261
Verification Method: Analysis
Legacy ID: 127382v3

It shall be possible to configure the router ECU's firewall to allow the Session Initiation Protocol, SIP traffic from external sources. Default state shall be Enabled.

661147 v2 Port Forwarding

Version history:
(2): Verification Method: Test -> Analysis.

Purpose: To ensure the support for static Port forwarding.
Source: RFC 3022
Verification Method: Analysis
Legacy ID: 352899v4

Static Port forwarding shall be implemented and configurable with the following parameters.

Parameter	Description
Friendly Name	A friendly name to identify the rule
Internal Port	Port number used by internal application
Internal IP Address	The IP Address of the internal IP-ECU
External Port	Port number opened towards WAN/Internet
External IP Address	The IP Address of the external host
Traffic Type	Used protocol for the rule: UDP, TCP or ANY

661153 v1 Routing Table Content

Version history:
(1):

Purpose: To ensure that the router ECUs have the necessary data to be able to make a forwarding decision
Source: RFC 1812
Verification Method: Analysis
Legacy ID: 54948v4

A routing table shall at least contain the following: *Network Destination, Netmask, Gateway, Interface*. The routing table shall have the capacity to hold 5K routes.

Network Destination: The destination network (or host).
Netmask: Network Mask.
Gateway: The gateway to use to reach the specified destination.
Interface: The identity of the network interface through which the packet is sent to reach the next hop.

661154 v2 Static Routing

Version history:
(2): Verification Method: Test -> Analysis.

Purpose: To ensure the means to configure router ECUs with static routing tables.
Source: RFC 1812
Verification Method: Analysis
Legacy ID: 54950v3

A router ECU shall support configuration of static routes.

661155 v2 Throughput

Version history:
(2): Verification Method: Test -> Analysis. AND Test.

Purpose: To ensure a high performance on the network.
Source:
Verification Method: Analysis AND Test
Legacy ID: 54897v5

The minimum throughput of a router ECU shall be at least 90 percent of the max bandwidth (full duplex) at Layer 3.

661140 v1 Translation

Version history:
(1):

Purpose: To set the mechanism for translating between an IPv6 network to a IPv4 network.

Source: RFC 3142
Verification Method: Test
Legacy ID: 55044v3

The router ECU that implements a dual stack according to [REQPROD 55042] shall also implement a translation mechanism to translate external IPv6 traffic into IPv4 traffic.

4.6.1.11 Time Sensitive Network

•Introduction

4.6.1.11.1 Introduction

4.6.1.11.1.1 Definitions & Abbreviations

Abbr.	Stands for
AAF	AVTP Audio Format
AED	Automotive Ethernet Device.
AED-A	Automotive Ethernet Audio Device.
AED-B	Automotive Ethernet Bridge Device
AVB	Audio & Video Bridging
AVTP	Audio Video Transport Protocol
CRF	Clock Reference Format
gPTP	generalized Precision Time Protocol
isGM	is GrandMaster
MTT	Maxmum Transit Time
PCP	Priority Code Point
PPS	pulse per second
SR	Stream Reservation

4.6.1.11.1.2 References

Reference ID	Name
[AVB1]	Avnu Functional and Interoprerability Specification Rev. 1.5
[AVB2]	IEEE 802.1AS
[AVB3]	AVnu 802.1AS Recovered Clock Quality Testing
[AVB4]	IEEE 1722a

•Requirements

4.6.1.11.2 Requirements

4.6.1.11.2.1 802.1AS / gPTP

4.6.1.11.2.1.1 Introduction

This document is a general specification of Datalink layer with focus on how time synchronization is achieved within an IP-based in-car network architecture. The strategy is to follow the IEEE standard 802.1AS as much as possible, but as the in-car network differs from network application that this standard was originally aimed for, there are some limitations introduced to better make use of the protocol.

802.1 AS also known as gPTP, generalized Precision Time Protocol is a profile of IEEE 1588 PTP protocol. The two IEEE standards describe techniques for synchronizing distributed clocks within a network. Even though both protocols are very similar, they are different in aspects that make them incompatible. While in a gPTP domain all network devices must support the protocol, that is not necessary for PTP but which comes at the cost of accuracy. In simplified terms, gPTP defines amongst others the roles, grandmaster, bridge and slave, which are of importance in a time-synchronized domain. The grandmaster is the network device that host the best clock, which is also the clock that all other clocks tune in. Bridges are typically gPTP enabled switches that connect the grandmaster to the slaves. The slaves are any hosts that want to synchronize their clocks to that of the grandmaster. Together all these types of devices create a gPTP domain.
An in-car network architecture differs from conventional public networks in the way that they are mostly static and changes are rare after production start, which is a fact that can be used to simplify the design of a time, synchronized in-car network

This document further describes how we plan to adopt it within the in-car network and the reader is assumed to have knowledge of the IEEE 802.1 AS standard.

This document will define the general Datalink requirements for implementing a time synchronized in-car network domain based on IEEE standard 802.1 AS.

4.6.1.11.2.1.2 General Requirement

871923 v1 **Announce and Signaling Message**

Version history:
(1):

Purpose: To avoid additional messages being used
Source:
Verification Method: Test
Legacy ID:

Announce and Signaling message shall not be used.

660904 v1 **BMCA, Best Master Clock Algorithm**

Version history:
(1):

Purpose: To use the fact that the in-car network is static to make the time synchronization procedure simpler and faster.
Source: [ETH REF 10]
Verification Method: Analysis
Legacy ID: 435829v0

As the in-car network is static, support for the BMCA, Best Master Clock Algorithm is not necessary. If BMCA is

available it shall be inactivated.

660905 v1 Clock Source Implementation

Version history:
(1):

Purpose: To ensure the compatibility with the IEEE 802.1AS standard.
Source: *[ETH REF 10]*
Verification Method: Analysis
Legacy ID: 356936v2

gPTP shall be implemented according to the *IEEE 802.1 AS,* [ETH REF 10]standard unless stated otherwise.

660907 v4 Configuration Update

Version history:
(4): Change Req ID to 660818.

Purpose: To state how the locally configurable parameters shall be handled.
Source: *[REQID 660818].*
Verification Method: Analysis
Legacy ID: 436223v0

In the requirements within this document, there are parameters required to be "configurable". Configurability of these parameters shall be implemented according to *[REQID 660818].*

908811 v1 Domain number

Version history:
(1):

Purpose: To define what domain we shall be used in time sync network.
Source:
Verification Method: Test
Legacy ID:

The domain number in PTP messages shall always be 0. There shall be only one time domain in the network.

660906 v1 Grandmaster Clock Selection

Version history:
(1):

Purpose: To avoid the grandmaster selection process and to instead configure the grandmaster clock used every time or after a reset, and to ensure an accelerated startup after a reset.
Source: *[ETH REF 24]*
Verification Method: Analysis
Legacy ID: 352857v3

ECUs shall support a solution to be preconfigured to be the grandmaster. The grandmaster role is allocated within *[ETH REF 24].*

871887 v1 Hardware timestamping for Ethernet

Version history:
(1):

Purpose: For better accuracy to Time Synchronization.
Source:
Verification Method: Analysis
Legacy ID:

The Ethernet controller shall support to capture hardware timestamp for ingress/egress PTP event message.

908805 v1 MAC address and ethertype

Version history:
(1):

Purpose: To define the destination MAC address and ethertype used for all PTP messages.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The destination MAC address for all PTP messages shall be 0x01-80-C2-00-00-0E, and the EtherType field shall be 0x88F7.

660909 v1 Multiple time domains

Version history:
(1):

Purpose: To be able to have multiple time domains in parallel.
Source: N/A
Verification Method: Analysis
Legacy ID: 435849v0

Time-aware systems shall be able to handle and respond to multiple time domains in parallel.

871924 v1 Reference Implementation

Version history:
(1):

Purpose: To ensure a good design for Generalized Precision Time Protocol (gPTP) in a static automotive network
Source:
Verification Method: Analysis AND Test
Legacy ID:

The ECU shall implement and follow the guidelines for Generalized Precision Time Protocol (gPTP) in *[AVB_1 : Automotive Ethernet AVB Functional and Interoperability Specification]*.

908810 v1 Sync receipt timeout

Version history:

(1):

Purpose: The Sync message shall be received periodically and error shall be reported when Sync receipt timeout happened according to this value.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Sync receipt timeout as defined in *[ETH13 : IEEE 802.1 AS]* shall be configured to 3 as default.

871926 v1 Synchronization Accuracy

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

Between the sync points, two endpoint systems (≤ 7 hops) have peak-to-peak difference within 1us.
Suggested verification method: measure the difference via PPS interface.

871925 v1 Time Interface for Upper Client

Version history:
(1):

Purpose: To define the interface for upper application getting the synchronized time.
Source:
Verification Method: Test
Legacy ID:

Time slave shall update the slave time(the synchronized time maintained at the slave) when receiving Sync & Follow_Up message through Ethernet time sync or CAN time sync and provide interface to acquire synchronized time for upper applications.

660912 v1 Timestamp Resolution

Version history:
(1):

Purpose: To ensure an accurate time stamping of the receipt and transmission of time synchronization packets.
Source: *[ETH REF 10]*
Verification Method: Analysis
Legacy ID: 363502v2

The resolution of the timestamp shall be in the order of 1 ns.

871921 v1 Timestamp Resolution

Version history:
(1):

Purpose: To ensure an accurate time stamping of the receipt and transmission of time synchronization packets.
Source:
Verification Method: Analysis
Legacy ID:

According to *[IEEE_8 : IEEE802.1AS]*, the resolution of timestamp measured by local hardware clock shall be less than or equal to 40ns.

908806 v1 **Untagged for PTP message**

Version history:
(1):

Purpose: As specified in IEEE 802.1AS, PTP messages shall be VLAN untagged.
Source:
Verification Method: Test
Legacy ID:

All the PTP messages(that is Sync, Follow_Up, Pdelay_Req, Pdelay_Resp, Pdelay_Resp_Follow_UP) used in time synchronization shall be untagged as defined in [ETH TS 1 : IEEE 802.1AS].

871922 v1 **gPTP Message Exchange**

Version history:
(1):

Purpose: To ensure that the correct exchange mechanism is used
Source:
Verification Method: Test
Legacy ID:

For gPTP message exchange, only two-step peer-to-peer mechanism shall be used.

4.6.1.11.2.1.3 Sync intervals

92885 v2 **Nominal Sync interval**

Version history:
(2): Add a purpose for the requirement.

Purpose: To define the parameters required for gPTP, Nominal Sync Interval.
Source:
Verification Method: Test
Legacy ID:

The actual Sync interval, i.e. the time interval between two consecutive Sync messages emitted by the Grandmaster, must conform to the initialLogSyncInterval and the operLogSyncInterval as defined in REQ92884 the allowed interval can be between 2^{-4} s = 62.5ms and 2^{-2} s = 250ms.

Nevertheless, for ease of analyzation during integration, it is recommended to use a nominal Sync interval of 125ms. This nominal interval would offer a margin for internal deviations of several ms within the Grandmaster ECU without exceeding the allowed intervals.

Proposal:

Nominal Sync Interval = 125ms

92884 v2 **initialLogSyncInterval and operLogSyncInterval**

Version history:
(2): Add a purpose for the requirement.

Purpose: To define the parameters required for gPTP, initialLogSyncInterval and operLogSyncInterval.

Source:
Verification Method: Test
Legacy ID:

The sync intervals are proposed to be parametrized as follows:

initialLogSyncInterval = -3

operLogSyncInterval = -3

Justification: the value of -3 is the default value according to the [AVB1] and has been commonly used in various (non-Automotive) AVB scenarios.

While it is possible to have the interval being extended, this would require the use of Signaling messages which will increase the complexity of the system without promising a noticeable benefit (only minor reduction of network overhead and CPU load).

Additionally, there is also an oversight in the gPTP specification that might lead to unexpected behaviour upon receiving Signaling Messages (see note in [AVB1] in section 6.2.4, lines 175-179).

For this reason, it is recommended to not use Signaling messages and instead use the same interval for the entire life cycle.

4.6.1.11.2.1.3.1 AVnu 802.1AS Recovered Clock Quality Testing

This functionality is in fact not specified in [AVB2], but it would enable testing of the recovered clock of a gPTP slave. An alternative approach would be a PPS output line that is externally accessible for measurements.

94974 v2 **Testablity function**

Version history:
(2): Add a purpose for the requirement.

Purpose: To clarify that testability function in [AVB3] should be supported.
Source:
Verification Method: Analysis
Legacy ID:

Proposal:

Testability function according to [AVB3] should be supported

4.6.1.11.2.1.4 Pdelay_Req intervals

92889 v2 **InitialLogPdelayReqInterval and operLogPdelayReqInterval**

Version history:
(2): Add a purpose for the requirement.

Purpose: To define the parameters required for gPTP,InitialLogPdelayReqInterval and operLogPdelayReqInterval.

Source:
Verification Method: Analysis
Legacy ID:

The Pdelay_Req intervals are proposed to be parametrized as follows:

initialLogPdelayReqInterval = 0

operLogPdelayReqInterval = 0

Justification: the value of 0 is required according to the [AVB1] for the initialLogSyncInterval. While it is allowed that the Pdelay_Req interval is increased to up to 3 after a short time of stabilization, it is not recommended to change the interval as this will introduce the complexity of the system without promising a noticeable benefit (only minor reduction of network overhead and CPU load).

92891 v2 **Nominal Pdelay_Req interval**

Version history:
(2): Add a purpose for the requirement.

Purpose: To define the parameters required for gPTP, Nominal Pdelay_Req interval.

Source:
Verification Method: Test
Legacy ID:

The actual Pdelay_Req interval, i.e. the time interval between two consecutive Pdelay_Req messages emitted by an ECU, must conform to the initialLogPdelayReqInterval and the operLogPdelayReqInterval as defined in REQ92889.

For a value of 0, the nominal interval would be 2^{-0} s =1s. However, it is permitted by the standard that the actual interval varies such as if the LogPdelayReqInterval value varies by 1, i.e. for a value of 0, the allowed interval can be between 2^{-1} s = 500ms and 2^{1} s = 2s.

Nevertheless, for ease of analyzation during integration, it is recommended to use a nominal Pdelay_Req interval of 1s. This nominal interval would offer a margin for internal deviations of several ms within the ECU without exceeding the allowed intervals.

Proposal:

Nominal Pdelay_Req Interval = 1s

4.6.1.11.2.1.5 Storing measured gPTP values for next startup

92893 v1b1 **NeighborPropDelay**

Version history:
(1b1): Add a purpose for the requirement.

Purpose: To define the parameter required for gPTP after production and service, neighborPropDelay.

Source:
Verification Method: Analysis
Legacy ID:

Before the Pdelay mechanism starts and shows a first result for the measured path delay, every ECU should use a predefined value. This value can be updated with the results from the Pdelay algorithm, but at least at the very first startup after production or service, the ECU must use a predefined value.

This can be any regular value above 0,

e.g. derived from measurements in the integration phase, or it also can be a value of exactly 0. If it is 0, then the ECU must wait until the first measured delay of the Pdelay algorithm.

In the field, this situation only applies for the very first startup after production or after service and delays the regular gPTP operation only by the duration of one Pdelay_Req interval. For this reason a value of 0 can be justified. Alternatively, an appropriate value must be determined in a later phase of specification or integration.

Proposal:

Set stored value of neighborPropDelay to 0 after production and service.

92895 v2 **RateRatio and NeighborRateRatio**

Version history:
(2): Add a purpose for the requirement.

Purpose: To make sure the value of rateRatio and neighborRateRatio after sleeping.
Source:
Verification Method: Analysis
Legacy ID:

Storing the rateRatio and neighborRateRatio values before sleeping is optional according to the [AVB1].

Both values highly depend on the temperature of the ECUs in the car. As the temperature is expected to differ significantly between letting the car sleep and waking it up, any values of the rateRatio and neighborRateRatio that have been stored before sleeping will most likely be imprecise.

Proposal:

Set stored rateRatio to 0 after sleeping

Set stored neighborRateRatio to 0 after sleeping

4.6.1.11.2.1.6 Pdelay Requests from AED-B master ports

An AED-B may send Pdelay Requests from Master ports according to [AVB1], but it is not necessary.

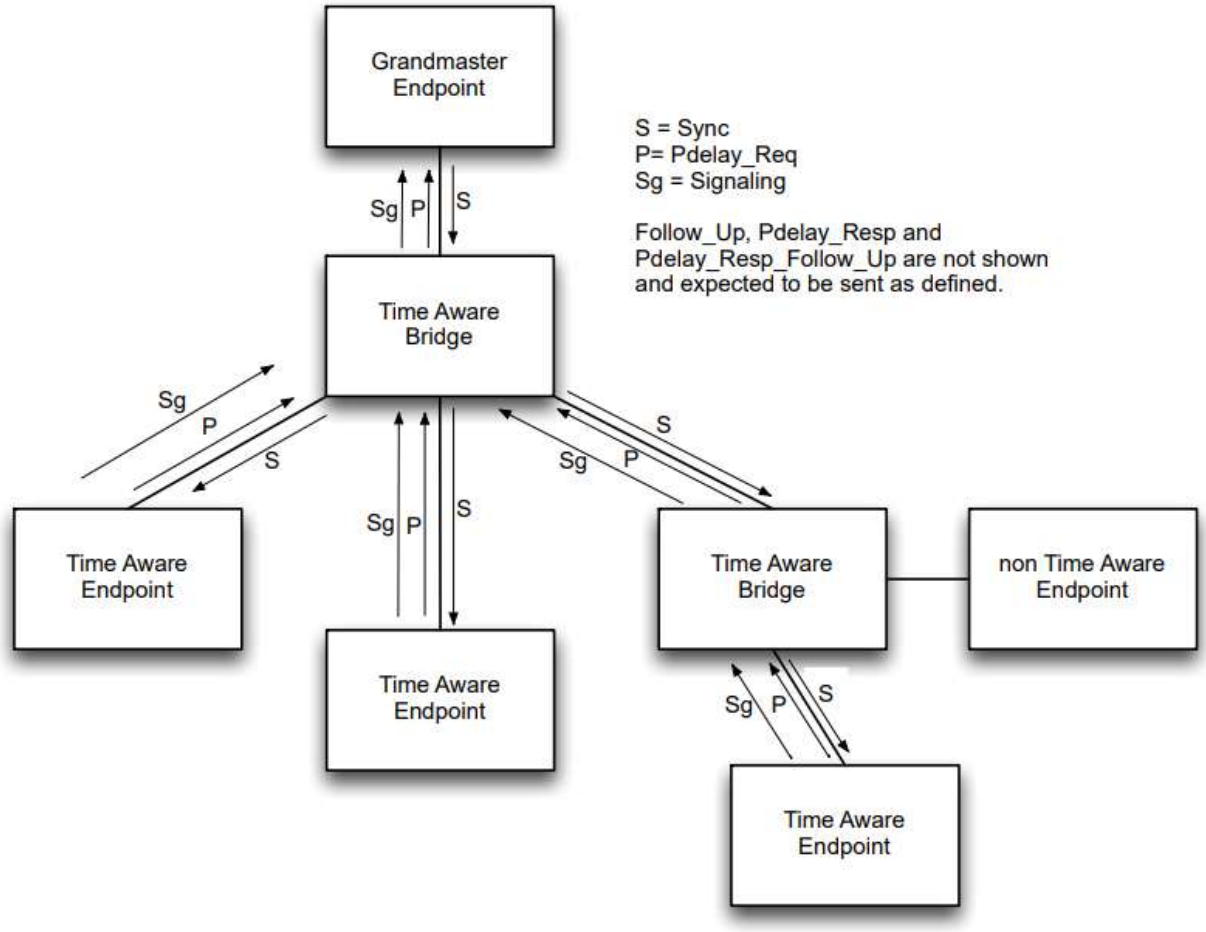


Figure: Flow of Timing Messages

94967 v2 **Pdalay Request send**

Version history:
(2): Add a purpose for the requirement.

Purpose: Specify the method of link delay measurement.
Source:
Verification Method: Test
Legacy ID:

As there is no need to perform delay measurement in both directions, for reduction of complexity, it is recommended that an AED-B sends Pdelay_Req messages only on its slave port.

4.6.1.11.2.1.7 Time to wait for initial Sync message in AED-B

After a specified time to wait for receiving an initial Sync message on its slave port, an AED-B shall start emitting Sync and Follow_Up messages based on its own local clock as described in [AVB1] in section 6.3 lines 215-219.

Early implementations of Grandmaster ECUs might require a longer time for startup than in later stages of integration. Of course, the manufacturer of the Grandmaster ECU should be encouraged to provide short startup times for the gPTP Grandmaster functionality even at early stages of integration. However, it might be necessary to adjust the time to wait for initial Sync messages in AED-B devices to an appropriate value. An important aspect of the time to wait is the required individual AVB startup time of each ECU; ECUs that are involved in rear end camera image transmission should have an overall AVB startup time (i.e. until the first image has arrived at the display) below 2s.

92897 v3 **Time to wait for initial Sync message**

Version history:
(3): Delete the time requirement during the development phase, and change the time requirement after SOP to 2 seconds

Purpose: To define Time To Wait for initial Sync message value.
Source:
Verification Method: Test
Legacy ID:

AED-B: time to wait for an initial Sync message = 2s.

4.6.1.11.2.2 IEEE1722 / AVTP

4.6.1.11.2.2.1 AAF parameters

4.6.1.11.2.2.1.1 Supported audio sample rate

According to the [AVB1], at least one sample rate of 44.1kHz and 48kHz are required for AAF (AVTP Audio Format) over [AVB4], while additional rates are also allowed.

Thus, the actual audio sample rate that should be used in an Automotive AVB system significantly depends on the audio use cases. This should be reconsidered when the actual use case scenarios have been defined.

As telephony is expected to be an important use case,

e.g. for assistant services like E-Call, it is reasonable to adapt audio sample rates that are typical for mobile telephony. Such typical sample rates are 8kHz or 16kHz, but other audio codecs used in mobile telephony provide rates of up to 48kHz. This sample rate can also be suitable for other audio use cases like entertainment.

92899 v2 **Supported audio sample rate**

Version history:
(2): Add a purpose for the requirement.

Purpose: Give a proposal of AFF audio sample rate.
Source:
Verification Method: Analysis
Legacy ID:

Proposal:

AAF audio sample rate = 48kHz

4.6.1.11.2.2.1.2 AED-A Talker Timestamp mode

The full timestamp mode is less complex than the sparse timestamp mode.

92902 v2 **AED-A Talker Timestamp mode**

Version history:
(2): Add a purpose for the requirement.

Purpose: Give a proposal of AED-A talker timestamp mode.
Source:
Verification Method: Analysis
Legacy ID:

Proposal:

AED-A Talker Timestamp mode: full

4.6.1.11.2.2.1.3 Dynamic Channel Layout EventField

If telephone audio is the only use case for audio, only one channel (mono) is required.

However, as soon as other audio use cases are planned, the EventField has to be set to a value according to table 14 in the [AVB1].

92904 v2 **Dynamic Channel Layout EventField**

Version history:
(2):

Purpose: Give a proposal to define dynamic channel layout eventfield.
Source:
Verification Method: Analysis
Legacy ID:

Dynamic Channel Layout EventField: 0x1

DRAFT NOTE:

Further research about using the value 0x0 for static layouts according to [AVB4] are required.

4.6.1.11.2.2.1.4 Number of channels

If telephone audio is the only use case for audio, only one channel (mono) is required.

However, as soon as other audio use cases are planned, additional channels must be considered.

92907 v2 **Number of channels**

Version history:
(2): Add a purpose for the requirement.

Purpose: Give a proposal of number of channels.
Source:
Verification Method: Analysis
Legacy ID:

Proposal:

Number of channels: 1

4.6.1.11.2.2.2 Video format

AVnu recommends H.264 or Motion JPEG (MJPEG) as video format or MPEG2-TS as container format. However, specification of the format (and its parameters) for transmitting video is outside of this scope.

4.6.1.11.2.2.3 Clock Reference Format (CRF)

The necessity of a Clock Reference Stream highly depends on the use cases and on the network topology.

DRAFT NOTE: tbd

4.6.1.11.2.3 Traffic Class parametrization

DRAFT NOTE:

Further research for individual use cases are required in this clause.

4.6.1.11.2.3.1 SR classes

The number of required SR classes highly depends on the use cases and on the network topology. With only a telephony use case, it is sufficient to use only one SR class. For additional use cases, additional SR classes might become necessary.

92918 v2 **SR classes**

Version history:
(2): Add a purpose for the requirement.

Purpose: To define the value of Number of SR classes and SR class(When only a telephony use case)
Source:
Verification Method: Analysis
Legacy ID:

Proposal:

Number of SR classes: 1 (telephony only)

SR class: A

4.6.1.11.2.3.2 SR Class Template(s)

As the SR class template for the telephony audio use case, tpl_1333 is recommended.

In an AVTPDU in the tpl_1333 template, 64 samples, sampled with a rate of 48kHz, are transmitted which is designed for efficient usage of DSP and DMA modules in AED-A devices. This template leads to an AVTPDU rate of 750 frames per second.

92919 v2 **SR Class Template(s)**

Version history:
(2): Add a purpose for the requirement.

Purpose: To define SR class template for the telephony audio.
Source:
Verification Method: Analysis
Legacy ID:

Proposal:

SR class template for 48kHzaudio: tpl_1333

4.6.1.11.2.3.3 Mapping between PCP and SR Class

The Mapping between PCP and SR Class should follow the recommendation in the [AVB1] (see clause 8.1, lines 32-40). If telephony is the only use case, then the telephony stream should use SR Class A, PCP3 and SR Class ID 6 in order to guarantee the required bandwidth and short delays for this stream.

92920 v2 **Mapping between PCP and SR Class**

Version history:
(2): Add a purpose for the requirement.

Purpose: To define the mapping between PCP and SR class
Source:
Verification Method: Test
Legacy ID:

Proposal:

Telephony:

SR Class: A

SR Class ID: 6

PCP (VLAN priority): 3

4.6.1.11.2.3.4 Maximum Transit Time (MTT)

For each SR class, an individual MTT has to be defined.

Principally, reasonable values for this should be calculated in order to reduce the delay between the Talker and Listener application layers. However, exact calculation requires knowledge about the network topology, including delay times within all involved ECUs, especially in an AED-B.

A big part of the delay within one 100Mbps link is basically defined by the Store&Forward delay at the ingress and by the queueing delay at the egress, each of these two components to be accounted to be 123.360us in the MTT calculation, summing up for every AED-B on the path.

Additionally, the AVTP stack implementations on Talkers and Listeners will cause additional, individual delays, as well as the PHYs on every involved ECU and as well as the switching fabrics in every AED-B along the path. As long as these individual delays are not known for every component that is going to be used in the network.

The AVnu specification states default MTT values for the four SR Class templates; the default value for the tpl_1333 is 15ms. Although this value might be much higher than expected/necessary in a realistic Automotive network scenario, it is a suitable and sufficient value for an initial AVB concept

and might be reduced when detailed delays for the delaying components of the involved ECUs are known.

92922 v2 **Maximum Transit Time (MTT)**

Version history:
(2): Add a purpose for the requirement.

Purpose: To define the value of MTT for audio.
Source:
Verification Method: Test
Legacy ID:

Proposal:

MTT for audio: 15ms

4.6.1.11.2.4 Exception handling

4.6.1.11.2.4.1 Behaviour upon detection of lost or late data

4.6.1.11.2.4.1.1 Lost or late video data

94968 v2 **Lost or late video data**

Version history:
(2): Add a purpose for the requirement.

Purpose: To define the exception handing rules for rear view camera display.
Source:
Verification Method: Test
Legacy ID:

A display must not show video rear view camera images that are out of date as this is a safety-relevant use case. Thus, if such a use case will be defined, an ECU must show a black screen when detecting lost or late video data from a rear view camera.

4.6.1.11.2.4.1.2 Lost or late audio data

In case that an AED-A Listener ECU detects lost or late audio data, the AVnu specification recommends to use suitable and practical methods to gracefully mitigate the loss of data.

However, a detailed definition of such methods is outside the scope of AVB and thus outside the scope of this doc.

4.6.1.11.2.5 802.1CB

4.6.1.11.2.5.1 Introduction

As defined in IEEE 802.1CB-2017, Ethernet packet can be replicated into different pieces at the sender ECU/switch side, and can be eliminated into just one packet at the receiver ECU/switch side. By transmitting frames through multiple paths, it improves reliability for end-to-end transmission.

This container defines the general requirements for IEEE 802.1CB, which shall be followed by each ECU that shall support 802.1CB.

4.6.1.11.2.5.2 Requirements

871930 v1 **FRER Functions**

Version history:
(1):

Purpose: Define which FRER functions that shall be supported in an CB enabled ECU.
Source:
Verification Method: Test
Legacy ID:

For the CB enabled ECU the following *[IEEE_2 : IEEE802.1CB]* functions is mandatory:

-Sequence generation and sequence recovery

-Individual recovery

-Stream splitting

-Sequence encode/decode

-Stream identification

871933 v1 **Latent Error Detection Support**

Version history:
(1):

Purpose: Latent error detection function shall be supported to monitor the status for FRER function.
Source:
Verification Method: Test
Legacy ID:

Latent error detection function as defined in *[IEEE_2 : IEEE802.1CB]* shall be supported per sequence recovery function.

871935 v1 **Redundancy Tag Support**

Version history:
(1):

Purpose: Redundancy tag must be supported for CB frames and used for sequence recovery.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Redundancy tag shall be supported by sequence encode/decode function. HSR-tag and PRP-tag is optional.

871932 v1 Sequence Recovery Algorithm

Version history:
(1):

Purpose: Define the algorithm supported for sequence recovery.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Both vector recovery algorithm and match recovery algorithm shall be supported by the recovery function.

871928 v1 Standard Compatibility

Version history:
(1):

Purpose: The implementation of frame replication and elimination shall be compatible with IEEE standards.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The implementation of frame replication and elimination for reliability shall follow *[IEEE_2 : IEEE802.1CB]*.

871931 v1 Stream Identification Function

Version history:
(1):

Purpose: Define stream identification function type which is needed for FRER.
Source:
Verification Method: Analysis AND Test
Legacy ID:

For the switch is it mandatory to support following stream identification functions as defined in *[IEEE_2 : IEEE802.1CB]*:

1. Null Stream identification
2. Source MAC and VLAN Stream identification

3. Active Destination MAC and VLAN Stream identification
4. IP Stream identification

871929 v2 **Support for Switch and End-point Chip**

Version history:
(2):

Purpose: Define which kind of node shall support 802.1CB.
Source:
Verification Method: Analysis
Legacy ID:

For a In-vehicle network with a ring topology it is mandatory for the selected switches to support *[IEEE_2 : IEEE802.1CB]*.

Note: For endpoints, such as MCU, MPU or SoCs the support for *[IEEE_2 : IEEE802.1CB]* is optional.

4.6.1.11.2.6 802.1Qav / Credit-Based Shaper(CBS)

IEEE802.1Qav是一种基于信用的整形机制，应用在数据链路层，用于音视频等数据流的缓存队列及转发，保证时间敏感流传输低延迟的同时，也让普通流有机会传输。
本规范定义了以太网网络系统 Qav 协议的相关需求，部署 Qav 整形的网桥设备、端节点应满足此标准。
IEEE 802.1Qav is a credit-based shaping mechanism operating at the data link layer. It is used for buffer queuing and forwarding of data streams such as audio and video, ensuring low-latency transmission of time-sensitive streams while allowing ordinary streams to be transmitted.
This specification defines the relevant requirements for the Qav protocol in Ethernet network systems. Bridge devices and end nodes deploying Qav shaping shall comply with this standard.

4.6.1.11.2.6.1 Introduction

IEEE 802.1Qav（时间敏感流的转发及队列协议，简称FQTSS）定义了AVB/TSN网络上保证数据快速可靠传输所必要的流量整形方法。该协议使网桥或终端节点能满足时间敏感（有界延时及延时抖动）、低丢包率及实时性高的数据流的传输需求。
IEEE 802.1Qav (Forwarding and Queuing for Time-Sensitive Streams, FQTSS) defines the traffic shaping mechanisms required to guarantee real-time and reliable data transmission over AVB/TSN networks.
This protocol enables bridges or end stations to meet the transmission demands of time-sensitive data streams characterized by bounded latency and latency jitter, low packet loss rate, and high real-time performance.

IEEE802.1Qav指定如何使用优先级信息来决定时间敏感流的流量类别，在为数据流预留带宽的前提下，定义Credit-Based shaper（CBS）算法作为网桥设备或终端节点设备的传输选择算法以实现对数据流进行整形。
IEEE 802.1Qav specifies how priority information is used to determine the traffic class of time-sensitive streams. On the premise of bandwidth reservation for data streams, it defines the Credit-Based Shaper (CBS) algorithm as the transmission selection algorithm for bridge devices or end stations to perform traffic shaping on data streams.

4.6.1.11.2.6.2 General Requirements

4.6.1.11.2.6.2.1 MAC Requirements

1069505 v1 **Disable the operation of MAC control PAUSE**

Version history:

Purpose: Disable the operation of MAC control PAUSE.
Source:
Verification Method: Analysis
Legacy ID:

任何使用实时数据流的端口均应禁用MAC的PAUSE操作。具体数据流和端口定义参见子系统需求。
MAC PAUSE operation shall be disabled on all ports carrying real-time data streams. For specific definitions of data streams and ports, refer to the subsystem requirements.

1069504 v1 **Max MAC PDU size**

Version history:

Purpose: Define the max MAC PDU size.
Source:
Verification Method: Analysis AND Test
Legacy ID:

MAC层PDU大小应小于等于1522个字节。
The size of the MAC-layer PDU shall be no greater than 1522 bytes.

1069519 v1 **Support VLAN and queues**

Version history:

Purpose: Bridge, Talker, and Listener shall support VLAN and eight priority queues.
Source:
Verification Method: Analysis
Legacy ID:

网桥设备、Talker 和 Listener 必须支持 VLAN，网桥设备、Talker 同时应该支持 8 个优先级队列。
Bridge devices, Talkers and Listeners shall support VLAN. Bridge devices and Talkers shall additionally support 8 priority queues.

4.6.1.11.2.6.2.2 Bridge Requirements

1069507 v1 **Support CBS as transmission selection algorithm for Bridge**

Version history:

Purpose: Define the transmission selection algorithm of bridge.
Source:
Verification Method: Analysis AND Test
Legacy ID:

网桥的MAC层应支持IEEE802.1Qav(CBS)，见[ETH REF 8]，第5.4.1.5章节。
The MAC layer of the bridge shall support IEEE 802.1Qav (CBS), as specified in [ETH REF 8], Clause 5.4.1.5.

网桥的所有端口应支持两种或更多的traffic class，并在支持基于信用整形算法（CBS）的端口上将其作为traffic class的传输选择算法。支持的traffic class的数量和传输选择算法应在子系统需求中明确。
All ports of the bridge shall support two or more traffic classes, and the Credit-Based Shaper (CBS) algorithm shall

be used as the transmission selection algorithm for traffic classes on ports that support CBS. The number of supported traffic classes and the corresponding transmission selection algorithms shall be clearly defined in the subsystem requirements.

1069506 v1 **VLAN-aware Bridge**

Version history:

Purpose: Define a VLAN-aware Bridge.
Source:
Verification Method: Analysis
Legacy ID:

网桥应能够识别VLAN和支持[ETH REF 8] 5.4（a~r）中定义VLAN-aware网桥组件要求。
The bridge shall be capable of identifying VLAN and supporting the requirements for VLAN-aware bridge components as defined in 5.4 (a–r) of [ETH REF 8].

4.6.1.11.2.6.2.3 Talker Requirements

1069509 v1 **Number of data streams for Talker**

Version history:

Purpose: Define the number of data streams of talker.
Source:
Verification Method: Analysis
Legacy ID:

Talker应能够传输至少一种指定VLAN ID和PCP值的数据流。Talker支持的数据流数量应遵循子系统需求。
The Talker shall be capable of transmitting at least one data stream with a specified VLAN ID and PCP value. The number of data streams supported by the Talker shall comply with the subsystem requirements.

1069508 v1 **Support CBS as transmission selection algorithm for Talker**

Version history:

Purpose: Define the transmission selection algorithm of talker.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Talker的MAC层建议支持IEEE802.1Qav(CBS)，应在子系统需求中明确，参见[ETH REF 8]，第5.4.1.5章节。
It is recommended that the MAC layer of Talker support IEEE 802.1Qav (CBS) and clearly defined in the subsystem requirements, as specified in [ETH REF 8], Clause 5.4.1.5.

Talker的所有端口应支持两种或更多的traffic class，并在支持基于信用整形算法（CBS）的端口上将其作为traffic class的传输选择算法。支持的traffic class的数量和传输选择算法应在子系统需求中明确。
All ports of the Talker shall support two or more traffic classes, and the Credit-Based Shaper (CBS) algorithm shall be used as the transmission selection algorithm for traffic classes on ports that support CBS. The number of supported traffic classes and the corresponding transmission selection algorithms shall be clearly defined in the subsystem requirements.

4.6.1.11.2.6.2.4 Listener Requirements

1070713 v1 CBS shall not be supported for Listener

Version history:

Purpose: CBS shall not be supported for the Listener.
Source:
Verification Method: Analysis
Legacy ID:

作为数据流的接收方，Listener不需要支持802.1Qav(CBS)相关的要求。
As the data stream receiver, the Listener is not required to support 802.1Qav (CBS).

1069510 v1 Number of data streams for Listener

Version history:

Purpose: Define the number of data streams of listener.
Source:
Verification Method: Analysis
Legacy ID:

Listener应能够接收最少一种数据流。Listener支持的数据流数量应遵循子系统需求。
The Listener shall be capable of receiving at least one data stream. The number of streams supported by the Listener shall comply with subsystem requirements.

4.6.1.11.2.6.3 Mapping Requirements

1069511 v1 Classify frames into traffic classes

Version history:

Purpose: A bridge or a talker should classify frames into traffic classes.
Source:
Verification Method: Analysis
Legacy ID:

Talker或网桥应将各数据帧分配给不同的流量类别。流量类别需从0开始标号，直到N - 1，其中N是网桥端口的出站队列数量，1 ≤ N ≤ 8。每个流量类别与该端口的特定出站队列一一对应。
The Talker or Bridge shall classify each data frame into traffic classes. Traffic classes shall be numbered starting from 0 up to N−1, where N is the number of egress queues on the bridge port (1 ≤ N ≤ 8). Each traffic class shall be in one-to-one correspondence with a specific egress queue on the port.

应使用固定的映射关系来关联数据帧的VLAN优先级（PCP）及流量类别，以此定义一个数据帧会分配哪个流量类别，具体定义见子系统需求。
A fixed mapping shall be used to associate the VLAN priority(PCP) of data frames with traffic classes, thereby classifying a data frame into the corresponding traffic class, and the specific definition shall be specified in the subsystem requirements.

对于网桥和 Talker，VLAN PCP 和流量类别的映射关系必须能够配置。Listener 不做配置要求。
For bridges and Talkers, the mapping between VLAN PCP and traffic classes shall be configurable. No configuration requirement applies to the Listener.

1069512 v1 **Support a minimum of two traffic classes**

Version history:

Purpose: A bridge or a talker shall support a minimum of two traffic classes
Source:
Verification Method: Analysis
Legacy ID:

网桥和Talker应支持至少两种traffic class，其中一种支持严格优先级算法，另一种支持基于信用整形（CBS）算法。
Bridges and Talkers shall support at least two traffic classes: one shall support the strict priority algorithm, and the other shall support the Credit-Based Shaping (CBS) algorithm.

网桥和Talker支持的traffic class数量及对应的传输选择算法在子系统需求中说明。
The number of traffic classes supported by bridges and Talkers, as well as the corresponding transmission selection algorithms, shall be specified in the subsystem requirements.

当网桥和Talker的某个端口有多个流属于同一个traffic class，CBS算法适用于该traffic class的所有流。
When multiple streams on a port of a bridge or Talker belong to the same traffic class, the CBS algorithm shall apply to all streams of that traffic class.

4.6.1.11.2.6.4 Traffic Shaping Implementation Requirements

1069516 v1 **CBS Configuration requirement**

Version history:

Purpose: Define the idleSlope and portTransmitRate parameter.
Source:
Verification Method: Analysis
Legacy ID:

网桥设备和 Talker 必须能够配置 idleSlope和portTransmitRate参数。
Bridge devices and Talkers shall be configurable for the idleSlope and portTransmitRate parameters.

idleSlope 的配置必须小于等于 portTransmitRate。
The configured idleSlope shall be less than or equal to portTransmitRate.

idleSlope 数值上等于为这个traffic class所预留的实际带宽 operIdleSlope。
idleSlope shall be numerically equal to operIdleSlope, the actual bandwidth reserved for this traffic class.

1071924 v1 **CBS diagnostic logging requirement**

Version history:

Purpose: This section does identify logging requirements used for diagnostic.
Source:
Verification Method: Analysis AND Test
Legacy ID:

当MAC层因为CBS整形出现丢帧时，需要日志记录便于测试问题排查。
When frame loss occurs at the MAC layer due to CBS shaping, logging shall be performed to facilitate test debugging and fault troubleshooting.

当CBS失效时，日志记录当前的adminIdleSlope和operIdleSlope参数。
When CBS fails, the current values of the adminIdleSlope and operIdleSlope parameters shall be logged.

1069514 v1 **Credit-based shaper(CBS) algorithm**

Version history:

Purpose: Introduce the Credit-based shaper algorithm.
Source:
Verification Method: Analysis
Legacy ID:

基于信用的整形算法用来整形数据流的传输，需遵循[ETH REF 8]的8.6.8.2和附录L。
The Credit-Based Shaper (CBS) algorithm is used to shape the transmission of data streams and shall comply with Clause 8.6.8.2 and Annex L of [ETH REF 8].

基于信用的整形机制：

- 应用 CBS 的流量类别的帧必须在信用值大于等于 0 时才可以传输，根据sendSlope的值减少信用，sendSlope = idleSlope - portTransmitRate。
- 当信用值等于 0 且该应用 CBS 的流量类别可以传输时，该流量类别只能传输一帧报文。
- 当信用值等于 0 且该应用 CBS 的流量类别的帧等待传输时，该流量类别的信用值必须根据 idleSlope 的值累计信用。
- 当信用值小于 0 且该应用 CBS 的流量类别没有帧正在传输时，流量类别的信用值必须根据 idleSlope 的值累计信用直至信用值达到 0。
- 当信用值大于或等于 0 且该应用 CBS 的流量类别没有帧等待传输时，流量类别的信用值必须设置为 0。

Credit-Based Shaping mechanism:

- Frames of a traffic class to which CBS is applied shall only be transmitted when the credit value is greater than or equal to 0. The credit shall be decremented according to the sendSlope value, where sendSlope = idleSlope − portTransmitRate.
- When the credit value equals 0 and the traffic class to which CBS is applied is eligible for transmission, only one frame shall be transmitted for that traffic class.
- When the credit value equals 0 and frames of the traffic class to which CBS is applied are waiting for transmission, the credit value of that traffic class shall be accumulated in accordance with the idleSlope value.
- When the credit value is less than 0 and no frames of the traffic class to which CBS is applied are being transmitted, the credit value of that traffic class shall be accumulated in accordance with the idleSlope value until the credit value reaches 0.
- When the credit value is greater than or equal to 0 and no frames of the traffic class to which CBS is applied are waiting for transmission, the credit value of that traffic class shall be set to 0.

1069518 v1 **Disable SRP for bandwidth reservation**

Version history:

Purpose: Disable SRP for stream bandwidth reservation.
Source:
Verification Method: Analysis AND Test
Legacy ID:

网桥、Talker和Listener禁止使用SRP，带宽预留采用静态配置。
The Bridge, Talker and Listener shall not use SRP and the bandwidth reservation shall be configured statically.

网桥和Talker应使用基于信用整形（CBS）进行流带宽预留，在子系统需求中明确。
The Bridge and Talker shall support CBS for stream bandwidth reservation, and clearly defined in the subsystem requirements.

4.6.1.11.2.7 802.1Qbv / Time-Aware Shaper(TAS)

IEEE802.1Qbv 是一种基于时间的整形机制，应用在数据链路层，通过开关传输门控制以太网报文的传输选择过程。
本规范定义了以太网网络系统 Qbv 协议的相关需求，部署 Qbv 整形的网桥设备、端节点应满足此标准。
IEEE 802.1Qbv is a time-based shaping mechanism operating at the data link layer, which controls the transmission selection process of Ethernet frames by switching transmission gates.
This specification defines the relevant requirements for the Qbv protocol in Ethernet network systems. Bridge devices and end nodes deployed with Qbv shaping shall comply with this standard.

4.6.1.11.2.7.1 Introduction

本文档的范围是提供汽车设备上IEEE802.1Qbv功能的基本规范，以作为功能有效实施的基础。 IEEE802.1Qbv提供了一种用于时间触发通信的技术，其使用一种根据预定义的调度表控制出口队列进行帧选择的时间感知整形器。 调度考虑了应用程序的延迟、抖动和缓冲区要求，并保证了网络上数据流的确定性传输。
The scope of this document is to provide the basic specification for IEEE 802.1Qbv functionality on automotive devices, serving as a foundation for the effective implementation of the feature.
IEEE 802.1Qbv provides a technique for time-triggered communication using a time-aware shaper that controls egress queues for frame selection according to a predefined schedule.
The scheduling accounts for application latency, jitter, and buffer requirements, and guarantees deterministic transmission of data streams over the network.

IEEE 802.1Qbv “调度流量的增强功能” 定义了一种在网络终端节点之间实现确定性通信延迟的机制，称为时间感知整形器（Time-Aware Shaper，TAS）。IEEE 802.1Qbv 还引入了称为调度流量 （Scheduled Traffic，ST） 的新型流量。
IEEE 802.1Qbv, Enhancements for Scheduled Traffic, defines a mechanism for achieving deterministic communication latency between network end points, referred to as the Time-Aware Shaper (TAS).
IEEE 802.1Qbv also introduces a new traffic type known as Scheduled Traffic (ST).

IEEE 802.1Qbv 中定义的时间感知整形器本质上是一种门机制，它根据预定义的循环调度表，也称为门控表（Gate Control List，GCL），动态启用或禁用从出口队列中选择所传输的帧。更准确地说，802.1Qbv 在出口端为端口中的每个队列定义了一个定时门，在给定时间，该队列可以处于两种定义的状态之一：打开或关闭。当门处于打开状态时，可以从相应的队列中选择帧，以先进先出 （FIFO） 顺序传输到物理链路。当门处于关闭状态，则不会从相应的队列中选择帧。
The Time-Aware Shaper (TAS) defined in IEEE 802.1Qbv is essentially a gating mechanism that dynamically enables or disables the selection of frames for transmission from egress queues according to a predefined cyclic schedule, also known as a Gate Control List (GCL).
More specifically, 802.1Qbv defines a timed gate for each queue within a port at the egress side. At any given time, a queue may be in one of two defined states: open or closed.
When the gate is open, frames can be selected from the corresponding queue and transmitted onto the physical link in first-in, first-out (FIFO) order.
When the gate is closed, no frames are selected from the corresponding queue.

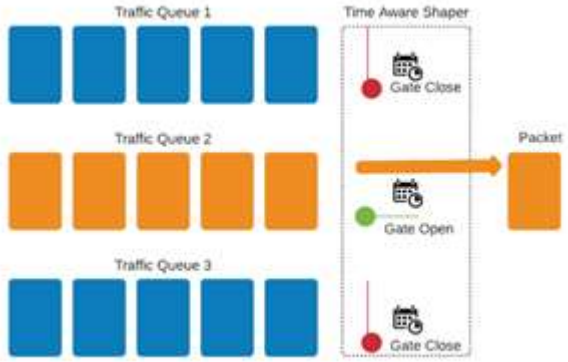


Figure - Gate is open then frame is selected to transmit

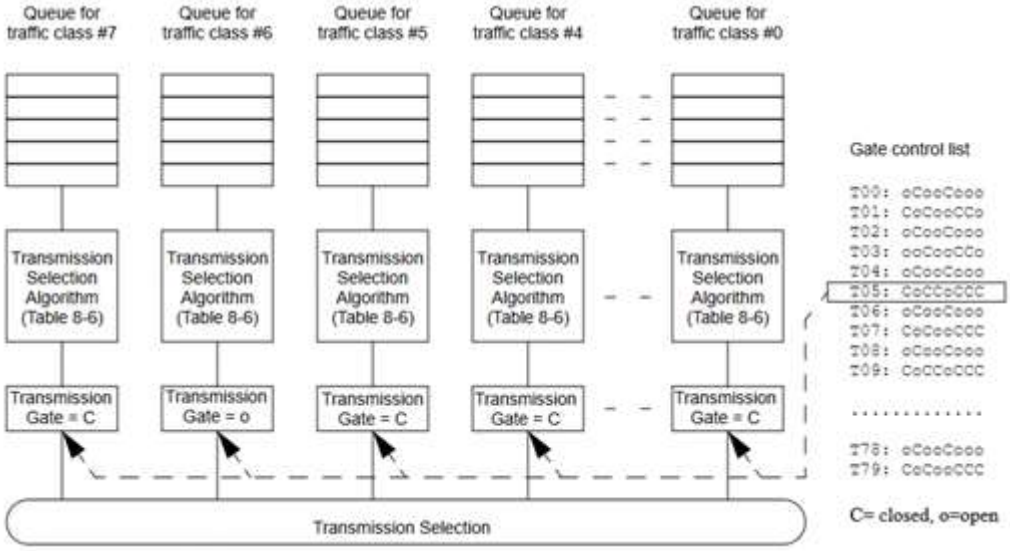


Figure - Gate control list(GCL)

4.6.1.11.2.7.2 General Requirements

4.6.1.11.2.7.2.1 MAC Requirements

1069505 v1 **Disable the operation of MAC control PAUSE**

Version history:

Purpose: Disable the operation of MAC control PAUSE.
Source:
Verification Method: Analysis
Legacy ID:

任何使用实时数据流的端口均应禁用MAC的PAUSE操作。具体数据流和端口定义参见子系统需求。
MAC PAUSE operation shall be disabled on all ports carrying real-time data streams. For specific definitions of data streams and ports, refer to the subsystem requirements.

1069504 v1 **Max MAC PDU size**

Version history:

Purpose: Define the max MAC PDU size.
Source:
Verification Method: Analysis AND Test
Legacy ID:

MAC层PDU大小应小于等于1522个字节。
The size of the MAC-layer PDU shall be no greater than 1522 bytes.

1069519 v1 **Support VLAN and queues**

Version history:

Purpose: Bridge, Talker, and Listener shall support VLAN and eight priority queues.
Source:
Verification Method: Analysis
Legacy ID:

网桥设备、Talker 和 Listener 必须支持 VLAN，网桥设备、Talker 同时应该支持 8 个优先级队列。
Bridge devices, Talkers and Listeners shall support VLAN. Bridge devices and Talkers shall additionally support 8 priority queues.

4.6.1.11.2.7.2.2 Bridge Requirements

1069706 v1 **Support TAS as transmission selection algorithm for Bridge**

Version history:

Purpose: Define the transmission selection algorithm of bridge.
Source:
Verification Method: Analysis
Legacy ID:

网桥应支持[ETH REF 8]的 8.6.8.4 中规定的调度流量的增强功能（802.1Qbv）。
The bridge shall support the enhancements for scheduled traffic (802.1Qbv) as specified in Clause 8.6.8.4 of [ETH REF 8].

网桥应支持[ETH REF 8]的 8.6.9 中规定的调度流量的状态机。
The bridge shall support the state machine for scheduled traffic as specified in Clause 8.6.9 of [ETH REF 8].

1069506 v1 **VLAN-aware Bridge**

Version history:

Purpose: Define a VLAN-aware Bridge.
Source:
Verification Method: Analysis
Legacy ID:

网桥应能够识别VLAN和支持[ETH REF 8] 5.4（a~r）中定义VLAN-aware网桥组件要求。

The bridge shall be capable of identifying VLAN and supporting the requirements for VLAN-aware bridge components as defined in 5.4 (a–r) of [ETH REF 8].

1071923 v1 **gPTP time synchronization for Bridge**

Version history:

Purpose: Time synchronization is necessary for Bridge.
Source:
Verification Method: Analysis
Legacy ID:

如果Bridge支持 802.1Qbv，则它还应支持 802.1AS。
If the Bridge supports 802.1Qbv, it shall also support 802.1AS.

4.6.1.11.2.7.2.3 Talker Requirements

1069509 v1 **Number of data streams for Talker**

Version history:

Purpose: Define the number of data streams of talker.
Source:
Verification Method: Analysis
Legacy ID:

Talker应能够传输至少一种指定VLAN ID和PCP值的数据流。Talker支持的数据流数量应遵循子系统需求。
The Talker shall be capable of transmitting at least one data stream with a specified VLAN ID and PCP value. The number of data streams supported by the Talker shall comply with the subsystem requirements.

1069711 v1 **Support TAS as transmission selection algorithm for Talker**

Version history:

Purpose: Define the transmission selection algorithm of talker.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Talker是调度流量的数据源，在完成gPTP同步的前提下，时间关键型流量从Talker周期性发出。
The Talker acts as the data source for scheduled traffic. Provided that gPTP synchronization has been accomplished, time-critical traffic is sent periodically from the Talker.

如果 Talker 支持 802.1Qbv，则它应支持[ETH REF 8]的 8.6.8.4 中指定的调度流量的增强功能。
If the Talker supports 802.1Qbv, it shall support the enhancements for scheduled traffic as specified in Clause 8.6.8.4 of [ETH REF 8].

如果 Talker 支持 802.1Qbv，则它应该支持[ETH REF 8]的 8.6.9 中指定的调度流量的状态机。
If the Talker supports 802.1Qbv, it shall support the state machine for scheduled traffic as specified in Clause 8.6.9 of [ETH REF 8].

1069709 v1 **gPTP time synchronization for Talker**

Version history:

Purpose: Time synchronization is necessary for Talker.
Source:
Verification Method: Analysis
Legacy ID:

如果Talker支持 802.1Qbv，则它还应支持 802.1AS。
If the Talker supports 802.1Qbv, it shall also support 802.1AS.

4.6.1.11.2.7.2.4 Listener Requirements

1069510 v1 **Number of data streams for Listener**

Version history:

Purpose: Define the number of data streams of listener.
Source:
Verification Method: Analysis
Legacy ID:

Listener应能够接收最少一种数据流。Listener支持的数据流数量应遵循子系统需求。
The Listener shall be capable of receiving at least one data stream. The number of streams supported by the Listener shall comply with subsystem requirements.

1069712 v1 **TAS shall not be supported for Listener**

Version history:

Purpose: TAS shall not be supported for the Listener.
Source:
Verification Method: Analysis
Legacy ID:

作为调度流量的接收方，在Listener端不需要支持802.1Qbv。
As the recipient of scheduled traffic, the Listener is not required to support 802.1Qbv.

4.6.1.11.2.7.3 Traffic Shaping Implementation Requirements

1070052 v1 **CBS and TAS are typically not co-deployed**

Version history:

Purpose: CBS and TAS are typically not co-deployed on egress ports.
Source:
Verification Method: Analysis
Legacy ID:

一般情况下，部署 TAS 功能的出端口不同时部署 CBS 功能。
Generally, egress ports where TAS functionality is deployed shall not implement CBS functionality simultaneously.

CBS 用于限制高优先级数据过度占用带宽，如果应用了 CBS 的队列从属的端口同时部署 TAS，可能会进一步增加应用了 CBS 的队列的数据流的延时。

CBS is used to limit the excessive bandwidth usage of high-priority data. If TAS is deployed at the same time on queue-dependent ports on which CBS is applied, the latency of the data flow on queues on which CBS is applied may be further increased.

当网桥设备或端节点同时使用 Qbv 和 Qav 的情况下，idleSlope 必须满足[ETH REF 8]的第 8.6.8.2 章节要求。当网桥设备或端节点同时使用 Qbv 和 Qav 且 GateEnabled 为 TRUE 时，idleSlope = (operIdleSlope(N) × OperCycleTime / GateOpenTime)，队列的传输门打开是 credit 归零的前提之一。其中，OperCycleTime按[ETH REF 8]的 8.6.9.4.20 节中定义执行，GateOpenTime是指在一个门控周期内，该队列的门控状态为打开的总时长。其他情况应按照802.1Qav规范处理，即指定队列的idleSlope 取值等于该队列对应的operIdleSlope参数值，参见[ETH REF 8] 第 34.3 节。
When Qbv and Qav are used simultaneously on a bridge device or end node, idleSlope shall comply with the requirements specified in Clause 8.6.8.2 of [ETH REF 8].
When Qbv and Qav are used simultaneously on a bridge device or end node with GateEnabled set to TRUE, idleSlope shall be calculated as:
idleSlope = (operIdleSlope(N) × OperCycleTime / GateOpenTime)
The opening of the transmission gate for the queue is one of the prerequisites for the credit to be reset to zero. Wherein, OperCycleTime shall be implemented in accordance with the definition in Clause 8.6.9.4.20 of [ETH REF 8], and GateOpenTime refers to the total duration during which the gate state of the queue is open within one gating cycle.
For all other cases, processing shall follow the IEEE 802.1Qav specification: the idleSlope value for a given queue shall equal the corresponding operIdleSlope parameter value of that queue, as specified in Clause 34.3 of [ETH REF 8].

1069736 v1 Disable dynamic GCL

Version history:

Purpose: Disable config GCL dynamically.
Source:
Verification Method: Analysis
Legacy ID:

门控列表应静态配置，启动时加载，运行过程中不更新。
The Gate Control List (GCL) shall be statically configured, loaded at startup, and not updated during operation.

当时间敏感型应用程序正在运行并发送时间关键型流量时，布尔变量 ConfigChange 不应为 TRUE。只有在网络时钟同步完成后，才允许将 ConfigChange 设置为 TRUE。
When time-sensitive applications are running and transmitting time-critical traffic, the Boolean variable ConfigChange shall not be TRUE. Setting ConfigChange to TRUE shall only be permitted after network clock synchronization has completed.

1069732 v1 GCL cycle and slot length

Version history:

Purpose: Define the GCL cycle and slot length.
Source:
Verification Method: Analysis
Legacy ID:

端口的门控列表周期长度是所有时间关键型流量周期的最小公倍数。
The cycle length of a port's Gate Control List (GCL) shall be the least common multiple(LCM) of all time-critical traffic cycles.

支持时间关键型流量的队列的打开时隙应足够宽，以允许帧通过。
The gate open intervals for queues supporting time-critical traffic shall be sufficiently wide to allow frames to pass through.

1069744 v1 **TAS Configuration requirement**

Version history:

Purpose: The devices that support 802.1Qbv shall be configured as specified in this section.
Source:
Verification Method: Analysis AND Test
Legacy ID:

应为支持 802.1Qbv 的每个设备的每个端口定义以下一组参数。
The following set of parameters shall be defined for each port of each device that support 802.1Qbv.

No.	Name	Data type	Cvt	Device	Default value
1	queueMaxSDUTable	sequence of queueMaxSDU	M	BE	—
2	GateEnabled	Boolean	M	BE	FALSE
3	AdminGateStates	gateStatesValue	M	BE	The transmission gates for all queues are open.
4	AdminControlList	sequence of GateControlEntry	M	BE	—
5	AdminControlListLength	unsigned integer	M	BE	0
6	AdminCycleTime	RationalNumber (seconds)	M	BE	—
7	AdminBaseTime	PTPtime	O	BE	—
8	AdminCycleTimeExtension	Integer (nanoseconds)	O	BE	0
9	ConfigChange	Boolean	O	BE	TRUE

Note:
Cvt: Convention； M: Mandatory； O: Optional; B: Bridge; E: Endpoint;

queueMaxSDUTable 最多包含 8 个条目，一个条目对应实现支持的每个流量类，每个条目由一个 queueMaxSDU 值组成。queueMaxSDU 是一个无符号整数值，表示队列支持的最大 SDU 大小。
The queueMaxSDUTable shall contain a maximum of 8 entries, with one entry corresponding to each traffic class supported by the implementation. Each entry consists of a queueMaxSDU value, which is an unsigned integer representing the maximum SDU size supported by the queue.

GateEnabled 是一个布尔变量，用于指示调度流量状态机的操作是启用 （TRUE） 还是禁用 （FALSE）。此变量的默认值为 FALSE。
GateEnabled is a Boolean variable that indicates whether the operation of the scheduled traffic state machine is enabled (TRUE) or disabled (FALSE). The default value of this variable is FALSE.

AdminGateStates 变量确定与端口的每个队列关联的传输门的初始状态。AdminGateStates 的默认值为所有队列传输门均为“打开”。
The AdminGateStates variable determines the initial state of the transmission gates associated with each queue on

a port. The default value of AdminGateStates is that all queue transmission gates are set to "open".

AdminControlList 是包含 AdminControlListLength 个条目的有序列表，每个条目代表一个门操作，如[ETH REF 8]的表 8-6 中定义。列表中的每个条目都结构为 GateControlEntry。
The AdminControlList is an ordered list containing AdminControlListLength entries, each representing a gate operation as defined in Table 8-6 of [ETH REF 8]. Each entry in the list is structured as a GateControlEntry.

GateControlEntry 由一个操作名称组成，后跟 2 个与操作关联的参数，如[ETH REF 8]的表 8-6 中所述。第一个参数是 gateStatesValue; 第二个参数是 timeIntervalValue。
A GateControlEntry consists of an operation name followed by two parameters associated with the operation, as described in Table 8-6 of [ETH REF 8]. The first parameter is gateStatesValue; the second parameter is timeIntervalValue.

gateStatesValue是最多包含 8 个元组的列表，每个元组对应端口支持的每个流量类，每个元组由一个 0-7 范围内的流量类编号和一个门状态组成，该门状态是一个枚举值，表示该流量类的门状态（打开或关闭）。
gateStatesValue is a list containing up to 8 tuples, each corresponding to a traffic class supported by the port. Each tuple consists of a traffic class number in the range 0–7 and a gate state, which is an enumerated value indicating whether the gate for that traffic class is open or closed.

timeIntervalValue是无符号整数，表示两个连续操作之间的 TimeInterval（以ns为单位）。
timeIntervalValue is an unsigned integer that represents the time interval (in nanoseconds) between two consecutive operations.

AdminControlListLength 指示 AdminControlList 中的条目数。
AdminControlListLength indicates the number of entries in the AdminControlList.

AdminCycleTime 是端口的门控周期。它是由整数分子和整数分母定义的有理秒数。
AdminCycleTime is the gating cycle of a port. It is a rational number of seconds defined by an integer numerator and an integer denominator.

AdminBaseTime 表示基准时间，以 IEEE 1588 精确时间协议 （PTP） 时间刻度表示。
AdminBaseTime denotes the base time, expressed in the time scale of the IEEE 1588 Precision Time Protocol (PTP).

AdminCycleTimeExtension 是一个整数，单位ns，用于定义在安装一个新的门控周期配置时允许延长端口的门控周期的最长时间。
AdminCycleTimeExtension is an integer value in nanoseconds, used to define the maximum time by which the port's gating cycle may be extended when installing a new gating cycle configuration.

ConfigChange 是一个布尔变量，它充当List Config状态机的启动信号，表明端口的管理变量值已准备好复制到其相应的操作变量中。此变量由用户管理设置为 TRUE，并由 List Config 状态机设置为 FALSE。
ConfigChange is a Boolean variable that acts as a start signal for the List Config state machine, indicating that the port’s management variable values are ready to be copied into their corresponding operational variables. This variable is set to TRUE by user management and set to FALSE by the List Config state machine.

1069739 v1 **TAS diagnostic logging requirement**

Version history:

Purpose: This section does identify logging requirements used for diagnostic.
Source:
Verification Method: Analysis AND Test
Legacy ID:

如果设备检测到给定队列的门关闭时仍在传输帧，则每个流量类别计数器 TransmissionOverrun 应递增。默认值为 0。
If a device detects that a frame is still being transmitted while the gate for a given queue is closed, the TransmissionOverrun counter for each traffic class shall be incremented. The default value is 0.

设备应提供读取 TransmissionOverrun 计数器的机制。
The device shall provide a mechanism for reading the TransmissionOverrun counter.

应设置每个流量类队列 queueMaxSDU 参数，该参数定义每个队列的最大服务数据单元大小;超过 queueMaxSDU 的帧将被丢弃。
The queueMaxSDU parameter shall be configured for each traffic class queue, which defines the maximum Service Data Unit (SDU) size of each queue, frames exceeding queueMaxSDU shall be discarded.

queueMaxSDU 的默认值为 1500 字节。
The default value of queueMaxSDU is 1500 bytes.

当MAC层因为TAS整形出现丢帧时，需要日志记录便于测试问题排查。
When frame loss occurs at the MAC layer due to TAS shaping, logging shall be performed to facilitate test debugging and fault troubleshooting.

当GCL失效时，日志记录当前的AdminControlList。
When GCL fails, the current AdminControlList shall be logged.

1069734 v1 Time-Aware Shaper(TAS) algorithm

Version history:

Purpose: Introduce the Time-Aware shaper algorithm.
Source:
Verification Method: Analysis
Legacy ID:

网络中支持调度流量的所有设备都应维护以 gPTP 时间刻度 表示的 CurrentTime。
All devices supporting scheduled traffic in the network shall maintain CurrentTime expressed on the gPTP time scale.

网络中支持调度流量的所有设备都应支持基于 gPTP 的同步时基。
All devices supporting scheduled traffic in the network shall support a gPTP-based synchronized time base.

当网桥设备和端节点不支持或关闭了 Qbv 功能时，其所有的门默认处于打开状态。
When bridge devices and end nodes do not support Qbv functionality or have it disabled, all their gates shall default to the open state.

每个实施 Qbv 的网桥设备和端节点的端口必须支持 Qbv 门控列表配置。
Each port of bridge devices and end nodes implementing Qbv shall support the configuration of Qbv gate control lists (GCL).

一个门对应一个队列，每个端口至多包含 8 个队列。
Each gate corresponds to one queue, and each port shall contain a maximum of 8 queues.

当门处于打开状态时，队列选择一条新帧的传输。
When the gate is open, the queue selects the transmission of a new frame.

当门处于关闭状态时，队列禁用一条新帧的传输。
When the gate is closed, the queue disables the transmission of a new frame.

当队列对应的下一个门关闭时刻前剩余时间不足够传输一个完整的帧时，队列中的这个帧不应该选择或等下一个门

控周期传输。
If the remaining time before the next gate-close instant for the corresponding queue is insufficient to transmit a complete frame, this frame in the queue shall not be selected for transmission and shall wait until the next gate control cycle.

支持 Qbv 功能的网桥设备和端节点可通过设置保护带（Guard Band，GB）为下一门开启前预留充足时间。
Bridge devices and end nodes supporting Qbv functionality can reserve sufficient time before the next gate opens by configuring a Guard Band (GB).

4.6.1.12 IP Link Manager requirement

•Introduction

4.6.1.12.1 Introduction

4.6.1.12.1.1 Abbreviations & Definitions

Abbr.	Stands for
API	Application Programming Interface
ARS	Active Request Session
BYTE	8 bits, permitting values between 0 to 255
DLL	Dynamic-Link Library
FMEA	Failure Mode and Effects Analysis
IP	Internet Protocol
ISO	International Organization for Standardization
ITU-T	International Telecommunication Union – Telecommunication Standardization Sector ITU-T
LM	IP Link Manager
LSC	Local Software Component
OR	Ongoing Request
OSI	Open Systems Interconnection
PDU	Protocol Data Unit
RG	Resource Group
RMT	Request monitoring timeout
TCP	Transmission Control Protocol
TBD	To be defined
UDP	User Datagram Protocol

Definition	Description
ARS	State of single IP links describing weather there exist an active communication path towards an external LM instance.
Client	A software program which access services provided/enabled by the server
LM	Link Manager is a software module based on IP Command Protocol, distributed over the vehicle network and manages IP based communication links.
Local Configuration File	ECU local configuration file where parameters are configured. See [REQPROD 357337]
Offset	Indicates the distance from the beginning of an object up until a given point. Denotes the number of address location added to a base address in order to get a specific absolute address.

Operation	A function, method (or an instruction)
OperationID	Unique identifier of an operation, used to identify a function/method.
Service	A service is a subsystem, which is an independent component of a larger system. In this specification, it's a logical combination of methods or events. E.g. Telematics, WLAN …
ServiceID	Unique identifier of a service, used to identify a specific service.
Server	A software program that offers one or many services
Request	A message form the client to the server invoking a operation
Response	A message from the server to the client transporting the result of a operation invocation
Request/Response	A remote procedure call that consists of booth a request and a response.
Resource Group	A set of ECUs that together provide one or more services.

4.6.1.12.1.2 Requisite Documents

Document Nr.	LC	Description
31835911	LC : Internetworks General Specification	Internetworks General Specification
31836302	LC : IP Routing and Firewalling	IP Routing and Firewalling
	LC : Diagnostics and ECU Platform	Diagnostics and ECU Platform Specification

•**Requirements**

4.6.1.12.2 Requirements

680935 v1 **ARS timers**

Version history:
(1):

Purpose: To define usage of the monitoring timers in the LM module.
Source:
Verification Method: Test
Legacy ID: 347881v3

The LM module shall implement one timer for each Active Request Session, ARS. This timer shall assume the value given by *Request_monitoring_timeout*. The timer shall start when the LM module has registered an ARS upon reception of the IP_activity message.
Each time LM receives an *IP_activity* message it shall restart the interface-specific timer and when an interface is declared dead its timer shall stop.

This example illustrated the relationship between the above mentioned states and timers. Main focus is to show the state transitions “entering OR” à “exit OR” and “setup ARS” à “maintain ARS” à “invalidate ARS”. Note, all signaling is not shown in the figure to make the sequence easier to understand.

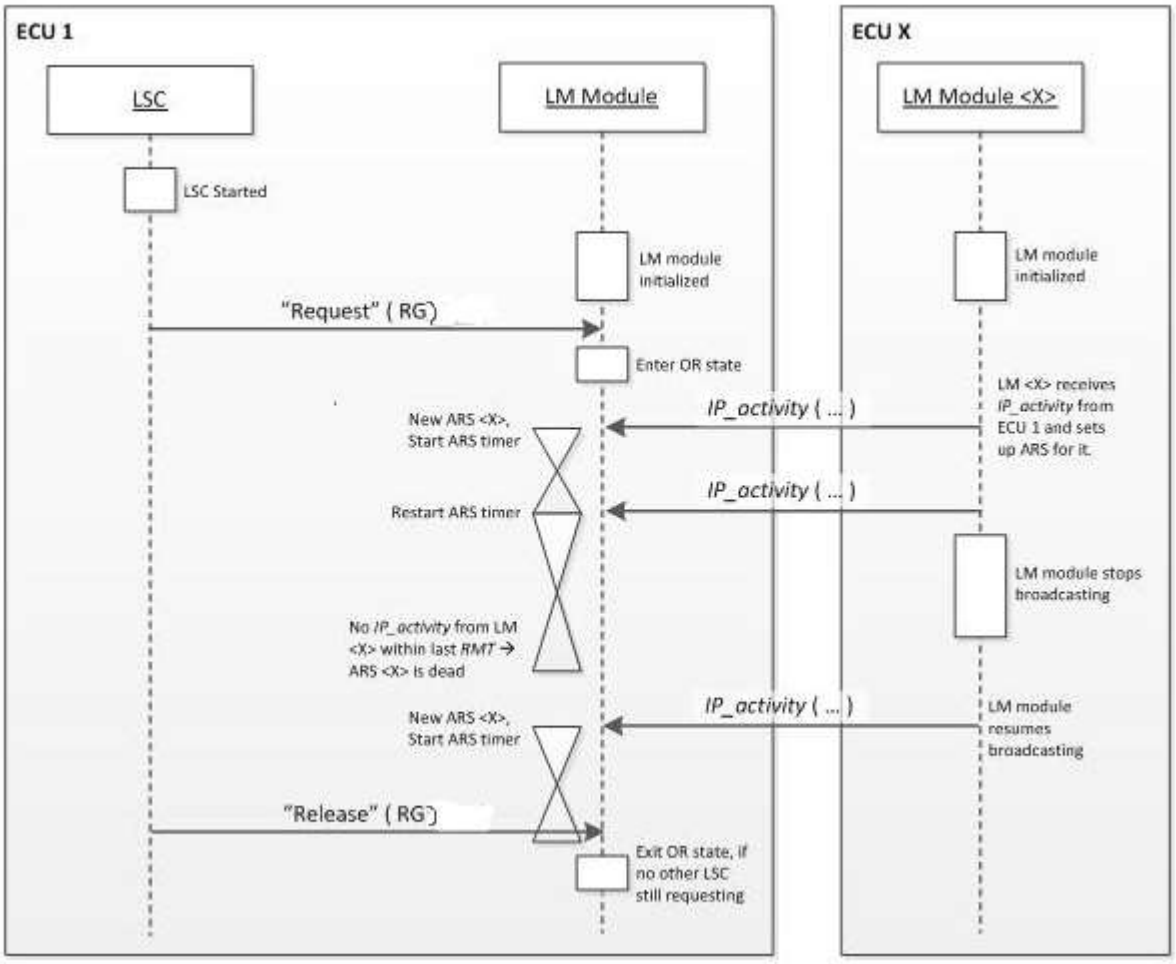


Figure: Example sequence showing the relationship between different states and timers.

680933 v1 **Active Request Session**

Version history:
(1):

Purpose: To define the term Active Request Session, ARS.
Source:
Verification Method: Test
Legacy ID: 347883v2

Active Request Session, ARS shall be implemented according to the following definition. The ARS is a state bound to single IP links. Whenever an *IP_activity* message is received, with ACTION-field = AVAILABLE | REQUEST_RG_X an ARS shall be set for the IP link. The ARS exist as long as new *IP_activity* messages are received from the same IP link within the time defined by *Request_monitoring_timeout.* A new ARS about the request can be Seen in [REQPROD 347881] and figure [*Example sequence showing the relationship between different states and timers.*]

1365 v4 **ECU wakeup detection**

Version history:
(4):

Purpose: To define how to handle the vehicle bus ECU wakeup signal
Source:
Verification Method: Test
Legacy ID: 347899v2

When the property *ResourceGroup = RG_X* is detected on the vehicle bus ECU wakeup signal in [REQPROD 347897]), and the *X* in *RG_X* refers to the Resource Group number which the LM module is a member of according to [REQPROD 347895], the following shall happen:
Initialize the vehicle internal IP bus according to chapter [*Link Manager - Initialization*], once the ECU has started.

680927 v2 **IP Bus release**

Version history:
(2):

Purpose: To define how the LM instance releases the vehicle internal IP Bus.
Source:
Verification Method: Test
Legacy ID: 347892v2

When a RG is no longer needed, the LM instance needs to change the parameters in the *IP_activity* message being broadcasted to not include the RG not requested any longer. If no RG is requested the parameters shall assume the default values ACTION = *AVAILABLE(*[REQPROD 347120] *).*
Note: Observe the example below, [*Example - IP Bus release*].
Scenario:
· LM on ECU1 no longer needs to keep the Resource Group active, only needs to broadcast its own availability until it becomes unavailable.
· LM on ECU X has started broadcasting availability after initialization and will do so until it becomes unavailable.

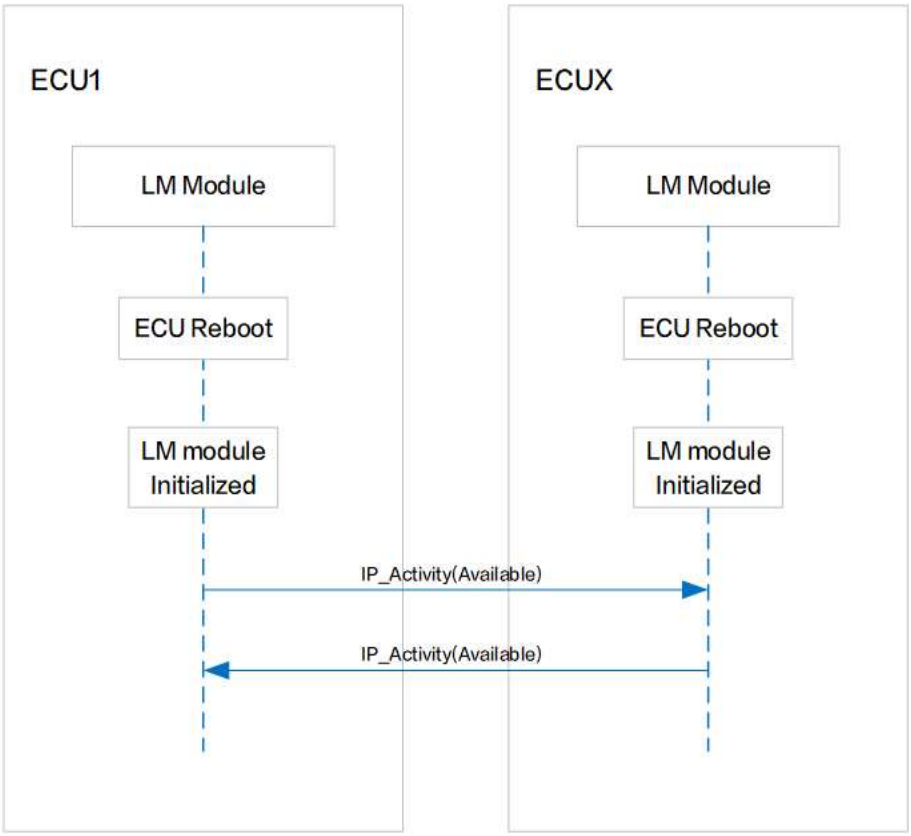


Figure: Example showing the two LM instance only broadcasts availability.

680928 v2 **IP Bus request**

Version history:
(2):

Purpose: To define how to request the vehicle internal IP bus
Source:
Verification Method: Test
Legacy ID: 347890v4

When the vehicle internal IP bus (by RG) is requested by an LSC and the ECU is ready to send *IP_activity* messages, the LM module shall:

1. Check if the vehicle internal IP bus already is up, if it is go to step 3.
2. Request to activate the vehicle internal IP bus according to [REQPROD 347898].
3. Change parameters in the *IP_activity* message already being broadcasted to include the bit (*REQUEST_RG_X*) corresponding to the requested RG such as ACTION = *AVAILABLE* | REQUEST_RG_X.

Note: Observe the example below, [*Example 1 - IP Bus request*].

Scenario:
· LM on ECU 1 has been requested to keep active a certain Resource Group which it will continue doing until requested not to.
· LM on ECU X has started broadcasting availability after initialization and will do so until it becomes unavailable.

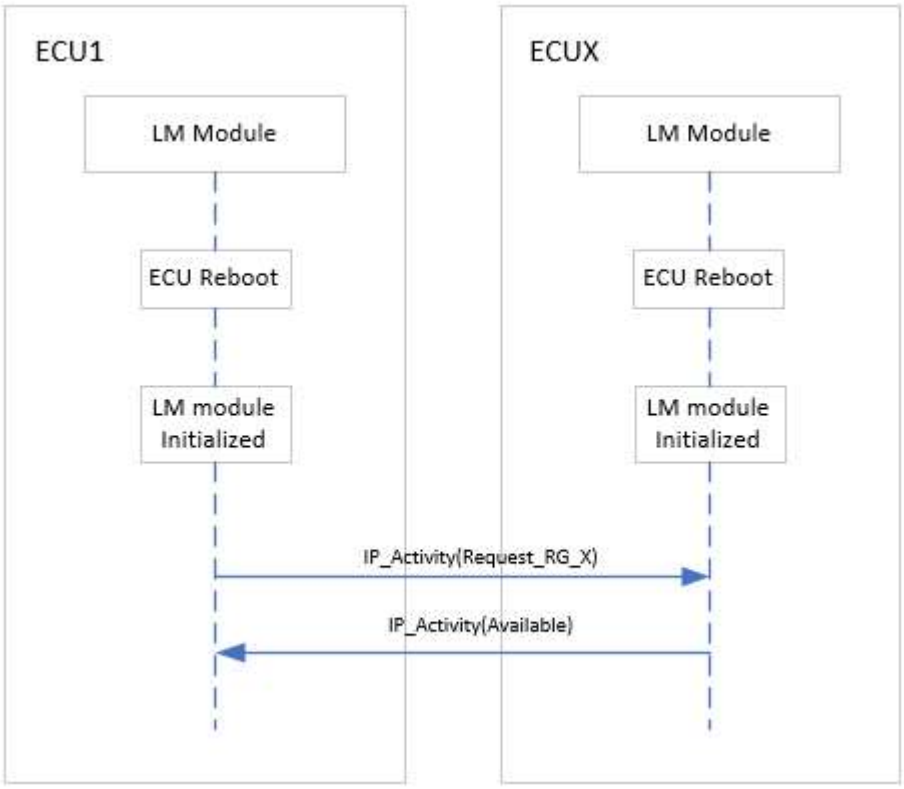


Figure: Example showing the scenario when one LM instance requests to keep alive the IP Bus while the other only broadcasts availability.

680924 v4 **LM IP_activity message structure**

Version history:
(4):

Purpose: To define the LM (IP_activity) message structure
Source:
Verification Method: Test
Legacy ID: 347120v4

The figure below shows the relation between the PDU header on the application layer, the UDP/TCP handling on the transport layer and also the IP handling on the network layer.

The following structure shall be used when constructing the LM *IP_activity* message.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
Service ID[16 bit]																**Operation ID[16 bit]**															
Length[32 bit]																															
Client ID[16 bit]																**Session ID[16 bit]**															
SDB Protocol Version [32 bit]																															
ACTION [8 bit]								**Reserved(24bit)**																							

Figure: Overview figure of the PDU proprietary data structure using UDP

Note: The ACTION-field shall be implemented as a bit-field, which can take on values representing all combinations of the bits addressed. This is required to make possible scenarios when more than one Resource Group is required. The AVAILABLE-bit just indicates the Link status is OK.

Header fields:

ACTION:

Values	Name	Description
0x01	AVAILABLE	The Link status is OK.
0x02	REQUEST_RG_1	Request at least one service on Resource Group 1.
0x04	REQUEST_RG_2	Request at least one service on Resource Group 2.
0x08	REQUEST_RG_3	Request at least one service on Resource Group 3.
0x10	REQUEST_RG_4	Request at least one service on Resource Group 4.
0x20	REQUEST_RG_5	Request at least one service on Resource Group 5.
0x40	REQUEST_RG_6	Request at least one service on Resource Group 6.
0x80	REQUEST_RG_7	Request at least one service on Resource Group 7.

Description of the IP Command Protocol Header		
ServiceID	16 bit (0xFFFF)	The 16 bit service identifier (ServiceID) is used to identify a specific service that a server provides. Examples of services provided by the subsystems are Telematics, Phone, Connectivity etc.
OperationID	16 bit (0xFF01)	The 16 bit OperationID shall be used to identify which service operation that shall be used. The OperationID must be unique for each operation that is supported in the vehicle.
Length	32 bit (0x00000008 – 0xFFFFFFFF)	The Length-field consists of 32 bits containing the length in Byte of the payload. The beginning of the length calculation is at the beginning of the **Client ID**, offset 64.
ClientID	16 bit (0x0000)	Default value is 0x0000.
Session ID	16 bit	Session ID shall be incremented for every message sent.It shall start with 1 and be 1 even after wrapping.
SDB Protocol Version	32 bit (0x00 – 0xFFFFFF)	Protocol version is an identifier defining the version of the SOME/IP Arxml version which is used.For example SDB2243100,it should be set to 0x02243100.

Length of IP_Activity is at least 16 bytes (128 bits).

681000 v2 LM broadcast

Version history:
(2): Revise requirement ID to [REQPROD 660805].

Purpose: To define how the LM messages shall be sent among the ECUs.
Source:
Verification Method: Test
Legacy ID:

IP_activity messages shall use the subnet-directed broadcast. The address is defined in [REQPROD 674228].

680929 v1 LM initial discovery mechanism

Version history:
(1):

Purpose: To define the LM module's initial discovery mechanism.
Source:
Verification Method: Test
Legacy ID:

The LM module shall start broadcasting the *IP_activity* message periodically **once every second** after it has been initialized according to [REQPROD 347878]. If during initialization all services could not be started, the LM module shall not broadcast *IP_activity* [REQPROD 347120].
The IP_activity message shall have the following default values in the header field;
ACTION = AVAILABLE

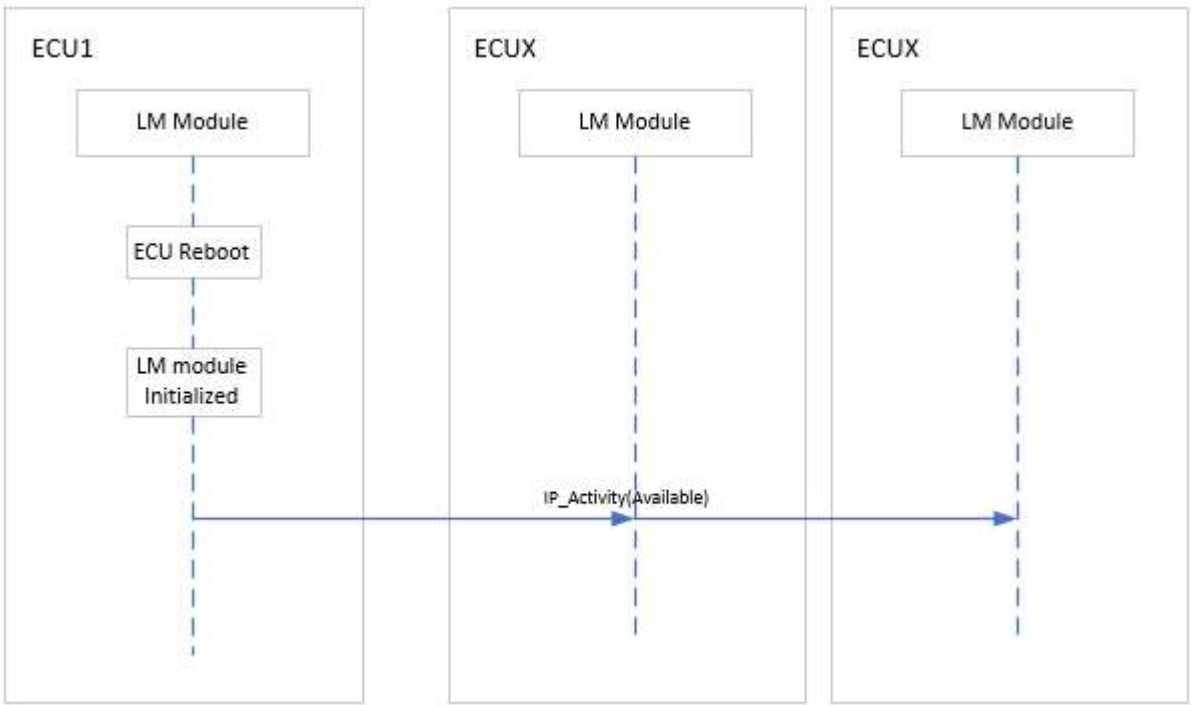


Figure: Showing the LM module broadcasting *IP_activity*.

1372 v3 **LM initialization**

Version history:
(3):

Purpose: To define the order in which services and the LM module shall be initialized at ECU startup.
Source:
Verification Method: Test
Legacy ID: 347878v2

At ECU startup, the LM module shall be initialized **afterall** preconfigured services have been started and ready to process received messages. All previous data shall be flushed during initialization.

1362 v5 **LM purpose**

Version history:
(5):

Purpose: To define the purpose of the LM module
Source:
Verification Method: Analysis
Legacy ID: 347118v4

Each LM module shall be responsible for the following tasks;
- Inform other LM modules across the vehicle internal IP network when the ECU has all its services available.
- Request services by Resource Group, from other ECUs.
- Handle RG requests from external LM modules.
- Notify LSCs about the availability of RGs and external LM modules.
- Initiate activation of the IP buses by starting the required ECUs via the vehicle buses.

1373 v4 **LSC IP Bus release**

Version history:
(4):

Purpose: To state that an LSC shall be able to request to release the vehicle internal IP bus.
Source:
Verification Method: Test
Legacy ID: 347888v2

LSCs shall be able to inform the local LM instance that the vehicle internal IP bus is not needed by the LSC anymore. If LM sees that the IP bus is no longer requested by any LSC it shall exit the OR state. When the vehicle internal IP Bus is no longer needed, the LM instance shall release it according to [REQPROD 347892].

1374 v3 LSC IP bus request

Version history:
(3):

Purpose: To state that an LSC shall be able to request the vehicle internal IP bus.
Source:
Verification Method: Test
Legacy ID: 347887v2

LSCs shall be able to request the local LM instance to start the complete vehicle internal IP bus or the part required for a given RG. Such request shall make the LM enter the OR state. LM shall activate the IP bus according to [REQPROD 347890].

680930 v1 LSC notifying

Version history:
(1):

Purpose: To define which information the LM module shall provide to LSCs.
Source:
Verification Method: Analysis
Legacy ID:

It shall be possible for an LSC to be notified if the requested RG is *Available*, *Partly available* or *Unavailable*. Were
1. *Available* means that the *IP_activity* messages from all *Number_of_Nodes_in_RG_X* nodes have been received on the vehicle internal IP bus within the last *Request_monitoring_timeout* seconds. The number of expected messages (*Number_of_Nodes_in_RG_X*) depends on the requested Resource Group.
2. *Partly available* means that at least one *IP_activity* message ([REQPROD 347120]) has been received on the vehicle internal IP bus within the last *Request_monitoring_timeout* seconds.
3. Unavailable means that no *IP_activity* messages have been received on the vehicle internal IP bus within the last *Request_monitoring_timeout* seconds.

Note:
1.An LSC may choose to try using the IP network even if the requested RG only is partly available, i.e., all IP services between the awakened nodes can communicate, but the IP services communication between the awakened node and the unawakened node is unavailable.

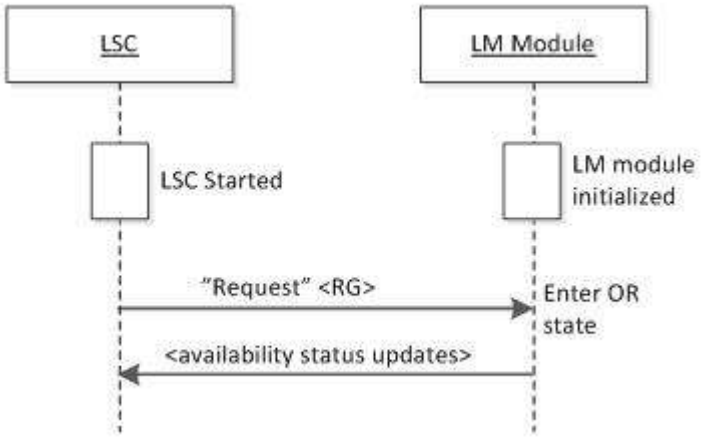


Figure: Showing ECU local interface between an LSC and the LM module.

1388 v2 Local Configuration file

Version history:
(2):

Purpose: To define the locally configurable parameters for this specification, [LC : IP Link Manager]
Source:
Verification Method: Analysis
Legacy ID: 357337v1

The parameters stated in the following table shall be implemented and configurable on each ECU that deploys a LM.

Parameter	DataType	Description	REQPROD
ResourceGroup	Bit-field	Defines which resource groups the specific ECU is member of	347895
Number_of_Nodes_in_RG_1	Integer	Defines how many ECUs is member of the resource group.	347879
Number_of_Nodes_in_RG_2	Integer	- Same as above -	- Same as above -
Number_of_Nodes_in_RG_3	Integer	- Same as above -	- Same as above -
Number_of_Nodes_in_RG_4	Integer	- Same as above -	- Same as above -
Number_of_Nodes_in_RG_5	Integer	- Same as above -	- Same as above -
Number_of_Nodes_in_RG_6	Integer	- Same as above -	- Same as above -
Number_of_Nodes_in_RG_7	Integer	- Same as above -	- Same as above -
Request_monitoring_timeout	double	Defines the time it takes for an ARS to be declared dead. Range 1.5 – 8, Precision 100ms.	347880

Default 3 seconds

1379 v5 **Number of Nodes in RG**

Version history:
(5): Change Verification Method from test to analysis

Purpose: To define constants used locally by the LM module.
Source:
Verification Method: Analysis
Legacy ID: 347879v3

The constants *Number_of_Nodes_in_RG_X*, where X is a number 1 – 7, shall be implemented and specified in the Local Configuration File.
*Number_of_Nodes_in_RG_*X is the number of vehicle internal IP ECUs that are included in a specific Resource Group, thus expected to broadcast their availability. It is used to conclude to what extent the vehicle internal IP network is available.

680936 v2 **Ongoing Request**

Version history:
(2): delete "During the OR state, if the request has high priority, normal power management shall be ignored."

Purpose: To define the Active Request, AR state.
Source:
Verification Method: Test
Legacy ID:

The Ongoing Request, OR state is defined as:

- The time between receiving a vehicle bus signal *Vehicle_bus_ECU_wakeup* (defined in [REQPROD 347897]) with *ResourceGroup* = RG_X **,** until *Request_monitoring_timeout* seconds after the LM module has started to broadcast the *IP_activity* message.
- The time from an LSC making an RG request until it releases the RG [REQPROD 347888].

1386 v4 **Port**

Version history:
(4): 增加描述：(Include Destination port and Source Port)，以避免歧义。

Purpose: To define a software port for the LM messages.
Source:
Verification Method: Test
Legacy ID: 347837v2

IP_activity messages shall be sent using software port number 50001(Include Destination port and Source Port) .

680937 v1 **Request Monitoring Timeout**

Version history:

(1):

Purpose: To define variables used locally by the LM module.
Source:
Verification Method: Test
Legacy ID:

The LM Module shall implement the configurable variable *Request_monitoring_timeout*, RMT. *Request_monitoring_timeout* represents the time it takes to declare an Active Request Session, ARS (defined in [REQPROD 347883]) dead. The parameter shall be configurable (with range 1.5 to 8 seconds and precision of 100ms) via the Local Configuration File.
Default value shall be 3 seconds.

1382 v4 **Resource Groups**

Version history:
(4):

Purpose: To define the usage of Resource Groups.
Source:
Verification Method: Analysis
Legacy ID: 347895v3

The vehicle internal IP ECUs are assigned as members of one or more Resource Groups, RG **.** The parameter *ResourceGroup* determines which Resource Groups the local LM module is a member of. *ResourceGroup* shall be implemented as a bit field of 1 byte, thus being able to represent an ECU as member of multiple Resource Groups. It shall be configurable in a Local Configuration File.

Decoding of *ResourceGroup* shall be performed according to the following table.

Value	Name	Description
0x01	Reserved	Reserved for future use
0x02	RG_1	Member of Resource Group 1.
0x04	RG_2	Member of Resource Group 2.
0x08	RG_3	Member of Resource Group 3.
0x10	RG_4	Member of Resource Group 4.
0x20	RG_5	Member of Resource Group 5.
0x40	RG_6	Member of Resource Group 6.
0x80	RG_7	Member of Resource Group 7.

680931 v1 **Software interface**

Version history:
(1):

Purpose: To define the interface between the Local Software Component, LSC and the LM module.
Source:
Verification Method: Analysis
Legacy ID:

The LM module shall offer Local Software Components, LSCs an interface (visualized in figure [*Showing ECU local interface between an LSC and the LM module*]) for determining whether there exists any ARSs or ORs.
The LM module shall also offer information about the currently requested Resource Groups .

680999 v1 **Target ECUs**

Version history:
(1):

Purpose: To define which ECU's that shall deploy the LM module
Source: LC : Diagnostics and ECU Platform
Verification Method: Test
Legacy ID:

The LM module shall be deployed throughout the vehicle internal IP network according to the requirement *IP Deployment Matrix* , [REQPROD 680997 in LC : Diagnostics and ECU Platform].

680923 v1 **Transport Protocol**

Version history:
(1):

Purpose: To define the LM protocol
Source:
Verification Method: Test
Legacy ID:

IP_activity messages shall be over **UDP**.

680926 v1 **VFC IP wakeup**

Version history:
(1):

Purpose: To define how to wake up the vehicle internal IP bus via the vehicle bus
Source:
Verification Method: Test
Legacy ID: 347898v2

How to request wakeup of the vehicle internal IP bus via the vehicle bus signaling

1. VFC " *IP Wakeup*" shall be activated, see [REQPROD 347896]
2. *ResourceGroup = RG_X* shall be set in the V*ehicle_bus_ECU_wakeup* signal as long as VFC *"IP Wakeup"* is activated, and then return to default values as specified in [REQPROD 347897].

680925 v1 **Vehicle bus signal**

Version history:

(1):

Purpose: To define the content of the vehicle bus signaling
Source:
Verification Method: Test
Legacy ID: 347897v2

A V*ehicle_bus_ECU_wakeup* signal with content "< *ResourceGroup*>" shall be used as wakeup signal when the vehicle internal IP bus is not available. Each ECU will publish its own signal. The actual V*ehicle_bus_ECU_wakeup* from each vehicle internal IP ECU is specified in the Infotainment Platform SRD.

<ResourceGroup> - Parameter

Value	Name	Description
0x00	Default	Default, no specific RG is requested
0x01	Reserved	Reserved for future use
0x02	RG_1	Request at least one service on Resource Group 1.
0x04	RG_2	Request at least one service on Resource Group 2.
0x08	RG_3	Request at least one service on Resource Group 3.
0x10	RG_4	Request at least one service on Resource Group 4.
0x20	RG_5	Request at least one service on Resource Group 5.
0x40	RG_6	Request at least one service on Resource Group 6.
0x80	RG_7	Request at least one service on Resource Group 7.

1366 v4 Wakeup of internal IP Bus

Version history:
(4): Revise the timeout from 3s to30 s.

Purpose: To define how to wake up the vehicle internal IP bus via the vehicle bus
Source:
Verification Method: Test
Legacy ID: 347896v2

The IP Wakeup VFC shall be implemented according to the following description.

Specification	On Criteria		Off Criteria	
IP Link Manager	**Signal/Timer**	**Description**	**Signal/Timer**	**Description**
	An LSC (Local Software Component) requests a remote service AND IP interface to the service providing ECU(s) is not active.	Wakeup of the IP link between the local and the remote ECU.	VFCTimeOutDelay = 30 sec Or Ethernet related functions RG release.	The IP link VFC release.

Note: (Timers)

Deactivation criteria consisting of a VFCTimeOutDelay indicate that each VFC activation shall reset a timer called VFCTimeOutDelay. The VFCTimeOutDelay is false until the timer reaches its preset value (30 seconds unless otherwise specified in the table) when it turns to true. The VFC shall be deactivated when the entire expression is true.

4.6.1.13 SignalToUDP

•Genral Requirement

4.6.1.13.1 Genral Requirement

4.6.1.13.1.1 Definitions and Abbreviations

Definition	Description
I-PDU	Interaction Layer PDU. An I-PDU consists of data (buffer), length and I-PDU ID.
I-PDU ID	I-PDU Identifier.
nPdu Feature	The concepts in SoAd which can be understood as a combination of multiple I-PDUs.
Last -Is-Best	A Buffer policy in which the latest value overwrites the oldest value.
FIFO	First in first out Buffer strategy

4.6.1.13.1.2 Overview

This scheme specifies the scheme requirements for SignalToUDP of EEA4.0 platform and above. All associated Ethernet IP nodes need to implement corresponding configurations according to this scheme.

4.6.1.13.1.3 Requisite documents

The following specifications and documents shall be referred to for the formulation of this scheme:

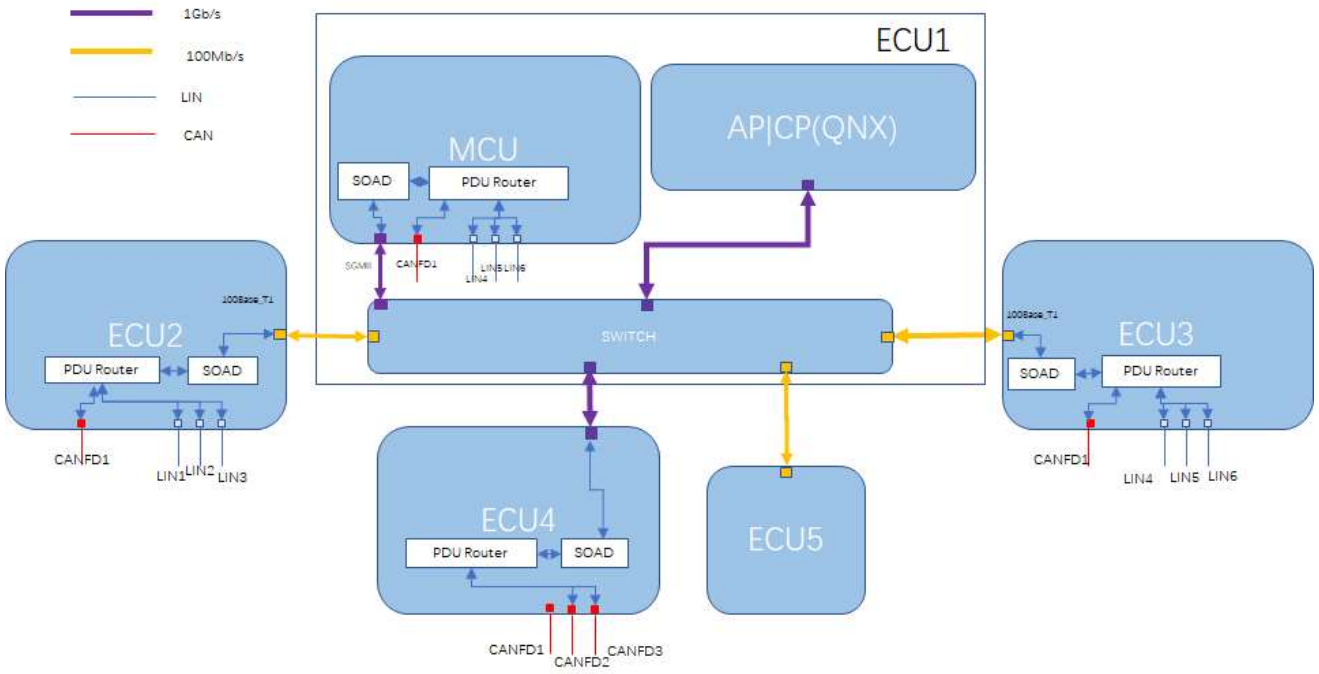
Reference	Source	Document
[STUDP_1]	External	AUTOSAR_SWS_IPDUMultiplexer
[STUDP_2]	External	AUTOSAR_SWS_PDURouter
[STUDP_3]	External	AUTOSAR_SWS_SocketAdaptor
[STUDP_4]	Internal	Project Specific LC for Diagnostics and ECU Platform

•SignalToUDP Topology

4.6.1.13.2 SignalToUDP Topology

4.6.1.13.2.1 Topology

Topology reference



4.6.1.13.2.2 Node Type

In this topology, nodes involving SignalToUDP are divided into two categories:

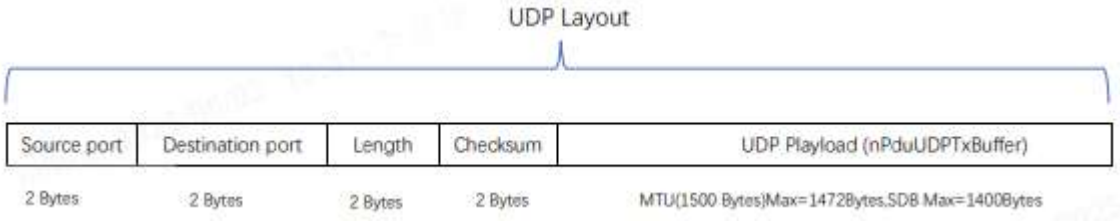
a) SignalToUDP Routing Node
In addition to the following functions of the Endpoint node, it is responsible for I-PDU routing between different protocols and I-PDU forwarding of CAN, LIN, and Ethernet through the PDU Router and SoAd modules.

b) SignalToUDP Endpoint Node
An IP node that receives or sends UDP packets. The node packages or parses UDP packets according to the defined format.

•UDP message Layout

4.6.1.13.3 UDP message Layout

UDP Layout as shown in the following figure：



4.6.1.13.3.1 nPduUdpTxBuffer

The Layout of the nPduUdpTxBuffer (later abbreviated as nPdu) is as follows：

There is no fixed Layout for nPDU. The I-PDUs in the nPduUdpTxBuffer before sending depend on which I-PDUs are placed in it.

nPDU can be cached for different Socket connections. In no special case, an I-PDU can be defined to only one Socket Connection.

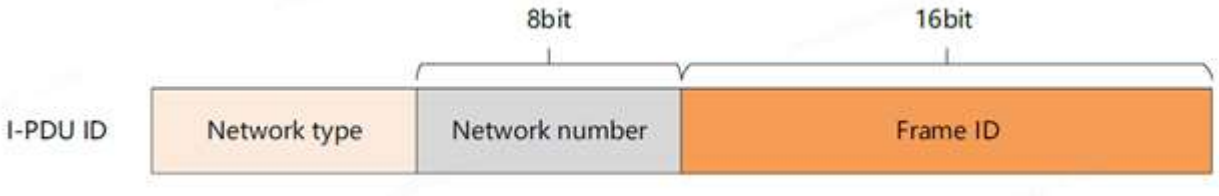
The receiver uses the Header to identify the I-PDU.

4.6.1.13.3.2 I-PDU Header

In the I-PDU Header, the 32-bit ID uniquely identifies an I-PDU and the 32-bit Length indicates the data Length of the I-PDU.

4.6.1.13.3.3 I-PDU ID

Each I-PDU has a globally unique ID. The high 8 bits in the I-PDU ID indicate the type of network segment (the network segment where the I-PDU source resides), the middle 8 bits indicate the specific network segment, and the lower 16 bits correspond to the Frame ID. The I-PDU ID Format is as follows:



When designing I-PDU IDs, observe the following rules:

a) Network Type:

Type	Value
LIN	0x01
CAN(FD)	0x02
Ethernet	0x03

b) Network Number:

LIN Network number fields are as follows：

Type	Value	Type	Value	Type	Value
DIM_LIN1	0x01	ZCUP_LIN1	0x11	SMB_LIN1	0x21
IHU_LIN2	0x02	ZCUP_LIN2	0x12	SMBR_LIN1	0x22
DHU_LIN3	0x03	CCM_LIN1	0x13	SMBL_LIN1	0x23
	0x04	CCM_LIN2	0x14		
	0x05	CCM_LIN3	0x15		
	0x06	CCM_LIN4	0x16		
	0x07	CEM_LIN5	0x17		
	0x08	CEM_LIN8	0x18		
	0x09				

CAN(FD) Network number fields are as follows:

Type	Value
ZCU_CANFD1	0x01
ZCU_CANFD2	0x02
ConnectivityCANFD	0x03
Propulsion_CAN	0x04
Private info_CAN	0x05
PrivateBNCMCANFD	0x06
PrivateRPDMCANFD1	0x07
Private RDHU CANFD1	0x08
Private RDHU CANFD2	0x09
TCAM_PrivateCAN FD2	0x0A
PropulsionCANFD	0x0C
ChassicCANFD1	0X0D

Ethernet Network number fields are as follows:

Type	Sub type	Value
Ethernet Cluster	Ethernet to ZCU_CANFD2	0x01
Ethernet Cluster	Ethernet to ZCU_CANFD1	0x02
Ethernet Cluster	Ethernet to PrivateRPDMCANFD1	0x03
Ethernet Cluster	Ethernet to Private RDHU CANFD1	0x04
Ethernet Cluster	Ethernet to Private RDHU CANFD2	0x05

c) Low 16bit field:
Corresponds to the Frame ID.

4.6.1.13.3.4 UDP Unused bit rules

In UDP data, unused bits are filled with 0x00.

•Routing Node Requirements

4.6.1.13.4 Routing Node Requirements

4.6.1.13.4.1 CAN/LIN transfer UDP requirements

857535 v1 **Buffer configuration**

Version history:
(1):

Purpose: to define the QueueDepth of a single I-PDU and storge mode.
Source:
Verification Method: Analysis
Legacy ID:

When I-PDUs are transferred to SoAd, FIFO buffer is used for caching. Last is best Is not allowed. The FIFO depth is configured based on a single I-PDU , The QueueDepth of a single I-PDU is shown in the following table:

Parameter	Value

QueueDepth	6(Number of I-PDUs)-recommend value

857536 v1 **I-PDU sequence mechanism**

Version history:
(1):

Purpose: to define the packing sequence of I-PDUs.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The packing sequence of I-PDUs in nPDU must be the same as that of I-PDUs entering SoAd.
Do not use the priority sequence.

857534 v1 **UDP Length**

Version history:
(1):

Purpose: Define the UDP Length.
Source:
Verification Method: Test
Legacy ID:

The Threshold Size(SoAdSocketnPduUdpTxBufferMin) is set to 1400 bytes, which is defined by SDB Team.

857540 v1 **VLAN Priorit y Setting Rules for nPDU**

Version history:
(1):

Purpose: to define encapsulate I-PDUs rules.
Source:
Verification Method: Inspection
Legacy ID:

For signals with different delay requirements, in the design of frame packer, existing rules encapsulate I-PDUs with different cycles or different transmission types according to their delay requirements.
To minimize the switch forwarding delay, I-PDUs with different cycles or transmission types should be encapsulated in different nPdu, and then different priorities should be set by associating VLANs of different network endpoints.
For specific VLAN usage, please refer to SDB.

4.6.1.13.4.1.1 UDP Message triggering mechanism

a) nPdu and I-PDU timeout Triggering
b) SoAdSocketnPduUdpTxBufferMin Triggering
c) I-PDU with TRIGGER_ALWAYS Triggering
The combination of the preceding three mechanisms can be used to trigger UDP packet sending , and it is defined by SDB Team.

857539 v1 **I-PDU with TRIGGER_ALWAYS Triggering**

Version history:
(1):

Purpose: For I-PDUs with special latency requirements, such as less than 1ms, this mechanism should be used.

Source:

Verification Method: Test

Legacy ID:

Whenever an I-PDU with TRIGGER_ALWAYS is placed in an nPdu, the nPdu will be sent. If an I-PDU is TRIGGER_ALWAYS, the TimeOut parameter should not be set, and vice versa.

For I-PDUs with special latency requirements, such as less than 1ms, this mechanism should be used.

For I-PDUs to set TRIGGER_ALWAYS, this requirement should be reflected at the signal or signal group level in the SCR of the subsystem design.

857538 v1 **SoAdSocketnPduUdpTxBufferMin Triggering**

Version history:
(1):

Purpose: to define how to trigger to sending out of the nPDU.

Source:

Verification Method: Analysis AND Test

Legacy ID:

As long as the effective payload of the nPDU exceeds the size of the parameter SoAdSocketnPduUdpTxBufferMin, the sending of the nPDU will be triggered.
All nPdu should use this mechanism and use the recommended values in the following table:

Parameter	Value
SoAdSocketnPduUdpTxBufferMin	1400Bytes

857537 v1 **nPdu and I-PDU timeout Triggering**

Version history:
(1):

Purpose: Timeout triggering can be set for nPdu and I-PDU.

Source:

Verification Method: Analysis AND Test

Legacy ID:

Timeout triggering can be set for nPdu and I-PDU respectively. If both values are defined, UDP packets will be sent when either nPdu or I PDU times out.
These two parameters can be understood as requirements for forwarding delay. If nPdu or I-PDU uses thi s mechanism, the recommended values in the following table should be used:

Type	Timeout Value
I-PDU	5ms(min)
nPDU	5ms(min)

4.6.1.13.4.2 UDP Transfer CAN /LIN requirement

857543 v1 **Buffer Configuration**

Version history:

(1):

Purpose: to define the storge of received a single I-PDU.
Source:
Verification Method: Analysis AND Test
Legacy ID:

When receiving a UDP packet containing I-PDU, the Routing node uses FIFO buffer instead of Last Is Best. The FIFO depth is configured based on a single I-PDU , The QueueDepth of a single I-PDU is shown in the following table:

Parameter	Value
QueueDepth	12(Number of I-PDUs)-recommend value

857544 v1 **I-PDU forwarding triggering mechanism**

Version history:
(1):

Purpose: to define forward I-PDUs to specific CAN/LIN network segments.
Source:
Verification Method: Test
Legacy ID:

Routing nodes should forward I-PDUs to specific CAN/LIN network segments according to TRIGGER_ALWAYS.

857545 v1 **UDP receiving process**

Version history:
(1):

Purpose: How to deal with stop processing nPdu.
Source:
Verification Method: Test
Legacy ID:

When receiving a UDP packet, the Routing node uses the Header to identify the unique I-PDU.
For the received I-PDU without Header ID, the Routing node stops processing nPdu and discards subsequent data.

•Endpoint Node Requirements

4.6.1.13.5 Endpoint Node Requirements

4.6.1.13.5.1 Endpoint Node Requirements

857550 v1 **UDP Length**

Version history:
(1):

Purpose: to define UDP Length to lack the delay.
Source:
Verification Method: Analysis AND Test
Legacy ID:

When the network communication team defines SDB, the Length of UDP packets is defined based on the actual situation.
The UDP Length is set to 1400 bytes, which is defined by SDB Team.

857551 v1 UDP Message triggering mechanism

Version history:
(1):

Purpose: the UDP Message triggering mechanism.
Source:
Verification Method: Demonstration
Legacy ID:

The UDP packet Sending mechanism can be triggered using the combination of the following three mechanisms , which is defined by SDB Team.
a) nPdu and I-PDU timeout Triggering:

Type	TimeOut Value
nPdu	5ms(min)
I-PDU	5ms(min)

b) SoAdSocketnPduUdpTxBufferMin Triggering:

Parameter	Value(Byte)
SoAdSocketnPduUdpTxBufferMin	1400

c) I-PDU with TRIGGER_ALWAYS Triggering:

The TRIGGER_ALWAYS attribute is recommended for event based I-PDUs or I-PDUs that require high latency.

857548 v1 UDP Packaging mechanism

Version history:
(1):

Purpose: The Endpoint how to deal with the signal.
Source:
Verification Method: Analysis
Legacy ID:

The Endpoint should use the same packaging format as defined in UDP message Layout: Encapsulate the Signal as I-PDU, put it into nPdu, and transmit it over UDP.

857554 v1 UDP receiving process

Version history:
(1):

Purpose: UDP receiving process in Endpoint.
Source:
Verification Method: Test
Legacy ID:

When receiving a UDP packet, the Endpoint node uses the Header to identify the unique I-PDU.
If the received I-PDU without Header ID,the Endpoint node stops processing nPdu and discards subsequent data.

857553 v1 **VLAN Priority Setting Rules for nPDU**

Version history:
(1):

Purpose: VLAN Priority Setting Rules for nPDU
Source:
Verification Method: Demonstration
Legacy ID:

The VLAN Priority Setting Rules for nPDU is the same with ReqID 857540, and it is defined by SDB Team.

4.7 Communication_WLAN

4.7.1 Communication_Wlan_Requirement_Specification

4.7.1.1 WLAN General

•Requirements

4.7.1.1.1 Requirements

1032777 v1 **Roaming Support**

Version history:
(1):

Purpose: 支持WIFI漫游模式
Source:
Verification Method: Analysis
Legacy ID:

The WLAN transceiver shall support roaming within ESS when connected to OEM Access Points.

1032778 v2 **Test Report Requirements**

Version history:
(2): Optimization description and add Purpose description.

Purpose: 确保覆盖Wi-Fi模组射频测试和整车无线网络性能测试
Source:
Verification Method: Analysis
Legacy ID:

It shall be possible to enable debug mode on WLAN transceiver to enable them to connect to WLAN testers to perform RF parameter testing.

ECUs with integrated Wi-Fi modules shall provide module-level Radio Frequency (RF) test reports and vehicle-level wireless performance assessment reports, and shall confirm the test results with the GRI Basetech team.

1032790 v2 WLAN Parameter Monitor

Version history:
(2): change Verification Method from Analysis and Test to Analysis

Purpose: 确保日志中能记录WIFI相关参数
Source:
Verification Method: Analysis
Legacy ID:

At minimum, it shall be possible to monitor (online analysis):

- Used Frequency
- Used channel
- SSID (if connected)
- Data rate (Bitrate)
- MAC address of BSSID
- SNR
- PER
- Link Level (RSSI)
- Link Noise
- Link Quality for a specific BSS when connected
- Tx-power
- Missed beacon
- Discarded packets
- Security Credentials

The ECU that integrates the Wi-Fi module shall support logging the above parameters in a log file and shall confirm the log-reading method with the GRI Basetech team.

集成Wi-Fi模组的ECU需支持在日志文件中记录上述参数，并与GRI Basetech团队确认日志读取方式。

1032774 v1 WLAN Performance Requirements

Version history:
(1):

Purpose: 确认整车环境下车载WIFI性能满足工厂使用需求
Source:
Verification Method: Analysis AND Test
Legacy ID:

In the full-vehicle environment, Wi-Fi performance shall be optimized from dimensions such as vehicle layout and antenna design. When the on-board Wi-Fi is operating in STA mode, the RX signal must remain above –70 dBm within a 15 m range. At a 15 m distance (with receiver sensitivity ≥ –70 dBm), the RX/TX throughput test results shall still achieve at least 60% of the values measured at a 5 m distance.

在整车环境下，需从整车布置、天线设计等维度优化Wi-Fi性能，当车载Wi-Fi处于STA模式下，需保证15m范围内RX信号高于-70dBm，在15m距离（接收灵敏度 ≥ –70 dBm）条件下，吞吐量的RX/TX测试结果仍需达到5m距离时

测得值的60%。

4.8 Diagnostics Transport & Network Layer

4.8.1 Transport & Network Layer

4.8.1.1 DoCAN

•Introduction

4.8.1.1.1 Introduction

This document describes specific adaptations for the CAN transport protocol. This document specifies the OSI network layer 3 and transport layer 4 requirements for the diagnostic software component.

Figure – Document structure

Note
This document covers both the older CAN standard, referred to as classic CAN, and the updated standard CAN Flexible Data-Rate, referred to as CAN FD. Unless explicitly specifying the standard of CAN in this document, any reference to CAN refers to both the classic CAN and CAN FD standards.

•Requisite documents

4.8.1.1.2 Requisite documents

Reference	Source	Title
[DoCAN_2]	ISO	ISO 15765-2:2016 Road vehicles – Diagnostic communication over Controller Area Network (DoCAN) - Part 2: Transport protocol and network layer services
[DoCAN_3]	ISO	ISO 11898-1:2015 Road vehicles - Controller area network (CAN) - Part 1: Data link layer and physical signalling

Information obtaining ISO documents may be found on the internet: www.iso.org

•Definitions and Abbreviations

4.8.1.1.3 Definitions and Abbreviations

Definition	Description
Client	Software entity which uses services of a server. Normally this is a tester.
Client side	The ECU sending a diagnostic request on the specific network. The ECU may either be a gateway sending a diagnostic request from a higher level network or a tester directly connected to the network.
FC.CTS	See [DoCAN_2].
FC.OVFLW	See [DoCAN_2].
FC.Wait	See [DoCAN_2].
N_Ar	See [DoCAN_2].
N_As	See [DoCAN_2].
N_Br	See [DoCAN_2].
N_Bs	See [DoCAN_2].
N_Cr	See [DoCAN_2].
N_Cs	See [DoCAN_2].
N_SA	See [DoCAN_2].
N_TA	See [DoCAN_2].
N_WFTmax	See [DoCAN_2]
Server	Software entity which provides services for clients
Server side	The ECU sending a diagnostic response on the specific network. The ECU may either be a server or a gateway transferring a diagnostic response from a lower level network.
STMin	See [DoCAN_2].
Tester	A system that controls functions such as test, inspection, monitoring, or diagnosis of an on-vehicle electronic control unit and may be dedicated to a specific type of operators (e.g. a scan tool dedicated to garage mechanics

	or a test tool dedicated to assembly plant agents). The tester is also referenced as the client.

Abbreviation	Description
CF	See [DoCAN_2].
DLC	Data Length Code
ECU	Electronic Control Unit
FF	See [DoCAN_2].
OBD	On Board Diagnostics
TP	Transport Protocol
USDT	Unacknowledged Segmented Data Transfer

•Transport layer protocol

4.8.1.1.4 Transport layer protocol

The transport/network layer required by this document shall be compliant to *[DoCAN_2] Road vehicles – Diagnostic communication over Controller Area Network (DoCAN) - Part 2: Transport protocol and network layer services*, with the restrictions/additions as defined by the subsequent sub-sections.

This document specifies OEM specific requirements. If there are other kinds or requirements like legislative requirements (for example OBD requirements), these requirements may have higher priority than the requirements in this specicifation. However if the other requirements have performance requirements lower than the performance requirements specified in this document, the performance requirements in this document shall be followed. This is valid as long as the performance requirement in this document does not contradict timeout requirements in the other document.

660946 v1 **Precedence of requirements**

Version history:
(1):

Purpose: Define which set of requirements to use for which circumstances.
Source:
Verification Method: Analysis
Legacy ID: 60004v1

If there are other kinds or requirements like legislative requirements (for example OBD requirements), these requirements may have higher priority than the requirements in this specification. However if the other requirements have performance requirements lower than the performance requirements specified in this document, the performance requirements in this document shall be followed. This is valid as long as the performance requirement in this document does not contradict timeout requirements in the other document.

660945 v1 **Transport and Network layer**

Version history:
(1):

Purpose: Standardise communication to ensure all ECUs uses the same diagnostic

communication specifications. ISO standard shall be followed as far as possible unless otherwise specified to reduce cost and make implementation easier.

Source:
Verification Method: Analysis
Legacy ID: 60003v1

The transport/network layer shall be compliant to *[DoCAN_2] Road vehicles – Diagnostic communication over Controller Area Network (DoCAN) - Part 2: Transport protocol and network layer services* with the restrictions/additions as defined by this document. If there are contradictions between this specification and *[DoCAN_2] Road vehicles – Diagnostic communication over Controller Area Network (DoCAN) - Part 2: Transport protocol and network layer services,* then this specification shall override *[DoCAN_2] Road vehicles – Diagnostic communication over Controller Area Network (DoCAN) - Part 2: Transport protocol and network layer services*.

4.8.1.1.4.1 Protocol control information specification

4.8.1.1.4.1.1 BlockSize (BS) parameter definition

When in a non-programming session, both the server and client side shall be able to receive a complete diagnostic message without sending any intermediate flow control frames, to ensure as swift communication as possible. Client side shall be able to receive 4095 bytes diagnostic message without sending any flow control frames i.e. BlockSize = 0x00.

Block size for CAN using 8 bytes frame length
When in a non-programming session the server shall be able to receive diagnostic messages of at least 100 bytes without sending flow control frames. Translated into block size the server shall be able to receive 100/(7 bytes / CF), which rounds up to 15 Consecutive Frames, i.e. the server shall have a BlockSize parameter set to 15 (0x0F) or larger, e.i. BlockSize 1 to 14 is not allowed. Server may also set BlockSize to 0x00 if it is capable of receiving 4095 bytes diagnostic message without sending Flow Control frames.

Block size for CAN FD using 64 byte frame length
When in a non-programming session the server shall be able to receive diagnostic messages of at least 200 bytes without sending flow control frames. Translated into block size the server shall be able to receive 300/(64 bytes / CF), which rounds up to 5 Consecutive Frames, i.e. the server shall have a BlockSize parameter set to 5 (0x05) or larger, e.i. BlockSize 1 to 4 is not allowed. Server may also set BlockSize to 0x00 if it is capable of receiving 4095 bytes diagnostic message without sending Flow Control frames.

When in programming session server side shall be able to receive a complete diagnostic message of 4095 bytes without sending any intermediate Flow Control frames i.e. BlockSize shall always be 0x00 for all services the server supports.

660949 v1	BlockSize parameter non-programming session client side

Version history:
(1):

Purpose: Define BlockSize for non-programming session client side. For more information see section BlockSize (BS) parameter definition.
Source:
Verification Method: Test
Legacy ID: 60007v2

For ECU gatewaying responses from CAN to another network:
In non-programming session the receiver of an upstream message must respond with BS=0 so that there are never more than one FlowControl N_PDUs per 4095 bytes of data (including the first FlowControl N_PDU) when buffer resources makes 4095 bytes available.
In non-programming session, if the largest available RAM buffer is less than the length of the message, that buffer must be used, and BS set so that all of that buffer is used. This will reduce the number of FlowControl N_PDU used for reception of the complete message.
It is mandatory to use the smallest RAM buffer available that can hold the complete message, when such buffer is available.
Note:
Example A: A 300 bytes message is received, and the only RAM buffer available is 4095 bytes. BS is 0 or at least 42.
Example B: A 300 bytes message is received, and RAM buffers are available with 50 bytes and 100 bytes, so none of the buffers can hold the complete message. A 100 bytes RAM buffer is used in favour of 50 bytes buffers, which reduce the number of FlowControl N_PDUs. BS will be 13.

660947 v1 BlockSize parameter non-programming session server side

Version history:
(1):

Purpose: Define BlockSize for non-programming session server side. For more information see section BlockSize (BS) parameter definition
Source: Section "BlockSize (BS) parameter definition"
Verification Method: Test
Legacy ID: 60006v2

BlockSize shall be 15 (0x0F) or higher for server side. If the server can receive 4095 bytes without intermediate FlowControl frames, the BlockSize parameter shall be set to 0x00.

660951 v1 BlockSize parameter programming session client side

Version history:
(1):

Purpose: Define BlockSize for programming session client side. For more information see section BlockSize (BS) parameter definition.
Source:
Verification Method: Test
Legacy ID: 60009v1

BlockSize (BS) shall always be 0x00 in programming session for client side

660950 v1 **BlockSize parameter programming session server side**

Version history:
(1):

Purpose: Define BlockSize for programming session server side. For more information see section BlockSize (BS) parameter definition.
Source:
Verification Method: Test
Legacy ID: 60008v1

BlockSize (BS) shall always be 0x00 in programming session for server side.

660948 v1 **CAN FD BlockSize parameter non-programming session server side**

Version history:
(1):

Purpose: Define BlockSize for non-programming session server side
Source:
Verification Method: Test
Legacy ID:

BlockSize shall be 5 (0x05) or higher for server side. If the server can receive 4095 bytes without intermediate FlowControl frames, the BlockSize parameter shall be set to 0x00.

4.8.1.1.4.1.2 Separation (STmin) parameter definition

The STmin parameter shall reflect the maximum processing time the ECU needs to have in between each CAN frame, regardless of ECU state or activity. For example if an ECU sets a STmin time of 1ms, the supplier must guarantee that the ECU is always able to process consecutive frames at this rate. Preferably the STmin time shall be as low as possible, but the server is allowed to set a maximum of 15ms. Client side shall be dimensioned to handle STmin time of 0 which means CAN frames are being sent back to back. The reason why STmin requirement is tougher for gateways than the ECU's on the network is:

- When in non-programming session, most requests requiring some performance are small enough to be fitted into one frame.
- More lenient requirements lessen the burden on the servers, making it possible to use ECU's with less performance as servers.

When in programming session both server and client side shall be able to handle receiving frames with no separation time primarily for 2 purposes.

- Considering the amount of data to transfer via diagnostics when in programming session compared to the other sessions, the need for speed is greater when in programming session.
- Since when in bootloader mode the ECU will not be burden by simultaneously running the normal functionality supported by the ECU, the ECU shall be able to handle this extra processor load.

660953 v3 **Separation time (STmin) non-programming session client side**

Version history:

(3): Delete the Note, limit STmin to 1.

Purpose: Define STmin for non-programming session client side. For more information see section Separation (STmin) parameter definition.
Source:
Verification Method: Test
Legacy ID: 60011v2

When in non-programming session the ECU shall be able to receive CF N_PDU with minimum 1ms separation time (i.e. STmin set to 1ms).

660952 v2 **Separation time (STmin) non-programming session server side**

Version history:
(2):

Purpose: Define STmin for non-programming session server side. For more information see section Separation (STmin) parameter definition.
Source: Section "Separation (STmin) parameter definition"
Verification Method: Test
Legacy ID: 60010v0

Separation time (STmin) for server side when in non-programming session shall be maximum 15ms.

To consider FOTA update scenario, for ECU gatewaying requests from CAN to another network, separation time (STmin) for server side when in non-programming session shall be maximum 0ms.

660955 v2 **Separation time (STmin) programming session client side**

Version history:
(2):

Purpose: Define STmin for programming session. For more information see section Separation (STmin) parameter definition.
Source:
Verification Method: Test
Legacy ID: 60013v2

Separation time (STmin) for programming session shall be maximum 0ms for client side.

660954 v1 **Separation time (STmin) programming session server side**

Version history:
(1):

Purpose: Define STmin for programming session server side. For more information see section Separation (STmin) parameter definition.
Source:
Verification Method: Test
Legacy ID: 60012v1

Separation time (STmin) for programming session shall be maximum 0ms for server side.

4.8.1.1.4.1.3 Separation time between single frames

In programming session the ECU shall be able to handle receiving single frames (SF) with no separation time. This applies regardless of whether the single frames are received from one or more servers.

Examples for a server:

a: queued request where both the first and second request each fits in a single frame.

Examples for a gateway ECU:

b: a server sends negative response NRC 0x78 and immediately after it sends final response.

c: several servers sends negative response NRC 0x78 and immediately after they send final responses.

In these examples and if using 500kbps bit rate up to four single frames shall be received within one millisecond.

660956 v2 **Separation time between single frames - programming session**

Version history:
(2): 对于编程会话下的单帧间隔处理，区分支持队列的和不支持队列的情况。

Purpose: To ensure no messages are lost and good timing performance.
Source:
Verification Method: Test
Legacy ID: 380118v0

If ECU supports queued requests in programming session. The ECU shall handle receiving single frames (SF N_PDU) with as short time apart as the CAN protocol allows in programming session. This applies both when the single frames are from the same sender and from different senders.
Note1: the shortest separation time of CAN frames is the minimum Interframe Space.
Note2: In programming session, ECUs which do not support queued request shall handle receiving single frames with interval not exceeding P2server_max.

4.8.1.1.4.2 Maximum number of FC. Wait frame transmission (N_WFTmax)

The N_WFTmax counter shall be set to 255 (0xFF). In programming session, the server may use FC.Wait if it receives a request to be queued in server, when it is processing a previously received request. A gateway ECU may use FC.Wait in any situation and session when it does not have enough buffers available for receiving the complete message.

660959 v1 **Flowcontrol Wait usage**

Version history:
(1):

Purpose: Use FC.Wait to halt message reception in appropriate situations.
Source:
Verification Method: Test
Legacy ID: 115798v2

Non-gateway server:
In programming session, the server may use FC.Wait if it receives a request when it is processing a previously received request. In all other situations in all sessions usage of FC.Wait is not allowed.
Gateway ECU:
The gateway ECU shall use FC.Wait if it receives a message when buffers are not available for receiving the complete message. This applies to server side and client side.

660958 v1 **N_WFTmax value for client side**

Version history:
(1):

Purpose: Define the value to be used as N_WFTmax for client side. For more information see section Maximum number of FC.Wait frame transmissions (N_WFTmax).
Source:
Verification Method: Test
Legacy ID: 60016v1

N_WFTmax counter shall be set to 255 (FF hex) for client side. This requirement is applicable for all sessions.

660957 v1 **N_WFTmax value for server side**

Version history:
(1):

Purpose: N_WFTmax counter shall be set to 255 (FF hex) for server side. This requirement is applicable for all sessions. For more information see section Maximum number of FC.Wait frame transmissions (N_WFTmax).
Source:
Verification Method: Analysis AND Test
Legacy ID: 60015v1

N_WFTmax counter shall be set to 255 (FF hex) for server side. This requirement is applicable for all sessions.

4.8.1.1.4.3 Network layer timing

4.8.1.1.4.3.1 Timing parameters

The time it takes to send one CAN frame (N_Ar and N_As) depends on the bit rate of the CAN network and if the bit rate is set to 500kbit/s the lowest possible value for N_Ar and N_As is 250 µs. If a CAN with a different bit rate is used, performance of N_Ar and N_As will have to be recalculated.
Different parameter values applies depending on if the ECU is in programming or non-programming session.
The network layer timing parameters for non-programming session is described in Table – Timing parameters non-programming session.

Item	Parameter	Timeout value	Performance requirement value
1	N_As	1000 ms	-
2	N_Ar	1000 ms	-
3	N_Bs	1000 ms	-
4	N_Br	-	N_Br < 20 ms for FC.CTS and FC.OVFLW. For FC.Wait the periodicity shall be N_Bs * 0.9 = 900ms ± 50ms.
5	N_Cs	-	N_Cs < 20 ms.
6	N_Cr	1000ms	-

Table – Timing parameters non-programming session The network layer timing parameters for programming session is described in Table – Timing parameters programming session.

Item	Parameter	Timeout value	Performance requirement value
1	N_As	1000 ms	-
2	N_Ar	1000 ms	-
3	N_Bs	1000 ms	-
4	N_Br		N_Br < 2000 µs for FC.CTS and FC.OVFLW. For FC.WT the periodicity shall be N_Bs * 0.9 = 900ms ± 50ms.
5	N_Cs		N_Cs < 100 µs
6	N_Cr	1000 ms	-

Table – Timing parameters programming session All requirements from this section are specified in section Requirements from section Timing parameters.

Note: When performance timing is verified, timestamp information normally shows time at start of each frame. In such case it is important to know that N_Ar/N_As is included when e.g. evaluating N_Br and N_Cs performance.

660961 v1 **N_Ar timeout in non-programming session**

Version history:
(1):

Purpose: From a system perspective it is important that both sender and receiver side times out roughly the same time. The timeout value shall be high enough to not be affected by situations like occasional high busloads and low enough to get a user friendly system if for example an ECU is not connected.

Source:

Verification Method: Test

Legacy ID: 60019v1

N_Ar timeout value shall be 1000ms in non-programming session.

660968 v1 **N_Ar timeout in programming session**

Version history:
(1):

Purpose: From a system perspective it is important that both sender and receiver side

times out roughly the same time. The timeout value shall be high enough to not be affected by situations like occasional high busloads and low enough to get a user friendly system if for example an ECU is not connected.

Source:
Verification Method: Test
Legacy ID: 60103v2

N_Ar timeout value shall be 1000ms in programming session.

660960 v1 **N_As timeout in non-programming session**

Version history:
(1):

Purpose: From a system perspective it is important that both sender and receiver side times out roughly the same time. The timeout value shall be high enough to not be affected by situations like occasional high busloads and low enough to get a user friendly system if for example an ECU is not connected.

Source:
Verification Method: Test
Legacy ID: 60017v1

N_As timeout value shall be 1000ms in non-programming session.

660967 v1 **N_As timeout in programming session**

Version history:
(1):

Purpose: From a system perspective it is important that both sender and receiver side times out roughly the same time. The timeout value shall be high enough to not be affected by situations like occasional high busloads and low enough to get a user friendly system if for example an ECU is not connected.

Source:
Verification Method: Test
Legacy ID: 60102v1

N_As timeout value shall be 1000ms in programming session.

660963 v1 **N_Br Performance requirement in non-programming session for FC.CTS and FC.OVFLW**

Version history:
(1):

Purpose: The time spent waiting shall be minimized. Although N_Br time does not affect the transmission time as much as N_Cs time, the diagnostic kernel scheduling shall still be done with the same frequency
Source: Table - Timing parameters non-programming session, item 4
Verification Method: Test
Legacy ID: 60062v3

N_Br shall be N_Br < 20 ms for FC.CTS and FC.OVFLW when in non-programming session.
Note:
a. If other CAN frames with higher priority exist on CAN, then a FC N_PDU may be delayed due to CAN protocol arbitration. The requirement is applicable and verifyable only when no such frame exist.
b. For a client side of a gateway sending parallel requests, the CAN frames will have different CAN identifiers and hence different priority, and actual N_Cs may be higher than the requirement, this is expected.
c. For a server, if other servers are sending responses in parallel then a FC N_PDU may be delayed due to CAN protocol arbitration, this is expected.
d. The Autosar CAN Transport Layer configuration parameter CanTpNbr is not configuring the N_Br performance for the first transmitted FlowControl after a FirstFrame although CanTpNbr is stated to be a perfomance configuration parameter. Instead CanTpNbr is configuring an internal timeout of getting internal buffer resources for use in subsequent reception of Consecutive Frames and affect the time between a previous FlowControl and a subsequent FlowControl.

660964 v1 **N_Br Performance requirement in non-programming session for FC.Wait**

Version history:
(1):

Purpose: The time that an ECU may wait before sending a FC.Wait frame must be lower than the specified timeout to ensure the sending ECU does not time out the message. To prevent unnecessary bus load and minimize the risk that N_WFTmax counter is overrun, the periodicity of the FC.Wait frame shall be as high as possible.

Source:

Verification Method: Test

Legacy ID: 60063v4

FC.Wait frames shall be sent with a periodicity of N_Cr * 0.9 = 900ms ± 50ms non-programming session.
Note:
a. If other CAN frames with higher priority exist on CAN, then a FC N_PDU may be delayed due to CAN protocol arbitration. The requirement is applicable and verifyable only when no such frame exist.
b. For a client side of a gateway sending parallel requests, the CAN frames will have different CAN identifiers and hence different priority, and actual N_Cs may be higher than the requirement, this is expected.
c. For a server, if other servers are sending responses in parallel then a FC N_PDU may be delayed due to CAN protocol arbitration, this is expected.
d. The Autosar CAN Transport Layer configuration parameter CanTpNbr is not configuring the N_Br performance for the first transmitted FlowControl after a FirstFrame although CanTpNbr is stated to be a perfomance configuration parameter. Instead CanTpNbr is configuring an internal timeout of getting internal buffer resources for use in subsequent reception of Consecutive Frames and affect the time between a previous FlowControl and a subsequent FlowControl.

660971 v1 **N_Br Performance requirement in programming session for FC.Wait**

Version history:
(1):

Purpose: The time that an ECU may wait before sending a FC.Wait frame must be lower than the specified timeout to ensure the sending ECU does not time out the message. To prevent unnecessary bus load and minimize the risk that N_WFTmax counter is overrun, the periodicity of the FC.Wait frame shall be as high as possible.
Source:
Verification Method: Test
Legacy ID: 60106v2

FC.Wait frames shall be sent with a periodicity of N_Bs * 0.9 = 900ms ± 50ms when in programming session.
Note:
a. If other CAN frames with higher priority exist on CAN, then a FC N_PDU may be delayed due to CAN protocol arbitration. The requirement is applicable and verifyable only when no such frame exist.
b. For a client side of a gateway sending parallel requests, the CAN frames will have different CAN identifiers and hence different priority, and actual N_Cs may be higher than the requirement, this is expected.
c. For a server, if other servers are sending responses in parallel then a FC N_PDU may be delayed due to CAN protocol arbitration, this is expected.

660970 v1 **N_Br performance requirement in programming session for FC.CTS and FC.OVFLW**

Version history:
(1):

Purpose: When in programming session the ECU will not be burdened by simultaneously running an application and hence the ECU shall be able to respond faster than in non-programming session.
Source: Table - Timing parameters programming session, item 4
Verification Method: Test
Legacy ID: 60105v2

N_Br shall be N_Br < 2000 microseconds for FC.CTS and FC.OVFLW when in programming session.
Note:
a. If other CAN frames with higher priority exist on CAN, then a FC N_PDU may be delayed due to CAN protocol arbitration. The requirement is applicable and verifyable only when no such frame exist.
b. For a client side of a gateway sending parallel requests, the CAN frames will have different CAN identifiers and hence different priority, and actual N_Cs may be higher than the requirement, this is expected.
c. For a server, if other servers are sending responses in parallel then a FC N_PDU may be delayed due to CAN protocol arbitration, this is expected.

660962 v1 **N_Bs timeout in non-programming session**

Version history:
(1):

Purpose: From a system perspective it is important that both sender and receiver side times out roughly the same time. The timeout value shall be high enough to not be affected by situations like occasional high busloads and low enough to get a user friendly system if for example an ECU is not connected.
Source:
Verification Method: Test
Legacy ID: 60021v1

N_Bs timeout value shall be 1000ms in non-programming session.

660969 v1 **N_Bs timeout in programming session**

Version history:
(1):

Purpose: From a system perspective it is important that both sender and receiver side times out roughly the same time. The timeout value shall be high enough to not be affected by situations like occasional high busloads and low enough to get a user friendly system if for example an ECU is not connected.
Source:
Verification Method: Test
Legacy ID: 60104v1

N_Bs timeout value shall be 1000ms in programming session.

660966 v1 **N_Cr timeout in non-programming session**

Version history:
(1):

Purpose: From a system perspective it is important that both sender and receiver side times out roughly the same time. The timeout value shall be high enough to not be affected by situations like occasional high busloads and low enough to get a user friendly system if for example an ECU is not connected.
Source:
Verification Method: Test
Legacy ID: 60081v1

N_Cr timeout value shall be 1000ms in non-programming session.

660973 v1 **N_Cr timeout in programming session**

Version history:
(1):

Purpose: From a system perspective it is important that both sender and receiver side times out roughly the same time. The timeout value shall be high enough to

not be affected by situations like occasional high busloads and low enough to get a user friendly system if for example an ECU is not connected.

Source:

Verification Method: Test

Legacy ID: 60108v1

N_Cr timeout value shall be 1000ms in programming session.

660965 v2 **N_Cs Performance requirement in non-programming session**

Version history:
(2):

Purpose: The time spent waiting shall be minimized and especially for N_Cs since N_Cs wait time is the factor that affects the transfer rates the most. If the send mechanism is performed in application schedule context (compared to sending on Tx interrupt), the ECU needs to poll the send function more often, and thus less time will be spent on the applications normal functionality.

Source:

Verification Method: Test

Legacy ID: 60064v4

When in non-programming session, ECU shall be able to send Consecutive Frames with N_Cs less than 20ms.
And for ECU that support gatewaying, N_Cs will be less than 900ms due to the impact of protocol overhead from low network to upper network, i.e. CAN to CANFD, LIN to CAN/CANFD. It is important that the Gateway ECU shall send Consecutive Frames as quickly as possible.

Note:
a. If other CAN frames with higher priority exist on CAN, then a FC N_PDU may be delayed due to CAN protocol arbitration. The requirement is applicable and verifyable only when no such frame exist.
b. For a client side of a gateway sending parallel requests, the CAN frames will have different CAN identifiers and hence different priority, and actual N_Cs may be higher than the requirement, this is expected.
c. For a server, if other servers are sending responses in parallel then a FC N_PDU may be delayed due to CAN protocol arbitration, this is expected.
d. The Autosar CAN Transport Layer configuration parameter CanTpNcs is not configuring the N_Cs performance defined by ISO-15765 although it is stated to be a perfomance configuration parameter. Instead CanTpNcs is configuring an internal timeout of getting data to be placed in the next Consecutive Frame. I.e. if no data can be retrieved internally for sending in a new Consecutive Frame, the sender will abort the transmission after CanTpNcs elapsed.

660972 v2 **N_Cs Performance requirement in programming session**

Version history:
(2):

Purpose: In programming session the consecutive frames shall be sent as close to back

to back as possible for maximum speed.
Source: Table - Timing parameters programming session, item 5
Verification Method: Test
Legacy ID: 60107v2

For ECU, in programming session N_Cs shall be N_Cs < 100 microseconds.
For ECU that support gatewaying, in programming session N_Cs will be less than 200ms due to the impact of protocol overhead from low network to upper network, i.e. CAN to CANFD, LIN to CAN/CANFD. It is important that the Gateway ECU shall send Consecutive Frames as quickly as possible.

Note:
a. If other CAN frames with higher priority exist on CAN, then a FC N_PDU may be delayed due to CAN protocol arbitration. The requirement is applicable and verifyable only when no such frame exist.
b. For a client side of a gateway sending parallel requests, the CAN frames will have different CAN identifiers and hence different priority, and actual N_Cs may be higher than the requirement, this is expected.
c. For a server, if other servers are sending responses in parallel then a FC N_PDU may be delayed due to CAN protocol arbitration, this is expected.

4.8.1.1.4.3.2 Unexpected arrival of N_PDU

In programming session, full duplex shall be supported. In all other sessions half duplex shall be supported.
The reason why full-duplex is to be supported is because of full-duplex makes it possible to queue requests on both the gateways and the servers.

660974 v2 **Duplex communication**

Version history:
(2): In order to achieve ReqID661376, GW where diagnostic routing bidirectional network is CAN shall also support full duplex in applicaiton mode.

Purpose: Full duplex shall be supported to make it possible to queue requests on both the gateways and the servers in programming session.
Source:
Verification Method: Test
Legacy ID: 60109v3

In programming session, full duplex shall be supported.
In all other sessions half duplex shall be supported, except for gateways where diagnostic routing bidirectional network is CAN network.

***Note**: Since the functionally addressed Tester Present (0x3E) commands are sent by the tester in parallel to physical requests/responses(to keep the session alive), there should always be a dedicated receive buffer allocated for this use case.*

•Data link layer usage

4.8.1.1.5 Data link layer usage

4.8.1.1.5.1 Adressing formats

The addressing format shall be “normal addressing” as specified in *[DoCAN_2] Road vehicles – Diagnostic communication over Controller Area Network (DoCAN) - Part 2: Transport protocol and network layer services*. Only 11bit CAN frame identifier (N_TA and N_SA) shall be used. Both physically and functionally addressed requests shall be supported.

660975 v1 **Addressing format**

Version history:
(1):

Purpose: Define addressing format.
Source:
Verification Method: Analysis
Legacy ID: 60110v1

“Normal addressing” as specified in *[DoCAN_2] Road vehicles – Diagnostic communication over Controller Area Network (DoCAN) - Part 2: Transport protocol and network layer services* shall be used.

660976 v1 **CAN frame identifier length**

Version history:
(1):

Purpose: Define length of CAN frame identifier.
Source:
Verification Method: Analysis AND Test
Legacy ID: 60111v1

11-bit CAN frame identifiers shall be used.

660977 v1 **Supporting functional requests**

Version history:
(1):

Purpose: Define support for functional requests.
Source:
Verification Method: Test
Legacy ID: 60112v1

Functional requests shall be supported.

4.8.1.1.5.2 CAN frame data padding

For Classic CAN the DLC (Data Length Code) contained in every CAN frame shall always be set to eight. Any CAN frame with a DLC different from eight shall be considered invalidly formatted and ignored by the recipient.

For CAN FD the DLC (Data Length Code) contained in every CAN frame shall always be set to a value within the range 8 to 15. Any CAN frame with a DLC outside this range shall be considered invalidly formatted and ignored by the recipient.

Unused data shall be padded with 0x00 and the receiver of the frame shall not check the padding.

Note
The number of bytes in the data field of a CAN frame shall be indicated by the DLC. This DLC shall consist of 4 bits. The admissible number of data bytes for Classic CAN frames shall range from 0 to 8. DLCs in the range of 0 to 7 shall indicate data fields of length of 0 to 7 bytes. In Classical CAN frames, all other DLCs shall indicate that the data field is 8 bytes long. In CAN FD frames, DLCs in the range of 0 to 8 shall indicate data fields of length 0 to 8 bytes, while the other DLCs code longer data fields, according to the table below (see *[DoCAN_3] ISO 11898-1*).

Frame types	Data Lenght Code				Number of data bytes
	DLC3	DLC2	DLC1	DLC0	
Classic CAN and CAN FD	0	0	0	0	0
	0	0	0	1	1
	0	0	1	0	2
	0	0	1	1	3
	0	1	0	0	4
	0	1	0	1	5
	0	1	1	0	6
	0	1	1	1	7
	1	0	0	0	8
Classic CAN	1	0 or 1	0 or 1	0 or 1	8
CAN FD	1	0	0	1	12
	1	0	1	0	16
	1	0	1	1	20
	1	1	0	0	24
	1	1	0	1	32
	1	1	1	0	48
	1	1	1	1	64

660980 v1 **Handling of CAN requests in regards of DLC**

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID: 60130v1

USDT request frames with a DLC not equal to eight (8) for Classic CAN, or outside the range 8-15 for CAN FD, shall be considered invalidly formatted and be ignored by the recipient.

660979 v2 **Length of CAN FD frames**

Version history:
(2):

Purpose: Define the length of USDT frames
Source:
Verification Method: Test
Legacy ID:

The DLC (DataLengthCode) of USDT frames on CAN FD format shall be in the range 8 to 15.

Note: The diagnostic message is filled up to the nearest valid message length (8/12/16/20/24/32/48/64) based on the diagnostic data length. For details, please refer to [DoCAN_2].

660978 v1 **Length of Classic CAN frames**

Version history:
(1):

Purpose: Define the length of USDT frames.
Source:
Verification Method: Test
Legacy ID: 60129v1

The DLC (DataLengthCode) of USDT frames on Classic CAN format shall always be set to eight.

660981 v1 **Unused data in the CAN frame**

Version history:
(1):

Purpose: Define what to do with data not used in the USDT frame.
Source:
Verification Method: Test
Legacy ID: 60133v1

Unused data shall be padded with 0x00 and the receiver of the frame shall ignore padding.

4.8.1.1.5.3 Use data frame by frame

While receiving a segmented request for TransferData 0x36 the transport protocol shall send all N_Data from each N_PDU immediately to upper layers that contain the flash programming functions (optional for all other services). The ECU must not start flashing the TransferData block after the complete TrasferData block has been received. If the N_Data (transferRequestParameterRecord) is compressed, all N_Data from each N_PDU shall be sent to the layer where decompression is performed, and that layer shall in turn send data as soon as it is decompressed to upper layers that contain the flash programming functions.
This contradicts normal specified transport layer behaviour since it states that the transport

layer shall only send complete messages to upper layers. This d ecrease the time it takes to perform TransferData by writing frame by frame to flash memory. The ECU shall be able to write the data while receiving new data and therefore the ECU will not need to write as much data at the last message and thus the P4 time will be shorter leading to shorter SWDL time.

660982 v1 **Forward N_Data from each N_PDU to upper layer**

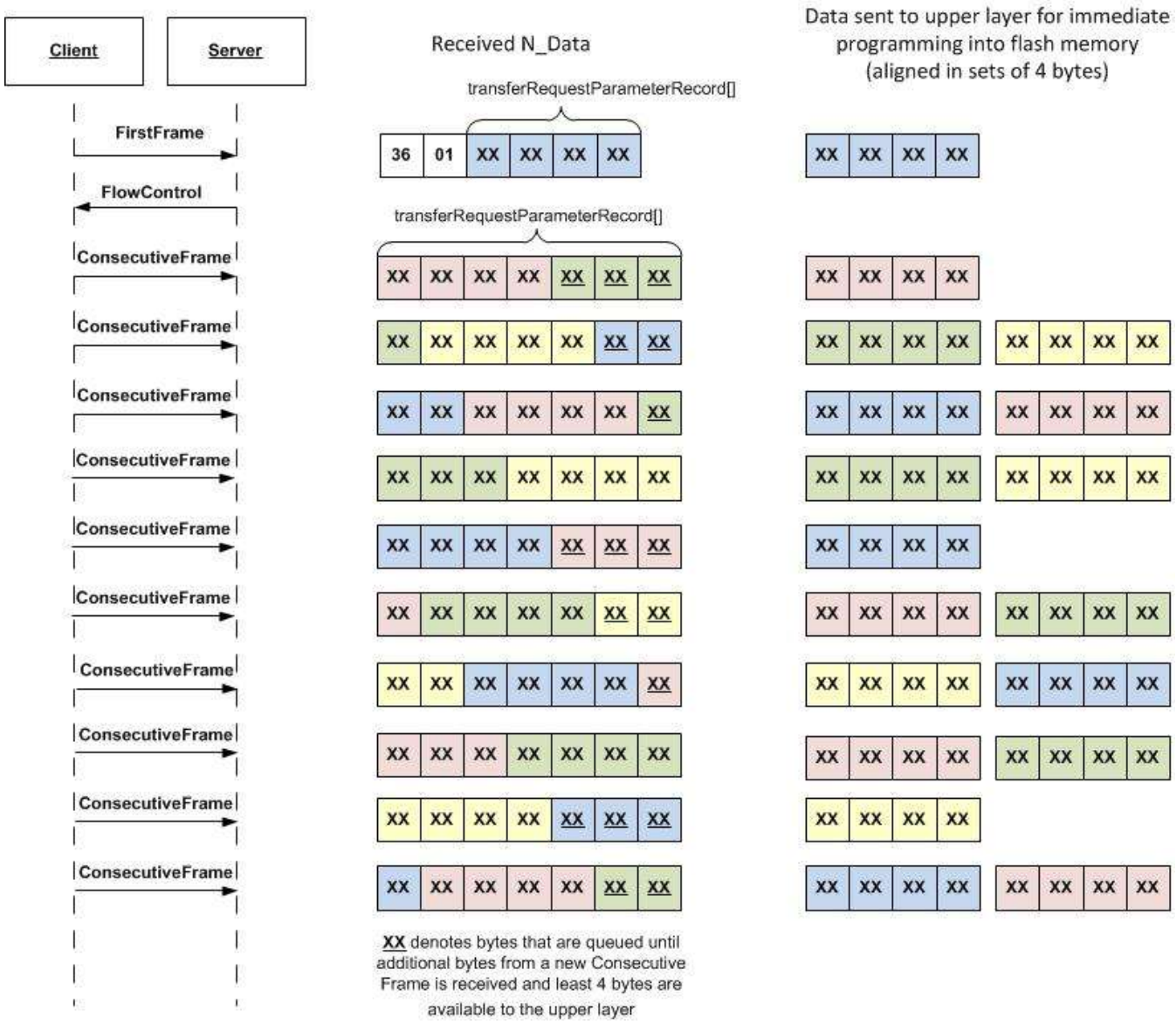
Version history:
(1):

Purpose: Decrease the time it takes to perform TransferData via writing frame by frame. The ECU will be able to write the data while receiving new data and thus the ECU will not need to write as much data at the last message and thus the P4 time will be shorter leading to shorter SWDL time.
Source:
Verification Method: Analysis
Legacy ID: 128908v1

While a diagnsotic server is receiving a segmented request for TransferData 0x36 the transport protocol layer shall send all N_Data from each N_PDU to upper layers that contain the flash programming functions (optional for all other services). If the N_Data is compressed, all N_Data from each N_PDU shall be sent to the layer where decompression is performed, and that layer shall in turn send data as soon as it is decompressed to upper layers that contain the flash programming functions.
Note: this contradicts and shall override ISO specified transport layer behaviour which states that the transport layer shall only send complete diagnostic messages to upper layers.
Depending on programming alignment constrains of the flash memory, the actual programming of bytes may have to be aligned according to the flash memory and not performed immediately when data bytes are available from transport layer or decompression layer.

Example 1: Uncompressed N_Data; a microcontroller has capability to write only 4 bytes at a time into program memory, e.i. not 2 bytes in one event and adjacent 2 bytes in another event. In this case the flash programming function has to reassemble groups of 4 bytes for programming, and must queue any received bytes less than 4 until further bytes are received to get at a group of 4 bytes. In this case 4 bytes of the 6 bytes N_Data from FirstFrame can be programmed immediately, and 2 bytes (6-4) queued and joined with first 2 bytes from the first Consecutive Frame and so on. If the length of the data to be programmed is such that at the end, a group of 4 bytes can not be made, the reaining bytes, less than 4, are written anyway.
Example 2: Uncompressed N_Data; a microcontroller has capability to write only 256 bytes at a time time into program memory. In this case the flash programming function has to reassemble groups of 256 bytes for programming, and must queue any received bytes less than 256. In this case FirstFrame + 36 Consecutive Frames (7 bytes N_Data in FF and 252 bytes N_Data in 36 CF:s for a total of 259 bytes) will be received until flash programming can start when the beginning of TransferData request is received, and 3 bytes (259-256) will be queued and reassembled with the following Consecutive Frames until at least 256 new bytes are received. If the length of the data to be programmed is such that at the end, a group of 256 bytes can not be made, the reaining bytes, less than 256, are written anyway.

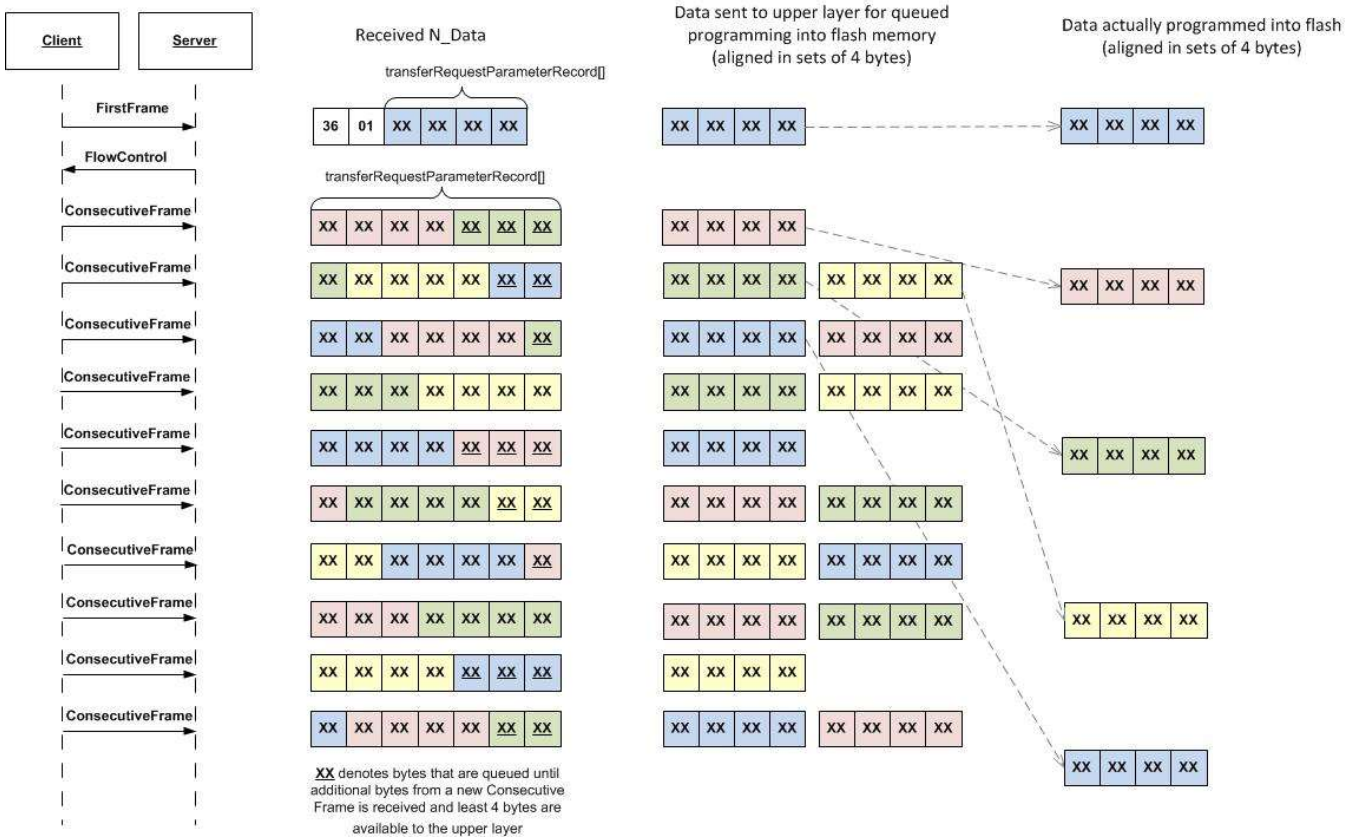
4.8.1.1.5.3.1 Example A



Example A: Alignment of flash memory is on 4 byte boundary with flash programming time which is less than the time until next Consecutive Frame. Bytes are queued and aligned until sets of 4 bytes can be forwarded to the upper layer for immediate programming. The ECU can flash memory at least at the same speed as data from Consecutive Frames are received.

Note: Example A is for Classic CAN (DLC = 8) or CAN FD where DLCs above 8 are not used. A CAN FD example with DLCs above 8 would still require that bytes are queued and aligned for the last fraction of the 4 bytes set in every Consecutive Frame. However, more sets of 4 bytes can immediately be programmed for every Consecutive Frame due to that the DLC is above 8.

4.8.1.1.5.3.2 Example B



Example B: Alignment of flash memory is on 4 byte boundary with flash programming time which is more than the time until next Consecutive Frame. Bytes are aligned in sets of 4 bytes and forwarded to the upper layer for queued programming. The ECU can not flash memory at the same speed as data from Consecutive Frames are received.

Note: Example B is for Classic CAN (DLC = 8) or CAN FD where DLCs above 8 are not used. A CAN FD example with DLCs above 8 would still require that bytes are queued and aligned for the last fraction of the 4 bytes set in every Consecutive Frame. More sets of 4 bytes sets in every Consecutive Frame imply that more data per Consecutive Frame will be forwarded to the upper layer for queued programming.

4.8.1.2 DoLIN

•Introduction

4.8.1.2.1 Introduction

This document describes OEM specific adaptations for the LIN transport layer specification.

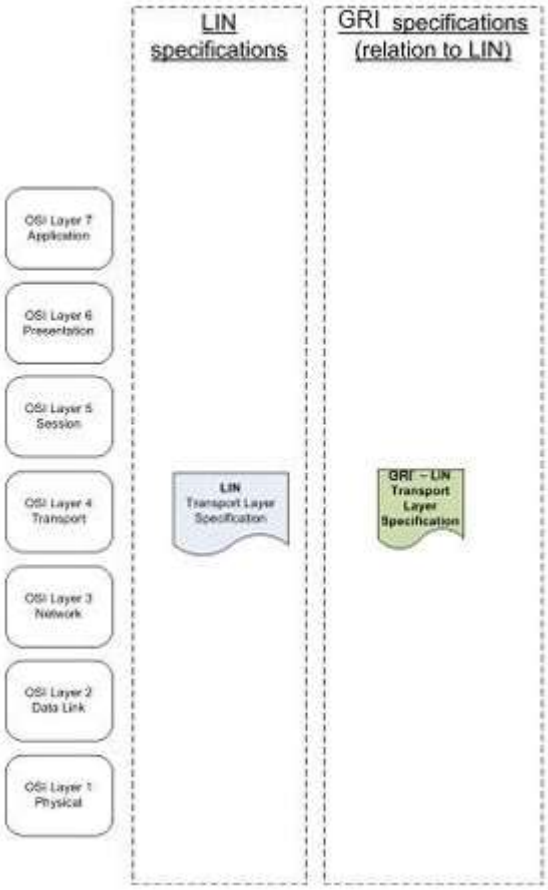


Figure - Document structure

•Requisite documents

4.8.1.2.2 Requisite documents

Reference	Source	Title
[LIN_TP_1]	LIN Consortium	LIN Specification Package Vers 2.1
[LIN_TP_2]	Internal	LIN Data Link Layer - Master Requirements

•Definitions and Abbreviations

4.8.1.2.3 Definitions and Abbreviations

Abbreviation	Description
ECU	Electronic Control Unit
NAD	Node Address. See LIN specification package for further reference
PCI	Protocol Control Information. See LIN specification package for further reference.

•Transport Layer

4.8.1.2.4 Transport Layer

1427 v2	Transport layer

Version history:
(2):

Purpose: Define the transport protocol to use
Source: LIN Transport layer specification
Verification Method: Test
Legacy ID: 82241v1

Diagnosable and/or re-programmable LIN slaves shall support the transport protocol specified in *[LIN_TP_1] LIN Specification Package* with the restrictions/additions as defined in this document.

If there are diagnosable and/or re-programmable LIN slaves on a LIN network the LIN master shall support the transport protocol specified in *[LIN_TP_1] LIN Specification Package* with the restrictions/additions as defined in this document.

4.8.1.2.4.1 LIN Master requirements

1428 v2 **LIN master N_Cs performance in programming session**

Version history:
(2): Consider that the LIN schedule is 15ms.

Purpose: To ensure a certain performance of the LIN master during SWDL.
Source:
Verification Method: Test
Legacy ID: 342937v0

In programming session a LIN master shall have a performance N_Cs not exceeding 20ms and allowed jitter is defined in *[LIN_TP_2] LIN Datalink Layer - Master reqirements.* It is important that the LIN master shall send Consecutive Frames as quickly as possible.

1429 v1 **LIN master usage of N_Cr**

Version history:
(1):

Purpose: Prevent the LIN master from timing out TP messages while the enhanced P2 timer runs.
Source:
Verification Method: Test
Legacy ID: 125501v0

The LIN master shall not time out N_Cr transport protocol timer on first frames (FF). However N_Cr timer is still applicable for consecutive frames (CF).

4.8.1.2.4.2 LIN Slave requirements

1430 v2 **Block size**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: The LIN protocol does not have a flow control which the LIN slave can use to

pause a message reception so the size of the LIN slave message buffer shall not limit the size of supported diagnostic services.

Source:
Verification Method: Analysis
Legacy ID: 82201v0

The LIN slave Rx message buffer shall be large enough to handle any diagnostic request the LIN slave is required to handle. Note that if possible the LIN slave may implement a strategy where earlier received data is freed to make room for new data and thus lowering the need for buffer.

1431 v2 LIN slave proceed with current message if receiving an invalid frame

Version history:
(2):

Purpose: The diagnostic communication shall not be affected by invalid frames on the protocol level.

Source:
Verification Method: Test
Legacy ID: 82207v1

If an segmented diagnostic reception is ongoing, the slave node shall proceed with the reception after:

- Reception of an invalid frame (failure in header, checksum error, framing error).

Note: if there is a failure in header, checksum or framing error it can not be determined what type of frame it is, i.e. it is unknown if it is an application frame or diagnostic frame since it is unknown what parts of the frame are correct and not correct.
Example A; a frame that was intended to be an application frame but was corrupted for any reason shall not disturb diagnostic communication.
Example B; a frame that was intended to be an diagnostic Consecutive Frame from LIN Master but was corrupted for any reason shall be ignored. However, if subsequent Consecutive Frames are received without failure in header, checksum error, framing error, they will probably have unexpected sequence numbers and due to that the diagnostic message reception shall be aborted according to LIN Specification Package, section Transport Layer - Error Handling.

1432 v3 LIN slave receiving a new message while processing a message

Version history:
(3):

Purpose: If LIN master for some reason do not read a response for a request which has been sent to a LIN slave and for which the LIN slave has a response available, it shall be possible to make a new try to communicate with LIN slave by sending a new request. The LIN slave shall the discard the response not read and accept the new request and process it.

Source:
Verification Method: Test
Legacy ID: 82204v2

The slave node shall abort processing of a diagnostic request (discard any reponse if such was

already prepared) that was correctly received after:

- Reception of a valid master request
- Reception of a master request that is valid concerning the LIN protocol, but with absurd data, e.g. wrong PCI.

1433 v1 **Use data frame by frame**

Version history:
(1):

Purpose: Decrease the time it takes to perform TransferData via writing frame by frame. The ECU will be able to write the data while receiving new data and thus the ECU will not need to write as much data at the last message and thus the P4 time will be shorter leading to shorter SWDL time.
Source:
Verification Method: Analysis
Legacy ID: 125542v0

The transport protocol shall send frame by frame to upper layer for diagnostic service TransferData 0x36 (optional for all other services). Note that this contradicts normal specified transport layer behaviour since normal transport layer behaviour states that the transport layer shall only send complete messages to upper layers.

4.8.1.3 DoIP - Generic Info

•Introduction

4.8.1.3.1 Introduction

Diagnostic communication Over IP (DoIP) is a standard developed by ISO for vehicle communication over the Internet Protocol (IP). It introduces a session layer protocol in the OSI model *(see diagram in chapter Overview)* for sending and receiving data over.

This document will for the most part provide specifications regarding optional features in the DoIP standard.
It will also specify hard limits for some "open" values and specify a variety of manufacturer specific values.
It will have two Chapters: One for specifying the DoIP requirements for the ECU called “Edge Node”.
This is the only DoIP ECU in the vehicle.
The other Chapter contains requirements for how to handle DoIP communication with the remaining IP ECUs in the vehicle network.

The document shall be seen as a companion to the official DoIP specification, giving directions on how we expect the DoIP protocol to behave. This document always has higher precedence over ISO-13400 documents and for conflicting information, this document shall be valid.

Anyone who implements the Edge Node, IP ECUs or Tester equipment that shall communicate with these ECUs should read this document. It can also be used as a starting point for testers.

The DoIP standard is very open-ended regarding its network design. A vehicle can e.g. expose

several DoIP ECUs to the outside world, individually or organized in groups and so forth.

The main idea in the vehicle network is to expose a single DoIP ECU and to provide only one IP address to the tester. This makes things like GID synchronization superfluous. The only ECU exposed to a tester is called the Edge Node and all DoIP communication with the vehicle will pass through this ECU. This gives several advantages. With only one point of information flow into the vehicle, one can implement several security mechanisms, which are normally associated with firewalls and intrusion detection systems in "normal" computer network systems. Another advantage with this design is conservation of IP-addresses. There is no ambiguity regarding the mapping between vehicle and IP-addresses. The routing of DoIP messages and diagnostic traffic is also generalized in the Edge Node, e.g. some packets are destined for a Flexray network and some are destined for an internal Ethernet network. For further information regarding routing and gateways please see *[IP Routing*] and [Diagnostic and Bootloader Gateway Specification].

•Requisite Documents

4.8.1.3.2 Requisite Documents

Reference	Source	Title
[IP Routing]	Internal	IP Routing Specification
[Diagnostic and Bootloader Gateway Specification]	Internal	Diagnostic and Bootloader Gateway Specification
[Internetworks]	Internal	Internetworks General Specification
[Diagnostic Mgmt]	Internal	Diagnostic Management
[ISO13400-1]	ISO	ISO 13400-1: Communication between test equipment and vehicles over Internet Protocol – Part 1: General information and use case definition (2011)
[ISO13400-2]	ISO	ISO 13400-2: Communication between test equipment and vehicles over Internet Protocol – Part 2: Network and Transport Layer Requirements and Services (2012)
[RFC2131]	IETF	RFC-2131: Dynamic Host Configuration Protocol (October 1993)
[RFC2132]	IETF	RFC-2132: DHCP Options and BOOTP Vendor Extensions (October 1993)
[RFC2133]	IETF	RFC-4361: Node-specific Client Identifiers for Dynamic Host Configuration Protocol Version Four (February 2006)
[RFC4361]	IETF	RFC-1469: IP Multicast over Token-Ring Local Area Networks (June 1993)

•Definitions & Abbreviations

4.8.1.3.3 Definitions & Abbreviations

Abbreviation	Stands for	Description
DHCP	Dynamic Host Configuration Protocol	DHCP is an autoconfiguration protocol used on IP networks, eliminating the need for intervention by a

		network administrator. This prevents two computers from accidentally being configured with the same IP address.
DoIP	Diagnostics over IP	The goal of the DoIP standard is the complete compatibility to the existing Ethernet standard.
ECU	Electronic Control Unit	Electronic control unit with a computer/microcontroller.
GID	Group ID	Diagnostic Management
IP	Internet Protocol	The IP is the principal communications protocol used for relaying datagrams (packets) across an internetwork using the Internet Protocol Suite.
IPv4	Internet Protocol version 4	IPv4 is a connectionless protocol for use on packet-switched Link Layer networks (e.g., Ethernet)
IPv6	Internet Protocol version 6	IPv6 is a version of the Internet Protocol that is designed to succeed IPv4.
MAC address	Media Access Control address	MAC address is a identifier assigned to network interfaces for communications on the physical network segment.
NM	Network Management	Responsible for start-up, operation, shutdown and communication of a network e.g. CAN or Flexray.
TCP	Transmission Control Protocol	TCP is one of the core protocols of the Internet Protocol Suite. TCP is one of the two original components of the suite, complementing the IP, and therefore the entire suite is commonly referred to as TCP/IP.
UDP	User Datagram Protocol	UDP is one of the core members of the Internet Protocol Suite, the set of network protocols used for the Internet.
VIN	Vehicle Identification Number	
EID	Entity Identification	
UHWID	Unique Hardware ID	

Definition	Description
DoIP Session	In this document a DoIP session is when a TCP connection is established between the Edge Node and the Tester on port 13400.
Edge Node	An ECU (internal vehicle network node) which is the network node that is exposed towards the external network. The node implements a diagnostic over IP-protocol (DoIP - ISO13400) and provides diagnostic access to test equipment.
Ethernet MAC	The Ethernet MAC in this document is referring to the unique and global routable MAC address of the external Ethernet interface of the Edge Node.
Hostname	A string (for example, google.com) representing an IP-address.
Internet Protocol Suite	The group of protocols used on the Internet. Usually a synonym for TCP/IP.
Network	A group of ECUs that are connected together.
Port	A number representing a specific application in the TCP/UDP transport layers.

Tester	A system that controls functions such as test, inspection, monitoring, or diagnosis of an on-vehicle electronic control unit and may be dedicated to a specific type of operators (e.g. a scan tool dedicated to garage mechanics or a test tool dedicated to assembly plant agents). The tester is also referenced as the client.

4.8.1.4 DoIP - Edge Node

•Edge Node Requirements

4.8.1.4.1 Edge Node Requirements

661004 v1 **Accept DoIP TCP Connection Requests From Already Connected IP ECUs**

Version history:
(1):

Purpose: Handle reconnects after an undetected disconnect.
Source:
Verification Method: Test
Legacy ID: 54316v1

If the Edge Node receives a TCP Connection request for DoIP communication from an already connected ECU, the Edge Node shall accept the request and establish a TCP connection to the IP ECU and remove the old entry.

661002 v1 **Accept TCP Connection requests for DoIP Messages**

Version history:
(1):

Purpose: To enable DoIP server access for DoIP messages.
Source:
Verification Method: Test
Legacy ID: 54314v2

If the Edge Node receives a TCP Connection request for DoIP communication from one or several IP ECUs, the Edge Node shall accept the request and establish a TCP Connection.

660994 v1 **Asynchronous request-response handling**

Version history:
(1):

Purpose: To enable the tester to send messages to several ECU:s in parallel
Source:
Verification Method: Test
Legacy ID: 442597v0

The DoIP server shall be able to handle parallel messages to multiple ECU:s as long as the tester does not send more than one request to the same ECU address before the DoIP server has acknowledged the previous request

660983 v1 **Concurrent DoIP Sessions**

Version history:
(1):

Purpose: To ensure that only one TCP data connection is supported in the Edge Node for DoIP sessions.
Source: DoIP-002 [ISO13400-2]
Verification Method: Test
Legacy ID: 54305v2

Only one TCP data connection shall be supported in the Edge Node for DoIP sessions.

660984 v2 **Connected with configured AutoIP Address**

Version history:
(2): Verification Method: Test change to Analysis AND Test, for investigating the code implementation methods of suppliers.

Purpose: If a DoIP session is established with a link-local IP the search for a DHCP server shall be halted. In other case, if a DHCP server is found the vehicle will configure the DHCP IP which in turn will disconnect all connections. This can be an inconvenience for the tester, especially if the tester is in the process of a software download.
Source:
Verification Method: Analysis AND Test
Legacy ID: 54312v1

If a DoIP session is established with a link-local IP the search for a DHCP server shall be halted.

660987 v1 **DHCP Host name option with Ethernet MAC**

Version history:
(1):

Purpose: VIN is not always available and therefore another option is defined that can be used in these cases.
Source: DoIP-017 [ISO13400-2]
Verification Method: Test
Legacy ID: 54287v2

If no VIN is available, all vehicles shall set the DHCP optional host name to: DoIP-$MAC where $MAC is the external Ethernet MAC of the Edge Node.

660986 v1 **DHCP Host name option with VIN**

Version history:
(1):

Purpose: Requirement DoIP-017 in [ISO13400-2] specifies that the host name option shall contain at minimum "DoIP-<manufacturer_specific>" whereas the

<manufacturer_specific> part can be replaced with any text that meets the specific requirements of the manufacturer.

Source: DoIP-017 [ISO13400-2]
Verification Method: Test
Legacy ID: 54286v2

All vehicles shall, when available, request DHCP data with the optional host name set to: DoIP-$VIN where $VIN is the VIN of the vehicle.

660989 v1 DHCP Retry

Version history:
(1):

Purpose: To define how many DHCPDISCOVER messages the DHCP client sends before giving up.
Source: [RFC2131]
Verification Method: Test
Legacy ID: 54288v2

The DHCP shall send 3 DHCPDISCOVER before giving up.

660988 v1 DHCP interval timing

Version history:
(1):

Purpose: According to the DHCP references there should be a delay between each request to avoid congestion and packet loss.
Source: [RFC2131] [RFC4361]
Verification Method: Test
Legacy ID: 54289v2

The DHCP Interval Timer shall be 2 seconds.

660990 v1 Diagnostic Message ACK - Previous data

Version history:
(1):

Purpose: The "Previous Diagnostic Message Data" item in the Diagnostic Message Positive Acknowledgement response is optional in the DoIP specification [ISO13400-2] and has no clear use case at this time.
Source: DoIP-066 [ISO13400-2]
Verification Method: Analysis
Legacy ID: 54292v2

The Diagnostic Message Positive Acknowledgment response item "Previous Diagnostic Message Data" shall not be implemented.

660991 v1 Diagnostic Message NACK - Previous Data

Version history:
(1):

Purpose: The "Previous Diagnostic Message Data" item in the Diagnostic Message Negative Acknowledgement response is optional and has no clear use case at this time.
Source: DoIP-068 [ISO13400-2]
Verification Method: Analysis
Legacy ID: 54293v2

The Diagnostic Message Negative Acknowledgment response item "Previous Diagnostic Message Data" shall not be implemented

660992 v1 **Diagnostic message negative acknowledge code 0x06**

Version history:
(1):

Purpose: To define which of the optional error codes that shall be used.
Source: DoIP-103 [ISO13400-2]
Verification Method: Test
Legacy ID: 54310v2

DoIP-103 in *[ISO13400-2]*, shall be supported. If a link is lost on the target bus connected to the Edge Node, then the Edge Node shall send a DoIP NACK with error code 0x06, target unreachable.

660985 v1 **Disconnected with configured AutoIP Address**

Version history:
(1):

Purpose: If no DoIP session is established with the vehicle, it shall search for a DHCP server even though an AutoIP has been configured. An IP address from a DHCP server is regarded as more important since it enables communication on an external network. Therefore the search must not stop.
Source: DoIP-023 [ISO13400-2]
Verification Method: Test
Legacy ID: 54313v3

If no DoIP session is established the Edge Node shall search for a DHCP server in accordance to REQPROD 54286, REQPROD 54287 and DoIP-023 *[ISO13400-2]* .

660993 v1 **DoIP ACK**

Version history:
(1):

Purpose: Consensus on use of DoIP ACK's
Source: DoIP-066 [ISO13400-2]
Verification Method: Test
Legacy ID: 54309v2

The DoIP ACK shall be sent when the entire message has been received, processed correctly and verified that the destination network is active.

660999 v1 **Group ID Default Value**

Version history:
(1):

Purpose: The Group ID (GID) shall be preconfigured and static since only one DoIP node will send the Vehicle Announcement Message, namely the Edge Node. Therefore a deviation from DoIP-142 in [ISO13400-2] is made, even though it is not guaranteed that the vehicle can be identified by the VIN at any time no support is offered for GID.
Source: DoIP-046 [ISO13400-2]
Verification Method: Test
Legacy ID: 54299v3

The GID shall always be set to 0x000000000001.

661000 v1 **ISO 13400**

Version history:
(1):

Purpose: This specification require that the DoIP standard is implemented according to ISO 13400.
Source: [ISO13400-2]
Verification Method: Analysis
Legacy ID: 54275v4

The ISO 13400 standard shall be implemented according to *[ISO13400-2]*

661003 v1 **Keep track of TCP connections**

Version history:
(1):

Purpose: To keep track of all available TCP connections
Source:
Verification Method: Test
Legacy ID: 54315v1

The Edge Node shall maintain a table of all available TCP connections.

661007 v4 **Manufacturer specific Further action required information**

Version history:
(4): Remove the WLAN flag and update the definition of FAR to adapt the usage of diagnostic channel.

Purpose: To inform the diagnostic client about the availability of the vehicle and if secure mode is required or not.
Source:

Verification Method: Test
Legacy ID: 441982v0

The value 0x11 to 0xFF of Further action required is available for additional OEM-specific use.

Specific values for 'Further action required' (FAR) in the Vehicle announcement/VehicleIdentificationResponse message shall be supported according to the definitions below.

FAR byte in Vehicle announcement/VehicleIdentificationResponse:

Bit	7	6	5	4	3	2	1	0
Mnem	OEM specific		AUTH	CHNL			BUSY	DoIPs

Mnem	Value	Description
OEM specific	0b10	
AUTH	0	Authentication UDS $29 not required to diagnostic handler
AUTH	1	Authentication UDS $29 requred to diagnostic handler
CHNL	0b000 ~ 0b111	0b000 = Channel is empty 0b001 = Vehicle is connected locally by wire (Ethernet P2P or LAN) 0b010 = Vehicle is connected locally by wireless (WLAN) 0b011 = Vehicle is diagnosed by OTA 0b100 = Vehicle is diagnosed by Diagnostic APP ……
BUSY	0	Vehicle is not in DoIP session phase
BUSY	1	Vehicle is in DoIP session phase
DoIPs	0	TLS not required to establish DoIP session
DoIPs	1	TLS required to establish DoIP session
Note: When BUSY = 0, the CHNL flag is not of importance and shall be set to 0. When BUSY = 1, the CHNL flag should not be 0.		

661001 v1 **Power-mode**

Version history:
(1):

Purpose: To ensure that the Diagnostic Power Mode Information Request returns "Ready" when a UDS connection is made with the vehicle
Source: [ISO13400-2] (DoIP-116, DoIP-117, DoIP-118)
Verification Method: Test
Legacy ID: 54297v3

The Diagnostic Power Mode Information Response shall be 0x01 – Ready.

841909 v1 **Routing activation request activation types for**

Version history:
(1): 法规标准要求的路由激活请求类型需要为0x01。

Purpose: Diagnostic communication required by regulation.
Source: E.g. ISO 27145, the ISO 20730 series, SAE J1979-3 2022, etc.
Verification Method: Analysis AND Test

Legacy ID:

Specific values for Routing activation request activation types shall be supported according to the regulation.

Value	Description	Required action	Support
0x01	Diagnostic communication required by regulation	None	MC

661006 v2 **Specific DoIP payload types**

Version history:
(2):

Purpose: To define details about the payload types.
Source: Functional Realization "External Diagnostic Client Connection" (M-204)
Verification Method: Analysis AND Test
Legacy ID: 241714v2

The range 0xF000 to 0xFFFF of the "Manufacturer specific" payload types values shall be reserved for manufacturer specific payload types.

[ISO13400-2] reserves “Payload type” values in the range 0xF000 to 0xFFFF for manufacturer-specific use.

The following tables shows the current specific DoIP payload types that shall be supported:

Payload type value	Payload type name	Support	Port and protocol
0xF002	PT_CHAT_TEXT_FROM_VEHICLE	Optional	TCP_DATA
0xF003	PT_CHAT_TEXT_TO_VEHICLE	Optional	TCP_DATA
0xF004	SET_TEMP_UTC_TIME_REQUEST	Optional	TCP_DATA
0xF005	SET_TEMP_UTC_TIME_RESPONSE	Optional	TCP_DATA
0xF006	CLIENT_AUTHENTICATION_REQUEST	Optional	TCP_DATA
0xF007	CLIENT_AUTHENTICATION _RESPONSE	Optional	TCP_DATA

Note; TCP_DATA is defined in [ISO13400-2].

The payload types shall support the following data parameters:
(Pos. = position of first byte in parameter Len. = length of parameter in bytes, if not defined otherwise)

Payload type PT_CHAT_TEXT_FROM_VEHICLE

Item	Pos.	Len.	Description	Values	Support
ChatText	0	256	Text string sent from the vehicle to the external client	UTF8	mandatory

Payload type PT_CHAT_TEXT_TO_VEHICLE

Item	Pos.	Len.	Description	Values	Support
ChatText	0	256	Text string sent from the external client to the vehicle	UTF8	mandatory

Payload type SET_TEMP_UTC_TIME_REQUEST

Item	Pos.	Len.	Description	Values	Support
UTC Time	0	24	Set value for UTC_TIME (UTC time value used in Client authentication for DoIP security). Format example: 2014-07-06T11:56:00.000Z	String yyyy-mm-ddThh:mm:ss.000Z	mandatory

Payload type SET_TEMP_UTC_TIME_RESPONSE

Item	Pos.	Len.	Description	Values	Support
Status	0	1	Response defining the result of the UTC time request	ENUM 0: Not accepted 1: Accepted	mandatory
UTC Time Ref	1	24	Current value of the local UTC time reference (UTC time reference based the CarTiGbl signal as the clock tick) Format example: 2014-07-06T11:56:00.000Z	String yyyy-mm-ddThh:mm:ss.000Z	mandatory

Payload type CLIENT_AUTHENTICATION_REQUEST

Item	Pos.	Len.	Description	Values	Support
SAML_Assertion	0	Variable Max 4k	SAML ticket authenticating the diagnostic client	UTF8	mandatory

Payload type CLIENT_AUTHENTICATION_RESPONSE

Item	Pos.	Len.	Description	Values	Support
Status	0	1	Response defining the result of the Client authentication	ENUM 0: Authenticated 1: Rejected – invalid time 2: Rejected – invalid IP 3: Rejected – invalid VIN 4: Rejected – invalid signature 5: Rejected - other	mandatory

661008 v2 **Specific DoIP routing activation request activation types**

Version history:
(2):

Purpose: To define a range of routing activation request codes for specific use.
Source: Functional Realization "External Diagnostic Client Connection" (M-204)
Verification Method: Test
Legacy ID: 241716v2

Specific values for Routing activation request activation types shall be supported according to the definitions below. Further requirements are also defined in *[Diagnostic Mgmt]*

Specific Routing activation request Activation types:

Value	Description	Required action	Support
0xE2	RAREQC_LOCAL	Await permission from the SyncDiagcCtrl to activate routing	optional
0xE3	RAREQC_REMOTE	None	optional

661009 v2 **Specific DoIP routing activation response codes**

Version history:
(2):

Purpose: To define a range of DoIP routing activation response codes for specific use.
Source: Functional Realization "External Diagnostic Client Connection" (M-204)
Verification Method: Test
Legacy ID: 241717v2

Specific Routing activation response codes shall be supported according to the definitions below.
Further requirements are also defined in *[Diagnostic Mgmt]*

Specific Routing activation response code values:

Value	Description	Required action	Support
0xE1	RARESPC_BUSY Another client (internal or external) is currently using the vehicle diagnostic system	Do not activate routing	optional
0xE2	RARESPC_WRONG_MODE The activation type does not match the current DoIP mode (Local; Remote)	Do not activate routing	optional

660995 v1 Vehicle Announcement Message Interval

Version history:
(1):

Purpose: To specify a new announcement interval. In order not to fill the network with announcements messages the value must be increased from the suggested value in [ISO13400-2]
Source: DoIP-050; [ISO13400-2]
Verification Method: Test
Legacy ID: 54294v2

The Vehicle Announcement Interval timing parameter (A_DoIP_Announce_Interval) shall be 1000ms.

660996 v1 Vehicle Announcement Message Number

Version history:
(1):

Purpose: To increase the probability that the tester(s) picks up the Vehicle Announcement Message.
Source: DoIP-050; [ISO13400-2]
Verification Method: Analysis AND Test
Legacy ID: 54295v2

The Vehicle Number Message parameter (A_DoIP_Announce_Num) shall be set to unlimited.

716529 v1 Vehicle Announcement Message/Identification Response Message UDP Port

Version history:
(1): 对车辆广播声明和车辆识别请求/响应的UDP端口号做了要求。此前无明确要求。

Purpose: In the WiFi diagnostics scenario, in order to be compatible with the software code implementation (e.g. NAT routing implementation) of Vehicle Announcement and Vehicle Identification Request&Response.
Source:
Verification Method: Test
Legacy ID:

Although the Vehicle Announcement Message Source Port is allowed to be UDP_DISCOVERY or dynamically assigned according to [ISO13400-2], it is required that the Vehicle Announcement Message Source Port shall be dynamically assigned. The dynamically assigned source port should be within the range 49152 to 65535.

Note1: the method and rule of dynamically changing source port is not constrained.
Note2: ECU shall receive and response the vehicle identification request with destination port UDP_DISCOVERY.

660998 v2 Vehicle Announcement When No VIN Available

Version history:
(2):

Purpose: To specify what content the Vehicle Announcement Message shall have in the VIN field when no VIN is available.
Source: DoIP-049 [ISO13400-2]
Verification Method: Test
Legacy ID: 68463v2

When no VIN is available the Vehicle Announcement Message shall set the VIN field to all 0xFF.

660997 v1 Vehicle Announcement when Connected

Version history:
(1):

Purpose: To avoid unnecessary broadcasts of the Vehicle Announcement Message
Source: [ISO13400-2] (DoIP-049, DoIP-050)
Verification Method: Test
Legacy ID: 54296v2

Vehicle Announcement Message shall be halted when a connection has been established with a tester (i.e when routing activation is ok) .

661005 v1 Vehicle Identification Request Message with EID UHWID

Version history:
(1):

Purpose: For identification of a vehicle without a VIN-number, the Vehicle Identification Request with EID is needed.
Source: [ISO13400-2]
Verification Method: Test
Legacy ID: 54290v2

The Vehicle Identification Request Message with EID / UWHID shall be supported where the

identifier shall be the external Ethernet MAC of the Edge Node.

4.8.1.5 DoIP - Client

•Client Requirements

4.8.1.5.1 Client Requirements

When a client initiate a DoIP session it must remain connected unless the session is completed. This is to prevent unexpected consequences and for higher performance. If the Edge Node disconnects, the client must reestablish a TCP connection once the Edge Node is online again which can be seen on e.g. vehicle announcements being broadcasted.

661010 v2 — Client Behaviour During DoIP Session

Version history:
(2): Verification Method: Anaylsis -> Analysis AND Test.

Purpose: To ensure that the client does not disconnect during a DoIP session.
Source:
Verification Method: Analysis AND Test
Legacy ID: 93683v2

The client ECU shall remain connected during the complete DoIP Session. If the Edge Node disconnects then the client shall try to reestablish the connection.

4.8.1.6 DoIP - InVehicle requirements

•IP-ECU Requirements

4.8.1.6.1 IP-ECU Requirements

661018 v1 — Close TCP port on connection reset

Version history:
(1):

Purpose: To ensure that the TCP port is closed when the Edge node is resetting
Source:
Verification Method: Test
Legacy ID: 442454v0

The IP ECU shall close the TCP port when the Edge node resets

661012 v1 — DoIP Ports for Internal Communication

Version history:
(1):

Purpose: To define which port to use when connecting to the Edge Node for vehicle DoIP communication.
Source: DoIP-001 [ISO 13400-2]
Verification Method: Test
Legacy ID: 84751v2

All in-vehicle DoIP communication shall use the target/destination TCP port 13400 when

connecting to and communicating with the Edge Node.

661014 v1 **DoIP Support in Internal ECUs**

Version history:
(1):

Purpose: To define which DoIP types that shall be supported for in vehicle DoIP communication.
Source: DoIP-064 [ISO 13400-2]
Verification Method: Test
Legacy ID: 84741v3

All internal ECUs in the vehicle network shall only support diagnostic message, payload type 0x8001, according to DoIP-064 *[ISO13400-2]*

661011 v1 **DoIP header**

Version history:
(1):

Purpose: To define the header used for in-vehicle DoIP communication
Source: DoIP-036, DoIP-064 [ISO13400-2]
Verification Method: Test
Legacy ID: 84740v3

The DoIP header shall follow DoIP-036 and DoIP-064 in *[ISO13400-2]* during its transportation through the internal vehicle network.

661015 v2 **Lost TCP connection for DoIP Messages**

Version history:
(2):

Purpose: Establish connection for DoIP messages when not available.
Source:
Verification Method: Test
Legacy ID: 84759v1

If the IP ECU detects that a DoIP connection is not available to the Edge Node then it shall establish a DoIP connection to the Edge Node within 500ms.

661016 v3 **Open TCP Connection for DoIP Messages**

Version history:
(3):

Purpose: Establish connection for DoIP messages at startup.
Source:
Verification Method: Test
Legacy ID: 84757v2

At startup, all TCP based target ECUs within the vehicle shall initiate and open TCP Connections for DoIP communication to/from the Edge Node. The TCP Connection shall remain until the Edge node sends a reset or shut down request.

If programming session is entering, IP ECU shall wakeup and maintain Ethernet to establish TCP Connection to Edge node.

661017 v2 **Reconnect With Same IP and Port Number**

Version history:
(2): 当连接尝试被拒绝，更新重连的时间标准，放宽至50ms内需要再次连接即可。同时对于源端口号为相同或随机都可以接受。

Purpose: To define the IP ECU behaviour when connection attempt is refused.
Source:
Verification Method: Test
Legacy ID: 84763v2

The IP ECU shall try to connect again within 50 ms using IP address and port number after a connection attempt has been refused. The source port number shall be the same or randomized as when the ECU started.

661013 v1 **Source Port for Internal Communication**

Version history:
(1):

Purpose: To define which source(local) port the in-vehicle IP ECUs shall use for DoIP communication.
Source:
Verification Method: Test
Legacy ID: 84752v3

The source port number shall be shall be randomized by the IP ECU.

•IP-ECU Fast SWDL General Requirement

4.8.1.6.2 IP-ECU Fast SWDL General Requirement

692068 v2 **Deactivation the specific DoIP TCP connection**

Version history:
(2): 1、更新描述；2、放宽性能；3、不需要过安全访问；4、APP和Boot下都支持。

Purpose: Define a method to deactivate the feature.
Source:
Verification Method: Test
Legacy ID:

A UDS RoutineControl service with startRoutine will be used to end the specific DoIP TCP connection, restore the original state.

After returning a positive response, IP-ECU shall disconnect this DoIP TCP Connection according to the wavehand

process, and restore DoIP TCP Connection defined in *ReqID 661016* "*Open TCP Connection for DoIP Messages*" within 5s.

If this diagnostic request is received when ECU is in the normal state which meets ReqID 661016 requirements , ECU shall give a positive response to Client and no action to execute.

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	0249		
Name	Deactivate a specific DoIP TCP Connection		
Description	Deactivate the specific DoIP TCP Connection for IP-ECU to restore the original DoIP TCP Connection.		
Type	1		
$P4_{Server_Max}$	200ms		
Security access	/		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	N
Request parameters	-		
Response parameters	**Start [0x01] RoutineStatusRecords**	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	
	Results	[1 byte, Hex]	
Available in application		Available in bootloader	
M (All sessions)		M	

692129 v1 **Gateway Routing Capability**

Version history:
(1):

Purpose: Gateway routing is defined above the TP layer, the differences in DoIP TCP Connection do not affect the reception and processing of diagnostic messages.
Source:
Verification Method: Test
Legacy ID:

The gateway routing capability of diagnostic messages from Ethernet to subnetwork or vice versa shall not change.

•IP-ECU Fast SWDL Requirement for WiFi Diagnostics

4.8.1.6.3 IP-ECU Fast SWDL Requirement for WiFi Diagnostics

Scheme: implement diagnostics through car mounted WiFi connection
This chapter describes the scenario of performing diagnostics through an in car WiFi connection.To improve the upgrade efficiency of IP-ECU in this scenarios, a fast SWDL method and the requirements for IP-ECU are described here. **This scheme is applicable to ECU based on file system, and whole VBFs are greater than 500MBytes.**

Different from “the IP-ECU Fast SWDL Requirements for OBD Connection”, the communication medium for vehicle diagnostics here is based on wireless network connection.

The vehicle gateway plays a role in isolating the internal and external networks, but it also limits the transmission rate of DoIP data from external diagnostic tools and IP-ECU, as it involves ECU address recognition and IP conversion routing, see figure "Routing based on TP layer for Vehicle Gateway". To improve data transmission speed, activate the In-Vehicle IP ECU to disconnect original connection with Edge Node and establish a new TCP connection with external diagnostic tool through wireless network adapter, see "Routing based on Switch for Vehicle Gateway during wireless network adapter". This chapter just describes the requirements of IP-ECU in Fast SWDL.

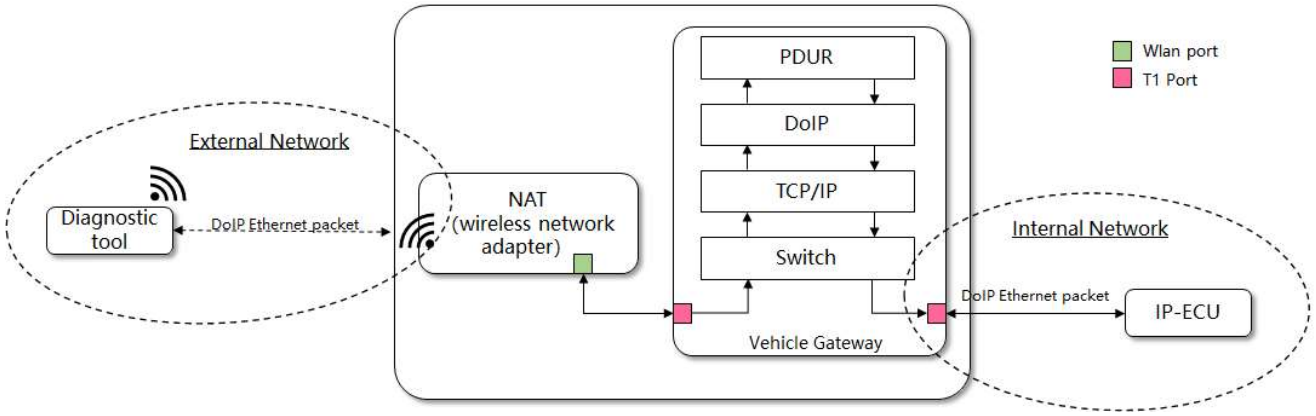


Figure: Routing based on TP layer for Vehicle Gateway during wireless network adapter

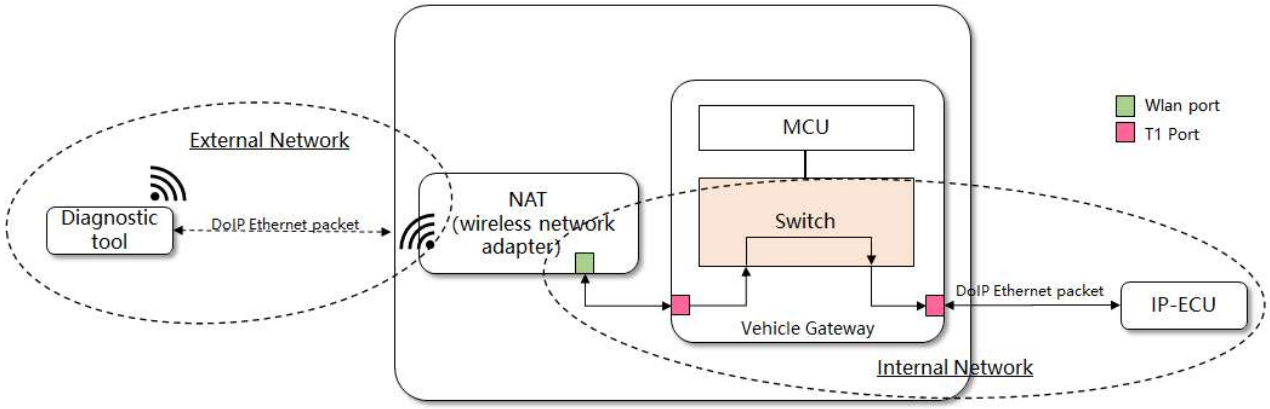


Figure: Routing based on Switch for Vehicle Gateway during wireless network adapter

717582 v1 **Create Wireless DoIP TCP Connection in specific scenario**

Version history:
(1):

Purpose: As a TCP client role, create TCP Connections for DoIP communication to/from the external tester through wireless network adapter.

Source:

Verification Method: Test

Legacy ID:

In-vehicle DoIP ECU shall be able to support creating DoIP TCP connection with the target/destination TCP port 13400 tagged specific “VLAN ID” to external tester when specific activation is received.

Note1: The specific “VLAN ID” aboved refers to Use Case(s) Service Name, S-643-01 WiFi Diagnostics(Wifi model

from/to DHU) defined in the requirement *ReqID 674216 "VLAN and Priority Allocation table"*.
Note2: When establishing a wireless DoIP TCP connection with an external diagnostic tester, NAT configuration is required and this part is not within the scope on the Basetech SWRS.

717580 v1 Disconnect DoIP TCP Connection with Edge Node

Version history:
(1):

Purpose: To ensure that only one DoIP connection is used for DoIP message between IP-ECU and client.
Source:
Verification Method: Test
Legacy ID:

IP-ECU will disconnect the DoIP TCP connection with Edge Node actively defined in *ReqID 661016* "*Open TCP Connection for DoIP Messages*", when specific activation is received.

Note: the disconnection of DoIP TCP shall comply with the standard process of TCP protocol, at least on its own side.

717577 v1 The definition of wireless TCP connection activation

Version history:
(1): Pre-condition: It shall be executed after enabling wifi diagnostics function.

Purpose: Define a method to implement a wireless DoIP TCP Connection with external tool through wireless network adapter.
Source:
Verification Method: Test
Legacy ID:

A UDS RoutineControl service with startRoutine will be as an activation, carrying the IP Information parameter with external tester IP address.

After returning a positive response, IP-ECU shall execute the actions defined in *ReqID 717580* and *ReqID 717582* in sequence. Firstly disconnect DoIP TCP Connection, secondly create a wireless DoIP TCP Connection within 10s.

If this diagnostic command is received when ECU is in the current new state, ECU shall give a positive response to Client and no action to execute.

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	091A
Name	Create a wireless DoIP connection to external tester
Description	Activate IP-ECU as TCP client to create a wireless DoIP connection to external tester through wireless network adapter.
Type	1
$P4_{Server_Max}$	200ms
Security access	/

Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	N
Request parameters	**Start [0x01] RoutineStatusRecords** External Tester IPv4 Example: If IPv4 is 192.168.2.200, the raw value will be 0xC0A802C8.	**Length, Data Type** [4 byte, Hex]	
Response parameters	**Start [0x01] RoutineStatusRecords** Routine Type Routine Status Results	**Length, Data Type** [4 bits, Unsigned] [4 bits, Unsigned] [1 byte, Hex]	

Available in application	Available in bootloader
M (All sessions)	M

717598 v1 **The handle of wireless DoIP TCP Connection**

Version history:
(1):

Purpose: Describe how the DoIP TCP connection is handled.
Source:
Verification Method: Test
Legacy ID:

When IP-ECU is in wireless TCP state, if the wireless DoIP TCP connection opened by the IP-ECU is not successfully estblished within 5s, it is considered a timeout, and the DoIP connection to the edge node is actively restored by IP-ECU within 1s.

Note1: If the IP ECU detects that a wireless DoIP connection is not available to the external tester then it shall establish a wireless DoIP connection again within 500ms until timeout or connection is established.
Note2: The IP ECU shall try to connect again within 50 ms using IP address and port number after a connection attempt has been refused. The source port number shall be the same or randomized when the ECU started.

Before resetting, IP-ECU shall store the current DoIP TCP connection to ensure the next connection at start up. For example, when a wireless DoIP connection has existed, restarting due to entering programming session or ECU hardreset, IP-ECU shall attemp to establish a wireless DoIP connection after initialization.

Note: The requirements defined in "Internetworks General Specification" are still applicable.

4.8.1.7 DoIP - Edge Node - Transport Layer Secutiry

•Diagnostic over Network

4.8.1.7.1 Diagnostic over Network

652167 v1 **Ethernet point-to-point connection**

Version history:
(1):

Purpose: Clarify Ethernet point-to-point connection.
Source:
Verification Method: Analysis

Legacy ID:

An Ethernet connection where the IP address is assigned as a fixed address or by AutoIP address assignment is defined as Ethernet point-to-point connection (also called Ethernet peer-to-peer connection).

652170 v1 **Safety TCP connection on trusted network**

Version history:
(1):

Purpose: Although in a trusted network, the probability of DoIP communication being attacked is small. The security of transparent data needs to be protected , e.g. using DoIP TLS.
Source:
Verification Method: Analysis
Legacy ID:

The doip diagnostic data interaction in an trusted network (e.g. Ethernet point-to-point) is expected to be on a secure TCP connection if necessary.

652169 v1 **Safety TCP connection on untrusted network**

Version history:
(1):

Purpose: In order to protect the doip communication, the security policy of the transport layer is necessary, e.g. DoIP-TLS.
Source:
Verification Method: Analysis
Legacy ID:

The doip diagnostic data interaction in an untrusted network (e.g. WLAN and LAN) shall be on a secure TCP connection. The secure diagnostic communication shall implement the TLS protocol for the transport channel between client and a server.

652157 v1 **Trusted network**

Version history:
(1):

Purpose: Clarify how it is a trusted network, in which the interaction of diagnostic communication can be considered safe.
Source:
Verification Method: Analysis
Legacy ID:

The trusted network is only allowed to be point-to-point wired direct connection defined in ReqID 652167, that is, the client is directly connected to the server through Ethernet cable.

652168 v1 **Untrusted network**

Version history:
(1):

Purpose: Clarify how it is a untrusted network, in which the interaction of diagnostic communication can be considered to be safe.
Source:

Verification Method: Analysis

Legacy ID:

Any router or network switch between the client and the server, such as the diagnostic equipment connected to the vehicle edge node through the router, is considered as an untrusted network.

•TLS secure diagnostic communication for DoIP

4.8.1.7.2 TLS secure diagnostic communication for DoIP

TLS allows to establish an authenticated (ensures authenticity and integrity) and encrypted (confidentiality protection) communication channel between the DoIP client and the DoIP server.

The secure diagnostic communication implements the TLS protocol for the transport channel between DoIP client and a DoIP server.

Before the DoIP client and DoIP server begin exchanging DoIP message over TLS, first the secured TCP connection is negotiated within the TLS handshake: the DoIP client and the DoIP server agree on the version of the TLS protocol, transfer certificate, choose the common cipher suite, verify certificates, and complete the TLS session key exchange.

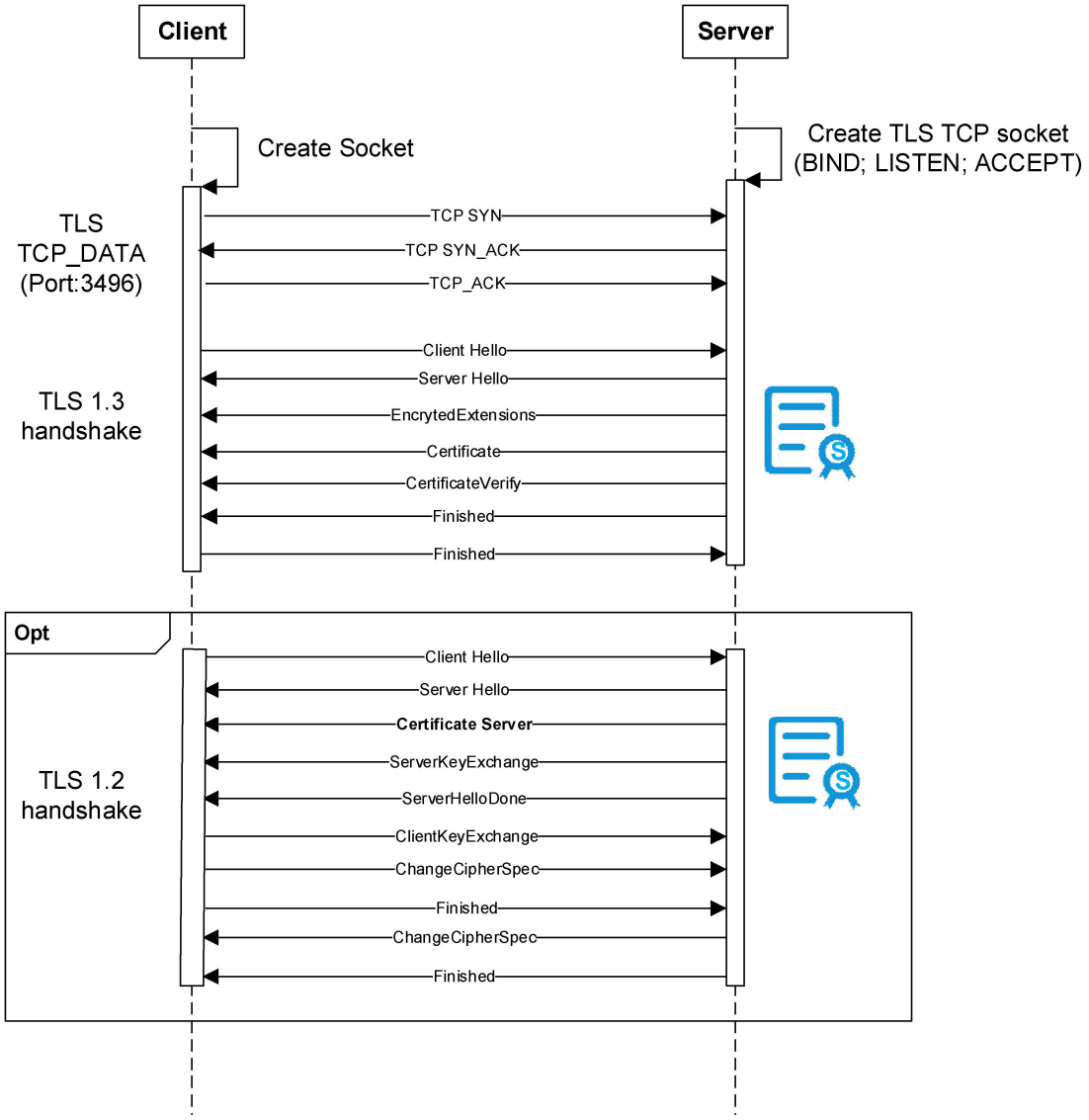


Figure: A simplified TLS handshake preocedure (server authentication type)

652192 v1 **DoIP TLS TCP port**

Version history:
(1):

Purpose: Define the TCP port used for DoIP TLS.
Source:
Verification Method: Test
Legacy ID:

In case of secured diagnostic communication, only the TCP connection over the dedicated port 3496 for TLS shall be used. For Vehicle discovery and UDP protocol based DoIP communication, no secured communication and no dedicated ports are applied.

Note: If the TLS TCP port is not used to establish a TCP connection, the specified return code defined in [ISO13400-2] shall be returned when execute routing activation process.

535480 v1 **TLS authentication type**

Version history:
(1):

Purpose: Depending on the TLS authentication type, the DoIP client and DoIP server may require different serurity information.But it needs to be considered to save calculation and resources on the server side.
Source:
Verification Method: Test
Legacy ID:

Edge Node as a server shall support the server authentication type, that is, only the client needs to verify the authenticity of the server.

Note: additional supporting client authentication is optional, which need to approved by GRI Basetech Team.

535473 v1 **TLS handshake process**

Version history:
(1):

Purpose: Clarify general requirements.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Before the client DoIP entity and the server DoIP entity begin exchanging DoIP messages over TLS, first the secured TCP connection is negotiated within the TLS handshake: the DoIP client and the DoIP server agree on the version of the TLS protocol, choose the common cipher suite, transfer certificate, verify certificates, and complete the TLS session key exchange.

855011 v1 **Usage of DoIP TLS**

Version history:
(1): 保证边缘节点TLS解密成功，至少分配16kB的HSM Buffer。将其作为需求约束。

Purpose: Ensure that decryption of TLS does not fail due to buffer size being too small.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The DoIP TLS communication will be used in Diagnostic and SWDL. The hardware security module shall be allocated enough buffer at least 16kBytes to decrpyt TLS message.

Note: The hardware security module should support the capability of updates.

4.8.1.7.2.1 TLS for DoIP client side

652229 v2 **Client TLS cipher suites**

Version history:
(2):

Purpose: The client should have stronger capabilities to adapt to the requirements of the peer.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Depending on the TLS protocol version, the client shall support multiple cipher suites under the TLS version.

652213 v2 **Client TLS process**

Version history:
(2):

Purpose: Define the Client implementation.
Source:
Verification Method: Test
Legacy ID:

ECU or tools as doip TLS client shall initial TLS handshake.

652225 v2 **Client TLS version**

Version history:
(2):

Purpose: The client should have stronger capabilities to adapt to the requirements of the peer.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The client (e.g. external device) should support multiple TLS versions, such as TLS1.2 and TLS1.3, to adapt to the server's capabilities.

652247 v2 **Requirements to DoIP client**

Version history:
(2):

Purpose: Define the ability and action of Client.
Source:
Verification Method: Analysis AND Test
Legacy ID:

In the TLS authentication process, the Doip client shall have a complete CA certificate chain issued by the PKI system and be able to verify the authenticity of the DoIP server.

4.8.1.7.2.2 TLS for DoIP server side

652246 v4 Requirements to DoIP server

Version history:
(4): 更新需求，TLS交互时需要发送中间证书。

Purpose: Define the ability and action of Server.
Source:
Verification Method: Analysis AND Test
Legacy ID:

In the TLS handshake the DoIP server shall use the ECUs server X.509v3 certificate and the associated private key in the authentication process, and send the DoIP server certificate.
Note: The root certificate shall not be transmitted.

652231 v2 Server TLS cipher suites

Version history:
(2):

Purpose: Define the requirement of TLS cipher suites for DoIP server.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Depending on the TLS protocol version, the server should support at least one cipher suite under the TLS version.

652215 v2 Server TLS process

Version history:
(2):

Purpose: Define the Server implementation.
Source:
Verification Method: Test
Legacy ID:

ECU as doip TLS server shall perform TLS handshake.

652228 v2 Server TLS version

Version history:
(2):

Purpose: TLS version 1.1 is obsolete, expect to use higher version than TLS 1.1.
Source:
Verification Method: Analysis AND Test
Legacy ID:

ECU supporting DoIP Protocol shall support TLS 1.2 and above, and at least one version.

•TLS DoIP configuration profile

4.8.1.7.3 TLS DoIP configuration profile

The TLS configuration profile defines the list of the accepted TLS versions and the prioritized list of the supported TLS cipher suites for server and the client.

4.8.1.7.3.1 Supported TLS versions for DoIP

TLS protocol versions include TLS 1.0 (disabled), TLS 1.1, TLS 1.2, and TLS 1.3. In order to ensure the security and reliability of the connection, the protocol of TLS version 1.2 or above shall be used.

Table: TLS versions for IP

TLS version	RFC	Supported	Notes
TLS 1.2	IETF RFC 5246	O	Note 1
TLS 1.3	IETF RFC 8446	MC	Note 1

Note 1: At least TLS version 1.2 or version 1.3 shall be supported. The TLS version 1.3 is recommended and the use of TLS 1.2 needs to be evaluated by the Base Technology team and Cyber Security team.

535492 v1 **TLS version selection**

Version history:
(1):

Purpose: Clarify the usage of different versions.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The latest TLS 1.3 provides a higher security level, and the Doip ECU shall be recommended to support the TLS version 1.3 protocol.
If use TLS 1.2, it needs to be evaluated by the Base Technology team and Cyber Security team.

4.8.1.7.3.2 Supported TLS cipher suites

This chapter provides a brief overview of the TLS cipher suites that can be supported.

Table: TLS 1.2 cipher suites

Cipher suite	RFC	Supported	Notes
TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256	IETF RFC 5289	Mandatory	Note 1
TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384	IETF RFC 5289	Optional	
TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256	IETF RFC 7905	Optional	
TLS_ECDHE_ECDSA_WITH_AES_128_CCM	IETF RFC 7251	Optional	
TLS_ECDHE_ECDSA_WITH_AES_128_CCM_8	IETF RFC 7251	Optional	
TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256	IETF RFC 4492	Optional	
TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA384	IETF RFC 5289	Optional	
TLS_ECDHE_ECDSA_WITH_NULL_SHA	IETF RFC 4492	NA	

Table: TLS 1.3 cipher suites

Cipher suite	RFC	Supported	Notes
TLS_AES_128_GCM_SHA256	IETF RFC 8446	Mandatory	Note 1
TLS_AES_256_GCM_SHA384	IETF RFC 8446	Optional	
TLS_CHACHA20_POLY1305_SHA256	IETF RFC 8446	Optional	
TLS_AES_128_CCM_SHA256	IETF RFC 8446	Optional	
TLS_AES_128_CCM_8_SHA256	IETF RFC 8446	Optional	

Note1: the server needs to support this one at least.

535493 v1 **Cipher suites selection for TLS 1.2**

Version history:
(1):

Purpose: Define the cipher suite that need to support.
Source:
Verification Method: Test
Legacy ID:

When the Doip ECU uses TLS 1.2 protocol, it shall support *TLS_ ECDHE_ ECDSA_ WITH_ AES_ 128_ GCM_ Sha256* cipher Suite.

652293 v1 **Cipher suites selection for TLS 1.3**

Version history:
(1):

Purpose: Define the cipher suite that need to support.
Source:
Verification Method: Test
Legacy ID:

When the Doip ECU uses TLS 1.3 protocol, it shall support *TLS_AES_128_GCM_SHA256* cipher Suite.

4.8.1.7.3.3 Supported TLS extensions

This chapter provides a brief overview of the TLS extension that can be supported.

Table: TLS 1.2 extension

Cipher suite	RFC	Supported	Notes
Record_size_limit	IETF RFC 8449	M	Note 1
Encrypt_then_MAC:	IETF RFC 7366	O	
server_name	IETF RFC 6066	O	
Client certificate URL	IETF RFC 6066	NA	
Trusted_ca_keys	IETF RFC 6066	NA	
truncated_hmac	IETF RFC 6066	NA	
Renegotiation indication	IETF RFC 5746	NA	
Session hash and extended master secret extension	IETF RFC 7627	NA	
ClientHello padding extension	IETF RFC 7685	NA	
Data compression	IETF RFC 5246	NA	
Signature algorithms	IETF RFC 5246	NA	

Table: TLS 1.3 extension

Cipher suite	RFC	Supported	Notes
Record_size_limit	IETF RFC 8446	M	Note 1
Supported_versions	IETF RFC 8446	M	
Server_name_identification	IETF RFC 8446	O	
0-RTT	IETF RFC 8446	NA	
Cookies	IETF RFC 8446	NA	

Note1: the server needs to support this one at least.

652304 v1 **Extension for TLS 1.2**

Version history:
(1):

Purpose: Define the extension that need to support.
Source:
Verification Method: Test
Legacy ID:

When the Doip ECU uses TLS 1.2 protocol, it shall support *Record_size_limit* TLS extension.

652306 v1 **Extension for TLS 1.3**

Version history:
(1):

Purpose: Define the extension that need to support.
Source:
Verification Method: Test
Legacy ID:

When the Doip ECU uses TLS 1.3 protocol, it shall support two TLS extensions, *Record_size_limit* and *Support_versions*.

4.8.1.7.3.4 Compression Method

675091 v1 **TLS compression for DoIP**

Version history:
(1):

Purpose: ClientHello message can contain a list of compression algorithms, but the compression method needs to be disabled.
Source:
Verification Method: Test
Legacy ID:

The support in TLS for data compression shall not be used (CompressionMethod.null) for DoIP.

•Additional Requirements

4.8.1.7.4 Additional Requirements

652173 v1 **Default safety communication configuration**

Version history:
(1):

Purpose: In order to facilitate the use of manufacturer and compatibility with equipment.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The default state is not to activate the Doip secure communication mechanism, meaning that the Doip TLS is disabled. The security state can change to enabled stated through the diagnostic service.

652176 v1 **Initial safety communication for DoIP**

Version history:
(1):

Purpose: Define when DoIP TLS protection will take effect.
Source:
Verification Method: Test
Legacy ID:

After configuring DoIP TLS through data record, DoIP TLS shall take effect in next connection (e.g. a restarted DoIP session).

652424 v1 **TLS handshake establishment**

Version history:
(1):

Purpose: Complete the transport protection rapidly.
Source:
Verification Method: Test
Legacy ID:

From the time a client initiates a TLS handshake until the handshake is completed shall take maximum 5 seconds.

Note: This requirement has the precondition that TCP connection between both ends has established.

4.9 Diagnostics Gateway

4.9.1 Diagnostic and Bootloader Gateway Specification

4.9.1.1 Gateway Generic

•Overview

4.9.1.1.1 Overview

This document describes and defines requirements for in-vehicle gateways for diagnostic communication for next

generation of GRI with the electrical architecture for the SPA platform.
The interface for an external test tool to the vehicle for UDS diagnostic will be IP based using the DoIP standard in *[GW_26] ISO 13400-2*.
This document does not specify any formal requirements for the physical communication link to the vehicle, it could e.g. be DoIP over Ethernet or DoIP over WLAN. The scope of the document is when a DoIP message is received in the vehicle, independent on used communication link. Communication link to the vehicle is covered by other specification as e.g. *[GW_12] DoIP*, *[GW_13] IP Routing Specification* and *[GW_14] Internetworks General Specification*].

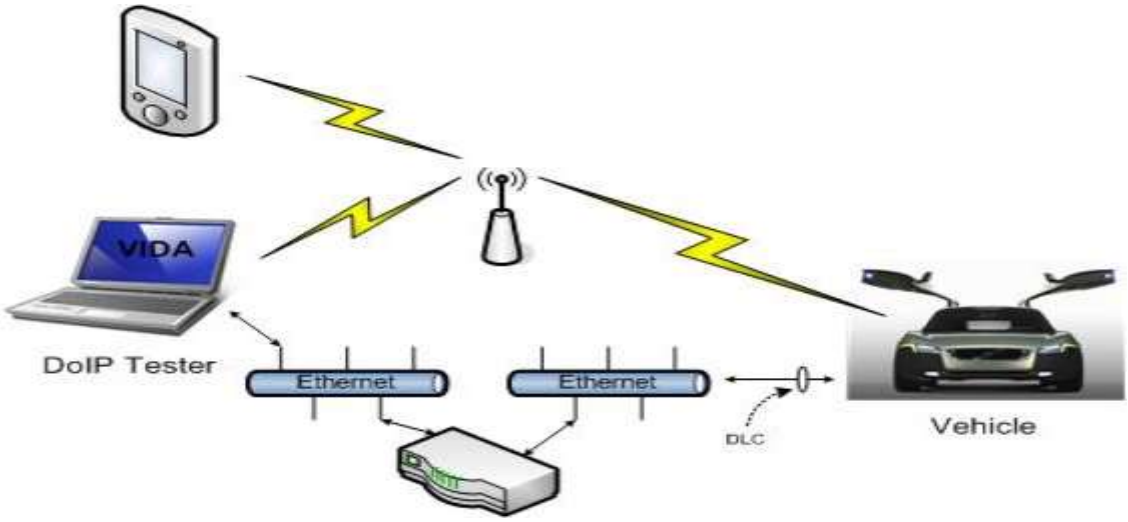


Figure: Overview external UDS tester connected to vehicle

Note: A legislated OBD tester for CAN, e.g. scan tool, could be directly connected to CAN and is not included in the UDS view in Figure " *Overview external UDS tester connected to vehicle"*. A J2534 test tool for CAN could also be connected to the vehicle through CAN (due to other legislated requirements), this traffic will pass the in-vehicle gateways. OBD tester should use 11 bit CAN ID and a J2534 tester should use 29 bit CAN ID. The vehicle GW will have a filter to receive for 29 bit CAN IDs for request from J2534, the 11 bit CAN IDs from a OBD test tool will not pass the vehicle GW. See section ' *J2534 standard tester connected to Vehicle GW through CAN'* for J2534 requirements.

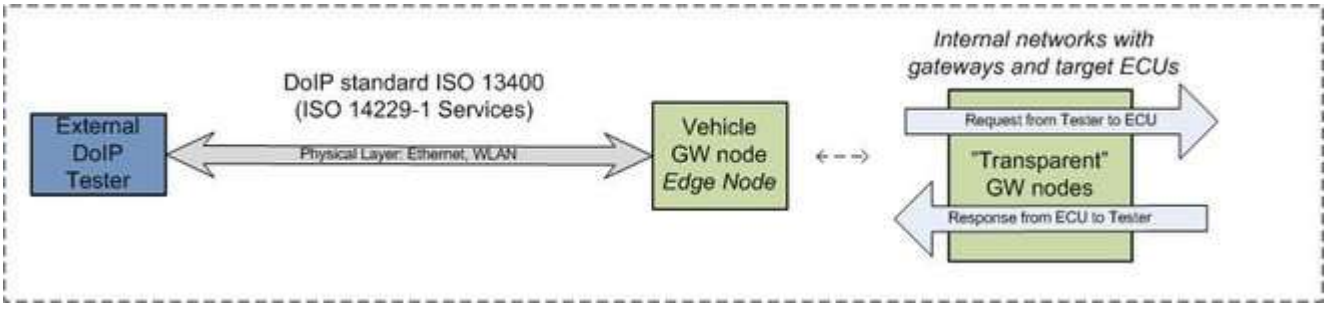


Figure: External tester connected to vehicle with DoIP ISO 13400 protocol

4.9.1.1.1.1 Definitions and Abbreviations

Definition	Description
Backbone Network	Internal network to connect zones on a high level
Client	Software entity which uses services of a server
Client-Server Communication	A specific form of communication in a possibly distributed system in which software entities act as clients, servers or both, where 1...n clients are requesting services via a specific protocol from typically one server.
Communication channel	A logical connection for sending signals over a multiplexed medium, as e.g.

	FlexRay, CAN Network.
Diagnostic communication	Cover both ' Diagnostic' and 'Software Download' communication
Zone ID	Defines which zone that shall be addressed.
Zone Network	Internal network for connection of nodes within a zone
Zone Network GW	Gateway within a vehicle between Backbone Network (or other internal Network as e.g. internal IP Network) and Zone Network
ECU ID	Defines which ECU that shall be addressed.
Edge Node	An ECU implementing both DoIP edge node functionality and DoIP gateway functionality. The ECU implements DoIP and is connected to external tester via IP and to the internal backbone network and IP network within the car.
External Network	Network for connection of an external tester. Used for software download and diagnostics
Final response	A final response is a positive response or a negative response other than response code 0x78 "requestCorrectlyReceived-ResponsePending". In case of a request to schedule periodic responses, the initial positive or negative response that indicates the acceptance or non-acceptance of the request shall be considered the final response.
Gateway	An ECU that is connected to two or more independent communication networks that allows information to flow between the networks.
gatewaying-on-the-fly	Gateway capability; routing between two TP modules where forwarding of data is started (when a specified threshold is reached) before all data have been received. If larger amount of data is transported between two interfaces it is desirable to be able to start the transmission on the destination network before receiving all data from the source network. This saves memory and time.
Internal Network	Network for communication within the vehicle
In-vehicle GW	Generic gateway within a vehicle, could be of type ' Vehicle GW', ' Zone Network GW', 'InVehicle DoIP GW' or ' Sub Network GW'
Network ID	Defines which network that shall be addressed.
Program mode	Program mode means that the ECU executing the primary or secondary Bootloader.
Server	Software entity which provides services for clients
Software Download	Services in *[GW_1] UDS Services* implemented in bootloader mode, e.g. used by a tester to download new software to ECU.
Sub Network	Internal network for low level communication within a domain
Sub Network GW	Gateway within a vehicle between Domain Network and Sub Network
Target	Destination of a diagnostic message
Tester	A system that controls functions such as test, inspection, monitoring, or diagnosis of an on-vehicle electronic control unit and may be dedicated to a specific type of operators (e.g. a scan tool dedicated to garage mechanics or a test tool dedicated to assembly plant agents). The tester is also referenced as the client.
TesterPresent	Service as specified in *[GW_19] ISO 14229-1*]. This service is used to keep one or multiple servers in a diagnostic session other than the defaultSession.
Routing Table	A routing table list routes to particular network destinations and contains information about the topology of the network immediately around it, e.g. which Domain Network IDs that are connected to a port.
Vehicle GW	Gateway within a vehicle between External Network and Backbone Network (or other Internal Network as e.g. internal IP Network)

Abbr.	Stands for	Description (where appropriate)
AUTOSAR	AUTOmotive Open System Architecture	The objective of the partnership is the establishment of an open standard for automotive E/E architecture. It will serve as a basic infrastructure for the management of functions within both future applications and standard software modules. [ref www.autosar.org]

CC	Communication channel	See definition in table ' *Definitions'*
CTS	Continue To Send	Flow control message, ok to send complete or part of message, dependent on parameters in CTS
DIAG	Diagnostic	See definition in *table ' Definitions'*
DLC	Diagnostic Link Connector	Connector of a external network to the vehicle
DoIP	Diagnostic over IP	According to ISO 13400
Eth/ETH	Ethernet	External and in vehicle network
Fr/FR	FlexRay	In vehicle network
GW	Gateway	See definition in table ' *Definitions'*
I-PDU	Interaction Layer PDU	Interaction Layer PDU. An I-PDU consists of data (buffer), length and I-PDU ID.
I-PDU ID	I-PDU Identifier	
LAN	Local Area Network	
L-SDU	Data Link Layer SDU	SDU of the data link layer
L-PDU	Data Link Layer PDU	PDU of the data link layer .The L-PDU is bus specific, e.g. CAN frame.
MHP	MOST High Protocol	Alternative protocol to TCP/IP on MOST. MHP should not be used for diagnostic messages, where TCP/IP and 'Ethernet over MOST' should be used.
MOST	Media Oriented Systems Transport	MOST is a multimedia fibre-optic network optimized for automotive applications which is used in the infotainment domain. [ref http://www.mostnet.de]
MTU	Maximum transmission unit	
N-PDU	Network Layer PDU	Used by transport protocol modules to fragment an I-PDU
OBD , OBDII	On-Board Diagnostics	According to ISO 15031, ISO 15765-4
PCI	Protocol Control Information	This Information is needed to pass a SDU from one instance of a specific protocol layer to another instance. E.g. it contains source and target information.
PDU	Protocol Data Unit	In layered systems, it refers to a data unit that is specified in the protocol of a given layer. The PDU contains user data of that layer (SDU) and possible protocol control information (PCI). The PDU of layer X is the SDU of its lower layer X-1 (i.e. (X)-PDU = (X-1)-SDU).
SA	Source Address	Address of the sender of a message
SDU	Service Data Unit	In layered systems, this refers to a set of data that is sent by a user of the services of a given layer, and is transmitted to a peer service user, whilst remaining semantically unchanged It is the data passed by an upper layer, with the request to transmit the data. It is as well the data, which is extracted after reception by the lower layer and passed to the upper layer.
SPRMIB	SuppressPositiveResponseMessageIndicationBit	As defined in *[GW_19] ISO 14229-1* If set to TRUE, a positive response should be suppressed.
TA	Target Address	Address of the receiver of a message
USDT	Unacknowledged Segmented Data Transfer	According to ISO 14229
UUDT	Unacknowledged Unsegmented Data Transfer	According to ISO 14229
WLAN	Wireless LAN	

4.9.1.1.1.2 Requisite documents

4.9.1.1.1.2.1 Documents

Internal references Documents

Reference number	Source	Title
[GW_1]	Internal	UDS Services
[GW_2]	Internal	UDS Data
[GW_3]	Internal	UDS Session
[GW_4]	Internal	UDSonCAN
[GW_5]	Internal	Bootloader - UDSonCAN
[GW_6]	Internal	DoCAN
[GW_7]	Internal	UDSonFR
[GW_8]	Internal	Bootloader - UDSonFR
[GW_9]	Internal	CoFlexRay
[GW_10]	Internal	UDSonIP
[GW_11]	Internal	Bootloader - UDSonIP
[GW_12]	Internal	DoIP
[GW_13]	Internal	IP Routing Specification
[GW_14]	Internal	Internetworks General Specification
[GW_15]	Internal	General Software Download Specification
[GW_16]	Internal	FlexRay Datalink Layer
[GW_17]	Internal	UDSonLIN
[GW_18]	Internal	Bootloader - UDSonLIN

External publications

Reference number	Source	Title	Version or Date	Document number / Short Name
[GW_19]	ISO	Road vehicles – Unified diagnostic services Part 1: Specification and requirements	See [1]* and [2]*	ISO 14229-1
[GW_20]	ISO	Road vehicles – Unified diagnostic services Part 2: Session layer services	See [3]*	ISO 14229-2
[GW_21]	ISO	Road vehicles – Unified diagnostic services Part 3: UDS on CAN implementation	See [4]*	ISO 14229-3
[GW_22]	ISO	Road vehicles - Unified diagnostic services Part 4: UDS on FlexRay implementation	See [7]*	ISO 14229-4
[GW_23]	ISO	Road vehicles – Unified diagnostic services Part 5: UDS on internet Protocol implementation	See [10]*	ISO 14229-5
[GW_24]	ISO	Road vehicles – Diagnostic Communication over CAN (DoCAN)	See [6]*	ISO 15765-2
[GW_25]	ISO	Road vehicles - Communication over FlexRay (CoFR)	See [9]*	ISO 10681-2
[GW_26]	ISO	Road vehicles – Diagnostic Communication over Internet Protocol (DoIP)	See [12]*	ISO 13400-2
[GW_27]	SAE	Recommended Practice for Pass-Thru Vehicle Programming	2004-12-01	SAE J2534-1

*: See *Diagnostic and Bootloader Gateway Specification - Requisite documents*.
Information obtaining ISO documents may be found on the internet: http://www.iso.org/ and standards.sae.org

For deploying gateway functionality on an AUTOSAR platform (i.e. in application and DIAG mode), following specifications are also relevant:

Reference number	Source	Title	Version or Date	Document number
	AUTOSAR	Specification of PDU Router	R4.0 Rev2 or later	AR Doc ID No: 035
	AUTOSAR	Specification of CAN Transport Layer	R4.0 Rev2 or later	AR Doc ID No: 014
	AUTOSAR	Specification of FlexRay Transport Layer	R4.0 Rev2 or later	AR Doc ID No: 029
	AUTOSAR	Specification of LIN Interface	R4.0 Rev2 or later	AR Doc ID No: 073
	AUTOSAR	Specification of Socket Adaptor	R4.0 Rev2 or later	AR Doc ID No: 416
	AUTOSAR	Specification of Ethernet Interface	R4.0 Rev2 or later	AR Doc ID No: 417

Information obtaining AUTOSAR documents may be found on the internet: http://www.autosar.org/

4.9.1.1.1.2.2 Relation to other documents

This document specifies OEM specific requirements on in-vehicle gateways for diagnostic communication.

The document shall serve as the cross car line compliance reference for implementation of gateway functionality in all GRI vehicles. Adherence to the specifications contained within this document is required to ensure compatibility within the corporate product line from vehicle to vehicle, model year to model year and maintain compatibility with the industry driven standards.

This document alone does not cover all requirements for the in-vehicle gateway functionality. The document shall be seen as a guideline and companion to other listed specifications, OEM specification and ISO specifications, which all together defines the in-vehicle gateway behavior.

Figure *Gateway specification relation to ISO Specifications* and *other OEM Specifications* shows the specifications that are related to this document.

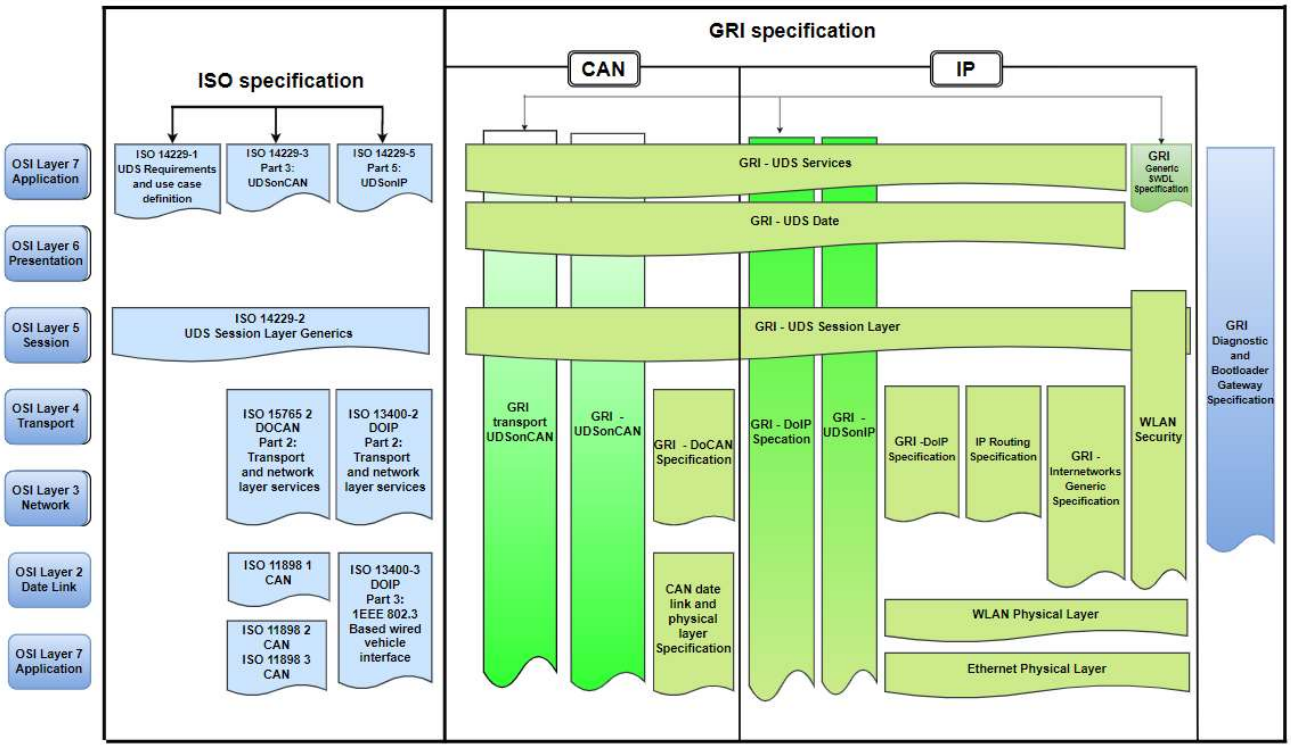


Figure: Gateway specification relation to ISO Specifications and other OEM Specifications

•Constraints and Assumptions

4.9.1.1.2 Constraints and Assumptions

This document assumes that infotainment network will use Ethernet or MOST150 network, and diagnostic communication should be IP based. I.e. if MOST150 is used, diagnostic communication shall be IP based and sent on the 'Ethernet over MOST150' channel. Diagnostic communication over MOST25 is not covered in this version. If MOST25 should be used it would require e.g. diagnostic communication over MHP or similar protocol with flow control mechanism to be able to fulfill all requirements specified in this document.

•Gateway description for DIAG and SWDL

4.9.1.1.3 Gateway description for DIAG and SWDL

This document describes and defines requirements for Diagnostic (DIAG) and Software download (SWDL) Gateway functionality in the vehicle.

An example topology for diagnostic communication, UDS view, according to Figure " *Example topology with routing tables, UDS view"* is used as a base for other examples in this document.

Note: The addresses used in to Figure " *Example topology with routing tables, UDS view"* is only example, allocation of addresses, e.g. CAN ID and IP addresses etc. will done in a project specific document for the project specific topology.

Note: Legislated test tool for OBD diagnostic, e.g. scan tool, can be connected directly to CAN for the emission related ECU. This communication will not go trough gateways and are not included in the UDS view in Figure " *Example topology with routing tables, UDS view"*.

[GW_1] UDS Services specifies that an external tester shall have logical address 0x0E80 (in address range specified by *[GW_26] ISO 13400-2*).

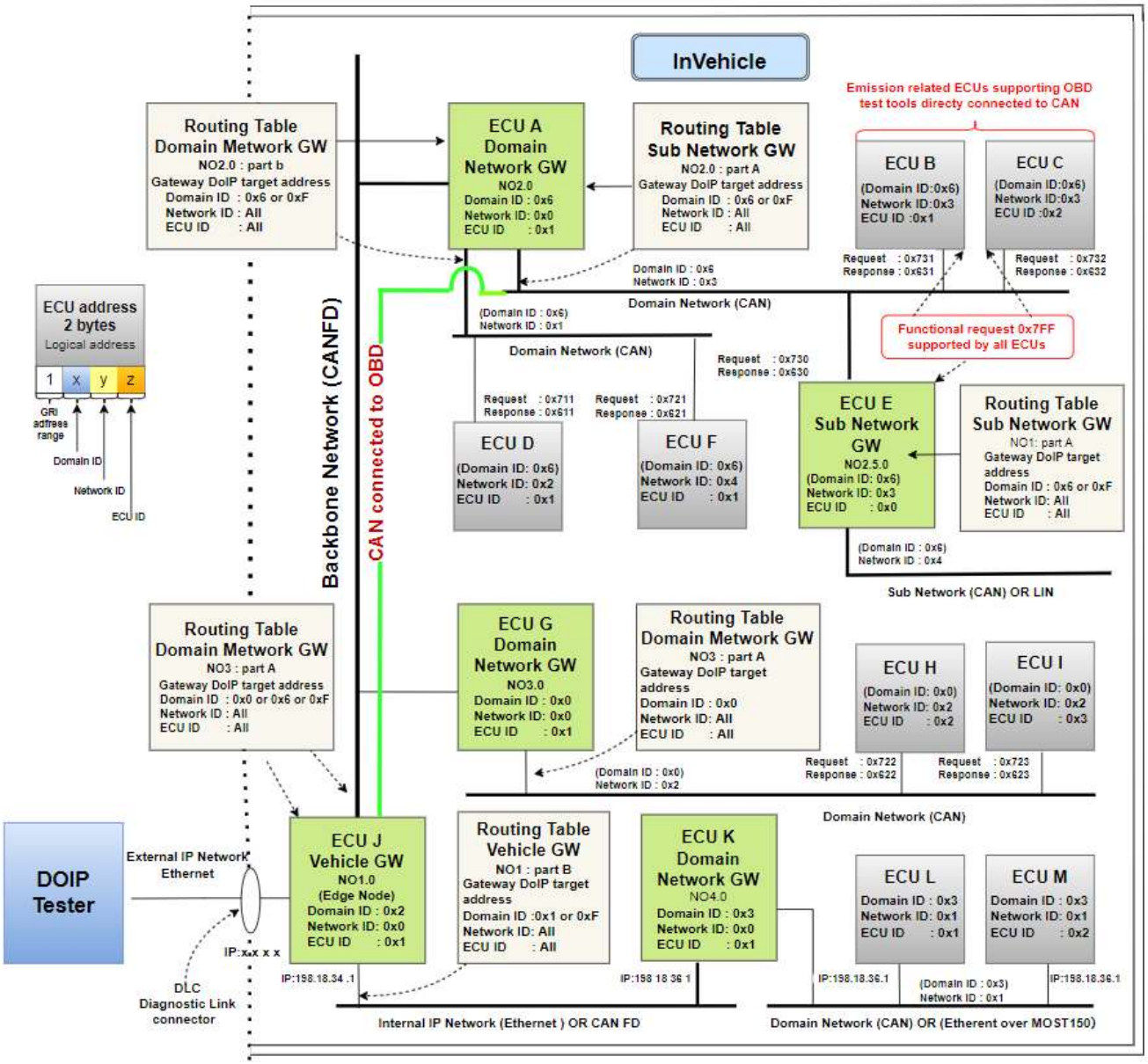


Figure: Example topology with routing tables, UDS view

Note: In the example in Figure " *Example topology with routing tables, UDS view"* ECU B and ECU H have same CAN ID for request and response. On 11 bit CAN normal addressing format the Domain ID information is lost.

Both DIAG and SWDL use services specified in *[GW_19] ISO 14229-1*, however there are differences in which services in ISO 14229-1 that are used for DIAG and SWDL and how much data that are sent, see *[GW_1] UDS Services* and *[GW_2] UDS Data*.

DIAG is executed in the application software, “application mode”, hence resources as memory, CPU and networks capacity are limited and must be shared with other applications.

SWDL is usually executed in the bootloader software, “program mode”, where no other software is executed, hence SWDL has more memory, CPU and network capacity available than DIAG.
In GEEA3.0 Electrical and Electronic Architecture, SWDL is also executed in the Gateway application software, hence certain resourcs (e.g. memory, CPU) need to be reserved.

For DIAG there are usually small requests from tester to vehicle, and greater responses from vehicle to tester. For SWDL the situation is normally reversed, large request from tester to vehicle, and small responses from vehicle to

tester.

Due to these differences between SWDL and DIAG, requirements for e.g. allocation of number of buffers and buffers sizes will differ between SWDL and DIAG.

The goal for this specification is that, where possible, have same requirements for DIAG and SWDL.

For enhanced communication performance between tester and vehicle, the following features are supported in the in-vehicle gateways:
For DIAG and SWDL:

- Parallel requests to different ECUs
- Functional requests to all ECUs

For SWDL:

- Queued requests to same ECU

•General In-vehicle gateway requirements

4.9.1.1.4 General In-vehicle gateway requirements

The diagnostic protocol between the tester and the vehicle is specified in *[GW_12] DoIP* and *[GW_26] ISO DoIP ISO 13400-2*.

The in-vehicle gateways should be as simple and robust as possible, and e.g. leave complexity as re-sending a request to the tester (e.g. if no response from an ECU and tester times out).

The in-vehicle gateways (Three types of gateways: Vehicle GW, Domain Network GW and Sub Network GW) should not do any validation or checks on responses from ECUs on Session layer level or above. The in-vehicle gateways should only work on transport protocol layer level and below, i.e. layer below where session layer timing parameters as e.g. P2, see *[GW_3] UDS Session*, is defined.
An exception of this generic requirement may be needed for the Vehicle GW connected to the external tester, which may need to work in application level to e.g. differentiate if a response should be sent different external network as Ethernet, WLAN or CAN, or to an in-vehicle tester.

The gateways in the vehicle shall have separate independent communication channels and buffers for downstream/request messages and upstream/response messages with flow control mechanisms where needed. The indepent allocation of resources for request/downstream and response/upstream is important so that one direction does not starve out the other direction.see Figure *"Gateway ECU handling network level"*.

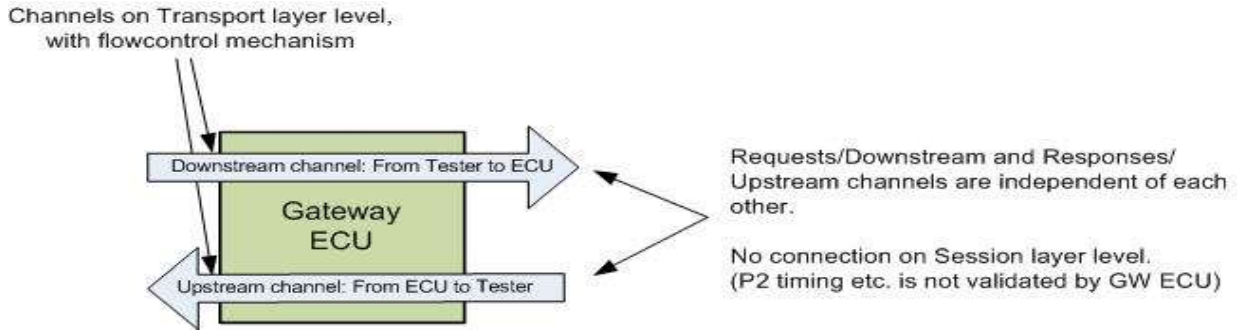


Figure: Gateway ECU handling network level

An functional request, e.g. TesterPresent (see *[GW_19] ISO 14229-1*) to keep the ECUs in a non-default session, shall always have a higher priority through the in-vehicle gateways over a physical request.

When dimension number communication channels and buffers in the in-vehicle gateways it should be guaranteed that messages are not lost during normal communication, e.g. during handling of parallel requests described later in this document.

To be able to handle different speed and different size on frames on different networks, all segmented messages

should be routed through a transport layer.

An in-vehicle gateway should never re-send messages if some error occurs, as. e.g. time-out errors (this is valid for both session layer time-outs and transport layer time-outs). One reason why the gateways in the vehicle should not do any re-sending is that the DP2, see *[GW_3] UDS Session*, would be unpredictable and a external tester would not know how long it should wait for response before a session layer time out.
If a communication error occur, e.g. time-out in CAN network or connection lost on IP, the in-vehicle GWs should throw away the message(s) and release corresponding buffer(s).

All possible errors from an ECU should be visible for the external tester, the gateways in the vehicle should not hide or correct responses from ECUs. The external tester should take the actions as e.g. re-send a message if needed.

If a functional request is received that would lead to reset of the in-vehicle gateway, the in-vehicle gateway must send the functional request to all connected networks before the in-vehicle gateway to performs the reset, so that no functional requests are lost.

A in-vehicle gateway can not guarantee that responses is not lost if the tester send a request that would lead to reset of the in-vehicle gateway. In the case when the tester expect a response, the tester should not send a request that would lead to reset of an in-vehicle gateway before the expected response is receive or a time-out on the expected response.

To reduce latency, "gatewaying-on-the-fly" should where ever possible be used in the in-vehicle gateways. A diagnostic message should be sent as soon as possible, i.e. before a complete segmented message is received. If the network inferface support this, e.g. CAN FD and CAN, "gatewaying-on-the-fly" shall be used for these networks. A exception from this generic requirement is communication to and from the vehicle which will be message based due to the *[GW_26] ISO DoIP protocol ISO 13400-2*, i.e. only exception of this requirement if the network/standard does not support "gatewaying-on-the-fly".

All in-vehicle gateway should have an routing table, describing the network topology behind the gateway.

If a multiple CPU solution is used in the GW ECU for implementing the gateway functionality, the internal interface between the CPUs shall be design to still fulfil the GW ECU requirements.

661367 v1 Flow control mechanism for diagnostic messages

Version history:
(1):

Purpose: Flow control mechanism needed for queued request, functional request functionality.
Source:
Verification Method: Test
Legacy ID: 74082v2

An in-vehicle gateway shall support flow control mechanism on the connected networks, both for request/downstream and for responses/upstream.

Rationale:
No messages or frames shall be lost, if the receiver cannot receive the complete message directly due to out of “high level” buffers, it should be negotiated between sender and receiver when the remaining part can be sent. E.g. on CAN FD and CAN network using flow control frames WAIT and CTS. E.g. on IP network using TCP windows.

661368 v2 Gatewaying-on-the-fly

Version history:
(2):

Purpose: Reduce latency of forwarding diagnostic messages and possibilty to save buffer memory (i.e.

buffers less than the complete message can also be used to gateway messages)

Source:

Verification Method: Test

Legacy ID: 74089v3

When both connected networks supports gatewaying-on-the-fly, the gateway shall use the gatewaying-on-the-fly functionality to reduce latency.

Rationale:
A diagnostic message should be sent as soon as possible, i.e. before a complete segmented message is received. If both network inferface support this, e.g. CANFD and CAN, "gatewaying-on-the-fly" shall be used for these networks to reduce latency.
Addition, parallel requests to different target ECU

Comment:
The TP threshold for gatewaying-on-the-fly shall be used to for the complete transmission.

Example if CANFD frame is 64 bytes, and 100 bytes of data needs to be transfer, send data on CANFD when 62 bytes of data is available from CAN (the data part will be 62 bytes + 2 bytes "overhead" with e.g. Network protocol control information in a CANFD CF frame).
I.e. not only start to send the message when 62 bytes is received, and then continue with 7 bytes of data on CANFD buffer when CANFD “catches” up on CAN.
Fill the CF frames on CANFD with 64 bytes of data during the complete transmission, i.e. use as less overhead as possible in the CANFD frame.

Comment:
Communication between an external tester and the vehicle is an exception of this rule. This communication shall follow the DoIP protocol as specified in *[GW_12] DoIP* and *[GW_26] ISO 13400-2*.

661369 v2 Generic gateway latency requirement

Version history:
(2): Added minimum requirements for IP communication.

Purpose: Generic requirement for latency thru the gateway ECU.

Source:

Verification Method: Test

Legacy ID: 74168v2

When the gateway has received enough data on the receive (Rx) communication channel to fill an L-PDU on the transmit (Tx) side and the gateway can allocate a communication channel on the Tx side, start of transfer on the Tx side shall be within 10ms.

Comment:
For IP to IP communication the requirement is higher, 5ms, see *[GW_14] Internetworks General Specification*.
For IP to IP communication, the performance time of the TCP ack for ethernet data received shall be faster than 3ms.
For IP to IP communication, the transfer rate shall be faster than 7.5kB/ms during transferring DoIP data (Excluding TCP ack time).

661370 v1 Multiple CPU design in the GW ECU

Version history:
(1):

Purpose:

Source:

Verification Method: Analysis
Legacy ID: 366540v0

If a multiple CPU solution is used in the GW ECU for implementing the gateway functionality, the internal interface between the CPUs shall be design to still fulfil the GW ECU requirements.

Rationale:
Latency requirement on the GW ECU shall be kept, which e.g implies that gateway-on-the-fly shall be used on the internal interface to keep the GW ECU latency requirement (e.g. 10ms requirement on GW ECU level).
Any internal data transfer of a GW ECU shall never restrict the flow of gatewayed data, i.e. the internal data throughput must surpass the throughput of connected interfaces/networks by sufficient margin.

If the IP infterface is realized in one CPU and the CAN FD interface is realized in an other CPU, the CPU on the IP side must be aware of current available resources in the CPU on the CAN FD side regarding available communication channels and buffers. If the CAN FD side runs out of resources for transmission of messages, the messages on the IP side must be halted until the resources on the CAN FD side is released and available.
No messages shall be dropped due to out of resources in one of the CPUs. Messages shall only be halted until resources are available and then continue with the transmission.

661371 v1 — No re-sending of diagnostic requests

Version history:
(1):

Purpose: Keep internal gateways as simple as possible, and have a predictable in vehicle (delta)P2 time.
Source:
Verification Method: Test
Legacy ID: 74075v2

If a communication error occurs, an in-vehicle gateway shall not re-send a diagnostic request.

Comment:
Resending of diagnostic requests shall only be allowed for the external tester.

661372 v1 — No re-sending of diagnostic responses

Version history:
(1):

Purpose: Keep internal target ECUs and gateway ECUs as simple as possible, and have a predictable in vehicle (delta)P2 time.
Source:
Verification Method: Test
Legacy ID: 74077v2

If a communication error occurs, an in-vehicle gateway shall not re-send a diagnostic response.

661373 v1 — Release buffers on communication errors

Version history:
(1):

Purpose: Not lock buffers, if a communication error occurs and the message cannot be sent or received. Requirement is on both sender and receiver side.
Source:
Verification Method: Test

Legacy ID: 74080v2

If a communication error occurs, the in-vehicle gateway shall release corresponding buffers used for the transmit (Tx) and/or receive (Rx) communication channels.

661374 v1 Routing table

Version history:
(1):

Purpose: To be able to only route diagnostic messages which is supposed to be addressed to a specific Domain Network. For less complexity and maintenance is the diagnostic messages routed on network ID level.
Source:
Verification Method: Test
Legacy ID: 74171v2

An in-vehicle gateway shall have routing table for all possible paths to the ECUs after the gateway in the network topology, functionally addressed request included.

Comment:
See example in Figure " *Example topology with routing tables"*.

661376 v1 Separate CCs for request and responses

Version history:
(1):

Purpose: Request and response communication channels must be independent of each other for supporting functionality as parallel requests and queued requests
Source:
Verification Method: Test
Legacy ID: 74085v2

An in-vehicle gateway shall have separate and independent communication channels (CCs) for diagnostic request/downstream and for diagnostic responses/upstream.

661375 v1 Separate buffers for requests and responses

Version history:
(1):

Purpose: Request and response buffers must be independent of each other for supporting functionality as parallel requests and queued requests
Source:
Verification Method: Test
Legacy ID: 366516v0

An in-vehicle gateway shall have separate and independent buffers for diagnostic request/downstream and for diagnostic responses/upstream.

Rationale:
All buffers can not be used for one way so that the other way is starved out and messages are dropped.

661377 v1 Session layer time-out handling

Version history:
(1):

Purpose: Keep in-vehicle gateways as simple as possible. In-vehicle gateways shall not unpack and analyze data part of messages. Handling of Session layer time-outs, as e.g. P2 and P4, would require analysis of data and a higher level according to the OSI levels and more complex implementation.
Source:
Verification Method: Test
Legacy ID: 74072v3

An in-vehicle gateway shall not handle Session layer time-outs.

Comment:
Session layer time-outs handling (time-out between request and response message) shall only be handled by the external and/or internal tester.

661378 v1 **Transfer all functional request before a reset of gateway**

Version history:
(1):

Purpose: Secure that no data is lost in the gateway due to reset of the gateway.
Source:
Verification Method: Test
Legacy ID: 74087v2

If an in-vehicle gateway receives a functional request that will lead to a reset of the in-vehicle gateway, the gateway shall first transmit all functional diagnostic to the connected network(s) before the gateway performs the reset.

Comment:
Implementation may differ depending on network type, e.g. CAN or IP network, depending on which functionality the network stack support, e.g. type of acknowledge that a message is sent.

There is also requirements on the tester to avoid that messages are lost. The tester should not send a functional request that leads to reset with positive response expected (SPRMIB=FALSE), in this case there is a risk that the responses from the ECUs will be lost. A request to reset in-vehicle GW (functional or physical addressed) shall not be sent if there is pending request/responses in the vehicle. If the tester sends a request to reset a in-vehicle GW before all expected response from ECUs are received, there is a risk that responses are lost.

661379 v1 **Transport layer routing**

Version history:
(1):

Purpose: To be able to handle different speed and messaged sizes on different networks
Source:
Verification Method: Test
Legacy ID: 74704v2

All segmented diagnostic messages shall be routed thru a transport layer with flow control mechanism.

•General allocation of buffers requirements

4.9.1.1.5 General allocation of buffers requirements

661380 v1 **Allocation of buffers from buffer pools**

Version history:
(1):

Purpose: To be able to specify max values for latency thru a gateway, there must also be a max time for allocating buffers.
Source:
Verification Method: Analysis
Legacy ID: 74649v2

If the total size of messages does not exceed the total size of available memory for buffers, an in-vehicle gateway shall be able to allocate a suitable buffer for the message within 1ms.

Rationale:
Fragmentation of memory for buffers must be taken in account when implementing an algorithm for allocation of buffers. Memory reserved for buffers should not be so fragmented so this time is exceeded, e.g. due to a need to run a de-fragmentation algorithm.

Comment:
Internal behaviour that is hard to measure from outside. Requirement should be verified with data/test result from supplier or inspection of implemented algorithm.

661381 v1 **Static allocated buffers vs dynamic allocated buffers**

Version history:
(1):

Purpose: Indication to ECU supplier how much more memory that is needed if a static sized buffer algorithm is used instead of a dynamic sized buffer algorithm
Source:
Verification Method: Analysis
Legacy ID: 436123v0

If static sized buffers are used instead of dynamic sized buffers, it shall be estimated that at least 50% more memory is needed.

Rationale:
All buffer requirements assume a dynamic sized buffer algorithm, i.e. with efficient use of available memory without unused gaps in the memory.
If using static sized message buffers and/or not possible to reserve separate buffers for request and responses: the project and the ECU supplier shall come to an agreement on used buffer structure to achieve similar performance, supplier shall count with that at least 50% more memory is needed when selecting hardware

4.9.1.1.5.1 General allocation of buffers requirements, part 1

This section will describe and give grounds for the following requirements:

- REQPROD 74649 Allocation of buffers from buffer pools
- REQPROD 436123 Static allocated buffers vs dynamic allocated buffers

This document does not specify exactly how buffer allocation shall be implemented, this is up to the implementer. Allocation could e.g. be from dynamically sized message buffers or static sized message buffers in a buffer pool. However, the requirements on minimum RAM for buffers in the specification assume an efficient use of the available RAM, implicitly some sort of dynamic buffer allocation to be able to use most or the memory at all times. If a simpler static sized buffer allocation is used, the memory needs to be increased by at least 50% to achieve equal throughput as if a more complex and dynamic sized message buffer allocation is used.

Since this a generic cross-platform specification, it cannot contain optimizations for all type of possible gateways.

E.g. A gateway with only one network and one ECU connected does in practice not need same amount of memory as a gateway with two networks and 25 ECUs connected.
Hence, optimization of buffer usage may be need to be handled by project by project and by gateway by gateway to get a compromise between efficiency and cost.

There are also some rules that must be followed to get a predictable behavior with known maximum times for allocation. The time for allocating a buffer for a message must be within a defined maximum time, i.e. buffer pool shall not be fragmented so that the time to allocate a buffer could increase to some unpredictable and unknown time.

There must be some reserved buffers in buffer pools for a minimum number of request and response messages, to be able to receive messages from both directions request/downstream and response/upstream. I.e. all buffers cannot be allocated for requests/downstream, and vice versa not all buffers can be allocated for response/upstream so that one direction is starved out due to all buffers are allocated for the other direction.

Below are two examples of two different buffer allocation algorithms:

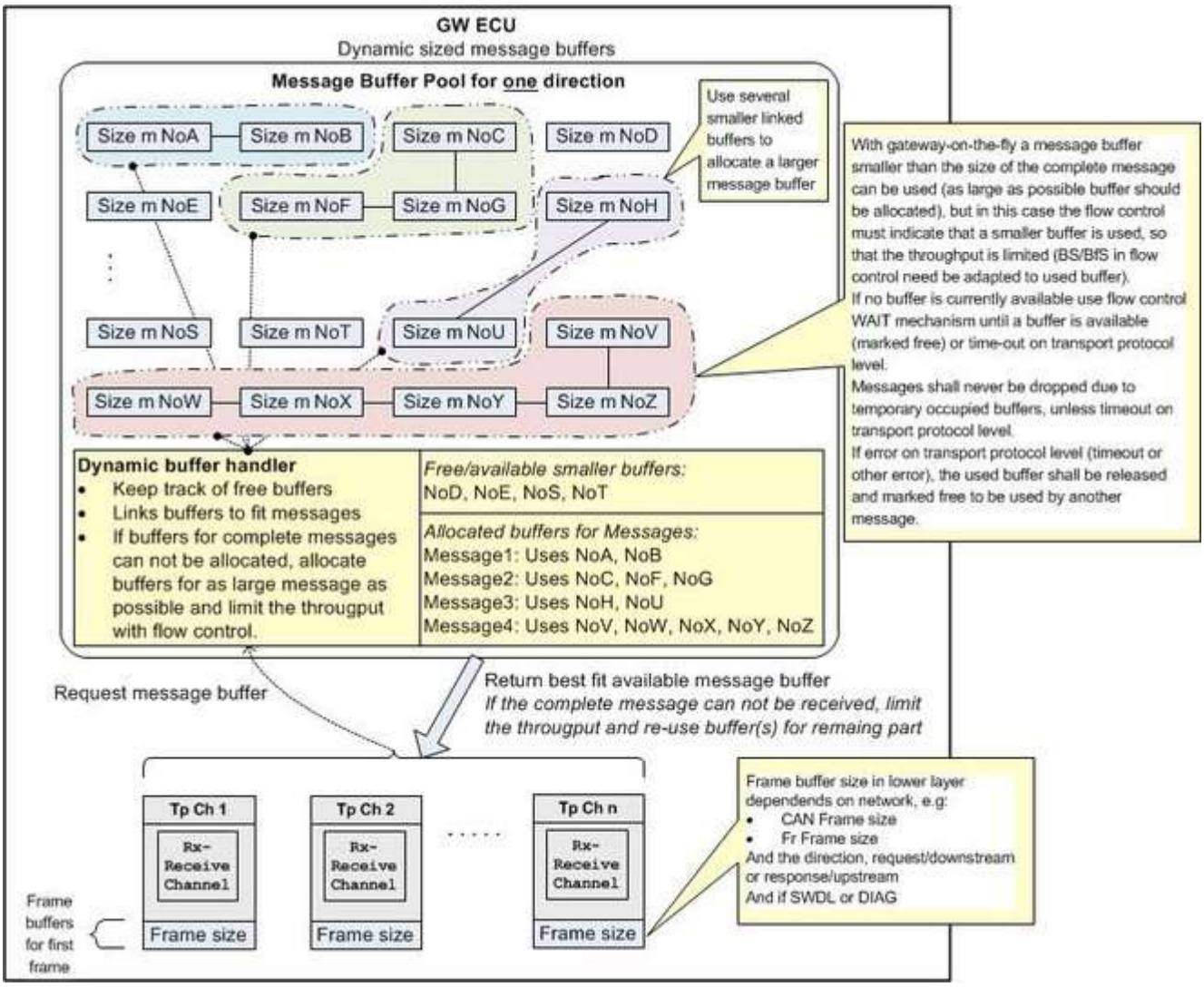


Figure: Example buffer pool with dynamic sized message buffers

4.9.1.1.5.2 General allocation of buffers requirements, part 2

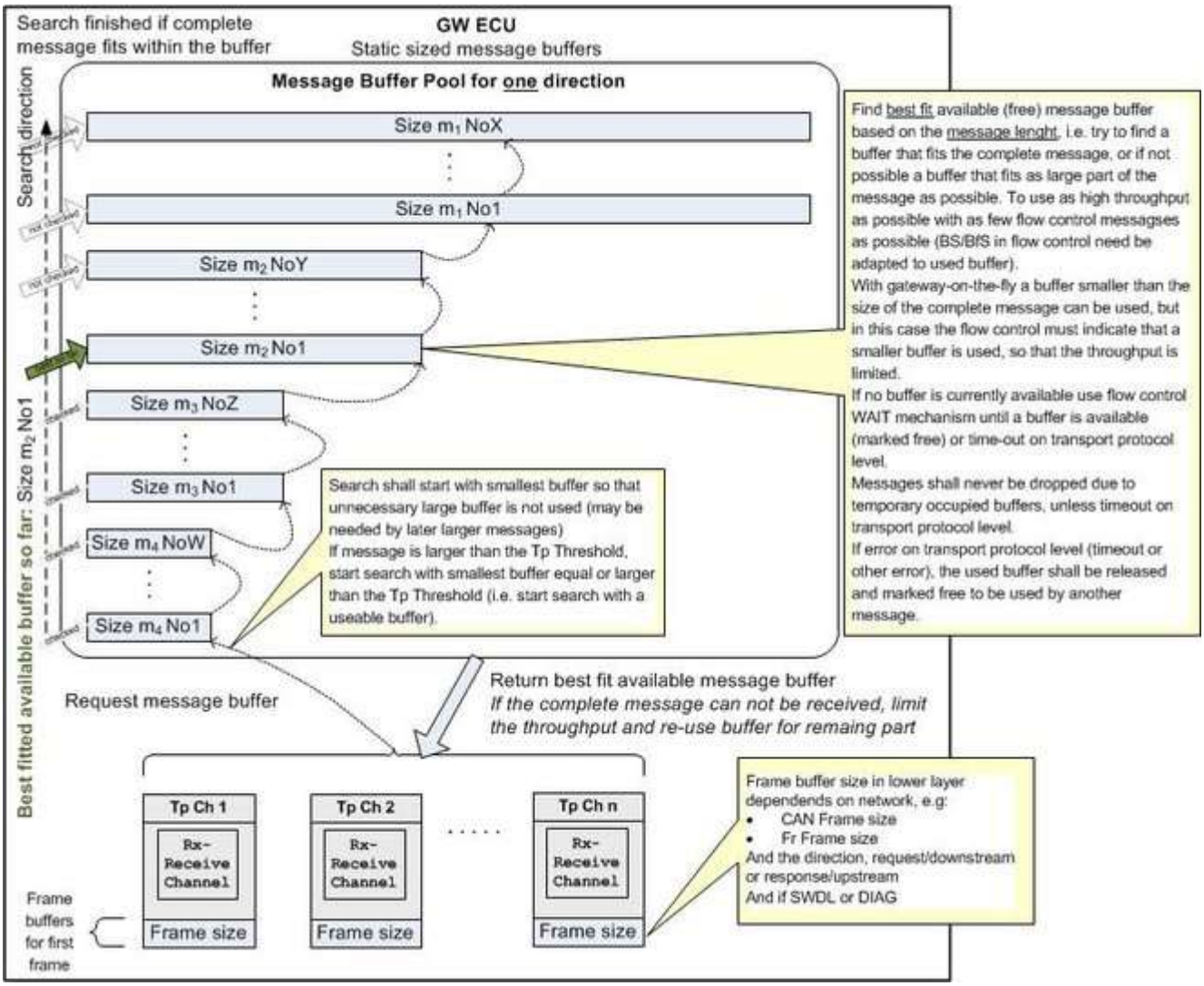


Figure: Example buffer pool with static sized message buffers

•Parallel physical requests to different ECUs

4.9.1.1.6 Parallel physical requests to different ECUs

The external tester can send requests to different ECUs in parallel without waiting for response between the requests.

Example: If external tester shall send two requests to two different targets, TA 0x211 and TAx311, external tester does not need to wait for response from first request to TA 0x211 before sending the next request to TA 0x311.

The Backbone Network CAN FD supports multiple instances of transport protocol (TP) communication channels (CCs) to be able to send messages in parallel/simultaneously.

If the Vehicle GW can allocate a free communication channel onCAN FD, the next request can be sent to the Backbone Network as soon as the next DoIP request is received. If no communication channel can be allocated, i.e. all communication channels occupied, the Vehicle GW must wait for next CAN FD cycle before re-trying to allocate a communication channel.

The sequence of physical requests that is received in Vehicle GW from external tester shall be kept when Vehicle GW tries to allocate a communication channel.

Example: Vehicle GW receives RequestA and RequestB, and requestA was received first but no communication channels are currently available. When a communication channel is available RequestA should be connected to this communication channel, and RequestB has to wait for next communication channel.

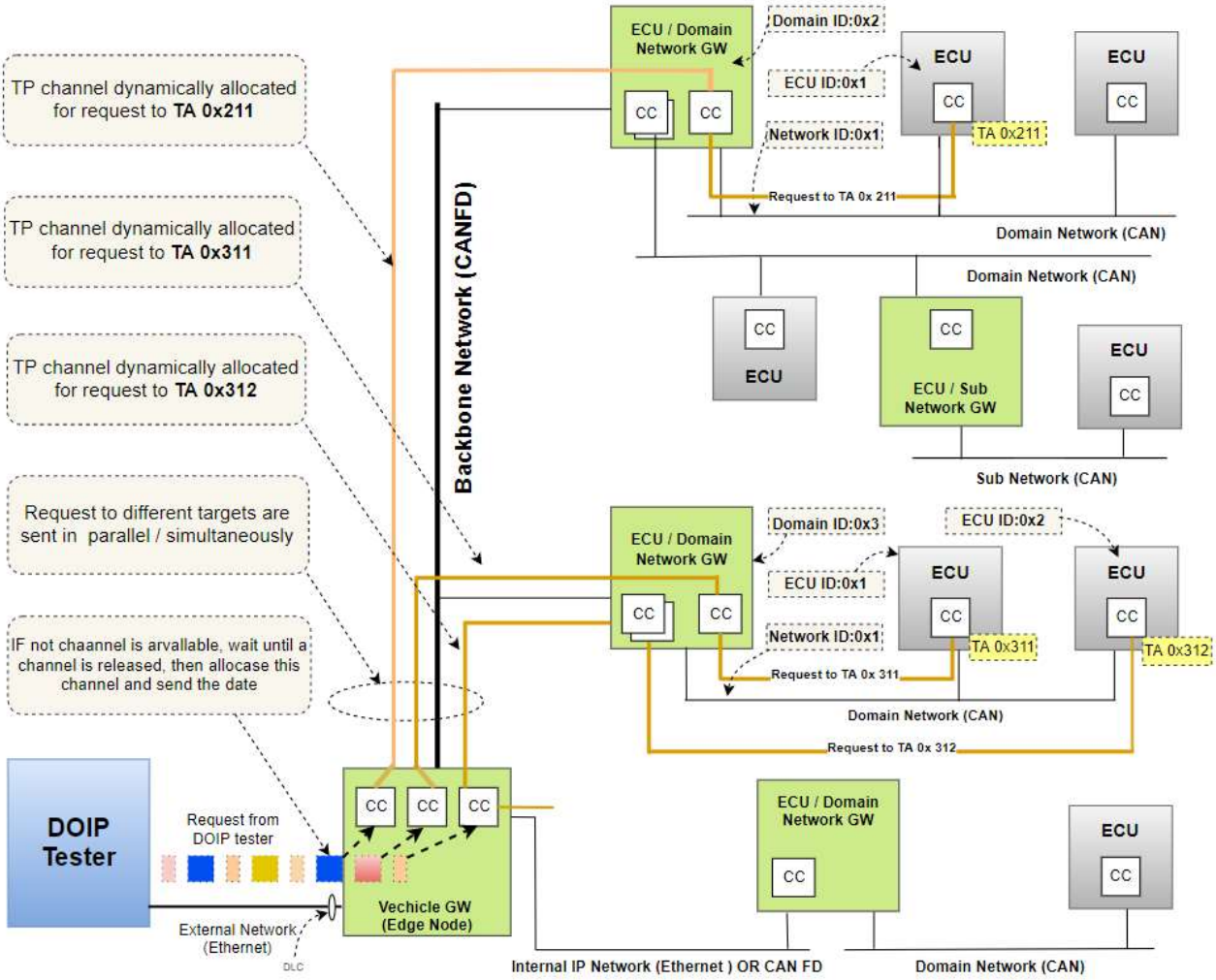


Figure: Overview three parallel request to three different target ECUs

Sequence and more details regarding traffic on the networks for parallel requests, see section ' *Parallel physical requests to different ECUs'* and Figure " *Sequence diagram three parallel request to three different target ECUs"*.

661382 v2 **Keep chronologic order of parallel physical messages**

Version history:
(2): 增加说明，明确使用FIFO策略。

Purpose: Keep sequence of parallel physical requests.
Source:
Verification Method: Test
Legacy ID: 74201v3

The in-vehicle GW shall preserve the chronological order of received parallel physical request/responses when sending out the request/response on the networks.

Rationale:

Fair arbitration shall be implemented in the gateway, i.e. one channel may not starve another channel, and use first in first out (FIFO) strategy for transport protocol.

Comment:
A functional request as e.g. a tester present should have highest priority, and may be sent out before a physical request even if it arrives later to the gateway.

661383 v1 **Parallel physical requests to different ECUs**

Version history:
(1):

Purpose: Speed up communication between tester and vehicle, by sending request to several networks and ECU in parallel/concurrent.
Source:
Verification Method: Test
Legacy ID: 74200v4

The in-vehicle GW shall support parallel requests to different ECUs (different target addresses).

Rationale:
Parallel request also leads to parallel responses from the target ECUs, including response pending and final response.
Response pending and final response from a target ECU can come close an, i.e. without separation time between the two frames.

Different target addresses includes the GW ECU, and if request is SWDL request the consequence is that GW ECU must be able to re-programming itself at the same time as requests are gatewayed to other ECUs.
For example flashing of the GW ECU shall not disturb the gateway functionality to other ECUs.

661384 v1 **Wait for available communication channel**

Version history:
(1):

Purpose: Clarify that messages shall not be dropped if all communication channels are currently occupied. Message shall be kept and send as soon as a communication channel is released
Source:
Verification Method: Test
Legacy ID: 366460v0

If no communication channel can be allocated, i.e. all communication channels are currently occupied by other messages, the in-vehicle GW must wait until a communication channel is released and then send the message on the released communication channel.

Rationale:
No messages shall be dropped due to that there is currently no available communication channel.

•Functional requests to all ECUs

4.9.1.1.7 Functional requests to all ECUs

The external tester can send the same request to all ECUs in the vehicle, see example in Figure " *Overview functional request to all target ECUs".* A variant of this could be to send functional request to all ECUs on a specific domain or network. See *[GW_1] UDS Services* for possible functional request addresses.

Example: external tester sends a requests to all ECUs within the vehicle with TA 0xFFF, to all ECUs on specific domain with 0x<Domain ID>FF.

Functional request should be unsegmented, hence it needs to be unsegmented on all types of networks, and this is specified in *[GW_1] UDS Services*.

The in-vehicle gateway must be able to send a functional request to all connected network and the gateway ECU it self.

An functional request, e.g. TesterPresent to keep the ECUs in a non-default session, shall always have a higher priority through the in-vehicle gateways over a physical request.

The in-vehicle gateway should be able to open a separate transmit (Tx) communication channel (CC) for functional request for the respective network(s) connected to the gateway. I.e. separate CC(s) from the CCs for physical request so that a functional request (TesterPresent) is not jammed in the gateway.

Note: the in-vehicle gateway does not check the data in the request, i.e. which service that are included in the functional request, hence this requirement will cover functional TesterPresent to keep ECUs in a non-default session ("keep alive logic").

With a functional request with positive response suppressed (SPRMIB=TRUE), e.g. a functional TesterPresent for keeping a non-default session, there are (normally) not any expected responses back from the ECUs.

The in-vehicle gateways should also support functional request with positive response expected (SPRMIB=FALSE) and the in-vehicle gateways must be able to handle all responses from the ECUs.

For the responses from a functional request (with many responses) the in-vehicle GWs must be able to send flow control WAIT if not all complete responses can be received directly due to out of buffers.

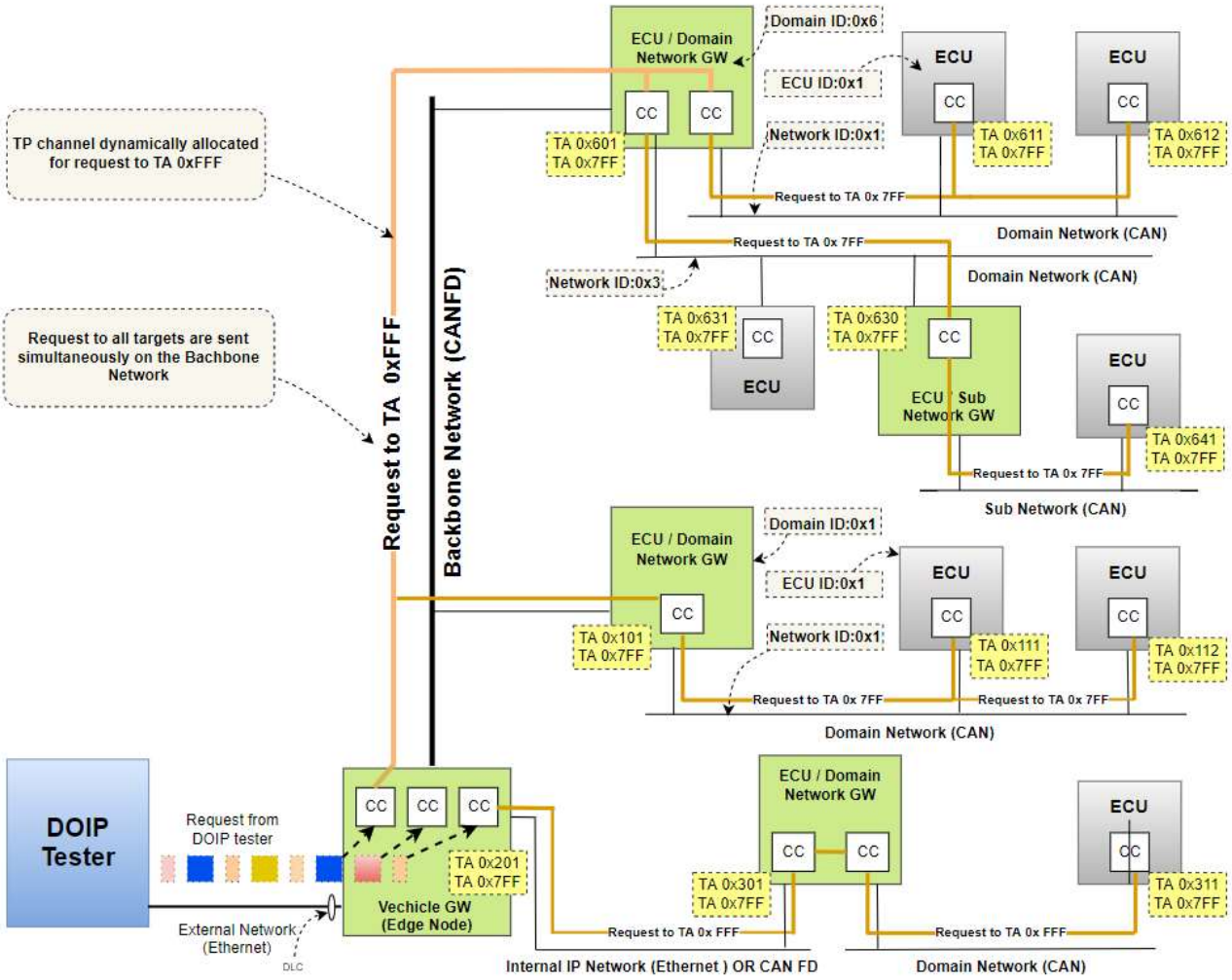


Figure: Overview functional request to all target ECUs

661385 v1 Functional requests transmit (Tx) com. channel(s)

Version history:
(1):

Purpose: A TesterPresent may not be delay/stopped in the gateway due to queued and/or parallel physical request.
Source:
Verification Method: Test
Legacy ID: 74208v2

The in-vehicle gateway shall be able to open separate transmit (Tx) communication channel(s) for functional request for the respective network(s) connected to the gateway.

661386 v1 Internal functional request handling

Version history:
(1):

Purpose: A functional request could be to both the in-vehicle GW and the ECUs connected to the in-vehicle GW, i.e. the request should be received and executed by the in-vehicle GW and sent to other ECUs.

Source:
Verification Method: Test
Legacy ID: 74215v2

The in-vehicle GW shall be able route a functional request to all connected networks and to the in-vehicle GW it self.

Comment:
If the request will lead to a reset, the in-vehicle should first send out the request(s) on the network(s) before performing a reset of the in-vehicle GW, see requirement REQPROD 74087.

661387 v1 **Prioritize functional request**

Version history:
(1):

Purpose: A functional TesterPresent must be able to reach a target ECU in non-default session, to keep the ECU in non-default session, independent of other possible physical request that could be queued in the gateway.
Source:
Verification Method: Test
Legacy ID: 74213v2

An in-vehicle gateway shall always prioritize a functional requests over a physical request.

Rationale:
An in-vehicle gateway does not check the content of a functional request, i.e. does not read the data field of the request, hence all functional request will include functional TesterPresent.

•Gateway sequence diagrams

4.9.1.1.8 Gateway sequence diagrams

This section contain no new requirements, but describes in some more details example of traffic on the networks as consequence of the functionality described in sections *'Parallel physical requests to different ECUs'* and *'Functional requests to all ECUs'*

4.9.1.1.8.1 Functional requests to all ECUs

The functionality described in section ' *Functional requests to all ECUs'* implies that in-vehicle GWs must be able to send flow control WAIT frames if there are not enough buffers available to receive all responses in parallel/simultaneously.

Note: For simplicity, the responses from the gateways on functional request is not included in the figures.

Figure: " *Sequence diagram, functional request and 1 available response buffer in GW"* gives an overview of the traffic on the networks when there is not response buffers available to receive all responses in parallel/simultaneously, and flow control WAIT frames must be sent.

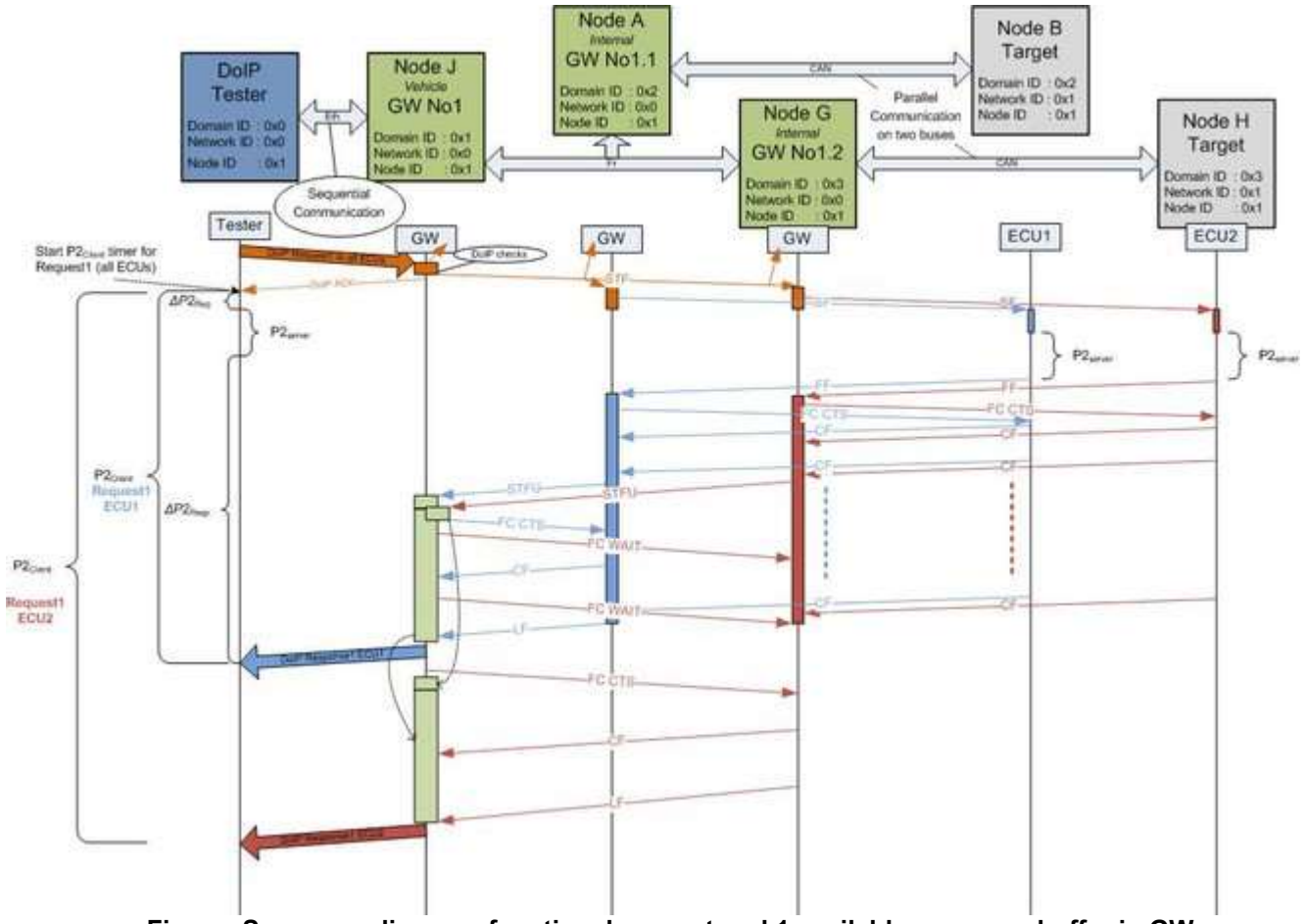


Figure: Sequence diagram, functional request and 1 available response buffer in GW

Figure: " *Sequence diagram, functional requests and 2 available response buffer in GW"* shows same use case, but with response buffers available, hence no flow control WAIT frames need to be sent.

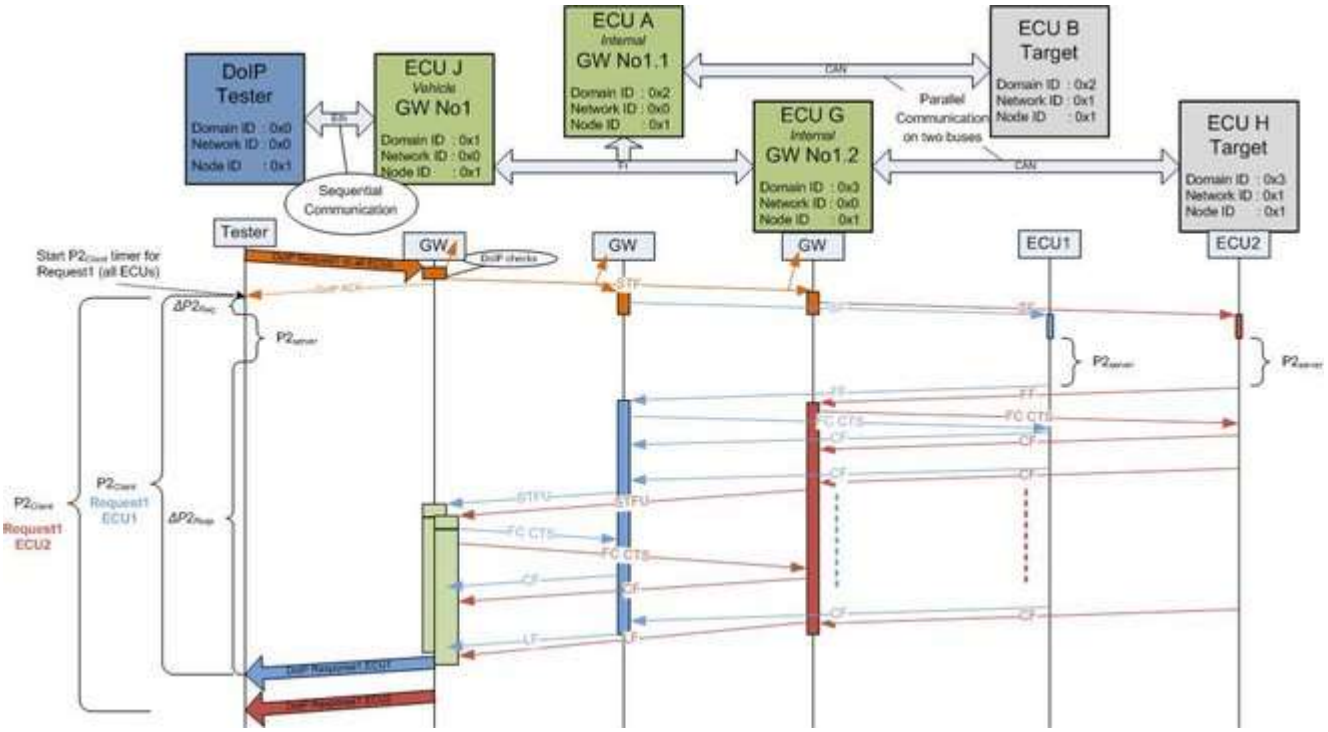


Figure: Sequence diagram, functional requests and 2 available response buffer in GW

4.9.1.1.8.2 Parallel physical requests to different ECUs

The functionality described in section ' *Parallel physical requests to different ECUs'* implies multiple communication channels on the Backbone Network (FlexRay) and parallel traffic on different Domain Networks.

Figure: ' *Sequence diagram two parallel request to two different target ECUs'* gives an overview of the traffic in the different networks.

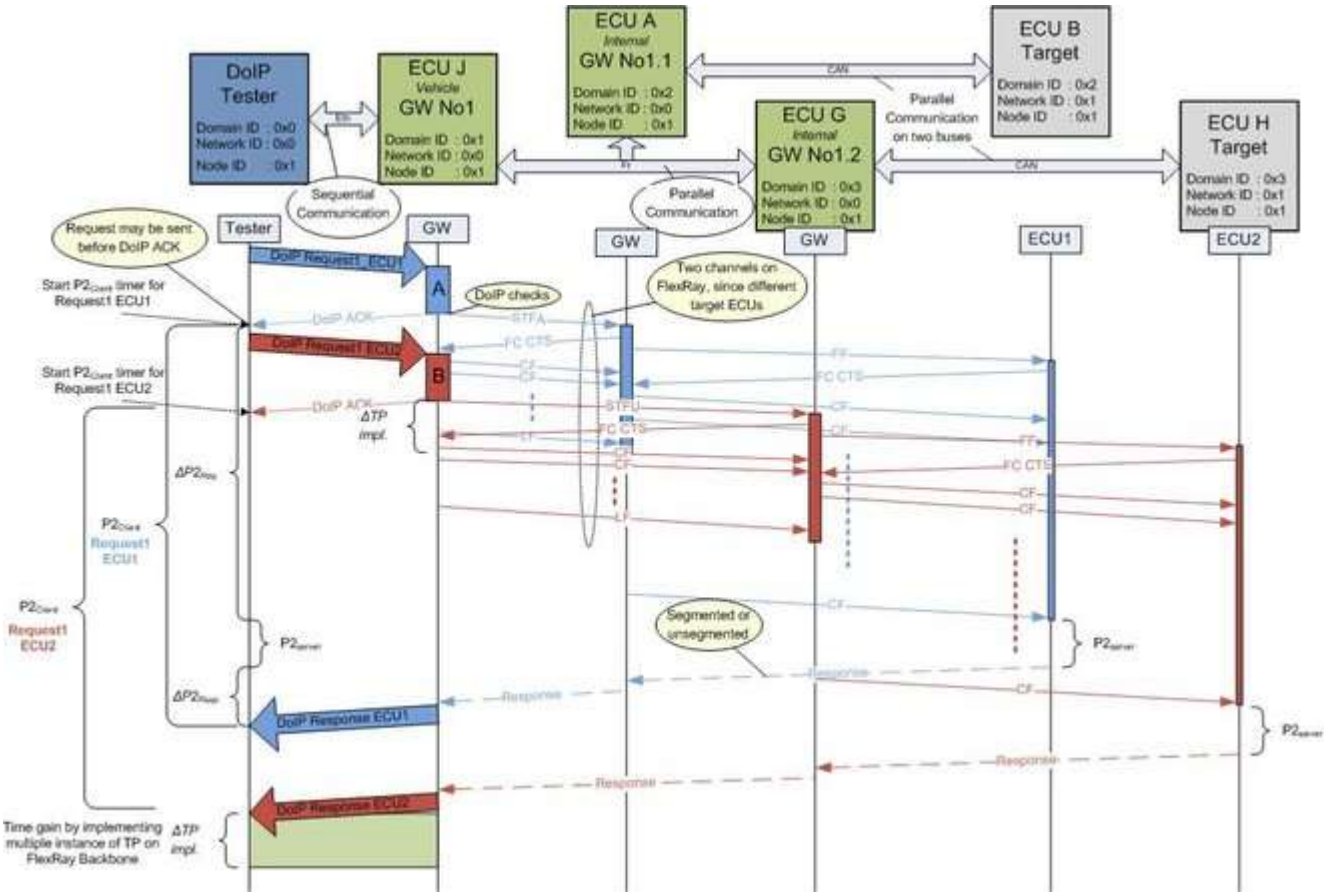


Figure: Sequence diagram two parallel request to two different target ECUs

4.9.1.2 Gateway Generic DIAG

•Routing table

4.9.1.2.1 Routing table

For application mode and diagnostic, it should be possible to update this routing table post build.

The routing table for diagnostic messages should be static and not dependent on the content in the diagnostic message.

661388 v1 **Protection of routing table**

Version history:
(1):

Purpose: Protection of routing table of gateway when it is in use.
Source:
Verification Method: Analysis
Legacy ID: 77681v2

An in-vehicle gateway shall only allow reconfiguration of the routing table when the routing table is not in use (e.g. during programming mode).

Rationale:
Reconfiguration during normal operation (application running and routing table is in use) shall not be possible. Changes during normal operation are regarded as safety risk.

661389 v1 **Static routing rules**

Version history:
(1):

Purpose: No dynamic routing, dependent on the content of a message.
Source:
Verification Method: Analysis
Legacy ID: 77682v2

An in-vehicle gateway shall only support statically defined routing rules.

Rationale:
The gateway shall not support dynamic routing rules for diagnostic messages. All routing paths shall be statically defined, and do not depend on the content of a diagnostic message.
Reduction of complexity, realization of deterministic gateways.

664776 v1 **Updateable routing table**

Version history:
(1):

Purpose: Configuration routing table in gateway shall be updateable if a new ECU is added which is not included in the current routing table.
Source:
Verification Method: Analysis
Legacy ID: 77683v2

The routing configuration table in DIAG shall be updateable at post-build time.

Rationale:
The goal is to avoid re-compiling and re-loading of huge application binaries in case of minimal routing changes.

Use case:
A diagnostic request has to be routed to an additional path, e.g. a new ECU is added on a Domain network.

4.9.1.3 Gateway Generic SWDL

•Routing table

4.9.1.3.1 Routing table

For bootloader and software download it is not possible to update the routing table burned in to the non-volatile memory, so the routing table must be able to cover all possible connections from the start, so that a ECU can be added to the network topology with out updating the bootloader.

661391 v1 **Static routing configuration**

Version history:
(1):

Purpose: If a new ECU is added, the Bootloader, PBL and SBL, shall be able to handle this without update of bootloader. Bootloader shall use Domain ID, Network ID and ECU ID ranges when forwarding messages.
Source:
Verification Method: Test

Legacy ID: 77685v3

The routing configuration table in SWDL shall be able to route all possible ECU IDs in a network.
This shall be implemented by either a range based approach or a table with defined spare addresses.

Note:
The table with defined spare addresses, if it exist for the project, are project specific and defined in 'Diagnostics and ECU platform' specification.

Rationale:
The goal is to avoid updating bootloader if ECUs are removed or added to a Domain network.

Use case:
A diagnostic/SWDL request has to be routed to an additional path, e.g. a new ECU is added on a Domain network, but the bootloader (PBL) is not updated.

•Queued physical requests to one ECU

4.9.1.3.2 Queued physical requests to one ECU

Messages to/from an external tester to/from an ECU in a topology as shown in Figure " *Example topology with routing tables, UDS view"* may pass one or several gateways, depending on where in the topology the ECU is connected. Latency is introduced in each gateway step.

Queued physical request is a feature to reduce the impact from this latency caused by the gateways (∆P2, see *[GW_3] UDS Session*) for SWDL. Next request can be queued and already "loaded" in the vehicle and as soon the target ECU has processed the previous request, there is a new request ready in the queue. Next request can be received to the target ECU at the same time as the response from previous request is sent to the external tester. This implies that gateways and target ECU must have full duplex transport protocols for SWDL (or at least be able to send e.g. a flow control WAIT until the next request can be processed). Full duplex requirement is covered in other specifications for respective network.

This implies that the external tester can send several requests to same ECUs for SWDL without waiting for response between the requests.

Example: If an external tester shall send two requests to same target ECU, Request#1 and Request#2 to TA 0x211, the external tester does not need to wait for response from Request#1 before sending Request#2.

The sequence of physical requests that is received in the vehicle from the external tester must be kept on all networks down to the target ECU. Same rule apply for the responses. Two queued messages in the vehicle should not use two different communication channels.

On transport protocol layer level on the Domain Network, last frame of the first request must be sent before the first frame of the second request.

Example: Request#1 is larger than Request#2, and if requests where sent in parallel in two independent communication channels on the CAN FD backbone, Request#2 may arrive before Request#1 in target ECU. This is not allowed!

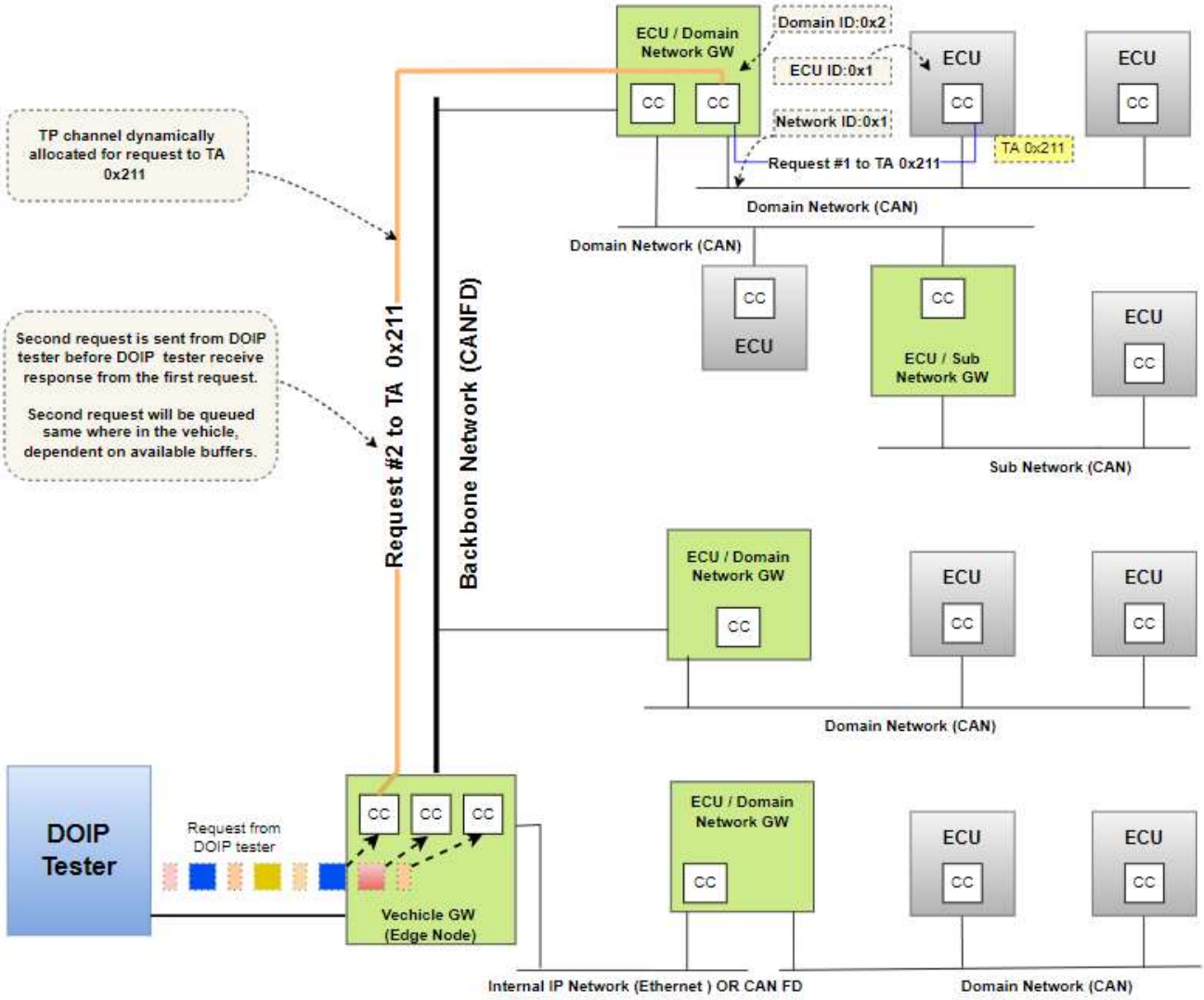


Figure: Overview two queued request to one target ECU

To be able to have a more predictable and not to long P2Client timer on the external tester side for the next/queued request, the external tester should not start the P2Client timer for the next/queued request when the DoIP ACK is received. When the response for the first request is received, or time out for the first request, then the external tester shall start the P2Client timer for the second request.

Requirements regarding the P2Client timer can be found in *[GW_3] UDS Session*.

Sequence and more details regarding traffic on the networks for queued requests, see section ' *Queued physical requests to same ECU'* and Figure " *Sequence diagram two queued request to one target ECU"*.

661392 v1	**Keep chronologial order for queued messages**

Version history:
(1):

Purpose: If several queued diagnostic requests to one ECU, with same target address are sent thru the gateway, the chronological order of the request must be preserved.
Source:
Verification Method: Test
Legacy ID: 74205v3

The in-vehicle GW for SWDL shall preserve the chronological order of queued physical request/responses to/from a target ECU.

Comment:
Similar as REQPROD 74201 for parallel requests, but a queued request should not allocate a new available communication channel if the there is an ongoing transfer to same ECU.
The queued (second) message shall never be sent out on a network before the first message.

661393 v1 **Queued physical request to one ECU**

Version history:
(1):

Purpose: Speed up communication between tester and vehicle for SWDL, by reducing latency between requests to same ECU.
Source:
Verification Method: Test
Legacy ID: 74202v3

The in-vehicle GW for SWDL shall support at least one queued physical request to a target ECU (i.e. same target address in two requests).

Rationale:
Queued request also leads to queued responses from the target ECUs, including response pending and final response.
Response pending and final response from a target ECU can come close an, i.e. without separation time between the two frames.

Comment:
Tester can send two requests to same target ECU without waiting for the response from the first request. The second request is called queued request.
When tester receive the response from the first request, the tester can send next request, which now will be the queued request and so on.

•Gateway sequence diagrams

4.9.1.3.3 Gateway sequence diagrams

This section contain no new requirements, but describes in some more details example of traffic on the networks as consequence of the functionality described in section *'Queued physical requests to one ECU'*

4.9.1.3.3.1 Queued physical requests to same ECU

The functionality described in section ' *Queued physical requests to one ECU'* implies that flow control mechanism must be used, i.e. sending of flow control WAIT frames if next (queued) request not can be received yet, due to no available buffer.

Figure: " *Sequence diagram two queued request to one target ECU"* gives an overview of the traffic on networks with flow control WAIT frames for queued requests.

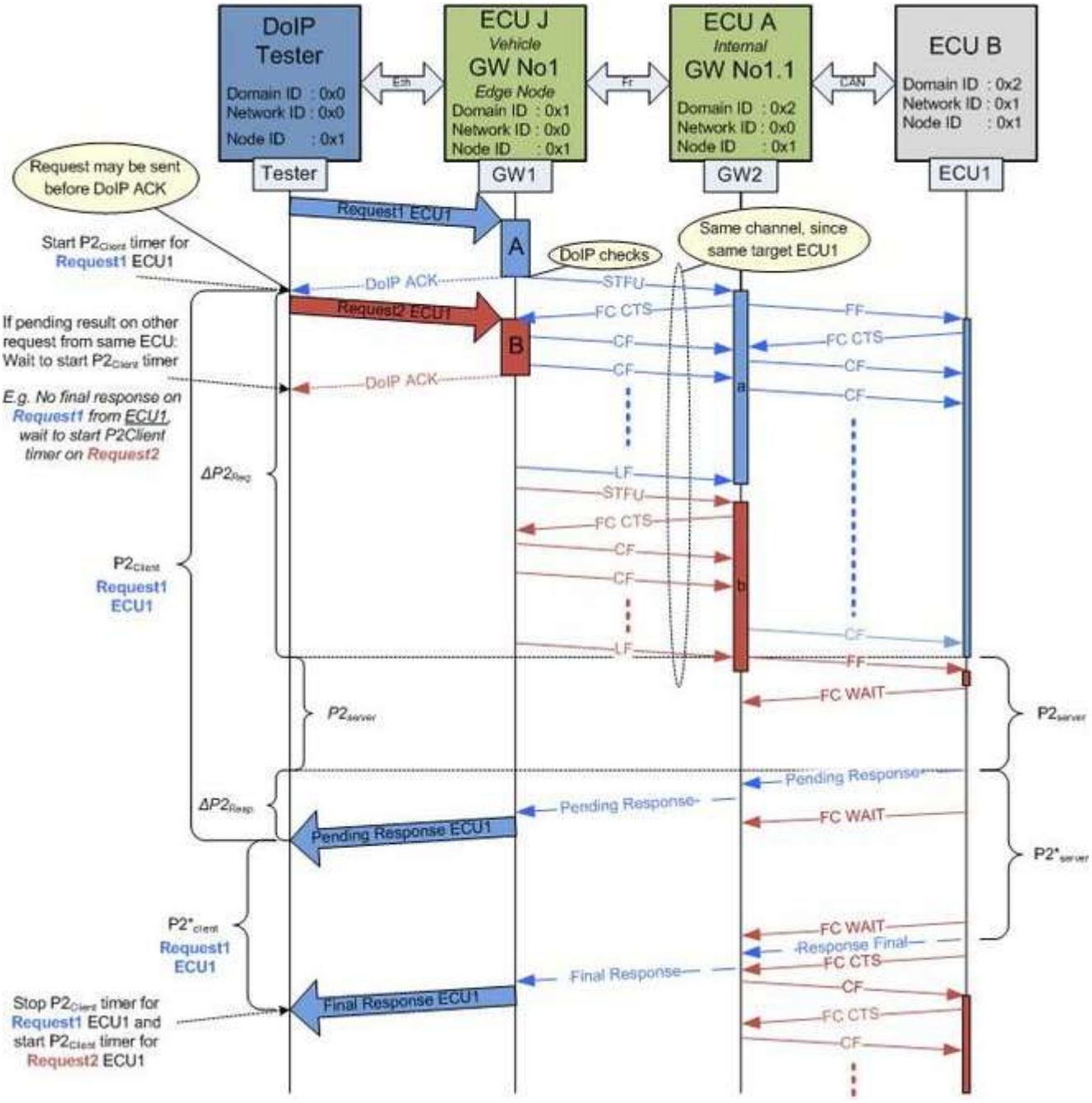


Figure: Sequence diagram two queued request to one target ECU

4.9.1.4 Gateway connected to External IP

•IP diagnostic format on External IP

4.9.1.4.1 IP diagnostic format on External IP

For communication between a tester and the vehicle, the protocol specified in DoIP *[GW_26] ISO 13400-2* will be used.
For communication between a tester and the vehicle, IPv4 or IPv6 may be used.
Examples in this section uses IPv4, if IPv6 is used (for communication between tester and vehicle) the IPv4 header shall be replaced with an IPv6 header.

TCP is used as transport protocol for all diagnostic messages, requests and responses including periodic

responses.

Figure *Example single DoIP message in a Ethernet frame* shows an example where a single DoIP message is packed in an Ethernet/IPv4/TCP frame.

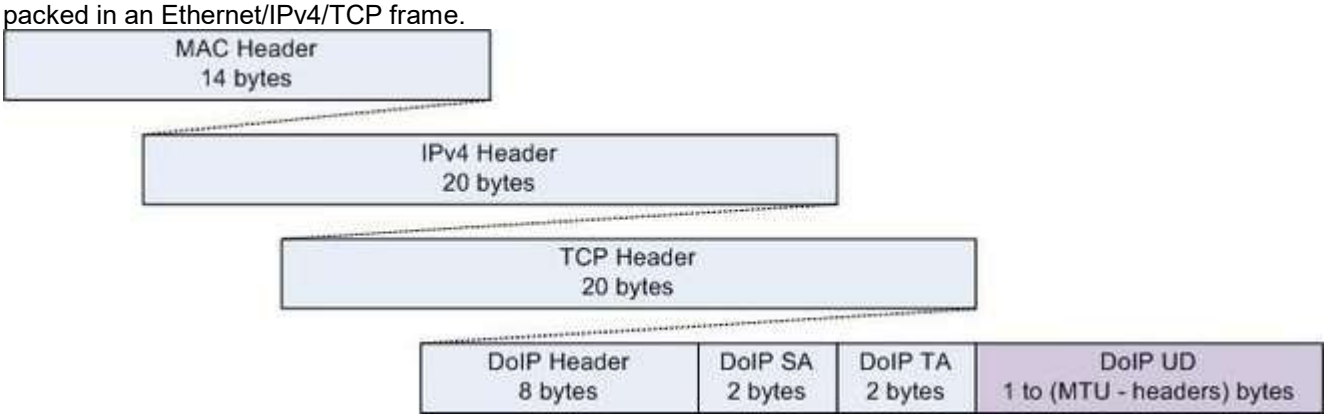


Figure: Example single DoIP message in a Ethernet frame

Sections ' *Diagnostic request on IP network'* to ' *Periodic response (SID 2A) on IP based network'* gives an overview of the frames used for diagnostic message communication in IP network, request, response and periodic responses with DoIP payload type 0x8001.

When TCP is used (stream of bytes), an Ethernet frame could contain several DoIP messages, and one DoIP message could be sent over several Ethernet frames, depending in the DoIP UD and MTU size.

For simplicity in the overviews of DoIP messages, it only shows the case where one single DoIP message is included in a TCP stream that fits within one Ethernet frame and where the MTU size is 1500 bytes.
And only header and data in the frames are showed, i.e. frame trailers containing e.g. CRC checksums etc. are not shown.

For more detailed information about other payload types, e.g. how to establish a connection between the tester and the vehicle, see specifications *[GW_10] UDSonIP*, *[GW_11] Bootloader - UDSonIP* and *[GW_12] DoIP*.

4.9.1.4.1.1 Diagnostic request on external IP network

External IP network request message structure (external tester to Vehicle):

DoIP payload type 8001, can be packed in an Ethernet frame from the external tester into the vehicle and Vehicle GW (Edge Node), if the complete DoIP messages fits within one Ethernet frame. If the DoIP message is larger than MTU size bytes, the DoIP message will be sent in several Ethernet frames.

An Ethernet frame can also contain several DoIP messages.

Figure " *External IP diagnostic request frame structure"* shows the external IP network Ethernet frame format for a diagnostic request (in the case of one single DoIP message within an Ethernet frame).

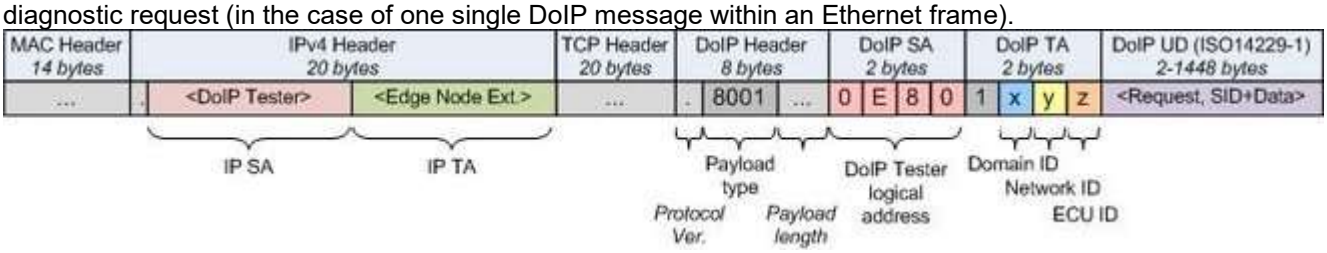


Figure: External IP diagnostic request frame structure

661394 v1	**Supported request format from external IP network**

Version history:
(1):

Purpose: Define supported request frame format on external IP network, including ECU logical addresses in the frame, which should transported to an internal network.
Source:
Verification Method: Test
Legacy ID: 77667v2

To be able to transport a diagnostic request to an internal network and ECU, the Vehicle GW (Edge Node) shall receive all DoIP diagnostic requests with payload type 8001 as defined in DoIP *[GW_26] ISO 13400-2* with ECU logical addresses as defined in *[GW_1] UDS Services* and analyse if the DoIP target address (DoIP TA) is within a valid range.

Rationale:
The gateway shall not only filter and handle request that are target to the gateway.

Comment:
See example in Figure: " *External IP diagnostic request frame structure"*.

4.9.1.4.1.2 Diagnostic response on external IP network

Vehicle to external IP network response message structure:

A response on an external IP network should contain a DoIP header with payload type 8001, as received from the internal IP network.

The Vehicle GW (Edge Node) should change the internal IP address to the external IP address, where the IP TA is the external tester IP address and IP SA is the Vehicle GWs external IP address.

Figure " *External IP diagnostic response frame structure"* shows the external IP network Ethernet frame format for a diagnostic response (in the case of one single DoIP message within an Ethernet frame).

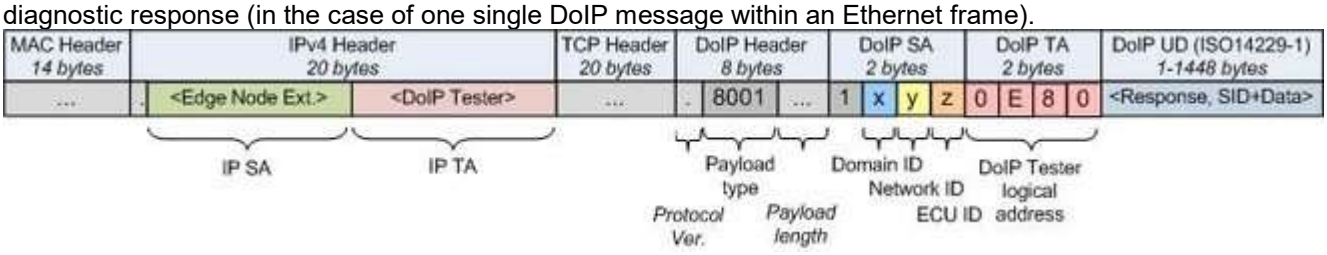


Figure: External IP diagnostic response frame structure

661395 v1	**Response format to external IP**

Version history:
(1):

Purpose: Define the response format out from the gateway to a external IP network, including the ECU logical address in the frame.
Source:
Verification Method: Test
Legacy ID: 77670v2

When a diagnostic response is received that shall be transported to an external IP network and tester, the Vehicle GW (Edge Node) shall send out the diagnostic response with a DoIP header with payload type 8001 as defined in DoIP *[GW_26] ISO 13400-2* with ECU logical addresses as defined in *[GW_1] UDS Services* and with the external tester address in the IP TA and the Vehicle GW external IP address in the IP SA.

Comment:
See example in Figure: " *External IP diagnostic response frame structure"*.

4.9.1.5 Gateway connected to Internal IP

•IP diagnostic format on Internal IP

4.9.1.5.1 IP diagnostic format on Internal IP

For internal IP communication, the protocol specified in DoIP *[GW_26] ISO 13400-2* will be used.
For in vehicle communication only IPv4 shall be used.

TCP is used as transport protocol for all diagnostic messages, requests and responses including periodic responses.

Figure *Example single DoIP message in a Ethernet frame* shows an example where a single DoIP message is packed in an Ethernet/IPv4/TCP frame.

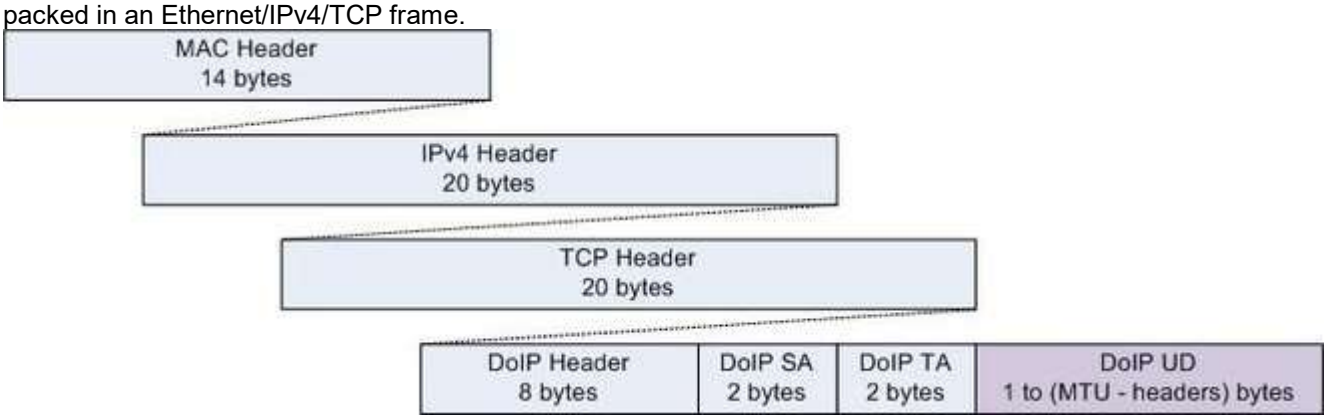


Figure: Example single DoIP message in a Ethernet frame

Sections ' *Diagnostic request on IP network'* to ' *Periodic response (SID 2A) on IP based network'* gives an overview of the frames used for diagnostic message communication in IP network, request, response and periodic responses with DoIP payload type 0x8001.

When TCP is used (stream of bytes), an Ethernet frame could contain several DoIP messages, and one DoIP message could be sent over several Ethernet frames, depending in the DoIP UD and MTU size.

For simplicity in the overviews of DoIP messages, it only shows the case where one single DoIP message is included in a TCP stream that fits within one Ethernet frame and where the MTU size is 1500 bytes.
And only header and data in the frames are showed, i.e. frame trailers containing e.g. CRC checksums etc. are not shown.

For more detailed information about other payload types, e.g. how to establish a connection between the tester and the vehicle, see specifications *[GW_10] UDSonIP*, *[GW_11] Bootloader - UDSonIP* and *[GW_12] DoIP*.

4.9.1.5.1.1 Diagnostic request on internal IP network

Internal IP network request message structure:

For the internal IP network diagnostic request format, The DoIP header should be kept.

By keeping the DoIP header in the request, it is possible to gateway the request from an internal IP ECU (e.g. a Sub Network GW) to some other networks as e.g. a CAN or LIN network.

The Vehicle GW (Edge Node) should change the external IP address to an internal IP address, where the three lowest nibbles in the IP TA is matches the three lowest nibbles in the DoIP TA and IP SA is the Vehicle GWs internal IP address.
If the target ECU is a non-IP ECU (e.g. CAN or LIN ECU) the IP TA shall be same as the Domain GW IP-IP IP address.

Figure " *Internal IP diagnostic request frame structure"* shows the internal IP network Ethernet frame format for a diagnostic request (in the case of one single DoIP message within an Ethernet frame).

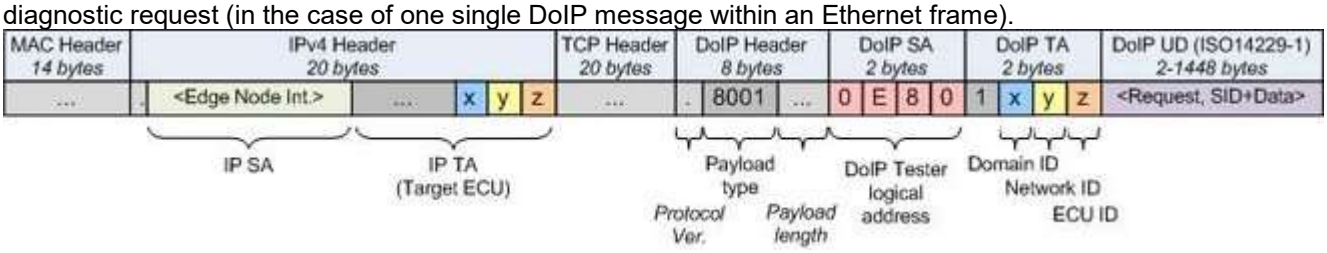


Figure: Internal IP diagnostic request frame structure

661396 v1 **Diagnostic request format on internal IP network**

Version history:
(1):

Purpose: Define the request format out from the gateway to an internal IP network, including the ECU logical address in the frame.
Source:
Verification Method: Test
Legacy ID: 77668v3

All in-vehicle GW connected to internal IP network shall send/receive diagnostic request with a DoIP header with payload type 8001 as defined in DoIP *[GW_26] ISO 13400-2* with ECU logical addresses as defined in *[GW_1] UDS Services* and with the three lowest nibbles in the ECU logical address in the three lowest nibbles in the IP TA and the Vehicle GW internal IP address in the IP SA.

Rationale:
By keeping the DoIP header in the request, it is possible to gateway the request from an internal IP ECU (e.g. a Sub Network GW) to some other networks as e.g. a CAN or LIN network.

Comment:
See example in Figure: " *Internal IP diagnostic request frame structure".*

4.9.1.5.1.2 Diagnostic response on internal IP network

In Internal IP network response message structure:

A response on an internal IP network should contain a DoIP header with payload type 8001.

Figure " *Internal IP diagnostic response frame structure"* shows the internal IP network Ethernet frame format for a diagnostic response (in the case of one single DoIP message within an Ethernet frame).

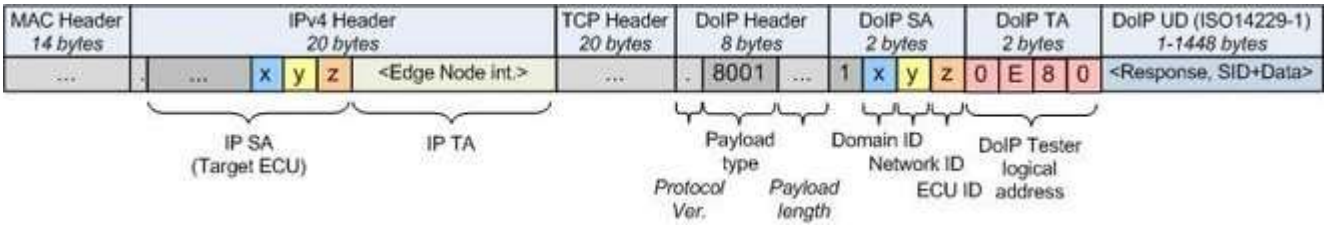


Figure: Internal IP diagnostic response frame structure

661397 v1 **Diagnostic response format on internal IP**

Version history:

(1):

Purpose: Define supported response frame format on internal IP network, including ECU logical addresses in the frame, which should transported to an external network and tester or to an internal tester.

Source:

Verification Method: Test

Legacy ID: 77669v3

All in-vehicle GW connected to internal IP network shall send/receive diagnostic responses with a DoIP header with payload type 8001 as defined in DoIP *[GW_26] ISO 13400-2* with ECU logical addresses as defined in *[GW_1] UDS Services*.

Comment:
See example in Figure: " *Internal IP diagnostic response frame structure".*

4.9.1.6 Gateway connected to CAN

•CAN diagnostic frame format

4.9.1.6.1 CAN diagnostic frame format

This section gives an overview of the CAN frame format and how it should be used in a GEELY vehicle. For more detailed information, see *[GW_6] DoCAN* and *[GW_24] ISO 15765-2*.

For simplicity, only header and data in the frames are showed, i.e. frame trailers containing e.g. CRC checksums etc. are not shown.

Note：
This document covers both the older CAN standard, referred to as classic CAN, and the updated standard CAN Flexible Data-Rate, referred to as CAN FD. Unless explicitly specifying the standard of CAN in this document, any reference to CAN refers to both the classic CAN and CAN FD standards.

4.9.1.6.1.1 Diagnostic Request frame on CAN network

Figure " *CAN diagnostic request frame structure"* shows an overview of the diagnostic request frame format on CAN and which PCI values that are used.

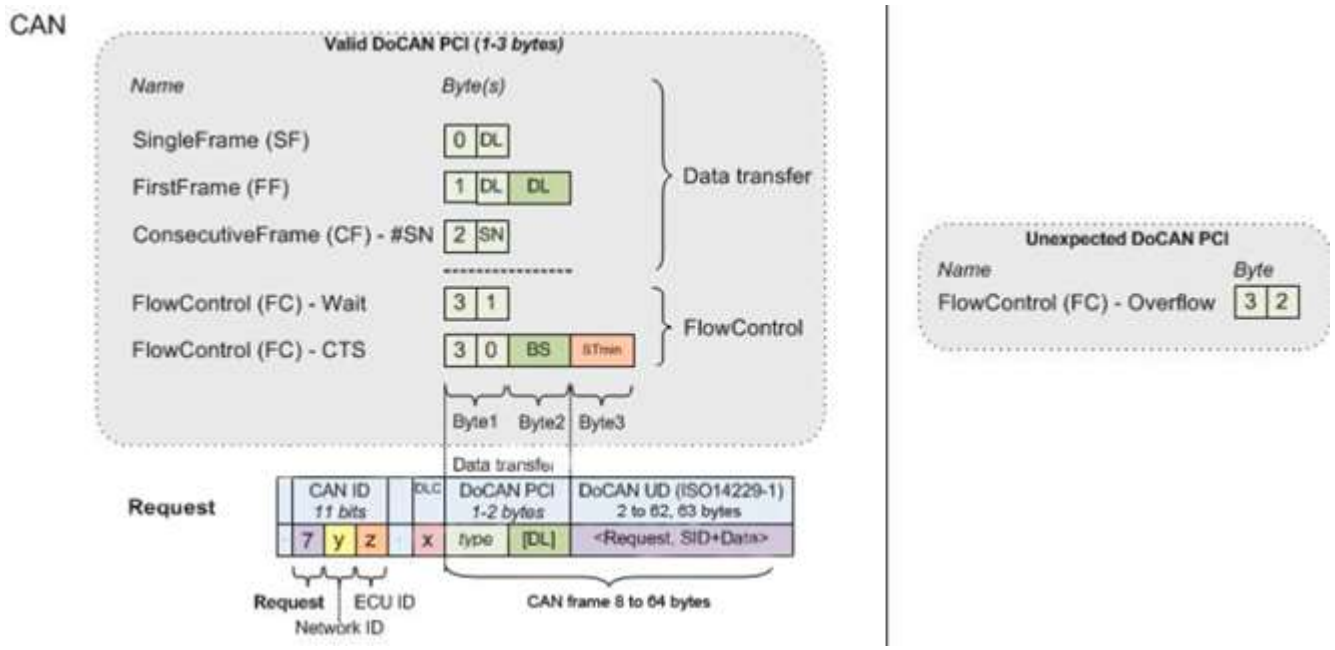


Figure: CAN diagnostic request frame structure

661398 v1 **Diagnostic request format on CAN**

Version history:
(1):

Purpose: Define the request format out from the gateway to a Domain Network (CAN), including the ECU logical address in the frame.

Source:

Verification Method: Test

Legacy ID: 77678v3

All in-vehicle GW connected to CAN network shall send/receive diagnostic request as defined in DoCAN *[GW_24] ISO 15765-2* normal addressing 11 bit CAN ID with ECU logical addresses as defined in *[GW_1] UDS Services*.

Comment:
See example in Figure: " *CAN diagnostic request frame structure".*

4.9.1.6.1.2 Diagnostic Response frame on CAN network

Figure " *CAN diagnostic response frame structure"* shows an overview of the diagnostic response frame format on CAN (for CAN PCI see Figure " *CAN diagnostic request frame structure"*).

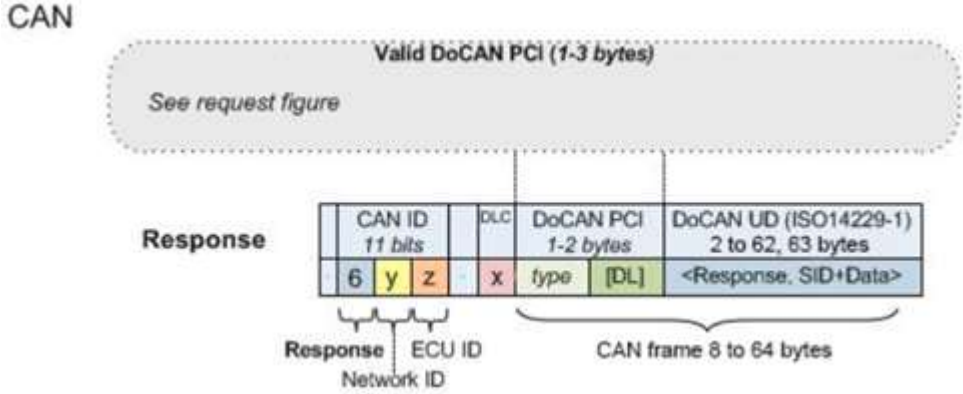


Figure: CAN diagnostic response frame structure

661399 v1 **Diagnostic response format on CAN**

Version history:
(1):

Purpose: Define supported response frame format from a Domain Network (CAN), including ECU logical addresses in the frame, which should transported the Backbone Network (CAN FD).
Source:
Verification Method: Test
Legacy ID: 77679v3

All in-vehicle GW connected to CAN FD network shall send/receive diagnostic response as defined in DoCAN *[GW_24] ISO 15765-2* normal addressing 11 bit CAN ID with ECU logical addresses as defined in *[GW_1] UDS Services.*

Comment:
See example in Figure: " *CAN diagnostic response frame structure"*.

4.9.1.7 Gateway connected to X-LIN

•Conversion from TA to NAD to SA

4.9.1.7.1 Conversion from TA to NAD to SA

661471 v1 **Conversion from TA to NAD to SA**

Version history:
(1):

Purpose: Standard method shall be used for routing IDs in between LIN (i.e. NAD) and the other networks (i.e. logical address).
Source:
Verification Method: Test
Legacy ID: 82161v1

The LIN master shall convert all diagnostic requests from TA logical ECU address to LIN NAD and convert LIN slave NAD to SA logical ECU address, as specified in Table - Network and ECU ID mapping to and from NAD.

Network ID (Hex)	ECU ID (Hex)	NAD (Hex)	Explanation
0x0	0x0		Reserved by LIN
0x0	0x1…0xE	0x01…0xE	Physical NAD address range in network 0
0x1…0x6	0x0…0xE	0x10…0x6E	Physical NAD address ranges in networks 1…6
0x7	0x0…0xD	0x70...0x7D	Physical NAD address range in network 7
0x7	0xE		Reserved by LIN
0x7	0xF		Reserved by LIN
0x8…0xF	0x0…0xE	0x80…0xFE	Physical NAD address ranges in networks 8…F
0xF	0xF	0x7E	Functional address domain or complete vehicle

Table - Network and ECU ID **mapping to and from NAD**

Example:
The Logical address (i.e. TA and SA) 0x1234 (Domain ID= 0x2, Network ID = 0x3 and ECU ID = 0x4). The LIN master will route a diagnostic request to NAD 0x34.
In terms of diagnostic addressing the LIN master shall be transparent in regards of not knowing which LIN slaves there are on a LIN network.

661472 v1 **Physical request NAD validation**

Version history:
(1):

Purpose: The LIN master shall not block physically addressed diagnostic requests.
Source:
Verification Method: Test
Legacy ID: 82301v1

The LIN master shall be transparent in regards of physically addressed diagnostic requests.

•Functional request

4.9.1.7.2 Functional request

661473 v1 **LIN master routing functional requests**

Version history:
(1):

Purpose: The LIN master shall route functional diagnostic requests and since the SPRMIB is placed in the diagnostic request, the LIN slave might as well perform the interpretation of the SPRMIB.
Source:
Verification Method: Test
Legacy ID: 95031v1

The LIN master shall always route functional requests .

661474 v1 **LIN master time to poll time for functional addressed responses**

Version history:
(1):

Purpose: To save bus bandwidth the LIN master shall not poll for responses on diagnostic requests.
Source:
Verification Method: Test
Legacy ID: 95040v1

The LIN master shall consider a functionally addressed diagnostic request completed as soon as the LIN master have sent the request on LIN i.e. the LIN master shall not poll for a diagnostic response (send frame identifier 0x3D) for functionally addressed requests.

661475 v1 **LIN master to prioritize routing functionally addressed requests**

Version history:
(1):

Purpose: The diagnostic request TesterPresent must be prioritized to prevent LIN slaves from timing out.
Source:
Verification Method: Test
Legacy ID: 95361v1

LIN master shall prioritize sending functional requests. Even if a message is being transferred on LIN, the LIN

master shall pause the message transmission (in between two LIN frames), send the functionally addressed request and then continue sending the message.

•LIN master time out

4.9.1.7.3 LIN master time out

661477 v1 **LIN master time out not receiving consecutive 0x78 NRCs**

Version history:
(1):

Purpose: Define max time a LIN master shall wait before the LIN master shall consider a diagnostic request to LIN to be completed, if no subsequent NRC 0x78 (requestCorrectlyReceivedResponsePending) are received.
Source:
Verification Method: Test
Legacy ID: 94607v1

The LIN master shall consider a diagnostic request as timed out if the LIN master does not receive a new NRC 0x78 (requestCorrectlyReceivedResponsePending) 5000ms after sending the previous NRC 0x78 (requestCorrectlyReceivedResponsePending).

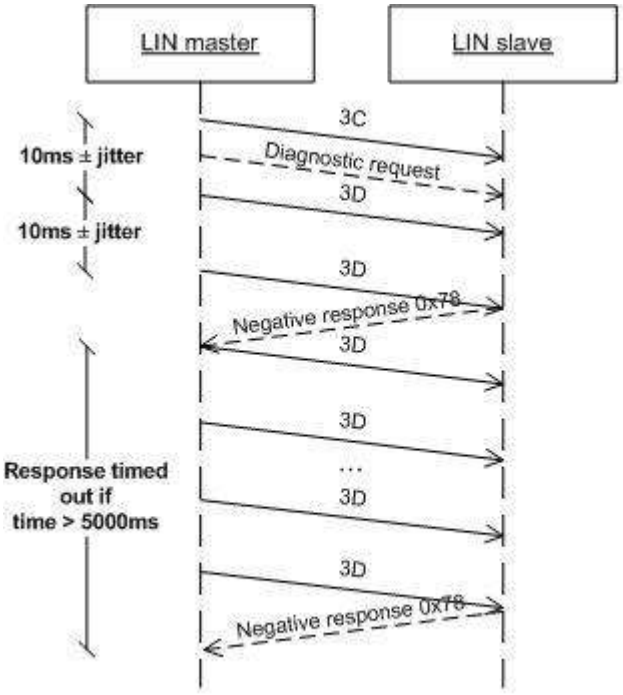


Figure Time out not receiving consecutive 0x78 NRCs

661476 v1 **LIN master time out not receiving first NRC 0x78**

Version history:
(1):

Purpose: Define max time a LIN master shall wait before the LIN master shall consider a diagnostic request to LIN to be completed, if no NRC 0x78 (requestCorrectlyReceivedResponsePending)

is received.

Source:

Verification Method: Test

Legacy ID: 94606v1

The LIN master shall consider a physically addressed diagnostic request as timed out if the LIN master does not receive a 0x78 NRC (requestCorrectlyReceivedResponsePending) 100ms after the diagnostic request was sent. For functionally addressed diagnostic request the time shall be 0 (i.e. the LIN master shall continue sending next diagnostic request immediately after the functionally addressed diagnostic request).

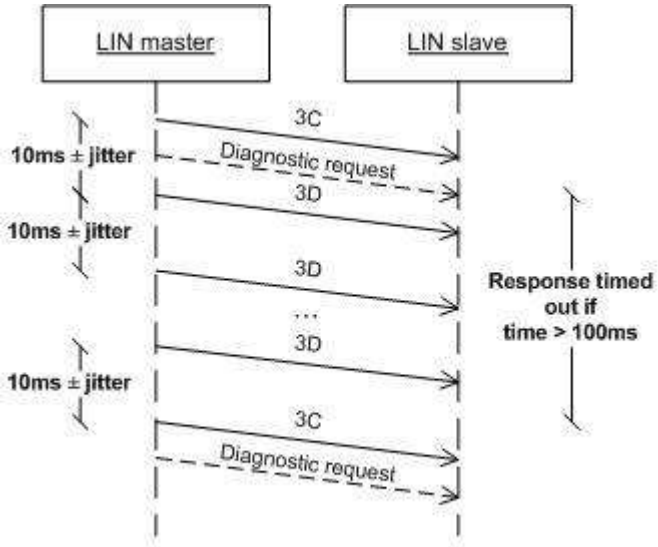


Figure Time out not receiving first 0x78 NRC

•Parallel requests

4.9.1.7.4 Parallel requests

661479 v1 **Concurrent LIN master and LIN slave diagnostic requests**

Version history:
(1):

Purpose: Speed up communication between tester and vehicle, by sending request to several networks and ECU in parallel/concurrent.

Source:

Verification Method: Test

Legacy ID: 372704v4

The LIN master shall concurrent be able to receive and process diagnostic requests addressed to the LIN master itself during gateway to/from LIN slaves. This means the LIN master shall be able to be re-programmed during the same time as the LIN slave.

661478 v1 **Parallel requests on LIN**

Version history:
(1):

Purpose: The tester shall not need to consider limitations on LIN.

Source:

Verification Method: Test
Legacy ID: 82084v3

The LIN master shall be able to handle parallel physical addressed diagnostic requests to a LIN network.
On one LIN network there shall only be one ongoing physical address diagnostic request, the LIN master needs to wait until response is received or time out before next request is sent on the LIN network.

Rationale:
The LIN master shall serialize incomming parallel request to one LIN network. Next request(s) need be stored in buffer(s) in the LIN master until previous message is completed then take next parallel request in the queue and send on LIN etc.
Similar behaviour is also valid for queued request in bootloader, i.e. a physical addressed diagnostic request on LIN shall always be completed before next physical addressed diagnostic request can be sent.

4.9.1.8 Vehicle Gateway - IP-IP

•Requirements

4.9.1.8.1 Requirements

The IP based diagnostic communication shall follow the requirements specified in *[GW_13] IP Routing Specification* and *[GW_14] Internetworks General Specification*.

4.9.1.8.1.1 Translation of diagnostic messages

4.9.1.8.1.1.1 Diagnostic request from external IP network to internal IP network

Internal IP addresses are not visible outside the vehicle, and an external test tool cannot directly address an ECU behind the Vehicle GW (Edge Node).

The vehicle GW should replace the external IP address with an internal IP address for the internal IP network. The target address in the DoIP header shall be mapped to an internal IP TA, three lowest nibbles shall match, and the Vehicle GW internal IP address should be used as IP SA.

See example in Figure " *Diagnostic request from external IP to internal IP"*.

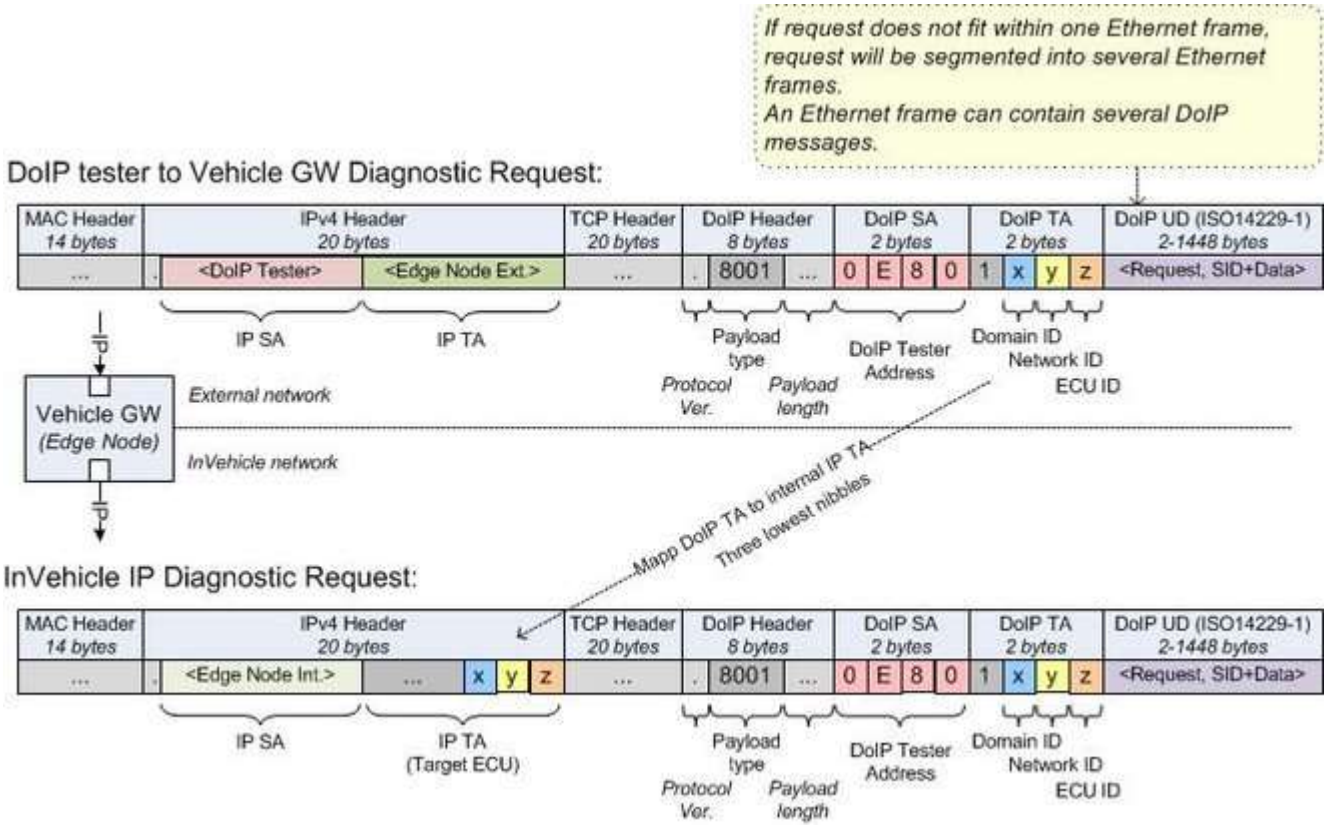


Figure: Diagnostic request from external IP to internal IP

661400 v1	**External IP format to internal IP format in request**

Version history:
(1):

Purpose: Define the change from an external IP request to an internal IP request, using the DoIP header information for set up of a internal IP TA in diagnostic request and using the Vehicle GW (Edge Node) as an internal IP SA.

Source:

Verification Method: Test

Legacy ID: 77645v2

If a request is received from an external IP network that should be transported to an internal IP network, the Vehicle GW (Edge Node) shall translate the DoIP target address (DoIP TA) to an internal IP target address (IP TA) as specified by *[GW_14] Internetworks General Specification* and use the Vehicle GW internal IP address as the internal IP target address (IP SA), format on frames as defined in DoIP *[GW_26] ISO 13400-2*.

Comment:
See example in Figure: " *Diagnostic request from external IP to internal IP"*.

4.9.1.8.1.1.2 Diagnostic response from internal IP network to external IP network

Before the response is sent out to an external IP network, the internal IP addresses will be replaced with the external IP addresses, DoIP tester address and the Vehicle GW (Edge Node) external IP address.

See example in Figure " *Diagnostic response from internal IP to external IP"*.

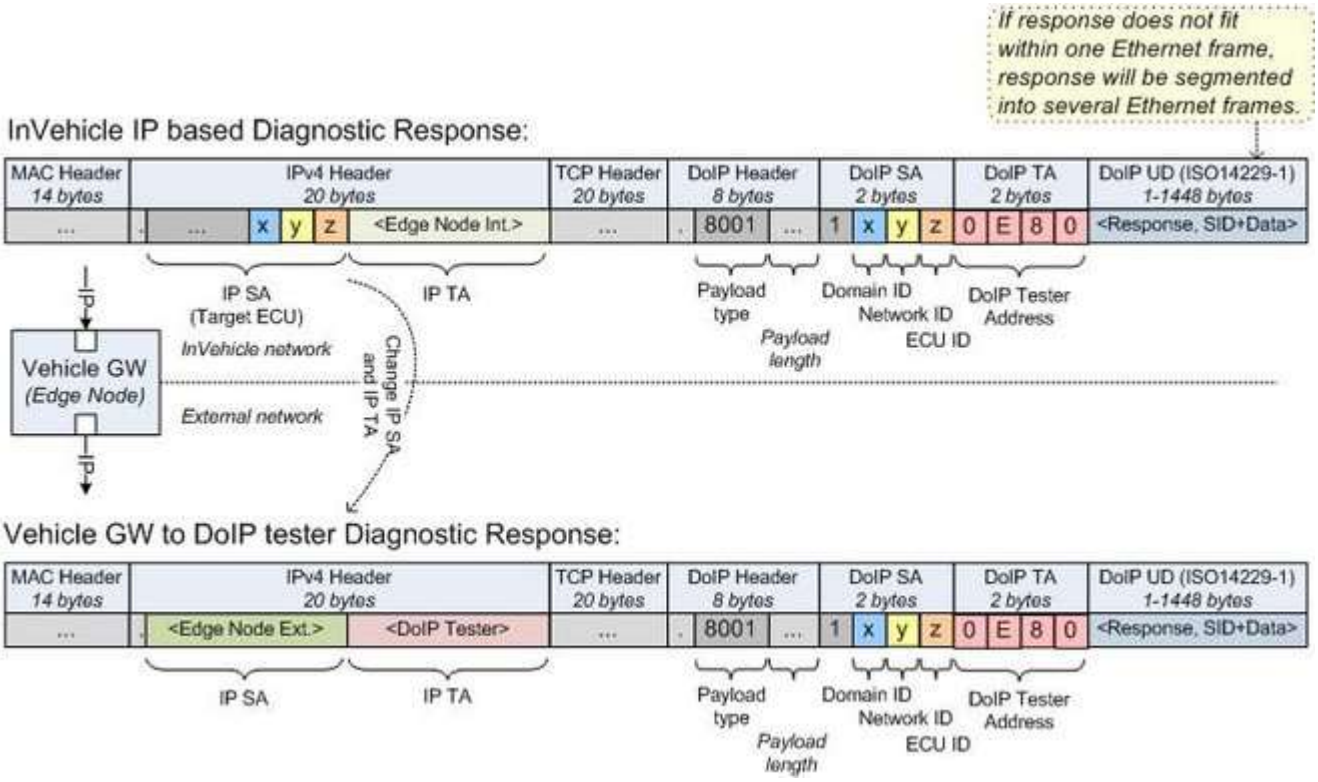


Figure: Diagnostic response from internal IP to external IP

661401 v1	**Internal IP to external IP address in response**

Version history:
(1):

Purpose: Define the change from an internal IP address in response to an external IP address in response.
Source:
Verification Method: Test
Legacy ID: 77647v2

If a response is received from an internal IP network that should be transported to the external IP network, the Vehicle GW (Edge Node) shall change the internal IP source address to the Vehicle GW external IP address (IP SA) and change the Vehicle GW internal IP address to the tester tool IP address (IP TA), IP address that was allocated during establishing connection between external tester and vehicle.

Rationale:
Internal IP addresses are not visible outside the vehicle.

Comment:
See example in Figure: " *Diagnostic response from internal IP to external IP".*

4.9.1.8.1.1.3 Functional request External IP to Internal IP

[GW_12] DoIP specifies how TCP connections shall be handled for internal IP network(s), e.g. how an ECU should initiate a TCP connection, re-connection of TCP connection, etc.

All diagnostic communication on IP, Ethernet or Ethernet over MOST 150, will utilize established TCP connections, see *[GW_10] UDSonIP* and *[GW_11] Bootloader - UDSonIP*.

All IP nodes within the vehicle will initiate and open TCP connections for diagnostic communication to/from the Vehicle GW(Edge Node).
See example in Figure *IP - IP Diagnostic Gateway via TCP Connection*, Domain Network GW (ECU K), Target ECUs (ECU L and ECU M) will initiate and open TCP connections to Vehicle GW (ECU J, Edge Node).

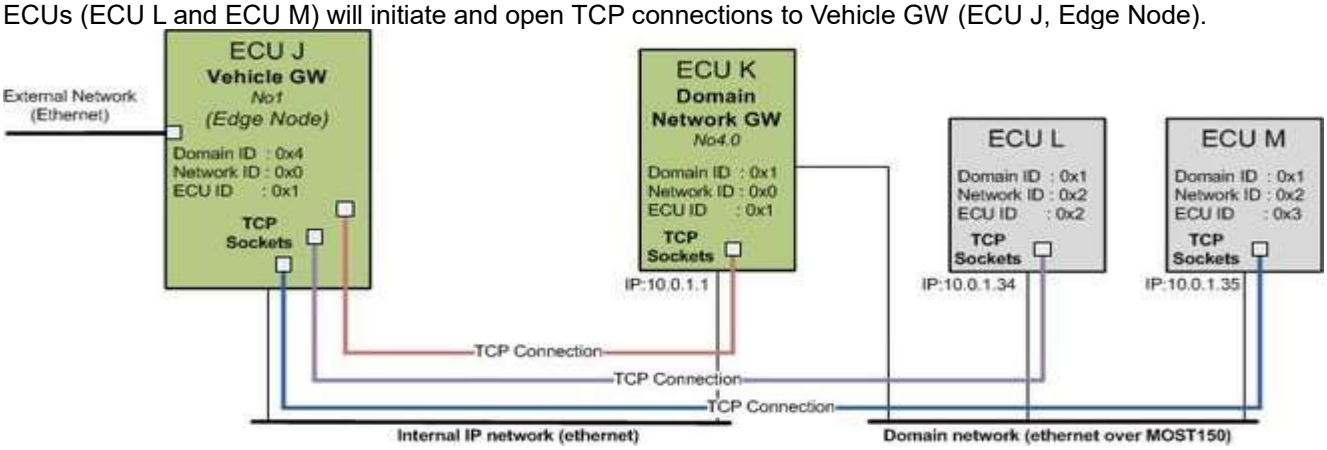


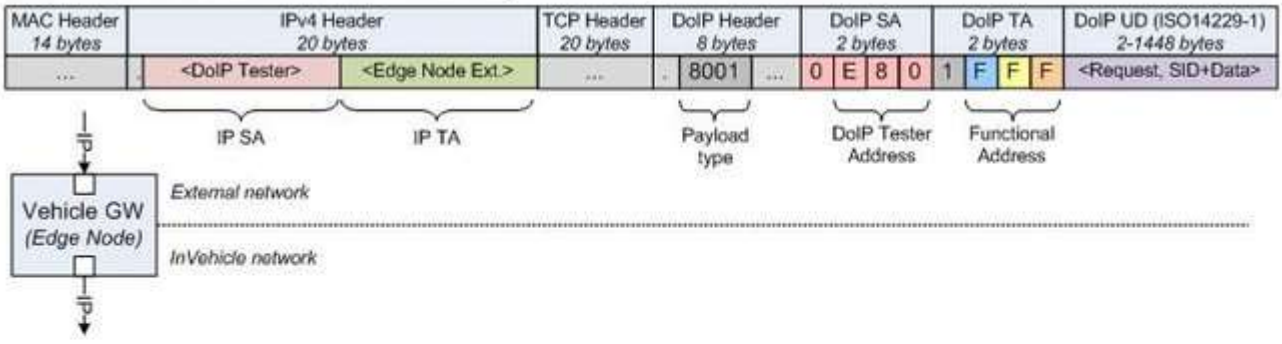
Figure: IP - IP Diagnostic Gateway via TCP Connection

Note: IP addresses in figure above only example to show the relation between 3 lowest nibbles in ECU address and IP address.

For in vehicle diagnostic communication, diagnostic request and response messages with DoIP payload type 8001 and messages according to *[GW_19] ISO 14229-1*, the DoIP header will be kept. The target address from the DoIP header will be set as an IP target address, see Figure: " *Internal IP diagnostic request frame structure"*.

For functional request, for example DoIP request to all ECUs in the vehicle with DoIP TA 0x1FFF, the Vehicle GW (Edge Node) will send the request to all TCP Connections. I.e. in this case the DoIP TA will result in several IP TA for all established TCP connections, see example in Figure " *IP - IP Functional Diagnostic requests via TCP Connection"*.

DoIP tester to Vehicle GW Functional Diagnostic Request:



InVehicle IP Diagnostic Functional Requests, sent out on each TCP Connection:

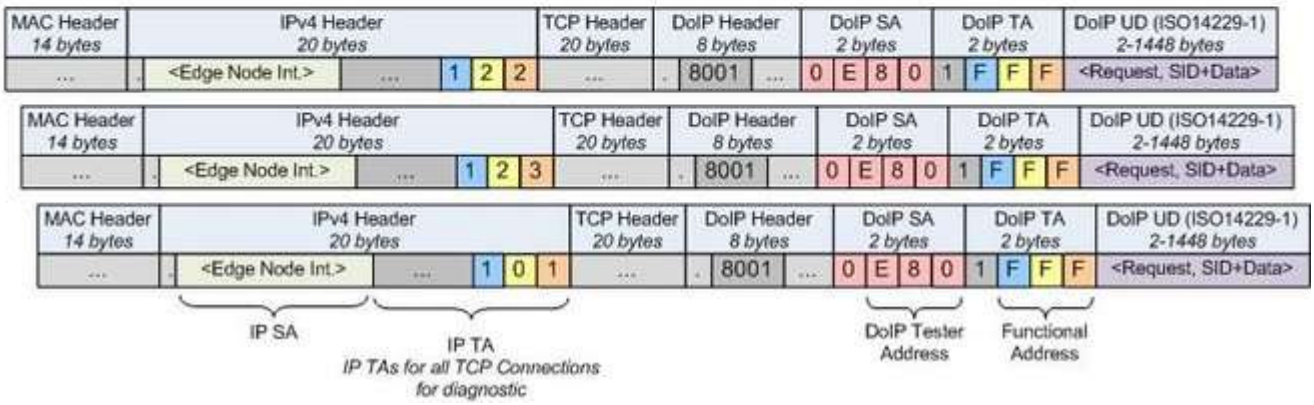


Figure: IP - IP Functional Diagnostic requests via TCP Connection

The IP based diagnostic communication shall follow the requirements specified in *[GW_13] IP Routing Specification* and *[GW_14] Internetworks General Specification*.

661402 v1 **Handling functional requests on IP**

Version history:
(1):

Purpose: Define how a functional request shall be handled on an IP based network. TCP is also used for functional request on IP, and a request can not be broadcasted to a TCP connection reaching all IP based target ECUs. The Vehicle GW need to send the functional request on all established TCP connections.
Source:
Verification Method: Test
Legacy ID: 77666v2

If the Vehicle GW (Edge Node) receives a functional request to all IP based target ECUs, the Vehicle GW shall send the request thru all established internal TCP connections, with functional address kept in the DoIP header target address (DoIP TA) but with different IP target addresses (IP TA). The request(s) to gateway(s) shall be sent last, with the gateway closest to the Vehicle GW last of all.

Comment:
If the functional request would lead to a reset of an in-vehicle GW, the request should be sent last so that the gateway does not perform the reset before all request to other ECUs is transported.
See example in Figure: " *IP – IP Functional Diagnostic requests via TCP Connection"*.

4.9.1.9 Vehicle Gateway - IP-CAN(Network)

•Translation of diagnostic messages

4.9.1.9.1 Translation of diagnostic messages

4.9.1.9.1.1 Diagnostic request from external IP network to CAN network

For gateway of request from external IP to CAN, The target address in the DoIP header (DoIP TA) should be translate the 11 bit CAN ID. The bit 10~8 of CAN ID set as 0x7 and bit 7-0 containing Network ID and ECU ID to part of the DoIP header (DoIP TA) including the Network ID and ECU ID.

See example in Figure "*Diagnostic request from external IP to CAN"*.

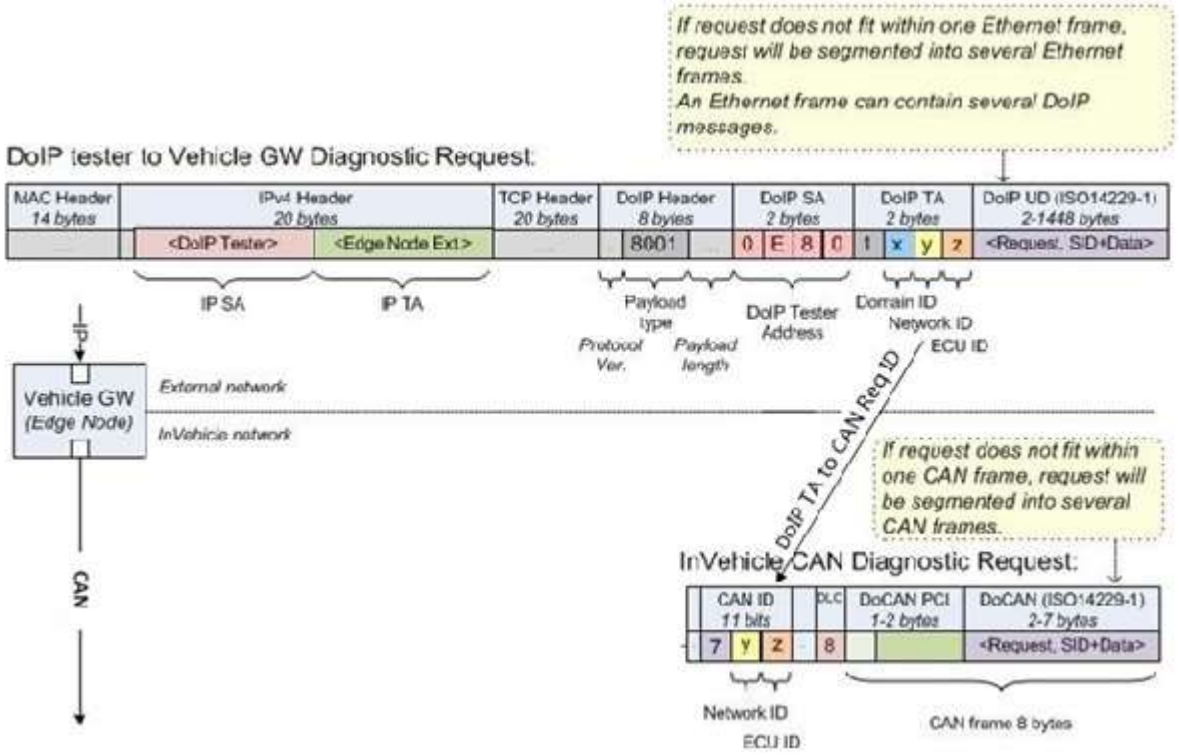


Figure: Diagnostic request from external IP to CAN

665099 v1 **External IP format to CAN format in request**

Version history:
(1):

Purpose: Define the change in format from an external IP request to a CAN request, using the DoIP header information to set up the CAN request identifier (CAN Req ID).

Source:

Verification Method: Test

Legacy ID: 118094

If a request is received from the external IP network that should be transported to the CAN Network, the Vehicle GW (Edge Node) shall translate the DoIP target address (DoIP TA) to the 11 bit CAN request identifier (CAN Req ID), format on frames as defined in DoIP [*ISO 13400-2*] and in DoCAN [*ISO 15765-2*].

Comment:
See example in Figure: "*Diagnostic request from external IP to CAN*".

4.9.1.9.1.2 Diagnostic response from CAN network to external IP network

This section will describe CAN format to external IP format in response and CAN response add IP address in extern.

See example in Figure *Diagnostic response from CAN to external IP*.

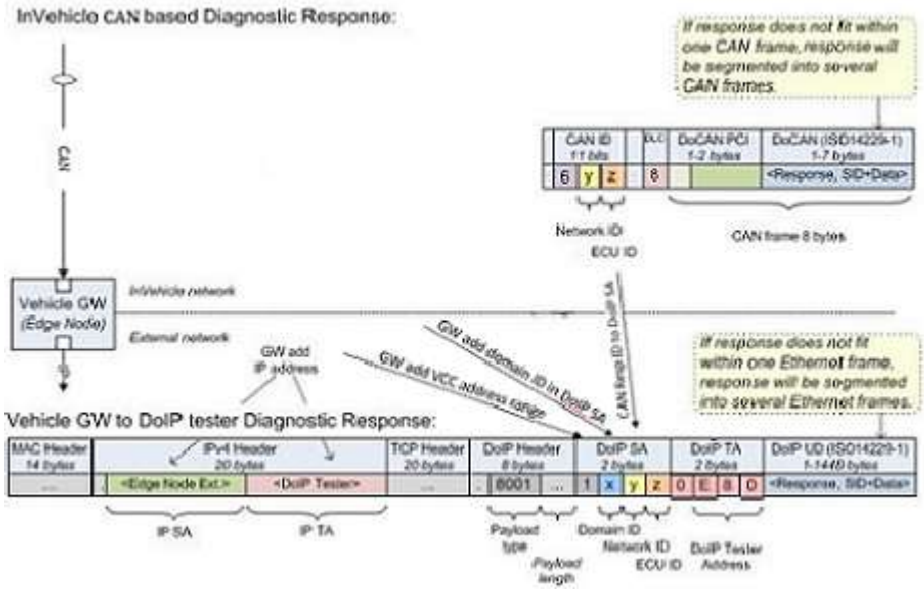


Figure: Diagnostic response from CAN to external IP

661404 v1 — CAN format to external IP format in response

Version history:
(1):

Purpose: Define the change from CAN format to DoIP format in the response, CAN response identifier to DoIP header source address.
Source:
Verification Method: Test
Legacy ID: 118102

If a response is received from the CAN network that should be transported to the external IP network, the Vehicle GW (Edge Node) shall translate the CAN response identifier (CAN Resp ID) to a DoIP source address (DoIP SA).

Comment:
See example Figure: "*Diagnostic response from CAN to external IP*".

661405 v1 — CAN response add IP address in external IP

Version history:
(1):

Purpose: External IP address must be added in a response from sub Network (CAN) to external IP network.
Source:
Verification Method: Test
Legacy ID: 118105

If a response is received from the sub Network (CAN) that should be transported to the external IP network, the Vehicle GW (Edge Node) shall add the Vehicle GW external IP address (IP SA) and add external test tool IP address (IP TA), IP address that was allocated during establishing connection between external tester and vehicle.

Comment:
See example in Figure: "*Diagnostic response from CAN to external IP"*.

•Gateway allocation of communication channels

4.9.1.9.2 Gateway allocation of communication channels

This section describes the allocation of communication channels for physical request and responses.

Note: Functional requests, including TesterPresent, should not be delayed in the gateway due to e.g. queued or parallel requests in the gateway. Functional requests need to have separate communication channel, so that TesterPresent can be sent though the gateways and keep ECUs in non-default session.

4.9.1.9.2.1 Network (CAN(FD)) - Client side

This section will describe and give grounds for the following requirements:

- REQPROD 661406 Transmit(Tx) CCs on Network
- REQPROD 661407 Receive(Rx) CCs on Network

The in-vehicle gateway should not do any re-sending of messages, e.g. if time-out due to no flow control frames is received.
It should always be guaranteed when allocating a transmit (Tx) communication channel (CC) on the sender side, the receiver side should be able to allocate a receive (Rx) communication channel (CC), so that no data (first frame) will be lost.

There is no handshake mechanism when allocating a Tx CC on the sender side with the allocation of an Rx CC on the receiver side, hence it must always be assured that a Rx CC can be allocated to receive at least the first frame so that no data will be lost.

Allocation of transmit (Tx) communication channels (CCs)
Vehicle GW for requests to Network (CAN(FD)):

- The Vehicle Network GW should have six(6) communication channels (CCs) for sending physical request to the ECUs on each Backbone Network (CANFD).
- The Vehicle Network GW should have three(3) communication channels (CCs) for sending physical request to the ECUs on each Sub Network (CAN).

Allocation of receive (Rx) communication channels (CCs)
Vehicle GW for responses from Network (CAN(FD)):

- The Vehicle Network GW should have same number of receive (Rx) communication channels (CCs) as transmit (Tx) communication channels (CCs) on the Network (Note: The number of Rx CCs are project specific, e.g. same number as ECUs connected on the Network).
- The Vehicle GW must be able to receive and handle first frame from all ECUs.
- If there is no buffer in the buffer pool available for receiving the complete response on the CC, the Vehicle GW should send flow control WAIT until a buffer that fits the response is available. When a buffer is available, the Vehicle GW should send a flow control CTS for receiving the remaining part of the response.

Note: When dimension the system with number of supported communication channels and number of messages that can be allocated from the buffer pool, it must be assured that flow control WAIT frames can be sent long enough time so that the sending side does not time-out.

See Figure: "Allocation communication channels (CCs) for physical request on Backbone Network (CANFD) - Client side".

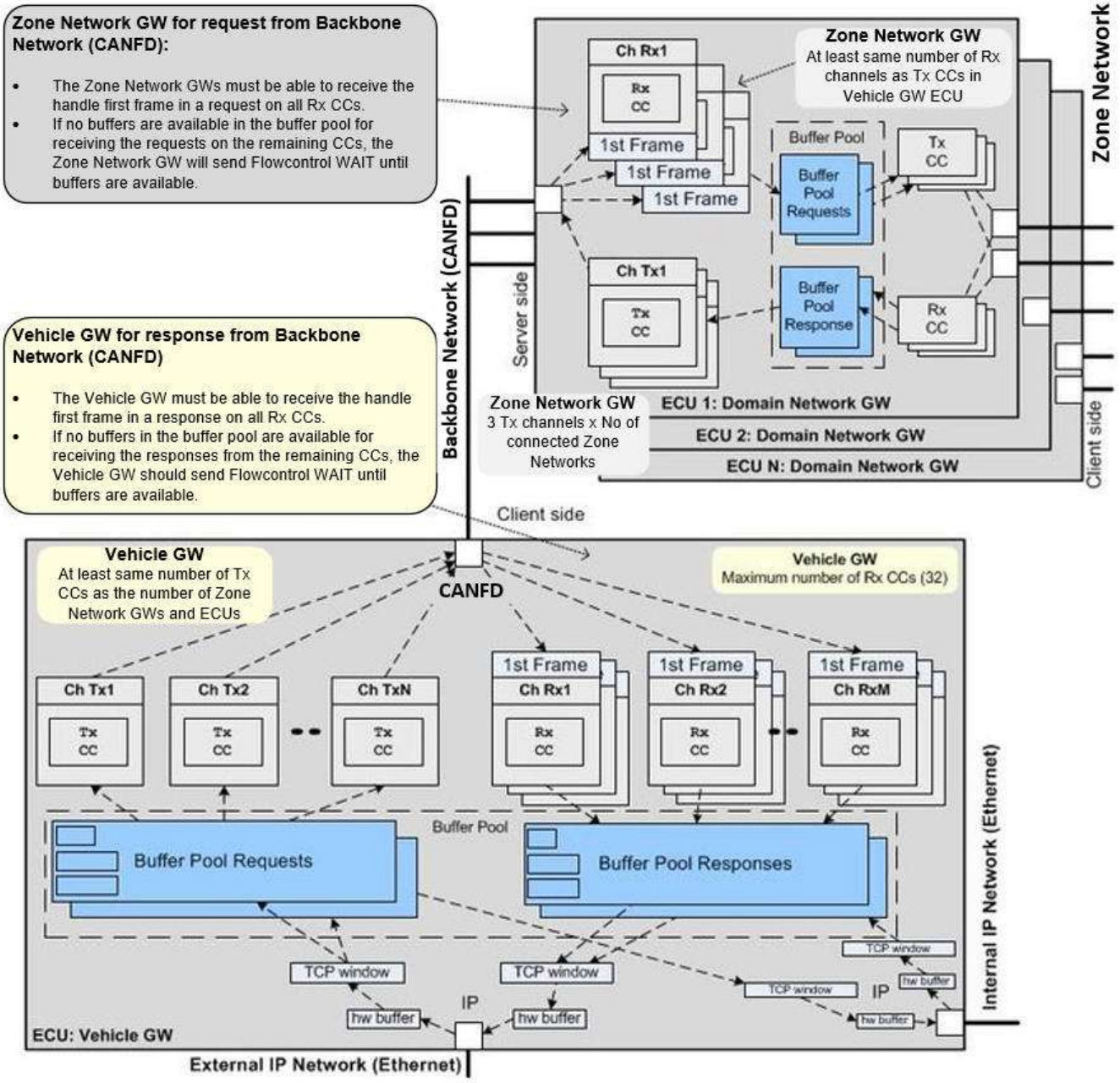


Figure: Allocation communication channels (CCs) for physical request on Backbone Network (CANFD) - Client side

661407 v3 **Receive(Rx) CCs on Network (CAN(FD))**

Version history:
(3):

Purpose: Ensure that no diagnostic responses are dropped, a first frame shall always be stored, and dependent on available buffers flow control CTS or WAIT should be sent.
Source:
Verification Method: Test
Legacy ID:

The Vehicle GW connect to the Network (CAN(FD)) shall be able to open as many receive (Rx) communication channels (CCs) for receiving diagnostic responses as the total number of ECUs on the network connected to Vehicle GW and subnetwork connected to Zone GW.

Comment:
The number of Rx CCs are project specific, and at least 32 (Rx) communication channels (CCs).
The number of Rx CCs will be the number of ECUs on the network, since the requirement on Tx communication channels on a ECU is one (1).

661406 v4 **Transmit(Tx) CCs on Network (CAN(FD))**

Version history:
(4): Tx communication channels change from 5 to 6, matching the number of OTA parallels.

Purpose: It should be possible to send request as soon as possible so that timing requirement for reading information from ECUs can be fulfilled
Source:
Verification Method: Test
Legacy ID:

For each Backbone Network (CANFD) behind the GW: The Vehicle GW shall be able to open six(6) transmit (Tx) communication channels (CCs) for sending diagnostic physical requests to the ECUs.

For each Sub Network (CAN) behind the GW: The Vehicle GW shall be able to open three(3) transmit (Tx) communication channels (CCs) for sending diagnostic physical requests to the ECUs.

For each network behind the GW: The Vehicle GW shall be able to open one additional transmit (Tx) communication channel (CC) for functional requests.

The ECU supplier shall confirm with OEM, the exact number of communication channels for the specific project.

•Gateway latency requirements

4.9.1.9.3 Gateway latency requirements

4.9.1.9.3.1 IP - CAN diagnostic messages

[*DoCAN*] specifies requirement on transport protocol level as e.g. maximum time between c consecutive frames, C_Cs, performance requirements.

4.9.1.9.3.1.1 Diagnostic requests IP to CAN

The IP to CAN Vehicle GW shall receive and validate the complete request from IP before the request is transmitted on the Sub network (CAN), as specified in [*ISO 13400-2*]. See example in Figure: "*Vehicle GW: IP to CAN request*".

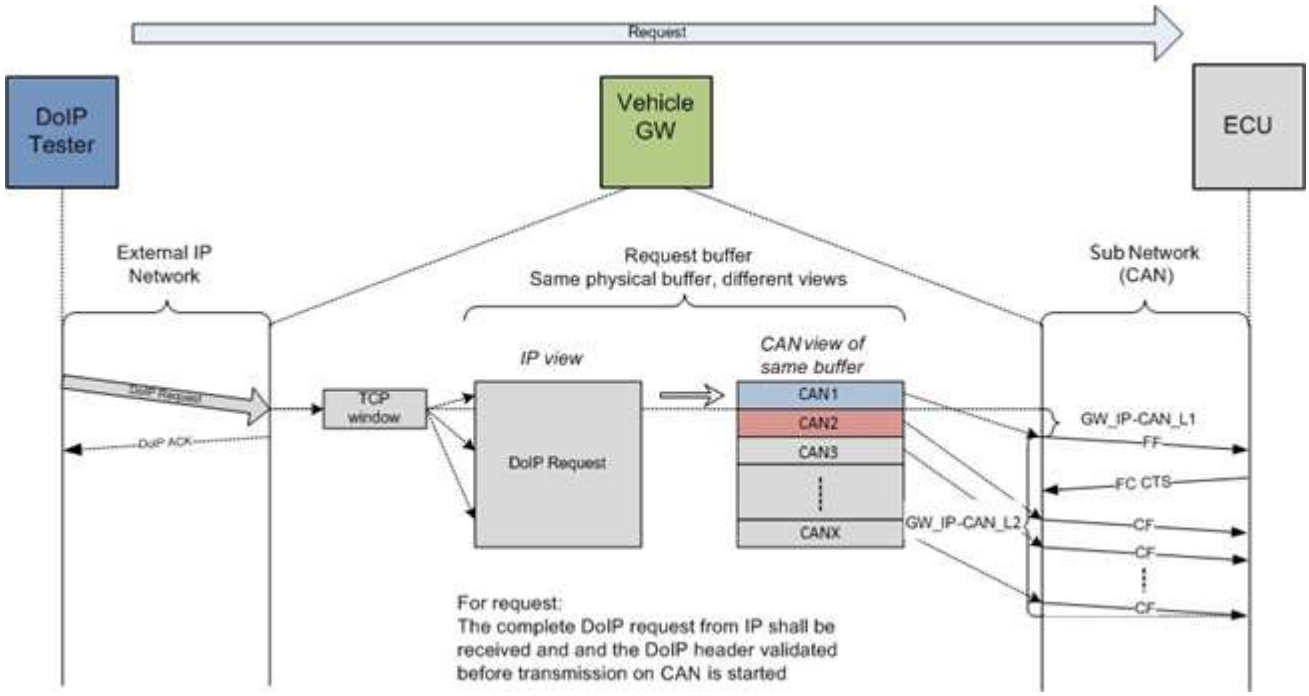


Figure: Vehicle GW: IP to CAN request

Definition of latency timing parameters:
GW_IP-CAN_L1: Time between receiving the complete request from IP until first frame is sent out on Sub Network (CAN).

GW_IP-CAN_L2: Time for transfer the complete request on Sub Network (CAN).
This time depends on:

- Size of the message, how the message is packed on the Sub Network (CAN), i.e. how many frames that shall be sent.
- Timing parameters for the Sub Network (CAN) is specified in [*DoCAN*].

Flow control WAIT can only be sent a limited time, so GW_IP-CAN_L2 has impact on how the system shall be dimension regarding parallel communication channels and how many messages that can be received without sending Flow control WAIT frames.

Timing Parameter	SWDL	DIAG
	Max Value	Max Value
GW_IP-CAN_L1	10 ms [1)]	10 ms [2)]
GW_IP-CAN_L2	See [1*] and Formula(1)	See [1*] and Formula(1)
Formula(1): · unsegmented messages: C_As · segmented messages: C_As + C_Bs + ((Message size) – (L-SDU size))/(L-SDU size) * C_Cr		

Table: Vehicle GW: IP to CAN request timing parameters

*: [*DoCAN*]

661408 v1 **Latency IP to CAN, GW_IP-CAN_L1**

Version history:
(1):

Purpose: Clarify and define a maximum latency time.
Source:
Verification Method: Test
Legacy ID:

If a communication channel can be allocated, the time between receiving the complete request from IP/Ethernet until first frame is sent out on CAN shall be max 10ms.

4.9.1.9.3.1.2 Diagnostic responses CAN to IP

The CAN to IP Vehicle GW shall receive the complete response on CAN before response is transmitted on the IP network.

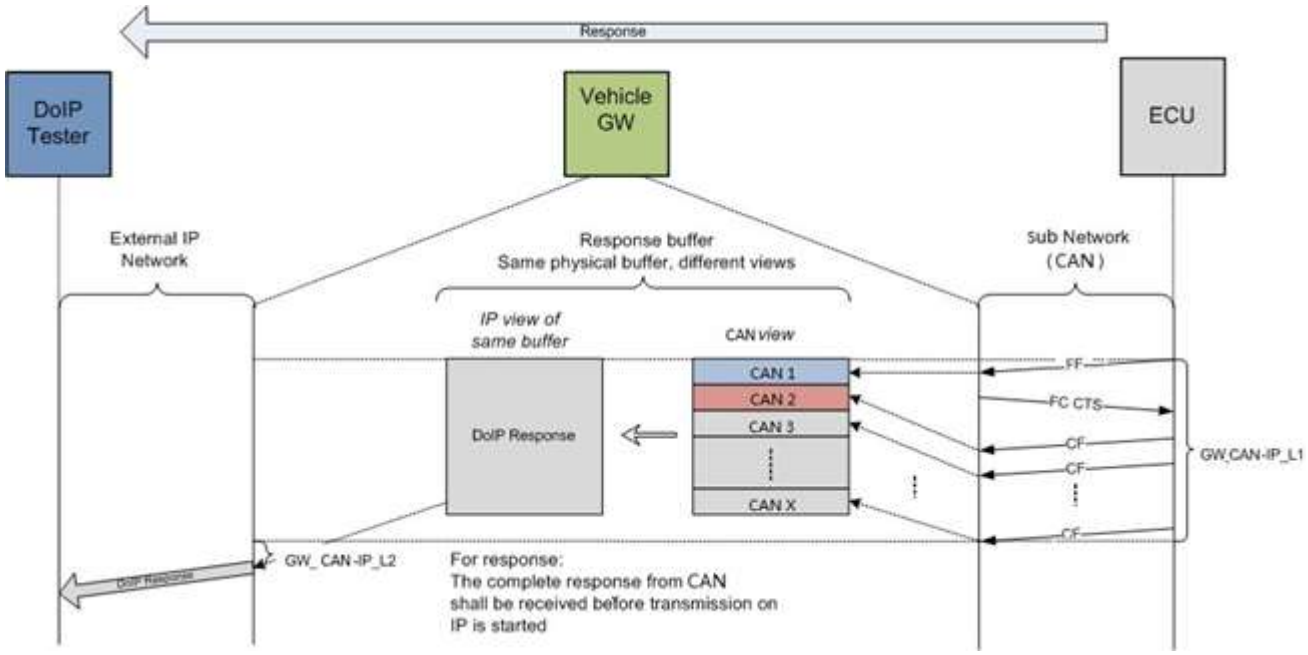


Figure: Vehicle GW: CAN to IP response

Definition of latency timing parameters:
GW_CAN-IP_L1: Time for transfer the complete response on Sub Network (CAN).
This time depends on:

- Size of the message, how the message is packed on the Sub Network (CAN), i.e. how many frames that shall be sent.
- Timing parameters for the Sub Network (CAN) specified in [*DoCAN*].

GW_CAN_IP_L2: Time between receiving complete response from Sub Network (CAN) until DoIP response shall be triggered on the IP Network.

Flow control WAIT can only be sent a limited time, so GW_CAN_IP_L1 has impact on how the system shall dimension regarding parallel communication channels and how many messages that can be received without sending Flow control WAIT frames.

Timing Parameter	SWDL	DIAG
	Max Value	Max Value
GW_CAN_IP_L1	See [*] and Formula(1)	See [*] and Formula(1)
GW_CAN_IP_L2	5 ms	5 ms
Formula(1): · unsegmented messages: C_As		

·	segmented messages: C_As + C_Bs + ((Message size) – (L-SDU size))/(L-SDU size) * C_Cr

Table: Vehicle GW: FlexRay to IP response timing parameters

*: [*DoCAN*]

661409 v1	**Latency CAN to IP, GW_CAN-IP_L2**

Version history:
(1):

Purpose: Clarify and define a maximum latency time.
Source:
Verification Method: Test
Legacy ID:

If a communication channel can be allocated, time between receiving complete response from CAN network until DoIP response shall be triggered shall be max 5ms.

4.9.1.10 Vehicle Gateway DIAG

•Gateway allocation of buffers for DIAG

4.9.1.10.1 Gateway allocation of buffers for DIAG

This section will describe and give grounds for the following requirements:

- ReqID 661414 Minimum RAM for request buffers
- ReqID 666416 Minimum number of buffers for requests
- ReqID 661410 Large Buffers for request
- ReqID 661415 Minimum RAM for response buffers
- ReqID 661413 Minimum number of buffers for responses
- ReqID 661411 Large Buffers for response

Available memory for allocation of buffers for DIAG messages (UDS Layer) are a compromise between hardware cost and performance, e.g. how much traffic in parallel the gateway should be able to handle.
For DIAG messages, the Vehicle GW will execute in application mode, and have to share resources as RAM for buffers with other applications.

For each network behind the GW: The Vehicle GW should be able to allocate at least CANFD: 8kB or CAN: 4kB, or Ethernet: 8kB RAM reserved for buffers for requests, and at least CANFD:32 KB or CAN: 8kB or Ethernet: 64kB RAM reserved for buffers for responses.

The Vehicle GW must be able to allocate buffer for the largest possible request, CANFD or CAN: 4kB or Ethernet: 8kB, and CANFD or CAN: 4kB or Ethernet: 16kB for response.

There should be a minimum number of available buffers so that the gateway can handle parallel communication, as long as the total size for buffers is not exceeded. If there is a small request or response the smallest possible buffer should be used, e.g. do not allocate an 4 kB buffer for a 2 bytes long request. This specification does not specify exactly which buffer allocation algorithm that should be used, this is up to the implementer, but there should be a minimum of available buffers for small requests.

To insure that parallel DIAG can be performed for small messages (dynamic sized buffer allocation):

- For each network behind the GW: If the total size of requests are less or equal to CANFD:8kB or CAN: 4kB RAM, the minimum number for available request buffers should be CANFD: 80 or CAN: (6~8).
- For each network behind the GW: If the total size of responses are less or equal to CANFD: 32 KB or CAN: 8kB RAM, the minimum number for available response buffers should be CANFD: 80 or CAN: (6~8).

See example in Figure: "*Vehicle GW Buffers for Internal IP and Backbone Network (CAN FD) DIAG*".

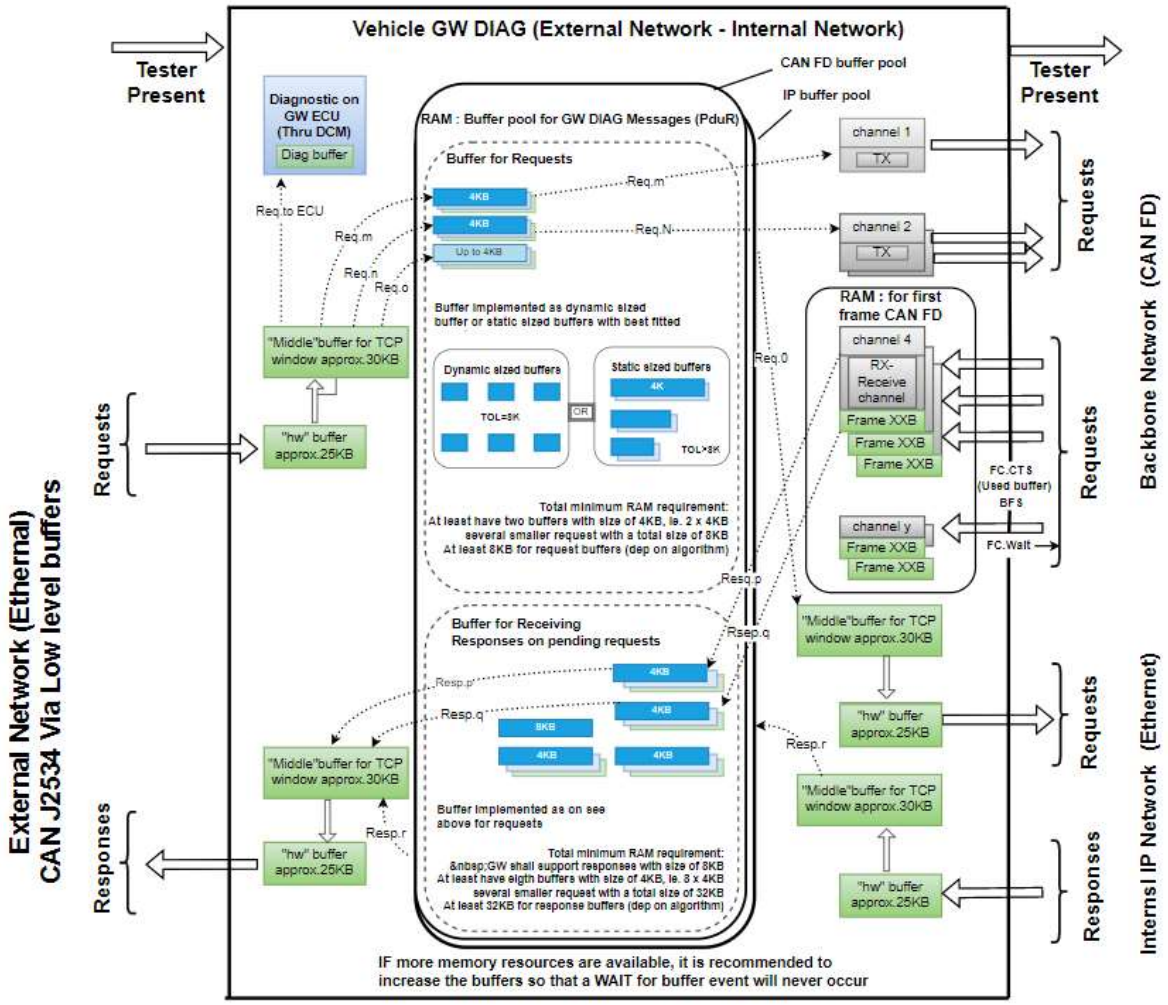


Figure: Vehicle GW Buffers for Internal IP and Backbone Network (CAN FD) DIAG

661410 v2 **Large buffers for request**

Version history:
(2):

Purpose: Set a minimum requirement for available memory for requests and response. To ensure a minimum performance requirement for parallel communication.

Source:

Verification Method: Test

Legacy ID: 77453v4

For each network behind the GW: In application mode, the Vehicle GW shall be able to allocate up to CANFD or CAN: 4 kB or Ethernet: 8kB large buffers for request.

661411 v4 **Large buffers for response**

Version history:
(4): 适配法规EDR数据（超过4kB）的读取，增加最大响应Buffer分配。

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum

performance requirement for parallel communication.

Source:

Verification Method: Test

Legacy ID: 77456v4

For each network behind the GW: In application mode, the Vehicle GW shall be able to allocate up to CANFD or CAN: 8kB or Ethernet: 16kB large buffers for response.

Note: 8 kB for 1 UDS response read out of EDR data.

661414 v3 Minimum RAM for request buffers

Version history:
(3):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication.

Source:

Verification Method: Test

Legacy ID: 77451v4

For each network behind the GW: In application mode, the Vehicle GW shall be able to reserve at least a CANFD: 8kB or a CAN: 4kB or Ethernet: 8kB RAM buffer pool only for requests where several smaller buffers can be allocated from.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for requests.
If using static sized message buffers and/or not possible to reserve buffer pool only for requests: OEM and the ECU supplier shall come to an agreement on used buffer sizes to achieve similar performance, more memory for buffers are needed.

Note:
There is a separate requirement for minimum number of buffers that can be allocated form this buffer pool.

Example (dynamic sized message buffers and 4 KB buffer pool):
A) One network behind the gateway: 4 KB RAM reserved for requests (with possibility to allocate different buffer sizes).
B) Two networks behind the gateway: Implies requirement of 2 x 4 KB RAM reserved for requests.

Example (static sized message buffers):
A) Two messages with size < 4 KB received, but uses large 4 KB buffers, 2 x 4 KB = 8 KB, more memory than the minimum requirement are needed to be able to fulfil the requirement for number of buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers"*. For static sized message buffers it can be larger gaps in the usable memory, hence this minimum RAM requirement may need to be increased dependent on how the static sized message buffers are chosen.

661415 v3 Minimum RAM for response buffers

Version history:
(3):

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication.

Source:

Verification Method: Test

Legacy ID: 77454v4

For each network behind the GW: In application mode, the Vehicle GW shall be able to reserve at least a CANFD: 32kB or a CAN: 8KB or Ethernet: 64kB RAM buffer pool only for responses where several smaller buffers can be allocated from.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for responses.
If using static sized message buffers and/or not possible to reserve buffer pool only for responses: OEM and the ECU supplier shall come to an agreement on used buffer sizes to achieve similar performance, more memory for buffers are needed.

Note:
There is a separate requirement for minimum number of buffers that can be allocated form this buffer pool.

Example (dynamic sized message buffers and 32 KB buffer pool):
A) One network behind the gateway: 32 KB RAM reserved for responses (with possibility to allocate different buffer sizes).
B) Two networks behind the gateway: Implies requirement of 2 x 32 KB RAM reserved for responses.

Example (static sized message buffers):
A) Four messages with size < 4 KB received, but uses large 4 KB buffers, 8 x 4 KB = 32 KB, more memory than the minimum requirement are needed to be able to fulfil the requirement for number of buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers"*. For static sized message buffers it can be larger gaps in the usable memory, hence this minimum RAM requirement may need to be increased dependent on how the static sized message buffers are chosen.

666416 v4 Minimum number of buffers for requests

Version history:
(4): Minimum number of buffers for requests on CANFD: change from 80 to 40.

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication.
Source:
Verification Method: Test
Legacy ID: 77452v3

For each network behind the GW: In application mode and if the total size of all current requests are less or equal to CANFD: 8kB or CAN: 4KB RAM, the Vehicle GW shall be able to allocate at least CANFD: 40 or CAN: (6~8) buffers for requests.
Note: Ethernet is a connection oriented protocol. There are sockets which need to be established and behave as a full duplex channel for a port and IP pair.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for requests.

If using static sized message buffers and/or not possible to reserve buffer pool only for requests: OEM and the ECU supplier shall come to an agreement on the number of buffers to achieve similar performance.

Example (dynamic sized message buffers and 8 KB buffer pool):
A) One network behind the gateway: Several 200 bytes messages received, 40 x 200 bytes = total size 8000 bytes < 8 KB, at least 40 buffers shall be possible to allocate.
B) One network behind the gateway: One 4 KB message received -> 4 KB left in buffer pool. This reduce the number of buffers that can be allocated, e.g. only 20 x 200 bytes due to remaining available memory.
C) Two networks behind the gateway: Implies requirement of 2 x 40 buffers reserved for requests.

Example (static sized message buffers):

A) Two messages with size < 4 KB received, but uses large 4 KB buffers, 2 x 4 KB = 8 KB, more memory than the minimum requirement are needed to be able to allocate more smaller buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers"*.

661413 v4 **Minimum number of buffers for responses**

Version history:
(4): Minimum number of buffers for response on CANFD: change from 80 to 40.

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication.
Source:
Verification Method: Test
Legacy ID: 77455v3

For each network behind the GW: In application mode and if the total size of all current responses are less or equal to CANFD: 32 KB or CAN: 8KB RAM, the Vehicle GW shall be able to allocate at least CANFD: 40 or CAN: (6~8) buffers for responses.
Note: Ethernet is a connection oriented protocol. There are sockets which need to be established and behave as a full duplex channel for a port and IP pair.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for responses.

If using static sized message buffers and/or not possible to reserve buffer pool only for responses: OEM and the ECU supplier shall come to an agreement on the number of buffers to achieve similar performance.

Example (dynamic sized message buffers and 32 KB buffer pool):
A) One network behind the gateway: Several 500 bytes messages received, 40 x 500 bytes = total size 20000 bytes < 32 KB, at least 40 buffers shall be possible to allocate.
B) One network behind the gateway: Four 4 KB message received -> 16 KB left in buffer pool. This reduce the number of buffers that can be allocated, e.g. only 32 x 500 bytes due to remaining available memory.
C) Two networks behind the gateway: Implies requirement of 2 x 40 buffers reserved for responses.

Example (static sized message buffers):
A) Eight messages with size < 4kB received, but uses large 4 KB buffers, 8 x 4 KB = 32 KB, more memory than the minimum requirement are needed to be able to allocate more smaller buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers"*.

4.9.1.11 Vehicle Gateway SWDL

•Gateway allocation of buffers for SWDL

4.9.1.11.1 Gateway allocation of buffers for SWDL

This section will describe and give grounds for the following requirements:

- ReqID 661419 Minimum RAM for request buffers
- ReqID 661418 Minimum number of buffers for requests
- ReqID 661416 Large buffers for request
- ReqID 661420 Minimum RAM for response buffers
- ReqID 661421 Minimum number of buffers for responses
- ReqID 661417 Large buffers for response

Available memory for allocation of buffers for SWDL messages (UDS Layer) are a compromise between hardware cost and performance, e.g. how much traffic in parallel the gateway should be able to handle.
For SWDL messages the Vehicle GW will execute a bootloader software, PBL or SBL, **also will execute in application mode** (i.e. expected that large RAM resources are available).

For each network behind the GW: **In program mode,** the Vehicle GW should be able to allocate at least CANFD: 48KB or CAN: 24 kB or Ethernet: 128kB RAM reserved for buffers for requests, and at least CANFD: 8kB or CAN: 4kB or Ethernet: 8kB RAM reserved for buffers for responses.
For each network behind the GW: **In application mode,** The Vehicle GW should be able to allocate at least CANFD: 32KB or CAN: 12 kB or Ethernet: 128kB RAM reserved for buffers for requests, and at least CANFD: 8kB or CAN: 4kB or Ethernet: 8kB RAM reserved for buffers for responses.

The Vehicle GW must be able to allocate buffer for the largest possible request, CANFD or CAN: 4kB or Ethernet: 64kB, and CANFD or CAN: 4kB or Ethernet: 8kB for responses.

There should be a minimum number of available buffers so that the gateway can handle parallel communication, as long as the total size for buffers is not exceeded. If there is a small request or response the smallest possible buffer should be used, e.g. do not allocate an 4 kB buffer for a 2 bytes long request. This specification does not specify exactly which buffer allocation algorithm that should be used, this is up to the implementer, but there should be a minimum of available buffers for small requests.

To insure that parallel SWDL can be performed for small messages (dynamic sized buffer allocation):

- For each network behind the GW: If the total size of requests are less or equal to the agreed CANFD or CAN RAM in two modes mentioned above, the minimum number for available request buffers should be CANFD: 80 or CAN: (6~8).
- For each network behind the GW: If the total size of responses are less or equal to CANFD or CAN: 4kB RAM, the minimum number for available response buffers should be CANFD: 80 or CAN: (6~8).

See example in Figure: "*Vehicle GW buffers for Internal IP and Backbone Network (CAN FD) SWDL"*.

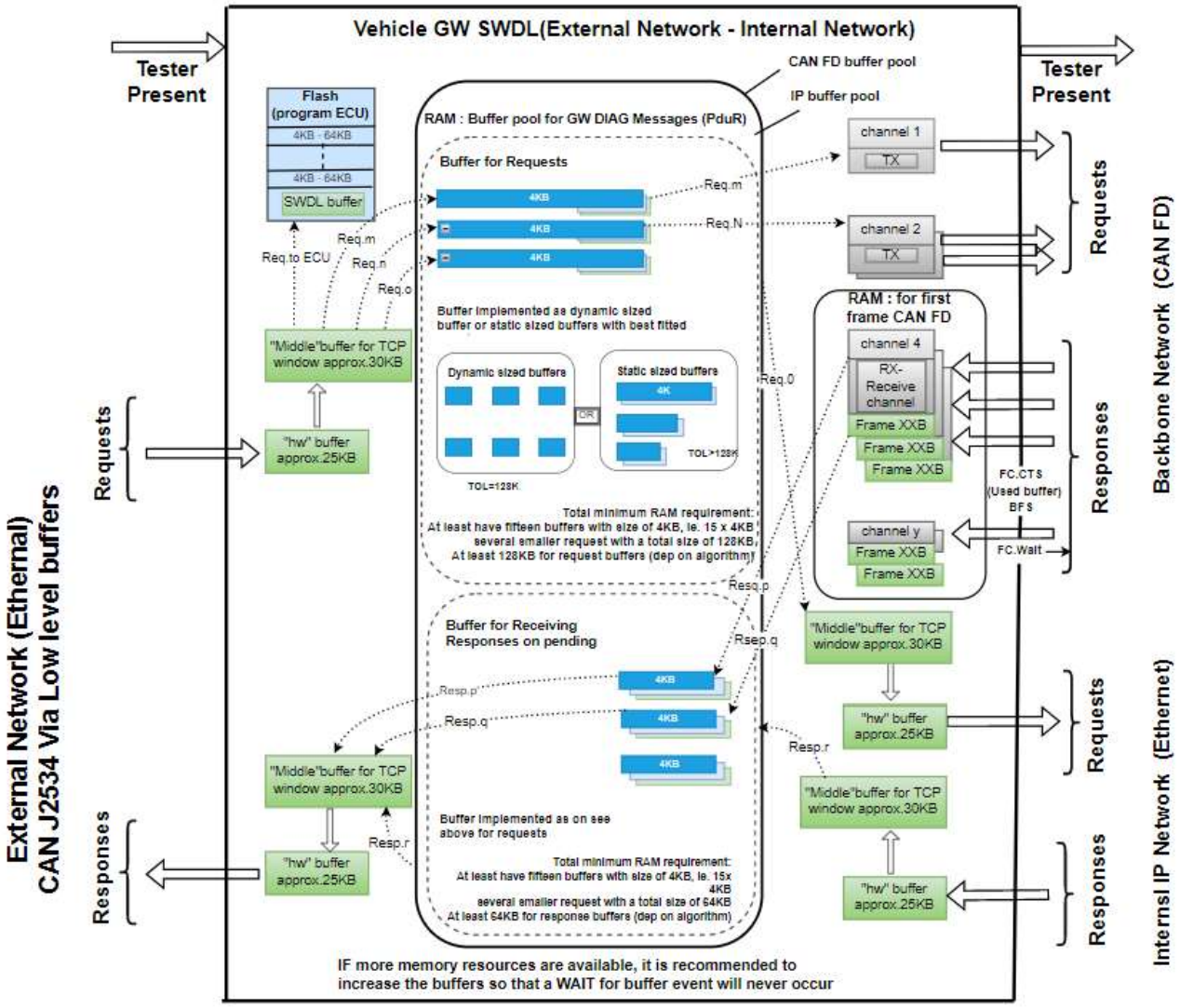


Figure: Vehicle GW buffers for Interal IP and Backbone Network SWDL

661416 v3 Large buffers for request

Version history:
(3): 补充需求，对于VBF软件包超过10MB以上的，且通过CANFD升级的ECU，GW应配置16kB+2的最大Buffer用于诊断路由。

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.

Source:

Verification Method: Test

Legacy ID: 77434v4

For each network behind the GW: In bootloader mode, also in application mode, the Vehicle GW shall be able to allocate up to CANFD or CAN: 4KB or Ethernet: 64kB large buffers for request.

Note: In order to improve the SWDL speed of the ECU on CANFD bus, ECUs with software size larger than 10M will support at least 16kB+2 (SID + blockSequenceCounter) data transmission, Gateway shall be able to allocate up to CANFD: 16386 Bytes large buffers for request. Specific depends on the project.

661417 v2 Large buffers for response

Version history:
(2):

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:
Verification Method: Test
Legacy ID: 77437v4

For each network behind the GW: In bootloader mode, also in application mode, the Vehicle GW shall be able to allocate up to CANFD or CAN: 4kB or Ethernet: 8kB large buffers for response.

661419 v3 Minimum RAM for request buffers

Version history:
(3):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:
Verification Method: Test
Legacy ID: 77432v4

For each network behind the GW: In bootloader mode, the Vehicle GW shall be able to reserve at least a CANFD: 48KB or a CAN: 24KB or Ethernet: 128KB RAM buffer pool only for requests where several smaller buffers can be allocated from.
For each network behind the GW: **In application mode**, the Vehicle GW shall be able to reserve at least a CANFD: 32 KB or a CAN: 12KB or Ethernet: 128KB RAM buffer pool only for requests where several smaller buffers can be allocated from.

This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for requests.
If using static sized message buffers and/or not possible to reserve buffer pool only for requests: GRI Basetech Team and the ECU supplier shall come to an agreement on used buffer sizes to achieve similar performance, more memory for buffers are needed.

Note:
There is a separate requirement for minimum number of buffers that can be allocated form this buffer pool.

Example (dynamic sized message buffers and 48 KB buffer pool):
A) One network behind the gateway: 48 KB RAM reserved for requests (with possibility to allocate different buffer sizes).
B) Two networks behind the gateway: Implies requirement of 2 x 48 KB RAM reserved for requests.

Example (static sized message buffers):
A) Twelve messages with size < 4 KB received, but uses large 4 KB buffers, 12 x 4 KB = 48 KB, more memory than the minimum requirement are needed to be able to allocate more smaller buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers"*. For static sized message buffers it can be larger gaps in the usable memory, hence this minimum RAM requirement may need to be increased dependent on how the static sized message buffers are chosen.

661420 v3 Minimum RAM for response buffers

Version history:
(3):

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:
Verification Method: Test
Legacy ID: 77435v4

For each network behind the GW: In both bootloader mode and application mode, the Vehicle GW shall be able to reserve at least a CANFD: 8KB or CAN:4KB or Ethernet: 8KB RAM buffer pool only for responses where several smaller buffers can be allocated from.

This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for responses.
If using static sized message buffers and/or not possible to reserve buffer pool only for response: OEM and the ECU supplier shall come to an agreement on used buffer sizes to achieve similar performance, more memory for buffers are needed.

Note:
There is a separate requirement for minimum number of buffers that can be allocated form this buffer pool.

Example (dynamic sized message buffers and 8 KB buffer pool):
A) One network behind the gateway: 8 KB RAM reserved for responses (with possibility to allocate differebuffer sizes).
B) Two networks behind the gateway: Implies requirement of 2 x 8 KB RAM reserved for responses.

Example (static sized message buffers):
A) Two messages with size < 4 KB received, but uses large 4 KB buffers, 2 x 4 KB = 8 KB, more memory than the minimum requirement are needed to be able to fulfil the requirement for number of buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers".* For static sized message buffers it can be larger gaps in the usable memory, hence this minimum RAM requirement may need to be increased dependent on how the static sized message buffers are chosen.

661418 v3 Minimum number of buffers for requests

Version history:
(3):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:
Verification Method: Test
Legacy ID: 77433v3

For each network behind the GW: If the total size of all current requests are less or equal to CANFD: 48 KB or CAN: 24KB RAM in bootloader mode, or CANFD: 32 KB or CAN: 12KB RAM in application mode, the Vehicle GW shall be able to allocate at least CANFD: 80 or CAN: (6~8) buffers for requests.
Note: Ethernet is a connection oriented protocol. There are sockets which need to be established and behave as a full duplex channel for a port and IP pair.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for requests.

If using static sized message buffers and/or not possible to reserve buffer pool only for requests: OEM and the ECU supplier shall come to an agreement on the number of buffers to achieve similar performance.

Example for CAN network connected behind gateway (dynamic sized message buffers and 48 KB buffer pool):
A) One network behind the gateway: Several 400 bytes messages received, 80 x 400 byte = total size 32000 bytes < 48 KB, at least 80 buffers shall be possible to allocate.
B) One network behind the gateway: Six 4 KB message received -> 24 KB left in buffer pool. This reduce the number of buffers that can be allocated, e.g. only 40 x 500 bytes due to remaining available memory.
C) Two networks behind the gateway: Implies requirement of 2 x 80 buffers reserved for requests.

Example for CAN network connected behind gateway (static sized message buffers):
A) Twelve messages with size < 4 KB received, but uses large 4 KB buffers, 12 x 4 KB = 48 KB, more memory than the minimum requirement are needed to be able to allocate more smaller buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers"*.

661421 v3 **Minmum number of buffers for responses**

Version history:
(3):

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:
Verification Method: Test
Legacy ID: 77436v3

For each network behind the GW: If the total size of all current requests are less or equal to CANFD: 8KB or CAN: 4KB RAM in both bootloader mode and application mode, the Vehicle GW shall be able to allocate at least CANFD: 80 or CAN: (6~8) buffers for responses.
Note: Ethernet is a connection oriented protocol. There are sockets which need to be established and behave as a full duplex channel for a port and IP pair.

This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for responses.

If using static sized message buffers and/or not possible to reserve buffer pool only for responses: OEM and the ECU supplier shall come to an agreement on the number of buffers to achieve similar performance.

Example (dynamic sized message buffers and 8 KB buffer pool):
A) One network behind the gateway: Several 100 bytes messages received, 80 x 100 bytes = total size 8000 bytes < 8 KB, at least 80 buffers shall be possible to allocate.
B) One network behind the gateway: Two 2 KB message received -> 4 KB left in buffer pool. This reduce the number of buffers that can be allocated, e.g. only 40 x 100 bytes due to remaining available memory.
C) Two networks behind the gateway: Implies requirement of 2 x 80 buffers reserved for responses.

Example (static sized message buffers):
A) Two message with size < 4 KB received, but use large 4 KB buffers, 2 x 4 KB = 8 KB, more memory than the minimum requirement are needed to be able to allocate more smaller buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers".*

4.9.1.12 Vehicle Gateway J2534

•J2534 CAN diagnostic frame format

4.9.1.12.1 J2534 CAN diagnostic frame format

4.9.1.12.1.1 Diagnostic Request frame on J2534 CAN network

29 bit CAN identifier is used to utilize address format defined in *[GW_2] UDS Data* for transmission the message to the in-vehicle 16 bit address format networks e.g. IP.

The Vehicle GW shall only listen to 29 bit CAN identifier frames with bit26 to bit28 set to '1', see Figure: " *29 bit CAN Identifier format for J2534 Diagnostic request"*.

29 bit CAN identifier frames from legislated tester tools, as e.g. scan tool OBD and CAN ID 0x18DB33F1, should not be received by the Vehicle GW.
Scan tool OBD should use 11 bit CAN ID for direct access to emission related ECUs.

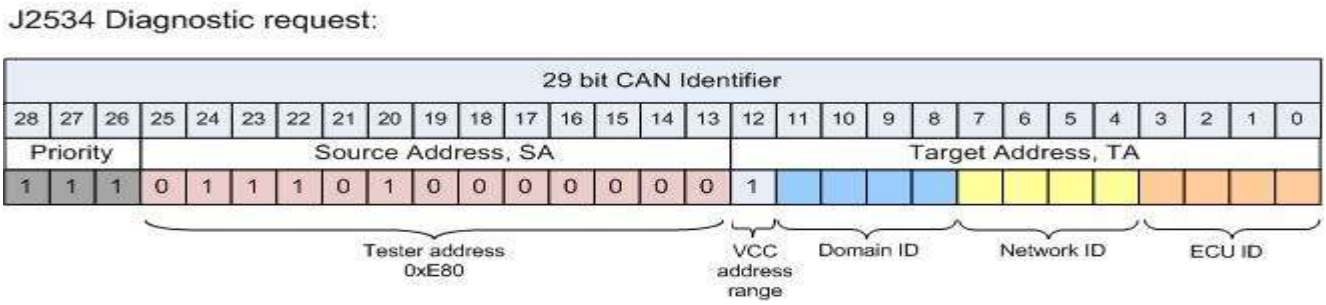


Figure: 29 bit CAN Identifier format for J2534 Diagnostic request

661422 v2 **J2534 CAN Identifier format request**

Version history:
(2):

Purpose: To define the CAN frame format.
Source:
Verification Method: Test
Legacy ID: 77364v3

The Vehicle GW shall only receive 29 bit CAN messages with bit 26 to 28 set to '1'. The format shall follow Table: J2534 29 bit CAN frame format for request:

Bit 26 to 28	All bits shall have the value of “1” (lowest priority) to not interfere with the normal network communication.
Bit 25	'0'
Bit 13 to 24	Source address
Bit 12	'1'
Bit 0 to 11	Target address

Table: J2534 29 bit CAN frame format for request

4.9.1.12.1.2 Diagnostic Response frame on J2534 CAN network

29 bit CAN identifier is used to utilize address format defined in *[GW_2] UDS Data* for transmission the message to the in-vehicle 16 bit address format networks e.g. IP.

J2534 Diagnostic response:

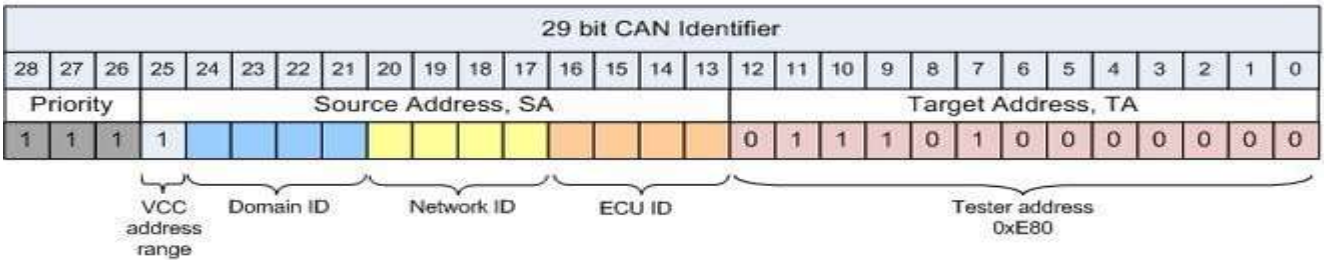


Figure: 29 bit CAN Identifier format for J2534 Diagnostic response

661423 v2 **J2534 CAN Identifier format response**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID: 345704v1

The Vehicle GW shall only receive 29 bit CAN messages with bit 26 to 28 set to '1'. The format shall follow Table 7 - J2534 29 bit CAN frame format for response

Bit 26 to 28	All bits shall have the value of “1” (lowest priority) to not interfere with the normal network communication.
Bit 25	'1'
Bit 13 to 24	Source address
Bit 12	'0'
Bit 0 to 11	Target address

Table: J2534 29 bit CAN frame format for response

•J2534 Latency and Message based requirements

4.9.1.12.2 J2534 Latency and Message based requirements

[GW_27] SAE J2534-1 specifies that the transport protocol *[GW_24] ISO 15765-2* shall be used on the External CAN Network.

The Vehicle GW between the external CAN network and the in-vehicle networks shall receive the complete message before transmitting the request to the vehicle networks. When the complete request is received in the Vehicle GW, internal communication should be the same as if the request was receive from IP.

See Figure: " *Vehicle GW: J2534 CAN to CANFD request"* for receiving a request from J2534 device on CAN.

Same method shall be applied for the response.

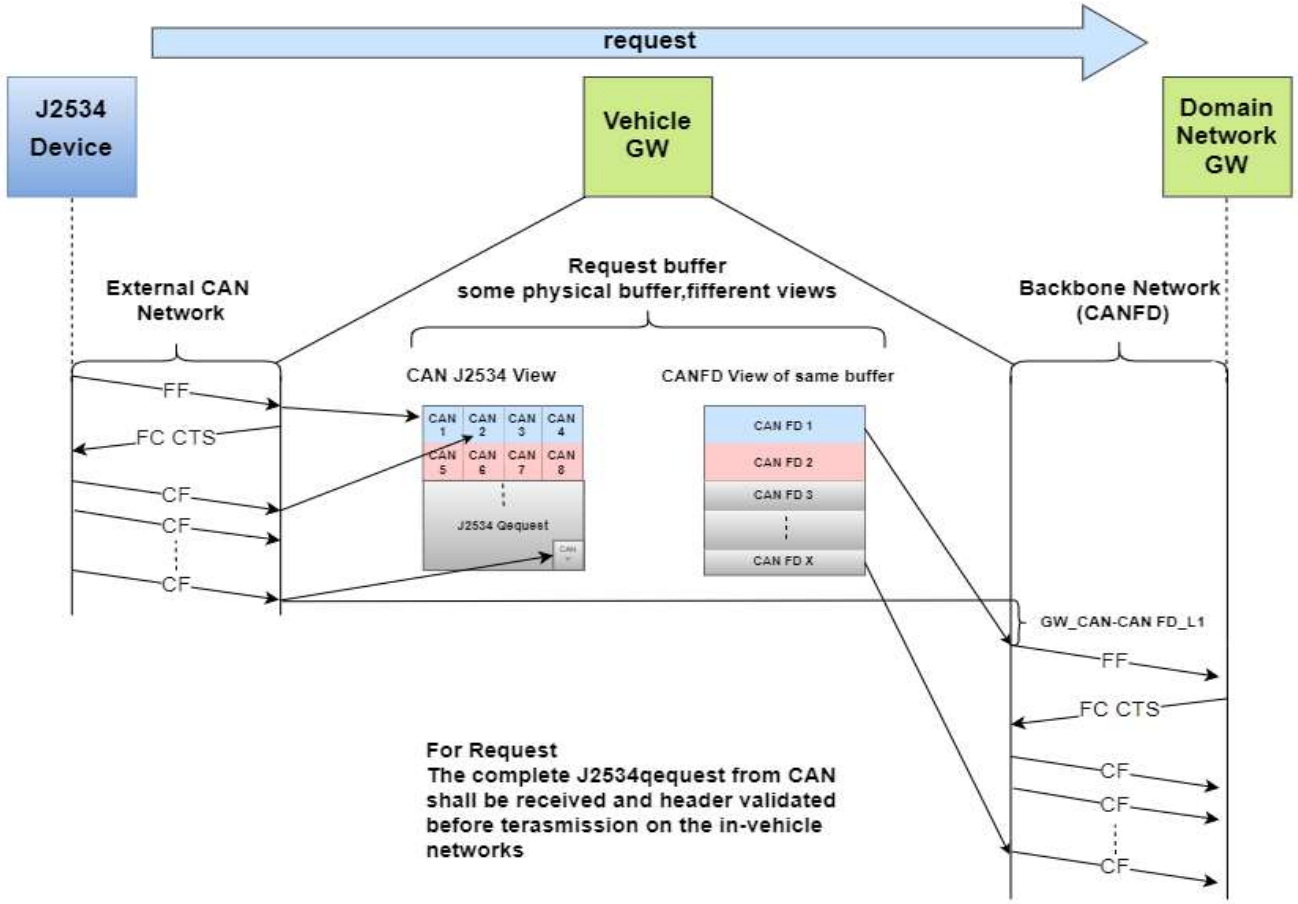


Figure: Vehicle GW: J2534 CAN to CAN FD request

661424 v1 Gateway timing

Version history:
(1):

Purpose: To define a maximum gateway latency time.
Source:
Verification Method: Test
Legacy ID: 77369v2

The time between receiving the complete request until first frame is sent out on the in-vehicle network and vice versa shall be according to the following requirements REQPROD 74168.

661425 v1 Message based gateway

Version history:
(1):

Purpose: To have same internal behaviour in the J2534 gateway as if the diagnostic request is received from the external Ethernet network.
Source:
Verification Method: Test
Legacy ID: 77367v2

The Vehicle GW between the external CAN network and the in-vehicle network shall use a message based gateway for both requests and responses.

•J2534 standard tester connected to Vehicle GW through CAN

4.9.1.12.3 J2534 standard tester connected to Vehicle GW through CAN

The European and U.S. legislation require vehicle manufactures to support the *[GW_27] SAE J2534-1* for re-programming of the vehicle ECUs for the independent aftermarket service environment. This means that both emission and non-emission related ECUs in the vehicle shall be possible to re-programming with a J2534 compliant tool.

The Vehicle GW shall act as a diagnostic communication gateway between the J2534 device and the vehicle via a CAN interface according to J2534.

If the Vehicle GW also has additional external communication interface connected, e.g. Ethernet and a TCP connection, the Vehicle GW should be able to distinguish where the request come from and send the response back on same communication channel.

Note that same external CAN network is also connected to the domain which has the emission related ECUs, see Figure: " *J2534 tester connected to vehicle"*. To prevent an ECU on this domain to get the request from two directions both, from the backbone via the Vehicle GW and direct via the J2534 device, the CAN frame format different; 11bit CAN identifiers is used in-vehicle and 29bit CAN identifier to the Vehicle GW. See also next section ' *J2534: 29 bit CAN identifier format'*.

The re-programming sequence shall be conducted according to *[GW_15] General Software Download Specification*.

J2534 tester limitation:
Due to the external CAN network also is connected to domains containing emission related ECUs is parallel download of data to multiple ECUs or queued request NOT allowed to prevent high busload on the domain network. It is therefore only allowed re-program/communicate with one ECU at a time.

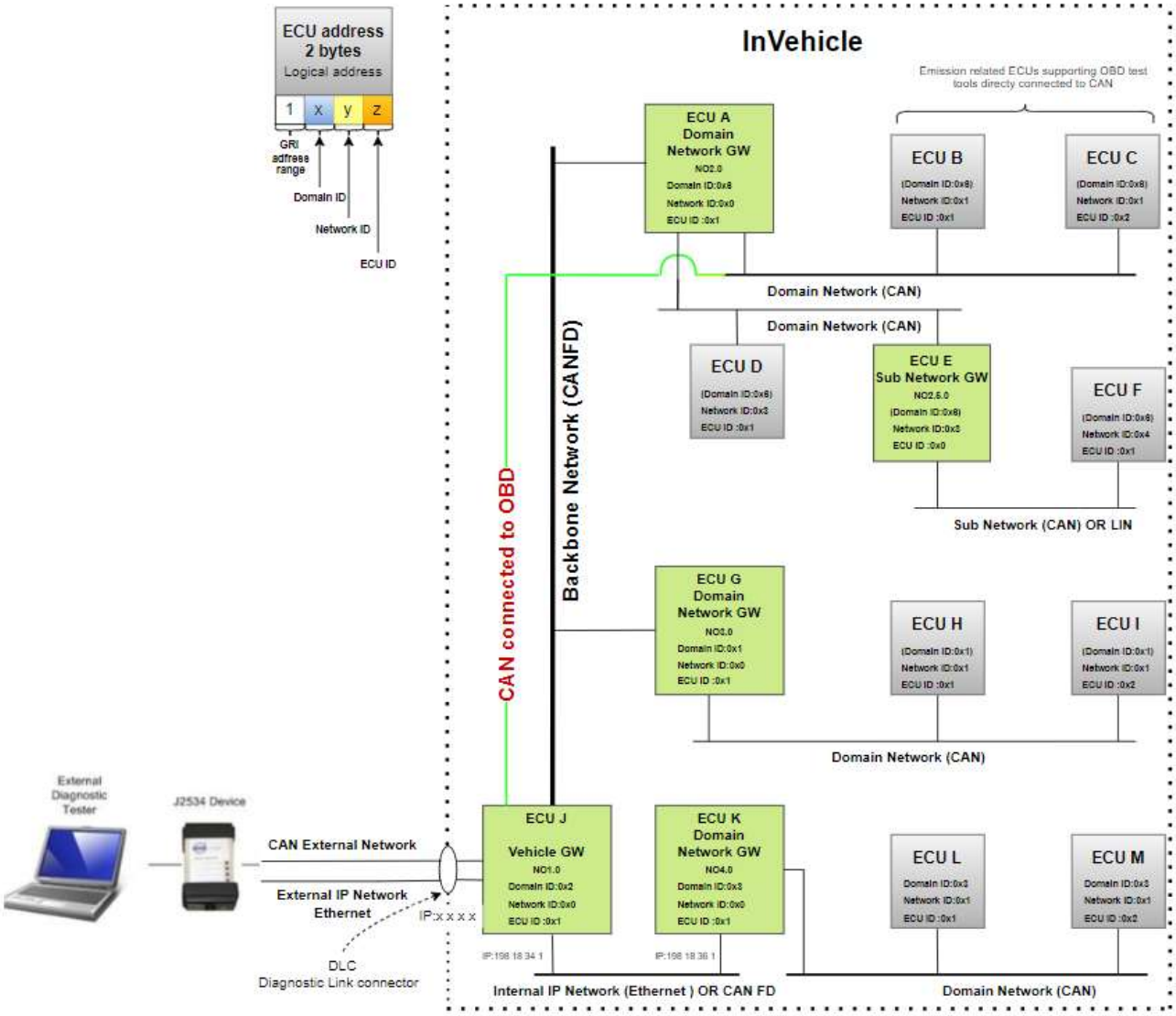


Figure: J2534 tester connected to vehicle

704857 v1 **Diagnostic Routine Channels**

Version history:
(1): New add the requirement to clarify that the routing path is same as OEM UDS diagnostic routing path.

Purpose: To be able to gateway diagnostic messages using the same UDS diagnostic routing path as the internal network.

Source:

Verification Method: Test

Legacy ID:

The Vehicle GW shall implement the J2534 gateway between the external CAN network and the in-vehicles networks through the same UDS diagnostic routing path as the internal network.

661426 v1 **Differentiate response to external network**

Version history:
(1):

Purpose: To transmit the response to the same network as the request was originated from.
Source:
Verification Method: Test
Legacy ID: 77361v2

The Vehicle GW shall transmit the response from the vehicle to the external network which the request was originated from.

Rationale:
If the request was a DoIP request from Ethernet, the response should be sent on the Ethernet connection. If the request was a J2534 from CAN, the response should be sent on the CAN connection.

661427 v1 **Gateway mode**

Version history:
(1):

Purpose: To be able to gateway diagnostic messages in both the application and bootloader.
Source:
Verification Method: Test
Legacy ID: 77365v2

The Vehicle GW shall implement the J2534 gateway between the external CAN network and the in-vehicles networks in both bootloader and application mode.

661428 v1 **Generic gateway requirements**

Version history:
(1):

Purpose: To define the generic gateway requirements applied also for the J2534 gateway.
Source:
Verification Method: Test
Legacy ID: 77368v2

The Vehicle GW for J2534 messages shall where possible implement the same generic GW behavior as if the diagnostic messages was to be transported on an external IP network defined in this document, except the IP specific requirements for addressing and format to and from the external IP network.

4.9.1.13 InVehicle DoIP Gateway - IP - CAN

•Translation of diagnostic messages

4.9.1.13.1 Translation of diagnostic messages

4.9.1.13.1.1 Diagnostic request from internal IP network to CAN network

For gateway of request from internal IP to CAN, The target address in the DoIP header (DoIP TA) should be translate the 11 bit CAN ID. The bit 10~8 of CAN ID set as 0x7 and bit 7-0 containing Network ID and ECU ID to part of the DoIP header (DoIP TA) including the Network ID and ECU ID.

See example in Figure "*Diagnostic request from internal IP to CAN*".

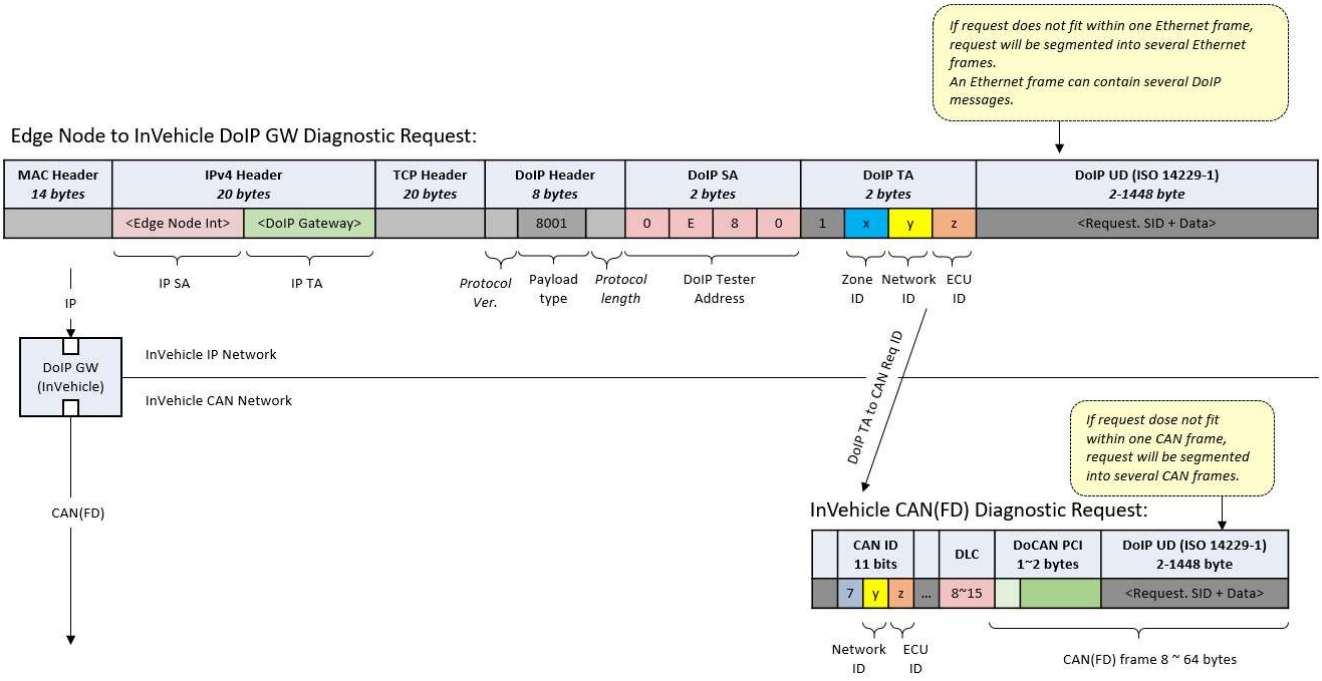


Figure: Diagnostic request from internal IP to CAN

681666 v1 **Internal IP format to CAN format in request**

Version history:
(1):

Purpose: Define the change in format from an internal IP request to a CAN request, using the DoIP header information to set up the CAN request identifier (CAN Req ID).

Source:

Verification Method: Test

Legacy ID:

If a request is received from the internal IP network that should be transported to the CAN Network, the in-vehicle DoIP GW shall translate the DoIP target address (DoIP TA) to the 11 bit CAN request identifier (CAN Req ID), format on frames as defined in DoIP [*ISO 13400-2*] and in DoCAN [*ISO 15765-2*].

Comment:
See example in Figure: "*Diagnostic request from internal IP to CAN*".

4.9.1.13.1.2 Diagnostic response from CAN network to internal IP network

This section will describe CAN format to internal IP format in response and CAN response add IP address in extern.

See example in Figure *Diagnostic response from CAN to internal IP*.

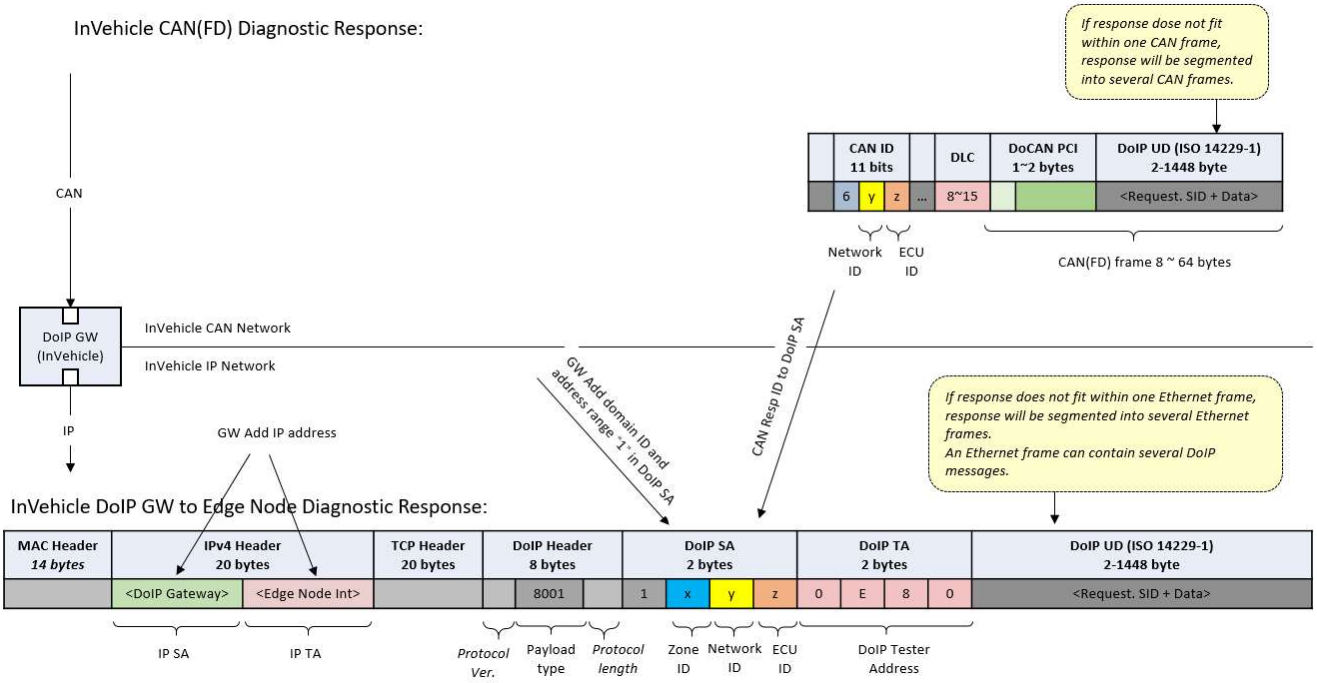


Figure: Diagnostic response from CAN to internal IP

681691 v1 CAN format to internal IP format in response

Version history:
(1):

Purpose: Define the change from CAN format to DoIP format in the response, CAN response identifier to DoIP header source address.

Source:

Verification Method: Test

Legacy ID:

If a response is received from the CAN network that should be transported to the internal IP network, the in-vehicle DoIP GW shall translate the CAN response identifier (CAN Resp ID) to a DoIP source address (DoIP SA).

Comment:
See example Figure: "*Diagnostic response from CAN to internal IP*".

681698 v1 CAN response add IP address in internal IP

Version history:
(1):

Purpose: Internal IP address must be added in a response from sub Network (CAN) to internal IP network.

Source:

Verification Method: Test

Legacy ID:

If a response is received from the sub Network (CAN) that should be transported to the internal IP network, the in-vehicle DoIP GW shall add the DoIP GW itself internal IP address (IP SA) and add Edge Node internal IP address (IP TA), IP address that was statically allocated during establishing connection between IP ECU and Edge Node.

Comment:
See example in Figure: "*Diagnostic response from CAN to internal IP*".

•Gateway allocation of communication channels

4.9.1.13.2 Gateway allocation of communication channels

This section describes the allocation of communication channels for physical request and responses.

Note: Functional requests, including TesterPresent, should not be delayed in the gateway due to e.g. queued or parallel requests in the gateway. Functional requests need to have separate communication channel, so that TesterPresent can be sent though the gateways and keep ECUs in non-default session.

4.9.1.13.2.1 Network (CAN) - Client side

This section will describe and give grounds for the following requirements:

- REQPROD 681716 Transmit(Tx) CCs on Network
- REQPROD 681717 Receive(Rx) CCs on Network

The in-vehicle gateway should not do any re-sending of messages, e.g. if time-out due to no flow control frames is received.
It should always be guaranteed when allocating a transmit (Tx) communication channel (CC) on the sender side, the receiver side should be able to allocate a receive (Rx) communication channel (CC), so that no data (first frame) will be lost.

There is no handshake mechanism when allocating a Tx CC on the sender side with the allocation of an Rx CC on the receiver side, hence it must always be assured that a Rx CC can be allocated to receive at least the first frame so that no data will be lost.

Allocation of transmit (Tx) communication channels (CCs)
InVehicle DoIP GW for requests to Network (CAN):

- The InVehicle DoIP GW should have three(3) communication channels (CCs) for sending physical request to the ECUs on each Network (CAN).

Allocation of receive (Rx) communication channels (CCs)
InVehicle DoIP GW for responses from Network (CAN):

- The InVehicle DoIP GW should have same number of receive (Rx) communication channels (CCs) as transmit (Tx) communication channels (CCs) on the Network (Note: The number of Rx CCs are project specific, e.g. same number as ECUs connected on the Network).
- The InVehicle DoIP GW must be able to receive and handle first frame from all ECUs.
- If there is no buffer in the buffer pool available for receiving the complete response on the CC, the InVehicle DoIP GW should send flow control WAIT until a buffer that fits the response is available. When a buffer is available,InVehicle DoIP GW should send a flow control CTS for receiving the remaining part of the response.

Note: When dimension the system with number of supported communication channels and number of messages that can be allocated from the buffer pool, it must be assured that flow control WAIT frames can be sent long enough time so that the sending side does not time-out.
See Figure: "Allocation communication channels (CCs) for physical request on Network (CAN) - Client side".

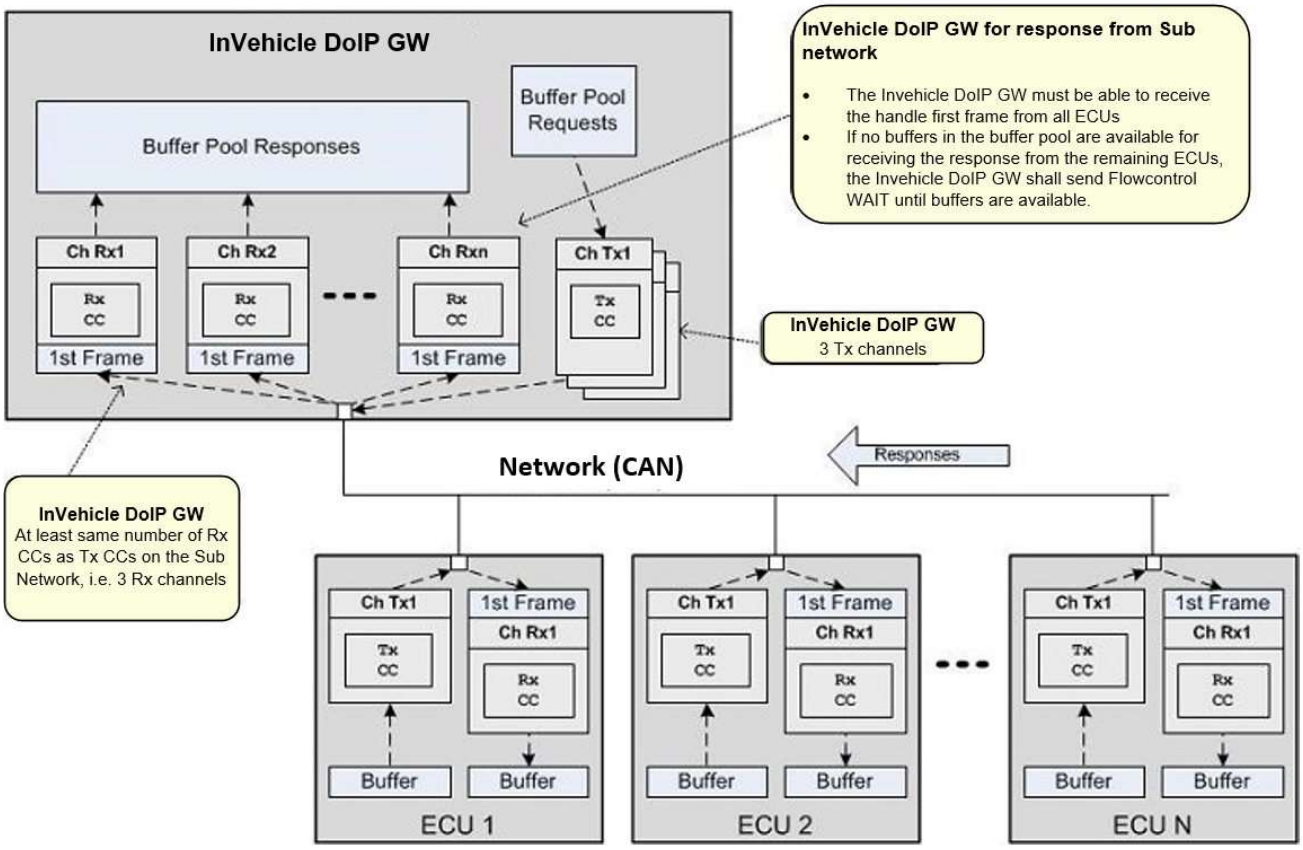


Figure: Allocation communication channels (CCs) for physical request on Network (CAN) - Client side

681717 v1 **Receive(Rx) CCs on Network (CAN)**

Version history:
(1):

Purpose: Ensure that no diagnostic responses are dropped, a first frame shall always be stored, and dependent on available buffers flow control CTS or WAIT should be sent.

Source:

Verification Method: Test

Legacy ID:

The InVehicle DoIP GW connect to Network (CAN) shall be able to open as many receive (Rx) communication channels (CCs) for receiving diagnostic responses as the total number of transmit (Tx) communication channels (CCs) that can open by all ECUs on the network.

Comment:
The number of Rx CCs are project specific.

The number of Rx CCs will be the number of ECUs on the sub network, since the requirement on Tx communication channels on a ECU is one (1).

681716 v1 **Transmit(Tx) CCs on Network (CAN)**

Version history:
(1):

Purpose: It should be possible to send request as soon as possible so that timing requirement for reading information from ECUs can be fulfilled
Source:
Verification Method: Test
Legacy ID:

For each Network (CAN) behind the GW: The InVehicle DoIP GW shall be able to open three(3) transmit (Tx) communication channels (CCs) for sending diagnostic physical requests to the ECUs.

For each Network (CAN) behind the GW: The InVehicle DoIP GW shall be able to open one additional transmit (Tx) communication channel (CC) for functional requests.

The ECU supplier shall confirm with OEM, the exact number of communication channels for the specific project.

•Gateway latency requirements

4.9.1.13.3 Gateway latency requirements

4.9.1.13.3.1 IP - CAN diagnostic messages

[GW_6 *DoCAN*] specifies requirement on transport protocol level as e.g. maximum time between consecutive frames, C_Cs, performance requirements.

4.9.1.13.3.1.1 Diagnostic requests IP to CAN

The IP to CAN InVehicle DoIP GW shall receive and validate the complete request from IP before the request is transmitted on the network (CAN), as specified in [*ISO 13400-2*]. See example in Figure: "*InVehicle DoIP GW: IP to CAN request*".

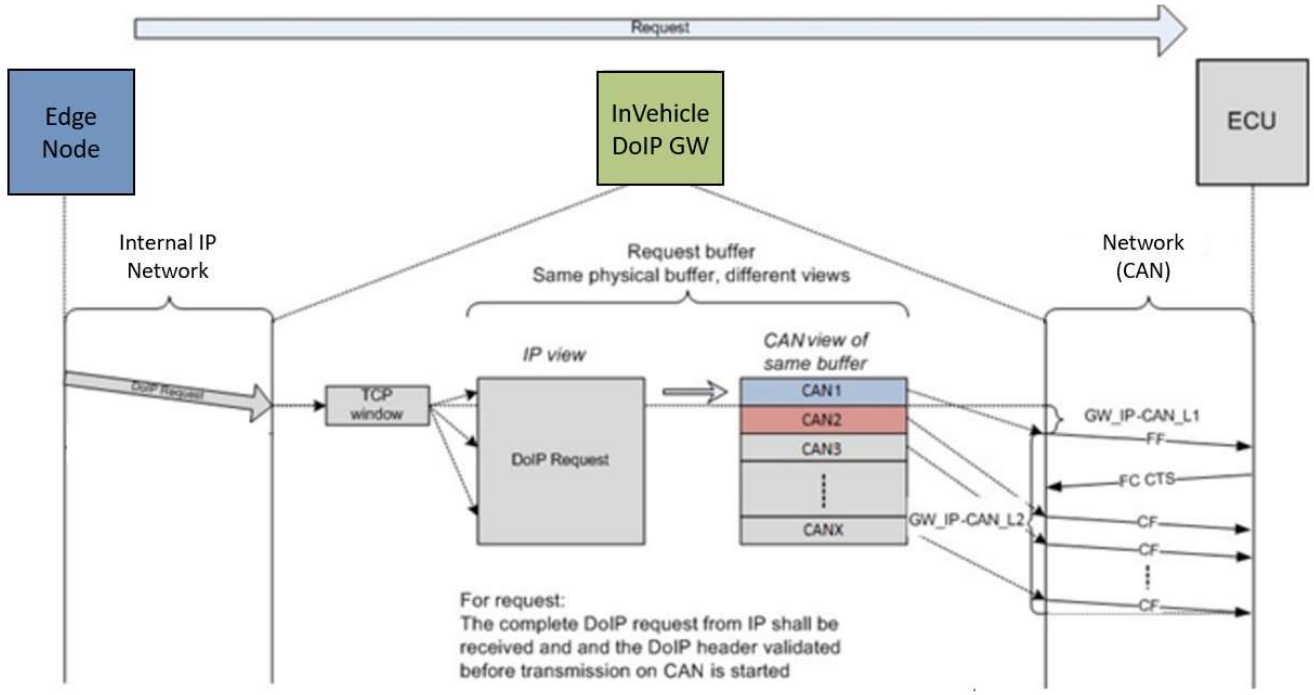


Figure: InVehicle DoIP GW: IP to CAN request

Definition of latency timing parameters:
GW_IP-CAN_L1: Time between receiving the complete request from IP until first frame is sent out on Network (CAN).

GW_IP-CAN_L2: Time for transfer the complete request on Network (CAN).
This time depends on:

- Size of the message, how the message is packed on the Network (CAN), i.e. how many frames that shall be sent.
- Timing parameters for the Network (CAN) is specified in [*GW_6*].

Flow control WAIT can only be sent a limited time, so GW_IP-CAN_L2 has impact on how the system shall be dimension regarding parallel communication channels and how many messages that can be received without sending Flow control WAIT frames.

Timing Parameter	SWDL	DIAG
	Max Value	Max Value
GW_IP-CAN_L1	10 ms 1)	10 ms 2)
GW_IP-CAN_L2	See [1*] and Formula(1)	See [1*] and Formula(1)
Formula(1): · unsegmented messages: C_As · segmented messages: C_As + C_Bs + ((Message size) – (L-SDU size))/(L-SDU size) * C_Cr		

Table: InVehicle DoIP GW: IP to CAN request timing parameters

*: [*DoCAN*]

661408 v1 **Latency IP to CAN, GW_IP-CAN_L1**

Version history:
(1):

Purpose: Clarify and define a maximum latency time.
Source:
Verification Method: Test
Legacy ID:

If a communication channel can be allocated, the time between receiving the complete request from IP/Ethernet until first frame is sent out on CAN shall be max 10ms.

4.9.1.13.3.1.2 Diagnostic responses CAN to IP

The CAN to IP InVehicle DoIP GW shall receive the complete response on CAN before response is transmitted on the IP network.

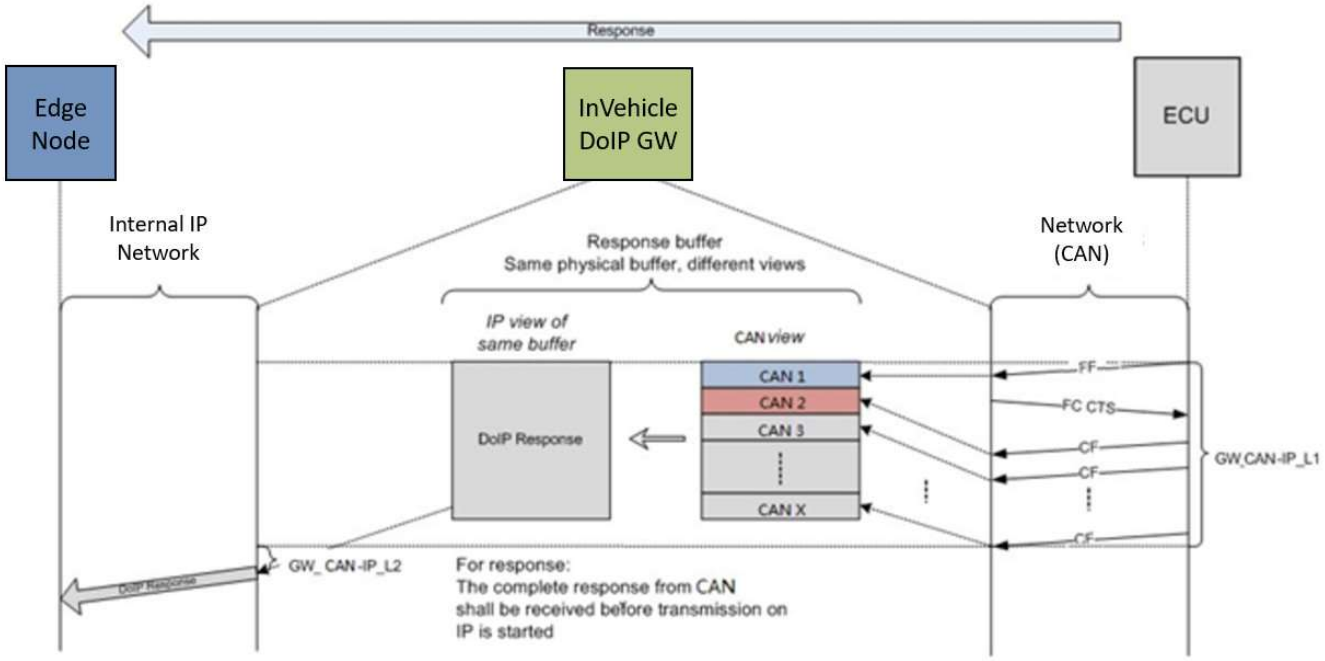


Figure: Vehicle GW: CAN to IP response

Definition of latency timing parameters:
GW_CAN-IP_L1: Time for transfer the complete response on Network (CAN).
This time depends on:

- Size of the message, how the message is packed on the Network (CAN), i.e. how many frames that shall be sent.
- Timing parameters for the Network (CAN) specified in [*GW_6*].

GW_CAN_IP_L2: Time between receiving complete response from Network (CAN) until DoIP response shall be triggered on the IP Network.

Flow control WAIT can only be sent a limited time, so GW_CAN_IP_L1 has impact on how the system shall dimension regarding parallel communication channels and how many messages that can be received without sending Flow control WAIT frames.

Timing Parameter	SWDL	DIAG
	Max Value	Max Value
GW_CAN_IP_L1	See [*] and Formula(1)	See [*] and Formula(1)
GW_CAN_IP_L2	5 ms	5 ms
Formula(1): · unsegmented messages: C_As · segmented messages: C_As + C_Bs + ((Message size) – (L-SDU size))/(L-SDU size) * C_Cr		

Table: Vehicle GW: FlexRay to IP response timing parameters

*: [*DoCAN*]

661409 v1	**Latency CAN to IP, GW_CAN-IP_L2**

Version history:
(1):

Purpose: Clarify and define a maximum latency time.
Source:
Verification Method: Test
Legacy ID:

If a communication channel can be allocated, time between receiving complete response from CAN network until DoIP response shall be triggered shall be max 5ms.

4.9.1.14 InVehicle DoIP GW DIAG

•Gateway allocation of buffers for DIAG

4.9.1.14.1 Gateway allocation of buffers for DIAG

This section will describe and give grounds for the following requirements:

- ReqID 681722 Minimum RAM for request buffers
- ReqID 681723 Minimum number of buffers for requests
- ReqID 681724 Large Buffers for request
- ReqID 681725 Minimum RAM for response buffers
- ReqID 681740 Minimum number of buffers for responses
- ReqID 681741 Large Buffers for response

Available memory for allocation of buffers for DIAG messages (UDS Layer) are a compromise between hardware cost and performance, e.g. how much traffic in parallel the gateway should be able to handle.
For DIAG messages, the InVehicle DoIP GW will execute in application mode, and have to share resources as RAM for buffers with other applications.

For each network behind the GW: The InVehicle DoIP GW should be able to allocate at least CAN: 4kB RAM reserved for buffers for requests, and at least CAN: 8kB RAM reserved for buffers for responses.

The InVehicle DoIP GW must be able to allocate buffer for the largest possible request, CAN: 4kB, and CAN: 4kB for response.

There should be a minimum number of available buffers so that the gateway can handle parallel communication, as long as the total size for buffers is not exceeded. If there is a small request or response the smallest possible buffer should be used, e.g. do not allocate an 4 kB buffer for a 2 bytes long request. This specification does not specify exactly which buffer allocation algorithm that should be used, this is up to the implementer, but there should be a minimum of available buffers for small requests.

To insure that parallel DIAG can be performed for small messages (dynamic sized buffer allocation):

- For each network behind the GW: If the total size of requests are less or equal to CAN: 4kB RAM, the minimum number for available request buffers should be CAN: 32.
- For each network behind the GW: If the total size of responses are less or equal to CAN: 8kB RAM, the minimum number for available response buffers should be CAN: 32.

See example in Figure: "*InVehicle DoIP GW Buffers for Internal IP and Network (CAN) DIAG*".

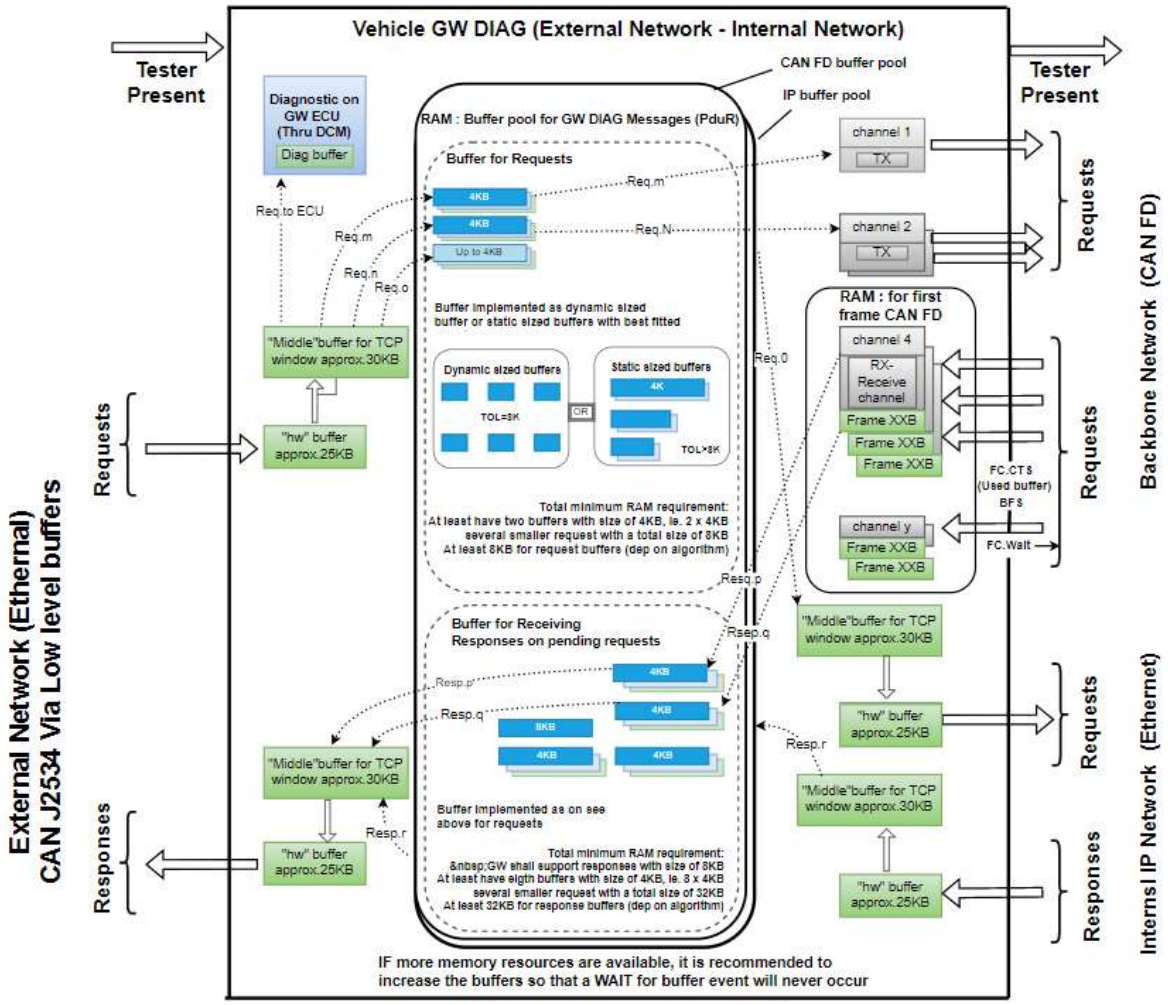


Figure: InVehicle DoIP GW Buffers for Internal IP and Network (CAN) DIAG

681724 v1 **Large buffers for request**

Version history:
(1):

Purpose: Set a minimum requirement for available memory for requests and response. To ensure a minimum performance requirement for parallel communication.

Source:

Verification Method: Test

Legacy ID:

For each network behind the GW: In application mode, the GW shall be able to allocate up to CAN: 4 kB large buffers for request.

681741 v1 **Large buffers for response**

Version history:
(1):

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum

performance requirement for parallel communication.

Source:

Verification Method: Test

Legacy ID:

For each network behind the GW: In application mode, the GW shall be able to allocate up to CAN: 4kB large buffers for response.

681722 v1 Minimum RAM for request buffers

Version history:
(1):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication.

Source:

Verification Method: Test

Legacy ID:

For each network behind the GW: In application mode, the GW shall be able to reserve at least a CAN: 4kB RAM buffer pool only for requests where several smaller buffers can be allocated from.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for requests.
If using static sized message buffers and/or not possible to reserve buffer pool only for requests: OEM and the ECU supplier shall come to an agreement on used buffer sizes to achieve similar performance, more memory for buffers are needed.

Note:
There is a separate requirement for minimum number of buffers that can be allocated form this buffer pool.

Example (dynamic sized message buffers and 4 KB buffer pool):
A) One network behind the gateway: 4 KB RAM reserved for requests (with possibility to allocate different buffer sizes).
B) Two networks behind the gateway: Implies requirement of 2 x 4 KB RAM reserved for requests.

Example (static sized message buffers):
A) One messages with size < 4 KB received, but uses large 4 KB buffers, 1 x 4 KB = 4 KB, more memory than the minimum requirement are needed to be able to fulfil the requirement for number of buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers".* For static sized message buffers it can be larger gaps in the usable memory, hence this minimum RAM requirement may need to be increased dependent on how the static sized message buffers are chosen.

681725 v1 Minimum RAM for response buffers

Version history:
(1):

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication.

Source:

Verification Method: Test

Legacy ID:

For each network behind the GW: In application mode, the GW shall be able to reserve at least a CAN: 8KB RAM buffer pool only for responses where several smaller buffers can be allocated from.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for responses.
If using static sized message buffers and/or not possible to reserve buffer pool only for responses: OEM and the ECU supplier shall come to an agreement on used buffer sizes to achieve similar performance, more memory for buffers are needed.

Note:
There is a separate requirement for minimum number of buffers that can be allocated form this buffer pool.

Example (dynamic sized message buffers and 8 KB buffer pool):
A) One network behind the gateway: 8 KB RAM reserved for responses (with possibility to allocate different buffer sizes).
B) Two networks behind the gateway: Implies requirement of 2 x 8 KB RAM reserved for responses.

Example (static sized message buffers):
A) Four messages with size < 4 KB received, but uses large 4 KB buffers, 8 x 4 KB = 32 KB, more memory than the minimum requirement are needed to be able to fulfil the requirement for number of buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers"*. For static sized message buffers it can be larger gaps in the usable memory, hence this minimum RAM requirement may need to be increased dependent on how the static sized message buffers are chosen.

681723 v1 **Minimum number of buffers for requests**

Version history:
(1):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication.
Source:
Verification Method: Test
Legacy ID:

For each network behind the GW: In application mode and if the total size of all current requests are less or equal to CAN: 4KB RAM, the GW shall be able to allocate at least CAN: 32 buffers for requests.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for requests.

If using static sized message buffers and/or not possible to reserve buffer pool only for requests: OEM and the ECU supplier shall come to an agreement on the number of buffers to achieve similar performance.

Example (dynamic sized message buffers and 4 KB buffer pool):
A) One network behind the gateway: Several 100 bytes messages received, 32 x 100 bytes = total size 3200 bytes < 4 KB, at least 32 buffers shall be possible to allocate.
B) One network behind the gateway: One 2 KB message received -> 2 KB left in buffer pool. This reduce the number of buffers that can be allocated, e.g. only 20 x 100 bytes due to remaining available memory.
C) Two networks behind the gateway: Implies requirement of 2 x 32 buffers reserved for requests.

Example (static sized message buffers):
A) One messages with size < 4 KB received, but uses large 4 KB buffers, 1 x 4 KB = 8 KB, more memory than the minimum requirement are needed to be able to allocate more smaller buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers"*.

681740 v1 **Minimum number of buffers for responses**

Version history:
(1):

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication.
Source:
Verification Method: Test
Legacy ID:

For each network behind the GW: In application mode and if the total size of all current responses are less or equal to CAN: 8KB RAM, the GW shall be able to allocate at least CAN: 32 buffers for response.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for responses.

If using static sized message buffers and/or not possible to reserve buffer pool only for responses: OEM and the ECU supplier shall come to an agreement on the number of buffers to achieve similar performance.

Example (dynamic sized message buffers and 8 KB buffer pool):
A) One network behind the gateway: Several 200 bytes messages received, 32 x 200 bytes = total size 64000 bytes < 8 KB, at least 32 buffers shall be possible to allocate.
B) One network behind the gateway: One 4 KB message received -> 4 KB left in buffer pool. This reduce the number of buffers that can be allocated, e.g. only 20 x 200 bytes due to remaining available memory.
C) Two networks behind the gateway: Implies requirement of 2 x 32 buffers reserved for responses.

Example (static sized message buffers):
A) Two messages with size < 4kB received, but uses large 4 KB buffers, 2 x 4 KB = 8 KB, more memory than the minimum requirement are needed to be able to allocate more smaller buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers"*.

4.9.1.15 InVehicle DoIP GW SWDL

•Gateway allocation of buffers for SWDL

4.9.1.15.1 Gateway allocation of buffers for SWDL

This section will describe and give grounds for the following requirements:

- ReqID 681742 Minimum RAM for request buffers
- ReqID 681744 Minimum number of buffers for requests
- ReqID 681745 Large buffers for request
- ReqID 681746 Minimum RAM for response buffers
- ReqID 681747 Minimum number of buffers for responses
- ReqID 681748 Large buffers for response

Available memory for allocation of buffers for SWDL messages (UDS Layer) are a compromise between hardware cost and performance, e.g. how much traffic in parallel the gateway should be able to handle.
For SWDL messages the InVehicle DoIP GW will execute a bootloader software, PBL or SBL, **also will execute in application mode** (i.e. expected that large RAM resources are available).

For each network behind the GW: **In program mode,** the InVehicle DoIP GW should be able to allocate at least CAN: 24 kB RAM reserved for buffers for requests, and at least CANFD: 8kB or CAN: 4kB or Ethernet: 8kB RAM reserved for buffers for responses.
For each network behind the GW: **In application mode,** The InVehicle DoIP GW should be able to allocate at

least CAN: 12 kB RAM reserved for buffers for requests, and at least CANFD: 8kB or CAN: 4kB or Ethernet: 8kB RAM reserved for buffers for responses.

The Vehicle GW must be able to allocate buffer for the largest possible request, CANFD or CAN: 4kB or Ethernet: 64kB, and CANFD or CAN: 4kB or Ethernet: 8kB for responses.

There should be a minimum number of available buffers so that the gateway can handle parallel communication, as long as the total size for buffers is not exceeded. If there is a small request or response the smallest possible buffer should be used, e.g. do not allocate an 4 kB buffer for a 2 bytes long request. This specification does not specify exactly which buffer allocation algorithm that should be used, this is up to the implementer, but there should be a minimum of available buffers for small requests.

To insure that parallel SWDL can be performed for small messages (dynamic sized buffer allocation):

- For each network behind the GW: If the total size of requests are less or equal to the agreed CANFD or CAN RAM in two modes mentioned above, the minimum number for available request buffers should be CANFD: 80 or CAN: (6~8).
- For each network behind the GW: If the total size of responses are less or equal to CANFD or CAN: 4kB RAM, the minimum number for available response buffers should be CANFD: 80 or CAN: (6~8).

See example in Figure: "*InVehicle DoIP GW buffers for Internal IP and Network (CAN) SWDL*".

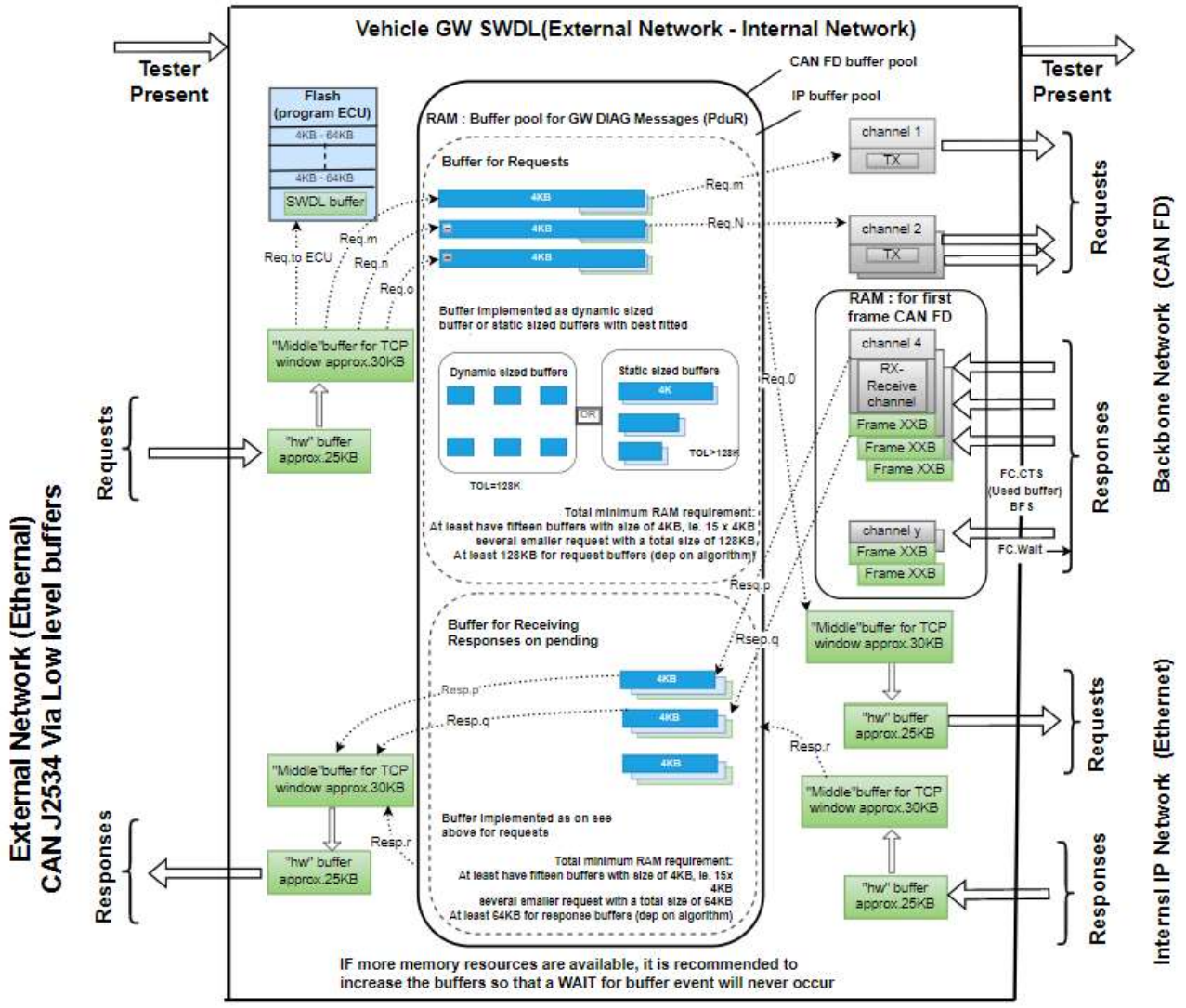


Figure: InVehicle DoIP GW buffers for Interal IP and Network SWDL

681745 v1 Large buffers for request

Version history:
(1):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:
Verification Method: Test
Legacy ID:

For each network behind the GW: In bootloader mode, also in application mode, the GW shall be able to allocate up to CAN: 4KB large buffers for request.

681748 v1 Large buffers for response

Version history:
(1):

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:
Verification Method: Test
Legacy ID:

For each network behind the GW: In bootloader mode, also in application mode, the GW shall be able to allocate up to CAN: 4kB large buffers for response.

681742 v1 Minimum RAM for request buffers

Version history:
(1):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:
Verification Method: Test
Legacy ID:

For each network behind the GW: In bootloader mode, the GW shall be able to reserve at least a CAN: 24KB RAM buffer pool only for requests where several smaller buffers can be allocated from.
For each network behind the GW: **In application mode**, the GW shall be able to reserve at least a CAN: 12KB RAM buffer pool only for requests where several smaller buffers can be allocated from.

This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for requests.
If using static sized message buffers and/or not possible to reserve buffer pool only for requests: GRI Basetech Team and the ECU supplier shall come to an agreement on used buffer sizes to achieve similar performance, more memory for buffers are needed.

Note:
There is a separate requirement for minimum number of buffers that can be allocated form this buffer pool.

Example (dynamic sized message buffers and 24 KB buffer pool):
A) One network behind the gateway: 24 KB RAM reserved for requests (with possibility to allocate different buffer sizes).
B) Two networks behind the gateway: Implies requirement of 2 x 24 KB RAM reserved for requests.

Example (static sized message buffers):
A) Six messages with size < 4 KB received, but uses large 4 KB buffers, 6 x 4 KB = 24 KB, more memory than the minimum requirement are needed to be able to allocate more smaller buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers"*. For static sized message buffers it can be larger gaps in the usable memory, hence this minimum RAM requirement may need to be increased dependent on how the static sized message buffers are chosen.

681746 v1 Minimum RAM for response buffers

Version history:
(1):

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:
Verification Method: Test
Legacy ID:

For each network behind the GW: In both bootloader mode and application mode, the GW shall be able to reserve at least a CAN:4KB RAM buffer pool only for responses where several smaller buffers can be allocated from.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for responses.

If using static sized message buffers and/or not possible to reserve buffer pool only for response: OEM and the ECU supplier shall come to an agreement on used buffer sizes to achieve similar performance, more memory for buffers are needed.

Note:
There is a separate requirement for minimum number of buffers that can be allocated form this buffer pool.

Example (dynamic sized message buffers and 4 KB buffer pool):
A) One network behind the gateway: 4 KB RAM reserved for responses (with possibility to allocate differebuffer sizes).
B) Two networks behind the gateway: Implies requirement of 4 x 8 KB RAM reserved for responses.

Example (static sized message buffers):
A) One messages with size < 4 KB received, but uses large 4 KB buffers, 1 x 4 KB = 4 KB, more memory than the minimum requirement are needed to be able to fulfil the requirement for number of buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers"*. For static sized message buffers it can be larger gaps in the usable memory, hence this minimum RAM requirement may need to be increased dependent on how the static sized message buffers are chosen.

681744 v1 Minimum number of buffers for requests

Version history:
(1):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:

Verification Method: Test

Legacy ID:

For each network behind the GW: If the total size of all current requests are less or equal to CAN: 24KB RAM in bootloader mode, or CAN: 12KB RAM in application mode, the GW shall be able to allocate at least CAN: 32 buffers for requests.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for requests.

If using static sized message buffers and/or not possible to reserve buffer pool only for requests: OEM and the ECU supplier shall come to an agreement on the number of buffers to achieve similar performance.

Example for CAN network connected behind gateway (dynamic sized message buffers and 24 KB buffer pool):
A) One network behind the gateway: Several 600 bytes messages received, 32 x 600 byte = total size 19200 bytes < 24 KB, at least 32 buffers shall be possible to allocate.
B) One network behind the gateway: Three 4 KB message received -> 12 KB left in buffer pool. This reduce the number of buffers that can be allocated, e.g. only 20 x 600 bytes due to remaining available memory.
C) Two networks behind the gateway: Implies requirement of 2 x 32 buffers reserved for requests.

Example for CAN network connected behind gateway (static sized message buffers):
A) Six messages with size < 4 KB received, but uses large 4 KB buffers, 6 x 4 KB = 24 KB, more memory than the minimum requirement are needed to be able to allocate more smaller buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers"*.

681747 v1 Minmum number of buffers for responses

Version history:
(1):

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.

Source:

Verification Method: Test

Legacy ID:

For each network behind the GW: If the total size of all current requests are less or equal to CAN: 4KB RAM in both bootloader mode and application mode, the GW shall be able to allocate at least CAN: 32 buffers for responses.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for responses.

If using static sized message buffers and/or not possible to reserve buffer pool only for responses: OEM and the ECU supplier shall come to an agreement on the number of buffers to achieve similar performance.

Example (dynamic sized message buffers and 8 KB buffer pool):
A) One network behind the gateway: Several 100 bytes messages received, 32 x 100 bytes = total size 3200 bytes < 4 KB, at least 32 buffers shall be possible to allocate.
B) One network behind the gateway: One 2 KB message received -> 2 KB left in buffer pool. This reduce the number of buffers that can be allocated, e.g. only 20 x 100 bytes due to remaining available memory.
C) Two networks behind the gateway: Implies requirement of 2 x 32 buffers reserved for responses.

Example (static sized message buffers):
A) One message with size < 4 KB received, but use large 4 KB buffers, 1 x 4 KB = 4 KB, more memory than the minimum requirement are needed to be able to allocate more smaller buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers".*

4.9.1.16 Zone Network Gateway - CAN-CAN

•Gateway allocation of communication channels

4.9.1.16.1 Gateway allocation of communication channels

4.9.1.16.1.1 Backbone network - Server side

This section will describe and give grounds for the following requirements:

- REQPROD 661444 Transmit(Tx) CCs on Backbone Network (Server side requirement)
- REQPROD 661443 Receive(Rx) CCs on Backbone Network (Server side requirement)

It should always be guaranteed when allocating a transmit (Tx) communication channel (CC) on the sender side, the receiver side should be able to allocate a receive (Rx) communication channel (CC), so that no data (first frame) will be lost.

There is no handshake mechanism when allocating a Tx CC on the sender side with the allocation of an Rx CC on the receiver side, hence it must always be assured that a Rx CC can be allocated to receive at least the first frame so that no data will be lost.

Allocation of transmit (Tx) communication channels (CCs)
Zone Network GW for responses to Backbone Network:

- The Zone Network GW should have three(3) communication channels (CCs) for sending responses to the Vehicle GW for each connected network behind the gateway.

Allocation of receive (Rx) communication channels (CCs)
Zone Network GW for requests from Backbone Network:

- The Zone Network GW should have at least the same number of receive (Rx) communication channels (CCs) as transmit (Tx) communication channels (CCs) in the Vehicle GW (Note: The number of Rx CCs are porject specific).
- The Zone Network GW must at able to receive and handle first frame in a request on all Rx CCs.
- If there is no buffer in the buffer pool available for receiving the complete request on the CC, the Zone Network GW should send flow control WAIT until a buffer that fits the request is available. When a buffer is available, the Zone Network GW should send a flow control CTS for receiving the remaining part of the request.

Note: When dimension the system with number of ECUs, max size of complete response and number of complete response that can be allocated from the buffer pool, it must be assured that flow control WAIT frames can be sent long enough time so that the sending side does not time-out.

Note: If it is known that the Zone Network GW will have less than three(3) ECUs connected, it may be allowed with a deviation from the above stated requirements, but that needs to be approved by OEM.

See Figure: *"Allocation communication channels (CCs) on Backbone Network - Server side"*.

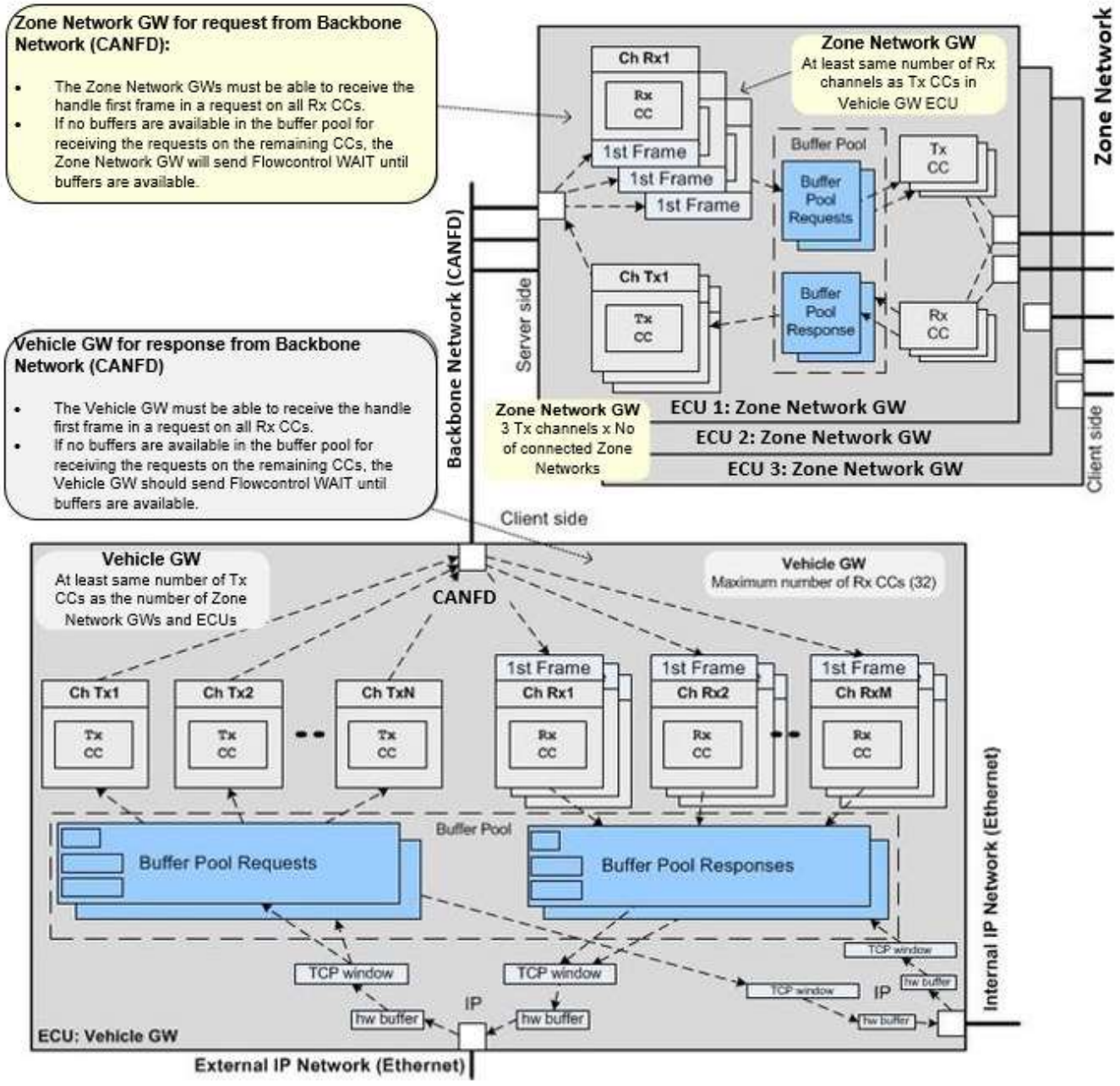


Figure: Allocation communication channels (CCs) on Domain Network - Server side

661443 v2 **Receive(Rx) CCs on Backbone Network**

Version history:
(2):

Purpose: Ensure that no diagnostic requests are lost, a first frame shall always be stored, and dependent on available buffers flow control CTS or WAIT should be sent.

Source:

Verification Method: Test

Legacy ID: 238934v2

For each network behind the GW: The Zone Network GW shall be able to open at least three(3) receive (Rx)

communication channels (CCs) for receiving diagnostic request from the Backbone Network (CANFD) in addition to one(1) Rx CC for request to the GW and one(1) Rx CC for functional request (testerPresent).

The ECU supplier shall confirm the exact number of communication channels for the specific project, e.g. for a GW with many ECUs behind the GW this number may need to be increased due to performance requirements.

Example:
One network behind the gateway: 3 + 1 + 1(testerPresent) = 5 Rx Communication Channels on the Backbone.
Two networks behind the gateway: 2 x 3 + 1 + 1(testerPresent) = 8 Rx Communication Channels on the Backbone

661444 v2 **Transmit(Tx) CCs on Backbone Network**

Version history:
(2):

Purpose: It should be possible to send responses so that timing requirement for reading information from ECUs can be fulfilled, e.g. DRO requirement.
Source:
Verification Method: Test
Legacy ID: 238933v2

For each network behind the GW: The Zone Network GW shall be able to open three(3) and not more than three(3) transmit (Tx) communication channels (CCs) for sending diagnostic responses on the Backbone Network (CANFD) to the Vehicle GW (Edge Node) in addition to one(1) Tx CC for response from the GW.

The ECU supplier shall confirm the exact number of communication channels for the specific project, e.g. for a GW with many ECUs behind the GW this number may need to be increased due to performance requirements.

Rationale:
If more responses shall be sent than available Tx CCs, responses need to be queued until a Tx CC is released and available for next response etc.

Note:
There is a limitation in how many parallel responses the Vehicle GW can receive, hence limitation of parallel Tx CCs is needed in GW for responses on a functional request.

Example:
One network behind the gateway: 1 + 3 = 4 Tx Communication Channels on the Backbone.
Two networks behind the gateway: 1+ 2 x 3 = 7 Tx Communication Channels on the Backbone.

4.9.1.16.1.2 Zone Network - Client side

This section will describe and give grounds for the following requirements:

- REQPROD 661446 Transmit(Tx) CCs on Zone network
- REQPROD 661445 Receive(Rx) CCs on Zone network

It should always be guaranteed when allocating a transmit (Tx) communication channel (CC) on the sender side, the receiver side should be able to allocate a receive (Rx) communication channel (CC), so that no data (first frame) will be lost.

There is no handshake mechanism when allocating a Tx CC on the sender side with the allocation of an Rx CC on the receiver side, hence it must always be assured that a Rx CC can be allocated to receive at least the first frame so that no data will be lost.

Allocation of transmit (Tx) communication channels (CCs)
Zone Network GW for requests to Zone Network:

- The Zone Network GW should have three(3) communication channels (CCs) for sending physical request to the ECUs on the Zone Network

Allocation of receive (Rx) communication channels (CCs)

Zone Network GW for responses from Zone Network:

- The Zone Network GW should have same number of receive (Rx) communication channels (CCs) as transmit (Tx) communication channels (CCs) on the Zone Network (Note: The number of Rx CCs are project specific, e.g. same number as ECUs connected on the Zone Network).
- The Zone Network GW must at able to receive and handle first frame from all ECUs.
- If there is no buffer in the buffer pool available for receiving the complete response on the CC, the Zone Network GW should send flow control WAIT until a buffer that fits the response is available. When a buffer is available, the Zone Network GW should send a flow control CTS for receiving the remaining part of the response.

Note: When dimension the system with number of ECUs, max size of complete response and number of complete response that can be allocated from the buffer pool, it must be assured that flow control WAIT frames can be sent long enough time so that the sending side does not time-out.

Note: If it is known that the Zone Network GW will have less than three(3) ECUs connected, it may be allowed with a deviation from the above stated requirements, but that needs to be approved by OEM.

See Figure: "Allocation communication channels (CCs) on Zone Network".

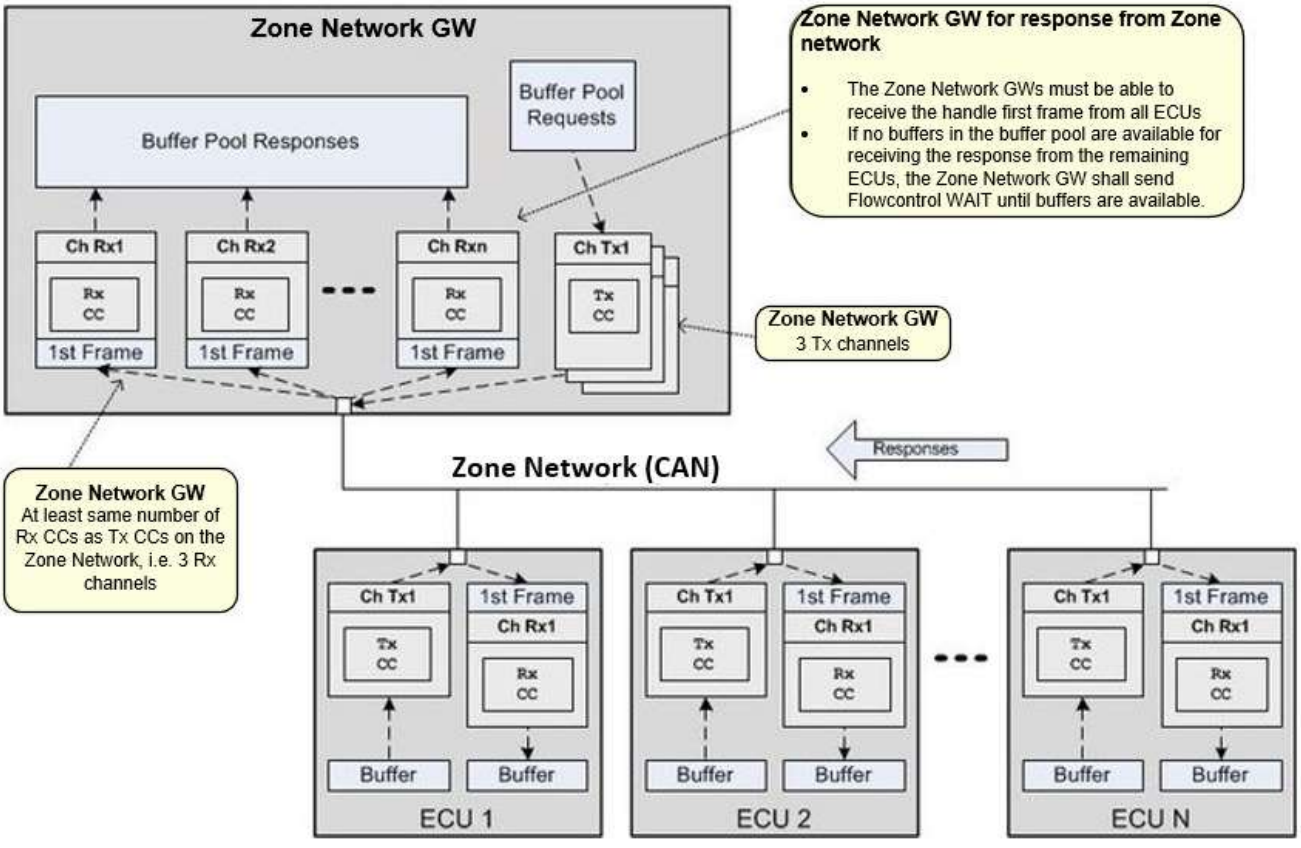


Figure: Allocation communication channels (CCs) on Zone Network.

661445 v2	Receive(Rx) CCs on Zone network

Version history:
(2):

Purpose: Ensure that no diagnostic responses are dropped, a first frame shall always be stored, and dependent on available buffers flow control CTS or WAIT should be sent.

Source:
Verification Method: Test
Legacy ID: 238973v2

The Zone Network GW shall be able to open as many receive (Rx) communication channels (CCs) for receiving diagnostic responses as the total number of transmit (Tx) communication channels (CCs) that can be open by all ECUs on the Zone network.
The ECU supplier shall confirm with OEM the exact number of communication channels for the specific project.

Comment:
The number of Rx CCs are project specific.
The number of Rx CCs will be the number of ECUs on the Zone network, since the requirement on Tx communication channels on a ECU is one (1).

661446 v3 **Transmit(Tx) CCs on Zone network**

Version history:
(3): Revised description of errors in Purpose.

Purpose: To match the Rx communication channel requirement on the Backbone Network (CANFD), where three(3) complete request could be received without flow control WAIT, Zone Network GW should be able to forward the requests as soon as possible.
Source:
Verification Method: Test
Legacy ID: 238936v2

For each network behind the GW: The Zone Network GW shall be able to open three (3) transmit (Tx) communication channels (CCs) for sending diagnostic physical requests to the ECUs on the Zone network.
For each network behind the GW: The Zone Network GW shall be able to open one additional transmit (Tx) communication channel (CC) for functional requests.

The ECU supplier shall confirm with OEM the exact number of communication channels for the specific project.

•Gateway latency requirements

4.9.1.16.2 Gateway latency requirements

[GW_6] DoCAN specifies requirement on transport protocol level as e.g. maximum time between consecutive frames, N_Cs and C_Cs, performance requirements.

How many CAN L-SDUs that can be packed in one CANFD L-SDU for requests or responses depends on the type of CAN communication (Classic CAN or CAN FD), see *[GW_6] DoCAN*.

4.9.1.16.2.1 Diagnostic requests CAN to CAN

This section will describe and give grounds for the following requirements:

- REQPROD 661447 Latency CANFD Backbone to CAN Zone, GW_Backbone_CANFD-Zone_CAN_L

The Zone Network GW, CANFD to CAN, shall not wait for the complete request before the request is started to be transmitted on the Zone Network (CAN). Since the Backbone Network CANFD L-SDU is greater or at least same size as a Zone Network CAN L-SDU, the transmission on CAN shall start as soon as possible after the Backbone Network CANFD L-SDU is received.

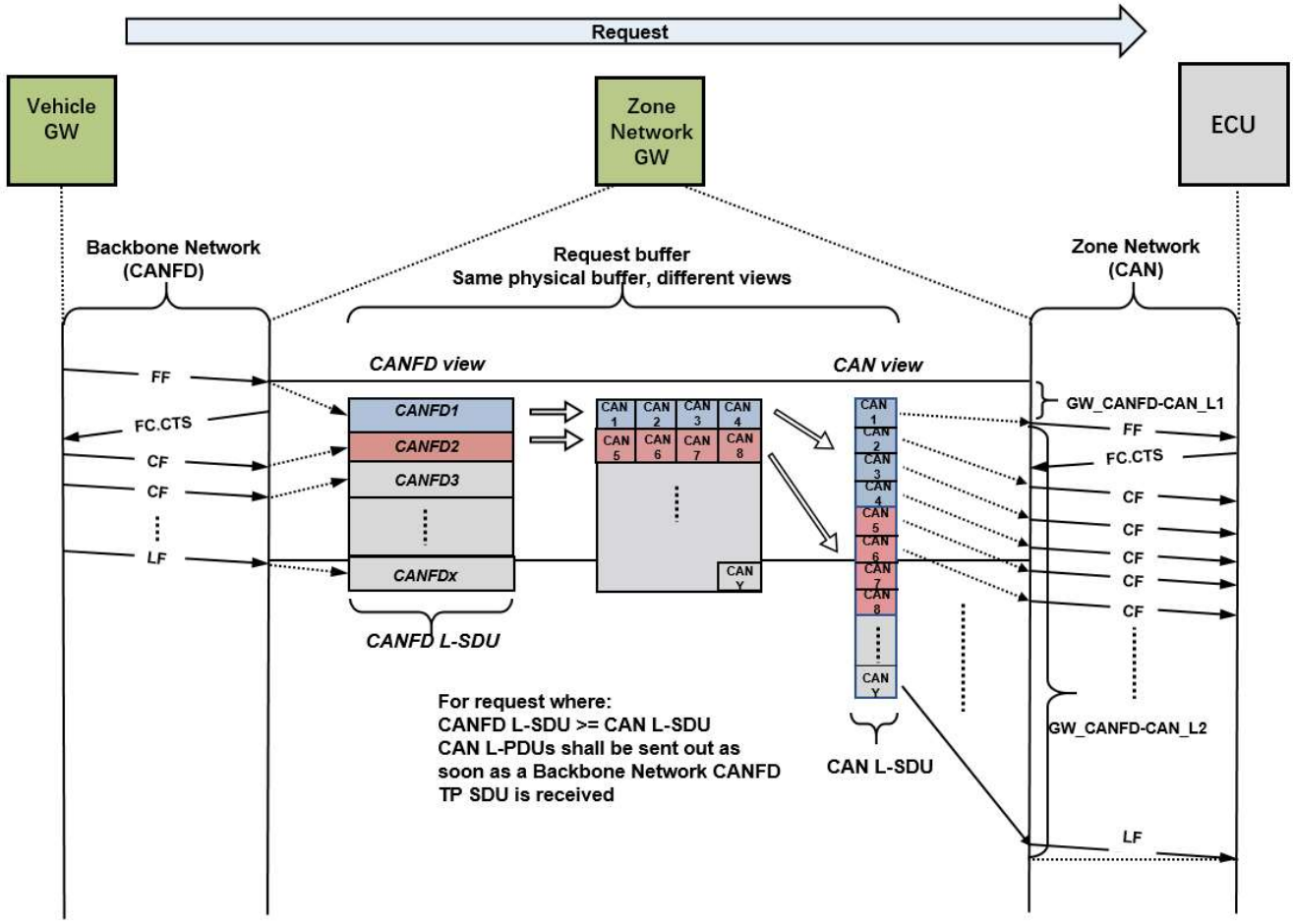


Figure: Zone Network GW: CANFD to CAN

Definition of latency timing parameter:

GW_CANFD-CAN_L1: Time between receiving the first frame in request from Backbone Network (CANFD) until first frame is sent out on Domain Network (CAN).

GW_FR-CAN_L2: Time for transfer the complete request on Zone Network (CAN).
This time depends on:

- Size of the message, i.e. how many frames that shall be sent.
- Timing parameters for the Zone Network (CAN) specified in *[GW_6] DoCAN*.

Flow control WAIT can only be sent a limited time, so GW_CANFD-CAN_L2 has impact on how the system shall be dimension regarding parallel communication channels and how many messages that can be received without sending Flow control WAIT frames.

Timing Parameter	SWDL	DIAG
	Max Value	Max Value
GW_CANFD-CAN_L1	10 ms	10 ms
GW_CANFD-CAN_L2	See [*] and Formula(1)	See [*] and Formula(1)
Formula(1): · unsegmented messages: N_As · segmented messages: N_As + N_Bs + ((Message size) – (L-SDU size))/(L-SDU size) * N_Cr		

Table: Domain Network GW: FlexRay to CAN request timing parameters

***:** *[GW_6] DoCAN*

661447 v2 **Latency CAN Backbone to CAN Zone, GW_Backbone_CAN-Zone_CAN_L**

Version history:
(2):

Purpose: Clarify and define a maximum latency time.
Source:
Verification Method: Test
Legacy ID: 238993v1

If a communication channel can be allocated, time between receiving the first frame in request from CANFD Backbone network until first frame is sent out on CAN Zone network shall be max 10ms.

4.9.1.16.2.2 Diagnostic responses CAN to CAN

How many CAN L-SDUs that can be packed in one CANFD L-SDU for responses depends on the type of CAN communication (Classic CAN or CAN FD), see *[GW_6] DoCAN, [GW_4] UDSonCAN* and *[GW_5] Bootloader - UDSonCAN*.

661448 v2 **Latency CAN Zone to Backbone CAN, GW_Backbone_CAN-Zone_CAN_L**

Version history:
(2):

Purpose: Clarify and define a maximum latency time.
Source:
Verification Method: Test
Legacy ID: 238995v1

If a communication channel can be allocated, time between receiving enough data to pack a CANFD frame until first frame is sent out on CANFD backbone shall be max 10ms.

Rationale:
If the Zone network is a CANFD network there is immediately enough data for sending a CANFD frame on the backbone (the L-SDU on the Zone network is the same size or larger than on the Backbone network).
If the Zone network is a Classic CAN network several CAN frames shall be received before sending a CANFD frame on the backbone since CANFD L-SDU is larger than Classic CAN L-SDU.

4.9.1.16.2.2.1 Diagnostic Response CAN to CANFD, when CAN L-SDU < CANFD L-SDU

Diagnostic Response CAN to CANFD, when CAN L-SDU < CANFD L-SDU

The protocol overhead on CANFD is larger than the protocol overhead on Classic CAN, so in most use cases it makes sense to pack several CAN frames into one CANFD frame to get an optimization between bandwidth utilization on CANFD and timing. So normally the CANFD L-SDU is greater than CAN L-SDU, and several CAN frames will be received and packed before frames are sent out on the CANFD backbone, see example in Figure "*Zone Network GW: CAN to CANFD response, different SDU size*".

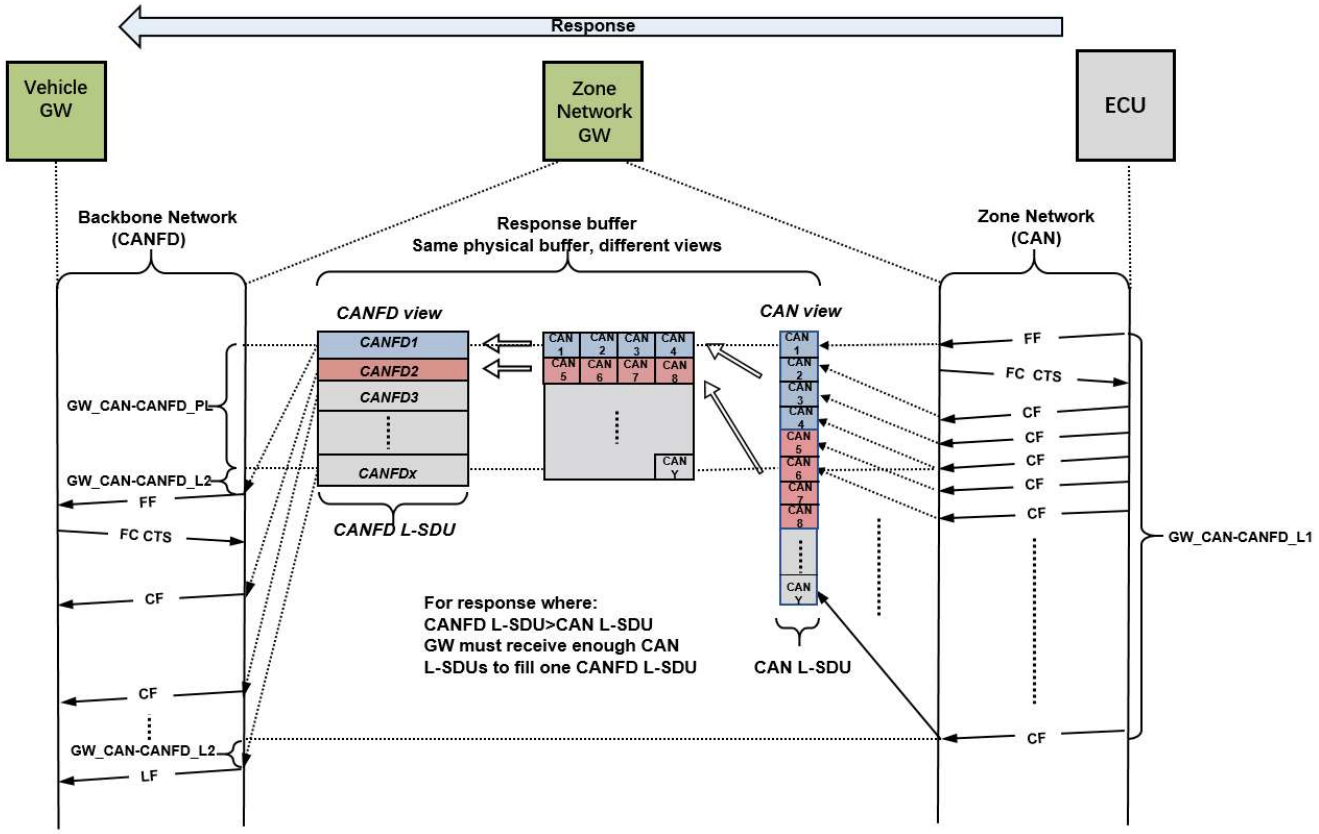


Figure: Zone Network GW: CAN to CANFD response, different SDU size

Definition of latency timing parameters:
GW_CAN-CANFD_PL: Packing Latency, Time to receive enough CAN frames to fill a CAND frame. How many CAN L-SDUs that will be packed in a CANFD L-SDU depends on the type of CAN communication (Classic CAN or CAN FD), see *[GW_6] DoCAN.*

GW_CAN-FR_L1: Time for transfer the complete response on Zone Network (CAN).
This time depends on:

- Size of the message, i.e. how many CAN frames that will be sent.
- Timing parameters for the Zone Network (CAN) specified in *[GW_6] DoCAN*].

GW_CAN-FR_L2: Time between receiving the last frame from Zone Network (CAN) to pack a CANFD frame until this frame is sent out on Backbone Network (CANFD).

Flow control WAIT can only be sent a limited time, so GW_CAN-CANFD_L1 has impact on how the system shall dimension regarding parallel communication channels and how many messages that can be received without sending Flow control WAIT frames.

Timing Parameter	SWDL	DIAG
	Max Value	Max Value
GW_CAN-CANFD_PL	See [*] and Formula(2)	See [*] and Formula(2)
GW_CAN-CANFD_L1	See [*] and Formula(1)	See [*] and Formula(1)
GW_CAN-CANFD_L2	10 ms	10 ms

Formula(1):
· Unsegmented messages: N_As
· Segmented messages: N_As + N_Bs + ((Message size) – (L-SDU size))/(L-SDU size) * N_Cr

Formula(2):

·	Approximately: (CANFD L-SDU size) / (CAN L-SDU size) * N_Cr

Table: Zone Network GW: CAN to CANFD Response timing parameters, CANFD L-SDU>CAN L-SDU

:[GW_6] DoCAN*

4.9.1.16.2.2.2 Diagnostic Response CAN to CANFD, when CAN L-SDU = CANFD L-SDU

Diagnostic Response CAN to CANFD, when CAN L-SDU = CANFD L-SDU

For most use cases, several Classic CAN L-SDUs will be packed in one CANFD L-SDU as showed in section '*Diagnostic Response CAN to CANFD, when CAN L-SDU < CANFD L-SDU*', since the CANFD L-SDU is normally larger than a CAN L-SDU.

The size of CANFD L-SDU is not specified in this document. In this case CANFD L-SDU would have the same size as CAN L-SDU, the CANFD L-SDU should be sent out as soon as the CAN L-SDU is received. See example in Figure: " *Zone Network GW: CAN to CANFD response same SDU size, segmented response"*.

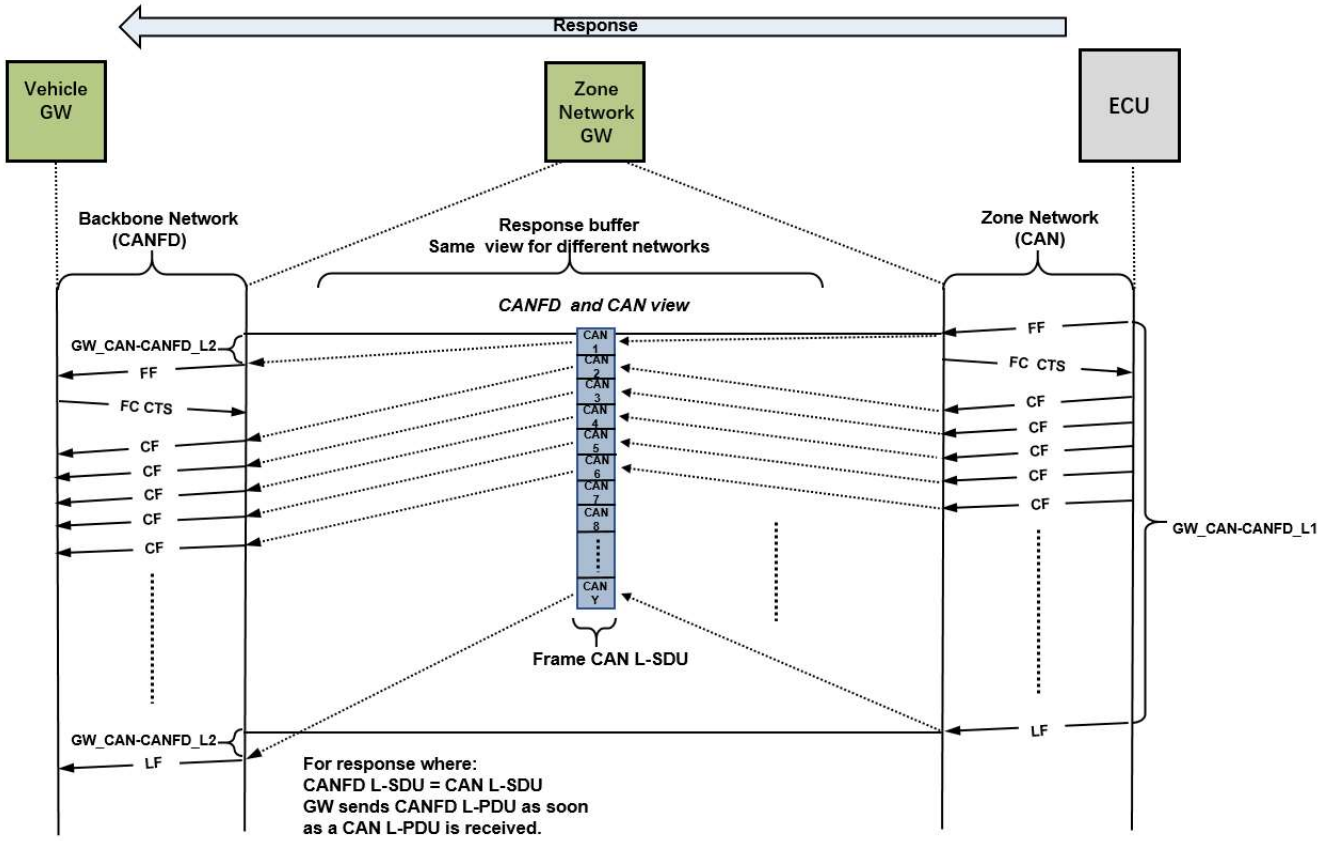


Figure: Zone Network GW: CAN to CANFD response same SDU size, segmented response

Definition of latency timing parameters specified in Table *Zone Network GW: CAN to CANFD response timing parameters, CANFD L-SDU=CAN L-SDU*:

GW_CAN-CANFD_PL: Packing Latency is zero if CANFD L-SDU=CAN L-SDU, i.e. N/A.

GW_CAN-CANFD_L1: Time for transfer the complete request on Zone Network (CAN).
This time depends on:

- Size of the message, i.e. how many CAN frames that will be sent.
- Timing parameters for the Zone Network (CAN) specified in [GW_6] *DoCAN*.

GW_CAN-CANFD_L2: Time between receiving a frame from Zone Network (CAN) until the frame is sent out on Backbone Network (CANFD).

Flow control WAIT can only be sent a limited time, so GW_CAN-CANFD_L1 has impact on how the system shall dimension regarding parallel communication channels and how many messages that can be received without sending Flow control WAIT frames.

Timing Parameter	SWDL	DIAG
	Max Value	Max Value
GW_CAN-CANFD_PL	N/A	N/A
GW_CAN-CANFD_L1	See [*] and Formula(1)	See [*] and Formula(1)
GW_CAN-CANFD_L2	10 ms	10 ms
Formula(1): · Unsegmented messages: N_As · Segmented messages: N_As + N_Bs + ((Message size) – (L-SDU size))/(L-SDU size) * N_Cr		

Table: Zone Network GW: CAN to CANFD response timing parameters, CANFD L-SDU=CAN L-SDU

*: *[GW_6] DoCAN*

4.9.1.16.2.2.3 Periodic responses CAN to CANFD

Periodic responses CAN to CANFD

A periodic response received from Zone Network (CAN), message type #2 – UUDT on CAN, will be sent out as fast as possible on the Backbone Network (CANFD).

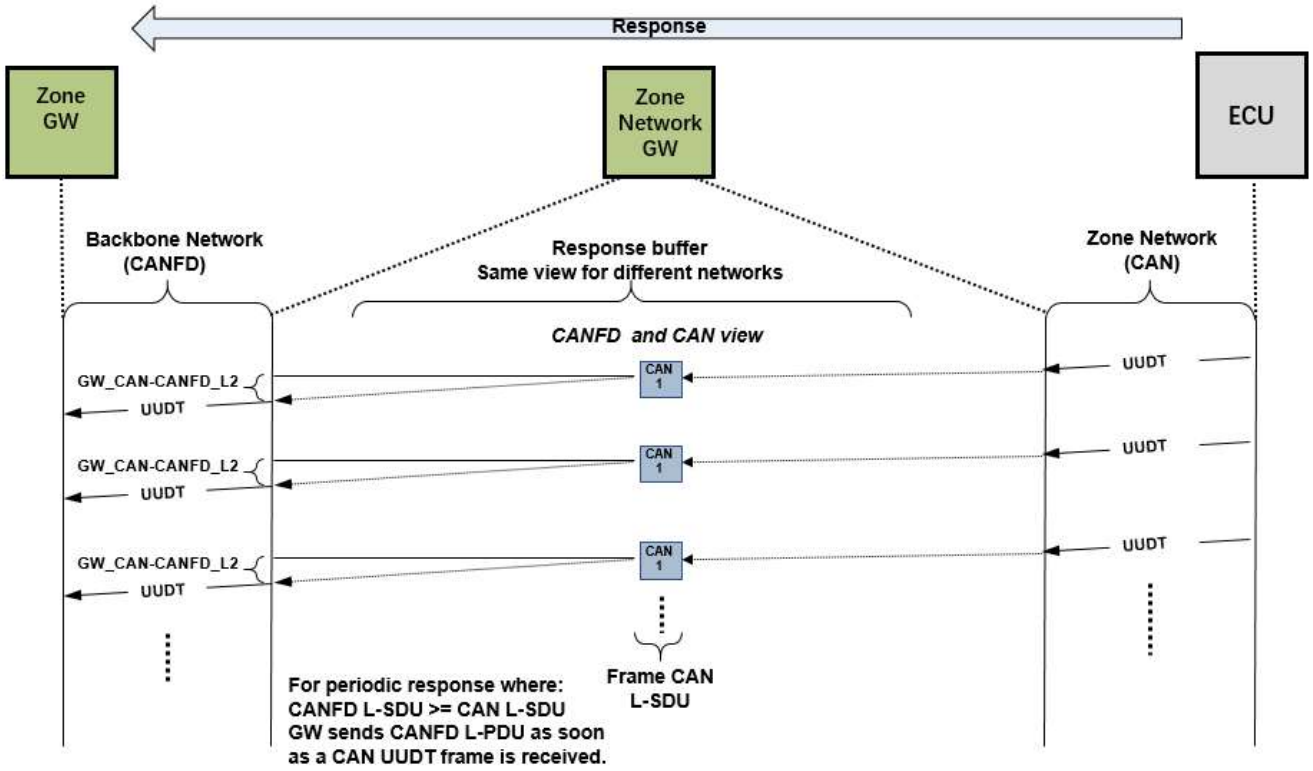


Figure: Zone Network GW: CAN to CANFD periodic responses when CANFD L-SDU = CAN L-SDU

Definition of latency timing parameters:

GW_CAN-FR_L2: Time between receiving a response in an UUDT frame from Zone Network (CAN) until the complete response is sent out on Backbone Network (CANFD).

Timing Parameter	SWDL	DIAG

	Max Value	Max Value
GW_CAN-CANFD_PL	N/A	N/A
GW_CAN-CANFD_L1 (CANFD L-SDU = CAN L-SDU)	N/A	N/A
GW_CAN-CANFD_L2 (CANFD L-SDU = CAN L-SDU)	N/A	10 ms

Table: Zone Network GW: CAN to CANFD periodic response timing parameters

4.9.1.17 Zone Network Gateway DIAG

•Gateway allocation of buffers for DIAG

4.9.1.17.1 Gateway allocation of buffers for DIAG

This section will describe and give grounds for the following requirements:

- REQPROD 661434 Minimum RAM for request buffers
- REQPROD 661432 Minimum number of buffers for requests
- REQPROD 661430 Large buffer size for request
- REQPROD 661435 Minimum RAM for response buffers
- REQPROD 661433 Minimum number of buffers for responses
- REQPROD 661431 Large buffer size for response

Available memory for allocation of buffers for DIAG messages (UDS Layer) are a compromise between hardware cost and performance, e.g. how much traffic in parallel the gateway should be able to handle.
For DIAG messages the Zone Network GW will execute in application mode, and have to share resources as RAM for buffers with other applications.

For each network behind the GW: The Zone Network GW should be able to allocate at least CANFD or CAN: 4 KB RAM reserved for buffers for requests, and at least 8 KB RAM reserved for buffers for responses.

The Zone Network GW must be able to allocate 4 KB buffer for diagnostic messages.

There should be a minimum number of available buffers so that the gateway can handle parallel communication, as long as the total size for buffers is not exceeded. If there is a small request or response the smallest possible buffer should be used, e.g. do not allocate a 4 KB buffer for a 2 bytes long request. This specification does not specify exactly which buffer allocation algorithm that should be used, this is up to the implementer, but there should be a minimum of available buffers for small requests.

To insure that parallel DIAG can be performed for small messages (dynamic sized buffer allocation):

- For each network behind the GW: If the total size of requests are less or equal to 4 KB RAM, the minimum available request buffers should be 32.
- For each network behind the GW: If the total size of responses are less or equal to 8 KB RAM, the minimum available response buffers should be 32.

See example in Figure: " *Zone Network Gateway Buffers for DIAG"*.

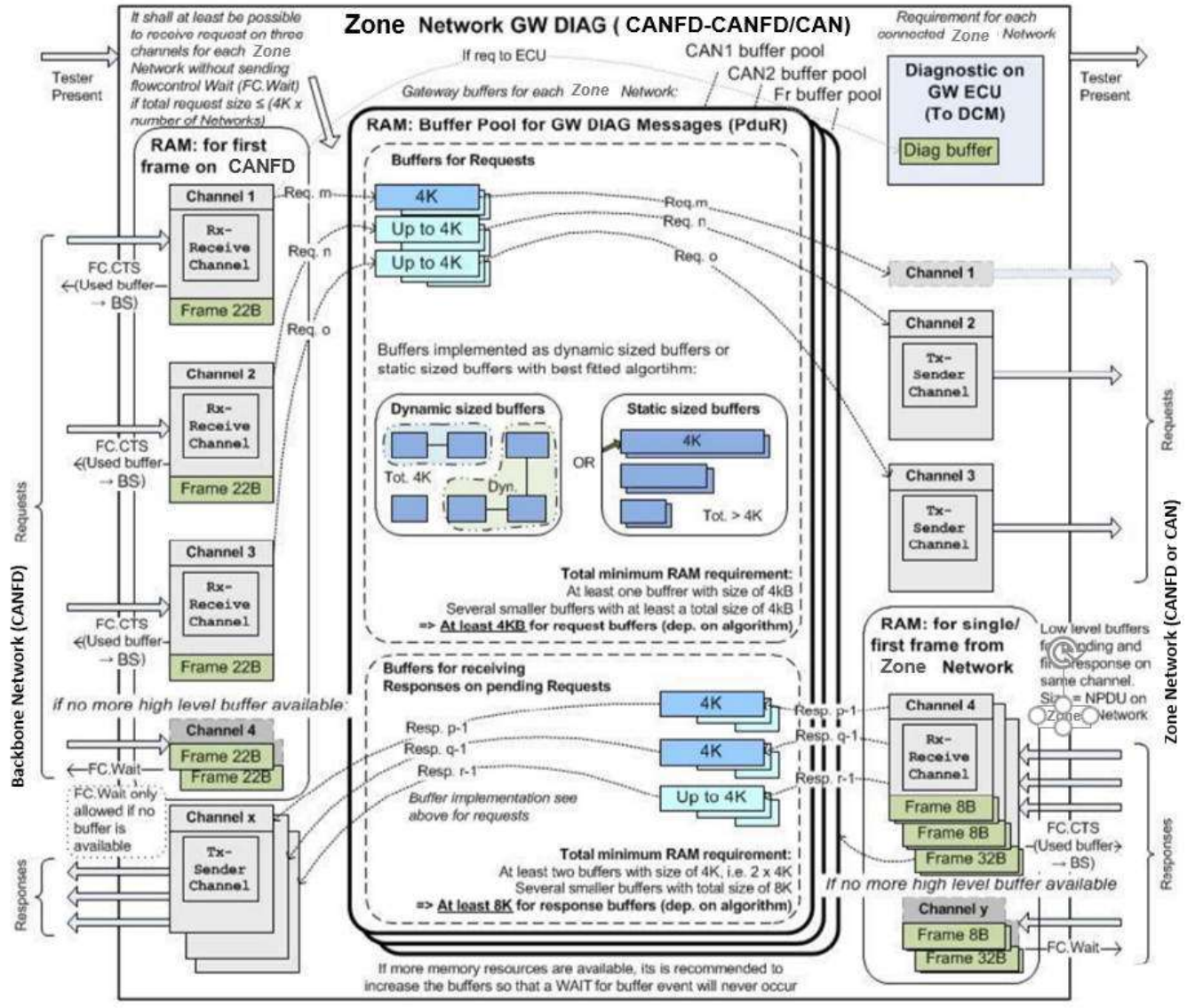


Figure: Zone Network Gateway Buffers for DIAG

661430 v2 Large buffers for request

Version history:
(2):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication.

Source:

Verification Method: Test

Legacy ID: 77624v4

For each network behind the GW: In application mode, the Zone Network GW shall be able to allocate up to 4 kB large buffers for request.

661431 v4 Large buffers for response

Version history:
(4): Description update.

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication.
Source:
Verification Method: Test
Legacy ID: 77627v5

For each network behind the GW: In application mode, the Zone GW shall be able to allocate up to 4 kB large buffers for response.

Note: for one specifical network behind the GW: 8 kB for 1 UDS response read out of EDR data.

661434 v2 Minimum RAM for request buffers

Version history:
(2):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication.
Source:
Verification Method: Test
Legacy ID: 77621v4

For each network behind the GW: In application mode, the Zone Network GW shall be able to reserve at least a 4 KB RAM buffer pool only for requests where several smaller buffers can be allocated from.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for requests.
If using static sized message buffers and/or not possible to reserve buffer pool only for requests: GRI Basetech Team and the ECU supplier shall come to an agreement on used buffer sizes to achieve similar performance, more memory for buffers are needed.

Note:
There is a separate requirement for minimum number of buffers that can be allocated form this buffer pool.

Example (dynamic sized message buffers and 4 KB buffer pool):
A) One network behind the gateway: 4 KB RAM reserved for requests (with possibility to allocate different buffer sizes).
B) Two networks behind the gateway: Implies requirement of 2 x 4 KB RAM reserved for requests.

Example (static sized message buffers):
A) One message with size < 4 KB received, but uses large 4 KB buffer, 1 x 4 KB = 4 KB, more memory than the minimum requirement are needed to be able to fulfil the requirement for number of buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers"*. For static sized message buffers it can be larger gaps in the usable memory, hence this minimum RAM requirement may need to be increased dependent on how the static sized message buffers are chosen.

661435 v2 Minimum RAM for response buffers

Version history:
(2):

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication.

Source:
Verification Method: Test
Legacy ID: 77625v4

For each network behind the GW: In application mode, the Zone Network GW shall be able to reserve at least a 8 KB RAM buffer pool only for responses where several smaller buffers can be allocated from.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for responses.
If using static sized message buffers and/or not possible to reserve buffer pool only for responses: GRI Basetech Team and the ECU supplier shall come to an agreement on used buffer sizes to achieve similar performance, more memory for buffers are needed.

Note:
There is a separate requirement for minimum number of buffers that can be allocated form this buffer pool.

Example (dynamic sized message buffers and 8 KB buffer pool):
A) One network behind the gateway: 8 KB RAM reserved for responses (with possibility to allocate different buffer sizes).
B) Two networks behind the gateway: Implies requirement of 2 x 8 KB RAM reserved for responses.

Example (static sized message buffers):
A) Two messages with size < 4 KB received, but uses large 4 KB buffers, 2 x 4 KB = 8 KB, more memory than the minimum requirement are needed to be able to fulfil the requirement for number of buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers"*. For static sized message buffers it can be larger gaps in the usable memory, hence this minimum

661432 v2 Minimum number of buffers for requests

Version history:
(2):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication.
Source:
Verification Method: Test
Legacy ID: 77622v3

For each network behind the GW: In application mode and if the total size of all current requests are less or equal to 4 KB RAM, the Zone Network GW shall be able to allocate at least 32 buffers for requests.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for requests.
If using static sized message buffers and/or not possible to reserve buffer pool only for requests: GRI Basetech Team and the ECU supplier shall come to an agreement on the number of buffers to achieve similar performance.

Example (dynamic sized message buffers and 4 KB buffer pool):
A) One network behind the gateway: Several 100 bytes messages received, 32 x 100 bytes = total size 3200 bytes < 4 KB, at least 32 buffers shall be possible to allocate.
B) One network behind the gateway: One 2 KB message received -> 2 KB left in buffer pool. This reduce the number of buffers that can be allocated, e.g. only 20 x 100 bytes due to remaining available memory.
C) Two networks behind the gateway: Implies requirement of 2 x 32 buffers reserved for requests.

Example (static sized message buffers):
A) One messages with size < 4 KB received, but uses large 4 KB buffer, 1 x 4 KB = 4 KB, more memory than the minimum requirement are needed to be able to allocate more smaller buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers".*

661433 v2 **Minimum number of buffers for responses**

Version history:
(2):

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication.
Source:
Verification Method: Test
Legacy ID: 77626v3

For each network behind the GW: In application mode and if the total size of all current responses are less or equal to 8 KB RAM, the Zone Network GW shall be able to allocate at least 32 buffers for responses.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for responses.
If using static sized message buffers and/or not possible to reserve buffer pool only for responses: GRI Basetech Team and the ECU supplier shall come to an agreement on the number of buffers to achieve similar performance.

Example (dynamic sized message buffers and 8KB buffer pool):
A) One network behind the gateway: Several 200 bytes messages received, 32 x 200 bytes = total size 6400 bytes < 8 KB, at least 32 buffers shall be possible to allocate.
B) One network behind the gateway: One 4 KB message received -> 4 KB left in buffer pool. This reduce the number of buffers that can be allocated, e.g. only 20 x 200 bytes due to remaining available memory.
C) Two networks behind the gateway: Implies requirement of 2 x 32 buffers reserved for responses.

Example (static sized message buffers):
A) Two messages with size < 4 KB received, but uses large 4 KB buffers, 2 x 4 KB = 8 KB, more memory than the minimum requirement are needed to be able to allocate more smaller buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: "Example buffer pool with dynamic sized message buffers" and Figure: "Example buffer pool with static sized message buffers".

4.9.1.18 Zone Network Gateway SWDL

•Gateway allocation of buffers for SWDL

4.9.1.18.1 Gateway allocation of buffers for SWDL

This section will describe and give grounds for the following requirements:

- REQPROD 661440 Minimum RAM for request buffers
- REQPROD 661438 Minimum number of buffers for requests
- REQPROD 661442 Support for queued request
- REQPROD 661436 Large buffer size for request
- REQPROD 661441 Minimum RAM for response buffers
- REQPROD 661439 Minimum number of buffers for responses
- REQPROD 661437 Large buffer size for response

Available memory for allocation of buffers for SWDL messages (UDS Layer) are a compromise between hardware cost and performance, e.g. how much traffic in parallel the gateway should be able to handle.
For SWDL messages the Zone GW will execute a bootloader software, PBL or SBL, **also will execute in application mode** (i.e. expected that large RAM resources are available).

For each network behind the GW: **In program mode,** the Zone Network GW should be able to allocate at least CANFD or CAN:24 KB RAM reserved for buffers for requests, and at least 4 KB RAM reserved for buffers for responses.
For each network behind the GW: **In application mode,** the Zone Network GW should be able to allocate at least CANFD or CAN: 12 kB RAM reserved for buffers for requests, and at least 4 KB RAM reserved for buffers for responses.

The Zone Network GW must be able to allocate 4 KB buffer for diagnostic messages.

There should be a minimum number of available buffers so that the gateway can handle parallel communication, as long as the total size for buffers is not exceeded. If there is a small request or response the smallest possible buffer should be used, e.g. do not allocate an 4 KB buffer for a 2 bytes long request. This specification does not specify exactly which buffer allocation algorithm that should be used, this is up to the implementer, but there should be a minimum of available buffers for small requests.

To insure that parallel SWDL can be performed for small messages (dynamic sized buffer allocation):

- For each network behind the GW: If the total size of requests are less or equal to CANFD or CAN:24 KB RAM, the minimum available request buffers should be 32.
- For each network behind the GW: If the total size of responses are less or equal to 4 KB RAM, the minimum available response buffers should be 32.

For supporting queued requests, if the request size is less or equal to 24 kB, the Zone Network GW should able to receive two(2) queued request on three(3) communication channels without sending flow control Wait (FC.Wait).

See example in Figure " *Zone Network Gateway Buffers for SWDL"*.

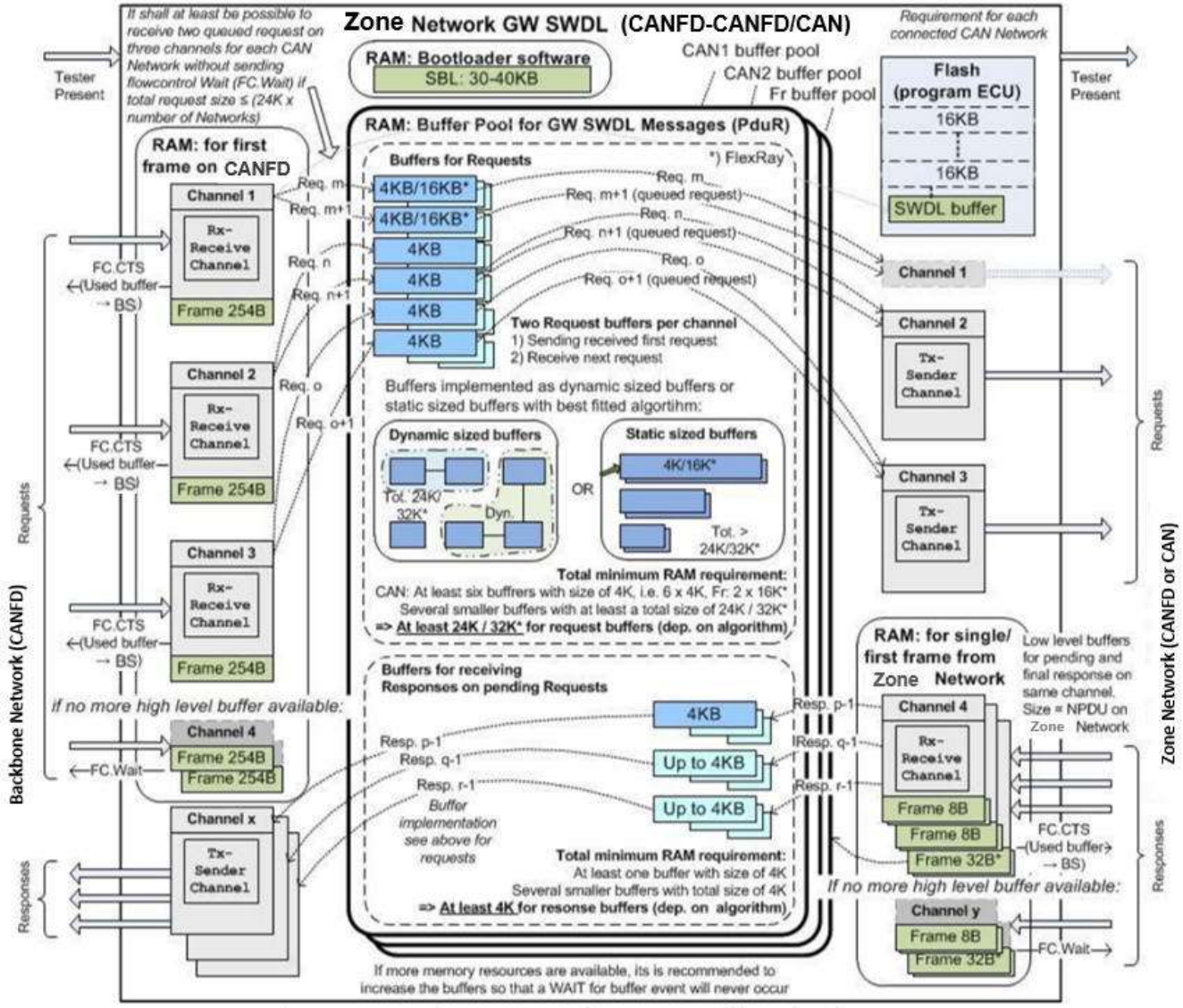


Figure: Zone Network Gateway Buffers for SWDL

661436 v2 **Large buffers for request**

Version history:
(2):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:
Verification Method: Test
Legacy ID: 77445v5

For each network behind the GW: In bootloader mode, **also in application mode**, the Zone Network GW shall be able to allocate up to 4 KB large buffers for request for CANFD or CAN networks.

661437 v2 **Large buffers for response**

Version history:
(2):

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:
Verification Method: Test
Legacy ID: 77448v4

For each network behind the GW: In bootloader mode, **also in application mode**, the Zone Network GW shall be able to allocate up to 4 kB large buffers for response.

661440 v4 Minimum RAM for request buffers

Version history:
(4):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:
Verification Method: Test
Legacy ID: 77442v5

For each network behind the GW: In bootloader mode, the Zone Network GW shall be able to reserve at least a 24KB RAM buffer pool for each CANFD or CAN network, reserved only for requests where several smaller buffers can be allocated from.
For each network behind the GW: **In application mode**, the Zone Network GW shall be able to reserve at least a 12KB RAM buffer pool for each CANFD or CAN network, reserved only for requests where several smaller buffers can be allocated from.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for requests.
If using static sized message buffers and/or not possible to reserve buffer pool only for requests: the project and the ECU supplier shall come to an agreement on used buffer sizes to achieve similar performance, more memory for buffers are needed.

Note:
There is a separate requirement for minimum number of buffers that can be allocated form this buffer pool.

Example for CAN network connected behind gateway (dynamic sized message buffers and 24 KB buffer pool):
A) One network behind the gateway: 24 KB RAM reserved for requests (with possibility to allocate different buffer sizes).
B) Two networks behind the gateway: Implies requirement of 2 x 24 KB RAM reserved for requests.

Example for CAN network connected behind gateway (static sized message buffers):
A) Twelve messages with size < 4 KB received, but uses large 4 KB buffers, 12 x 4 KB = 48 KB, more memory than the minimum requirement are needed to be able to fulfil the requirement for number of buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers"*. For static sized message buffers it can be larger gaps in the usable memory, hence this minimum RAM requirement may need to be increased dependent on how the static sized message buffers are chosen.

661441 v2 Minimum RAM for response buffers

Version history:
(2):

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum

performance requirement for parallel communication and optimize SWDL performance.

Source:

Verification Method: Test

Legacy ID: 77446v4

For each network behind the GW: In bootloader mode, **also in application mode**, the Zone Network GW shall be able to reserve at least a 4 KB RAM buffer pool only for responses where several smaller buffers can be allocated from.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for responses.
If using static sized message buffers and/or not possible to reserve buffer pool only for response: the project and the ECU supplier shall come to an agreement on used buffer sizes to achieve similar performance, more memory for buffers are needed.

Note:
There is a separate requirement for minimum number of buffers that can be allocated form this buffer pool.

Example (dynamic sized message buffers and 4 KB buffer pool):
A) One network behind the gateway: 4 KB RAM reserved for responses (with possibility to allocate different buffer sizes).
B) Two networks behind the gateway: Implies requirement of 2 x 4 KB RAM reserved for responses.

Example (static sized message buffers):
A) One message with size < 4 KB received, but use one large 4 KB buffer, 1 x 4 KB = 4 KB, more memory than the minimum requirement are needed to be able to fulfil the requirement for number of buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers"*. For static sized message buffers it can be larger gaps in the usable memory, hence this minimum RAM requirement may need to be increased dependent on how the static sized message buffers are chosen.

661438 v3 Minimum number of buffers for requests

Version history:
(3):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.

Source:

Verification Method: Test

Legacy ID: 77443v4

For each network behind the GW: If the total size of all current requests are less or equal to 24 KB RAM for CANFD or CAN in bootloader mode, or 12 KB RAM for CANFD or CAN in application, the Zone Network GW shall be able to allocate at least 32 buffers for requests.

This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for requests.
If using static sized message buffers and/or not possible to reserve buffer pool only for requests: the project and the ECU supplier shall come to an agreement on the number of buffers to achieve similar performance.

Example for CAN network connected behind gateway (dynamic sized message buffers and 24 KB buffer pool):
A) One network behind the gateway: Several 600 bytes messages received, 32 x 600 bytes = total size 19200 bytes < 24 KB, at least 32 buffers shall be possible to allocate.
B) One network behind the gateway: Three 4 KB message received -> 12 KB left in buffer pool. This reduce the number of buffers that can be allocated, e.g. only 20 x 600 bytes due to remaining available memory.
C) Two networks behind the gateway: Implies requirement of 2 x 32 buffers reserved for requests.

Example for CAN network connected behind gateway (static sized message buffers):
A) Six messages with size < 4 KB received, but uses large 4 KB buffers, 6 x 4 KB = 24 KB, more memory than the minimum requirement are needed to be able to allocate more smaller buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers".*

661439 v2 Minimum number of buffers for responses

Version history:
(2):

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:
Verification Method: Test
Legacy ID: 77447v3

For each network behind the GW: In bootloader mode, **also in application mode**, and if the total size of all current responses are less or equal to 4 KB RAM, the Zone Network GW shall be able to allocate at least 32 buffers for responses.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for responses.
If using static sized message buffers and/or not possible to reserve buffer pool only for responses: the project and the ECU supplier shall come to an agreement on the number of buffers to achieve similar performance.

Example (dynamic sized message buffers and 4 KB buffer pool):
A) One network behind the gateway: Several 100 bytes messages received, 32 x 100 bytes = total size 3200 bytes < 4 KB, at least 32 buffers shall be possible to allocate.
B) One network behind the gateway: One 2 KB message received -> 2 KB left in buffer pool. This reduce the number of buffers that can be allocated, e.g. only 20 x 100 bytes due to remaining available memory.
C) Two networks behind the gateway: Implies requirement of 2 x 32 buffers reserved for responses.

Example (static sized message buffers):
A) One message with size < 4 KB received, but use one large 4 KB buffer, 1 x 4 KB = 4 KB, more memory than the minimum requirement are needed to be able to allocate more smaller buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers".*

661442 v2 Support for queued request

Version history:
(2):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:
Verification Method: Test
Legacy ID: 77444v3

For each network behind the GW: In bootloader mode and if the total request size is less or equal to 24kB, the Zone Network GW shall able to receive two(2) queued request on three(3) communication channels without

sending flow control Wait (FC.Wait)

4.9.1.19 Firewall CANX-CAN11 Generic

•Buffer requirements

4.9.1.19.1 Buffer requirements

661462 v1 **Firewall CANX-CAN11 Minimum buffer structure for requests**

Version history:
(1):

Purpose: Assure that buffers are availiable for OBD communication and "end-of-life activation of on-board pyrotechnic devices"
Source:
Verification Method: Test
Legacy ID: 436095v1

Firewall CANX-CAN11 shall have minimum request buffer structure as:

- 1 x 50 bytes buffer (for UDS "end-of-life activation of on-board pyrotechnic devices" request)
- 20 x 8 bytes (for OBD functional request and flow control to 20 ECUs)

661463 v2 **Firewall CANX-CAN11 Minimum buffer structure for responses**

Version history:
(2): 适配法规EDR数据（超过4KB）的读取，增加1个8kB长度的响应Buffer分配。

Purpose: Assure that buffers are availiable for OBD communication, end-of-life activation of on-board pyrotechnic devices and crash data.
Source:
Verification Method: Test
Legacy ID: 436096v1

Firewall CANX-CAN11 shall have minimum response buffer structure as:

- 20 x 200 bytes (for 20 OBD responses or 1 UDS "end-of-life activation of on-board pyrotechnic devices" response)
- 1 x 4095 bytes (for 1 UDS response read out of crash data)
- 1 x 8192 bytes (for 1 UDS response read out of EDR data)

•Client side filter

4.9.1.19.2 Client side filter

661464 v1 **Firewall CANX-CAN11 Client side filter**

Version history:
(1):

Purpose: Only gateway messages from vechicle to OBD contact that are needed for OBD, end-of-life activation of on-board pyrotechnic devices, crash data and flight recorder.
Source:
Verification Method: Test
Legacy ID: 436072v0

On the client side (side closet to target ECUs) Firewall CANX-CAN11 shall filter/hinder all 11-bit and 29-bit CAN messages except for the following.
11-bits CAN ID pass filter:

- **0x7E8 - 0x7EF** (OBD responses, according to ISO 15765-4)
- **0x7F9** (end-of-life activation of on-board pyrotechnic devices response, according to ISO 26021)
- **0x6E0 – 0x6FF** (UUDT frames for flight recorder)
- **0x520** (ECM NM frame for triggering of flight recorder)

•Timing and flow control parameters

4.9.1.19.3 Timing and flow control parameters

661465 v1 **Firewall CANX-CAN11 Flow Control parameters**

Version history:
(1):

Purpose: From scantool tester view the firewall shall behave as OBD ECUs, from ECU view the firewall shall behave as a scantool tester, as specified in ISO 15765-2 and 15764-4.
Source:
Verification Method: Test
Legacy ID: 436019v0

Flow control parameters:

- WaitFrameTransmission, N_WFTmax, counter shall be set to 0.
- BlockSize parameter shall be 0x00
- SeparationTime, STmin, shall be set to 0

Rationale:
No FlowControl wait frames are allowed.
A single FlowControl frame shall be transmitted by the Firewall CAN-CAN for the duration of a segmented transfer.
Firewall CAN-CAN shall be able to receive CAN frames with no separation time.

661466 v2 **Firewall CANX-CAN11 Timing parameter values**

Version history:
(2): 1、根据ISO 15765-4 2021版，更新网络层参数。2、对于OBD口的网关要求，详细区分车内和车外的TP层要求。

Purpose: From scantool tester view the firewall shall behave as OBD ECUs, from ECU view the firewall shall behave as a scantool tester, as specified in ISO 15765-2 and 15764-4.
Source:
Verification Method: Test
Legacy ID: 436018v0

From scantool tester view the firewall shall behave as OBD ECUs, the following paramenter shall be set:
Timeout values:

- N_As/N_Ar timeout value shall be 33ms
- N_Bs timeout value shall be 75ms
- N_Cr timeout value shall be 150ms

Performance requirement values:

- (N_Br + N_Ar) < 33ms
- (N_Cs + N_As) < 50ms (Note 2)

Note1: Detailed description of timing parameters can be found in ISO 15765-4.
Note2: For the tolerance of GW's internal network CANTp parameter N_Cr, the external network CANTp parameter N_Cs shall be less than 150ms, (N_Cs + N_As) < 150ms.

From ECU view, the firewall shall behave as a scantool tester, the following paramenter shall be set:
Timeout values:

- N_As/N_Ar timeout value shall be 50ms
- N_Bs timeout value shall be 75ms
- N_Cr timeout value shall be 150ms

Performance requirement values:

- (N_Br + N_Ar) < 50ms
- N_Cs is less than 150ms, (N_Cs + N_As) < 150ms

•Transport layer and latency

4.9.1.19.4 Transport layer and latency

661468 v1 **Firewall CANX-CAN11 Transport layer**

Version history:
(1):

Purpose: From scantool tester view the firewall shall behave as OBD ECUs, from ECU view the firewall shall behave as a scantool tester, as specified in ISO 15765-2 and 15764-4.
Source:
Verification Method: Test
Legacy ID: 436023v0

Gateway shall be implemented with a Transport layer according to ISO 15765-2 and ISO 15765-4 on each CAN network and a separate TP channel for each OBD ECU and configured as gateway on-the-fly with TP level one (1) frame.

Rationale:
The TP layer shall not wait for the complete message, as soon one (1) frame is received, start send on next the network(s) immediately.
Gateway need to be able to handle all OBD responses in parallel.

661467 v1 **Firewall CANX-CAN11 latency**

Version history:
(1):

Purpose: Set latency requirement on gateway so that OBD requirement of 50ms from scantool point-of-view can be met.
Source:
Verification Method: Test
Legacy ID: 436026v1

When a CAN SingleFrame or FirstFrame is received from one network and the CAN ID passes the filter, the gateway shall start arbitration for sending the frame on next network within 0.5ms

Rationale:

Implies gateway on-the-fly functionality in the TP layer, immediately send a SingleFrame or FirstFrame to next network.

4.9.1.20 Firewall CAN11-CAN11

•Possibilty to disable Firewall functionality

4.9.1.20.1 Possibilty to disable Firewall functionality

661469 v1 **Firewall CAN11-CAN11 disable functionality**

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 436098v0

It shall be possible to disable the Firewall CAN11-CAN11 functionality.

Rationale:
It shall be possibly to disable Firewall functionality if Firewall functionality shall be handled by another ECU, depending on network topologi. In this requirement not exactly specified how disable functionality shall be implemented but e.g. via post-build select.

•Service side filter

4.9.1.20.2 Service side filter

661470 v1 **Firewall CAN11-CAN11 Server side filter**

Version history:
(1):

Purpose: Only gateway messages from OBD contact to vechicle that are needed for OBD, end-of-life activation of on-board pyrotechnic devices and crash data.
Source:
Verification Method: Test
Legacy ID: 436074v0

On the server side (side closet to tester) Firewall CAN11-CAN11 shall filter/hinder all 11-bit and 29-bit CAN messages except for the following
11-bits CAN ID pass filter:

- **0x7DF** (OBD functional request, according to ISO 15765-4)
- **0x7E0 - 0x7E7** (OBD flow control, according to ISO 15765-4)
- **0x7F1** (end-of-life activation of on-board pyrotechnic devices request, according to ISO 26021 and crash data)

4.9.1.21 Gateway X-LIN DIAG

•Gateway latency requirements

4.9.1.21.1 Gateway latency requirements

661480 v1 **Time routing incoming Diagnostic request**

Version history:
(1):

Purpose: The LIN slaves must be given a chance to start the operation cycle within the start of operation cycle timing.
Source:
Verification Method: Test
Legacy ID: 77403v1

A diagnostic request shall be routed to the LIN network maximum 50ms after the LIN master have received the request.

•Gateway allocation of buffers for DIAG

4.9.1.21.2 Gateway allocation of buffers for DIAG

661481 v1 **X-LIN DIAG Minimum RAM for request buffers**

Version history:
(1):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication.
Source:
Verification Method: Test
Legacy ID: 442643v0

For each LIN network behind the LIN master: In application mode, the LIN master shall be able to reserve at least a 264 bytes RAM buffer pool only for requests.
The buffers for each LIN network shall be structured as:

Buffers size	Number of Buffers	Memory need	Total Memory need
200	1	200	
32	2	64	
# of buffers	3		**264**

661482 v1 **X-LIN DIAG Minimum RAM for response buffers**

Version history:
(1):

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication.
Source:
Verification Method: Test
Legacy ID: 442646v0

For each LIN network behind the LIN master: In application mode, the LIN master shall be able to reserve at least a 900 bytes RAM buffer pool only for responses.
The buffers for each LIN network shall be structured as:

Buffers size	Number of Buffers	Memory need	Total Memory need
500	1	500	
200	2	400	
# of buffers	3		**900**

4.9.1.22 Gateway X-LIN SWDL

•Diagnostic schedule execution

4.9.1.22.1 Diagnostic schedule execution

661484 v3 LIN scheduling interval

Version history:
(3): 编程会话下的诊断调度表调度周期由15ms修改为10ms。

Purpose: Maximum utilization of the LIN network
Source:
Verification Method: Test
Legacy ID: 95054v2

The LIN master shall run the LIN schedule table at least once every 10ms and the allowed jitter is defined in either the corresponding platform specification or the LIN Data Link Specification.

661483 v1 LIN scheduling method

Version history:
(1):

Purpose: Minimize SWDL time
Source:
Verification Method: Test
Legacy ID: 82125v1

The LIN master shall run LIN scheduling in Diagnostics Only Mode. The diagnostic implementer is allowed to define the LIN master to switch LIN schedule if there are no pending diagnostic requests or responses as long as the LIN master is able to switch back to Diagnostic Only mode when the LIN master receives a diagnostic request addressed to the LIN network.

•Gateway allocation of buffers for SWDL

4.9.1.22.2 Gateway allocation of buffers for SWDL

661485 v3 X-LIN SWDL Minimum RAM for request buffers

Version history:
(3):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:
Verification Method: Test
Legacy ID: 442649v0

Available memory for allocation of buffers for SWDL messages are a compromise between hardware cost and performance, e.g. how much traffic in parallel the gateway should be able to handle.
For SWDL messages the GW will execute a bootloader software, PBL or SBL, **also will execute in application mode** (i.e. expected that large RAM resources are available).

For each LIN network behind the LIN master: The LIN master shall be able to reserve at least a 8200 bytes RAM buffer pool only for requests in bootloader mode, and reserve at least a 4100 bytes RAM buffer pool only for requests in application mode.

The buffers for each LIN network shall be structured as:

Mode	Buffers size	Number of Buffers	Memory need	Total Memory need
Bootloader mode	2050	4	8200	8200
Application mode	2050	2	4100	4100

661487 v3 **X-LIN SWDL Minimum RAM for response buffers**

Version history:
(3):

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:
Verification Method: Test
Legacy ID: 442653v0

For each LIN network behind the LIN master: In both bootloader mode and application, the LIN master shall be able to reserve at least a 300 and 24 bytes RAM buffer pool only for responses.
The buffers for each LIN network shall be structured as:

Mode	Buffers size	Number of Buffers	Memory need	Total Memory need
Bootloader mode	100	3	300	300
Application mode	8	3	24	24

661486 v3 **X-LIN SWDL Support for queued request**

Version history:
(3): 修改为对LIN网段的队列请求无要求，仅对不同ECU寻址的并行要求。

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:
Verification Method: Test
Legacy ID: 442651v0

For each LIN network behind the LIN master: In bootloader mode and the LIN master shall be able to receive two(2) different address request on two(2) communication channels without sending flow control Wait (FC.Wait).

4.9.1.23 Domain Network Gateway - IP-IP/IP Routing

•Requirements

4.9.1.23.1 Requirements

The IP based diagnostic communication shall follow the requirements specified in *[GW_13] IP Routing Specification* and *[GW_14] Internetworks General Specification*.
•Gateway allocation of buffers for DIAG

4.9.1.23.2 Gateway allocation of buffers for DIAG

1072314 v1 **Minimum RAM for buffers**

Version history:

Purpose: Set a minimum requirement for available memory for requests and responses. To ensure a minimum performance requirement for parallel communication.
Source:
Verification Method: Test
Legacy ID:

For each network behind the GW: In application mode, the InVehicle DoIP GW shall be able to reserve at least a Ethernet: 8kB RAM buffer pool only for **requests** where several smaller buffers can be allocated from.

For each network behind the GW: In application mode, the InVehicle GW shall be able to reserve at least a Ethernet: 16kB RAM buffer pool only for **responses** where several smaller buffers can be allocated from.

•Gateway allocation of buffers for SWDL

4.9.1.23.3 Gateway allocation of buffers for SWDL

1072320 v1 **Minimum RAM for buffers**

Version history:

Purpose: Set a minimum requirement for available memory for requests and responses. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:
Verification Method: Test
Legacy ID:

For each network behind the GW: the InVehicle GW shall be able to reserve at least a Ethernet: 64KB RAM buffer pool only for **requests** where several smaller buffers can be allocated from.

For each network behind the GW: In both bootloader mode and application mode, the InVehicle GW shall be able to reserve at least a Ethernet: 8KB RAM buffer pool only for **responses** where several smaller buffers can be allocated from.

4.9.1.24 InVehicle Network Gateway - IP-IP/DoIP GW

•Translation of diagnostic messages

4.9.1.24.1 Translation of diagnostic messages

4.9.1.24.1.1 Diagnostic request from Internal IP network to Sub IP network

When the Sub DoIP ECU diagnostic request is received, the InVehicle GW should replace the EdgeNode internal IP source address with the InVehicle GW Sub IP source address for the Sub IP network. The target address in the DoIP header shall be mapped to a sub IP TA, and the InVehicle GW Sub IP address should be used as Sub IP SA.

See example in Figure " *Diagnostic request from Internal IP to Sub IP"*.

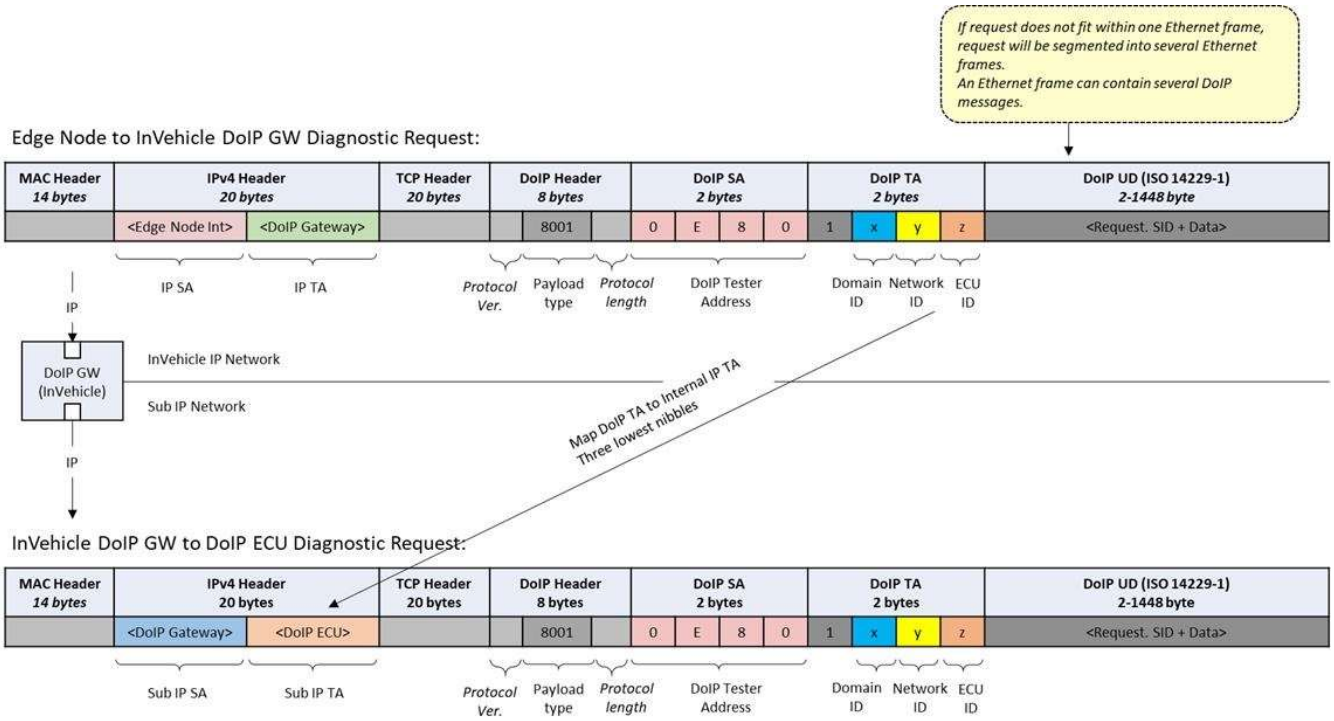


Figure: Diagnostic request from Internal IP to Sub IP

661429 v2 **Internal IP format to Sub IP format in request**

Version history:
(2): Update InVehicle DoIP GW DoIP to DoIP routing requirements between internal IP network and sub IP network.

Purpose: Define the change from an internal IP request to a sub IP request, using the DoIP header information for set up of the Sub IP TA in diagnostic request and using the InVehicle GW as Sub IP SA.

Source:

Verification Method: Test

Legacy ID:

If a request is received from an internal IP network that should be transported to a sub IP network, the InVehicle GW shall translate the DoIP target address (DoIP TA) to the Sub IP target address (IP TA) as specified by *[GW_14] Internetworks General Specification* and use the InVehicle GW Sub IP address as the Sub IP target address (IP SA), format on frames as defined in DoIP *[GW_26] ISO 13400-2*.

Comment:
1. See example in Figure: " *Diagnostic request from InVehicle IP to Sub IP"*.
2. The allocation of subnetwork IP addresses are defined in ECU SWRS requirements "ECU Addresses in GEEA3.0&EEA4.0 - Superset".

4.9.1.24.1.2 Diagnostic response from internal IP network to InVehicle IP network

When the Sub DoIP ECU diagnostic response is received, the InVehicle GW should replace the Sub IP target address with the EdgeNode internal IP target address for the internal IP network. The target address in the DoIP header shall be mapped to EdgeNode IP TA, and the InVehicle GW Internal IP address should be used as Internal IP SA.

See example in Figure " *Diagnostic response from Sub IP to Internal IP"*.

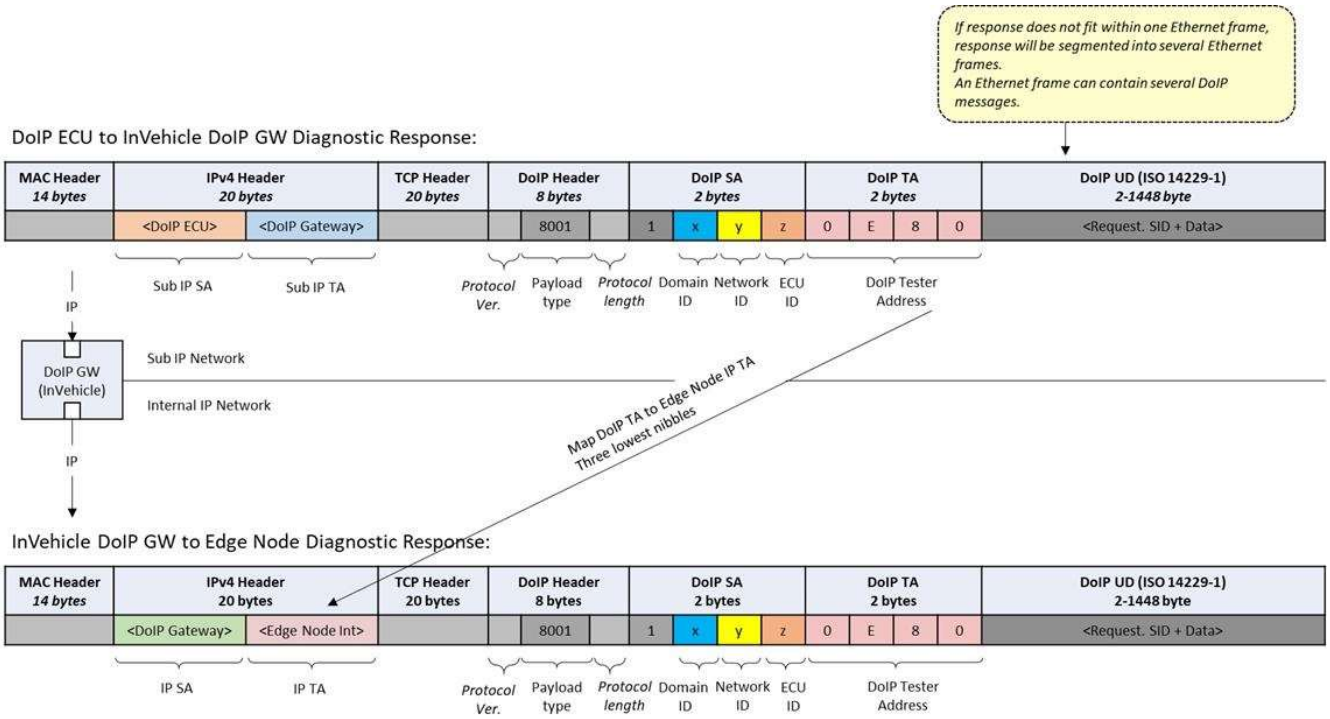


Figure: Diagnostic response from Sub IP to Internal IP

1072236 v1 **Sub IP format to Internal IP format in response**

Version history:

Purpose: Define the change from an InVehicle IP request to a Sub IP request, using the DoIP header information for set up of a Sub IP TA in diagnostic request and using the InVehicle GW as Sub IP SA.

Source:

Verification Method: Test

Legacy ID:

If a response is received from a sub IP network that should be transported to an internal IP network, the InVehicle GW shall translate the DoIP target address (DoIP TA) to the EdgeNode internal IP target address (IP TA) as specified by *[GW_14] Internetworks General Specification* and use the InVehicle GW Internal IP address as the Internal IP target address (IP TA), format on frames as defined in DoIP *[GW_26] ISO 13400-2*.

Comment:
1. See example in Figure: " *Diagnostic response from Sub IP to Internal IP"*.
2. The allocation of subnetwork IP addresses are defined in ECU SWRS requirements "ECU Addresses in GEEA3.0&EEA4.0 - Superset".

4.9.1.25 Domain Network Gateway DIAG

•Gateway allocation of buffers for DIAG

4.9.1.25.1 Gateway allocation of buffers for DIAG

This section will describe and give grounds for the following requirements:

- ReqID 1687 Minimum RAM for request buffers
- ReqID 1685 Minimum number of buffers for requests
- ReqID 1683 Large buffers size for request

- ReqID 1688 Minimum RAM for response buffers
- ReqID 1686 Minimum number of buffers for responses
- ReqID 1684 Large buffers size for response

Available memory for allocation of buffers for DIAG messages are a compromise between hardware cost and performance, e.g. how much traffic in parallel the gateway should be able to handle.
For DIAG messages the Domain Network GW will execute in application mode, and have to share resources as RAM for buffers with other applications.

For each network behind the GW: The Domain Network GW should be able to allocate at least 4 KB RAM reserved for buffers for requests, and at least 8 KB RAM reserved for buffers for responses.

The Domain Network GW must be able to allocate 4 KB buffer for diagnostic messages.

There should be a minimum number of available buffers so that the gateway can handle parallel communication, as long as the total size for buffers is not exceeded. If there is a small request or response the smallest possible buffer should be used, e.g. do not allocate a 4 KB buffer for a 2 bytes long request. This specification does not specify exactly which buffer allocation algorithm that should be used, this is up to the implementer, but there should be a minimum of available buffers for small requests.

To insure that parallel DIAG can be performed for small messages (dynamic sized buffer allocation):

- For each network behind the GW: If the total size of requests are less or equal to 4 KB RAM, the minimum available request buffers should be 32.
- For each network behind the GW: If the total size of responses are less or equal to 8 KB RAM, the minimum available response buffers should be 32.

976008 v5 **Large buffers for request**

Version history:
(5):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication.
Source:
Verification Method: Test
Legacy ID: 77624v4
Zeekr Requirement ID: 1683

For each network behind the GW: In application mode, the Domain Network GW shall be able to allocate up to 4 kB large buffers for request.

976009 v6 **Large buffers for response**

Version history:
(6):

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication.
Source:
Verification Method: Test
Legacy ID: 77627v5
Zeekr Requirement ID: 1684

For each network behind the GW: In application mode, the Domain GW shall be able to allocate up to 4 kB large buffers for response.

976012 v6 **Minimum RAM for request buffers**

Version history:
(6):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication.
Source:
Verification Method: Test
Legacy ID: 77621v4
Zeekr Requirement ID: 1687

For each network behind the GW: In application mode, the Domain Network GW shall be able to reserve at least a 4 KB RAM buffer pool only for requests where several smaller buffers can be allocated from.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for requests.
If using static sized message buffers and/or not possible to reserve buffer pool only for requests: ZEEKR and the ECU supplier shall come to an agreement on used buffer sizes to achieve similar performance, more memory for buffers are needed.

Note:
There is a separate requirement for minimum number of buffers that can be allocated form this buffer pool.

Example (dynamic sized message buffers and 4 KB buffer pool):
A) One network behind the gateway: 4 KB RAM reserved for requests (with possibility to allocate different buffer sizes).
B) Two networks behind the gateway: Implies requirement of 2 x 4 KB RAM reserved for requests.

Example (static sized message buffers):
A) One message with size < 4 KB received, but uses large 4 KB buffer, 1 x 4 KB = 4 KB, more memory than the minimum requirement are needed to be able to fulfil the requirement for

number of buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers"*. For static sized message buffers it can be larger gaps in the usable memory, hence this minimum RAM requirement may need to be increased dependent on how the static sized message buffers are chosen.

976013 v6 **Minimum RAM for response buffers**

Version history:
(6):

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication.
Source:
Verification Method: Test
Legacy ID: 77625v4
Zeekr Requirement ID: 1688

For each network behind the GW: In application mode, the Domain Network GW shall be able to reserve at least a 8 KB RAM buffer pool only for responses where several smaller buffers can be allocated from.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for responses.
If using static sized message buffers and/or not possible to reserve buffer pool only for responses: ZEEKR and the ECU supplier shall come to an agreement on used buffer sizes to achieve similar performance, more memory for buffers are needed.

Note:
There is a separate requirement for minimum number of buffers that can be allocated form this buffer pool.

Example (dynamic sized message buffers and 8 KB buffer pool):
A) One network behind the gateway: 8 KB RAM reserved for responses (with possibility to allocate different buffer sizes).
B) Two networks behind the gateway: Implies requirement of 2 x 8 KB RAM reserved for responses.

Example (static sized message buffers):
A) Two messages with size < 4 KB received, but uses large 4 KB buffers, 2 x 4 KB = 8 KB, more memory than the minimum requirement are needed to be able to fulfil the requirement for number of buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message*

buffers". For static sized message buffers it can be larger gaps in the usable memory, hence this minimum

976010 v5 **Minimum number of buffers for requests**

Version history:
(5):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication.
Source:
Verification Method: Test
Legacy ID: 77622v3
Zeekr Requirement ID: 1685

For each network behind the GW: In application mode and if the total size of all current requests are less or equal to 4 KB RAM, the Domain Network GW shall be able to allocate at least 32 buffers for requests.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for requests.
If using static sized message buffers and/or not possible to reserve buffer pool only for requests: ZEEKR and the ECU supplier shall come to an agreement on the number of buffers to achieve similar performance.

Example (dynamic sized message buffers and 4 KB buffer pool):
A) One network behind the gateway: Several 100 bytes messages received, 32 x 100 bytes = total size 3200 bytes < 4 KB, at least 32 buffers shall be possible to allocate.
B) One network behind the gateway: One 2 KB message received -> 2 KB left in buffer pool. This reduce the number of buffers that can be allocated, e.g. only 20 x 100 bytes due to remaining available memory.
C) Two networks behind the gateway: Implies requirement of 2 x 32 buffers reserved for requests.

Example (static sized message buffers):
A) One messages with size < 4 KB received, but uses large 4 KB buffer, 1 x 4 KB = 4 KB, more memory than the minimum requirement are needed to be able to allocate more smaller buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers".*

976011 v5 **Minimum number of buffers for responses**

Version history:
(5):

Purpose: Set a minimum requirement for available memory for responses. To ensure a

minimum performance requirement for parallel communication.

Source:
Verification Method: Test
Legacy ID: 77626v3
Zeekr Requirement ID: 1686

For each network behind the GW: In application mode and if the total size of all current responses are less or equal to 8 KB RAM, the Domain Network GW shall be able to allocate at least 32 buffers for responses.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for responses.
If using static sized message buffers and/or not possible to reserve buffer pool only for responses: ZEEKR and the ECU supplier shall come to an agreement on the number of buffers to achieve similar performance.

Example (dynamic sized message buffers and 8KB buffer pool):
A) One network behind the gateway: Several 200 bytes messages received, 32 x 200 bytes = total size 6400 bytes < 8 KB, at least 32 buffers shall be possible to allocate.
B) One network behind the gateway: One 4 KB message received -> 4 KB left in buffer pool. This reduce the number of buffers that can be allocated, e.g. only 20 x 200 bytes due to remaining available memory.
C) Two networks behind the gateway: Implies requirement of 2 x 32 buffers reserved for responses.

Example (static sized message buffers):
A) Two messages with size < 4 KB received, but uses large 4 KB buffers, 2 x 4 KB = 8 KB, more memory than the minimum requirement are needed to be able to allocate more smaller buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: "Example buffer pool with dynamic sized message buffers" and Figure: "Example buffer pool with static sized message buffers".

4.9.1.26 Domain Network Gateway SWDL

•Gateway allocation of buffers for SWDL

4.9.1.26.1 Gateway allocation of buffers for SWDL

This section will describe and give grounds for the following requirements:

- REQPROD 829267 Minimum RAM for request buffers
- REQPROD 829265 Minimum number of buffers for requests
- REQPROD 1695 Support for queued request
- REQPROD 829264 Maximum buffer size for request
- REQPROD 1694 Minimum RAM for response buffers
- REQPROD 1692 Minimum number of buffers for responses
- REQPROD 1690 Maximum buffer size for response

Available memory for allocation of buffers for SWDL messages are a compromise between

hardware cost and performance, e.g. how much traffic in parallel the gateway should be able to handle.
For SWDL messages the Domain Network GW will execute a bootloader software, PBL or SBL, and no other application should be executed (i.e. expected that large RAM resources are available).

For each network behind the GW: The Domain Network GW should be able to allocate at least CAN:24 KB or FlexRay:32KB RAM reserved for buffers for requests, and at least 4 KB RAM reserved for buffers for responses.

The Domain Network GW must be able to allocate 4 KB buffer for diagnostic messages.

There should be a minimum number of available buffers so that the gateway can handle parallel communication, as long as the total size for buffers is not exceeded. If there is a small request or response the smallest possible buffer should be used, e.g. do not allocate an 4 KB buffer for a 2 bytes long request. This specification does not specify exactly which buffer allocation algorithm that should be used, this is up to the implementer, but there should be a minimum of available buffers for small requests.

To insure that parallel SWDL can be performed for small messages (dynamic sized buffer allocation):

- For each network behind the GW: If the total size of requests are less or equal to CAN:24 KB or FlexRay:32KB RAM, the minimum available request buffers should be 32.
- For each network behind the GW: If the total size of responses are less or equal to 4 KB RAM, the minimum available response buffers should be 32.

For supporting queued requests, if the request size is less or equal to 24 kB, the Domain Network GW should able to receive two(2) queued request on three(3) communication channels without sending flow control Wait (FC.Wait).

See example in Figure " *Domain Network Gateway Buffers for SWDL"*.

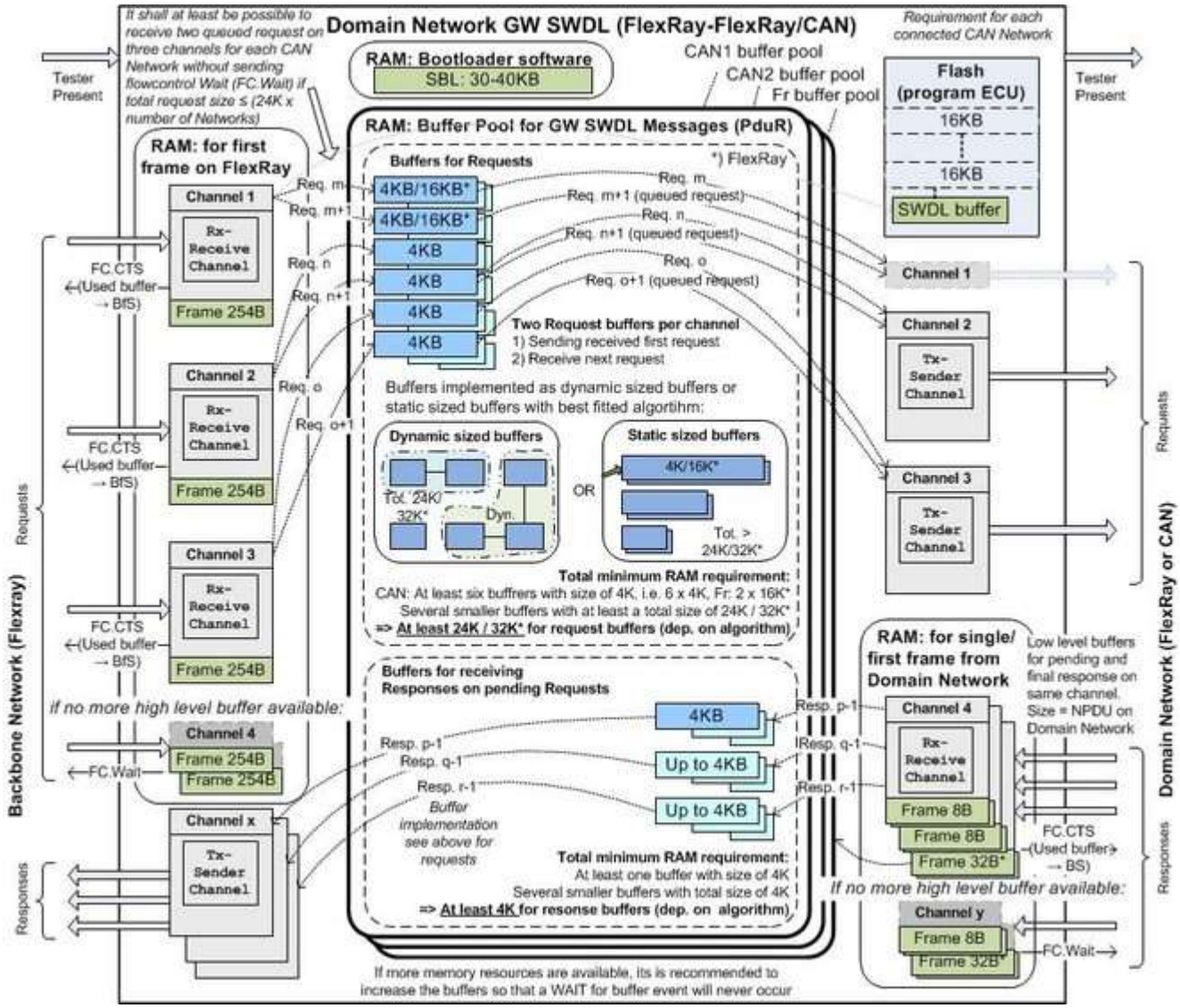


Figure: Domain Network Gateway Buffers for SWDL

976055 v1 **Large buffers for request**

Version history:
(1):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.

Source:

Verification Method: Test

Legacy ID:

Zeekr Requirement ID: 829264

For each network behind the GW: In bootloader mode, the Domain Network GW shall be able to allocate up to 4 KB large buffers for request for CAN networks.

976056 v5 **Large buffers for response**

Version history:
(5):

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:
Verification Method: Test
Legacy ID: 77448v4
Zeekr Requirement ID: 1690

For each network behind the GW: In bootloader mode, the Domain Network GW shall be able to allocate up to 4 kB large buffers for response.

976058 v1 **Minimum RAM for request buffers**

Version history:
(1):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 829267

For each network behind the GW: In bootloader mode, the Domain Network GW shall be able to reserve at least a 24 KB RAM buffer pool for each CAN network,reserved only for requests where several smaller buffers can be allocated from.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for requests.
If using static sized message buffers and/or not possible to reserve buffer pool only for requests: the project and the ECU supplier shall come to an agreement on used buffer sizes to achieve similar performance, more memory for buffers are needed.

Note:
There is a separate requirement for minimum number of buffers that can be allocated form this buffer pool.

Example for CAN network connected behind gateway (dynamic sized message buffers and 24 KB buffer pool):
A) One network behind the gateway: 24 KB RAM reserved for requests (with possibility to allocate different buffer sizes).
B) Two networks behind the gateway: Implies requirement of 2 x 24 KB RAM reserved for requests.

Example for CAN network connected behind gateway (static sized message buffers):
A) Twelve messages with size < 4 KB received, but uses large 4 KB buffers, 12 x 4 KB = 24 KB, more memory than the minimum requirement are needed to be able to fulfil the requirement for number of buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers".* For static sized message buffers it can be larger gaps in the usable memory, hence this minimum RAM requirement may need to be increased dependent on how the static sized message buffers are chosen.

976060 v5 **Minimum RAM for response buffers**

Version history:
(5):

Purpose:	Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.
Source:	
Verification Method:	Test
Legacy ID:	77446v4
Zeekr Requirement ID:	1694

For each network behind the GW: In bootloader mode, the Domain Network GW shall be able to reserve at least a 4 KB RAM buffer pool only for responses where several smaller buffers can be allocated from.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for responses.
If using static sized message buffers and/or not possible to reserve buffer pool only for response: the project and the ECU supplier shall come to an agreement on used buffer sizes to achieve similar performance, more memory for buffers are needed.

Note:
There is a separate requirement for minimum number of buffers that can be allocated form this buffer pool.

Example (dynamic sized message buffers and 4 KB buffer pool):
A) One network behind the gateway: 4 KB RAM reserved for responses (with possibility to allocate different buffer sizes).
B) Two networks behind the gateway: Implies requirement of 2 x 4 KB RAM reserved for responses.

Example (static sized message buffers):
A) One message with size < 4 KB received, but use one large 4 KB buffer, 1 x 4 KB = 4 KB, more memory than the minimum requirement are needed to be able to fulfil the requirement for number of buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers"*. For static sized message buffers it can be larger gaps in the usable memory, hence this minimum RAM requirement may need to be increased dependent on how the static sized message buffers are chosen.

976057 v1 **Minimum number of buffers for requests**

Version history:
(1):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.

Source:

Verification Method: Test

Legacy ID:

Zeekr Requirement ID: 829265

For each network behind the GW: In bootloader mode and if the total size of all current requests are less or equal to 24 KB RAM for CAN, the Domain Network GW shall be able to allocate at least 32 buffers for requests.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for requests.
If using static sized message buffers and/or not possible to reserve buffer pool only for requests: the project and the ECU supplier shall come to an agreement on the number of buffers to achieve similar performance.

Example for CAN network connected behind gateway (dynamic sized message buffers and 24 KB buffer pool):
A) One network behind the gateway: Several 600 bytes messages received, 32 x 600 bytes = total size 19200 bytes < 24 KB, at least 32 buffers shall be possible to allocate.
B) One network behind the gateway: Three 4 KB message received -> 12 KB left in buffer pool. This reduce the number of buffers that can be allocated, e.g. only 20 x 600 bytes due to remaining available memory.
C) Two networks behind the gateway: Implies requirement of 2 x 32 buffers reserved for requests.

Example for CAN network connected behind gateway (static sized message buffers):
A) Six messages with size < 4 KB received, but uses large 4 KB buffers, 6 x 4 KB = 24 KB, more memory than the minimum requirement are needed to be able to allocate more smaller buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers"*.

976059 v4 **Minimum number of buffers for responses**

Version history:
(4):

Purpose: Set a minimum requirement for available memory for responses. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.

Source:

Verification Method: Test

Legacy ID: 77447v3

Zeekr Requirement ID: 1692

For each network behind the GW: In bootloader mode and if the total size of all current responses are less or equal to 4 KB RAM, the Domain Network GW shall be able to allocate at least 32 buffers for responses.
This generic requirement assumes the use of an efficient algorithm for dynamic sized message buffers reserved for responses.
If using static sized message buffers and/or not possible to reserve buffer pool only for responses: the project and the ECU supplier shall come to an agreement on the number of buffers to achieve similar performance.

Example (dynamic sized message buffers and 4 KB buffer pool):
A) One network behind the gateway: Several 100 bytes messages received, 32 x 100 bytes = total size 3200 bytes < 4 KB, at least 32 buffers shall be possible to allocate.
B) One network behind the gateway: One 2 KB message received -> 2 KB left in buffer pool. This reduce the number of buffers that can be allocated, e.g. only 20 x 100 bytes due to remaining available memory.
C) Two networks behind the gateway: Implies requirement of 2 x 32 buffers reserved for responses.

Example (static sized message buffers):
A) One message with size < 4 KB received, but use one large 4 KB buffer, 1 x 4 KB = 4 KB, more memory than the minimum requirement are needed to be able to allocate more smaller buffers.

Comment:
Example of dynamic sized buffers vs. static sized buffers, see Figure: *"Example buffer pool with dynamic sized message buffers"* and Figure: *"Example buffer pool with static sized message buffers"*.

976061 v4 **Support for queued request**

Version history:
(4):

Purpose: Set a minimum requirement for available memory for requests. To ensure a minimum performance requirement for parallel communication and optimize SWDL performance.

Source:
Verification Method: Test
Legacy ID: 77444v3
Zeekr Requirement ID: 1695

For each network behind the GW: In bootloader mode and if the total request size is less or equal to 24kB, the Domain Network GW shall able to receive two(2) queued request on three(3) communication channels without sending flow control Wait (FC.Wait)

4.9.1.27 Gateway Generic supporting periodic responses

•Logical address format

4.9.1.27.1 Logical address format

4.9.1.27.1.1 Logical address format for periodic response

Response message type #2: Including the periodicDataIdentifier and the data of the periodicDataIdentifier, see *[ISO_1 : ISO 14229-1]*, should be used for periodic response to SID 2A-ReadDataByPeriodicIdentifer on all networks. This is required in the specifications for respective network (CAN, IP).

The address format for periodic responses is different from the address format for diagnostic request and ("single") response and is also defined in *[INTL_6 : UDS Services]*.

See Figure "*ECU address format for DIAG periodic response (SID 2A)"* for an overview of the logical address format used for periodic response and how this is used in this document.

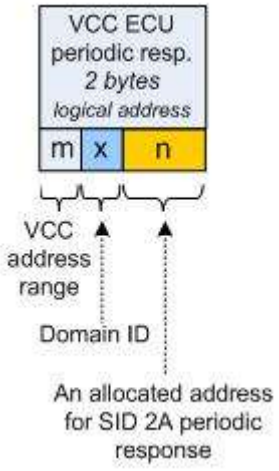


Figure: ECU address format for DIAG periodic response (SID 2A)

4.9.1.27.1.2 Example of adding time stamp in periodic response

ISO specifications,14229-x, does not specify how a periodic response could be time stamped, when a periodic response is sent thru gateways.

The in-vehicle GWs will not do any time stamp on periodic responses received from in-vehicle networks, e.g. a Domain network GW will not time stamp a periodic response received on CAN

before it is sent out on Flexray and Vehicle GW will not time stamp a periodic response received on Flexray before it is sent to tester on an IP network.

If there is a need for time stamping of periodic responses, e.g. during development for measuring data in the ECU, the time stamp must be added in the data part of the periodic response in the source ECU. The in-vehicle GWs handling is independent of if a user defined time stamp is added in the data part or not, the in-vehicle GWs does not look at the data part.

To add a time stamp in the data in periodic responses is an optional feature, no requirement in this specification. The section only gives some hints/suggestions how this could be solved, when there is no support for time stamping in the in-vehicle gateways.

Example, when creating a 'Dynamically defined periodic DataIdentifier' with service 2C – DynamicallyDefineDataIdentifier a request to include a time stamp could be added in the request, if the ECU has an internal clock that can be read and used for time stamping. An internal clock that could e.g. be read directly from a memory position, see example in Figure " *Example 1 of user defined time stamp in DIAG periodic response (SID 2A)"*.

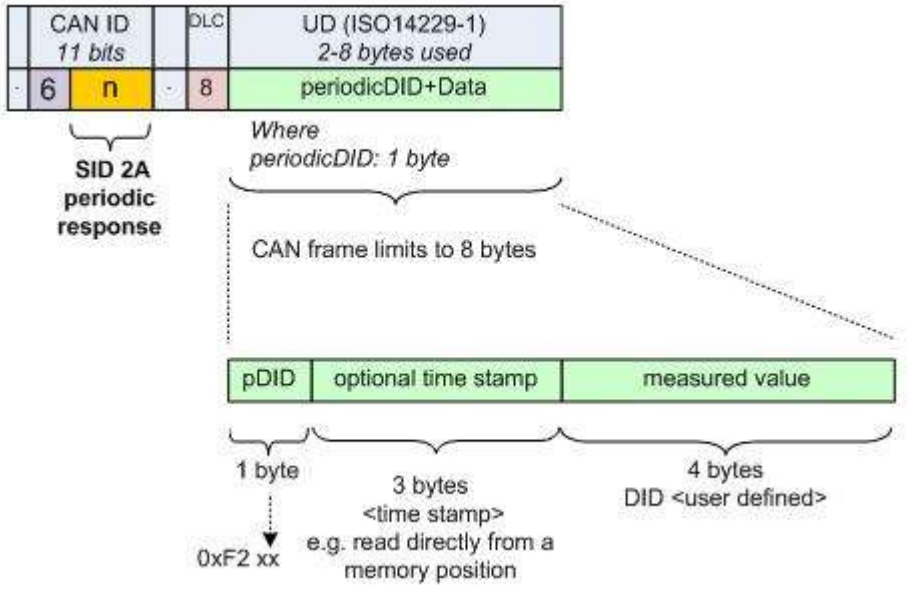


Figure: Example 1 of user defined time stamp in DIAG periodic response (SID 2A)

If a memory range should be monitored, e.g. 130 bytes, sent out as UUDT frames with periodic ID 0xF200 – F213, a time stamp could be added for this block of data and only sent in one UUDT frame, see example in Figure " *Example 2 of user defined time stamp in DIAG periodic response (SID 2A)"*.

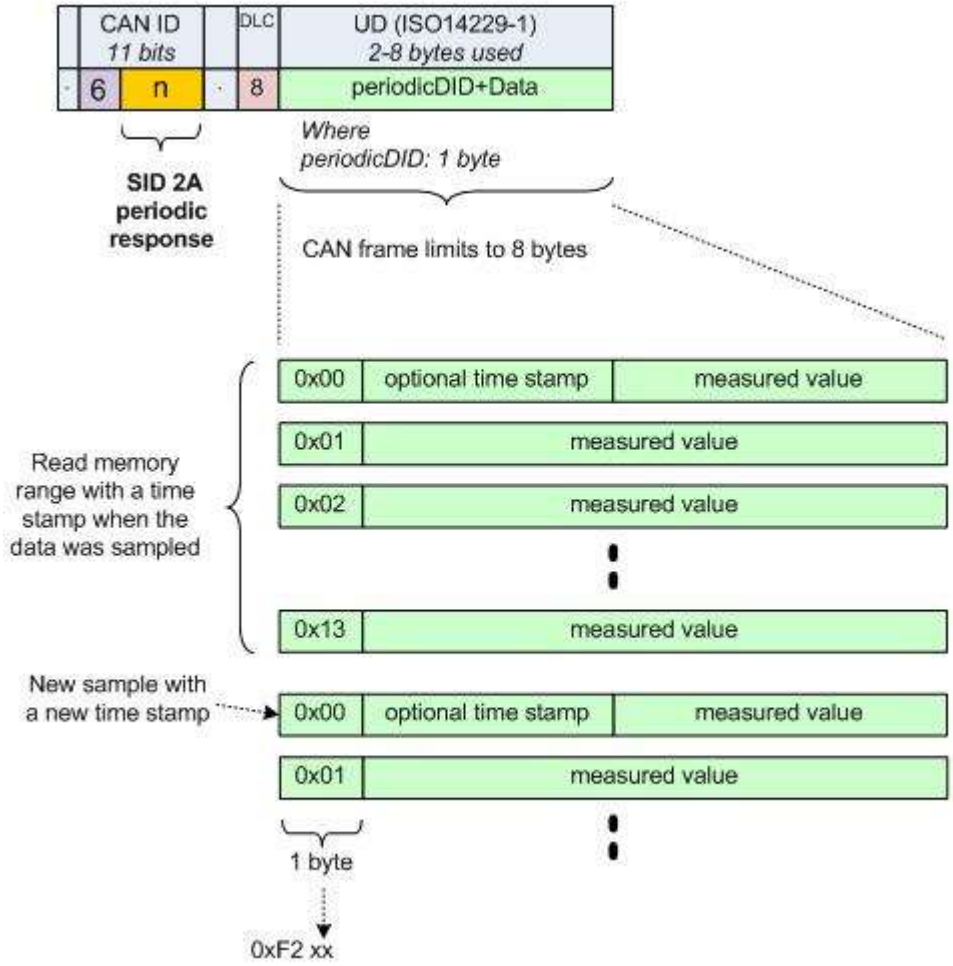


Figure: Example 2 of user defined time stamp in DIAG periodic response (SID 2A)

4.9.1.28 Gateway connected to External IP supporting periodic responses

•IP diagnostic format on External IP

4.9.1.28.1 IP diagnostic format on External IP

4.9.1.28.1.1 Periodic response (SID 2A) on IP based network

This section will describe and give grounds for the following requirements:

- REQPROD 713538 Periodic response format to external IP

Response message type #2: Including the periodicDataIdentifier and the data of the periodicDataIdentifier, see *[ISO_1 : ISO 14229-1]*, should be used for periodic response to SID 2A-ReadDataByPeriodicIdentifer.

Vehicle to external IP network periodic response message structure:

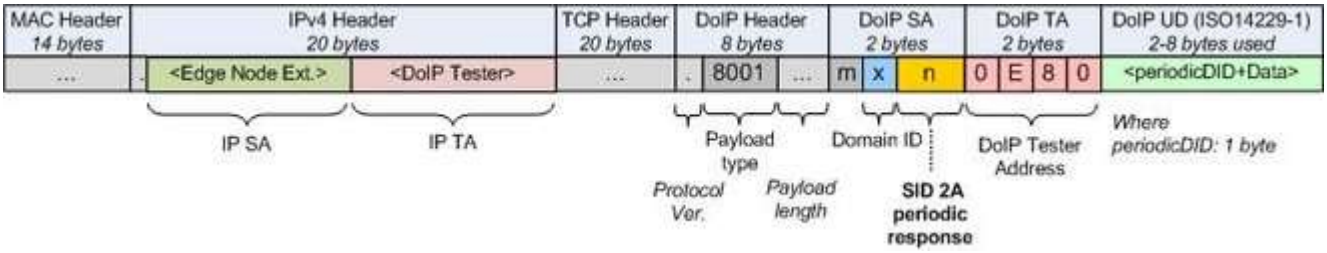


Figure: External IP periodic response (SID 2A) frame structure

976051 v3 **Periodic response format to external IP**

Version history:
For3.0(3):

Purpose: Define the periodic response format out from the gateway to an external IP network, including the SID 2A logical address in the frame.

Source:

Verification Method: Test

Legacy ID:

Zeekr Requirement ID: 713538

When a diagnostic periodic response is received that shall be transported to an external IP network and tester, the Vehicle GW (Edge Node) shall send out the diagnostic periodic response with a DoIP header with payload type 8001 as defined in DoIP *[ISO_15 : ISO 13400-2]* with SID 2A logical addresses as defined in *[INTL_6 : UDS Services]* and with the external tester address in the IP TA and the Vehicle GW external IP address in the IP SA.

Comment:
See example in Figure: " *External IP periodic response (SID 2A) frame structure"*.

4.9.1.29 Gateway connected to Internal IP supporting periodic responses

•IP diagnostic format on Internal IP

4.9.1.29.1 IP diagnostic format on Internal IP

4.9.1.29.1.1 Periodic response (SID 2A) on IP based network

This section will describe and give grounds for the following requirements:

- ReqID 713541 Supported periodic response format from internal IP

Response message type #2: Including the periodicDataIdentifier and the data of the periodicDataIdentifier, see *[ISO_1 : ISO 14229-1]*, should be used for periodic response to SID 2A-ReadDataByPeriodicIdentifer.

Internal IP network periodic response message structure:

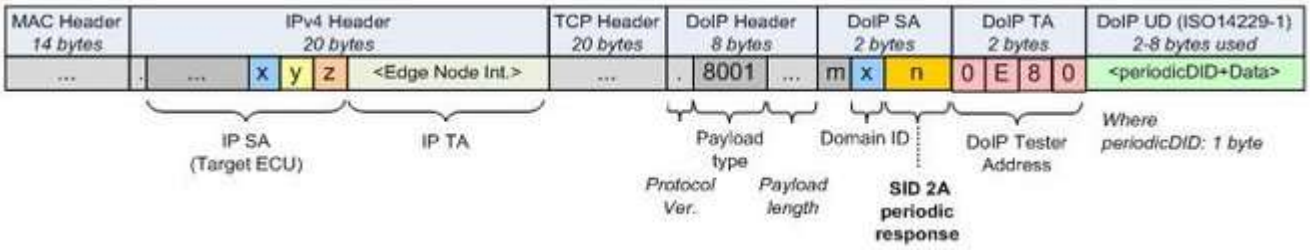


Figure: Internal IP periodic response (SID 2A) frame structure

976052 v3 **Supported periodic response format from internal IP**

Version history:
For3.0(3):

Purpose: Define supported periodic response frame format on internal IP network, including SID 2A logical addresses in the frame, which should transported to an external network and tester or to an internal tester.

Source:

Verification Method: Test

Legacy ID:

Zeekr Requirement ID: 713541

To be able to transport a diagnostic periodic response to an external network and tester or an internal tester, the Vehicle GW (Edge Node) shall receive all DoIP periodic responses with payload type 8001 as defined in DoIP *[ISO_15 : ISO 13400-2]* with SID 2A logical addresses as defined in *[INTL_6 : UDS Services]* and analyse the destination for the periodic response.

Comment:
See example in Figure: " *Internal IP periodic response (SID 2A) frame structure"*.

4.9.1.30 Gateway connected to CAN supporting periodic responses

•CAN diagnostic frame format

4.9.1.30.1 CAN diagnostic frame format

4.9.1.30.1.1 Periodic response (SID 2A) on CAN Network

This section will describe and give grounds for the following requirements:

- ReqID 713548 Supported periodic response format from CAN

Response message type #2: Including the periodicDataIdentifier and the data of the periodicDataIdentifier, see *[ISO_1 : ISO 14229-1]*, should be used for periodic response to SID 2A-ReadDataByPeriodicIdentifer.

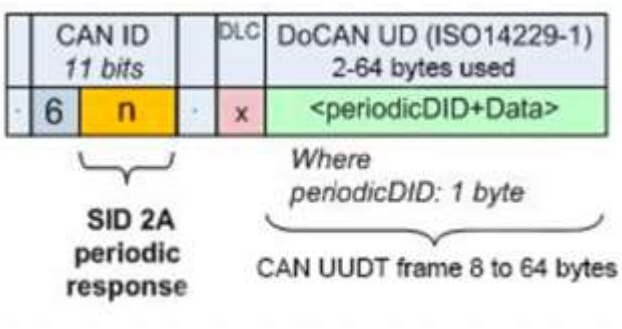


Figure: CAN periodic response (SID 2A) frame structure

976053 v3 **Supported periodic response format from CAN**

Version history:
For3.0(3):

Purpose: Define supported periodic response frame format from a Domain Network (CAN), including SID 2A logical addresses in the frame, which should transported the Backbone Network.

Source:

Verification Method: Test

Legacy ID:

Zeekr Requirement ID: 713548

To be able to transport a diagnostic periodic response from the Domain Network (CAN) to the Backbone Network. the Domain Network GW shall be able to interpret and handle a diagnostic periodic response from the Domain Network (CAN) as defined in DoCAN *[ISO_17 : ISO 15765-2]* normal addressing 11 bit CAN ID with SID 2A logical addresses as defined in *[INTL_6 : UDS Services]*.

Comment:
See example in Figure: " *CAN periodic response (SID 2A) frame structure".*

4.9.1.31 Vehicle Gateway DIAG - IP-IP supporting periodic responses

•Translation of diagnostic messages

4.9.1.31.1 Translation of diagnostic messages

4.9.1.31.1.1 Diagnostic periodic response from internal IP Network to external IP Network

This section will describe and give grounds for the following requirements:

- ReqID 713549 Internal IP to external IP address in periodic response

Periodic responses are handled in a similar way as ("single") response described in ' *Diagnostic response from internal IP network to external IP network'*.

See example in Figure " *Periodic response from internal IP to external IP"*.

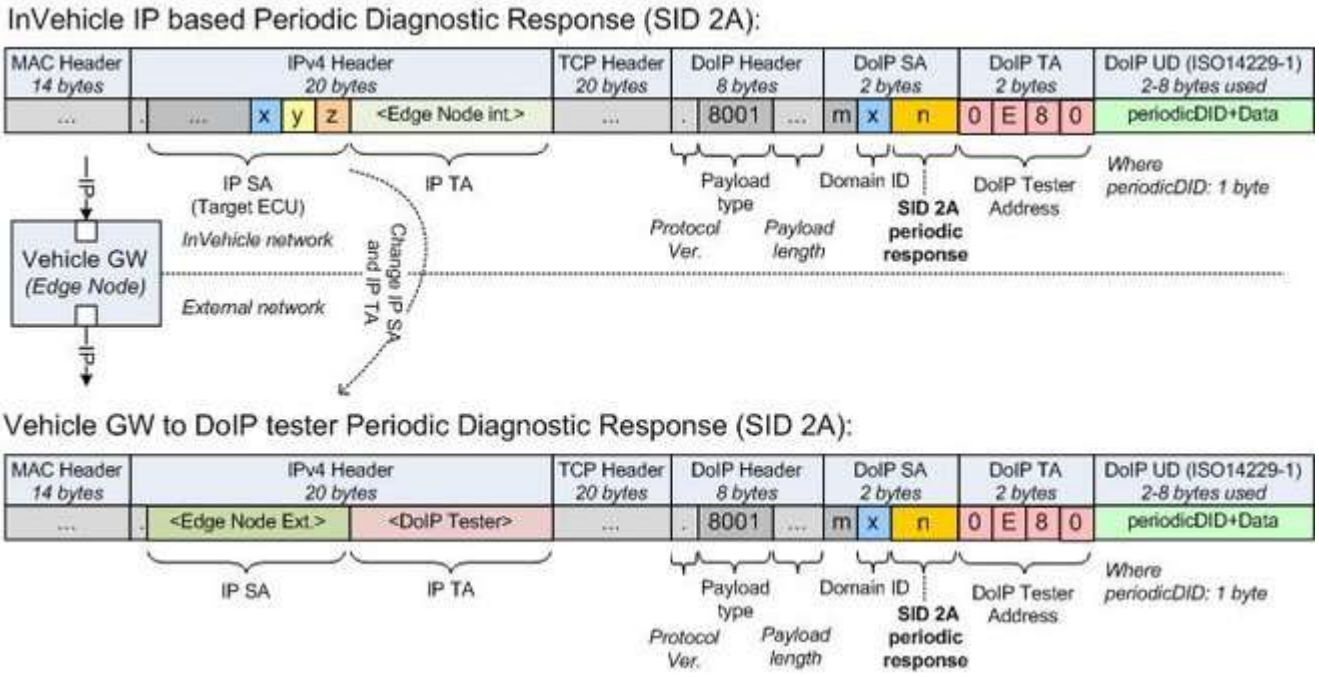


Figure: Periodic response from internal IP to external IP

976054 v2	Internal IP to external IP address in periodic response

Version history:
For3.0(2):

Purpose: Define the change from an internal IP address in periodic response to an external IP address in periodic response.

Source:

Verification Method: Test

Legacy ID:

Zeekr Requirement ID: 713549

If a periodic response is received from an internal IP network that should be transported to the external IP network, the Vehicle GW (Edge Node) shall change the internal IP source address to the Vehicle GW external IP address (IP SA) and change the Vehicle GW internal IP address to the tester tool IP address (IP TA), IP address that was allocated during establishing connection between external tester and vehicle.

4.9.1.32 Sub Network Gateway DIAG - CAN-CAN supporting perdiodic responses

•Gateway latency requirements

4.9.1.32.1 Gateway latency requirements

4.9.1.32.1.1 Periodic diagnostic responses CAN to CAN

Periodic responses CAN to CAN

A periodic response received from Sub Network (CAN), message type #2 – UUDT on CAN, will be sent out as fast as possible on the Domain Network (CAN).

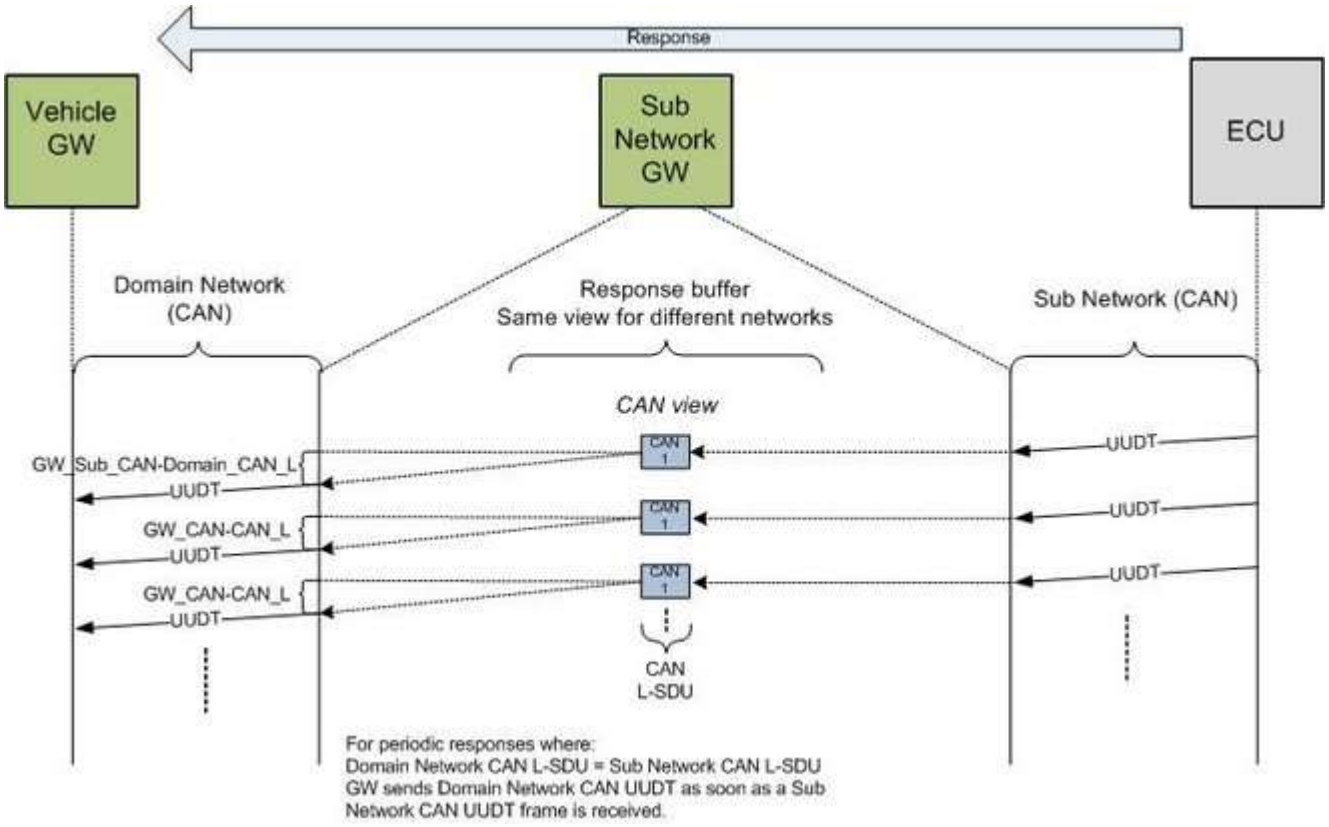


Figure: Sub Network GW: CAN to CAN periodic responses

Definition of latency timing parameter:

GW_Sub_CAN-Domain_CAN_L: Time between receiving an UUDT frame from Sub Network (CAN) until the frame is sent out on Domain Network (CAN).

Timing Parameter	SWDL	DIAG
	Max Value	Max Value
GW_Sub_CAN-Domain_CAN_L	N/A	10 ms

Table: Sub Network GW: CAN to CAN periodic response timing parameter

4.10 Diagnostics

4.10.1 UDS Session

4.10.1.1 UDS Session generic reqs

•Overview

4.10.1.1.1 Overview

What is the purpose of this document?
This document specifies OEM specific requirements for OSI layer 5 – Session Layer Services for the diagnostic

software component. The session layer services shall be compliant to ISO 14229-2, Road vehicles – Unified diagnostic services - Part 2: Session layer services see requisite documents, with the restrictions/additions as defined by the subsequent sub-sections.

This document is intended to provide guidance and requirements to architects, system designers, application developers, and suppliers that are to deliver diagnostics compliant with production vehicle programs. It shall serve as the cross car line compliance reference for implementation of vehicle diagnostics. Adherence to the specifications contained within this document is required to ensure compatibility within the corporate product line from vehicle to vehicle, model year to model year and maintain compatibility with industry driven standards.

This document specifies OEM specific requirements. If there are other kinds or requirements like legislative requirements (for example OBD requirements), these requirements may have higher priority than the requirements in this specification. However if the other requirements have performance requirements lower than the performance requirements specified in this document, the performance requirements in this document shall be followed. This is valid as long as the performance requirement in this document does not contradict timeout requirements in the other document.

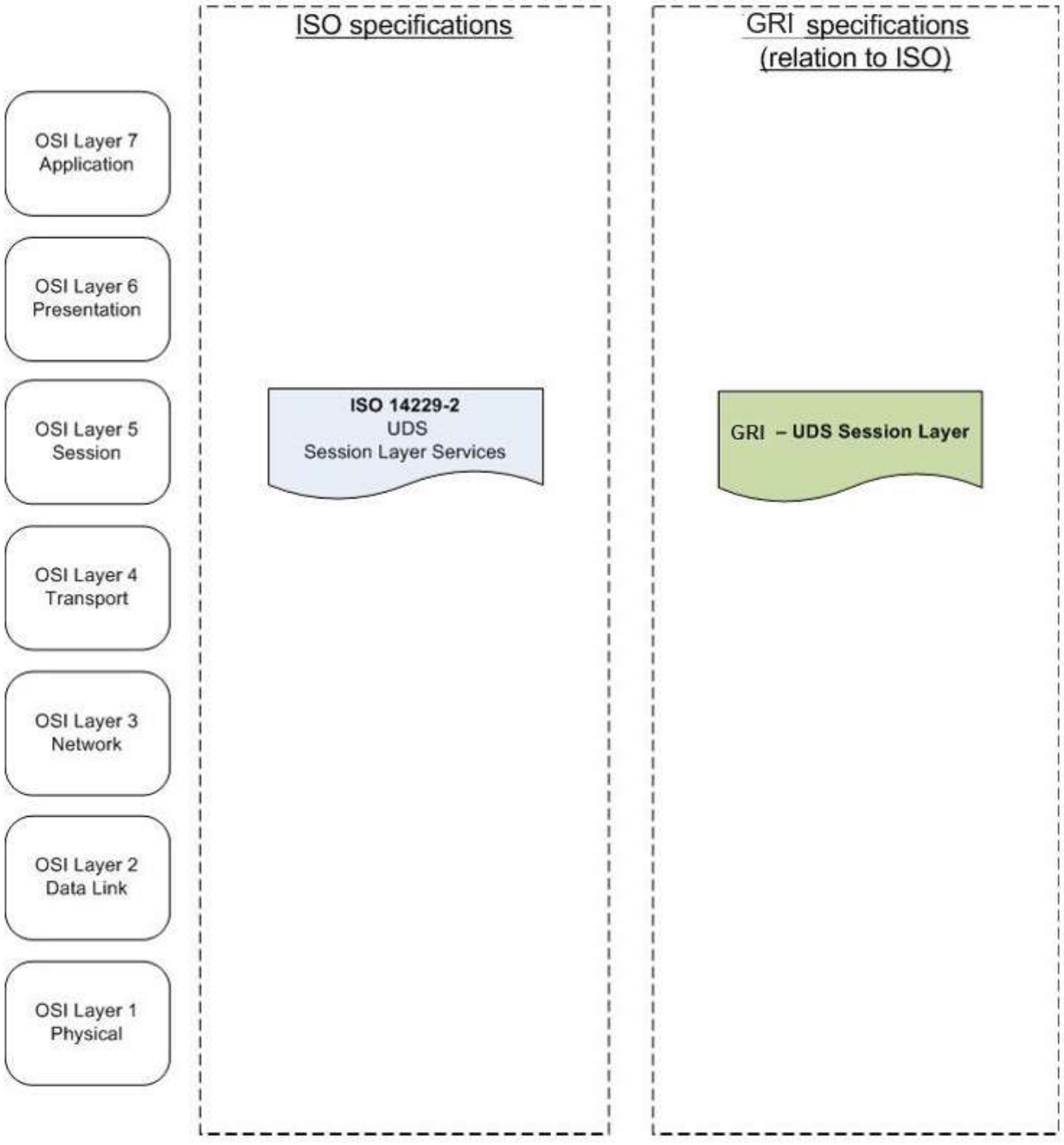


Figure - Relation to ISO document

Who should read this document?
The document is mainly intended for the developer of the actual software.

If the software product is a part of a larger system and interacts with other software products, this document also should be read and approved by the system designers and the designers responsible for each interacting software product.

•Requisite documents

4.10.1.1.2 Requisite documents

Reference	Source	Title
[1]	Internal	UDS Services
[2]	Internal	DoIP Communication
[3]	ISO	ISO 14229-2: Road vehicles – Unified diagnostic services - Part 2: Session layer services. Edition 1: 2013
[4]	ISO	ISO 13400-2: Road vehicles – Diagnostic Communication over Internet Protocol (DoIP)

Note:
Internal references are typically available as separate chapters in ECU specific Base Technology SWRS.
Information obtaining ISO documents may be found on the internet: http://www.iso.org/

•Definitions and Abbrevations

4.10.1.1.3 Definitions and Abbrevations

Definition	Description
A_Data	A_Data is a string of byte data defined for each individual application layer service: The A_Data string shall start with the Service Identifier followed by all service specific parameters from the application layer service data unit.
Client	Function that is part of the tester and that makes use of the diagnostic services.
Edge node	An ECU (internal vehicle network node) which is the network node that is exposed towards the external network. The node implements a diagnostic over IP-protocol (DoIP - ISO13400) and provides diagnostic access to test equipment.
Final response	A final response is a positive response or a negative response other than response code 0x78 "requestCorrectlyReceived-ResponsePending".
Server	Software entity which provides services for clients
Tester	A system that controls functions such as test, inspection, monitoring, or diagnosis of an in-vehicle electronic control unit and may be dedicated to a specific type of operators (e.g. a scan tool dedicated to garage mechanics or a test tool dedicated to assembly plant agents). The tester is also referenced as the client.

Abbr.	Description
ECU	Electronic Control Unit
SPRMIB	SuppressPositiveResponseMessageIndicationBit

•Message timing definition

4.10.1.1.4 Message timing definition

The timing handling during communication required by this document shall be compliant to *[3] ISO 14229-2, Road vehicles – Unified diagnostic services - Part 2: Session layer services*, section “Timing handling during communication”, with the restrictions/additions as defined by this specification.

665752 v2 **Session Layer**

Version history:
(2): Add the purpose and verification method.

Purpose: Describe the requirement of session layer.
Source:
Verification Method: Analysis
Legacy ID:

The session layer defined in *[3] 14229-2 Road vehicles — Unified diagnostic services — Part 2: Session layer services* shall be implemented with restriction/additions defined by the requirements in this specification.

4.10.1.2 UDS Session ECU reqs

•Timing parameters

4.10.1.2.1 Timing parameters

The definition of the session layer timing parameters required by this document is compliant to *[3] ISO 14229-2* with the restrictions/additions as defined by the subsequent sub-sections.

4.10.1.2.1.1 Message timing definition

4.10.1.2.1.1.1 P2Server

662497 v1 P2Server_max - ProgrammingSession

Version history:
(1):

Purpose: P2Server_max is the maximum time for the server to start with the response message after the reception of a request message
Source:
Verification Method: Test
Legacy ID: 56150v1

The maximum time for $P2_{Server}$ in programmingSession is 25 ms.

662496 v1 P2Server_max - non programming session

Version history:
(1):

Purpose: P2Server_max is the maximum time for the server to start with the response message after the reception of a request message.
Source:
Verification Method: Test
Legacy ID: 56151v2

The maximum time for $P2_{Server}$ in all sessions except programmingSession is 50 ms.

662498 v1 P2server_min

Version history:
(1):

Purpose: P2Server_min is the minimum time for the server to start with the response message after the reception of a request message.
Source:
Verification Method: Analysis
Legacy ID: 56148v1

The minimum time for $P2_{Server}$ in all sessions is 0ms.

4.10.1.2.1.1.2 P2(star)Server

662499 v1 P2(star)Server_max

Version history:
(1):

Purpose: P2*Server_max is the maximum time for the server to start with the response message after the transmission of a negative response 0x78 (enhanced response timing).
Source:
Verification Method: Test
Legacy ID: 56155v1

The maximum time for P2* $_{Server}$ in all sessions is 5000 ms.

662500 v4 P2(star)Server_min

Version history:
(4): 对于”close to P2server_max“明确可允许的最大间隔。Verification Method: Test更新为Analysis AND Test。

Purpose: P2*Server_min is specified differently for final response and pending response. For final response it shall be as short as possible to allow good timing performance. For pending response it must be a suitable value so that a server do not significantly increase bus load by consecutive pending responses.
Source:
Verification Method: Analysis AND Test
Legacy ID: 56154v2

For a final response the minimum time for P2* $_{Server}$ in all sessions is 0ms.
During the enhanced response timing, the range time between the transmission of consecutive negative messages (each with negative response code 0x78) shall be 0.3 * P2* $_{Server_max}$ to 0.8 * P2* $_{Server_max}$, in order to avoid flooding the data link with unnecessary negative response messages. E.g. It may involve parameter *DcmTimStrP2StarServerAdjust.*

Note: the first negative message with response code 0x78 to diagnostic request should be close to P2server_max, with a maximum difference of one scheduling cycle. E.g. It may involve parameter *DcmTimStrP2ServerAdjust.*

•Timing handling during communication

4.10.1.2.2 Timing handling during communication

The timing handling during communication required by this document is compliant to *[3] ISO 14229-2, Road vehicles – Unified diagnostic services - Part 2: Session layer services*, section “Timing handling during communication” with the restrictions/additions as defined by the subsequent sub-sections.

4.10.1.2.2.1 Queued request messages to one server (Zone ECUs Support)

Messages sent in-between the client and the server may pass several gateways which results in greater latency, compared to if the client was directly connected to same network as the server. To make the client server communication faster, a feature is introduced named “queued request”. The method used is to send an extra request directly after sending the first request (i.e. receiving the response of the first request). The time saved by using queued request is the response time up to the client + client processing the response and then send a new request + the time for the second request to be sent from the client to the server through all gateways.
A LIN slave shall not implement support for queued request. The LIN network/transport layer standard is not allowing this feature. The LIN master is therefore required to buffer the second (queued) request.

Note: The Queued request requirement only be supported by Zone ECU Nodes.

662503 v1 Order of responses in programming session

Version history:
(1):

Purpose: The order the server responds to queued requests must be defined.

Source:
Verification Method: Test
Legacy ID: 56197v1

In programming session, physical diagnostic requests addressed to a server shall be processed and responded to in the order they were received.

662504 v1 Server behavior for additional client requests in programming session

Version history:
(1):

Purpose: If the server is processing a request and receives a request that is supposed to be placed in the receive queue and the receive queue is full, a predictable behaviour is required.
Source:
Verification Method: Test
Legacy ID: 56200v5

In programming session, if the server is processing a request and it receives a request that is supposed to be placed in the receive queue and the receive queue is full; the request just received shall be ignored.
Note: This requirement applies regardless of the actual size of the receive queue.
Example assuming minimum 1 queued request is required; If the server is processing a first request and queues a second request, and then a third request appears, the third request shall be ignored unless it can be queued as well.

662501 v2 Server support of queued requests in programming session

Version history:
(2):

Purpose: To reduce the (delta)P2Request time.
Source:
Verification Method: Test
Legacy ID: 56192v3

In programming session the server shall be able to handle the minimum of one (1) queued physical request during processing of an already received physical request which do not lead to a server reset , meaning the server shall store the second request in a FIFO queue for later processing.
If the server isn't able to receive the complete second message due to lack of receive buffer space the server shall halt the message with the help of the network/transport layer e.g. WAIT on CAN/FlexRay or window size on TCP. Messages that can be fully received in addition to the minimum one queued request shall be stored in queue. If the queue is full, any further requests shall be ignored.

Note: Queued physical requests aboved shall include Service-TransferData UDS $36 and Service-RequestTransferExit UDS $37 services at least.

662502 v1 Start of P2Server for queued request in programming session

Version history:
(1):

Purpose: To reduce latency when the server shall start process the queued request.
Source:
Verification Method: Test
Legacy ID: 56194v1

In programming session, the server shall start the P2 $_{Server}$ timer for the queued request once the final response (indicated via Server T_Data.con) of the previous request is transmitted.
If the queued request is not completely received at the server when the final response of the previous request is transmitted (e.g. the message is halted or a large queued request is received) shall the server start the P2 $_{Server}$ timer when the complete queued message is received (indicated via Server T_Data.ind).

4.10.1.2.2.2 P4 definition

P4Server_max is the maximum time allowed for a server from reception of a request until start of transmission of the final response. P4Server_max is specific for each diagnostic request the server supports.

662505 v1 **P4Server_max definition**

Version history:
(1):

Purpose: To define and set the performance timing conditions how long time the server is allowed to process a diagnostic request.
Source:
Verification Method: Test
Legacy ID: 56203v1

The timing parameter P4 $_{Server_max}$ is the maximum time between the reception of a request (Server T_Data.ind) and the start of transmission of the final response (Server T_Data.req).
Note: A final response is a positive response or a negative response other than response code 0x78 “requestCorrectlyReceived-ResponsePending". In case of a request to schedule periodic responses, the initial positive or negative response that indicates the acceptance or non-acceptance of the request to schedule periodic responses shall be considered the final response.

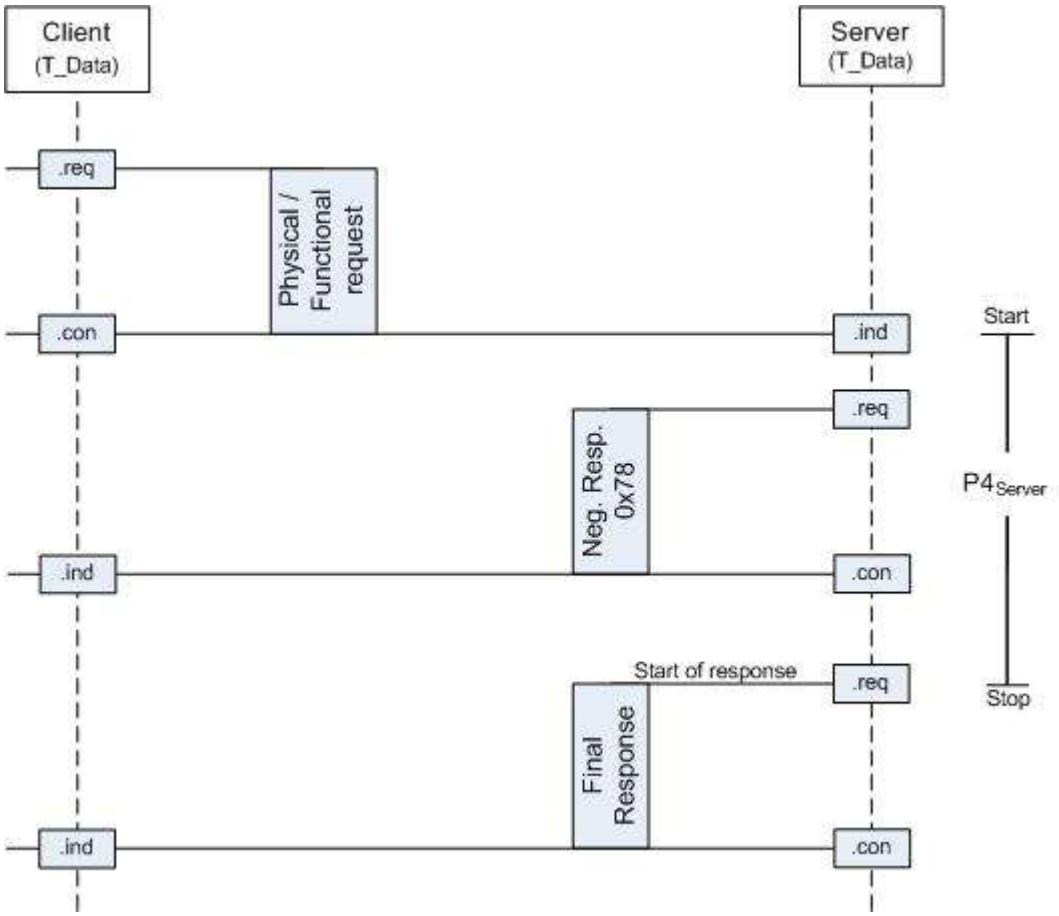


Figure – Definition P4 $_{Server}$ is the actual time which is allowed to be maximum P4 $_{Server_max}$

662506 v1 **P4Server_max equal to P2Server_max**

Version history:
(1):

Purpose: To define the behaviour when the P4Server_max is equal to P2Server_max.
Source:
Verification Method: Test
Legacy ID: 56204v1

The server is not allowed to response with a negative response code 0x78 (requestCorrectlyReceived-ResponsePending) if P4 $_{Server_max}$ is the same as P2 $_{Server_max}$.
Note: The value of P2 $_{Server_max}$ is defined in section Timing parameters. P4 $_{Server_max}$ is defined for each diagnostic service in *[1] UDS Services*.

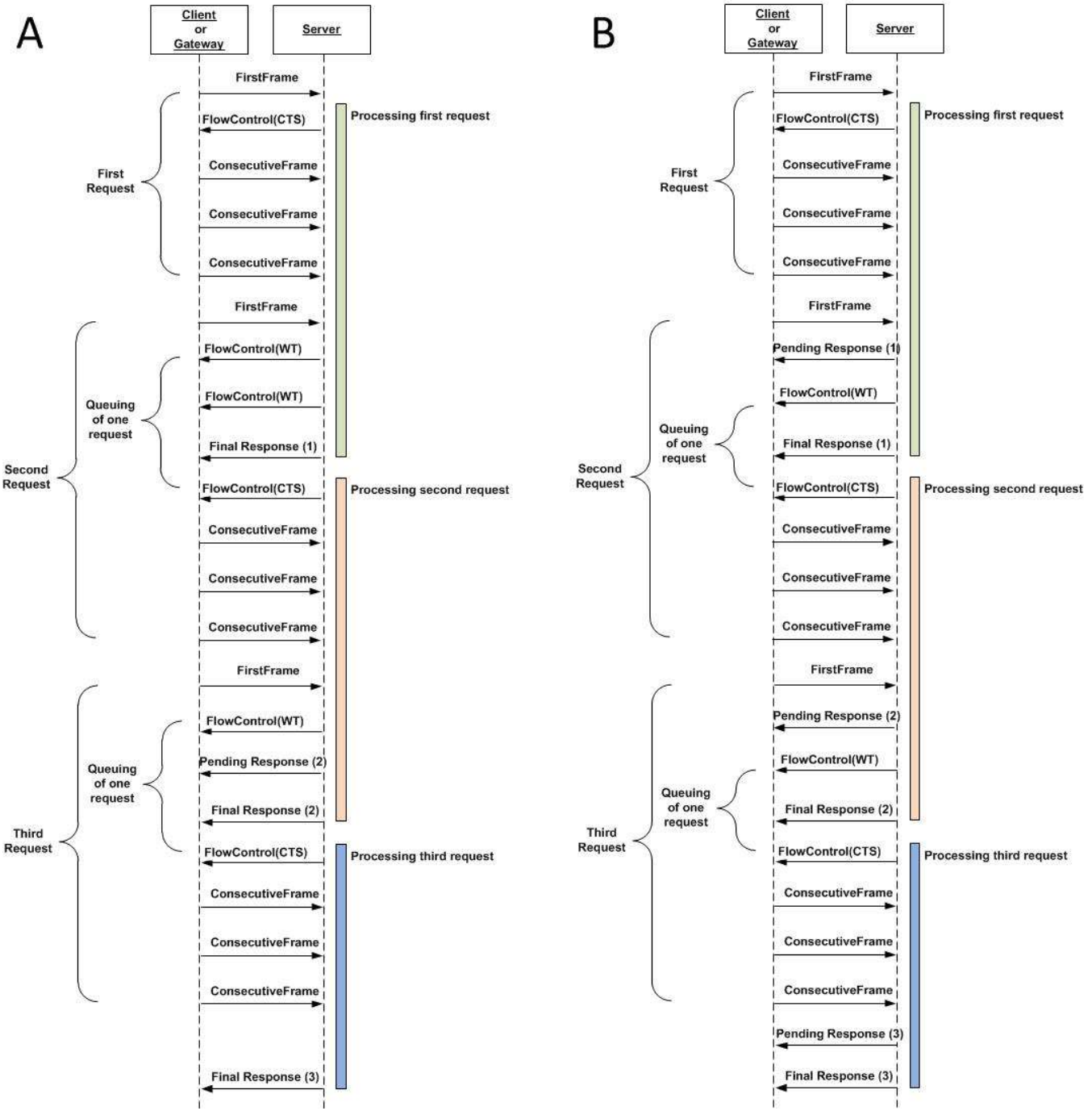
•Appendix

4.10.1.2.3 Appendix

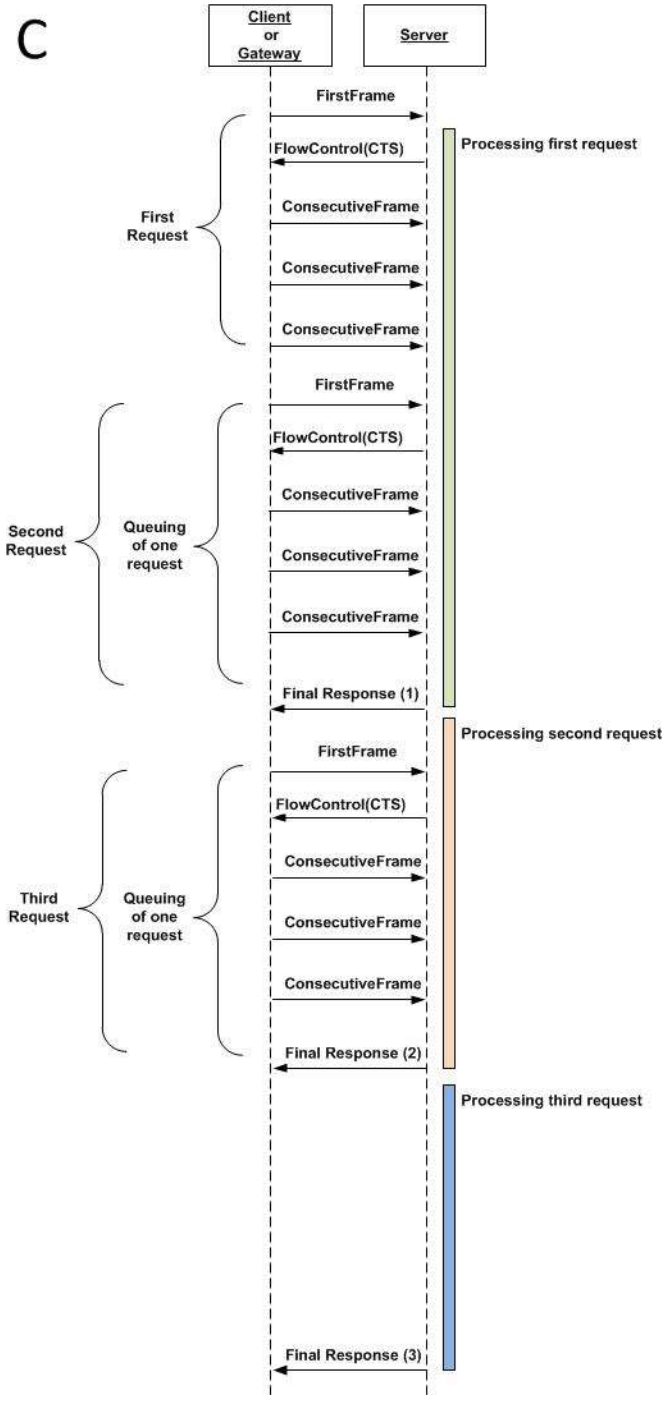
4.10.1.2.3.1 Example, queuing of requests in a server

The sequence below shows examples where multiple requests are sent to a server, a typical case is in programming session where the client will send additional request to be queued in the server. The requests are typically TransferData with Block 1, 2, 3 and so on where the data is known in advance and do not depend on any

data received in a previous response.



Examples of queuing of requests towards a CAN server.
The server is not able to receive complete queued message while processing the current message. A pending response, NRC 0x78, for the current message is sent by ECU in various order compared to the FC.WAIT for the queued message.

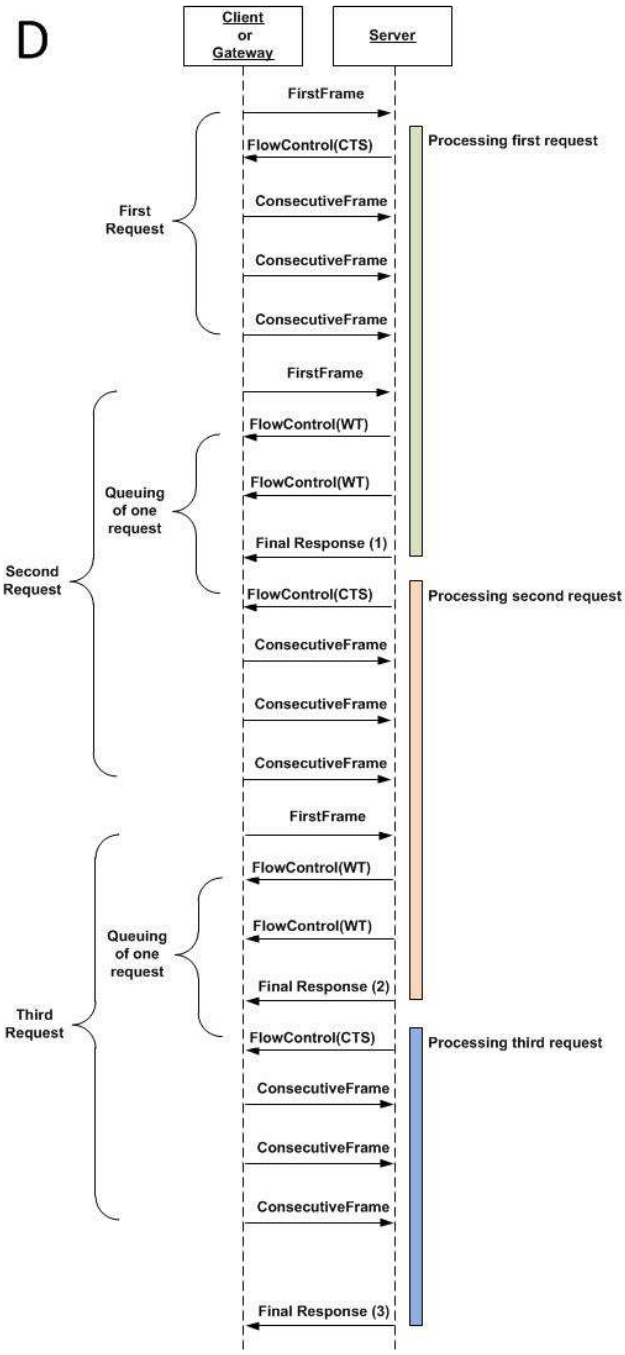


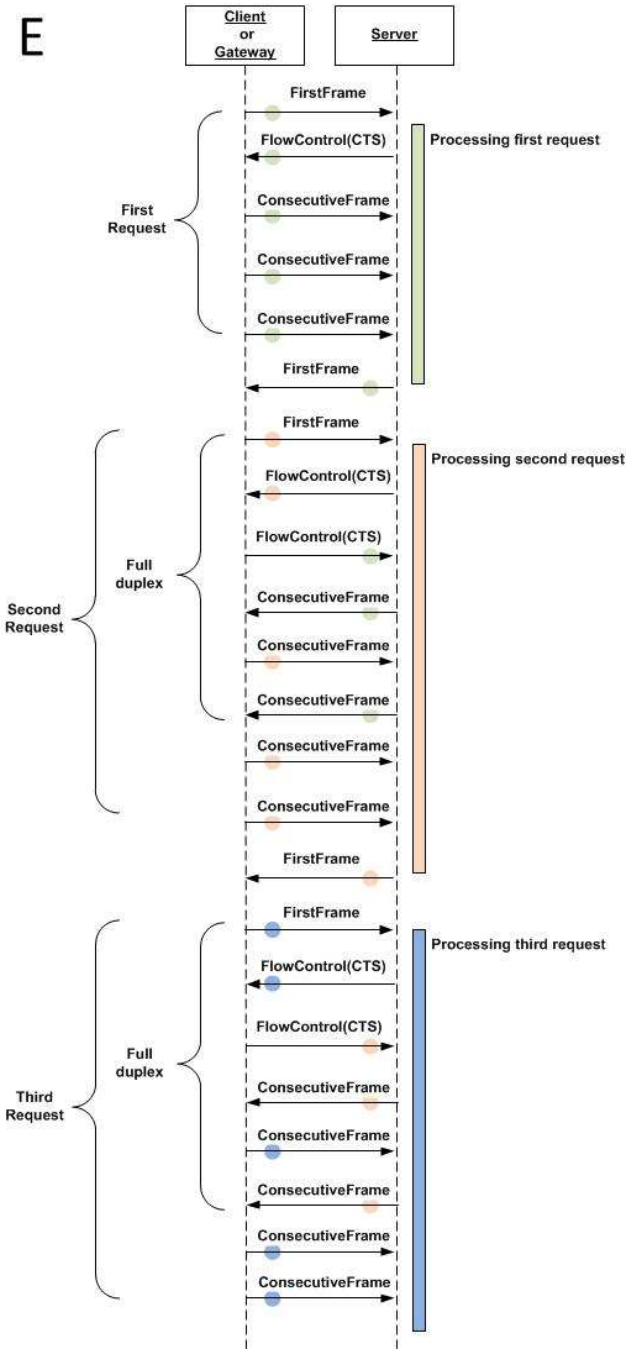
Examples of queuing of requests towards a CAN server.
The server is able to receive complete queued message, and stores it internally until the previous message has been processed and its response sent.

4.10.1.2.3.2 Example, queuing of requests versus full duplex

The purpose of the sequences below is to clarify the difference between queued requests, an OEM specific feature, and full duplex communication. If the server has not yet begun responding (i.e. has not yet sent a NRC 0x78 or a final reponse) to a previous request and a new request has been transmitted to the server, or begin to be transmitted in case of segmented request to server, the second request is considered queued. Since it is not possible to determine by observing network communication between a request that has been processed completely inside a server but not yet responded to and a request that has not yet been processed completely by a server, the

observable network communication will be used to judge if a specific sequence is queued requests or full duplex communication.
The sequences shows examples where multiple requests are sent to a server, a typical case in programming session where the client will send additional request to be queued in the server. If gateways are located between client and server, due to delays in gateways, the second request (believed by client to be queued in vehicle) may arrive to the server slightly differently now and then in time relation to the server responses. The server may already have started responding to a previous request. In that case the second request is not considered to be queued, as percieved by server, but is instead only full duplex communication.





Examples of queuing of requests towards a CAN server and full duplex communication.
In sequence to left is the server has not yet begun responding to the second request and that request is

therefore considered to be a queued request.
In sequence to right the server has begun the response by a First Frame (processing of request is completed but response not yet completely sent) when a second request is received. Since the (segmented) response is in progress, and a new request is received within the response transmission it is full duplex instead of queued request.

4.10.1.3 UDS Session client reqs

•Timing parameters

4.10.1.3.1 Timing parameters

The session layer defined in *[3] Road vehicles — Unified diagnostic services — Part 2: Session layer services* shall be implemented with restriction/additions defined by the requirements in this specification.

4.10.1.3.1.1 ISO 13400 DoIP message timing definition

Definition	Description
D P2 $_{External_Request}$	The time is introduced by the external network. The time is started when the DoIP request is transmitted from the DoIP client T_Data.req and stops when the DoIP Ack or Nack is received T_Data.con.
D P2 $_{External_Response}$	The time is off-vehicle network dependent. The DP2 $_{External_Response}$ includes the transmission and delays introduced in the external network which the external diagnostic tester is connected to.
D P2 $_{Vehicle}$	D P2 $_{Vehicle}$ is in-vehicle network dependent and is divided into the time to transmit the request from the DoIP gateway to the addressed server (DP2 $_{Vehicle_Request}$) and the time to transmit the response to the DoIP gateway (DP2 $_{Vehicle_Response}$). Depending on length of requests and responses and number of simultaneously used communication channels, one of the D P2 $_{Vehicle_singleframe}$, D P2 $_{Vehicle_multiframe}$ or D P2 $_{Vehicle_multichannel}$ shall be used. Which value to use will be determined dynamically depending on how the client uses the vehicle. Therefore other timing parameters that are derived partly from DP2 $_{Vehicle}$ will also be determined dynamically. D P2 $_{Vehicle}$ = D P2 $_{Vehicle_Request}$ + D P2 $_{Vehicle_Response}$.
D P2 $_{Vehicle_singleframe}$	Replacement value for D P2 $_{Vehicle}$ when single frame request/response message is used. See also definition of D P2 $_{Vehicle}$.
D P2 $_{Vehicle_multiframe}$	Replacement value for D P2 $_{Vehicle}$ when multi frame request/response message is used. See also definition of D P2 $_{Vehicle}$.
D P2 $_{Vehicle_multichannel}$	Replacement value for D P2 $_{Vehicle}$ when multi channel request/response message is used. See also definition of D P2 $_{Vehicle}$.
D P2 $_{Vehicle_Request}$	The in-vehicle time to transmit the request from the DoIP gateway “Edge node” to the addressed server.
D P2 $_{Vehicle_Response}$	T he in-vehicle time to transmit the response to the DoIP gateway “Edge node” from a server.
D P6	This is the same as DP2 in this document.
P6 $_{Client}$.	Timeout for the client to wait after the successful transmission of a request message (indicated via T_Data.con) for the complete reception of the corresponding response message (indicated via T_Data.ind). If the transport/network layer doesn't support the T_DataSOM.ind (e.g. ISO 13400 DoIP), P6 $_{Client}$ shall be used instead of P2 $_{Client}$.
P6* $_{Client}$.	Enhanced timeout for the client to wait after the reception of a negative response message with negative response code 0x78 (indicated via T_Data.ind). If the transport/network layer doesn't support the T_DataSOM.ind interface (e.g. ISO 13400 DoIP), P6* $_{Client}$ shall be used instead of P2* $_{Client}$.

Table – Definition of message timing ISO 13400 DoIP

The ISO-13400 DoIP P6 $_{Client}$ timeout is message length dependent due to the ISO-13400 DoIP transport control protocol provides an indication to the application layer at the end of a received message while e.g. ISO 15765-2 DoCAN transport protocol provides a FirstFrame indication at the beginning of the response message; see section Example, message length dependent parameter P6$_{Client}$.
To reduce time between diagnostic requests the client/tester is allowed to use different timeout values for the P6

$_{Client}$ which is a message length dependent timeout.
Note: *The time delay for the diagnostic request introduced by the external network is not included in the P6 $_{Client}$*calculation since the P6 $_{Client}$is started when the client receives the DoIP Ack or Nack. The client must therefore add time introduced by the external network. This time delay is defined by this document as *D*P2 $_{External_Request}$.

Timing Parameter	**ProgrammingSession**		**Other sessions** (All supported sessions except programmingSession)	
	Value		Value	
	Min	Max	Min	Max
DP2	0 ms	DP2 $_{Vehicle_max}$ + DP2 $_{Ext_Response_max}$	0 ms	DP2 $_{Vehicle_max}$ + DP2 $_{Ext_Response_max}$
DP2 $_{External_Response}$	0 ms	5 ms	0 ms	5 ms
$_{D}$P2Vehicle_singleframe	0 ms	200 ms	0 ms	200 ms
DP2 $_{Vehicle_multiframe}$	0 ms	4970 ms	0 ms	13945 ms
$_{D}$P2Vehicle_multichannel	0 ms	90000 ms	0 ms	90000 ms
P2 $_{Server}$	See requirement in *UDS Session ECU reqs*	See requirement in *UDS Session ECU reqs*	See requirement in *UDS Session ECU reqs*	See requirement in *UDS Session ECU reqs*
P2* $_{Server}$	See requirement in *UDS Session ECU reqs* Note: shall be 0.3 x P2*server_max for NRC 0x78	See requirement in *UDS Session ECU reqs*	See requirement in *UDS Session ECU reqs* Note: shall be 0.3 x P2*server_max for NRC 0x78	See requirement in *UDS Session ECU reqs*
P3 $_{Client_Phys}$	P4 $_{Server_max}$ + DP2 $_{max}$	P3 $_{Client_Phys_min}$ + 5ms	P4 $_{Server_max}$ + DP2 $_{max}$	P3 $_{Client_Phys_min}$ + 5ms
P3 $_{Client_Func}$	P4 $_{Server_max_}$ + DP2 $_{max}$	P3 $_{Client_Func_min}$ + 5ms	P4 $_{Server_max}$ + DP2 $_{max}$	P3 $_{Client_Func_min}$ + 5ms
P6 $_{Client}$	P2 $_{Server_max}$ +DP2 $_{max}$	P6 $_{Client_min}$x 1.1	P2 $_{Server_Max}$ +DP2 $_{max}$	P6 $_{Client_min}$ x 1.1
P6* $_{Client}$	P2* $_{Server_max}$ +DP2 $_{max}$	P6* $_{Client_min}$ x 1.1	P2* $_{Server_max}$ +DP2 $_{max}$	P6* $_{Client_min}$ x 1.1

Table – Overview, message timing definition for ISO 13400 DoIP

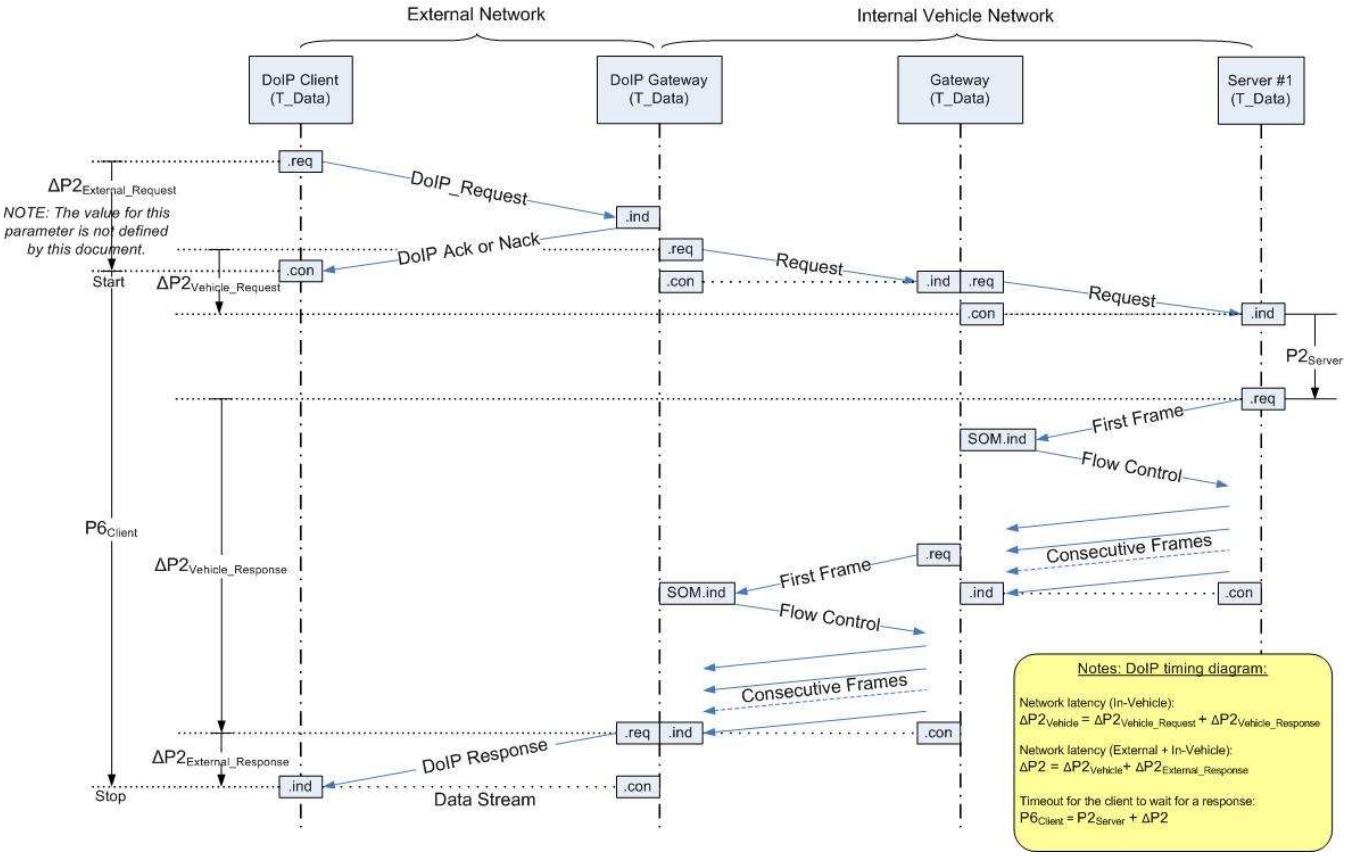


Figure – ISO 13400 DoIP timing diagram

4.10.1.3.1.1.1 (delta)P2

The DP2 parameter is defined to be the worst case message transmission delay such as delays introduced by gateways, networks and bus-load arbitration. The value of DP2 is divided into the time to transmit the request from the DoIP gateway to the addressed server (DP2 $_{Vehicle_Request}$) and the time to transmit the response to the DoIP client/tester (DP2 $_{Vehicle_Response}$ + DP2 $_{External_Response}$), see the figure above – ISO 13400 DoIP timing diagram .

662508 v1 **(delta)P2max**

Version history:
(1):

Purpose: The time to transport the request and response message from the edge node to the server back to the client needs to be defined, so the client may set up a timeout in case the message is lost along the way.

Source:

Verification Method: Test

Legacy ID: 56113v3

The maximum time for D P2 in all sessions is equal to D P2 $_{Vehicle_max}$ + D P2 $_{External_Response_max}$.
Note: D P2 $_{Vehicle_max}$ is a generic parameter for the transfer delay of requests and responses in the vehicle. However in other requirements the value of D P2 $_{Vehicle_max}$ shall be replaced by the value of D P2 $_{Vehicle_singleframe}$, D P2 $_{Vehicle_multiframe}$ or D P2 $_{Vehicle_multichannel}$ depending on the actual client usage of the vehicle.

662507 v1 **(delta)P2min**

Version history:
(1):

Purpose: The client shall not be the bottleneck in the client/server communication
Source:
Verification Method: Test
Legacy ID: 56111v1

The minimum time for D P2 in all sessions is 0ms. That means that the client (i.e. tester) shall be able handle response messages, being sent back to back.
NOTE: D P2 $_{min}$ is the minimum transmission time to transmit the request to the addressed server/ECU and the time to transmit the response to the client/tester.

4.10.1.3.1.1.2 (delta)P2External_Response

662510 v1 (delta)P2External_Response_max

Version history:
(1):

Purpose: (delta)P2External_Response_max is a part of the total deltaP2 time and must therefore be kept to a minimum.
Source:
Verification Method: Test
Legacy ID: 56116v1

The maximum time for DP2 $_{External_Response}$ in all sessions is off-vehicle network dependent.
Recommended max time is 5 ms to transmit a message with the length of 4095 data bytes on an Ethernet cable connected IP based data link.
Note:
1) D P2 $_{External_Response_max}$ is the maximum transmission time introduced by the external network which the external diagnostic tester is connected to. The time is off-vehicle dependent and is a performance requirement on the external network connected to the vehicle
2) D P2 which is equal to D P2 $_{Vehicle_max}$ + D P2 $_{External_Response_max}$, must be re-calculated if the external network introduces longer transmission time than the recommended.
3) The DP2 $_{External_Response_max}$ is based on two different situations:

- A point-to-point connection between the vehicle and the tester.
- Tester and the Vehicle are located within same LAN with maximum one switch/router in-between.

662509 v1 (delta)P2External_Response_min

Version history:
(1):

Purpose: The client shall not be the bottleneck in the client/server communication.
Source:
Verification Method: Test
Legacy ID: 56115v1

The minimum time for D P2 $_{External_Response}$ in all sessions is 0ms. That means that the client (i.e. tester) shall be able handle response messages, being sent back to back.
NOTE: DP2 $_{External_Response_min}$ is the minimum transmission time introduced by the external network which the external diagnostic tester is connected to.

4.10.1.3.1.1.3 (delta)P2Vehicle

The value of DP2 $_{Vehicle}$ is in-vehicle network dependent and is divided into the time to transmit the request from the DoIP gateway “Edge node” to the addressed server (DP2 $_{Vehicle_Request}$) and the time to transmit the response to the

DoIP gateway “Edge node” (DP2 $_{Vehicle_Response}$). To improve timing performance during detection of a missing response the client shall use different values of DP2 $_{Vehicle}$ depending on message length and session type (A shorter time for short messages and longer times for longer messages and multi channel usage).

662520 v1 **(delta)P2Vehicle_multichannel_max in other than programming session**

Version history:
(1):

Purpose: Setting a specific (delta)P2Vehicle for multi channel request and response improves timing performance by only using the long timeout when needed.
Source:
Verification Method: Test
Legacy ID: 110651v1

The maximum time for D P2 $_{Vehicle_multichannel_max}$ in all sessions except programming session is 90000 ms.
Note: D P2 $_{Vehicle_multichannel_max}$ = D P2 $_{Vehicle_Request}$ + D P2 $_{Vehicle_Response}$.

662519 v1 **(delta)P2Vehicle_multichannel_max in programming session**

Version history:
(1):

Purpose: Setting a specific (delta)P2Vehicle for multi channel request and response improves timing performance by only using the long timeout when needed.
Source:
Verification Method: Test
Legacy ID: 110649v1

The maximum time for D P2 $_{Vehicle_multichannel_max}$ in programming session is 90000 ms.
Note: D P2 $_{Vehicle_multichannel_max}$ = D P2 $_{Vehicle_Request}$ + D P2 $_{Vehicle_Response}$

662518 v1 **(delta)P2Vehicle_multichannel_min**

Version history:
(1):

Purpose: To clarify primarily for the client that it shall handle the case where a response is received immediately after the request.
Source:
Verification Method: Test
Legacy ID: 110648v1

The minimum time for D P2 $_{Vehicle_multichannel_min}$ in all sessions is 0 ms.
Note: D P2 $_{Vehicle_multichannel_min}$ = D P2 $_{Vehicle_Request}$ + D P2 $_{Vehicle_response}$

662517 v1 **(delta)P2Vehicle_multiframe_max in other than programming session**

Version history:
(1):

Purpose: Setting a specific (delta)P2Vehicle for long request and response improves timing performance by only using the long timeout when needed. Compare with (delta)P2Vehicle _singleframe_max.
Source:

Verification Method: Test
Legacy ID: 110647v1

The minimum time for D P2 $_{Vehicle_multiframe_max}$ in all sessions except programming session is 13945 ms.
Note: DP2 $_{Vehicle_multiframe_max}$ = DP2 $_{Vehicle_Request}$ + DP2 $_{Vehicle_Response}$.

662516 v1 **(delta)P2Vehicle_multiframe_max in programming session**

Version history:
(1):

Purpose: Setting a specific (delta)P2Vehicle for multiframe request and response improves timing performance by only using the longer timeout when needed. Compare with (delta)P2Vehicle _singleframe_max.
Source:
Verification Method: Test
Legacy ID: 110644v1

The minimum time for D P2 $_{Vehicle_multiframe_max}$ in programming session is 4970 ms.
Note: D P2 $_{Vehicle_multiframe_max}$ = D P2 $_{Vehicle_request}$ + D P2 $_{Vehicle_Response}$.

662515 v1 **(delta)P2Vehicle_multiframe_min**

Version history:
(1):

Purpose: To clarify primarily for the client that it shall handle the case where a response is received immediately after the request.
Source:
Verification Method: Test
Legacy ID: 128609v0

The minimum time for D P2 $_{Vehicle_multiframe_min}$ in all sessions is 0 ms.
Note: D P2 $_{Vehicle_multiframe_min}$ = D P2 $_{Vehicle_Request}$ + D P2 $_{Vehicle_Response}$.

662514 v1 **(delta)P2Vehicle_singleframe_max in other than programming session**

Version history:
(1):

Purpose: Setting a specific (delta)P2Vehicle for short request and response improves timing performance when detecting an error, e.g. missing response. Compare with (delta)P2Vehicle _multiframe_max.
Source:
Verification Method: Test
Legacy ID: 110643v1

The maximum time for D P2 $_{Vehicle_singleframe_max}$ in all sessions except programming session is 200 ms.
Note: D P2 $_{Vehicle_singleframe_max}$ = D P2 $_{Vehicle_Request}$ + D P2 $_{Vehicle_Response}$

662513 v1 **(delta)P2Vehicle_singleframe_max in programming session**

Version history:
(1):

Purpose: Setting a specific (delta)P2Vehicle for short request and response improves timing performance when detecting an error, e.g. missing response. Compare with (delta)P2Vehicle _multiframe_max.
Source:
Verification Method: Test
Legacy ID: 110642v1

The maximum time for D P2 $_{Vehicle_singleframe_max}$ in programming session is 200 ms.
Note: D P2 $_{Vehicle_singleframe_max}$ = D P2 $_{Vehicle_Request}$ + D P2 $_{Vehicle_Response}$

662512 v1 (delta)P2Vehicle_singleframe_min

Version history:
(1):

Purpose: To clarify primarily for the client that it shall handle the case where a response is received immediately after the request.
Source:
Verification Method: Test
Legacy ID: 110641v1

The minimum time for D P2 $_{Vehicle_singleframe_min}$ in all sessions is 0 ms.
Note: D P2 $_{Vehicle_singleframe_min}$ = D P2 $_{Vehicle_Request}$ + D P2 $_{Vehicle_Response}$.

662511 v1 Dynamic selection of (delta)P2Vehicle based on message length

Version history:
(1):

Purpose: For the client to know when to use (delta)P2Vehicle_singleframe, (delta)P2Vehicle_multiframe or (delta)P2Vehicle_multichannel. To improve timing performance during error detection a short and two longer timeouts are used. When it is known that the request will fit in a single frame on all transport layers that may be involved between client and servers a shorter (delta) P2Client shall be used which reduce error detection time.
Source:
Verification Method: Test
Legacy ID: 110584v1

The client shall use $\Delta P2_{Vehicle}$ based on the length of the request and corresponding response, length of A_Data according to *[3] ISO-14229-1* according to the table below. Length of request and response and whether multi channel requests are used shall affect which parameter to use as $\Delta P2_{Vehicle}$ and therefore being the base for ΔP2, $P6_{Client}$/$P6^*_{Client}$. In any situation where a reference is made to $\Delta P2_{Vehicle}$ in general it means the actual $\Delta P2_{Vehicle_singleframe}$, $\Delta P2_{Vehicle_multiframe}$ or $\Delta P2_{Vehicle_multichannel}$ parameter determined by this requirement. Those 3 parameters in turn each have a minimum and a maximum value which are defined in other requirements.

	Requests to more than 1 server simultaneously (multi channel usage)	
Length of A_Data in request and response	No	Yes
Request and response is maximum 6 bytes each	$\Delta P2_{Vehicle_singleframe}$	$\Delta P2_{Vehicle_multichannel}$
Request or response or both are more than 6 bytes each	$\Delta P2_{Vehicle_multiframe}$	$\Delta P2_{Vehicle_multichannel}$

Request length is known and response length is unknown	$\Delta P2_{Vehicle_multiframe}$	$\Delta P2_{Vehicle_multichannel}$

Note: the threshold of 6 bytes is derived from the transport layer protocol that has the lowest number of payload bytes; LIN Transport Protocol implemented on LIN in a single frame, to ensure ΔP2 Vehicle_singleframe is used when single frames are used on all networks. The client does not know which transport layer will be used inside the vehicle or even how many transport layers in case of gatewaying. Despite now knowing actual transport layer protocols used the simple algorithm described in the requirement shall be used in the application layer.

4.10.1.3.1.1.4 P6Client

Timeout for the client to wait after the successful transmission of a request message (indicated via T_Data.con) for the complete reception of the corresponding response message (indicated via T_Data.ind). If the transport/network layer doesn't support the T_Data.SOM.ind interface (e.g. ISO 13400 DoIP), $P6_{Client}$ shall be used instead of $P2_{Client}$.

662522 v1 **P6Client_max**

Version history:
(1):

Purpose: Ensures that detection of a missing final response is performed as soon as possible and still allowing for some timing flexibility in the client.
Source:
Verification Method: Test
Legacy ID: 56158v1

The maximum time a client waits for a response message to start, is at the discretion of the client, provided that $P6_{Client}$ is greater than the specified minimum value of $P6_{Client}$.
Recommended max time is $P6_{Client_min}$ x 1.1.

662521 v1 **P6Client_min**

Version history:
(1):

Purpose: Ensures that maximum time for transmission of request and a final response message is allowed and enabling a client to detect a missing response as soon as possible to improve timing performance.
Source:
Verification Method: Test
Legacy ID: 56157v2

The minimum time for $P6_{Client}$ in all sessions is equal to $P2_{Server_max}$ + $D\ P2_{_max}$.

4.10.1.3.1.1.5 P6(star)Client

Enhanced timeout for the client to wait after reception negative response code 0x78 (indicated via (T_Data.ind) for the complete reception of the corresponding response messages (indicated via T_Data.ind). If the transport/network layer doesn't support the T_DataSOM.ind interface (e.g. ISO 13400 DoIP, $P6^*_{Client}$ shall be used instead of $P2^*_{Client}$.

662524 v1 **P6(star)Client_max**

Version history:
(1):

Purpose: Ensures that detection of a missing response during enhanced timing is performed as soon as

possible and still allowing for some timing flexibility in the client.

Source:

Verification Method: Test

Legacy ID: 56162v1

The maximum time a client waits for a response message to start, is at the discretion of the client, provided that P6* $_{Client}$ is greater than the specified minimum value of P6* $_{Client}$.
Recommended max time is P6* $_{Client_min}$ x 1.1.

662523 v1 **P6(star)Client_min**

Version history:
(1):

Purpose: Ensures that maximum time for sending a new response during enhanced timing is allowed and enabling a client to detect a missing response as soon as possible to improve timing performance.

Source:

Verification Method: Test

Legacy ID: 56161v1

The minimum time for P6* $_{Client}$ in all sessions is equal to P2* $_{Server_max}$ + D P2 $_{_max}$.

4.10.1.3.1.1.6 P3Client_Phys

662526 v1 **P3Client_Phys_max**

Version history:
(1):

Purpose: P3Client_Phys_max is the maximum time for the client to wait after the successful transmission of a physically addressed request message with no response required before it is allowed to transmit the next request message.

Source:

Verification Method: Test

Legacy ID: 56168v1

The maximum time in all sessions a client shall wait until the client may transmit the next request message, is at the discretion of the client, provided it is greater than the specified minimum value of P3 $_{Client_Phys_min}$.
Recommended max time is P3 $_{Client_Phys_min}$ + 5ms.

662525 v1 **P3Client_Phys_min**

Version history:
(1):

Purpose: P3Client_Phys_min is the minimum time for the client to wait after the successful transmission of a physically addressed request message with no response required before it is allowed to transmit the next request message.

Source:

Verification Method: Test

Legacy ID: 56166v1

The minimum time in all sessions for P3 $_{Client_Phys_min}$ is P4 $_{Server_max}$ + D P2 $_{max}$.

4.10.1.3.1.1.7 P3Client_Func

662528 v1 **P3Client_Func_max**

Version history:
(1):

Purpose: P3Client_Func_max is the maximum time for the client to wait after the successful transmission of a functionally addressed request message with no response required before it is allowed to transmit the next request message.
Source:
Verification Method: Test
Legacy ID: 56172v1

The maximum time in all sessions a client shall wait until the client may transmit the next request message, is at the discretion of the client, provided it is greater than the specified minimum value of $P3_{Client_Func_min}$.
Recommended max time is $P3_{Client_Func_min}$ + 5ms.

662527 v1 **P3Client_Func_min**

Version history:
(1):

Purpose: P3Client_Func_min is the minimum time for the client to wait after the successful transmission of a functionally addressed request message with no response required before it is allowed to transmit the next request message.
Source:
Verification Method: Test
Legacy ID: 56171v1

The minimum time in all sessions for $P3_{Client_Func_min}$ is $P4_{Server_max}$ + $D\ P2_{max}$.

4.10.1.3.1.1.8 P4 definition

The client shall not implement a timer that is directly based on $P4_{Server}$ and D P2 . This is due to the difficulty to calculate a precise message transportation delay; $D\ P2_{Vehicle_max}$, (i.e. $D\ P2_{Vehicle_singleframe}$, $D\ P2_{Vehicle_multiframe}$ or $D\ P2_{Vehicle_multichannel}$) when using gateways and multichannel requests. However, during extended software download sequences, especially when multiple servers are requested, it may be useful to implement some form of timeout-based watchdog functionality for each communication channel to detect if a server do not respond. The timeout time could still be generous, but may be small compared to the overall software download time of all servers. Monitoring for a maximum number of negative response codes, 0x78, to detect servers getting stuck in this response behavior could also be used.

Some actions can be made when detecting a server not responding or indefinitely sends negative response:
- Notify a user about a problem who could investigate the problem while the rest of the servers are being downloaded or otherwise perform special handling of the vehicle.
- Reallocate the communication channel to a download job to another server, this can be automatic or manually activated.

•Timing handling during communication

4.10.1.3.2 Timing handling during communication

The timing handling during communication required by this document shall be compliant to *[3] ISO 14229-2, Road vehicles – Unified diagnostic services - Part 2: Session layer services*, section “Timing handling during communication”, with the restrictions/additions as defined by this specification.

4.10.1.3.2.1 Queued request messages to one server

Messages sent in-between the client and the server may pass several gateways which results in greater latency, compared to if the client was directly connected to same network as the server. To make the server client communication faster, a feature is introduced named “queued request”. The method used is to send an extra request directly after sending the first request (i.e. receiving the response of the first request). The time saved by using queued request is the response time up to the client (ΔP2External_Response + ΔP2Vehicle_Response) + client processing the response and then send a new request + the time for the second request to be sent from the client to the server (ΔP2External_Request + ΔP2Vehicle_Request) through all gateways.

662530 v1 **Client sending additional requests to same server in programming session**

Version history:
(1):

Purpose: To reduce the (delta)P2External_Request time and only send queued requests in a few situations.
Source: REQPROD 56192 - Server behaviour for additional client requests; REQPROD 56200 - Server support of queued requests
Verification Method: Test
Legacy ID: 56184v3

When in programming session, for each server where one physical request with SPRMIB=false which does not lead to a server reset has been sent and for which a final response has not yet been received, the client shall send one (1) additional request if it is physical and SPRMIB=false and the content of that request can be determined before the arrival of the previous response. Additional requests shall not be sent to same server in other situations.
Note: 1) In some situations it may be required to ignore sending an additional request since the response of the previous request must be evaluated before proceeding with the next request. This is contrary to this requirements and is accepted. During software download where the progress across multiple servers shall be synchronized at discrete steps, e.g. to wait for SBL to be downloaded in all servers before each SBL is activated.
2) If the client violates this requirement the vehicle behavior may be unpredictable.

662529 v1 **Client shall wait for the DoIP Ack or Nack before queue any request**

Version history:
(1):

Purpose: When queued requests is used it is vital to have the first request Ack:ed before sending the next one. Otherwise, if two requests is sent to the same ECU and a request is Nack:ed there will be no way in telling which request that failed and complex roll back mechanisms would have to be implemented.
Source:
Verification Method: Test
Legacy ID: 56178v1

Before sending a queued request the client must always wait for the DoIP Ack/Nack, Road vehicles – Diagnostic Communication over Internet Protocol (DoIP) , indicated via (Client T_Data.con) of the previous request.

662531 v1 **P6Client and P6(star)Client behavior during queued request**

Version history:
(1):

Purpose: To handle large (delta)P2Vehicle without expiring the P2Client unintentionally.
Source:
Verification Method: Test
Legacy ID: 56177v2

When queued request messages are used the client shall use $P6_{Client}$ and $P6^*_{Client}$ timers. The $P6_{Client}$ timer for the subsequent request shall be started when the final response is received for the previous request according to the figures below.

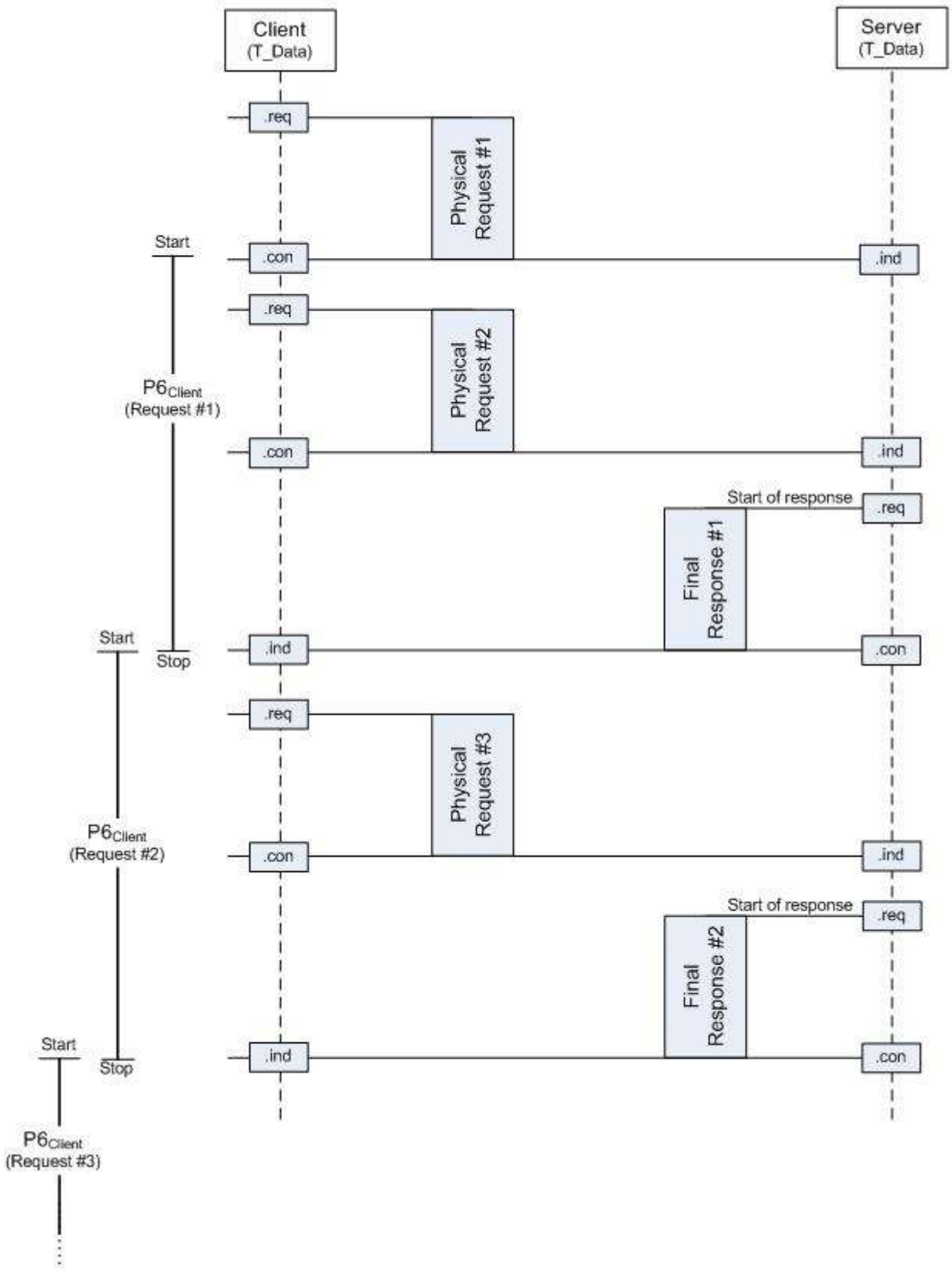


Figure - P6 $_{Client}$ and P6* $_{Client}$ behavior when queued requests are used (only final responses).

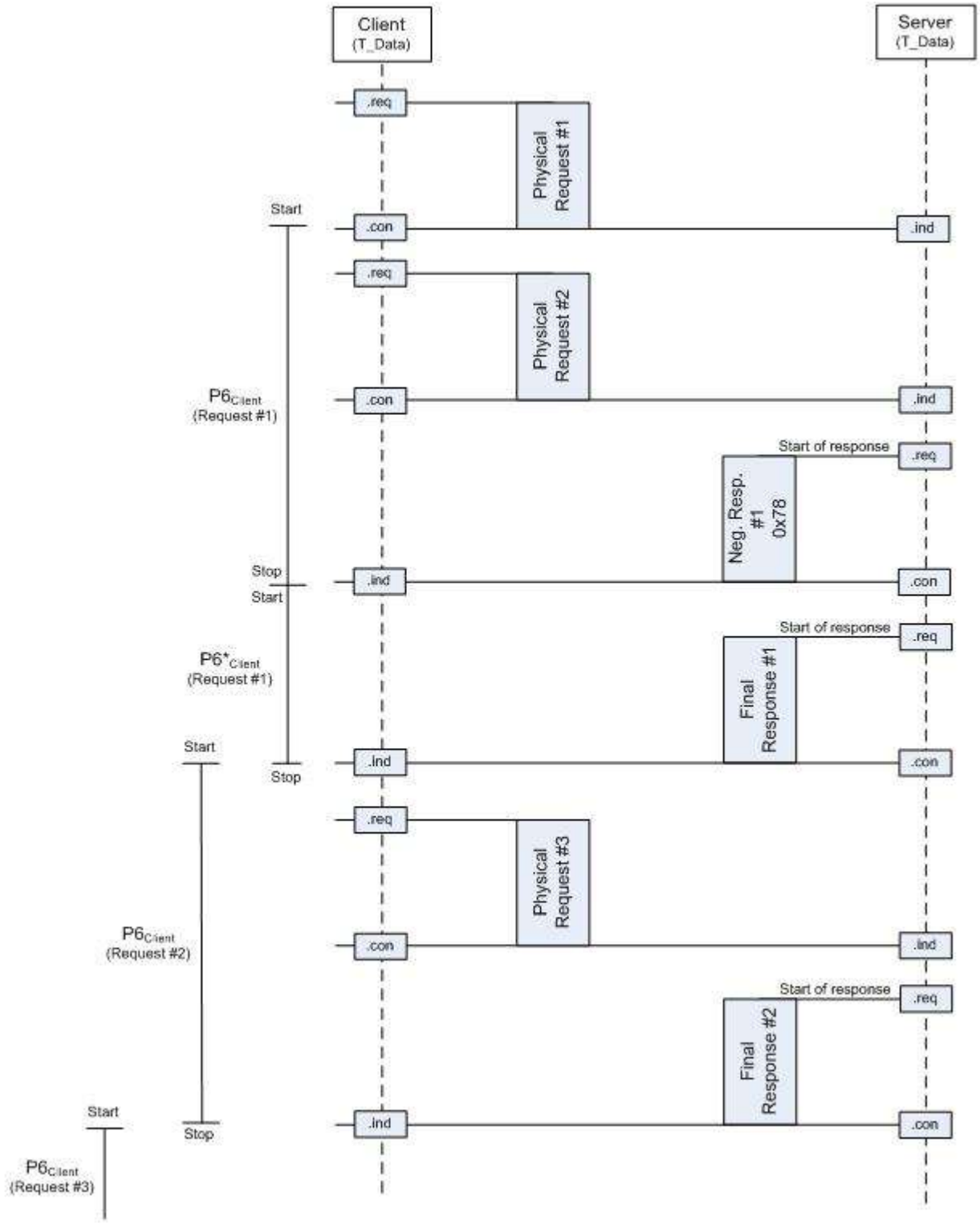


Figure - P6 $_{Client}$ and P6* $_{Client}$ behavior when queued requests are used (mixed negative and final responses).

•Appendix

4.10.1.3.3 Appendix

4.10.1.3.3.1 Example, message length dependent parameter P6Client

Since the gateway “Edge node” which is connected to the external network stores the complete message before transition the message to the in-vehicle networks shows this section a number of examples why the $P6_{Client}$ is message length dependent.

4.10.1.3.3.1.1 Single frame request and single frame response, one server

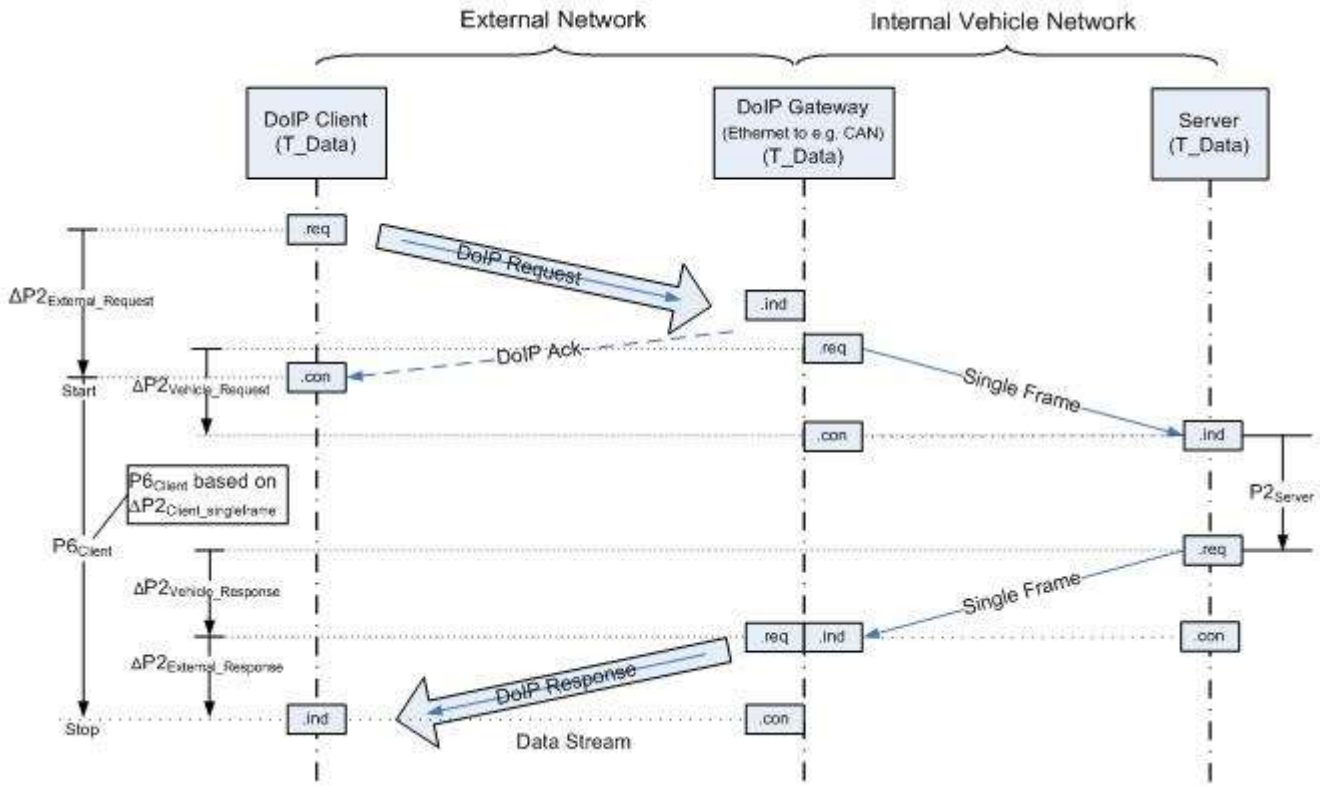


Figure - Single frame request and single frame response, one server.

4.10.1.3.3.1.2 Multi frame request and single frame response, one server

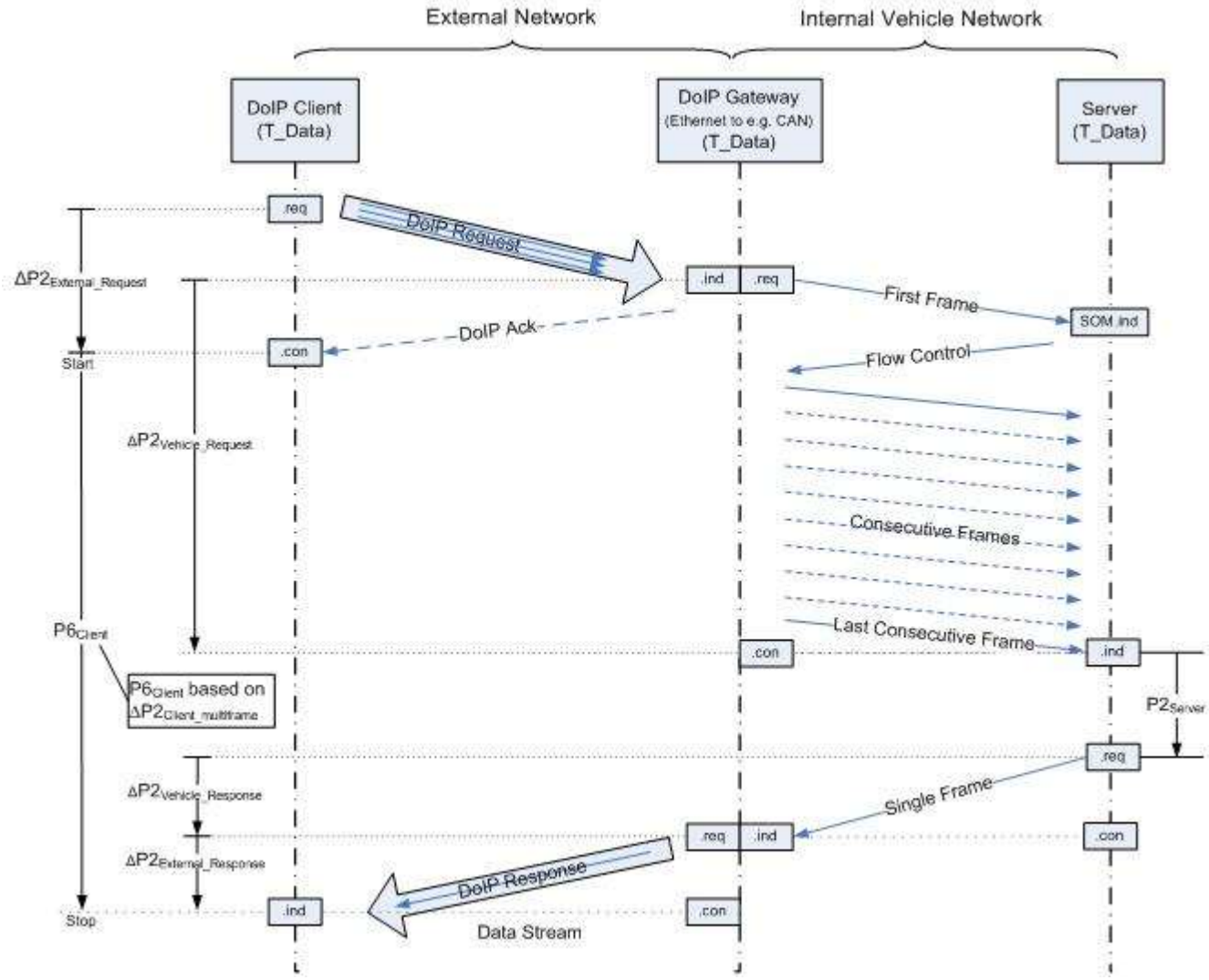


Figure - Multi frame request and single frame response, one server.

4.10.1.3.3.1.3 Single frame request and multi frame response, one server

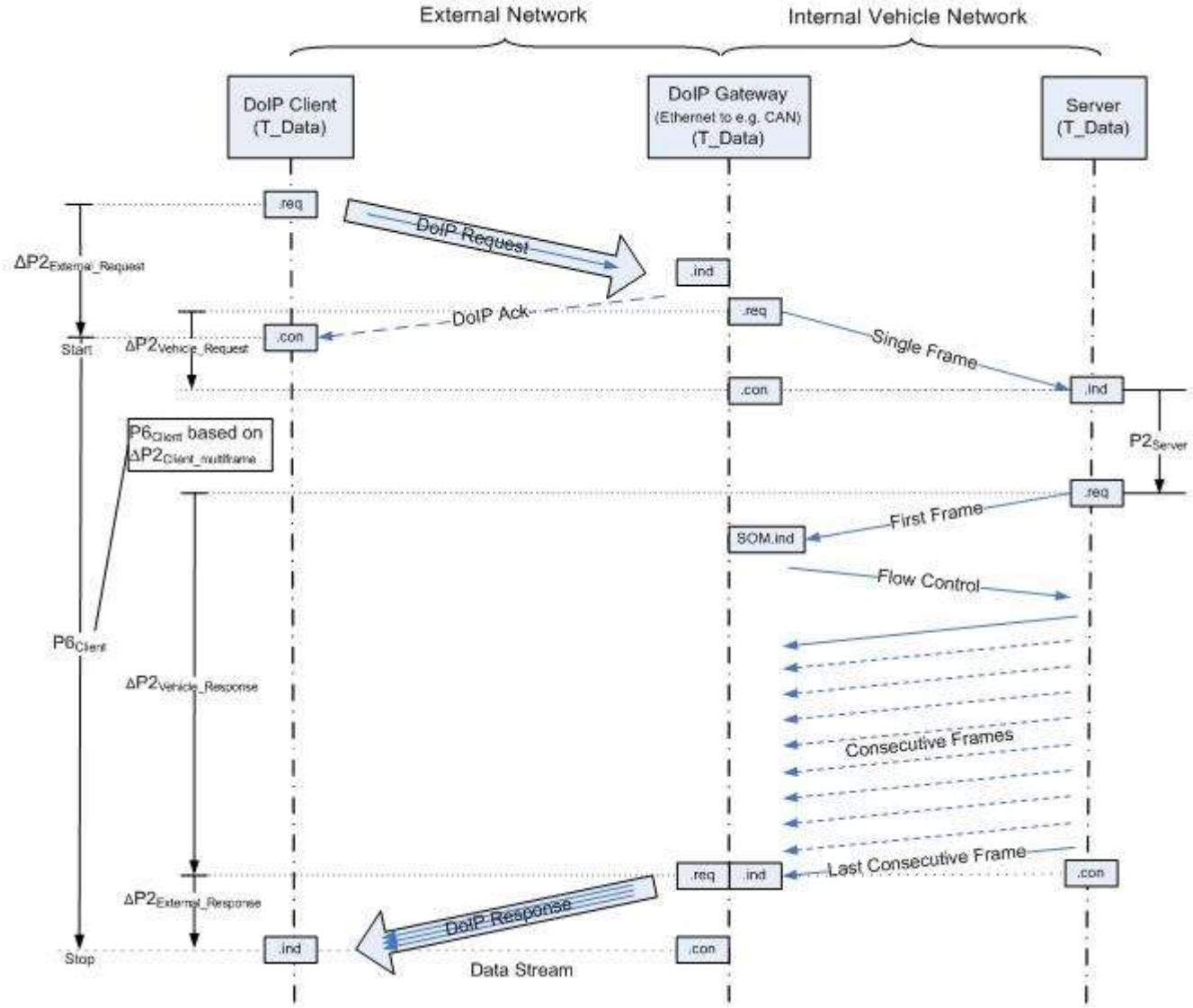


Figure - Single frame request and multi frame response, one server.

4.10.1.3.3.1.4 Single frame request and multi frame response, multiple servers

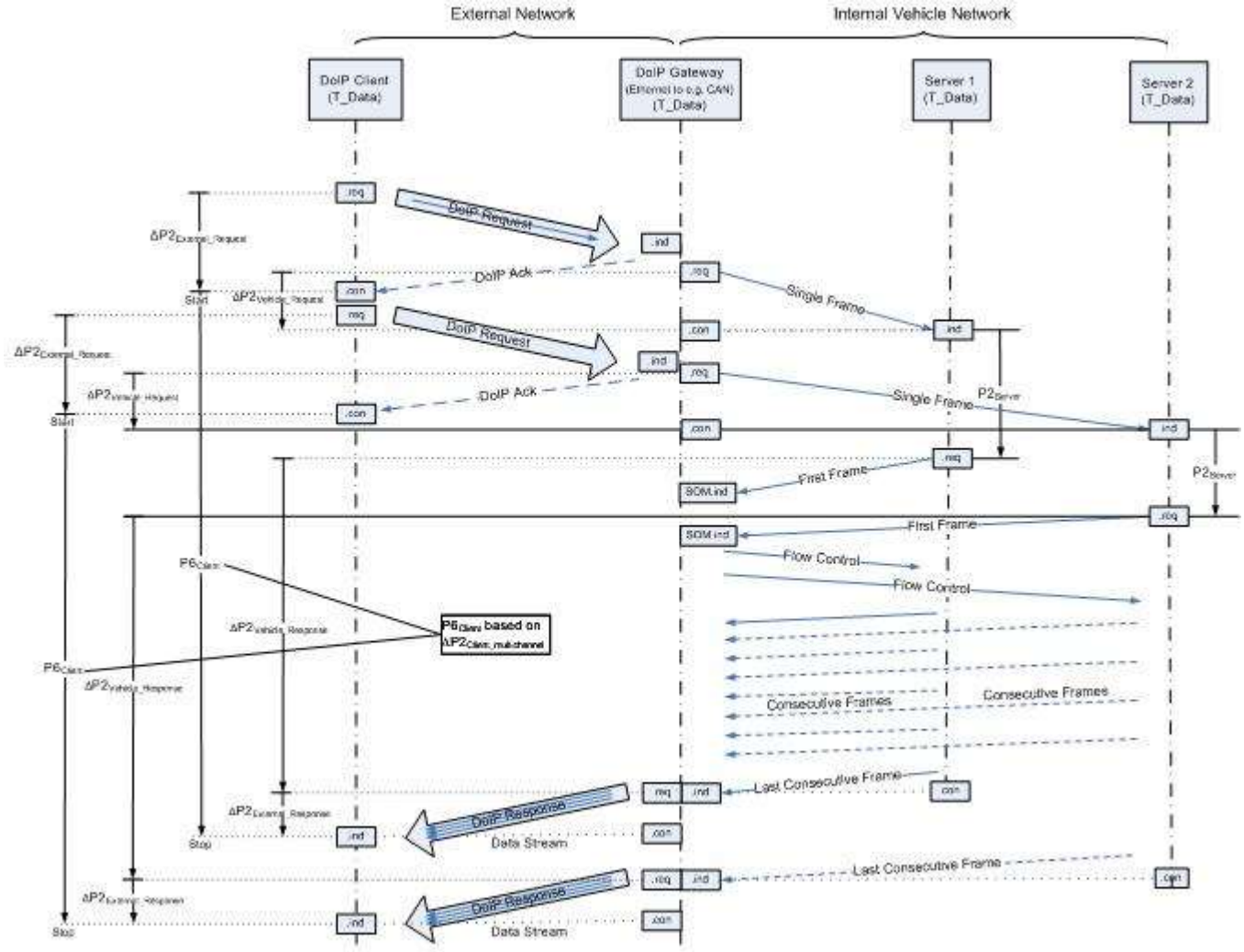


Figure - Single frame request and multi frame response, multiple servers.

4.10.2 UDS Services

4.10.2.1 UDS Services

•Overview

4.10.2.1.1 Overview

•Requisite Documents

4.10.2.1.2 Requisite Documents

This specification shall be followed and the requirements specified in this document are not negotiable.

In the case of any requirement conflict between this document and any of the referenced documents, the proposed solution to the conflict shall be approved by GRI Electrical Architecture.

Reference	Source	Document

[1]	Internal	UDS Data
[2]	Internal	UDS Session
[3]	Internal	Security access algorithm
[4]	Internal - CarCom	Global Master Reference Database
[5]	Internal	Data Compression and Encryption
[6]	Internal	Delta Encoding
[7]	Internal	General SWDL Specification
[8]	Internal	General Software Authentication
[9]	Internal	Authentication with PKI Certificate Exchange
[10]	External	ISO 14229-1
[11]	External	ISO 26021-1
[12]	External	ISO 26021-2
[13]	External	ISO 26262

•Definitions and Abbreviations

4.10.2.1.3 Definitions and Abbreviations

Definition	Description
Backbone Network	Internal network to connect domains on a high level
Domain ID	Part of the ECU Address, states which Domain the ECU is physical connected to.
Domain Network	Internal network for connection of nodes within a domain
Domain Network GW	Gateway between Backbone Network (or other internal Network as e.g. internal IP Network) and Domain Network
ECU ID	Part of the ECU Address, unique identifier for each ECU on a network.
ECU sleep mode	ECU sleep mode is defined as an ECU activity level when the ECU has no (functional) activity and has reached its absolute lowest power consumption, but still need to be powered e.g. to maintain long term memory or check for wake-up requests.
Entry condition	An entry condition for a specific diagnostic service is defined as any condition that that is necessary to meet for the diagnostic service to be successfully executed. This shall include ECU and vehicle operation conditions, surrounding environment conditions and other conditions that is necessary to be meet. Note that this includes also conditions that are initiated by other diagnostic services or obtained by the technician performs a certain actions.
Exit condition	An exit condition for a specific diagnostic service is defined as any condition that will cause active diagnostic functionality requested by the diagnostic service to abort.
External Network	Network for connection of an external tester. Used for software download and diagnostics
Implementer	Person responsible for the diagnostic implementation of the ECU. It may be the vehicle manufacturer or the ECU supplier diagnostic software designer.
Internal Network	Network for communication within the vehicle
Network ID	Part of the ECU Address, states which Network the ECU is physical connected to.
Network frame ID	Part of a data frame, transmitted on a network, which is used to identify the content of the frame, and/or the origin of the frame, and/or the destination of the frame.
Operation cycle	Refer to section *ECU ability to receive requests* for definition of an operation cycle and requirements on an ECU to start and stop an operation cycle.
Public ECU	An ECU addressable from a client/tester point of view.
Public ECU master	A public ECU responsible to act as a diagnostic server for one or more private ECUs.
Private ECU	An ECU not addressable from a client/tester point of view. Note: An ECU can partly be public and partly private i.e. some diagnostic services and

	diagnostic data supported by the ECU are addressable and some are not. Refer to the implementation specification for respective network (e.g. UDSonLIN, UDSonCAN, etc.) for which diagnostic services and diagnostic data supported by the ECU that may be or may not be addressable.
Responsible authority appointed by OEM	A person, appointed by OEM for a specific vehicle program, which is responsible for support, review and approval of the implementation of the diagnostic part of VDS4 in the vehicle.
Server	Software entity of an ECU which provides services for clients (testers)
Sub Network	Internal network for low level communication within a domain
Sub Network GW	Gateway between Domain Network and Sub Network
OEM tools	OEM tools include all tools used by OEM development, manufacturing and aftersales. These include, but are not limited to: • Development tools (used for e.g. analysis, test, calibration etc.) • Manufacturing tools • Aftersales tools • Databases • Embedded software components

Abbreviation	Description
ASIL	Automotive Safety Integrity Level. Defined by *Road vehicles - Functional safety, International Organization for Standardization* , UDS Services - External publications
D	Do not care. Implementation of the requirement is not required and not forbidden. The implementer may decide if to implement or not. If implemented it will not be verified by OEM and shall not be expected to be supported by OEM tools.
DTC	Diagnostic Trouble Code
ECU	Electronic Control Unit
M	Mandatory. Shall be supported by the ECU.
MC	Mandatory on Condition(s). The condition(s) is specified separately for each occurrence.
MPS	Mandatory for primary and secondary bootloader. Shall be supported by the ECU.
MS	Mandatory for secondary bootloader. Shall be supported by the ECU.
N	Not allowed. Shall not be supported by the ECU.
N/A	Not applicable
O	Optional. May be supported by the ECU. Whether or not the ECU shall support it is defined by *Road vehicles – Diagnostics systems – Diagnostic services*, UDS Services - External publications , if it is not defined by this reference , by the implementer (if not otherwise is explicitly specified by this document).
OS	Optional in secondary bootloader. Not allowed in primary bootloader. May be supported by the ECU. Whether or not the ECU shall support it is defined by *Road vehicles – Diagnostics systems – Diagnostic services*, UDS Services - External publications , if it is not defined by this reference , by the implementer (if not otherwise is explicitly specified by this document).
OPS	Optional for primary and secondary bootloader. May be supported by the ECU. Whether or not the ECU shall support it is defined by *Road vehicles – Diagnostics systems – Diagnostic services*, UDS Services - External publications , if it is not defined by this reference , by the implementer (if not otherwise is explicitly specified by this document).
PBL	Primary Bootloader
QM	Quality Management. Defined by *Road vehicles - Functional safety, International Organization for Standardization*, UDS Services - External publications
SA	Source Address
SBL	Secondary Bootloader
SWDL	Software Download
TA	Target Address
UDS	Unified Diagnostic Services

•Diagnostic message addressing

4.10.2.1.4 Diagnostic message addressing

The normal vehicle network configuration will be a multi level network, as described in the figure *Typical vehicle network topology* below , with both public and private ECUs on the different networks.
This chapter defines requirements for diagnostic message addressing for all public ECUs on any network. It is however not applicable to ECUs on networks where the network frame ID is less than 16 bits. For further information regarding addressing on networks with a network frame ID less than 16 bits, see the corresponding implementation specification.

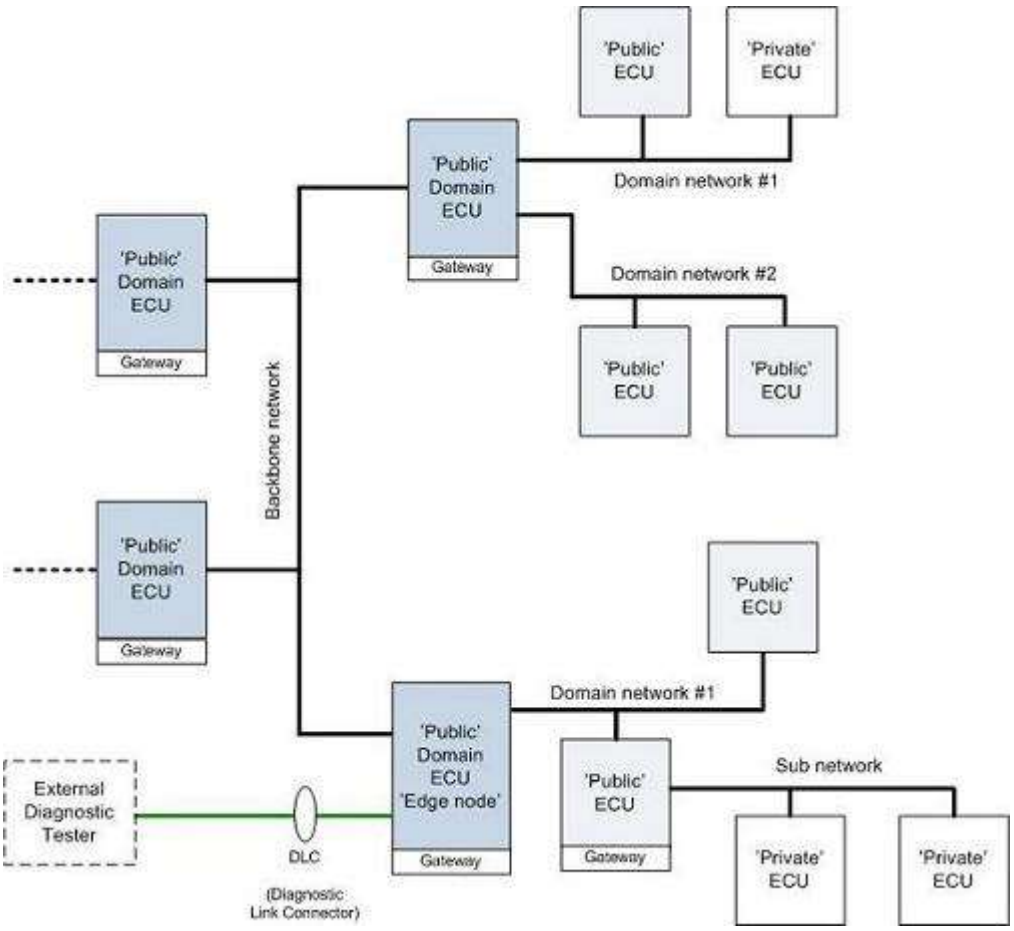


Figure - Typical vehicle network topology

A distributed server system shall be used where all public ECUs shall have their own server.
To make it possible for the tester to send a specific request to a public ECU on a network in the vehicle, each public ECU needs to have its own unique address. The address will be dependent on which network the ECU is located on and which domain the network belongs to.

The logical address range 0x1000 – 0x7FFF (Vehicle manufacturer specific) defined in *[13] ISO-13400-2 Road vehicles – Diagnostic Communication over Internet Protocol (DoIP) – Part 2: Network and Transport Layer Requirements and Services*, where the range 0x1000 to 0x1FFF is used as addresses for the public ECUs in the platforms using this specification.

The logical address range 0x0E80 – 0x0EFF (External vehicle manufacture / aftersales enhanced diagnostic test equipment) defined in *[13] ISO-13400-2 Road vehicles – Diagnostic Communication over Internet Protocol (DoIP) – Part 2: Network and Transport Layer Requirements and Services*, is used for the external diagnostic tester. Only 0x0E80 can be used, (using another address would affect internal routing tables).

4.10.2.1.4.1 Logical address format for diagnostic request and response

The addressing format used for diagnostic request (target address) and response (source address) is divided into three parts where the combination of Domain ID, Network ID and ECU ID is utilized, see the figure below. This addressing format gives the advantage of less complex gateway implementations.
Note: All diagnostic services defined in this document shall be supported for both physical and functional addressing unless otherwise is specified in this document.

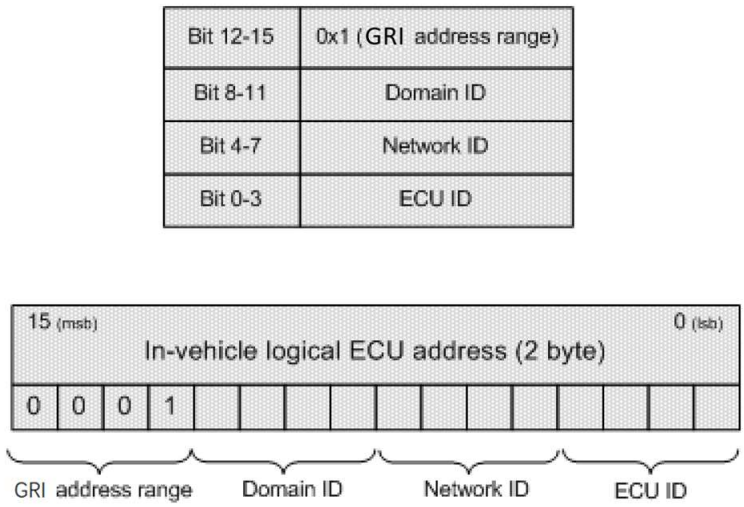


Figure – addressing format

The maximum number of ECUs per network range is limited to 15 since the 16'th (bit 0-3 equal to 0xF) is allocated for functional requests. If there is a need to have more than 15 ECUs within a network the network can be extended by assigning two or more logical Network IDs to the same physically network i.e. the ECU's will physically be on the same network but will have different Network IDs.

The maximum number of networks per domain is limited to 7 since Networks IDs (0x0 to 0x6) are assigned for communication with external diagnostic tester and Networks IDs (0x7 to 0xD) reserved for future communication with internal diagnostic tester. If there is a need to have more than 7 physical networks within a domain the domain can be extended by assigning two or more logical Domain IDs.

666002 v1 **Unique ECU address**

Version history:
(1):

Purpose: Define how the unique ECU ID is decided for each network.
Source:
Verification Method: Analysis
Legacy ID: 2336v1

Of the 16 bits that is used for TA and SA bit 0-3 shall define ECU ID, bit 4-7 shall define Network ID, bit 8-11 shall define Domain ID and bit 12-15 shall define the address range and shall be set to 0x1.

4.10.2.1.4.2 Physical logical address range

Each ECU shall have one vehicle unique physical logical address assigned within the ranges defined by the table below.
The system architect defining each ECU physical logical address needs to be aware of the parallel request

algorithm which the tester uses to distribute the diagnostic request messages during software download based on the logical address. The ECUs which takes the longest time to re-program shall have the lowest Domain/Network/ECU ID.
The system architect must also make sure that a Network ID equal to 0xD is not assigned to a CAN network including emissions related ECUs (this is necessary in order to avoid possible conflicts with request messages sent from an OBD scan tool).
The system architect shall check respective transport/network layer specification (e.g. CAN, LIN, IP, FlexRay, MOST150), regarding limitations when assigning a physical address to an ECU.

Table – Physical logical address range

(msb) Physical logical address range (lsb)				Comment
Address range Bit 15 – 12	**Domain ID Bit 11 – 8**	**Network ID Bit 7 – 4**	**ECU ID Bit 3 – 0**	
0x1	0x0 to 0xE	0x0 to 0x6	0x0 to 0xE	Reserved for external diagnostic tester communication
0x1	0x0 to 0xE	0x7 to 0xD	0x0 to 0xE	Reserved for future internal diagnostic tester communication

666003 v1 **Allocating ECU addresses**

Version history:
(1):

Purpose: Describe how to define server ECU address (i.e. TA for request and SA for response). The ECU address distribution method chosen shall:
- make as easy gateway implementation as possible
- support a method for load balancing certain types of system requests.

Source:
Verification Method: Analysis
Legacy ID: 2408v3

The ECU shall have one vehicle unique physical logical address assigned within the ranges defined by the table *Physical logical address range* .

Table – Physical logical address range

(msb) Physical logical address range (lsb)				Comment
Address range Bit 15 – 12	**Domain ID Bit 11 – 8**	**Network ID Bit 7 – 4**	**ECU ID Bit 3 – 0**	
0x1	0x0 to 0xE	0x0 to 0x6	0x0 to 0xE	Reserved for external diagnostic tester communication
0x1	0x0 to 0xE	0x7 to 0xD	0x0 to 0xE	Reserved for future internal diagnostic tester communication

666006 v1 **Assigning ECU ID address**

Version history:
(1):

Purpose: The scheduling method, for order that SWDL shall be performed, relies on that larger (in regards of SWDL time) ECU¿s shall be programmed first. The method shall be vehicle platform independent.
Source:
Verification Method: Analysis
Legacy ID: 2410v1

ECU ID’s range shall be assigned based on the time it takes to make a SWDL for the specific ECU. The ECU that

takes the longest time to perform a SWDL shall have the lowest ID and so on.

666004 v1 **Assigning domain ID address**

Version history:
(1):

Purpose: The scheduling method, for the order that SWDL shall be performed, relies on that larger (in regards of SWDL time) domains shall be programmed first. The method shall be vehicle platform independent.
Source:
Verification Method: Analysis
Legacy ID: 2409v1

Domain ID's range shall be assigned based on the time it takes to make a SWDL for the specific domain. The domain that takes the longest time to perform a SWDL shall have the lower ID's and so on.

666005 v1 **Assigning network ID address**

Version history:
(1):

Purpose: The scheduling method, for order that SWDL shall be performed, relies on that larger (in regards of SWDL time) networks shall be programmed first. The method shall be vehicle platform independent.
Source:
Verification Method: Analysis
Legacy ID: 2411v1

Network ID's range shall be assigned based on the time it takes to make a SWDL for the specific network. The network that takes the longest time to perform a SWDL shall have the lowest ID and so on.

4.10.2.1.4.3 Functional logical address range

It shall be possible via a functional logical address, to send a request to all (public) ECUs in the complete vehicle or a request to all (public) ECUs in the Zone. However, in programmingSession there may be ECUs on a specific network not supporting functional addressed requests where a response is expected.Refer to the implementation specification for respective network (e.g. UDSonLIN) for more information.
A functional request message must be unsegmented on all networks, i.e. the tester shall only send functional request messages that can be transported in a single frame on all networks (with public ECUs) in the vehicle or in specific zone. Eg: for a LIN network the largest functional request message is 6 bytes (payload data).

The ECU shall support the Geely functional address specified in the table below.

Table – Functional logical address range

(msb) Functional logical address range (lsb)				Comment
address range Bit 15 – 12	Zone ID Bit 11 – 8	Network ID Bit 7 – 4	ECU ID Bit 3 – 0	
0x1	0xF	0xF	0xF	All ECUs on all networks within all domains are functionally addressed
0x1	<Zone ID>	0xF	0xF	All ECUs on all networks within **specific zones** are functionally addressed.

If there are applicable regulatory requirements on the project, the ECU shall support the legislated functional address specified in the table below:

Table – Functional logical address range in use case-specific standard

(msb) Functional logical address range (lsb)	Comment

address range Bit 15 – 12	Zone ID Bit 11 – 8	Network ID Bit 7 – 4	ECU ID Bit 3 – 0	
0xE	0x0	0x0	0x0	All ZEV-relevant server(s)/ECU(s)
0xE	0x0	0x0	0x1	All ePTI-relevant server(s)/ECU(s)
0xE	0x0	0x0	0x2	All PCU-relevant server(s)/ECU(s) (PCU: pyrotechnic control unit)
0xE	0x0 to 0x3	0x0 to 0xF	0x3 to 0xF	Reserved

841703 v1 **Functional addressing in use case-specific standard**

Version history:
(1): 对于边缘节点和法规相关的DoIP ECU(s)，新增对法规相关的DoIP功能寻址要求。

Purpose: From a testing point of view it might be good to read information from several ECUs simultaneously in use case-specific standard.
Source:
Verification Method: Analysis AND Test
Legacy ID:

If there are applicable regulatory requirements in the project, the **Edge Node** and the **regulation related DoIP ECU(s)** shall support the legislated functional address specified in the table below:

Table – Functional logical address range in use case-specific standard

(msb) Functional logical address range (lsb)				Comment
address range Bit 15 – 12	Zone ID Bit 11 – 8	Network ID Bit 7 – 4	ECU ID Bit 3 – 0	
0xE	0x0	0x0	0x0	All ZEV-relevant server(s)/ECU(s)
0xE	0x0	0x0	0x1	All ePTI-relevant server(s)/ECU(s)
0xE	0x0	0x0	0x2	All PCU-relevant server(s)/ECU(s) (PCU: pyrotechnic control unit)
0xE	0x0 to 0x3	0x0 to 0xF	0x3 to 0xF	Reserved

666012 v3 **Functional addressing specific networks**

Version history:
(3): Add notes to facilitate receiving requirements for Ethernet ECUs that support DoIP.

Purpose: From a testing point of view it might be good to read information from several ECUs simultaneously for example when testing network management status.
Source: Section "Functional logical address range"
Verification Method: Test
Legacy ID: 2401v3

The ECU shall support the functional logical address as specified in table *Functional logical address range* .

Table – Functional logical address range

(msb) Functional logical address range (lsb)				Comment
address range Bit 15 – 12	Zone ID Bit 11 – 8	Network ID Bit 7 – 4	ECU ID Bit 3 – 0	
0x1	0xF	0xF	0xF	All ECUs on all networks within all domains are functionally addressed
0x1	<Zone ID>	0xF	0xF	All ECUs on all networks within **specific zones** are functionally addressed.

Note: Ethernet ECUs that support DoIP shall support the above functional logical address. The definition of Zone ID is defined in Function SWRS *ReqID 682784*

666013 v1 **Functional request shall be unsegmented on all networks**

Version history:
(1):

Purpose: A functional request must be unsegmented on all networks. For CAN network the largest value is 6 bytes.
Source: Section "Functional logical address range"
Verification Method: Analysis
Legacy ID: 2402v3

The tester shall not send larger functional request than 6 bytes in the data field.

4.10.2.1.4.4 Logical address range for periodic response messages

666024 v1 **Logical address range for periodic response messages**

Version history:
(1):

Purpose: To define the source address range used for periodical response messages.
Source:
Verification Method: Analysis
Legacy ID: 2404v2

The logical address range for periodic responses message type #2 (see *[10] ISO 14229-1 Road vehicles – Diagnostics systems – Diagnostic services*), used as source address for the ReadDataByPeriodicIdentifiers service (0x2A), is defined in the table below. The Domain ID shall represent the Domain which the ECU is logically connected to.

Table– Periodic response logical address range

(msb) Periodical response address range (lsb)		
address range Bit 15 – 12	Domain ID Bit 11 – 8	Bit 7 – 0
0x1 to 0x7	0x0	0xE0 to 0xFF
0x1 to 0x7	0x1	0xE0 to 0xFF
0x1 to 0x7	0x2	0xE0 to 0xFF
0x1 to 0x7	0x3	0xE0 to 0xFF
0x1 to 0x7	0x4	0xE0 to 0xFF
0x1 to 0x7	0x5	0xE0 to 0xFF
0x1 to 0x7	0x5	0xE0 to 0xFF
0x1 to 0x7	0x6	0xE0 to 0xFF
0x1 to 0x7	0x7	0xE0 to 0xFF
0x1 to 0x7	0x8	0xE0 to 0xFF
0x1 to 0x7	0x9	0xE0 to 0xFF
0x1 to 0x7	0xA	0xE0 to 0xFF
0x1 to 0x7	0xB	0xE0 to 0xFF
0x1 to 0x7	0xC	0xE0 to 0xFF
0x1 to 0x7	0xD	0xE0 to 0xFF
0x1 to 0x7	0xE	0xE0 to 0xFF

4.10.2.1.4.5 Logical address range for external diagnostic tester

The logical address range used for the external diagnostic tester is defined in the table below.

Table 1 – External vehicle-manufacturer-/aftermarket-enhanced diagnostic tester logical address range

(msb) External tester logical address range (lsb)				Comment
Bit 15 – 12	Bit 11 – 8	Bit 7 – 4	Bit 3 – 0	
0x0	0xE	0x8	0x0	External diagnostic tester
0x0	0xE	0x8	0x1 to 0xF	Reserved
0x0	0xE	0x9 to 0xF	0x0 to 0xF	Reserved

Table 2 - External legislated diagnostic tester logical address range

(msb) External tester logical address range (lsb)				Comment
Bit 15 – 12	Bit 11 – 8	Bit 7 – 4	Bit 3 – 0	
0x0	0xE	0x0	0x0	ZEV external diagnostic tester
0x0	0xE	0x0	0x1	ePTI external diagnostic tester
0x0	0xE	0x0	0x2	PDT external diagnostic tester (PDT: pyrotechnic device deployment tool)
0x0	0xE	0x0	0x3 to 0xF	Reserved
0x0	0xE	0x1 to 0x7	0x0 to 0xF	Reserved

666023 v2 **Logical address for external diagnostic tester**

Version history:
(2): Verification Method由Analysis修改为Test。

Purpose: To define the logical address used for the external diagnostic tester.
Source:
Verification Method: Test
Legacy ID: 2403v1

The external diagnostic tester shall be assigned the logical address 0x0E80.

841273 v2 **Logical address for external legislated diagnostic tester**

Version history:
(2): Verification Method由Analysis修改为Test。

Purpose: To define the logical address used for the external legislated diagnostic tester.
Source:
Verification Method: Test
Legacy ID:

If there are applicable regulatory requirements in the project, the external legislated diagnostic tester shall be assigned the logical address:

- 0x0E00, ZEV external diagnostic tester
- 0x0E01, ePTI external diagnostic tester
- 0x0E02, PDT external diagnostic tester

Note: It is applicable for Edge Node and regulation related DoIP ECU(s).

4.10.2.1.4.6 ECU ability to receive requests

666031 v1 ECU ability to receive a complete message

Version history:
(1):

Purpose: To ensure performance and predictable response times. This requirement does not apply to request and response messages gatewayed thru a gateway.
Source:
Verification Method: Analysis
Legacy ID: 2359v2

The ECU shall be able to receive a complete request message, receiving a queued request excluded (refer to *[2] UDS Session* for definition of queued request), without to halt the message, while sending of it is in progress, with the help of services in lower OSI-layer e.g network/transport layer.

666028 v1 ECU ability to receive requests

Version history:
(1):

Purpose: Define a time the ECU is allowed to be unavailable in regards of diagnostic communication when powering up and the time shall be short enough to not be a problem for manufacturing and aftersales.
Source: Section "ECU ability to receive requests"
Verification Method: Analysis
Legacy ID: 2360v4

As long as the ECU is within an operation cycle, the ECU shall all the time be able to receive diagnostic service requests and send a response (positive or negative) on the requests. Refer to section *Entry conditions* for requirements on when the ECU shall be able to execute the received requests.

1032594 v1 ECU diagnostic data handling

Version history:
(1): 新增需求，吉利定义的UDS数据（除法规数据外）仅允许被吉利定义的ECU地址（如，UDS诊断地址）访问。

Purpose: To prevent the core data used by OEMs from being obtained/operated through other means.
Source:
Verification Method: Analysis AND Test
Legacy ID:

UDS data (excluding regulatory data) defined by Geely is only allowed to be accessed by ECU address (e.g. UDS diagnostic addresses) defined by Geely.

666030 v3 ECU operation cycle

Version history:
(3): 对于非常电且无NM的ECU，Master ECU应该负责此类从ECU的上电/断电信号指示输入。从ECU以此作为驾驶循环。

Purpose: Define a time the ECU is allowed to be unavailable in regards of diagnostic communication when powering up and the time shall be short enough to not be a problem for manufacturing and aftersales.
Source: Section "ECU ability to receive requests"
Verification Method: Analysis

Legacy ID: 2361v2

The ECU shall start an operation cycle at the latest when the ECU has completed its start-up sequence, while start DTC tests within 2.5s, that time may run simultaneously with the requirement of the transition inhibit criteria defined in the section "Generic Test Run Criteria".

The ECU shall stop the operation cycle at the earliest when the ECU start to perform its shut-down sequence and at the latest when the ECU has completed its shut-down sequence and prior to the following ECU events:
1. Power down (the power supply is disconnected from the ECU).
2. Sleep mode (the ECU goes to sleep mode) or other equivalent modes defined by the implementer.

The detailed criteria for operation cycle start and stop are defined by the implementer for review and approval by Electrical Architecture.

Note: If the slave ECU is not powered by constant power and does not support NM mechanism, then the Master ECU shall be responsible for sending the power-off signal to slave ECU.

666029 v1 **ECU start up time**

Version history:
(1):

Purpose: Define a time the ECU is allowed to be unavailable in regards of diagnostic communication when powering up and the time shall be short enough to not be a problem for manufacturing and aftersales.
Source: Section "ECU ability to receive requests"
Verification Method: Test
Legacy ID: 2362v2

The ECU shall complete its start-up sequence within 2500 ms after an event that initiates a start-up sequence. The implementer defines the events that initiate a start-up sequence. However, minimum the following events shall initiate a startup sequence:
1. Power up (the power supply is connected to the ECU) or waking up from sleep (the ECU receives a request to wake up) or other equivalent modes defined by the implementer.
2. An ECU hard reset is triggered. In case of an ECU hard reset is triggered by a diagnostic service and if not otherwise is specified by the diagnostic service that trig the ECU hard reset, the ECU reset shall be triggered latest when the diagnostic service has been executed successfully. See reference *[10] ISO 14229 Road vehicles – Diagnostics systems – Diagnostic services for* definition of 'ECU hard reset'.

•Diagnostic service requirements

4.10.2.1.5 Diagnostic service requirements

Diagnostic services shall be supported by the ECU as specified in this section. Note that requirements on diagnostic services are defined for both diagnostic application software (i.e. software that execute services in other sessions than the programmingSession) and bootloader software (i.e. software that execute services in the programmingSession).

The diagnostic services required by this document are defined by *[10] ISO 14229-1*. This section specifies additions and restrictions that are compliant to *[10] ISO 14229-1.*

Additionally requirements on diagnostic services, which are network dependent, are specified in the implementation specification for the respective networks (e.g. internal specifications UDSonCAN, UDSonFR and UDSonIP).

The diagnostic services that shall be supported by the ECU in the safetySystemDiagnosticSession are specified by *[11] ISO 26021-1* and *[12] ISO 26021-2 "End-of-life activation of on-board pyrotechnic devices"*

The ECU is not allowed to support other diagnostic services without approval from GRI Electrical Architecture.

If the ECU implements safety requirements with an ASIL higher than QM, it shall be ensured that the diagnostic services supported by the ECU in all sessions except in programmingSession cannot violate any of those safety requirements, i.e. to demonstrate independence as required by *[13] ISO 26262*. Exception to this requirement is allowed only when approved by GRI Electrical Architecture and will require that the diagnostic services are developed according to *[13] ISO 26262* and the allocated ASIL of those safety requirement for which independence cannot be demonstrated.

853765 v4 General negative response code

Version history:
(4): Delete the signal description in NRC 0xF5.

Purpose: Define the general negative response code for every diagnostic service.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The general supported negative response code shall be within the following list:

Diagnostic Services	Allowed Negative Response Code
DiagnosticSessionControl (0x10)	0x12, 0x13, 0x22, 0xF0~0xFE
ECUReset (0x11)	0x12, 0x13, 0x22, 0xF0~0xFE
SecurityAccess (0x27)	0x12, 0x13, 0x24, 0x31, 0x35, 0x36, 0x37, 0x7F/0x7E
CommunicationControl (0x28)	0x12, 0x13, 0x22, 0x31, 0x7F/0x7E, 0xF0~0xFE
Authentication (0x29)	0x12, 0x13, 0x22, 0x24, 0x50, 0x51, 0x52, 0x53, 0x54, 0x55, 0x56, 0x57, 0x58, 0x5D
TesterPresent (0x3E)	0x12, 0x13
ControlDTCSetting (0x85)	0x12, 0x13, 0x22, 0x31, 0x7F/0x7E, 0xF0~0xFE
ReadDataByIdentifier (0x22)	0x13, 0x14, 0x22, 0x31, 0x33, 0x7F/0x7E
ReadMemoryByAddress (0x23)	0x13, 0x14, 0x31, 0x33, 0x7F/0x7E
ReadDataByPeriodicIdentifier (0x2A)	0x13, 0x31
DynamicallyDefineDataIdentifier (0x2C)	0x12, 0x13, 0x31, 0x33, 0x7F/0x7E
WriteDataByIdentifier (0x2E)	0x13, 0x22, 0x31, 0x33, 0x72, 0x7F/0x7E
WriteMemoryByAddress (0x3D)	0x13, 0x22, 0x31, 0x33, 0x72, 0x7F/0x7E
ClearDiagnosticInformation (0x14)	0x13, 0x22, 0x31, 0x72
ReadDTCInformation (0x19)	0x12, 0x13, 0x31
InputOutputControlByIdentifier (0x2F)	0x13, 0x22, 0x31, 0x33, 0x7F/0x7E
RoutineControl (0x31)	0x12, 0x13, 0x22, 0x24, 0x31, 0x33, 0x72
RequestDownload (0x34)	0x13, 0x22, 0x31, 0x33, 0x70
RequestUpload (0x35)	0x13, 0x22, 0x31, 0x33, 0x70
TransferData (0x36)	0x13, 0x24, 0x31, 0x71, 0x72, 0x73, (0x92), (0x93)
RequestTransferExit (0x37)	0x13, 0x24, 0x31, 0x72
RequestFileTransfer (0x38)	0x13, 0x22, 0x24, 0x31, 0x33, 0x70

For the negative response code NRC 0x22 which conditions not correct, it is not possible to identify which condition is not met. Therefore, utilize negative response code - VehicleManufactureSpecificConditionNotCorrect range to match the conditions with each negative response code one by one.

Negateive Response Code (NRC) for VehicleManufactureSpecificConditionNotCorrect	Descrpition
0xF0	Condition "Vehicle speed < 3km/h and VehSpdLgtQf is equal to 2 or 3" is not met.
0xF1	Condition "Main propulsion system is not active which means that the signal UsgModSts is not equal to "Driving"" is not met.

0xF2	Condition "The power supply voltage of ECU for SWDL" is not met.
0xF3	Condition "Main propulsion system is not active which means that the signal HvSysRlySts is equal to as "HvActvnSts_Open"" is not met.
0xF4	Condition "High voltage is less than 60V (The high voltage will be detected by MGM/IEM/IGM itself)" is not met.
0xF5	Condition "Shifter is in "P"or "N" position" is not met.
0xF6	Condition "PowerPackStatus is under flashable status which means that the signal EngSt1WdSts is equal to as any one of "EngSt1_Ini", "EngSt1_Awake" and "EngSt1_Rdy" is not met.
0xF7 ~ 0xFE	Undefined

Note1: NRC 0x78 is in general supported by each diagnostic service, as not otherwise stated in the data link specific implementation document, therefore it is not listed in the list of applicable response codes of the diagnostic services.
Note2: NRC 0x34 is in general supported by each diagnostic service for ECU supporting Authentication (0x29) service, therefore it is not listed in the list of applicable response codes of the diagnostic services.
Note3: Regarding to negative response code for VehicleManufactureSpecificConditionNotCorrect, the smaller the value, the higher the priority.
Note4: Which specific negateive response code to use shall follow specific requirements.

665436 v2 General service requirements

Version history:
(2): 补充否定响应码NRC10和NRC21通用情况下不允许支持。

Purpose: Define standardized diagnostic services.
Source:
Verification Method: Analysis AND Test
Legacy ID: 2357v3;2356v2;2358v2;2407v4

The following general service requirements shall be supported:

- The ECU shall be compliant to *[10] ISO 14229-1*: Road vehicles – Diagnostics systems – Diagnostic services.
- The ECU is not allowed to support other diagnostic services than specified in section *Diagnostic service requirements* of the applicable implementation specification for the respective networks and by the *[12] ISO 26021-2* "End-of-life activation of on-board pyrotechnic devices" unless approved by GRI Electrical Architecture.
- If the ECU implements safety requirements with an ASIL higher than QM, it shall be ensured that the diagnostic services supported by the ECU in all sessions except in programmingSession cannot violate any of those safety requirements, i.e. to demonstrate independence as required by *[13] ISO 26262*. Exception to this requirement is allowed only when approved by GRI Electrical Architecture and will require that the diagnostic services are developed according to *[13] ISO 26262* and the allocated ASIL of those safety requirement for which independence cannot be demonstrated.
- The negative response code generalReject (0x10) sent by negative responses on diagnostic services requests is not allowed.
- The negative response code busyRepeatRequest (0x21) sent by negative responses on diagnostic services requests is not allowed unless otherwise is specified by this document or in the implementation specification for the respective networks (e.g. UDSonCAN, UDSonFR and UDSonIP).
- Negative response code other than GeneralReject (10) and BusyRepeatRequest (21) sent by negative responses on diagnostic services requests is allowed as specified by *[10] ISO 14229-1* unless otherwise is specified by this document or in the implementation specification for the respective networks (e.g. UDSonCAN, UDSonFR and UDSonIP).

666414 v1 Response on the diagnostic service requests

Version history:

(1):

Purpose: Define ECU response time and special response requirements.
Source:
Verification Method: Test
Legacy ID: 2436v2;2412v3

Exceptions to P4 $_{Server_max}$ are not allowed unless approved By GRI Electrical Architecture.

If a diagnostic service is allowed to make a reset (affect the ECU's ability to execute non diagnostic tasks) , the response shall be sent before performing the reset.

666415 v1 **Response to accessing non-volatile memory**

Version history:
(1):

Purpose: Data written or cleared to an ECU shall be stored in long term memory immediately by the ECU so that it can be read out what is programmed to the ECUs.
Source:
Verification Method: Test
Legacy ID: 2413v5

A diagnostics service that needs to write data to non-volatile memory is completed only after the write operation to the non-volatile memory has been completed. This means that:
• For service RoutineControl, in case a routine needs to store data in non-volatile memory, the routine shall only respond with status = routineCompleted after the data has been written to non-volatile memory.
• For all other write services (WriteDataByIdentifier, WriteMemoryByAddress, ClearDiagnosticInformation) the positive response to the diagnostic request shall only be send after the data has been written to non-volatile memory.

4.10.2.1.5.1 Diagnostic services

Abbreviations in the table Requirements on diagnostic services that shall be supported by the ECU:

MC=If the ECU supports 29 services, parts of diagnostic request is required to be protected by authenticaion.
MC1= Mandatory if the ECU is not emission related. Mandatory if the ECU is emission related and support "calibration DTCs" (one or more). The emission related ECU shall in this case support the groupOfDTC set to value of a "calibration DTC" that is supported by the ECU.
If the ECU is emission related the ECU is allowed to support the groupOfDTC set to value of a non-emission DTCs other than a "calibration DTC".
If the ECU is emission related the ECU is not allowed to support the groupOfDTC set to value of an emission related DTC.
A "calibration DTC" is a non-emission related DTC that indicates that a specific action (e.g. request a diagnostic service for initiating or calibrating a function controlled by the ECU) that must be performed as a part of the car manufacturing process has not yet been performed and that shall be cleared (by the diagnostic service ClearDiagnosticInformation with the data parameter groupOfDTC set to a value of the "calibration DTC") as soon as the action has been completed.
MC2= Mandatory if the ECU is emission related. Not allowed if the ECU is not emission related.
MC3= Mandatory if the dataRecord consists of more than one parameter (i.e. the dataRecord is packeted or bitmapped). Not allowed if the dataRecord consists of one parameter.
MC4= Mandatory if *[1] UDS Data* specifies that the value of a data record, pointed out by a dataIdentifier, shall be possible to write (by diagnostic service specified by this document).
Otherwise, the service WriteDataByIdentifier may be supported by the ECU in one, some or all sessions. When in programming session WriteDataByIdentifier shall only be supported in the secondary bootloader.
MC5= Mandatory if the ECU is emission related and support permanent DTCs.

4.10.2.1.5.1.1 Diagnostic and Communication Unit

Service Name(Hex)	Sub-Function Name(Hex)	Session			Addressing		Comment
		Default	Extended	Programming	Physical	Functional	
DiagnosticSessionControl (0x10)		M	M	MPS	M	M	
ECUReset (0x11)		M	M	MPS	M	M	
SecurityAccess(0x27)		N	O	MPS	M	O	
CommunicationControl (0x28)		N	M	N	M	M	Note2
Authentication (0x29)		MC	MC	MCPS	M	O	Note1
	DeAuthenticate(0x00)	M	M	MPS	M	M	
	VerifyCertificateUnidirectional(0x01)	M	M	MPS	M	O	
	VerifyCertificateBidirection(0x02)	O	O	OPS	M	O	
	ProofOfOwnership(0x03)	M	M	MPS	M	O	
	TransmitCertificate(0x04)	O	O	OPS	M	O	
	AuthenticationConfiguration(0x08)	M	M	MPS	M	M	
TesterPresent (0x3E)		M	M	MPS	M	M	
ControlDTCSetting (0x85)		N	M	N	M	M	

Note:
1. The ECU that supports authentication with PKI Certificate Exchange need to support bidirectional authentication or unidirectional authentication. The Authentication service shall be implemented as defined in *[10] ISO 14229-1:2020* (Third Edition).
2. For LIN slave ECU, CommunicationControl (0x28) service is not manadatory.

663982 v5 **Authentication (0x29)**

Version history:
(5): 1、添加诊断服务寻址的需求便于高效开发。

Purpose: The purpose of this service is to provide a means for the client to prove its identity, allowed it to access data and/or diagnostic services, which have restricted access for.

Source:

Verification Method: Analysis AND Test

Legacy ID: 538608v1;639216v1;538611v1;538613v1;538732v1;538738v1;655065v1;652545v1;538615v1;538617v1;538637v1;538704v1;538737v1;539067v1;654131v1;654132v1;654129v1;654130v1;654135v1;654137v1;654134v1;655067v1;539615v1;539616v1;539640v1;539643v1;655070v1;654162v1;655073v1;539611v1;539645v1;539653v1;539654v1;539655v1;654141v1;640033v1;637888v1;637890v1;637891v1;637900v1;653843v1;655562v1;656703v1

Service Name (Hex)	Sub-function Name(Hex)	Data Paremeters Name	Hex	Session Default	Extended	Programming	Addressing Physical	Functional	Authentication access 29	Security access 27	P4 Server
Authentication(	DeAuthen	N/A	N/A	M	M	MPS	M	M	N/A	N	10000

0x29)	ticate(00, 80)										ms
	VerifyCertificateUnidirectional(01, 81)	CommunicationConfiguration	00	M	M	MPS	M	O	N/A	N	
			11	O	O	O					
			01-10, 12-FF	O	O	O					
		LengthOfCertificateClient		M	M	MPS					
		CertificateClient		M	M	MPS					
		LengthOfChallengeClient		M	M	MPS					
		ChallengeClient		C	C	C					
	VerifyCertificateBidirection(02, 82)	CommunicationConfiguration	00	M	M	MPS	M	O	N/A	N	
			11	O	O	OPS					
			01-10, 12-FF	O	O	OPS					
		LengthOfCertificateClienta		M	M	M					
		CertificateClient		M	M	M					
		LengthOfChallengeClient		M	M	M					
		ChallengeClient		M	M	M					
	ProofOfOwnership(03, 83)	LengthOfProofOfOwnershipClient		M	M	MPS	M	O	N/A	N	
		ProofOfOwnershipClient		M	M	MPS					
		LengthOfEphermeralPublicKeyClient	0000	M	M	MPS					
	TransmitCertificate(04, 84)	CertificateEvaluationId		M	M	MPS	M	O	N/A	N	
		LengthOfCertificateData		M	M	MPS					
		CertificateData		M	M	MPS					
	AuthenticationConfiguration(08, 88)	N/A	N/A	M	M	MPS	M	M	N/A	N	

Note:

- The verifyCertificateUnidirectional 0x01, verifyCertificateBidirectional 0x02 and proofOfOwnership 0x03 shall use the OEM standardized authentication algorithm specified in reference *[9] Authentication with PKI Certificate Exchange*.
- CommunicationConfiguration 0x00 : no secure diagnostic communication.
- CommunicationConfiguration 0x11 - Default key exchange method and default encryption method as specified in *[9] Authentication with PKI Certificate Exchange*.(The high nibble specifies the "keyExchangeMethod " and the low nibble specifies the "encryptingMethod", as specified in *[9] Authentication with PKI Certificate Exchange*.)

- The lengthOfChallengeClient represents the length of client challenge, according to the authentication method *[9] Authentication with PKI Certificate Exchange*.
- The service Authentication - proofOfOwnership with lengthOfEphemeralPublicKeyClient of this parameter shall be 0x0000 indicating that no private/public key pair is present if the communicationConfiguration is set to 0x00, otherwise length information of ephemeralPublicKeyClient shall be represented, e.g. communicationConfiguration 0x11.
- The service Authentication - proofOfOwnership with ephemeralPublicKeyClient is equal to 0x0000, then the parameter shall not be transmitted.The parameter shall be according to the authentication method *[9] Authentication with PKI Certificate Exchange*.
- The service Authentication - transmitCertificate with certificateEvaluationId If subsequent diagnostic requests with the same evaluationTypeId is received by server successfully, it will overwrite the certificate data of the previous requests.
- The authenticationReturnParameter shall be returned depending on the client authentication service request, according to *[10] ISO 14229*.

Entry conditions:

- Entry conditions for service Authentication(0x29) are not allowed.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- The service Authentication - verifyCertificateUnidirectional or verifyCertificateBidirectional response with lengthOfCertificateServer represents the length of server challenge, according to the authentication method *[9] Authentication with PKI Certificate Exchange*.
- The response with lengthOfEphemeralPublicKeyServer of this parameter shall be 0x0000 indicating that no private/public key pair is present if the communicationConfiguration is set to 0x00, otherwise length information of ephemeralPublicKeyServer shall be represented, e.g. communicationConfiguration 0x11.
- The service Authentication - verifyCertificateUnidirectional or verifyCertificateBidirectional response with ephemeralPublicKeyServer If lengthOfEphemeralPublicKeyServer is equal to 0x0000, then the parameter shall not be transmitted.
- If the ECU has received a request message for the service Authentication in delay time, this criteria for the request Authentication is also considered to be not met, the ECU shall send a negative response with response code 0x22 - conditionsNotCorrect.
- The service authentication (0x29) shall not affect the ECU's ability to execute non diagnostic tasks.
- ~~Define vehicle manufacturer specific authentication return parameter for the response of subfunction - AuthenticationConfiguration. If only support subfunction - VerifyCertificateUnidirectional, return value 0xA1 (APCE with Unidirectional) shall be responsed when request AuthenticationConfiguration. If only support subfunction - VerifyCertificateBidirection, return value 0xA2 (APCE with Bidirectional) shall be responsed when request AuthenticationConfiguration. If support both direction, return value 0x02 shall be responsed.~~

663980 v4 **CommunicationControl (0x28)**

Version history:
(4): 1、添加诊断服务寻址的需求便于高效开发。2、调整29认证访问的支持情况。3、调整P4server_max为200ms。4、考虑通讯控制的异步操作，返回肯定响应后的100ms内要执行完通讯控制。4、进入条件不满足使用的NRC参考ReqID 853765。

Purpose: The purpose of this service is to switch on/off the transmission and /or the reception of certain messages of server.

Source:

Verification Method: Analysis AND Test

Legacy ID: 239742v1;239744v1;239745v1;239746v1;239747v1;239749v1;239750v1;239751v1;239755v1;239757v1;239759v1;2400v1;239771v1;652805v1

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
Communication Control(0x28)	EnableRxAndTx(00,80)	NormalCommunicationMessages	01	N	M	N	M	M	M	N	200ms
		NetworkManagement CommunicationMessages (Note1)	02	N	C	N					
		NetworkManagement CommunicationMessages, NormalCommunicationMessages (Note2)	03	N	M	N					
	EnableRxAndDisableTx(01,81)	NormalCommunicationMessages	01	N	M	N	M	M	M	N	
		NetworkManagement CommunicationMessages (Note1)	02	N	C	N					
		NetworkManagement CommunicationMessages, NormalCommunicationMessages (Note2)	03	N	M	N					
	DisableRxAndEnableTx(02.82)	NormalCommunicationMessages	01	N	M	N	M	M	M	N	
		NetworkManagement CommunicationMessages (Note1)	02	N	C	N					
		NetworkManagement CommunicationMessages, NormalCommunicationMessages (Note2)	03	N	M	N					
	DisableRxAndTx(03,83)	NormalCommunicationMessages	01	N	M	N	M	M	M	N	
		NetworkManagement CommunicationMessages (Note1)	02	N	C	N					
		NetworkManagement CommunicationMessages, NormalCommunicationMessages (Note2)	03	N	M	N					

Note:

- 1. The ECU implements network management function must implement the CommunicationType 02.
- 2. For the ECUs without network management function implement the CommunicationType 03, the messages are same as CommunicationType 01.

Entry conditions:

- Entry conditions for service CommunicationControl (0x28) are allowed only if approved by GRI Electrical Architecture.

If the ECU implement safety requirements with an ASIL higher than QM (see *[13] ISO 26262)* it shall, in all situations when diagnostic services may violate any of those safety requirements, reject the critical diagnostic service requests.But that require an approval by GRI Electrical Architecture.

When at least one of these conditions is not met, the ECU shall reject the request message with a specific negative response code defined in *ReqID 853765*.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- The service CommunicationControl (0x28) is allowed to affect the ECU's ability to execute non diagnostic tasks. The service is only allowed to affect execution of the non-diagnostic tasks during the execution of the diagnostic service. After the diagnostic service is completed any effect on the non-diagnostic tasks is not allowed anymore (normal operational functionality resumes). The service is not allowed to reset the ECU.
- The function of Diasble Rx or Disable Tx shall apply for all channels, e.g. CAN, Ethernet, LIN. Exception (e.g. from functional requirements) to this item is allowed only when approved by GRI Basetech Team, and fill comments in SWRT document.
- For ECU connected multiple networks, bit 4 to 7 of communicationType parameter will be applied and subnet Number will be defined according to project.
- For ECU that support Partial Network Cluster , application messages may be suppressed when a request is received to disable Rx with communicationType - NetworkManagementCommunicationMessages, which is acceptable.
- The communication control shall be completed within 100ms after positive response returned.

663985 v3 ControlDTCSetting (0x85)

Version history:
(3): 1、添加诊断服务寻址的需求便于高效开发。2、调整29认证访问的支持情况。3、调整P4server_max为200ms。4、进入条件不满足使用的NRC参考ReqID 853765。

Purpose: The service shall be used by a client to stop or resume the updating of DTC status bits in server.

Source:

Verification Method: Analysis AND Test

Legacy ID: 239752v1;239753v1;239754v14239756v1;239758v1;239760v1;2400v3;239772v1;652806v1

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
ControlDTCSetting (0x85)	DTC setting - on(01,81)	N/A	N/A	N	M	N	M	M	M	N	200ms
	DTC setting - off(02,82)	N/A	N/A	N	M	N	M	M	M	N	

Entry conditions:

- Entry conditions for service ControlDTCSetting(0x85) are allowed only if approved by OEM Electrical Architecture.

If the ECU implement safety requirements with an ASIL higher than QM (see *[13] ISO 26262)* it shall, in all situations when diagnostic services may violate any of those safety requirements, reject the critical diagnostic service requests.But that require an approval by GRI Electrical Architecture.

When at least one of these conditions is not met, the ECU shall reject the request message with a specific negative response code defined in *ReqID 853765*.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- The service ControlDTCSetting (0x85)shall not affect the ECU's ability to execute non diagnostic tasks.
- When controlDTCSetting - off is requested, the fault detection shall be frozen, i.e. AUTOSAR DEM

DemDebounceBehavior=FREEZE.

663781 v2 DiagnosticSessionControl (0x10)

Version history:
(2): 1、添加诊断服务寻址的需求便于高效开发。2、调整29认证访问的支持情况。3、补充需求，仅进入扩展会话不允许功能降级，除非特定的诊断服务操作被执行。4、进入条件更新，以及条件不满足使用的NRC参考ReqID 853765。

Purpose: The DiagnosticSessionControl service is used to enable different diagnostic sessions in the services.

Source:

Verification Method: Analysis AND Test

Legacy ID: 2495v2;2496v2;2498v1;2497v1;2499v1;2416v1;2365v2;2364v3;2398v3;2399v1;2437v2;63789 2v1;

Service	Sub-function	Data Paremeters		Session			Adressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
DiagnosticSessionControl (0x10)	DefaultSession(01,81)	N/A	N/A	M	M	MPS	M	M	N	N	P2 Server_max
	ProgrammingSession(02,82)	N/A	N/A	M	M	MPS	M	M	M	N	
	ExtendedDiagnosticSession(03,83)	N/A	N/A	M	M	N	M	M	M	N	

Note:

- Refer to section *Additional session requirements* for additional session requirements.

Entry conditions: Only allowed when changing from non-programming session to programming session

- Vehicle speed is not over 3km/h;
- Main propulsion is not active which means that the UsageMode is not equal to “Driving”.

Note:

The implementer shall ensure that the vehicle is in a defined safe state based on the allocated functions, so it is not limited to the above two points.
In an impaired vehicle or in a stand alone scenario if the vehicle signal(s) used in the evaluation of the condition e.g. speed and/or "main propulsion system not active" shall the safety mechanism not prevent the ECU to change to programmingSession to allow SWDL.

If the ECU implement safety requirements with an ASIL higher than QM (see *[13] ISO 26262*) it shall, in all situations when diagnostic services may violate any of those safety requirements, reject the critical diagnostic service requests.But that require an approval by GRI Electrical Architecture.
When at least one of these conditions is not met, the ECU shall reject the request message with a specific negative response code defined in *ReqID 853765*.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- The ECU shall not use the service diagnosticSessionControl to alter the values of $P2_{Server_max}$ and $P2^*_{Server_max}$ from the values specified in *[2] UDS Session*. The response message data parameter SessionParameterRecord shall always contain these fixed values when the Session is requested.
- Only allowed transition from and to programmingSession (Note:Excluding programmingSession to programmingSession) is allowed to affect the ECU’s ability to execute non diagnostic tasks. The service is only allowed to affect execution of the non-diagnostic tasks during the execution of the diagnostic service. After the diagnostic service is completed any effect on the non-diagnostic tasks is not allowed anymore (normal operational functionality resumes).

- It is not allowed to peform function degradation when only extended session is entered, e.g. cause warning indicator on. If the ECU implement safety requirements with an ASIL higher than QM (see *[13] ISO 26262)* it shall, in all situations when specific diagnostic services may violate any of those safety requirements, turn on the warning indicator or reject the critical diagnostic service requests, e.g. CommunicationControl, InputOutputControl. But that require an approval by GRI Safety Team.

663978 v3 ECUReset (0x11)

Version history:
(3): 1、添加诊断服务寻址的需求便于高效开发。2、调整29认证访问的支持情况。3、增加执行复位至少的前提条件。4、条件不满足使用的NRC参考ReqID 853765。

Purpose: The ECUReset service is used by the client to request a server reset.
Source:
Verification Method: Analysis AND Test
Legacy ID: 2500V4;2501v1;2418v1;2367v2;2397v2;2440v2;637894v1

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
ECUReset (0x11)	HardReset(01,81)	N/A	N/A	M	M	MPS	M	M	M	N	P2 Server_max

Entry conditions:

Entry conditions for service ECUReset (0x11) are allowed only if approved by GRI Electrical Architecture. At least the following conditions shall be met:

- Vehicle speed is not over 3km/h;
- Main propulsion is not active which means that the UsageMode is not equal to “Driving”.

Note:

The implementer shall ensure that the vehicle is in a defined safe state based on the allocated functions, so it is not limited to the above two points.
In an impaired vehicle or in a stand alone scenario if the vehicle signal(s) used in the evaluation of the condition e.g. speed and/or "main propulsion system not active" shall the safety mechanism not prevent the ECU to reset
If the ECU implement safety requirements with an ASIL higher than QM (see *[13] ISO 26262)* it shall, in all situations when diagnostic services may violate any of those safety requirements, reject the critical diagnostic service requests.But that require an approval by GRI Electrical Architecture.
When at least one of these conditions is not met, the ECU shall reject the request message with a specific negative response code defined in *ReqID 853765*.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- The service ECUReset (0x11) is allowed to affect the ECU’s ability to execute non diagnostic tasks. The service is only allowed to affect execution of the non-diagnostic tasks during the execution of the diagnostic service. After the diagnostic service is completed any effect on the non-diagnostic tasks is not allowed anymore (normal operational functionality resumes).
- The reset shall be completed within 2 seconds after the positive response is returned or P2server_max is exceeded (no response due to SPRMIB being set), i.e. ECUs is capable of communication.

663979 v3 SecurityAccess (0x27)

Version history:

(3): 1、添加诊断服务寻址的需求便于高效开发。2、调整29认证访问的支持情况。3、调整P4server_max为200ms。

Purpose: The purpose of this service is to provide a means to access data and/or diagnostic services, which have restricted access for security,emissions,or safety reasons.

Source:

Verification Method: Analysis AND Test

Legacy ID: 2502v2;2506v4;2507v4;2511v2:2503v3;2504v2;2505v2;2508v2;2509v2;2510v3;2431v1;2375v2;2392v1;2400v3;637931v1;

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
SecurityAccess(0x27)	RequestSeed(01,81)	N/A	N/A	N	N	MPS	M	O	M	N/A	200ms
	SendKey(02,82)	XXX Byte Key		N	N	MPS	M	O	M	N/A	
	RequestSeed(03-41,83-C1)	N/A	N/A	N	O	N	M	O	U	N/A	
	SendKey(04-42,84-C2)	XXX Byte Key		N	O	N	M	O	U	N/A	
	RequestSeed(61-7E,E1-FE)	N/A	N/A	N	O	N	M	O	U	N/A	
	SendKey(62-7F,E2-FF)	XXX Byte Key		N	O	N	M	O	U	N/A	

Note:

- The requestSeed range 0x01-0x41 and corresponding sendKey range 0x02-0x42 shall use the OEM standardized SecurityAccess algorithm specified in reference *[3] Security access algorithm*.
- The requestSeed range 0x61-0x7E and corresponding sendKey range 0x62-0x7F shall use a SecurityAccess algorithm that is provided by the implementer. The number of bytes of the data parameter securityKey is specified by the implementer. In order to use this range, an approval by GRI Electrical Architecture is required. Note that GRI tools are not required to support the range.

Entry conditions:

- Entry conditions for service SecurityAccess(0x27) are not allowed.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- Whenever the ECU has responded with a seed on a requestSeed request, the seed shall be valid until:
  -Any new securityAccess request is received (except when the new request has a sub-function that is not supported by the ECU).
  -A valid request with sub-function sendKey which first unlocks the SecurityAccess level and then invalidates the seed.
  -ECU session transition regardless if it is to the same or another session.
- The service securityAccess (0x27) shall not affect the ECU's ability to execute non diagnostic tasks.

663984 v2 **TesterPresent (0x3E)**

Version history:
(2): 1、添加诊断服务寻址的需求便于高效开发。

Purpose: The service is used to indicate to a server that a client is still connected to the vehicle and that certain diagnostic services and/or communication that have been previously activated are to remain active.

Source:

Verification Method: Analysis AND Test

Legacy ID: 2512v2;2432v1;2376v2;2393v1;2400v3;2453v2;652769v1

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
TesterPresent(0x3E)	zeroSubFunction(00,80)	N/A	N/A	M	M	MPS	M	M	N	N	P2 Server_max

Entry conditions:

- Entry conditions for service TesterPresent (0x3E) are not allowed.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- The service testerPresent (0x3E) shall not affect the ECU's ability to execute non diagnostic tasks.

4.10.2.1.5.1.2 Data Transmission Functional Unit

Service	Sub-Function	Session			Addressing		Comment
Name(Hex)	Name(Hex)	Default	Extended	Programming	Physical	Functional	
ReadDataByIdentifier (0x22)		M	M	MPS	M	M	
ReadMemoryByAddress (0x23)		N	M	N	M	O	
ReadDataByPeriodicIdentifier (0x2A)		N	O	D	M	O	
DynamicallyDefineDataIdentifier (0x2C)		O	O	N	M	O	
	DefineByIdentifier(0x01)	O	O	N	M	O	
	DefineByMemoryAddress(0x02)	O	O	N	M	O	
	ClearDynamicallyDefinedDataIdentifier(0x03)	O	O	N	M	O	
WriteDataByIdentifier (0x2E)		MC4	MC4	OS	M	O	
WriteMemoryByAddress (0x3D)		O	O	N	M	O	

663992 v2 **DynamicallyDefineDataIdentifier (0x2C)**

Version history:
(2): 1、添加诊断服务寻址的需求便于高效开发。2、调整29认证访问的支持情况。

Purpose: The service allows the client to dynamically define in a server a data identifier that can be read via the ReadDataByIdentifier service at a later time.

Source:

Verification Method: Analysis AND Test

Legacy ID: 2457v2;2461v1;2462v1;2465v1;2464v1;2463v1;2471v1;2468v1;2466v1;2467v1;2469v1;2470v1;2458v2;2459v1;2460v1;2417v1;2366v2;2380v1;2439v2;652774v1

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
DynamicallyDefineDataIdentifier (0x2C)	DefineByIdentifier(01,81)	DynamicallyDefinedDataIdentifier		M	M	N	M	O	M	O	200ms
		SourceDataIdentifier		M	M	N					
		PositionInSourceDataRecord	01~FF	M	M	N					
		MemorySize	01~FF	M	M	N					
	DefineByMemoryAddress (02,82)	DynamicallyDefinedDataIdentifier		M	M	N	M	O	M	O	
		AddressAndLengthFormatIdentifier	14	M	M	N					
			15/24/25	O	O	N					
		MemoryAddress	U	M	M	N					
		MemorySize	U	M	M	N					
	ClearDynamicallyDefinedDataIdentifier(03,83)	DynamicallyDefinedDataIdentifier		O	O	N	M	O	M	O	

Note:

- The values of DynamicallyDefinedDataIdentifier and SourceDataIdentifier are defined by *[1] UDS Data*.
- The addressAndLengthFormatIdentifier values 0x15 and 0x25 are intended for memory addresses there the most significant byte of the address is used as a memoryIdentifier. The definition of the memoryIdentifier is specified by the implementer.
- The service dynamicallyDefineDataIdentifier (0x2C) is allowed to be protected by security / Authentication only in the following situation: If data that is read by service readDataByIdentifier (0x22) or readMemoryByAddress (0x23) is protected by security / Authentication access, then the service shall be protected when including this same data (completely or partly) in the dynamically defined dataIdentifier.

Entry conditions:

- Entry conditions for service DynamicallyDefineDataIdentifier (0x2C) are not allowed.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- The dynamically defined data records that the ECU supports shall be kept until they are cleared by the service DynamicallyDefineDataIdentifier –ClearDynamicallyDefinedDataIdentifier.

- The service dynamicallyDefineDataIdentifier (0x2C) shall not affect the ECU's ability to execute non diagnostic tasks.

663989 v2 ReadDataByIdentifier (0x22)

Version history:
(2): 1、添加诊断服务寻址的需求便于高效开发。2、调整29认证访问的支持情况。

Purpose: The service allows the client to request data record valued from the server identified by one or more dataIdentifiers.
Source:
Verification Method: Analysis AND Test
Legacy ID: 2472v2;2473v3;2420v2;2369v2;2382v2;2400v3;2443v3;2442v2;652770v1;652771v1

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
ReadDataByIdentifier(0x22)	N/A	DataIdentifier		M	M	MPS	M	M	MC	C	200ms

Note:
- The value of the dataIdentifier(s) is defined by *[1] UDS Data*.
- Read by system supplier specific dataIdentifiers, is allowed to be protected by security access or Authentication access only if approved by GRI Electrical Architecture.

Entry conditions:
- Entry conditions for service ReadDataByIdentifier (0x22) are not allowed.

Exit Conditions:
The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:
- Minimum 10 dataIdentifiers, or as many dataIdentifiers as implemented, in one single ReadDataByIdentifier request shall be supported by the ECU in default and extended diagnostic session. The ECU is allowed to reject requests with multiple dataIdentifiers when the response is larger than 61 bytes. For example, lager than 1-byte ReadDataByIdentifier response SID (Service Identifier) + 10·2-byte dataIdentifier + 10·4-byte dataRecords. In programmingSession only one dataIdentifier needs to be supported.
- The service readDataByIdentifier (0x22) shall not affect the ECU's ability to execute non diagnostic tasks.

663991 v2 ReadDataByPeriodicIdentifier (0x2A)

Version history:
(2): 1、添加诊断服务寻址的需求便于高效开发。2、调整29认证访问的支持情况。

Purpose: The service allows the client to request the periodic transmission of data record values from the server identified by one or more periodicDataIdentifiers.
Source:
Verification Method: Analysis AND Test
Legacy ID: 2474v2;2476v1;2479v1;2478v1;2477v2;2480v1;2475v2;2421v1;2370v2;2383v1;2444v2;652773v1

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access	Security access 27	P4 Server
Name (Hex)	Name(H	Name	Hex	Default	Extende	Program	Physica	Functi			

	ex)				d	ming	l	onal	29		
ReadDataByPeriodicIdentifier (2A)	N/A	TransmissionModeSlow	01	N	O	D	M	O	M	N	200ms
	N/A	TransmissionMode Medium	02	N	O	D					
	N/A	TtransmissionMode Fast	03	N	C	D					
	N/A	TransmissionModeStop	04	N	M	D					
	N/A	PriodicDataIdentifier #1~#n	U	N	O	D					

Note:

- **Slow(01):**The value of the transmission rate in the transmissionMode slow shall be defined by the implementer.
- **Medium(02):**The value of the transmission rate in the transmissionMode medium shall be defined by the implementer.
- **Fast(03):**The value of the transmission rate in the transmissionMode fast shall be defined by the implementer. The transmissionMode fast is allowed to be defined as "as fast as possible" i.e. must not be fixed, it may change over time (e.g. be CPU load dependent).
- **PriodicDataIdentifier:**The service readDataByPeriodicIdentifier with the data parameter periodicDataIdentifier may be supported by the ECU as specified by *[10] ISO 14229-1*.

Entry conditions:

- Entry conditions for service ReadDataByPeriodicIdentifier (0x2A) are not allowed.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- Only response message type #2 is allowed. See also *[10] ISO 14229-1*, and the implementation specification for the respective network (e.g. UDSonCAN, UDSonFR and UDSonIP) for a detailed description of response message type #2.
- The service readDataByPeriodicIdentifier (0x2A) shall not affect the ECU's ability to execute non diagnostic tasks.

663990 v5 ReadMemoryByAddress (0x23)

Version history:
(5): 1、添加诊断服务寻址的需求便于高效开发。2、调整29认证访问的支持情况。3、调整P4server_max为200ms。

Purpose: The service allows the client to request memory data from the service via provided starting address and size of memory to be read.

Source:

Verification Method: Analysis AND Test

Legacy ID: 2481v1;2482v1;2483v1;647340v1;2484v1;2485v1;2424v1;2373v2;2386v1;2447v2;652772v1

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
ReadMermoryByAddress(0x23)	N/A	AddmessAndLengthFormatIdentifier	24	N	M	N	M	O	M	M (Note2)	200ms
	N/A		14/15/25/44 (Note1)	N	O	N					
	N/A	MermoryAddress	U	N	M	N					

	N/A	MermorySize	U	N	M	N					

Note:

- The addressAndLengthFormatIdentifier values 0x15 and 0x25 are intended for memory addresses there the most significant byte of the address is used as a memoryIdentifier. The definition of the memoryIdentifier is specified by the implementer.
- The service readMemoryByAddress (0x23) shall be protected by security access level 05.

Entry conditions:

- Entry conditions for service ReadMemoryByAddress (0x23) are not allowed.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- The service readMemoryByAddress (0x23) shall not affect the ECU's ability to execute non diagnostic tasks.
- The ECU reserves independent storage space to record key log information, including but not limited to: communication module data records, software download exception records, functional module records, and static current exception records. These log information are used for analyzing controller malfunctions or vehicle malfunctions.
- There shall be a separate fault state history buffer. Each buffer shall be updated with the last value derived from the respective fault history algorithm. There shall be a time stamp that relates to each value in all history buffers. The source of timestamps shall be a global time received from GNSS. This enables comparing running historic events across all function for that node and across nodes. If a timestamp value shall be taken and there is temporarily no communication on the network it is acceptable to use the last received time. If the time signal is not available a local time source can be used, in that case resolution shall be 100ms. If more than one buffer is implemented, the same source shall always be used for all buffers. All history buffers shall operate according to FIFO, first-in-first-out. All historical buffers should store the latest 100 states or the states within the last 7 days. No changes in states or conditions may be missed. **Example**:

Time Stamp1	byte1					byte2	byte3	……	TimeStamp2	byte1					byte2	byte3	……
YYYY-MM-DDThh:mm:ss.ssZ	bit7	bit6	bit5	bit4	…	…	…	…	YYYY-MM-DDThh:mm:ss.sssZ	bit7	bit6	bit5	bit4	…	…	…	…
	State1 (simple state) 0= 1=	State2 (Multiple States) 000= 001= 010= …			xx	xx	xx	… …		State1(simple state) 0= 1=	State2 (Multiple States) 000= 001= 010= …			xx	xx	xx	… …

- The ECU should have hardware and software that allow for persistent storage of historical buffered data. If the number of write events supported by the hardware memory itself is insufficient, the loss balancing algorithm in the software should be used to meet the requirements.

663993 v3 WriteDataByIdentifier (0x2E)

Version history:
(3): 1、添加诊断服务寻址的需求便于高效开发。2、调整29认证访问的支持情况。

Purpose: The service allows the client to write information into the server at an internal location specified by the provided data identifier.

Source:

Verification Method: Analysis AND Test

Legacy ID: 2486v3;2487v1;2488v1;2434v1;2377v1;2395v2;2455v2;652775v1

Service	Sub-function	Data	Session	Addressing	Authentic	Security	P4

		Paremeters							ation access 29	access 27	Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
WriteDataByIdentifier (0x2E)	N/A	DataIdentifier		O	M	MS	M	O	M	O	5000ms

Note:

- The values of the dataIdentifier are defined by *[1] UDS Data*.
- It is mandatory for the WriteDataByIdentifier (0x2E) to be protected by security /Authentication access under SBL, and optional under other circumstances.

Entry conditions:

- Entry conditions for service WriteDataByIdentifier (0x2E) are allowed only if approved by GRI Electrical Architecture.

If the ECU implement safety requirements with an ASIL higher than QM (see *[13] ISO 26262*) it shall, in all situations when diagnostic services may violate any of those safety requirements, reject the critical diagnostic service requests.But that require an approval by GRI Electrical Architecture.

When at least one of these conditions is not met, the ECU shall reject the request message with a negative response code 0x22.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- The service writeDataByIdentifier (0x2E) is allowed to affect the ECU's ability to execute non diagnostic tasks. The service is only allowed to affect execution of the non-diagnostic tasks during the execution of the diagnostic service. After the diagnostic service is completed any effect on the non-diagnostic tasks is not allowed anymore (normal operational functionality resumes). The service is not allowed to reset the ECU .

663994 v2 WriteMemoryByAddress (0x3D)

Version history:
(2): 1、添加诊断服务寻址的需求便于高效开发。2、调整诊断会话和29认证访问的支持情况。

Purpose: The service allows the client to write information into the server at one or more contiguous memory locations.

Source:

Verification Method: Analysis AND Test

Legacy ID: 2489v1;2490v1;2491v1;2493v1;2494v1;2492v1;2435v1;2378v1;2396v1;2456v2;652793v1

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
WriteMemoryByAddress (0x3D)	N/A	AddressAndLengthFormatIdentifier	14	N	M	N	M	O	M	O	5000ms
	N/A		15/24/25	N	O	N					
	N/A	MemoryAddress	U	N	M	N					
	N/A	MemorySize	U	N	M	N					
	N/A	DataRecord	U	N	M	N					

Note:

- The addressAndLengthFormatIdentifier values 0x15 and 0x25 are intended for memory addresses there the most significant byte of the address is used as a memoryIdentifier. The definition of the memoryIdentifier is specified by the implementer.

- The values of the MemoryAddress/MemorySize/DataRecord shall be defined by the implementer.

Entry conditions:

- Entry conditions for service WriteMemoryByAddress (0x3D) are allowed only if approved by GRI Electrical Architecture.

If the ECU implement safety requirements with an ASIL higher than QM (see *[13] ISO 26262*) it shall, in all situations when diagnostic services may violate any of those safety requirements, reject the critical diagnostic service requests.But that require an approval by GRI Electrical Architecture.

When at least one of these conditions is not met, the ECU shall reject the request message with a negative response code 0x22.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- The service writeMemoryByAddress (0x3D) is allowed to affect the ECU's ability to execute non diagnostic tasks. The service is only allowed to affect execution of the non-diagnostic tasks during the execution of the diagnostic service. After the diagnostic service is completed any effect on the non-diagnostic tasks is not allowed anymore (normal operational functionality resumes). The service is not allowed to reset the ECU .

4.10.2.1.5.1.3 Stored Data Transmission Functional Unit

Service	Sub-Function	Session			Addressing		Comment
Name(Hex)	Name(Hex)	Default	Extended	Programming	Physical	Functional	
ClearDiagnosticInformation (0x14)		M	M	D	M	M	
ReadDTCInformation (0x19)		M	M	D	M	M	
	ReportDTCByStatusMask (0x02)	M	M	D	M	M	
	ReportDTCSnapshotIdentification(0x03)	M	M	D	M	M	
	ReportDTCSnapshotRecordByDTCNumber(0x04)	M	M	D	M	M	
	ReportDTCExtendedDataRecordByDTCNumber(0x06)	M	M	D	M	M	
	ReportSupportedDTC (0x0A)	M	M	D	M	M	
	ReadDTCInformation – reportDTCWithPermanentStatus(0x15)	MC5	MC5	D	M	M	
	ReportWWHOBDDTCWithPermanentStatus (0x55)	MC5	MC5	D	M	M	

663995 v2 **ClearDiagnosticInformation (0x14)**

Version history:
(2): 1、添加诊断服务寻址的需求便于高效开发。2、调整29认证访问的支持情况。3、考虑到NVM翻页导致的时间加长，将P4server_max由3500ms调整为5000ms。

Purpose: The service is used by the client to clear diagnostic information in one or multiple servers' memory.

Source:

Verification Method: Analysis AND Test

Legacy ID: 2534v1;2537v3;2535v1;2536v2;2415v2;2363v2;2379v2;2438v2;652794v1

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
ClearDiagnosticInformation (0x14)	N/A	GroupOfDTC Specific	Any 3 Byte	MC1	MC1	D	M	M	M	N	5000ms
	N/A	GroupOfDTC All DTCs in emission related ECU	FFF000	MC2	MC2	D	M	M	M	N	
	N/A	GroupOfDTC All Groups	FFFFFF	M	M	D	M	M	M	N	

Note:

- Refer to section *Additional ClearDiagnosticInformation requirements* for additional clear requirements.

Entry conditions:

- Entry conditions for service ClearDiagnosticInformation (0x14) are allowed only if approved by GRI Electrical Architecture.

If the ECU implement safety requirements with an ASIL higher than QM (see *[12] ISO 26262)* it shall, in all situations when diagnostic services may violate any of those safety requirements, reject the critical diagnostic service requests.But that require an approval by GRI Electrical Architecture.

When at least one of these conditions is not met, the ECU shall reject the request message with a negative response code 0x22.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- The service clearDiagnosticInformation (0x14) shall not affect the ECU's ability to execute non diagnostic tasks.

663996 v2 ReadDTCInformation (0x19)

Version history:
(2): 1、添加诊断服务寻址的需求便于高效开发。2、调整29认证访问的支持情况。3、新增19 55诊断服务。

Purpose: This service allows a client to read the status of server resident Diagnostic Trouble Code(DTC) information from any server, or group of servers within a vehicle.

Source:

Verification Method: Analysis AND Test

Legacy ID: 2542v1;2543v2;2544v1;2549v1;2550v2;2551v2;2553v1;2552v1;2545v2;2546v1;2547v1;2548v1;2555v1;2554v2;2422v2;2371v2;2384v1;2445v2;652795v1

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
ReadDTCInformation (0x19)	ReportDTCByStatusMask(02, 82)	DTCStatusMask		M	M	D	M	M	M	N	500ms

	ReportDTCSnapshotIdentification(03, 83)	N/A	N/A	M	M	D	M	M	M	N
	ReportDTCSnapshotRecordByDTCNumber (04, 84)	DTCMaskRecord		M	M	D	M	M	M	N
		DTCSnapshotRecordNumber	FF							
			U							
	ReportDTCExtendedDataRecordByDTCNumber (06, 86)	DTCMaskRecord		M	M	D	M	M	M	N
		DTCSnapshotRecordNumber	FF							
			U							
	ReportSupportedDTC(0A, 8A)	N/A	N/A	M	M	D	M	M	M	N
	ReportDTCWithPermanentStatus(15, 95)	N/A	N/A	MC5	MC5	D	M	M	M	N
	ReportWWHOBDDTCWithPermanentStatus (55, D5)	FunctionalGroupIdentifier	Emission-system group (33)	MC5	MC5	D	M	M	M	N
			Safety-system group (D0)	O	O	D				
			Legislative system group (D1 ~ DF)	O	O	D				
			VOBD system (FE)	O	O	D				
			ISO/SAE reserved (00~32, 34~CF, E0~FD, FF)	O	O	D				

Note:
The values of the DTCMaskRecord and the DTCSnapShotRecordNumber are defined by *[1] UDS Data*.

Entry conditions:

- Entry conditions for service ReadDTCInformation (0x19) are not allowed.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- The service ReadDTCInformation (0x19) shall not affect the ECU's ability to execute non diagnostic tasks.

4.10.2.1.5.1.4 InputOutput Control Functional Unit

Service	Sub-Function	Session			Addressing		Comment
Name(Hex)	Name(Hex)	Default	Extended	Programming	Physical	Functional	

InputOutputControlByIdentifier (0x2F)		N	M	N	M	O	Note 1

Note:
1. a. From the time the request is received by the ECU until the ECU regains control, the substituted values of the data record (referenced by the dataIdentifier) shall remain static. If the substituted values need to change over time, for example due to safety reasons, the service routineControl (0x31) shall be used.
b. The service InputOutputControlByIdentifier (0x2F) is the only service intended for temporarily controlling inputs (e.g., real world sensors) and outputs (e.g., real world actuators). Unlike service writeDataByIdentifier (0x2E) and writeMemoryByAddress (0x3D), the values substituted by the service 0x2F shall always be immediately effective and revert back to the normal value (as determined by the control system) when the ECU regains control over the values.

663998 v2 InputOutputControlByIdentifier (0x2F)

Version history:
(2): 1、添加诊断服务寻址的需求便于高效开发。2、调整29认证访问的支持情况。3、调整P4server_max为200ms。

Purpose: The service is used by the client to substitute a value for an input signal, internal server function and/or force control to a value for an output of an electronic system.

Source:

Verification Method: Analysis AND Test

Legacy ID: 2513v2;2519v2;2521v2;2520v2;2522v2;2517v3;2515v3;2523v1;2524v1;2514v1;2516v1;2518v1;2419v1;2368v1;2381v3;2441v3;652797v1

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
InputOutputControlByIdentifier (0x2F)	N/A	DataIdentifier		N	M	N	M	O	M	O	200ms
	N/A	InputOutputControlParameter returnControlToECU	00	N	M	N					
	N/A	InputOutputControlParameter freezeCurrentState	02	N	O	N					
	N/A	InputOutputControlParameter shortTermAdjustment	03	N	M	N					
	N/A	ControlState data record	U	N	M	N					
	N/A	ControlMask		N	MC3	N					

Note:

- The dataIdentifier is defined by *[1] UDS Data*.
- The values of ControlState data record are defined specified by the implementer .

- The service InputOutputControlByIdentifier with data parameter controlMask shall be supported by the ECU as specified in *[10] ISO 14229-1* if the dataRecord consists of more than one parameter (i.e. the dataRecord is packeted or bitmapped) but not if the dataRecord consists of only one parameter,The length of the controlMask depends on the dataRecord that are defined by *[1] UDS Data*.

Entry conditions:

- Entry conditions for service InputOutputControlByIdentifier (0x2F) are allowed only if approved by GRI Electrical Architecture.

If the ECU implement safety requirements with an ASIL higher than QM (see *[13] ISO 26262*) it shall, in all situations when diagnostic services may violate any of those safety requirements, reject the critical diagnostic service requests.But that require an approval by GRI Electrical Architecture.

When at least one of these conditions is not met, the ECU shall reject the request message with a negative response code 0x22.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- From the time the request is received by the ECU until the ECU regains control, the substituted values of the data record (referenced by the dataIdentifier) shall remain static. If the substituted values need to change over time, for example due to safety reasons, the service routineControl (0x31) shall be used.
- The service InputOutputControlByIdentifier (0x2F) is the only service intended for temporarily controlling inputs (e.g., real world sensors) and outputs (e.g., real world actuators). Unlike service writeDataByIdentifier (0x2E) and writeMemoryByAddress (0x3D), the values substituted by the service 0x2F shall always be immediately effective and revert back to the normal value (as determined by the control system) when the ECU regains control over the values.
- The ECU shall support several consecutive shortTermAdjustment requests without any returnControlToECU (0x00) requests in between.
- The ECU shall accept a correct returnControlToECU (0x00) request regardless of the value of the controlMask bits (0 or 1).
- The controlState parameter (of the controlStatusRecord) of the positive response shall always be equal to the size of the dataRecord (referenced by the dataIdentifier) and shall represent the actual current value(s) for each parameter within the dataIdentifier, independent of the value of the controlMask (i.e., the values that would be returned if the data were to be included in a positive response to a service readDataByIdentifier (0x22) request at that time).
- The service inputOutputControlByIdentifier (0x2F) is allowed to affect the ECU's ability to execute non diagnostic tasks. The service is only allowed to affect execution of the non-diagnostic tasks during the execution of the diagnostic service. After the diagnostic service is completed any effect on the non-diagnostic tasks is not allowed anymore (normal operational functionality resumes). The service is not allowed to reset the ECU .

4.10.2.1.5.1.5 Routine Functional Unit

Service	Sub-Function	Session			Addressing		Comment
Name(Hex)	Name(Hex)	Default	Extended	Programming	Physical	Functional	
RoutineControl (0x31)		M	M	MPS	M	M	Note 1
	StartRoutine(0x01)	M	M	MPS	M	M (Note2)	
	StopRoutine(0x02)	O	O	OPS	M	O	
	RequestRoutineResults(0x03)	O	O	OPS	M	O	

Note:
1. In defaultSession the only allowed RoutineControl type is type 1- Short routine. Refer to section *Additional routine requirements* for additional routine requirements. Note that whether or not the ECU shall support a specific sub-function depends on the routine type and is specified by section *Additional routine requirements*.

2. The RoutineControl with sub-function startRoutine shall support functional addressing, e.g. Check programming pre-conditions.

663999 v2 **RoutineControl (0x31)**

Version history:
(2): 1、添加诊断服务寻址的需求便于高效开发。2、调整29认证访问的支持情况。3、增加Note需求。

Purpose: The service is used by the client ti execute a defined sequence of steps and obtain any relevant results.
Source:
Verification Method: Analysis AND Test
Legacy ID: 2525v2;2529v1;2530v1;2528v2;2531v1;2533v1;2532v2;2526v1;2527v1;2430v1;2429v2;2374v1;2391v3;2452v2;652798v1

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
RoutineControl (0x31)	StartRoutine (01)	RoutineIdentifier		M	M	MPS	M	M	MC	O	5000ms
	StopRoutine (02)	RoutineIdentifier		O	O	OPS	M	O	M	O	200ms
	RequestRoutineResults (03)	RoutineIdentifier		O	O	OPS	M	O	M	O	

Note：

- In defaultSession the only allowed RoutineControl type is type 1- Short routine.
- The values of routineIdentifier are defined by *[1] UDS Data*.
- These values are general response time requirements for P4 servers. Exceptions from the general requirements are specified in *[1] UDS Data*.
- It is mandatory for the RoutineControl (0x31) to be protected by security /Authentication access in programming session, and optional under other circumstances.
- It is mandatory for the RoutineControl (0x31) to be protected by Authentication access in all diagnostic session, except for some specific routine dataIdentifer, e.g. Check programming pre-conditions, Certificate Validity Check.
- The RoutineControl with sub-function startRoutine shall support functional addressing, e.g. Check programming pre-conditions.

Entry conditions:

- Entry conditions for service RoutineControl (0x31) are allowed only if approved by GRI Electrical Architecture.

If the ECU implement safety requirements with an ASIL higher than QM (see *[13] ISO 26262)* it shall, in all situations when diagnostic services may violate any of those safety requirements, reject the critical diagnostic service requests.But that require an approval by GRI Electrical Architecture.

When at least one of these conditions is not met, the ECU shall reject the request message with a negative response code 0x22.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- The service routineControl (0x31) is allowed to affect the ECU's ability to execute non diagnostic tasks. The service is only allowed to affect execution of the non-diagnostic tasks during the execution of the diagnostic service. After the diagnostic service is completed any effect on the non-diagnostic tasks is not allowed anymore (normal operational functionality resumes). The service is not allowed to reset the ECU .

4.10.2.1.5.1.6 Upload download functional unit

Service	Sub-Function	Session			Addressing		Comment
Name(Hex)	Name(Hex)	Default	Extended	Programming	Physical	Functional	
RequestDownload (0x34)		N	N	MPS	M	O	Note:1
RequestUpload (0x35)		N	O	OPS	M	O	
TransferData (0x36)		N	O	MPS	M	O	
RequestTransferExit (0x37)		N	O	MPS	M	O	
RequestFileTransfer (0x38)		N	O	OPS	M	O	Note:2

Note:
1. The compression method "Delta encoding" is optional only if the size of the software file is under 10Mb or if the programming time is under 10 minutes. (See *[7] General SWDL Specification* for more info.)
2. If the ECU supports the RequestFileTransfer service which is an alternative method for file based data storage the RequestDownload service may not need to be supported.

664001 v3 **RequestDownload (0x34)**

Version history:
(3): 增加对于基于文件操作系统的ECU，AddressAndLengthFormatIdentifier值可以为0x44或0x88。

Purpose: The service is used by the client to initiate a data transfer from the client to the server.
Source:
Verification Method: Analysis AND Test
Legacy ID: 2583v2;2577v3;2579v3;2580v2;2578v4;2576v2;2581v2;2582v2;354180v1;2425v1;2387v1;2448v2;652800v1

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
RequestDownload (0x34)	N/A	DataFormatIdentifier	00	N	N	MPS	M	O	M	M	1000ms
	N/A		10	N	N	MPS					
	N/A		80	N	N	OPS					
	N/A		01-0F/11-7F/ 81-FF	N	N	OPS					
	N/A	AddressAndLengthFormatIdentifier	44	N	N	MPS					
	N/A		88	N	N	MPS					
	N/A	MemoryAddress	00000000 - FFFFFFFF	N	N	MPS					
	N/A		0000000000000000 - FFFFFFFFFFFFFFFF	N	N	MPS					
	N/A	MemorySize	00000000 - FFFFFFFF	N	N	MPS					
	N/A		0000000000000000 -	N	N	MPS					

			FFFFFFFFFFFFFF								

Note:

- DataFormatIdentifier 0x00 no compression and no encryption.
- DataFormatIdentifier 0x10 compression method #1, as specified in *[5] Data Compression and Encryption* and no encryption.
- DataFormatIdentifier 0x80 Delta encoding, as specified in *[6] Delta Encoding Specification* and no encryption.
- For address based ECU, AddressAndLengthFormatIdentifier, with the value 0x44 (4 bytes MemoryAddress and 4 bytes memorySize)
- For file based ECU supporting file operation system, AddressAndLengthFormatIdentifier, with the value 0x44 (4 bytes MemoryAddress and 4 bytes memorySize) or 0x88 (8 bytes MemoryAddress and 8 bytes memorySize). The external tool will depend on the value of address_and_length_format_identifier defined in ReqID 992348 to send correct diagnostic request.

Entry conditions:

- Entry conditions for service RequestDownload (0x34) are not allowed.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- The description about lengthFormatIdentifier in *[10] ISO14229-1*.
  lengthFormatIdentifier of Positive response message for RequestDownload.
- **lengthFormatIdentifier**
  This parameter is a one byte value with each nibble encoded separately:
  - bit 7 - 4: Length(number of bytes) of the maxNumberOfBlockLength parameter.
  - bit 3 - 0: reserved by document, to be set to '0'.
- **maxNumberOfBlockLength**
  This parameter is used by the requestDownload positive response message to inform the client how many data bytes (maxNumberOfBlockLength) to include in each TransferData request message from the client.
- **The specific limit for lengthFormatIdentifier from OEM client:**
  - 0x20 is commonly supported and allows for a maximum block of 0x 20 FF FF (i.e. 65535 bytes) wihich based on a single connection.
  - 0x40 is supportable but only when increase socket connection.
  - Others(Example 0x30) is **not allowed**.

664005 v3 RequestFileTransfer (0x38)

Version history:
(3): 1、添加诊断服务寻址的需求便于高效开发。

Purpose: The service is used by the client to initiate a file data transfer from either the client to the server or from the server to the client.This service has capabilities to retrieve information about the file system.

Source:

Verification Method: Analysis AND Test

Legacy ID: 2596v2;2593v1;2594v1;2595v1;2589v1;2588v1;2584v2;2586v3;2587v2;2585v3;2591v2;2592v2;2590v2;2426v2;2388v1;2449v1;652804v1

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
RequestFileTran	N/A	ModeOfOp	01-03	N	M	MPS	M	O	M	M	1000ms

sfer (0x38)	N/A	eration	04-05	N	O	OPS					
	N/A	FilePathAndNameLength	0000-FFFF	N	M	MPS					
	N/A	FilePathAndName	00-XX	N	M	MPS					
	N/A	DataFormatIdentifier	00	N	M	MPS					
	N/A		10	N	M	MPS					
	N/A		80	N	O	OPS					
	N/A		01-0F, 11-7F, 81-FF	N	O	OPS					
	N/A	FileSizeParameterLenght	00-FF	N	M	MPS					
	N/A	FileSizeUnCompressed	00-XX	N	M	MPS					
	N/A	FileSizeCompressed	00-XX	N	M	MPS					

Note:

- If the modeOfOperation parameter 0x03 ReplaceFile is used and the file not is stored at the location shall the file be added.
- The presence of the dataFormatIdentifier parameter depends on the modeOfOperation parameter.
- DataFormatIdentifier 0x10 compression method #1: Specified in reference *[5] Data Compression and Encryption* and no encryption.
- DataFormatIdentifier 0x80 Delta encoding: Specified in reference *[6] Delta encoding* and no encryption.
- FileSizeParameterLenght and fileSizeUnCompressed and fileSizeCompressed: The presence of the parameter depends on the modeOfOperation parameter.

Entry conditions:

- Entry conditions for service RequestFileTransfer (0x38) are not allowed.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- The RequestFileTransfer service is an alternative method if the ECU implementes a file system data storage and replaces the RequestDownload and RequestUpload service. The RequestFileTransfer service shall be implemented as defined in *[10] ISO 14229-1*.

664004 v2 RequestTransferExit (0x37)

Version history:
(2): 1、添加诊断服务寻址的需求便于高效开发。2、调整诊断会话的支持情况。

Purpose: The service is used by the client to terminate a data transfer between client and server.
Source:
Verification Method: Analysis AND Test
Legacy ID: 2597v2;2598v5;2599v4;2600v1;2601v5;2427v2;2389v1;2450v2;652803v1

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			

RequestTransferExit (0x37)	N/A	N/A	N/A	N	O	MPS	M	O	N	N	1000ms

Note:

- The use cases (a-d) for blockSequenceCounter, specified in *[10] ISO 14229-1*, shall be handled by the tester and the ECU.

Entry conditions:

- Entry conditions for service RequestTransferExit (0x37) are not allowed.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- If the ECU does not support Software Authentication (CheckMemory routine) as defined in *[8] General Software Authentication*, the transferResponseParameterRecord shall contain a checksum for downloaded or uploaded data block.
  If the ECU supports Software Authentication, no transferResponseParameterRecord shall be used for downloaded block but only for upload.
  Examples - Data transfer initiated by RequestDownload

Table - Example 1: RequestTransferExit positive response when CheckMemory is not implemented

Tester	Description	Data								Explanation
Tx	RequestTransferExit (S.F)	01	37	00	00	00	00	00	00	
Rx	Positive response	03	77	xx	xx	00	00	00	00	2-byte CRC

Table - Example 2: RequestTransferExit positive response when CheckMemory is implemented (i.e. Software Authentication supported)

Tester	Description	Data								Explanation
Tx	RequestTransferExit (S.F)	01	37	00	00	00	00	00	00	
Rx	Positive response	01	77	00	00	00	00	00	00	CRC omitted

- If the transfer of data is initiated by RequestDownload or RequestUpload the transferResponseParameterRecord checksum calculation shall use the CRC16-CCITT algorithm (initial value 0xFFFF and normal representation). The checksum shall be calculated on all data bytes as defined by the diagnostic services RequestDownload or RequestUpload. Hint. For algorithm compliance checking: use the example described in section *Appendix A- Suggestion code for transferResponseParameterRecord calculation*.
- If the transfer of data is initiated by the RequestFileTransfer service the (transferResponseParameterRecord) shall consist of a four byte checksum based on the CRC32 algorithm (initial value 0xFFFFFFFF and normal representation) calculation on all data bytes as defined by the diagnostic services RequestFileTransfer. Hint. For algorithm compliance checking use the example described in section *Appendix A- Suggestion code for transferResponseParameterRecord calculation* .
- The checksum (transferResponseParameterRecord) shall always be calculated on data read from the actual non-volatile memory as defined by the RequestDownload, RequestUpload or RequestFileTransfer .

664002 v2 RequestUpload (0x35)

Version history:
(2): 1、添加诊断服务寻址的需求便于高效开发。

Purpose: The service is used by the client to initiate a data transfer from the server to the client.
Source:

Verification Method: Analysis AND Test
Legacy ID: 2606v3;2603v2;2602v2;2604v2;2605v2;2428v1;2390v1;2451v2;652801v1

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
RequestUpload (0x35)	N/A	DataFormatIdentifier	00	N	M	MS	M	O	M	M	1000ms
	N/A	AddressAndLengthFormatIdentifier	44	N	M	MS					
	N/A	MemoryAdress	0000-FFFF	N	M	MS					
	N/A	MemorySize	0000-FFFF	N	M	MS					

Note:
- AddressAndLengthFormatIdentifier with the value 0x44 (4 bytes MemoryAddress and 4 bytes memorySize).

Entry conditions:
- Entry conditions for service RequestUpload (0x35) are not allowed.

Exit Conditions:
The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:
- --

664003 v2 TransferData (0x36)

Version history:
(2): 1、添加诊断服务寻址的需求便于高效开发。2、调整诊断会话的支持情况。

Purpose: The service is used by the client transfer data either from the client to the server or from the server to the client.
Source:
Verification Method: Analysis AND Test
Legacy ID: 2610v3;2611v2;2612v2;2607v2;2608v2;2609v2;2433v1;2390v1;2454v2;652802v1

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
TransferData (0x36)	N/A	BlockSequenceCounter	00-FF	N	O	MPS	M	O	N	N	5000ms
	N/A	TransferRequestParameterRecord	00-FF	N	O	MPS					

Note:
- The use cases (a-d) for blockSequenceCounter, specified in *[10] ISO 14229-1*, shall be handled by the tester and the ECU. It is recommended that a tester retransmits the transferData service (i.e. with the same blockSequenceCounter and data) at least two times if no response is received from the ECU.

Entry conditions:
- Entry conditions for service TransferData (0x36) are not allowed.

Exit Conditions:
The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.
Implementation Rules:
• If an ECU receives a transferData request during an active download sequence with the same blockSequenceCounter as the last accepted transferData request, it shall respond with a positive response without writing the data once again to its memory.

4.10.2.1.5.1.7 Additional routine requirements

The positive response on the service routineControl request, regardless of sub-function, shall have the data parameter RoutineStatusRecord. The first byte of the data parameter RoutineStatusRecord shall be the RoutineInfo byte. The RoutineInfo byte shall consist of two fields where bits 7-4 defines RoutineType (see table *RoutineType definition*) and bits 3-0 defines RoutineStatus (see table *RoutineStatus definition*).

The result of the routine shall (at least) be available, via requestRoutineResults, as long as the diagnostic session is kept or until the routine is re-started.

Table: Routine type definition

Index	Hex	Name	Definition
1	0	Reserved	Reserved by this document
2	1	Type 1- Short routine	The routine stops after a certain time when it is completed. The routine is completed before a positive response on startRoutine request is sent. A positive response on startRoutine request shall be sent and only be sent in the following situations : • The entry conditions have been fulilled and the routine has been completed (and stopped). • The entry conditions have not been fulfilled. • Any of exit conditions have occurred. A positive response shall include parameters for routine execution results. The sub-function stopRoutine shall not be supported. It is not possible to abort a short routine by diagostic service during its execution as the positive response on startRoutine request is sent after the routine has been stopped (aborted or completed). The sub-function requestRoutineResults may be supported. Note: The routine with Routine Type 1 shall be completed within 5 seconds.
3	2	Type 2- Long routine	The routine stops after a certain time when it is completed. The routine may be completed before or after a positive response on startRoutine request is sent. A positive response on startRoutine request shall be sent and only be sent in the following situations: • The entry conditions have been fulilled and the routine has been started or completed (and stopped). • The entry conditions have not been fulfilled. • Any of exit conditions have occurred. The sub-function stopRoutine should be supported. It should be possible to abort a routine during its execution, before it is completed, by stopRoutine requests (after a positive response on startRoutine request has been sent).

			The sub-function RequestRoutineResults shall be supported. It shall be possible to read the RoutineInfo byte by RequestRoutineResults requests (after a positive response on startRoutine request has been sent) both while the routine executes and after it has been stopped (aborted or completed). Note that the routine type is compliant with the "Method B" specified in *[10] ISO 14229-1*.
4	3	Type 3 - Continuous routine	The routine is stopped by stopRoutine request. The routine don't need a certain time in order to be completed. It is completed when it is stopped by stopRoutine request. A positive response on startRoutine request shall be sent and only be sent in the following situations: • The entry conditions have been fulilled and the routine has started. • The entry conditions have not been fulfilled. The sub-function stopRoutine shall be supported. The sub-function RequestRoutineResults shall be supported. It shall be possible to read the RoutineInfo byte by RequestRoutineResults requests both while the routine executes and after it has been stopped (aborted or completed). Note that the routine type is compliant with the "Method A" specified in *[10] ISO 14229-1*.
5	4-F	Reserved	Reserved by this document

Table: Routine status definition

Index	Hex	Name	Definition
1	0	RoutineCompleted	The routine completed all requested functionality (successfully or not). The routine results (if any), readable via service routineControl and/or any other service, are valid.
2	1	RoutineAborted	The routine aborted before completion of all requested functionality due to any of the following reasons: 1. The routine has not started due to the entry conditions are not fulfilled. 2. The routine has been aborted due to any of the exit conditions have occurred. 3. The routine has been aborted by a stopRoutine request (only applicable for routine type 2).
3	2	RoutineExcecutes	The routine is under execution and not yet completed (only applicable for routine type 2 and 3).
4	3-F	Reserved	Reserved by this document

665734 v3 **Routine Type 1- Short routine**

Version history:
(3): Verification Method设置为Test。

Purpose: Quick and easy method for perfoming simpler issues. One advantage with this routine is that it may be run in defaultSession.
Source:
Verification Method: Test
Legacy ID: 2338v1;2339v3;2344v2

The routine stops after a certain time when it is completed. The routine is completed before a positive response on startRoutine request is sent .

Routine Type 1- Short routine : A positive response on startRoutine request shall be sent and only be sent in the following situations:

- The entry conditions have been fulfilled and the routine has been completed (and stopped).
- The entry conditions have not been fulfilled.
- Any of exit conditions have occurred.

A positive response shall include parameters for routine execution results.

Routine Type 1- Short routine: The sub-function requestRoutineResults may be supported.

Note: The routine with Routine Type 1 shall be completed within 5 seconds.

665735 v2 **Routine Type 2- Long routine**

Version history:
(2):

Purpose: Implementation requirements for the long type of Routine.
Source:
Verification Method: Test
Legacy ID: 2345v1;2346v3;2348v1;2347v1

The routine stops after a certain time when it is completed. The routine is completed before or after a positive response on startRoutine request is sent.
Type 2- Long routine : A positive response on startRoutine request shall be sent and only be sent in the following situations:

- The entry conditions have been fulfilled and the routine has been started or completed (and stopped).
- The entry conditions have not been fulfilled.
- Any of exit conditions have occurred.

Type 2- Long routine :
The sub-function stopRoutine should be supported.
It should be possible to abort a routine under execution, before it is completed, by stopRoutine requests (after a positive response on startRoutine request has been sent).

The sub-function RequestRoutineResults shall be supported.
It shall be possible to read the RoutineInfo byte by RequestRoutineResults requests (after a positive response on startRoutine request has been sent) both while the routine executes and after it has been stopped (aborted or completed).

665736 v1 **Routine Type 3 - Continuous routine**

Version history:
(1):

Purpose: Implementation requirements for the continuous type of Routine.
Source:
Verification Method: Test
Legacy ID: 2349v1;2350v2;2352v1;2351v1

Type 3 - Continuous routine: The routine is stopped by stopRoutine request. The routine don't need a certain time in order to be completed. It is completed when it is stopped by stopRoutine request.
Type 3 - Continuous routine: A positive response on startRoutine request shall be sent and only be sent in the following situations:

- The entry conditions have been fulilled and the routine has started.
- The entry conditions have not been fulfilled.

Type 3 - Continuous routine: The sub-function stopRoutine and RequestRoutineResults shall be supported.
It shall be possible to read the RoutineInfo byte by RequestRoutineResults requests both while the routine executes and after it has been stopped (aborted or completed).

665731 v1 Routine byte

Version history:
(1):

Purpose: Defines the info format of the routine response.
Source:
Verification Method: Test
Legacy ID: 2340v2

The positive response on the service routineControl request, regardless of sub-function, shall have the data parameter RoutineStatusRecord. The first byte of the data parameter RoutineStatusRecord shall be the RoutineInfo byte. The RoutineInfo byte shall consist of two fields where bits 7-4 defines RoutineType and bits 3-0 defines RoutineStatus.

bit 7	bit 4	bit 3	bit 0
RoutineType		RoutineStatus	

The value of the RoutineType shall be one of the following:

Hex	RoutineType
0x1	Type 1 - Short Routine
0x2	Type 2 - Long Routine
0x3	Type 3 - Continuous Routine

The value of the RoutineStatus shall be one of the following:

Hex	RoutineStatus
0x0	RoutineCompleted
0x1	RoutineAborted
0x2	RoutineExecutes

665733 v1 Routine result availability

Version history:
(1):

Purpose: Define how long time the route shall keep the result of the routine.
Source:
Verification Method: Test
Legacy ID: 2337v1

The result of the routine shall (at least) be available, via requestRoutineResults as long as the diagnostic session is kept or until the routine is re-started.

665737 v1 RoutineStatus

Version history:
(1):

Purpose: Define three execution states of Routines.
Source:
Verification Method: Test
Legacy ID: 2342v2;2341v3;2343v1

RoutineStatus, routineCompleted:
The routine completed all requested functionality (successfully or not). The routine results (if any), readable via service routineControl and/or any other service, are valid.

RoutineStatus, RoutineAborted :
The routine aborted before completion of all requested functionality due to any of the following reasons:
1. The routine has not started due to the entry conditions are not fulfilled.
2. The routine has been aborted due to any of the exit conditions have occurred.
3. The routine has been aborted by a stopRoutine request (only applicable for routine type 2).

RoutineStatus, RoutineExcecutes:
The routine is under execution and not yet completed (only applicable for routine type 2 and 3).

943093 v1 **The relationship between routine and action**

Version history:
(1): 新增需求，说明例程和动作的关系。

Purpose: Clarify the relationship between routine and actual execution actions. Suppliers need to pay special attention when developing code implementations.
Source:
Verification Method: Analysis
Legacy ID:

Each routine always has a corresponding action to match. If a routine is executed, the corresponding actions can be actuator movement, relay on/off, calibration, or internal code flag setting. So for routine IDs that support the sub-function requestRoutineResults, the end of the action does not mean that the routine is terminated. It should be able to retrieve the routine result at any time until it is truly terminated, e.g. through session transition, or initialization, or new routine start, or in other way.

4.10.2.1.5.1.8 Additional session requirements

4.10.2.1.5.1.8.1 General

When the ECU transitions from any diagnostic session to another diagnostic session (including the currently active session), the ECU shall reset all active diagnostic functionality requested by diagnostic services that is not supported in the new diagnostic session or that are protected by security access. Note that this means that the ECU shall also reset the security level to the locked state if the ECU was previously unlocked.

When the ECU transitions from any non-default diagnostic session to the same non-default diagnostic session, the ECU shall maintain all active diagnostic functionality requested by diagnostic services, with the exception of the functionality that are protected by security access.

Note that written changes to non-volatile memory shall remain regardless of diagnostic session transitions.

665741 v1 **Maintain active diagnostic functionality at session transitions**

Version history:
(1):

Purpose: Define session transition behaviour.

Source:
Verification Method: Test
Legacy ID: 2353v1

When the ECU transitions from any non-default diagnostic session to the same non-default diagnostic session, the ECU shall maintain all active diagnostic functionality requested by diagnostic services, with the exception of the functionality that are protected by security access.
Note that written changes to non-volatile memory shall remain regardless of diagnostic session transitions.

665740 v1 **Reset active diagnostic functionality at session transitions**

Version history:
(1):

Purpose: Define session transition behaviour.
Source:
Verification Method: Test
Legacy ID: 2354v2

When the ECU transitions from any diagnostic session to another diagnostic session (including the currently active session), the ECU shall reset all active diagnostic functionality requested by diagnostic services that is not supported in the new diagnostic session or that are protected by security access. The ECU shall also reset the security level to the locked state if the ECU was previously unlocked.

4.10.2.1.5.1.8.2 Response message data parameter sessionParameterRecord

The structure and content of the positive diagnostic response message data parameter sessionParameterRecord is defined in table SessionParameterRecord structure definition and table SessionParameterRecord content definition . Thus, the following specified in *[Services_9] ISO 14229-1* regarding the sessionParameterRecord shall be ignored: "The content and structure of this parameter record is data-link-layer-specific and can be found in the implementation specification(s) of this part of ISO 14229".

Table: SessionParameterRecord structure definition

Byte pos. in record	Description	Value	M/O
	sessionParameterRecord[] = [		
#1	P2 $_{Server_max}$ (high byte)	0x00 – 0xFF	M
#2	P2 $_{Server_max}$ (low byte)	0x00 – 0xFF	M
#3	P2* $_{Server_max}$ (high byte)	0x00 – 0xFF	M
#4	P2* $_{Server_max}$ (low byte)]	0x00 – 0xFF	M

Table: SessionParameterRecord content definition

Parameter	Description	# of bytes	Resolution	Value	Note
P2 $_{Server_max}$	Default P2 $_{Server_max}$ timing supported by the server for the activated diagnostic session. Note that the activated diagnostic session is the same as the requested diagnostic session.	2	1 ms	See note 2	1
P2* $_{Server_max}$	Enhanced (NRC 0x78) P2 $_{Server_max}$ supported by the server for the activated diagnostic session. Note that the activated diagnostic session is the same as the requested diagnostic session.	2	10 ms	See note 2	1

Notes in table SessionParameterRecord content definition:

1. The ECU shall not use the service to alter the values of P2 $_{Server_max}$ and P2* $_{Server_max}$ from the values specified in *[2] UDS Session*. The response message data parameter sessionParameterRecord shall contain the fixed values of P2 $_{Server_max}$ and P2* $_{Server_max}$ defined in *[2] UDS Session*.
2. See *[2] UDS Session.*

665742 v1 **SessionParameterRecord definition**

Version history:
(1):

Purpose: Define SessionParameterRecord.
Source:
Verification Method: Test
Legacy ID: 2355v1

The structure and content of the positive diagnostic response message data parameter sessionParameterRecord is defined in table *SessionParameterRecord structure definition* and table *SessionParameterRecord content definition* .

4.10.2.1.5.1.9 Additional ClearDiagnosticInformation requirements

666488 v1 **GroupOfDTC All DTCs in emission related ECU**

Version history:
(1):

Purpose: To shorten the time the ECU is in the workshop or during the assembly process, it shall be possible to clear all DTCs in emission related ECUs with one diagnostic request.
Source:
Verification Method: Test
Legacy ID: 2535v1

If the ECU is emission related, the service ClearDiagnosticInformation with the data parameter groupOfDTC set to the value 0xFFF000 – “All DTCs in emission related ECU” shall be supported by the ECU in all sessions where the ECU supports the service ClearDiagnosticInformation but only for functional addressed diagnostic request messages. An emission related ECU is not allowed to support this for physical addressed diagnostic request messages. Refer to section Diagnostic message addressing for details regarding functional addressed requests. Note that the requirement “The server shall support its list of diagnostic services regardless of addressing mode (physical, functional addressing type)” specified in *[10] ISO 14229-1*, shall not apply in this case. Note that this data parameter groupOfDTC – “All DTCs in emission related ECU” is not defined in *[10] ISO 14229-1*.

If the ECU is not emission related, the service ClearDiagnosticInformation with the data parameter groupOfDTC set to the value 0xFFF000 – “All DTCs in emission related ECU” is not allowed to be supported by the ECU.

666491 v2 **GroupOfDTC All Groups**

Version history:
(2): The emission related ECU shall support functional addressed $14 FF FF FF, according to J1979-3 2022.

Purpose: To shorten the time the ECU is in the workshop or during the assembly process, it shall be possible to clear all DTCs with one diagnostic request.
Source:
Verification Method: Test
Legacy ID: 2536v2

If the ECU is not emission related, the service ClearDiagnosticInformation with the data parameter groupOfDTC set to All Groups shall be supported by the ECU in all sessions where the ECU supports the service ClearDiagnosticInformation.

If the ECU is emission related, the service ClearDiagnosticInformation with the data parameter groupOfDTC set to All Groups shall be supported by the ECU in all sessions where the ECU supports the service ClearDiagnosticInformation for functional addressed and physical addressed diagnostic request messages. Refer to section Diagnostic message addressing for details regarding functional addressed requests.

666487 v1 **GroupOfDTCSpecific DTC**

Version history:
(1):

Purpose: It shall be possible to remove specific DTCs.
Source:
Verification Method: Test
Legacy ID: 2537v3

If the ECU is not emission related, the service ClearDiagnosticInformation with the data parameter groupOfDTC set to a value of any specific DTC shall be supported by the ECU in all sessions where the ECU supports the service ClearDiagnosticInformation.

If the ECU is emission related, the service ClearDiagnosticInformation with the data parameter groupOfDTC set to a value of a “calibration DTC” shall be supported by the ECU in all sessions where the ECU supports the service ClearDiagnosticInformation.
A “calibration DTC” is a non-emission related DTC that indicates that a specific action (e.g. request a diagnostic service for initiating or calibrating a function controlled by the ECU) that must be performed as a part of the car manufacturing process has not yet been performed and that shall be cleared (by the diagnostic service ClearDiagnosticInformation with the data parameter groupOfDTC set to a value of the “calibration DTC”) as soon as the action has been completed. This requirement is only applicable if the ECU support “calibration DTCs” (one or more).

If the ECU is emission related, the service ClearDiagnosticInformation with the data parameter groupOfDTC set to a value a non-emission related DTC, other than a “calibration DTC”, is allowed to be supported by the ECU in all sessions where the ECU supports the service ClearDiagnosticInformation.

If the ECU is emission related, the service ClearDiagnosticInformation with the data parameter groupOfDTC set to a value of an emission related DTC is not allowed to be supported by the ECU.

•Appendix

4.10.2.1.6 Appendix

4.10.2.1.6.1 Appendix A- Suggestion code for transferResponseParameterRecord calculation

4.10.2.1.6.1.1 CRC16-CITT C-code example 1 (fast)

This example uses a look-up table with pre-calculated CRCs for fast calculation.

\+ fast calculation
\- large code size

```
/* function declarations */
unsigned int CalcCRC(unsigned int size,unsigned char *data);

/* test data array */
```

```
unsigned char data[256] =
{  0x00,0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08,0x09,0x0a,0x0b,0x0c,0x0d,0x0e,0x0f,
   0x10,0x11,0x12,0x13,0x14,0x15,0x16,0x17,0x18,0x19,0x1a,0x1b,0x1c,0x1d,0x1e,0x1f,
   0x20,0x21,0x22,0x23,0x24,0x25,0x26,0x27,0x28,0x29,0x2a,0x2b,0x2c,0x2d,0x2e,0x2f,
   0x30,0x31,0x32,0x33,0x34,0x35,0x36,0x37,0x38,0x39,0x3a,0x3b,0x3c,0x3d,0x3e,0x3f,
   0x40,0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48,0x49,0x4a,0x4b,0x4c,0x4d,0x4e,0x4f,
   0x50,0x51,0x52,0x53,0x54,0x55,0x56,0x57,0x58,0x59,0x5a,0x5b,0x5c,0x5d,0x5e,0x5f,
   0x60,0x61,0x62,0x63,0x64,0x65,0x66,0x67,0x68,0x69,0x6a,0x6b,0x6c,0x6d,0x6e,0x6f,
   0x70,0x71,0x72,0x73,0x74,0x75,0x76,0x77,0x78,0x79,0x7a,0x7b,0x7c,0x7d,0x7e,0x7f,
   0x80,0x81,0x82,0x83,0x84,0x85,0x86,0x87,0x88,0x89,0x8a,0x8b,0x8c,0x8d,0x8e,0x8f,
   0x90,0x91,0x92,0x93,0x94,0x95,0x96,0x97,0x98,0x99,0x9a,0x9b,0x9c,0x9d,0x9e,0x9f,
   0xa0,0xa1,0xa2,0xa3,0xa4,0xa5,0xa6,0xa7,0xa8,0xa9,0xaa,0xab,0xac,0xad,0xae,0xaf,
   0xb0,0xb1,0xb2,0xb3,0xb4,0xb5,0xb6,0xb7,0xb8,0xb9,0xba,0xbb,0xbc,0xbd,0xbe,0xbf,
   0xc0,0xc1,0xc2,0xc3,0xc4,0xc5,0xc6,0xc7,0xc8,0xc9,0xca,0xcb,0xcc,0xcd,0xce,0xcf,
   0xd0,0xd1,0xd2,0xd3,0xd4,0xd5,0xd6,0xd7,0xd8,0xd9,0xda,0xdb,0xdc,0xdd,0xde,0xdf,
   0xe0,0xe1,0xe2,0xe3,0xe4,0xe5,0xe6,0xe7,0xe8,0xe9,0xea,0xeb,0xec,0xed,0xee,0xef,
   0xf0,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7,0xf8,0xf9,0xfa,0xfb,0xfc,0xfd,0xfe,0xff
};

unsigned int crctab[256] =
{
   0x0000, 0x1021, 0x2042, 0x3063, 0x4084, 0x50A5, 0x60C6, 0x70E7,
   0x8108, 0x9129, 0xA14A, 0xB16B, 0xC18C, 0xD1AD, 0xE1CE, 0xF1EF,
   0x1231, 0x0210, 0x3273, 0x2252, 0x52B5, 0x4294, 0x72F7, 0x62D6,
   0x9339, 0x8318, 0xB37B, 0xA35A, 0xD3BD, 0xC39C, 0xF3FF, 0xE3DE,
   0x2462, 0x3443, 0x0420, 0x1401, 0x64E6, 0x74C7, 0x44A4, 0x5485,
   0xA56A, 0xB54B, 0x8528, 0x9509, 0xE5EE, 0xF5CF, 0xC5AC, 0xD58D,
   0x3653, 0x2672, 0x1611, 0x0630, 0x76D7, 0x66F6, 0x5695, 0x46B4,
   0xB75B, 0xA77A, 0x9719, 0x8738, 0xF7DF, 0xE7FE, 0xD79D, 0xC7BC,
   0x48C4, 0x58E5, 0x6886, 0x78A7, 0x0840, 0x1861, 0x2802, 0x3823,
   0xC9CC, 0xD9ED, 0xE98E, 0xF9AF, 0x8948, 0x9969, 0xA90A, 0xB92B,
   0x5AF5, 0x4AD4, 0x7AB7, 0x6A96, 0x1A71, 0x0A50, 0x3A33, 0x2A12,
   0xDBFD, 0xCBDC, 0xFBBF, 0xEB9E, 0x9B79, 0x8B58, 0xBB3B, 0xAB1A,
   0x6CA6, 0x7C87, 0x4CE4, 0x5CC5, 0x2C22, 0x3C03, 0x0C60, 0x1C41,
   0xEDAE, 0xFD8F, 0xCDEC, 0xDDCD, 0xAD2A, 0xBD0B, 0x8D68, 0x9D49,
   0x7E97, 0x6EB6, 0x5ED5, 0x4EF4, 0x3E13, 0x2E32, 0x1E51, 0x0E70,
   0xFF9F, 0xEFBE, 0xDFDD, 0xCFFC, 0xBF1B, 0xAF3A, 0x9F59, 0x8F78,
   0x9188, 0x81A9, 0xB1CA, 0xA1EB, 0xD10C, 0xC12D, 0xF14E, 0xE16F,
   0x1080, 0x00A1, 0x30C2, 0x20E3, 0x5004, 0x4025, 0x7046, 0x6067,
   0x83B9, 0x9398, 0xA3FB, 0xB3DA, 0xC33D, 0xD31C, 0xE37F, 0xF35E,
   0x02B1, 0x1290, 0x22F3, 0x32D2, 0x4235, 0x5214, 0x6277, 0x7256,
   0xB5EA, 0xA5CB, 0x95A8, 0x8589, 0xF56E, 0xE54F, 0xD52C, 0xC50D,
   0x34E2, 0x24C3, 0x14A0, 0x0481, 0x7466, 0x6447, 0x5424, 0x4405,
   0xA7DB, 0xB7FA, 0x8799, 0x97B8, 0xE75F, 0xF77E, 0xC71D, 0xD73C,
   0x26D3, 0x36F2, 0x0691, 0x16B0, 0x6657, 0x7676, 0x4615, 0x5634,
   0xD94C, 0xC96D, 0xF90E, 0xE92F, 0x99C8, 0x89E9, 0xB98A, 0xA9AB,
   0x5844, 0x4865, 0x7806, 0x6827, 0x18C0, 0x08E1, 0x3882, 0x28A3,
   0xCB7D, 0xDB5C, 0xEB3F, 0xFB1E, 0x8BF9, 0x9BD8, 0xABBB, 0xBB9A,
   0x4A75, 0x5A54, 0x6A37, 0x7A16, 0x0AF1, 0x1AD0, 0x2AB3, 0x3A92,
   0xFD2E, 0xED0F, 0xDD6C, 0xCD4D, 0xBDAA, 0xAD8B, 0x9DE8, 0x8DC9,
   0x7C26, 0x6C07, 0x5C64, 0x4C45, 0x3CA2, 0x2C83, 0x1CE0, 0x0CC1,
   0xEF1F, 0xFF3E, 0xCF5D, 0xDF7C, 0xAF9B, 0xBFBA, 0x8FD9, 0x9FF8,
   0x6E17, 0x7E36, 0x4E55, 0x5E74, 0x2E93, 0x3EB2, 0x0ED1, 0x1EF0
};

/* driver */
void main(void)
{
   unsigned int crc;

   crc=CalcCRC(256,data);

   while(1); /* the result of the above calculation shall be: crc=0x3FBD */
```

```
}

/* Calculate CRC */
unsigned int CalcCRC(unsigned int size, unsigned char *data)
{
   unsigned int crc=0xffff; /* initial value */
   int tmp;
   int i;

   for(i=0;i<size;i++) {
     tmp=(crc>>8)^data[i];
     crc=(crc<<8)^crctab[tmp];
   }
   return crc;
}
```

4.10.2.1.6.1.2 CRC16-CITT C-code example 2 (slow)

This example is not using any look-up table, only shift and XOR operations.

\+ small code size
\- slow calculation

```
#define                MASK                    0x1021                  /* CRC16-CITT
mask */

/* function declarations */
unsigned int CalcCRC(unsigned int size,unsigned char *data);

/* test data array */
unsigned char data[256] =
{  0x00,0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08,0x09,0x0a,0x0b,0x0c,0x0d,0x0e,0x0f,
   0x10,0x11,0x12,0x13,0x14,0x15,0x16,0x17,0x18,0x19,0x1a,0x1b,0x1c,0x1d,0x1e,0x1f,
   0x20,0x21,0x22,0x23,0x24,0x25,0x26,0x27,0x28,0x29,0x2a,0x2b,0x2c,0x2d,0x2e,0x2f,
   0x30,0x31,0x32,0x33,0x34,0x35,0x36,0x37,0x38,0x39,0x3a,0x3b,0x3c,0x3d,0x3e,0x3f,
   0x40,0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48,0x49,0x4a,0x4b,0x4c,0x4d,0x4e,0x4f,
   0x50,0x51,0x52,0x53,0x54,0x55,0x56,0x57,0x58,0x59,0x5a,0x5b,0x5c,0x5d,0x5e,0x5f,
   0x60,0x61,0x62,0x63,0x64,0x65,0x66,0x67,0x68,0x69,0x6a,0x6b,0x6c,0x6d,0x6e,0x6f,
   0x70,0x71,0x72,0x73,0x74,0x75,0x76,0x77,0x78,0x79,0x7a,0x7b,0x7c,0x7d,0x7e,0x7f,
   0x80,0x81,0x82,0x83,0x84,0x85,0x86,0x87,0x88,0x89,0x8a,0x8b,0x8c,0x8d,0x8e,0x8f,
   0x90,0x91,0x92,0x93,0x94,0x95,0x96,0x97,0x98,0x99,0x9a,0x9b,0x9c,0x9d,0x9e,0x9f,
   0xa0,0xa1,0xa2,0xa3,0xa4,0xa5,0xa6,0xa7,0xa8,0xa9,0xaa,0xab,0xac,0xad,0xae,0xaf,
   0xb0,0xb1,0xb2,0xb3,0xb4,0xb5,0xb6,0xb7,0xb8,0xb9,0xba,0xbb,0xbc,0xbd,0xbe,0xbf,
   0xc0,0xc1,0xc2,0xc3,0xc4,0xc5,0xc6,0xc7,0xc8,0xc9,0xca,0xcb,0xcc,0xcd,0xce,0xcf,
   0xd0,0xd1,0xd2,0xd3,0xd4,0xd5,0xd6,0xd7,0xd8,0xd9,0xda,0xdb,0xdc,0xdd,0xde,0xdf,
   0xe0,0xe1,0xe2,0xe3,0xe4,0xe5,0xe6,0xe7,0xe8,0xe9,0xea,0xeb,0xec,0xed,0xee,0xef,
   0xf0,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7,0xf8,0xf9,0xfa,0xfb,0xfc,0xfd,0xfe,0xff
};

/* driver */
void main(void)
{
   unsigned int crc;

   crc=CalcCRC(256,data);

   while(1); /* the result of the above calculation shall be: crc=0x3FBD */
}

/* Calculate CRC */
unsigned int CalcCRC(unsigned int size, unsigned char *data)
{
```

```
   unsigned int crc=0xffff; /* initial value */
   int tmp;
   int i,j;

   for(i=0;i<size;i++){
      tmp=data[i]<<8;
      for(j=0;j<8;j++){
         if((crc ^ tmp) & 0x8000) crc=(crc<<1)^MASK;
         else crc<<=1;
         tmp<<=1;
      }
   }
   return crc;
}
```

4.10.2.1.6.1.3 CRC32 C-code example

This example uses a look-up table with pre-calculated CRCs for fast calculation.

```
/* function declarations */
void generate_crc32_table(void);
unsigned long calc_crc32(unsigned long size, unsigned char *data);

/* variables */
unsigned long crc32;
static unsigned long crc32_table[256];

/* test data array */
unsigned char data[256] =
{  0x00,0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08,0x09,0x0a,0x0b,0x0c,0x0d,0x0e,0x0f,
   0x10,0x11,0x12,0x13,0x14,0x15,0x16,0x17,0x18,0x19,0x1a,0x1b,0x1c,0x1d,0x1e,0x1f,
   0x20,0x21,0x22,0x23,0x24,0x25,0x26,0x27,0x28,0x29,0x2a,0x2b,0x2c,0x2d,0x2e,0x2f,
   0x30,0x31,0x32,0x33,0x34,0x35,0x36,0x37,0x38,0x39,0x3a,0x3b,0x3c,0x3d,0x3e,0x3f,
   0x40,0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48,0x49,0x4a,0x4b,0x4c,0x4d,0x4e,0x4f,
   0x50,0x51,0x52,0x53,0x54,0x55,0x56,0x57,0x58,0x59,0x5a,0x5b,0x5c,0x5d,0x5e,0x5f,
   0x60,0x61,0x62,0x63,0x64,0x65,0x66,0x67,0x68,0x69,0x6a,0x6b,0x6c,0x6d,0x6e,0x6f,
   0x70,0x71,0x72,0x73,0x74,0x75,0x76,0x77,0x78,0x79,0x7a,0x7b,0x7c,0x7d,0x7e,0x7f,
   0x80,0x81,0x82,0x83,0x84,0x85,0x86,0x87,0x88,0x89,0x8a,0x8b,0x8c,0x8d,0x8e,0x8f,
   0x90,0x91,0x92,0x93,0x94,0x95,0x96,0x97,0x98,0x99,0x9a,0x9b,0x9c,0x9d,0x9e,0x9f,
   0xa0,0xa1,0xa2,0xa3,0xa4,0xa5,0xa6,0xa7,0xa8,0xa9,0xaa,0xab,0xac,0xad,0xae,0xaf,
   0xb0,0xb1,0xb2,0xb3,0xb4,0xb5,0xb6,0xb7,0xb8,0xb9,0xba,0xbb,0xbc,0xbd,0xbe,0xbf,
   0xc0,0xc1,0xc2,0xc3,0xc4,0xc5,0xc6,0xc7,0xc8,0xc9,0xca,0xcb,0xcc,0xcd,0xce,0xcf,
   0xd0,0xd1,0xd2,0xd3,0xd4,0xd5,0xd6,0xd7,0xd8,0xd9,0xda,0xdb,0xdc,0xdd,0xde,0xdf,
   0xe0,0xe1,0xe2,0xe3,0xe4,0xe5,0xe6,0xe7,0xe8,0xe9,0xea,0xeb,0xec,0xed,0xee,0xef,
   0xf0,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7,0xf8,0xf9,0xfa,0xfb,0xfc,0xfd,0xfe,0xff
};


/* driver */
void main(void)
{
   unsigned long crc32;

   generate_crc32_table();
   crc32=calc_crc32(256,data);

   while(1); /* the result of the above calculation shall be: crc32=0x29058c73 */
}


/* calculate the crc32 value */
unsigned long calc_crc32(unsigned long size, unsigned char *data)
{
```

```
   unsigned long i, crc;

   crc=0xffffffff;          // initial value

   for(i=0;i<size;i++) {
      crc = ((crc>>8)&0x00FFFFFF)^crc32_table[(int)(crc^data[i])&0xff];
   }

   return ~crc;
}



#define CRC32_POLY 0xedb88320 // 0x04c11db7 bit reversed

/* generate the lock-up table */
void generate_crc32_table(void)

{
   int i, j;
   unsigned long crc;

   for(i=0;i<256;i++) {
      crc=i;
      for(j=8;j>0;j--) {
         if(crc&1) {
            crc=(crc>>1)^CRC32_POLY;
         }
         else{
            crc>>=1;
         }
      }
      crc32_table[i]=crc;
   }
}

// As an alternative to the "generate_crc32_table()" function the following table
can be used

/*
static unsigned long crc32_tab[256] =
{  0x00000000L, 0x77073096L, 0xee0e612cL, 0x990951baL, 0x076dc419L, 0x706af48fL,
   0xe963a535L, 0x9e6495a3L, 0x0edb8832L, 0x79dcb8a4L, 0xe0d5e91eL, 0x97d2d988L,
   0x09b64c2bL, 0x7eb17cbdL, 0xe7b82d07L, 0x90bf1d91L, 0x1db71064L, 0x6ab020f2L,
   0xf3b97148L, 0x84be41deL, 0x1adad47dL, 0x6ddde4ebL, 0xf4d4b551L, 0x83d385c7L,
   0x136c9856L, 0x646ba8c0L, 0xfd62f97aL, 0x8a65c9ecL, 0x14015c4fL, 0x63066cd9L,
   0xfa0f3d63L, 0x8d080df5L, 0x3b6e20c8L, 0x4c69105eL, 0xd56041e4L, 0xa2677172L,
   0x3c03e4d1L, 0x4b04d447L, 0xd20d85fdL, 0xa50ab56bL, 0x35b5a8faL, 0x42b2986cL,
   0xdbbbc9d6L, 0xacbcf940L, 0x32d86ce3L, 0x45df5c75L, 0xdcd60dcfL, 0xabd13d59L,
   0x26d930acL, 0x51de003aL, 0xc8d75180L, 0xbfd06116L, 0x21b4f4b5L, 0x56b3c423L,
   0xcfba9599L, 0xb8bda50fL, 0x2802b89eL, 0x5f058808L, 0xc60cd9b2L, 0xb10be924L,
   0x2f6f7c87L, 0x58684c11L, 0xc1611dabL, 0xb6662d3dL, 0x76dc4190L, 0x01db7106L,
   0x98d220bcL, 0xefd5102aL, 0x71b18589L, 0x06b6b51fL, 0x9fbfe4a5L, 0xe8b8d433L,
   0x7807c9a2L, 0x0f00f934L, 0x9609a88eL, 0xe10e9818L, 0x7f6a0dbbL, 0x086d3d2dL,
   0x91646c97L, 0xe6635c01L, 0x6b6b51f4L, 0x1c6c6162L, 0x856530d8L, 0xf262004eL,
   0x6c0695edL, 0x1b01a57bL, 0x8208f4c1L, 0xf50fc457L, 0x65b0d9c6L, 0x12b7e950L,
   0x8bbeb8eaL, 0xfcb9887cL, 0x62dd1ddfL, 0x15da2d49L, 0x8cd37cf3L, 0xfbd44c65L,
   0x4db26158L, 0x3ab551ceL, 0xa3bc0074L, 0xd4bb30e2L, 0x4adfa541L, 0x3dd895d7L,
   0xa4d1c46dL, 0xd3d6f4fbL, 0x4369e96aL, 0x346ed9fcL, 0xad678846L, 0xda60b8d0L,
   0x44042d73L, 0x33031de5L, 0xaa0a4c5fL, 0xdd0d7cc9L, 0x5005713cL, 0x270241aaL,
   0xbe0b1010L, 0xc90c2086L, 0x5768b525L, 0x206f85b3L, 0xb966d409L, 0xce61e49fL,
   0x5edef90eL, 0x29d9c998L, 0xb0d09822L, 0xc7d7a8b4L, 0x59b33d17L, 0x2eb40d81L,
```

```
    0xb7bd5c3bL, 0xc0ba6cadL, 0xedb88320L, 0x9abfb3b6L, 0x03b6e20cL, 0x74b1d29aL,
    0xead54739L, 0x9dd277afL, 0x04db2615L, 0x73dc1683L, 0xe3630b12L, 0x94643b84L,
    0x0d6d6a3eL, 0x7a6a5aa8L, 0xe40ecf0bL, 0x9309ff9dL, 0x0a00ae27L, 0x7d079eb1L,
    0xf00f9344L, 0x8708a3d2L, 0x1e01f268L, 0x6906c2feL, 0xf762575dL, 0x806567cbL,
    0x196c3671L, 0x6e6b06e7L, 0xfed41b76L, 0x89d32be0L, 0x10da7a5aL, 0x67dd4accL,
    0xf9b9df6fL, 0x8ebeeff9L, 0x17b7be43L, 0x60b08ed5L, 0xd6d6a3e8L, 0xa1d1937eL,
    0x38d8c2c4L, 0x4fdff252L, 0xd1bb67f1L, 0xa6bc5767L, 0x3fb506ddL, 0x48b2364bL,
    0xd80d2bdaL, 0xaf0a1b4cL, 0x36034af6L, 0x41047a60L, 0xdf60efc3L, 0xa867df55L,
    0x316e8eefL, 0x4669be79L, 0xcb61b38cL, 0xbc66831aL, 0x256fd2a0L, 0x5268e236L,
    0xcc0c7795L, 0xbb0b4703L, 0x220216b9L, 0x5505262fL, 0xc5ba3bbeL, 0xb2bd0b28L,
    0x2bb45a92L, 0x5cb36a04L, 0xc2d7ffa7L, 0xb5d0cf31L, 0x2cd99e8bL, 0x5bdeae1dL,
    0x9b64c2b0L, 0xec63f226L, 0x756aa39cL, 0x026d930aL, 0x9c0906a9L, 0xeb0e363fL,
    0x72076785L, 0x05005713L, 0x95bf4a82L, 0xe2b87a14L, 0x7bb12baeL, 0x0cb61b38L,
    0x92d28e9bL, 0xe5d5be0dL, 0x7cdcefb7L, 0x0bdbdf21L, 0x86d3d2d4L, 0xf1d4e242L,
    0x68ddb3f8L, 0x1fda836eL, 0x81be16cdL, 0xf6b9265bL, 0x6fb077e1L, 0x18b74777L,
    0x88085ae6L, 0xff0f6a70L, 0x66063bcaL, 0x11010b5cL, 0x8f659effL, 0xf862ae69L,
    0x616bffd3L, 0x166ccf45L, 0xa00ae278L, 0xd70dd2eeL, 0x4e048354L, 0x3903b3c2L,
    0xa7672661L, 0xd06016f7L, 0x4969474dL, 0x3e6e77dbL, 0xaed16a4aL, 0xd9d65adcL,
    0x40df0b66L, 0x37d83bf0L, 0xa9bcae53L, 0xdebb9ec5L, 0x47b2cf7fL, 0x30b5ffe9L,
    0xbdbdf21cL, 0xcabac28aL, 0x53b39330L, 0x24b4a3a6L, 0xbad03605L, 0xcdd70693L,
    0x54de5729L, 0x23d967bfL, 0xb3667a2eL, 0xc4614ab8L, 0x5d681b02L, 0x2a6f2b94L,
    0xb40bbe37L, 0xc30c8ea1L, 0x5a05df1bL, 0x2d02ef8dL
};
*/
```

4.10.3 UDS Data

4.10.3.1 UDS Data - General Requirements

•Overview

4.10.3.1.1 Overview

The document *UDS Data* descript which DIDs/DTCs/RIDs should be supported by application layer or bootloader layer of the ECU, and define these diagnostic data parameter.

This section gives a short summary of the content of each concept container.

UDS Data - General Requirements:
This concept container contains a short summary of this chapter,and general requirement for data records.

UDS Data - Application:
This concept container contains all the requirements related to the implementation of DIDs/DTCs/RIDs in Application SW.

UDS Data - Bootloader:
This concept container contains all the requirements related to the implementation of DIDs/DTCs/RIDs in Bootloader SW.

•Requisite documents

4.10.3.1.2 Requisite documents

This specification shall be followed and the requirements specified in this document shall not be negotiable.

In the case of any requirement conflict between this document and any of the referenced external documents, the proposed solution to the conflict shall be approved by the responsible authority appointed by OEM before the conflict is recognized as resolved.

Number	Source	Title
Data_1	ECU Basetech SWRS	UDS Services
Data_2	ECU Basetech SWRS	UDS Session

Data_4	Carcom Database	Global Master Reference Database
Data_5	ECU Basetech SWRS	Central Car Configuration
Data_6	ECU Basetech SWRS	General Software Download Specification
Data_7	ECU Basetech SWRS	Security access algorithm
Data_8	ECU Basetech SWRS	Overview of vehicle configuration & customer settings.
Data_9	ECU SWRS	FltEgyCns
Data_10	ECU Basetech SWRS	General Software Authentication
Data_11	ISO/DIS 14229-1 (001, 2006-12-01 Corrected 2007-04-15)	Road vehicles – Diagnostics systems – Diagnostic services
Data_12	SAE J2012 (December 2007)	Diagnostic trouble code definitions
Data_13	The latest available version	OBD II regulation, section 1968.2 of title 13, California Code of Regulations
Data_14	ISO26021-1 (2008-05-15)	Road vehicles — End-of-life activation of on-board pyrotechnic devices — Part 1: General information and use case definitions
Data_15	ISO26021-2 (2008-07-15)	Road vehicles — End-of-life activation of on-board pyrotechnic devices — Part 2: Communication requirements
Data_16	ISO-26262 (2011)	Road vehicles — Functional safety , International Organization for Standardization, 2011
Data_17	ECU SWRS	Car Configuration Management

•Definitions and Abbreviations

4.10.3.1.3 Definitions and Abbreviations

Abbreviation or Definition	Description
ASIL	Automotive Safety Integrity Level
D	Don't Care Implementation of the requirement is not required and not forbidden. The implementer may decide if to implement or not. If implemented it will not be verified by OEM and shall not be expected to be supported by OEM tools.
DID	Data Identifier
DTC	Diagnostic Trouble Code
ECU	Electronic Control Unit
FM	Frequency Modulation
FTB	Failure Type Byte
GMRDB	Global Master Reference Data Base
HMI	Human Machine Interface
M	Mandatory
MC	Mandatory on Condition(s)
MCPS	Mandatory on Condition for primary and secondary bootloader.
MCS	Mandatory on Condition for secondary bootloader.
MCP	Mandatory on Condition for primary bootloader.
MPS	Mandatory for primary and secondary bootloader.
MS	Mandatory for secondary bootloader.

N	Not allowed
N/A	Not applicable
O	Optional
OEM	Original Equipment Manufacturer
OS	Optional in secondary bootloader. Not allowed in primary bootloader.
PCM	Pulse Code Modulation
PWM	Pulse Width Modulation
SBL	Secondary Bootloader
SWDL	Software Download
TRC	Test Run Criteria
UDS	Unified Diagnostic Services
Implementer	Person responsible for the diagnostic implementation of the ECU. It may be the vehicle manufacturer or the ECU supplier diagnostic software designer.
Long-term memory	A non-volatile memory (e.g., EEPROM or Flash).
Main circuit board	The circuit board in an ECU containing the micro-controller.
Public ECU master	A public ECU responsible to act as a diagnostic server for one or more private ECUs.
Operation cycle	Refer to section *ECU ability to receive requests* in *UDS Services* for definition of an operation cycle and requirements on an ECU to start and stop an operation cycle.
Private ECU	An ECU not addressable from a client/tester point of view.
Public ECU	An ECU addressable from a client/tester point of view.
Short-term memory	A volatile memory (e.g. RAM).
Responsible authority appointed by OEM	A person, appointed by OEM for a specific vehicle program, which is responsible for support, review and approval of the implementation of the diagnostic part in the vehicle.
Test Sample	A test sample represents the pass or fail result from a single instance of a DTC test execution when the test run criteria are met. This represents a single sample and therefore not generally a fully matured / qualified condition. A test sample representing a fail will increase the DTC Fault Detection Counter by a specific amount and a test sample representing a pass will decrease the DTC Fault Detection Counter by a specific amount. Note the difference between "test sample passed" and "test passed".
OEM tools	OEM tools include all tools used by OEM R&D, manufacturing and service. These include, but are not limited to: · Development tools (used for e.g. analysis, test, calibration etc.) · Manufacturing tools · Aftersales tools · Databases · Embedded software components

•DID General Requirement

4.10.3.1.4 DID General Requirement

Data records (containing current, frozen and static data) shall be supported by the ECU according to the table below (e.g. in the valid identifier ranges).

Unless specified by this document, the implementer shall specify, for each data identifier, if the data record shall be possible to read, write and/or control by the services specified in *UDS Services*.

The data records shall fit into one of the data types and classifications as described in *Data types and Classifications*. No other data types and classifications are allowed.

Table: Requirements on implementation of data records

Name	Valid identifier range	Definition, see note 5	Notes	Session, see note 6	
				Other	Program-ming
Vehicle Manufacturer specific	0100 – D8FF DE00 – E2FF E600 – ED1F ED80 - ED9F F010 - F0FF	The data items (e.g. external and internal signals) that shall be defined in these records are specified in sections *ECU external signals*, *ECU internal signals*, *ECU_Vehicle status*, *Network data* and *Local DTC snapshot data.*	1	M	D
Development Specific - implementer specified	D900 – DCFF E300 – E4FF EE00 – EFFF	Data records defined by the implementer. Data records that are needed only during the development of the ECU, for e.g. tuning, calibration and validation of the ECU's software, shall be defined in this range.	4	O	D
Development Specific	E500 - E5FF	Data records that are needed only during the development of the ECU, for e.g. tuning, calibration and validation of the ECU's software, are specified in this range.	4	O	D
DTC Global Snapshot Data	DD00 – DD7F	The data items that (e.g. external and internal signals) shall be defined in these records are specified in section *Global DTC snapshot data.*	1	M	D
Reserved by this document	DD80 – DDFF	This identifier range is reserved for future use.			
ECU Identification and Configuration	EDA0 – EDFF ED20 – ED7F	The data items that shall be defined in these records are specified in section *ECU_Vehicle identification.*	1	M	M
Identification Option – Vehicle Manufacturer Specific	F100 – F17F, F1A0 – F1EF	The data items that shall be defined in these records are specified in section *ECU_Vehicle identification*.	1	M	M
Identification Option – ISO specified	F180 – F19F	The data items that shall be defined in these records are specified in section *ECU_Vehicle status* and *ECU_Vehicle identification.*	1	M	M
Identification Options-System Supplier Specific	F1F0 – F1FF	Data records defined by the system supplier	4	O	D
Dynamically defined periodic DataIdentifier	F200 – F240	Periodic data records defined dynamically.	2	O	D
Reserved by this document	F241 – F2FF	This identifier range is reserved for future use.			
Dynamically Defined-DataIdentifier	F300 – F3FF	Data records defined dynamically.	3	O	D
OBD Identifier	F400 – F8FF	Reserved by ISO 14229-1 for OBD/EOBD PIDs, on-board monitoring result value and info types as defined in ISO 15031-5.		MC	
Airbag deployment dataidentifier	FA00 – FA0F	Reserved by ISO 14229-1 for safety-systems related airbag deployment DIDs.		MC	
Safety system dataidentifier	FA10 – FAFF	Reserved by ISO 14229-1 for safety-system related DIDs.		MC	

System Supplier Specific	FD00 – FEFF	Data records defined by the system supplier	4	O	D
Reserved by ISO 14229-1	0000 – 00FF F000 – F00F F900 – F9FF FB00 – FCFF FF00 – FFFF	Reserved by ISO 14229-1		MC	

Notes in the table above:
1. Data records within this range shall be implemented **exactly** as they are defined in *Global Master Reference Database*.

2. If the ECU supports only one single dynamically defined periodicDataIdentifier then its value shall be 0xF200, if the ECU supports two dynamically defined periodicDataIdentifiers then the values shall be 0xF200 and 0xF201, etc.

Any identifier implemented within this range shall not be supported by services (specified by *UDS Services*) that can write or control the dynamically defined data parameters.

Any identifier implemented within this range shall be supported by services (specified by *UDS Services*) that can dynamically define a data identifier and read the dynamically defined data parameter periodically.

3. If the ECU supports only one single dynamicallyDefinedDataIdentifier then its value shall be 0xF300, if the ECU supports two dynamicallyDefinedDataIdentifiers then the values shall be 0xF300 and 0xF301, etc.

Any identifier implemented within this range shall not be supported by services (specified by *UDS Services*) that can write or control the dynamically defined data parameters.

Any identifier implemented within this range shall be supported by services (specified by *UDS Services*) that can dynamically define a data identifier and read the dynamically defined data parameter..

4. The implementer shall not expect that OEM tools support the data records within this range.

5. The identifiers that the ECU supports are specified by the implementer.

6. Note that “M” means that the ECU shall support data records according to column “Definition” of the table which shall be defined within the data record identifier range according to column “valid identifier range”.

7. Note that “O” means that the ECU may support data records according to column “Definition” of the table which shall be defined within the identifier range according to column “valid identifier range”.

8. Note that "MC" means that Mandatory if the ECU is required to implement identifiers in these ranges due to emission or safety legislation. The implementation shall be according to specification in ISO, SAE or other legislative documentation. Otherwise optional.

660543 v1 **DID - Data types and Classifications**

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID:

All data records shall be defined according to the data types and classifications described below. No other data types and classifications are allowed.

Where multi-byte data records are specified and the byte order is not already specified in Global Master Reference Database, then the order shall be that the most significant byte of the multi-byte record shall be placed in the data byte with the lowest number according to the data byte numbering convention.

Data type 1- ASCII (American Standard Code for Information Interchange)
The ASCII data type contains one character per byte with a value between 0x00 and 0x7F using the standard ASCII code. ASCII parameters shall always be left justified (i.e., the most significant byte represents the first character) with any unused bytes filled with 0x00.

Data type 2- BCD (Binary Coded Decimal)
The BCD data type contains two numbers per byte with a value between 0 and 9 using the standard 4 bit hex value for each number. BCD parameters shall always be left justified (i.e., the most significant bit represents the first number) with any unused nibbles filled with 0xF.

Data type 3- State Encoded
The state encoded data type is used when hexadecimal values represent unique states (e.g., using different values to represent the states of the ignition switch). State encoded parameters shall be limited to a maximum size of 1 byte (i.e., 256 unique states).

Data type 4- Bit-Mapped
The bit-mapped data type contains the binary state (0/1) of a parameter in a single bit (e.g., the discrete state of an ECU output may be represented as "0" for "Off" and "1" for "On"). If a particular bit-mapped parameter is not supported within a data record that is being reported by the ECU, the value of this bit shall be set to zero.

Data type 5- Unsigned Numeric
The unsigned numeric data type is used to represent data that has a defined range of values and a specific resolution that can be represented through hexadecimal encoding. Each unsigned numeric parameter consists of a size in bits (n), a resolution (r), and an offset. The minimum value of the unsigned parameter is equal to the offset. The maximum value of the unsigned parameter is equal to (((2n-1) * r) + offset). The maximum value of n is 32.

Data type 6- Signed Numeric
The signed numeric data type is used to represent data that has a defined range of values and a specific resolution that can be represented through 2's complement encoding. Each signed numeric parameter consists of a size in bits (n) and a resolution (r). The minimum value of the signed parameter is equal to - (2(n-1) * r). The maximum value of the signed parameter is equal to ((2(n-1) - 1) * r). The maximum value of n is 32.

Data type 7- Binary floating point
The binary floating point data type is used to represent data that has a defined range of values and a specific resolution that can be represented through the IEEE 754 single precision standard for binary floating point arithmetic. Each binary floating point parameter consists of a sign bit, exponent, and mantissa that is stored in accordance with single precision IEEE 754.

Data type 8 - Hexadecimal
The hexadecimal data type contains two characters per byte with a value between 0 and 9 and between A and F using the standard 4 bit hex value for each character.

660523 v1 **DID - External Signals Requirements**

Version history:
(1):

Purpose:
Source:
Verification Method: Inspection
Legacy ID:

All external signals (i.e. the signals on all the external circuits connected to the ECU) shall be defined in data

records within the ECU according to the table below.

It shall be possible to read and control these data records, according to the table, by diagnostic services specified in *UDS Services* in all diagnostic sessions except the programming session..

Table: Requirements for definition of ECU external signals in data records

Type of signal	Item	Signals to be defined in data records by the implementer	Diagnostic service, see note 7		Notes
			Read	I/O Control	
Switch input	1.1	Actual digital input signal read by the ECU (i.e., the sampled raw signal), but after necessary HW and/or SW filtering of “normal” disturbances.	M	O	1, 6
	1.2	Value currently available for the ECU´s application software (e.g., a filtered value used by the ECU when in normal operation or the replacement value used when the ECU is in limp-home operation).	MC1	O	1
Analog input	2.1	Actual input signal read by the ECU (i.e., the sampled raw signal), but after necessary HW and/or SW filtering of “normal” disturbances.	M	O	1, 2, 6
	2.2	Value currently available for the ECU´s application software (e.g. a filtered value used by the ECU when in normal operation or the replacement value used when the ECU is in limp-home operation).	MC1	O	1
FM, PCM, PWM (or similar) input	3.1	Actual FM, PCM, PWM (or similar) input signal in suitable engineering units.	M	O	1, 3, 6
	3.2	Actual digital input signal read by the ECU (i.e., the sampled raw signal), but after necessary HW and/or SW filtering of “normal” disturbances.	MC1	O	1, 6
Output	4.1	Output signal (the desired set point) controlled by the ECU´s application software or by the ECU´s diagnostic software (from the tester via a diagnostic service).	M	M	5
	4.2	Actual output signal read back by the ECU as a digital or analog signal.	M	N/A	1
Public ECU network signals	5.1	Network signals (e.g. data items embedded in CAN- or FlexRay-frames) transmitted by the ECU.	MC1	MC1	4, 5
	5.2	Network signals (e.g. data items embedded in CAN- or FlexRay-frames) received by the ECU.	MC1	O	6
	5.3	The network signal that contain the data record 0xDD02 – “Vehicle battery voltage” transmitted by the ECU.	MC2	MC2	4
External electrical circuits connected to private ECUs	6	Note that from a diagnostic point of view, all Private ECUs shall be regarded as if they were internally connected I/O units, reading sensors and controlling actuators, within the Public ECU master itself. The Private ECU master shall, on behalf of the private ECUs, support all the diagnostic data required by this table and that are applicable for the private ECUs.			
Power supply	7	Actual voltage read as an analog input signal by the ECU (i.e. the sampled raw signal), but after necessary HW or/and SW filtering of “normal” disturbances.	M	O	1, 6
Other external signals	8	Specified by the implementer.	MC1	MC1	5, 6

Abbreviations in the table above:
MC1 = A feasibility study is required. The data record(s) shall be implemented if the study shows that a technical solution is available and its cost is justified. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM.
MC2 = Mandatory if the ECU is responsible for maintaining and distributing the network signal.

Notes in the table above:
1. The signal read by the diagnostic services shall be updated minimum each 100 ms.
2. By examining the raw signal it shall be possible to determine if the signal is correct or if a short to battery, short to ground or open circuit is present.
3. By examining the value read it will normally be possible to decide whether the circuit is correct or if a short to battery voltage, short to ground or open circuit fault is present. If this is not possible then the feasibility study required by item 3.2 of the table shall be performed to decide if the decision can be made by examining the value of the raw version of the signal instead.
4. If the network signal is normally sent periodically by the ECU then this must continue during diagnostic control of the signal.
5. It shall be possible to control all outputs individually and independently of each other unless there are special safety reasons not to do this. If any output is prohibited to be controlled due to such safety reason, this must be approved by the responsible authority appointed by OEM.
6. The ECU shall report the tester substituted input value to a diagnostic request to read the signal value while the input parameter value is under tester control.
7. Whether or not the session (in which the data record shall be supported) is all other sessions, or just one or some sessions, depends on which sessions the diagnostic services for read/write/control of the data is supported in.

660524 v1 DID - Internal Signals Requirements

Version history:
(1):

Purpose:
Source:
Verification Method: Inspection
Legacy ID:

All internal signals (i.e. the signals on all internal circuits connected to the processor of the ECU) shall be defined in data records within the ECU, according to the table below in all diagnostic sessions except the programming session.

It shall be possible to read and control these data records, according to the table, by the diagnostic services specified in *UDS Services*.

Table: Requirements for definition of ECU internal signals in data records

Type of signal	Item	Signals to be defined in data records by the implementer	Diagnostic service, see note 4		Notes
			Read	I/O Control	
Input from internal HMI (e.g., push button)	1.1	Actual digital input signal read by the ECU (i.e., the sampled raw signal), but after necessary HW and/or SW filtering of “normal” disturbances.	M	O	1, 3
	1.2	Value currently available for the ECU´s application software (e.g., a filtered value used by the ECU in normal operation or the replacement value if the ECU is in limp-home operation).	MC	O	1

Output to internal HMI (e.g., LED display)	2	Output signal (the desired set point) controlled by the ECU´s application software or by the ECU´s diagnostic software (from the tester via a diagnostic service).	M	M	2
Other internal signals (e.g., from internal sensors or to internal actuators)	3	Specified by the implementer.	MC	MC	3

Abbreviations in the table above:
MC = A feasibility study is required. This shall be implemented if a technical solution is available and its cost is justified. The implementer is responsible for the feasibility study which shall be approved by the responsible authority appointed by OEM.
Notes in the table above:
1. The signal read by the diagnostic services shall be updated minimum each 100 ms.
2. It shall be possible to control all outputs individually and independently of each other unless there are special safety reasons not to do this. If any output is prohibited to be controlled due to such safety reason, this must be approved by the responsible authority appointed by OEM.
3. The ECU shall report the tester substituted input value to a diagnostic request to read the signal value while the input parameter value is under tester control.
4. Whether or not the session (in which the data record shall be supported) is all other sessions, or just one or some sessions, depends on which sessions the diagnostic services for read/write/control of the data is supported in.

•RID General Requirement

4.10.3.1.5 RID General Requirement

Routines shall be supported by the ECU according to the table below (e.g. in the valid identifier ranges). No other routines are allowed unless it is approved by the responsible authority appointed by OEM.

Table: Requirements on implementation of routines

Item	Routines			Session, see note 5		Notes
	Name	**Valid identifier range, hex**	**Definition**	**Programming**	**Other, see note 3**	
1	Vehicle Manufacturer Specific	0200 – DBFF	Routines are defined in *Global Master Reference Database*, UDS - Data - Requisite documents. Routines that shall be implemented are specified in section *Supported routines*	M	M	1
2	Development Specific	DC00 – DFFF	Routines defined by the implementer. Routines that are needed only during the development of the ECU, or for analysis of ECUs returned to the vehicle manufacturer, shall be defined in this range.	D	O	2
3	System Supplier Specific	F000 – FEFF	Specified by the system supplier.	D	O	2, 4
4	ISO specified	FF00-FF02	Routines are defined in reference *Road vehicles – Diagnostics systems – Diagnostic services, UDS - Data - External publications,*Routines that shall	M	D	

			be implemented are specified in section *Supported routines*			

Notes in the table above:

1. Routines defined in these identifier ranges shall be implemented exactly as they are defined in *Global Master Reference Database*.

2. The implementer shall not expect that OEM tools support the routines within this range.

3. Whether or not the supported session is all other sessions or just one or some depends on which session the diagnostic services for control and read of the routines is supported in.

4. Routines in this range shall comply with this requirements, but are not required to comply with requirements in *Unified diagnostic services (UDS) - Part 1: Specification and requirements - Services*, UDS - Data - Requisite documents.

5. Note that “M” means that the ECU shall support routines according to column “Definition” of the table which shall be defined within the routine identifier range according to column “valid identifier range”.

6.Note that “O” means that the ECU may support routine according to column “Definition” of the table which shall be defined within the identifier range according to column “valid display value range”.

•DTC General Requirement

4.10.3.1.6 DTC General Requirement

4.10.3.1.6.1 DTC Information

The requirements in this section are not applicable for programming session.

4.10.3.1.6.1.1 General

DTCs shall be supported by the ECU according to the table below (e.g. within the valid display value ranges). The implementation must use DTCs exactly as they are defined in *[Data_4] Global Master Reference Database*.

Table: Requirements on implementation of DTCs

DTC				**M/O, see note 1**
Name	**Item**	**Valid display value range See [Data_51] Diagnostic trouble code definitions , UDS - Data - External publications .**	**Definition**	
Powertrain (P) DTCs	1.1	P0000YY to P0FFFYY P2000YY to P2FFFYY P3400YY to P3FFFYY	DTCs defined in *[Data_51] Diagnostic trouble code definitions* , and *[Data_4] Global Master Reference Database .*	MC

			Byte YY is the DTCFailureTypeByte specified by *Diagnostic trouble code definitions* . The requirements on detection of the faults (emission related) that shall be identified by DTCs in this range are not specified by this document. The DTCs shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services*, and be reported not more than once in any response.	
	1.2	P1000YY to P1FFFYY P3000YY to P33FFYY	DTCs defined in *[Data_4] Global Master Reference Database* Byte YY is the DTCFailureTypeByte specified by *[Data_51] Diagnostic trouble code definitions* . The requirements on detection of the faults that shall be identified by DTCs in this range are specified in section *Detection of faults.* The DTCs shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported not more than once in any response.	M
Body (B), Chassis (C), Network and Miscellaneous (U) DTCs	2.1	B0000YY to B0FFFYY B3000YY to B3FFFYY C0000YY to C0FFFYY C3000YY to C3FFFYY U0000YY to U0FFFYY U3000YY to U3FFFYY	DTCs defined in *[Data_51] Diagnostic trouble code definitions ,* and *[Data_4] Global Master Reference Database* Byte YY is the DTCFailureTypeByte specified by *[Data_51] Diagnostic trouble code definitions.* The requirements on detection of the faults that shall be identified by DTCs in this range are specified in section *Detection of faults .* The DTCs shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported not more than once in any response.	M
	2.2	B1000YY to B2FFFYY C1000YY to C2FFFYY U1000YY to U28FFYY	DTCs defined in *[Data_4] Global Master Reference Database .* Byte YY is the DTCFailureTypeByte specified by *[Data_51] Diagnostic trouble code definitions.* The requirements on detection of the faults that shall be identified by DTCs in this range are specified in section *Detection of faults .* The DTCs shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported not more than once in any response.	M
	2.3	U2C00YY to U2CFFYY U2600YY to U26FFYY	DTCs defined in the *[Data_4] Global Master Reference Database .* Byte YY is the DTCFailureTypeByte specified by *[Data_51] Diagnostic trouble code definitions* , The faults that shall be identified by DTCs in this range are faults caused by signal or data value received from another ECU where the signal/data is indicated as invalid by the transmitting ECU. The requirements on detection of these faults are specified in section *Detection of faults caused by received signals.* The DTCs shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported not more than once in any response.	M
	2.4	U2D00YY to U2DFFYY U2700YY to U27FFYY	DTCs defined in *[Data_4] Global Master Reference Database .* Byte YY is the DTCFailureTypeByte specified by *[Data_51] Diagnostic trouble code definitions* , . The faults that shall be identified by DTCs in this range are faults caused by signal or data value received from	M

			another ECU where the signal/data is not indicated as invalid by the transmitting ECU. The requirements on detection of these faults are specified in section *Detection of faults caused by received signals.* The DTCs shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported not more than once in any response.	
	2.5	U2E00YY to U2EFFYY	DTCs defined in *[Data_4] Global Master Reference Database* . Byte YY is the DTCFailureTypeByte specified by [Data_51] *Diagnostic trouble code definitions* . The faults that shall be identified by DTCs in this range are faults caused by the ECU operating environment or by customer actions The requirements on detection of these faults are specified in section *Detection of faults caused by the ECU environment.* The DTCs shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported not more than once in any response.	M
	2.6	U2F00YY to U2FFFYY	DTCs defined by the implementer. DTCs that are needed only during the development or quality tracking of the ECU shall be specified in this range. The requirements on detection of the faults that shall be identified by DTCs in this range are specified by the implementer and in section *Detection of faults caused by signals and data values received from other ECUs.* The FDC10 shall increase with the value 1 when the test sample failed criteria is met for the DTCs in this range. The maximum value of the FDC10 shall be set to +126 (which means that the value DTC status bits, except the bits 4 and 6, will never be set, they will remain unchanged) for the DTCs in this range. The implementer shall not expect that OEM tools support the DTCs within this range.	O
	2.7	U2800YY to U280FYY	DTCs defined in Global Master Reference Database. DTCs that are needed only during the development of the ECU for DTC data verification purposes shall be specified in this range. Byte YY is the DTCFailureTypeByte specified by Diagnostic trouble code definitions. The pre-passed and pre-failed status of these DTCs shall be set by means of a dedicated Control Routine.	M
	2.8	U2810YY to U2BFFYY	DTCs defined in *Global Master Reference Database*. DTCs that are needed only during the development or quality tracking of the ECU shall be specified in this range. Byte YY is the DTCFailureTypeByte specified by Diagnostic trouble code definitions. The requirements on detection of the faults that shall be identified by DTCs in this range are specified in the following sections. The FDC10 shall increase with the value 1 when the test sample failed criteria is met for the DTCs in this range. The maximum value of the FDC10 shall be set to +126 (which means that the value DTC status bits, except the bits 4 and 6, will never be set, they will remain unchanged)	O

			for the DTCs in this range. The implementer shall not expect that OEM tools support the DTCs within this range.	

Abbreviations in the table above:
MC = Mandatory for emission related ECUs. Otherwise optional

Notes in the table above:
1. Note that “M” means that the ECU shall support DTCs according to column “Definition” of the table which shall be defined within the DTC range according to column “valid display value range”. Note that “O” means that the ECU may support DTCs according to column “Definition” of the table which shall be defined within the DTC range according to column “valid display value range

Requirements from this section are specified in section "Requirements from section DTC information - General".

4.10.3.1.6.1.1.1 Calibration of DTCs

The DTCs specified by this document shall be calibrated individually according to following generic requirements if not otherwise is specified by this document or is approved by the responsible authority appointed by OEM.

1. Calibration of DTC test sample failed
The DTC test sample failed criteria shall be calibrated in such a way that the following is met:
• The DTC test sample shall fail when the monitored item does not meet the design requirements specified by the implementer.
Examples:
1. The DTC test monitors an external circuit connected to the ECU and it runs periodically each 100 ms. The DTC test sample fail when the circuit is short to ground.
2. The DTC test monitors the maximum end-to-end timing of a signal sent on the in-vehicle network. The DTC test sample fail as soon as the maximum end-to-end timing of the received signal is exceeded.
Refer to section *Definitions and Abbreviations* for definition of DTC test sample.

2. Calibration of DTC test sample passed
The DTC test sample passed criteria shall be calibrated in such a way that the following is met:
• The DTC test sample shall pass when the monitored item meet the design requirements specified by the implementer.
Examples:
1. The DTC test monitors an external circuit connected to the ECU and it runs periodically each 100 ms. The DTC test sample pass when the circuit is not short to ground.
2. The DTC test monitors the maximum end-to-end timing of a signal sent on the in-vehicle network. The DTC test sample pass when the maximum end-to-end timing of the received signal is met.
Refer to section *Definitions and Abbreviations* for definition of DTC test sample.

3. Calibration of unconfirmedDTCLimit
The unconfirmedDTCLimit shall be calibrated in such a way that the following is met:
• The DTC fault detection counter (FDC10) shall reach a value that is equal to or greater than the unconfirmedDTCLimit when it is likely that at least some customers observer a symptom (caused by the detected fault) but probably only some of them is bothered by the symptom in such extent that they require repair of the vehicle.
Typically, it is very likely that the detected fault is intermittent (temporary), i.e. sporadic and transient, and that it occurs and causes customer symptom during only certain (perhaps unusual) vehicle operation and environment conditions.
Note that customer symptoms may be caused by actions (e.g. reconfigurations) taken by the ECU that tries mitigating the effect (e.g. on customer function, engine emissions or system safety) of the detected fault. If the detected fault causes a driver indication (e.g. telltale, text message, etc.), it shall be regarded as a customer symptom.
Refer to *General Diagnostic Guideline*, UDS DATA, Requisite Documents for recommendations and guidelines on how this requirement shall be met.

4. Calibration of DTCTestFailedLimit
The DTCTestFailedLimit shall be calibrated, i.e. the size of the DTC fault detection counter (FDC 10) step-up (increase) value shall be adjusted, in such a way that the following is met:
• The DTC fault detection counter (FDC10) shall reach a value that is equal to the DTCTestFailedLimit (i.e. +127) when it is very likely that the most customers observe a symptom (caused by the detected fault) but probably not all of them is bothered by the symptom in such extent that they require repair of the vehicle.
Note that customer symptoms may be caused by actions (e.g. reconfigurations) taken by the ECU that tries mitigating the effect (e.g. on customer function, engine emissions or system safety) of the detected fault. If the detected fault causes a driver indication (e.g. telltale, text message, etc.), it shall be regarded as a customer symptom.
Refer to *General Diagnostic Guideline*, UDS DATA, Requisite Documents for recommendations and guidelines on how this requirement shall be met.

5. Calibration of DTCTestPassedLimit
The DTCTestPassedLimit shall be calibrated, i.e. the size of the DTC fault detection counter (FDC 10) step-down (decrease) value shall be adjusted, in such a way that the following is met:
• The DTC fault detection counter (FDC10) shall reach a value that is equal to the DTCTestPassedLimit (i.e. -128) when the fault has not been detected for a while and the monitored object can currently be considered free from the fault.
Typically, the FDC 10 step-down value is equal or a bit less than the FDC 10 step-up value.
Refer to *General Diagnostic Guideline*, UDS DATA, Requisite Documents for recommendations and guidelines on how this requirement shall be met.

6. Calibration of confirmedDTCLimit
The confirmedDTCLimit shall be calibrated in such a way that the following is met:
• The DTC trip counter (see OCC6 definition) shall reach a value that is equal to the confirmedDTCLimit when it is very likely that that the most customers observe a symptom (caused by the detected fault) and is bothered by the symptom in such extent that they require repair of the vehicle.
Typically, it is very likely that the detected fault can be considered as permanent (lasting), i.e. repetitive and persistent, and that it occurs and causes customer symptom during the most vehicle operations and environmental conditions.
Note that customer symptoms may be caused by actions (e.g. reconfigurations) taken by the ECU that tries mitigating the effect (e.g. on customer function, engine emissions or system safety) of the detected fault. If the detected fault causes a driver indication (e.g. telltale, text message, etc.), it shall be regarded as a customer symptom.
Refer to *General Diagnostic Guideline*, UDS DATA, Requisite Documents for recommendations and guidelines on how this requirement shall be met.

7. Calibration of agedDTCLimit
The AgedDTCLimit shall be calibrated in such a way that the following is met:
• The DTC aging counter (OCC2 or OCC5) shall reach the agedDTCLimit when the fault has not been detected at such a long time that it probably will not occur and be detected again.
Typically, it is likely that the detected fault has been self-healed or detected by mistake.
Refer to *General Diagnostic Guideline*, UDS DATA, Requisite Documents for recommendations and guidelines on how this requirement shall be met.

Requirements from this section are specified in section "Requirements from section Calibration of DTCs".

4.10.3.1.6.1.2 DTC status bits

DTC status bits shall be supported for all DTCs supported by the ECU according to the table below.
The requirements for storing and clearing of the status bits are specified in section *Store DTC information* and *Clear DTC information*.
The stored DTC status byte (that contains the DTC status bits) shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services*.
Table : Requirements on implementation of DTC status bits

DTC status bits			M/O
Bit no See note 1	**Name**	**Definition**	
0 (SB0)	testFailed	See note 2 with the following addition: *-[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* allows for additional reset conditions to be defined by the vehicle manufacturer or implementer. No such additional reset condition of this bit is allowed. Note that the value of this bit shall not be reset to 0 at start of an operation cycle (the value from previous operation cycle shall be kept).	M
1 (SB1)	testFailedThisOperationCycle	See note 2.	M
2 (SB2)	pendingDTC	See note 2.	M
3 (SB3)	confirmedDTC	See note 2 with the following addition: - The bit shall be set to 1 when the trip counter (OCC6) reaches the confirmedDTCLimit. The value of confirmedDTCLimit is specified by the implementer according to the requirements specified in section *Generic requirements on calibration of DTCs*. The value shall be in the range of [1, +15]. Refer to section *DTC operation cycle counters* for definition of the OCC6. - The bit shall be reset to 0 when the DTC is aged. Refer to table *Requirements in implementation of clear of DTC information* in section *Clear DTC information* for when a DTC is aged. - The bit shall not be set to 1 and it shall be reset to 0, in case of the bit is already set to 1, if there are no aging counter (OCC2 or OCC5) allocated to the DTC due to the long term memory overflow. Refer to table *Requirements in implementation of clear of DTC information in section Clear DTC information* for definition of long term memory overflow.	M
4 (SB4)	testNotCompletedSinceLast-Clear	See note 2.	M
5 (SB5)	testFailedSinceLastClear	See note 2.	M
6 (SB6)	testNotCompletedThis-OperationCycle	See note 2.	M
7 (SB7)	warningIndicatorRequested	See note 2 with the following addition: *-[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* allows for additional set and reset conditions to be defined by the vehicle manufacturer or implementer. No such additional reset condition of this bit is allowed. Note that if the bit 3 (SB3) above cannot be set due to there is no aging counter (OCC2 or OCC5) allocated to the DTC then neither can this bit (SB7) be set to 1	M

Notes in the table above:
1. The number is the DTC status bit number of the DTC status byte associated with the DTC.
2. Specified in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services*, UDS - Data - External publications.

Requirements from this section are specified in section "Requirements from section DTC status bits".

4.10.3.1.6.1.2.1 Requirements from section DTC status bits

661846 v1 **Support for DTC status bit confirmedDTC**

Version history:
(1):

Purpose: To provide information about the status of an individual DTC to the test tool.
Source: Table "Requirements on implementation of DTC status bits"
Verification Method: Test
Legacy ID: 71827v3

The DTC status bit no 3, confirmedDTC, shall be supported for all DTCs supported by the ECU, as defined in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* , UDS - Data - External publications, with the following addition:
The bit shall be set to 1 when the the trip counter (see REQPROD 72035) reaches the confirmedDTCLimit.The value of the confirmedDTCLimit shall be in the range of [1, +15].
The bit shall be reset to 0 when the DTC is aged.

The bit shall not be set to 1 and it shall be set to 0, in case of the bit is already set to 1, if there are no aging counter (OCC2 or OCC5) allocated to the DTC due to the long term memory overflow.
(The status bit number is the bit number of the DTC status byte associated with the DTC.)

661845 v1 Support for DTC status bit pendingDTC

Version history:
(1):

Purpose: To provide information about the status of an individual DTC to the test tool.
Source: Table "Requirements on implementation of DTC status bits"
Verification Method: Test
Legacy ID: 71824v2

The DTC status bit no 2, pendingDTC, shall be supported for all DTCs supported by the ECU, as defined in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* , UDS - Data - External publications. (The status bit number is the bit number of the DTC status byte associated with the DTC).

661843 v1 Support for DTC status bit testFailed

Version history:
(1):

Purpose: To provide information about the status of an individual DTC to the test tool.
Source: Table "Requirements on implementation of DTC status bits"
Verification Method: Test
Legacy ID: 71815v1

The DTC status bit no 0, testFailed, shall be supported for all DTCs supported by the ECU, as defined in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* , UDS - Data - External publications , with the following exception:
No additional reset conditions are allowed to be defined by the implementer.

Note that the value of this bit shall not be reset to 0 at start of an operation cycle, the value from previous operation cycle shall be kept.

(The status bit number is the bit number of the DTC status byte associated with the DTC.)

661848 v1 Support for DTC status bit testFailedSinceLastClear

Version history:
(1):

Purpose: To provide information about the status of an individual DTC to the test tool.

Source: Table "Requirements on implementation of DTC status bits"
Verification Method: Test
Legacy ID: 71829v1

The DTC status bit no 5, testFailedSinceLastClear, shall be supported for all DTCs supported by the ECU, as defined in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* , UDS - Data - External publications.
(The status bit number is the bit number of the DTC status byte associated with the DTC.)

661844 v1 **Support for DTC status bit testFailedThisOperationCycle**

Version history:
(1):

Purpose: To provide information about the status of an individual DTC to the test tool.
Source: Table "Requirements on implementation of DTC status bits"
Verification Method: Test
Legacy ID: 71823v1

The DTC status bit no 1, testFailedThisOperationCycle, shall be supported for all DTCs supported by the ECU, as defined in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* , UDS - Data - External publications.
(The status bit number is the bit number of the DTC status byte associated with the DTC.)

661847 v1 **Support for DTC status bit testNotCompletedSinceLast-Clear**

Version history:
(1):

Purpose: To provide information about the status of an individual DTC to the test tool.
Source: Table "Requirements on implementation of DTC status bits"
Verification Method: Test
Legacy ID: 71828v1

The DTC status bit no 4, testNotCompletedSinceLast-Clear, shall be supported for all DTCs supported by the ECU, as defined in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* , UDS - Data - External publications.
(The status bit number is the bit number of the DTC status byte associated with the DTC.)

661849 v1 **Support for DTC status bit testNotCompletedThis-OperationCycle**

Version history:
(1):

Purpose: To provide information about the status of an individual DTC to the test tool.
Source: Table "Requirements on implementation of DTC status bits"
Verification Method: Test
Legacy ID: 71830v1

The DTC status bit no 6, testNotCompletedThis-OperationCycle, shall be supported for all DTCs supported by the ECU, as defined in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* , UDS - Data - External publications.
(The status bit number is the bit number of the DTC status byte associated with the DTC.)

661850 v1 **Support for DTC status bit warningIndicatorRequested**

Version history:
(1):

Purpose: To provide information about the status of an individual DTC to the test tool.
Source: Table "Requirements on implementation of DTC status bits"
Verification Method: Test
Legacy ID: 71831v1

The DTC status bit no 7, warningIndicatorRequested, shall be supported for all DTCs supported by the ECU, as defined in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* , UDS - Data - External publications , with the following exception:
No additional reset conditions are allowed to be defined by the implementer.
(The status bit number is the bit number of the DTC status byte associated with the DTC.)

4.10.3.1.6.1.3 DTC snapshot data

4.10.3.1.6.1.3.1 DTC snapshot data identification

The DTC snapshot data, consisting of both global and local DTC snapshot data, is identified by using data record identifiers.

Global DTC snapshot data consists of data that is distributed over the vehicle networks and is received by all Public ECUs. The data record identifiers for global DTC snapshot data are specified in *[Data_4] Global Master Reference Database* as detailed in chapter Global DTC snapshot data.

Local DTC snapshot data is defined separately for each ECU and can include data that is only known to the specific ECU.The data record identifiers for local DTC snapshot data are specified by the implementer as detailed in chapter Local DTC snapshot data.

Data records defined either as SystemSupplierSpecific or dynamically defined, according to table "Requirements on implementation of data records" found in section Record data - General, shall not be included in local DTC snapshot data.

If the data is sampled multiple times (record type = 'Multiple samples') before the sampling criteria is satisfied, then the sampled values shall be repeated a defined number of times in the snapshot data record. The order of the DTC snapshot data in the DTC snapshot record shall be such that the data which is sampled first is put at the start of the record (in bytes #1 to n) and the next data sample is appended to data from the first sample (bytes #n+1 to m), and so on.

See the example in the table below. See *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services, UDS - Data - External publications* for further details.

Table: Example of identification of DTC snapshot data

DTC hex	DTC snapshot	No. of samples	DTC snapshot data			
	record number, hex See section *Global DTC snapshot data*		Type	Data record type	Data record identifier hex	Definition
990022	20	N/A	Global	See note 1.	DD00	Global real time. See section *Global DTC snapshot data.*
					DD01	Total distance. See section *Global DTC snapshot data.*

					DD02	Vehicle battery voltage. See section *Global DTC snapshot data.*
					DD06	Vehicle speed. See section *Global DTC snapshot data.*
					DD07	GNSS Time. See section *Global DTC snapshot data.*
					DD0A	Usage mode. See section *Global DTC snapshot data.*
					DD0B	Partial Network Cluster (PNC). See section *Global DTC snapshot data*.
					DD0C	ElPowerLevel. See section *Global DTC snapshot data.*
			Local	See note 1.	A472	Left Hand Mirror Up Down Switch
					D901	Input from push buttons x3 to x10
					D902	Input from sensor x11
				Development specific	EE00	Output to motor y1 and y2
990122	20	N/A	Global	See note 1.	DD00 – DD02, DD06 - DD07, DD0A - DD0C	See above
			Local	See above	A472, D901, D902, EE00	See above
990322	20	N/A	Global	See note 1.	DD00 – DD02, DD06 - DD07, DD0A - DD0C	See above
			Local	See note 1.	D904	Input from sensor x2
					D905	Input from sensor x12
990422	20	N/A	Global	See note 1.	DD00 – DD02, DD06 - DD07, DD0A - DD0C	See above
			Local	See note 1.	D906	Input from temperature sensor x13
					D907	Input from switches x20 to x27
				Development specific	EE01	Output to motor y3
	30	5	Local	See note 1.	DD02	Vehicle battery voltage.
				See note 1.	D908	Input from sensor x15, sample no. 1 (first sample)
				See note 1.	D908	Input from sensor x15, sample no. 2
				See note 1.	D908	Input from sensor x15, sample no. 3
				See note 1.	D908	Input from sensor x15, sample no.5 (last sample)

Notes in the table above:
1. Specified in *[Data_4] Global Master Reference Database*.

Requirements from this section are specified in section "Requirements from section DTC snapshot dataidentification".

4.10.3.1.6.1.3.1.1 Requirements from section DTC snapshot data identification

661851 v1 **Data records not included in local snapshot data**

Version history:
(1):

Purpose: To ensure consistent storage of data records in local DTC snapshot data records.
Source: Section "DTC snapshot data identification"
Verification Method: Analysis
Legacy ID: 71837v0

Data records defined either as SystemSupplierSpecific or dynamically defined (as defined according to the table *Requirements on implementation of data records*) shall not be included in local DTC snapshot data.

661852 v1 **Order of multple samples of data records in global and local snapshot data records**

Version history:
(1):

Purpose: To ensure consistent storage of data records in local DTC snapshot data records
Source:
Verification Method: Test
Legacy ID: 115401v0

If data is sampled multiple times (record type = 'Multiple samples') before the sampling criteria is satisfied, then the sampled values shall be repeated a defined number of times in the snapshot data record. The order of the DTC snapshot data in the DTC snapshot record shall be such that the data which is sampled first is put at the start of the record (in bytes #1 to n) and the next data sample is appended to data from the first sample (bytes #n+1 to m), and so on.

4.10.3.1.6.1.3.2 DTC snapshot sampling criteria

DTC snapshot sampling criteria shall be supported for all DTCs supported by the ECU according to the table below.

The DTC snapshot data that shall be sampled and stored when the criteria is satisfied is specified in section *Global DTC snapshot data* and *Local DTC snapshot data.*

Table: Requirements on implementation DTC snapshot sampling criteria - AUTOSAR 4.0.x

DTC snapshot record			
Number, hex see note 1	**Snapshot Type see note 2**	**The criteria which triggers the sampling of data (the snapshot)**	**M/O**
00	Specified by [Data_50] Road vehicles – Diagnostics systems – Diagnostic services, UDS - Data - External publications.	Specified by *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services, UDS - Data - External publications*.	O

01-09	N/A	Reserved by this document for future use.	N/A
10-13	N/A	Reserved by this document for future use.	N/A
20 (SS20)	Single sample	Each time the FDC10 reaches a value that is equal to or greater than its unconfirmedDTCLimit until the FDC10 reaches the value +127 (since DTC information was last cleared) and each time the FDC10 reaches the value +127 until the DTC status bit 3 – confirmedDTC is set to 1 (since DTC information was last cleared). If FDC10 reaches a value that is equal to or greater than its unconfirmedDTCLimit and less than +127 more than once during an operation cycle then the sampling of data shall be triggered the first time only. If FDC10 reach the value +127 more than once during an operation cycle then the sampling of data shall be triggered the first time only. Only the last sampled DTC snapshot data shall kept in the memory (the previously sampled DTC snapshot data is overwritten).	M
21-2F	Single sample	Reserved by this document for future use.	N/A
30-3F (SS 30-3F)	Multiple samples	Specified by the implementer. The DTC snapshot record shall only consist of DTC snapshot data that is needed during the development of the ECU. The implementer shall not expect that OEM tools support the DTC snapshot record.	O
40-4F (SS 40-4F)	Single sample	Specified by the implementer. The DTC snapshot record shall only consist of DTC snapshot data that is needed during the development of the ECU. The implementer shall not expect that OEM tools support the DTC snapshot record.	O
50-5F (SS 50-5F)	Other samples	Specified by the implementer. The DTC snapshot record shall only consist of DTC snapshot data that is needed during the development of the ECU. The implementer shall not expect that OEM tools support the DTC snapshot record.	O
60-FF	N/A	Reserved by this document for future use.	N/A
FF	N/A	Refer to *[Data_1] UDS Services*	M

Table: Requirements on implementation DTC snapshot sampling criteria - AUTOSAR 4.1 or newer

DTC snapshot record			
Number, hex see note 1	**Snapshot Type see note 2**	**The criteria which triggers the sampling of data (the snapshot)**	**M/O**
00	Specified by [Data_50] Road vehicles – Diagnostics systems – Diagnostic services, UDS - Data - External publications.	Specified by [Data_50] Road vehicles – Diagnostics systems – Diagnostic services, UDS - Data - External publications.	O
01-09	N/A	Reserved by this document for future use.	N/A
10-13	N/A	Reserved by this document for future use.	N/A
20 (SS20)	Single sample	When the DTC status bit 3 – confirmedDTC is set to 1 (since DTC information was last cleared).	M
21 (SS21)	Single Sample	The first time that the FDC10 reaches a value that is equal to or greater than its unconfirmedDTCLimit. (since DTC information was last cleared).	M
22-2F	Single sample	Reserved by this document for future use.	N/A
30-3F (SS 30-3F)	Multiple samples	Specified by the implementer. The DTC snapshot record shall only consist of DTC snapshot data that is needed during the development of the ECU. The implementer shall not expect that OEM tools support the DTC snapshot record.	O

40-4F (SS 40-4F)	Single sample	Specified by the implementer. The DTC snapshot record shall only consist of DTC snapshot data that is needed during the development of the ECU. The implementer shall not expect that OEM tools support the DTC snapshot record.	O
50-5F (SS 50-5F)	Other samples	Specified by the implementer. The DTC snapshot record shall only consist of DTC snapshot data that is needed during the development of the ECU. The implementer shall not expect that OEM tools support the DTC snapshot record.	O
60-FF	N/A	Reserved by this document for future use.	N/A
FF	N/A	Refer to *[Data_1] UDS Services*	N/A

Notes in the tables above:

1. The number is the DTCSnapshotRecordNumber that identifies the failure criteria which triggers the sampling and storage of DTC snapshot data. The text within brackets is the short name of the DTC snapshot record.

2 Single sample: The DTC snapshot data shall be sampled (and stored) once when the criteria is satisfied. Multiple samples: The DTC snapshot data shall be sampled periodically. A number of samplers before and/or after the criteria are satisfied shall be stored (the number of samples, and the periodicity, shall be defined by the implementer).

Requirements from this section are specified in section "Requirements from section DTC snapshot sampling criteria".

4.10.3.1.6.1.3.2.1 Requirements from section DTC snapshot sampling criteria

661853 v1 **Sampling criteria for DTCSnapshotRecordNumber 0x20, single sample**

Version history:
(1):

Purpose: To define the critera for sampling of snapshot data when the DTC has reached its unconfirmed limit.
Source: Table "Requirements on implementation DTC snapshot sampling criteria"
Verification Method: Test
Legacy ID: 72012v4

When implementing with AUTOSAR 4.0.x:

For all DTCs supported by the ECU DTC snapshot data shall be sampled and stored with DTCSnapshotRecordNumber 0x20 when the following criteria is satisfied for the data associated with the DTC: Each time the FDC10 reaches a value that is equal to or greater than its unconfirmedDTCLimit until the FDC10 reaches the value +127 (since DTC information was last cleared) and each time the FDC10 reaches the value +127 until the DTC staus bit 3 – confirmedDTC is set to 1 (since DTC information was last cleared).

If FDC10 reaches a value that is equal to or greater than its unconfirmedDTCLimit and less than +127 more than once during an operation cycle then the sampling of data shall be triggered the first time only. If FDC10 reaches the value +127 more than once during an operation cycle then the sampling of data shall be triggered the first time only.

Only the last sampled DTC snapshot data shall kept in the memory (the previously sampled DTC snapshot data is overwritten).

When implementing with AUTOSAR 4.1 or newer:

For all DTCs supported by the ECU DTC snapshot data shall be sampled and stored with

DTCSnapshotRecordNumber 0x20 when the following criteria is satisfied for the data associated with the DTC:

When the DTC status bit 3 – confirmedDTC is set to 1 (since DTC information was last cleared).

661854 v1 **Sampling criteria for DTCSnapshotRecordNumber 0x21, single sample**

Version history:
(1):

Purpose: To define the criteria for sampling of snapshot data record 21.
Source:
Verification Method: Test
Legacy ID: 441094v0

When implementing with AUTOSAR 4.0x:
This requirement is Not Applicable

When implementing with AUTOSAR 4.1 or newer:
For all DTCs supported by the ECU DTC snapshot data shall be sampled and stored with DTCSnapshotRecordNumber 0x21 when the following criteria is satisfied for the data associated with the DTC:

The first time that the FDC10 reaches a value that is equal to or greater than its unconfirmedDTCLimit (since DTC information was last cleared).

661855 v1 **Sampling criteria for DTCSnapshotRecordNumber 0x30 to 0x3F, multiple samples**

Version history:
(1):

Purpose: To define the critera for sampling of snapshot data when the DTC has reached its unconfirmed limit.
Source:
Verification Method: Test
Legacy ID: 115421v1

For DTCs supported by the ECU DTC snapshot data may be sampled and stored with DTCSnapshotRecordNumber in the range 0x30 to 0x3F according to the following:
The DTC snapshot data shall be sampled periodically. A number of samplers before and/or after the criteria are satisfied shall be stored (the number of samples, and the periodicity, shall be defined by the implementer). The sampling critera is specified by the implementer. The DTC snapshot record shall only consist of DTC snapshot data that is needed during the development of the ECU. The implementer shall not expect that OEM tools support the DTC snapshot record.

661856 v1 **Sampling criteria for DTCSnapshotRecordNumber 0x40 to 0x4F, single sample**

Version history:

(1):

Purpose: To define the critera for sampling of snapshot data.
Source: Table: Requirements on implementation DTC snapshot sampling criteria
Verification Method: Test
Legacy ID: 336996v0

For DTCs supported by the ECU DTC snapshot data may be sampled and stored with DTCSnapshotRecordNumber in the range 0x40 to 0x4F according to the following:
The DTC snapshot data shall be sampled (and stored) once when the criteria is satisfied. The sampling criteria is specified by the implementer. The DTC snapshot record shall only consist of DTC snapshot data that is needed during the development of the ECU. The implementer shall not expect that OEM tools support the DTC snapshot record.

661857 v1 **Sampling criteria for DTCSnapshotRecordNumber 0x50 to 0x5F, other samples**

Version history:
(1):

Purpose: To define the critera for sampling of snapshot data.
Source: Table: Requirements on implementation DTC snapshot sampling criteria
Verification Method: Test
Legacy ID: 336997v0

For DTCs supported by the ECU DTC snapshot data may be sampled and stored with DTCSnapshotRecordNumber in the range 0x50 to 0x5F according to the following:
The sampling critera is specified by the implementer. The DTC snapshot record shall only consist of DTC snapshot data that is needed during the development of the ECU. The implementer shall not expect that OEM tools support the DTC snapshot record.

4.10.3.1.6.1.3.3 Global DTC snapshot data

Global DTC snapshot data shall be supported for all DTCs supported by the ECU according to the table below.
The data records defined in this table must be implemented exactly as they are defined in *[Data_4] Global Master Reference Database*.

The requirements for storing and clearing of the DTC snapshot data are specified in sections *Store DTC information* and *Clear DTC information.*

The criteria that trigger the sampling and storage of DTC snapshot data are specified in section *DTC snapshot sampling criteria* .

If DTC snapshot data received from the vehicle network is not available when the sampling criteria is fulfilled, the latest received value shall be used and stored as DTC snapshot data. If no value has been received in the current operation cycle a value representing "correct value not available" shall be used and stored as DTC snapshot data.

Table: Requirements on implementation of global DTC snapshot data

Global DTC snapshot data			**M/O**
Data record identifier, hex	**Name**	**Definition**	
DD00	Global real time	Data record defined in *[Data_4] Global Master Reference Database* with the following additional requirements that are only applicable to the ECU responsible for maintaining and distributing the global real time data: - range 0 to 13.6 year / 0x00000000 to 0xFFFFFFFE, correct value not available = 0xFFFFFFFF	MC1

		- resolution 1 bit = 100 ms. - accuracy minimum ±10 minutes/week (±0.1%). - after the global time have reached 0xFFFFFFFE it shall restart from 0x00000000. - the current value of global real time shall be stored in long term memory at least once each operation cycle. Note: The sender should store the time in long term once by an hour, store this when usage mode changes and store this before sleep. - If the ECU loses the current value due to loss of power supply then, following reconnection of the power supply, it shall continue to count the global real time beginning from the stored value. The stored DTC snapshot data shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services*.	
DD01	Total distance	Data record defined in *[Data_4] Global Master Reference Database* with the following additional requirements that are only applicable to the ECU responsible for maintaining and distributing the total distance data: - range minimum 0 to 2 000 000 km / 0x000000 to 0x1E8480, correct value not available = 0xFFFFFF - resolution 1 bit = 1 km - the accuracy shall be the same as or better than the accuracy of the total distance displayed in the dash board. The stored DTC snapshot data shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services*.	M
DD02	Vehicle battery voltage	Data record defined in *[Data_4] Global Master Reference Database* with the following additional requirements that are only applicable to the ECU responsible for measuring and distributing the vehicle battery voltage: - range minimum 0 to 25 V / 0x00 to 0x64, correct value not available = 0xFF - resolution 1 bit = 0.25 V - see section *Detection of faults in external electrical circuits* The stored DTC snapshot data shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services*.	M
DD06	Vehicle speed	ECU received vehicle speed signal "VehSpdLgt" as DID DD06 value. The Vehicle Speed Time: - range minimum 0 to 125.0027m/s / 0x00 to 0x7CE2, correct value not available = 0xFFFF. - resolution 1 bit = 0.00391 m/s. The stored DTC snapshot data shall be reported DD06.	M
DD07	GNSS Time	ECU received GNSS time signal "TiAndDateIndn" as DID DD07 value. The GNSS Time: - Year range 0 to 99/0x0 to 0x63, correct value not available = 0xFF. - Month range 1 to 12/0x1 to 0xC, correct value not available = 0xFF. - Day range 1 to 31/0x1 to 0x1F, correct value not available = 0xFF. - Hour range 0 to 23/0x0 to 0x17, correct value not available = 0xFF. - Minute range 0 to 59/0x0 to 0x3B, correct value not available = 0xFF. - Second range 0 to 59/0x0 to 0x3B, correct value not available = 0xFF. - the current value of GNSS Time shall be stored in long term memory at least once each operation cycle. - If the ECU loses the current value due to loss of power supply then, following reconnection of the power supply, it shall continue to count the GNSS Time beginning from the stored value.	M

		The stored DTC snapshot data shall be reported DD07.	
DD0A	Usage mode	Data record defined by *[Data_4] Global Master Reference Database*. The stored DTC snapshot data shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services*.	M
DD0B	Partial Network Cluster (PNC)	Data record defined by reference *[Data_4] Global Master Reference Database*. The stored DTC snapshot data shall be reported on request of the diagnostic services specified in reference *[Data_1] UDS Services*.	MC2
DD0C	ElPowerLevel	Data record defined by reference *[Data_4] Global Master Reference Database*. The stored DTC snapshot data shall be reported on request of the diagnostic services specified in reference *[Data_1] UDS Services*.	M

Abbreviations in the table above:
MC1 = Mandatory if the ECU is responsible for maintaining and distributing the real time data.
MC2 = Mandatory if the ECU is the Vehicle Mode Management (VMM) master.

Global DTC snapshot data (see the table above) shall be transmitted to all public ECUs by an ECU (one or more) responsible for maintaining and distributing the data according to the following requirements:

- The ECU shall transmit current values of the data periodically each second on the network except the data record 0xDD02 – "Vehicle battery voltage" which shall be sent faster according to the requirements specified in section *Additional requirements for detection of low supply voltage.*
- The ECU shall transmit the data with a maximum age of 250 ms, except the data record 0xDD02 – "Vehicle battery voltage" which shall be sent with a lower maximim age according to the requirements specified in section *Additional requirements for detection of low supply voltage*.

4.10.3.1.6.1.3.3.1 Requirements from section Global DTC snapshot data

667096 v1 **Definition of GNSS time**

Version history:
(1):

Purpose: Ensure that the GNSS time data has a useful resolution and accuracy. And to ensure consistency in the distributed global real time value after power loss.
Source: Table "Requirements on implementation of global DTC snapshot data".
Verification Method: Test
Legacy ID:

The ECU responsible for maintaining and distributing the GNSS time data shall store the current value of GNSS time in long term memory at least once each operation cycle.

ECU received GNSS time signal "TiAndDateIndn" as DID DD07 value.
The GNSS Time:
- Year range 0 to 99/0x0 to 0x63, correct value not available = 0xFF.
- Month range 1 to 12/0x1 to 0xC, correct value not available = 0xFF.
- Day range 1 to 31/0x1 to 0x1F, correct value not available = 0xFF.
- Hour range 0 to 23/0x0 to 0x17, correct value not available = 0xFF.
- Minute range 0 to 59/0x0 to 0x3B, correct value not available = 0xFF.

- Second range 0 to 59/0x0 to 0x3B, correct value not available = 0xFF.

- the current value of GNSS Time shall be stored in long term memory at least once each operation cycle.
- If the ECU loses the current value due to loss of power supply then, following reconnection of the power supply, it shall continue to count the GNSS Time beginning from the stored value.

The stored DTC snapshot data shall be reported DD07.

661859 v2 Definition of global real time data record

Version history:
(2):

Purpose: Ensure that the global real time data has a useful resolution and accuracy.
Source: Table "Requirements on implementation of global DTC snapshot data"
Verification Method: Analysis
Legacy ID: 72018v1

The ECU responsible for maintaining and distributing the real time data shall maintain a global time data record with the following
range, resolution and accuracy:

range 0x00000000 to 0xFFFFFFFE
resolution 1 bit = 100 ms
accuracy minimum ±10 minutes/week (±0.1%)
correct value not available = 0xFFFFFFFF
after the global time have reached 0xFFFFFFFE it shall restart from 0x00000000

661861 v2 Definition of total distance data record

Version history:
(2): Total distance最大值错误更新，调整为0x1E8480（2000000）。

Purpose: Ensure that the total distance data has a useful resolution and accuracy.
Source: Table "Requirements on implementation of global DTC snapshot data"
Verification Method: Analysis
Legacy ID: 72020v2

The ECU responsible for maintaining and distributing the total distance data shall maintain a total distance data record with the following range, resolution and accuracy:

range minimum 0x000000 to 0x1E8480.
resolution 1 bit = 1 km
the accuracy shall be the same as or better than the accuracy of the total distance displayed in the dash board
correct value not available = 0xFFFFFF

661862 v1 Definition of vehicle battery voltage data record

Version history:
(1):

Purpose: Ensure that the vehicle battery voltage data has a useful resolution and accuracy.

Source: Table "Requirements on implementation of global DTC snapshot data"
Verification Method: Analysis
Legacy ID: 72021v1

The ECU responsible for maintaining and distributing the vehicle battery voltage data shall maintain a vehicle battery voltage data record with the following range, resolution and accuracy:

range minimum / 0x00 to 0x60
resolution 1 bit = 0.25 Volt
the accuracy shall be according to section *Requirements for detection of low supply voltage.*
correct value not available = 0xFF

676288 v2 Definition of vehicle speed

Version history:
(2): 错误修订：correct value not available修改为oxFFFF。

Purpose: Ensure the freezing of vehicle speed information when a malfunction occurs.
Source: Table "Requirements on implementation of global DTC snapshot data".
Verification Method: Test
Legacy ID:

ECU received vehicle speed signal "VehSpdLgt" as DID DD06 value.
The Vehicle Speed Time:
- range minimum 0 to 125.0027m/s / 0x00 to 0x7CE2, correct value not available = 0xFFFF.
- resolution 1 bit = 0.00391 m/s.

The stored DTC snapshot data shall be reported DD06.

661863 v1 Distribution of global DTC snapshot data

Version history:
(1):

Purpose: To ensure that global snaphsot data that is sampled and stored in each ECU is not too old.
Source: Section "Global DTC snapshot data"
Verification Method: Analysis
Legacy ID: 72024v1

Global DTC snapshot data shall be transmitted to all public ECUs by an ECU (one or more) responsible for maintaining and distributing the data according to the following requirements:
- The ECU shall transmit current values of the data periodically each second on the network except the data record 0xDD02 – "Vehicle battery voltage" which shall be sent faster according to the requirements specified in section Additional requirements for detection of low supply voltage.
- The ECU shall transmit the data with a maximum age of 250 ms, except the data record 0xDD02 – "Vehicle battery voltage" which shall be sent with a lower maximim age according to the requirements specified in section Additional requirements for detection of low supply voltage.

661860 v2 Maintaining global real time after power loss

Version history:
(2): 增加对于Global real time的存储频率要求。

Purpose: To ensure consistency in the distributed global real time value after power loss.
Source: Table "Requirements on implementation of global DTC snapshot data"
Verification Method: Analysis
Legacy ID: 72019v1

The ECU responsible for maintaining and distributing the real time data shall store the current value of global real time in long term memory at least once each operation cycle. If the ECU loses power supply then, following reconnection of the power supply, it shall continue to count the global real time beginning from the stored value.

Note: The sender should store the time in long term once by an hour, store this when usage mode changes and store this before sleep.

661864 v2 Storage of global DTC snapshot data when not available

Version history:
(2): 除了没有接收到数值，超出范围值或不可靠的数值也被表征为correct value not available。

Purpose: If data received from the network is not availabel at the sampling moment the last received value will be the most useful to store.
Source: Section "Global DTC snapshot data"
Verification Method: Test
Legacy ID: 72026v2

If global DTC snapshot data received from the vehicle network is not available when the sampling criteria is fulfilled, the latest received value shall be used and stored as DTC snapshot data.
If no value, or out-of-range value, or unreliable value has been received in the current operation cycle a value representing "correct value not available", according to specified in table: *Requirements on implementation of global DTC snapshot data*, shall be used and stored as DTC snapshot data.

661858 v3 Supported data records in global DTC snapshot data

Version history:
(3): 更新需求，全局DTC快照数据可选择的包含DID DDoo，不强制。

Purpose: To enhance fault tracing and root cause analysis specific vehicle global data shall be stored with each DTC.
Source: Table "Requirements on implementation of global DTC snapshot data".
Verification Method: Analysis
Legacy ID: 72017v8

For all DTCs supported by the ECU the global DTC snapshot data records shall include the following data records, as defined in reference *[Data_4] Global Master Reference Database* and the stored data records shall be reported on request of the diagnostic services specified in reference *[Data_1] UDS Services*:

0xDD00 Global real time. See note 2
0xDD01 Total distance.

0xDD02 Vehicle battery voltage.
0xDD06 Vehicle speed.
0xDD07 GNSS Time.
0xDD0A Usage mode.
0xDD0B Partial Network Cluster (PNC). See note 1
0xDD0C ElPowerLevel.

Note 1:
This DID is not mandatory for all ECU's. It shall only be included in the Global DTC snapshot data when the ECU is required to implement this DID.

Note 2:
This DID is not mandatory for all ECUs. It is optional for being included in the Global DTC snapshot data when the ECU is required to implement this DID.

4.10.3.1.6.1.3.4 Local DTC snapshot data

Local DTC snapshot data shall be supported by the ECU according to the table below :

The data record identifiers shall be defined within the range of identifiers specified for these data records in table "Requirements on implementation of data records", section *Record data - General* .
The requirements for storing and clearing of the DTC snapshot data are specified in sections *Store DTC information* and *Clear DTC information*.
The criteria that trigger the sampling and storage of DTC snapshot data are specified in section *DTC snapshot sampling criteria.*

If DTC snapshot data received from the vehicle network is not available when the sampling criteria is fulfilled, the latest received value shall be used and stored as DTC snapshot data. If no value has been received in the current operation cycle a value representing "correct value not available" shall be used and stored as DTC snapshot data

Table: Requirements on implementation of local DTC snapshot data

Local DTC snapshot data			**M/O**
Data record identifier, hex	**Name**	**Definition**	
Defined by the implementer	Defined by the implementer	Local DTC snapshot data is specified by the implementer. The data shall be chosen so that it will be useful in identification of the conditions (e.g. operation state and surrounding environment of the ECU) which are present whenever a fault (and the fault symptom) occurs. The local DTC snapshot data may be specific for each DTC (e.g. different data for different DTCs). The stored DTC snapshot data shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services*.	MC

Abbreviations in the table:
MC = A feasibility study is required. This shall be implemented if a technical solution is available and its cost is justified. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM.

4.10.3.1.6.1.3.4.1 Requirements from section Local DTC snapshot data

661866 v1 **Storage of local DTC snapshot data when not available**

Version history:

(1):

Purpose: If data received from the network is not availabel at the sampling moment the last received value will be the most useful to store.
Source: Section "Local DTC snapshot data"
Verification Method: Analysis
Legacy ID: 72028v0

If local DTC snapshot data received from the vehicle network is not available when the sampling criteria is fulfilled, the latest received value shall be used and stored as DTC snapshot data.
If no value has been received in the current operation cycle a value representing "correct value not available" shall be used and stored as DTC snapshot data.

661865 v1 **Support for local DTC snapshot data**

Version history:
(1):

Purpose: To enhance fault tracing and root cause analysis specific ECU local data shall be stored with each DTC.
Source: Table "Requirements on implementation of local DTC snapshot data"
Verification Method: Analysis
Legacy ID: 72027v1

For all DTCs supported by the ECU local DTC snapshot data shall be implemented as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the DTC snapshot data to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM). The stored local DTC snapshot data records shall be reported on request of the diagnostic services specified in reference *[Data_1] UDS Services*.

4.10.3.1.6.1.4 DTC extended data

4.10.3.1.6.1.4.1 DTC operation cycle counters

DTC operation cycle counters shall be supported for all DTCs supported by the ECU according to the table below :

The requirements for storing and clearing of the counters are specified in sections *Store DTC information* and *Clear DTC information.*

Table : Requirements on implementation of DTC operation cycle counters - AUTOSAR 4.0.x

DTC extended data record			**M/O**
Number hex, see note 1	**Name**	**Definition**	
01	DTC operation cycle counter #1 (OCC1)	Definition: Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 last reached a value that is equal to or greater than its unconfirmedDTCLimit or more (since DTC information was last cleared). All operation cycles, including those during which the test was not passed shall be included. Increment/clear criteria: The counter shall be incremented at the beginning of each operation cycle.	M

		The counter shall be cleared as soon as FDC10 reaches its unconfirmedDTCLimit. If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	
02	DTC operation cycle counter #2 (OCC2)	Definition: Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 last reached a value that is equal or greater than its unconfirmedDTCLimit (since DTC information was last cleared). The operation cycles during which the test was not passed shall be excluded. Increment/clear criteria: In each operation cycle the counter shall be incremented after completing an operation cycle in which the FDC10 reaches the value -128, unless FDC10 has already reached a value that is equal to or greater than its unconfirmedDTCLimit in the current operation cycle. The counter shall be cleared as soon as FDC10 reaches its unconfirmedDTCLimit. If any count operation occurs which would cause a couter to roll over past 0xFF then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	M
03	DTC operation cycle counter #3 (OCC3)	Definition: Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 first reached a value that is equal to or greater than its unconfirmedDTCLimit (since DTC information was last cleared). All operation cycles, including those in which the test has not been passed or the DTC fault detection counter 10 has not reach a value that is equal to or greater than its unconfirmedDTCLimit shall be included. Increment/clear criteria: The counter shall be incremented at the beginning of each operation cycle following the operation cycle where FDC10 first reached a value that is equal to or greater than its unconfirmedDTCLimit. If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	M
04	DTC operation cycle counter #4 (OCC4)	Definition: Number of operation cycles (including the current operation cycle) during which the DTC fault detection counter 10 reached a value that is equal to or greater than its unconfirmedDTCLimit (since DTC information was last cleared). Increment/clear criteria: In each operation cycle the counter shall be incremented as soon as FDC10 has reached a value that is equal to or greater than its unconfirmedDTCLimit for the first time. If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.	M

		Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	
05	DTC operation cycle counter #5 (OCC5)/Warm-up cycle counter	Definition: Number of warm-up cycles since the DTC commanded the MIL to switch off (since DTC information was last cleared). The warm-up cycle is defined in *[Data_52] OBD II regulation, section 1968.2 of title 13, California Code of Regulations*, UDS - Data - External publications. Increment/clear criteria: The counter shall be incremented when WUC changes from false to true. The counter shall be cleared when the DTC fault detection counter 10 reach the value of +127. If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte.	MC
06	DTC operation cycle counter #6 (OCC6)	Definition: The number of consecutive operation cycles (including the current operation cycle) during which the DTC fault detection counter 10 reached the value of +127 (since DTC information was last cleared). The operation cycles during which the test was not completed shall be excluded. The minimum range of the counter is 0-15. Increment/clear criteria: In each operation cycle the counter shall be incremented as soon as FDC10 has reached the value +127 for the first time. The counter shall be cleared in the beginning of the operation cycle following an operation cycle where FDC10 did not reach the value +127 at any time but -128 (the counter shall maintain its value if did not reach the value -128). The counter may also be cleared when the confirmedDTCLimit is reached. If any count operation occurs which would cause a counter to roll over past 0xFF, then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	M
07	DTC o peration cycle counter #7 (OCC7)	Definition: Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 first reached a value that is equal to or greater than its unconfirmedDTCLimit (since DTC information was last cleared). The operation cycles during which the test was not passed or the DTC fault detection counter 10 did not reach a value that is equal to or greater than its unconfirmedDTCLimit shall be excluded. Increment/clear criteria: The counter shall be incremented in each operation cycle, following the operation cycle where FDC10 first reached a value that is equal to or greater than its unconfirmedDTCLimit, as soon as the following criteria is meet: The FDC10 reach -128 or a value that is equal to or greater than its unconfirmedDTCLimit. If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.	M

		Reporting: The stored counter shall be reported on request of the diagnostic services specified in reference *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	

Table: Requirements on implementation of DTC operation cycle counters - AUTOSAR 4.1 or newer

DTC extended data record			**M/O**
Number hex, see note 1	**Name**	**Definition**	
01	DTC operation cycle counter #1 (OCC1)	Definition: Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 last reached the value +127 (since DTC information was last cleared). All operation cycles, including those during which the test was not passed shall be included. Increment/clear criteria: The counter shall be incremented at the beginning of each operation cycle. The counter shall be cleared as soon as FDC10 reaches the value +127. If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	M
02	DTC operation cycle counter #2 (OCC2)	Definition: Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 last reached the value +127 (since DTC information was last cleared). The operation cycles during which the test was not passed shall be excluded. Increment/clear criteria: In each operation cycle the counter shall be incremented after completing an operation cycle in which the FDC10 reaches the value -128, unless FDC10 has already reached the value +127 in the current operation cycle. The counter shall be cleared as soon as FDC10 reaches the value +127. If any count operation occurs which would cause a couter to roll over past 0xFF then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	M
03	DTC operation cycle counter #3 (OCC3)	Definition: Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 first reached the value +127 (since DTC information was last cleared). All operation cycles, including those in which the test has not been passed or the DTC fault detection counter 10 has not reached the value +127 shall be included. Increment/clear criteria: The counter shall be incremented at the beginning of each operation cycle following the operation cycle where FDC10 first reached the value +127.	M

		If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	
04	DTC operation cycle counter #4 (OCC4)	Definition: Number of operation cycles (including the current operation cycle) during which the DTC fault detection counter 10 reached the value +127 (since DTC information was last cleared). Increment/clear criteria: In each operation cycle the counter shall be incremented as soon as FDC10 has reached the value +127 for the first time. If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	M
05	DTC operation cycle counter #5 (OCC5)/Warm-up cycle counter	Definition: Number of warm-up cycles since the DTC commanded the MIL to switch off (since DTC information was last cleared). The warm-up cycle is defined in *[Data_52] OBD II regulation, section 1968.2 of title 13, California Code of Regulations*, UDS - Data - External publications. Increment/clear criteria: The counter shall be incremented when WUC changes from false to true. The counter shall be cleared when the DTC fault detection counter 10 reach the value of +127. If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services,* and be reported as 1 byte.	MC
06	DTC operation cycle counter #6 (OCC6)	Definition: The number of consecutive operation cycles (including the current operation cycle) during which the DTC fault detection counter 10 reached the value of +127 (since DTC information was last cleared). The operation cycles during which the test was not completed shall be excluded. Increment/clear criteria: In each operation cycle the counter shall be incremented as soon as FDC10 has reached the value +127 for the first time. The counter shall be cleared in the beginning of the operation cycle following an operation cycle where FDC10 did not reach the value +127 at any time but -128 (the counter shall maintain its value if did not reach the value -128). The counter may also be cleared when the confirmedDTCLimit is reached. If any count operation occurs which would cause a counter to roll over past 0xFF, then the count value shall instead be maintained at 0xFF. Reporting:	O

		The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	

Abbreviations in the table above:
MC = Mandatory for emission related DTCs supported by the ECU.

Notes in the table above:
1. The number is the DTCExtendedDataRecordNumber.

Requirements from this section are specified in section "Requirements from section DTC operation cycle counters".

4.10.3.1.6.1.4.1.1 Requirements from section DTC operation cycle counters

661867 v1 **Support for Operation Cycle Counter #1**

Version history:
(1):

Purpose: To provide enhanced information about the occurence of a DTC that may be useful in the analysis of the fault.
Source: Table "Requirements on implementation of DTC operation cycle counters"
Verification Method: Test
Legacy ID: 72030v3

For all DTCs supported by the ECU a counter identified by DTCExtendedDataRecordNumber=1 shall be implemented according to the following:

When implementing with AUTOSAR 4.0.x:
Definition:
Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 last reached a value that is equal to or greater than its unconfirmedDTCLimit or more (since DTC information was last cleared). All operation cycles, including those during which the test was not passed shall be included.

Increment/clear criteria:
The counter shall be incremented at the beginning of each operation cycle.
The counter shall be cleared as soon as FDC10 reaches a value that is equal to or greater than its unconfirmedDTCLimit.
If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.

Reporting:
The stored counter shall be reported as a 1 byte unsigned numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services.*

When implementing with AUTOSAR 4.1 or newer:
Definition:
Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 last reached the value +127 (since DTC information was last cleared). All operation cycles, including those during which the test was not passed shall be included.

Increment/clear criteria:
The counter shall be incremented at the beginning of each operation cycle.
The counter shall be cleared as soon as FDC10 reaches the value +127.
If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.

Reporting:
The stored counter shall be reported as a 1 byte unsigned numeric value on request of the diagnostic services

specified in reference *[Data_1] UDS Services*.

661868 v1 **Support for Operation Cycle Counter #2**

Version history:
(1):

Purpose: To provide enhanced information about the occurence of a DTC that may be useful in the analysis of the fault.
Source: Table "Requirements on implementation of DTC operation cycle counters"
Verification Method: Test
Legacy ID: 72031v4

For all DTCs supported by the ECU a counter identified by DTCExtendedDataRecordNumber=2 shall be implemented according to the following:

When implementing with AUTOSAR 4.0.x:
Definition:
Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 last reached a value that is equal to or greater than its unconfirmedDTCLimit (since DTC information was last cleared). The operation cycles during which the test was not passed shall be excluded.

Increment/clear criteria:
In each operation cycle the counter shall be incremented after completing an operation cycle in which FDC10 reachesthe value -128, unless FDC10 has already reached a value that is equal to or greater than its unconfirmedDTCLimit in the current operation cycle.
The counter shall be cleared as soon as FDC10 reaches a value that is equal to or greater than its unconfirmedDTCLimit.
If any count operation occurs which would cause a couter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.

Reporting:
The stored counter shall be reported as a 1 byte unsigned numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services*.

When implementing with AUTOSAR 4.1 or newer:
Definition:
Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 last reached the value +127 (since DTC information was last cleared). The operation cycles during which the test was not passed shall be excluded.

Increment/clear criteria:
In each operation cycle the counter shall be incremented after completing an operation cycle in which FDC10 reachesthe value -128, unless FDC10 has already reached the value +127 in the current operation cycle.
The counter shall be cleared as soon as FDC10 reaches the value +127.
If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.

Reporting:
The stored counter shall be reported as a 1 byte unsigned numeric value on request of the diagnostic services specified in reference [Data_1] *UDS Services.*

661869 v1 **Support for Operation Cycle Counter #3**

Version history:
(1):

Purpose: To provide enhanced information about the occurence of a DTC that may be useful in the analysis of the fault.
Source: Table "Requirements on implementation of DTC operation cycle counters"
Verification Method: Test
Legacy ID: 72032v4

For all DTCs supported by the ECU a counter identified by DTCExtendedDataRecordNumber=3 shall be implemented according to the following:

When implementing with AUTOSAR 4.0.x:
Definition:
Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 first reached a value that is equal to or greater than its unconfirmedDTCLimit (since DTC information was last cleared).
All operation cycles, including those in which the test has not been passed or the DTC fault detection counter 10 has not reached a value that is equal to or greater than its unconfirmedDTCLimit shall be included.

Increment/clear criteria:
The counter shall be incremented at the beginning of each operation cycle following the operation cycle where FDC10 first reached a value that is equal to or greater than its unconfirmedDTCLimit.
If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.

Reporting:
The stored counter shall be reported as a 1 byte unsigned numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services.*

When implementing with AUTOSAR 4.1 or newer:
Definition:
Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 first reached the value +127 (since DTC information was last cleared).
All operation cycles, including those in which the test has not been passed or the DTC fault detection counter 10 has not reached the value +127 shall be included.

Increment/clear criteria:
The counter shall be incremented at the beginning of each operation cycle following the operation cycle where FDC10 first reached the value +127.
If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.

Reporting:
The stored counter shall be reported as a 1 byte unsigned numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services.*

661870 v1 Support for Operation Cycle Counter #4

Version history:
(1):

Purpose: To provide enhanced information about the occurence of a DTC that may be useful in the analysis of the fault.
Source: Table "Requirements on implementation of DTC operation cycle counters"
Verification Method: Test
Legacy ID: 72033v3

For all DTCs supported by the ECU a counter identified by DTCExtendedDataRecordNumber=4 shall be implemented according to the following:

When implementing with AUTOSAR 4.0.x:
Definition:
Number of operation cycles (including the current operation cycle) during which the DTC fault detection counter 10 reached a value that is equal to or greater than its unconfirmedDTCLimit (since DTC information was last cleared).

Increment/clear criteria:
In each operation cycle the counter shall be incremented as soon as FDC10 has reached a value that is equal to or greater than its unconfirmedDTCLimit for the first time.
If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.

Reporting:
The stored counter shall be reported as a 1 byte unsigned numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services*.

When implementing with AUTOSAR 4.1 or newer:
Definition:
Number of operation cycles (including the current operation cycle) during which the DTC fault detection counter 10 reached the value +127 (since DTC information was last cleared).

Increment/clear criteria:
In each operation cycle the counter shall be incremented as soon as FDC10 has reached the value +127 for the first time.
If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.

Reporting:
The stored counter shall be reported as a 1 byte unsigned numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services*.

661871 v1 **Support for Operation Cycle Counter #5**

Version history:
(1):

Purpose: To provide enhanced information about the occurence of a DTC that may be useful in the analysis of the fault.
Source: Table "Requirements on implementation of DTC operation cycle counters"
Verification Method: Test
Legacy ID: 72034v3

For all emission related DTCs supported by the ECU a counter identified by DTCExtendedDataRecordNumber=5 shall be implemented according to the following:

Definition:
Number of warm-up cycles since the DTC commanded the MIL to switch off (since DTC information was last cleared).
The warm-up cycle is defined in *[Data_52] OBD II regulation, section 1968.2 of title 13, California Code of Regulations, UDS - Data - External publications*.

Increment/clear criteria:
The counter shall be incremented when warm-up cycle changes from false to true.
The counter shall be cleared when the DTC fault detection counter 10 reach the value of +127.
If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.

Reporting:

The stored counter shall be reported as a 1 byte unsigned numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services.*

661872 v1 **Support for Operation Cycle Counter #6**

Version history:
(1):

Purpose: To provide enhanced information about the occurence of a DTC that may be useful in the analysis of the fault.
Source: Table "Requirements on implementation of DTC operation cycle counters"
Verification Method: Test
Legacy ID: 72035v6

For all DTCs supported by the ECU a trip counter shall be implemented according to the following:

Definition:
The number of consecutive operation cycles (including the current operation cycle) during which the DTC fault detection counter 10 reached the value of +127 (since DTC information was last cleared). The operation cycles during which the test was not completed shall be excluded. The minimum range of the counter is 0-15.

Increment/clear criteria:
In each operation cycle the counter shall be incremented as soon as FDC10 has reached the value +127 for the first time.
The counter shall be cleared in the beginning of the operation cycle following an operation cycle where FDC10 did not reach the value +127 at any time but -128 (the counter shall maintain its value if the FDC10 did not reach the value -128). The counter may also be cleared when the confirmedDTCLimit is reached.
If any count operation occurs which would cause a counter to roll over past 0xFF, then the count value shall instead be maintained at 0xFF.

Reporting:
When implementing with AUTOSAR 4.0.x:

The stored trip counter value shall be reported in DTCExtendedDataRecordNumber=6 as a 1 byte value on request of the diagnostic services specified in reference [Data_1] UDS Services.

When implementing with AUTOSAR 4.1 or newer:

The stored trip counter value may be reported in DTCExtendedDataRecordNumber=6 as a 1 byte value on request of the diagnostic services specified in reference [Data_1] UDS Services.

661873 v1 **Support for Operation Cycle Counter #7**

Version history:
(1):

Purpose: To provide enhanced information about the occurence of a DTC that may be useful in the analysis of the fault.

Source:

Verification Method: Test

Legacy ID: 115561v5

For all DTCs supported by the ECU a counter identified by DTCExtendedDataRecordNumber=7 shall be implemented according to the following:

When implementing with AUTOSAR 4.0.x:
Definition:
Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 first reached a value that is equal to or greater than its unconfirmedDTCLimit (since DTC information was last cleared). The operation cycles during which the test was not passed or the DTC fault detection counter 10 did not reach a value that is equal to or greater than its unconfirmedDTCLimit shall be excluded.

Increment/clear criteria:
The counter shall be incremented in each operation cycle, following the operation cycle where FDC10 first reached a value that is equal to or greater than its unconfirmedDTCLimit, as soon as the following criteria is met: The FDC10 reach the value -128 or a value that is equal to or greater than its unconfirmedDTCLimit.
If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.

Reporting:
The stored counter shall be reported as a 1 byte unsigned numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services*.

When implementing with AUTOSAR 4.1 or newer:
This requirements is Not Applicable

4.10.3.1.6.1.4.2 DTC fault detection counters

DTC fault detection counters shall be supported for all DTCs supported by the ECU according to the table below:

The requirements for storing and clearing of the counters are specified in sections *Store DTC information* and *Clear DTC information* .

Table : Requirements for the implementation DTC fault detection counters

DTC extended data record			M/O
Number hex, see note 1	**Name**	**Definition**	
10	DTC fault detection counter 10 (FDC10)	The counter is specified by *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services*, UDS - Data - External publications, service 19 sub-function 14 and Appendix D.5 (Appendix D.5 shall be regarded as mandatory and not just an example) for both emission-related and non-emission-related DTCs with the following additions: 1. At the start of a new operation cycle the count value shall be 0. 2. The count value shall be maintained between test runs within one and same operation cycle (also when the test run is blocked due to the test run criteria is not satisfied). 3. The DTC test may be completed (the DTC status bit 4 and 6 is set to 0) before FDC10 reaches +127 or -128, and passed (the DTC status bit 0 is set to 0) before	M

		FDC10 reaches -128, where approved by the responsible authority appointed by OEM. 4. The maximum value of the counter may be set to +126 (which means that the value DTC status bits, except the bits 4 and 6, will never be set, they will remain unchanged) where approved by the responsible authority appointed by OEM or shall be set to +126 if that is specified by this document. 5. If a single test sample of a test returns a fail result, when the current value of the counter is negative, the new count value resulting from the fail result shall be set as if the current count value had been 0 (a test sample represents the fail result shall always causes fault detection counter to increment above 0). 6. If a single test sample of a test returns a pass result, when the current value of the counter is positive, the new count value resulting from the pass result shall be set as if the current count value had been 0 (a test sample represents the pass result shall always causes fault detection counter to decrement below 0). 7. The count value shall be updated (increased or decreased) each time the DTC test runs and produce a test sample failed or passed result. If the number of consecutive test sample failed results is more than 127 to reach the testFailedLimit +127 (from the start value 0) or if the number of consecutive test sample passed results is more than 128 to reach the testPassedLimit -128 (from the start value 0), the count value cannot be updated (increased or decreased) each time the DTC test runs and produce a test sample failed or passed result. In this case, and only in this case, may a pre-counter or similar (specified by the implementer) be used in order to reduce the number of updates to 127 (at increases) and 128 (at decreases). Exception to this requirement (e.g. due to legal requirements) is allowed but only when approved by the responsible authority appointed by OEM. Refer to section Definitions and Abbreviations for definition of a test sample. Refer to section DTC status bits for details of the DTC status bits. The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services*, and be reported as 1 byte signed numeric with a range of –128 to + 127.	
11	DTC fault detection counter 11 (FDC11)	The maximum value that the FDC10 has reached during the current operation cycle. The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte signed numeric with a range of –128 to + 127.	O
12	DTC fault detection counter 12 (FDC12)	The maximum value, which is equal to or greater than unconfirmedDTCLimit, that the FDC10 has reached since the last time DTC information was cleared. The stored counter shall be reported on request of the diagnostic services specified in*[Data_1] UDS Services* and be reported as 1 byte signed numeric with a range of –128 to + 127.	M

Notes in the table above:
1. The number is the DTCExtendedDataRecordNumber.

Requirements from this section are specified in section "Requirements from section DTC fault detection counters".

4.10.3.1.6.1.4.2.1 Requirements from section DTC fault detection counters

661874 v1 **Support for Fault Detection Counter #10**

Version history:
(1):

Purpose: To provide enhanced information about the occurence of a fault, that may be useful in the analysis of the fault, and enables detailed calibration of the corresponding DTC.
Source: Table "Requirements for the implementation DTC fault detection counters"
Verification Method: Test
Legacy ID: 72036v5

For all DTCs supported by the ECU a counter identified by DTCExtendedDataRecordNumber=10 shall be implemented as defined in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* (service 19 sub-function 14 and Appendix D.5) , UDS - Data - External publications and according to the following additional definitions:

1. At the start of a new operation cycle the count value shall be 0.

2. The count value shall be maintained between test runs within one and same operation cycle, also when the test run is blocked due to the test run criteria is not satisfied

3. The DTC test may be completed (i.e. the DTC status bit 4 and 6 is set to 0) before FDC10 reaches +127 or -128, and passed (i.e. the DTC status bit 0 is set to 0) before FDC10 reaches -128, where approved by the responsible authority appointed by OEM.

4. The maximum value of the counter may be set to +126 (which means that the value DTC status bits, except the bits 4 and 6, will never be set, they will remain unchanged) where approved by the responsible authority appointed by OEM or shall be set to +126 if that is specified by this document.

5. If a single test sample of a test returns a fail result, when the current value of the counter is negative, the new count value resulting from the fail result shall be set as if the current count value had been 0 (a test sample representing the fail result shall always cause fault detection counter to increment above 0).

6. If a single test sample of a test returns a pass result, when the current value of the counter is positive, the new count value resulting from the pass result shall be set as if the current count value had been 0 (a test sample representing the pass result shall always cause fault detection counter to decrement below 0).

7. The count value shall be updated (increased or decreased) each time the DTC test runs and produce a test sample failed or passed result. If the number of consecutive test sample failed results is more than 127 to reach the testFailedLimit +127 (from the start value 0) or if the number of consecutive test sample passed results is more than 128 to reach the testPassedLimit -128 (from the start value 0), the count value cannot be updated (increased or decreased) each time the DTC test runs and produce a test sample failed or passed result. In this case, and only in this case, may a pre-counter or similar (specified by the implementer) be used in order to reduce the number of updates to 127 (at increases) and 128 (at decreases). Refer to section Definitions and Abbreviations for definition of a test sample. Exception to this requirement (e.g. due to legal requirements) is allowed but only when approved by the responsible authority appointed by OEM.

The stored counter shall be reported as a 1 byte signed numeric value with a range of –128 to + 127 on request of the diagnostic services specified in reference *[Data_1] UDS Services*.

661875 v1 **Support for Fault Detection Counter #11**

Version history:
(1):

Purpose: To provide enhanced information about the occurence of a fault, that may be useful in the analysis of the fault.
Source: Table "Requirements for the implementation DTC fault detection counters"

Verification Method: Test
Legacy ID: 72038v3

For all DTCs supported by the ECU a counter identified by DTCExtendedDataRecordNumber=11 may be implemented and if it is implemented then shall be according to the following definition:
The count value shall be equal to the maximum value the FDC10 has reached during current operation cycle.
The stored counter shall be reported as a 1 byte signed numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services*.

661876 v1 **Support for Fault Detection Counter #12**

Version history:
(1):

Purpose: To provide enhanced information about the occurence of a fault, that may be useful in the analysis of the fault.
Source: Table "Requirements for the implementation DTC fault detection counters"
Verification Method: Test
Legacy ID: 72039v1

For all DTCs supported by the ECU a counter identified by DTCExtendedDataRecordNumber=12 shall be implemented according to the following definition:
The count value shall be equal to the maximum value, which is equal to or greater than unconfirmedDTCLimit, the FDC10 has reached since the last time DTC information was cleared.
The stored counter shall be reported as a 1 byte signed numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services*.

4.10.3.1.6.1.4.3 DTC time stamp

DTC time stamps shall be supported for all DTCs supported by the ECU according to following the table below : The requirements for storing and clearing of the time stamps are specified in sections *Store DTC information* and *Clear DTC information* .

Table: Requirements on implementation of DTC time stamps

DTC extended data record			**M/O**
Number hex, see note 1	**Name**	**Definition**	
20	DTC time stamp 20 (TS20)	The global real time that is taken the first time, since DTC information was last cleared, FDC10 reaches a value that is equal to or greater than unconfirmedDTCLimit. Refer to section *Global DTC snapshot data*, data identifier DD00, for definition of global real time. Refer to section *DTC status indicators* for definition of the unconfirmedDTCLimit. The stored time stamp shall be reported on request of the diagnostic services specified in *[Data_1] UDS - Services*, and be reported as 4 bytes.	O
21	DTC time stamp 21 (TS21)	The global real time that is taken the latest time, since DTC information was last cleared, FDC10 reaches a value that is equal to or greater than unconfirmedDTCLimit. If FDC10 is updated and reaches a value equal to or greater than unconfirmedDTCLimit more than once during an operation cycle then the time stamp shall be taken the first time only.	O

		Refer to section *Global DTC snapshot data*, data identifier DD00, for definition of global real time. Refer to section *DTC status indicators* for definition of the unconfirmedDTCLimit. The stored time stamp shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 4 bytes.	
22	DTC time stamp 22 (TS22)	The GNSS time that is taken the first time, since DTC information was last cleared, FDC10 reaches a value that is equal to or greater than unconfirmedDTCLimit. Refer to section *Global DTC snapshot data*, data identifier DD07, for definition of GNSS DTC time. Refer to section *DTC status indicators* for definition of the unconfirmedDTCLimit. The stored time stamp shall be reported on request of the diagnostic services specified in *[Data_1] UDS - Services*, and be reported as 6 bytes.	M
23	DTC time stamp 23 (TS23)	The GNSS time that is taken the latest time, since DTC information was last cleared, FDC10 reaches a value that is equal to or greater than unconfirmedDTCLimit. If FDC10 is updated and reaches a value equal to or greater than unconfirmedDTCLimit more than once during an operation cycle then the time stamp shall be taken the first time only. Refer to section *Global DTC snapshot data*, data identifier DD07, for definition of global real time. Refer to section *DTC status indicators* for definition of the unconfirmedDTCLimit. The stored time stamp shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 6 bytes.	M

Notes in Table 4.23:
1. The number is the DTCExtendedDataRecordNumber.

Requirements from this section are specified in section "Requirements from section DTC time stamp".

4.10.3.1.6.1.4.3.1 Requirements from section DTC time stamp

661877 v1 **Support for DTC time stamp #20**

Version history:
(1):

Purpose: To provide enhanced information about the occurence of a fault, that may be useful in the analysis of the fault.
Source: Table "Requirements on implementation of DTC time stamps"
Verification Method: Test
Legacy ID: 72041v2

For all DTCs supported by the ECU a data record identified by DTCExtendedDataRecordNumber=20 shall be implemented according to the following definition:
The record value shall be equal to the global real time (data record 0xDD00) that is taken the first time FDC10 reaches a value that is equal to or greater than unconfirmedDTCLimit, since DTC information was last cleared.
The stored data record shall be reported as a 4 byte value on request of the diagnostic services specified in reference *[Data_1] UDS Services*.

661878 v1 **Support for DTC time stamp #21**

Version history:
(1):

Purpose: To provide enhanced information about the occurence of a fault, that may be useful in the analysis of the fault.
Source: Table "Requirements on implementation of DTC time stamps"
Verification Method: Test
Legacy ID: 72042v2

For all DTCs supported by the ECU a data record identified by DTCExtendedDataRecordNumber=21 shall be implemented according to the following definition:
The record value shall be equal to the global real time (data record 0xDD00) that is taken the latest time FDC10 reaches a value that is equal to or greater than unconfirmedDTCLimit, since DTC information was last cleared.
If FDC10 is updated and reaches a value equal to or greater than unconfirmedDTCLimit more than once during an operation cycle then the time stamp shall be taken the first time only.
The stored data record shall be reported as a 4 byte value on request of the diagnostic services specified in reference *[Data_1] UDS Services*.

675447 v1 **Support for DTC time stamp #22**

Version history:
(1):

Purpose: To provide enhanced information about the occurence of a fault, that may be useful in the analysis of the fault.
Source: Table "Requirements on implementation of DTC time stamps".
Verification Method: Test
Legacy ID:

For all DTCs supported by the ECU a data record identified by DTCExtendedDataRecordNumber=22 shall be implemented according to the following definition:
The record value shall be equal to the GNSS time (data record 0xDD07) that is taken the first time FDC10 reaches a value that is equal to or greater than unconfirmedDTCLimit, since DTC information was last cleared.
The stored data record shall be reported as a 6 byte value on request of the diagnostic services specified in reference *[Data_1] UDS Services*.

675449 v2 **Support for DTC time stamp #23**

Version history:
(2):

Purpose: To provide enhanced information about the occurence of a fault, that may be useful in the analysis of the fault.
Source: Table "Requirements on implementation of DTC time stamps".
Verification Method: Test
Legacy ID:

For all DTCs supported by the ECU a data record identified by DTCExtendedDataRecordNumber=23 shall be implemented according to the following definition:
The record value shall be equal to the GNSS time (data record 0xDD07) that is taken the latest time FDC10 reaches a value that is equal to or greater than unconfirmedDTCLimit, since DTC information was last cleared.
If FDC10 is updated and reaches a value equal to or greater than unconfirmedDTCLimit more than once during an operation cycle then the time stamp shall be taken the first time only.
The stored data record shall be reported as a 6 byte value on request of the diagnostic services specified in

reference *[Data_1] UDS Services*.

4.10.3.1.6.1.4.4 DTC status indicators

DTC status indicators shall be supported for all DTCs supported by the ECU according to the table below :

The requirements for storing and clearing of the status indicators in long term memory are specified in sections *Store DTC information* and *Clear DTC information* .

Table: Requirements on implementation of DTC status indicators - AUTOSAR 4.0.x

DTC extended data record					**M/O**
Number hex, see note 1	**Name**	**Definition**			
30	DTC status indicator 30 (SI30)	The status indicator consists of the following bits:			
		No	Name	Definition	
		0	UnconfirmedDTC	The bit shall indicate whether or not the DTC is currently unconfirmed. The bit is set to 1 when the FDC10 reach a value in the range [unconfirmedDTCLimit, +127]. The bit is reset to 0 when the FDC10 reach the value -128. The value of the unconfirmedDTCLimit is specified by the implementer according to the requirements specified in section *Generic requirements on calibration of DTCs*. The value shall be in the range of [+1, +127]. Note that the value of this bit shall not be reset to 0 at start of an operation cycle (the value from previous operation cycle shall be kept).	M
		1	UnconfirmedDTC-ThisOperationCyle	The bit shall indicate whether or not the FDC10 has reached a value in the range [unconfirmedDTCLimit, +127] at any time during the current operation cycle. The bit is set to 1 when the FDC10 reach a value in the range [unconfirmedDTCLimit, +127]. The bit is reset to 0 at start of an operation cycle.	M
		2	UnconfirmedDTC-SinceLastClear.	The bit shall indicate whether or not the FDC10 has reached a value in the range [unconfirmedDTCLimit, +127] since DTC information was last cleared) The bit is set to 1 when the FDC10 reach a value in the range [unconfirmedDTCLimit, +127]..	M
		3	AgedDTC	The bit is set to 1 when the DTC is aged. The bit is reset to 0 when the aged DTC reoccurs. Refer to Table 4.27 for when a DTC is aged and when an aged DTC reoccurs.	M
		4	SymptomSinceLast-Clear	The fault detected by the DTC-test has caused a customer symptom since DTC information was last clear. The bit shall be set to 1 when the FDC10 is in the range [unconfirmedDTCLimit, +127] and specific conditions, that need to be fulfilled in order to cause a customer symptom, are satisfied. The specific conditions, specified by the implementer, can be that the customer use the defect function, ECU takes certain actions due to the fault, the ECU is in specific operation or environment condition, etc.	MC1
		5	WarningIndicator-Requested Since-LastClear	Maximum value of DTC status bit 7 since DTC information was last cleared. The bit is set to 1 when DTC status bit no 7 is set to 1. However, the bit may also be set to 1 even when DTC status bit not 7 – warningIndicatorRequested is not set to 1 (refer to reference [Data_50] Road vehicles – Diagnostics systems – Diagnostic services, UDS -	M

				Data - External publications regarding requirements on set of DTC status bit not 7 to 1) according to the following: The bit is set to 1 when the same criteria as required by the above mention reference on set of DTC status bit not 7 – warningIndicatorRequested to 1 is satisfied, but it may also be set to 1 and warning indicator may be requested before DTC status bit 3 – confirmed DTC is set to 1. Note that in all cases must FDC10 has reached the value +127, since DTC information was latest cleared, before the bit is set to 1 and warring indicator is requested (if not otherwise is specified in this document).	
		6	EmissionRelatedDTC	The bit is set to 0 when the DTC is not emission related. The bit is set to 1 when the DTC is emission related. Note that a DTC is not emission related as long as ithe FDC10 has not reached the value +127 since DTC information was last cleared and when it is aged i.e. the SI30:3 (bit 3 above) is set to1.	M
		7	TestFailedSinceLastClear/Aged	DTC test failed since the DTC was last aged or, as long as the DTC have not been aged, since the DTC information was last clear. The bit is set to set to 1 when DTC test is failed i.e. when the FDC10 reach the value +127. The bit is reset to 0 when DTC is aged i.e. the SI30:3 (bit 3 above) is set to1.	M
		The stored status indicators shall be reported on request of the diagnostic services specified in reference *[Data_1] UDS Services* and be reported as 1 byte bit mapped value.			

Table: Requirements on implementation of DTC status indicators - AUTOSAR 4.1 or newer

DTC extended data record					**M/O**
Number hex, see note 1	**Name**	**Definition**			
30	DTC status indicator 30 (SI30)	The status indicator consists of the following bits:			MC
		No	Name	Definition	
		0	UnconfirmedDTC	When implemented, the bit shall indicate whether or not the DTC is currently unconfirmed. The bit is set to 1 when the FDC10 reach a value in the range [unconfirmedDTCLimit, +127]. The bit is reset to 0 when the FDC10 reach the value -128. The value of the unconfirmedDTCLimit is specified by the implementer according to the requirements specified in section Generic requirements on calibration of DTCs. The value shall be in the range of [+1, +127]. Note that the value of this bit shall not be reset to 0 at start of an operation cycle (the value from previous operation cycle shall be kept). When not implemented, the bit shall be set to 0	O
		1	UnconfirmedDTC-ThisOperationCyle	When implemented, the bit shall indicate whether or not the FDC10 has reached a value in the range [unconfirmedDTCLimit, +127] at any time during the current operation cycle. The bit is set to 1 when the FDC10 reach a value in the range [unconfirmedDTCLimit, +127]. The bit is reset to 0 at start of an operation cycle. When not implemented,the bit shall be set to 0	O
		2	UnconfirmedDTC-	When implemented, the bit shall indicate whether or not the FDC10 has reached a value in the range [unconfirmedDTCLimit, +127] since	O

			SinceLast Clear.	DTC information was last cleared) The bit is set to 1 when the FDC10 reach a value in the range [unconfirmedDTCLimit, +127] When not implemented, the bit shall be set to 0	
		3	AgedDTC	When implemented, the bit is set to 1 when the DTC is aged. The bit is reset to 0 when the aged DTC reoccurs. Refer to Table 4.27 for when a DTC is aged and when an aged DTC reoccurs. When not implemented, the bit shall be set to 0	O
		4	SymptomSinceLast-Clear	When implemented, this bit indicates whether the fault detected by the DTC-test has caused a customer symptom since DTC information was last cleared. The bit shall be set to 1 when the FDC10 is in the range [unconfirmedDTCLimit, +127] and specific conditions, that need to be fulfilled in order to cause a customer symptom, are satisfied. The specific conditions, specified by the implementer, can be that the customer use the defect function, ECU takes certain actions due to the fault, the ECU is in specific operation or environment condition, etc. When not implemented, the bit shall be set to 0	MC1
		5	WarningIndicator-Requested Since-LastClear	When implemented, this bit indicates the maximum value of DTC status bit 7 since DTC information was last cleared. The bit is set to 1 when DTC status bit no 7 is set to 1. However, the bit may also be set to 1 even when DTC status bit not 7 – warningIndicatorRequested is not set to 1 (refer to reference Road vehicles – Diagnostics systems – Diagnostic services, UDS - Data - External publications regarding requirements on set of DTC status bit not 7 to 1) according to the following: The bit is set to 1 when the same criteria as required by the above mention reference on set of DTC status bit not 7 – warningIndicatorRequested to 1 is satisfied, but it may also be set to 1 and warning indicator may be requested before DTC status bit 3 – confirmed DTC is set to 1. Note that in all cases must FDC10 has reached the value +127, since DTC information was latest cleared, before the bit is set to 1 and warring indicator is requested (if not otherwise is specified in this document). When not implemented, teh bit shall be set to 0	MC2
		6	EmissionRelatedDTC	The bit is set to 0 when the DTC is not emission related. The bit is set to 1 when the DTC is emission related. Note that a DTC is not emission related as long as the FDC10 has not reached the value +127 since DTC information was last cleared and when it is aged i.e. the SI30:3 (bit 3 above) is set to1.	MC3
		7	TestFailedSinceLastClear/Aged	When implemented, this bit indicates DTC test failed since the DTC was last aged or, as long as the DTC have not been aged, since the DTC information was last cleared. The bit is set to set to 1 when DTC test is failed i.e. when the FDC10 reach the value +127. The bit is reset to 0 when DTC is aged i.e. the SI30:3 (bit 3 above) is set to1. When not implemented, this bit shall be set to 0	O
		The stored status indicators shall be reported on request of the diagnostic services specified in reference *[Data_1] UDS Services* and be reported as 1 byte bit mapped value.			

Abbreviations in the table above:

MC = SI30 shall be implemented in case at least one of the status indicator bits is being implemented. If none of the status bits are being implemented, then SI30 shall not be implemented either.
MC1 = A feasibility study is required. This bit shall be implemented if a technical solution is available and its cost is justified. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM.
MC2 = Only mandatory in case ECU has for DTC's that support "Warning Indicator Requested"
MC3 = Mandatory for Emission related ECU's

Notes in the table above:
1. The number is the DTCExtendedDataRecordNumber.

Requirements from this section are specified in section DTC status indicators "DTC status indicators".

4.10.3.1.6.1.4.4.1 Requirements from section DTC status indicators

661879 v1 **Support for DTC status indicator #30**

Version history:
(1):

Purpose: To provide enhanced information about the occurence of a fault, that may be useful in the analysis of the fault.
Source: Table "Requirements on implementation of DTC status indicators"
Verification Method: Test
Legacy ID: 72063v0

For all DTCs supported by the ECU a status indicator record identified by DTCExtendedDataRecordNumber=30 (SI30) shall be implemented, but only if at least one of the status indicator bits is being implemented.
The status indicator record shall be reported as a 1 byte value.

661883 v1 **Support for DTC status indicator #30, AgedDTC.**

Version history:
(1):

Purpose: To provide information about the maximum value of DTC status bit 7 since DTC information was last clearedaged DTC.
Source: Table "Requirements on implementation of DTC status indicators"
Verification Method: Test
Legacy ID: 72067v3

For all DTCs supported by the ECU, bit no 3 of SI30 shall indicate AgedDTC according to the following definition:

The bit is set to 1 when the DTC is aged.
The bit is reset to 0 when the aged DTC reoccurs.

When implementing with AUTOSAR 4.0.x:
This bit shall be implemented.

When implementing with AUTOSAR 4.1 or newer:

This bit is optional. In case this bit is not implemented, but SI30 is (in order to support one or more of the other status indicator bits) then bit no 3 of SI30 shall report the value 0.

661886 v1 **Support for DTC status indicator #30, EmissionRelatedDTC**

Version history:
(1):

Purpose: To allow for a tool to understand if the DTC is emission related or not.
Source: Table "Requirements on implementation of DTC status indicators"
Verification Method: Analysis
Legacy ID: 72070v1

For all DTCs supported by the ECU, bit no 6 of SI30 shall indicate EmissionRelatedDTC according to the following definition:

The bit shall be set to 1 when the DTC is emission related.

When implementing with AUTOSAR 4.1 or newer:
This bit is only mandatory for ECUs that support EmissionRelatedDTC.
For other ECUs, this bit only needs to be implemented in case SI30 is implemented (in order to support one or more of the other status indicator bits).

661884 v2 **Support for DTC status indicator #30, SymptomSinceLastClear**

Version history:
(2): Verification Method由Test调整为Analysis AND Test。

Purpose: To provide information that a fault indicated by the DTC has caused a customer symptom since DTC information was last clear.
Source: Table "Requirements on implementation of DTC status indicators"
Verification Method: Analysis AND Test
Legacy ID: 72069v1

For all DTCs supported by the ECU, bit no 4 of SI30 shall be supported as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

If implemented bit no 4 of SI30 shall indicate SymptomSinceLastClear according to the following definition:
The bit shall be set to 1 when FDC10 is in the range [unconfirmedDTCLimit, +127] and specific conditions specified by the implementer, that need to be fulfilled in order to generate customer symptom, are satisfied.

When implementing with AUTOSAR 4.1 or newer:
In case this bit is not implemented, but SI30 is (in order to support one or more of the other status indicator bits) then bit no 4 of SI30 shall report the value 0.

661880 v1 **Support for DTC status indicator #30, UnconfirmedDTC**

Version history:
(1):

Purpose: To provide an early indication that a fault may be currently present.
Source: Table "Requirements on implementation of DTC status indicators";
Verification Method: Test
Legacy ID: 72064v3

For all DTCs supported by the ECU, bit no 0 of SI30 shall indicate UnconfirmedDTC according to the following definition:

The bit shall be set to 1 when FDC10 reaches a value in the range [unconfirmedDTCLimit, +127]
The bit shall be reset to 0 when FDC10 reaches the value -128.

The value of the unconfirmedDTCLimit is specified by the implementer according to the requirements specified in section Generic requirements on calibration of DTCs.

The value shall be in the range of [+1, +127].

When implementing with AUTOSAR 4.0.x:
This bit shall be implemented.

When implementing with AUTOSAR 4.1 or newer:
This bit is optional. In case this bit is not implemented, but SI30 is (in order to support one or more of the other status indicator bits) then bit no 0 of SI30 shall report the value 0.

661882 v1 **Support for DTC status indicator #30, UnconfirmedDTCSinceLastClear**

Version history:
(1):

Purpose: To provide an early indication that a fault may be present or has been present since last clear.
Source: Table "Requirements on implementation of DTC status indicators"
Verification Method: Test
Legacy ID: 72066v2

For all DTCs supported by the ECU, bit no 2 of SI30 shall indicate UnconfirmedDTCSinceLastClear according to the following definition:

The bit is set to 1 when the FDC10 reach a value in the range [unconfirmedDTCLimit, +127].

When implementing with AUTOSAR 4.0.x:

This bit shall be implemented.

When implementing with AUTOSAR 4.1 or newer:
This bit is optional. In case this bit is not implemented, but SI30 is (in order to support one or more of the other status indicator bits) then bit no 2 of SI30 shall report the value 0.

661881 v1 **Support for DTC status indicator #30, UnconfirmedDTCThisOperationCyle**

Version history:
(1):

Purpose: To provide an early indication that a fault may be present in the current operation cycle.
Source: Table "Requirements on implementation of DTC status indicators"
Verification Method: Test
Legacy ID: 72065v2

For all DTCs supported by the ECU, bit no 1 of SI30 shall indicate UnconfirmedDTCThisOperationCyle according to the following definition:

The bit is set to 1 when the FDC10 reach a value in the range [unconfirmedDTCLimit, +127].
The bit shall be reset to 0 at the start of an operation cycle.

When implementing with AUTOSAR 4.0.x:
This bit shall be implemented.

When implementing with AUTOSAR 4.1 or newer:
This bit is optional. In case this bit is not implemented, but SI30 is (in order to support one or more of the other status indicator bits) then bit no 1 of SI30 shall report the value 0.

661885 v1 **Support for DTC status indicator #30, WarningIndicatorRequestedSinceLastClear**

Version history:
(1):

Purpose: To provide information about the maximum value of DTC status bit 5 since DTC information was last cleared.
Source:
Verification Method: Test
Legacy ID: 115621v1

For all DTCs supported by the ECU, bit no 5 of SI30 shall indicate WarningIndicatorRequestedSinceLastClear according to the following definition:
The bit shall be set to 1 when DTC status bit no 7 is set to 1.

However, the bit may also be set to 1 even when DTC status bit not 7 – warningIndicatorRequested is not set to 1 (refer to reference [Data_50] Road vehicles – Diagnostics systems – Diagnostic services, UDS - Data - External publications regarding requirements on set of DTC status bit not 7 to 1) according to the following:
The bit is set to 1 when the same criteria as required by the above mention reference on set of DTC status bit no 7 – warningIndicatorRequested to 1 is satisfied, but it may also be set to 1 and warning indicator may be requested before DTC status bit 3 – confirmed DTC is set to 1.
Note that in all cases FDC10 must have reached the value +127, since DTC information was latest cleared, before the bit is set to 1 and warning indicator is requested (if not otherwise is specified in this document).

When implementing with AUTOSAR 4.1 or newer:
This bit is only mandatory for ECUs that support DTC status bit no 7 – warningIndicatorRequested.
For other ECUs, this bit only needs to be implemented in case SI30 is implemented (in order to support one or more of the other status indicator bits).

661887 v1 **Support for DTC status indicator #30, testFailedSinceLastClear/Aged**

Version history:
(1):

Purpose: To provide information about the DTC test failed since the DTC was last aged or, as long as the DTC have not been aged, since the DTC information was last clear.
Source: Table "Requirements on implementation of DTC status indicators"
Verification Method: Test
Legacy ID: 72071v1

For all DTCs supported by the ECU, bit no 7 of SI30 shall indicate testFailedSinceLastClear/Aged according to the following definition:

The bit is set to set to 1 wthen DTC test is failed i.e. when the FDC10 reach the value +127.
The bit is reset to 0 when DTC is aged i.e. the SI30:3 is set to1.

When implementing with AUTOSAR 4.1 or newer:
This bit is optional. In case this bit is not implemented, but SI30 is (in order to support one or more of the other status indicator bits) then bit no 7 of SI30 shall report the value 0.

4.10.3.1.6.1.5 Store DTC information

The ECU shall be able to store the DTC information that are supported by the ECU, i.e. DTC status bits, DTC snapshot data and DTC extended data (refer to sections *DTC status bits* , *DTC snapshot data* and *DTC extended data* for which DTC information that the ECU shall support) according to the table below.

If DTC information is temporarily stored in short-term memory before being stored in long-term memory it shall be

possible to store at least the same amount of DTC information in short-term memory as in long-term memory.

The DTC information shall be stored within a maximum delay time of "DTC storage time" T1=400 ms. The parameter "DTC storage time" T1 is defined as the time from that an event occurs (e.g. a fault is detected) that cause DTC information to be stored (updated) until the DTC information is available for access by diagnostic service specified in reference [1]. If many events (e.g. 10 or more) occur within a time period of T1 then may the "DTC storage time" be increased but only if that is approved by the responsible authority appointed by OEM.

Table: Requirements on storage of the DTC information that are supported by the ECU - AUTOSAR 4.0.x

DTC information				**Number of DTC for which it shall be possible to store the DTC information supported by the ECU**				
				In short-term memory		**In long-term memory**		
Name		**Number, hex**		**Minimum number (see note 1 below)**	**All**	**Minimum number (see note 1 below)**	**All**	**Any number specified by the implementer**
DTC status bits (SB)		0, 2, 3, 4, 5					M	
		1, 6			M			
		7			M			O
DTC snapshot data (SD), see note 1		20				M		
		30						O
DTC extended data (ED)	DTC operation cycle counters (OCC)	1, 2, 3, 4, 6, 7				M		
		5				MC1		
	DTC fault detection counter (FDC)	10			M			
		11			O			
		12				M		
	DTC time stamp (TS)	20, 21, 22, 23				M		
	DTC status indicators (SI)	30	Bits 0, 2-7			M		
			Bit 1	M				

Table: Requirements on storage of the DTC information that are supported by the ECU - AUTOSAR 4.1 or newer

DTC information			**Number of DTC for which it shall be possible to store the DTC information supported by the ECU**				
			In short-term memory		**In long-term memory**		
Name		**Number, hex**	**Minimum number (see note 1 below)**	**All**	**Minimum number (see note 1 below)**	**All**	**Any number specified by the implementer**
DTC status bits (SB)		0, 2, 3, 4, 5				M	
		1, 6		M			
		7		M			O
DTC snapshot data (SD), see note 1		20, 21			M		
		30					O
DTC extended data (ED)	DTC operation cycle counters (OCC)	1, 2, 3, 4, 6			M		
		5			MC1		

	DTC fault detection counter (FDC)	10		M			
		11		O			
		12			M		
	DTC time stamp (TS)	20, 21, 22, 23			M		
	DTC status inidicators (SI)	30 : Bits0, 2-7			MC2		
		Bit 1	MC2				

Abbreviations in the table above:
MC1 = Mandatory if the DTC is emission related.
MC2 = Mandatory if the bit is implemented, otherwise N/A

Notes in the table above:
1. See the table "Requirements on minimum number of DTCs for which DTC information shall be possible to store in long-term memory" below.

Table: Requirements on minimum number of DTCs for which DTC information shall be possible to store in long-term memory

Number of supported DTCs	Minimum number of DTCs for which DTC information shall be possible to store	Notes
0-5	All supported DTCs	
6-50	5	
51-200	5 + 1/30 x (all supported DTCs minus 50)	1
201-	10	

Notes in the table above:
1 The value shall be rounded to the nearest integer value.

Requirements from this section are specified in section "Requirements from Store DTC information".

4.10.3.1.6.1.5.1 Requirements from section Store DTC information

661889 v1 **DTC storage time**

Version history:
(1):

Purpose: To ensure that DTC information is up to date when reported by diagnostic service.
Source:
Verification Method: Analysis
Legacy ID: 115623v0

The DTC information shall be stored within a maximum delay time of "DTC storage time" T1=400 ms. The parameter "DTC storage time" T1 is defined as the time from that an event occurs (e.g. a fault is detected) that cause DTC information to be stored (updated) until the DTC information is available for access by diagnostic service specified in reference [1]. If many events (e.g. 10 or more) occur within a time period of T1 then may the "DTC storage time" be increased but only if that is approved by the responsible authority appointed by OEM.

661898 v1 **Storage of DTC fault detection counters 10 and 11 in short term memory**

Version history:
(1):

Purpose: To ensure that DTC information is not lost when the ECU is powered down and to define the required storage capacity for the ECU.
Source:
Verification Method: Analysis
Legacy ID: 115663v1

I t shall be possible to store the value of the DTC fault detection counter 0x10 in short term memory for a all DTCs supported by the ECU. If the DTC fault detection counter 0x11 is supported by the ECU then shall it be possible to store the value of the DTC fault detection counter 0x11 in short term memory for all DTCs supported by the ECU.

661894 v1 Storage of DTC fault detection counters 12 in long term memory

Version history:
(1):

Purpose: To ensure that DTC information is not lost when the ECU is powered down and to define the required storage capacity for the ECU.
Source: Table "Requirements on storage of DTC information in long-term memory"
Verification Method: Analysis
Legacy ID: 72084v0

I t shall be possible to store the value of the DTC fault detection counter 0x12 in long term memory for a minimum number of DTCs according to table *Requirements on minimum number of DTCs for which DTC information shall be possible to store in long-term memory.*

661888 v1 Storage of DTC information in short term memory

Version history:
(1):

Purpose: To ensure that the requirements for storage in long term memory is also fulfilled by the short term memory.
Source: Section "Store DTC information"
Verification Method: Analysis
Legacy ID: 72072v1

If DTC information is temporarily stored in short term memory before being stored in long term memory it shall be possible to store at least the same amount of DTC information in short term memory as in long term memory.

661893 v1 Storage of DTC operation cycle counter 5 in long term memory

Version history:
(1):

Purpose: To ensure that DTC information is not lost when the ECU is powered down and to define the required storage capacity for the ECU.
Source: Table "Requirements on storage of DTC information in long-term memory"
Verification Method: Analysis
Legacy ID: 72083v0

If the DTC is emission related it shall be possible to store the value of the DTC operation cycle counter 5 in long term memory for a minimum number of DTCs according to table *Requirements on minimum number of DTCs for which DTC information shall be possible to store in long-term memory.*

661892 v1 **Storage of DTC operation cycle counters 1, 2, 3, 4, 6 and 7 in long term memory**

Version history:
(1):

Purpose: To ensure that DTC information is not lost when the ECU is powered down and to define the required storage capacity for the ECU.
Source: Table "Requirements on storage of DTC information in long-term memory"
Verification Method: Analysis
Legacy ID: 72081v1

It shall be possible to store the values of the DTC operation cycle counters 1, 2, 3, 4, 6 and 7 in long term memory for a minimum number of DTCs according to table *Requirements on minimum number of DTCs for which DTC information shall be possible to store in long-term memory.*

661891 v1 **Storage of DTC snapshot data in long term memory**

Version history:
(1):

Purpose: To ensure that DTC information is not lost when the ECU is powered down and to define the required storage capacity for the ECU.
Source: Table "Requirements on storage of DTC information in long-term memory"
Verification Method: Analysis
Legacy ID: 72074v1

It shall be possible to store the values of the DTC snapshot data defined by DTCSnapshotRecordNumber 0x20 in long term memory for a minimum number of DTCs according to table *Requirements on minimum number of DTCs for which DTC information shall be possible to store in long-term memory.*

661890 v1 **Storage of DTC status bits no 0, 2, 3, 4 and 5 in long term memory**

Version history:
(1):

Purpose: To ensure that DTC information is not lost when the ECU is powered down and to define the required storage capacity for the ECU.
Source: Table "Requirements on storage of DTC information in long-term memory"
Verification Method: Analysis
Legacy ID: 72073v1

It shall be possible to store the values of DTC status bits no 0, 2, 3, 4, and 5 in long term memory for all DTCs supported by the ECU.

661897 v1 Storage of DTC status bits no 1, 6 and 7 in short term memory

Version history:
(1):

Purpose: To ensure that DTC information is not lost when the ECU is powered down and to define the required storage capacity for the ECU.
Source:
Verification Method: Analysis
Legacy ID: 115662v1

It shall be possible to store the values of DTC status bits no 1, 6 and 7 in short term memory for all DTCs supported by the ECU.

661899 v1 Storage of DTC status indicator bit no 1 in short term memory

Version history:
(1):

Purpose: To ensure that DTC information is not lost when the ECU is powered down and to define the required storage capacity for the ECU.
Source:
Verification Method: Analysis
Legacy ID: 115664v0

It shall be possible to store the value of the DTC status indicator bit no 1 defined by DTCExtendedDataRecordNumber 0x30 in short term memory for a minimum number of DTCs according to Table *Requirements on minimum number of DTCs for which DTC information shall be possible to store* .

661896 v1 Storage of DTC status indicator bits no 0, 2-7 in long term memory

Version history:
(1):

Purpose: To ensure that DTC information is not lost when the ECU is powered down and to define the required storage capacity for the ECU.
Source: Table "Requirements on storage of DTC information in long-term memory"
Verification Method: Analysis
Legacy ID: 72121v1

It shall be possible to store the value of the DTC status indicator no 0, 2-7 defined by DTCExtendedDataRecordNumber 0x30 in long term memory for a minimum number of DTCs according to table *Requirements on minimum number of DTCs for which DTC information shall be possible to store in long-term memory.*

661895 v2 Storage of DTC time stamp in long term memory

Version history:
(2):

Purpose: To ensure that DTC information is not lost when the ECU is powered down and to define the required storage capacity for the ECU.

Source: Table "Requirements on storage of DTC information in long-term memory"
Verification Method: Analysis
Legacy ID: 72085v0

It shall be possible to store the values of the DTC time stamp defined by DTCExtendedDataRecordNumber 0x20, 0x21, 0x22 and 0x23 in long term memory for a minimum number of DTCs according to table *Requirements on minimum number of DTCs for which DTC information shall be possible to store in long-term memory*.

4.10.3.1.6.1.5.2 Transfer of DTC information to long term memory

The following requirements are applicable only when DTC information is temporarily stored in short-term memory (e.g. RAM) before being stored in long-term memory (e.g. Flash).

DTC information shall be transferred from short-term memory to long-term memory whenever necessary in order to ensure that no DTC information is lost during normal vehicle operation (examples of situations/events, which are not regarded as normal vehicle operation, are defined below). As a minimum, DTC information that has been added, deleted or changed shall be transferred from short-term memory to long-term memory at least at the end of each operation cycle.

There will be situations where an ECU resets either due to unexpected operating situations or as a result of requested diagnostic services. These ECU reset situations may result in that the DTC information is not transferred to long-term memory and thereby this DTC information is lost. In such situations only DTC information created in the current operation cycle, prior to the ECU reset, may be lost. DTC information from previous operation cycles shall always be retained.

Examples of acceptable causes which may result in lost DTC information are listed below:

1) Unexpected disconnection of the ECU power supply while the ECU is in operation, due to Vehicle battery removal, fuse blow, connector disconnection or other similar cause which does not occur as part of normal vehicle operation.

2) ECU supply voltage below minimum specification due to e.g. weak battery or other vehicle related fault.

3) Diagnostic service requests that results in an ECU reset. See *[Data_1] UDS Services* for services which are allowed to reset the ECU.

All other situations, not covered by the examples above, which may lead to lost DTC information, shall be identified by the implementer and are accepted only after approval by the responsible authority appointed by OEM.

Requirements from this section are specified in section "Requirements from section Transfer of DTC information to long term memory".

4.10.3.1.6.1.5.2.1 Requirements from section Transfer of DTC information to long term memory

661900 v3 **Transfer of DTC information to long term memory**

Version history:
(3): 对于诊断服务请求导致的复位，列出具体的诊断服务。

Purpose: To ensure that no DTC information is lost during normal vehicle operation.
Source: Section "Transfer of DTC information to long term memory"
Verification Method: Test
Legacy ID: 72141v1

If the ECU temporarily stores DTC information in short-term memory before storing it in long-term memory, any DTC information that has been added, deleted or changed during the operation cycle shall be transferred from short-term memory to long-term memory at the end of the operation cycle, and whenever required before it is erased from the short term memory if the erase is caused by normal vehicle operation.
Situations/events, which are not regarded as normal vehicle operation, are:

1) Unexpected disconnection of the ECU power supply while the ECU is in operation, due to Vehicle battery removal, fuse blow, connector disconnection or other similar cause which does not occur as part of normal vehicle operation.

2) ECU supply voltage below minimum specification due to e.g. weak battery or other vehicle related fault.

3) Additional events, not covered by items 1-2 above, which have been documented by the implementer and approved by the responsible authority by OEM.

Note: Diagnostic service requests that results in an ECU reset, e.g. ECUReset service (0x11) and DiagnosticSessionControl with programming session service (0x10 02/82), which is regarded as normal vehicle operation, means that it shall be transferred form short-term memory to long-term memory before ECU reset. See *[Data_1] UDS Services*, for services which are allowed to reset the ECU.

DTC information from previous operation cycle shall be retained if temporarily stores DTC information in short-term memory is lost, in the current operation cycle, due to vehicle operation that is not regarded as normal.

4.10.3.1.6.1.6 Clear DTC information

Clearing of stored DTC information (see sections *DTC status bits* , *DTC snapshot data* and *DTC extended data* for definition of the DTC information) shall be supported by the ECU according to the following the table below.

Note that the ECU is not allowed to support other ways to clear stored DTC information diagnostic data unless it is approved by the responsible authority appointed by OEM.

Table: Requirements in implementation of clear of DTC information

Clear of DTC information			
Item	**When DTC information shall be cleared**	**DTC information that shall be cleared**	**M/O**
1	When requested by diagnostic service specified in *[Data_1] UDS Services*.	All with some exceptions. The exceptions are specified by reference [Data_50] *Road vehicles – Diagnostics systems – Diagnostic services, UDS - Data - External publications*	MC
2	When the long term memory overflows. The potential for an overflow condition occurs when storage of DTC information is required but available storage for the DTC information in the long term memory is already full. An overflow condition shall be resolved as follows: If the data that is required to be stored has higher storage priority than any of the stored data then shall the stored data of lowest storage priority be cleared (see note 1 below) and replaced by the data that is required to be stored. Otherwise shall the stored data not be cleared and replaced. Note 1: If several data items (DTC information for several DTCs) of the same storage priority are stored then shall the data item that has the highest value of OOC2 be cleared. If several data items (DTC information for several DTCs) of the same priority and the same value of OCC2 are stored then is it up to the implementer to specify which of these data that shall be cleared. The order of storage priority is the following (1 is the highest priority): 1. The DTC is associated with a warningIndicator (i.e. support the value 1 of the	All DTC information except the DTC status bits and FDC10. Note that the DTC status bit 3 is set to 0, according to Table - *Requirements on implementation of DTC status bits*, if there are no aging counter (OCC2 or OCC5) allocated to the DTC due to the long term memory overflow.	MC

	DTC status bit 7) and is not aged. See note 2 below. 2. The DTC is not associated with a warningIndicator (i.e. do not support the value 1 of the DTC status bit 7). See note 2 below. 3. The DTC is aged (i.e. the DTC status indicator 30, bit 3 is set to 1). Note 2: Stored DTC information is not aged when DTC status indicator 30, bit 3 is set to 0. DTC information that is required to be stored is not aged because DTC information is required to be stored when the DTC fault detection counter 10 reach a value in the range [unconfirmedDTCLimit, +127]. Excluded from above stated storage priority are development specific DTCs (refer to "Table: Requirements on implementation of DTCs" regarding definition of development specific DTCs). They shall have the lowest storage priority (i.e, priority 4) of all DTCs supported by the ECU. However, development specific DTCs may have higher storage priority if the minimum number of DTCs that shall be possible to store in long term memory (according to Table: Requirements on minimum number of DTCs for which DTC information shall be possible to store in long-term memory) is increased with the number of the development specific DTCs that shall have higher storage priority. The mutual storage priority between developments specific DTCs is specified by the implementer. Safety and emission related DTCs and other specific DTCs may have additional storage priorities than above where approved by the responsible authority appointed by OEM. See section *Requisite documents* for handling of conflicts with legislated requirements. Note 3: If DTC information has not been stored due to memory overflow, then shall the DTC information be stored as soon as memory is available and the criteria for storing are satisfied again. Refer to section *DTC status bits* for details about the DTC status bits. Refer to section *DTC fault detection counters* for details about the DTC fault detection counter. Refer to section *DTC status indicators* for details about the DTC status indicators.		
3	When a DTC is aged. A DTC is aged when the counter used for aging reach the agedDTCLimit. The agedDTCLimit is a value in the range of [01, 255]. The counter used for aging shall either be operation cycle counter 2 or 5 (warm up cycle counter) if not otherwise is specified by this document. The implementer specifies the value of the agedDTCLimit according to the requirements specified in section *Generic requirements on calibration of DTCs* and which of the two counters shall be used for aging. An addtion to specified above is that emission related DTCs may use both counters for aging. I this case is the DTC aged when both counters has reached its agedDTCLimit . Refer to section *DTC operation cycle counters* for details about the operation cycle counters.	Autosar 4.0.x: No DTC information. Note that the bit SI 30:3 is set to 1, SI30:6 and SI30:7 are set to 0 according to Table - Requirements on implementation of DTC status indicators and SB3 is set to 0 according to Table - Requirements on implementation of DTC status bits when a DTC is aged. Autosar 4.1 or newer: All DTC Snapshot data and Extended data records shall be cleared. All DTC status bits shall be cleared as	MC

		well, except for status bit testFailedSinceLastClear, this bit will retain its status	
4	When an aged DTC reoccurs. An aged DTC reoccurs when the FDC10 reach a value equal to or greater than unconfirmedDTCLimit after that the DTC has been aged. Refer to item 3 of this table for details about aging of DTCs Refer to section *DTC status indicators* for details about the unconfirmedDTCLimit. Note that new DTC information, that replaces the cleared DTC information, shall be sampled and stored as specified in other section of this document, when an aged DTC reoccurs	Autosar 4.0.x: All DTC information except the DTC status bits and FDC10. Autosar 4.1 or newer: Not Applicable.	MC

Abbreviations in the table above:
MC = Mandatory for all DTCs supported by the ECU with some exceptions. The exceptions are specified in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services*.

Requirements from this section are specified in section "Requirements from section Clear DTC information".

4.10.3.1.6.1.6.1 Requirements from section Clear DTC information

661901 v1 **Clear DTC information by diagnostic service**

Version history:
(1):

Purpose: To allow all DTC information to be cleared by request from e.g. the service technician.
Source: Table "Requirements in implementation of clear of DTC information" - item 1
Verification Method: Analysis
Legacy ID: 72146v1

All stored DTC information for all DTCs supported by the ECU , with the exceptions defined in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* , UDS - Data - External publications , shall be cleared when requested by diagnostic service specified in *[Data_1] UDS Services*.

661902 v1 **Clear DTC information when a DTC is aged**

Version history:
(1):

Purpose: To allow the DTCConfirmation status to be reset if a failure has not been detected for a defined time.
Source: Table "Requirements in implementation of clear of DTC information" - item 3
Verification Method: Test
Legacy ID: 72152v4

When implementing with AUTOSAR 4.0.x:
When the counter used for aging of a DTC reaches the value agedDTCLimit defined for the DTC, no DTC information shall be cleared with some exception (reset of a DTC status bit and some DTC status indicators) that is specified in the document.

The agedDTCLimit shall be a value in the range of [1, 255]. The counter used for aging shall either be operation cycle counter 2 or 5. The implementer shall specify the value of the agedDTCLimit according to the requirements specified in section *Generic requirements on calibration of DTCs* and which of the two counters shall be used for aging. An addtion to specified above is that emission related DTCs may use both counters for aging. In this case the DTC is aged when both counters have reached their agedDTCLimit.

When implementing with AUTOSAR 4.1 or newer:
When the counter used for aging of a DTC reaches the value agedDTCLimit defined for the DTC, all DRC Snapshot data and Extended data records shall be cleared. All DTC status bits shall be cleared as well, except for status bit testFailedSinceLastClear, this bit will retain its status.

The agedDTCLimit shall be a value in the range of [1, 255]. The counter used for aging shall either be operation cycle counter 2 or 5. The implementer shall specify the value of the agedDTCLimit according to the requirements specified in section Generic requirements on calibration of DTCs and which of the two counters shall be used for aging. An addtion to specified above is that emission related DTCs may use both counters for aging. In this case the DTC is aged when both counters have reached their agedDTCLimit.

661903 v1 Clear DTC information when an aged DTC reoccurs

Version history:
(1):

Purpose: To ensure that new DTC information is captured when an aged DTC reoccurs.
Source: Table "Requirements in implementation of clear of DTC information" - item 4
Verification Method: Test
Legacy ID: 72154v1

When implementing with Autosar 4.0x:
If FDC10 for a DTC reaches a value equal to or greater than unconfirmedDTCLimit, after the DTC has been aged (i.e. the counter used for aging of the DTC has reached the value agedDTCLimit defined for the DTC), all DTC information, except the DTC status bits and FDC10, shall be cleared for that DTC.

When implementing with Autosar 4.1 or newer:
This requirement is Not Applicable

661904 v2 Clear DTC information when the long term memory overflows

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: To define the rules for what DTC information that shall be stored and what shall be discarded in the event that long term DTC memory is full when new DTC information needs to be stored.
Source: Table "Requirements in implementation of clear of DTC information" - item 2
Verification Method: Analysis
Legacy ID: 72148v4

If storage of DTC information is required but available storage for the DTC information in the long term memory is already full, all DTC information for all DTCs supported by the ECU, except the DTC status bits and FDC10, of lowest storage priority (see note 1 below) shall be cleared from the long term memory and replaced by the data that is required to be stored if the data that is required to be stored has higher storage priority according to the following priority order (1 is the highest priority):

1. The DTC is associated with a warningIndicator (i.e. support the value 1 of the DTC status bit 7) and is not

aged. See note 2 below.

2. The DTC is not associated with a warningIndicator (i.e. do not support the value 1 of the DTC status bit 7).See note 2 below.

3. The DTC is aged (i.e. the DTC status indicator 30, bit 3 is set to 1).

Excluded from above stated storage priority are developent specific DTCs (refer to table *Requirements on implementation of DTCs* regarding definition of development specific DTCs). They shall have the lowest storage priority (i.e, priority 4) of all DTCs supported by the ECU. However, development specific DTCs may have higher storage priority if the minimum number of DTCs that shall be possible to store in long term memory (according to Table: Requirements on minimum number of DTCs for which DTC information shall be possible to store in long-term memory) is increased with the number of the development specific DTCs that shall have higher storage priority. The mutual storage priority between developments specific DTCs is specified by the implementer.
Safety and emission related DTCs and other specific DTCs may have additional storage priorities than above where approved by the responsible authority appointed by OEM.

Note 1: If several data items (DTC information for several DTCs) of the same storage priority are stored then shall the data item that has the highest value of OOC2 be cleared. If several data items (DTC information for several DTCs) of the same priority and the same value of OCC2 are stored then it is up to the implementer to specify which of these data that shall be cleared.

Note 2: Stored DTC information is not aged when DTC status indicator 30, bit 3 is set to 0. DTC information that is required to be stored is not aged because DTC information is required to be stored when the DTC fault detection counter 10 reach a value in the range [unconfirmedDTCLimit, +127].

4.10.3.1.6.2 Requirement Content

4.10.3.1.6.2.1 Requirements from section Record Data - General

661611 v2 **DTC Global Snapshot Data records defined in GMRDB**

Version history:
(2): Verification Method由Analysis AND Test调整为Analysis。

Purpose: Mandatory data records in are specificed in GMRDB.
Source: Table "Requirements on implementation of data records"
Verification Method: Analysis
Legacy ID: 67764v0,67765v1

DTC Global Snapshot Data records with data identifiers in the range 0xDD00 to 0xDD7F, shall be implemented exactly as they are defined in *[Data_4] Global Master Reference Database*.
It shall be possible to access the DTC Global Snapshot Data records with data identifiers in the range 0xDD00 to 0xDD7F, by diagnostic services specified in [Data_1] UDS Services, in all diagnostic sessions in which services used for access the data records is supported, except programming session. Data items (e.g. external and internal signals) specified in section *Global DTC snapshot data* shall be defined in records with data identifiers in this range.

4.10.3.1.6.2.2 Requirements from section DTC information - General

661615 v1 **DTC Failure Type Byte**

Version history:
(1):

Purpose: A unified failure type description shall be used by all DTCs since it will be

displayed by the tools.

Source: Table "Requirements on implementation of DTCs"

Verification Method: Analysis

Legacy ID: 68265v0

DTCs supported by an ECU shall use Failure Type Byte as defined in *[Data_51] Diagnostic trouble code definitions* , UDS - Data - External publications.

661614 v1 **DTCs defined by GMRDB**

Version history:
(1):

Purpose: All DTC information shall be supported by OEM specific tools and must therefore be defined in GMRDB.

Source: Table "Requirements on implementation of DTCs"

Verification Method: Analysis

Legacy ID: 68264v1

DTCs supported by an ECU shall be implemented as defined in *[Data_4] Global Master Reference Database*. Development specific implementer specifed DTCs are excluded from from this requirement.

661620 v1 **DTCs for faults caused by the ECU operating environment or by customer**

Version history:
(1):

Purpose: DTCs identifying faults caused by the ECU operating environment or by customer actions shall be defined in a separate range in order for the tools to be able to mask them out if required.

Source:

Verification Method: Inspection

Legacy ID: 115161v0

DTCs that identifies faults caused by the ECU operating environment or by customer actions shall be defined in the base DTC range U2E00 to U2EFF and according to section *Detection of faults caused by the ECU operating environment or customer actions* .

661618 v1 **DTCs for secondary faults caused by data from other ECU indicated as invalid**

Version history:
(1):

Purpose: DTCs identifying secondary faults (i.e. not root cause9 shall be defined in a separate range in order for the tools to be able to mask them out if required.

Source: Table "Requirements on implementation of DTCs" - item 2.3

Verification Method: Analysis

Legacy ID: 68270v2

DTCs that identifies faults caused by signal or data value received from another ECU where the signal/data is indicated as invalid by the transmitting ECU, shall be defined in the base DTC range U2C00 to U2CFF or U2600 to U26FF and according to section *Detection of faults caused by signals and data values received from other ECUs* .

661619 v1 **DTCs for secondary faults caused by data from other ECU not indicated as invalid**

Version history:
(1):

Purpose: DTCs identifying secondary faults (i.e. not root cause9 shall be defined in a separate range in order for the tools to be able to mask them out if required.
Source: Table "Requirements on implementation of DTCs" - item 2.4
Verification Method: Analysis
Legacy ID: 68271v2

DTCs that identifies faults caused by signal or data value received from another ECU where the signal/data is **not** indicated as invalid by the transmitting ECU, shall be defined in the base DTC range U2D00 to U2DFF or U2700 to U27FF according to section *Detection of faults caused by signals and data values received from other ECUs* .

661622 v2 **DTCs that are needed only during the development of the ECU**

Version history:
(2):

Purpose: DTCs that are needed only during the development of the ECU shall be defined in a separate range in order for the tools to be able to mask them out if required.
Source: Table "Requirements on implementation of DTCs" - item 2.7
Verification Method: Analysis
Legacy ID: 392611v0

DTCs are defined in Global Master Reference Database. DTCs that are needed only during the development or quality tracking of the ECU shall be defined in the base DTC range U2810 to U2BFF.
The requirements on detection of the faults that shall be identified by DTCs in this range are specified in the following sections.
The FDC10 shall increase with the value 1 when the test sample failed criteria is met for the DTCs in this range.
The maximum value of the FDC10 shall be set to +126 (which means that the value DTC status bits, except the bits 4 and 6, will never be set, they will remain unchanged) for the DTCs in this range.

661621 v1 **DTCs that are needed only during the development of the ECU - implementer specified**

Version history:
(1):

Purpose: DTCs that are needed only during the development of the ECU shall be defined in a separate range in order for the tools to be able to mask them out if required.
Source: Table "Requirements on implementation of DTCs" - item 2.6
Verification Method: Inspection
Legacy ID: 115163v4

DTCs specified by the implementer that are needed only during the development or quality tracking of the ECU shall be defined in the base DTC range U2F00 to U2FFF.
The requirements on detection of the faults that shall be identified by DTCs in this range are specified in section *Requirements on detection of faults caused by signal/data content received from other ECUs.*
The FDC10 shall increase with the value 1 when the test sample failed criteria is met for the DTCs in this range.
The maximum value of the FDC10 shall be set to +126 (which means that the value DTC status bits, except the bits 4 and 6, will never be set, they will remain unchanged) for the DTCs in this range.

661623 v2 ECU operation cycle

Version history:
(2):

Purpose: Reference to requirements on an operation cycle that shall be supported by the ECU.
Source: Section "Definitions- Operation cycle"
Verification Method: Analysis
Legacy ID: 392752v1

The ECU shall start an operation cycle at the latest when the ECU has completed its start-up sequence.

The ECU shall stop the operation cycle at the earliest when the ECU start to perform its shut-down sequence and at the latest when the ECU has completed its shut-down sequence and prior to the following ECU events:
1. Power down (the power supply is disconnected from the ECU).
2. Sleep mode (the ECU goes to sleep mode) or other equivalent modes defined by the implementer.

The detailed criteria for operation cycle start and stop are defined by the implementer for review and approval by the responsible authority appointed by OEM.

661616 v1 Emissions related DTCs defined by SAEJ2012

Version history:
(1):

Purpose: Emission related DTCs are defined by SAEJ2012.
Source: Table "Requirements on implementation of DTCs" - item 1.1
Verification Method: Analysis
Legacy ID: 68266v0

For all emissions related ECUs, DTCs with base DTC value in any of the ranges P0000 to P0FFF, P2000 to P2FFF or P3400 to P3FFF shall be implemented as defined in *[Data_51] Diagnostic trouble code definitions* , UDS - Data - External publications.

661617 v1 Non emissions related DTCs defined by SAEJ2012

Version history:
(1):

Purpose: Standardized DTCs defined in SAEJ2012 shall be used.
Source: Table "Requirements on implementation of DTCs" - item 2.1
Verification Method: Analysis
Legacy ID: 68267v1

DTCs with base DTC value in any of the ranges B0000 to B0FFF, B3000 to B3FFF, C0000 to C0FFF, C3000 to C3FFF, U0000 to U0FFF or U3000 to U3FFF shall be implemented as defined in *[Data_51] Diagnostic trouble code definitions* , UDS - Data - External publications and in the sections *Detection of faults in external electrical circuits* to *Detection of faults due to configuration errors and software incompatibility* and *DTCs associated with driver indications* .

661613 v1 Reporting of DTC information

Version history:

(1):

Purpose: It must be possible to extract DTC information from the ECU by standardized services.
Source: Table "Requirements on implementation of DTCs"
Verification Method: Analysis
Legacy ID: 68262v1

DTCs supported by an ECU shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services*, and be reported not more than once in any response.

4.10.3.1.6.2.3 Requirements from section Calibration of DTCs

661624 v1 **Calibration of DTC test sample failed**

Version history:
(1):

Purpose: To ensure that DTCs is calibrated correctly.
Source: Section Calibration of DTCs, item 1.
Verification Method: Analysis
Legacy ID: 408292v0

If not otherwise is specified by this document or is approved by the responsible authority appointed by OEM, the DTC test sample failed criteria shall be calibrated in such a way that the following is met:
• The DTC test sample shall fail when the monitored item does not meet the design requirements specified by the implementer.
Examples:
1. The DTC test monitors an external circuit connected to the ECU and it runs periodically each 100 ms. The DTC test sample fail when the circuit is short to ground.
2. The DTC test monitors the maximum end-to-end timing of a signal sent on the in-vehicle network. The DTC test sample fail as soon as the maximum end-to-end timing of the received signal is exceeded.
Refer to section *Definitions and Abbreviations* for definition of DTC test sample.

661625 v1 **Calibration of DTC test sample passed**

Version history:
(1):

Purpose: To ensure that DTCs is calibrated correctly.
Source: Section Calibration of DTCs, item 2.
Verification Method: Analysis
Legacy ID: 408293v0

If not otherwise is specified by this document or is approved by the responsible authority appointed by OEM, the DTC test sample passed criteria shall be calibrated in such a way that the following is met:
• The DTC test sample shall pass when the monitored item meet the design requirements specified by the implementer.
Examples:
1. The DTC test monitors an external circuit connected to the ECU and it runs periodically each 100 ms. The DTC test sample pass when the circuit is not short to ground.
2. The DTC test monitors the maximum end-to-end timing of a signal sent on the in-vehicle network. The DTC test sample pass when the maximum end-to-end timing of the received signal is met.
Refer to section *Definitions and Abbreviations* for definition of DTC test sample.

661627 v1 Calibration of DTCTestFailedLimit

Version history:
(1):

Purpose: To ensure that DTCs is calibrated correctly.
Source: Setion Calibration of DTCs, item 4
Verification Method: Analysis
Legacy ID: 408296v0

If not otherwise is specified by this document or is approved by the responsible authority appointed by OEM, the DTCTestFailedLimit shall be calibrated, i.e. the size of the DTC fault detection counter (FDC 10) step-up (increase) value shall be adjusted, in such a way that the following is met:
• The DTC fault detection counter (FDC10) shall reach a value that is equal to the DTCTestFailedLimit (i.e. +127) when it is very likely that the most customers observe a symptom (caused by the detected fault) but probably not all of them is bothered by the symptom in such extent that they require repair of the vehicle.
Note that customer symptoms may be caused by actions (e.g. reconfigurations) taken by the ECU that tries mitigating the effect (e.g. on customer function, engine emissions or system safety) of the detected fault. If the detected fault causes a driver indication (e.g. telltale, text message, etc.), it shall be regarded as a customer symptom.
Refer to *[Data 12] General Diagnostic Guideline* for recommendations and guidelines on how this requirement shall be met.

661628 v1 Calibration of DTCTestPassedLimit

Version history:
(1):

Purpose: To ensure that DTCs is calibrated correctly.
Source: Setion Calibration of DTCs, item 5
Verification Method: Analysis
Legacy ID: 408299v0

If not otherwise is specified by this document or is approved by the responsible authority appointed by OEM, the DTCTestPassedLimit shall be calibrated, i.e. the size of the DTC fault detection counter (FDC 10) step-down (decrease) value shall be adjusted, in such a way that the following is met:
• The DTC fault detection counter (FDC10) shall reach a value that is equal to the DTCTestPassedLimit (i.e. -128) when the fault has not been detected for a while and the monitored object can currently be considered free from the fault.
Typically, the FDC 10 step-down value is equal or a bit less than the FDC 10 step-up value.
Refer to *[Data 12] General Diagnostic Guideline* for recommendations and guidelines on how this requirement shall be met.

661630 v2 Calibration of agedDTCLimit

Version history:
(2): Remove the CEVT

Purpose: To ensure that DTCs is calibrated correctly.
Source: Section Calibration of DTCs, item 7.
Verification Method: Analysis
Legacy ID: 408301v0

If not otherwise is specified by this document or is approved by Electrical Architecture, the AgedDTCLimit shall be calibrated in such a way that the following is met:
• The DTC aging counter (OCC2 or OCC5) shall reach the agedDTCLimit when the fault has not been detected at

such a long time that it probably will not occur and be detected again.
Typically, it is likely that the detected fault has been self-healed or detected by mistake.
Refer to *[Data 12] General Diagnostic Guideline* for recommendations and guidelines on how this requirement shall be met.

661629 v1 Calibration of confirmedDTCLimit

Version history:
(1):

Purpose: To ensure that DTCs is calibrated correctly.
Source: Section Calibration of DTCs. item 6
Verification Method: Analysis
Legacy ID: 408300v0

If not otherwise is specified by this document or is approved by the responsible authority appointed by OEM, the confirmedDTCLimit shall be calibrated in such a way that the following is met:
• The DTC trip counter (see REQPROD 72035) shall reach a value that is equal to the confirmedDTCLimit when it is very likely that that most customers will observe a symptom (caused by the detected fault) and is bothered by the symptom in such extent that they require repair of the vehicle.
Typically, it is very likely that the detected fault can be considered as permanent (lasting), i.e. repetitive and persistent, and that it occurs and causes customer symptom during most vehicle operations and environmental conditions.
Note that customer symptoms may be caused by actions (e.g. reconfigurations) taken by the ECU that tries mitigating the effect (e.g. on customer function, engine emissions or system safety) of the detected fault. If the detected fault causes a driver indication (e.g. telltale, text message, etc.), it shall be regarded as a customer symptom.
Refer to *[Data 12] General Diagnostic Guideline* for recommendations and guidelines on how this requirement shall be met.

661626 v1 Calibration of unconfirmedDTCLimit

Version history:
(1):

Purpose: To ensure that DTCs is calibrated correctly.
Source: Section Calibration of DTCs, item 3
Verification Method: Analysis
Legacy ID: 408294v0

If not otherwise is specified by this document or is approved by the responsible authority appointed by OEM, the unconfirmedDTCLimit shall be calibrated in such a way that the following is met:
• The DTC fault detection counter (FDC10) shall reach a value that is equal to or greater than the unconfirmedDTCLimit when it is likely that at least some customers observer a symptom (caused by the detected fault) but probably only some of them is bothered by the symptom in such extent that they require repair of the vehicle.
Typically, it is very likely that the detected fault is intermittent (temporary), i.e. sporadic and transient, and that it occurs and causes customer symptom during only certain (perhaps unusual) vehicle operation and environment conditions.
Note that customer symptoms may be caused by actions (e.g. reconfigurations) taken by the ECU that tries mitigating the effect (e.g. on customer function, engine emissions or system safety) of the detected fault. If the detected fault causes a driver indication (e.g. telltale, text message, etc.), it shall be regarded as a customer symptom.
Refer to *[Data 12] General Diagnostic Guideline* for recommendations and guidelines on how this requirement shall be met.

4.10.3.1.6.2.4 Requirements from section Detection of faults

661638 v1 **DTC values**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Section "Detection of faults"
Verification Method: Analysis
Legacy ID: 68892v1

Unless otherwise specified in the following sections, one unique DTC value shall be used to identify each fault that can be detected by a DTC test in an ECU. Grouping of faults to use one single common DTC is not allowed unless explicitly approved by the responsible authority appointed by OEM, or where the tables within this specification already shows the use of the same DTC. Where faults are combined, all of the associated stored DTC information (e.g. the fault detection counter) is also combined and no more than one instance of the DTC shall be reported in any response to the services specified in reference *[Data_1] UDS Services*. Where the implementer identifies a requirement for separate repair action or fault confirmation process for DTCs that are shown with the same DTC then the responsible authority appointed by OEM may permit the use of different DTCs.

661639 v1 **Failure type value for DTCs**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Section "Detection of faults"
Verification Method: Analysis
Legacy ID: 68893v1

Unless specifically defined in this document, DTCs shall use Failure Type values which are the same as the DTCFailureTypeByte defined by *[Data_51] Diagnostic trouble code definitions* , UDS - Data - External publications , as appropriate depending on the type of fault.
A failure type value of 0x00 may only be used if the Base DTC description includes the failure type and is approved in *[Data_4] Global Master Reference Database*. In such cases the Base DTC may not be combined with any other failure type values.

661637 v1 **Generic Test Run Criteria - car mode transition inhibit criteria**

Version history:
(1):

Purpose: Changing car mode may cause transitional effects that may trigger a DTC test. This must be avoided in order not to set false DTCs.
Source: Section "Detection of faults"
Verification Method: Test
Legacy ID: 68889v1

A ll DTC tests shall be inhibited during 5 seconds from any transition from one car mode to another. The inhibition shall start at notational consumption of the 'car mode' signal when a change of car mode is requested and shall be active for 5 s ± 2s. Exception is alllowed (e.g. due to safety reasons or legel requirements on emissionrelated faults) if the DTC test is able to detect faults in an accurate and reliable manner without use of these test run criteria but possibly other test run criteria, and if approved by the responsible authority appointed by OEM.

661635 v1 Generic Test Run Criteria - car modes inhibit criteria

Version history:
(1):

Purpose: In some car modes vehicle functions may be inhibited which may cause other behaviour than normally expected by the DTC tests.
Source: Section "Detection of faults"
Verification Method: Test
Legacy ID: 68891v1

A ll DTC tests shall be inhibited when the vehicle is set in any of the car modes Factory, Transport or Crash.

661632 v1 Generic Test Run Criteria - enable criteria

Version history:
(1):

Purpose: A general rule is that all DTC tests shall run all the time that the ECU is "in operation".
Source: Section "Detection of faults"
Verification Method: Analysis
Legacy ID: 68886v2

DTC tests shall be enabled all the time during an operation cycle provided that no inhibited criteria are met.
If the ECU performs any action due to fault detected by the DTC test (e.g. in order to mitigate the effect of the fault on customer functions or system safety), the DTC tests shall not be inhibited after the ECU has performed the action (e.g. has reduced or turned off the customer functions that are affected by the fault). However, if this is not appropriate and if approved by the responsible authority appointed by OEM, the DTC test may be inhibited in this case.
The DTC tests shall not be inhibited (while the ECU is in operation) when the functionality, which can be affected by fault detected by the DTC test, is not active (e.g. is disabled when the functionalty is not used). However, if this is not appropriate and if approved by the responsible authority appointed by OEM, the DTC test may be inhibited in this case.

661633 v2 Generic Test Run Criteria - low power inhibit criteria

Version history:
(2): 修改描述，避免认为只有ElPowerLevel 1的时候才抑制。实际只要ElPowerLevel 大于0就会受到抑制。并增加了该信号的描述。

Purpose: DTC tests shall be regarded as unreliable when electrical power availability is limited, i.e. the tests shall be inhibited.
Source: Section "Detection of faults"
Verification Method: Test
Legacy ID: 68887v3

If the ECU is power supplied from the 12 V system for which the ElPowerLevel signal is applicable, all DTC tests shall be inhibited when electrical power availability is limited, such as indicated by ElPowerLevel 1 or above (Signal PwrLvlElec, consists of two signals PwrLvlElecMai and PwrLvlElecSub). Exception: A test shall not be inhibited during engine cranking if the test can detect faults that can occur during cranking and that can prevent the transition to engine running. Other exception is alllowed (e.g. due to safety reasons or legel requirements on emissionrelated faults) if the DTC test is able to detect faults in an accurate and reliable manner without use of these test run criteria but possibly other test run criteria, and if approved by the responsible authority appointed by OEM

661636 v1 Generic Test Run Criteria - low voltage inhibit criteria

Version history:
(1):

Purpose: DTC tests shall be regarded as unreliable when the ECU supply voltage is low, i.e. the tests shall be inhibited.
Source: Section "Detection of faults"
Verification Method: Test
Legacy ID: 68890v0

A ll DTC tests shall be inhibited when voltage supplied to the ECU is lower than required for ensuring correct fault monitoring. The low voltage limit shall be defined by the implementer but shall not be higher than the lower limit of the supply voltage range specified for the function(s) which includes the monitored component/system, unless otherwise defined in this document.

661634 v1 Generic Test Run Criteria - usage mode transition inhibit criteria

Version history:
(1):

Purpose: Changing usage mode may cause transitional effects that may trigger a DTC test. This must be avoided in order not to set false DTCs.
Source: Section "Detection of faults"
Verification Method: Test
Legacy ID: 68888v3

If not otherwise is specified, all DTC tests shall be inhibited during 5 seconds from any transition from one usage mode to another. The inhibition shall start at notational consumption of the 'usage mode' signal when a change of usage mode is requested and shall be active for 5 s ± 2s. Exception is alllowed (e.g. due to safety reasons or legel requirements on emissionrelated faults) if the DTC test is able to detect faults in an accurate and reliable manner without use of these test run criteria but possibly other test run criteria, and if approved by the responsible authority appointed by OEM.

661631 v1 Test Run Criteria - Updating the DTC Fault Detection Counter

Version history:
(1):

Purpose: If a DTC test is stopped by an inhibit critera the actual test may not be allowed to stop due to safety or other functional requirements. In order to stop updateing the DTC information it is sufficient to stop updating FDC10. Note that if the actual test is not stopped and the ECU can take any action due to faults detected by the DTC test (e.g. reduce or shut off its functionality or request a warring indicator) that for sure can cause customer symptom the updating of the FDC10 should not be stopped (i.e. the test is not inhibited in this case).
Source: Section "Detection of faults"
Verification Method: Analysis
Legacy ID: 68885v2

If a DTC test is inhibited it shall stop updating the DTC Fault Detection Counter (FDC10).

666800 v1 **Test Run Criteria for fault detection**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Section "Detection of faults"
Verification Method: Analysis
Legacy ID:

Any fault detection test for external circuits using shall be inhibited when the fault is a secondary fault that appears as a result of another (primary) fault that is detected by another DTC test.

4.10.3.1.6.2.5 Requirements for detection of faults

4.10.3.1.6.2.5.1 Detection of faults in external electrical circuits

The ECU shall be able to detect faults in all external electrical circuits connected to the ECU. This includes both the parts of the circuits that are external to the ECU (e.g., sensors, actuators, cables and connectors) and the parts that are internal to the ECU (e.g., input filters, input protection and output drivers).
The ECU shall be able to detect open circuit, short to ground, short to battery voltage and other faults in these circuits as specified in the table below

Table: Requirements on detection of faults in external electrical circuits connected to the ECU

Type of item	Item no.	Detected faults/errors	Test Period (recommended value)	Test Run Criteria (TRC)	Failure type (FTB)	Base DTC (display value, excl. FTB)	M/O
Switch input	1.1	'Stuck at' fault (e.g., activation too long, button with spring return activated for longer than 1 minute).	500 ms		23 or 24	One for each input circuit.	MC1
	1.2	Other faults specified by the implementer (e.g., impossible value of the switch input when another input signal has a certain value).	100 ms See note 1	TRC1	See note 2	One for each input circuit.	MC1
Analog input	2.1	Short to battery voltage or open circuit.	100 ms		12 and 13 or 15	One for each input circuit.	M
	2.2	Short circuit to ground.	100 ms		11	One for each input circuit.	M
	2.3	Other faults specified by the implementer (e.g., too slow or too fast rate of change for a signal, value out of range, implausible combination of signals etc.).	100 ms See note 1	TRC1	See note 2	One for each input circuit.	MC1
FM, PCM, PWM or similar in-put, alternative 1	3.1	Short circuit to battery voltage or open circuit (low time>maximum or high time>maximum).	100 ms		12 and 13 or 33 or 34	One for each input circuit.	M
	3.2	Short circuit to ground.	100 ms		11	One for each input circuit.	M
	3.3	Other faults specified by the implementer (for examples see the analog input).	100 ms See note 1	TRC1	See note 2	One for each input circuit.	MC1

FM, PCM, PWM or similar in-put, alter-native 2	4.1	Short circuit to battery voltage.	100 ms		12	One for each input circuit.	M
	4.2	Short circuit to ground or open circuit (low time>maximum or high time>maximum).	100 ms		11 and 13 or 33 or 34	One for each input circuit.	M
	4.3	Other faults specified by the implementer (for examples see the analogue input).	100 ms See note 1	TRC1	See note 2	One for each input circuit.	MC1
Output with low side driver	5.1	Short circuit to battery voltage.	100 ms	TRC2	12	One for each output circuit.	M
	5.2	Short circuit to ground or open circuit.	100 ms	TRC3	11 and 13 or 14	One for each output circuit.	M
	5.3	Other faults specified by the implementer (e.g. over-temperature of the low side driver, etc.).	100 ms See note 1	TRC1	See note 2	One for each output circuit.	MC1
Output with high side driver	6.1	Short circuit to battery voltage or open circuit.	100 ms	TRC3	12 and 13 or 15	One for each output circuit.	M
	6.2	Short circuit to ground.	100 ms	TRC2	11	One for each output circuit.	M
	6.3	Other faults specified by the implementer (for examples see the low side driver).	100 ms See note 1	TRC1	See note 2	One for each output circuit.	MC1
Public ECU network interface	7	Refer to implementation specification for the respective network (e.g. UDSonCAN, UDSonFR, UDSonIP, UDS onMOST and UDSonLIN). Note that the implementation specification för LIN (UDSonLIN) and CAN (UDSonCAN) specifies a LIN and CAN respectively there an ECU can be regarded as partly public and partly private (some diagnostic services supported by the ECU can be addressed to the ECU while other can not).					
External electrical circuits connecte d to private ECUs (other than item 9 below)	8	Note that from a diagnostic point of view, all Private ECUs shall be regarded as if they were internally connected I/O units, reading sensors and controlling actuators, within the Public ECU master itself. The Private ECU master shall, on behalf of the private ECUs, support all the diagnostic data required by this table and that are applicable for the private ECUs. The fault detection may, however, be performed by the Private ECUs with the results being sent back to Public ECU master for processing and storage of DTC information.					
Private ECU network interface (except LIN and CAN, see item 7 above)	9.1	Transmitted message contain data that does not correspond to data read back by the ECU.		Each time a message is sent.	86	One for each network.	MC1, MC3
	9.2	The line is not free when no data transmission is expected.	100 ms. See note 1		81	One for each network.	MC1, MC3

	9.3	CS- or CRC-error on received message.		Each time a message is received	83	One for each network.	MC1, MC3
	9.4	Incorrect timing and content of bits, bytes or messages. Incorrect structure and format of bytes or messages.		Each time a message is received	86	One for each network.	MC1, MC3
	9.5	Received data has not been updated by the transmitting ECU.		Each time a message is received	82	One for each network.	MC1, MC3
	9.6	Periodic message is not received within maximum period.		Each time a message is expected to be received.	87	One for each network.	MC1, MC3
	9.7	The physical layer of the serial communication link has a short to ground, short to battery voltage or open circuit.	100 ms. See note 1		01	One for each network.	MC1, MC3
	9.8	Other faults identified by the implementer.	100 ms. See note 1	TRC1	See note 2	One for each network.	MC1, MC3
Power supply	10.1	Low supply voltage at the ECU. The ECU checks if its supply voltage is lower than the measured vehicle battery voltage received from the network. For additional requirements see section *Requirements for detection of low supply voltage*	See section *Requirements for detection of low supply voltage*	See section *Requirements for detection of low supply voltage*	62	U3003	M
Other external electrical circuit.	11.1	Faults identified by the implementer.	100 ms See note 1	TRC1 and additional conditions specified by the implementer	One for each fault. See note 2	One for each circuit.	MC1

Abbreviations in Table 4.7

TRC1 = Not when the fault is a secondary fault that appears as a result of another (primary) fault that is detected by another DTC test. For example a DTC test that has a plausibility check for an analogue sensor signal where the signal is compared with another similar signal. This test shall be inhibited if the DTC test for the compared signal detects a fault on that signal.

TRC2 = At least while output is activated.

TRC3 = At least while output is deactivated.

MC1 = Mandatory where the fault is detected by a fault monitor as part of the ECU functional requirements. Otherwise, a feasibility study shall be performed to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM.

MC2 = N/A

MC3 = Mandatory if the ECU is a Public ECU master and fulfills the conditions of MC1.

Notes in the table above

1. The defined Test Period Time is the general requirement for this type of fault. Depending of the actual fault detected the general requirement may not be achievable or may not be practical for the type of monitor used.

In such cases the implementer may use a different Test Period Time if approved by the responsible authority appointed by OEM.
2. See *[Data_51] Diagnostic trouble code definitions*, UDS - Data - External publications

Requirements from this section are specified in section "Requirements from section Detection of faults in external electrical circuits".

4.10.3.1.6.2.5.1.1 Detection of faults for switch inputs

661644 v1 **Base DTC value for switch inputs**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 1.1 and 1.2
Verification Method: Analysis
Legacy ID: 68899v0

The ECU shall implement one unique base DTC value for each switch input circuit for which fault detection is implemented.

661641 v2 **Detection of faults for switch inputs**

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 1.1 and 1.2
Verification Method: Analysis
Legacy ID: 68895v1

Where a fault in a switch input circuit connected to the ECU is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate Failure Type value for a corresponding DTC.
Otherwise, the ECU shall be able to detect faults in all switch input circuits connected to the ECU as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

661645 v1 **Failure type value for stuck at faults for switch inputs**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 1.1
Verification Method: Analysis
Legacy ID: 68900v0

DTCs for ‘stuck at’ faults for switch inputs shall use Failure Type value 0x23 or 0x24, as appropriate depending on the type of fault.

661640 v1 **Test Run Criteria for fault detection in external circuits**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Section "Detection of faults in external electrical circuits"
Verification Method: Analysis
Legacy ID: 68894v1

Any fault detection test for external circuits using shall be inhibited when the fault is a secondary fault that appears as a result of another (primary) fault that is detected by another DTC test.

661643 v1 **Test period time for other faults for switch inputs**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 1.2
Verification Method: Analysis
Legacy ID: 68898v0

The DTC tests for the detection of other (than 'stuck at') faults for switch inputs shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

661642 v1 **Test period time for stuck at faults for switch inputs**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 1.1
Verification Method: Analysis
Legacy ID: 68897v2

The DTC test for the detection of ‘stuck at’ faults for switch inputs shall produce Test Samples periodically every 500 ms or faster while the Test Run Criteria are met.

4.10.3.1.6.2.5.1.2 Detection of short circuit and open circuit faults in analog inputs

661648 v1 **Base DTC value for short circuit to battery or open circuit faults in an...**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 2.1 and 2.2

Verification Method: Analysis
Legacy ID: 68903v0

The ECU shall implement one unique base DTC value for each analog input circuit for detection short circuit to ground faults, short circuit to battery and open circuit faults.

661651 v3 Detection of other faults for analog inputs

Version history:
(3): Verification Method changes to Analysis from Test.

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 2.3
Verification Method: Analysis
Legacy ID: 68906v0

Where a fault in an analog input circuit connected to the ECU, other than short circuit and open circuit, is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate Failure Type value for the corresponding DTC.
Otherwise, the ECU shall be able to detect other faults in all analog input circuits connected to the ECU as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

661646 v2 Detection of short circuit and open circuit faults in analog inputs

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 2.1 and 2.2
Verification Method: Analysis
Legacy ID: 68901v0

The ECU shall be able to detect short circuit to ground faults, and short circuit to battery or open circuit faults, for all analog input circuits connected to the ECU.

661649 v1 Failure type value for short circuit to battery or open circuit faults i...

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 2.1
Verification Method: Analysis
Legacy ID: 68904v0

DTCs for detection for short circuit to battery or open circuit faults in analog input circuits that can differentiate between the two faults shall use Failure Type value 0x12 and 0x13, as appropriate depending on the type of fault. If the fault detection can not differentiate between the two faults then Failure Type value 0x15 shall be used for both faults.

661650 v1 **Failure type value for short circuit to ground faults in analog inputs**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 2.2
Verification Method: Analysis
Legacy ID: 68905v0

DTCs for short circuit to ground faults for analog inputs shall use Failure Type value 0x11.

661652 v1 **Test period time for other faults for analog inputs**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 2.3
Verification Method: Analysis
Legacy ID: 68907v0

The DTC tests for the detection of other (than short circuit and open circuit) faults for analog inputs shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

661647 v1 **Test period time for short circuit and open circuit faults in analog inputs**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 2.1 and 2.2
Verification Method: Analysis
Legacy ID: 68902v0

The DTC test for the detection short circuit to ground faults, short circuit to battery and open circuit faults in analog inputs shall produce Test Samples periodically every 100 ms or faster while the Test Run Criteria are met.

4.10.3.1.6.2.5.1.3 Detection of short circuit and open circuit faults in FM, PCM, PWM etc. ...

661655 v1 **Base DTC value for short circuit and open circuit faults in FM, PCM, PWM...**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - items 3.1, 3.2, 4.1 and 4.2
Verification Method: Analysis

Legacy ID: 68910v0

The ECU shall implement one unique base DTC value for each FM, PCM, PWM and similar input circuit for detection short circuit to ground faults, short circuit to battery and open circuit faults.

661657 v2 Detection of other faults for FM, PCM, PWM etc. inputs

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - items 3.3 and 4.3
Verification Method: Analysis
Legacy ID: 68912v0

Where a fault in a FM, PCM, PWM and similar input circuit connected to the ECU, other than short circuit and open circuit, is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate Failure Type value for the corresponding DTC.
Otherwise, the ECU shall be able to detect other (than short circuit and open circuit) faults in all FM, PCM, PWM and similar input circuits connected to the ECU as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

661653 v2 Detection of short circuit and open circuit faults in FM, PCM, PWM etc. ...

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - items 3.1, 3.2, 4.1 and 4.2
Verification Method: Analysis
Legacy ID: 68908v0

The ECU shall be able to detect short circuit and open circuit faults in FM, PCM, PWM and similar input circuits connected to the ECU according to one of the following alternatives.
Alternative 1: short circuit to ground faults and short circuit to battery or open circuit faults.
Alternative 2: short circuit to battery faults and short circuit to ground or open circuit faults.

661656 v1 Failure type value for short circuit and open circuit faults in FM, PCM,...

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - items 3.1, 3.2, 4.1 and 4.2
Verification Method: Analysis
Legacy ID: 68911v0

DTCs for faults in FM, PCM, PWM and similar input circuits shall use Failure Type value according to one of the following alternatives
Alternative 1: I nput circuits which have fault detection for short circuit to battery or open circuit faults implemented

that can differentiate between the two faults shall use Failure Type value 0x12 and 0x13, as appropriate depending on the type of fault. If the fault detection can not differentiate between the two faults then Failure Type value 0x33 or 0x34, as appropriate depending on the type of fault, shall be used for both faults.
DTCs for short circuit to ground faults shall use Failure Type value 0x11.
Alternative 2: I nput circuits which have fault detection for short circuit to ground or open circuit faults implemented that can differentiate between the two faults shall use Failure Type value 0x11 and 0x13, as appropriate depending on the type of fault. If the fault detection can not differentiate between the two faults then Failure Type value 0x33 or 0x34, as appropriate depending on the type of fault, shall be used for both faults.
DTCs for short circuit to battery faults shall use Failure Type value 0x12.

661658 v1 **Test period time for other faults for FM, PCM, PWM etc. inputs**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 3.3 and 4.3
Verification Method: Analysis
Legacy ID: 68913v0

The DTC tests for the detection of other (than short circuit and open circuit) faults for FM, PCM, PWM and similar input circuits shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

661654 v1 **Test period time for short circuit and open circuit faults in FM, PCM, P...**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" items 3.1, 3.2, 4.1 and 4.2
Verification Method: Analysis
Legacy ID: 68909v0

The DTC test for the detection short circuit to ground faults, short circuit to battery and open circuit faults in FM, PCM, PWM and similar input circuits shall produce Test Samples periodically every 100 ms or faster while the Test Run Criteria are met.

4.10.3.1.6.2.5.1.4 Detection of short circuit and open circuit faults in low side driver ou...

661663 v1 **Base DTC value for low side driver outputs**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - items 5.1, 5.2 and 5.3
Verification Method: Analysis
Legacy ID: 68918v0

Low side driver outputs circuits which has fault detection implemented shall have one unique base DTC value for

each output circuit.

661666 v2 Detection of other faults for low side driver outputs

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 5.3
Verification Method: Analysis
Legacy ID: 68922v0

Where a fault in a low side driver output circuit connected to the ECU, other than short circuit and open circuit, is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate Failure Type value for the corresponding DTC.
Otherwise, the ECU shall be able to detect other (than short circuit and open circuit) faults in all low side driver outputs circuits connected to the ECU as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

661659 v2 Detection of short circuit and open circuit faults in low side driver ou...

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - items 5.1 and 5.2
Verification Method: Analysis
Legacy ID: 68914v0

The ECU shall be able to detect short circuit to battery faults and short circuit to ground or open circuit faults, for all low side driver outputs circuits connected to the ECU.

661665 v1 Failure type value for short circuit to battery faults in low side drive...

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 5.1
Verification Method: Analysis
Legacy ID: 68920v0

DTCs for short circuit to battery faults for low side driver outputs circuits shall use Failure Type value 0x12.

661664 v1 Failure type value for short circuit to ground or open circuit faults in...

Version history:
(1):

Purpose: To enable easy identification of specific faults.

Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 5.2
Verification Method: Analysis
Legacy ID: 68919v0

Low side driver outputs circuits which have fault detection for short circuit to ground or open circuit faults implemented that can differentiate between the two faults shall use Failure Type value 0x11 and 0x13, as appropriate depending on the type of fault. If the fault detection can not differentiate between the two faults then Failure Type value 0x14 shall be used for both faults.

661661 v2 Test Run Criteria for short circuit to battery faults in low side driver...

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 5.1
Verification Method: Analysis
Legacy ID: 68916v0

The DTC test for detection of short circuit to battery faults in a low side driver output shall run at least while the output is activated.

661662 v2 Test Run Criteria for short circuit to ground or open circuit faults in ...

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 5.2
Verification Method: Analysis
Legacy ID: 68917v0

The DTC test for detection of short circuit to ground or open circuit faults in a low side driver output shall run at least while the output is deactivated.

661667 v1 Test period time for other faults for low side driver outputs

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 5.3
Verification Method: Analysis
Legacy ID: 68923v0

The DTC tests for the detection of other (than short circuit and open circuit) faults for low side driver outputs circuits shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

661660 v1 **Test period time for short circuit and open circuit faults in low side d...**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - items 5.1 and 5.2
Verification Method: Analysis
Legacy ID: 68915v0

The DTC test for the detection short circuit to ground faults, short circuit to battery and open circuit faults in low side driver outputs circuits shall produce Test Samples periodically every 100 ms or faster while the Test Run Criteria are met.

4.10.3.1.6.2.5.1.5 Detection of short circuit and open circuit faults in high side driver o...

661671 v1 **Base DTC value for faults in high side driver outputs**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - items 6.1, 6.2 and 6.3
Verification Method: Analysis
Legacy ID: 68961v0

High side driver outputs circuits which has fault detection implemented shall have one unique base DTC value for each output circuit.

661675 v2 **Detection of other faults for high side driver outputs**

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 6.3
Verification Method: Analysis
Legacy ID: 68965v0

Where a fault in a high side driver output circuit connected to the ECU, other than short circuit and open circuit, is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate Failure Type value for the corresponding DTC.
Otherwise, the ECU shall be able to detect other (than short circuit and open circuit) faults in all high side driver outputs circuits connected to the ECU as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

661668 v2 **Detection of short circuit and open circuit faults in high side driver o...**

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - items 6.1 and 6.2
Verification Method: Analysis
Legacy ID: 68924v0

The ECU shall be able to detect short circuit to ground faults, and short circuit to battery or open circuit faults, for all high side driver outputs circuits connected to the ECU.

661672 v1 Failure type value for short circuit to battery or open circuit faults in high side driver outputs

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 6.1
Verification Method: Analysis
Legacy ID: 68962v2

High side driver outputs circuits which have fault detection for short circuit to battery or open circuit faults implemented that can differentiate between the two faults shall use Failure Type value 0x12 and 0x13, as appropriate depending on the type of fault. If the fault detection can not differentiate between the two faults then Failure Type value 0x15 shall be used for both faults.

661673 v1 Failure type value for short circuit to ground faults in high side drive...

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 6.2
Verification Method: Analysis
Legacy ID: 68963v0

DTCs for short circuit to ground faults for high side driver outputs circuits shall use Failure Type value 0x11.

661670 v2 Test Run Criteria for short circuit to battery or open circuit faults in high side driver outputs

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table 4-7, item 6.1
Verification Method: Analysis
Legacy ID: 151986v0

The DTC test for detection of short circuit to battery or open circuit faults in a high side driver output shall run at least while the output is deactivated.

661674 v2 **Test Run Criteria for short circuit to ground faults in high side driver...**

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 6.2
Verification Method: Analysis
Legacy ID: 68964v0

The DTC test for detection of short circuit to ground faults in a high side driver output shall run at least while the output is activated.

661676 v1 **Test period time for other faults for high side driver outputs**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 6.3
Verification Method: Analysis
Legacy ID: 68966v0

The DTC tests for the detection of other (than short circuit and open circuit) faults for high side driver outputs circuits shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

661669 v1 **Test period time for short circuit and open circuit faults in high side ...**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - items 6.1 and 6.2
Verification Method: Analysis
Legacy ID: 68941v0

The DTC test for the detection short circuit to battery faults, short circuit to ground and open circuit faults in high side driver outputs circuits shall produce Test Samples periodically every 100 ms or faster while the Test Run Criteria are met.

4.10.3.1.6.2.5.1.6 Detection of other external electrical circuit faults

661682 v1 **DTC value for other external electrical circuit faults**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 11.1
Verification Method: Analysis
Legacy ID: 69025v0

There shall be one unique base DTC value for each output circuit which has a fault monitor for other faults, and for each fault detected on this circuit a unique Failure Type value shall be used

661680 v2 **Detection of other external electrical circuit faults**

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 11.1
Verification Method: Analysis
Legacy ID: 69023v0

Where a fault in electrical circuits connected to the ECU (e.g., cables, connectors, sensors and actuators including both the electrical and mechanical parts of sensors and actuators) is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate Failure Type value for the corresponding DTC.
Otherwise, the ECU shall be able to detect other faults in electrical circuits connected to the ECU as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

661681 v1 **Test period time for detection of other external electrical circuit faults**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 11.1
Verification Method: Analysis
Legacy ID: 69024v0

The DTC tests for the detection of other external electrical circuit faults shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

4.10.3.1.6.2.5.2 Detection of faults in internal electrical circuits

Where an ECU, or a non repairable (by the workshop) assembly unit containing the ECU, circuitry (actuators, sensors etc) internal to the ECU but not located on the main circuit board, the ECU shall be able to detect faults in these internal electrical circuits according to section *Detection of faults in external electrical circuits* as if they were external electrical circuits.

The ECU shall be able to detect faults in circuits and components located on the main circuit board of the ECU according to the table below where these have not been already covered by the requirements above.
Table: Requirements on detection of faults in internal electrical circuits

Type of item	Item no.	Detected faults/errors	Test Period	Test Run Criteria (TRC)	Failure Type (FTB)	Base DTC (display value, excl. FTB)	M/O
Memory devices	1.1	Data integrity fault in NVM (Non Volatile Memory e.g. ROM, FLASH used for storing executable code, calibration, configuration or adaptation data etc.). The specific data integrity verification methods used by the ECU are specified by the implementer. See note 2.	Specified by the implementer	Specified by the implementer	45 or 46	One for all NVM	MC
	1.2	Data integrity fault in VM (Volatile Memory, RAM). The specific data integrity verification methods used by the ECU are specified by the implementer.	Specified by the implementer	Specified by the implementer	44	One for all VM	MC
Internal Communi-cation devices	2.1	Internal device communication error (e.g. communication watchdog timeout, data integrity check, etc.)		At each communi-cation with the device	49	One for each device	MC
Programme Executing units	3.1	Programme execution fault The specific data execution monitoring methods used by the ECU are specified by the implementer. Examples of execution monitoring methods: Overall execution time exceeded, e.g. watchdog Task sequence error, e.g. checking the execution path of tasks. Task run error, e.g. checking the number of tasks run within a defined time Task timeout, e.g. checking the time between executions of the same task	Specified by the implementer	Specified by the implementer	47	One for each execution unit	MC
Internally connected actuator	4.1	Faults identified by the implementer. When determining the requirement for internal actuator failures the implementer shall consider the possible faults of the internal actuator and its internal physical and electrical interface with the ECU hardware.	100 ms See note 1		One for each fault. See note 3	One for each actuator	MC
Internally connected sensor	5.1	Faults identified by the implementer. When determining the requirement for internal sensor failures the implementer shall consider the possible faults of the internal sensor and its internal physical and electrical interface with the ECU hardware.	100 ms See note 1		One for each fault. See note 3	One for each sensor	MC
Other internal parts	6.1	Faults specified by the implementer.	100 ms See note 1	TRC1 and additional	One for each fault.	One for each internal part of	MC

				conditions specified by the implementer	See note 3	the ECU that is monitored by a test (e.g. component , circuit, system or functiont)	

Abbreviations in the table above:
TRC1 = Not when the fault is a secondary fault that appears as a result of another (primary) fault that is detected by another DTC test. For example a DTC test that has a plausibility check for an analogue sensor signal where the signal is compared with another similar signal. This test shall be inhibited if the DTC test for the compared signal detects a fault on that signal.
MC = Mandatory where the fault is detected by a fault monitor as part of the ECU functional requirements. Otherwise, a feasibility study shall be performed to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM.

Notes in the table above:
1. The defined Test Period Time is the general requirement. Depending of the actual fault detected the general requirement may not be achievable or may not be practical for the type of monitor used. In such cases the implementer may use a different Test Period Time if approved by the responsible authority at OEM.
2. Integrity check of Global ECU Configuration data is defined in *Detection of faults due to configuration errors.*
3. See *[Data_51] Diagnostic trouble code definitions*, UDS - Data - External publications.

Requirements from this section are specified in section "Requirements from section Detection of faults in internal electrical circuits".

4.10.3.1.6.2.5.2.1 Detection of internal faults not on the main circuit board

661692 v1 **Detection of internal faults not on the main circuit board**

Version history:
(1):

Purpose: Faults not on the main circuit board shall be treated as if they were external faults.
Source: Section "Detection of faults in internal electrical circuits"
Verification Method: Analysis
Legacy ID: 69540v1

Where an ECU, or a non repairable (by the workshop) assembly unit containing the ECU, contains circuitry (actuators, sensors etc) internal to the ECU but not located on the main circuit board, the ECU shall be able to detect faults in these internal electrical circuits according to section *Detection of faults in external electrical circuits* as if they were external electrical circuits.

661693 v1 **Test Run Criteria for fault detection in internal circuits**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Section "Detection of faults in internal electrical circuits"
Verification Method: Analysis
Legacy ID: 69547v1

Any fault detection test for internal circuits shall be inhibited when the fault is a secondary fault that appears as a result of another (primary) fault that is detected by another DTC test.

4.10.3.1.6.2.5.2.2 Detection of data integrity fault in non volatile memory

661696 v1 **Base DTC value for detection of data integrity fault in non volatile memory**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 1.1
Verification Method: Analysis
Legacy ID: 69556v0

Detection of data integrity fault in non volatile memory shall use one unique base DTC value for all non volatile memory in the ECU.

661694 v1 **Detection of data integrity fault in non volatile memory**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 1.1
Verification Method: Analysis
Legacy ID: 69551v0

Where a data integrity fault in non volatile memory is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect data integrity fault in non volatile memory as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).
The specific data integrity verification methods used by the ECU are specified by the implementer.

661697 v1 **Failure type value for detection of data integrity fault in non volatile...**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 1.1
Verification Method: Analysis
Legacy ID: 69557v0

DTCs for detection of data integrity fault in non volatile memory shall use Failure Type value 0x45 or 0x46.

661695 v1 **Test Run Criteria and Test Period time for detection of data integrity f...**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 1.1
Verification Method: Analysis
Legacy ID: 69554v0

The test run criteria and test period time for detection of data integrity fault in non volatile memory shall be defined by the implementer.

4.10.3.1.6.2.5.2.3 Detection of data integrity fault in volatile memory

661700 v1 **Base DTC value for detection of data integrity fault in volatile memory**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 1.2
Verification Method: Analysis
Legacy ID: 69561v0

Detection of data integrity fault in volatile memory shall use one unique base DTC value for all volatile memory in the ECU.

661698 v1 **Detection of data integrity fault in volatile memory**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 1.2
Verification Method: Analysis
Legacy ID: 69558v0

Where a data integrity fault in volatile memory is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect data integrity fault in volatile memory as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).
The specific data integrity verification methods used by the ECU are specified by the implementer.

661701 v1 **Failure type value for detection of data integrity fault in volatile memory**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 1.2
Verification Method: Analysis
Legacy ID: 69562v0

DTCs for detection of data integrity faults in volatile memory shall use Failure Type value 0x44.

661699 v1 **Test Run Criteria and Test Period time for detection of data integrity f...**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 1.2
Verification Method: Analysis
Legacy ID: 69560v0

The test run criteria and test period time for detection of data integrity fault in volatile memory shall be defined by the implementer.

4.10.3.1.6.2.5.2.4 Detection of internal device communication error

661704 v1 **DTC value for detection of internal device communication error**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 2.1
Verification Method: Analysis
Legacy ID: 69571v0

Detection of internal device communication error shall use on unique base DTC value for each monitored device to identify the fault.

661702 v1 **Detection of internal device communication error**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 2.1
Verification Method: Analysis
Legacy ID: 69564v1

Where internal device communication errors are detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect internal device communication errors as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

661705 v1 **Failure type value for detection of internal device communication error**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 2.1
Verification Method: Analysis

Legacy ID: 69572v0

DTCs for detection of internal device communication error shall use Failure Type value 0x49.

661703 v1 Test Run Criteria for detection of internal device communication error

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 2.1
Verification Method: Analysis
Legacy ID: 69565v0

If the detection of internal device communication error is implemented the DTC test shall run at each communication with the internal device.

4.10.3.1.6.2.5.2.5 Detection of programme execution fault

661708 v1 DTC value for detection of programme execution fault

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 3.1
Verification Method: Analysis
Legacy ID: 69576v0

Detection of programme execution fault shall use one unique base DTC value for each execution unit in the ECU to identify the fault.

661706 v1 Detection of programme execution fault

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 3.1
Verification Method: Analysis
Legacy ID: 69573v0

Where a programme execution fault is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect programme execution faults as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).
The specific data execution monitoring methods used by the ECU shall be specified by the implementer.

661709 v1 Failure type value for detection of programme execution fault

Version history:

(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 3.1
Verification Method: Analysis
Legacy ID: 69581v0

DTCs for detection of programme execution fault shall use Failure Type value 0x47.

661707 v1 Test Run Criteria and Test period time for detection of programme execution fault

Version history:
(1):

Purpose: The test run criteria and test period time for detection of programme execution fault are specific for each implementation and must be defined by the implementer.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 3.1
Verification Method: Analysis
Legacy ID: 69575v1

The test run criteria and test period time for detection of programme execution fault shall be defined by the implementer.

4.10.3.1.6.2.5.2.6 Detection of internally connected actuator fault

661712 v1 Base DTC value for detection of internally connected actuator fault

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 4.1
Verification Method: Analysis
Legacy ID: 69590v0

Detection of internally connected actuator fault shall use one unique base DTC value for each actuator to identify the fault.

661710 v1 Detection of internally connected actuator fault

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 4.1
Verification Method: Analysis
Legacy ID: 69584v0

Where an internally connected actuator fault is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect internally connected actuator faults as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

When determining the requirement for internal actuator failures the implementer shall consider the possible faults of the internal actuator and its internal physical and electrical interface with the ECU hardware.

661713 v1 **Failure type value for detection of internally connected actuator fault**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 5.1
Verification Method: Analysis
Legacy ID: 69595v0

DTCs for detection of internally connected actuator fault shall use a unique Failure Type value for each fault type as defined in *[Data_51] Diagnostic trouble code definitions* , UDS - Data - External publications .

661711 v1 **Test period time for detection of internally connected actuator fault**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 4.1
Verification Method: Analysis
Legacy ID: 69588v0

The DTC tests for the detection of internally connected actuator fault shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

4.10.3.1.6.2.5.2.7 Detection of internally connected sensor fault

661716 v1 **Base DTC value for detection of internally connected sensor fault**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 5.1
Verification Method: Analysis
Legacy ID: 69602v0

Detection of internally connected sensor fault shall use one unique base DTC value for each sensor to identify the fault.

661714 v1 **Detection of internally connected sensor fault**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 5.1
Verification Method: Analysis

Legacy ID: 69596v0

Where an internally connected sensor fault is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect internally connected sensor faults as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).
When determining the requirement for internal sensor failures the implementer shall consider the possible faults of the internal sensor and its internal physical and electrical interface with the ECU hardware.

661717 v1 **Failure type value for detection of internally connected sensor fault**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 5.1
Verification Method: Analysis
Legacy ID: 69605v0

DTCs for detection of internally connected sensor fault shall use a unique Failure Type value for each fault type as defined in *[Data_51] Diagnostic trouble code definitions* , UDS - Data - External publications .

661715 v1 **Test period time for detection of internally connected sensor fault**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 5.1
Verification Method: Analysis
Legacy ID: 69601v0

The DTC tests for the detection of internally connected sensor fault shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

4.10.3.1.6.2.5.2.8 Detection of faults in other internal parts

661720 v1 **Base DTC value for detection of faults in other internal parts**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 6.1
Verification Method: Analysis
Legacy ID: 69611v0

Detection of faults in other internal parts shall use one unique base DTC value for each internal part of the ECU that is monitored by a test (e.g. component , circuit, system or function) to identify the fault.

661718 v1 **Detection of faults in other internal parts**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 6.1
Verification Method: Analysis
Legacy ID: 69608v0

Where a fault in another internal part (other than identified in items 1.1 through 5.1 in table *Requirements on detection of faults in internal electrical circuits*) is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect faults in other internal parts as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

661721 v1 **Failure type value for detection of faults in other internal parts**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 6.1
Verification Method: Analysis
Legacy ID: 69612v0

DTCs for detection of faults in other internal parts shall use a unique Failure Type value for each fault type as defined in *[Data_51] Diagnostic trouble code definitions* , UDS - Data - External publications .

661719 v1 **Test period time for detection of faults in other internal parts**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 6.1
Verification Method: Analysis
Legacy ID: 69610v0

The DTC tests for the detection of faults in other internal parts shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

4.10.3.1.6.2.5.3 Detection of faults inferred from system behaviour

The ECU shall be able to detect faults inferred from system behaviour according to the table below.
Table: Requirements on detection of faults inferred from system behaviour.

Type of item	Item no.	Detected faults/errors	Test Period	Test Run Criteria (TRC)	Failure type (FTB)	Base DTC (display value, excl. FTB))	M/O

Any component or system	1.1	Failure to react after event. Sensor readings do not change as a result of some change actuated by the ECU (e.g. failure of evaporator temperature to fall after air conditioning compressor is activated).	100 ms	Changes from event are actively expected.	67	One DTC for each component, system or part of system.	MC
	1.2	The measured behaviour of a component or system does not follow the predictions of a simulation of expected behaviour which is being run by the ECU (e.g., the engine temperature does not match with predictions based on ambient temperature, engine power output and time since engine start).	100 ms	Whenever the function being simulated in the predictive model is active	61	One DTC for each component, system or part of system.	MC
	1.3	A closed loop control (e.g. of temperature, motor position, engine speed etc.) is not stable under conditions where stability would be expected	100 ms	Stable closed loop control actively expected.	06	One DTC for each component, system or part of system.	MC
	1.4	System adaptation out of range. A system where the ECU adapts to manufacturing tolerance, component ageing etc. has exceeded the limit of its adaptation range.		Whenever any adaptation value is updated.	91	One DTC for each component, system or part of system.	MC
	1.5	Faults specified by the implementer.	100 ms. See note 1.	TRC1 and additional conditions specified by the implementer	See note 2.	One DTC for each component, system or part of system.	MC

Abbreviations in the table above:
TRC1 = Not when the fault is a secondary fault that appears as a result of another (primary) fault that is detected by another DTC test. For example a DTC test that has a plausibility check for an analogue sensor signal where the signal is compared with another similar signal. This test shall be inhibited if the DTC test for the compared signal detects a fault on that signal.
MC = Mandatory where the fault is detected by a fault monitor as part of the ECU functional requirements. Otherwise, a feasibility study shall be performed to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM.

Notes in the table above:
1. The defined Test Period Time is the general requirement for this type of fault. Depending of the actual fault detected the general requirement may not be achievable or may not be practical for the type of monitor used. In such cases the implementer may use a different Test Period Time if approved by the responsible authority at OEM.
2 See *[Data_51] Diagnostic trouble code definitions*, UDS - Data - External publications.

Requirements from this section are specified in section "Requirements from section Detection of faults inferred from system behaviour".

4.10.3.1.6.2.5.3.1 Detection of failure to react after event

661727 v1 **Base DTC value for detection of failure to react after event**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 1.1
Verification Method: Analysis
Legacy ID: 69644v0

Detection of failure to react after event shall use a unique base DTC value for each component, system or part of system that is monitored, to identify the fault.

661723 v1 Detection of failure to react after event

Version history:
(1):

Purpose: Where no direct fault detection mechanism is available, or where it is inadequate, the ECU shall be able to indirectly detect faults in the system and components associated with the ECU.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 1.1
Verification Method: Analysis
Legacy ID: 69628v0

If the ECU is able to detect if sensor readings do not change as a result of some change actuated by the ECU, as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect if sensor readings do not change as a result of some change actuated by the ECU, as defined by a feasibility study performed by the implementer. (The feasibility study shall be performed to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

661726 v1 Failure type value for detection of failure to react after event

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 1.1
Verification Method: Analysis
Legacy ID: 69642v0

DTCs for detection of failure to react after event shall use Failure Type value 0x67.

661724 v2 Test Run Criteria for detection of failure to react after event

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 1.1
Verification Method: Analysis
Legacy ID: 69631v0

If the detection of failure to react after event is implemented the DTC test shall run whenever changes from event are actively expected.

661725 v1 **Test period time for detection of failure to react after event**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 1.1
Verification Method: Analysis
Legacy ID: 69639v0

If the detection of failure to react after event is implemented the DTC test shall run periodically every 100 ms or faster whenever the Test Run Criteria are met.

4.10.3.1.6.2.5.3.2 Detection of Comparison of system behaviour with predictive model

661732 v1 **Base DTC value for comparison of system behaviour with predictive model**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 2.1
Verification Method: Analysis
Legacy ID: 69657v0

Comparison of system behaviour with predictive model shall use a unique base DTC value for each component, system or part of system that is monitored, to identify the fault.

661728 v1 **Comparison of system behaviour with predictive model**

Version history:
(1):

Purpose: Where no direct fault detection mechanism is available, or where it is inadequate, the ECU shall be able to indirectly detect faults in the system and components associated with the ECU.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 2.1
Verification Method: Analysis
Legacy ID: 69647v0

If the ECU is able to detect if the measured behaviour of a component or system do not follow the predictions of a simulation of expected behaviour which is being run by the ECU , as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect if the measured behaviour of a component or system do not follow the predictions of a simulation of expected behaviour which is being run by the ECU , as defined by a feasibility study performed by the implementer. (The feasibility study shall be performed to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

661731 v1 **Failure type value for comparison of system behaviour with predictive model**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 2.1
Verification Method: Analysis
Legacy ID: 69653v0

DTCs for comparison of system behaviour with predictive model shall use Failure Type value 0x61.

661729 v2 Test Run Criteria for comparison of system behaviour with predictive model

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 2.1
Verification Method: Analysis
Legacy ID: 69648v0

If comparison of system behaviour with predictive model is implemented the DTC test shall run whenever the function being simulated in the predictive model is active.

661730 v1 Test period time for comparison of system behaviour with predictive model

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 2.1
Verification Method: Analysis
Legacy ID: 69652v0

If comparison of system behaviour with predictive model is implemented the DTC test shall run periodically every 100 ms or faster whenever the Test Run Criteria are met.

4.10.3.1.6.2.5.3.3 Detection of closed loop control system out of control

661737 v1 Base DTC value for detection of closed loop control system out of control

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 3.1
Verification Method: Analysis
Legacy ID: 69662v0

Detection of closed loop control system out of control shall use a unique base DTC value for each component, system or part of system that is monitored, to identify the fault.

661733 v1 Detection of closed loop control system out of control

Version history:
(1):

Purpose: Where no direct fault detection mechanism is available, or where it is inadequate, the ECU shall be able to indirectly detect faults in the system and components associated with the ECU.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 3.1
Verification Method: Analysis
Legacy ID: 69658v0

If the ECU is able to detect if a closed loop control is not stable under conditions where stability would be expected , as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect if a closed loop control is not stable under conditions where stability would be expected , as defined by a feasibility study performed by the implementer. (The feasibility study shall be performed to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

661736 v1 Failure type value for detection of closed loop control system out of co...

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 3.1
Verification Method: Analysis
Legacy ID: 69661v0

DTCs for detection of closed loop control system out of control shall use Failure Type value 0x06.

661734 v2 Test Run Criteria for detection of closed loop control system out of con...

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 3.1
Verification Method: Analysis
Legacy ID: 69659v0

If the detection of closed loop control system out of control is implemented the DTC test shall run whenever stable closed loop control is actively expected.

661735 v1 Test period time for detection of closed loop control system out of control

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 3.1
Verification Method: Analysis
Legacy ID: 69660v0

If the detection of closed loop control system out of control is implemented the DTC test shall run periodically every 100 ms or faster whenever the Test Run Criteria are met.

4.10.3.1.6.2.5.3.4 Detection of system adaptation out of range

661741 v1 **Base DTC value for detection of system adaptation out of range**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 4.1
Verification Method: Analysis
Legacy ID: 69666v0

Detection of system adaptation out of range shall use a unique base DTC value for each component, system or part of system that is monitored, to identify the fault.

661738 v1 **Detection of system adaptation out of range**

Version history:
(1):

Purpose: Where no direct fault detection mechanism is available, or where it is inadequate, the ECU shall be able to indirectly detect faults in the system and components associated with the ECU.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 4.1
Verification Method: Analysis
Legacy ID: 69663v0

If the ECU is able to detect if a ystem where the ECU adapts to manufacturing tolerance, component ageing etc., has exceeded the limit of its adaptation range , as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect if a ystem where the ECU adapts to manufacturing tolerance, component ageing etc., has exceeded the limit of its adaptation range , as defined by a feasibility study performed by the implementer. (The feasibility study shall be performed to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

661740 v1 **Failure type value for detection of system adaptation out of range**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 4.1
Verification Method: Analysis
Legacy ID: 69665v0

DTCs for detection of system adaptation out of range shall use Failure Type value 0x91.

661739 v2 **Test Run Criteria for detection of system adaptation out of range**

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.

Source: Table "Requirements on detection of faults inferred from system behaviour" - item 4.1
Verification Method: Analysis
Legacy ID: 69664v0

If the detection of system adaptation out of range is implemented the DTC test shall run whenever any adaptation value is updated.

4.10.3.1.6.2.5.3.5 Detection of other faults inferred from system behaviour

661744 v1 **Base DTC value for detection of other faults inferred from system behaviour**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 5.1
Verification Method: Analysis
Legacy ID: 69669v0

Detection of other faults faults inferred from system behaviour shall use one unique base DTC value for each component, system or part of system that is monitored, to identify the fault.

661742 v1 **Detection of other faults inferred from system behaviour**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 5.1
Verification Method: Analysis
Legacy ID: 69667v0

Where another fault inferred from system behaviour is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect faults inferred from system behaviour as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

661745 v1 **Failure type value for detection of other faults inferred from system be...**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 5.1
Verification Method: Analysis
Legacy ID: 69670v0

DTCs for detection of other faults faults inferred from system behaviour shall use a unique Failure Type value for each fault type as defined in *[Data_51] Diagnostic trouble code definitions* , UDS - Data - External publications.

661722 v2 **Test Run Criteria for detection of faults inferred from system behaviour**

Version history:
(2): Verification Method: Inspection change to Analysis.

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source:
Verification Method: Analysis
Legacy ID: 115341v0

Any fault detection test for detection of faults inferred from system behaviour shall be inhibited when the fault is a secondary fault that appears as a result of another (primary) fault that is detected by another DTC test.

661743 v1 **Test period time for detection of other faults inferred from system beha...**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item
Verification Method: Analysis
Legacy ID: 69668v0

The DTC tests specified by the implementer for the detection of othe faults inferred from system behaviour shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

4.10.3.1.6.2.5.4 Detection of faults due to configuration errors

An ECU shall be able to detect faults caused by errors in the configuration of the ECU, software incompatibility and hardware incompatibility, as specified in the tables found in this section .

Table: Requirements on detection of faults caused by local ECU Configuration (local parameters).

Type of item	Item no.	Detected faults/errors	Test Period	Test Run Criteria (TRC)	Failure type (FTB)	Base DTC (display value, excl. FTB)	M/O
Local Configuration data	1.1	Invalid configuration data. Compare value of configured item with allowed range of values for that item. The ECU has one or more configured parameters with unknown/invalid values. Refer to *[Data_8] Overview of vehicle configuration & customer* settnings for details regarding local configuration.		Whenever a configured parameter is used AND/OR whenever a configured parameter is changed	56	U2101	MC1

Table: Requirements on detection of faults caused by Global ECU Configuration (global parameters)

Type of item	Item no.	Detected faults/errors	Test Period	Test Run Criteria (TRC)	Failure type (FTB)	Base DTC (display value, excl. FTB)	M/O
Global Configuration data	2.1	Missing vehicle configuration data See *[Data_5]Central Car Configuration*, DTC 'Car Configuration – Not Configured', and table "Calibration of configuration DTCs" in section *Calibration of configuration DTCs.*		Enabled while ECU is in 'Bulk state'. Test runs each time a Car Configuration block is received. Test is inhibited once passed.	55	U2300	MC4
	2.2	Reserved for future use.					
	2.3	Invalid vehicle configuration data See *[Data_5] Central Car Configuration*, DTC 'Car Configuration – Invalid/Incompatible Configuration', and table "Calibration of configuration DTCs" in section *Calibration of configuration DTCs.*		Every time a global configuration parameter used for configuration of the ECU is received until test passed.	56	U2300	MC4
	2.4	New configuration not learned See *[Data_5] Central Car Configuration*, DTC 'Car Configuration – Not programmed', and table "Calibration of configuration DTCs" in section *Calibration of configuration DTCs.*		Every time a global configuration parameter used for configuration of the ECU is received until test passed	51	U2300	MC5
	2.5	Inconsistent configuration data See table "Calibration of configuration DTCs" in section *Calibration of configuration DTCs.*		Every time a global configuration parameter used for configuration of the ECU is received until test passed	57	U2300	MC2
Configuration data storage	3.1	The ECU has corrupt storage of configuration data. See *[Data_5] Central Car Configuration*, Req4, and table "Calibration of configuration DTCs" in section *Calibration of configuration DTCs.*		Specified by the implementer.	41	U2301	MC3
Car Configuration Logic	4.1	Incompatible Vehicle Configuration Data (VCD) and Car Configuration Logic (CCL) data loaded. See *[Data_5] Central Car Configuration* , DTC 'Car Configuration – VCP Not sustainable', and table "Calibration of configuration DTCs" in section *Calibration of configuration DTCs.*		Whenever the Vehicle Configuration Parameters (VCP) generation routine is executed	57	U2302	MC3
	4.2	No Car Configuration Logic (CCL) is present in CCL memory. See *[Data_5] Central Car Configuration* , DTC 'Car		Whenever the Vehicle Configuration Parameters (VCP)	55	U2302	MC3

		Configuration – CCL Not available', and table "Calibration of configuration DTCs" in section *Calibration of configuration DTCs.*		generation routine is executed			
Vehicle Configuration Data	5.1	No Vehicle Configuration Data (VCD) is present in VCD memory. See *[Data_5] Central Car Configuration*, DTC 'Car Configuration – VCD Not available', and table "Calibration of configuration DTCs" in section *Calibration of configuration DTCs.*		Whenever the Vehicle Configuration Parameters (VCP) generation routine is executed	55	U2303	MC3
Public ECU network interface	6.1	Configuration check resulted in disapproved. Refer to implementation specification for the respective network (e.g. UDSonCAN, UDSonFR and UDSonIP).					

Table: Requirements on detection of faults caused by incompatible or missing software components

Type of item	Item no.	Detected faults/errors	Test Period	Test Run Criteria (TRC)	Failure type (FTB)	Base DTC (display value, excl. FTB)	M/O
VIN	7.1	Incompatible VIN. The locally stored VIN does not match the VIN transmitted from another ECU		Once per operation cycle	00	P0630, P0631, C0545 or C0546	MC2
ECU Software	8.1	The ECU has a check mechanism where the application/strategy/data level of another ECU can be checked by the ECU. Data Indicates Software incompatibility with another ECU	Test time is determined by the functional need of the ECU to ensure correct operation		4A if module must be replaced, 57 If module must be reprogrammed or 00 if cause not known	U03XX or U2DXX (software incompatibility with ECU XX. See note 2 for definition.	MC2
	8.2	The ECU is delivered to OEM with supplier specific software that must be replaced by OEM authorised application software as part of the manufacturing process. See note 1.	N/A	N/A	57	U2400 Supplier software installed	MC6

Table: Requirements on detection of faults caused by production SW download data not programmed

Type of item	Item no.	Detected faults/errors	Test Period	Test Run Criteria (TRC)	Failure type (FTB)	Base DTC (display value, excl. FTB)	M/O
Security Constant	9.1	Production security constant used for security level 01/02 does not be programmed. Initial value be still used.		At least once per operation cycle	51	U114B	MC7
	9.2	Production security constant used for security level 19/1A		At least once per	51	U1634	MC9

		does not be programmed. Initial value be still used.		operation cycle			
Public Key	10.1	Production public key used for SWDL verification does not be programmed. Development public key be still used for SW verification		At least once per operation cycle	51	U114C	MC8

Abbreviations in the three tables above:

MC1 = Mandatory if the ECU is required to be programmed with local configuration (calibration) data for correct operation.

MC2 = Mandatory where the fault is detected by a fault monitor as part of the ECU functional requirements.

MC6 = Mandatory if the ECU is delivered to OEM with supplier specific software that must be replaced by OEM authorised application software as part of the manufacturing process.

MC7 = Mandatory if the ECU supports SWDL.

MC8 = Mandatory if the ECU supports SWDL authentication verification.

MC9 = Mandatory if the ECU supports Security Level 19/1A.

Notes in the three tables above:

1 This DTC shall always be set as long as the supplier specific software (not intended for use by OEM)§ is installed in the ECU. It shall not be possible to delete (clear) this DTC except by erasing or replacing the supplier specific software. The DTC shall not be supported by any OEM authorised application software.

2 See *[Data_4] Global Master Reference Database* and *[Data_51] Diagnostic trouble code definitions*.

Requirements from this section are specified in section "Requirements from section Detection of faults due to configuration errors".

4.10.3.1.6.2.5.4.1 Detection of software incompatibility with another ECU

661754 v1 **Base DTC value for detection of software incompatibility with another ECU**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by incompatible or missing software components" - item 8.1
Verification Method: Analysis
Legacy ID: 70842v0

The DTC test for detection of software incompatibility with another ECU shall use the base DTC value U03XX if defined in *[Data_51] Diagnostic trouble code definitions* , UDS - Data - External publications , otherwise base DTC value U2DXX shall be used to identify this fault.

661757 v1 **Clearing of DTC for detection of suppler software installed in ECU.**

Version history:
(1):

Purpose: The DTC for indicating that the ECU has supplier specific software installed shall not be

possible to clear except by replacing the software.
Source: Table "Requirements on detection of faults caused by incompatible or missing software components" - item 8.2
Verification Method: Analysis
Legacy ID: 70845v0

The DTC U2400-57 shall always be set as long as the supplier specific software is installed in the ECU. It shall not be possible to delete (clear) this DTC except by erasing or replacing the supplier software. The DTC shall not be supported by any OEM authorised application software.

661753 v1 Detection of software incompatibility with another ECU

Version history:
(1):

Purpose: To ensure that it can be detected if an ECU has software loaded that is incompatible with another ECU.
Source: Table "Requirements on detection of faults caused by incompatible or missing software components" - item 8.1
Verification Method: Analysis
Legacy ID: 70841v0

If t he ECU has a fault monitor implemented as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) where the application/strategy/data level of another ECU can be checked and indicate software incompatibility with the other ECU, this test hsall also be used as a DTC test.

661756 v1 Detection of suppler specific software installed in ECU

Version history:
(1):

Purpose: To ensure that an ECU with supplier specific software installed will not pass undetected through the manufacturing process.
Source: Table "Requirements on detection of faults caused by incompatible or missing software components" - item 8.2
Verification Method: Analysis
Legacy ID: 70844v0

If t he ECU is delivered to GRI with supplier specific software that must be replaced by GRI authorised application software as part of the manufacturing process this shall be identified by a confirmed DTC with the DTC value U2400-57.

661755 v1 Failure type value for detection of software incompatibility with anothe...

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by incompatible or missing software components" - item 8.1
Verification Method: Analysis
Legacy ID: 70843v0

The DTCs for detection of software incompatibility with another ECU shall use Failure Type value:
0x4A if module must be replaced
0x57 If module must be reprogrammed

0x00 if cause is not known

4.10.3.1.6.2.5.5 Detection of faults caused by received signals

The ECU shall be able to detect faults caused by signals and data values received from other ECUs as specified in the table below.
Note: the faults detailed in this table are intended to handle the situation where there may be a customer complaint and a symptom associated with the ECU generating the DTC but the root cause is located on another ECU which may or may not be exhibiting any symptoms.

Table: Requirements on detection of faults caused by signal/data content received from other ECUs.

Type of item	Item no.	Detected faults/errors	Test Period	Test Run Criteria (TRC)	Failure Type (FTB)	Base DTC (display value, excl. FTB)	M/O
Other ECU not operating normally	1.1	Another ECU sends data that indicates it has detected a fault which causes it to operate with reduced or not correct functionality. See note 1.		Each time the updated data (indicating reduced functionality in the other ECU) is received.	92	U2C00 to U2CFF One DTC per signal as defined in (see note 3).	MC1
Other ECU sending data tagged as invalid	2.1	Another ECU has detected a fault which causes data sent by the other ECU to be tagged as invalid. See note 2.		Each time updated data is received from another ECU. TRC1	81	U2C00 to U2CFF One DTC per signal as defined in (see note 3).	MC1
Implausible data from another ECU	3.1	Data from another ECU is not tagged as invalid but detected as faulty by the ECU, e.g. by plausibility check(i.e. when compared with one or more external or internal signals and/or the current operating conditions).		Each time updated data is received. TRC2	86	U2D00 to U2DFF One DTC per signal as defined in (see note 4).	MC1
Corrupt data from another ECU	4.1	Data value from other ECU detected as corrupt by the data protection mechanism “rolling counter” implemented at application level in the ECU. The data is received without the corresponding rolling count value being properly updated (e.g. incremented by the value 1 when expected).		Each time updated data is received.	82	U2D00 to U2DFF One DTC per signal as defined in (see note 4).	MC2
	4.2	Data value from other ECU detected as corrupt by the data protection mechanism “end-to-end checksum” implemented at application level in the ECU.		Each time updated data is received	83	U2D00 to U2DFF One DTC per signal as defined in (see note 4).	MC2
	4.3	Data value from other ECU detected as corrupt by the data protection mechanism “up-date bit” implemented at application level in the ECU. The data is received without the corresponding up-date bit value being properly set (e.g. set to the value 1 when expected).		Each time updated data is expected to be received.	87	U1000 to U2EFF if the DTC is not emission related. U0100 to U02FF if the DTC is emission related.	O See note 5.

		Note: The faults are identified by DTCs that are known as "LostComDTCs". If the DTC is not emission related the DTC status indicator 30:5 (SI30:5) – warningIndicatorRequestedSinceLastClear is allowed to be set to 1 for these DTCs despite the FDC10 never reach +127 as the DTCs are development specific. Refer to Table: "Requirements on implementation of DTC status indicators" for requirements on set of DTC SI 30:5 to 1. It is up to the implementer to specify the criteria for set the DTC SI 30:5 to 1				One DTC per signal as defined in (see note 3).	

Abbreviations in the table above:
MC1 = Mandatory where the fault is detected by a fault monitor as part of the ECU functional requirements, i.e. where the primary reason for the fault monitor is another than to support generation of DTC information. Otherwise, a feasibility study shall be performed to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM.
MC2 = Mandatory where the fault is detected by a fault monitor as part of the ECU functional requirements.
TRC1 = This test shall be inhibited if the received data is detected as corrupt by a data protection mechanism (item 4.1 to 4.3).
TRC2 = If plausibility check is used as fault detection logic the test shall be inhibited if a DTC test for one (or more) of the compared signals (used to evaluate the plausibility of the received signal) has detected a fault on that signal.
Notes in the table above:
1. In all cases the ECU transmitting the signal that indicates it is not operating normally shall set an appropriate DTC. This can be either a DTC relating to the root cause of the situation if the root cause exists in the transmitting ECU, or a secondary DTC if the root cause is not in the transmitting ECU.
2. In all cases the ECU transmitting the signal tagged as invalid shall set an appropriate DTC. This can be either a DTC relating to the root cause of the invalid data if the root cause exists in the transmitting ECU, or a secondary DTC if the root cause is not in the transmitting ECU.
3. *[Data_4] Global Master Reference Database* and *[Data_51] Diagnostic trouble code definitions*, UDS - Data - External publications
4. *[Data_4] Global Master Reference Database*
5. The DTC may be mandatory if the DTC is emission related but this is not specified by this document. Requirements from this section are specified in section "Requirements from section Detection of faults caused by received signals".

4.10.3.1.6.2.5.5.1 Detection of other ECU not operating normally

661782 v1 **Base DTC value for detection of other ECU not operating normally**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 1.1
Verification Method: Analysis
Legacy ID: 70872v0

The DTC test for detection of other ECU not operating normally shall use one unique base DTC value for each signal (indicating reduced functionality in the other ECU), in the range U2C00 to U2CFF or U2600 to U26FF, as defined in:
[Data_51] Diagnostic trouble code definitions
[Data_4] Global Master Reference Database

661780 v1 Detection of other ECU not operating normally

Version history:
(1):

Purpose: To handle the situation where there may be a customer complaint and a symptom associated with the ECU generating the DTC but the root cause is located on another ECU which may or may not be exhibiting any symptoms.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 1.1
Verification Method: Analysis
Legacy ID: 70867v0

If the ECU detects that another ECU sends data that indicates it has detected a fault which causes it to operate with reduced or not correct functionality, as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect that other ECU not operating normally as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

661783 v1 Failure type value for detection of other ECU not operating normally

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 1.1
Verification Method: Analysis
Legacy ID: 70875v0

The DTCs for detection of other ECU not operating normally shall use Failure Type value 0x92.

661784 v1 Root DTC for ECU not operating normally

Version history:
(1):

Purpose: To enable identification of the root cause of the DTC for other ECU not operating normally.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 1.1
Verification Method: Analysis
Legacy ID: 70877v0

The ECU transmitting a signal that indicates it is not operating normally shall set an appropriate DTC for this fault (this can be either a DTC relating to the root cause of the situation if the root cause exists in the transmitting ECU, or a secondary DTC if the root cause is not in the transmitting ECU).

661781 v1 **Test Run Criteria for detection of other ECU not operating normally**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 1.
Verification Method: Analysis
Legacy ID: 70869v0

The DTC test for detection of other ECU not operating normally shall run each time the updated data indicating reduced functionality in the other ECU is received.

4.10.3.1.6.2.5.5.2 Detection of other ECU sending data tagged as invalid

661787 v1 **Base DTC value for detection of other ECU sending data tagged as invalid**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 2.1
Verification Method: Analysis
Legacy ID: 71743v0

The DTC test for detection of other ECU sending data tagged as invalid shall use one unique base DTC value for each signal, in the range U2C00 to U2CFF or U2600 to U26FF, as defined in *[Data_51] Diagnostic trouble code definitions* and *[Data_4] Global Master Reference Database.*

661785 v2 **Detection of other ECU sending data tagged as invalid**

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: To handle the situation where there may be a customer complaint and a symptom associated with the ECU generating the DTC but the root cause is located on another ECU which may or may not be exhibiting any symptoms.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 2.1
Verification Method: Analysis
Legacy ID: 70878v0

If the ECU detects that another ECU sends data tagged as invalid, as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect that other ECU sends data tagged as invalid as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

661788 v1 **Failure type value for detection of other ECU sending data tagged as inv...**

Version history:

(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 2.1
Verification Method: Analysis
Legacy ID: 71744v0

The DTCs for detection of other ECU sending data tagged as invalid shall use Failure Type value 0x81.

661789 v1 Root DTC for ECU sending data tagged as invalid

Version history:
(1):

Purpose: To enable identification of the root cause of the DTC for other ECU sending data tagged as invalid.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 2.1
Verification Method: Analysis
Legacy ID: 71745v0

The ECU transmitting a signal that is tagged as invalid shall set an appropriate DTC for this fault (this can be either a DTC relating to the root cause of the situation if the root cause exists in the transmitting ECU, or a secondary DTC if the root cause is not in the transmitting ECU).

661786 v1 Test Run Criteria for detection of other ECU sending data tagged as invalid

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 2.1
Verification Method: Analysis
Legacy ID: 71423v0

The DTC test for detection of other ECU sending data tagged as invalid shall run each time updated data is received from another ECU.
The test shall be inhibited if the received data is detected as corrupt by a data protection mechanism.

4.10.3.1.6.2.5.5.3 Detection of other ECU sending faulty data

661792 v1 Base DTC value for detection of other ECU sending faulty data

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 3.1
Verification Method: Analysis
Legacy ID: 71748v0

The DTC test for detection of other ECU sending faulty data shall use one unique base DTC value for each tested

signal, in the range U2D00 to U2DFF or U2700 to U27FF, as defined in *[Data_51] Diagnostic trouble code definitions* and *[Data_4] Global Master Reference Database*.

661790 v2 Detection of other ECU sending faulty data

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: To handle the situation where there may be a customer complaint and a symptom associated with the ECU generating the DTC but the root cause is located on another ECU which may or may not be exhibiting any symptoms.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 3.1
Verification Method: Analysis
Legacy ID: 71746v0

If the ECU detects that another ECU sends data that is not tagged as invalid but still detected as faulty by the ECU, e.g. by plausibility check, as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect that other ECU sends faulty data as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

661793 v1 Failure type value for detection of other ECU sending faulty data

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 3.1
Verification Method: Analysis
Legacy ID: 71750v0

The DTCs for detection of other ECU sending faulty data shall use Failure Type value 0x86.

661791 v1 Test Run Criteria for detection of other ECU sending faulty data

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 3.1
Verification Method: Analysis
Legacy ID: 71747v0

The DTC test for detection of other ECU sending faulty data shall run each time updated data, for which a test i simplemented, is received from another ECU.
If plausibility check is used as fault detection logic the test shall be inhibited if a DTC test for one (or more) of the compared signals (used to evaluate the plausibility of the received signal) has detected a fault on that signal.

4.10.3.1.6.2.5.5.4 Detection of corrupt data received from other ECU by rolling counter

661796 v1 Base DTC value for detection of corrupt data received from other ECU by rolling counter

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 4.1
Verification Method: Analysis
Legacy ID: 71754v1

The DTC test for detection of corrupt data received from other ECU by rolling counter shall use one unique base DTC value for each tested signal, in the range U2D00 to U2DFF or U2700 to U27FF, as defined in *[Data_51] Diagnostic trouble code definitions* , UDS - Data - External publications and [Data_4] *Global Master Reference Database*.

661794 v1 Detection of corrupt data received from other ECU by rolling counter

Version history:
(1):

Purpose: To handle the situation where there may be a customer complaint and a symptom associated with the ECU generating the DTC but the root cause is located on another ECU which may or may not be exhibiting any symptoms.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 4.1
Verification Method: Analysis
Legacy ID: 71752v1

If the ECU detects, by the data protection mechanism rolling counter implemented at application level in the ECU, that a data value received from other ECU is corrupted (the data is received without the corresponding rolling count value being properly updated e.g. incremented by the value 1 when expected), as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.

661797 v1 Failure type value for detection of corrupt data received from other ECU by rolling counter

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 4.1
Verification Method: Analysis
Legacy ID: 71755v1

The DTCs for detection of corrupt data received from other ECU by rolling counter shall use Failure Type value 0x82.

661795 v1 Test Run Criteria for detection of corrupt data received from other ECU by rolling counter

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 4.1
Verification Method: Analysis
Legacy ID: 71753v1

The DTC test for detection of corrupt data received from other ECU by rolling counter shall run each time updated data, for which a test i simplemented, is received from another ECU.

4.10.3.1.6.2.5.5.5 Detection of corrupt data received from other ECU by end-to-end checksum

661800 v1 Base DTC value for detection of corrupt data received from other ECU by end-to-end checksum

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs"
Verification Method: Analysis
Legacy ID: 332536v0

The DTC test for detection of corrupt data received from other ECU by end-to-end checksum shall use one unique base DTC value for each tested signal, in the range U2D00 to U2DFF or U2700 to U27FF, as defined in *[Data_51] Diagnostic trouble code definitions*, UDS - Data - External publications and *[Data_4] Global Master Reference Database*.

661798 v1 Detection of corrupt data received from other ECU by end-to-end checksum

Version history:
(1):

Purpose: To handle the situation where there may be a customer complaint and a symptom associated with the ECU generating the DTC but the root cause is located on another ECU which may or may not be exhibiting any symptoms.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs"
Verification Method: Analysis
Legacy ID: 332533v0

If the ECU detects, by the data protection mechanism end-to-end checksum implemented at application level in the ECU, that a data value received from other ECU is corrupted each such fault shall also be identified by an appropriate DTC.

661801 v1 Failure type value for detection of corrupt data received from other ECU by end-to-end checksum

Version history:
(1):

Purpose: To enable easy identification of specific faults.

Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs"
Verification Method: Analysis
Legacy ID: 332537v1

The DTCs for detection of corrupt data received from other ECU by end-to-end checksum shall use Failure Type value 0x83.

661799 v1 Test Run Criteria for detection of corrupt data received from other ECU by end-to-end checksum

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs"
Verification Method: Analysis
Legacy ID: 332535v0

The DTC test for detection of corrupt data received from other ECU by end-to-end checksum shall run each time updated data, for which a test i simplemented, is received from another ECU.

4.10.3.1.6.2.5.5.6 Detection of corrupt data received from other ECU by up-date-bit

661804 v1 Base DTC value for detection of corrupt data received from other ECU by up-date-bit

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs"
Verification Method: Analysis
Legacy ID: 332571v2

The DTC test for detection of corrupt data received from other ECU by up-date-bit shall use one unique base DTC value for each tested signal, in the range U2F00 to U2FFF if the DTC is not emission related and U0100 to U02FF if the DTC is emission related, as defined in *[Data_51] Diagnostic trouble code definitions*, UDS - Data - External publications and *[Data_4] Global Master Reference Database*.

661802 v1 Detection of corrupt data received from other ECU by up-date-bit

Version history:
(1):

Purpose: To handle the situation where there may be a customer complaint and a symptom associated with the ECU generating the DTC but the root cause is located on another ECU which may or may not be exhibiting any symptoms.
Source: able "Requirements on detection of faults caused by signal/data content received from other ECUs"
Verification Method: Analysis
Legacy ID: 332551v2

If the ECU detects, by the data protection mechanism end-to-end checksum implemented at application level in the ECU (the data is received without the corresponding up-date bit value being properly set e.g. set to the value 1 when expected), that a data value received from other ECU is corrupted each such fault may also be identified by an appropriate DTC.
Note: The DTC may be mandatory if the DTC is emission related but this is not specified by this document.
Note: The faults are identified by DTCs that are known as "LostComDTCs".
If the DTC is not emission related the DTC status indicator 30:5 (SI30:5) – warningIndicatorRequestedSinceLastClear is allowed to be set to 1 for these DTCs despite the FDC10 never reach +127 as the DTCs are development specific. Refer to Table: “Requirements on implementation of DTC status indicators” for requirements on set of DTC SI 30:5 to 1. It is up to the implementer to specify the criteria for set the DTC SI 30:5 to 1.

661805 v2 Failure type value for detection of corrupt data received from other ECU by up-date bit

Version history:
(2): Failure Type 0x87 is more appliable.

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs"
Verification Method: Analysis
Legacy ID: 332572v0

The DTCs for detection of corrupt data received from other ECU by up-date bit shall use Failure Type value 0x87.

661803 v1 Test Run Criteria for detection of corrupt data received from other ECU by up-date bit

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: ble "Requirements on detection of faults caused by signal/data content received from other ECUs"
Verification Method: Analysis
Legacy ID: 332552v1

The DTC test for detection of corrupt data received from other ECU by up-date bit shall run each time updated data, for which a test i simplemented, is received from another ECU.

4.10.3.1.6.2.5.6 Detection of faults caused by the ECU environment

An ECU shall be able to detect faults caused by the ECU operating environment (e.g. temperature, operating voltage, etc.) or by certain customer actions (e.g. operating audio system continuously at maximum volume) where these faults cause symptoms that a customer would both notice and not expect given the (environmental or other) conditions observable by the customer. Detailed requirements and allowed exceptions are specified in the table below

The DTCs detailed in this table are intended to handle the situation where there may be a customer complaint and a symptom associated with the ECU (typically the ECU operates with reduced functionality) but the root cause is that the ECU is operating outside of its design specification. The DTCs in this table are generally set to provide a warning or explanation to the customer which describes the cause of the loss of function and to re-assure them that no repair is necessary. The DTCs in this table are generally set to prevent the ECU being incorrectly suspected as faulty and to prevent unnecessary repair work.

Note that during assembly of the vehicle, all DTCs associated with these faults shall not require any repair action. Furthermore, these faults will most likely not be presented in any of the manufacturing tools.

Table: Requirements on detection of faults caused by the ECU operating environment or customer actions

Type of item	Item no.	Detected faults/errors	Test Period	Test Run Criteria (TRC)	Failure Type (FTB)	Base DTC (display value, excl. FTB)	M/O
Ambient temperature	1.1	Ambient temperature too high for full and correct operation.	100 ms		68	U2E00	O
	1.2	Ambient temperature too low for full and correct operation.	100 ms		68	U2E01	O
Customer actions	2.1	Customer operates vehicle or vehicle equipment outside specification so that not correct, reduced or no system functionality is available, or a fallback strategy is activated (e.g. customer action causes temporary component overheating).	100 ms		68	U2E02	MC
Power supply voltage	3.1	ECU supply voltage is too high for full operation. ECU has symptom of not correct, reduced or no function for a period of time which the customer would notice.	100 ms		68	U2E03	MC
	3.2	ECU supply voltage is too low for full operation at a time when an average customer would not have expected low battery voltage (i.e. not during cranking or when the engine is not running). ECU has symptom of reduced or no function for a period of time which the customer would both notice and not expect (normally an obvious default behaviour latched for the complete operation cycle and/or a warning lamp indication would be required before this DTC may be set).	100 ms	TRC1	68	U2E04	MC
Other condition required for normal ECU functionality	4.1	A condition, not covered by other items in this table, is out of range for full (and correct) operation. ECU has the symptom of not correct, reduced or no function which the customer would both notice and not expect given the conditions observable by the customer.	100 ms. See note 1.	TRC2 and additional conditions specified by the implementer	68	U2E05 to U2EFF. One DTC per condition that is out of range as defined in reference	MC

						[Data_4] Global Master Reference Database.	

Abbreviations in the table above:
TRC1 Not when ECU supply voltage check with the vehicle battery voltage has detected a problem (See Table *Requirements on detection of faults in external electrical circuits connected to the ECU*- DTC U3003-62).

TRC2 Not when the fault is a secondary fault that appears as a result of another (primary) fault that is detected by another DTC test. For example a DTC test that has a plausibility check for an analogue sensor signal where the signal is compared with another similar signal. This test shall be inhibited if the DTC test for the compared signal detects a fault on that signal.

MC Mandatory where the fault is detected by a fault monitor as part of the ECU functional requirements. Otherwise, a feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM.

Notes in the table above:

1. The defined Test Period Time is the general requirement for this type of fault. Depending of the actual fault detected the general requirement may not be achievable or may not be practical for the type of monitor used. In such cases the implementer may use a different Test Period Time if approved by the responsible authority at OEM.

Requirements from this section are specified in section "Requirements from section Detection of faults caused by the ECU environment".

4.10.3.1.6.2.5.6.1 Detection of events caused by other condition that is out of range for f...

661837 v1 **Base DTC value for detection of events caused by other condition that is out of range for full opera**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 4.1
Verification Method: Analysis
Legacy ID: 71796v1

If DTC tests for events caused by other conditions are out of range for full operation is implemented the tests shall use a base DTC value in the range U2E05 to U2EFF. The tests shall use one unique base DTC value for each event.

661835 v1 **Detection of events caused by other condition that is out of range for f...**

Version history:
(1):

Purpose: To be able to handle the situation where there may be a customer complaint and a symptom associated with the ECU but the root cause is that the ECU is operating outside of its design

specification.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 4.1
Verification Method: Analysis
Legacy ID: 71792v0

If the ECU is able to detect that any other condition is out of range for full operation so that not correct, reduced or no system functionality is available and this detection is implemented as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such event shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect that such events as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

661838 v1 **Failure type value for detection of events caused by other condition tha...**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 4.1
Verification Method: Analysis
Legacy ID: 71797v0

If a DTC test for events caused by other condition is out of range for full operation is implemented the test shall use Failure Type value 0x68.

661836 v1 **Test period time for detection of events caused by other condition that is out of range for full ope**

Version history:
(1):

Purpose: To allow for fast detection of faults and the ability to detect intermittent faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 4.1
Verification Method: Analysis
Legacy ID: 71794v1

If a DTC test for events caused by other condition is out of range for full operation is implemented the test shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

4.10.3.1.6.2.5.6.2 Detection of faults caused by the ECU operating environment or customer action

661813 v1 **Base DTC value for detection of events caused by customer operating vehi...**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 2.1

Verification Method: Analysis
Legacy ID: 71773v0

If a DTC test for events caused by customer operating vehicle or vehicle equipment outside specification is implemented the test shall use the base DTC value U2E02.

661811 v1 Detection of events caused by customer operating vehicle or vehicle equi...

Version history:
(1):

Purpose: To be able to handle the situation where there may be a customer complaint and a symptom associated with the ECU but the root cause is that the ECU is operating outside of its design specification.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 2.1
Verification Method: Analysis
Legacy ID: 71767v0

If the ECU detects that the vehicle, or vehicle equipment (including the ECU itself), is operated outside of its design specification so that not correct, reduced or no system functionality is available, or a fallback strategy is activated, and this detection is implemented as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such event shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect that such events as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

661814 v1 Failure type value for detection of events caused by customer operating ...

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 2.1
Verification Method: Analysis
Legacy ID: 71777v0

If a DTC test for events caused by customer operating vehicle or vehicle equipment outside specification is implemented the test shall use Failure Type value 0x68.

661806 v2 Test Run Criteria for detection of faults caused by the ECU operating environment or customer action

Version history:
(2): Verification Method: Inspection change to Analysis.

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source:
Verification Method: Analysis
Legacy ID: 115361v0

Any fault detection test for detection of faults caused by the ECU operating environment or customer actions shall

be inhibited when the fault is a secondary fault that appears as a result of another (primary) fault that is detected by another DTC test.

661812 v1 **Test period time for detection of events caused by customer operating ve...**

Version history:
(1):

Purpose: To allow for fast detection of faults and the ability to detect intermittent faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 2.1
Verification Method: Analysis
Legacy ID: 71769v0

If a DTC test for events caused by customer operating vehicle or vehicle equipment outside specification is implemented the test shall produce Test Samples periodically every 100 ms or faster whenever the Test Run Criteria are met.

4.10.3.1.6.2.5.6.3 Detection of events caused by ambient temperature

661808 v1 **Base DTC value for detection of faults caused by ambient temperature too ...**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 1.1
Verification Method: Analysis
Legacy ID: 71760v0

If a DTC test for ambient temperature too high for full and correct operation is implemented the test shall use the base DTC value U2E00.

661809 v1 **Base DTC value for detection of faults caused by ambient temperature too low**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 1.2
Verification Method: Analysis
Legacy ID: 71762v0

If a DTC test for ambient temperature too low for full and correct operation is implemented the test shall use the base DTC value U2E01.

661810 v1 **Failure type value for detection of faults caused by ambient temperature**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 1.1 and 1.2

Verification Method: Analysis
Legacy ID: 71764v0

If a DTC test for ambient temperature too high or too low for full and correct operation is implemented the test shall use Failure Type value 0x68.

661807 v1 **Test period time for detection of faults caused by ambient temperature**

Version history:
(1):

Purpose: To allow for fast detection of fault and the ability to detect intermittent faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 1.1 and 1.2
Verification Method: Analysis
Legacy ID: 71757v0

If a DTC test for ambient temperature too high or too low for full and correct operation is implemented the test shall produce Test Samples periodically every 100 ms or faster whenever the Test Run Criteria are met.

4.10.3.1.6.2.5.6.4 Detection of other faults inferred from system behaviour

661839 v2 **Base DTC value for detection of other faults inferred from system behaviour**

Version history:
(2): Verification Method: Inspection change to Analysis.

Purpose: To enable easy identification of specific faults.
Source:
Verification Method: Analysis
Legacy ID: 115381v0

Detection of other faults faults inferred from system behaviour shall use one unique base DTC value for each component, system or part of system that is monitored, to identify the fault.

4.10.3.1.6.2.5.7 Detection of events associated with driver indications

If a driver indication (e.g. telltale, text message, etc.) exists which, when activated, recommends (e.g. by a service book instruction, text message, etc.) the driver to bring the car to a workshop, a study shall be performed by the implementer to investigate if the event that triggers the activation of the driver indication shall also be detected by a DTC test. The result of the study shall be approved by the responsible authority appointed by OEM.

Requirements from this section are specified in section "Requirements from section DTCs associated with driver indications".

661840 v3 **Detection of events associated with driver indications**

Version history:
(3): Clarify the specific time range requirement for DTC record.

Purpose: If a driver indication (e.g. telltale, text message, etc.) exists which, when activated, recommends (e.g. by a service book instruction, text message, etc.) the driver to bring the car to a workshop, the workshop may need to be able to find the root cause for this indication by a DTC.
Source: Section "Detection of events associated with driver indications"
Verification Method: Analysis

Legacy ID: 71801v1

A DTC test indicating the cause of the event that triggers an activation of a driver indication (which when activated, recommends the driver to bring the car to a workshop) shall be implemented as required by a study perfomed by the implementer and approved by the responsible authority appointed by OEM.

Comment: A confirmed DTC shall be set within 2s before the driver indication is actiavted, unless otherwise specific by regulations.

4.10.3.1.6.2.5.8 Detection of faults due to LIN related circuit faults for LIN Master

The diagnostic trouble codes each LIN master shall support are described in table Summary of LIN related circuit faults.

Item	Detected fault	Test Period Time	Test Run Criteria (TRC)	Failure type (hex)	DTC ID	M/O
1	**Faulty transmitted data.** Data sent by the LIN master (either header or response from LIN master) is faulty. The DTC fault detection counter counts up when data read back does not match data sent. The DTC fault detection counts down when data read back matches data sent.		Every sent LIN frame	86	Defined in Global Master Reference Database.	M
2	**Faulty LIN response** LIN frame checksum error. The DTC fault detection counter counts up when a checksum in one LIN frame from one LIN slave differs from expected value. The DTC fault detection counter counts down when any LIN frame with correct checksum from a specific LIN slave is received.		Every received LIN frame	83	Defined in Global Master Reference Database.	M
3	**Missing LIN response** No response (i.e. no bytes received at all) when the master expects to receive a LIN frame response. The DTC fault detection counter counts up when no response is received from a LIN slave. The DTC fault detection counter counts down when a response is received from a LIN slave.		Every expected LIN response except frame identifier 0x3D triggered responses	87	Defined in Global Master Reference Database.	M
4	**Faulty LIN physical bus.** LIN physical bus short to ground or battery voltage. The DTC fault detection counter counts up when LIN physical bus short to ground or battery voltage is detected (e.g. when master fail to detect the synchronisation break or the start-bit of the synchronisation/identifier field). The DTC fault detection counts down when not LIN physical bus short to ground or battery voltage is detected (e.g. when master succeeds to detect the synchronisation break or the start-bit of the synchronisation/identifier field).		Every sent LIN frame	01	Defined in Global Master Reference Database.	M
5	**Other LIN network faults.** Other faults identified by the implementer	TBD by implementer but at least every 100ms	TBD by implementer	TBD by implem-enter	Defined in Global Master Reference Database.	MC

Table - Summary of LIN related circuit faults

Abbreviations in Table - Summary of LIN related circuit faults
M= Mandatory
MC = Mandatory where the fault is detected by a fault monitor as part of the ECU functional requirements.

662731 v1 **Base DTC value for LIN slave node failure**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Summary of LIN related circuit faults"
Verification Method: Analysis
Legacy ID: 94731v0

LIN network failure detection shall have one base DTC for each LIN slave and one base DTC for the LIN master.

4.10.3.1.6.2.5.9 Detection of faults caused by too high quiescent current

The ECU shall be able to detect faults caused by too high quiescent current according to Table below.

Requirements on detection of faults caused by too high quiescent current.

Type of item	Item no.	Detected faults/errors	Test Period	Test Run Criteria (TRC)	Failure type (FTB)	Base DTC (display value, excl. FTB)	M/O
See note 1.	1	Quiescent current is too high in the ECU. See note 1.	See note 1.	See note 1.	See note 1.	See note 1.	See note 1.

Notes in the table above:
1. See reference *[Data_9] FltEgyCns*.

4.10.3.1.6.2.5.10 DTC Data Verification DTCs

In order to support the verification of DTC data, like sampling of snapshot data and updating of the various extended data records, an ECU shall implement a few dedicated test DTCs that can be controlled by means of control routine B000 - Set DTC prepassed/prefailed.

Carcom name	AUTOSAR parameter	Tested events	
Identifier	DemUdsDTC	U280000 (0xE80000)	U280100 (0xE80100)
Software label	DemEventParamter	EventA	EventB
DTC Event priority	DemEventPriority	100	100
Warning Indicator	DemIndicatorAttribute	None	None
N/A	DemDebounceCounterJumpUp	True	True
N/A	DemDebounceCounterJumpUpValue	0	0
Step Up	DemDebounceCounterIncrementStepSize	4	4
unconfirmedDTCLimit	DemDebounceCounterUnconfirmedThreshold	20	20
Max value	DemDebounceCounterFailedThreshold	127	127
ConfirmedDTCLimit	DemEventFailureCycleCounterThreshold	1	3
Jump Down	DemDebounceCounterJumpDown	True	True
N/A	DemDebounceCounterJumpDownValue	0	0
Step Down	DemDebounceCounterDecrementStepSize	4	4
Min Value	DemDebounceCounterPassedThreshold	-128	-128
agedDTCLimit	DemAgingCycleCounterThreshold	3	3

Requirements from this section are specified in section "Requirements from section Data Verification DTCs"

4.10.3.1.6.2.5.11 Detection of faults in private ECUs

662020 v2 **Detection of CS- or CRC-error on private ECU network**

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.3
Verification Method: Analysis
Legacy ID: 68988v0

If the ECU is a Public ECU master on a network and it is able to detect CS- or CRC-error on received messages on that network as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) this fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect this fault as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

662026 v2 **Detection of data not updated by transmitter on private ECU network**

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.5
Verification Method: Analysis
Legacy ID: 68994v0

If the ECU is a Public ECU master on a network and it is able to detect that received data has not been updated by the transmitting ECU on that network as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) this fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect this fault as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

662023 v2 **Detection of incorrect timing and format on private ECU network**

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.4
Verification Method: Analysis
Legacy ID: 68991v0

If the ECU is a Public ECU master on a network and it is able to detect incorrect timing, content, structure and/or format of bits, bytes or messages on that network as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) this fault shall also be identified by an appropriate DTC.

Otherwise, the ECU shall be able to detect this fault as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

662029 v2 Detection of missed deadline for message on private ECU network

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.6
Verification Method: Analysis
Legacy ID: 68998v0

If the ECU is a Public ECU master on a network and it is able to detect that a periodic message is not received within its maximum period on that network as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) this fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect this fault as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

662035 v2 Detection of other faults on private ECU network

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.8
Verification Method: Analysis
Legacy ID: 69004v0

If the ECU is a Public ECU master on a network and another fault (than the faults identified in items 9.1 through 9.7 in table *Requirements on detection of faults in external electrical circuits connected to the ECU*) on that network is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate Failure Type value for the corresponding DTC.
Otherwise, the ECU shall be able to detect other faults in the private ECU network as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

662032 v2 Detection of physical layer faults on private ECU network

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.7
Verification Method: Analysis
Legacy ID: 69001v0

If the ECU is a Public ECU master on a network and it is able to detect that the physical layer of the communication link has a short to ground, short to battery voltage or open circuit as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) this fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect this fault as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

662014 v2 Detection of transmit error on private ECU network

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.1
Verification Method: Analysis
Legacy ID: 68982v3

If the ECU is a Public ECU master on a network and it is able to detect if transmitted message contain data that does not correspond to data read back by the ECU from the network as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) this fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect this fault as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM) .

662017 v2 Detection of unexpected transmissions on private ECU network

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.2
Verification Method: Analysis
Legacy ID: 68985v0

If the ECU is a Public ECU master on a network and it is able to detect if the transmission line is not free when no data transmission is expected on that network as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) this fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect this fault as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

662022 v2 Failure type value for CS- or CRC-error on private ECU network

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.3

Verification Method: Analysis
Legacy ID: 68990v0

DTCs for CS- or CRC-error on private ECU network shall have a Failure Type value 0x83.

662025 v2 Failure type value for incorrect timing and format on private ECU network

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.4
Verification Method: Analysis
Legacy ID: 68993v0

DTCs for incorrect timing and format on private ECU network shall have a Failure Type value 0x86.

662031 v2 Failure type value for missed deadline for message on private ECU network

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.6
Verification Method: Analysis
Legacy ID: 69000v0

DTCs for missed deadline for message on private ECU network shall have a Failure Type value 0x87.

662034 v2 Failure type value for physical layer faults on private ECU network

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.7
Verification Method: Analysis
Legacy ID: 69003v0

DTCs for physical layer faults on private ECU network shall have a Failure Type value 0x01.

662016 v2 Failure type value for transmit error on private ECU network

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.1
Verification Method: Analysis
Legacy ID: 68984v0

DTCs for transmit error on private ECU network shall have a Failure Type value 0x86.

662019 v2 **Failure type value for unexpected transmissions on private ECU network**

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.2
Verification Method: Analysis
Legacy ID: 68987v0

DTCs for unexpected transmissions on private ECU network shall have a Failure Type value 0x81.

662021 v1 **Test Run Criteria for CS- or CRC-error on private ECU network**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.3
Verification Method: Analysis
Legacy ID: 68989v0

The DTC test for CS- or CRC-error on private ECU network shall run each time a message is received on the network by the ECU.

662027 v1 **Test Run Criteria for data not updated by transmitter on private ECU net...**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.5
Verification Method: Analysis
Legacy ID: 68995v0

The DTC test for data not updated by transmitter on private ECU network shall run each time a message is received on the network by the ECU.

662015 v1 **Test Run Criteria for detection of transmit error on private ECU network**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.1
Verification Method: Analysis
Legacy ID: 68983v0

The DTC test for transmit error on private ECU network shall run each time a message is transmitted on the network by the ECU.

662024 v1 **Test Run Criteria for incorrect timing and format on private ECU network**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.4
Verification Method: Analysis
Legacy ID: 68992v0

The DTC test for incorrect timing and format on private ECU network shall run each time a message is received on the network by the ECU.

662030 v1 **Test Run Criteria for missed deadline for message on private ECU network**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.6
Verification Method: Analysis
Legacy ID: 68999v0

The DTC test for missed deadline for message on private ECU network shall run each time a message is expected to be received on the network by the ECU.

662036 v1 **Test period time for detection of other faults on private ECU network**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.8
Verification Method: Analysis
Legacy ID: 69005v0

The DTC tests for the detection of faults in the private communication network shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

662018 v1 **Test period time for detection of unexpected transmissions on private EC...**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.2
Verification Method: Analysis
Legacy ID: 68986v0

The DTC tests for the detection of unexpected transmissions on private ECU network shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

662033 v1 **Test period time for physical layer faults on private ECU network**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.7
Verification Method: Analysis
Legacy ID: 69002v0

The DTC tests for the detection of physical layer faults on private ECU network shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

662028 v2 **type value for data not updated by transmitter on private ECU network**

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.5
Verification Method: Analysis
Legacy ID: 68996v0

DTCs for data not updated by transmitter on private ECU network shall have a Failure Type value 0x82.

4.10.3.2 UDS Data - Basetech Data Requirements

•DID data

4.10.3.2.1 DID data

4.10.3.2.1.1 DID of Vehicle Identification

664243 v2 **DID - ED20**

Version history:
(2):

Purpose: To enable readout of the Geely Complete set of ECU part/serial numbers.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	ED20
Name	Complete set of ECU part/serial numbers - Geely
Description	The data record (called "composite" data record) shall contain several data items in an arbitrary order. Each data item shall consist of a data identifier followed by the record data that is identified by the data identifier. The "composite" data record with identifier 0xED20 shall contain the following data items (the data items is depending of the currently active executing software): **Identifier** **Description** F1A0 or F1A1 or F1A2 Application- or PBL- or SBL Diagnostic Database Part number F1AA ECU Core Assembly Part Number F1AB ECU Delivery Assembly Part Number F18C ECU Serial Number F1AE or F1A5 or F124 ECU- or PBL Software Part Number(s) or SBL Software VersionNumber F13F Private ECU(s) / components Serial Number(s)
Content	**SubParameter Name** **Length, Data Type** F1A0 or F1A1 or F1A2 [8 Bytes, 5 BCD+3 Ascii] F1AA [8 Bytes, 5 BCD+3 Ascii] F1AB [8 Bytes, 5 BCD+3 Ascii] F18C [4 Bytes, BCD] F1AE or F1A5 or F124 [see GMRDB] F13F [see GMRDB]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
M	N	MPS	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664244 v2 **DID - EDA0**

Version history:
(2):

Purpose: To enable readout of the other brands Complete set of ECU part/serial numbers.
Source:
Verification Method: Test

Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	EDA0
Name	Complete set of ECU part/serial numbers
Description	The data record (called "composite" data record) shall contain several data items in an arbitrary order. Each data item shall consist of a data identifier followed by the record data that is identified by the data identifier. The "composite" data record with identifier 0xEDA0 shall contain the following data items (the data items is depending of the currently active executing software): **Identifier** **Description** F120 or F121 or F122 Application- or PBL- or SBL Diagnostic Database Part Number F12A ECU Core Assembly Part Number F12B ECU Delivery Assembly Part Number F18C ECU Serial Number F12E or F125 or F124 ECU- or PBL Software Part Number(s) or SBL Software Version Number. F13F Private ECU(s) / components Serial Number(s) All ECU:s shall implement both 8 digit and 10 digit format DID:s independently of what brand they are planned to be used in.
Content	**SubParameter Name** **Length, Data Type** F120 or F121 or F122 [7 Bytes, 4 BCD+3 Ascii] F12A [7 Bytes, 4 BCD+3 Ascii] F12B [7 Bytes, 4 BCD+3 Ascii] F18C [4 Bytes, BCD] F12E or F125 or F124 [see GMRDB] F13F [see GMRDB]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
M	N	MPS	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

663824 v2 **DID - F106**

Version history:
(2):

Purpose: To enable readout of the Vehicle Configuration Parameters.
Source:

Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F106	
Name	Vehicle Configuration Parameters	
Description	It shall be implemented in the Car Configuration Domain Master, on networks with ECU configuration controlled by an on-board master configuration file.	
Content	**SubParameter Name**	**Length, Data Type**
	Number of Bytes	[2 Bytes, Unsigned]
	Checksum	[1 Byte, Unsigned]
	Carconfig parameters	[1743 bytes(Every parameter 1 Byte), Unsigned]
Default	/	
Security access required	Level 19/1A	

Other Session		Programming Session	
Read	Write	Read	Write
MC	**MC**	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

677100 v2 **DID - F109**

Version history:
(2):

Purpose: For security reasons it must not be possible to read the value of the Security Constant in any diagnostic session except SBL.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F109
Name	Security Constant Level 01/02

Description	Security constant according to reference *Security access algorithm* used to calculate the key that is sent in programming session. It shall not be possible to read the value of the data record by any diagnostic service defined in other sessions or any kind of external debug interface that may be used without destroying the sealing of the ECU in any diagnostic session. If it is possible to change the value, the value is allowed to be changed only one single time. Once the value has been changed, it shall not be possible to change the value again. Change of the value is optional only when the ECU is executing the secoundary bootloader - programming session. There is below description for ECU Security Constant Level 01/02 about DID usage.

ECU status	Mode	Read DID(22 F1 09)	Write DID(2E F1 09)
With development security constant	PBL	Not Supported	Service not support and return NRC 11
	SBL	DID not supported and return NRC 31	Support and return 6E F1 09
	Application	Not Supported	Not Supported
With production security constant	PBL	Not Supported	Service not support and return NRC 11
	SBL	DID not supported and return NRC 31	DID support and return NRC 22(Conditions Not Correct)
	Application	Not Supported	Not Supported

Note1: It is mandatory for SWDL ECU.
Note2: If ECU does not support SBL (Secondary bootloader), it shall meet the requirements required by SBL within the PBL state.
Note3: F109 can be written with production security constant once only after activated SBL, that means F109 can be executed after step- "Activate SBL" or before step-"check complete&compatible" or after step- "check complete&compatible".
Note4: The programmed security constant shall be applied immediately, i.e. without requiring an ECU reset.

Content	**SubParameter Name** — **Length, Data Type** Security Constant — [16 Bytes, Hex]
Default	/
Security access required	Level 01/02

Other Session		Programming Session	
Read	Write	Read	Write
N	N	N	**MS**
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

677101 v2 **DID - F10B**

Version history:
(2):

Purpose: For security reasons it must not be possible to read the value of the Security Constant in any

diagnostic session.

Source:

Verification Method: Analysis AND Test

Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F10B			
Name	Security Constant Level 19/1A			
Description	Security constant according to reference *Security access algorithm* used to calculate the key that is sent in extended session. It shall not be possible to read the value of the data record by any diagnostic service defined in other sessions or any kind of external debug interface that may be used without destroying the sealing of the ECU in any diagnostic session. If it is possible to change the value, the value is allowed to be changed only one single time. Once the value has been changed, it shall not be possible to change the value again. Note1: It is mandatory for ECU supporting Security Level 19/1A. Note2: The programmed security constant shall be applied immediately, i.e. without requiring an ECU reset. Note3: F10B can be written with production security constant once. Otherwise, multiple write attempts after successful writing will return NRC 22 (incorrect condition).			
Content	**SubParameter Name**	**Length, Data Type**		
	Security Constant	[16 Bytes, Hex]		
Default	/			
Security access required	Level 19/1A			
Other Session		Programming Session		
Read	Write	Read	Write	
N	**MC**	N	N	
Periodic Read	I/O Control	Periodic Read	I/O Control	
N	N	N	N	

663871 v2 **DID - F114**

Version history:
(2):

Purpose: To enable readout of Vehicle Information Section.

Source:

Verification Method: Test

Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F114
Name	Vehicle Information Section
Description	It shall be implemented in the Car Configuration Domain Master, on networks with ECU configuration controlled by an on-board master configuration file.
Content	**SubParameter Name** **Length, Data Type** Number of Bytes [1 Byte, Unsigned] VCD inforamtions(Every subparameter please see GMRDB) [255 Bytes, BCD or Ascii]
Default	/
Security access required	Level 19/1A

Other Session		Programming Session	
Read	Write	Read	Write
MC	**MC**	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

663873 v1 **DID - F120**

Version history:
(1):

Purpose: To enable readout of a database key for the diagnostic database used by the other brands ECU application SW.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F120
Name	Application Diagnostic Database Part Number

Description	Part number for diagnostic database used by ECU application SW. It have 8 digits for part number + 3 characters (version suffix). The part number digits shall be coded in BCD and the 3 characters shall be coded in ASCII. The data record used for 8 digit format shall have a fixed length of 7 bytes, be right justified with any unused digit(s) filled with 0. If version suffix contains one character, there shall be two space characters between the part number digits and the version suffix. If version suffix contains two characters, there shall be one space character between the part number digits and the version suffix. Example (applicable to data records with identifier used by 8 digit format): 1234567<space><space>A -> (hex) 01 23 45 67 20 20 41 12345678<space><space>A-> (hex) 12 34 56 78 20 20 41 1234567<space>AB -> (hex) 01 23 45 67 20 41 42 12345678<space>AB -> (hex) 12 34 56 78 20 41 42 1234567ABC -> (hex) 01 23 45 67 41 42 43 12345678ABC -> (hex) 12 34 56 78 41 42 43 Note:This is mandatory on condition for other Brands which is support 8 digits part number.
Content	**SubParameter Name** — **Length, Data Type** Application Diagnostic Database Part Number — [7 Bytes, 4 BCD + 3 Ascii]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
MC	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664201 v1 DID - F121

Version history:
(1):

Purpose: To enable readout of a database key for the diagnostic database used by the other brands ECU's primary bootloader SW.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F121
Name	Primary Bootloader Diagnostic Database Part Number

Description	Part number for diagnostic database used by ECU primary bootloader SW. It have 8 digits for part number + 3 characters (version suffix). The part number digits shall be coded in BCD and the 3 characters shall be coded in ASCII. The data record used for 8 digit format shall have a fixed length of 7 bytes, be right justified with any unused digit(s) filled with 0. If version suffix contains one character, there shall be two space characters between the part number digits and the version suffix. If version suffix contains two characters, there shall be one space character between the part number digits and the version suffix. Example (applicable to data records with identifier used by 8 digit format): 1234567<space><space>A -> (hex) 01 23 45 67 20 20 41 12345678<space><space>A-> (hex) 12 34 56 78 20 20 41 1234567<space>AB -> (hex) 01 23 45 67 20 41 42 12345678<space>AB -> (hex) 12 34 56 78 20 41 42 1234567ABC -> (hex) 01 23 45 67 41 42 43 12345678ABC -> (hex) 12 34 56 78 41 42 43 Note1: This is mandatory on condition for other Brands which is support 8 digits part number. Note2: It is allowed to change the value of the data records, but only once. Only the secondary bootloader is allowed to support change of the value.
Content	**SubParameter Name** — **Length, Data Type** Primary Bootloader Diagnostic Database Part Number — [7 Bytes, 4 BCD + 3 Ascii]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
O	N	MPS	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664203 v1 **DID - F122**

Version history:
(1):

Purpose: To enable readout of the complete set of other brands ECU Part/Serial numbers.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F122
Name	Secondary Bootloader Diagnostic Database Part Number

Description	Part number for diagnostic database used by ECU secondary bootloader SW. It have 8 digits for part number + 3 characters (version suffix). The part number digits shall be coded in BCD and the 3 characters shall be coded in ASCII. The data record used for 8 digit format shall have a fixed length of 7 bytes, be right justified with any unused digit(s) filled with 0. If version suffix contains one character, there shall be two space characters between the part number digits and the version suffix. If version suffix contains two characters, there shall be one space character between the part number digits and the version suffix. Example (applicable to data records with identifier used by 8 digit format): 1234567<space><space>A -> (hex) 01 23 45 67 20 20 41 12345678<space><space>A-> (hex) 12 34 56 78 20 20 41 1234567<space>AB -> (hex) 01 23 45 67 20 41 42 12345678<space>AB -> (hex) 12 34 56 78 20 41 42 1234567ABC -> (hex) 01 23 45 67 41 42 43 12345678ABC -> (hex) 12 34 56 78 41 42 43 Note1: This is mandatory on condition for other Brands which is support 8 digits part number.
Content	**SubParameter Name** — **Length, Data Type** Secondary Bootloader Diagnostic Database Part Number — [7 Bytes, 4 BCD + 3 Ascii]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
N	N	MS	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664204 v1 **DID - F124**

Version history:
(1):

Purpose: To enable correct interpretation of the Secondary Bootloader Software Version Number.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F124
Name	Secondary Bootloader Software Version Number

Description	Software Version Number for the Secondary Bootloader SW. The Secondary Bootloader Software Version Number shall contain version information values specified by the supplier which allow a tool to uniquely identify the secondary bootloader software resident on the ECU. The Secondary Bootloader Software Version Number 0xF124 shall be BCD encoded, right justified, all unused digit shall be filled with 0 in total 7 bytes. Note: We can use different SBL software version number for different brands, but both use F124.
Content	**SubParameter Name** **Length, Data Type** Secondary Bootloader Software Version Number [7 Bytes, BCD]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
N	N	MS	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664214 v2 **DID - F125**

Version history:
(2):

Purpose: To enable readout of the part number of the other brands Primary Bootloader SW.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F125
Name	Primary Bootloader Software Part Number

Description	Part number for the Primary Bootloader SW. It have 8 digits for part number + 3 characters (version suffix). The part number digits shall be coded in BCD and the 3 characters shall be coded in ASCII. The data record used for 8 digit format shall have a fixed length of 7 bytes, be right justified with any unused digit(s) filled with 0. If version suffix contains one character, there shall be two space characters between the part number digits and the version suffix. If version suffix contains two characters, there shall be one space character between the part number digits and the version suffix. Example (applicable to data records with identifier used by 8 digit format): 1234567<space><space>A -> (hex) 01 23 45 67 20 20 41 12345678<space><space>A-> (hex) 12 34 56 78 20 20 41 1234567<space>AB -> (hex) 01 23 45 67 20 41 42 12345678<space>AB -> (hex) 12 34 56 78 20 41 42 1234567ABC -> (hex) 01 23 45 67 41 42 43 12345678ABC -> (hex) 12 34 56 78 41 42 43 Note: This is mandatory on condition for other Brands which is support 8 digits part number.
Content	**SubParameter Name** — **Length, Data Type** Primary Bootloader Software Part Number — [7 Bytes, 4 BCD + 3 Ascii]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
M	N	MPS	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664226 v1 **DID - F126**

Version history:
(1):

Purpose: To enable readout of the specific version of the AUTOSAR cluster(s) implemented in the ECU.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F126
Name	AUTOSAR BSW Vendor IDs and Cluster Versions

Description	Autosar Basic Software cluster versions. To support diagnostic procedures it must be possible to read out the AUTOSAR cluster(s) implemented in the ECU.
Content	**SubParameter Name** — **Length, Data Type** Diagnosis cluster vendor ID and version — [11 Bytes, Hex or unsigned] Mode management cluster vendor ID and version — [8 Bytes, Hex or unsigned] COM cluster vender ID and version — [8 Bytes, Hex or unsigned] CAN cluster vender ID and version — [8 Bytes, Hex or unsigned] Flexray cluster vender ID and version — [8 Bytes, Hex or unsigned] LIN cluster vender ID and version — [8 Bytes, Hex or unsigned]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664228 v1 **DID - F12A**

Version history:
(1):

Purpose: To enable readout of other brands part number that identifies the combination of the ECU hardware and any non-replaceable = software (bootloaders and other fixed software).

Source:

Verification Method: Test

Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F12A
Name	ECU Core Assembly Part Number

Description	A number that identifies the combination of the ECU hardware (PCB circuit board micro-controller and memory) and any non-replaceable software (bootloaders and other fixed software). The ECU core assembly part number will NOT represent: 1) Any variable aspect of the ECU that is not associated with the circuit board, e.g. its color or associated internal actuators or mechatronic parts so long as these parts do not affect the compatibility of any downloadable component. 2) Any downloadable software component regardless of whether or not this component is preloaded into the ECU. For ECUs not including any items in 1) or 2) above the core assembly number will fully define the part. Note that the ECU core assembly part number will provide sufficient information to fully define the compatibility but not necessarily the correct vehicle applicability of any downloadable software component. It have 8 digits for part number + 3 characters (version suffix). The part number digits shall be coded in BCD and the 3 characters shall be coded in ASCII. The data record used for 8 digit format shall have a fixed length of 7 bytes, be right justified with any unused digit(s) filled with 0. If version suffix contains one character, there shall be two space characters between the part number digits and the version suffix. If version suffix contains two characters, there shall be one space character between the part number digits and the version suffix. Example (applicable to data records with identifier used by 8 digit format): 1234567<space><space>A -> (hex) 01 23 45 67 20 20 41 12345678<space><space>A-> (hex) 12 34 56 78 20 20 41 1234567<space>AB -> (hex) 01 23 45 67 20 41 42 12345678<space>AB -> (hex) 12 34 56 78 20 41 42 1234567ABC -> (hex) 01 23 45 67 41 42 43 12345678ABC -> (hex) 12 34 56 78 41 42 43 Note1:This is Mandatory for other Brands which is support 8 digits part number. Note2:It is allowed to change the value of the data records, but only once. Only the secondary bootloader is allowed to support change of the value.
Content	**SubParameter Name** — **Length, Data Type** ECU Core Assembly Part Number — [7 Bytes, 4 BCD + 3 Ascii]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
M	N	MPS	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664230 v1 **DID - F12B**

Version history:
(1):

Purpose: To enable readout of other brands part number that identifies the complete ECU at the point of delivery to the assembly plant.

Source:

Verification Method: Test

Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F12B
Name	ECU Delivery Assembly Part Number
Description	A number that identifies the complete ECU at the point of delivery to the assembly plant or the service bay. The number shall not be altered or updated after initial delivery of the ECU. ECU Delivery Assembly Number shall represent the ECU Core Assembly Number and additionally: 1) Any variable aspect of the ECU, e.g. its color, associated internal actuators or mechatronic parts. 2) All downloadable software components that are pre-loaded into the ECU. Note that any software download process will alter the ECU software content such that the content referred to by this part number will be incorrect. Therefore the number is only valid at the point of ECU delivery. It have 8 digits for part number + 3 characters (version suffix). The part number digits shall be coded in BCD and the 3 characters shall be coded in ASCII. The data record used for 8 digit format shall have a fixed length of 7 bytes, be right justified with any unused digit(s) filled with 0. If version suffix contains one character, there shall be two space characters between the part number digits and the version suffix. If version suffix contains two characters, there shall be one space character between the part number digits and the version suffix. Example (applicable to data records with identifier used by 8 digit format): 1234567<space><space>A -> (hex) 01 23 45 67 20 20 41 12345678<space><space>A-> (hex) 12 34 56 78 20 20 41 1234567<space>AB -> (hex) 01 23 45 67 20 41 42 12345678<space>AB -> (hex) 12 34 56 78 20 41 42 1234567ABC -> (hex) 01 23 45 67 41 42 43 12345678ABC -> (hex) 12 34 56 78 41 42 43 Note1:This is Mandatory for other Brands which is support 8 digits part number. Note2:It is allowed to change the value of the data records, but only once. Only the secondary bootloader is allowed to support change of the value.
Content	**SubParameter Name** — **Length, Data Type** ECU Delivery Assembly Part Number — [7 Bytes, 4 BCD + 3 Ascii]
Default	/
Other Session	Programming Session

Read	Write	Read	Write
M	N	MPS	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664232 v2 **DID - F12C**

Version history:
(2):

Purpose: To enable readout of the other brands part number of the ECU Software Structure.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F12C	
Name	ECU Software Structure Part Number	
Description	Software Part Number for the ECU Software Structure, which is used by ECUs supporting Software Authentication concept where each software part is signed individually. It have 10 digits for part number + 3 characters (version suffix). The part number digits shall be coded in BCD and the 3 characters shall be coded in ASCII. The data record used for 10 digit format shall have a fixed length of 8 bytes, be right justified with any unused digit(s) filled with 0. If version suffix contains one character, there shall be two space characters between the part number digits and the version suffix. If version suffix contains two characters, there shall be one space character between the part number digits and the version suffix. Example (applicable to data records with identifier used by 10 digit format): 123456789<space><space>A -> (hex) 01 23 45 67 89 20 20 41 1234567890<space><space>A-> (hex) 12 34 56 78 90 20 20 41 123456789<space>AB -> (hex) 01 23 45 67 89 20 41 42 1234567890<space>AB -> (hex) 12 34 56 78 90 20 41 42 123456789ABC -> (hex) 01 23 45 67 89 41 42 43 1234567890ABC -> (hex) 12 34 56 78 90 41 42 43 Note:This is Mandatory for Geely Brands which is support 10 digits part number.	
Content	**SubParameter Name**	**Length, Data Type**
	ECU Software Structure Part Number	[7 Bytes, 4 BCD + 3 Ascii]
Default	/	
Other Session		Programming Session

Read	Write	Read	Write
N	N	MCPS	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664234 v4 **DID - F12E**

Version history:
(4): Verification Method由Test修改为Analysis AND Test。

Purpose: To enable readout of the other brands part number for the ECU software.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F12E
Name	ECU Software Part Numbers
Description	Complete set of part numbers for the ECU software. The Part Number(s) is only allowed to be used for item(s) that can be separately downloaded to the ECU. A data record with identifier 0xF12E shall be implemented containing 8 digit format Part numbers. The content of the dataRecord with identifier 0xF12E shall be the following: Byte Description #1 Total number of ECU Software Part Numbers #2 - 8 ECU Software #1 Part Number #9 - 15 ECU Software #2 Part Number . . . . #N - (N+6) ECU Software #X Part Number **Note1:** If part numbers are not obtained on both formats DID:s on either format may be filled with zeros. The ECU shall however always have at least one set of valid part numbers. **Note2:** This is Mandatory for other Brands which is support 8 digits part number. **Note3:** The part number should be filled with zeros when the software parts which is not on the list of complete and compatibility check is damaged. The deviation related to certificate data software parts part number shall be approved by GRI Basetech Team. **Note4:** If the part number can not be obtained from secondary processor, e.g. communication not being established between processors, it should be filled with all 0xFF. **Note5:** If both ECU addresses independently support diagnostics in application mode, but one main ECU address is responsible for updating softwares of both ECU addresses, then the ECU address that only supports diagnostics in application mode shall keep the same ECU software part numbers as the other ECU address. E.g. The ECU has two processors, SOC and MCU.

Content	SubParameter Name	Length, Data Type
	Total number of ECU Software Part Numbers	[1 Byte, Unsigned]
	ECU Software Part Number (Every software part number as a subparameter)	[7 Bytes, 4 BCD +3 Ascii]
Default	/	

Other Session		Programming Session	
Read	Write	Read	Write
M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664236 v2 **DID - F12F**

Version history:
(2): 响应参数字节长度错误修订。

Purpose: To enable readout of the other brands extended part numbers for the ECU softwares.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F12F
Name	ECU Software Extended Part Numbers
Description	Set of Extended part number(s) for the ECU software. The Extended Part Number(s) is only allowed to be used for item(s) that can not be separately downloaded to the ECU. Mandatory if required by the responsible authority appointed by Geely. Otherwise optional. A data record with identifier 0xF12F shall be implemented containing 8 digit format Part numbers. The content of the dataRecord with identifier 0xF12F shall be the following: Byte Description #1 Total number of ECU Extended Software Part Numbers #2 - 8 ECU Extended Software #1 Part Number #9 - 15 ECU Extended Software #2 Part Number . . . . #N - (N+6) ECU Extended Software #X Part Number Note: If Extended part numbers are not obtained on both formats DID:s on either format may be filled with zeros. The ECU shall however always have at least one set of valid part numbers.

Content	SubParameter Name	Length, Data Type
	Total number of ECU Extended Software Part Numbers	[1 Byte, Unsigned]
	ECU Extended Software Part Number (Every Extended software part number as a subparameter)	[7 Bytes, 4 BCD +3 Ascii]
Default	/	

Other Session		Programming Session	
Read	Write	Read	Write
MC	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664239 v1 **DID - F13B**

Version history:
(1):

Purpose: To enable readout of the other brands Private ECU(s) Delivery Assembly Part Number.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F13B	
Name	Private ECU(s) Delivery Assembly Part Number(s)	
Description	Every Private ECU(s) Delivery Assembly Part Number(s) for which connect to a Public ECU master. Mandatory if required by the responsible authority appointed by Geely. Otherwise optional. The content of the dataRecord with identifier 0xF13B shall be the following: Byte Description #1 Total number of Private ECUs #2 - 8 Private ECU #1 Delivery Assembly Part Number #9 - 15 Private ECU #2 Delivery Assembly Part Number . . . . #N - (N+6) Private ECU #N Delivery Assembly Part Number Note: If Private ECU(s) Delivery Assembly Part Number(s) are not obtained on both formats DID:s on either format may be filled with zeros. The ECU shall however always have at least one set of valid part numbers.	
Content	**SubParameter Name**	**Length, Data Type**
	Total number of Private ECUs	[1 Byte, Unsigned]
	Private ECU Delivery Assembly Part Number (Every Private ECU as a subparameter)	[7 Bytes, 4 BCD + 3 Ascii]

Default	/		
Other Session		Programming Session	
Read	Write	Read	Write
MC	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664238 v5 **DID - F13F**

Version history:
(5): Description中错误的字节长度修订。

Purpose: To enable read out of the serial numbers for Private ECU(s) or component(s).
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F13F
Name	Private ECU(s)/components Serial Number(s)
Description	A supplier unique Serial number per Private ECU Core Assembly Part Number for a Private ECU or component for which the ECU is Public ECU master. Mandatory if required by the responsible authority appointed by Geely. Otherwise optional. The content of the dataRecord with identifier 0xF13F shall be the following: Byte Description #1 Total number of Private ECU(s) / component(s) Serial Numbers #2 - 13 Private ECU / component #1 Serial Number #14 - 25 Private ECU / component #2 Serial Number . . . . #N - (N+11) Private ECU / component #N Serial Number Note: If Private ECU / component Serial Number are not obtained on both formats DID:s on either format may be filled with zeros. The ECU shall however always have at least one set of valid Serial numbers.
Content	**SubParameter Name** — **Length, Data Type** Total number of Private ECU(s) / component(s) Serial Numbers — [1 Byte, Unsigned] Private ECU / component Serial Number — [12 Bytes, Ascii] (Every Private ECU as a subparameter)
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
MC	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664240 v1 **DID - F186**

Version history:
(1):

Purpose: To enable readout of the active diagnostic session.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F186	
Name	Active Diagnostic Session	
Description	To support diagnostic procedures it must be possible to read out the active diagnostic session from the ECU, independent of the diagnostic session.	
Content	**SubParameter Name**	**Length, Data Type**
	Active Diagnostic Session	[1 Byte, Unsigned]
Default	/	

Other Session		Programming Session	
Read	Write	Read	Write
M	N	M	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

676865 v2 **DID - F18A**

Version history:
(2): Verification Method: Test.

Purpose: To enable readout of the system supplier identifier of ECU.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F18A
Name	System Supplier Identifier
Description	”System Supplier Identifier" refers to the codes assigned by Geely, after the supplier enters Geely system through the quelified approval process. It is allowed to change the value of the data record, but only once. Only the secondary bootloader is allowed to support the write. The 6 digits for System Supplier Identifier shall be coded in ASCII. Example: 012345 -> (hex) 30 31 32 33 34 35 123456 -> (hex) 31 32 33 34 35 36
Content	**SubParameter Name** — **Length, Data Type** System Supplier Identifier — [6 Bytes, ASCII]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
M	N	MPS	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664241 v3 **DID - F18C**

Version history:
(3):

Purpose: To enable readout of the supplier unique ECU serial number.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F18C
Name	ECU Serial Number

Description	A supplier unique ECU serial number per ECU Core Assembly Part Number. It is allowed to change the value of the data record, but only once. Only the secondary bootloader is allowed to support the write. 8 digits for serial number. All coded in BCD, right justified with any unused digit filled with 0, in total 4 bytes Example: 12345678 -> (hex) 12 34 56 78 123456 -> (hex) 00 12 34 56 Note: ECU Serial Number shall not be filled with all zero.
Content	**SubParameter Name** — **Length, Data Type** ECU Serial Number — [4 Bytes, BCD]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
M	N	MPS	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664242 v2 **DID - F190**

Version history:
(2):

Purpose: To enable readout of the standard 17 character VIN (as written on the vehicle identification plate).
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F190
Name	Vehicle Identification Number
Description	The standard 17 character VIN as written on the vehicle identification plate. Note that the requirement to store and report Vehicle Identification Number is usually dependent upon legislated regulations and/or on the vehicle security (theft deterrence) policy. Mandatory if the ECU has a requirement to store and report VIN.
Content	**SubParameter Name** — **Length, Data Type** Vehicle Identification Number — [17 Bytes, Ascii]
Default	All 0xFF (17 Byte)

Other Session	Programming Session

Read	Write	Read	Write
MC	N	D	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

663872 v1 **DID - F1A0**

Version history:
(1):

Purpose: To enable readout of a database key for the diagnostic database used by the Geely ECU application SW.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F1A0	
Name	Application Diagnostic Database Part Number - Geely	
Description	Part number for diagnostic database used by ECU application SW. It have 10 digits for part number + 3 characters (version suffix). The part number digits shall be coded in BCD and the 3 characters shall be coded in ASCII. The data record used for 10 digit format shall have a fixed length of 8 bytes, be right justified with any unused digit(s) filled with 0. If version suffix contains one character, there shall be two space characters between the part number digits and the version suffix. If version suffix contains two characters, there shall be one space character between the part number digits and the version suffix. Example (applicable to data records with identifier used by 10 digit format): 123456789<space><space>A -> (hex) 01 23 45 67 89 20 20 41 1234567890<space><space>A-> (hex) 12 34 56 78 90 20 20 41 123456789<space>AB -> (hex) 01 23 45 67 89 20 41 42 1234567890<space>AB -> (hex) 12 34 56 78 90 20 41 42 123456789ABC -> (hex) 01 23 45 67 89 41 42 43 1234567890ABC -> (hex) 12 34 56 78 90 41 42 43 Note:This is Mandatory for Geely Brands which is support 10 digits part number.	
Content	**SubParameter Name**	**Length, Data Type**
	Application Diagnostic Database Part Number - Geely	[8 Bytes, 5BCD + 3Ascii]
Default	/	
Other Session		Programming Session

Read	Write	Read	Write
M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664200 v1 **DID - F1A1**

Version history:
(1):

Purpose: To enable readout of a database key for the diagnostic database used by the Geely ECU's primary bootloader SW.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F1A1	
Name	Primary Bootloader Diagnostic Database Part Number - Geely	
Description	Part number for diagnostic database used by ECU primary bootloader SW. It have 10 digits for part number + 3 characters (version suffix). The part number digits shall be coded in BCD and the 3 characters shall be coded in ASCII. The data record used for 10 digit format shall have a fixed length of 8 bytes, be right justified with any unused digit(s) filled with 0. If version suffix contains one character, there shall be two space characters between the part number digits and the version suffix. If version suffix contains two characters, there shall be one space character between the part number digits and the version suffix. Example (applicable to data records with identifier used by 10 digit format): 123456789<space><space>A -> (hex) 01 23 45 67 89 20 20 41 1234567890<space><space>A-> (hex) 12 34 56 78 90 20 20 41 123456789<space>AB -> (hex) 01 23 45 67 89 20 41 42 1234567890<space>AB -> (hex) 12 34 56 78 90 20 41 42 123456789ABC -> (hex) 01 23 45 67 89 41 42 43 1234567890ABC -> (hex) 12 34 56 78 90 41 42 43 Note1:This is Mandatory for Geely Brands which is support 10 digits part number. Note2:It is allowed to change the value of the data records, but only once. Only the secondary bootloader is allowed to support change of the value.	
Content	**SubParameter Name**	**Length, Data Type**
	Primary Bootloader Diagnostic Database Part Number - Geely	[8 Bytes, 5 BCD + 3 Ascii]
Default	/	
Other Session		Programming Session

Read	Write	Read	Write
O	N	MPS	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664202 v2 **DID - F1A2**

Version history:
(2):

Purpose: To enable readout of the complete set of Geely ECU Part/Serial numbers.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F1A2	
Name	Secondary Bootloader Diagnostic Database Part Number - Geely	
Description	Part number for diagnostic database used by ECU secondary bootloader SW. It have 10 digits for part number + 3 characters (version suffix). The part number digits shall be coded in BCD and the 3 characters shall be coded in ASCII. The data record used for 10 digit format shall have a fixed length of 8 bytes, be right justified with any unused digit(s) filled with 0. If version suffix contains one character, there shall be two space characters between the part number digits and the version suffix. If version suffix contains two characters, there shall be one space character between the part number digits and the version suffix. Example (applicable to data records with identifier used by 10 digit format): 123456789<space><space>A -> (hex) 01 23 45 67 89 20 20 41 1234567890<space><space>A-> (hex) 12 34 56 78 90 20 20 41 123456789<space>AB -> (hex) 01 23 45 67 89 20 41 42 1234567890<space>AB -> (hex) 12 34 56 78 90 20 41 42 123456789ABC -> (hex) 01 23 45 67 89 41 42 43 1234567890ABC -> (hex) 12 34 56 78 90 41 42 43 Note1:This is Mandatory for Geely Brands which is support 10 digits part number. Note2:It is allowed to change the value of the data records, but only once. Only the secondary bootloader is allowed to support change of the value.	
Content	**SubParameter Name**	**Length, Data Type**
	Secondary Bootloader Diagnostic Database Part Number - Geely	[8 Bytes, 5 BCD + 3 Ascii]
Default	/	
Other Session		Programming Session

Read	Write	Read	Write
N	N	MS	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664209 v2 **DID - F1A5**

Version history:
(2):

Purpose: To enable readout of the part number of the Geely Primary Bootloader SW.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F1A5	
Name	Primary Bootloader Software Part Number - Geely	
Description	Part number for the Primary Bootloader SW. It have 10 digits for part number + 3 characters (version suffix). The part number digits shall be coded in BCD and the 3 characters shall be coded in ASCII. The data record used for 10 digit format shall have a fixed length of 8 bytes, be right justified with any unused digit(s) filled with 0. If version suffix contains one character, there shall be two space characters between the part number digits and the version suffix. If version suffix contains two characters, there shall be one space character between the part number digits and the version suffix. Example (applicable to data records with identifier used by 10 digit format): 123456789<space><space>A -> (hex) 01 23 45 67 89 20 20 41 1234567890<space><space>A-> (hex) 12 34 56 78 90 20 20 41 123456789<space>AB -> (hex) 01 23 45 67 89 20 41 42 1234567890<space>AB -> (hex) 12 34 56 78 90 20 41 42 123456789ABC -> (hex) 01 23 45 67 89 41 42 43 1234567890ABC -> (hex) 12 34 56 78 90 41 42 43 Note1:This is Mandatory for Geely Brands which is support 10 digits part number. Note2:It is allowed to change the value of the data records, but only once. Only the secondary bootloader is allowed to support change of the value.	
Content	**SubParameter Name**	**Length, Data Type**
	Primary Bootloader Software Part Number - Geely	[8 Bytes, 5 BCD + 3 Ascii]
Default	/	
Other Session		Programming Session

Read	Write	Read	Write
M	N	MPS	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664227 v1 **DID - F1AA**

Version history:
(1):

Purpose: To enable readout of Geely part number that identifies the combination of the ECU hardware and any non-replaceable = software (bootloaders and other fixed software).
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F1AA
Name	ECU Core Assembly Part Number - Geely

Description	A number that identifies the combination of the ECU hardware (PCB circuit board micro-controller and memory) and any non-replaceable software (bootloaders and other fixed software). The ECU core assembly part number will NOT represent: 1) Any variable aspect of the ECU that is not associated with the circuit board, e.g. its color or associated internal actuators or mechatronic parts so long as these parts do not affect the compatibility of any downloadable component. 2) Any downloadable software component regardless of whether or not this component is preloaded into the ECU. For ECUs not including any items in 1) or 2) above the core assembly number will fully define the part. Note that the ECU core assembly part number will provide sufficient information to fully define the compatibility but not necessarily the correct vehicle applicability of any downloadable software component. It have 10 digits for part number + 3 characters (version suffix). The part number digits shall be coded in BCD and the 3 characters shall be coded in ASCII. The data record used for 10 digit format shall have a fixed length of 8 bytes, be right justified with any unused digit(s) filled with 0. If version suffix contains one character, there shall be two space characters between the part number digits and the version suffix. If version suffix contains two characters, there shall be one space character between the part number digits and the version suffix. Example (applicable to data records with identifier used by 10 digit format): 123456789<space><space>A -> (hex) 01 23 45 67 89 20 20 41 1234567890<space><space>A-> (hex) 12 34 56 78 90 20 20 41 123456789<space>AB -> (hex) 01 23 45 67 89 20 41 42 1234567890<space>AB -> (hex) 12 34 56 78 90 20 41 42 123456789ABC -> (hex) 01 23 45 67 89 41 42 43 1234567890ABC -> (hex) 12 34 56 78 90 41 42 43 Note1:This is Mandatory for Geely Brands which is support 10 digits part number. Note2:It is allowed to change the value of the data records, but only once. Only the secondary bootloader is allowed to support change of the value.
Content	**SubParameter Name** — **Length, Data Type** ECU Core Assembly Part Number - Geely — [8 Bytes, 5 BCD + 3 Ascii]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
M	N	MPS	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664229 v1 **DID - F1AB**

Version history:
(1):

Purpose: To enable readout of Geely part number that identifies the complete ECU at the point of delivery to the assembly plant.

Source:

Verification Method: Test

Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F1AB
Name	ECU Delivery Assembly Part Number - Geely
Description	A number that identifies the complete ECU at the point of delivery to the assembly plant or the service bay. The number shall not be altered or updated after initial delivery of the ECU. ECU Delivery Assembly Number shall represent the ECU Core Assembly Number and additionally: 1) Any variable aspect of the ECU, e.g. its color, associated internal actuators or mechatronic parts. 2) All downloadable software components that are pre-loaded into the ECU. Note that any software download process will alter the ECU software content such that the content referred to by this part number will be incorrect. Therefore the number is only valid at the point of ECU delivery. It have 10 digits for part number + 3 characters (version suffix). The part number digits shall be coded in BCD and the 3 characters shall be coded in ASCII. The data record used for 10 digit format shall have a fixed length of 8 bytes, be right justified with any unused digit(s) filled with 0. If version suffix contains one character, there shall be two space characters between the part number digits and the version suffix. If version suffix contains two characters, there shall be one space character between the part number digits and the version suffix. Example (applicable to data records with identifier used by 10 digit format): 123456789<space><space>A -> (hex) 01 23 45 67 89 20 20 41 1234567890<space><space>A-> (hex) 12 34 56 78 90 20 20 41 123456789<space>AB -> (hex) 01 23 45 67 89 20 41 42 1234567890<space>AB -> (hex) 12 34 56 78 90 20 41 42 123456789ABC -> (hex) 01 23 45 67 89 41 42 43 1234567890ABC -> (hex) 12 34 56 78 90 41 42 43 Note1:This is Mandatory for Geely Brands which is support 10 digits part number. Note2:It is allowed to change the value of the data records, but only once. Only the secondary bootloader is allowed to support change of the value.
Content	**SubParameter Name** — **Length, Data Type** ECU Delivery Assembly Part Number - Geely — [8 Bytes, 5 BCD + 3 Ascii]
Default	/
Other Session	Programming Session

Read	Write	Read	Write
M	N	MPS	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664231 v2 **DID - F1AC**

Version history:
(2):

Purpose: To enable readout of the Geely part number of the ECU Software Structure.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F1AC	
Name	ECU Software Structure Part Number - Geely	
Description	Software Part Number for the ECU Software Structure, which is used by ECUs supporting Software Authentication concept where each software part is signed individually. It have 10 digits for part number + 3 characters (version suffix). The part number digits shall be coded in BCD and the 3 characters shall be coded in ASCII. The data record used for 10 digit format shall have a fixed length of 8 bytes, be right justified with any unused digit(s) filled with 0. If version suffix contains one character, there shall be two space characters between the part number digits and the version suffix. If version suffix contains two characters, there shall be one space character between the part number digits and the version suffix. Example (applicable to data records with identifier used by 10 digit format): 123456789<space><space>A -> (hex) 01 23 45 67 89 20 20 41 1234567890<space><space>A-> (hex) 12 34 56 78 90 20 20 41 123456789<space>AB -> (hex) 01 23 45 67 89 20 41 42 1234567890<space>AB -> (hex) 12 34 56 78 90 20 41 42 123456789ABC -> (hex) 01 23 45 67 89 41 42 43 1234567890ABC -> (hex) 12 34 56 78 90 41 42 43 Note:This is Mandatory for Geely Brands which is support 10 digits part number.	
Content	**SubParameter Name**	**Length, Data Type**
	ECU Software Structure Part Number - Geely	[8 Bytes, 5 BCD + 3 Ascii]
Default	/	
Other Session		Programming Session

Read	Write	Read	Write
N	N	MCPS	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664233 v5 **DID - F1AE**

Version history:
(5): Verification Method由Test修改为Analysis AND Test。

Purpose: To enable readout of the Geely part number for the ECU software.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F1AE
Name	ECU Software Part Numbers - Geely
Description	Complete set of part numbers for the ECU software. The Part Number(s) is only allowed to be used for item(s) that can be separately downloaded to the ECU. A data record with identifier 0xF1AE shall be implemented containing 10 digit format Part numbers. The content of the dataRecord with identifier 0xF1AE shall be the following: Byte Description #1 Total number of ECU Software Part Numbers #2 - 9 ECU Software #1 Part Number #10 - 17 ECU Software #2 Part Number . . . . #N - (N+7) ECU Software #X Part Number **Note1:** If part numbers are not obtained on both formats DID:s on either format may be filled with zeros. The ECU shall however always have at least one set of valid part numbers. **Note2:** This is Mandatory for Geely Brands which is support 10 digits part number. **Note3:** The part number should be filled with zeros when the software parts which is not on the list of complete and compatibility check is damaged. The deviation related to software parts part number shall be approved by GRI Basetech Team, e.g. certificate software part. **Note4:** If the part number can not be obtained from secondary processor, e.g. communication not being established between processors, it should be filled with all 0xFF. **Note5:** If both ECU addresses independently support diagnostics in application mode, but one main ECU address is responsible for updating softwares of both ECU addresses, then the ECU address that only supports diagnostics in application mode shall keep the same ECU software part numbers as the other ECU address. E.g. The ECU has two processors, SOC and MCU.

Content	SubParameter Name	Length, Data Type
	Total number of ECU Software Part Numbers	[1 Byte, Unsigned]
	ECU Software Part Number (Every software part number as a subparameter)	[8 Bytes, 5 BCD+3 Ascii]
Default	/	

Other Session		Programming Session	
Read	Write	Read	Write
M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664235 v1 **DID - F1AF**

Version history:
(1):

Purpose: To enable readout of the Geely extended part numbers for the ECU softwares.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F1AF
Name	ECU Software Extended Part Numbers - Geely
Description	Set of Extended part number(s) for the ECU software. The Extended Part Number(s) is only allowed to be used for item(s) that can not be separately downloaded to the ECU. Mandatory if required by the responsible authority appointed by Geely. Otherwise optional. A data record with identifier 0xF1AF shall be implemented containing 10 digit format Part numbers. The content of the dataRecord with identifier 0xF1AF shall be the following: Byte Description #1 Total number of ECU Extended Software Part Numbers #2 - 9 ECU Extended Software #1 Part Number #10 - 17 ECU Extended Software #2 Part Number . . . . #N - (N+7) ECU Extended Software #X Part Number Note: If Extended part numbers are not obtained on both formats DID:s on either format may be filled with zeros. The ECU shall however always have at least one set of valid part numbers.

Content	SubParameter Name	Length, Data Type
	Total number of ECU Extended Software Part Numbers	[1 Byte, Unsigned]
	ECU Extended Software Part Number (Every Extended software part number as a subparameter)	[8 Bytes, 5 BCD+3 Ascii]
Default	/	

Other Session		Programming Session	
Read	Write	Read	Write
MC	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664237 v1 **DID - F1BB**

Version history:
(1):

Purpose: To enable readout of the Geely Private ECU(s) Delivery Assembly Part Number.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	F1BB
Name	Private ECU(s) Delivery Assembly Part Number(s) - Geely
Description	Set of Private ECU Delivery Assembly part number(s) for the ECU software. The Private ECU Delivery Assembly Part Number(s) is only allowed to be used for item(s) that can not be separately downloaded to the ECU. Mandatory if required by the responsible authority appointed by Geely. Otherwise optional. A data record with identifier 0xF1BB shall be implemented containing 10 digit format Part numbers. The content of the dataRecord with identifier 0xF1BB shall be the following: Byte Description #1 Total number of Private ECUs #2 - 9 Private ECU #1 Delivery Assembly Part Number #10 - 17 Private ECU #2 Delivery Assembly Part Number . . . . #N - (N+7) Private ECU #N Delivery Assembly Part Number Note: If Private ECU Delivery Assembly part numbers are not obtained on both formats DID:s on either format may be filled with zeros. The ECU shall however always have at least one set of valid part numbers.

Content	SubParameter Name	Length, Data Type
	Total number of Private ECUs	[1 Byte, Unsigned]
	Private ECU Delivery Assembly Part Number (Every Private ECU as a subparameter)	[8 Bytes, 5 BCD+3 Ascii]
Default	/	

Other Session		Programming Session	
Read	Write	Read	Write
MC	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

4.10.3.2.1.2 DID of Vehicle Status

665715 v1 **DID - C000**

Version history:
(1):

Purpose: The identifier for a IEEE 802.11 wireless LAN. The SSID is sent into beacon messages from AP and STA can associate to the wireless network using the SSID.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	C000	
Name	Service Set Identifier STN	
Description	Identifier for the external AP that vehicle shall connect to. A data record with identifier 0x C000 shall be implemented by the ECU containing WLAN used by workshops and manufacturing exactly. The data record shall consist of 32 character, all coded in ASCII. The data record shall have a fixed length of 32 bytes, be right justified with any unused character(s) filled with 0. Mandatory and applicable only if the ECU is a edge node which has a IEEE 802.11 wireless LAN.	
Content	**SubParameter Name**	**Length, Data Type**
	ServiceSet Identifier STN	[32 bytes, Ascii]
Default	/	

Other Session		Programming Session	
Read	Write	Read	Write
MC	MC	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control

N	N	N	N

665714 v1 **DID - C001**

Version history:
(1):

Purpose: To be able to set the SSID for the wireless network when the vehicle is configured as AP. This SSID shall be used by nomadic devices when associating with wireless network vehicle.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	C001	
Name	Service Set Identifier AP	
Description	A data record with identifier 0x C001 shall be implemented by the ECU containing AP, The data record shall consist of 32 character, all coded in ASCII. The data record shall have a fixed length of 32 bytes, be right justified with any unused character(s) filled with 0. Mandatory and applicable only if the ECU is a edge node which has a IEEE 802.11 wireless LAN.	
Content	**SubParameter Name** Service Set Identifier AP	**Length, Data Type** [32 bytes, Ascii]
Default	/	

Other Session		Programming Session	
Read	Write	Read	Write
MC	MC	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

665717 v2 **DID - C002**

Version history:
(2): 修改为适用所有以太网节点。

Purpose: To be able to change WLAN Passphrase.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	C002
Name	WLAN Passpharse
Description	A data record with identifier 0x C002 shall be implemented by the ECU containing WLAN and used as AP. The data record shall have a fixed length of 64 bytes, consist of 64 character, all coded in ASCII, be right justified with any unused character(s) filled with 0. Mandatory and applicable only if the ECU is a ethernet node which has a IEEE 802.11 wireless LAN.
Content	**SubParameter Name** — **Length, Data Type** WLAN Passpharse — [64 bytes, Ascii]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
MC	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

665712 v2 **DID - C003**

Version history:
(2): 扩大该DID的支持范围。除了边缘节点外，车内以太网ECU也需要支持。该MAC地址主要指示得是用于DoIP连接的那个MAC地址。而像TCAM、DHU还有WiFi的网卡地址，则不是用该DID来指示。

Purpose: To be able to read the Ethernet interface MAC address of the Ethernet Node for DoIP connection.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	C003
Name	MAC Adress Ethernet
Description	A data record with identifier 0xC003 shall be implemented by the Ethernet ECU for Ethernet interface MAC address. The data record shall consist of 6 bytes, all coded in HEX. The data record shall have a fixed length of 6 bytes, be right justified with any unused digit(s) filled with 0. Note1: Mandatory and applicable only if the ECU is a ethernet node. Note2: The MAC address will be used in DoIP connection.
Content	**SubParameter Name** — **Length, Data Type** MAC Adress Ethernet — [6 bytes, Hex]

Default	/		
Other Session		Programming Session	
Read	Write	Read	Write
MC	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664683 v3 **DID - C011**

Version history:
(3):

Purpose: The workshop tool must be able to read out the Parameter-spec for the vehicle.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	C011		
Name	Build Llist		
Description	Mandatory if the ECU is Car Configuration Domain master. The workshop tool must be able to read out the Parameter specification for the vehicle.		
Content	**SubParameter Name**	**Length, Data Type**	
	Carconfig parameter as a subparameter #1	[1 Byte, Unsigned]	
	:	:	
	Carconfig parameter as a subparameter #2048	[1 Byte, Unsigned]	
Default	/		
Security access required	Write - Level 19/1A		
Other Session		Programming Session	
Read	Write	Read	Write
MC	**MC**	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

667217 v3 **DID - C03B**

Version history:
(3): 增加DID C03B的默认值。

Purpose: To enable readout and change of secure DoIP TCP communication configuration.

Source:
Verification Method: Test
Legacy ID: 657039v1, 657040v1, 652174v1

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	C03B			
Name	Secure DoIP TCP Communication Configuration			
Description	The data record shall be implemented in the ECU supporting secure DoIP TCP communication (i.e. DoIP TLS), and to readout or configure the use of secure DoIP TCP communication. It shall be possible to changel the value of the data record in extended session by WriteDataByidentifier service when conditions are met (i.e. valid PKI certificate stored). Once the value has been changed, it shall be stored in non-volatile memory and never be affected by reprogramming (SWDL). The function will be applied in next TCP reconnection. In case PKI certificate stored in ECU supporting secure DoIP (i.e. DoIP TLS) is revoked or damaged, the ECU shall revert to initial configuration. Note 1: Mandatory for DoIP Node if required by function requirements specification, e.g. WiFi diagnostics function.			
Content	**SubParameter Name**		**Length, Data Type**	
	Secure DoIP TCP Communication Configuration		[1 Byte, Unsigned]	
Default	0x00			
Security access required	Write - Level 05/06			
Other Session			Programming Session	
Read	Write		Read	Write
MC	**MC**		**MC**	N
Periodic Read	I/O Control		Periodic Read	I/O Control
N	N		N	N

667216 v2 **DID - C03C**

Version history:
(2):

Purpose: To enable readout and change of secure diagnostic function configuration for UDS service - Authentication service 29.
Source:
Verification Method: Test
Legacy ID: 656845v1, 656887v2, 656886v1

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	C03C
Name	Secure Diagnostic Function Configuration For UDS Service 29
Description	The data record shall be implemented in the ECU supporting UDS service - $29 Authentication service, and to readout or configure the use of secure diagnostics function for Authentication service. It shall be possible to change the value of the data record in extended session by WriteDataByidentifier service when condition is met (i.e. valid PKI certificate stored when configure bidrectional authentication). Once the value has been changed, it shall be stored in non-volatile memory and never be affected by reprogramming (SWDL). The function will be applied immediately. In case certificate supported by ECU is revoked or damaged, the ECU shall revert to initial configuration. Once the value have already in activated configuration, controlling the data record to inactivated configuration shall be protected by authentication. In case PKI certificate stored in ECU supporting bidrectional authentication is revoked or damaged, the ECU shall revert to initial configuration. Note 1: Mandatory for DoIP Node if UDS Service 29 is used.
Content	**SubParameter Name** — **Length, Data Type** Secure Diagnostic Function Configuration for Authentication — [1 Byte, Unsigned]
Default	/
Security access required	Write - Level 05/06

Other Session		Programming Session	
Read	Write	Read	Write
MC	**MC**	**MC**	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

667215 v3 **DID - C03D**

Version history:
(3): C03D的认证状态，未来也可以按位来定义哪一个角色已经过了认证。当前的实现可以认为是只使用了Bit0这一bit，Bit0为1时，认证通过；Bit0为0时，认证不通过。
未来根据以下证书的Roles角色扩展后，当Bit0为1时，认为Admin认证权限通过；Bit0为0时，Admin权限不通过。
最低位Bit0：Admin；
Bit1：比如，Supplier；
Bit2：比如，AfterSales；
Bit3：比如，Manufacture；
Bit4：预留；
Bit5：预留；
Bit6：预留；
Bit7：预留。

Purpose: To enable readout of Authentication state for UDS service - Authentication service 29.
Source:
Verification Method: Test
Legacy ID: 657045v1, 657065v1

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	C03D
Name	Authentication State For UDS Service 29
Description	The data record shall be implemented in the ECU supporting UDS service - $29 Authentication service, and to know the currently active authentication state through each bit. Note 1: Mandatory for DoIP Node if UDS Service 29 is used.
Content	**SubParameter Name** — **Length, Data Type** Authentication State for Authenticaion Service — [1 Byte, Unsigned]
Default	0x01 (Admin Authenticated state)
Security access required	/

Other Session		Programming Session	
Read	Write	Read	Write
MC	N	**MC**	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

1037501 v1 **DID - C098**

Version history:
(1): Add DID requirement to control whether to skip time validation during the authentication process.

Purpose: To meet the scenario where some vehicles have no network configuration and cannot perform time synchronization, methods are used to control the activation and deactivation of time verification during the authentication process.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	GMRDB
Name	Time Validation Function Configuration of Authentication Process

Description	The data record shall be implemented in the ECU supporting UDS service - $29 Authentication service, and to readout or configure the time validation function of Authentication process. When activate the time validation function, it means that it is necessary for ECU to validate the time information from OCSPResponse and client certificate during authentication process. On the contrary, if ideactivation configuration is in, it means ECU does not validate the time information. It shall be possible to change the value of the data record in extended session by WriteDataByidentifier service. Once the value has been changed, it shall be stored in non-volatile memory and never be affected by reprogramming (SWDL). The function will be applied immediately. Once the value have already in activated configuration, controlling the data record to inactivated configuration shall be protected by authentication. Note 1: Mandatory for DoIP Node if UDS Service 29 is used.
Content	**SubParameter Name** — **Length, Data Type** Time Validation Function Configuration Of Authentication — [1 Byte, Unsigned]
Default	/
Security access required	Write - Level 05/06

Other Session		Programming Session	
Read	Write	Read	Write
MC	**MC**	**MC**	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

665713 v1 **DID - C106**

Version history:
(1):

Purpose: Public-key cryptography uses asymmetric key algorithms which have a public and private key property. These keys are used to enable communication on a secure (e.g. WLAN, 3G) link between the vehicle and an external network. This data record is used to enable the change of the private key and shall be located in the ECU that handles such a link in the vehicle. The protection of private keys is crucial and keys must be stored in a secure memory area of the ECU.

Source:

Verification Method: Test

Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	C106
Name	X.509 Privat Key Property

Description	The Private Key is used to enable communication on a secure wireless link between vehicle and external network. A data record with identifier 0x C106 shall be implemented by the ECU containing WLAN and used as AP. The data record shall consist of 2048 unsigned bytes, be right justified with any unused bit(s)/bytes filled with 0.
Content	**SubParameter Name** — **Length, Data Type** X.509 Privat Key Property — [2048 bytes, Unsigned]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
N	O	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664684 v4 **DID - D01C**

Version history:
(4): Verification Method: Test change to Analysis AND Test.

Purpose: To enable programming of the Public Key, for ECUs supporting Software Authentication where only the key (and not certificates) is used to verify the authenticity of the software.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D01C
Name	Public Key
Description	Public Key used by ECUs supporting Software Authentication concept, where only the key itself (not certificates) is used for authenticity verification of the software. If the ECU is supporting the Software Authentication concept where the data file is verified using a Public Key, the ECU shall implement: 1 - Write the Public Key data record with identifier 0xD01C, it shall be supported when the ECU is executing the secondary bootloader - programming session. It shall only be possible to write once. 2 - Read the Public Key Checksum (other parts of that key is excluded when reading) with identifier 0xD01C when the ECU is executing in all diagnostic sessions. There is below description for ECU Public Key about DID usage.

ECU status	Mode	Read DID(22 D0 1C)	Write DID(2E D0 1C)
With development Public Key	PBL	DID support and return NRC 22(Conditions Not Correct)	Service not support and return NRC 11

		SBL	DID support and return NRC 22(Conditions Not Correct)	Support and return 6E D0 1C
		Application	DID support and return NRC 22(Conditions Not Correct)	Not Supported
	With production Public Key	PBL	Support	Service not support and return NRC 11
		SBL	Support	DID support and return NRC 22(Conditions Not Correct)
		Application	Support	Not Supported

Note1: See "Req ID 661313: Public Key Types" for key type representation and "Req ID 662082: Program and read of the public key" for ECU bootloader requirements.
Note2: D01C can be written with production Public Key once only after activated SBL, that means D01C can be executed after step- "Activate SBL" or before step-"check complete&compatible" or after step- "check complete&compatible".

Content	SubParameter Name	Length, Data Type
	Public Key Modulus	[256 Bytes, Hex]/ [384 Bytes, Hex]/[512 Bytes, Hex]
	Public Key Exponent	[4 Bytes, Hex]
	Public Key CheckSum	[32 Bytes, Hex]

Note1: The length of Public Key Modulus depends on signing method used, e.g. SHA256_RSA2048.
Note2: The Public Key CheckSum will use Hash algorithm, SHA256.

Default	/
Security access required	Write - Level 01/02

Other Session		Programming Session	
Read	Write	Read	Write
MC	N	MCPS	MCS
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

908828 v1 **DID - D09D**

Version history:
(1): 新增DID，在Application下可读。用于特定项目记录双电源电压。

Purpose: Record the dual power voltage if the specific project has a dual power strategy.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D09D

Name	Vehicle Battery Voltage
Description	Data record defined in *Global Master Reference Database* with the following additional requirements that are only applicable to the ECU responsible for measuring and distributing the vehicle battery voltage: - range minimum 0 to 63.6 V / 0x00 to 0xFE, correct value not available = 0xFF - resolution 1 bit = 0.25 V - see section *Detection of faults in external electrical circuits* The stored DTC snapshot data shall be reported D09D.
Content	**SubParameter Name** — **Length, Data Type** Vehicle Battery Voltage_KL30A — [1 Byte, Unit32] Vehicle Battery Voltage_KL30B — [1 Byte, Unit32]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

665710 v3 **DID - D0AF**

Version history:
(3): add VLAN ID #13 #14 #111

Purpose: To enable control of the priority of a given VLAN ID for test and verification during development.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D0AF
Name	VLAN Priority for Platform
Description	The identifier contains information of the priority for a specific VLAN ID. Mandatory and applicable only if the ECU is an Ethernet ECU. It shall be possible to manipulate the VLAN priority using service InputOutputControlByIdentifier.

Content	SubParameter Name	Length, Data Type
	VLAN ID #1 Priority	[4 bits, Unsigned]
	VLAN ID #2 Priority	[4 bits, Unsigned]
	VLAN ID #3 Priority	[4 bits, Unsigned]
	VLAN ID #4 Priority	[4 bits, Unsigned]
	VLAN ID #5 Priority	[4 bits, Unsigned]
	VLAN ID #6 Priority	[4 bits, Unsigned]
	VLAN ID #7 Priority	[4 bits, Unsigned]
	VLAN ID #8 Priority	[4 bits, Unsigned]
	VLAN ID #9 Priority	[4 bits, Unsigned]
	VLAN ID #10 Priority	[4 bits, Unsigned]
	VLAN ID #11 Priority	[4 bits, Unsigned]
	VLAN ID #12 Priority	[4 bits, Unsigned]
	VLAN ID #13 Priority	[4 bits, Unsigned]
	VLAN ID #14 Priority	[4 bits, Unsigned]
	VLAN ID #111 Priority	[4 bits, Unsigned]
Default	/	

Other Session		Programming Session	
Read	Write	Read	Write
MC	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	MC	N	N

697475 v2 **DID - D0B0**

Version history:
(2): Add VLAN DI #13 #14 #111

Purpose: To support test and verification during development.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D0B0
Name	VLAN Membership ECU Information
Description	A data record containing information of the members that are part of the VLAN shall be implemented. If the ECU is member in more than one VLAN, the ECU shall implement a data record for each VLAN. Mandatory and applicable only if the ECU has an Ethernet ECU.

Content	SubParameter Name	Length, Data Type
	VLAN ID #1 ECU #1	[1 bit, Unsigned]
	VLAN ID #1 ECU #2	[1 bit, Unsigned]
	……	
	VLAN ID #1 ECU #15	[1 bit, Unsigned]
	VLAN ID #1 ECU #16	[1 bit, Unsigned]
	VLAN ID #2 ECU #1	[1 bit, Unsigned]
	VLAN ID #2 ECU #2	[1 bit, Unsigned]
	……	
	VLAN ID #2 ECU #15	[1 bit, Unsigned]
	VLAN ID #2 ECU #16	[1 bit, Unsigned]
	……	
	……	
	VLAN ID #12 ECU #1	[1 bit, Unsigned]
	VLAN ID #12 ECU #2	[1 bit, Unsigned]
	……	
	VLAN ID #12 ECU #15	[1 bit, Unsigned]
	VLAN ID #12 ECU #16	[1 bit, Unsigned]
	……	
	VLAN ID #111 ECU #1	[1 bit, Unsigned]
	VLAN ID #111 ECU #2	[1 bit, Unsigned]
	……	
	VLAN ID #111 ECU #15	[1 bit, Unsigned]
	VLAN ID #111 ECU #16	[1 bit, Unsigned]
Default	/	

Other Session		Programming Session	
Read	Write	Read	Write
MC	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664701 v2 **DID - D0B5**

Version history:
(2): 针对GEEA3.0 SDB分支情况，对于SDB版本号超过5位，则使用SDB的后6位数字填充。

Purpose: To be able to read out the SDB version number from ECUs implementing ECU extract.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D0B5
Name	SDB version

Description	Data records intended for SDB version tracking of the ECU is defined by Geely. The SignalDataBase version part number 0xD0B5 shall be coded in BCD, right justified, with any unused digits filled with 0 if the SDB version does not exceed 5 digits. Otherwise, fill in the last 6 digits of the SDB version. Mandatory if required by the responsible authority appointed by Geely.
Content	**SubParameter Name** — **Length, Data Type** Unused — [4 bits, BCD] SDB version — [20 bits, BCD]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

688989 v2 **DID - D105**

Version history:
(2):

Purpose: To enable readout the seamless update support information based on dual bank.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D105
Name	A/B Swap Support Information
Description	Used to identify whether seamless update is supported based on AB swap.
Content	**SubParameter Name** — **Length, Data Type** A/B Swap Support Information — [1 Byte, Unsigned]
Default	/
Security access required	/

Other Session		Programming Session	
Read	Write	Read	Write
M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control

N	N	N	N

714778 v1 **DID - D122**

Version history:
(1): 新增DID，用于搭配23服务使用。

Purpose: To get the memory address and length information of ReadMemoryByAddress service request parameter.
Source: Diagnostic services - Data Transmission Functional Unit "ReadMemoryByAddress (0x23)".
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D122
Name	Read Memory By Address (0x23) Service 14
Description	A data record set contains information of the memoryAddress and memorySize used in the MemoryByAddress (0x23) service, and is applicable that the address and length format identifier is 0x14. Complete set of memoryAddress and memorySize for the request parameter. The content of the dataRecord shall be the following: Byte Description #1 Total number of ECU Stage Read Momory By Address Service #2 Address And Length Format Identifier, i.e. 0x14 #3 First Stage Read Memory Size #4 - 7 First Stage Read Start Address #8 Second Stage Read Memory Size #9 - 12 Second Stage Read Start Address . . . . #N Stage #X Read Memory Size #N+1 - (N+4) Stage #X Read Start Address Note1: Parameter - Total number of ECU Stage Read Memory By Address Service shall be defined as the total number of used and unused stages. Note2: The memory address and memory size of unused stage shall default to all zeros.
Content	**SubParameter Name** — **Length, Data Type** Total number of ECU Stage Read Memory By Address Service — [1 byte, Unsigned] Address And Length Format Identifier — [1 byte, Unsigned] First Stage Read Memory Size — [1 byte, Unsigned] First Stage Read Start Address — [4 byte, Unsigned] …… More Stage Read Memory Size and Start Address
Default	/

Other Session		Programming Session	
Read	Write	Read	Write

M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

714886 v1 **DID - D123**

Version history:
(1): 新增DID，用于搭配23服务使用。

Purpose: To get the memory address and length information of ReadMemoryByAddress service request parameter.
Source: Diagnostic services - Data Transmission Functional Unit "ReadMemoryByAddress (0x23)".
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D123
Name	Read Memory By Address (0x23) Service 24
Description	A data record set contains information of the memoryAddress and memorySize used in the MemoryByAddress (0x23) service, and is applicable that the address and length format identifier is 0x24. Complete set of memoryAddress and memorySize for the request parameter. The content of the dataRecord shall be the following: Byte Description #1 Total number of ECU Stage Read Momory By Address Service #2 Address And Length Format Identifier, i.e. 0x24 #3 - 4 First Stage Read Memory Size #5 - 8 First Stage Read Start Address #9 - 10 Second Stage Read Memory Size #11 - 14 Second Stage Read Start Address . . . . #N - (N+1) Stage #X Read Memory Size #N+2 - (N+5) Stage #X Read Start Address Note1: Parameter - Total number of ECU Stage Read Memory By Address Service shall be defined as the total number of used and unused stages. Note2: The memory address and memory size of unused stage shall default to all zeros.
Content	**SubParameter Name** — **Length, Data Type** Total number of ECU Stage Read Memory By Address Service — [1 byte, Unsigned] Address And Length Format Identifier — [1 byte, Unsigned] First Stage Read Memory Size — [2 byte, Unsigned] First Stage Read Start Address — [4 byte, Unsigned] …… More Stage Read Memory Size and Start Address

Default	/		
Other Session		Programming Session	
Read	Write	Read	Write
M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

714887 v2 **DID - D124**

Version history:
(2): Verification Method: Test.

Purpose: To get the memory address and length information of ReadMemoryByAddress service request parameter.
Source: Diagnostic services - Data Transmission Functional Unit "ReadMemoryByAddress (0x23)".
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D124
Name	Read Memory By Address (0x23) Service 44
Description	A data record set contains information of the memoryAddress and memorySize used in the MemoryByAddress (0x23) service, and is applicable that the address and length format identifier is 0x44. Complete set of memoryAddress and memorySize for the request parameter. The content of the dataRecord shall be the following: Byte Description #1 Total number of ECU Stage Read Momory By Address Service #2 Address And Length Format Identifier, i.e. 0x44 #3 - 6 First Stage Read Memory Size #7 - 10 First Stage Read Start Address #11 - 14 Second Stage Read Memory Size #15 - 18 Second Stage Read Start Address . . . . #N - (N+3) Stage #X Read Memory Size #N+4 - (N+7) Stage #X Read Start Address Note1: Parameter - Total number of ECU Stage Read Memory By Address Service shall be defined as the total number of used and unused stages. Note2: The memory address and memory size of unused stage shall default to all zeros.

Content	SubParameter Name	Length, Data Type
	Total number of ECU Stage Read Memory By Address Service	[1 byte, Unsigned]
	Address And Length Format Identifier	[1 byte, Unsigned]
	First Stage Read Memory Size	[4 byte, Unsigned]
	First Stage Read Start Address	[4 byte, Unsigned]
	……	
	More Stage Read Memory Size and Start Address	
Default	/	

Other Session		Programming Session	
Read	Write	Read	Write
M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

908405 v1 **DID - D12C**

Version history:
(1): 新增DID，在Application下可读可写。用于控制开发阶段研发报文的发送和停止发送。

Purpose: To control the transmission of communication development message to avoid a large number of development messages affecting the CAN bus load.

Source:

Verification Method: Test

Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D12C	
Name	Communication Development Message State	
Description	If the ECU owns the development messages, it shall use the communication development message state with identifier 0xD12C to control the persistent activating and deactivating of the sending of development messages. And the current status can be obtained by reading this data record. The control result of the development message state shall be stored in the non-volatile memory, and can take effect in the current cycle or in the next cycle. Note1: Development messages are only used as assistance in troubleshooting during the development phase and shall be uniformly closed after mass production.	
Content	**SubParameter Name**	**Length, Data Type**
	Communication Development Message State	[1 Bytes, Hex]
Default	/	

Security access required	Write - Level 05/06		
Other Session		Programming Session	
Read	Write	Read	Write
MC	MC	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

908214 v3 **DID - D131**

Version history:
(3):

Purpose: Track the identity of the ECUs on the vehicle to avoid illegal replacement.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D131
Name	Security Constant Level 01/02 Hash Value
Description	The value shall be calculated as Hash(the value of Security Constant Level 01/02) by ECU itself when security constant level 01/02 has been written successfully in programming session, and ECU stores the value to non-volatile memory. The algorithm used to calculate the checksum shall use the hash function SHA-256. If the ECU that support the DID 0xF109 - Security Constant Level 01/02 shall support this DID, and implement following requirements 1 - Read the Security Constant Level 01/02 Hash Value with identifier 0xD131 when the ECU is executing in the default or extended sessions. There is below description for ECU about DID usage.

ECU status	Mode	Read DID(22 D1 31)
With development security constant	PBL	Not Supported
	SBL	Not Supported
	Application	**Support**
With production security constant	PBL	Not Supported
	SBL	Not Supported
	Application	**Support**

Note1: When the value has been programmed, it is not allowed to be changed through any SWDL or Update.
Example: Hash_SHA256 (68 81 f5 af f8 d4 6b dd 71 ad e8 41 e8 2d b3 24) = a0 12 46 ef 70 b3 b0 f0 77 4a 3a 9e 30 b6 0e f4 86 e9 6e c4 54 93 8f 19 8a de 95 69 9a 7d 46 b9

Content	**SubParameter Name**	**Length, Data Type**
	Security Constant Level 01/02 Hash Value	[32 Bytes, Hex]
Default	All 0xFF	
Security access required	/	

Other Session		Programming Session	
Read	Write	Read	Write
MC	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664686 v2 **DID - D134**

Version history:
(2):

Purpose: To enable readout and control of the active Car mode.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D134	
Name	Car Mode	
Description	Currently active Car Mode. Mandatory for ECUs that are publisher or subscriber of the car mode signal. Mandatory for car mode publisher should be support I/O control by DID D134.	
Content	**SubParameter Name**	**Length, Data Type**
	Car Mode	[1 Byte, Unsigned]
Default	/	
Security access required	I/O Control - Level 03/04	

Other Session		Programming Session	
Read	Write	Read	Write
M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	MC	N	N

1078814 v1 **DID - D1A7**

Version history:

Purpose: Due to security concerns, this is used to enable and disable the routing function of the edge node SAE J2534.

Source:

Verification Method: Analysis AND Test

Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D1A7
Name	SAE J2534 Routing Enable Switch
Description	Used to enable and disable the routing function of the J2534 protocol for the gateway.
Content	**SubParameter Name** **Length, Data Type** SAE J2534 Routing Enable Switch [1 Byte, Unsigned] Note 1: Mandatory for Edge DoIP Node .
Default	0x00 (Close)
Security access required	Write - Level 19/1A

Other Session		Programming Session	
Read	Write	Read	Write
M	M	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664689 v3 **DID - DD00**

Version history:
(3): 1、删除电源丢失处理的需求描述，此描述已经在需求ReqID 661860定义了。2、增加DID的支持情况。

Purpose: To enhance fault tracing and root cause analysis specific vehicle global data shall be stored with each DTC.

Source:

Verification Method: Test

Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	DD00

Name	Global Real Time
Description	ECU received global real time signal "CarTiGlb" as DID DD00 value. The global real time data: - range 0 to 13.6 year / 0x00000000 to 0xFFFFFFFE, correct value not available = 0xFFFFFFFF - resolution 1 bit = 100 ms = 1/600 min. - accuracy minimum ±10 minutes/week (±0.1%). - after the global time have reached 0xFFFFFFFE it shall restart from 0x00000000. The stored DTC snapshot data may be reported DD00. **Note:** This DID is not mandatory for all ECU's. Mandatory if the ECU is responsible for maintaining and distributing the real time data.
Content	**SubParameter Name** — **Length, Data Type** Global Real Time — [4 Bytes, Unsigned]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
O	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664690 v3 **DID - DD01**

Version history:
(3): 依据SDB信号，更新DID最大值参数。

Purpose: To enhance fault tracing and root cause analysis specific vehicle global data shall be stored with each DTC.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	DD01
Name	Total Distance
Description	ECU received total distance data signal "BkpOfDstTrvld" as DID DD01 value. The total distance data: - range minimum 0 to 2 000 000 km / 0x000000 to 0x1E8480, correct value not available = 0xFFFFFF - resolution 1 bit = 1 km - the accuracy shall be the same as or better than the accuracy of the total distance displayed in the dash board. The stored DTC snapshot data shall be reported DD01.

Content	**SubParameter Name**	**Length, Data Type**
	Total Distance/Used by AS	[3 Bytes, Unsigned]
Default	/	

Other Session		Programming Session	
Read	Write	Read	Write
M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664691 v3 **DID - DD02**

Version history:
(3): 依据SDB信号，更新DID最大值参数。SDB的scaling为0.1，需求中的scaling为0.25，所以不能直接取信号VehBattU为DD02的值，应做一个转换。

Purpose: To enhance fault tracing and root cause analysis specific vehicle global data shall be stored with each DTC.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	DD02	
Name	Vehicle Battery Voltage	
Description	ECU received vehicle battery voltage signal "VehBattU" as DID DD02 **conversion** value. The vehicle battery voltage: - range minimum 0 to 25 V / 0x00 to 0x64, correct value not available = 0xFF. - resolution 1 bit = 0.25 V. - see section Detection of faults in external electrical circuits. The stored DTC snapshot data shall be reported DD02.	
Content	**SubParameter Name**	**Length, Data Type**
	Vehicle Battery Voltage	[1 Byte, Unsigned]
Default	/	

Other Session		Programming Session	
Read	Write	Read	Write
M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

676168 v4 DID - DD06

Version history:
(4): 更新DID数据在不同单位下的转换关系。

Purpose: To enhance fault tracing and root cause analysis specific vehicle global data shall be stored with each DTC.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	DD06			
Name	Vehicle Speed			
Description	ECU received vehicle speed signal "VehSpdLgt" as DID DD06 value. The Vehicle Speed Time: - range minimum 0 to 125.0027m/s (450.00972 km/h) / 0x00 to 0x7CE2, correct value not available = 0xFFFF. - resolution 1 bit = 0.00391 m/s = 0.014076 km/h. The stored DTC snapshot data shall be reported DD06.			
Content	**SubParameter Name** Vehicle Speed		**Length, Data Type** [2 Byte, Unsigned]	
Default	/			
Other Session			Programming Session	
Read	Write		Read	Write
M	N		N	N
Periodic Read	I/O Control		Periodic Read	I/O Control
N	N		N	N

666817 v6 DID - DD07

Version history:
(6): Verification Method: Test change to Analysis AND Test.

Purpose: To enhance fault tracing and root cause analysis specific vehicle global data shall be stored with each DTC.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	DD07
Name	GNSS Time
Description	ECU received GNSS time signal "TiAndDateIndcn" as DID DD07 value. The GNSS Time: - Year range 0 to 99/0x0 to 0x63, correct value not available = 0xFF. - Month range 1 to 12/0x1 to 0xC, correct value not available = 0xFF. - Day range 1 to 31/0x1 to 0x1F, correct value not available = 0xFF. - Hour range 0 to 23/0x0 to 0x17, correct value not available = 0xFF. - Minute range 0 to 59/0x0 to 0x3B, correct value not available = 0xFF. - Second range 0 to 59/0x0 to 0x3B, correct value not available = 0xFF. Note for ECU responsible for maintaing and distributing the GNSS time data: • The current value of GNSS Time shall be stored in long term memory at least once each operation cycle. • If the ECU loses the current value due to loss of power supply then, following reconnection of the power supply, it shall execute the predetermined functional logic according to the ECU SWRS *ReqID 157155 Fault handling for the ClockCalculation*. The stored DTC snapshot data shall be reported DD07.
Content	**SubParameter Name** — **Length, Data Type** Year — [1 Byte, Unsigned] Month — [1 Byte, Unsigned] Day — [1 Byte, Unsigned] Hour — [1 Byte, Unsigned] Minute — [1 Byte, Unsigned] Second — [1 Byte, Unsigned]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664692 v3 **DID - DD0A**

Version history:
(3): 更新描述，避免歧义。

Purpose: To enhance fault tracing and root cause analysis specific vehicle global data shall be stored with each DTC.

Source:

Verification Method: Test

Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	DD0A
Name	Usage Mode
Description	The Usage Mode: - Abandoned - Inactive - Convenience - Active - Driving The stored DTC snapshot data shall be reported DD0A. I/O control must be mandatory for VMM master. Note: This usage mode should remain the last value which after using I/O control even if the non-DefaultSession transition, i.e. Extended session to Extended session.
Content	**SubParameter Name** — **Length, Data Type** Usage Mode — [1 Byte, Unsigned]
Default	/
Security access required	I/O Control - Level 03/04

Other Session		Programming Session	
Read	Write	Read	Write
M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	MC	N	N

664693 v1 **DID - DD0B**

Version history:
(1):

Purpose: To enhance fault tracing and root cause analysis specific vehicle global data shall be stored with each DTC.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	DD0B
Name	Partial Network Cluster (PNC)
Description	The record data is a part of the network management user data and consists of a bit array with one bit for each PNC which indicate whether or not the PNC shall currently be active. This DID is not mandatory for all ECU's. Mandatory if the ECU is the Vehicle Mode Management (VMM) master.

Content	SubParameter Name	Length, Data Type
	Partial Network #N(Every PNC as a subparameter)	[4 Bytes, Unsigned]
Default	/	

Other Session		Programming Session	
Read	Write	Read	Write
M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664694 v1 **DID - DD0C**

Version history:
(1):

Purpose: To enhance fault tracing and root cause analysis specific vehicle global data shall be stored with each DTC.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	DD0C	
Name	Electric Power Level	
Description	Data record defined by reference GMRDB. The stored DTC snapshot data shall be reported DD0C.	
Content	**SubParameter Name**	**Length, Data Type**
	Electric Power Level	[1 Byte, Hex]
Default	/	

Other Session		Programming Session	
Read	Write	Read	Write
M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664687 v1 **DID - DED0**

Version history:
(1):

Purpose: The purpose is to support the ECU replacement process at workshop.

Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	DED0
Name	Customer Setting Parameters
Description	Data record intended for customer setting parameters. The data record shall only consist of record data that contain information of customer settings stored by the ECU. The data record is used for restoring customer settings in the ECU replacement process at a workshop. A customer setting is defined as: • A function that is open for alteration by the customer and/or tester tool it will be possible to change and adapt behavior of the functionality in the car to suit the customer. • A long term alteration that only is initiated by the customer himself. The ECU is not allowed to change the customer setting parameter. Short term adjustments that, due to safety or other reasons, only are valid in one driving cycle or less and will revert to a predefined value are not to be considered as customer setting parameter. A setting that can be done by the customer himself and by the tester tool or only by a tester tool (at workshop or remote). Option if the ECU stores customer settings that is expected to be rewritten to a new ECU in the ECU replacement process at a workshop.
Content	**SubParameter Name** — **Length, Data Type** Number of bytes — [2 Bytes, Unsigned] CRC — [1 Byte, Unsigned] Value to be rewritten: Parameter #1~#N (Every parameter as a subparameter) — [1 Byte, Unsigned]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
O	O	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664688 v1 **DID - DED1**

Version history:
(1):

Purpose: The purpose is to support the ECU replacement process at workshop.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	DED1
Name	System Adaption Data
Description	Data records intended adaption parameters.The data record shall only contain information of adaption data stored by the ECU. The data record is used for restoring adaption data in the ECU replacement process in workshop.Adaption parameters are adjusted to the customers usage/driving and/or components performance/lifespan in order to optimize the performance and drivability in the vehicle. The data record shall be used where it is considered suitable that adaption values are transferred from an old to a new ECU in the ECU replacement process at a workshop. Option if the ECU stores adaption parameters that is expected to be rewritten to a new ECU in the ECU replacement process at a workshop.
Content	**SubParameter Name** — **Length, Data Type** Number of bytes — [2 Bytes, Unsigned] CRC — [1 Byte, Unsigned] Value to be rewritten: Parameter #1~#N — [1 Byte, Unsigned] (Every parameter as a subparameter)
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
O	O	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

664695 v3 **DID - E103**

Version history:
(3):

Purpose: To support a more advanced analysis of the vehicle behaviour it must be possible to understand how many incorrect configuration parameters that the ECU has received.

Source:

Verification Method: Test

Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	E103
Name	Car Configuration Parameter Faults
Description	List of invalid car configuration parameter values identified by the ECU. Mandatory if the ECU support the DTC U230056 (0xE30056) - Invalid vehicle configuration data.

Content	SubParameter Name	Length, Data Type
	Number of invalid car configuration parameter values recognized	[1 Byte, Unsigned]
	#N parameter number of the car configuration data that is invalid	[2 Bytes, Unsigned]
	#N parameter invalid value (N=1~10)	[1 Byte, Unsigned]
Default	/	

Other Session		Programming Session	
Read	Write	Read	Write
MC	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

62 E1 03 XX YY YY ZZ YY YY ZZ YY YY ZZ …

3bytes 1byte 30bytes

Note:
1、DTC and DID should disappear at the same time if there one pre-condition happen as follow:
- **The ECU receive the valid value of all the parameters they need.**

2、The DID should report all 00（31bytes） if the ECU receive all valid value they need.
3、The DID should only report the max 10 invalid values if ECU received above 10 incorrect parameters，But the XX above should report the actual total number of invalid parameters.

Example:
Pre-condition:
1)ECU only receive parameter #04;
2)Applicable for both states in the ECU

Case1：
At first 9s of the 30s, ECU received the correct value, No DTC/E103 report all 00/Don't detect the config signal next 21s any more
Case2：
At first 9s of the 30s, ECU received the incorrect value，Detect second 9s of 30s
At second 9s of the 30s,ECU received the correct value, Don't detect the config signal next 12s any more
Case3：
At first 9s of the 30s, ECU received the incorrect value，Detect second 9s of 30s
At second 9s of the 30s,ECU still received the incorrect value,Detect last 9s of 30s
At last 9s of the 30s,ECU received the correct value, Don't detect the config signal any more
Case4：
At first 9s of the 30s, ECU received the incorrect value，Detect second 9s of 30s
At second 9s of the 30s,ECU still received the incorrect value，Detect last 9s of 30s
At last 9s of the 30s,ECU still received the incorrect value again, Record DTC/E103 report / 01 00 04 FF 00 00 00..../Don't detect the config signal any more。

675441 v3 **DID - E104**

Version history:
(3):

Purpose: To support a more advanced analysis of the vehicle behaviour it must be possible to understand which Car Configuration Logic(CCL) version has been used in the vehicle.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	E104
Name	Car Configuration Logic(CCL) Version
Description	The Car Configuration Logic(CCL) version has been used in the vehicle. Mandatory if the ECU is the Car Configuration Master.
Content	**SubParameter Name** — **Length, Data Type** CCL part NO. — [5 Bytes, BCD] CCL part version — [3 Bytes, ASCII] Example: 0x66 0x08 0x18 0x31 0x78 0x20 0x20 0x43 ---- 6608183178 A
Default	/
Security access required	Write - Level 19/1A

Other Session		Programming Session	
Read	Write	Read	Write
MC	MC	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

665697 v4 DID - Ethernet Error Message Counter

Version history:
(4): Verification Method: Analysis change to Analysis AND Test.

Purpose: To support test and verification during development.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D240, D241, D242, D243, D244, D245, D246, D247, D248
Name	Ethernet Error Message Counter #0 ~ #8

Description	The identifier contains the accumulated value of counted error messages. The 4 byte counter shall stop counting when it has reached its maximum value, i.e. it shall not act as a rolling counter. Mandatory and applicable only if the ECU is an Ethernet connector.
Content	**SubParameter Name** — **Length, Data Type** Ethernet Error Message Counter — [4 bytes, Unsigned]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
MC	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

665699 v2 DID - Ethernet Error Message Counter History

Version history:
(2): 增加相同内容但适用不同port需要的所有DID号。

Purpose: To enable evaluation and statistic of error message on Ethernet.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D230, D23F, D249, D24A, D24B, D24C, D24D, D24E, D24F
Name	Ethernet Error Message Counter History #0 ~ #8
Description	Histogram over the ten latest driving cycles with error message occurrence. The data record shall consist of the ten latest driving cycles where error messages on the Ethernet link occurred. Each error message event shall consist of a 4 byte counter value and a timestamp. The 4 byte counter shall stop counting when it has reached its maximum value, i.e. it shall not act as a rolling counter. The timestamp shall be the global real time value. Mandatory and applicable only if the ECU is an Ethernet connector.

Content	SubParameter Name	Length, Data Type
	Counter Value Latest Driving Cycle With Error Message	[4 bytes, Unsigned]
	Global Real Time Value Latest Cycle	[4 bytes, Unsigned]
	Counter Value 2nd Latest Driving Cycle With Error Message	[4 bytes, Unsigned]
	Global Real Time Value 2nd Latest Cycle	[4 bytes, Unsigned]
	Counter Value 3rd Latest Driving Cycle With Error Message	[4 bytes, Unsigned]
	Global Real Time Value 3rd Latest Cycle	[4 bytes, Unsigned]
	Counter Value 4th Latest Driving Cycle With Error Message	[4 bytes, Unsigned]
	Global Real Time Value 4th Latest Cycle	[4 bytes, Unsigned]
	Counter Value 5th Latest Driving Cycle With Error Message	[4 bytes, Unsigned]
	Global Real Time Value 5th Latest Cycle	[4 bytes, Unsigned]
	Counter Value 6th Latest Driving Cycle With Error Message	[4 bytes, Unsigned]
	Global Real Time Value 6th Latest Cycle	[4 bytes, Unsigned]
	Counter Value 7th Latest Driving Cycle With Error Message	[4 bytes, Unsigned]
	Global Real Time Value 7th Latest Cycle	[4 bytes, Unsigned]
	Counter Value 8th Latest Driving Cycle With Error Message	[4 bytes, Unsigned]
	Global Real Time Value 8th Latest Cycle	[4 bytes, Unsigned]
	Counter Value 9th Latest Driving Cycle With Error Message	[4 bytes, Unsigned]
	Global Real Time Value 9th Latest Cycle	[4 bytes, Unsigned]
	Counter Value 10th Latest Driving Cycle With Error Message	[4 bytes, Unsigned]
	Global Real Time Value 10th Latest Cycle	[4 bytes, Unsigned]
Default	/	

Other Session		Programming Session	
Read	Write	Read	Write
MC	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

665701 v2 **DID - Ethernet Link Status**

Version history:
(2): 增加相同内容但适用不同port需要的所有DID号。

Purpose: To support test and verification during development and fault tracing for aftersales.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D250, D25E, D25F, D704, D705, D706, D707, D708, D709
Name	Ethernet Link Status #0 ~#8

Description	The identifier contains information regarding the status of the link. Mandatory and applicable only if the ECU is an Ethernet connector.
Content	**SubParameter Name** **Length, Data Type** Ethernet Link Status [1 byte, Unsigned]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
MC	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

665702 v2 **DID - Extended Ethernet Link Status**

Version history:
(2): 增加相同内容但适用不同port需要的所有DID号。

Purpose: The record identifier shall be accessible by using the service ReadDataByPeriodicIdentifier.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D251, D252, D253, D254, D25A, D25B, D25C, D25D, D70A
Name	Extended Ethernet Link Status #0 ~ #8
Description	The identifier contains information regarding the status of the link. Mandatory and applicable only if the ECU is an Ethernet connector.
Content	**SubParameter Name** **Length, Data Type** Extended Ethernet Link Status [1 byte, Unsigned]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
MC	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

665703 v3 **DID - Signal Quality Index**

Version history:
(3): Signal Quality Index的DID“应该”通过$2A服务读取调整为“能够”通过$2A服务读取。不强制要求。增加Note关于SQI值的定义。

Purpose: To support test and verification during development and fault tracing for aftersales.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

Note：If SQI continuously maintains values below 4(for SQI range from 0~7) / 8(for SQI range from 0~15), consider it is a potential risk.

DID [Hex]	D260, D268, D269, D26A, D26B, D26D, D26E, D26F, D70B
Name	Signal Quality Index #0 ~ #8
Description	The identifier contains information regarding the quality index. The record identifier may be accessible by using the service ReadDataByPeriodicIdentifier. Mandatory and applicable only if the ECU is an Ethernet connector.
Content	**SubParameter Name** **Length, Data Type** Index Value [1 byte, Unsigned]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
MC	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
O	N	N	N

665704 v2 DID - VLAN Membership Port #0 ~ #11

Version history:
(2):

Purpose: To support test and verification during development.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D255, D256, D257, D258, D259, D261, D262, D263, D264, D265, D266, D267
Name	VLAN Membership Port #0 ~ #11

Description	A data record containing information of the members that are part of the VLAN per used port shall be implemented. A data record per switch port will list the vlan information. Mandatory and applicable only if the ECU has an Ethernet Switch.
Content	**SubParameter Name** — **Length, Data Type** VLAN ID #1 — [12 bits, Unsigned] VLAN Priority #1 — [4 bits, Unsigned] VLAN ID #2 — [12 bits, Unsigned] VLAN Priority #2 — [4 bits, Unsigned] VLAN ID #3 — [12 bits, Unsigned] VLAN Priority #3 — [4 bits, Unsigned] VLAN ID #4 — [12 bits, Unsigned] VLAN Priority #4 — [4 bits, Unsigned] VLAN ID #5 — [12 bits, Unsigned] VLAN Priority #5 — [4 bits, Unsigned] VLAN ID #6 — [12 bits, Unsigned] VLAN Priority #6 — [4 bits, Unsigned] VLAN ID #7 — [12 bits, Unsigned] VLAN Priority #7 — [4 bits, Unsigned] VLAN ID #8 — [12 bits, Unsigned] VLAN Priority #8 — [4 bits, Unsigned] VLAN ID #9 — [12 bits, Unsigned] VLAN Priority #9 — [4 bits, Unsigned] VLAN ID #10 — [12 bits, Unsigned] VLAN Priority #10 — [4 bits, Unsigned] VLAN ID #11 — [12 bits, Unsigned] VLAN Priority #11 — [4 bits, Unsigned] VLAN ID #12 — [12 bits, Unsigned] VLAN Priority #12 — [4 bits, Unsigned]
Default	/

Other Session		Programming Session	
Read	Write	Read	Write
MC	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

•RID data

4.10.3.2.2 RID data

665392 v3 **RID - 0205**

Version history:
(3): 将P4server_max由等于P2server_max调整为200ms。

Purpose: All ECUs must support routines defined for SWDL.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account, and exactly as defined in *Global Master Reference Database.*

RID [Hex]	0205		
Name	Check Complete And Compatible		
Description	The Check Complete And Compatible routine shall check that the downloaded block has been transferred without manipulation.		
Type	1		
$P4_{Server_Max}$	200ms		
Security access	Level 01/02		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	N
Response parameters	**Start [0x01] RoutineStatusRecords**	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	
	Complete	[1 bit, Unsigned]	
	Compatible	[1 bit, Unsigned]	
	Executable[EXE]	[1 bit, Unsigned]	
	Signal Configuration [SIGCFG]	[1 bit, Unsigned]	
	Car Configuration [CARCFG]	[1 bit, Unsigned]	
	User Defined SW [e.g. DATA]	[1 bit, Unsigned]	
	...		
	User Defined SW [e.g. DATA]	[1 bit, Unsigned]	
Available in application		Available in bootloader	
$40 - Update Session (If support seamless update)		MPS	

665393 v1 **RID - 0206**

Version history:
(1):

Purpose: All ECUs must support routines defined for SWDL.
The tester will send requests to all ECU to find out if they are prepared to enter program mode. This enables a robust and secure transition from the application to the bootloader by ensuring that all pre-conditions to transition to the bootloader are fulfilled when requested.

Source:

Verification Method: Test

Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	0206
Name	Check programming pre-conditions
Description	The routine shall check if the ECU, under its current operating conditions, is able to enter program mode. If the ECU is not able to enter program mode, i.e. the criteria for entering program mode are not fulfilled at the moment of the routineControl request, this shall be indicated by the corresponding RoutineStatusRecord response parameter. Possible circumstances that can cause an ECU to reject to enter program mode when requested are for example certain safety conditions that may need to be satisfied (e.g. vehicle shall not be moving or engine shall not be running). If the ECU need time to store data to non-volatile memory before making a mode transition, caused by the service DiagnosticSessionControl (programmingSession), this shall be performed within the time allowed for the mode change as given by the requirement for diagnostic availability.
Type	1
$P4_{Server_Max}$	50ms
Security access	-

Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	N

Request parameters	-
Response parameters	**Start [0x01] RoutineStatusRecords** — **Length, Data Type** Routine Type — [4 bits, Unsigned] Routine Status — [4 bits, Unsigned] Condition Fullfilled To Enter Programming Session — [1 byte, Hex]

Available in application	Available in bootloader
M	N

665397 v2 **RID - 0212**

Version history:
(2): 有无感升级的ECU，需要在UpdateSession(0x40)会话下支持该RID。

Purpose: To initiate the authenticity verification of the downloaded data file at SWDL.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	0212

Name	Check Memory		
Description	If the ECU is supporting the Software Authentication concept , the ECU shall implement the Check Memory routine with routine identifier 0x0212.		
Type	1		
$P4_{Server_Max}$	5000ms		
Security access	Level 01/02		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	N
Request parameters	**Start [0x01] Request parameter** Software Signature	**Length, Data Type** [256 bytes, Hex]	
Response parameters	**Start [0x01] RoutineStatusRecords** Routine Type Routine Status Result Value	**Length, Data Type** [4 bits, Unsigned] [4 bits, Unsigned] [1 byte, Hex]	

Available in application	Available in bootloader
MC[1]	MPS

Note 1: ECU which implement seamless update shall support this RID in the DiagnosticSessionControl (0x10) service with diagnosticSessionType equal to UpdateSession (0x40/0xC0).

665398 v2 **RID - 0235**

Version history:
(2):

Purpose: To start installation of a downloaded data file which requires to be installed.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	0235
Name	Self-installation Control
Description	The Software Installation routine with routine identifier 0x0235 shall be implemented. The routine is optional and shall be used for ECUs supporting file system storage and distributed download to start an installation of a downloaded data file. The parameter GLID in this startRoutine request shall match the GLID in previous after-sales assignment push routine. **Note:** The requirements of distributed download (i.e. downloading software part(s) through HTTPS protocol) is defined in ECU SWRS.
Type	2

$P4_{Server_Max}$	$P2_{Server_Max}$ = 25ms		
Security access	Level 01/02		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	M	M
Request parameters	**Start [0x01] Request parameter** OTA assignment UUID/Aftersale assignment GLID	**Length, Data Type** [36 bytes, Hex]	
Response parameters	**Start [0x01] RoutineStatusRecords** Routine Type Routine Status **Stop [0x02] RoutineStatusRecords** Routine Type Routine Status Control Status **Result [0x03] RoutineStatusRecords** Routine Type Routine Status Installation progress Abort reason Reason statement	**Length, Data Type** [4 bits, Unsigned] [4 bits, Unsigned] **Length, Data Type** [4 bits, Unsigned] [4 bits, Unsigned] [1 byte, Unsigned] **Length, Data Type** [4 bits, Unsigned] [4 bits, Unsigned] [1 byte, Unsigned] [1 byte, Hex] [49 bytes, Hex]	
Available in application		Available in bootloader	
N		MCPS	

665399 v3 **RID - 0236**

Version history:
(3):

Purpose: To start downloaded data file.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	0236
Name	File Download For Aftersales

Description	The Software data transmission routine with routine identifier 0x0236 shall be implemented. The routine is optional and shall be used for ECUs supporting file system storage and distributed download to download data file. The routineContolOptionRecord has a dynamical length and contains the link that ECU needs to obtain the files from the cloud backend server. If the ECU has not finished receiving file data for more than 30 minutes, it shall actively end this routine process. **Note:** The requirements of distributed download (i.e. downloading software part(s) through HTTPS protocol) is defined in ECU SWRS.		
Type	2		
$P4_{Server_Max}$	$P2_{Server_Max}$ = 50ms		
Security access required	Level 05/06		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	M	M
Request parameters	**Start [0x01] Request parameter** — **Length, Data Type** FileRecordNumber — [1 byte, Unsigned] URL Length — [2 bytes, Unsigned] URL Address — [The length of this parameter is defined by the URL Length, Unsigned]		
Response parameters	**Start [0x01] RoutineStatusRecords** — **Length, Data Type** Routine Type — [4 bits, Unsigned] Routine Status — [4 bits, Unsigned] Reserved — [2 bytes, Unsigned] **Stop [0x02] RoutineStatusRecords** — **Length, Data Type** Routine Type — [4 bits, Unsigned] Routine Status — [4 bits, Unsigned] Reserved — [2 bytes, Unsigned] **Result [0x03] RoutineStatusRecords** — **Length, Data Type** Routine Type — [4 bits, Unsigned] Routine Status — [4 bits, Unsigned] Abort Reason — [1 byte, Hex] Download Progress — [1 byte, Hex]		
Available in application	MC	Available in bootloader	N

665400 v2 **RID - 0237**

Version history:
(2):

Purpose: To start an after-sales assignment.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	0237		
Name	Assignment Push Control For After-sales		
Description	The routine is optional and shall be used for ECUs supporting file system storage and distributed download to start the after-sales assignment. The parameter GLID indicates an after-sales assignment for file system ECU. **Note:** The requirements of distributed download (i.e. downloading software part(s) through HTTPS protocol) is defined in ECU SWRS.		
Type	1		
$P4_{Server_Max}$	$P2_{Server_Max}$ = 50ms		
Security access	Level 05/06		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	N
Request parameters	**Start [0x01] Request parameter** Aftersales Assignment GLID	**Length, Data Type** [36 bytes, Hex]	
Response parameters	**Start [0x01]RoutineStatusRecords** Routine Type Routine Status Assignment Status	**Length, Data Type** [4 bits, Unsigned] [4 bits, Unsigned] [1 byte, Hex]	
Available in application		Available in bootloader	
MC		N	

665401 v2 **RID - 0240**

Version history:
(2):

Purpose: To start the digital envelope transfer.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	0240
Name	Digital Envelope Download For Aftersales

Description	The Digital Envelope Transfer routine with routine identifier 0x0240 shall be implemented. The routine is optional and shall be used for ECUs supporting file system storage and distributed download to receive digital envelope information. **Note:** The requirements of distributed download (i.e. downloading software part(s) through HTTPS protocol) is defined in ECU SWRS.		
Type	1		
$P4_{Server_Max}$	200ms		
Security access	Level 05/06		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	N
Request parameters	**Start [0x01] Request parameter**	**Length, Data Type**	
	Digital Envelope To FileRecordNumber	[1 byte, Unsigned]	
	File Encryption Type	[1 byte, Unsigned]	
	Digital Envelope Information Length	[2 bytes, Unsigned]	
	Digital Envelope Information Unsigned]	[The length of this parameter is defined by the Digital Envelope Information Length,	
Response parameters	**Start [0x01]RoutineStatusRecords**	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	
	Abort Reason	[1 byte, Hex]	

Available in application	Available in bootloader
MC	N

665402 v1 **RID - 0301**

Version history:
(1):

Purpose: All ECUs must support routines defined for SWDL.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	0301
Name	Activate Secondary Boot-loader

Description	The ActivateSBL routine with routine identifier 0x0301 shall be implemented. The routine shall be used to activate the Secondary Bootloader after it has been downloaded to volatile memory. The ECU shall start executing the secondary bootloader from the memory address defined in the data file containing the Secondary Bootloader. The final positive response message from the ActivateSBL routine shall be sent when the SBL has been activated, i.e. from the SBL. If the SBL is already activated at the time of the request the ECU shall respond with a positive response message with RoutineCompleted=0 (Routine is completed).		
Type	1		
$P4_{Server_Max}$	200ms		
Security access	Level 01/02		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	N
Request parameters	**Start [0x01] Request parameter**	**Length, Data Type**	
	memoryAdress (SBL, start adress byte 1, MSB)	[1 byte, Hex]	
	memoryAdress (SBL, start adress, byte 2)	[1 byte, Hex]	
	memoryAdress (SBL, start adress, byte 3)	[1 byte, Hex]	
	memoryAdress (SBL, start adress, byte 4, LSB)	[1 byte, Hex]	
Response parameters	**Start [0x01] RoutineStatusRecords**	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	

Available in application	Available in bootloader
N	MPS

706127 v2 **RID - 7027**

Version history:
(2): 更新Description一栏的描述，以及参数固化长度。

Purpose: Routine that Certificate Validity Check.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account, and exactly as defined in *Global Master Reference Database.*

RID [Hex]	7027
Name	Certificate Validity Check

Description	The Certificate Validity Check routine shall be able to: • Check whether the OCSP Information and the certificate (if carried, optional) are properly formatted. • Store the OCSP Information for further check in the process of authentication. • Check whether the OCSP Information matches the certificate (if carried), and check whether the certificate has been revoked (optional). Note 1: Mandatory for DoIP Node if UDS Service $29 is used. Note 2: If the certificate Validity Check is transmitted multiple times, the last received one shall prevail. Note 3: One Certificate Validity Check, one attempt to proceed the authentication process.
Type	1
$P4_{Server_Max}$	5000ms
Security access	-

Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	N

Request parameters	**Start [0x01] Request parameter**	**Length, Data Type**
	OCSP Length	[2 bytes, Unsigned]
	OCSP Response	[2048 bytes, Unsigned]
	Certificate Length	[2 bytes, Unsigned]
	Certificate	[1024 bytes, Unsigned]
	Note: Parameter "Length" represents the payload of the data, and the remaining insufficient bytes are filled with 0x00.	

Response parameters	**Start [0x01] RoutineStatusRecords**	**Length, Data Type**
	Routine Type	[4 bits, Unsigned]
	Routine Status	[4 bits, Unsigned]
	Result	[1 byte, Unsigned]

Available in application	Available in bootloader
MC	MC

665403 v2 **RID - A000**

Version history:
(2): Level 19/1A can also be used if ECU only supports Level 19/1A.

Purpose: Routine for manipulating register values.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	A000
Name	Substitue Value In Ethernet Register

Description	Routine for manipulating register values. The value of the register shall only temporarily be substituted by the routine. After a restart, register values shall return to the previous value stored in long term memory. Note：Ethernet node can be used based on internal requirements.		
Type	2		
$P4_{Server_Max}$	200ms		
Security access	Either **Level 05/06** or **Level 19/1A**		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	M	M
Request parameters	**Start [0x01] Request parameter**	**Length, Data Type**	
	Register	[1 byte, Unsigned]	
	Page	[1 byte, Unsigned]	
	Value	[2 bytes, Unsigned]	
Response parameters	**Start [0x01] RoutineStatusRecords**	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	
	Additional Result Info	[1 byte, Unsigned]	
	Stop [0x02] RoutineStatusRecords	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	
	Reserved	[1 byte, Unsigned]	
	Result [0x03] RoutineStatusRecords	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	
	Additional Result Info	[1 byte, Unsigned]	

Available in application	Available in bootloader
O	N

665404 v2 **RID - A001**

Version history:
(2): Level 19/1A can also be used if ECU only supports Level 19/1A.

Purpose: Routine that activates transmit droop test mode.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	A001
Name	Ethernet Test Mode 1

Description	Transmitter droop test mode. Valid only for BroadR-Reach/100BASE-T1. Note1：This is mandatory for Ethernet node. Note2：The test mode routine will disable all diagnostic communication on the Ethernet link, when started it is not possible to stop the routine using service RoutineControlType - stopRoutine. The routine shall stop either when the routine reached the time limit given in the test duration parameter in the request or when the ECU is resetted.		
Type	2		
$P4_{Server_Max}$	200ms		
Security access	Either **Level 05/06** or **Level 19/1A**		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	M
Request parameters	**Start [0x01] Request parameter**	**Length, Data Type**	
	Test Duration Setting	[2 bytes, Unsigned]	
	Test Parameter N	[2 bytes, Unsigned]	
Response parameters	**Start [0x01] RoutineStatusRecords**	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	
	Additional Result Info	[1 byte, Unsigned]	
	Result [0x03] RoutineStatusRecords	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	
	Additional Result Info	[1 byte, Unsigned]	

Available in application	Available in bootloader
MC	N

665405 v2 **RID - A002**

Version history:
(2): Level 19/1A can also be used if ECU only supports Level 19/1A.

Purpose: Routine that activates transmit jitter test in MASTER mode.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	A002
Name	Ethernet Test Mode 2

Description	Transmitter jitter test in MASTER mode. Note1：This is mandatory for Ethernet node. Note2：The test mode routine will disable all diagnostic communication on the Ethernet link, when started it is not possible to stop the routine using service RoutineControlType - stopRoutine. The routine shall stop either when the routine reached the time limit given in the test duration parameter in the request or when the ECU is resetted.		
Type	2		
$P4_{Server_Max}$	200ms		
Security access	Either **Level 05/06** or **Level 19/1A**		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	M
Request parameters	**Start [0x01] Request parameter**	**Length, Data Type**	
	Test Duration Setting	[2 bytes, Unsigned]	
Response parameters	**Start [0x01] RoutineStatusRecords**	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	
	Additional Result Info	[1 byte, Unsigned]	
	Result [0x03] RoutineStatusRecords	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	
	Additional Result Info	[1 byte, Unsigned]	

Available in application	Available in bootloader
MC	N

665406 v2 **RID - A003**

Version history:
(2): Level 19/1A can also be used if ECU only supports Level 19/1A.

Purpose: Routine that activates transmit jitter test in SLAVE mode
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	A003
Name	Ethernet Test Mode 3
Description	Transmit jitter test in SLAVE mode. Note1：This is mandatory for Ethernet node. Note2：The test mode routine will disable all diagnostic communication on the Ethernet link, when started it is not possible to stop the routine using service RoutineControlType - stopRoutine. The routine shall stop either when the routine reached the time limit given in the test duration parameter in the request or when the ECU is resetted.

Type	2		
$P4_{Server_Max}$	200ms		
Security access	Either **Level 05/06** or **Level 19/1A**		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	M
Request parameters	**Start [0x01] Request parameter** Test Duration Setting	**Length, Data Type** [2 bytes, Unsigned]	
Response parameters	**Start [0x01] RoutineStatusRecords** Routine Type Routine Status Additional Result Info **Result [0x03] RoutineStatusRecords** Routine Type Routine Status Additional Result Info	**Length, Data Type** [4 bits, Unsigned] [4 bits, Unsigned] [1 byte, Unsigned] **Length, Data Type** [4 bits, Unsigned] [4 bits, Unsigned] [1 byte, Unsigned]	

Available in application	Available in bootloader
MC	N

665407 v2 **RID - A004**

Version history:
(2): Level 19/1A can also be used if ECU only supports Level 19/1A.

Purpose: Routine that activates transmitter distortion test.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	A004
Name	Ethernet Test Mode 4
Description	Transmitter distortion test. Note1：This is mandatory for Ethernet node. Note2：The test mode routine will disable all diagnostic communication on the Ethernet link, when started it is not possible to stop the routine using service RoutineControlType - stopRoutine. The routine shall stop either when the routine reached the time limit given in the test duration parameter in the request or when the ECU is resetted.
Type	2
$P4_{Server_Max}$	200ms
Security access	Either **Level 05/06** or **Level 19/1A**

Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	M

Request parameters	**Start [0x01] Request parameter**	**Length, Data Type**
	Test Duration Setting	[2 bytes, Unsigned]
Response parameters	**Start [0x01] RoutineStatusRecords**	**Length, Data Type**
	Routine Type	[4 bits, Unsigned]
	Routine Status	[4 bits, Unsigned]
	Additional Result Info	[1 byte, Unsigned]
	Result [0x03] RoutineStatusRecords	**Length, Data Type**
	Routine Type	[4 bits, Unsigned]
	Routine Status	[4 bits, Unsigned]
	Additional Result Info	[1 byte, Unsigned]

Available in application	Available in bootloader
MC	N

665408 v2 **RID - A005**

Version history:
(2): Level 19/1A can also be used if ECU only supports Level 19/1A.

Purpose: Routine that activates normal operation at full power.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	A005
Name	Ethernet Test Mode 5
Description	Normal operation at full power (for the PSD mask). Note1：This is mandatory for Ethernet node. Note2：The test mode routine will disable all diagnostic communication on the Ethernet link, when started it is not possible to stop the routine using service RoutineControlType - stopRoutine. The routine shall stop either when the routine reached the time limit given in the test duration parameter in the request or when the ECU is resetted.
Type	2
$P4_{Server_Max}$	200ms
Security access	Either **Level 05/06** or **Level 19/1A**

Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	M

Request parameters	**Start [0x01] Request parameter** Test Duration Setting	**Length, Data Type** [2 bytes, Unsigned]
Response parameters	**Start [0x01] RoutineStatusRecords** Routine Type Routine Status Additional Result Info **Result [0x03] RoutineStatusRecords** Routine Type Routine Status Additional Result Info	**Length, Data Type** [4 bits, Unsigned] [4 bits, Unsigned] [1 byte, Unsigned] **Length, Data Type** [4 bits, Unsigned] [4 bits, Unsigned] [1 byte, Unsigned]

Available in application	Available in bootloader
MC	N

681784 v1 **RID - A040**

Version history:
(1):

Purpose: Routine that activates switch port mirror.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account, and exactly as defined in *Global Master Reference Database.*

RID [Hex]	A040		
Name	Switch Port Mirror Setting		
Description	For ethernet switch, to realize the switch port mirroring to specific port. E.g. The ethernet packets in internal IP network mirror to 100 Base-Tx port. Comment: Switch Port Mirror function depends on the hardware implementation of the Switch. Note1: This is mandatory for Edge Node. This is optional for other ethernet node with switch. Note2: Port mirror function will also exit when the mirrored port diagnostic connection is interrupted.		
Type	1		
$P4_{Server_Max}$	200ms		
Security access	-		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	N

Request parameters	**Start [0x01] Request parameter**	**Length, Data Type**
	Configure port mirroring for port 0~15	[2 bytes, Unsigned]
	Mirrored port configuration	[1 bytes, Unsigned]
	Configure port mirroring direction for selected ports	[4 bits, Unsigned]
	Multiple switches selection	[4 bits, Unsigned]
Response parameters	**Start [0x01] RoutineStatusRecords**	**Length, Data Type**
	Routine Type	[4 bits, Unsigned]
	Routine Status	[4 bits, Unsigned]

Available in application	Available in bootloader
MC	N

665394 v1 **RID - B000**

Version history:
(1):

Purpose: To have a possibility to create a predictable handling of diagnostic trouble code data in an ECU.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	B000
Name	Set DTC prepassed/prefailed
Description	This routine shall be able to set the prepassed/prefailed status of one of the Test DTCs in range U2800 – U280F. The routineControlOptionRecord "DTC Number" contains the identifier of the DTC that shall be affected by this control routine. This can only be one of the dedicated DTC Data Verification DTCs. When the routineControlOptionRecord "Set DTC prepassed/prefailed" is set to "Passed" then the routine shall report "Test sample passed" for the DTC Data Verification DTC specified in the "DTC Number" routineControlOptionRecord. When the routineControlOptionRecord "Set DTC prepassed/prefailed" is set to "Failed" then the routine shall report "Test sample failed" for the DTC Data Verification DTC specified in the "DTC Number" routineControlOptionRecord. The routineControlOptionRecord "Repeat Counter" defines the repeat counter for the "Set DTC prepassed/prefailed" routineControlOptionRecord. A repeat count of 0 means that the "Test sample passed" or "Test sample failed" is reported only once. Note:The control DTC parameter please refer to requirement of "DTC Data Verification DTCs".
Type	1
$P4_{Server_Max}$	500ms

Security access	Level 05/06		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	N
Request parameters	**Start [0x01] Request parameter**	**Length, Data Type**	
	DTC number	[3 bytes, Hex]	
	Set DTC prepassed/prefailed	[1 byte, Hex]	
	Repeat Counter	[1 byte, Unsigned]	
Response parameters	**Start [0x01]RoutineStatusRecords**	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	

Available in application	Available in bootloader
O	N

665395 v2 **RID - B001**

Version history:
(2): Level 19/1A can also be used if ECU only supports Level 19/1A.

Purpose: To have a possibility to create a predictable handling of diagnostic trouble code data in an ECU.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	B001		
Name	Start New Operation Cycle		
Description	The routine shall be able to: 1. End the current operation cycle: the ECU shall perform all actions it performs when ending an operation cycle (saving data to non-volatile memory, etc). 2. Start a new operation cycle: the ECU shall perform all actions it performs when starting an operation cycle. Note:It shall be possible to execute it in the extended diagnostic session only.		
Type	1		
$P4_{Server_Max}$	5000ms		
Security access	Either **Level 05/06** or **Level 19/1A**		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	N

Request parameters	**Start [0x01] Request parameter**	**Length, Data Type**
	DTC number	[3 bytes, Hex]
	Set DTC prepassed/prefailed	[1 byte, Hex]
	Repeat Counter	[1 byte, Unsigned]
Response parameters	**Start [0x01]RoutineStatusRecords**	**Length, Data Type**
	Routine Type	[4 bits, Unsigned]
	Routine Status	[4 bits, Unsigned]

Available in application	Available in bootloader
O	N

665409 v4 **RID - FF00**

Version history:
(4): 增加Note3，带文件操作系统的ECU，不要求支持擦除例程指令。

Purpose: All ECUs must support routines defined for SWDL.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	FF00
Name	Flash Erase

Description	The routine shall perform a non-volatile memory erase of the memory blocks defined by the memoryAddress and memorySize requestNparameters. The routine does not to be supported if the ECU has file system data storage and implements an alternative method e.g RequestFileTransfer – modeOfOperation parameter DeleteFile or ReplaceFile. If the ECU is delivered to OEM with the complete programmable memory already erased, the routine shall include an "already erased" detection in order to reduce the overall software download time. If the complete programmable memory is "already erased" and all other preconditions are met (e.g. SBL is downloaded and activated), the routine shall not perform any erase operation and immediately send a final positive response message with RoutineCompleted=0 (Routine is completed). Once one or more bytes of the ECU programmable memory area has been programmed, the "already erased" status shall be invalidated. Any erase memory requests when the "already erased" status is invalid shall erase the requested memory area. Note1: The response time P4 server_max for the Erase Memory routine shall be increased by an additional 20s for each megabyte range of data over 1 megabyte that is requested to be erased. Note2: The complete programmable memory will be erased when the ECU Software Structure, ESS, is updated. The Erase Memory routine needs longer time.The P4 server_max for an ESS update shall be defined by the size of the total memory, e.g. the sum of all erase requests for all programmable data files. **Note3**: The ECU is having a file based memory structure, **not** requiring an explicit erase memory request prior to request download. This means that this type of ECU does not need to support this Routine identifier.
Type	1
$P4_{Server_Max}$	60000ms
Security access	Level 01/02

Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	N

Request parameters	**Start [0x01] Request parameter**	**Length, Data Type**
	memoryAdress (Erase start adress, byte1, MSB)	[1 byte, Hex]
	memoryAdress (Erase start adress, byte 2)	[1 byte, Hex]
	memoryAdress (Erase start adress, byte 3)	[1 byte, Hex]
	memoryAdress (Erase start adress, byte 4, LSB)	[1 byte, Hex]
	memorySize (Erase length, byte 1, MSB)	[1 byte, Hex]
	memorySize (Erase length, byte 2)	[1 byte, Hex]
	memorySize (Erase length, byte 3)	[1 byte, Hex]
	memorySize (Erase length, byte 4, LSB)	[1 byte, Hex]

Response parameters	**Start [0x01]RoutineStatusRecords**	**Length, Data Type**
	Routine Type	[4 bits, Unsigned]
	Routine Status	[4 bits, Unsigned]

Available in application	Available in bootloader
MC[1]	MS

Note 1: ECU which implement seamless update shall support this RID in the DiagnosticSessionControl (0x10) service with diagnosticSessionType equal to UpdateSession (0x40/0xC0).

•**DTC data**

4.10.3.2.3 DTC data

666776 v3 **DTC - D14B51**

Version history:
(3):

Purpose: To ensure that it can be detected if the security constant for level 01/02 has never been written in ECU.
Source: Table "Requirements on detection of faults caused by production SW download data not programmed" - item 9.1
Verification Method: Analysis AND Test
Legacy ID: 661758,661759,661760,661761,661762,661763,661764,661765,661766,661767,661768

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	0xD14B	Failure type (Low Byte)	0x51
Description	Production SW Download Security Constant System Programming Failures Not programmed **Note:** Mandatory for ECU supporting SWDL.		
Test run criteria (TRC)	**TRC:** The DTC test for detection of no production security constant shall run at least once every operation cycle.		
Detection Criteria	If the ECU is required to be programmed production security constant for security level 01/02, the ECU shall be able to detect if it has be written in protected memory area successfully. For test sample failed: No Production Security Constant for security level 01/02 is stored in the protected memory. For test sample passed: Production Security Constant for security level 01/02 is stored in the protected memory.		

DTC CALIBRATION				
comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
1	1	OCC2	No	N/A
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	127	Yes	127	128

666781 v3 **DTC - D14C51**

Version history:
(3):

Purpose: To ensure that it can be detected if the public key has never been written in ECU.
Source: Table "Requirements on detection of faults caused by production SW download data not programmed" - item 10.1
Verification Method: Analysis AND Test
Legacy ID: 661769,661770,661771,661772,661773,661774,661775,661776,661777,661778,661779

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	0xD14C	Failure type (Low Byte)	0x51
Description	Production SW Download Signature Public Key System Programming Failures Not programmed **Note:** Mandatory for ECU supporting SWDL and software authentication.		
Test run criteria (TRC)	**TRC1:** The DTC test for detection of no production public key shall run at least once every operation cycle.		
Detection Criteria	If the ECU is required to be programmed production public key for software verification, the ECU shall be able to detect if it has be written in protected memory area successfully. For test sample failed: No production public key is stored in the protected memory. For test sample passed: Production public key is stored in the protected memory.		

DTC CALIBRATION				
comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
1	1	OCC2	No	N/A
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	127	Yes	127	128

1074518 v1 **DTC - D63451**

Version history:

Purpose: To ensure that it can be detected if the security constant for level 19/1A has never been written in ECU.
Source: Table "Requirements on detection of faults caused by production SW download data not programmed" - item 9.2
Verification Method: Analysis AND Test
Legacy ID:

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	0xD634	Failure type (Low Byte)	0x51
Description	Production SW Cybersecurity Security Constants System Programming Failures Not programmed **Note:** Mandatory for ECU supporting security level 19/1A.		
Test run criteria (TRC)	**TRC:** The DTC test for detection of no production security constant shall run at least once every operation cycle.		

Detection Criteria	If the ECU is required to be programmed production security constant for security level 19/1A, the ECU shall be able to detect if it has be written in protected memory area successfully. For test sample failed: No Production Security Constant for security level 19/1A is stored in the protected memory. For test sample passed: Production Security Constant for security level 19/1A is stored in the protected memory.

DTC CALIBRATION				
comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
1	1	OCC2	No	N/A
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	127	Yes	127	128

666763 v2 DTC - Detection of incompatible VIN

Version history:
(2):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults caused by incompatible or missing software components" - item 7.1
Verification Method: Analysis AND Test
Legacy ID: 70838v1,70839v0,70840v1

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	0x0630 (ECM/PCM) 0x0631 (TCM) 0x4545 (BSCM) 0x4546	Failure type (Low Byte)	0x00
Description	VIN Not Programmed or Incompatible **Note:** Mandatory where the fault is detected by a fault monitor as part of the ECU functional requirements.		
Test run criteria (TRC)	**TRC1:** The DTC test for detection of incompatible VIN shall run at least once every operation cycle. **TRC2:** The TRC shall be according to the General Test Run Criteria specified in *"Requirements from section Detection of faults"*.		
Detection Criteria	The ECU shall be able to detect if the locally stored VIN does not match the VIN transmitted from another ECU, this shall also be used as a DTC test.		

DTC CALIBRATION				
comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
Note 1				
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
Note 1				

Notes in the table above:

1. The DTC Calibration that the ECU supports are specified by the implementer.

666761 v3 **DTC - E10156**

Version history:
(3):

Purpose: To ensure that it can be detected if an ECU has invalid local configuration data.
Source: Table "Requirements on detection of faults caused by local ECU Configuration (local parameters)" - item 1.1
Verification Method: Analysis AND Test
Legacy ID: 69701v0,69702v0,69703v0,69704v0

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	0xE101	Failure type (Low Byte)	0x56
Description	Control Module Configuration Incompatible System Programming Failures Invalid / incompatible configuration **Note:** Mandatory if the ECU is required to be programmed with local configuration (calibration) data (e.g. VBF local configuration, or calibration data through diagnostic service) for correct operation.		
Test run criteria (TRC)	**TRC1:** If the detection of invalid local configuration data is implemented the DTC test shall run whenever a local configuration parameter is used or changed. **TRC2:** The TRC shall be according to the General Test Run Criteria specified in *"Requirements from section Detection of faults"*.		
Detection Criteria	The ECU shall be able to detect if it has one or more required local configuration parameters with unknown or invalid value comparing with allowed range of values for the item. Refer to *[Data_8] Overview of vehicle configuration & customer* settings for details regarding local configuration.		

DTC CALIBRATION				
comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
1	1	OCC2 or OCC5	No	N/A
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	127	Yes	127	128

667763 v2 **DTC - E30051**

Version history:
(2): 增加TRC条件。

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 2.4
Verification Method: Analysis AND Test
Legacy ID: 661990

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	0xE300	Failure type (Low Byte)	0x51
Description	Central Configuration System Programming Failures Not programmed **Note:** Mandatory if the ECU requires an additional control routine to instruct the ECU to learn a change configuration. (Security ECU, i.e. SRS)		
Test run criteria (TRC)	**TRC1:** The TRC shall be according to the General Test Run Criteria specified in *"Requirements from section Detection of faults"*. However, the inhibit criteria "During 5 seconds from any transition from one usage mode to another" and "During 5 seconds from any car mode from one car mode to another according to the Generic Test Run Criteria is not applicable for this DTC. **TRC2:** The DTC test for new configuration not learned shall run every time a global configuration parameter used for configuration of the ECU is received, until the test has passed. **TRC3:** Within 30 seconds after the usage mode transitions from below "Active" usage mode to "Active" or "Driving".		
Detection Criteria	If the ECU requires an additional control routine to instruct the ECU to learn a changed configuration, the ECU shall be able to detect that a new valid configuration has been received but not learned, according to *[Data_5] Central Car Configuration*. For test sample failed: A changed configuration has been evaluated as valid and the ECU requires an additional control routine to instruct the ECU to learn a changed configuration. For test sample passed: The routine required to instruct the ECU to learn a changed configuration has been successfully executed.		

DTC CALIBRATION				
comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
1	1	OCC2	No	N/A
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	127	Yes	127	128

667692 v2 **DTC - E30055**

Version history:
(2): 增加一条TRC条件：从低于Active切换到Active或Driving后的30s内。

Purpose: To ensure that it can be detected if an ECU has never received the required VCP data.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 2.1; Central Car Configuration - SPA Req18, UDS - Data - Requisite documents
Verification Method: Analysis AND Test
Legacy ID: 661970

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	0xE300	Failure type (Low Byte)	0x55
Description	Central Configuration System Programming Failures Not configured **Note:** Mandatory for ECU that is a Car Configuration Subscriber according to *[Data_5] Central Car Configuration*.		
Test run criteria (TRC)	**TRC1:** The TRC shall be according to the General Test Run Criteria specified in *"Requirements from section Detection of faults"*. However, the inhibit criteria "During 5 seconds from any transition from one usage mode to another" and "During 5 seconds from any car mode from one car mode to another according to the Generic Test Run Criteria is not applicable for this DTC. **TRC2:** The DTC test for detection of missing vehicle configuration data shall be enabled while ECU is in 'Bulk state' and shall run each time a Car Configuration block is received. The test shall be inhibited once it has passed. **TRC3:** Within 30 seconds after the usage mode transitions from below "Active" usage mode to "Active" or "Driving".		
Detection Criteria	The ECU is in 'Bulk state' it shall be able to detect missing vehicle configuration data, according to *[Data_5] Central Car Configuration*. For test sample failed: No valid configuration (i.e. any required parameter missing or with value 0x00 or 0xFF) has been stored in VCP memory. For test sample passed: A valid configuration (all required parameters have valid values) has been stored in VCP memory.		

DTC CALIBRATION				
comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
1	1	OCC2	No	N/A
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	127	Yes	127	128

667698 v2 **DTC - E30056**

Version history:
(2): 增加TRC条件。

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 2.3; Central Car Configuration - SPA, UDS - Data - Requisite documents
Verification Method: Analysis AND Test
Legacy ID: 661980

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	0xE300	Failure type (Low Byte)	0x56
Description	Central Configuration System Programming Failures Invalid / incompatible configuration **Note:** Mandatory for ECU that is a Car Configuration Subscriber according to *[Data_5] Central Car Configuration*.		

Test run criteria (TRC)	**TRC1:** The TRC shall be according to the General Test Run Criteria specified in *"Requirements from section Detection of faults"*. However, the inhibit criteria "During 5 seconds from any transition from one usage mode to another" and "During 5 seconds from any car mode from one car mode to another according to the Generic Test Run Criteria is not applicable for this DTC. **TRC2:** The DTC test for detection of invalid configuration data shall run every time a global configuration parameter used for configuration of the ECU is received, until the test has passed. **TRC3:** Within 30 seconds after the usage mode transitions from below "Active" usage mode to "Active" or "Driving".
Detection Criteria	The ECU shall be able to detect invalid vehicle configuration data, according to *[Data_5] Central Car Configuration*. For test sample failed: One or more parameters used for configuration of the ECU is received with an unrecognized or invalid value (included values 0x00 and 0xFF). For test sample passed: All parameter used for configuration of the ECU have been received with valid values.

DTC CALIBRATION				
comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
1	1	OCC2	No	N/A
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	127	Yes	127	128

667780 v2 **DTC - E30057**

Version history:
(2): 增加TRC条件。

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 2.5
Verification Method: Analysis AND Test
Legacy ID: 662000

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	0xE300	Failure type (Low Byte)	0x57
Description	Central Configuration System Programming Failures Invalid / Incompatible Software Component **Note:** Mandatory where the fault is detected by a fault monitor as part of the ECU functional requirements.		

Test run criteria (TRC)	**TRC1:** The TRC shall be according to the General Test Run Criteria specified in *"Requirements from section Detection of faults"*. **TRC2:** The DTC test for detection of inconsistent or incompatible configuration shall run every time a global configuration parameter used for configuration of the ECU is received until test the test has passed. **TRC3:** Within 30 seconds after the usage mode transitions from below "Active" usage mode to "Active" or "Driving".
Detection Criteria	If the ECU has a fault monitor implemented as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) to detect if any loaded configuration parameter has a valid value but is internally inconsistent or incompatible with some other data or configuration in the ECU or an ECU hardware option, this test shall also be used as a DTC test. For test sample failed: One or more parameters used for configuration of the ECU has a valid value but is internally inconsistent or incompatible with some other data or configuration in the ECU or an ECU hardware option. For test sample passed: All parameters used for configuration of the ECU have valid values and are internally consistent and compatible with all other data, configuration and hardware options in the ECU.

DTC CALIBRATION				
comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
1	1	OCC2	No	N/A
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	127	Yes	127	128

682163 v2 **DTC - E30141**

Version history:
(2): Update the Verification Method to Test.

Purpose:
Source:
Verification Method: Test
Legacy ID:

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	0xE301	Failure type (Low Byte)	0x41
Description	Configuration Data Storage System Internal Failures General checksum failure **Note:** Mandatory for ECU that is the Car Configuration Domain Master		
Test run criteria (TRC)	**TRC1:** The TRC shall be according to the General Test Run Criteria specified in "Requirements from section Detection of faults". **TRC2:** The DTC test shall run at least once every operation cycle.		

Detection Criteria	If the ECU is Car Configuration Domain Master, it should be able to detect the validity of DID F106. For test sample failed: No CCP/VCP data is stored or CCP/VCP checksum is error. For test sample passed: CCP/VCP data stored and CCP/VCP checksum is correct.

DTC CALIBRATION

comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
1	1	OCC2	No	NA
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	127	Yes	127	128

682161 v1 **DTC - E30355**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source:
Verification Method: Test
Legacy ID:

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database*.

DTC Code (High Byte and Middle Byte)	0xE303	Failure type (Low Byte)	0x55
Description	Vehicle Configuration Data System Programming Failures Not configured. **Note:** Mandatory for ECU that is the Car Configuration Domain Master.		
Test run criteria (TRC)	**TRC1:** The TRC shall be according to the General Test Run Criteria specified in "Requirements from section Detection of faults". **TRC2:** The DTC test shall run at least once every operation cycle.		
Detection Criteria	If the ECU is Car Configuration Domain Master, it should be able to detect the validity of DID C011. For test sample failed: No VDN data is stored or VDN checksum is error. For test sample passed: VDN data stored and VDN checksum is correct.		

DTC CALIBRATION

comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
1	1	OCC2	No	NA
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	127	Yes	127	128

666797 v3 **DTC - EE0368**

Version history:
(3):

Purpose: To be able to handle the situation where there may be a customer complaint and a symptom associated with the ECU but the root cause is that the ECU is operating outside of its design specification.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 3.1
Verification Method: Analysis AND Test
Legacy ID: 661815,661816,661817,661818,661819,661820,661821,661822,661823,661824

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	0xEE03	Failure type (Low Byte)	0x68
Description	ECU Supply Voltage Too High Algorithm Based Failure Event Information **Note:** Mandatory where the fault (i.e. supply voltage is too high for full operation so that not correct, reduced or no system functionality is available) is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information), each such event shall also be identified by an appropriate DTC. Otherwise, a feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM.		
Test run criteria (TRC)	The TRC to perform the fault check shall be according to the General Test Run Criteria specified in "*Requirements from section Detection of faults*"		
Detection Criteria	ECU supply voltage is too high for full operation. ECU has symptom of not correct, reduced or no function for a period of time which the customer would notice. For test sample failed: ECU supply voltage is too high for full operation at a time. For test sample passed: ECU supply voltage is normal supply voltage for full operation at a time. **Note**: For ECU as VMM Master, this fault shoud be detected when the supply voltage is 16V.		

DTC CALIBRATION				
comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
3	255	OCC2	Yes or No	100ms or faster
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	6	Yes	3	6

666798 v4 **DTC - EE0468**

Version history:
(4): 修改TRC1条件，低电压DTC需要满足通用TRC套件，除低电压抑制外。

Purpose: To be able to handle the situation where there may be a customer complaint and a symptom associated with the ECU but the root cause is that the ECU is operating outside of its design specification.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 3.2
Verification Method: Analysis AND Test
Legacy ID: 661825,661826,661827,661828,661829,661830,661831,661832,661833,661834

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	0xEE04	Failure type (Low Byte)	0x68
Description	ECU Supply Voltage Too Low Algorithm Based Failure Event Information **Note:** Mandatory where the fault (i.e. supply voltage is too low for full operation so that not correct, reduced or no system functionality is available) is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information), each such event shall also be identified by an appropriate DTC. Otherwise, a feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM.		
Test run criteria (TRC)	**TRC1:** The TRC to perform the fault check shall be according to the General Test Run Criteria specified in "*Requirements from section Detection of faults",* except Generic Test Run Criteria - low voltage inhibit criteria. **TRC2:** Not when ECU supply voltage check with the vehicle battery voltage has detected a problem (See ReqID 664181 "*DTC - F00362*")		
Detection Criteria	ECU supply voltage is too low for full operation. ECU has symptom of not correct, reduced or no function for a period of time which the customer would notice. For test sample failed: ECU supply voltage is too low for full operation at a time. For test sample passed: ECU supply voltage is normal supply voltage for full operation at a time. **Note**: For ECU as VMM Master, this fault shoud be detected when the supply voltage is 9V.		

DTC CALIBRATION				
comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
3	255	OCC2	Yes or No	100ms or faster
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	6	Yes	3	6

664181 v5 **DTC - F00362**

Version history:
(5): Add TRC.

Purpose: Low supply voltage at the ECU. The ECU checks if its supply voltage is lower than the measured vehicle battery voltage received from the network.

Source:

Verification Method: Test

Legacy ID:

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	0xF003	Failure type (Low Byte)	0x62

Description	Supply Voltage Algorithm Based Failures Signal compare failure **Note:** In each ECU supporting DTC U3003-62, the internal ECU supply voltage, Uecu, shall be measured and compared with the vehicle battery voltage, Ubat, received from the network. A robust strategy for measurement and filtering of the voltages shall be implemented. The actual method shall be designed by the implementer and shall comply with the following requirements.
Test run criteria (TRC)	**TRC1:** The TRC shall be according to the General Test Run Criteria specified in *"Requirements from section Detection of faults*". **TRC2:** Low supply voltage fault detection shall only be done when all of following conditions are fulfilled: • (Ubat ≥ 12 V) AND (UsageMode == Driving) • Received and process valid Ubat value at least once
Detection Criteria	The calculated Uecu value shall be compared with the received Ubat value from bus, • For test sample failed: Uecu < (Ubat - 3V) is detected. • For test sample passed: Uecu >= (Ubat - 3V) is detected. **Note1: Vehicle battery voltage:** The vehicle battery voltage Ubat shall be measured, by a dedicated ECU, as close to the actual battery terminals as possible (such that the measurement is unaffected by any internal energy storage in the measuring ECU). The Ubat shall be measured with a minimum accuracy of ±0.25 V and shall be sampled every 50 ms or faster. A new Ubat value shall be calculated every time a sample is taken. It shall be calculated as the average value of the samples taken during the last 500 ms period. The calculated Ubat values shall be sent periodically and received by all Public ECUs with maximum end-to-end time (including sensing, signaling and actuating time) of 100 ms. **Note2: ECU supply voltage:** The ECU supply voltage Uecu shall be measured with a minimum accuracy of ±0.25V and shall be sampled every 50 ms or faster. A new Uecu value shall be calculated every time a sample is taken. It shall be calculated as the average value of the samples taken during the last 500 ms period.

DTC CALIBRATION				
comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
3	255	OCC2 or OCC5	Yes or No	Less than 50ms
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	6	Yes	3	6

4.10.3.2.3.1 DTC of faults in external electrical circuits for CAN bus

666521 v1 **DTC - Common CAN electrical fault**

Version history:
(1):

Purpose: Defining what the cause of the electrical fault may help in the fault tracing process. It is enough if the ECU closest to the tester checks electrical faults on the bus since if the fault is present, it will only be the ECU closest to the tester that will be able to report the status.

Source:
Verification Method: Test
Legacy ID:

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	GMRDB	Failure type (Low Byte)	0x00
Description	Common CAN electrical fault Note: Mandatory if the ECU is the missing frame master. Means that if the ECU is connected to more then one public network and the ECU is the ECU transferring diagnostic messages in between the two networks, the ECU shall support detecting Common CAN electrical fault		
Test run criteria (TRC)	The TRC to perform this fault check shall be according to the general Test Run Criteria specified in "*Requirements from section Detection of faults*".		
Detection Criteria	When the ECU detects that at least one of the physical bus error flags is set.		

DTC CALIBRATION				
comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
3	255	OCC2 or OCC5	Yes or No	at least every 100 ms
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	5	Yes	3	2

666478 v5 DTC - Control module communication bus off

Version history:
(5): The word 3.5 has been revised to 4.0.

Purpose: The bus-off state means there has been a fault in sending or receiving CAN frames and usually this will make customer observable symptoms and hence a DTC shall be set showing that the network has been in bus-off state.

Source:
Verification Method: Test
Legacy ID:

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	GMRDB	Failure type (Low Byte)	0x00 (the ECU is emission) 0x88 (the ECU is not emission related)
Description	Control module communication bus off Note: Mandatory that all Public CAN ECUs on the network shall be able to detect and set bus-off DTCs when the CAN controller has gotten into the state bus-off.		

Test run criteria (TRC)	**TRC1:** The TRC to perform the bus-off check shall be according to the General Test Run Criteria specified in "*Requirements from section Detection of faults*". However, the inhibit criteria "during 5 seconds from any transition from one usage mode to another" and "During 5 seconds from the car mode of factory, transport and crash transmition to dyno,normal" according to the Generic Test Run Criteria is not applicable for this DTC. **TRC2**: The detection shall be within specific voltage range defined in *ReqID 660853 Operating voltage range hysteresis.*
Detection Criteria	**For all Autosar versions:** **For GEEA3.0 Architecture:** There shall be **32** detections of CAN bus off in CANSM until a DTC for CAN bus off is set to testFailed. **For EEA4.0 Architecture:** There shall be **10** detections of CAN bus off in CANSM until a DTC for CAN bus off is set to testFailed. Then a diagnostic trouble code (DTC) for CAN bus off shall be set within 100 ms. Depending on Autosar version, a fault detection counter is implemented in CANSM or DEM. The number of CAN bus off events required until a DTC is set to testFailed shall be same regardless of Autosar version. **Specific for Autosar 4.0.2:** If the bus off counter in CANSM exceeds CanSMBorCounterL2Err, and is defined in ReqID 663093, a diagnostic trouble code (DTC) shall be set to testFailed (FDC=+127). **Specific for Autosar 4.0.3 and later:** where no bus off counter exist in CANSM but a fault detection counter FDC10 in DEM shall acts as a prescaler until DTC is set: The fault detection counter FDC10 in DEM shall be increased by (127/ (CanSMBorCounterL2Err+1)) at every test call from CANSM to DEM with EventStatus := DEM_EVENT_STATUS_PRE_FAILED. This results in that 32 consecutive calls are required to make the FDC10 counter reach +127 and to set DTC to testFailed.

DTC CALIBRATION for GEEA3.0 Architecture

comfirmedDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
Note 1	255	OCC2 or OCC5	Yes or No	10
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	4	Yes	4	128

DTC CALIBRATION for EEA4.0 Architecture

comfirmedDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
Note 1	255	OCC2 or OCC5	Yes or No	100
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	13	Yes	13	128

Notes in the table above:
1. Parameter confirmedDTCLimit of DTC Calibration is usually configured as 1. The Emission ECU can configure parameter confirmedDTCLimit according to legislation regulation, e.g. may be different from 1.

666523 v2 DTC - Permanent missing frame

Version history:
(2):

Purpose: Permanent missing frame DTCs shall be used since it may in some cases help pinpointing the location of a network problem as well as proving a rough functional test on operability of an ECU.

Source:

Verification Method: Test
Legacy ID:

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	GMRDB	Failure type (Low Byte)	0x87
Description	Permanent missing frame **Note1:** It is mandatory for missing frame masters to detect missing frames and set permanent missing frame DTCs on CAN Bus. **Note2:** The missing frame master shall only monitor one periodically transmitted frame from each missing frame slave within the supervised network. **Note3:** Each missing frame slave shall periodically send at least one frame with a shorter periodicity than 100 ms. To save bandwidth on the CAN net, first check if there is an existing frame the gateway can perform the DTC test on. Secondly the send frequency of an existing frame shall be increased to match the 100 ms requirement.		
Test run criteria (TRC)	**TRC1:** The missing frame master, and not the missing frame slaves, shall manage all the enabling and disable conditions (i.e. set TRC to true or false) for the missing frame DTCs, both for permanent and intermittent missing frame. **TRC2:** The missing frame detection shall only be based on periodic frames sent in at least the usage mode "Driving". **TRC3:** All parameters used as test run criteria for missing frame tests shall be received outside the network that is being tested. **TRC4:** The testing for missing frames shall only be done when the frame is expected; i.e. missing frame master shall be configured such that it is aware if an optional missing frame slave is present (mounted or not) on the bus or not. **TRC5:** The missing frame master shall be aware of different operation modes preventing the missing frame slaves to send frames. For example if a missing frame slave is in power-off, low-power, sleep or equivalent mode, the missing frame slave might be prevented from sending frames. The missing frame master shall be aware when the missing frame slave is prevented from sending frames and shall not set any missing frame DTCs during this these modes. **TRC6:** Missing frame tests shall only be done when the ECU is the usage mode "Driving" **TRC7:** Missing frame tests shall never be done during combustion engine crank. **TRC8:** The missing frame master shall never set TRC true until 5±2 seconds after all criteria for a DTC test, for permanent and intermittent missing frame DTCs, to run, is satisfied. This inhibit time period include the inhibit time period, that starts after transition from one usage to another or one car mode to another, specified as Generic Test Run Criteria in "*Requirements from section Detection of faults*". **TRC9:** The voltage of the vehicle battery Ubat, shall be within: 10,0 ≤ Ubat ≤ 15,4V. Note: The battery voltage shall be measured as close to the battery as possible in order to minimize the measuring fault. **TRC10:** If a missing frame master does not receive the missing frame TRC no missing frame DTCs shall be set. **TRC11:** Missing frame tests shall never be done when fault is detected on the parameters used as test run criteria (e.g., when the value of parameters, used as test run criteria, distributed on an in-vehicle network is not updated (indicated by an update bit) or not correct (indicated by a quality factor)) and the parameter value adopted by ECU software is not met test run criteria after this fault.		

Detection Criteria	When the missing frame master have not received the periodic missing frame slave frame, for that specific ECU, in the test period time.			
DTC CALIBRATION				
comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
3	255	OCC2 or OCC5	Yes or No	150 ms ~ 170 ms
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	18	Yes	9	9

4.10.3.2.3.2 DTC of LIN related circuit faults for LIN Master

666691 v5 **DTC - Faulty LIN physical bus**

Version history:
(5): Verification Method: Analysis AND Test change to Test.

Purpose: Detect electrical faults on LIN.
Source:
Verification Method: Test
Legacy ID:

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	GMRDB	Failure type (Low Byte)	0x01	
Description	Faulty LIN physical bus **Note1:** Mandatory for LIN master. **Note2:** LIN physical bus failure detection shall have one base DTC for each LIN Network.			
Test run criteria (TRC)	**TRC1:** The TRC to perform this fault check shall be according to the general Test Run Criteria specified in "*Requirements from section Detection of faults*". **TRC2:** The LIN master shall perform Faulty LIN physical bus DTC test on every transmitted LIN frame. **TRC3:** The voltage of the vehicle battery Ubat, shall be within: 10,0 ≤ Ubat ≤ 15,4V.			
Detection Criteria	When LIN master detects LIN physical bus short to ground or battery voltage.			
DTC CALIBRATION				
comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
3	255	OCC2 or OCC5	Yes or No	100
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	5	Yes	3	2

666683 v5 **DTC - Faulty LIN response**

Version history:
(5): Reduce the sensitivity of confirming faults, adjust ConfirmedDTLimit from 1 to 3. Verification Method: Analysis AND Test change to Test. Update unconfirmedDTCLimit to 26.

Purpose: Detect LIN slave response disturbances.
Source:
Verification Method: Test
Legacy ID:

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	GMRDB	Failure type (Low Byte)	0x83
Description	Fault LIN response **Note1:** Mandatory for LIN master. **Note2:** LIN network failure detection shall have one base DTC for each LIN slave and one base DTC for the LIN master.		
Test run criteria (TRC)	**TRC1:** The TRC to perform this fault check shall be according to the general Test Run Criteria specified in "*Requirements from section Detection of faults*". **TRC2:** The LIN master shall perform a Faulty LIN response DTC test on every received LIN frame. **TRC3:** This tests shall only be done when the ECU is the usage mode "Driving" . **TRC4:** The voltage of the vehicle battery Ubat, shall be within: 10,0 ≤ Ubat ≤ 15,4V.		
Detection Criteria	When a checksum in any LIN frame from one specific LIN slave differs from expected value.		

DTC CALIBRATION				
comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
3	255	OCC2 or OCC5	Yes or No	100
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	26	Yes	13	13

.

666664 v5 DTC - Faulty transmitted data

Version history:
(5): Verification Method: Analysis AND Test change to Test.

Purpose: Detect network disturbances or electrical faults on LIN.
Source:
Verification Method: Test
Legacy ID:

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	GMRDB	Failure type (Low Byte)	0x86

Description	Fault transmitted data **Note1:** Mandatory for LIN master. **Note2:** LIN network failure detection shall have one base DTC for each LIN slave and one base DTC for the LIN master.
Test run criteria (TRC)	**TRC1:** The TRC to perform this fault check shall be according to the general Test Run Criteria specified in "*Requirements from section Detection of faults*". **TRC2:** The LIN master shall perform a Faulty transmitted DTC test on every transmitted LIN frame. **TRC3:** This tests shall only be done when the ECU is the usage mode "Driving" . **TRC4:** The voltage of the vehicle battery Ubat, shall be within: 10,0 ≤ Ubat ≤ 15,4V.
Detection Criteria	When transmitted data by LIN master does not match the data being read back. **Note1:** Data sent by the LIN master (either header or response from LIN master) is faulty.

DTC CALIBRATION				
comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
3	255	OCC2 or OCC5	Yes or No	100
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	20	Yes	10	20

666686 v6 DTC - Missing LIN response

Version history:
(6): Verification Method: Analysis AND Test change to Test.

Purpose: Detect electrical problems or missing LIN slaves. LIN slave diagnose response messages shall not be monitored since the LIN slaves may skip some frame requests.

Source:

Verification Method: Test

Legacy ID:

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	GMRDB	Failure type (Low Byte)	0x87
Description	Missing LIN response **Note1:** Mandatory for LIN master. **Note2:** LIN network failure detection shall have one base DTC for each LIN slave and one base DTC for the LIN master.		
Test run criteria (TRC)	**TRC1:** The TRC to perform this fault check shall be according to the general Test Run Criteria specified in "*Requirements from section Detection of faults*". **TRC2:** The LIN master shall perform a Faulty LIN response DTC test on every received LIN frame. **TRC3:** The missing frame detection shall only be based on periodic frames sent in at least the usage mode "Driving". **TRC4:** The voltage of the vehicle battery Ubat, shall be within: 10,0 ≤ Ubat ≤ 15,4V.		

Detection Criteria	When the master expects to receive but does not receive any LIN data at all. **Note1:** Diagnose response frames (LIN frame identifier 0x3D) shall not be monitored.

DTC CALIBRATION				
comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
3	255	OCC2 or OCC5	Yes or No	100
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	18	Yes	9	9

666805 v2 **DTC - Other LIN network faults**

Version history:
(2):

Purpose: Support extended DTC coverage.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	GMRDB	Failure type (Low Byte)	Defined by implementer
Description	Other LIN network faults **Note1:** Mandatory where the fault is detected by a fault monitor as part of the ECU functional requirements.		
Test run criteria (TRC)	The TRC to perform this fault check shall be defined by implementer.		
Detection Criteria	Other faults identified by the implementer.		

DTC CALIBRATION				
comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
Note 1				
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
Note 1				

Notes in the table above:
1. The DTC Calibration that the ECU supports are specified by the implementer.

4.10.3.2.3.3 DTC of Ethernet link failure

The IP Router/switch ECU shall discover when there is no Ethernet link to the IP ECU. This is done by polling if Link is up or down in control register. The polling shall be performed periodically when the IP ECU's conditions for communicating as well as the conditions for performing diagnostics are met.

666823 v2 **DTC - No Ethernet link to ECU for IP Router Switch ECU**

Version history:

(2):

Purpose: The IP Routing/switch ECU shall monitor the ECUs connected to it via Ethernet to see if the Ethernet link is up.

Source:

Verification Method: Analysis AND Test

Legacy ID:

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	GMRDB	Failure type (Low Byte)	0x00
Description	No Ethernet link to ECU **Note1:** Mandatory for IP Router/switch ECU. **Note2:** The IP Router/switch ECU shall implement one unique base DTC value for each ECU connected to it via Ethernet.		
Test run criteria (TRC)	**TRC1:** The TRC to perform this fault check shall be according to the general Test Run Criteria specified in "*Requirements from section Detection of faults*". **TRC2:** Besides the TRC1, also voltage for communication on controller shall be fulfilled to be able to set DTC for no link for point-to-point connections on Ethernet. The operating voltage range for Ethernet network shall also be according to *[UDSonIP_6] Ethernet Physical Layer*.		
Detection Criteria	No Ethernet link between it self and the ECUs connected to it self via a point-to-point connection. **Note1:** This is done by polling if Link is up or down in control register. The polling shall be performed periodically when the IP ECU's conditions for communicating as well as the conditions for performing diagnostics are met. **Note2:** To detect if link exists for point-to-point connection on Ethernet, a link status bit on the Ethernet transciever shall be used.		

DTC CALIBRATION				
comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
3	255	OCC2 or OCC5	Yes or No	100
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	20	Yes	10	20

4.10.3.2.3.4 Electrical fault trouble code information

872118 v2 **CANH circuit short to CANL circuit**

Version history:
(2): 1、修改Failure type 0x00为0x01。2、Description中描述勘误。

Purpose: Defining what the cause of the electrical fault may help in the fault tracing process. It is enough if the ECU closest to the tester checks electrical faults on the bus since if the fault is present, it will only be the ECU closest to the tester that will be able to report the status.

Source:

Verification Method: Test

Legacy ID:

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	GMRDB	Failure type (Low Byte)	0x01
Description	If the ECU is connected to more then one public network and the ECU is the ECU transferring diagnostic messages in between the two networks, the ECU shall support detecting CANH circuit short to CANL circuit. Note that there shall be one unique DTC for each public network in the vehicle.		
Test run criteria (TRC)	The TRC to perform the fault check shall be according to the General Test Run Criteria specified in "*Requirements from section Detection of faults*"		
Detection Criteria	The reported value of the DTC fault detection counter shall increase when CANH circuit short to CANL circuit is detected.		

DTC CALIBRATION

comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
3	255	OCC2 or OCC5	Yes or No	100
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	5	Yes	3	2

872115 v2 CANH circuit short to battery

Version history:
(2): 修改Failure type 0x00为0x12。

Purpose: Defining what the cause of the electrical fault may help in the fault tracing process. It is enough if the ECU closest to the tester checks electrical faults on the bus since if the fault is present, it will only be the ECU closest to the tester that will be able to report the status.

Source:
Verification Method: Test
Legacy ID:

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	GMRDB	Failure type (Low Byte)	0x12
Description	If the ECU is connected to more then one public network and the ECU is the ECU transferring diagnostic messages in between the two networks, the ECU shall support detecting CANH circuit short to battery voltage. Note that there shall be one unique DTC for each public network in the vehicle.		
Test run criteria (TRC)	The TRC to perform the fault check shall be according to the General Test Run Criteria specified in "*Requirements from section Detection of faults*"		
Detection Criteria	The reported value of the DTC fault detection counter shall increase when CANH circuit short to battery voltage is detected.		

DTC CALIBRATION

comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
3	255	OCC2 or OCC5	Yes or No	100
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	5	Yes	3	2

872119 v2 CANH circuit short to ground

Version history:
(2): 1、修改Failure type 0x00为0x11。2、Description中描述勘误。

Purpose: Defining what the cause of the electrical fault may help in the fault tracing process. It is enough if the ECU closest to the tester checks electrical faults on the bus since if the fault is present, it will only be the ECU closest to the tester that will be able to report the status.

Source:

Verification Method: Test

Legacy ID:

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	GMRDB	Failure type (Low Byte)	0x11
Description	If the ECU is connected to more then one public network and the ECU is the ECU transferring diagnostic messages in between the two networks, the ECU shall support detecting CANH circuit short to ground. Note that there shall be one unique DTC for each public network in the vehicle.		
Test run criteria (TRC)	The TRC to perform the fault check shall be according to the General Test Run Criteria specified in "*Requirements from section Detection of faults*"		
Detection Criteria	The reported value of the DTC fault detection counter shall increase when CANH circuit short to ground is detected.		

DTC CALIBRATION

comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
3	255	OCC2 or OCC5	Yes or No	100
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	5	Yes	3	2

872120 v2 CANL circuit short to battery

Version history:
(2): 1、修改Failure type 0x00为0x11。2、Description中描述勘误。

Purpose: Defining what the cause of the electrical fault may help in the fault tracing process. It is enough if the ECU closest to the tester checks electrical faults on the bus since if the fault is present, it will only be the ECU closest to the tester that will be able to report the status.

Source:

Verification Method: Test

Legacy ID:

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	GMRDB	Failure type (Low Byte)	0x12
Description	If the ECU is connected to more then one public network and the ECU is the ECU transferring diagnostic messages in between the two networks, the ECU shall support detecting CANL circuit short to battery voltage. Note that there shall be one unique DTC for each public network in the vehicle.		
Test run criteria (TRC)	The TRC to perform the fault check shall be according to the General Test Run Criteria specified in "*Requirements from section Detection of faults*"		
Detection Criteria	The reported value of the DTC fault detection counter shall increase when CANL circuit short to battery voltage is detected.		

DTC CALIBRATION				
comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
3	255	OCC2 or OCC5	Yes or No	100
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	5	Yes	3	2

872121 v2 CANL circuit short to ground

Version history:
(2):

Purpose: Defining what the cause of the electrical fault may help in the fault tracing process. It is enough if the ECU closest to the tester checks electrical faults on the bus since if the fault is present, it will only be the ECU closest to the tester that will be able to report the status.

Source:
Verification Method: Test
Legacy ID:

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	GMRDB	Failure type (Low Byte)	0x11
Description	If the ECU is connected to more then one public network and the ECU is the ECU transferring diagnostic messages in between the two networks, the ECU shall support detecting CANL circuit short to ground. Note that there shall be one unique DTC for each public network in the vehicle.		
Test run criteria (TRC)	The TRC to perform the fault check shall be according to the General Test Run Criteria specified in "*Requirements from section Detection of faults*"		
Detection Criteria	The reported value of the DTC fault detection counter shall increase when CANL circuit short to ground is detected.		
DTC CALIBRATION			

comfirmendDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
3	255	OCC2 or OCC5	Yes or No	100
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	5	Yes	3	2

975220 v4 **Separating type of electrical fault**

Version history:
(4):

Purpose: Defining what the cause of the electrical fault may help in the fault tracing process. It is enough if the ECU closest to the tester checks electrical faults on the bus since if the fault is present, it will only be the ECU closest to the tester that will be able to report the status.

Source:
Verification Method: Analysis
Legacy ID: 179113v1
Zeekr Requirement ID: 2789

The following faults for external circuit faults may be identified by one common DTC, instead of one single DTC for each fault. This shall be agreed between OEM and Tier-1.

The following faults may be combined
- CANH circuit short to battery voltage
- CANL circuit short to ground
- CANH circuit short to ground
- CANL circuit short to battery voltage
- CANH circuit short to CANL circuit

4.10.4 UDSonCAN

4.10.4.1 UDSonCAN - Generic

•Introduction

4.10.4.1.1 Introduction

This document specifies OEM specific requirements for an ECU on the CAN network. It intends to provide guidance and requirements to architects, system designers, application developers, and suppliers who are implementing an ECU connected to a CAN network. Figure - " *Document structure*" , shows the specifications that are related for a CAN ECU.

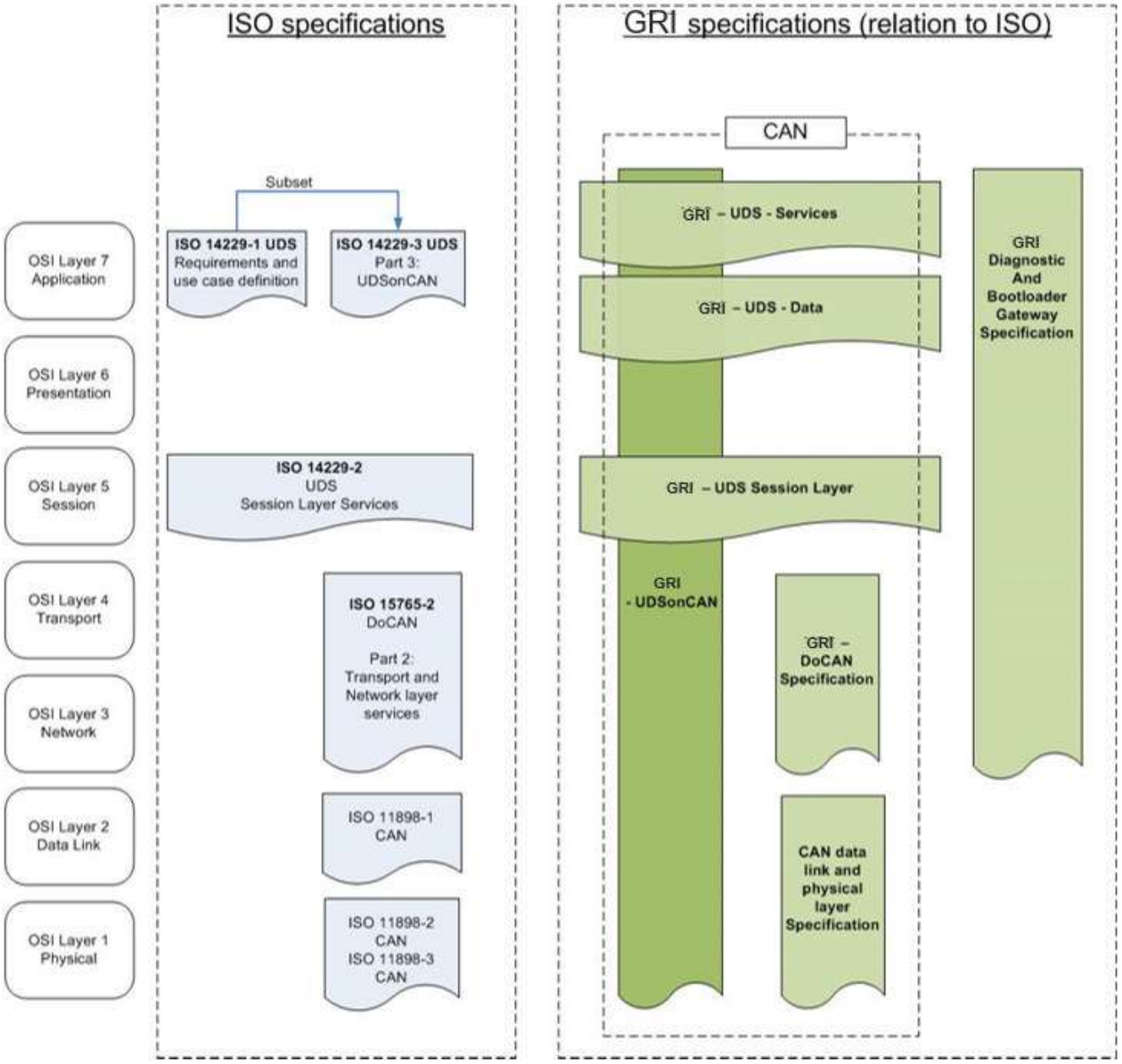


Figure - Document structure

4.10.4.1.1.1 Overview

This document specifies the CAN specific diagnostic requirements that shall apply to a CAN based ECU. All diagnostic messages on CAN shall be sent and received according to this and referenced documents in section Requisite documents .

Additional requirements are added if the ECU shall act as a diagnostic gateway to a sub network which is defined in the *[UDSonCAN_6] Diagnostic And Bootloader Gateway Specification*.

Depending on configuration, this document may consist in different combinations of the following sections:
- – Generic CAN ECU (Applicable to all diagnosable and/or programmable CAN ECUs)
- – Public CAN ECU (Applicable to all CAN ECUs supporting a total diagnostic implementation)
- – Private Programmable CAN ECU (Applicable for CAN ECUs with "limited" diagnostic implementation)

For those ECUs who implements a "limited" diagnostic functionality to support software download the Public ECU

master is responsible on the private programmable CAN ECU behalf to response on DTCs etc.

Note
This document covers both the older CAN standard, referred to as classic CAN, and the updated standard CAN Flexible Data-Rate, referred to as CAN FD. Unless explicitly specifying the standard of CAN in this document, any reference to CAN refers to both the classic CAN and CAN FD standards.

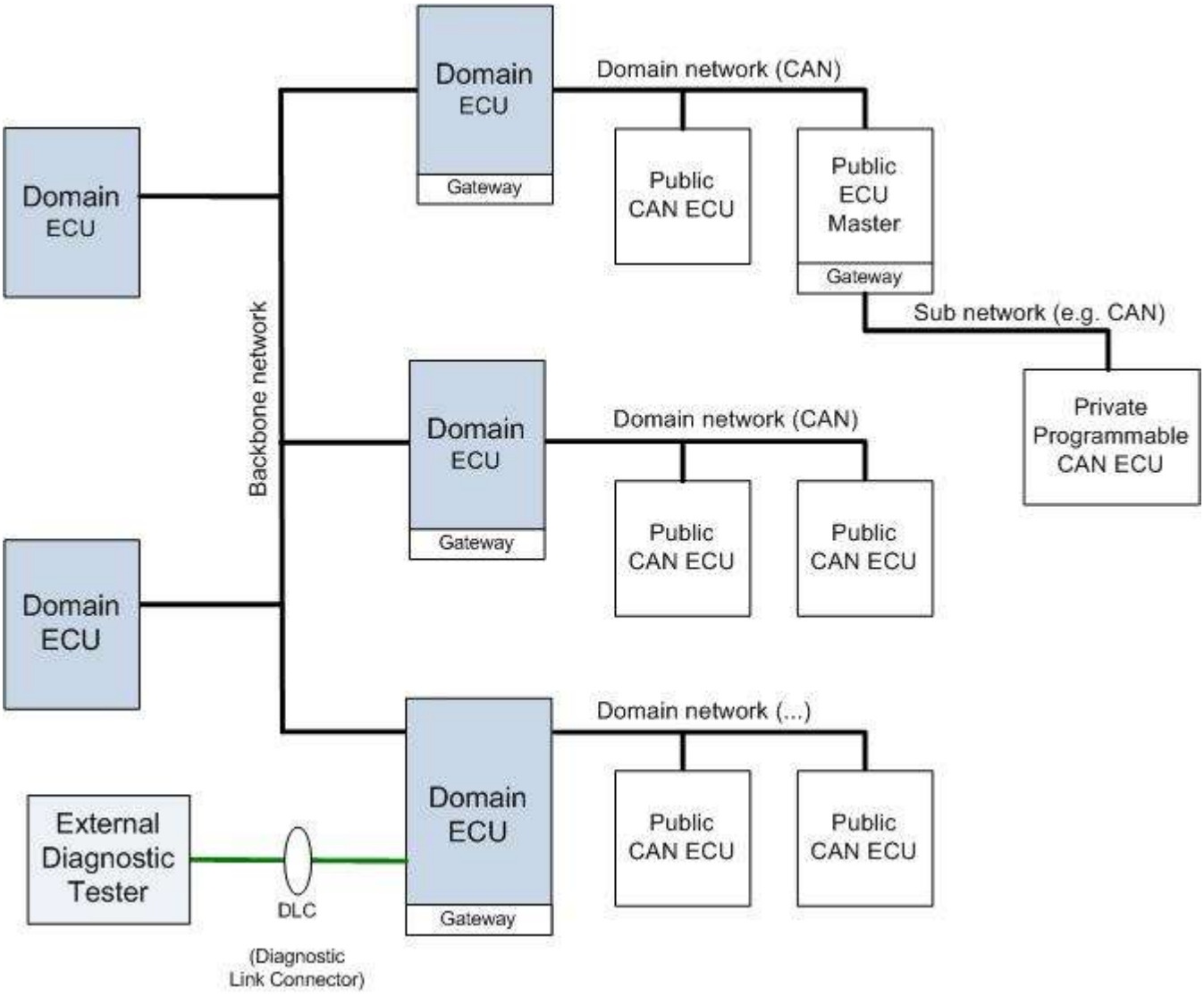


Figure: Example of the vehicle network topology

4.10.4.1.1.2 Requisite documents

Note that in the case of any requirement conflict between this document and any of the referenced documents, the proposed solution to the conflict shall be approved by GRI Basetech Team.

Reference number	Source	Title
[UDSonCAN_1]	Internal	UDS Services
[UDSonCAN_2]	Internal	UDS Data
[UDSonCAN_3]	Internal	UDS Session
[UDSonCAN_4]	Internal	DoCAN
[UDSonCAN_5]	Internal	CAN Data Link Layer Specification

[UDSonCAN_6]	Internal	Diagnostic And Bootloader Gateway Specification
[UDSonCAN_7]	Internal	Autosar Network Management- additional requirements
[UDSonCAN_8]	Internal - Base Technology DPR	CAN Physical Layer Specification
[UDSonCAN_9]	Internal	Bootloader_UDSonCAN
[UDSonCAN_10]	Internal - CarCom Database	Global Master Reference Database
[UDSonCAN_11]	Internal	General diagnostic guideline
[UDSonCAN_12]	ISO	ISO 14229-3: Road vehicles - Unified diagnostic services (UDS) - Part 3: UDS on CAN implementation (UDSonCAN) Edition 1, 2012

Note:
Internal references are typically available as separate chapters in ECU specific Base Technology SWRS.
Information obtaining ISO documents may be found on the internet: www.iso.org

4.10.4.1.1.3 Definitions and Abbreviations

Definition	Description
CANH	HIGH-level CAN bus line
CANL	LOW-level CAN bus line
Do not care	Implementation of the requirement is not required and not forbidden. The implementer may decide if to implement or not. If implemented it will not be verified and shall not be expected to be supported by tools.
Missing frame master	The missing frame master for a network is the ECU which is connected to a higher hierarchal network and act as a gateway for diagnostic request messages from the higher hierarchal network to the ECUs on the network. See *General diagnostic guideline.*
Missing frame slave	All ECUs, except the missing frame master, on a network. See *Missing frame guideline* requisite documents.
Network configuration master	The ECU that is responsibly for the network configuration.
Private ECU	An ECU not addressable from a client/tester point of view. Note: *An ECU can partly be public and partly private i.e. some diagnostic services and diagnostic data supported by the ECU are addressable and some are not.*
Public ECU	An ECU addressable from a client/tester point of view.
Public ECU Master	A public ECU responsible to act as a diagnostic server for one or more private ECUs.
Tester	A system that controls functions such as test, inspection, monitoring, or diagnosis of an on-vehicle electronic control unit and may be dedicated to a specific type of operators (e.g. a scan tool dedicated to garage mechanics or a test tool dedicated to assembly plant agents). The tester is also referenced as the client.
TRC	The Test Run Criteria defines the criteria that must be met for the DTC test to run, i.e. for the DTC test fault detection logic to execute and the DTC fault detection counter to be up-dated. *See UDS Services* requisite documents for further detailed explanation.

Abbreviation	Description
CAN	Controller Area Network
DTC	Diagnostic Trouble Code
ECU	Electronic Control Unit
MFCS	Missing frame Counter Signal. 1 byte counter sent by the missing frame master which all missing frame slaves shall use to ensure that the missing frame slave has continuous network access.
TRC	Test Run Criteria
UDS	Unified Diagnostic Services

•Application Layer

4.10.4.1.2 Application Layer

662533 v1 **Application bootloader requirement**

Version history:
(1):

Purpose: Define application behavior when transition to the bootloader and vice versa.
Source:
Verification Method: Analysis
Legacy ID: 55626v1

The application shall comply with the requirements as specified in *[UDSonCAN_9] Bootloader_UDSonCAN, chapter Flow chart diagram (Application),* in regards of what to do when making an ECU reset as well as when receiving a request to go to programming session.

662532 v1 **Application layer**

Version history:
(1):

Purpose: To define the application layer used for communicating on CAN
Source:
Verification Method: Analysis
Legacy ID: 55619v0

The application layer shall be compliant to *[UDSonCAN_12] Road vehicles - Unified diagnostic services (UDS) - Part 3: UDS on CAN implementation (UDSonCAN).*

•Session Layer

4.10.4.1.3 Session Layer

662534 v1 **Diagnostic Session Layer**

Version history:
(1):

Purpose: To define the Session layer used by the application.
Source:
Verification Method: Analysis
Legacy ID: 55627v0

The session layer required by this document shall be compliant to *[UDSonCAN_3] UDS Session*.

4.10.4.1.3.1 Addressing USDT frames

Since 11 bit CAN address mode with normal addressing format (see *[UDSonCAN_4] DoCAN)* is chosen, a fixed transmission ID is used. The ECU shall use the response ID specified in the table below - ECU Diagnostic Reception / Transmission CAN IDs as N_TA (Session layer Target Address).

ECU Reception ID Tester Transmit Request)	ECU Transmission ID (Tester Receive Request)
0x700–0x70F (1)	0x600–0x60F (1)
0x710	0x610
0x711	0x611
0x712	0x612
…	…
0x71E	0x61E
0x7FF	Functional request to all networks within all domains

Table - ECU Diagnostic Reception / Transmission CAN IDs

Note (1): The ECU reception ID from 0x700 to 0x70F is reserved due to the domain ECU always shall have network ID equal to zero(0).

Each ECU shall support a total of three (3) USDT CAN diagnostic IDs:
- Physically addressed requests (or ECU Diagnostic Reception ID).
- Physically and functionally addressed responses or (ECU Diagnostic Transmission ID). The ECU Transmission ID is always equal to the value of the
  ECU Reception ID minus 0x100.
- Functionally addressed requests to all ECUs in all domains. Value 0x7FF.

Conversion from the Logical address (ECU Address) to 11 bit CAN ID.
Example: If the logical address (i.e. TA and SA) is 0x1234 (Domain ID=0x2, Network ID=0x3 and ECU ID=0x4) the CAN ID 0x734 shall be used.

662535 v1 **Addressing USDT frames**

Version history:
(1):

Purpose: Define which CAN IDs an ECU shall listen on
Source:
Verification Method: Analysis
Legacy ID: 55628v2

Each ECU shall support a total of three (3) USDT CAN diagnostic IDs.
· Physically addressed requests (or ECU Diagnostic Reception ID).
· Physically and functionally addressed responses or (ECU Diagnostic Transmission ID). The ECU Transmission ID is always equal to the value of the ECU Reception ID minus 0x100.
· Functionally addressed requests to all ECUs in all domains. Value 0x7FF.

•Transport_Network layer

4.10.4.1.4 Transport_Network layer

662536 v1 **Transport_Network layer**

Version history:
(1):

Purpose: To define the transport/network layer used by the application.

Source:
Verification Method: Analysis
Legacy ID: 55630v0

The CAN transport/network layer required by this document shall be compliant with *[UDSonCAN_4] DoCAN*.

•Data Link Layer

4.10.4.1.5 Data Link Layer

662537 v1 **CAN data link layer**

Version history:
(1):

Purpose: To define the data link layer used by the application.
Source:
Verification Method: Analysis
Legacy ID: 55631v1

The data link layer required by this document shall be compliant to *[UDSonCAN_4] CAN Data Link Layer Specification*.

•Physical Layer

4.10.4.1.6 Physical Layer

662538 v1 **CAN physical layer**

Version history:
(1):

Purpose: To define the physical link layer used by the application.
Source:
Verification Method: Analysis
Legacy ID: 240274v0

The physical layer required by this document shall be compliant to *[UDSonCAN_8] CAN Physical Layer Specification*.

4.10.4.2 UDSonCAN - Public CAN ECU

•Diagnostic Services

4.10.4.2.1 Diagnostic Services

662539 v1 **Diagnostic services**

Version history:
(1):

Purpose: Define which services each CAN ECU shall or may support.
Source: Section "Diagnostic Services"
Verification Method: Analysis
Legacy ID: 55596v1

The CAN application shall support the diagnostic services as defined in *[UDSonCAN_1] UDS Services*.

•Diagnostic Data

4.10.4.2.2 Diagnostic Data

662540 v1 **Application diagnostic data**

Version history:
(1):

Purpose: Define which data parameters each CAN ECU shall or may support.
Source:
Verification Method: Analysis
Legacy ID: 55606v1

The CAN application shall support the diagnostic data as defined in *[UDSonCAN_2] UDS Data*.

4.10.4.3 UDSonCAN - Private Programmable CAN ECU

•Diagnostic Data

4.10.4.3.1 Diagnostic Data

All private programmable CAN ECUs shall at least support the data records as described in table " *Mandatory data records for private programmable CAN ECUs*".

The diagnostic implementer may design to support more data records than required as the implementation follows requirements and limitations as defined in this and in referenced specifications. Observe that this is considered to be handled as 'Do not care'.

Data record Item	Name
1	ECU Serial Number
2	Application Diagnostic Database Part Number
3	ECU Core Assembly Part Number
4	ECU Delivery Assembly Part Number
5	ECU Software Part Number(s)
6	Complete ECU Part/Serial Number
7	Active diagnostic session

Table – Mandatory data records for private programmable CAN ECUs

662674 v1 **ReadDataByIdentifier - Active diagnostic session**

Version history:
(1):

Purpose: The private programmable CAN ECU shall handle ReadDataByIdentifier requests on CAN.
Source: Item 7 in table "Mandatory data records for private programmable CAN ECUs".
Verification Method: Test
Legacy ID: 222562v0

All private programmable CAN ECUs shall support the dataIdentifier(Active diagnostic session, 0xF186).

662675 v1 **ReadDataByIdentifier - Application Diagnostic Database Part Number**

Version history:
(1):

Purpose: The private programmable CAN ECU shall handle ReadDataByIdentifier requests on CAN.
Source: Item 2 in table "Mandatory data records for private programmable CAN ECUs".
Verification Method: Test
Legacy ID: 222557v1

All private programmable CAN ECUs shall support the dataIdentifier (Application Diagnostic Database Part Number).

662676 v1 **ReadDataByIdentifier - Complete ECU Part Serial Number**

Version history:
(1):

Purpose: The private programmable CAN ECU shall handle ReadDataByIdentifier requests on CAN.
Source: Item 6 in table "Mandatory data records for private programmable CAN ECUs".
Verification Method: Test
Legacy ID: 222561v1

All private programmable CAN ECUs shall support the dataIdentifier (Complete ECU Part/Serial Number).

662677 v1 **ReadDataByIdentifier - ECU Core Assembly Part Number**

Version history:
(1):

Purpose: The private programmable CAN ECU shall handle ReadDataByIdentifier requests on CAN.
Source: Item 3 in table "Mandatory data records for private programmable CAN ECUs".
Verification Method: Test
Legacy ID: 222558v1

All private programmable CAN ECUs shall support the dataIdentifier (ECU Core Assembly Part Number).

662678 v1 **ReadDataByIdentifier - ECU Delivery Assembly Part Number**

Version history:
(1):

Purpose: The private programmable CAN ECU shall handle ReadDataByIdentifier requests on CAN.
Source: Item 4 in table "Mandatory data records for private programmable CAN ECUs".
Verification Method: Test
Legacy ID: 222559v1

All programmable private CAN ECUs shall support the dataIdentifier ECU (Delivery Assembly Part Number).

662679 v1 **ReadDataByIdentifier - ECU Serial Number**

Version history:
(1):

Purpose: The private programmable CAN ECU shall handle ReadDataByIdentifier requests on CAN.
Source: Item 1 in table "Mandatory data records for private programmable CAN ECUs".
Verification Method: Test
Legacy ID: 222556v0

All private programmable CAN ECUs shall support the dataIdentifier (ECU Serial Number, 0xF18C).

662680 v1 **ReadDataByIdentifier - ECU Software Part Number(s)**

Version history:
(1):

Purpose: The private programmable CAN ECU shall handle ReadDataByIdentifier requests on CAN.
Source: Item 5 in table "Mandatory data records for private programmable CAN ECUs".
Verification Method: Test
Legacy ID: 222560v1

All programmable private CAN ECUs shall support the dataIdentifier (ECU Software Part Number(s)).

4.10.4.3.1.1 RoutineIdentifier

All private programmable CAN ECUs shall at least support the routineIdentifier as described in table “ *Mandatory routines for private programmable CAN ECUs*.”

The diagnostic implementer may design to support more routines than required as long as the implementation follows requirements and limitations as defined in this and in referenced specifications. Observe that this is considered to be handled as 'Do not care'.

RoutineIdentifier		
Item	**Name**	**Hex**
1	Check Programming Pre-conditions	0206

Table – Mandatory routines for private programmable CAN ECUs

662681 v1	RoutineControl - Check Programming Pre-conditions

Version history:
(1):

Purpose: The private programmable CAN ECU shall handle RoutineControl requests on CAN.
Source: Item 1 in table "Mandatory routines for private programmable CAN ECUs".
Verification Method: Test
Legacy ID: 222563v0

All private programmable CAN ECUs shall support the routineIdentifier(Check Programming Pre-conditions, 0x0206).

•Diagnostic Services

4.10.4.3.2 Diagnostic Services

All private programmable CAN ECUs shall at least support the diagnostic services as described in table “ *Mandatory diagnostic services for private programmable CAN ECUs*”.

The diagnostic implementer may design to support more diagnostic services than required as long as the implementation follows requirements and limitations as defined in this and in referenced specifications. Observe that this is considered to be handled as 'Do not care'.

Item	Service	Hex	Sub function/parameter	Hex
1	DiagnosticSessionControl	10	DiagnosticSessionType – programmingSession	02 and 82
2	ReadDataByIdentifier	22	See section “ *DataRecords*”	-
3	RoutineControl	31	RoutineControlType - startRoutine See section “	01 and 81

RoutineIdentifier
s" for
parameter.

Table – Mandatory diagnostic services for private programmable CAN ECUs

662682 v1 **DiagnosticSessionControl DiagnosticSessionType-programming session**

Version history:
(1):

Purpose: The programmable private CAN ECU shall handle DiagnosticSessionControl programmingSession requests on CAN.
Source: Item 1 in table "Mandatory diagnostic services for private programmable CAN ECUs".
Verification Method: Test
Legacy ID: 222553v0

All private programmable CAN ECUs shall support the DiagnosticSessionControl (0x10) service with diagnosticSessionType (0x02 and 0x82).

662683 v1 **ReadDataByIdentifier**

Version history:
(1):

Purpose: The private programmable CAN ECU shall handle ReadDataByIdentifier requests on CAN.
Source: Item 2 in table "Mandatory diagnostic services for private programmable CAN ECUs".
Verification Method: Test
Legacy ID: 222554v0

All private programmable CAN ECUs shall support the ReadDataByIdentifier (0x22) service.

662684 v1 **RoutineControl**

Version history:
(1):

Purpose: The private programmable CAN ECU shall handle RoutineControl requests on CAN.
Source: Item 3 in table "Mandatory diagnostic services for private programmable CAN ECUs".
Verification Method: Test
Legacy ID: 222555v0

All private programmable CAN ECUs shall support the RoutineControl (0x31) service with routineControlType - startRoutine (0x01 and 0x81).

4.10.5 UDSonLIN

4.10.5.1 UDSonLIN - Generic

•Introduction

4.10.5.1.1 Introduction

This document specifies the LIN ECU diagnostic requirements. It intends to provide guidance and requirements to architects, system designers, application developers, and suppliers. The requirements are applicable for all LIN masters and diagnosable and reprogrammable LIN slaves.

Figure - Document structure, shows the specifications that are related for a LIN ECU.

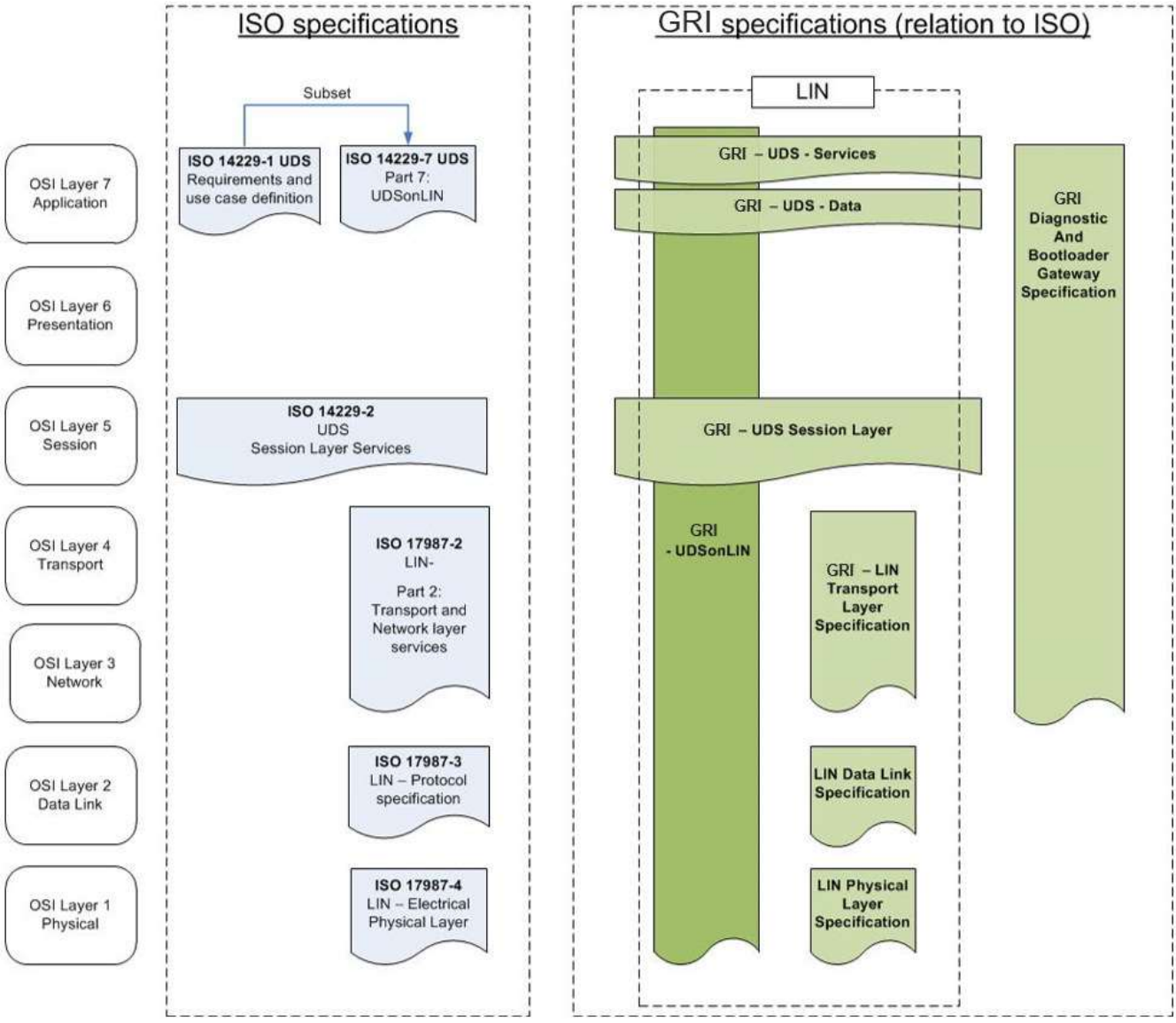


Figure - Document structure

•Overview

4.10.5.1.2 Overview

Depending on configuration, this document may consist in different combinations of the following sections:

- Generic. Applicable to all ECUs on LIN.
- Generic LIN master. Applicable to all LIN masters.
- LIN slave. Applicable for all diagnosable and reprogrammable LIN slaves.

The requirements in this document are applicable for all ECUs with LIN, regardless of higher level network (i.e. network the LIN master is connected to).

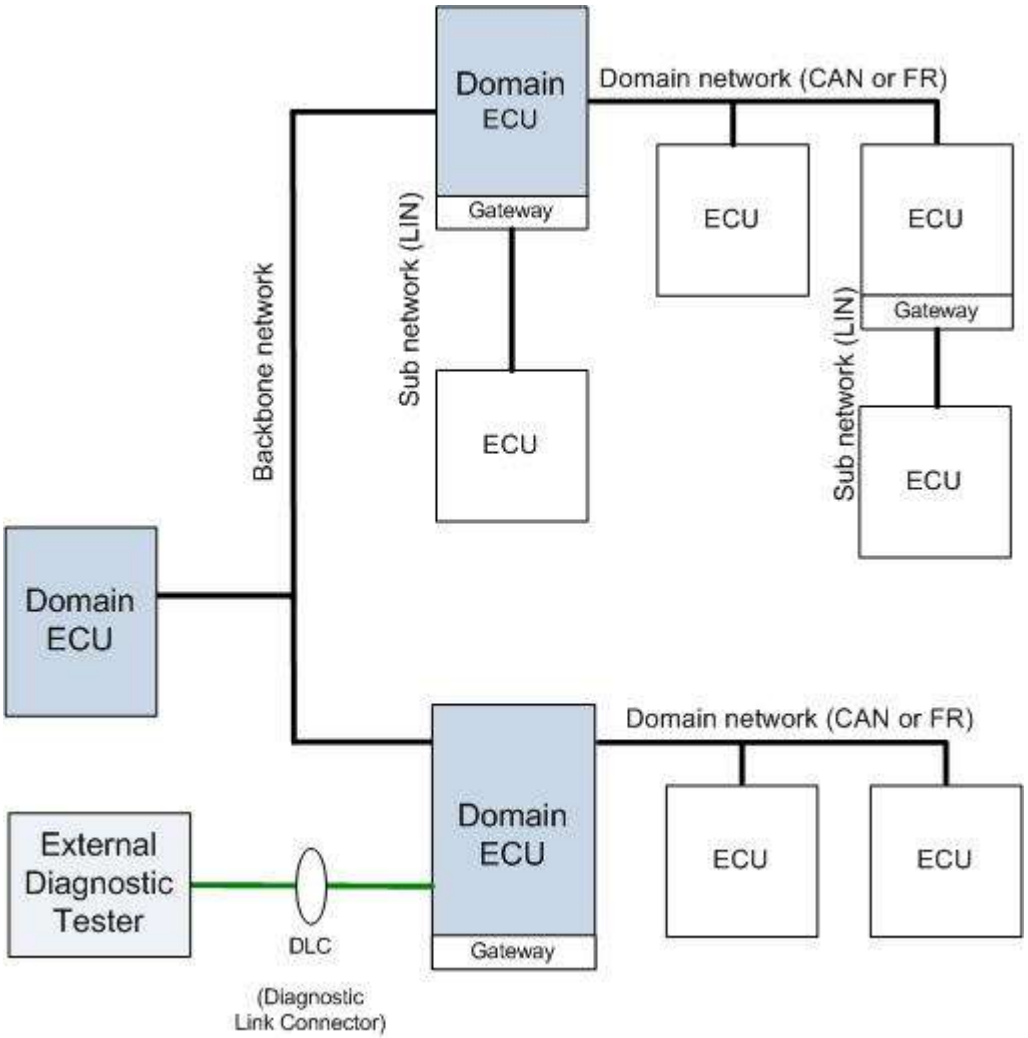


Figure - Example of the vehicle network topology

•Requisite documents

4.10.5.1.3 Requisite documents

Reference	Source	LC in Elektra
[UDSonLIN_1]	Internal	UDS Services

[UDSonLIN_2]	Internal	UDS Data
[UDSonLIN_3]	Internal	UDS Session
[UDSonLIN_4]	Internal	LIN Transport Layer
[UDSonLIN_5]	Internal	LIN Data Link Layer
[UDSonLIN_6]	Internal - Base Technology DPR	LIN Physical Layer
[UDSonLIN_7]	Internal	General Software Download
[UDSonLIN_8]	ISO	ISO 14229-7: Road vehicles - Unified diagnostic services (UDS) - Part 7: Unified diagnostic services on Local Interconnect Network implementation (UDSonLIN). (2015-02-01)

Note:
Internal references are typically found as a seperate chapter in the ECU specific Base Technology SWRS.

•Definitions and Abbreviations

4.10.5.1.4 Definitions and Abbreviations

Definition	Description
ECU ID	Part of the ECU Address, unique identifier for each ECU on a network.
Frame identifier	See reference [ISO 14229-7] Frame identifier definition.
Header	See reference [ISO 14229-7] header definition.
Implementer	Person responsible for the diagnostic implementation of the ECU. It may be the vehicle manufacturer or the ECU supplier diagnostic software designer.
LIN Master	See reference [ISO 14229-7] Master node definition.
LIN Slave	See reference [ISO 14229-7] Slave node definition.
Network ID	Part of the ECU Address, states which Network the ECU is physical connected to.
Schedule table	See reference [ISO 14229-7] schedule table definition.
Tester	A system that controls functions such as test, inspection, monitoring, or diagnosis of an on-vehicle electronic control unit and may be dedicated to a specific type of operators (e.g. a scan tool dedicated to garage mechanics or a test tool dedicated to assembly plant agents). The tester is also referenced as the client.

Abbreviation	Description
ECU	Electronic Control Unit
CAN	Controller Area Network
FR	FlexRay
NAD	Node Address
NRC	Negative Response Code
SA	Source Address

SPRMIB	Suppress Positive Response Message Indication Bit
TA	Target Address
UDS	Unified Diagnostic Services

4.10.5.2 UDSonLIN - Generic LIN master

•Diagnostic Data

4.10.5.2.1 Diagnostic Data

666815 v1 **Application diagnostic data**

Version history:
(1):

Purpose: Define which data parameters each LIN Master ECU shall or may support.
Source:
Verification Method: Analysis
Legacy ID:

The application shall support the diagnostic data as defined in *[UDSonLIN_2] UDS Data*.

4.10.5.3 UDSonLIN - LIN Slave

•Unified diagnostic services implementation

4.10.5.3.1 Unified diagnostic services implementation

4.10.5.3.1.1 Mandatory LIN services and data records

All programmable ECUs on LIN shall at least support the diagnostic services as described in table “Mandatory diagnostic services for private programmable ECUs on LIN network” in application mode.

Item	Service	Hex	Sub function/parameter	Hex
1	DiagnosticSessionControl	10	DiagnosticSessionType	02 and 82
2	ReadDataByIdentifier	22	See section "DataRecords"	-
3	ReadDTCInformation	19	ReportDTCByStatusMask	02 and 82
4	ClearDiagnosticInformation	14	GroupOfDTC All Groups	FFFFFF

Table – Mandatory diagnostic services for private programmable ECUs on LIN network

All reprogrammable ECUs shall at least support the data records as described in table “Mandatory data records for all private reprogrammable ECUs on LIN” in application mode

Item	Data Record
1	ECU Serial Number
2	Application Diagnostic Database Part Number
3	ECU Core Assembly Part Number
4	ECU Delivery Assembly Part Number
5	ECU Software Part Number(s)
6	Complete ECU Part/Serial Number
7	Active diagnostic session

Table – Mandatory data records for all private reprogrammable ECUs on LIN.

662746 v1 **DiagnosticSessionControl diagnosticSessionType programmingSession**

Version history:
(1):

Purpose: The LIN slave shall handle DiagnosticSessionControl programmingSession requests on LIN.
Source: General Software Download
Verification Method: Test
Legacy ID: 80115v1

The LIN slave shall support DiagnosticSessionControl programmingSession requests.

662753 v1 **LIN slave ReadDataByIdentifier Active diagnostic session**

Version history:
(1):

Purpose: There shall be a standardized request/response method a tester may use to be sure the ECU is in assumed session.
Source: Item 7 in table Mandatory diagnostic services and data records for ECUs on LIN.
Verification Method: Test
Legacy ID: 125543v1

All reprogrammable LIN slaves shall support Active diagnostic session (F186) requests.

662748 v1 **LIN slave ReadDataByIdentifier Application Diagnostic Database Part Number**

Version history:
(1):

Purpose: The diagnostic interface shall be used even when not directly used by the tester to primarily standardize the method used for retrieving information from the LIN slaves.
Source: Item 3 in table Mandatory diagnostic services and data records for ECUs on LIN.
Verification Method: Test
Legacy ID: 80118v2

All reprogrammable LIN slaves shall support Application Diagnostic Database Part Number requests.

662752 v1 **LIN slave ReadDataByIdentifier Complete ECU Part/Serial Number**

Version history:
(1):

Purpose: The diagnostic interface shall be used even when not directly used by the tester to primarily standardize the method used for retrieving information from the LIN slaves.
Source: Item 6 in table Mandatory diagnostic services and data records for ECUs on LIN.
Verification Method: Test
Legacy ID: 80125v2

All reprogrammable LIN slaves shall support Complete set of ECU part/serial numbers requests.

662749 v1 **LIN slave ReadDataByIdentifier ECU Core Assembly Part Number**

Version history:
(1):

Purpose: The diagnostic interface shall be used even when not directly used by the tester to primarily standardize the method used for retrieving information from the LIN slaves.
Source: Item 2 in table Mandatory diagnostic services and data records for ECUs on LIN.
Verification Method: Test
Legacy ID: 80119v2

All reprogrammable LIN slaves shall support ECU Core Assembly Part Number requests.

662750 v1 **LIN slave ReadDataByIdentifier ECU Delivery Assembly Part Number**

Version history:
(1):

Purpose: The diagnostic interface shall be used even when not directly used by the tester to primarily standardize the method used for retrieving information from the LIN slaves.
Source: Item 4 in table Mandatory diagnostic services and data records for ECUs on LIN.
Verification Method: Test
Legacy ID: 80121v2

All reprogrammable LIN slaves shall support ECU Delivery Assembly Part Number requests.

662751 v1 **LIN slave ReadDataByIdentifier ECU Software Part Number(s)**

Version history:
(1):

Purpose: The diagnostic interface shall be used even when not directly used by the tester to primarily standardize the method used for retrieving information from the LIN slaves.
Source: Item 5 in table Mandatory diagnostic services and data records for ECUs on LIN.
Verification Method: Test
Legacy ID: 80122v3

All reprogrammable LIN slaves shall support ECU Software Part Number(s) requests.

662747 v1 **LIN slave ReadDataByIdentifier ECU serial number**

Version history:
(1):

Purpose: The diagnostic interface shall be used even when not directly used by the tester to primarily standardize the method used for retrieving information from the LIN slaves.
Source: Item 1 in table Mandatory diagnostic services and data records for ECUs on LIN.
Verification Method: Test
Legacy ID: 80116v1

All reprogrammable LIN slaves shall support ECU serial number (0xF18C) requests.

1029903 v1 **ReadDTCInformation and ClearDiagnosticInformation**

Version history:
(1): 新增需求，支持刷写的LIN Slave节点需要支持$19读故障码和$14清故障码。

Purpose: The LIN slave shall handle ReadDTCInformation and ClearDiagnosticInformation requests on LIN.
Source: Item 3 and 4 in table Mandatory diagnostic services for private programmable ECUs on LIN network.
Verification Method: Test
Legacy ID:

The reprogrammable LIN slave shall support ReadDTCInformation with subfunction ReportDTCByStatusMask and ClearDiagnosticInformation with parameter GroupOfDTC All Groups requests.

4.10.5.3.1.2 Defining NAD for LIN slaves

662754 v1 **LIN slave supported NAD types**

Version history:
(1):

Purpose: The LIN slaves needs to support both physically and functional addressed NAD addresses.
Source:
Verification Method: Test
Legacy ID: 82429v0

Each reprogrammable LIN slave shall support at least two (2) USDT diagnostic NADs: functionally addressed requests and physically addressed requests and responses.

•Requirements from section Detection of faults due to configuration errors

4.10.5.3.2 Requirements from section Detection of faults due to configuration errors

975457 v2 **Detection of no production public key and Security Constant**

Version history:
(2): 修订需求，生产公钥和安全常数未写入的DTC故障，由Slave LIN ECU通过$19服务报告，而非通过Master上报。

Purpose: To ensure that it can be detected if the public key has never been written in ECU.
Source:

Verification Method: Test

Legacy ID:

Zeekr Requirement ID: 843574

If the ECU is required to be programmed production public key and security constant for software download, the ECU shall be able to detect if it has be written in protected memory area successfully, and report the DTC fault through ReportDTCByStatusMask service.

4.10.6 UDSonIP

4.10.6.1 UDSonIP - Generic reqs

•Introduction

4.10.6.1.1 Introduction

This document specifies OEM specific diagnostic requirements for an ECU connected to an IP based network. It intends to provide guidance and requirements to architects, system designers, application developers, and suppliers who are implementing the diagnostics to an ECU connected to an IP based network.

It is important to understand that this document only specifies the IP specific requirements and not the complete diagnostics, the figure below shows the specifications that are related to the implementation specification for an ECU connected to IP based network.

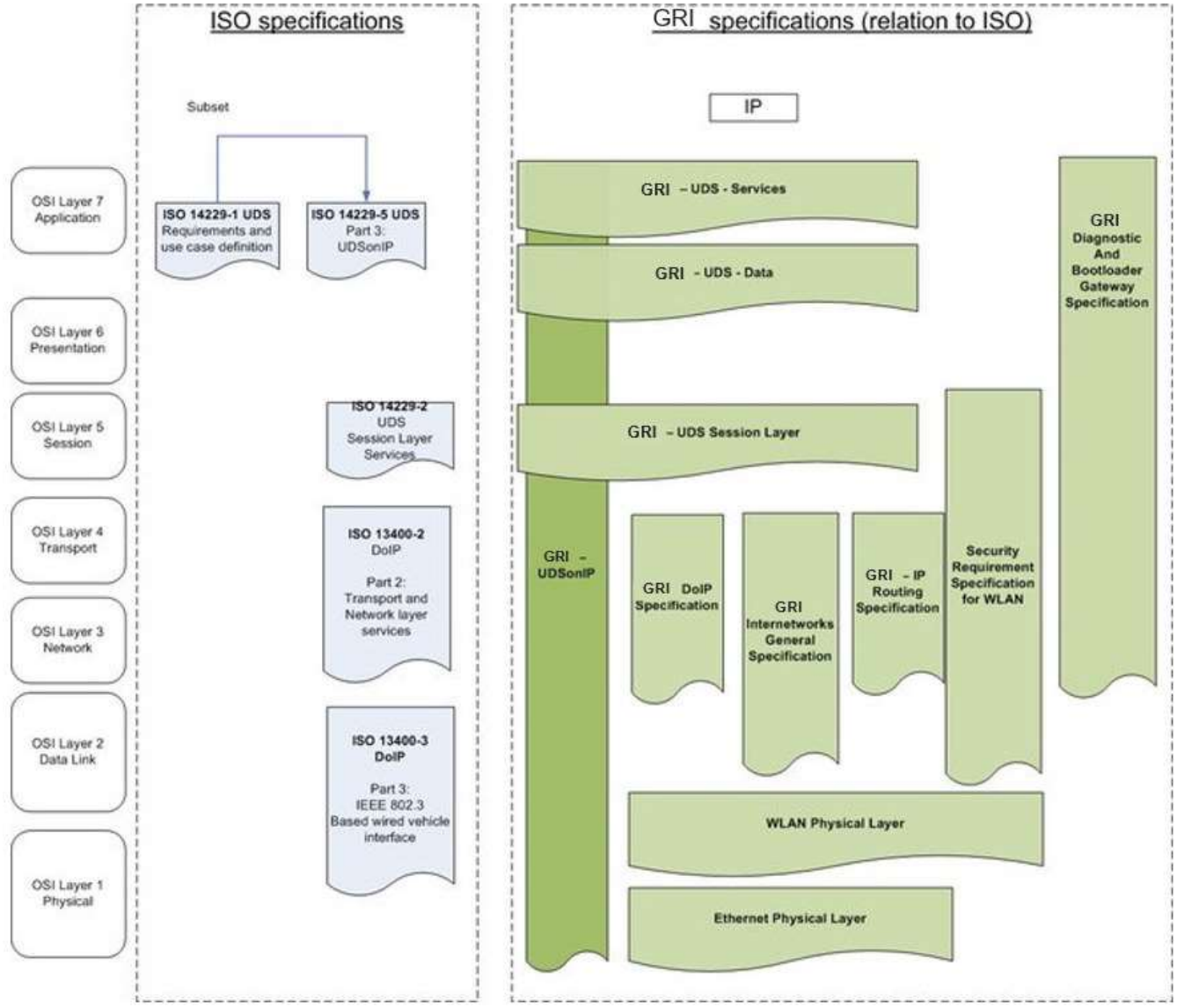


Figure: Document structure overview
•Overview

4.10.6.1.2 Overview

This document defines IP diagnostic implementation specific requirements that shall apply to an IP based ECU. There are three different types of ECUs implementing IP diagnostics: Edge Node, IP Router/switch ECU and IP ECU. The Edge Node handles the communication towards external testers as well as forwarding diagnostic messages to in vehicle ECUs via backbone network and internal networks. One part of the communication with external tester is to send messages used to identify the vehicle (vehicle announcements). The IP Router/switch ECU is an ECU that act as an IP router or IP switch. The other type of ECUs is those connected to internal TCP/IP based networks such as Ethernet. These ECUs are called IP ECUs in this document.

Additional requirements are added for diagnostic if the Edge node or IP ECU shall act as a diagnostic gateway or router or switch to a sub network. These requirements are defined in the

[UDSonIP_8] Diagnostic And Bootloader Gateway Specification and *[UDSonIP_9] IP Routing Specification*.

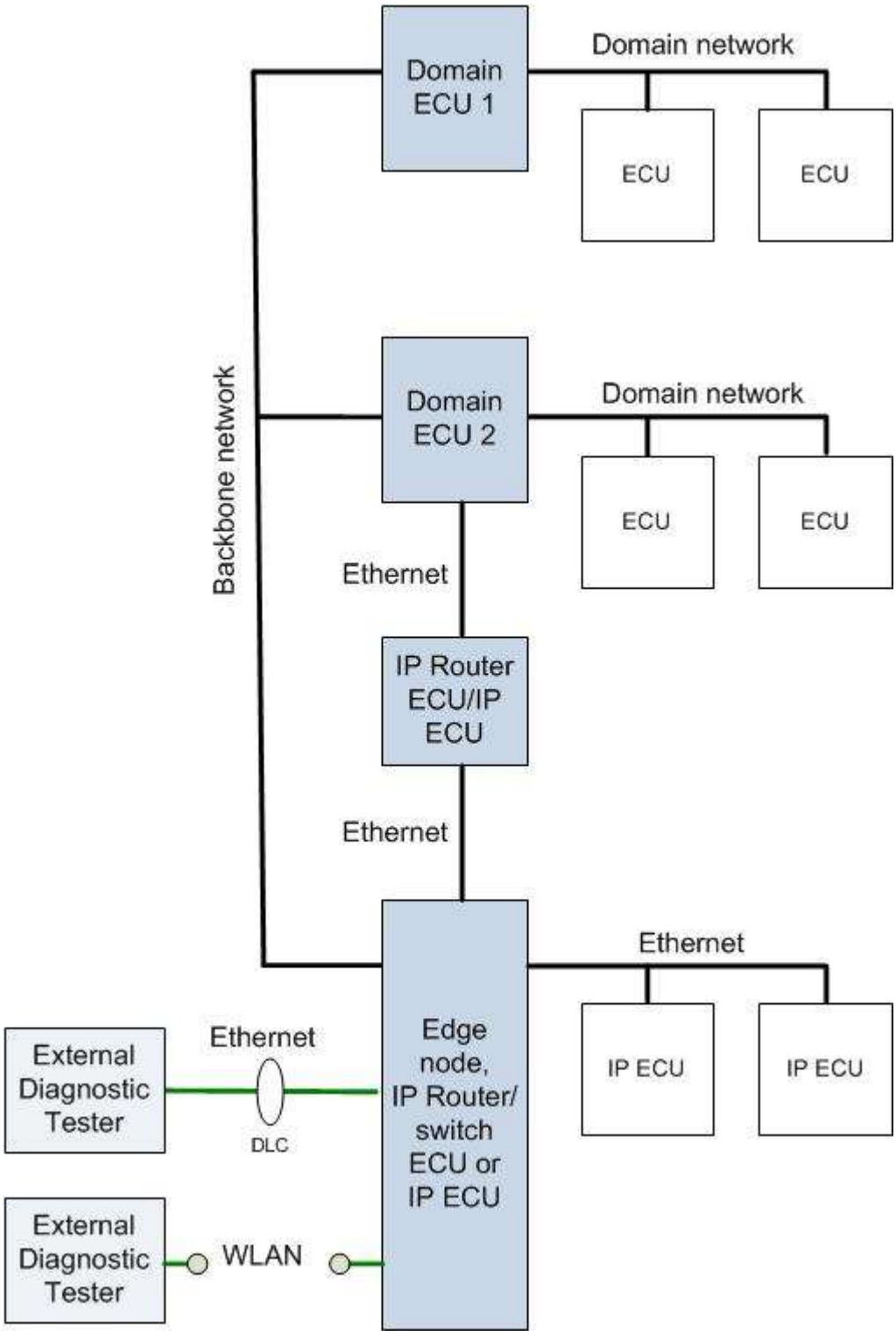


Figure: An example topology

662685 v1 **Diagnostic gateway**

Version history:
(1):

Purpose: To be able to gateway diagnostic messages in the diagnostic kernel.
Source: Section "Overview", Diagnostic And Bootloader Gateway Specification
Verification Method: Analysis
Legacy ID: 60589v0

ECUs which shall gateway diagnostic messages shall implement the applicable diagnostic requirements specified in *Diagnostic And Bootloader Gateway Specification*.

662686 v2 **Diagnostic routing or switching**

Version history:
(2):

Purpose: To define how to gateway signals from one IP based network to another.
Source: Section "Overview", IP Routing Specification
Verification Method: Analysis
Legacy ID: 60592v1

ECUs that shall route or switch diagnostic messages shall support applicable parts of routing or switching in accordance to IP Routing and Firewalling, UDSonIP, Requisite documents.

•Requisite documents

4.10.6.1.3 Requisite documents

In the case of any requirement conflict between this document and any of the referenced documents, the proposed solution to the conflict shall be approved by the responsible authority appointed by GRI Basetech Team before the conflict is recognized as resolved.

Reference number	Source	Title
[UDSonIP_1]	Internal	UDS Services
[UDSonIP_2]	Internal	UDS Data
[UDSonIP_3]	Internal	UDS Session
[UDSonIP_4]	Internal	DoIP
[UDSonIP_5]	Internal	Internetworks General Specification
[UDSonIP_6]	Internal	Ethernet Physical Layer
[UDSonIP_7]	Internal	WLAN Data link and Physical layer
[UDSonIP_8]	Internal	Diagnostic And Bootloader Gateway Specification
[UDSonIP_9]	Internal	IP Routing and Firewalling
[UDSonIP_10]	Internal	WLAN Security Specification
[UDSonIP_11]	Internal	Bootloader - UDSonIP
[UDSonIP_12]	Internal	Global Master Reference Database
[UDSonIP_13]	Internal	Security Access Algortihm
[UDSonIP_14]	ISO	ISO/DIS 14229-5: Road vehicles — Unified diagnostic services (UDS) — Part 5: Unified diagnostic services on Internet Protocol implementation (UDSonIP) 2011-11-13, Edition: 1

[UDSonIP_15]	ISO	ISO 13400-3: Road vehicles — Diagnostic communication over Internet Protocol (DoIP) — Part 3: Wired vehicle interface based on IEEE 802.3 2012-01-11, Edition: 1

Notes:

Internal references are typically available as separate chapters in ECU specific Base Technology SWRS.
In the case of any requirement conflict between this document and any of the referenced documents, the proposed solution to the conflict shall be approved by GRI Basetech Team before the conflict is recognized as resolved.

Information obtaining ISO documents may be found on the internet: http://www.iso.org/

•Definitions and Abbreviations

4.10.6.1.4 Definitions and Abbreviations

Abbreviation	Description
AP	Access Point
D	Don't care
DLC	Diagnostic Link Connector
DoIP	Diagnostic communication over IP
DTC	Diagnostic Trouble Code
ECU	Electric Control Unit
GMRDB	Global Master Reference Database
IP	Internet Protocol
M	Mandatory
NA	Not Applicable
O	Optional
OBD	On-Board Diagnostics
SSID	Service Set Identifier
STA	Station
TRC	Test Run Criteria
VLAN	Virtual Local Area Network
WLAN	Wireless Local Area Network

Definition	Description
Edge Node	An ECU (internal vehicle network node) which is the network node that is exposed towards the external network. The node implements a diagnostic over IP-protocol (DoIP - ISO13400) and provides diagnostic access to test equipment.
IP Router/switch ECU	An ECU that support an IP router or switch.
Nomadic Device	A handheld wireless device such as a PDA or smart phone.
Other session	The sessions used by diagnostic when application is running: • Default Session • Extended Session Programming Session is not included since used by bootloader.
WLAN link	An associated and authenticated link on WLAN between vehicle and an external AP where the vehicle acts as a station.

•Unified diagnostic service implementation

4.10.6.1.5 Unified diagnostic service implementation

4.10.6.1.5.1 Diagnostic services

The required diagnostic service are defined in *[UDSonIP_1] UDS Services* with the restrictions and additions in subsequent subsections.

The services and data (defined in *[UDSonIP_1] UDS Services* and *[UDSonIP_2] UDS Data*) sent on IP between tester and Edge node are encapsulated by the Diagnostic Message defined in *[UDSonIP_4] DoIP*.

A TCP connection is used for in vehicle communication and for external communication with vehicle as well as between a stand alone ECU and tester. When a reset occurs the TCP connection is lost and needs to be established again after the reset. The only time it is ok for an ECU to do a reset is when tester sends ECUReset or when DiagnosticSessionControl(programmingSession) is received.

4.10.6.1.5.1.1 ReadDataByPeriodicIdentifier

ReadDataByPeriodicIdentifier is sent on TCP as well as all other services. The Response Message Type for this service shall be of type#2 format.

662687 v1 **ReadDataByPeriodicIdentifiers Response Message Type**

Version history:
(1):

Purpose: To define ReadDataByPeriodicIdentifier since ISO does not as well as treat this service in the same way as on other networks.
Source: Section "ReadDataByPeriodicIdentifier", UDS Services
Verification Method: Test
Legacy ID: 60599v0

The Response Message Type for ReadDataByPeriodicIdentifier(0x2A) shall be a type #2 format.

4.10.6.1.5.2 Diagnostic data

The diagnostic data required are specified in *[UDSonCAN_2] UDS Data.*

•Application Layer

4.10.6.1.6 Application Layer

662701 v1 **Application layer**

Version history:

(1):

Purpose: To define the application layer used for communicating on IP based networks.
Source:
Verification Method: Analysis
Legacy ID: 60615v1

The application layer shall be compliant with *[UDSonIP_14] ISO 14229-5: Road vehicles — Unified diagnostic services (UDS) — Part 5: Unified diagnostic services on Internet Protocol implementation (UDSonIP).*

4.10.6.2 UDSonIP - Edge node reqs

•Unified Diagnostic Service Implementation Requirements

4.10.6.2.1 Unified Diagnostic Service Implementation Requirements

4.10.6.2.1.1 Diagnostic services

666821 v1 **Diagnostic services**

Version history:
(1):

Purpose: Define which services each Ethernet ECU shall or may support.
Source:
Verification Method: Analysis
Legacy ID:

The Ethernet ECU application shall support the diagnostic services as defined in *[UDSonCAN_1] UDS Services*.

4.10.6.2.1.2 Diagnostic Data

666820 v2 **ECU Ethernet identification data**

Version history:
(2): Error description revision.

Purpose: Define which data parameters each IP ECU shall or may support.
Source:
Verification Method: Analysis
Legacy ID:

The Ethernet ECU application shall support the diagnostic data as defined in *[UDSonIP_2] UDS Data*.

4.10.6.2.1.3 Physical Connection

The ECU containing the Edge Node functionality is connected to the external tester either via the OBD connector(Ethernet) or via WLAN. The Edge Node shall therefore support detection of active link and IP assignment as well as sending vehicle announcements. The Ethernet active link is described in accordance to *[UDSonIP_15] ISO 13400-3: Road vehicles — Diagnostic communication over Internet Protocol (DoIP) — Part 3: IEEE802.3 based wired*.

The Edge Node shall request an IP address and send vehicle announcement messages according to *[UDSonIP_4] DoIP*.

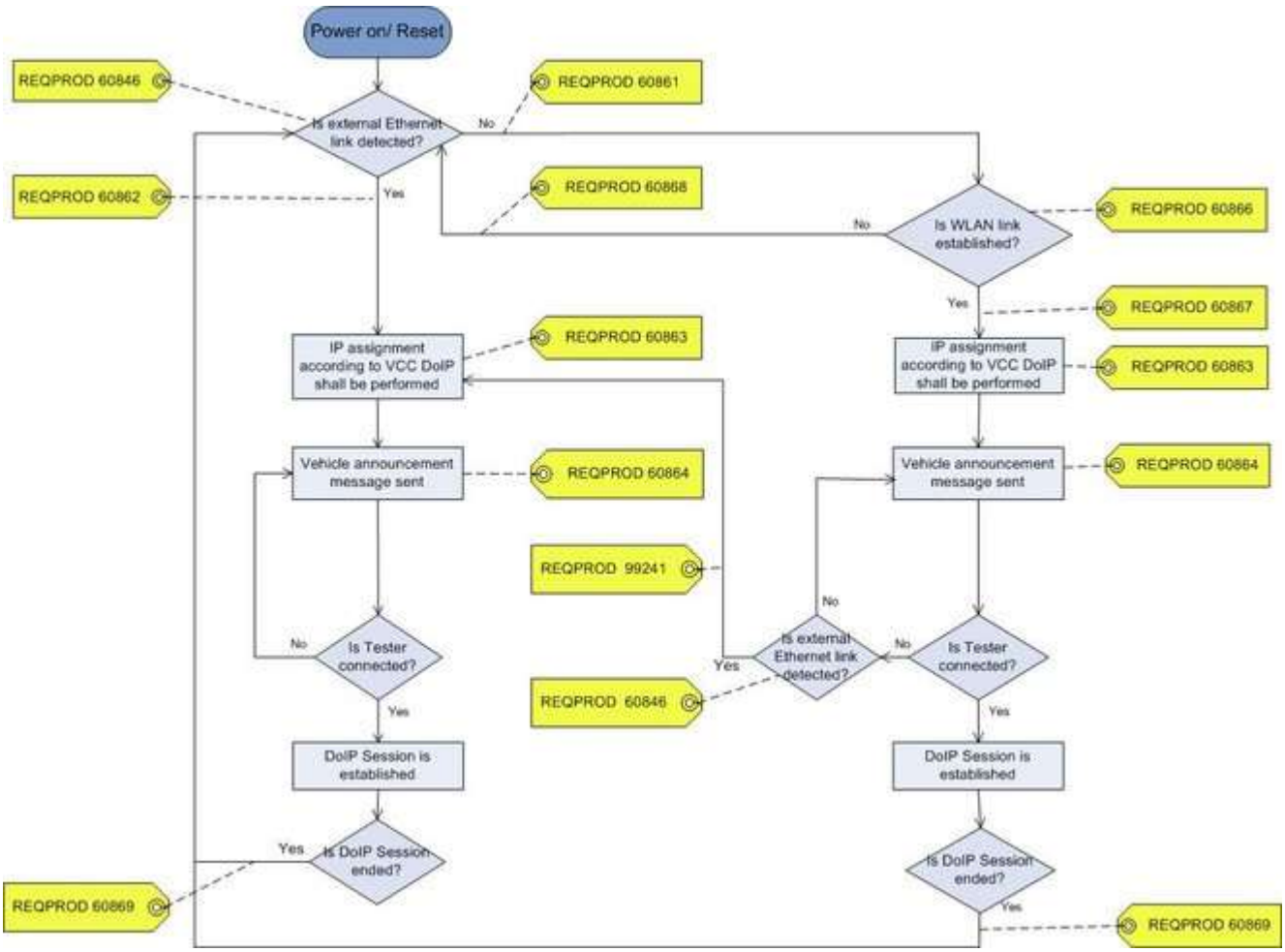


Figure: Link connection and IP assignment

662708 v2 **Edge Node - Check external Ethernet link**

Version history:
(2): Verification Method: Analysis change to Analysis AND Test.

Purpose: To be able to communicate and receive diagnostic services from an external tester over Ethernet, an Ethernet link shall exist
Source: Road vehicles - Diagnostic communication over Internet Protocol (DoIP) - Part 3: IEEE802.3 based wired
Verification Method: Analysis AND Test
Legacy ID: 60846v0

The ECU shall detect an external Ethernet link by using the OBD activation line *[UDSonIP_15] Road vehicles — Diagnostic communication over Internet Protocol (DoIP) — Part 3: IEEE802.3 based wired*.

662709 v1 Edge Node - Detection of WLAN Link

Version history:
(1):

Purpose: Since it is no use to request for IP address or send vehicle announcements if there is no WLAN link detected the Edge Node checks for WLAN link before doing so.
Source:
Verification Method: Analysis
Legacy ID: 60866v1

Check if an authenticated and associated WLAN link exists between the vehicle(configured as AP) and an external AP.

662710 v1 Edge Node - Ethernet link is detected

Version history:
(1):

Purpose: When Ethernet link is detected, IP address assignment is done is used to set up TCP connection with tester as well as sending vehicle announcement.
Source:
Verification Method: Analysis
Legacy ID: 60862v0

If external Ethernet link is detected, next step is to enter IP address assignment state.

662711 v1 Edge Node - Ethernet link not detected

Version history:
(1):

Purpose: If no Ethernet link is detected next step is to check if WLAN link is detected.
Source:
Verification Method: Analysis
Legacy ID: 60861v0

If external Ethernet link is not detected, try to detect WLAN link.

662712 v1 Edge Node - External Ethernet cable rejects WLAN link

Version history:
(1):

Purpose: It shall be possible for an Ethernet cable/OBD cable(with ethernet support) to terminate a WLAN link if no DoIP sessionis established. The reason is to give workshops a possibility to change link.
Source:
Verification Method: Analysis
Legacy ID: 99241v2

WLAN link shall be shutdown if an external Ethernet cable/OBD cabel(with ethernet support) is detected and no DoIP session has been established on the WLAN link. The next step is to enter IP assignment state.

662713 v1 **Edge Node - IP address assignment**

Version history:
(1):

Purpose: To be able to communicate with external tester an IP address is required.
Source:
Verification Method: Analysis
Legacy ID: 60863v0

After detection of active link t he Edge Node shall perform an IP address assignment in accordance to *[UDSonIP_4] DoIP*.

662714 v1 **Edge Node - Is DoIP session ended?**

Version history:
(1):

Purpose: To start checking again which link is connected if active link, or DoIP session is disconnected.
Source:
Verification Method: Test
Legacy ID: 60869v1

A restart of checking which link is connected to vehicle shall be made when the DoIP session ends or times out according to *[UDSonIP_4] DoIP*.

662715 v1 **Edge Node - Physical link connection and DoIP connection**

Version history:
(1):

Purpose: To describe detection of link, how to handle the IP address and sending of vehicle announcements
Source:
Verification Method: Analysis
Legacy ID: 60865v0

The Edge Node shall implement the flow chart *"Link connection and IP assignment*", found in UDSonIP, Physical Connection.

662716 v1 **Edge Node - Vehicle announcement messages**

Version history:
(1):

Purpose: To enable communication with external tester, the Edge Node has to indicate that the vehicle exists. This is done by announcement messages.

Source:
Verification Method: Test
Legacy ID: 60864v1

The Edge Node shall send vehicle announcement messages when an active link is detected in accordance to *[UDSonIP_4] DoIP*.

662717 v1 **Edge Node - WLAN link does not exists**

Version history:
(1):

Purpose: To define which next step is when WLAN link does not exists.
Source:
Verification Method: Analysis
Legacy ID: 60868v1

If WLAN link does not exist, the next step is to check external Ethernet link.

662718 v1 **Edge Node - WLAN link exists**

Version history:
(1):

Purpose: To define which next step is when WLAN link exists.
Source:
Verification Method: Analysis
Legacy ID: 60867v0

If WLAN link exists next step is to check for IP Address.

4.10.7 Data Security

4.10.7.1 Random number generation

•Introduction

4.10.7.1.1 Introduction

When a random number is needed for a cryptographic application both algorithm and seed need to be chosen with care. Examples of cryptographic application in the automotive context is:

- Challegne response protocols like security access
- TLS
- Passwords

when possible, autosar standard should be used.

Recommendations can be found in NIST SP 800-90A-C
http://csrc.nist.gov/groups/ST/toolkit/random_number.html
or
BSI AIS 20 & BSI AIS 31
https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/Zertifizierung/Interpretationen/AIS_31

_Functionality_classes_for_random_number_generators_e.pdf?__blob=publicationFile

and tool https://www.fourmilab.ch/random

•Requirements

4.10.7.1.2 Requirements

661282 v1 **Entropy for random numbers**

Version history:
(1):

Purpose: Avoid predictable outcome from random number generation, by ensuring that the random number generator is seeded with sufficient entropy.
Source:
Verification Method: Analysis
Legacy ID: 419311v1

The output from the entropy source shall be tested using the tool ENT (https://www.fourmilab.ch/random/). The entropy generated shall when tested with ent shall be:

- - Entropy: At least 7.5 bits per character or higher.
- - Compression: 0%
- - Chi-Square: Between 200 and 400 and randomly exceed more than 10% and less than 90% of the time.
- - Mean value: Between 126 and 128
- - Monte Carlo: Max 0.5% error
- - Serial correlation: 0.1 or lower

Note; recommendations can be found in NIST SP 800-90A-C or BSI AIS 20 & BSI AIS 31 https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/Zertifizierung/Interpretationen/AIS_31_Functionality_classes_for_random_number_generators_e.pdf?__blob=publicationFile

and tool https://www.fourmilab.ch/random/

661283 v2 **Random number generator algorithms**

Version history:
(2): Verification Method: Analysis change to Analysis AND Test.

Purpose: Ensure that generated random numbers are suitable for use in cryptographic and other security related use cases.
Source: NIST Special Publication 800-22 Revision 1a
Verification Method: Analysis AND Test
Legacy ID: 434662v0

ASIL: QM

Random numbers shall be generated by a deterministic random bit generator (DRBG), also known as a cryptographically safe pseudo random number generator. The generator shall be seeded and periodically reseeded from entropy provided by the entropy source.

The generator used shall be one of the hash based or block cipher based generators approved by NIST, see SP 800-90A. That is Hash_DRBG, HMAC_DRBG eller CTR_DRBG. The generator Dual_EC_DRBG shall not be used.

The output from the generator shall be tested with the tool Dieharder (https://www.phy.duke.edu/~rgb/General/dieharder.php). When tested, no test shall fail independent of ECU mode (like after a reboot).

4.10.7.2 Security Access algorithm

•Introduction

4.10.7.2.1 Introduction

Purpose

The purpose of this document is to define the security algorithm to be used with diagnostic service $27 (request security access) for all ECUs supporting diagnostics, regardless of protocol.

Diagnostic service $27 is based upon the tester requesting a random seed from the ECU. The tester then utilizes the algorithm described in this document to calculate a key which is transmitted to the ECU. The ECU utilizes the same algorithm and compares its calculated key to the key submitted by the tester to determine if security access is granted or not.

The algorithm described in this document generates a response (key) to an challenge. The challenge consists of random seed and fixed bytes. Note that the fixed bytes are not transmitted on the network and must be known in advance by both the ECU and tester. These fixed bytes are also referred to as the Security Constant.

The seed bytes are transmitted from S1 (most significant byte of the seed) to S_n (least significant byte of the seed) in a $67 positive response to a service $27 (security access) diagnostic request. For example, $67 $01 $S1 $S2 $S3..... $$S_n$.

Key (response) bytes are transmitted from R1 (most significant byte of the key) to R_n (least significant byte of the key) in a $27 submit key request. For example, $27 $02 $R1 $R2 $R3..... $$R_n$.

(Refer to specific algorithm documents for specific random number Seed /Fixed Byte /Key lengths and mixed numbers.)

•Algorithm Description

4.10.7.2.2 Algorithm Description

661291 v2	Security Access Algorithm Description

Version history:
(2):

Purpose: Define the algorithm used for Security Access
Source:
Verification Method: Analysis
Legacy ID: 362857v0

The Security Access algorithm description is provided by GRI Base Technology Team upon request.

•Security Constant

4.10.7.2.3 Security Constant

661284 v2 **Distribution of the Fixed Bytes**

Version history:
(2):

Purpose: To define the source provider of the fixed bytes.
Source: -
Verification Method: Test + Inspection
Legacy ID: 353878v0
ASIL: QM - Not safety relevant

The fixed bytes are to be supplied from GRI. These are not to be chosen by supplier.

703288 v1 **Fixed Bytes Program Frequency Limit**

Version history:
(1): Add the requirements for Fixed bytes writing.

Purpose: Avoid Security Constant repeatedly overwritten by values from unauthorized sources.
Source:
Verification Method: Test
Legacy ID:

When receving a programming fixed bytes request, if the fixed bytes have already been programmed to the ECU, the ECU shall reject the request and return negative response code 0x22. All fixed bytes can only be programmed once and cannot be read under any circumstances.

850391 v1 **Fixed Bytes Run**

Version history:
(1): 通用需求，Fixed Bytes写入后立即生效。适用于非Basetech要求的其他安全访问Fixed Bytes的通用需求。

Purpose: Ensure correct implementation and process safety.
Source:
Verification Method: Test
Legacy ID:

The programmed fixed bytes shall be applied immediately once programming successfully, i.e. without requiring an ECU reset.

661285 v2 **Fixed Bytes default value**

Version history:
(2):

Purpose: To define the default value of the fixed bytes when no Security Constant has been programmed to the ECU.
Source: Required by diagnostic tools.

Verification Method: Analysis
Legacy ID: 353877v0
ASIL: QM - Not safety relevant

If the fixed bytes has not yet been programmed to the ECU and unlocking by the $27 Security Access service is requested, the ECU shall use a default value where all fixed bytes are set to 0xFF.

661286 v2 **Key Storage**

Version history:
(2):

Purpose: The security of the system depends on the secrecy of the key that is used.
Source:
Verification Method: Test + Inspection
Legacy ID: 360246v0

The Fixed Bytes used shall be stored in a protected area, typically in tamper resistant memory, and must not be possible to read out. If a key update process is required, the security of that process must be ensured.

•Configuration of the Level of Protection

4.10.7.2.4 Configuration of the Level of Protection

The level of protection by using diagnostic service Security Access together with the Security Access Algorithm is configurable with respect to:
1. Fixed Bytes:
Common. The Fixed Bytes are used for a security level by an ECU in several vehicles. Typically it is already programmed when the ECU is delivered or . Hence, a compromised ECU (disclosure of fixed bytes from an ECU) will affect the security of other vehicles.
Unique. ECU individual (unique) Fixed Bytes are used (forward-secrecy, i.e. a compromised ECU will not affect the security of another ECU).
2. Delay Timer introduced after a number of false security access attempts.
3. Delay Timer could be applied upon every start-up.
To guide the system designer, this logical component will specify some basic configurations with respect to the level of protection. Which level to choose is based on the information (asset) to be protected.

Notes:
- The terminology "level of protection" is used in favor to "level of security" as security level is already in use (ISO 14229-1) to define the "area" to be protected in an ECU using Security Access. Each Security Level could be configured separately.
- There should be dedicated Fixed Bytes for different purposes (roles), e.g., for accessing enhanced diagnostic functionality or for ECU reprogramming.
- The level of protection is dependent of the random number generator capabilities (seed generation) and the secure storage of the Fixed Bytes.

Protection Level	**Delay Timer** (activated when max number of false attempts detected)	**Number of false security access attempts**	**Fixed Bytes**
1	N/A	N/A	Common
2	10 seconds	2	Common
3	N/A	N/A	Unique
4	5-60 seconds	2	Unique

Table - Levels of Protection

Level 1: To prevent an unauthorized user that is having limited time and capabilities, or to avoid accidental errors, as the common key must for "Level 1" be assumed to be known by an attacker at some stage.
Level 2: See Level 1. But, adds additional protection (time effort) to reveal the Fixed Bytes used the first time.

Level 3: Using this level limits that unauthorized access (Fixed Bytes revealed) to one ECU also affects the security of a second ECU.
Level 4: See Level 3. But, adds additional protection (time effort) to reveal the Fixed Bytes used.

Physical attacks not considered here, i.e. where Fixed Bytes are extracted from the ECU memory.

661292 v5 — Default values - Delay timers and counters for false attempts

Version history:
(5): 描述措辞更新，避免需求理解歧义。

Purpose: To define default values for the parameters used to configure the level of protection
Source:
Verification Method: Test
Legacy ID: 355163v2

The default values shall be set to:
- Delay Timer activated after a number of false Security Access attempts (T_1): 10 seconds.
- Delay Timer activated at every start (T_2): 0 seconds.
- Number of false Security Access attempt before a delay time is activated: 2.

A function/system designer is free to increase the delay timer activated after a number of false attempts, but decrasing that parameter or change of the other parameters must require an approved deviation.

ECUs having the proper hardware capabilities shall store (in non-volatile memory) the information whether the delay timer after a number of false attempts has been fully activated or not, i.e. that information shall not be lost at a reset, in order to determine whether to restart the delay timer (T_1) after initialization.

The false attempts counter shall be reset to zero (0):
- Any successful security access attempts (valid key), or
- Delay timer elapsed, or
- ECU Reset to start again, applicable when the previously false attemps counter value was 1.

Clarification: "Number of false Security Access attempt before a delay time is activated: 2".
1. First false attempt, the negative response code of invalidKey shall be responed.
2. Seconde false attempt, the negative response code of exceedNumberOfAttempts shall be responed.

661288 v2 — SecurityAccess (27) - time delay after false securityAccess attempts

Version history:
(2):

Purpose: For some scenarios the tools will require two attempts to unlock the ECU.
Source:
Verification Method: Test
Legacy ID: 76145v2

It shall be possible to make at least 1 false SecurityAccess attempt without being penalized by a time delay before the next attempt is accepted.

•Random Number Generation

4.10.7.2.5 Random Number Generation

The generation of the random number, i.e. the Seed the sent by the ECU, is important with respect to the security of algorithm, e.g. identical Seed values will result in that identical Key values (Response Bytes) are computed.

661289 v2 **Seed - Random Number Generation**

Version history:
(2):

Purpose: Random and pseudo-random numbers generated for cryptographic applications should be unpredictable.
Source:
Verification Method: Test + Analysis
Legacy ID: 354418v0
ASIL: QM - Not safety relevant

The Seed generated must be a cryptographically secure random number, meaning that it's generated using either a True Random Number Generator (TRNG) or Pseudo-Random Number Generator (PRNG). This implicit implies that the Seed outputs must not be repeated after an ECU reset. Hence, static Seed shall never be used.

•Security Levels (Areas)

4.10.7.2.6 Security Levels (Areas)

An ECU is divided into several security access levels ("areas"), where each level is protected by a dedicated security access constant. Information or services are by that separated and only accessible to the user having access to the security access constant to a dedicated level. For example, an user shall only be allowed to read some information but not be able to re-program an ECU.

This logical component defines some basic security access levels that shall be applied unless there is a specific need to add some more level in order to separate some information further.

661293 v3 **Security Level usage**

Version history:
(3): Due to information security considerations, the security access level has been switched to 19/1A as the default level for security diagnostic services, with 05/06 level reserved.

Purpose: Define a basic set of security access levels. Other security access levels are still supported, but needs to specified elsewhere.
Source:
Verification Method: Analysis
Legacy ID: 436226v0

The ECU shall use the security access levels defined in *Table - Security Access Levels.* Other levels are however also allowed, but requires an analysis to identify the need of separating information or services further.

Security Access Level	Purpose	Comment
01	Used for software download	A vehicle unique security access constant shall be generated in the manufacturing process.
03	Car mode transition	A common security access constant shall be applied.
05	General for diagnostic services **(Reserved)**	A common security access constant shall be applied. If the ECU has a specific need, it shall be discussed and be aligned with the Basetech Team.
11	Immo and ITP	A vehicle unique security access constant shall be generated in the manufacturing process.
19	**Security diagnostic services**	A vehicle unique security access constant shall be generated in the manufacturing process.

		This is the protected area that shall be applied by the ECU.
1D	Test and development only	Uses the algorithm supported by OEM tool-chain (but not used at manufacturing or workshops)
1F	Legal requirements for ERA-GLONASS	-
31	Vehicle Speed Limitation	A common security access constant shall be applied.
33	Engine Oil Maintenance & EEPROM Removal	Used for engine-related ECU nodes. Only applicable in aftersale scene.
5F	Pyrotechnic end-of-life activation	ISO 26021 requirement

Table - Security Access Levels

4.10.8 Authentication (0x29) with PKI Certificate Exchange

4.10.8.1 Authentication General Requirements

•Authentication function

4.10.8.1.1 Authentication function

666444 v3 **Authentication protection configuration**

Version history:
(3): 增加法律和法规相关的诊断总是被认为是授权的客户端，无需执行认证流程即可访问。

Purpose: In the EOL production stage, ECU has not owned its own certificate file, so the authentication protection shall be turned off in advance. It can only be enabled after the certificate file is updated.
Source:
Verification Method: Test
Legacy ID: 656703v1

The authentication protection function for diagnostic service access and/or diagnostic routing handling (i.e. through external diagnostics) of ECU shall be activated through configuring data record define in "***ReqID 656886***".

There are exceptions which shall be treated as authorized client:
— Onboard diagnostics (i.e. OTA, RVDC) shall always be considered as authorized client, without concerning about whether the authentication protection configuration has been turned on. And it shall diagnose the vehicle without going through the authentication process.
— Diagnostics related to laws and regulations

666443 v3 **Authentication protection for diagnostic routing handling**

Version history:
(3): 仅制造厂商定义的DoIP才需要过认证。

Purpose: The usage of authentication to protect diagnostic routing handling through legal client shall be used.
Source:
Verification Method: Test
Legacy ID: 655562v1

If the ECU is as a gateway, all UDS diagnostic routing handling need to be required to be protected by authentication when through manufacture specific DoIP.

666440 v3 **Authentication protection for diagnostic services on Edge Node**

Version history:
(3): 对于边缘节点，明确哪些诊断信息是不需要0x29认证保护。

Purpose: The usage of authentication to protect diagnostic services through legal client shall only be used when required and/or judged as needed.
Source:
Verification Method: Test
Legacy ID: 653843v1

If the ECU as an Edge Node is established diagnostic communication in external connection (e.g. OBD connection, WiFi Diagnostic connection), most diagnostic services supported by Edge Node except 0x29 need to be required to be protected by authentication.

The following diagnostic contents of the vehicle gateway should not be protected by authentication:

- Diagnostic session control with sub function default session (0x10 01);
- Read data by identifier (0x22): 0xC03C, 0xC03D, 0xF186, 0xF190, 0xEDA0, 0xED20, 0x9007, 0xDD07, 0xF1AE;
- Routine control (0x31): 0x0206, 0x7027.

Protection by authentication in other type of diagnostic connection shall be evaluated and confirmed by Base Technology team.

•Authentication state for UDS service 29

4.10.8.1.2 Authentication state for UDS service 29

When secure diagnostics function for authentication service is disabled, ECU shall always in authenticated state. The diagram of transitions is as follow:

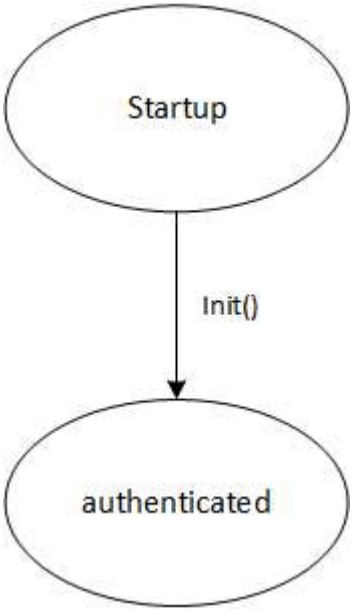


Figure - Authenticated state transitions with persistent state

When secure diagnostics function for authentication service is enabled, ECU shall refer to the following diagram of transitions.

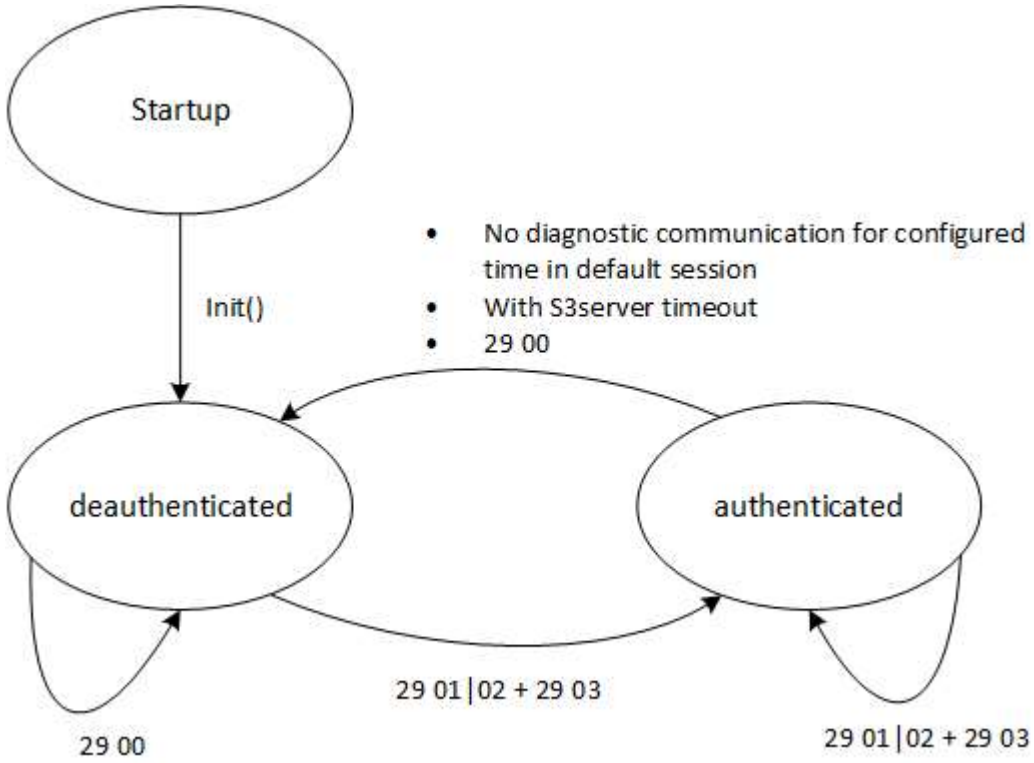


Figure - Authenticated state transitions without persistent state

665955 v2 **Authenticated state**

Version history:
(2):

Purpose: To define the authenticated state.
Source:
Verification Method: Test
Legacy ID: 640200v1

If the whole authentication process was completed and the final proof of ownership for client was successfully authenticated by ECU, which that is defined in reference *[Authentication with PKI Certificate Exchange],* the ECU shall set the authenticated state on this connection.

If authentication funciton is disabled (e.g. configuration for authentication service is disabled, or PKI certificate is revoked or damaged) , it means persistent authenticated state is set.

665957 v2 **Deauthenticated state**

Version history:
(2): Add handling of failed authentication.

Purpose: To define the deauthenticated state.
Source:
Verification Method: Test
Legacy ID: 652559v1

When ECU initialization is completed, it shall be in deauthenticated state by default until the whole authentication process is executed successfully.

Once deauthenticated state is in or authentication process is failed, all data generated and used during authentication process shall be destruction if the authentication process has been performed.

665960 v3 **Fallback to deauthenticated state on connection**

Version history:
(3): A installed timer由3 min调整为1 min。

Purpose: The implicit requirement to leave authenticated state shall be supported.
Source:
Verification Method: Test
Legacy ID: 640201v1

The ECU shall make a transition from authenticated into deauthenticated state for a configured connection if any of the following condition apply:

- The ECU was in default session when the last diagnostic response was send on that connection and a installed timer times out after no valid diagnostic data was received or handled (e.g. gateway routing) on that connection after the last valid diagnostic data. A installed timer shall be configured to 1 min. At least every request message of the same diagnostic protocol received by the transport protocol shall keep the authenticated state active and reset the timeout period.
- The ECU is in non-default session and a S3server timeout occurs.

668963 v2 **Maintain authenticated state functionality at ssion transitions**

Version history:
(2): To adjust that the authenticated status is associated with the diagnostic connection.

Purpose: Authenticated state will be kept at session transition unless the exit conditions are met.
Source:
Verification Method: Test
Legacy ID: 658312v1

The ECU shall keep authenticated state between session changes unless diagnostic connection is broken.

665959 v1 **Receiving de-authentication request**

Version history:
(1):

Purpose: The explicit requirement to leave authenticated state shall be supported.
Source:
Verification Method: Test
Legacy ID: 640202v1

The ECU shall exit the authenticated state intentionally when receiving the "deAuthenticate" request from client.

•Appendix C- Authentication roles and White lists for UDS service 29

4.10.8.1.3 Appendix C- Authentication roles and White lists for UDS service 29

665964 v1 **Authentication role in Server**

Version history:
(1):

Purpose: Defined the initial role in deauthentication state. There may be a fully restricted role actually used in untrusted scenario.
Source:
Verification Method: Analysis AND Test
Legacy ID: 647325v1

In the deauthentication state, ECU internal static role configuration shall be configured as the authentication role of Supplier. Supplier authentication roles will be defined in GMRDB.

The exception is that a fully restricted role can be configured, seeing **ReqID 653843**.

665966 v1 Authentication role of Admin

Version history:
(1):

Purpose: The authentication role right need to be defined clearly.
Source:
Verification Method: Analysis AND Test
Legacy ID: 654302v1

Client whose certificate is assigned to the Admin authentication role shall be allowed to access all diagnostic services/data supported by ECU software implementation.

The other authentication roles will be used in further consideration.

665965 v2 Authentication roles for External Diagnostic Client

Version history:
(2): 修改更新，仅描述外部诊断客户端的认证角色和权限。

Purpose: The external diagnostic client can provide certificates with multiple roles to authorize access to diagnostic services or data.
Source:
Verification Method: Analysis AND Test
Legacy ID: 647397v1

The external diagnostic client can transmit the PKI certificate of the authentication role described in **ReqID 647400** to server. The certificate will grant the client (e.g. external diagnostic tool) rights for the specific roles.

665962 v2 Authentication roles per client certificate

Version history:
(2):

Purpose: The certificate role is defined in advance, but only one authentication role is deployed per certificate on one connection.
Source:
Verification Method: Analysis
Legacy ID: 647400v1

Different authentication roles are defined on each client PKI certificate, including Admin, Supplier, Development, Production, AfterSales, ExtendedUser.

716322 v1 Authentication roles verification definition

Version history:
(1): 新增对认证角色Role的说明和定义。

Purpose: Defined the size and bit position of roles.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The size of role shall be defined 4 bytes. The value of role can refer to the data element “O” of Subject in the certificate, which is interpreted in the little endian format.

The bit position definition of role is shown as follows.

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24	Bit23~Bit16	Bit15~Bit8	Bit7~Bit0
--	--	**Admin (admi)**	--	--	--	**Supplier (SUPR)**	--	--	--	--

Note: "--" indicates for future use.

The Bit29 indicates the Admin role, and Bit25 indicates the Supplier role. Upon each role verification, the ECU shall make a bitwise "and" operation on the role provided in the certificate for that connection and the role assigned to the service. If the result is equal to 0, the ECU will stop the processing of subsequent diagnostic service and send a negative response 0x34 - "authenticationRequired".

716317 v1 Omitting Authentication of Internal Diagnostic Clients

Version history:
(1):

Purpose: The internal diagnostic client is integrated into the ECU controller and already has a high level of cyber security, no need for redundant authentication processes.
Source:
Verification Method: Test
Legacy ID:

For in-vehicle module with diagnostic client capability, e.g. OTAMaster and RVDC, vehicle gateway shall authorize their permissions to diagnose the vehicle without going through the authentication process.

Note: diagnosing the vehicle means that vehicle gateway shall be able to response to its own diagnostic request and route diagnostic messages to/from in-vehicle ECU.

665968 v1 White lists in certificate

Version history:
(1):

Purpose: Define white lists to extend access rights of the certificates.
Source:
Verification Method: Analysis
Legacy ID: 654308v1

The white lists of diagnostic services or diagnostic data inside certificate with extended field will be used in further consideration.

4.10.8.2 Authentication with PKI Certificate Exchange

•Introduction

4.10.8.2.1 Introduction

The purpose of Authentication is to provide a means for the client and server to prove their identify, allowing it to access data and/or diagnostic services, which have restricted access for, for example security, emissions, or safety reasons. For example, Diagnostic services for downloading/uploading routines or data into a server and reading specific memory locations from a server are situations where authentication may be required.

This security concepts is based on PKI certificate exchange procedures using asymmetric cryptography. As certificate format, X.509 according to ISO/IEC 9594-8, RFC5280 and RFC5755 shall be used.

•Definitions and Abbreviations

4.10.8.2.2 Definitions and Abbreviations

Definition	Description
Authentication	Authentication is the process of assuring and verifying authenticity.
Authenticity	The property of being genuine and being able to be verified and trusted.
Digital Signature	Used to assure originator authenticity and data integrity (of a software)
Hash	Hash algorithms are also called digital fingerprinting algorithms. They are irreversible functions that provide a fixed-size hash based on various inputs. Irreversibility and collision resistance are necessary attributes for successful hash functions. Examples of hash functions are Secure Hash Algorithm 2
Integrity	The property that specified information remains unchanged between observations
Challenge	Specific formatted data (likely containing randomized information) or a random number.

Table - Identification Authentication Definitions

Abbreviation	Description
ECU	Electronic Control Unit
PKI	Public Key Infrastructure. See definition of Trust Centre
PKIX	Public Key Infrastructure X.509
OCSP	Online Certificate Status Protocol
ECC	Ellipse Curve Ctyptograhpy

Table - Identification Authentication Abbreviations

•Requisite documents

4.10.8.2.3 Requisite documents

The following documents are either referenced by this specification, or contain information that is relevant to this specification.

Source	LC in Elektra
Internal	UDS Services
External	ISO/IEC 9594-8
External	RFC5280

External	RFC5755
External	ISO/IEC 9798-3
External	RFC6960
External	SEC 2: Recommended Elliptic Curve Domain Parameters, January 27, 2010, Version 2
External	RFC3279

•Authentication Process Description

4.10.8.2.4 Authentication Process Description

702544 v1 **Authentication method #1**

Version history:
(1): Add authentication configuration for GEEA3.0.

Purpose: State the authentication method and parameter.
Source:
Verification Method: Test
Legacy ID:

The authenticaiton method #1 shall be Unidirectional authentication. The certification process , request, and response must be consistent with the requirements.

Authentication method #1 parameters:

- communicationConfiguration: 0x00, no secure diagnostic communication.
- lengthOfChallengeClient: 0x0000, no client challenge to send.
- EphemeralPublicKeyClient: no key agreement, meaning lengthOfEphemeralPublicKeyClient shall be set to 0x0000.
- sessionKeyInfo: no key agreement indicates that there will be no calculated information, meaning lengthOfSessionKeyInfo shall be set to 0x0000.

4.10.8.2.4.1 Unidirectional authentication

The process below is a unidirectional authentication.

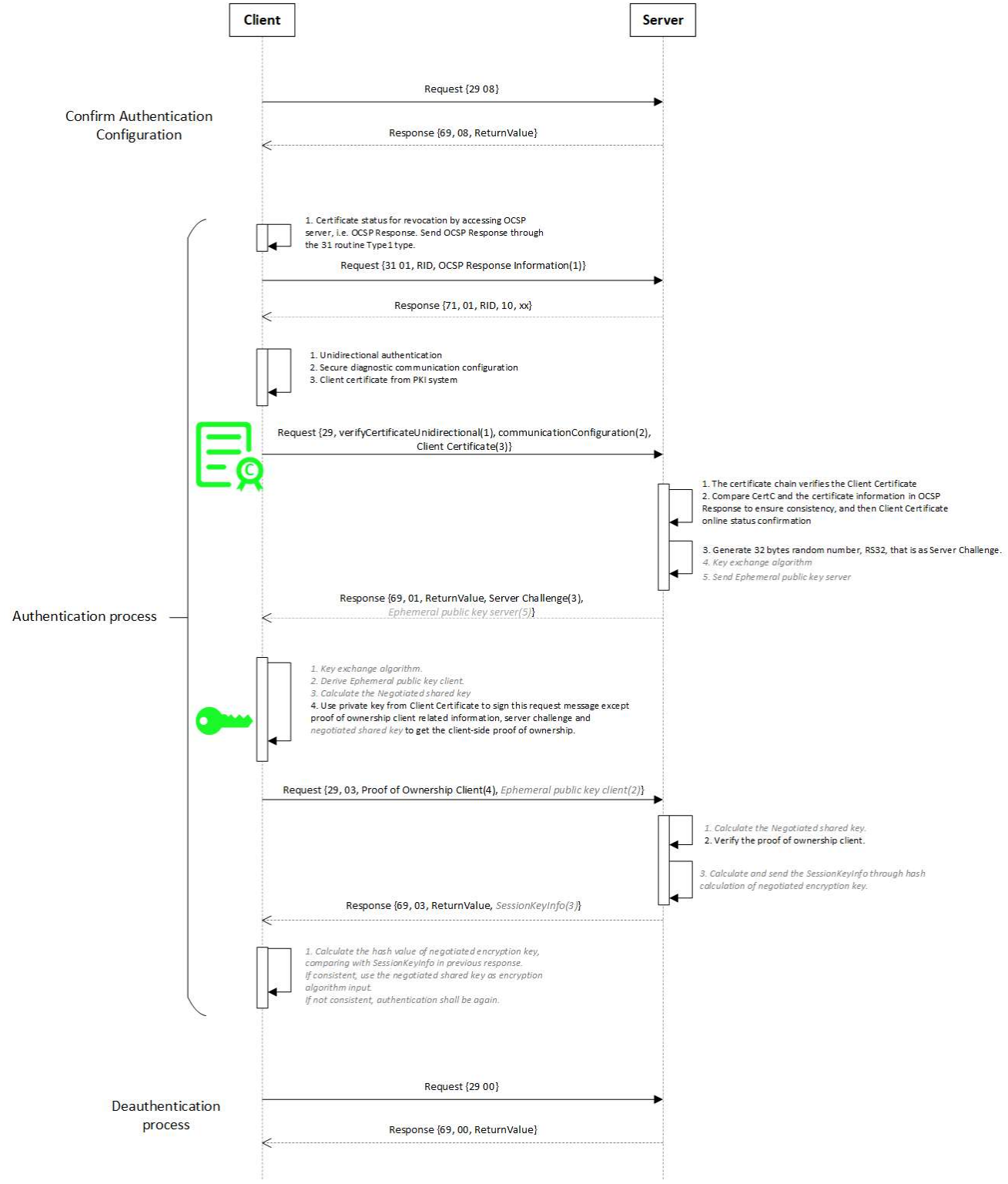


Figure - Unidirectional authentication sequence

Authentication process:
(0) Firstly, Client gets certificate which belongs to it and gets the OCSP response information. Client will send the OCSP response information and certificate to Sever through routine with RID 0x7027.
(i) Client sends certificate to Server.
(ii) Server verifies the receiving client certificate whether is legal and revocation through certificate chain stored in

Server. This can be done either during the progress of the routine, or after receiving certificate via service 29. If it is checked during routine progress, Server shall also check the certificate in service 29 request is same as certificate send in routine request.
(iii) Server creates its challenge, generates ephemeral private/public key, then responds to Client.
(iv) Client generates client ephemeral private/public key, and calculates signature as its proof of ownership, then sends to Server.
(v) Server verifies the proof of ownership client, and calculates new session key.
(vi) Server responds the result to Client.
(vii) Client confirms the result.

Note: The part described above in gray italic font will be implemented if secure diagnostic communication is used.

4.10.8.2.4.2 Bidirectional authentication

The process below is a bidirectional authentication.

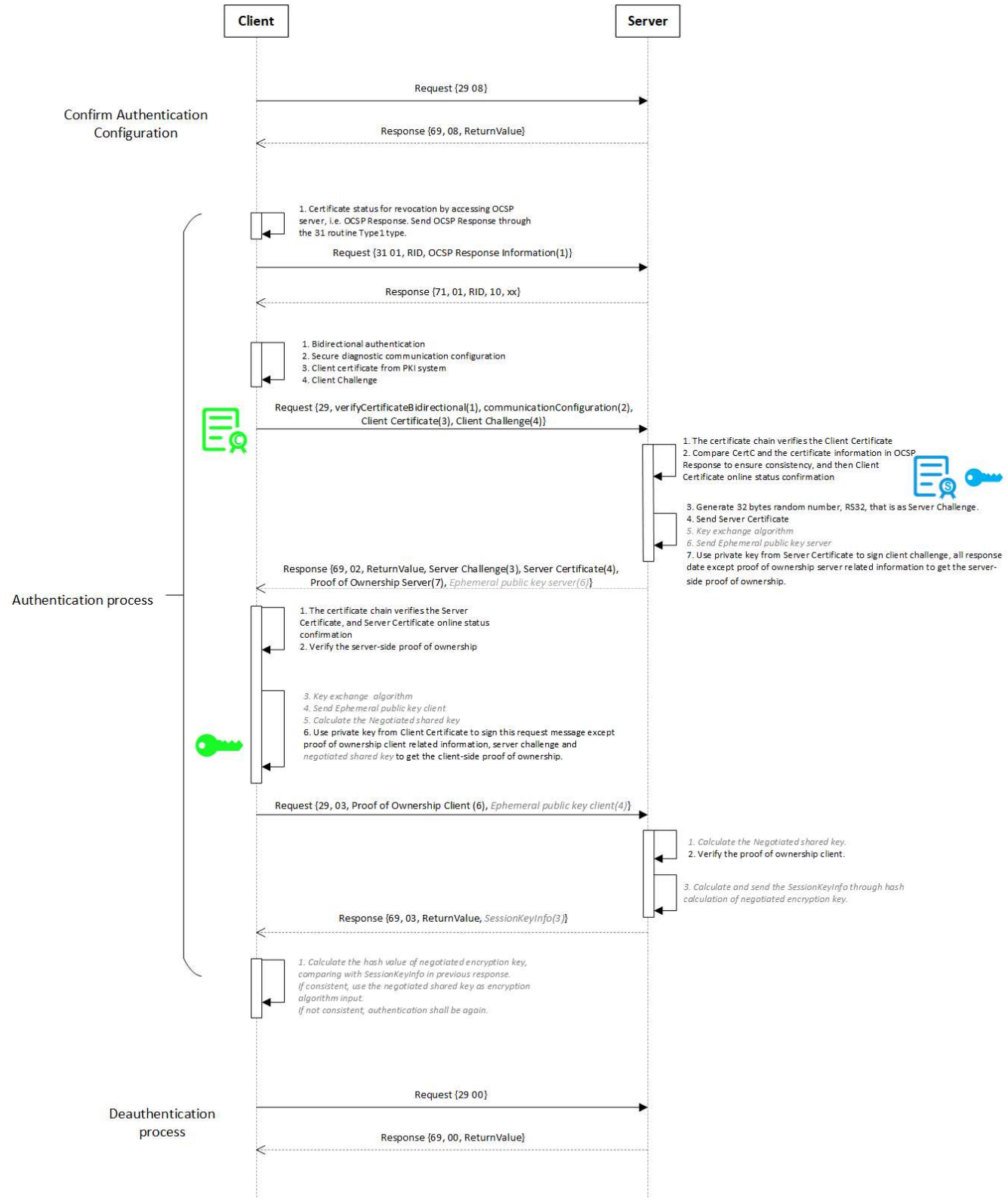


Figure - Bidirectional authentication sequence

Authentication process:
(0) Firstly, Client gets certificate which belongs to it and gets the OCSP response information. Client will send the OCSP response information and certificate to Sever through routine with RID 0x7027.

(i) Client sends certificate and client challenge to Server.
(ii) Server verifies the receiving client certificate whether is legal and revocation through certificate chain stored in Server. This can be done either during the progress of the routine, or after receiving certificate via service 29. If it is checked during routine progress, Server shall also check the certificate in service 29 request is same as certificate send in routine request.
(iii) Server creates its challenge, generates ephemeral private/public key and calculates signature as its proof of ownership.
(iv) Server responds to Client, carrying server challenge, server certificate, proof of ownership server and server ephemeral public key
(v) Client verifies the receiving server certificate whether is legal and revocation through certificate chain, as well as proof of ownership server.
(vi) Client also generates client ephemeral private/public key, and calculates signature as its proof of ownership, then sends to Server.
(vii) Server verifies the proof of ownership client, and calculates new session key.
(viii) Server responds the result to Client.
(ix) Client confirms the result.

Note: The part described above in gray italic font will be implemented if secure diagnostic communication is used.
•Challenge Description

4.10.8.2.5 Challenge Description

661336 v1 **Client Challenge**

Version history:
(1):

Purpose: Define the client challenge.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Client shall use the 32 bytes random number as client-side challenge.

661335 v1 **Random Number Generation**

Version history:
(1):

Purpose: Random and pseudo-random numbers generated for cryptographic applications should be unpredictable.
Source:
Verification Method: Test
Legacy ID:

The 32 bytes random number generated by client and server must be a cryptographically secure random number, meaning that it's generated using a True Random Number Generator (TRNG) or Pseudo-Random Number Generator (PRNG). This implicit implies that the Random outputs must not be repeated after a reset, e.g. ECUReset.

661337 v1 **Server Challenge**

Version history:
(1):

Purpose: Define the server challenge.

Source:
Verification Method: Analysis AND Test
Legacy ID:

Server shall use the 32 bytes random number as server-side challenge.

•Certificate

4.10.8.2.6 Certificate

The X.509 certificates shall be version 3 and follow RFC 5280. Detailed description is out of scope of this basetech requirement, and is defined in cyber security requirement.

976119 v2 **Action on revoked or unknown status of certificates**

Version history:
(2):

Purpose: To be able to replace certificates
Source:
Verification Method: Analysis AND Test
Legacy ID:
Zeekr Requirement ID: 712678

If the certificate is revoked or if the status is unknown , it shall be possible to write a new certificate.

Note: This requirement is for tool supplier.

976115 v2 **Chain of trust**

Version history:
(2):

Purpose: Define trust anchor
Source:
Verification Method: Analysis
Legacy ID:
Zeekr Requirement ID: 712489

Certificates for service 29 shall be issued by OEM trust center chain of trust. Chain of trust shall be pre-configured by supplier.

661340 v5 **Check Client Certificate for revocation**

Version history:
(5): Adding the ability to activate or deactivate time validation through DID configuration.

Purpose: Considering that there is no ability and environment to access the internet for

Server.

Source:

Verification Method: Analysis AND Test

Legacy ID:

After receiving the authentication service with subfunciton verifyCertificateUnidirectional ($29 01) or subfunciton verifyCertificateBidirectional ($29 02), server shall also use the OCSP Response received by Server in previous routineControl command to check client certificate for revocation. If there is no OCSP response available, negative response code 0x24 shall be returned.

The OCSP response will be validated in the following general rules sequencely. If there is a conflict with CSRS document, please refer to the CSRS document.

- ECU extracts OCSP server certificate from OCSP response.
- The ECU uses a preset certificate chain to chain and verify the OCSP server certificate.
- ECU extracts the response data and signature information from OCSP response.
- ECU uses the public key of the OCSP server certificate to verify the response data and signature of the OCSP Response.
- After the ECU successfully verifies the OCSP response signature, ECU shall check the current time (e.g. UTC time, TiAndDateIndcn) is within the time period defined by the OCSP response's thisUpdate time and the configured validity period. ECU shall ignore the nextUpdate time even if the nextUpdate time is present. The default value of the configured validity period is set to 18 hours. Refer to section “OCSP Response Time Validation Confirmation”. **(Note: this check depending on the configuration from *ReqID 1037501*)**
- ECU extract the certificate status from the response data.

Except for the current time not within the validity period and the certificate status not being Good, NRC 0x57 is returned. All other OCSP Response verification issues are returned to NRC 0x22, i.e. the client certificate data in OCSP Response does not match the certificate of the authentication service.

Note: OCSP is an internet protocol used to obtain the revocation status of X.509 digital certificates, which is defined in RFC 6960.

985775 v3 **Obtain Certificate**

Version history:
(3): 增加Client需求。另，补充增加子增加双向认证verifyCertificateBidirectional的子功能。

Purpose: To send the certificate to server

Source:

Verification Method: Analysis AND Test

Legacy ID:

Zeekr Requirement ID: 713049

The client obtain the certificate and public-private key pair from cloud server, and sends certificate to server by service 29 with the Sub-Function parameter ‘verifyCertificateUnidirectional’ or 'verifyCertificateBidirectional.

661339 v4 **Parse and verify Client Certificate**

Version history:
(4): Adding the ability to activate or deactivate time validation through DID configuration.

Purpose: Legal client certificate shall be verified and stored by server.

Source:

Verification Method: Analysis AND Test

Legacy ID:

The server shall parse and verify the client certificate, including:

- Verify the certificate signature
- Check certificate chain of trust
- Check certificate type
- Check certificate format
- Check certificate content scope
- Check certificate for revocation
- Check certificate time period, i.e. use the time signal "TiAndDateIndcn" to compare. If this signal is lost or abnormal (e.g. signal data invalid) during the process, or the physical value year for this value is less than or equal to 2020, the time validation is allowed to pass. **(Note: this check depending on the configuration from *ReqID 1037501*)**

If the whole verification operation is successful, ECU shall implement the next step within internal software, otherwise specific negative response shall be returned.

661341 v2 Providing Server Certificate

Version history:
(2): 更新说明在29认证过程中证书的格式要求。

Purpose: In bidirectional authentication, it is necessary for server to share its certificate.
Source:
Verification Method: Test
Legacy ID:

In bidirectional authentication process, the ECU shall distribute its certificate (X509.3 certificate with binary DER encoding) to client in diagnostic communication process.

661342 v1 Server Certification verification

Version history:
(1):

Purpose: This is for Client-side to verify server certificate.
Source:
Verification Method: Analysis
Legacy ID:

Client will use CA chain of trust to verify server certificate and access OCSP server to confirm server certificate for revocation.

661338 v2 Storage of Client Certificate

Version history:
(2): 更新说明在29认证过程中证书的格式要求。

Purpose: The server receives the client certificate for general confirmation, e.g. certificate size.
Source:
Verification Method: Analysis
Legacy ID:

The ECU shall receive the client certificate (X509.3 certificate with binary DER encoding) from the diagnostic service request, and temporarily store the client certificate within dedicated module (e.g. Csm) on one connection waiting for further validation.

If client certificate is finally verified to be valid ECU shall store this legal client certificate until leaving the authenticated state, otherwise client certificate shall be discard.

•Proof of Ownership

4.10.8.2.7 Proof of Ownership

Proof of ownership shall be provided for identify authentication both client(Tester) and server(ECU).

976129 v2 **Authentication Successful**

Version history:
(2): 增加需求，适用Client。

Purpose: To get the access to server
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713444

After verifying that the signature is valid, now client has proven that it has ownership of a certificate that was signed by a higher level certificate authority.

661345 v1 **Proof of ownership client in bidirectional authentication**

Version history:
(1):

Purpose: Client calculates proof of ownership client by building an appropriate authentication token in bidirectional authentication.
Source:
Verification Method: Analysis
Legacy ID:

The Client will build the authentication token through using the client certificate private key to sign the combination of this request message except proof of ownership client related information (e.g. length), server challenge, client challenge, and negotiated shared key if secure diagnostic communication is used, then get the calculated the client-side proof of ownership.

661344 v3 **Proof of ownership client in unidirectional authentication**

Version history:
(3): 更新计算Client proof of ownership的入参。

Purpose: Client calculates proof of ownership client by building an appropriate authentication token in unidirectional authentication.

Source:

Verification Method: Analysis

Legacy ID:

The Client will build the authentication token through using the client certificate private key to sign the data "server challenge" and get the calculated the client-side proof of ownership.

661346 v1 **Proof of ownership client verification**

Version history:
(1):

Purpose: In any authentication mode, server shall verify proof of ownership client to check client identity.

Source:

Verification Method: Analysis AND Test

Legacy ID:

Server shall verify the client's proof of ownership data provide in the authentication service with sub-function equal to proofOfOwnership request defined in *[UDS Service]* using client certificate public key.

The data to be verified needs to distinguish the authentication mode, as specified in *ReqID 639906* and *ReqID 653922*.

661343 v1 **Proof of ownership server**

Version history:
(1):

Purpose: Server calculates proof of ownership client by building an appropriate authentication token in bidirectional authentication.

Source:

Verification Method: Test

Legacy ID:

The Server shall build the authentication token through using the server certificate private key to sign the combination of client challenge and all response message to previous request except proof of ownership server related information (e.g. length), then get the calculated server-side proof of ownership.

This requirement is not applicable for the unidirectional authentication, but for bidirectional authentication.

661347 v1 **Proof of ownership server verification**

Version history:
(1):

Purpose: This is for Client-side to verify proof of ownership server.

Source:
Verification Method: Analysis
Legacy ID:

Client will verify the server's proof of ownership data responded by server using server certificate public key when the bidirectional authentication mode is selected.

•Secure diagnostic communication

4.10.8.2.8 Secure diagnostic communication

661348 v1 **Secure diagnostic communciation**

Version history:
(1):

Purpose: State that the secure diagnostic communication shall be required when communication configuration is configured.
Source:
Verification Method: Test
Legacy ID:

Secure diagnostic communication is established between client and server if communication configuration in Authentication service defined in [UDS Services] is not set to zero.

4.10.8.2.8.1 Ephemeral Private/Public Key

In order to prevent the communication session from attacking, it is recommended to use key agreement algorithm with forward security, e.g. Diffie-Hellma key agreement. The public key will be calculated using the temporarily generated private key.

661351 v1 **Client Ephemeral Public Key**

Version history:
(1):

Purpose: Define the client public key.
Source:
Verification Method: Analysis
Legacy ID:

Client shall use the generated random number private key to derive client public key.

661349 v1 **Ephemeral Private Key**

Version history:
(1):

Purpose: Define the random generated private key to apply the key exchange algorithm with forward secrecy.
Source:
Verification Method: Analysis
Legacy ID:

When every authentication request, client and server shall generate their owner private key according to True Random Number Generator (TRNG) for further key agreement.

The generated private key shall be stored in a protected memory area to avoid accessing and tampering.

661352 v1 **Negotiated shared key**

Version history:
(1):

Purpose: Define the negotiated shared key for session encryption to protect diagnostic communication.
Source:
Verification Method: Test
Legacy ID:

Client and server shall calculate the shared key based on their own private key and the peer public key for session encryption.

661350 v2 **Server Ephemeral Public Key**

Version history:
(2): Verification Method: Analysis AND Test change to Analysis.

Purpose: Define the server public key.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Server shall use the generated random number private key to derive server public key.

4.10.8.2.8.2 Session Key Information

661353 v1 **Session Key Information**

Version history:
(1):

Purpose: Defined the calculation of session key information for confirmation by both side.
Source:
Verification Method: Analysis AND Test
Legacy ID:

It shall be a proof value using hash algorithm - SHA256 for the validation of the session key on the client side.

•Signing Methods

4.10.8.2.9 Signing Methods

4.10.8.2.9.1 Signing Method - Variability control

661354 v1 **Default Signing Method**

Version history:
(1):

Purpose: Define the default signing method. The ECU shall use the default signing method, unless else is explicitly defined elsewhere, e.g. in the LC : Diagnostics and ECU Platform.
Source:
Verification Method: Analysis
Legacy ID:

The default signing method shall be SHA256_ECDSA (NIST Curve: P-256) for authentication.

4.10.8.2.9.2 Signing Method - Algorithm Definitions

661356 v1 **Signing Method SHA256_ECDSA - Elliptic Curve**

Version history:
(1):

Purpose: Define the elliptic curve used.
Source:
Verification Method: Analysis
Legacy ID:

The Elliptic Curve shall be NIST Curve: P-256.

661355 v1 **Signing Method SHA256_ECDSA - Hash function**

Version history:
(1):

Purpose: Define the hash function.
Source:
Verification Method: Analysis
Legacy ID:

The hash function to be used is the SHA-256 (256 bits) as defined in *FIPS PUB 180-4, Secure Hash Standard (SHS), March 2012*.

711819 v1 **Signing Method SHA256_ECDSA - Signature format**

Version history:
(1): 新增需求说明Proof of ownership签名的格式。

Purpose: Define the format of generated signature. It is defined in the IEEE P.1363 standard.For DSA, the size in bytes of the signature is N/4 bytes (e.g. 64 for N=256). For ECDSA, the signature is always twice the length of a point coordinate (e.g. 64 bytes for P-256).

Source:
Verification Method: Analysis
Legacy ID:

The format of generated signature through SHA256_ECDSA algorithm shall be "binary", i.e. the raw concatenation of "r" and "s".

661357 v1 **Signing Method SHA256_ECDSA - key size**

Version history:
(1):

Purpose: Define the key size for the private key.
Source:
Verification Method: Analysis
Legacy ID:

The key size shall be 256 bits.

•Key Exchange Methods

4.10.8.2.10 Key Exchange Methods

4.10.8.2.10.1 Key Exchange Method - Variability control

661358 v1 **Default Key Exchange Method**

Version history:
(1):

Purpose: Define the default key exchange method. The ECU shall use the default key exchange method, unless else is explicitly defined elsewhere, e.g. in the LC : Diagnostics and ECU Platform.
Source:
Verification Method: Analysis
Legacy ID:

The default key exchange method shall be ECDHE (NIST Curve: P-256) for key agreement protocol.

4.10.8.2.10.2 Key Exchange Method - Algorithm Definitions

661359 v1 **Key Exchange Method ECDHE - Elliptic Curve**

Version history:
(1):

Purpose: Define the elliptic curve used.
Source:
Verification Method: Analysis
Legacy ID:

The Elliptic Curve shall be NIST Curve: P-256.

661360 v1 **Key Exchange Method ECDHE - Negotiated shared key**

Version history:
(1):

Purpose: Define the key related information for the negotiated shared key through key exchange method.
Source:
Verification Method: Analysis
Legacy ID:

The x-coordinate value from calculated point is as the shared key for session encryption. The shared key size shall be 256 bits.

•Encryption Methods

4.10.8.2.11 Encryption Methods

4.10.8.2.11.1 Encryption Method - Variability control

661361 v1 **Default Encryption Method**

Version history:
(1):

Purpose: Define the default encryption method. The ECU shall use the default encryption method, unless else is explicitly defined elsewhere, e.g. in the LC : Diagnostics and ECU Platform.
Source:
Verification Method: Analysis
Legacy ID:

Both side for client and server, the default encryption method shall be AES_GCM which is an algorithm with authentication and encryption.

4.10.8.2.11.2 Encryption Method - Algorithm Definitions

661362 v1 **Encryption Method AES_GCM - Key and IV size**

Version history:
(1):

Purpose: Define the key size and iv size.
Source:
Verification Method: Analysis
Legacy ID:

The key size shall be 128 bits. The iv size shall be also 128 bits.

661363 v1 **Encryption Method AES_GCM - addtional authenticated data**

Version history:
(1):

Purpose: Define the additional authenticated data used for AES_GCM.
Source:
Verification Method: Analysis
Legacy ID:

The additional authenticated is not used in this encryption method, and shall be NULL.

4.10.8.2.11.3 Encryption Method - Additional requirements

661365 v1 **Encryption Method AES_GCM - Data arrangement**

Version history:
(1):

Purpose: Define the layout of the ciphertext and massage authentication code, in order to extract and validate the real value.
Source:
Verification Method: Analysis
Legacy ID:

Ciphertext and message authentication code - tag will always be obtained after calculation via AES_GCM algorithm.

The final encrypted message will consist of ciphertext and vmessage authentication code - tag which follows ciphertext closely.

661364 v1 **Encryption Method AES_GCM - Key and IV**

Version history:
(1):

Purpose: Define the key and iv.
Source:
Verification Method: Analysis
Legacy ID:

The negotiated shared key from key exchange method need to be 256 bit. Its first 128 bits are applied to the KEY, and the last 128 bits are applied to the IV when AES_GCM algorithm is used.

•Configuration of the authentication of Protection

4.10.8.2.12 Configuration of the authentication of Protection

661366 v2 **Default values - Delay timers and counters for false attempts**

Version history:
(2): Delay Timer延时由10s增加到60s。

Purpose: To define default values for the parameters used to configure the authentication of protection
Source:
Verification Method: Test

Legacy ID:

The default values shall be set to:
- Number of false authentication attempt before a delay time is activated: 2.
- Delay Timer activated after number of false authentication attempts (T_1): 60 seconds.

ECUs having the proper hardware capabilities shall store (in non-volatile memory) the information and whether the delay timer after number of false attempts has been activated/completed or not, i.e. that information shall not be lost at a reset, in order to determine whether to restart the delay timer (T_1) after initialization.

After delay timer has been activated or a successful authentication has been completed, the number of false authentication attempt shall be set to zero.

•Examples

4.10.8.2.13 Examples

4.10.8.2.13.1 Key exchange example

The purpose of this example is to show how to calculate the shared key on both sides. Client and server randomly generated elliptic curve private key and public key derived from private key based on elliptic curve P-256, are as follows.

Client:

```
Private-Key: (256 bit)
priv:
    0f:83:66:e0:88:6c:0e:a8:a4:81:27:0f:bf:16:b1:
    0f:58:df:1a:dc:db:95:5e:70:bf:fc:d9:fd:0c:db:
    8d:89

Public-Key: [Public point (Uncompressed)]
pub:
    04:d0:00:17:29:97:98:d9:04:aa:41:35:3b:ab:7d:
    73:d7:21:c1:66:cc:4c:87:78:fc:a7:30:82:d6:fd:
    01:24:a3:da:f3:58:f8:cd:72:50:1a:09:bd:a4:91:
    aa:aa:41:2d:fb:4a:bd:3b:7d:4f:c6:5b:c8:4c:30:
    73:eb:67:27:90
```

Server:

```
Private-Key: (256 bit)
priv:
    45:a0:7f:c8:e3:bb:5a:da:78:c2:b5:3d:ff:e0:21:
    e9:d4:71:83:66:dc:e0:a7:72:26:d2:38:0c:05:a3:
    9f:f2

Public-Key: [Public point (Uncompressed)]
pub:
    04:8e:4a:d9:1b:bf:4f:c6:75:48:a0:96:00:23:2e:
    48:db:ab:e5:11:eb:8e:58:48:56:90:0a:03:51:73:
    56:4b:17:a3:13:ba:2f:37:b3:7c:a0:ee:bc:b8:29:
    86:2e:ce:c4:4f:ca:70:d7:6b:72:df:c2:6f:91:0d:
    69:73:9e:60:f8
```

Shared Key:

```
Client = client_private_key * server_public_key =
0x072c553a866c1332fb621400b560cb227104407bed6e3e3695eac4990a44dde3

Server = server_private_key * client_public_key =
0x072c553a866c1332fb621400b560cb227104407bed6e3e3695eac4990a44dde3
```

4.10.8.2.13.2 Encrpytion of UDS Data example

The purpose of this example is to show how to encrypt UDS data plaintext into ciphertext with message authentication code.

Assuming UDS plaintext:

```
Diagnostic request = 22 f1 86
Diagnostic response = 62 f1 86 03
key = 53 b2 12 16 f6 13 dd d0 6e 14 48 39 63 7f 0f 86
iv = 45 7d 08 64 d9 a3 b7 2a 9b dd 6c 8c 8c bf f7 9c
```

After calculation, ciphertext with GMAC:

```
Encrypted Diagnostic request = 01 fa f3
tag = f1 1a 8a 70 20 13 d1 4a 46 e4 56 29 90 f8 af c9

Encrypted Diagnostic response = 41 fa f3 af
tag = c5 8d 5a 47 25 00 12 31 85 d5 25 35 51 8d 1f 62
```

4.10.8.2.13.3 Proof of ownership server in bdirectional authentication example

The purpose of this example is to show how to calulate the server-side proof of ownership in **bidirectional authentication**, when SHA256_ECDSA is used, and no secure diagnostic communication is configured (e.g. the parameter communication configuration is set to 0x00).

Note: server proof of ownership, as a calculated signature, and is different every time when it is generated.

```
Server proof of ownerhip = ECDSAwithSHA256 (client challenge || Server response
message except proof of ownership server related information)

= ECDSAwithSHA256 ([1F 76 9F 37 F2 00 12 F8 AD FC FD A3 58 E7 5E E0 E0 30 CA E6 1C
D3 99 A1 F7 0C B5 59 0A 47 7B F3] 69 02 [11] [00 20] [EB A0 A6 65 1F 55 47 39 C5 23
C2 72 AA 2F EF BE C2 68 84 79 81 1F 80 52 2E 78 75 90 06 22 EC 8D] [03 5B] [30 82 03
57 30 82 02 FC A0 03 02 01 02 02 11 00 F4 B6 AF D2 C4 09 6D 66 5B A1 F6 .. .. .. D3
66 8B 2D 5E 18 0F 9D 90 7A CF ED 27 17 A3 C8 0B 15 58 74 51 AD 71 BB 8E 56 D2 4A 54
12 B6 41] [00 00])
```

Note: the ellipsis dots aboved represents the server certificate related information.

4.10.8.2.13.4 Proof of ownership client in unidirectional authentication example

The purpose of this example is to show how to calulate the client-side proof of ownership in **unidirectional authentication**, when SHA256_ECDSA is used and no secure diagnostic communication is needed (e.g. the parameter communication configuration is set to 0x00)

Note: client proof of ownership, as a calculated signature, and is different every time when it is generated.

```
Client proof of ownerhip = ECDSAwithSHA256 (Server challenge)

= ECDSAwithSHA256 (1F 76 9F 37 F2 00 12 F8 AD FC FD A3 58 E7 5E E0 E0 30 CA E6 1C D3
99 A1 F7 0C B5 59 0A 47 7B F3)
```

4.10.8.2.13.5 OCSP Response Time Validatation Confirmation

If Certificate status is good, the server shall check the current time (i.e. UTC time and TiAndDateIndcn) is within the time period defined by the OCSP response's thisUpdate time and the configured validity period. Sever shall ignore the nextUpdate time even if the nextUpdate time is present.

The ECU shall support the following items to determine whether the OCSP Response This Update is within the range.

- Client can use DID 0x9007 to read the UTC time from the ECU.
- ECU may store the UTC time after completing an operation cycle.
- If UTC Time cannot be obtained or UTC is invalid or the physical value year for UTC time is less than or equal to 2020, the time validation is allowed to pass.

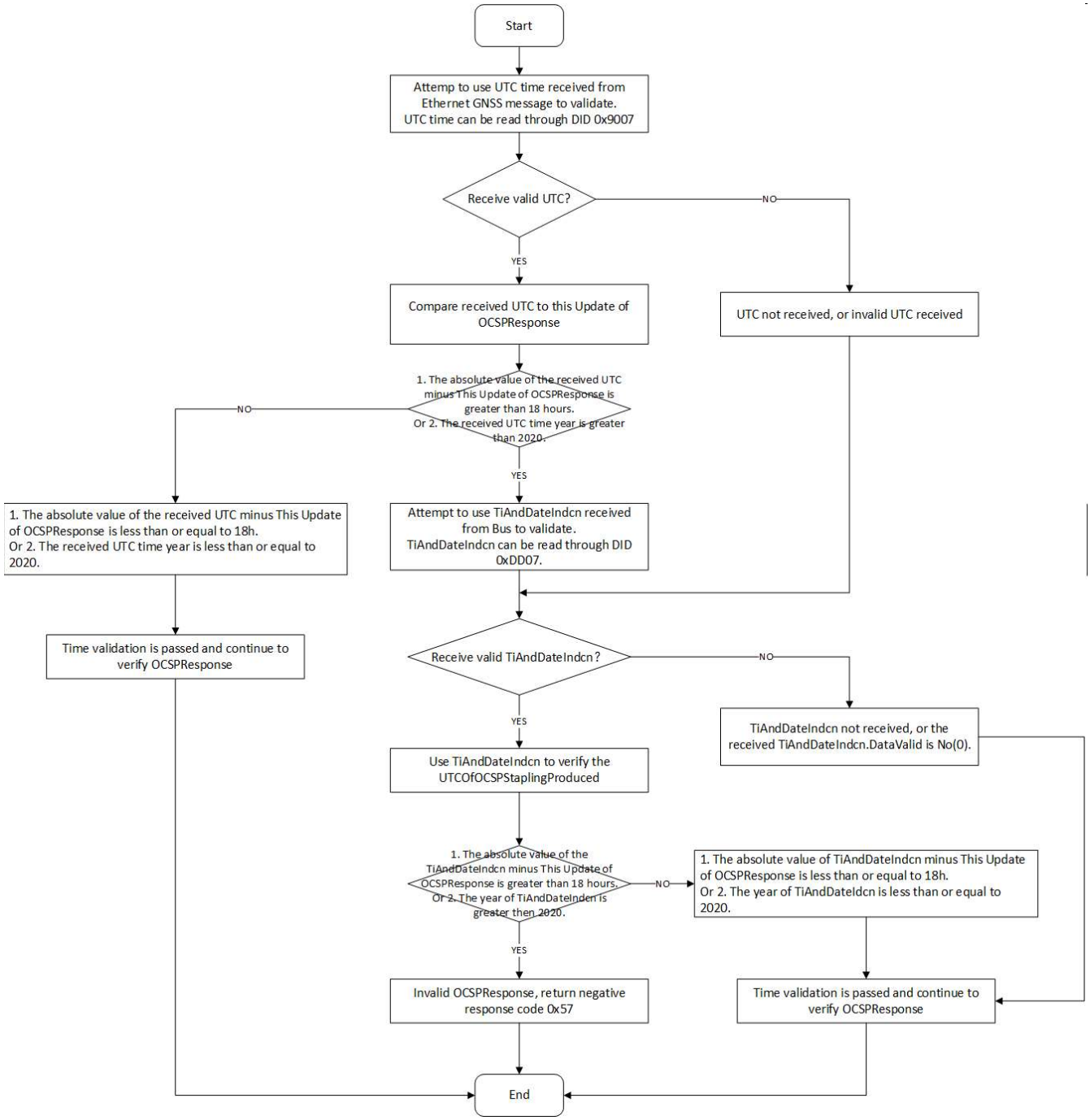


Figure — Overview of Time Validatation Confirmation

4.10.9 Regulatory certification based on UDS

4.10.9.1 Scope

4.10.9.2 ZEVonUDS

•Introduction

4.10.9.2.1 Introduction

SAE J1979-3 specifies diagnostic services required to be supported by motor vehicles and external test equipment

for diagnostic purposes which pertain to **Zero Emission Vehicle (ZEV)** propulsion related data.
ZEVonUDS provides an implementation guidance based on the principles of ISO 14229, focusing on standardization of data to ensure access to important repair information by independent repair facilities. The purpose of this standard is to focus on providing the necessary content and performance requirements for ZEV powertrain communications. ZEVonUDS diagnostic requirements can be satisfied over either CAN or Ethernet datalinks.
Scope of application of the regulations: Overseas regions such as California of the United States, South Korea, and Canada. The specific implementation will be subject to the notification of the project regulations.
•ZEVonUDS Diagnostic Link Definition

4.10.9.2.2 ZEVonUDS Diagnostic Link Definition

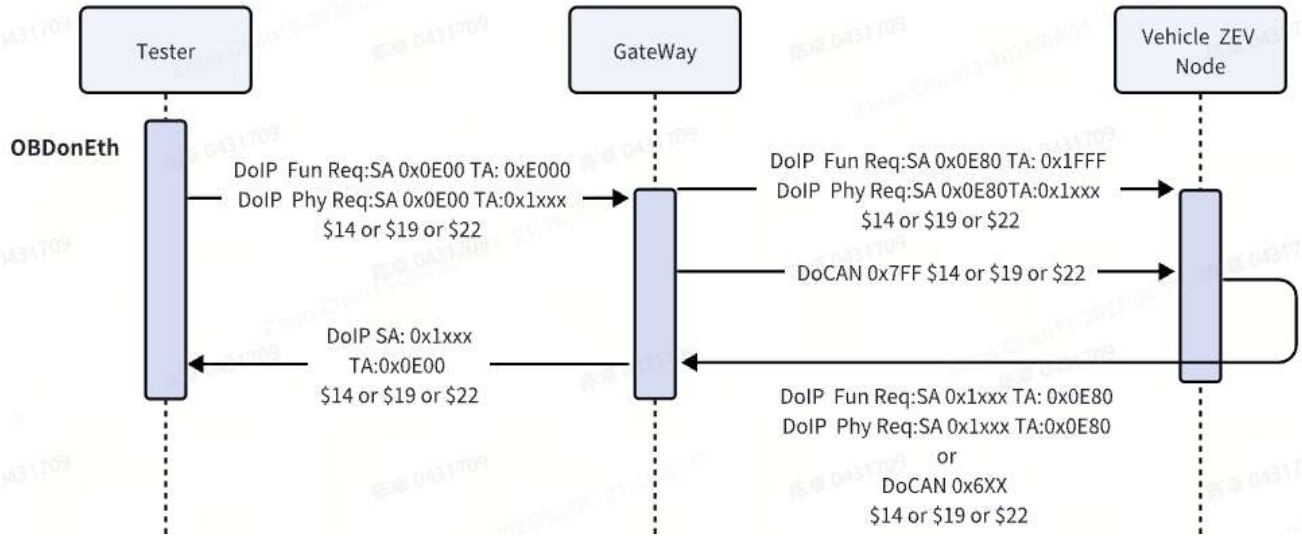
The ZEV specification defines that CAN or Ethernet can be used. Considering the number of nodes and the requirement of the response time of the specification service,Platform defines Ethernet links at present

857568 v2 **Diagnostic communication requirements after link confirmation**

Version history:
(2): Add a demand description, and do not force Geely's overseas cooperation projects to use it.

Purpose: Detailed rules for gateway address translation are defined.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The address conversion logic is only applicable to the Geely platform project. It is recommended to develop it in a platform-based manner. If there are any cooperation projects that do not require the implementation of this plan, they can also be accepted. Carry out according to the specific requirements of the project.
Considering a universal platform solution for diagnostic addresses, design an address translation scheme: For requirement 857565, step 4, when the route between the vehicle and the ZEV certification device is activated, the gateway needs to remember this scenario and meet the following requirements:
1. Translate the external diagnostic addresses and diagnostic requests into OEM-defined addresses and requests, and broadcast or unicast them within the vehicle.
2. Conversely, when receiving diagnostic responses from in-vehicle nodes, the gateway should support translating them into the corresponding ZEV addresses and send them to the external device.



Function Addressing Scenario:

Function address request(DoIP→DoIP)		
ZEV Tester request	Gateway Switch	In-Vehicle Command
SA:0x0E00		**SA:0x0E80**

Function address:**0xE000**		Function address:**0x1FFF**
Function address Respond(DoIP→DoIP)		
ZEV Node Respond	Gateway Switch	Out-Vehicle Command
SA:0x1xxx		SA:0x1xxx
TA: **0x0E80**		TA: **0x0E00**

Function address request(DoIP→DoCAN)		
ZEV Tester request	Gateway Switch	In-Vehicle Command
SA:0x0E00		DiagCAN:**0x7FF**
TA:function address:**0x0E00**		
Function address Respond(DoCAN→DoIP)		
ZEV Node Respond	Gateway Switch	Out-Vehicle Command
DiagCAN:**0x6xx**		SA:0x1xxx
		TA: **0x0E00**

In this scenario, the gateway needs to convert the source address(0x0E00) of the received ZEV device into a device address defined(0x0E80) by Geely.
At the same time, it is necessary to support the conversion of external function addressing(0xE000) commands to 0x1FFF and 0x7FF and route them to the Eth node and CAN node.
Conversely, support is also required.

Physical Addressing Scenario:

Physical address Respond(DoIP→DoIP) ADCU-Eth address as an example		
ZEV Tester request	Gateway Switch	In-Vehicle Command
SA:0x0E00		SA:0x0E80
TA: 0x1301		TA:0x1301

Physical address Respond(DoIP→DoIP)		
ZEV Node Respond	Gateway Switch	Out-Vehicle Command
SA:0x1301		SA:0x1301
TA: 0x0E80		TA: 0x0E00

Physical address request(DoIP→DoCAN) BCM-CAN address as an example		
ZEV Tester request	Gateway Switch	In-Vehicle Command
SA:0x0E00		DiagCAN:0x781
TA:0x1D81		

Physical address Respond(DoCAN→DoIP)		
ZEV Node Respond	Gateway Switch	In-Vehicle Command
DiagCAN:0x681		SA:0x1D81
		TA:0x0E00

In this scenario, the gateway needs to convert the source address(0x0E00) of the received ZEV device into a device address defined(0x0E80) by Geely.Conversely, support is also required.

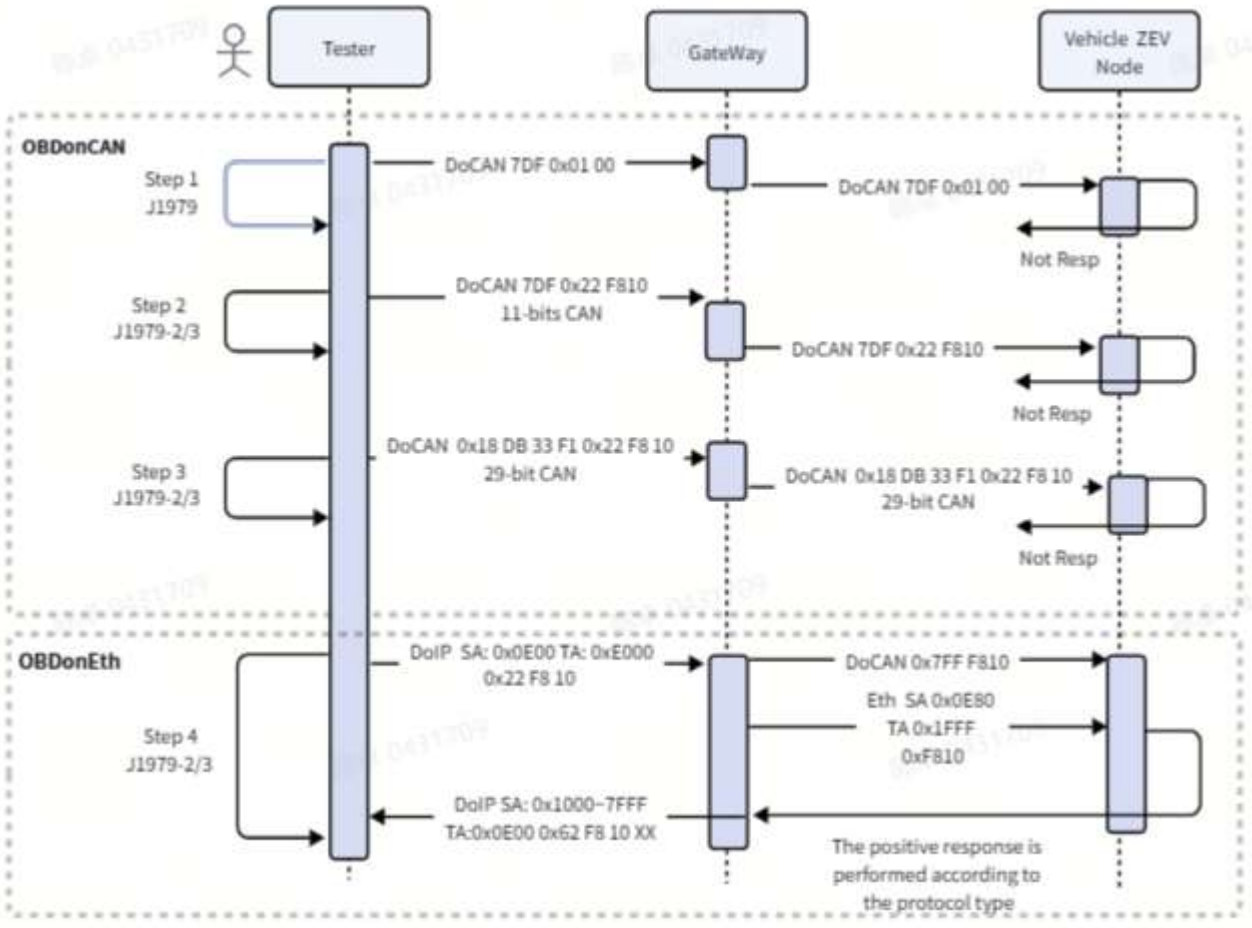
1030394 v2 **Regulation link identification process requirment-ECU**

Version history:
(2): In step 1, the instruction sent was corrected. The values 0x7DF 0x10 00 were changed to 0x7DF 0x01 00.

Purpose: Introduce the method for identifying the diagnostic link by the certification device, and define the different processing required by the gateway for different vehicle models.

Source:
Verification Method: Analysis AND Test
Legacy ID:

According to the 1979-3 regulations, external devices need to identify whether the vehicle's diagnostic link is CAN or Eth. The specific process is roughly as follows: the external device will broadcast different protocol addresses within the vehicle to determine the specific link. Currently, the Geely Research Institute specifies that the ZEV link should use Ethernet. The process is illustrated in the following diagram.



The diagnostic link mentioned above only supports one scenario on a vehicle that will not coexist.
The ZEV nodes involved cannot provide any positive or negative feedback in steps 1 to 3.

	Request message	Except ZEV ECU responds
Step1	0x7DF 0x01 00	No Respond
Step2	0x7DF 0xF8 10	No Respond
Step3	0x18 DB 33 F1 0x22 F8 10	No Respond
step4	CAN:0x7FF 0x22 F810 FR&Eth：SA 0x1FFF TA :0x0E80 0x22 F810	Positive Respond

857889 v1 **ZEV logical address assignment**

Version history:
(1):

Purpose: These addresses ensure proper communication and compliance with ZEV standards.
Source:
Verification Method: Analysis AND Test
Legacy ID:

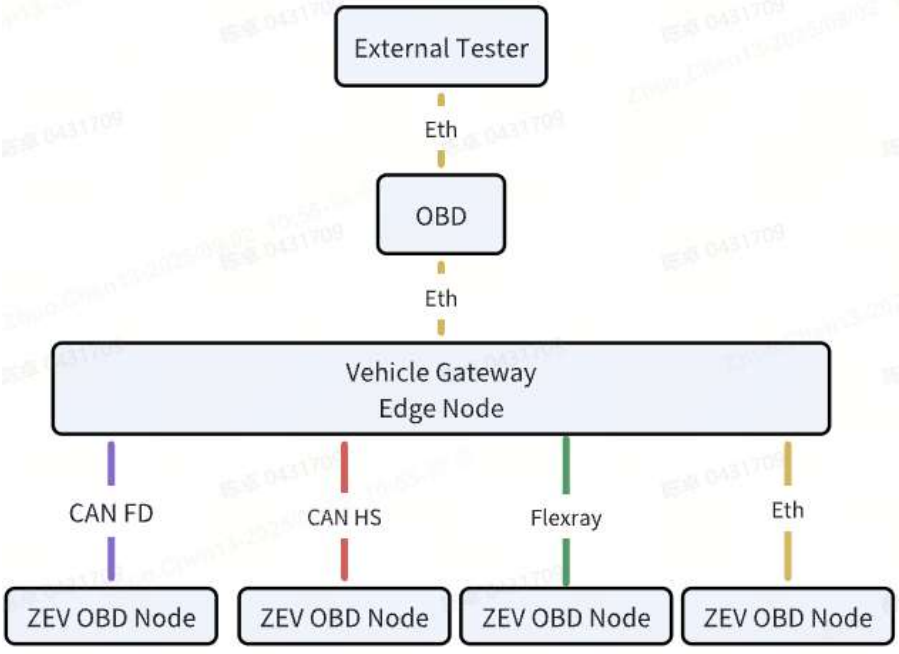
Addressing Message Type	Target Address (TA)	Source Address(SA)	Description
Functional request	0xE000	0x0E00	SA = External test equipment TA = all ZEV-relevant server(s)/ECU(s)
Physical request	0x1000~0x7FFF	0x0E00	SA = External test equipment TA = one ZEV-relevant server/ECU
Physical response	0x0E00	0x1000~0x7FFF	SA = one ZEV-relevant server/ECU TA = External test equipment

857549 v2 **ZEVonUDS base on Ethernet**

Version history:
(2): Adapt to all current bus types involved, and add FlexRay support for other platforms

Purpose: Currently, for the ZEV (Zero Emission Vehicle) link, we have chosen the DoIP (Diagnostic over Internet Protocol) link.
Source:
Verification Method: Analysis
Legacy ID:

The ZEV specification defines that CAN or Eth can be used. Considering the number of nodes and the requirement of the response time of the specification service, Platform defines Ethernet links at present.



•Genenral Requirments

4.10.9.2.3 Genenral Requirments

857676 v2 **Requirements for the hardware attributes of the OBD interface**

Version history:
(2): Update the "Purpose" information

Purpose: The Ethernet interface of the gateway must comply with ISO 13400-3
Source:
Verification Method: Analysis
Legacy ID:

The interface of the gateway must meet the requirements of 13400-3 2016.Ver Wired vehicle interface based on IEEE 802.3 .

857678 v2 ZEVonUDS DoIP routing acivation type

Version history:
(2): Add "Purpose" information

Purpose: Define the routing activation type
Source:
Verification Method: Analysis AND Test
Legacy ID:

According to regulatory requirements,The gateway must support therouting activation type 0x01.

0x01 is the only routing activation within scope of this document and has priority over other activation types.

•ZEVonUDS Services Application

4.10.9.2.4 ZEVonUDS Services Application

The aforementioned services only meet the minimum requirements for regulatory certification. In addition to meeting these minimum requirements, the ECU also needs to comply with the basic requirements of base technology. If there are any parts of the regulations that conflict with the design of basetech, then the regulations should take precedence and contact the basetech team.

857858 v1 ZEV Propulsion ClearDiagnosticInformation (0x14)

Version history:
(1):

Purpose: The service is used by the client to clear diagnostic information in one or multiple servers' memory.
Source:
Verification Method: Analysis AND Test
Legacy ID:

In addition to the basic service functions that meet the basetech requirements, it is also necessary to support additional diagnostic sub-service requirements, although these additional ones also comply with ISO 14229-1.

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
ClearDiagnosticInformation	N/A	GroupOfDTC All DTCs in zero emission	FFFF33	MC1	M	D	M	M	N	N	5000ms

(0x14)		related ECU									
		GroupOfDTC All Groups	FFFFFF	M	M	D	M	M	N	N	

Note:

- Due to regulatory requirements, the diagnostic tasks for ZEV must not be intercepted by any certification($29 & $27). Therefore, when the gateway identifies Tester SA: 0x0E00 sending $14, the gateway must not intercept it.

Entry conditions:

- Entry conditions for service ClearDiagnosticInformation (0x14) are allowed only if approved by GRI Electrical Architecture.

If the ECU implement safety requirements with an ASIL higher than QM (see *[12] ISO 26262)* it shall, in all situations when diagnostic services may violate any of those safety requirements, reject the critical diagnostic service requests.But that require an approval by GRI Electrical Architecture.

All ZEV ECUs shall keep the same entry conditions. When at least one of these conditions is not met, the ECU shall reject the request message with a negative response code 0x22.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- The service clearDiagnosticInformation (0x14) shall not affect the ECU's ability to execute non diagnostic tasks.
- MC1=Mandatory if the ECU is Zero emission related

857688 v1 ZEV Propulsion ReadDTCInformation (0x19)

Version history:
(1):

Purpose: This service allows a client to read the status of server resident Diagnostic Trouble Code(DTC) information from any server, or group of servers within a vehicle.

Source:

Verification Method: Analysis AND Test

Legacy ID:

In addition to the basic service functions that meet the basetech requirements, it is also necessary to support additional diagnostic sub-service requirements, although these additional ones also comply with ISO 14229-1.

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
ReadDTCInformation (0x19)	Read DTC Information Report WWHOBD DTC By Mask Record(42)	DTCStatusMask		MC	MC	D	M	M	N	N	500ms
		FunctionalGroupIdentifier	33								
		DTCSeverityMask	02								

Note:

- Due to regulatory requirements, the diagnostic tasks for ZEV must not be intercepted by any certification($29 & $27). Therefore, when the gateway identifies Tester SA: 0x0E00 sending $19, the gateway must not intercept it.

Entry conditions:

- Entry conditions for service ReadDTCInformation (0x19) are not allowed.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- The service ReadDTCInformation (0x19) shall not affect the ECU's ability to execute non diagnostic tasks.
- MC=Mandatory if the ECU is Zero emission related
- According to the ZEV regulation requirements, the emission node must support DTCs in the 0x04 format as specified in SAE-J2012. These DTCs must not only support service 1942 but also other sub-functions required by basetech for service 19, such as 1902/1903, etc. Additionally, they must support basetech's basic snapshot functionality.
- If a ZEV DTC function is similar to a basetech-defined DTC function, the basetech DTC must not be replaced with the ZEV DTC; the basetech DTC must **not be altered or deleted** by any regulation.
- The DTC fault code for ZEV must meet the SAE J2012DA 0x04 format requirements.

857861 v2 ZEV Propulsion ReadDataByIdentifier (0x22)

Version history:
(2): Remove the length limit (61 bytes) for responses to multiple DID requests

Purpose: The service allows the client to request data record valued from the server identified by one or more dataIdentifiers.

Source:

Verification Method: Analysis AND Test

Legacy ID:

In addition to the basic service functions that meet the basetech requirements, it is also necessary to support additional diagnostic sub-service requirements, although these additional ones also comply with ISO 14229-1.

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
ReadDataByIdentifier(0x22)	N/A	DataIdentifier		M	M	MPS	M	M	N	N	200ms

Note:

- The DID requirements for ZEV, which originate from SAE J1979DA, have a range of 0xF400 to 0xF8FF.
- Due to regulatory requirements, the diagnostic tasks for ZEV must not be intercepted by any certification($29 & $27). Therefore, when the gateway identifies Tester SA: 0x0E00 sending $22 0xF400~0xF8FF , the gateway must not intercept it.

Entry conditions:

- Entry conditions for service ReadDataByIdentifier (0x22) are not allowed.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- Minimum 10 dataIdentifiers, or as many dataIdentifiers as implemented, in one single ReadDataByIdentifier request shall

be supported by the ECU in default and extended diagnostic session.

- The service readDataByIdentifier (0x22) shall not affect the ECU's ability to execute non diagnostic tasks.

857865 v1 **ZEV Propulsion RoutineControl (0x31)**

Version history:
(1):

Purpose: The service is used by the client ti execute a defined sequence of steps and obtain any relevant results.

Source:

Verification Method: Analysis AND Test

Legacy ID:

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
RoutineControl (0x31)	StartRoutine (01)	RoutineIdentifier		M	M	MPS	M	M	N	N	5000ms
	StopRoutine (02)	RoutineIdentifier		O	O	OPS	M	O	N	N	200ms
	RequestRoutineResults (03)	RoutineIdentifier		O	O	OPS	M	O	N	N	

Note:

- Due to regulatory requirements, the diagnostic tasks for ZEV must not be intercepted by any certification($29 & $27). Therefore, when the gateway identifies Tester SA: 0x0E00 sending $31 0xE000 to 0xE1FF , the gateway must not intercept it.

Entry conditions:

- Entry conditions for service RoutineControl (0x31) are allowed only if approved by GRI Electrical Architecture.

If the ECU implement safety requirements with an ASIL higher than QM (see *[13] ISO 26262)* it shall, in all situations when diagnostic services may violate any of those safety requirements, reject the critical diagnostic service requests.But that require an approval by GRI Electrical Architecture.

When at least one of these conditions is not met, the ECU shall reject the request message with a negative response code 0x22.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- The service routineControl (0x31) is allowed to affect the ECU's ability to execute non diagnostic tasks. The service is only allowed to affect execution of the non-diagnostic tasks during the execution of the diagnostic service. After the diagnostic service is completed any effect on the non-diagnostic tasks is not allowed anymore (normal operational functionality resumes). The service is not allowed to reset the ECU .

•ZEVonUDS Data security

4.10.9.2.5 ZEVonUDS Data security

838556 v3 **ZEVonUDS Data protection**

Version history:
(3): Add the ZEV scenario to only allow the routing of 1942 sub-functions

Purpose: The gateway needs to identify diagnostic messages for the Ethernet link.
Source:
Verification Method: Analysis AND Test
Legacy ID:

If the gateway supports the correlation diagnostics encryption design (TLS,$29,$31.etc), ZEV diagnostics are not affected by the encryption design when external regulatory certified ZEV device**(SA 0x0E00)** read the relevant ZEV information from the gateway.
Combined with the J1979-3 regulations of the service application and the above requirements, gateway only allows routing service:

- ReadDataByIdentifier (0x22)
- ClearDiagnosticInformation (0x14)
- ReadDTCInformation (0x19)
- Routine Control(0x31)
- TesterPresent (0x3E)

Requests for ZEV Regulation $22 services can only be routed to the in-Vehicle nodes with 0xF400 to 0xF8FF segments as DID;
Requests for ZEV Regulation $31 services can only be routed to the in-Vehicle nodes with 0xE000 to 0xE1FF segments as RID;
Requests for ZEV Regulation $19 services can only be routed to the in-Vehicle nodes with 0x42 as subfunction;
Requests for ZEV Regulation $14 services can only be routed to the in-Vehicle nodes with groupOfDTC equal to 0xFFFF33 or 0xFFFFFF;
Request with other SID, subfuction or parameter shall not be routed, and the Gateway shall give a DoIP negative response with Payload Type = 0x8003 (Diagnostic message negative acknowledgement), and Diagnostic Message Negative Acknowledgement Code = 0x06 (Target unreachable).

4.10.9.3 OBDonUDS

•Introduction

4.10.9.3.1 Introduction

SAE J1979-2 specifies diagnostic services required to be supported by motor vehicles and external test equipment for diagnostic purposes which pertain to propulsion related data.
OBDonUDS provides an implementation guidance based on the principles of ISO 14229, focusing on standardization of data to ensure access to important repair information by independent repair facilities. The purpose of this standard is to focus on providing the necessary content and performance requirements for powertrain communications.
Scope of application of the regulations: Overseas regions such as Europe. Specific implementation projects will be subject to the project regulations notification.
•OBDonUDS Diagnostic Link Definition

4.10.9.3.2 OBDonUDS Diagnostic Link Definition

OBDonUDS implements regulatory certification work based on the CAN link.

1029361 v2 **OBDonUDS address assignment**

Version history:
(2): The word 3.5 has been revised to 4.0.

Purpose: Explanation of Diagnostic Address Allocation for OBD Nodes
Source:
Verification Method: Analysis AND Test
Legacy ID:

The OBD diagnostic addresses and function addressing of the ECU are defined in ISO 15765-4;
The specific address allocation for the ECU is as follows:

- GEEA3.0 & EEA4.0 ECU SWRS Req 667016 url:swap://10.24.24.227:27000/x04000000209E3DE1
- GEEA2.0 ECU SWRS Req 267539 url:swap://10.24.24.227:27000/x04000000197AB54D

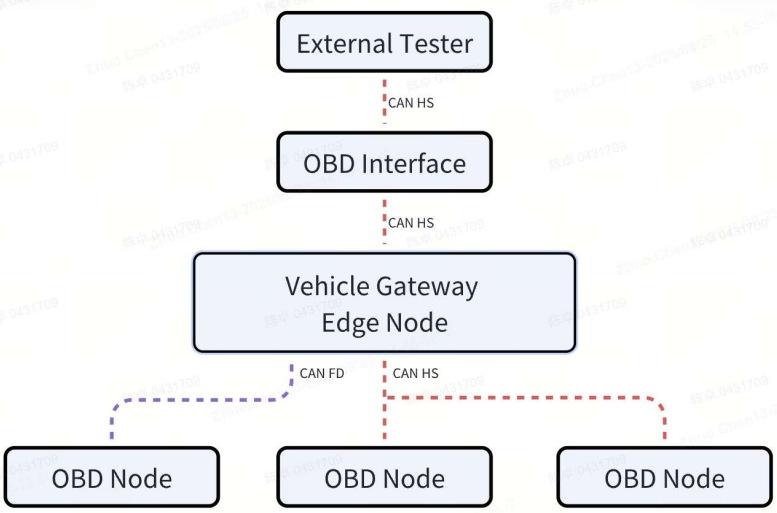
If cannot find the ECU from the above requirements, please contact the Basetech team .

1029358 v1 OBDonUDS base on CAN bus

Version history:
(1):

Purpose: Describe the communication link of OBDonUDS
Source:
Verification Method: Analysis
Legacy ID:

The OBDonUDS protocol stipulates that the communication between external devices and vehicles is conducted via the CAN bus protocol for diagnostic purposes.

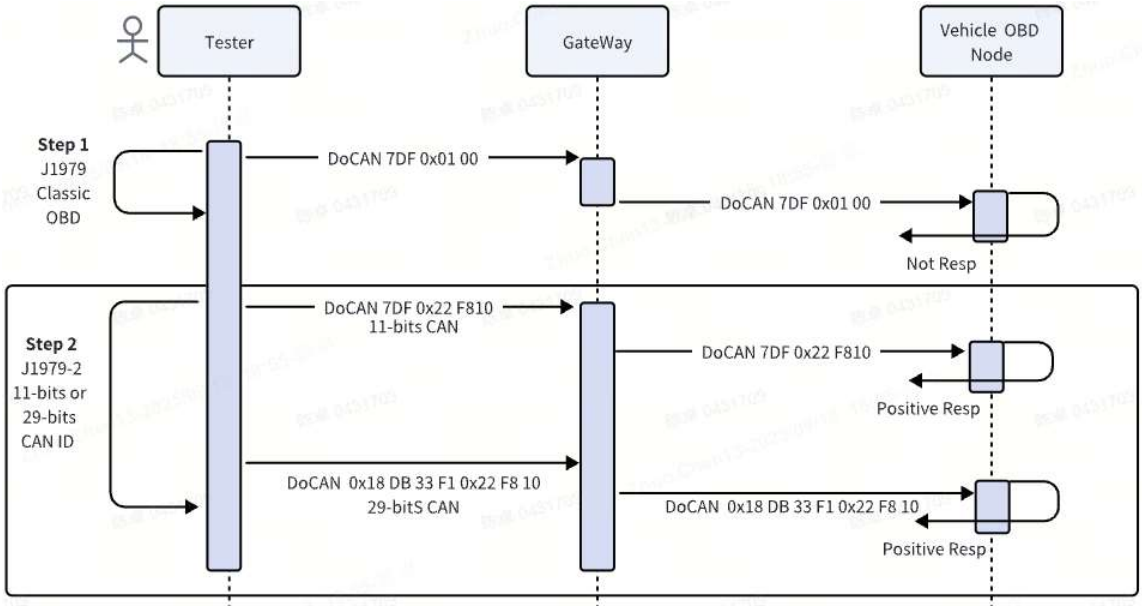


1029369 v1 Regulation link identification process requirment

Version history:
(1):

Purpose: Define the link identification process of OBDonUDS
Source:
Verification Method: Analysis AND Test
Legacy ID:

According to the SAE 1979-2 regulations, external devices need to identify the vehicle's diagnostic link is CAN. The specific process is roughly as follows: the external device will broadcast different protocol addresses within the vehicle to determine the specific link. Step 2 offers two ways to request using CAN IDs. The 11-bit CAN ID is mandatory for OBD nodes.



The diagnostic link mentioned above only supports one scenario on a vehicle that will not coexist.
The OBDonUDS nodes involved cannot provide any positive or negative feedback in step 1 .

	Request message	Expect OBD ECU responds
Step1	0x7DF 0x10 00	No Respond
Step2 11bit CANID	0x7DF 0xF8 10	Positive Respond
Step2 29bit CANID	0x18 DB 33 F1 0x22 F8 10	Not mandatory. Can be not supported.

•OBDonUDS Services Application

4.10.9.3.3 OBDonUDS Services Application

Diagnostic services shall be supported by the ECU as specified in this section. The diagnostic services required by this document are defined by [10] ISO 14229-1. This section specifies additions and restrictions that are compliant to [10] ISO 14229-1.
The aforementioned services only meet the minimum requirements for regulatory certification. In addition to meeting these minimum requirements, the ECU also needs to comply with the basic requirements of base technology. If there are any parts of the regulations that conflict with the design of basetech, then the regulations should take precedence and contact the basetech team.

1029458 v1 **OBDonUDS Propulsion ClearDiagnosticInformation (0x14)**

Version history:
(1):

Purpose: The service is used by the client to clear diagnostic information in one or multiple servers' memory.
Source:
Verification Method: Analysis AND Test
Legacy ID:

In addition to the basic service functions that meet the basetech requirements, it is also necessary to support additional diagnostic sub-service requirements, although these additional ones also comply with ISO 14229-1.

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			

ClearDiagnosticInformation (0x14)	N/A	GroupOfDTC All DTCs in zero emission related ECU	FFFF33	M	M	Not applicable in regulations.	M	M	N	N	50ms
		GroupOfDTC All Groups	FFFFFF	M	M		M	M	N	N	

Note:

- Only when the regulatory scenario does not apply will programming sessions be prohibited.

Entry conditions:

- If there are regulations, then follow the regulations. If there are no regulations, then implement in accordance with the requirements of basetech.

Exit Conditions:

- If there are regulations, then follow the regulations. If there are no regulations, then implement in accordance with the requirements of basetech.

Implementation Rules:

- The service clearDiagnosticInformation (0x14) shall not affect the ECU's ability to execute non diagnostic tasks.
- MC1=Mandatory if the ECU is Zero emission related

1029373 v1 OBDonUDS Propulsion ReadDTCInformation (0x19)

Version history:
(1):

Purpose: This service allows a client to read the status of server resident Diagnostic Trouble Code(DTC) information from any server, or group of servers within a vehicle.

Source:

Verification Method: Analysis AND Test

Legacy ID:

In addition to the basic service functions that meet the basetech requirements, it is also necessary to support additional diagnostic sub-service requirements, although these additional ones also comply with ISO 14229-1.

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
ReadDTCInformation (0x19)	Read DTC Information Report WWHOBD DTC By Mask Record(42)	DTCStatusMask		M	M	Not applicable in regulations.	M	M	N	N	50ms
		FunctionalGroupIdentifier	33								
		DTCSeverityMask	02								
	reportDTCSnapshotRecordByDTCNumber 04	DTCMaskRecord		M	M		M	M	N	N	
		DTCSnapshotRecordNumber	00								
			F0								
	ReportWWHOBDDTCWithPermanentStatus (55,	FunctionalGroupIdentifier	Emissionsystem	M	M		M	M	N	N	

	D5)		group (33)							
	ReportDTCExtendedDataRecordByDTCNumber (06, 86)	DTCMaskRecord		MC[1]	MC[1]		M	M	N	N
		DTCExtDataRecordNumber	90~9F							
	reportSupportedDTCExtDataRecord(1A)	DTCExtDataRecordNumber	90~9F	MC[1]	MC[1]		M	M	N	N
	Request DTCs for a ReadinessGroup (56)	ReadinessGroupIdentifier	00~FE	MC[2]	MC[2]		M	M	N	N

Note:

- Only when the regulatory scenario does not apply will programming sessions be prohibited.

Entry conditions:

- Entry conditions for service ReadDTCInformation (0x19) are not allowed.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- The service ReadDTCInformation (0x19) shall not affect the ECU's ability to execute non diagnostic tasks.
- MC[1]=DTCExtDataRecordNumber 90~9F is detailedly defined in SAE 1979 DA. This requirement does not specify which specific expansion types are to be used. Information needs to be provided according to local regulations or by the certification team.
- MC[2]=ReadinessGroupIdentifier 00~FE is detailedly defined in SAE 1979 DA. This requirement does not specify which specific expansion types are to be used. Information needs to be provided according to local regulations or by the certification team.
- According to the ZEV regulation requirements, the emission node must support DTCs in the 0x04 format as specified in SAE-J2012. These DTCs must not only support service 1942 but also other sub-functions required by basetech for service 19, such as 1902/1903, etc. Additionally, they must support basetech's basic snapshot functionality.
- If a ZEV DTC function is similar to a basetech-defined DTC function, the basetech DTC must not be replaced with the ZEV DTC; the basetech DTC must **not be altered or deleted** by any regulation.
- The DTC fault code for ZEV must meet the SAE J2012DA 0x04 format requirements.

Requirements for OBD snapshots

- DTCSnapshotRecordNumber 0x00 when the malfunction was detected for the first occurrence
- DTCSnapshotRecordNumber 0xF0 when the malfunction was detected during the latest occurrence.
- Snapshot 0x00 0xF0 is triggered by DTCStatusBit #2 = 1 or DTCStatusBit #3 = 1. The OBD node needs to satisfy either of these two triggering methods. This is in accordance with the regulations and recommendations.

1029459 v1 **OBDonUDS Propulsion ReadDataByIdentifier (0x22)**

Version history:
(1):

Purpose: The service allows the client to request data record valued from the server identified by one or more dataIdentifiers.

Source:

Verification Method: Analysis AND Test
Legacy ID:

Additional diagnostic sub-service requirements, although these additional ones also comply with ISO 14229-1.

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
ReadDataByIdentifier(0x22)	N/A	DataIdentifier		M	M	Not applicable in regulations.	M	M	N	N	50ms

Note:

- The DID requirements for OBDonUDS, which originate from SAE J1979DA, have a range of 0xF400 to 0xF8FF.
- P4server = 5000ms. This setting is applicable to 0xF800 to 0xF8FF. Additionally, for DID 0xF810, it must meet the requirement of 50ms.
- Only when the regulatory scenario does not apply will programming sessions be prohibited.

Entry conditions:

- Entry conditions for service ReadDataByIdentifier (0x22) are not allowed.

Exit Conditions:

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by GRI Electrical Architecture.

Implementation Rules:

- Minimum 10 dataIdentifiers, or as many dataIdentifiers as implemented, in one single ReadDataByIdentifier request shall be supported by the ECU in default and extended diagnostic session.
- The service readDataByIdentifier (0x22) shall not affect the ECU's ability to execute non diagnostic tasks.

1029460 v1 OBDonUDS Propulsion RoutineControl (0x31)

Version history:
(1):

Purpose: The service is used by the client ti execute a defined sequence of steps and obtain any relevant results.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Service	Sub-function	Data Paremeters		Session			Addressing		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional			
RoutineControl (0x31)	StartRoutine (01)	RoutineIdentifier		M	M	Not applicable in regulations.	M	M	N	N	50ms

Note:

- The RID requirements for OBDonUDS, which originate from SAE J1979DA, have a range of 0xE000 to 0xE1FF.

Entry conditions:

- If there are regulations, then follow the regulations. If there are no regulations, then implement in accordance with the requirements of basetech.

Exit Conditions:

- If there are regulations, then follow the regulations. If there are no regulations, then implement in accordance with the requirements of basetech.

Implementation Rules:

- The service routineControl (0x31) is allowed to affect the ECU's ability to execute non diagnostic tasks. The service is only allowed to affect execution of the non-diagnostic tasks during the execution of the diagnostic service. After the diagnostic service is completed any effect on the non-diagnostic tasks is not allowed anymore (normal operational functionality resumes). The service is not allowed to reset the ECU .

•OBDonUDS Data security

4.10.9.3.4 OBDonUDS Data security

1029552 v1 **OBDonUDS Data protection**

Version history:
(1):

Purpose:
ECU's requirements for data protection
Source:
Verification Method: Analysis AND Test
Legacy ID:

For DoCAN based links that cannot perform data recognition and interception like DoIP links, relying on gateways is required because the overall diagnostic link duration of DoCAN is required to be within 50ms. Therefore, ECU that supports OBDonUDS needs to be configured in software development. The specific requirements are as follows

- The use of OBD addresses should only answer the following service requests

1、ReadDataByIdentifier (0x22)
2、ClearDiagnosticInformation (0x14)
3、ReadDTCInformation (0x19)
4、Routine Control(0x31)
5、TesterPresent (0x3E)

- For services $22 and $31 with limited support for DID&RID

1、Only the $22 data content within this range can be read 0xF400 to 0xF8FF of DID；
1、Only the $31 data content within this range can be read 0xE000 to 0xE1FF of RID；

4.10.9.4 ADAS_L3onUDS

•Introduction

4.10.9.4.1 Introduction

<GB 44497—2024> Intelligentandconnectedvehicle—Datastoragesystemforautomateddriving stipulates the technical requirements, test methods, and type approval for the autonomous driving data recording system of intelligent connected vehicles.
Scope of application of the regulations: China market. Vehicles of type M and N, along with those equipped with L3 or higher-level autonomous driving systems, are required to meet this requirement mandatorily.The specific implementation will be subject to the notification of the project regulations.

•ADAS_L3onUDS Diagnostic Link Definition

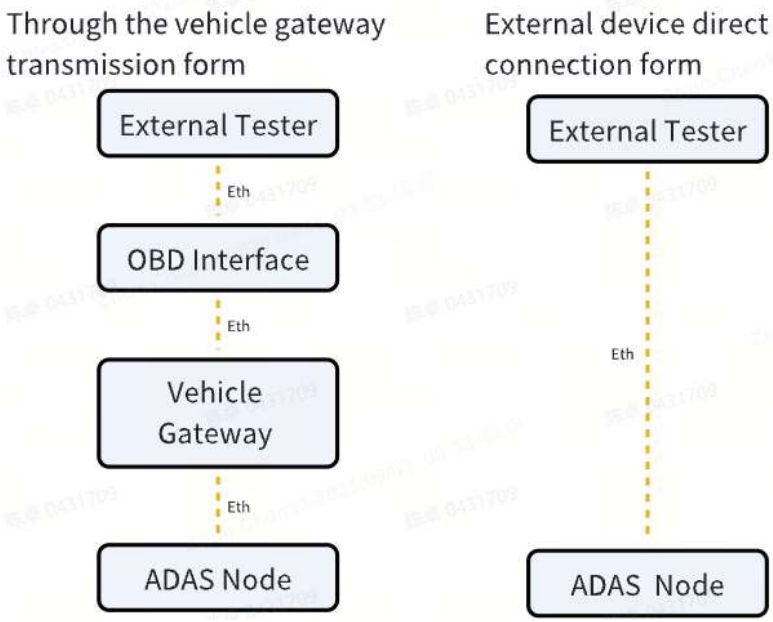
4.10.9.4.2 ADAS_L3onUDS Diagnostic Link Definition

1029718 v1 **ADAS_L3 base on Ethernet**

Version history:
(1):

Purpose: Define the two ways of reading L3 data as required by the regulations
Source:
Verification Method: Analysis
Legacy ID:

The regulatory definition requires that there will be two ways to access the data information of the autonomous driving controller. One method is to transmit through the edge nodes of the vehicle gateway, and the other is for external devices to connect directly.



1029736 v1 **L3 logical address assignment**

Version history:
(1):

Purpose: Define the logical address for communication between external tester and edge nodes.
Source:
Verification Method: Analysis AND Test
Legacy ID:

External tester shall have logical address 0x0F80, and use the logical address to communicate with the edge node.

Addressing Message Type	Target Address (TA)	Source Address(SA)	Description
Physical request	0x1000~0x7FFF	0x0F80	SA = External L3 test equipment TA = one ADAS ECU
Physical response	0x0F80	0x1000~0x7FFF	SA = one ADAS ECU TA = External L3 test equipment

This regulation does not define the logical address range of the ECU. The logical address of the ECU is described in the user manual of the motor vehicle product. This can be defined as the address definition for the all platform.

•Genenral Requirments

4.10.9.4.3 Genenral Requirments

1029812 v1 **DoIP Ports for Communication**

Version history:
(1):

Purpose: To define which port to use when connecting to the Edge Node for vehicle
Source:
Verification Method: Test
Legacy ID:

All in-vehicle DoIP communication shall use the target/destination TCP port 13400 when connecting to and communicating with the Edge Node.

1029813 v1 **L3onUDS DoIP routing acivation type**

Version history:
(1):

Purpose: Define acivation type
Source:
Verification Method: Test
Legacy ID:

The vehicle gateway ECU shall allow routing activation requests with activation type 0x00 (Default).

•L3onUDS Services Application

4.10.9.4.4 L3onUDS Services Application

1029761 v1 **Services related to regulations**

Version history:
(1):

Purpose: Define UDS services that allow physical addressing of routing for vehicle gateways
Source:
Verification Method: Analysis AND Test
Legacy ID:

List the information of the diagnostic services that the regulations require support for.The vehicle L3 system ECU shall allow the following list of UDS services for communication between data storage system for automated driving and external prolonged data collection equipment.

Service	Sub-function	Data Paremeters	Session	Addressing	Authe	Securi	P4

							0x0F80		nticati	ty	Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Programming	Physical	Functional	on access 29	access 27	MC[2]
DiagnosticSessionControl(0x10)	DefaultSession(01,)	N/A	N/A	M	M	Not applicable in the context of regulations.	M	N	N	N	P2 Server_
	ExtendedDiagnosticSession(03)	N/A	N/A	M	M		M	N	N	N	
TesterPresent(0x3E)	zeroSubFunction n(00 80)	N/A	N/A	M	M		M	N	N	N	P2 Server_
RequestFileTransfer (0x38)	N/A	AddFile	01	NA	M		M	N	N	MC[1]	1000ms
	N/A	DeleteFile	02	NA	M		M	N	N	MC[1]	
	N/A	ReplaceFile	03	NA	M		M	N	N	MC[1]	
	N/A	ReadFile	04	NA	M		M	N	N	N	
	N/A	ReadDir	05	NA	M		M	N	N	N	
	N/A	ResumeFile	06	NA	M		M	N	N	MC[1]	
TransferData(0x36)	N/A	BlockSequenceCounter	00-FF	N	M		M	N	N	N	5000ms
		TransferRequestParameterRecord	00-FF	N	M		M	N	N	N	
RequestTransferExit (0x37)	N/A	N/A	N/A	N	M		M	N	N	N	1000ms
RoutineControl (0x31)	StartRoutine (01)	RoutineIdentifier	01	N	M		M	N	N	N	5000ms

Note：

- Only when the regulatory scenario does not apply will programming sessions be prohibited.
- The vehicle gateway shall not apply any form of Authentication, Authorization or Security Access mechanisms to prevent external prolonged data collection equipment (DoIP tester address 0x0F80) from accessing data in the data storage system for automated driving.
- MC[1]=the data storage system for automated driving required； The regulatory scenario only requires support for reading files. The information security requirements prohibit any form of modification or deletion.
- MC[2]=If the regulations do not specify otherwise, the P4 time should be implemented in accordance with the requirements of basitech. If there are any requirements in the regulations, then the regulations shall prevail.

- The aforementioned services only meet the minimum requirements for regulatory certification. In addition to meeting these minimum requirements, the ECU also needs to comply with the basic requirements of base technology. If there are any parts of the regulations that conflict with the design of basetech, then the regulations should take precedence and contact the basetech team.
-

1029814 v1 **Vehicle gateway allows functional addressing of UDS services for routing**

Version history:
(1):

Purpose: Define UDS services that allow functional addressing of routing for vehicle gateways
Source:
Verification Method: Test
Legacy ID:

For the functional addressing request of external prolonged data collection equipment (DoIP logical address 0x0F80), the edge node does not route it to the vehicle and discards it.

•L3onUDS Data security

4.10.9.4.5 L3onUDS Data security

1029553 v1 **L3onUDS Data protection**

Version history:
(1):

Purpose: The gateway needs to identify diagnostic messages for the Ethernet link.
Source:
Verification Method: Analysis AND Test
Legacy ID:

If the gateway supports the correlation diagnostics encryption design (TLS,$29,$31.etc),L3 diagnostics are not affected by the encryption design when external regulatory certified L3 device**(SA 0x0F80**) read the relevant L3 information from the gateway.
Combined with the GB44497 regulations of the service application and the above requirements, the requirements of the gateway allows only routing service:

- DiagnosticSessionControl (0x10)
- TransferData (0x36)
- RequestTransferExit (0x37)
- RequestFileTransfer (0x38)
- Routine Control(0x31)
- TesterPresent (0x3E)

Requests for Regulation $10 services : only be routed to the in-Vehicle for subfunction default session(01) and extend session (03) ;
Requests for Regulation $31 services : only be routed to the in-Vehicle for 0xF1AA~0xF1AC segments of RID;
Request with other SID, subfuction or parameter shall not be routed, and the Gateway shall give a DoIP negative response with Payload Type = 0x8003 (Diagnostic message negative acknowledgement), and Diagnostic Message Negative Acknowledgement Code = 0x06 (Target unreachable).

4.10.9.5 ePTIonUDS

•Introduction

4.10.9.5.1 Introduction

ISO 20730-1 Roadworthiness testing is a part of a wider regime designed to ensure that road vehicles are kept in a safe and environmentally acceptable condition during their use. This regime covers periodic roadworthiness testing of vehicles and technical roadside inspections of vehicles used for commercial road transport activities and provides a vehicle registration procedure allowing for the suspension of a vehicle's authorisation to be used in road traffic where the vehicle constitutes an immediate risk to road safety. Periodic testing is the main tool to ensure roadworthiness. Technical roadside inspections of commercial vehicles are merely complementary to periodic testing.
Scope of application of the regulations: The market and application projects have not yet been clearly defined.The specific implementation will be subject to the notification of the project regulations.

•ePTIonUDS Diagnostic Link Definition

4.10.9.5.2 ePTIonUDS Diagnostic Link Definition

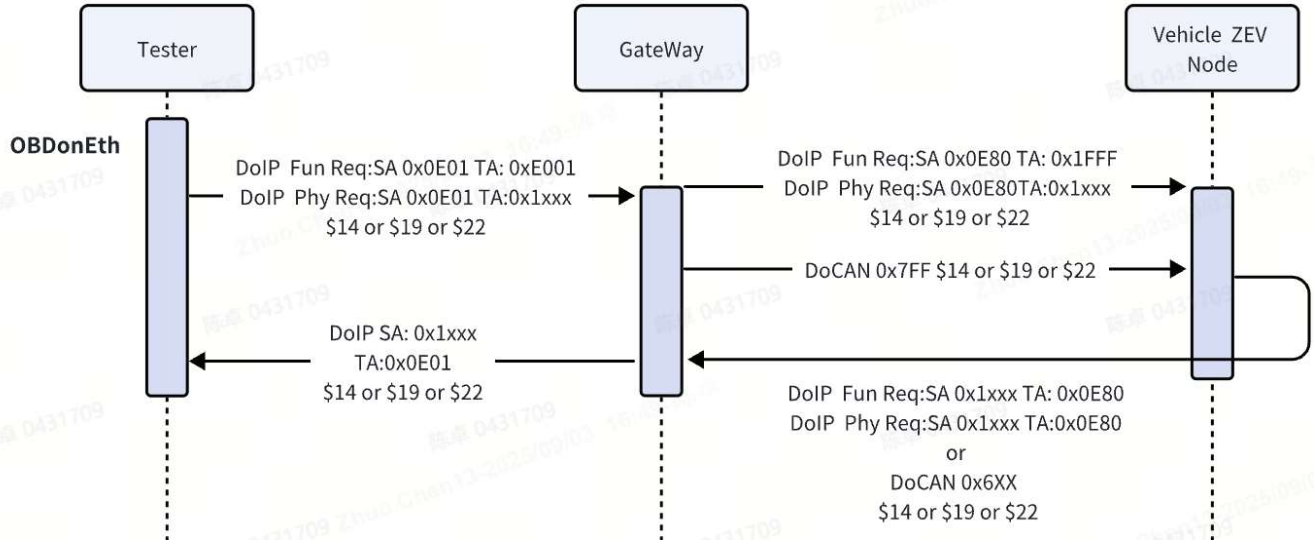
1030709 v1 **Diagnostic communication requirements after link confirmation**

Version history:
(1):

Purpose: Detailed rules for gateway address translation are defined.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The address conversion logic is only applicable to the Geely platform project. It is recommended to develop it in a platform-based manner. If there are any cooperation projects that do not require the implementation of this plan, they can also be accepted. Carry out according to the specific requirements of the project.

1. Translate the external diagnostic addresses and diagnostic requests into OEM-defined addresses and requests, and broadcast or unicast them within the vehicle.
2. Conversely, when receiving diagnostic responses from in-vehicle nodes, the gateway should support translating them into the corresponding ePTI addresses and send them to the external device.



Function Addressing Scenario:

Function address request(DoIP→DoIP)

ePTI Tester request	Gateway Switch	In-Vehicle Command
SA:0x0E01		**SA:0x0E80**
Function address:**0xE001**		Function address:**0x1FFF**

Function address Respond(DoIP→DoIP)		
ePTI Node Respond	Gateway Switch	Out-Vehicle Command
SA:0x1xxx		SA:0x1xxx
TA: **0x0E80**		TA: **0x0E01**

Function address request(DoIP→DoCAN)		
ePTI Tester request	Gateway Switch	In-Vehicle Command
SA:0x0E00		DiagCAN:**0x7FF**
TA:function address:**0x0E01**		

Function address Respond(DoCAN→DoIP)		
ePTI Node Respond	Gateway Switch	Out-Vehicle Command
DiagCAN:**0x6xx**		SA:0x1xxx
		TA: **0x0E01**

In this scenario, the gateway needs to convert the source address(0x0E01) of the received ePTI device into a device address defined(0x0E80) by Geely.
At the same time, it is necessary to support the conversion of external function addressing(0xE001) commands to 0x1FFF and 0x7FF and route them to the Eth node and CAN node.
Conversely, support is also required.

Physical Addressing Scenario:

Physical address Respond(DoIP→DoIP) ADCU-Eth address as an example		
ePTI Tester request	Gateway Switch	Out-Vehicle Command
SA:0x0E01		SA:0x0E80
TA: 0x1301		TA:0x1301

Physical address Respond(DoIP→DoIP)		
ePTI Node Respond	Gateway Switch	Out-Vehicle Command
SA:0x1301		SA:0x1301
TA: 0x0E80		TA: 0x0E01

Physical address request(DoIP→DoCAN) BCM-CAN address as an example		
ePTI Tester request	Gateway Switch	In-Vehicle Command
SA:0x0E01		DiagCAN:0x781
TA:0x1D81		

Physical address Respond(DoCAN→DoIP)		
ePTI Node Respond	Gateway Switch	In-Vehicle Command
DiagCAN:0x681		SA:0x1D81
		TA:0x0E01

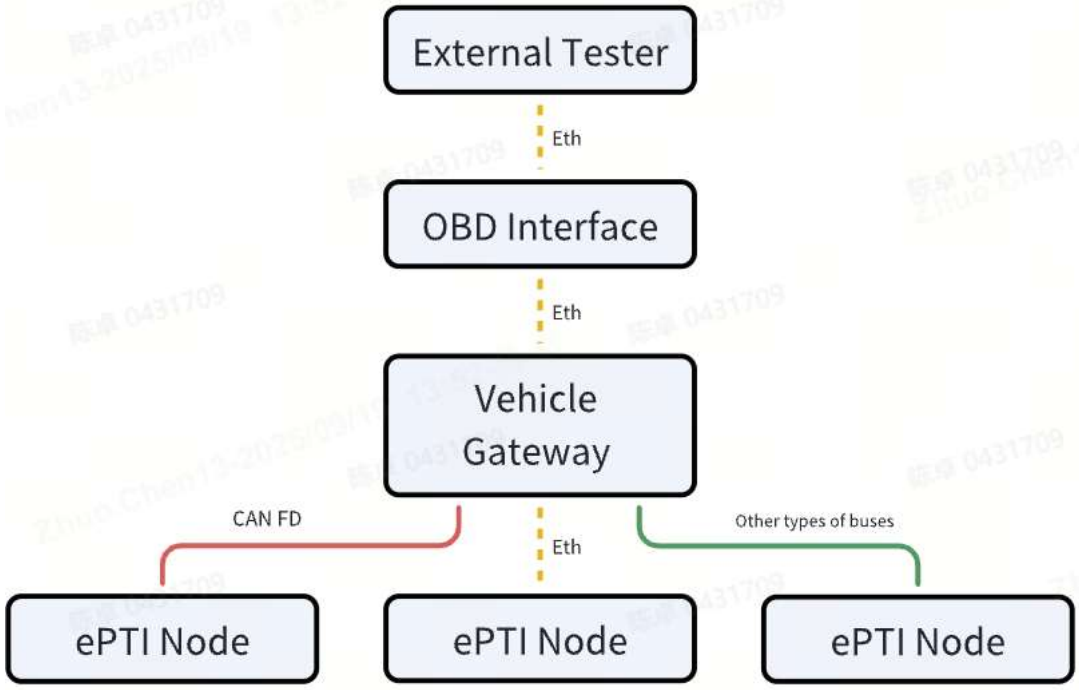
In this scenario, the gateway needs to convert the source address(0x0E01) of the received ePTI device into a device address defined(0x0E80) by Geely.Conversely, support is also required.

1037608 v1 **ePTI base on Ethernet**

Version history:
(1):

Purpose: Currently, for the ePTI link, we have chosen the DoIP (Diagnostic over Internet Protocol) link.
Source:
Verification Method: Analysis
Legacy ID:

The ePTI specification defines that CAN or Eth can be used. Considering the number of nodes and the requirement of the response time of the specification service, Platform defines Ethernet links at present.



1029901 v1 **ePTI logical address assignment**

Version history:
(1):

Purpose: These addresses ensure proper communication and compliance with ePTI standards.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Addressing Message Type	Target Address (TA)	Source Address(SA)	Description
Functional request	0xE001	0x0E01	SA = External test equipment TA = all ZEV-relevant server(s)/ECU(s)
Physical request	0x1000~0x7FFF	0x0E01	SA = External test equipment TA = one ZEV-relevant server/ECU

Physical response	0x0E01	0x1000~0x7FFF	SA = one ZEV-relevant server/ECU TA = External test equipment

•Genenral Requirment

4.10.9.5.3 Genenral Requirment

1030715 v1 **ePTIonUDS DoIP routing acivation type**

Version history:
(1):

Purpose: Define the routing activation type
Source:
Verification Method: Analysis AND Test
Legacy ID:

According to regulatory requirements,The gateway must support therouting activation type 0x01

•ePTIonUDS Services Application

4.10.9.5.4 ePTIonUDS Services Application

Diagnostic services shall be supported by the ECU as specified in this section. The diagnostic services required by this document are defined by [10] ISO 14229-1. This section specifies additions and restrictions that are compliant to [10] ISO 14229-1.
The aforementioned services only meet the minimum requirements for regulatory certification. In addition to meeting these minimum requirements, the ECU also needs to comply with the basic requirements of base technology. If there are any parts of the regulations that conflict with the design of basetech, then the regulations should take precedence and contact the basetech team.

1030289 v1 **Services related to regulations**

Version history:
(1):

Purpose: The definition of regulations involves service requirements.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Service	Sub-function	Data Paremeters		Session		Addressing 0x0E01		Authentication access 29	Security access 27	P4 Server
Name (Hex)	Name(Hex)	Name	Hex	Default	Extended	Physical	Functional			
DiagnosticSessionControl(0x10)	DefaultSession(01,)	N/A	N/A	M	M	M	M	N	N	5000ms
	ExtendedDiagnosticSession(03)	N/A	N/A	M	M	M	M	N	N	
ReadDataByIdentifier(0x22)	N/A	DataIdentifier		M	M	M	M	N	N	200ms

TesterPresent(0x3E)	zeroSubFunction(00 80)	N/A	N/A	M	M	M	N	N	N	P2 Server_max
RoutineControl (0x31)	StartRoutine(01)	RoutineIdentifier	N/A	MC	M	M	N	N	MC	5000ms
	stopRoutine(02)	RoutineIdentifier	N/A	MC	M	M	N	N	MC	200ms
	requestRoutineResults(03)	RoutineIdentifier	N/A	MC	M	M	N	N	MC	200ms
SecurityAccess(0x27)	SubFunctions are vehicle-manufacturer specific.									
ReadDTCInformation(0x19)	Develop in accordance with the requirements of basetech 663996									
InputOutputControlByIdentifier (0x2F)	Develop in accordance with the requirements of basetech 663998									

Note:

- Only when the regulatory scenario does not apply will programming sessions be prohibited.

•ePTIonUDS Data security

4.10.9.5.5 ePTIonUDS Data security

1029554 v1 **ePTIonUDS Data protection**

Version history:
(1):

Purpose: The gateway needs to identify diagnostic messages for the Ethernet link.
Source:
Verification Method: Analysis AND Test
Legacy ID:

If the gateway supports the correlation diagnostics encryption design (TLS,$29,$31.etc), ePTI diagnostics are not affected by the encryption design when external ePTI regulatory certified device**(SA 0x0E01)** read the relevant ePTI information from the gateway.
Combined with the 20730-1 regulations of the service application and the above requirements, the requirements of the gateway allows only routing service:

- ReadDataByIdentifier (0x22)
- DiagnosticSessionControl (0x10)
- ReadDTCInformation (0x19)
- Routine Control(0x31)
- TesterPresent (0x3E)
- InputOutputControlByIdentifier (0x2F)

Requests for Regulation $22/$2F services can only be routed to the in-Vehicle for 0xFA19 to 0xFACF/F18F/F190/F4A6/F40D/F442 segments of DID；
Requests for Regulation $31 services can only be routed to the in-Vehicle for 0xE200 to 0xE2FF segments of RID；

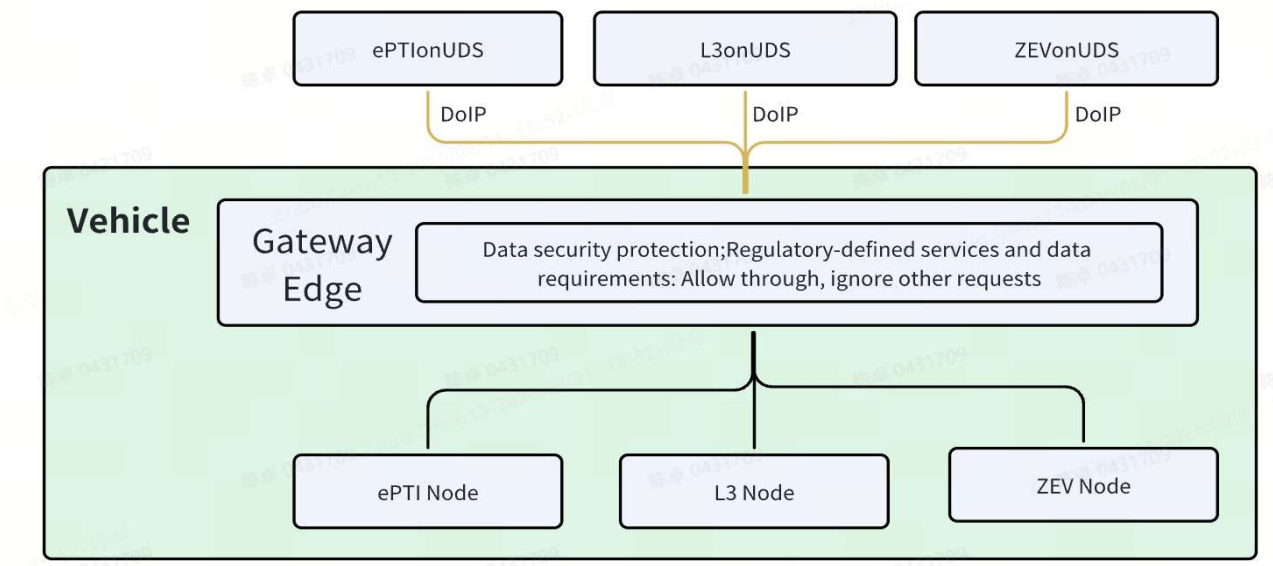
Requests for Regulation $10 services : only be routed to the in-Vehicle for subfunction default session(01) and extend session (03) ;
Request with other SID, subfuction or parameter shall not be routed, and the Gateway shall give a DoIP negative response with Payload Type = 0x8003 (Diagnostic message negative acknowledgement), and Diagnostic Message Negative Acknowledgement Code = 0x06 (Target unreachable).

4.10.9.6 Data security

•Platform-based data protection solution based on Ethernet

4.10.9.6.1 Platform-based data protection solution based on Ethernet

This is a rough schematic diagram, which briefly describes that based on the DoIP regulations for diagnosis, the gateway needs to identify the content of external requests. Only requests related to regulations are allowed to be routed to the vehicle. More detailed requirements will be defined in the subsequent sections.

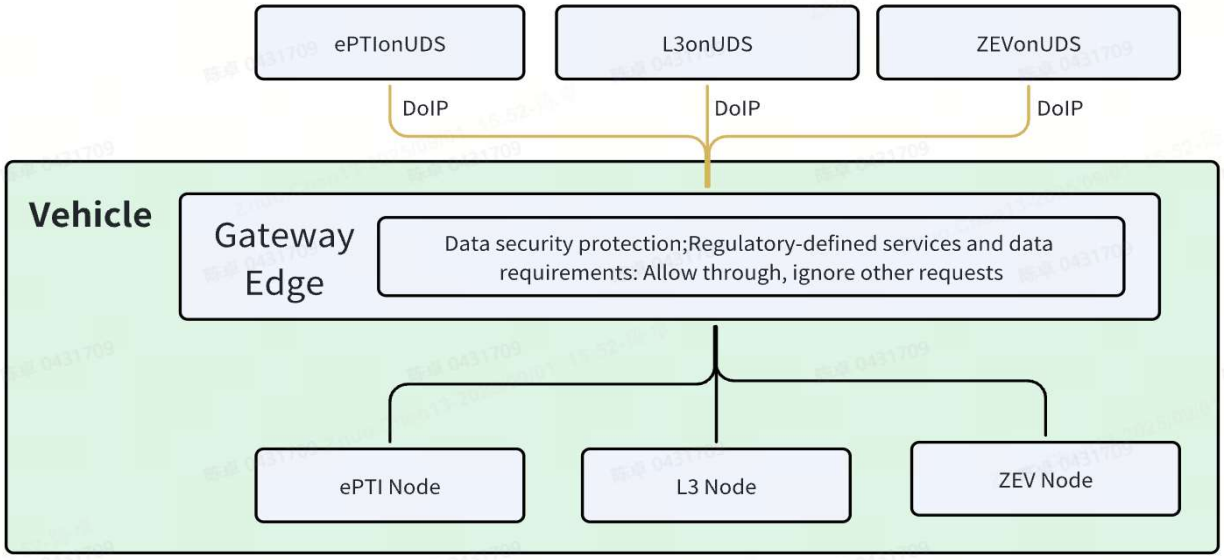


1037652 v1 **Regarding the data protection requirements for DoIP links**

Version history:
(1):

Purpose: Regarding the data protection requirements for DoIP links
Source:
Verification Method: Analysis
Legacy ID:

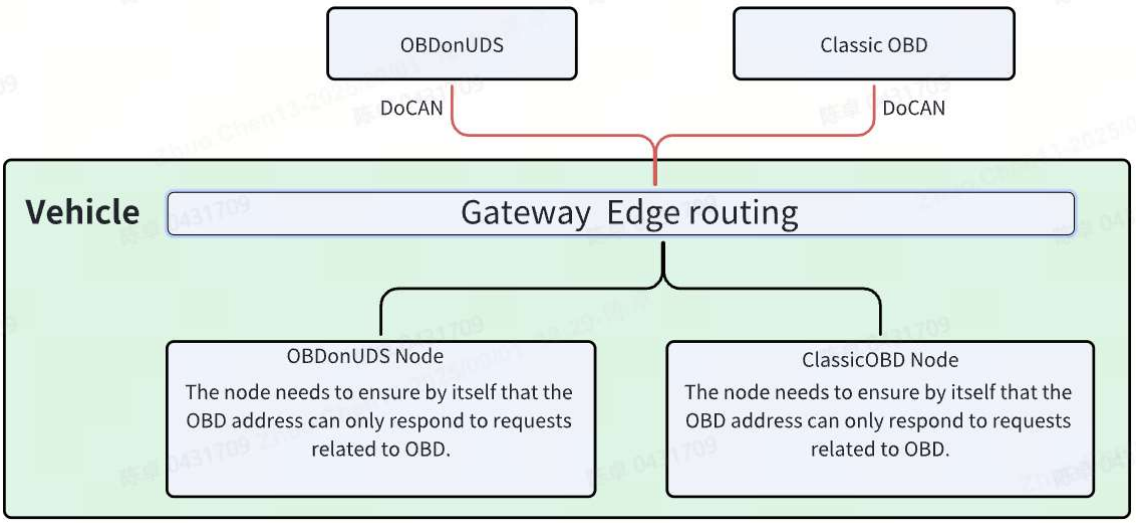
This is a rough schematic diagram, which briefly describes that based on the DoIP regulations for diagnosis, the gateway needs to identify the content of external requests. Only requests related to regulations are allowed to be routed to the vehicle. More detailed requirements will be defined in the subsequent sections.



•Platform-based data protection solution based on CAN

4.10.9.6.2 Platform-based data protection solution based on CAN

This is a simple schematic diagram, which roughly describes the regulatory protocol based on the DoCAN link. It requires that regulatory nodes should not respond to requests that are not related to regulations.



1030387 v1 **ClassicOBD Data protection**

Version history:
(1):

Purpose: ECU's requirements for data protection
Source:
Verification Method: Analysis AND Test
Legacy ID:

The classic OBD is not based on the UDS protocol, so the introduction is relatively small. Here, it is required to only support the ECU of the classic OBD, the OBD address can only support OBD services, and can only read OBD related data (data defined in SEA 1979DA&SAE 2012)

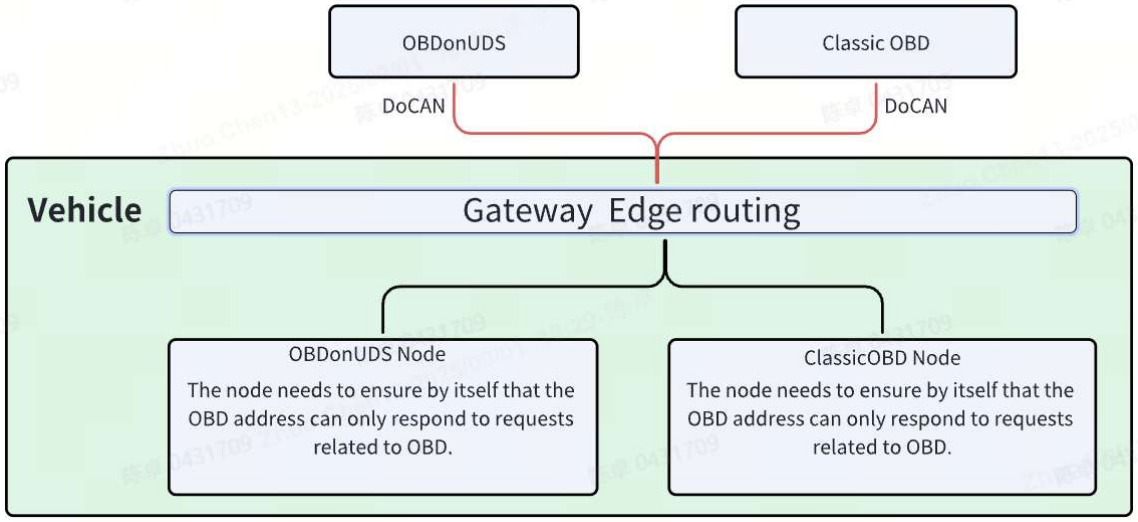
For ECU devices that are compatible with the classic OBD standard, it is required that the OBD address can only access the OBD service($01~$0A) and cannot access the UDS service(ISO 14229-1).

1037654 v1 **Regarding the data protection requirements for DoCAN links**

Version history:
(1):

Purpose: Regarding the data protection requirements for DoCAN links
Source:
Verification Method: Analysis
Legacy ID:

This is a simple schematic diagram, which roughly describes the regulatory protocol based on the DoCAN link. It requires that regulatory nodes should not respond to requests that are not related to regulations.



4.10.10 Universal Calibration Protocol

4.10.10.1 General Requirements for XCP

•XCP Introduction

4.10.10.1.1 XCP Introduction

ECU Measurement capability is a mission critical task for the development of automotive ECUs. XCP avoids the need to create specific ECU implementations for specific measurement tasks. Essentially, the protocol supports all typical measurement tasks without the need to recompile to ECU application.

XCP ensures interoperability between measurement systems and ECUs. Two typical customer-supplier collaboration scenarios are supported:

- an OEM measures on a vehicle consisting of multiple ECUs from different suppliers.
- a supplier provides its ECU to OEMs, and the OEMs use different calibration systems.

Application Areas

XCP can be used in all stages of ECU development, such as function development, ECU calibration and testing of ECUs. As XCP is able to realize high data rates and short measurement cycle times in the microsecond range, XCP is very helpful when analyzing the dynamic behavior of electromechanical systems relevant for automotive use cases such as engine or transmission systems or electric drives.

Besides measurement data acquisition and calibration use cases, XCP is also used for reprogramming in early stages of ECU development where the required diagnostic services have not been implemented yet.

Introduction

A key functionality of ASAM MCD-1 XCP is to enable read and write access to the memory of the ECU. Read access, commonly called "measurement" in the standard, lets users measure the time response of an internal ECU variable. ECUs are systems with discrete time behavior, whose variables only change at specific time intervals. One of the great strengths of XCP lies in acquiring measured values from RAM only at specific periodic points of time or at specific events. This time or event driven access, commonly called "event-synchronous measurement", allows users to directly evaluate the ECU's behavior over time with recorded values from internal variables and external sensors.

XCP was designed with these main objectives:

- Minimal resource usage in the ECU
- Efficient communication
- Simple Slave implementation
- Plug-and-play configuration with just a small number of parameters
- Scalability: XCP is also very scalable in its implementation, it is not necessary to implement every command described by the standard
- Transferability: XCP can be implemented on major bus systems and is open for future bus systems.

•XCP requirements

4.10.10.1.2 XCP requirements

975465 v2 **General XCP requirements**

Version history:
(2):

Purpose: Enable efficient function validation and verification during development and dynamic data collection post start of production.

Source:

Verification Method: Analysis

Legacy ID:

Zeekr Requirement ID: 733245

There shall be an XCP server implemented according ASAM MCD-1 XCP standard.

The XCP server shall be active and support measuring during normal operation during development and in production vehicles.

The XCP server shall handle invalid commands and addresses gracefully, i.e., Maintaining normal ECU functionality if access to invalid addresses is commanded/requested.

The XCP server must fulfill setup procedures and measurement according to performance requirements, without affecting ECU's ability to maintaining normal ECU functionality.

Memory areas that contains security and privacy information should not be possible to access via XCP measurement system.

In production vehicles, the XCP server shall protect against unauthorized changes of the content in the memory.

- o The calibration functionality of XCP shall be possible to enable/disable using a separate authentication seed/key or a separate software delivery.
- o If the flash functionality of XCP is supported, then it shall be possible to enable using a separate authentication seed/key or a separate software delivery.

Note : If alternative technical solutions for calibration during development are available, this part can be ignored.

During development only, i.e., non-production vehicles, the calibration and flash capability of XCP shall be possible to enable.

Authentication shall be in accordance with OEM security concept.

The XCP server shall support checksum calculation of program and calibration memory area using algorithm CRC16-CRITT.

The XCP server must ensure freedom of interference from ASIL A to D related software.

Calibration : The process of tuning parameters in ECU software. The values of internal ECU parameters are determined as a result of this process.

Intention of calibration is to tailor made software (set parameters) to fit specific applications/context without compiling new software.

975466 v2 **Measurement specific XCP requirements**

Version history:
(2):

Purpose: Alignment of XCP server implementations such that a central XCP client can handle setup and measurements with XCP post start of production.

Source:
Verification Method: Analysis
Legacy ID:
Zeekr Requirement ID: 733246

• Transfer object size:

- o Command Transfer Object - CTO: 255 bytes
- o Data Transfer Object - DTO:

§Ethernet: UDP MTU minus XCP header length but size shall be optimized based on hardware.

§ CAN FD: 64 bytes minus XCP header length

§ CAN FD @500 kbps: 64 bytes minus XCP header length

• Dynamic DAQ lists supported according to ASAM MCD-1 (XCP) standard.

• Ethernet port number to be configurable (for example controlled by Car config).

• Number of XCP events supported: Matching the number of scheduling frequencies and requested rasters, recommended as at least 5.

• All scheduling frequencies shall be connected to an event.

• Minimum number of DAQ lists supported shall be at least the number of supported events and shall be optimized to support “REQ: Performance requirements for Universal Measurement and Calibration Protocol (XCP)”.

• Identification Field Type: Absolute. Note! Shall be included for CAN FD and Ethernet connected ECU’s .

• Byte Order: MSB LAST

• Address Granularity: Byte

• Granularity ODT Entry Size: Byte (shall match Max CTO size)

• Timestamp & counter supported. Format and resolution will be affected by which time sync scenario from ASAM MCD-1 (XCP) standard is selected.

• Time sync according to one of the time sync scenarios in ASAM MCD-1 (XCP) standard. OEM to decide which scenario to be used.

• Supported Commands

o Mandatory according to ASAM MCD-1 (XCP) standard

o The Commands below, listed as optional in ASAM MCD-1 (XCP) standard, shall be supported:

§ OPTIONAL_CMD ALLOC_ODT_ENTRY

§ OPTIONAL_CMD ALLOC_ODT

§ OPTIONAL_CMD ALLOC_DAQ

§ OPTIONAL_CMD FREE_DAQ

§ OPTIONAL_CMD GET_DAQ_RESOLUTION_INFO

§ OPTIONAL_CMD GET_DAQ_PROCESSOR_INFO

§ OPTIONAL_CMD START_STOP_SYNCH

§ OPTIONAL_CMD GET_DAQ_CLOCK

§ OPTIONAL_CMD START_STOP_DAQ_LIST

- § OPTIONAL_CMD GET_DAQ_LIST_MODE
- § OPTIONAL_CMD SET_DAQ_LIST_MODE
- § OPTIONAL_CMD WRITE_DAQ
- § OPTIONAL_CMD SET_DAQ_PTR
- § OPTIONAL_CMD CLEAR_DAQ_LIST
- § OPTIONAL_CMD SHORT_UPLOAD
- § OPTIONAL_CMD UPLOAD
- § OPTIONAL_CMD SET_MTA
- § OPTIONAL_CMD GET_ID
- § OPTIONAL_CMD GET_COMM_MODE_INFO
- § OPTIONAL_CMD BUILD_CHECKSUM
- § OPTIONAL_CMD SET_SEGMENT_MODE
- § OPTIONAL_CMD GET_SEGMENT_MODE
- § OPTIONAL_CMD SET_REQUEST
- § OPTIONAL_CMD GET_SEED
- § OPTIONAL_CMD UNLOCK
- § OPTIONAL_CMD COPY_CAL_PAGE - Note: Only if Calibration part is implemented

975467 v2 **Performance XCP requirements**

Version history:
(2):

Purpose: Ensure sufficient resources to meet OEM's XCP measurement needs.
Source:
Verification Method: Analysis
Legacy ID:
Zeekr Requirement ID: 733247

Timing Requirement :

Maximum time for initialization of XCP measurement including setup procedure is 1 second per individual step (seed&key, checksum calculation, DAQ setup etc).

Measurement capability Requirements :

Dependent of number of domains/functional areas covered by the ECU, there are different requirement sets

applicable:

For ECU's covering one (or few) functional areas:

• The bandwidth of XCP server on the ECU shall be able to handle at least 30 4-byte signals, every 10 ms (100Hz).

Note: For CAN this corresponds to approx 212 kbit/s, including XCP-protocol and CAN-frame overhead.

• The XCP server on the ECU shall be able to handle at least 150 4-byte signals in total.

Note: The requested maximum bandwidth does not need to exceed bandwidth requirement stated above.

For ECU's covering multiple functional areas:

• The bandwidth of XCP server on the ECU shall be able to handle at least 100 4-byte signals, every 10 ms (100Hz).

• The XCP server on the ECU shall be able to handle at least 300 4-byte signals in total.

Note: The requested maximum bandwidth does not need to exceed bandwidth requirement stated above.

4.10.11 ECU Running Log Requirements

4.10.11.1 Running Log Requirements

•Introduction

4.10.11.1.1 Introduction

4.10.11.1.1.1 Guidelines for logging

The meaning of Logging within SW components is therefore different to how people in the automotive industry typically relate to "logging" (which often involves getting a log file containing all communication on e.g. a complete CAN bus).

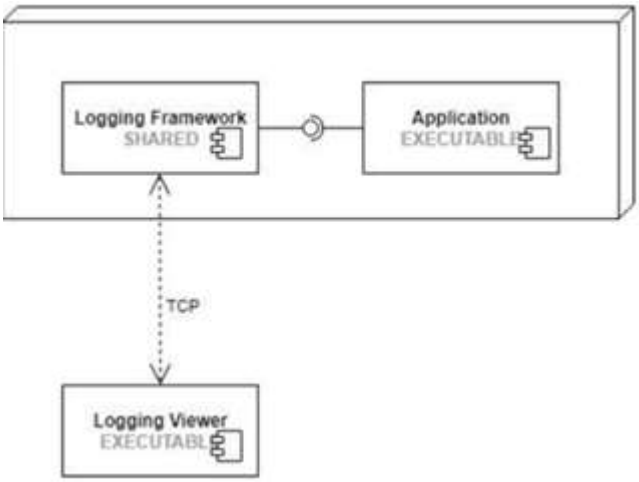
Basic guidelines when it comes to Logging Severity Levels:

- **Fatal** - The application can't continue to execute and cannot recover (with included message about why).
- **Error** - Something is not working as expected and the requirements can't be met. Error should not happen during execution.
- **Warning** - Typically something has occured that doesn't stop the execution flow of the app, but might be of importance. Warnings may happen during execution and the severity increase in case of multiple warning messages.
- **Info** - All the things that is not a problem but adds context to the system state, such as the application flow.
- **Debug** - More detailed or frequent logging which could help a programmer understand unexpected code behaviour.
- **Verbose** - The most detailed logging level. Info such as "package content" and detailed

"struct" info. Typically more detailed and frequent outputs than Debug level.

•Requirements

4.10.11.1.2 Requirements



Application - user application that can be implemented by ZEEKR or Third-party developers.
Logging Framework - overall name for a set of components that enables possibility to log messages.
Logging Viewer - Receiving and storing log messages from the Logging Framework and sending control commands.

976154 v1 **Application registration**

Version history:
(1):

Purpose: To be able to filter and associate logging information with the origin, it is necessary that applications register themselves at the logging framework.
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738384

The Logging Framework shall provide a possibility to register an Application using a unique four-character Application ID and a description.

Note:
If an Application is not registered in the Logging Framework log messages from the ring-buffer shall not be transmitted to the Logging Viewer

976160 v1 **Application unregistration**

Version history:
(1):

Purpose: To free the resources that are not needed for the application at the moment.
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738393

Logging Framework shall unregister itself at the de-initialization stage of an Application.

The Logging Framework shall take care that all remaining messages in the buffer would be sent to the Logging Server.

All logging registered contexts, structures, ring-buffer and receiver object must be de-initialized.

976163 v1 **Check current severity level**

Version history:
(1):

Purpose: To avoid unnecessary CPU and/or memory consumption, which is needed for the generation of logging information, applications shall have the possibility to first check whether or not its created logging information will be filtered out anyway by the underlying logging framework.

Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738398

Logging Framework shall provide the possibility for an Application to check the current severity level to avoid unnecessary computations.

976156 v1 **Context initialization**

Version history:
(1):

Purpose: The context is used to log application messages with specified severity.
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738386

The Logging Framework shall provide a possibility to register a unique four-character Context ID and its description.

Note:

Without the registration of a Context ID the log message cannot be created.

976166 v1 **Core dumps configuration**

Version history:
(1):

Purpose: Ability to create postmortem memory data of crashed application(s) for the future analysis.
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738404

The ECU SOC platform and its applications should be configured in a proper way to generate a postmortem core dump file when the application was crashed with debug information that is needed for future analysis.

Note: Shall be removed from the production cars

976145 v1 **Default log level**

Version history:
(1):

Purpose: To avoid unnecessary CPU, memory and network consumption, which is needed for the generation and transferring of logging information, applications shall have the default logging level.
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738354

The default logging level for Logging Framework on the SOC shall be "Info".

Note:
All logs with lower severities ("Debug", "Verbose") shall be dropped to improve performance.

976175 v1 **Delete locally stored logs**

Version history:
(1):

Purpose: To enable deletion of persistent log data to free up memory
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738426

The Logging Viewer shall provide a possibility to clean up locally stored logs.

976152 v1 Destination of logging information

Version history:
(1):

Purpose: There are different possibilities where the logging information can be stored.
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738376

Configuration of the Logging Framework shall allow to select the destination of the provided logging information -forward logging information to the console or send it via the communication bus.

976158 v1 Early logging

Version history:
(1):

Purpose: This requirement shall guarantee that the logging library can provide its services to other applications before the complete initialization of all SOC platform.
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738390

If an Application cannot connect to the Logging Framework, all log messages shall be buffered locally in a memory ring-buffer of configurable size.

976161 v1 Grouping of logging messages

Version history:
(1):

Purpose: The Application id and Context id are used to logically group logging messages generated by an application.

Source:

Verification Method: Test

Legacy ID:

Zeekr Requirement ID: 738394

The Logging Framework and Logging Viewer tool shall provide a possibility to logically group and filter log messages by using Application ID and Context ID.

976177 v1 **Length of the logging message**

Version history:
(1):

Purpose: Its important that the traffic produced by the Logging component will not cause an overload of the network and the SOC resources.

Source:

Verification Method: Test

Legacy ID:

Zeekr Requirement ID: 738428

The maximum length of the Logging Framework message shall be 1024 bytes.

Note:

Messages longer than configured will be truncated.

976173 v1 **Local log storage**

Version history:
(1):

Purpose: To prevent loss of important log data for some special use-cases.

Source:

Verification Method: Test

Legacy ID:

Zeekr Requirement ID: 738424

The Logging Framework shall have a possibility to store logs locally on the SOC file system. When the logs are full, no further logging storage shall be possible.

976174 v1 Local log storage configuration

Version history:
(1):

Purpose: To enable more fault tracing information in systems where we experience intermittent problems in the field

Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738425

The Logging Framework shall support a configuration of local log storage and shall include:

· application name;

· context name;

· log level:

· file size;

· synchronization strategy;

Note:

Default configuration shall be to not store logs persistently.

976147 v1 Logging API

Version history:
(1):

Purpose: Forward logging information to the logging framework.
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738358

The Logging Framework shall provide a Logging API that can be used by any Application.

976153 v1 Logging server temporary memory storage

Version history:
(1):

Purpose: To prevent a lack of communication e.g. at startup or if no external client is

connected, the logging system shall store some messages to be dumped as soon as communication channel is available.

Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738379

If there no Logging Viewer is connected to the Logging Framework for dumping log messages, log messages shall be stored locally in a memory ring buffer. The size of the buffer shall be set at configuration time.

976180 v1 **Message discarding notification**

Version history:
(1):

Purpose: To inform platform user in case of dropping some application's messages
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738432

Logging Framework shall inform the developer if the application’s message can’t be sent.

Note:

The notification way should not conflict with *[Req ID : 738363]* .

976168 v1 **Number of core dumps**

Version history:
(1):

Purpose: In order to not impact the performance by transfering big amount of files
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738411

The Logging Framework shall process up to 3 core dump files at same time for an Application. If the Application produced more core dumps, the new ones should be skipped.

976162 v1 **Number of logging contexts**

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID:
Zeekr Requirement ID: 738397

The Logging Framework shall allocate 500 logging context entries for an Application.

976155 v1 Providing logging severity

Version history:
(1):

Purpose: While developing applications, it is important to get additional information what is going on internally of an application.

Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738385

The Logging Framework shall include information about severity in the logging message.

976150 v1 Raw data

Version history:
(1):

Purpose: Enable applications and ZEEKR ARK OS services to log raw binary data. The original content of a buffer might be of interest for developing purpose.

Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738362

To log raw binary data, data size and pointer to the first element should be provided to the corresponding function of the Logging Framework.

976169 v1 Request for transferring core dump

Version history:
(1):

Purpose: In order to not impact the performance by transfering binary files

Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738415

The Logging Framework shall transmit core dump files only by the request from the Logging Viewer

976176 v1 **Retrieve locally stored logs**

Version history:
(1):

Purpose: To enable offline analysis of data collected during a long time.
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738427

The Logging Viewer shall provide a possibility to retrieve locally stored logs.

976167 v1 **Retrieve saved core dumps**

Version history:
(1):

Purpose: Ability to transfer postmortem memory data of crashed application(s) for the future analysis.
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738406

The Logging Framework shall transmit core dumps to the connected Logging Viewer using DLT file transfer protocol.

976165 v1 **Run-time configuration of severity level**

Version history:
(1):

Purpose: To adapt logging behavior to SOC and network resources it is important to have the capability of configuring it.

Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738402

ID:

Logging Framework shall permit to turn off and set log levels for registered Application IDs and Context IDs at runtime.

976171 v1 **Shaping network traffic**

Version history:
(1):

Purpose: Since Logging shall be used also in production phase, it is important that the traffic produced by the Logging component will not cause an overload of the network and the SOC resources.

Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738418

The Logging Framework may shape the network traffic bandwidth depending on which transport interface is used for communication. The traffic shaping shall be done at runtime.

976151 v1 **Silence execution**

Version history:
(1):

Purpose: To comply ASIL-B requirements and do not disturb the application with events or exceptions

Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738363

Any errors occurring inside the Logging Framework or underlying system, shall not bother the applications and silently discard the log messages.

976178 v1 **Size of core dump file**

Version history:
(1):

Purpose: In order to not impact the file system by storing big core dump files

Source:
Verification Method: Test
Legacy ID:

Zeekr Requirement ID: 738430

Configured maximum size of the core dump file shall be 100 MB.

976179 v1 **Size of core dumps storage**

Version history:
(1):

Purpose: In order to not impact the file system by storing big amount of files
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738431

The Logging Framework shall ensure that save core dumps don’t exceeds a configurable max size for core dump storage. Configured maximum size of the storage shall be 1 GiB.

976164 v1 **Size of logging API temporary memory storage**

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID:
Zeekr Requirement ID: 738400

Configured Logging API buffer size shall be 500 000 bytes.

976159 v1 **Size of logging server temporary memory storage**

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID:
Zeekr Requirement ID: 738392

Configured Logging Server buffer shall be 500 000 bytes.

976170 v1 **Stream log data to the Security Gateway**

Version history:
(1):

Purpose: To support debugging during development and fault tracing in production vehicles.
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738417

The Logging Framework shall provide logs over the Security Gateway to SOC network on the TCP port 3490.

976149 v1 **Supported data types**

Version history:
(1):

Purpose: Enable applications and ZEEKR ARK OS services to log events and additional data.
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738361

Logging Framework shall support next messages data types:
· generic C data types
· pointer to the char data (C string)
· const std::string (C++ string)
· pointer to void array (raw binary data)

976172 v1 **Supported network protocols**

Version history:
(1):

Purpose: To be able investigate logs from the application
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738422

The Logging Viewer shall be able to extract the logs from the Logging Framework via TCP protocol

976146 v1 Supported programming languages

Version history:
(1):

Purpose:
Source:
Verification Method: Inspection
Legacy ID:
Zeekr Requirement ID: 738357

The Logging Framework shall support following program languages:
· C99 (ISO/IEC 9899:1999)
· C++14 (ISO/IEC 14882:2014

976148 v1 Suppressing logging messages

Version history:
(1):

Purpose: To reduce redundant computation and intermediate API calls.
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738360

Calls from an Application to the Logging Framework with a disabled log level shall be suppressed in runtime to avoid redundant computations and operations.

976157 v1 Timestamp

Version history:
(1):

Purpose: The optional Timestamp is used to add timing information on when a log message has been generated.
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 738388

Each log message should be timestamped with a Vehicle Time in the Logging Framework from the moment an Application generates the message.

4.11 Software download

4.11.1 General Software Download

4.11.1.1 Generic SWDL Overview

•Introduction

4.11.1.1.1 Introduction

Electronic Control Units (ECUs) have demonstrated continued increase in feature, count, memory size, I/O circuit count, and part number complexity. All areas of the vehicle electrical/electronic system are affected. Electronic Control Units are experiencing late and frequent changes many of which affect software. These trends will undoubtedly continue into the foreseeable future.

There is a need to reduce vehicle build and service complexity, reduce ECU supplier complexity, and allow for maximum flexibility in software design by establishing designs and procedures providing the ability to configure and program ECUs in assembly and in service. Configuration and module programming will allow module design and release activities to minimize part complexity, economies of scale and allow bookshelf software to be utilized.

By having software download possibility in all ECUs with complex functionality the cost of correcting software errors is significantly reduced and new functions can be added after production. The software download functionality is used both in the production line to customize the hardware for the specific car and in workshops to correct errors or improve functions. The software download concept is based on a primary bootloader that is fixed in the ECU and always executed at power on before the application is started and a secondary bootloader, including non-volatile memory programming routines that is downloaded before downloading the rest of the applications and configuration files.

Important is to reduce the download time as much as possible to enable a convenient user experience and reduced assembly and workshop time/cost spent on the programming phase. Important technologies such as compression and delta encoding of data reduce the software size as well as using available bandwidth by parallel download of multiple ECUs is therefore motivated.

•What is the purpose of this document

4.11.1.1.2 What is the purpose of this document

This specification is the top document of the SoftWare DownLoad (SWDL) concept. The specification describes the background and the non-data link dependent requirements for a bootloader as well as the programming sequences used for software download. For details on specific protocols, data links, components and algorithms, please read the respectively documents.
This specification is intended to provide guidance and requirements to architects, system designers, application developers, and ECU suppliers. Adherence to the specifications contained within this document is required to ensure compatibility within the corporate product line from vehicle to vehicle, model year to model year and maintain compatibility with the industry driven standards.

Important to understand is that this specification only specifies the general non network specific requirements for a bootloader.

•Overview

4.11.1.1.3 Overview

Software Download involves the transfer of information from a tester to an ECU using data stored in a file. The data transferred may contain configuration or calibration data for the ECU or completely new software. It also includes uploading of data from the ECU to the tester for debugging.
The Software Download concept provides a means for a tester to send multiple bytes of data to an ECU.

•Requisite documents

4.11.1.1.4 Requisite documents

The following documents are either referenced by this specification, or contain information that is relevant to this

specification.

Reference	Source	Document
[SWDL_1]	Internal	VBF (Versatile Binary Format)
[SWDL_2]	Internal	Data Compression and Encryption
[SWDL_3]	Internal	Delta encoding
[SWDL_4]	Internal	UDS Services
[SWDL_5]	Internal	UDS Data
[SWDL_6]	Internal	UDS Session
[SWDL_7]	Internal	Diagnostic and Bootloader Gateway
[SWDL_8]	Internal	Software Authentication
[SWDL_9]	Internal	General Software Authentication
[SWDL_10]	ISO	ISO14229-1: Road vehicles — Unified diagnostic services (UDS) — Part 1: Specification and requirements. Revision as stated in *[SWDL56] UDS Data.*

Note:
Internal references is typically found as a separate chapter in the ECU specific Base Technology SWRS.
Information obtaining ISO documents may be found on the internet: www.iso.org

4.11.1.1.4.1 Referenced document, structure and relations

The Software Download concept includes a list of specifications that has a tight relation to each other, the Figure "*SWDL specification structure*" , shows the overview and their relation to the ISO documents which is the base.

All listed documents are not applicable for each and every ECU. Which specifications that are applicable depends of type of data link, whether the ECU is a gateway or not etc. It is each SW-responsible to request the applicable requirements to each ECU supplier. Note that the specific data link, network and transport protocol specifications not are referenced from this document.

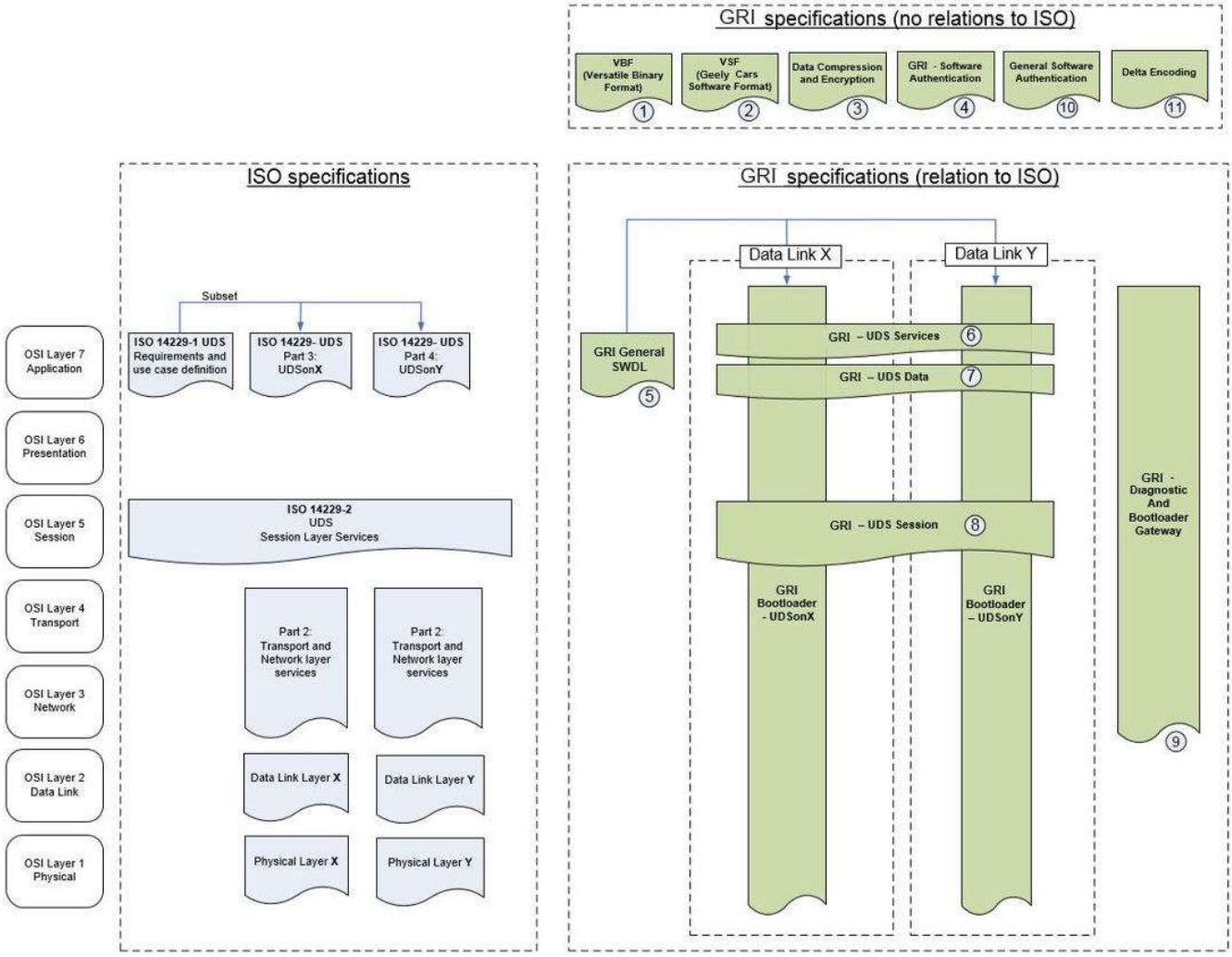


Figure - SWDL specification structure

1 - The "*VBF (Versatile Binary Format*)" data file format is supporting ECUs with addressed based memory data storage.

2 - The "*Data Compression and Encryption*" specifies how data stored in the data file shall be compressed and how the data shall be decompressed by the ECU. The specification includes a reference implementation of the two algorithms.

3 - The "*Software Authentication*" is defining a method to protect that the data file origins from a legitimated source.

4 - The "*General Software Download*", is the top document for the software download concept. It describes the background and the non-network dependent requirements for a bootloader as well as the programming sequences used for software download.

5 - The "*UDS Services*" specifies which diagnostic services that is applicable for the bootloader (programmingSession).

6 - The "*UDS Data*" specifies the data used by each diagnostic service that is applicable for the bootloader (programmingSession).

7 - The "*UDS Session*" specifies the requirements for the session layer.

8 - The "*Diagnostic and Bootloader Gateway"* specifies the gateway requirements for those ECUs that act as a

diagnostic gateway.

9 - The "*General Software Authentication*" specifies the Software Authentication concepts, i.e. methods used to ensure that the software is authentic.

10 - The "*Delta Encoding"* specifies the requirements to enable delta encoding as compression method for data files.

•Definitions and Abbreviations

4.11.1.1.5 Definitions and Abbreviations

Definition	Description
Broken software	The software present in the ECU is only partly downloaded.
Cold Start	The start-up of the ECU from a powered-down or off state.
Compatible check	Check that all data files fit together and the ECU is capable to starts its normal application functionality.
Complete check	Check that every individual data file is present, this can be done that each file has a signature placed in the beginning and end of each file that are checked.
Tester	A system that controls functions such as test, inspection, monitoring, or diagnosis of an on-vehicle electronic control unit and may be dedicated to a specific type of operators (e.g. a scan tool dedicated to garage mechanics or a test tool dedicated to assembly plant agents). The tester is also referenced as the client.
Final response	Positive or a negative response other than response code 0x78 "requestCorrectlyReceived-ResponsePending"
Program mode	The ECU executing the primary or secondary bootloader
Public ECU	An ECU addressable from a client/tester point of view.
Private ECU	An ECU not addressable from a client/tester point of view.
Warm start	The start-up of the ECU initiated by the application.

Abbreviation	Description
CRC	Cyclic Redundancy Check
ECU	Electronic Control Unit
HW	HardWare
NRC	Negative Response Code
PBL	Primary Bootloader
SBL	Secondary Bootloader
SPRMIB	SuppressPositiveResponseMessageIndicationBit
SW	SoftWare
SWDL	Software Download
UDS	Unified Diagnostic Services

4.11.1.2 General Bootlaoder requirements

•Target

4.11.1.2.1 Target

Allow any microprocessor based ECU to be configured and fully programmed at any assembly plant or dealership, via approved data links.
The following requirement principles shall be met for software download to an ECU.

857424 v3 **Memory Requirements**

Version history:
(3): Increase the constraint requirement on dedicated memory for ECU during the upgrade process.

Purpose: To ensure the smooth progress of SWDL, the requirements has been made on the storage memory for ECU software part or more.

Source:
Verification Method: Analysis
Legacy ID:

ECU shall allocate reasonable and dedicated memory for each software part during update. These memory shall be dedicated to software part storage to avoid overlapping use with other data partitions. If the software part needs to share the storage partition with other data due to memory size limitations, it needs to be approved by the Basetech team.

Comment:
Based address memory ECU, receives software part data in RAM, and after processing (decompression/decryption), it is directly written into the program flash partition.

Based file memory ECU, requires software part storage memory and another storage memory for the original installation files obtained by decoding/refactoring software parts (in most cases), which means there will be at least two storage memory. Also additonal buffer memory.
If these two storage memory share storage areas with other temporay data, the temporary data shall be cleared and the storage memory for update shall be organized before enabling software part transfer. The clearing action shall be performed asynchronously to avoid diagnostic response timeout.

For ECU supporting OTA distributed update, it should have independent and sufficient memory to store and update software during SWDL process, rather than being shared with other upgrade methods such as OTA distributed upgrades.

662755 v1 **Non-volatile memory for software storage**

Version history:
(1):

Purpose: To be able to modify/upgrade stored software(s)
Source:
Verification Method: Analysis AND Test
Legacy ID: 3163v3

All software(s) shall be stored in a non-volatile memory, SBL excluded.

681678 v1 **Reprogramming ability**

Version history:
(1):

Purpose: To prevent ECU from not responding to any requests.
Source:
Verification Method: Test
Legacy ID:

The ECU software shall be designed in such way that an aborted download or install, or an wrong download or install with incorrect data, or an wrong download or install with mismatch hardware, or removing ECU power at any time during the download and install operation, shall not prevent subsequent reprogramming of the same ECU software file.

Note1: Subsequent reprogramming means that the diagnostic tester is restarting the download from the beginning.

•Power consumption

4.11.1.2.2 Power consumption

662756 v4 **Power Consumption**

Version history:
(4): Adjust current consumption standard based on different types of ECUs.

Purpose:
- To minimize the power consumption in programming mode.
- To avoid damage to the ECUs, actuators, sensors, chips, etc.
- Verified by OTA Team.

Source:
Verification Method: Analysis AND Test
Legacy ID: 3121v3

When an ECU executing either the primary or secondary bootloader, all of the ECU's I/O and controls shall be set into a safe and minimize power consumption where components cannot be damaged and the area is safe for people working on the vehicle.
Note: "safe" means that ECU shall ensure all controls (e.g. actuators, sensors, etc) are in a safe state before and after entering and exiting the primary or secondary bootloader.

Minimize power consumption means that the ECU only shall supply the necessary components to perform software download. The requirements for ECU(s) are as follows

Item	ECU Type	Power consumption
Type 1	General ECU	≤ 150 mA
Type 2	Zone ECU Integrated multiple ECU capabilites	≤ 200 mA
Type 3	ECU with auxiliary processors	≤ 250 mA
Type 4	ECU with POSIX system	≤ 300 mA
Type 5	ECU with POSIX system and OTA-distributed capability	≤ 2000 mA

The implementation of minimizing power consumption, for example:
- Control the LIN Slave Node to enter sleep mode.
- Output shutdown signal for controlled business.
- Disable any SWDL unrelated actuators, sensors, chips, and so on.

Note1: It shall be allowed with a deviation through clarifying the composition of power consumption to OTA Team.
Note2: The supplier shall provide a complete and detailed report explaining the current consumption of the ECU during the programming session.

•Internal data transfer speed

4.11.1.2.3 Internal data transfer speed

662757 v1 **Internal data transfer speed**

Version history:
(1):

Purpose: To improve the total programming time
Source:
Verification Method: Analysis
Legacy ID: 3120v3

To improve the total programming time the internal transfer time between the bus communication controller and the

memory (where the data is stored) shall not exceed the memory writing time.

Note: The non-volatile memory itself shall set the dimensioning factor for a SWDL operation.

•Handling of distribution relays

4.11.1.2.4 Handling of distribution relays

662758 v1 **Handling of distribution relays**

Version history:
(1):

Purpose: To avoid interruptions on any ongoing communication.
Source:
Verification Method: Analysis AND Test
Legacy ID: 3119v6

An ECU controlling the supply to other ECUs must ensure a stable supply during the transition from the application to the bootloader and vice versa. No glitches or other interruptions are allowed if the supply was enabled in application state. While the bootloader executes, all supplies that feed other ECUs shall be enabled.

•High Voltage Control

4.11.1.2.5 High Voltage Control

716395 v1 **High Voltage Logic Function Control**

Version history:
(1): 增加下高压指令的Basetech要求。

Purpose: To ensure upgrading the high-voltage ECUs under low-voltage power supply
Source:
Verification Method: Test
Legacy ID:

There is an ECU (e.g. XCU or VCU/PCMU or ECM) on the vehicle that controls the high-voltage strategy of the power supply, and this ECU needs to support controlling and requesting the disconnection of high-voltage power supply after receiving specific diagnostic routines.

The following data identifiers should be taken into account, and exactly as defined in *Global Master Reference Database.*

RID [Hex]	0208
Name	Request BECM to Open HV Battery Contactor and inhibit HV On during the whole programming

Description	The routine shall request the power battery ECU to open HV battery contactor and inhibit HV On function. Note1: The positive response shall be returned after executing the HV disconnection logic flow and receiving result feedback. If successful, return routine status - RoutineCompleted. Otherwise, routine status - RoutineAborted shall be returned. Note2: Only when UsageMode is "Active", the routine shall be executed. Note3: If condition is not met, NRC 0x22 shall be responded. Note4: The inhibition of HV On function is only effective for the current cycle. It means that ECU reset or ECU sleep will clear the state. Attention: The function of RID 4000 with 01 shall be ended by RID 4000 with 00. Although the HV enabled function through RID 4000 with 01 has been activated, RID 0208 can still be used to request the power battery ECU to open HV battery contactor and inhibit HV On during this period.		
Type	1		
$P4_{Server_Max}$	10000ms		
Security access	Level 05/06		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	N
Response parameters	**Start [0x01] RoutineStatusRecords**	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	

Available in application	Available in bootloader
M	N

•Calibration and customer data

4.11.1.2.6 Calibration and customer data

662759 v1 **Calibration and Customer data**

Version history:
(1):

Purpose: To preserve calibration and customer data after the ECU is re-programmed.
Source:
Verification Method: Analysis AND Test
Legacy ID: 3111v3

To avoid re-calibration or re-installation of customer data after ECU programming, the calibration- or customer data shall be preserved. Typical cases are immobilizer codes, mp3 files, navigation waypoints, driver settings, necessary settings in driveline ECU or adaption's etc.

•Two level bootloader

4.11.1.2.7 Two level bootloader

The bootloader is divided into two separate parts; a primary bootloader (PBL) and a secondary bootloader (SBL). The PBL is designed to be permanently placed in a protected flash boot sector, which is activated from reset. The SBL is intended to be downloaded by the PBL into the CPU internal volatile memory, from where it is executed and

thereafter deleted after each use. The SBL can be described as a superset of the PBL, adding functions for erase and program the non-volatile memory.

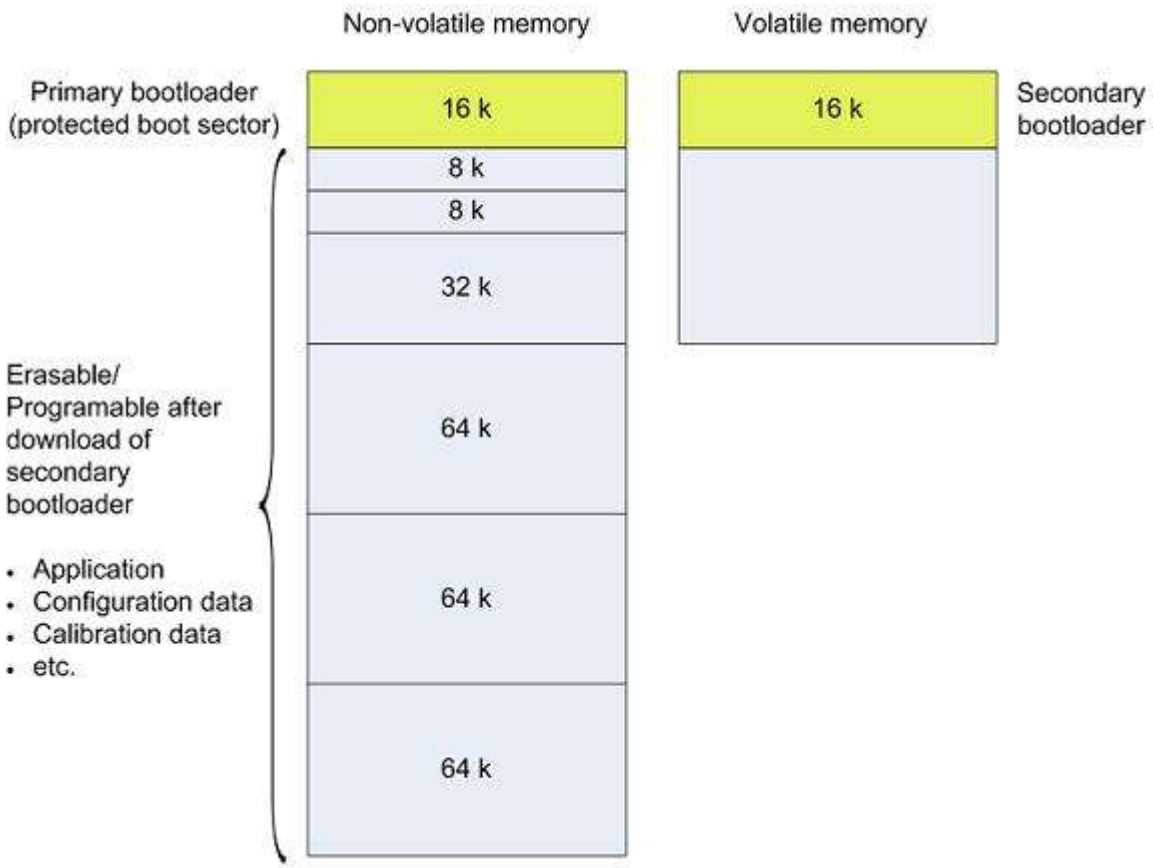


Figure - Typical ECU memory map

852897 v1 **Stucked state handling**

Version history:
(1): 新增需求ECU要有能力避免卡死在PBL或SBL中。推荐增加看门狗的机制，或其他方式。

Purpose: If the ECU is in a state where it cannot process diagnostic programs, in addition to disconnecting the power supply, it should have a way to restore and re-enter a state where it can process diagnostic programs.

Source:

Verification Method: Analysis

Legacy ID:

ECU shall have the ability to reset the state which the program is in an infinite loop in PBL or SBL (including entering and exiting PBL and SBL) . It is recommended to add a watchdog monitoring mechanism in PBL or SBL to ensure that the bootloader program does not get stuck in a loop. If there are other ways, they shall be confirmed by Basetech team.

Attention: Suppliers need to strictly review the loop logic in PBL and SBL codes, and take protectve method to break out of the infinite loop.

662760 v2 **Two level bootloader**

Version history:
(2): LIN slave nodes has low programming speed. If SBL is omitted, PINCode writing in EOL can be efficiently

executed.

Purpose: For security and data integrity purpose the ECU shall support two levels of bootloaders, PBL and SBL.
Source:
Verification Method: Analysis AND Test
Legacy ID: 3165v3

An ECU shall support the software download concept by the use of two bootloaders; primary- and secondary bootloader.

Note: LIN slave nodes are not recommended to support secondary bootloader. If necessary, an approval by GRI Basetech Team is required.

4.11.1.2.7.1 Primary Bootloader (PBL)

The memory area containing the PBL shall be protected from erasure to eliminate the possibility of accidentally erasing it, but if the processor architecture permits this protection to be configured such that it is removable then the protection shall be set up such that it could be removed if needed. The PBL shall not be able to overwrite or modify any of its own memory and the standard SBL shall not be able to overwrite or erase the PBL.

The PBL shall due to security and data integrity only be capable of writing data to the volatile memory. Erasure and programming to the non-volatile memory shall only be possible when the secondary bootloader is downloaded and activated.

When an ECU executes the bootloader all I/O shall be set into state where components cannot be damaged and the area is safe for people working on the vehicle. The PBL must also enable any I/O that is required to power other ECUs that support software download.

662761 v2 **PBL can only write data to volatile memory**

Version history:
(2): 增加特例情况的说明，避免供应商对安全访问延时计数器存储激活信息在PBL中的需求理解矛盾。

Purpose: Due to security and integrity purpose shall the PBL not be able to write or erase data in the non-volatile memory.
Source:
Verification Method: Test
Legacy ID: 3166v2

The primary bootloader shall only be capable of writing data to the volatile memory. Erasure or writing to non-volatile memory is not allowed, with the following exception:
-- In security access the information whether the delay timer after a number of false attempts has been fully activated or not shall be stored.

662762 v1 **PBL in non-volatile memory**

Version history:
(1):

Purpose: The PBL shall always be present in an ECU.
Source:
Verification Method: Test
Legacy ID: 3167v2

The primary bootloader shall be pre-programmed in the non-volatile memory by the ECU manufacturer.

662763 v2 **Pre-condition state of the ECUs IO**

Version history:
(2): Verification Method: Analysis AND Test change to Analysis.

Purpose: To prevent unwanted states of the ECU's I/O.
Source:
Verification Method: Analysis AND Test
Legacy ID: 3168v3

When an ECU executes the bootloader all I/O shall be set into state where components cannot be damaged and the area is safe for people working on the vehicle. The PBL must also enable any I/O that is required to power other ECUs that support software download.

662764 v1 **Protected boot sector**

Version history:
(1):

Purpose: To avoid unintentional erase of the PBL.
Source:
Verification Method: Analysis AND Test
Legacy ID: 3169v3

The memory area containing the PBL shall be protected from erasure to eliminate the possibility of unintentional erasure .

Note: The protection shall be set up such that it could be removed if a special SBL were to be written for this purpose, however the standard SBL or the application shall not be able to overwrite or erase the PBL.

4.11.1.2.7.1.1 Primary bootloader diagnostic data

662765 v1 **Primary bootloader diagnostic data**

Version history:
(1):

Purpose: To define the diagnostic data supported in the primary bootloader.
Source:
Verification Method: Test
Legacy ID: 3170v2

The PBL shall support the diagnostic data required in the programmingSession which are specified in *[SWDL_5] UDS Data*.

4.11.1.2.7.1.2 Primary bootloader diagnostic services

662766 v1 **Primary bootloader diagnostic services**

Version history:
(1):

Purpose: To define the diagnostic services supported in the primary bootloader.
Source:

Verification Method: Test
Legacy ID: 3171v2

The PBL shall support the diagnostic services required in the programmingSession which are specified in *[SWDL_4] UDS Services*.

4.11.1.2.7.2 Secondary Bootloader (SBL)

The SBL includes all routines for erase and program of data to the non-volatile memory.

The SBL is downloaded and activated with the help of the PBL into the volatile memory. The SBL diagnostic services are now added (super set to the PBL). This means that all PBL diagnostic services shall be capable of being executed in the SBL, exception is reprogramming and activation of the SBL itself.

After download activation of the SBL in the volatile memory it shall not be possible to make a new download to volatile memory, the ECU shall reject to execute the request with an NRC. Before a new download to the volatile memory can be performed, an exit from the programmingSession is required (see Figure " *Simplified bootloader state diagram"*). The reason for this is to avoid overwriting the SBL.

662767 v1 **Prevent the possibility to download the SBL when already activated**

Version history:
(1):

Purpose: To avoid overwriting the SBL.
Source:
Verification Method: Test
Legacy ID: 3172v4

After download and activation of the SBL in the volatile memory it shall not be possible to make a new SBL download to volatile memory, the ECU shall reject to execute the request with an NRC. Before a new SBL download to the volatile memory can be performed a hard reset of the ECU is required.
Before the SBL activation request is received a new download of the SBL shall be possible to perform.

662768 v1 **SBL in volatile memory**

Version history:
(1):

Purpose: To avoid resident functionality stored in the ECU which has the possibility to erase and write data to the non-volatile memory.
Source:
Verification Method: Analysis AND Test
Legacy ID: 3173v3

The Secondary bootloader shall be stored in the volatile memory.

662769 v1 **Support of erase and write data to non-volatile memory**

Version history:
(1):

Purpose: Only the Secondary Bootloader (SBL) shall be able to erase and write data to the non-volatile memory.
Source:
Verification Method: Test
Legacy ID: 3174v2

The Secondary Bootloader shall upon request erase and write data to the ECU non-volatile memory, except for the non-volatile memory area containing the primary bootloader.

662770 v1 **The SBL shall include all PBL functionality**

Version history:
(1):

Purpose: To simplify the bootloader implementation, to have the possibility to do a workaround if an error has occurred in the PBL and that no calls are necessary from the SBL to the PBL.
Source:
Verification Method: Analysis AND Test
Legacy ID: 3175v4

The SBL shall include the complete PBL functionality.

4.11.1.2.7.2.1 Secondary bootloader diagnostic data

662771 v1 **Secondary bootloader diagnostic data**

Version history:
(1):

Purpose: To define the diagnostic data supported in the secondary bootloader.
Source:
Verification Method: Test
Legacy ID: 3176v2

The SBL shall support the diagnostic data required in programmingSession which are specified in *[SWDL_6] UDS Data*.

4.11.1.2.7.2.2 Secondary bootloader diagnostic services

662772 v1 **Secondary bootloader diagnostic services**

Version history:
(1):

Purpose: To define the diagnostic services supported in the secondary bootloader.
Source:
Verification Method: Test
Legacy ID: 3177v2

The SBL shall support the diagnostic services required in programmingSession which are specified in *[SWDL_5] UDS Services*.

•Additional bootloader requirements for secondary processors

4.11.1.2.8 Additional bootloader requirements for secondary processors

Some ECU hardware architectures may include additional microcontrollers (secondary processors). If these microcontrollers have programmable memory areas additional bootloader functionality executing on the secondary processor is necessary. In these cases the bootloader will need to establish communication with the secondary processor.

This specification does not include any detail for this communication method and any such additional bootloader resource shall be defined by the implementer. However, the following requirements shall be complied.

662773 v1 **Interrupted communication to the secondary processor**

Version history:
(1):

Purpose: Interrupted communication to the secondary processor shall not prevent subsequent reprogramming.
Source:
Verification Method: Test
Legacy ID: 3109v2

The download communication method implemented on the secondary processor shall be protected such that aborting download or removing ECU power at any time during the download shall not prevent subsequent reprogramming. In addition the complete or partial downloading of any non operational or partly operational software into the secondary processor shall not prevent the subsequent reprogramming of any normally programmable memory area.

662774 v1 **Programming sequence for secondary processors**

Version history:
(1):

Purpose: To keep the same tester programming sequence as for a single processor ECU.
Source:
Verification Method: Analysis AND Test
Legacy ID: 3110v3

The programming and erasure of the additional memory areas shall be implemented such that the tester programming sequence have the same format as for a memory area supported on the main processor (in other words the memory of the secondary processors shall appear to be mapped into the memory space of the main processor for the purposes of software download).

Note: To keep the same programming sequence shall the secondary processors share the same SBL since only one SBL per ECU is allowed.

•Additional bootloader requirements for multiple networks

4.11.1.2.9 Additional bootloader requirements for multiple networks

Some ECU hardware architectures may include additional networks which are connected to one or several internal chips of the ECU. When developing the bootloader shall each requirement for respectively network be considered. For each additional network port , it shall be consistent with the network communication behavior to which it is connected.

This specification does not include any detail for how to merge the different bootloader implementation requirements and any such additional bootloader functionality shall be defined by the implementer. However, the following requirements shall be complied.

662775 v1 **ECUs with multiple networks**

Version history:
(1):

Purpose: To define the implementation of ECUs with multiple networks.
Source:

Verification Method: Analysis AND Test
Legacy ID: 3104v4

The bootloader shall only support software download from the network which is defined to be the incoming network for diagnostic communication.

Note: An ECU is only allowed to have one cluster for software download. if there is any specific case, it shall be approved by GRI Basetech Team.

662776 v2 **Gateway requirement**

Version history:
(2): 引用文档编号修订。

Purpose: To define the requirements with shall apply for an ECU which acts as a diagnostic gateway in the bootloader.
Source:
Verification Method: Test
Legacy ID: 3105v2

An ECU which has more than one network connected that acts as a diagnostic gateway shall implement the requirements defined in *[SWDL_7] Diagnostic and Bootloader Gateway*.

662777 v1 **Multiple networks communication request**

Version history:
(1):

Purpose: Define the multiple network communication behavior
Source:
Verification Method: Test
Legacy ID: 354576v1

The network with diagnostic communication shall always active in programming session. The additional networks without diagnostic communication requirement shall not send or route any undefined message in programming session except there is any special function.

•Additional bootloader requirements for private ECU

4.11.1.2.10 Additional bootloader requirements for private ECU

An ECU may not be addressable from a tester point of view (private ECU). If this private ECU have programmable memory areas that supports software download then additional bootloader functionality executing on the public ECU will be required. In these cases the bootloader needs to establish communication with the private ECU for the purposes of software download to the ECU.

This specification does not include any detail for this communication method and any such additional bootloader resource shall be defined by the implementer. However, the following requirements shall be complied.

662778 v2 **Interrupted communication to private ECU**

Version history:
(2): Verification Method: Test change to Analysis.

Purpose: Interrupted communication to the private network ECU shall not prevent subsequent reprogramming
Source:
Verification Method: Analysis

Legacy ID: 3106v2

The communication method implemented between the public ECU and the private ECU shall be protected such that erasing any or all of the programmable memory areas of the private ECU and removing the ECU power at any time shall not prevent the subsequent reprogramming of any normally programmable memory area in the private ECU. In addition, the complete or partial downloading of any non operational or partly operational software into the programmable memory areas of the private ECU shall not prevent the subsequent reprogramming of any normally programmable memory area.

662779 v1 **Programming sequence for private ECU**

Version history:
(1):

Purpose: To keep the same tester programming sequence as for the public ECU.
Source:
Verification Method: Analysis AND Test
Legacy ID: 3107v3

The programming and erasure of any memory area on a private ECU shall be implemented such that the tester programming sequence have the same format as for a memory area supported on the public connected ECU (i.e., the memory of the private ECU shall appear to be mapped into the memory space of the processor on the public ECU for the purposes of software download).

Note: To keep the same programming sequence shall the private ECU share the same SBL as for the public ECU since only one SBL per ECU address is allowed.

662780 v2 **Read information from a private ECU**

Version history:
(2): Verification Method: Test change to Analysis AND Test.

Purpose: To be able to read information from a private ECU.
Source:
Verification Method: Analysis AND Test
Legacy ID: 3108v2

The public ECU is responsible to respond on the behalf of the private ECU regarding software and hardware part numbers as well as if the software stored on the private ECU is complete and compatible.

•Data compression and encryption

4.11.1.2.11 Data compression and encryption

662781 v1 **Support for (un)compressed data**

Version history:
(1):

Purpose: To decrease the time to download data the ECU shall support decompression of the received compressed data.
Source:
Verification Method: Test
Legacy ID: 3112v4

To decrease the software download time of an ECU,compression of data stream to an ECU shall be used, while the ECU shall still support non-compressed data files.The ECU shall implement the decompression algorithm specified in the reference [Data Compression and Encryption].

Only the tool that makes the data file and the ECU that is receiving the content of the compressed data file need to implement the compression and decopression algorithms respectively.

The tester informs the ECU whether the data is compressed or not.

•Delta encoding

4.11.1.2.12 Delta encoding

The purpose with Delta encoding is to compress the size of the data to be transmitted to the ECU in order to reduce the time/cost spent during the programming phase. The delta encoding method is to calculate the difference between the new and the old software version, where only the difference between the two versions is transferred to the target ECU.

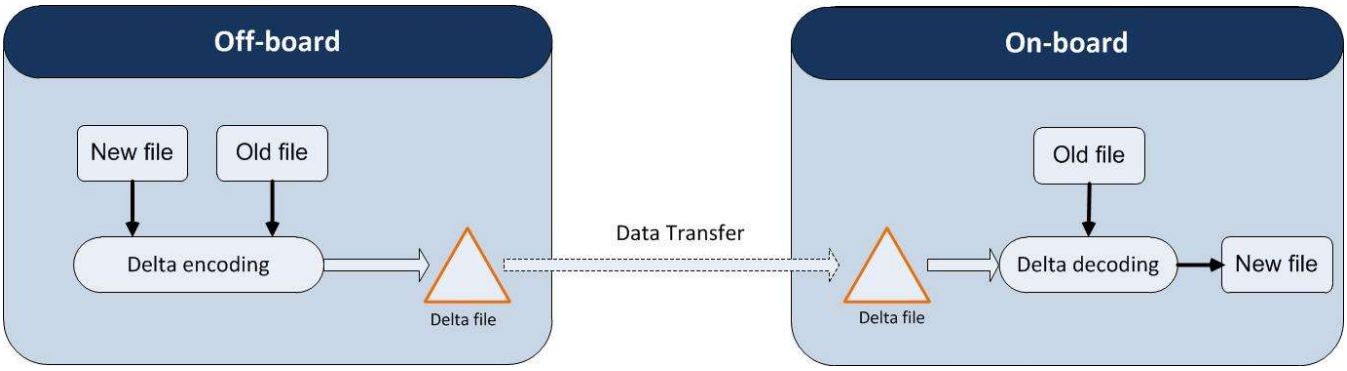


Figure - Principal of delta encoding and decoding

662782 v1 **Delta encoding**

Version history:
(1): 增加差分的需求。

Purpose: To reduce the time spent on the programming phase and the time spent in transfer over data communication networks.
Source:
Verification Method: Analysis
Legacy ID: 396753v2

Delta encoding and decoding method is required when a data file that is contained within a software delivery exceeds 10MB in size.

•ECU program mode

4.11.1.2.13 ECU program mode

Program mode means that the ECU executing the primary or secondary bootloader. This section describes the bootloader state and flow chart diagram in a general way. The specific implementation requirements for the bootloader are data link dependent and are therefore specified in respectively data link bootloader specification.

662783 v2 **Disable monitoring and logging of DTCs**

Version history:
(2): Verification Method: Test change to Analysis AND Test.

Purpose: To prevent logging and monitoring of DTCs when the bootloader executing.
Source:
Verification Method: Analysis AND Test
Legacy ID: 3114v2

The ECU shall ensure that the monitoring and logging of all diagnostic trouble codes (DTCs) is disabled when executing the bootloader (programmingSession).

662784 v1 **Suspend non diagnostic communication**

Version history:
(1):

Purpose: To only have diagnostic communication between tester and ECU when executing the bootloader.
Source:
Verification Method: Test
Legacy ID: 3115v3

Whenever an ECU is executing the bootloader, it shall ensure that suspends non diagnostic communication e.g. network management messages between ECUs are suspended.

4.11.1.2.13.1 Simplified bootloader state and flow chart diagram

The bootloader state diagram, Figure " *Simplified bootloader state diagram"* shows a simplified example of the different states the ECU can enter in the bootloader. The state diagram is in detailed described in each bootloader implementation specification for respectively network.

- From power on or reset initiated by the application the PBL shall start executing the initialization state where the PBL decide whether the application shall be started or not.
- The primary bootloader (PBL) shall have a time window (Timeout_Prog) for detection of the DiagnosticSessionControl service. This backdoor solution shall be used if the ECU believes it has a valid software(s) present, but the application software is not jumping to the PBL or if the application is not able to process a diagnostic request (see section "*Time window for PROG detection"*).
- The programmingSession state is entered if the reset was initiated by the diagnostic request DiagnosticSessionControl(programmingSession) in the application (Warm start) or if no valid software(s) occurs (see section "*Enter the bootloader (PBL) from the application"* and "*Validation of ECU Software(s)"*).
- The application is started if the start-up was a cold start and valid software(s) is present.

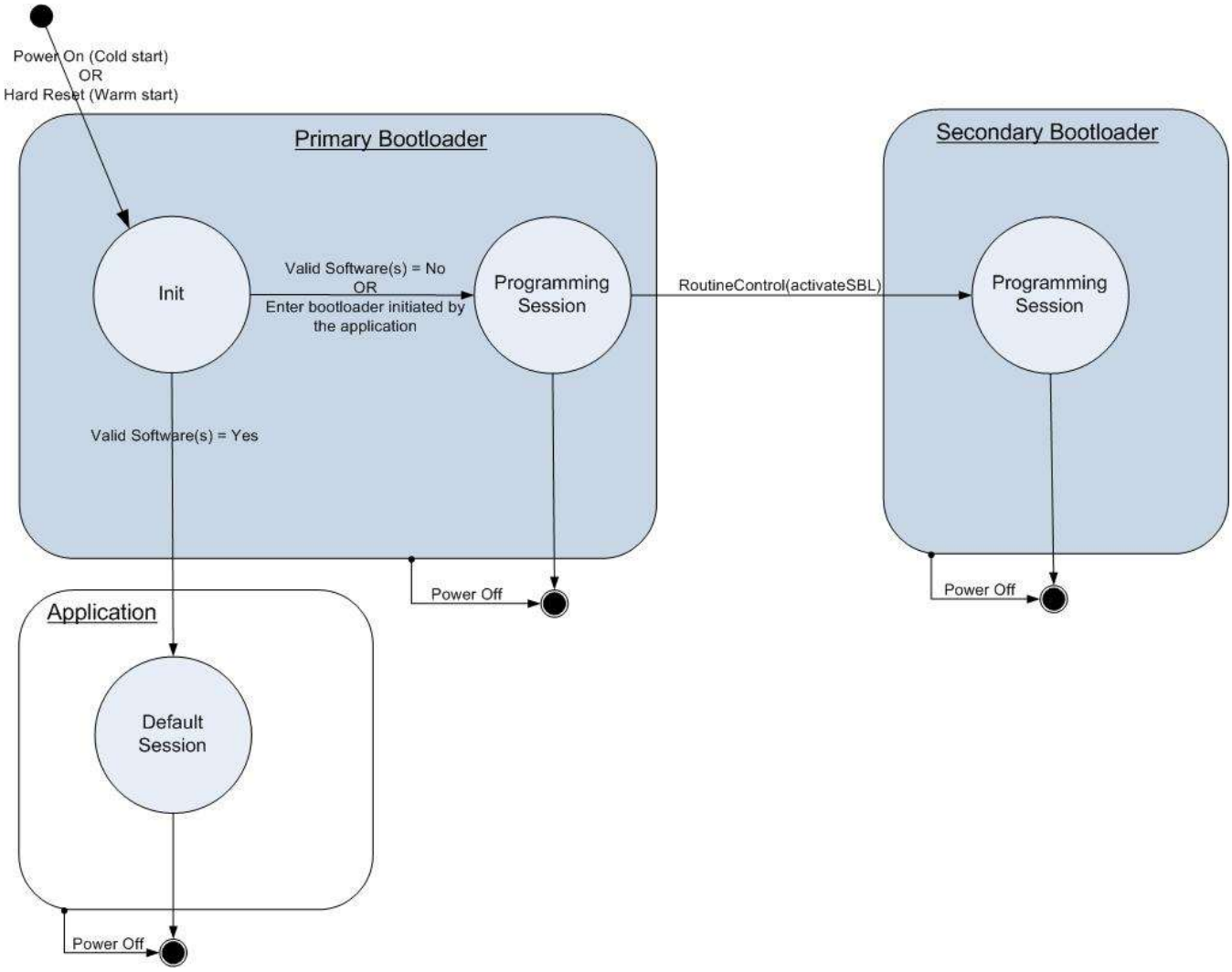


Figure - Simplified bootloader state diagram

The bootloader flow chart diagram, Figure " *Simplified bootloader flow chart diagram"* below shows a simplified example. The flow chart diagram is in detail described in each bootloader implementation specification for respectively data link network.

The Complete and Compatible function which is responsible to make the decision whether valid software(s) is present is in more detail described in section " *Validation of ECU Software(s)"*.

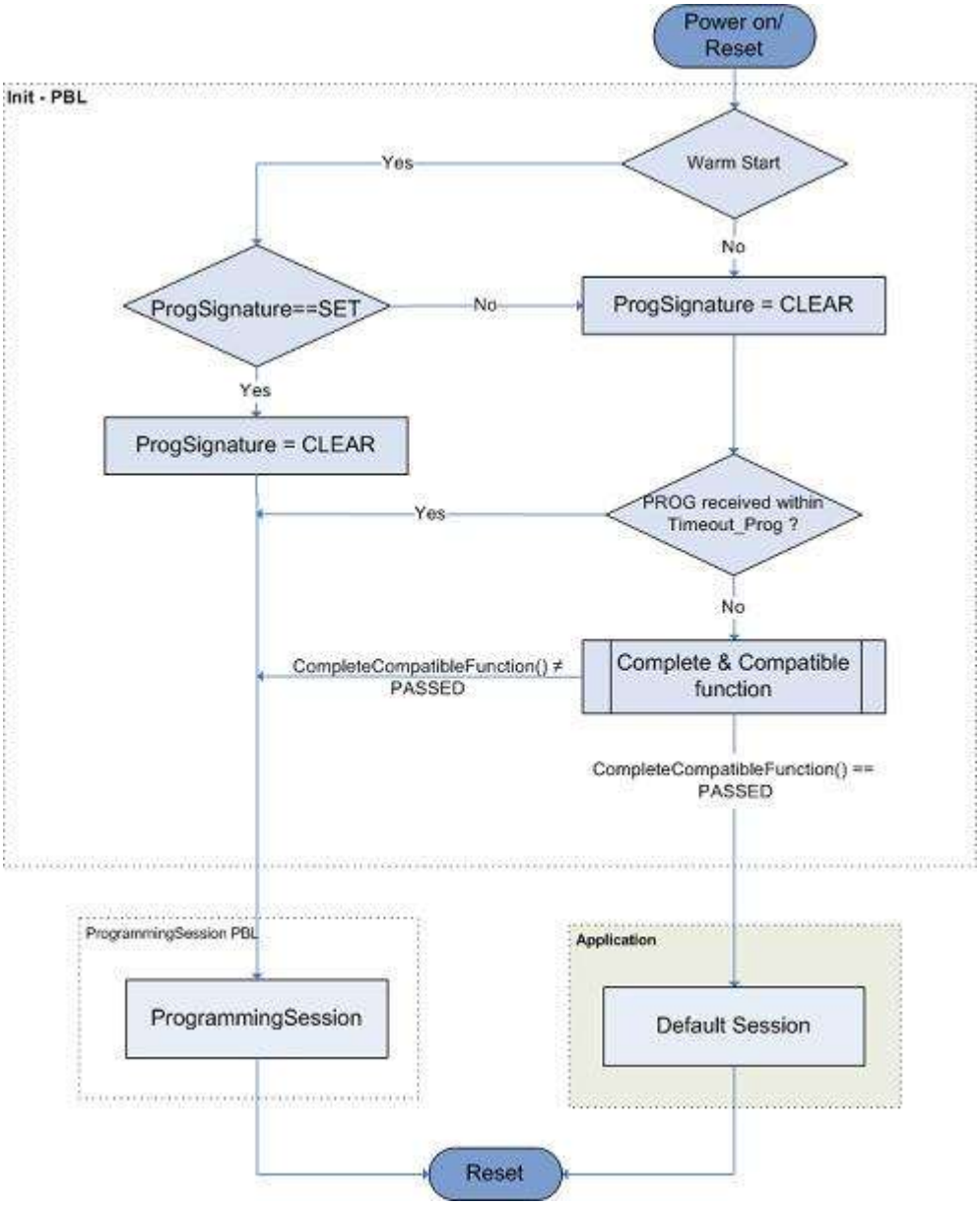


Figure - Simplified bootloader flow chart diagram

•Enter the bootloader (PBL) from the application

4.11.1.2.14 Enter the bootloader (PBL) from the application

If not explicitly stated in respectively bootloader implementation specification the application shall write a signature to the volatile memory after receiving the DiagnosticSessionControl(programmingSession) request.

Signature	**Type**	**Value** (Hex)	**Description**
ProgSignature	SET	0x50726f675369676e	The ProgSignature shall be written to a fixed address in the volatile memory, with a length of 8 bytes and a recommended value of 0x50726f675369676e (equal to text string "ProgSign" in ASCII-format).
ProgSignature	CLEAR	0x0000000000000000	ProgSignature shall be cleared by set the fixed address to zero (0).

Table - ProgSignature value and format

It must be possible to enter the primary bootloader by only receiving the DiagnosticSessionControl(programmingSession) service once, which is the normal way of entering the bootloader. To prevent the application to restart after reset must the application therefore indicate to the bootloader with a signature that programmingSession state where requested.

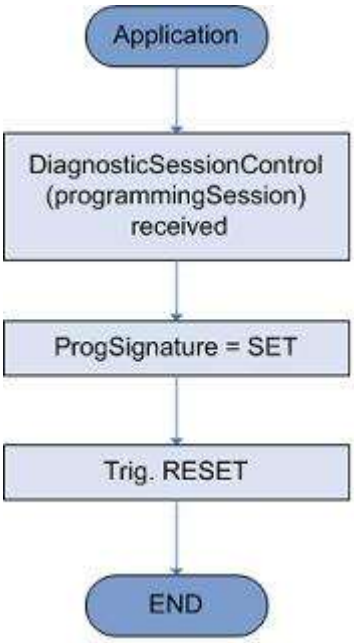


Figure - Write ProgSignature flow chart diagram

662785 v1 **ProgSignature - CLEAR**

Version history:
(1):

Purpose: To prevent unwanted bootloader activation
Source:
Verification Method: Analysis
Legacy ID: 3117v3

The ProgSignature shall be cleared with a recommended value of 0x0000000000000000 by the bootloader before starting the application.

662786 v1 **ProgSignature - SET**

Version history:
(1):

Purpose: To be able to enter the primary bootloader from the application by sending the DiagnosticSessionControl(programmingSession) service once.
Source:
Verification Method: Analysis
Legacy ID: 3116v3

The ProgSignature shall be written to a fixed address in the volatile memory, with a length of 8 bytes and a recommended value of 0x50726f675369676e (equal to text string "ProgSign" in ASCII-format) by the application after receiving the DiagnosticSessionControl(programmingSession) request.

Note: The fixed address must be synchronized between the application and bootloader.

•Time window for PROG detection

4.11.1.2.15 Time window for PROG detection

If not explicitly stated in respectively bootloader implementation specification shall the primary bootloader (PBL) have a time window (Timeout_Prog) for detection of the DiagnosticSessionControl service. This backdoor solution shall be used if the ECU believes it has valid software(s) present, but the application software is not correctly jumping to the PBL. This method may only be used in a stand alone scenario, due to the impossibility for the gateway ECUs to transmit the DiagnosticSessionControl service in time.

The tester shall send out the DiagnosticSessionControl service with diagnosticSessionType set to programmingSession and suppressPosRspMsgIndicationBit equal to TRUE during a minimum time of 5 milliseconds. The repetition rate of the DiagnosticSessionControl service shall be sent within 5ms but preferably at 2 ms or faster.

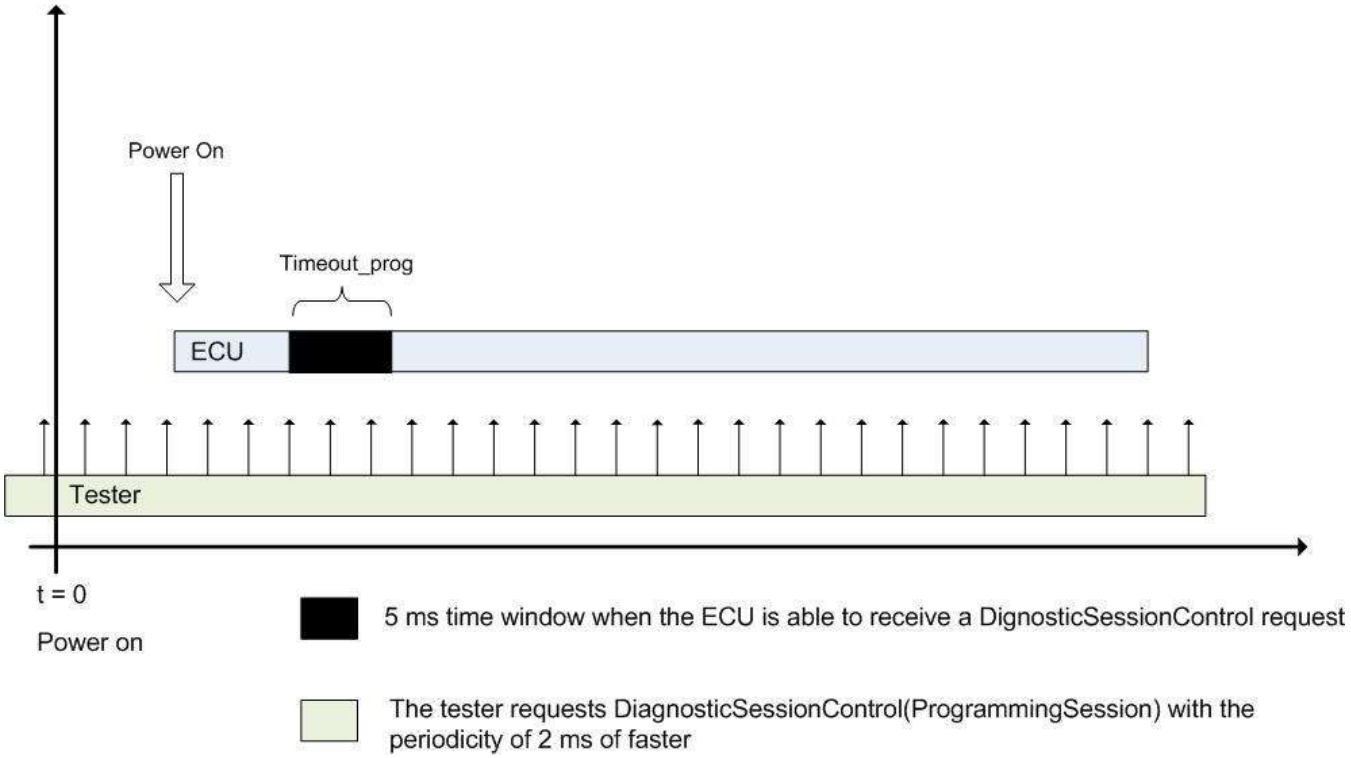


Figure - Typical boot sequence of ECU's in a vehicle

662787 v1 **Time window (Timeout_Prog)**

Version history:
(1):

Purpose: To have the possibility to reprogram the ECU believes if it believes it has valid software(s) present, but the application software is not correctly jumping to the PBL.
Source:
Verification Method: Analysis AND Test
Legacy ID: 3164v3

The primary bootloader (PBL) shall have a time window (Timeout_Prog) of 5 ms (-0% / +10%) for detection of the DiagnosticSessionControl service. If a DiagnosticSessionControl service with diagnosticSessionType equal to programmingSession is received during the time window the ECU shall enter programmingSession state.

The Timeout_prog timer is started when the ECU is able to receive incoming messages.

•File download order

4.11.1.2.16 File download order

The data files which are downloadable to the ECU shall be capable of being downloaded to the ECU individually or in any grouping combination independent of order. The only exception to this is that the secondary bootloader shall always be required to be downloaded first when supported. As an example, if the ECU supports a secondary bootloader (SW - A), Application (SW - B), and a Signal configuration (SW - C), then the following download sequences shall be supported:

- Transition to programmingSession, download SW - A then B
- Transition to programmingSession, download SW - A then C
- Transition to programmingSession, download SW - A, B, then C
- Transition to programmingSession, download SW - A, C, then B

662837 v1 **File download order**

Version history:
(1):

Purpose: To be able to download data files to the ECU individually or in any grouping combination independent of order.
Source:
Verification Method: Test
Legacy ID: 3118v4, 38391a1

No restrictions on the download order/sequence of the downloadable data files are allowed, with the exception of the SBL which shall be downloaded first.

It shall be possible to download each data file separately without any requirements of downloading other data files before, exception is the SBL.

Note: *If an ECU Software Structure (ESS) supported by ECU is to be downloaded, it shall always be downloaded next to the SBL. Added here for information, it is a client requirement.*

•Separate memory sectors

4.11.1.2.17 Separate memory sectors

To decrease the total programming time it shall be possible to replace only the specific data file that shall be replaced and not being forced to replace all software(s) that already are present in the ECU. Since the erase of the memory only can be performed for a complete memory sector shall all data files which are possible to download to the ECU not share same memory sector.

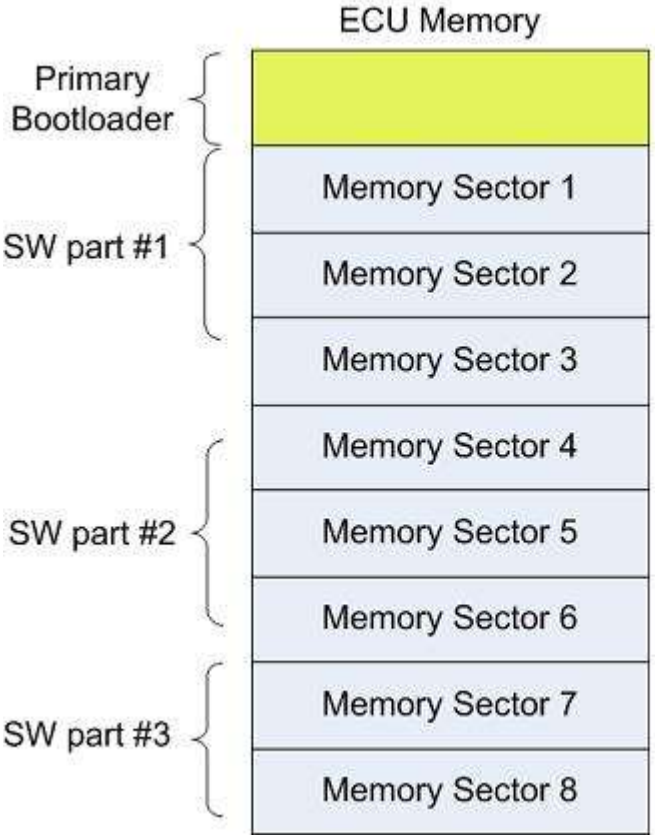


Figure - Separate memory sectors

662838 v1 **Separate memory sectors**

Version history:
(1):

Purpose: To decrease the total programming time it shall be possible to replace only the specific data file that shall be replaced and not being forced to replace all software(s) that already is present in the ECU.
Source:
Verification Method: Test
Legacy ID: 3123v2

Two or more data files shall not share a memory sector.

•General Request Download

4.11.1.2.18 General Request Download

850700 v1 **Checking of received data**

Version history:
(1): 新增需求，ECU在接收36传输的数据时，也对部分数据特征做了识别校验，提前报告失败。这个是可以接受，但需要做出说明。

Purpose: Feasibility of software feature recognition.
Source:
Verification Method: Analysis

Legacy ID:

In addition to meeting OEM software verification requirements, the supplier may verify the characteristics of the received data during the TransferData diagnostic request, which needs to clarify with the Basetech Team.

850799 v1 **Request Download process when download SBL**

Version history:
(1): 新增需求，约定在34/36/37传输过程的状态跳转。

Purpose: Defined the process flowchart to ensure that logical block can be downloaded again successfully.
Source:
Verification Method: Test
Legacy ID:

The ECU shall check that only one logical block is being updated for a single downloaded software part, i.e. one logical block is allowed to be addressed from a valid RequestDownload to a CheckMemory request is detected. If any errors occur during the download sequence, the ECU shall abort the download sequence on a rejected request and enter a state where no logical block is active to recover to wait for next request to program the memory, for example trying download the same software part once again or continue downloading the next software part (another logical block).

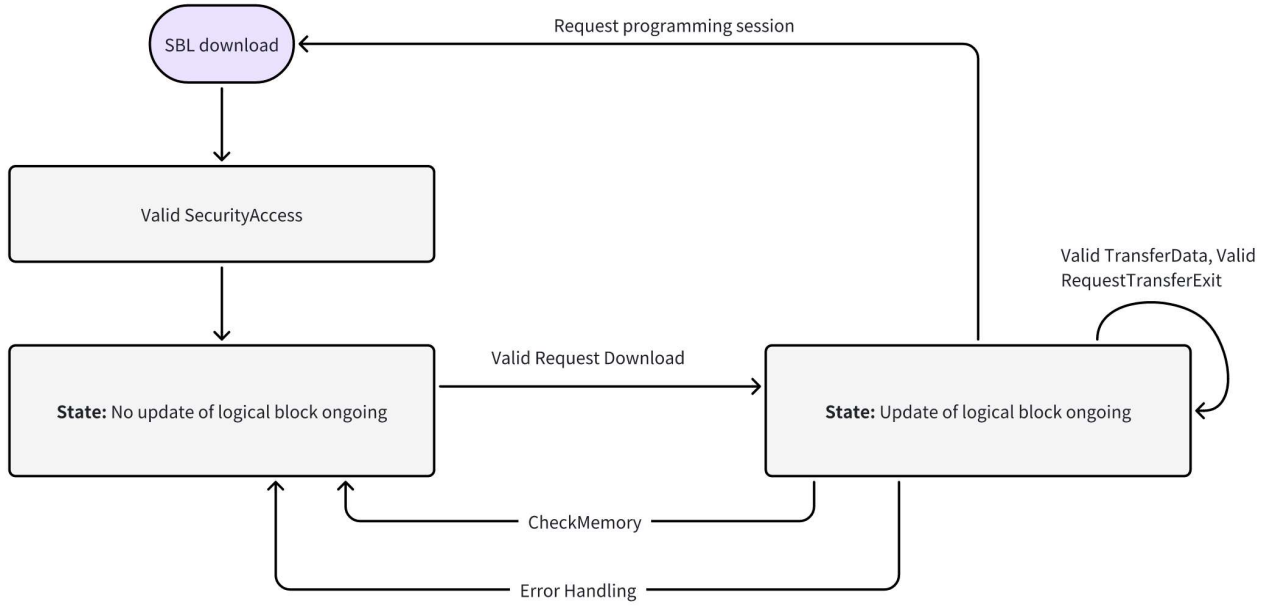


Figure - Simplified logical block active states overview

Notes:

- *"Figure Simplified logical block active states overview" shows the principle only, details are omitted.*
- *"Error handling" includes any negative response during update ongoing, for example:*
  1) Transfer data failure or invalid through TransferData service 0x36.
  2) If a server receives a request for RequestDownload service 0x34 while in the process of receiving a download of a data block, NRC 0x22 shall be returned.
  3) The programming process is not completed when a request for RequestTransferExit service 0x37 is receivied, NRC 0x24 shall be returned.
  4) Other failures, return NRC.

- *If a server receives two consecutive valid requests for RequestDownload service 0x34, for the second request, the server should return NRC 0x22 as "Error handling" or a positive response as a new valid download start.*
- *The figure does not show the actual verification status of each logical block.*

850800 v1 **Request Download process when download to non-volatile memory**

Version history:
(1): 新增需求，约定在34/36/37传输过程的状态跳转。

Purpose: Defined the process flowchart to ensure that logical block can be downloaded again successfully.
Source:
Verification Method: Test
Legacy ID:

The ECU shall check that only one logical block is being updated for a single downloaded software part, i.e. one logical block is allowed to be addressed from a valid RequestDownload to a CheckMemory request is detected. If any errors occur during the download sequence, the ECU shall abort the download sequence on a rejected request and enter a state where no logical block is active to recover to wait for next request to erase or program the memory, for example trying download the same software part once again or continue downloading the next software part (another logical block) after logical block "already erased".

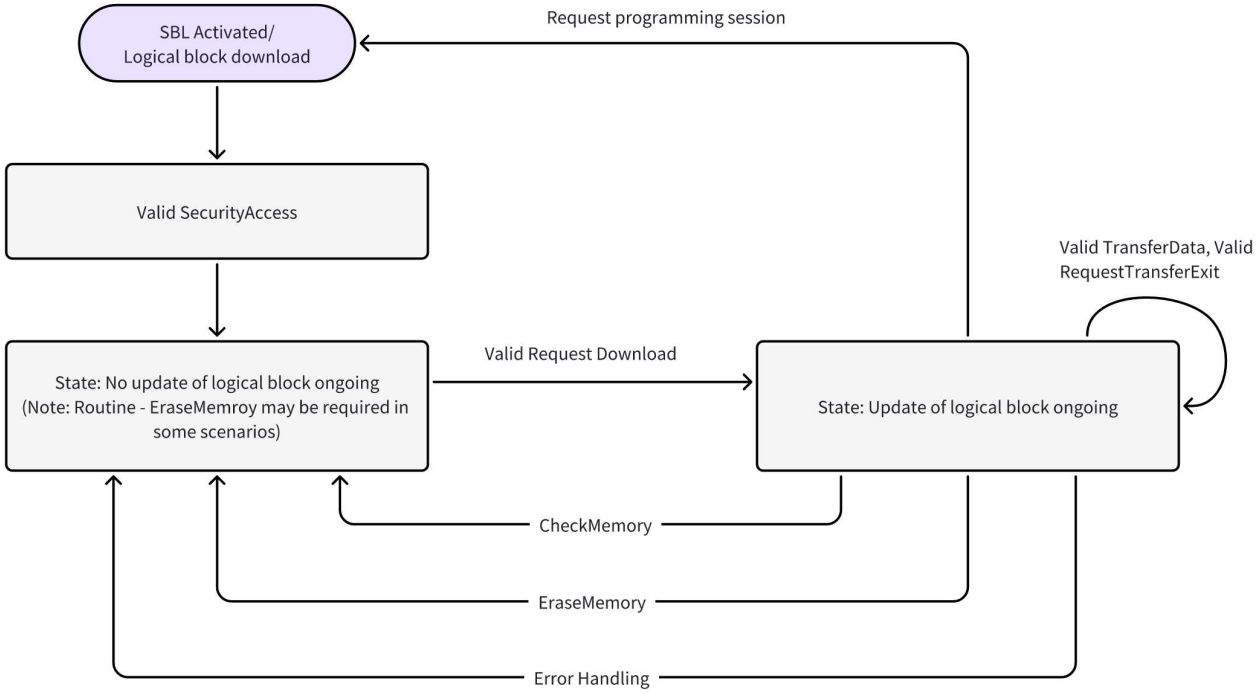


Figure - Simplified logical block active states overview

Notes:

- *"Figure Simplified logical block active states overview" shows the principle only, details are omitted.*
- *"Error handling" includes any negative response during update ongoing, for example:*
  1) Transfer data failure or invalid through TransferData service 0x36.
  2) If a server receives a request for RequestDownload service 0x34 while in the process of receiving a download of a data block, NRC 0x22 shall be returned.

3) The programming process is not completed when a request for RequestTransferExit service 0x37 is receivied, NRC 0x24 shall be returned.

4) Other failures, return NRC.

- *If a server receives two consecutive valid requests for RequestDownload service 0x34, for the second request, the server should return NRC 0x22 as "Error handling" or a positive response as a new valid download start.*
- *The figure does not show the actual verification status of each logical block.*
- *The ECU is having a file based memory structure, not requiring an explicit erase memory request prior to request download.*

•Writing of received data to non-volatile memory

4.11.1.2.19 Writing of received data to non-volatile memory

To decrease the total programming time it is important to start writing data to the non-volatile memory before the complete message is received, decompression included if supported. The positive response shall be sent after all data included in the TransferData request is written to the non-volatile memory.

If the message is interrupted or no response is given to the client the block will be resent. As non-volatile memories normally only allow one write operation for each erase operation the ECU must know how many data bytes it has written to the non-volatile memory. If the client transfers the same block again the ECU shall count the number of received data bytes and start to write data at the first unwritten non-volatile memory position.

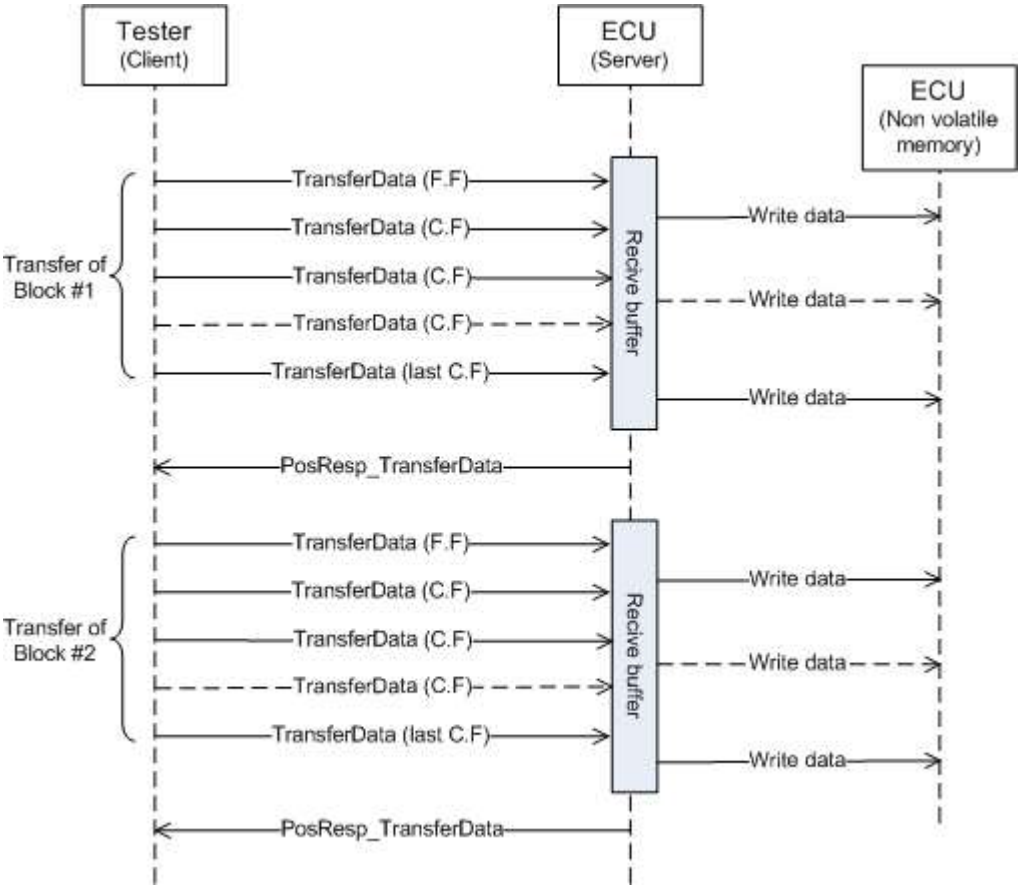


Figure - Writing of data to non-volatile memory

662839 v1	**Writing of received data**

Version history:
(1):

Purpose: To decrease the total programming time the ECU shall start writing data to the non-volatile memory before the complete message is received.
Source:
Verification Method: Analysis
Legacy ID: 3188v2

The ECU shall start writing data to the non-volatile memory before the complete message is received, decompression included if supported. The positive response shall be sent after all data included in the TransferData request is written to the non-volatile memory.

As non-volatile memories normally only allow one write operation for each erase operation the ECU must know how many data bytes it has written to the non-volatile memory. If the client transfers the same block again the ECU shall count the number of received data bytes and start to write data at the first unwritten non-volatile memory position.

•Pre-calculated hash

4.11.1.2.20 Pre-calculated hash

To decrease the total programming time the ECU could pre-calculate, i.e. calculate hash of data before receiving the diagnostic CheckMemory request.

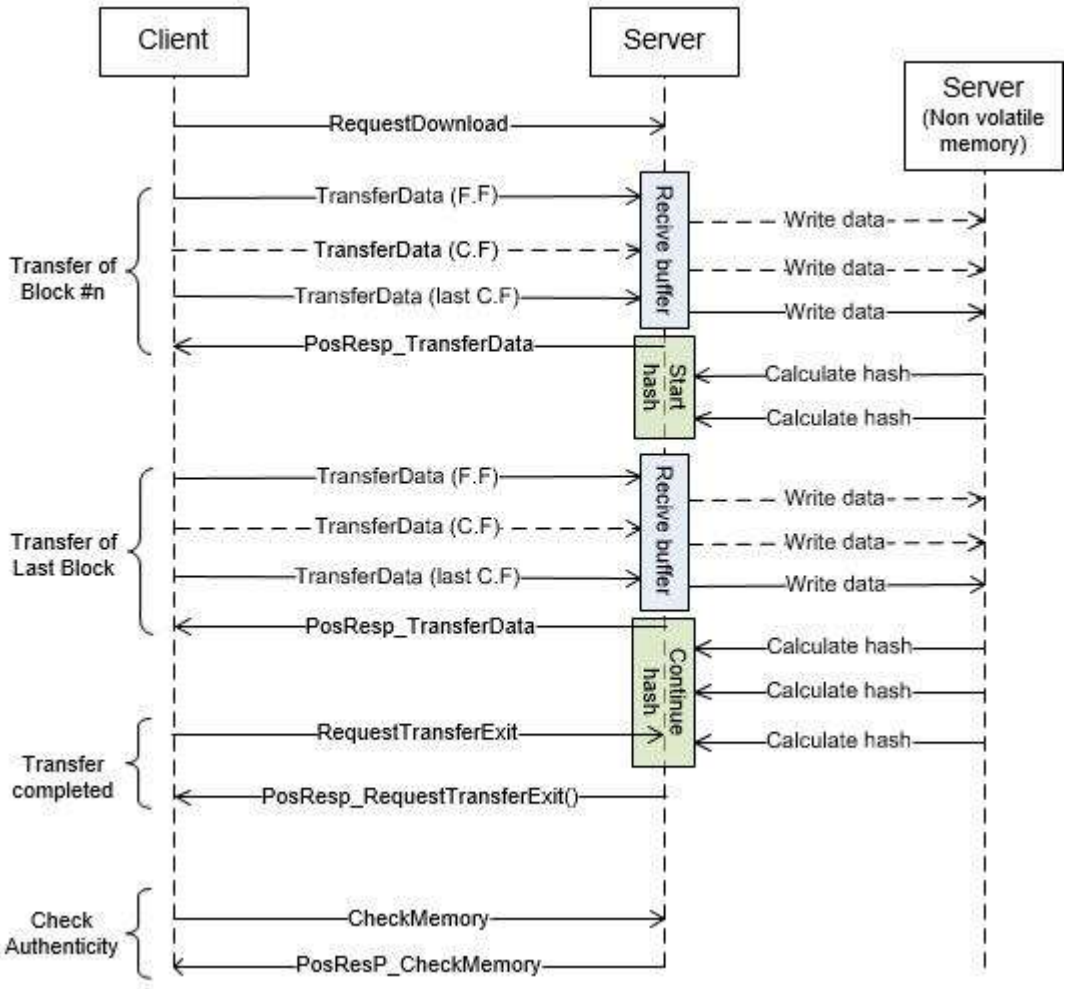


Figure - Pre-Calculated Checksum

This could be realized in several ways, e.g. for check memory, by always having the verification block table transferred first (first RequestDownload) and having the data blocks in the block table sorted according to the

downloaded order. then each downloaded block could then be checked against the block table.

662840 v1 **Pre-calculated hash**

Version history:
(1):

Purpose: To decrease the total programming time.
Source:
Verification Method: Analysis
Legacy ID: 3122v3, 3840v1a1

The ECU shall in between each TransferData request calculate the checksum, i.e. hash value or checksum, when next TransferData request is received the calculation shall be halted and continue after the data block is written to the non-volatile memory. The ECU must therefore remember where the calculation shall continue.

Important is that the pre-calculation must be done on data in the non-volatile memory. It is not allowed to pre-calculate received data in the volatile memory.

4.11.1.3 Bootoalder requirements for SWDL with check memory

•Software Authentication

4.11.1.3.1 Software Authentication

The purpose of Software Authentication is described in *[General Software Authentication]*, i.e. to verify that the installed software is authentic (originates from a legitimate source and has not been altered). To perform the authenticity verification, using digital signatures, the ECU must (at least) know:
• The data that must be checked for authenticity and where it is located in the memory. Verification Block Tables are used *[General Software Authentication]*. The verification block table (one per software part) contains the placement and hash value for the expected programmed data blocks.
• How to get the corresponding digital signature.
• Conditions for starting the verification and what is required for the verification to pass or fail respectively. If a complete verification is not performed at every application start, information about the verification status (validity status information) must be stored.
• How many software parts (and their location) that must be successfully verified in order to start the application: ECU Software Structure and Logical Blocks are introduced.

The ECU authenticity verification (for a single software part, i.e. a logical block) is performed according to *Figure - Authenticity verification principle*:
(1) - The data blocks of the software part is programmed to the ECU. The corresponding digital signature is transferred using a Check Memory control routine, which will also initiate the verification.
(2) - The signed verification block table is verified. The placement of the verification block is ensured as its start address and length parameters are part of the signature generation, i.e. the length (unprocessed) must be calculated by the ECU (the start address is known for the updated logical block).
(3) - The programmed data blocks, according to the authenticated verification block, are then verified. The placement of all ingoing data blocks are ensured as their start address and lengths together with the corresponding hash values are part of the verification block table.
(4) - The remaining memory area within the logical block shall not have been programmed (the verification block table only contains the programmed data blocks). The ECU verifies that these parts are "blank" (not programmed), e.g. the value of erased memory (hardware dependent) or filled with some known value (e.g. for the SBL). It must be ensured that erased state is a valid state for the used HW.

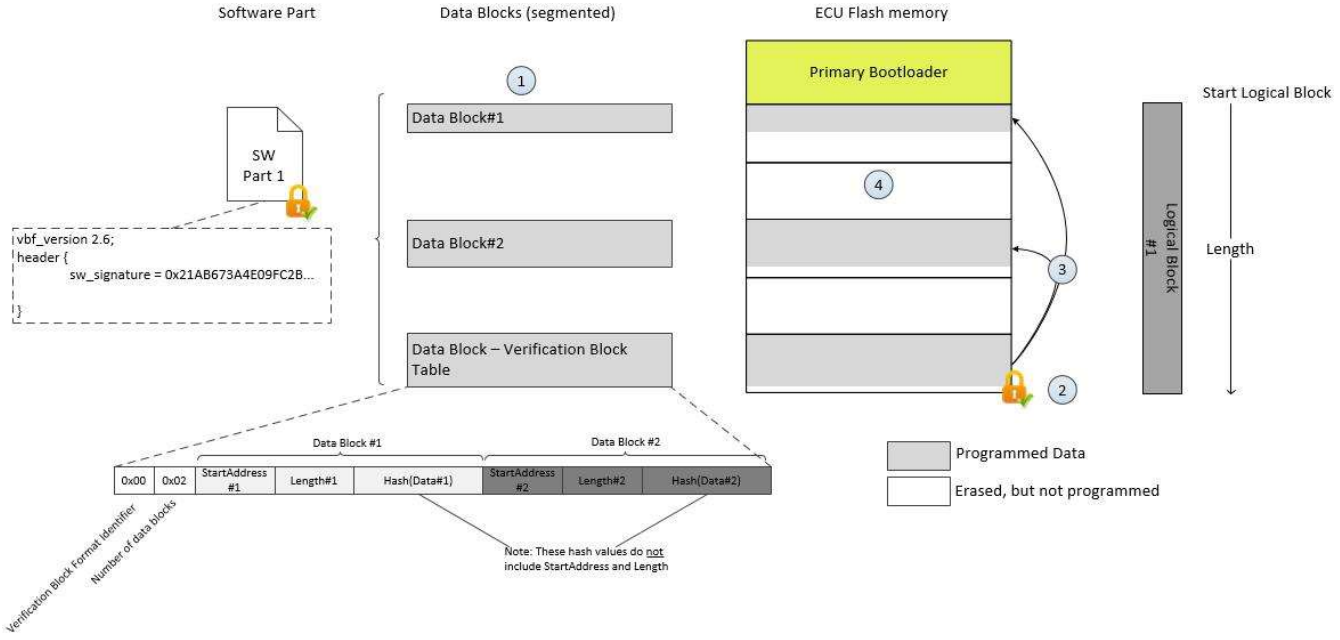


Figure - Authenticity verification principle

The *Figure - Authenticity verification principle* does not show all details of the software preparation (off-board) nor the signing. The purpose is to introduce the reader of how the verification is performed.

At software **download**, the ECU must:

- Check that the erase request matches a logical block range (to ensure that previously installed data is erased)
- Mark the logical block as invalid (i.e. it has not passed the signature verification), prior to any data erase/write operations.
- Check at every Request Download that the logical block is marked as invalid.
- Check at Request Download and at Transfer Data that only one logical block has been addressed. Note that the "compressed" (processed) memory size is used as input parameter to the Request Download, i.e. actual programmed length might differ (typically longer).
- Check at Request Download and at Transfer Data that no memory area is addressed that is exclusive to the bootloader. It must also be ensured that no memory is addressed that is not define, e.g. outside the range of where the application data is allowed to be programmed.
- At Check Memory, verify according to (1) to (4) in *Figure - Authenticity verification principle*.
- Only upon a successful authenticity verification, set the validity status for the verified logical block to valid.

At every ECU **start**, the bootloader must:

- Verify that all logical block are present and valid (i.e. signature verification is passed). The number of logical blocks and corresponding locations are defined by the ECU Software Structure, i.e. this must be authenticated first. All software must have passed the authenticity verification in order to be executed in the ECU.
- If the verification is failed, the complete part of the complete and compatible function must fail and the application is not started.

4.11.1.3.1.1 Software Structure

The ECU is to be organized into different "logical blocks", the ECU software structure. Each logical block is to be verified (and signed) individually and because of that it must only be one software part (e.g. a VBF) allocated per logical block. A software part is typically segmented into one or many data blocks. That means that all data blocks must reside in the dedicated logical block in order to be able to verify each logical block individually.

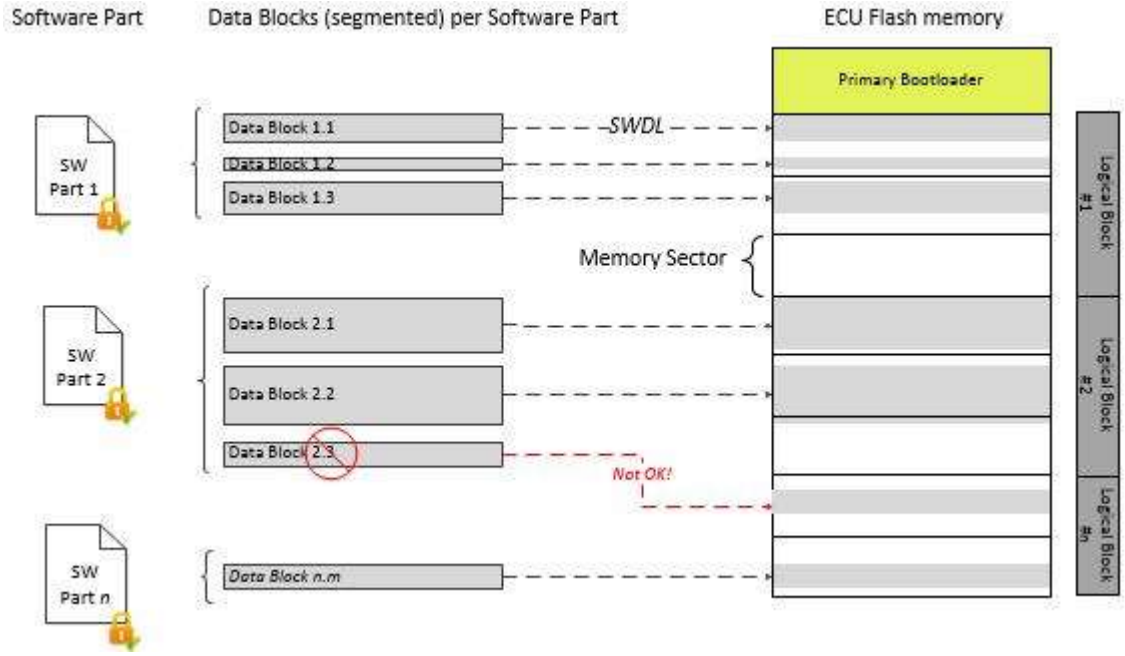


Figure - Logical Block Definition
In *Figure - Logical Block Definition,* a software part *n* is segmented into *m* data blocks and is dedicated to the logical block *n*.

662791 v1 **Logical Block - Properties**

Version history:
(1):

Purpose: Define the constraints of a logical block
Source:
Verification Method: Analysis
Legacy ID: 397291v3

The non-volatile memory for software download shall be divided into logical blocks that must start and end at the boundary of a flash sector. The logical blocks must not share a common flash sector, i.e. logical blocks must not overlap or contain another logical block. Logical blocks must also not cross different memory devices. It shall be contiguous.
Clarification for ECUs having file-based data storage: erasure or modification of a single logical block should not affect contents of other logical blocks (unless explicitly specified in other requirements (e.g. for ESS update)).

662792 v1 **Logical Block Declaration**

Version history:
(1):

Purpose: To define the properties for each logical block
Source:
Verification Method: Analysis AND Test
Legacy ID: 397432v4

For each individual logical block, following properties must be defined:
1. its start address and length, i.e. the location (mandatory)
2. where the verification block table is located (mandatory). The verification block contains information (location and hash value) of the data blocks that shall have been programmed to the logical block. It is typically the first

downloaded data block of a Software Part.

3. how to get the corresponding signature (mandatory) and where is shall be stored (optional). Storing the signature in a non-volatile memory is only applicable for use cases where it is required to perform a signature verification at a later time, i.e. the verification is not only performed at the programming sequence.
4. where the validity status information is located (mandatory)
5. where the erase status information is located (mandatory), used to identify if a logical block is erased or not, to e.g. not erase memory when it is already erased (time consuming).
6. where the compatibility information is located (optional). Used to decide the compatibility to other downloaded parts. This must be known to the bootloader *if* the check is done by the bootloader itself. See section for Complete&Compatible function for further details.

Example.
A logical block will contain 1:m data blocks and 1 verification block table.

662790 v1 Number of logical blocks

Version history:
(1):

Purpose: Define the minimum number of logical block. The actual number of logical blocks are defined by the ECU Software Structure.
Source:
Verification Method: Analysis AND Test
Legacy ID: 397293v1

The ECU shall contain at least one logical block. The maximum number of blocks shall be statically configured in the bootloader.

662788 v1 Programming of Logical Blocks

Version history:
(1):

Purpose: Define properties for a logical block, that are required for the ECU to be able to manage each logical block independently but also to simplify the signing process at the trust centre.
Source:
Verification Method: Analysis AND Test
Legacy ID: 400425v2

Each logical block shall be possible to sign separately, i.e. each logical block must be possible to erase, program and verify independently in the ECU.

662789 v1 Software parts per logical block

Version history:
(1):

Purpose: Define the software structure to enable that each software part is possible to individually sign and verify.
Source:
Verification Method: Analysis
Legacy ID: 397292v2

The content (data blocks) of a software part must be within a single logical block range, i.e. two different logical blocks must not be addressed by a single software part.

4.11.1.3.1.1.1 ECU Software Structure (ESS)

The **E**CU **S**oftware **S**tructure (**ESS**) carries the information of all logical block ranges and properties. It shall be stored in a dedicated logical block which position is known to the primary bootloader, e.g. a flash sector exclusively reserved to the ESS. It is however re-programmable as any other software part, according to the Programming sequence.

Typical use cases for updating the ESS are when a software part growths beyond its previously reserved logical block or when a software part is divided into two parts.

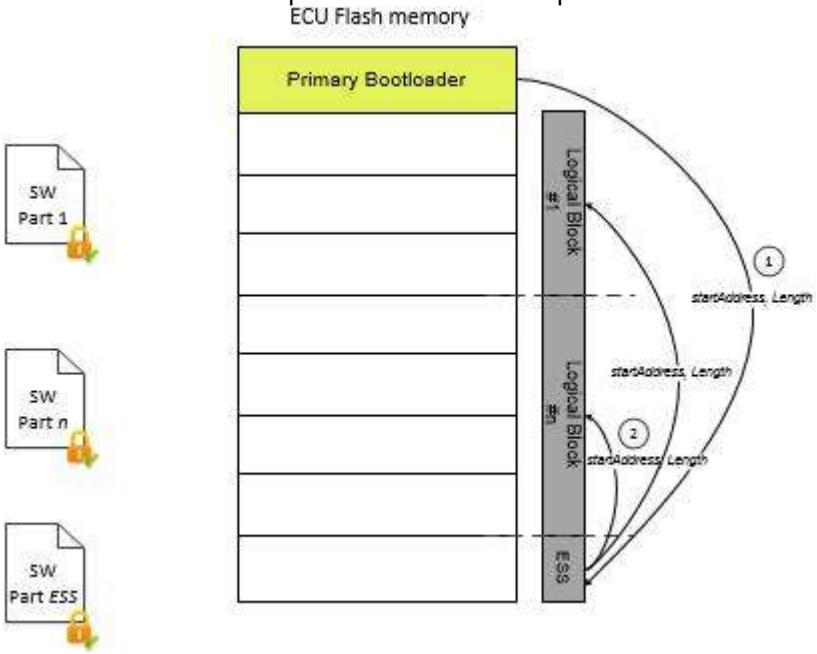


Figure - Re-programmable ECU Software Structure

(1) The location of the ESS is known by the primary bootloader.
(2) The bootloader uses the definitions in the ESS to identify the properties of the other logical blocks. The ESS must first be verified (valid) before its definitions are applied.

When there is no valid ESS stored in the dedicated logical block, the ECU must only allow software download of the SBL and to ESS as the information in the ESS is required in order to validate other logical blocks.

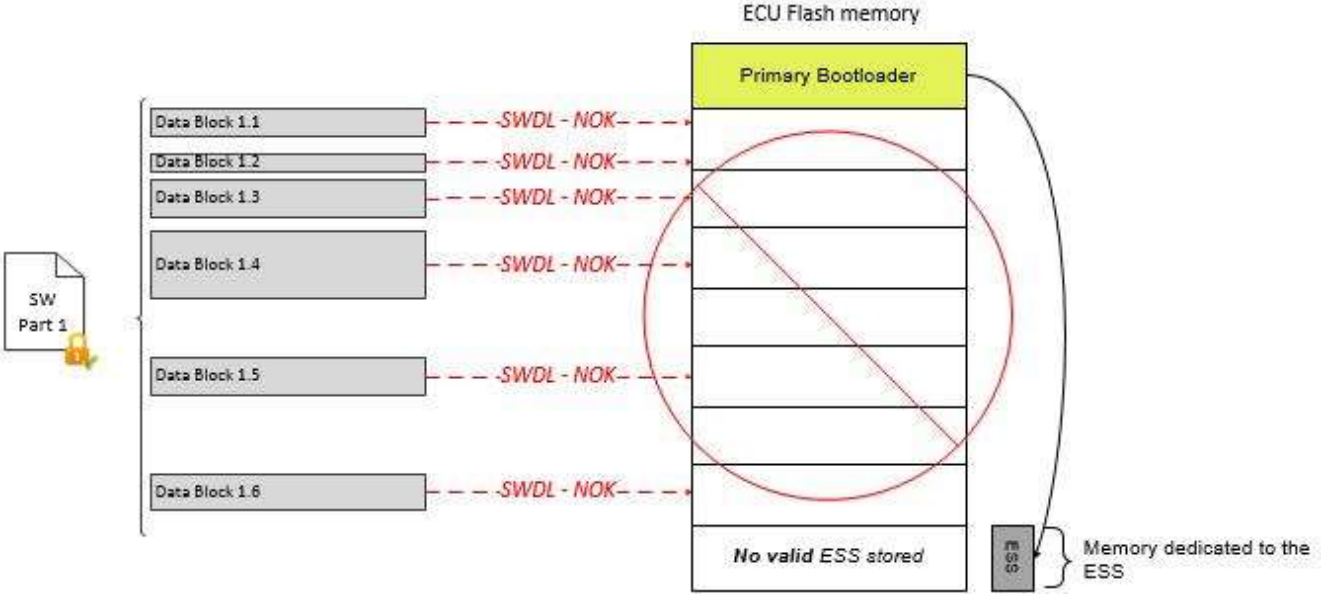


Figure - Restricted SWDL when no valid ESS stored
To ensure a known memory state when the ESS is updated, the bootloader shall erase the complete flash upon every ESS update.

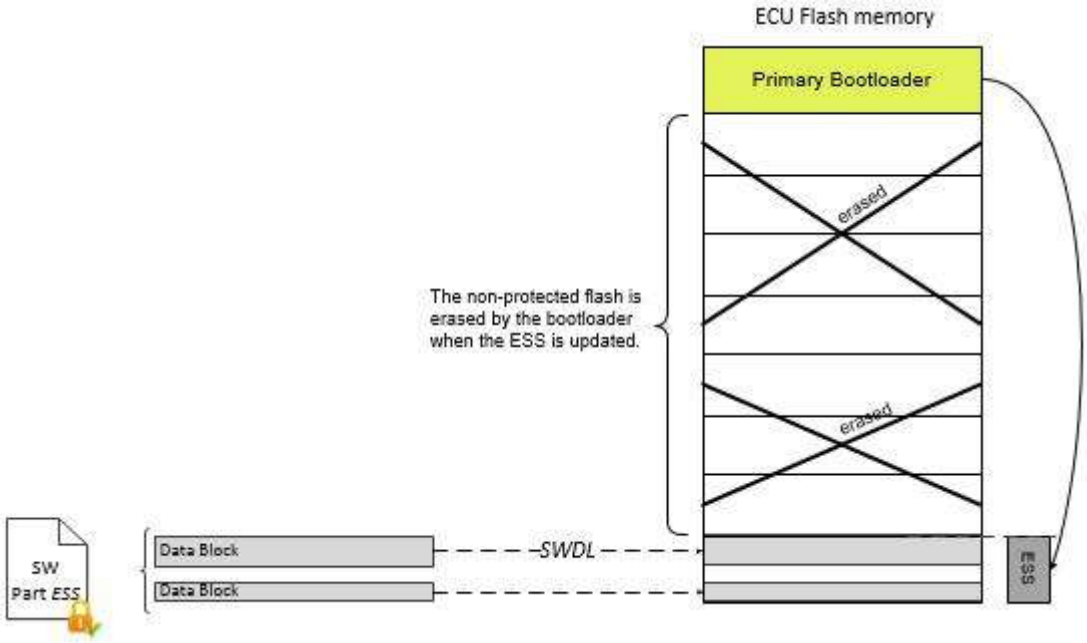


Figure - Non-protected flash to be erased when the ESS is updated

662798 v1 **Erase memory when ECU Software Structure (ESS) is updated**

Version history:
(1):

Purpose: To ensure a known memory state when the ESS is updated.
Source:
Verification Method: Analysis AND Test
Legacy ID: 397440v2

The complete programmable flash memory shall be erased when the logical block dedicated to the ESS is (internally) addressed for erasure, i.e. being updated and its validity is changed from valid to invalid.

Example.
The bootloader detects that the ESS is addressed for erasure. The bootloader will ensure that all flash memory is erased according to a defined physical flash layout. Note that a logical block shall be set to 'invalid' prior to it is erased. The erase might be initiated by either EraseMemory or RequestDownload.

662795 v2 **Immediately apply ESS update**

Version history:
(2): Verification Method: Test change to Analysis AND Test.

Purpose: The ESS is downloaded as any other software parts. For example, an ECU reset cannot be added to the programming sequence every time an ESS is updated.
Source:
Verification Method: Analysis AND Test
Legacy ID: 411891v1

The ESS updates must be applied immediately after being verified correctly, i.e. an initiation of the bootloader must not be required. This means that another software part (e.g. EXE) shall be possible to program next to the ESS without requiring an ECU reset.

662794 v2 Reprogrammable ECU Software Structure (ESS)

Version history:
(2): Verification Method: Test change to Analysis AND Test.

Purpose: To enable updates of the ESS without requiring a hardware or bootloader update. Typical use cases are when software parts growths in size, the software structure is changed (e.g. a software part is divided into two parts or vice versa).
Source:
Verification Method: Analysis AND Test
Legacy ID: 397436v2

The ESS shall be reprogrammable according to the Programming Sequence described in this document, i.e. as any other software part.

662796 v2 Restricted software download when no valid ESS stored

Version history:
(2): Verification Method: Test change to Analysis AND Test.

Purpose: To enable ECU Software Structure (ESS) update when there is no valid ESS stored.
Source:
Verification Method: Analysis AND Test
Legacy ID: 397438v3

If there is no valid ESS stored, the software download must be restricted to the SBL and the logical block dedicated to the ESS only. That means that attempts to address other logical blocks shall be rejected. The ECU shall inform the tester of the aborted erase memory attempt or aborted request download.
Note: *The SBL is verified by the primary bootloader.*

662793 v1 Storage of the ECU Software Structure (ESS)

Version history:
(1):

Purpose: To reduce the complexity of the bootloader implementation with respect to ESS update use case and for the bootloader to know where to find the logical block definitions
Source:
Verification Method: Analysis
Legacy ID: 397435v1

The ECU Software Structure (ESS) shall be stored at a reserved logical block (flash sector), i.e. dedicated to the ESS only.

662797 v2 Validation of the ECU Software Structure

Version history:
(2): Verification Method: Test change to Analysis AND Test.

Purpose: The verification of the programmed data could be bypassed if the logical block carrying the ESS is not valid.
Source:

Verification Method: Analysis AND Test
Legacy ID: 404771v3

The ESS must be validated before the bootloader applies the ESS definitions. The minimum requirements are that it shall be verified as any other logical blocks and ensure that the complete memory is covered by the defined logical blocks.

Note: *The ECU is recommended to add additional checks to e.g. ensure that the format or length is correct, even though these parts are to be verified prior to the software part is signed.*

4.11.1.3.1.1.2 ECU Software Structure - Declaration

The format of the ECU Software Structure is design specific and will not be specified in details. Some parts might be explicitly defined in the structure itself or derived from another known location, e.g. the logical block range.

Example.

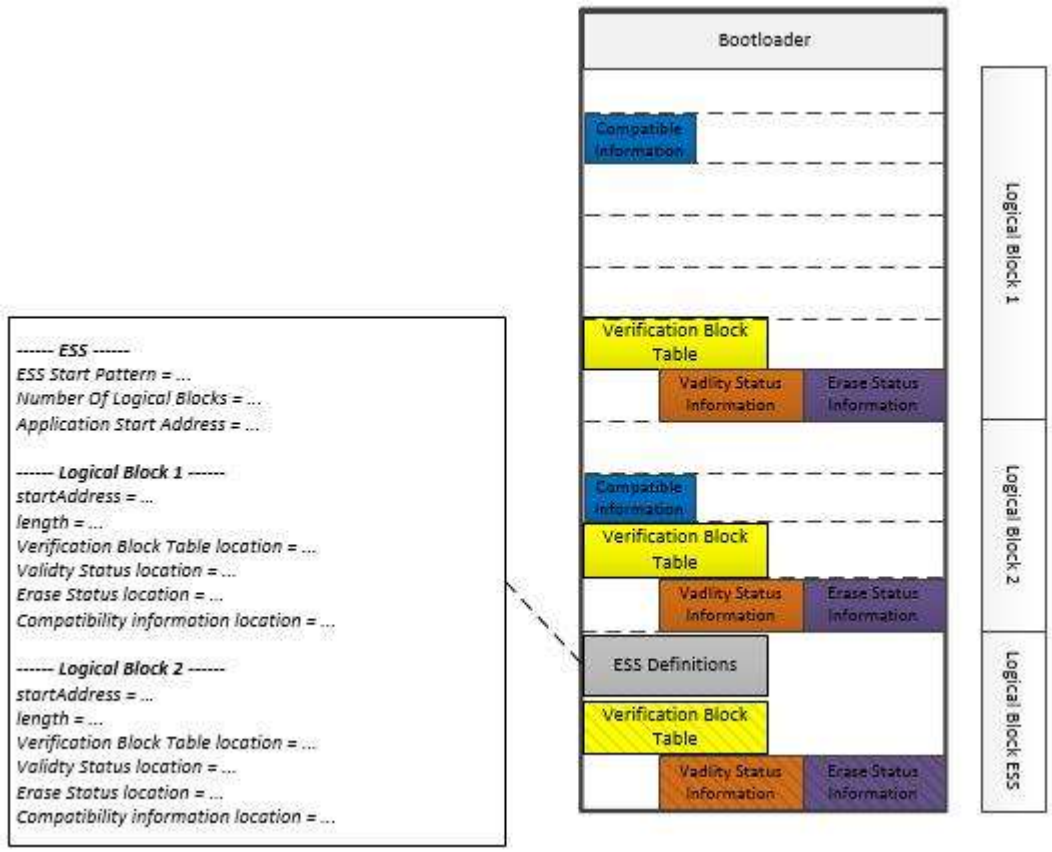


Figure - Example of ESS Declaration

To have a simplified view, the Figure - Example of ESS Declaration shows when Validity Status information and Erase Status information are stored within each logical block range. These could also be stored elsewhere, e.g. in a dedicated memory exclusive to the bootloader. See separate section for Validity Status as it has dependencies to the ECU verification being executed at start (boot). Previous figure shows the case where the Validity Status is used to indicate the latest software download status. The recommendation is to verify all software also at boot.

662799 v2 **ECU Software Structure Declaration**

Version history:
(2): Verification Method: Analysis AND Test change to Analysis.

Purpose: Define the properties of the ECU Software Structure. The supplier is responsible to specify the

format of the ESS in detail. This degree of freedom is left to the supplier in order to not interfere with supplier specific tool chain and implementations.

Source:

Verification Method: Analysis

Legacy ID: 399873v2

The *ECU Software Structure* shall at least contain:

- A pattern known to the bootloader, defining the existence of the structure, i.e. always starting with a known pattern.
- The number of logical blocks defined in the ESS.
- The definitions of all logical blocks, see *REQPROD Logical Block Declaration*.

1077149 v1 **ESS software solidification**

Version history:

Purpose:

Source:

Verification Method: Analysis AND Test

Legacy ID:

ESS software is applicable for logic blocks that are not fully fixed in the development stage.

In the formal production stage of TT, the ECU software structure is basically frozen and it is not allowed to submit ESS software again through Test Order.

4.11.1.3.1.1.3 Root ECU Software Structure

The *ECU Software Structure* is stored in a dedicated logical block. But, the bootloader must know the properties of that logical block. Hence, the term ***Root** ECU Software Structure* is introduced. This structure is stored in a protected area known to the bootloader (e.g. in the bootloader area itself).

662800 v2 **Root ECU Software Structure Declaration**

Version history:
(2): Verification Method: Analysis AND Test change to Analysis.

Purpose: Define ESS properties that must be stored in a protected area known to, or part of, the bootloader. This properties are required when there is no ESS stored and for the bootloader to be able to verify that the stored ESS is valid (i.e. has passed the authenticity verification).

Source:

Verification Method: Analysis

Legacy ID: 399878v2

The ECU shall in a protected (not possible to alter nor read) area statically store the definitions required to verify the logical block dedicated to the ECU Software Structure, i.e:
(i) start address and length (mandatory)
(ii) where the verification block table is located (mandatory)
(iii) where the validity status information is located (mandatory)
(iv) where the erase status information is located (mandatory)
(v) where the compatibility information is located (optional)
(vi) where the signature of the verification block table is to be stored (optional).

4.11.1.3.1.2 Erase Status Information

Per logical block, the bootloader shall able to identify if the logical block is erased or not. This is typically used by

the bootloader to not erase memory (time consuming) when it's already erased, but also to ensure that the memory has been erased as expected for use cases where this is required.

662801 v2 **Erase Status Information**

Version history:
(2): Verification Method: Analysis change to Analysis AND Test.

Purpose: To reduce software download time and to ensure that the memory has been erased as expected.
Source:
Verification Method: Analysis AND Test
Legacy ID: 401108v3

The ECU shall reserve some dedicated memory to the Erase Status Information. This information must be exclusively handled by the bootloader. If the logical block is "already erased" and an valid erase request is detected, the ECU shall not perform any erase operations. Once one or more bytes of the programmable area has been programmed, the addressed logical block must be considered as "not erased" until it is ensured that the complete block has been erased again. The ECU supplier must ensure that a "virgin" ECU has this information set to erased and that a completely programmed ECU has the erase information set to "not erased".

4.11.1.3.1.3 Verification Block Table

A *Verification Block Table* is introduced, representing the expected segmented data blocks (including address, length and hash values per data block) of a complete software part. By that, the signature could be created on the verification block table. When that block is verified correctly at the ECU, the ECU could continue to verify also the other data blocks that has been programmed. Hence, the security principle of "chain-of-trust" is applied. Note that the verification block table is downloaded as any other data block, i.e. the verification block is a separate data block with defined properties.

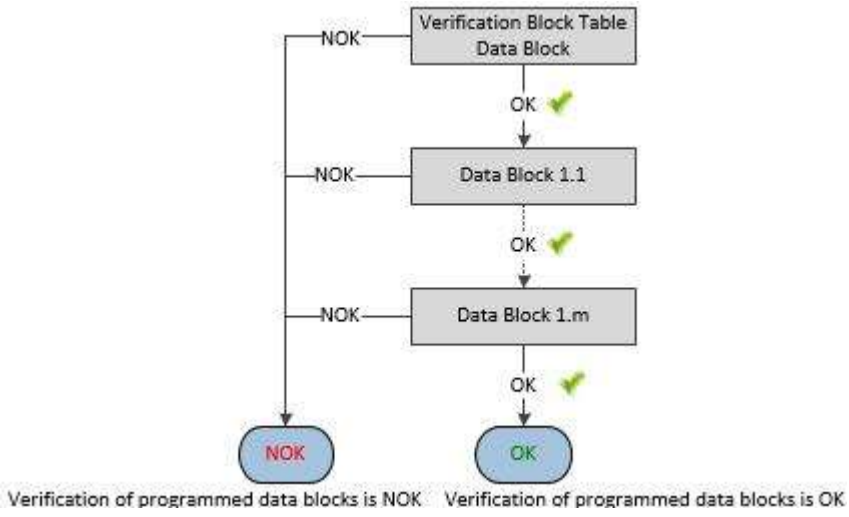


Figure - Verification of programmed data blocks

The verification block table must be put in a separate data block (i.e. downloaded as a separate data block to the ECU) and it shall contain:

- Verification Block Table format identifier, e.g. a version number if the format of the table is changed in the future
- Number of data blocks (the verification block table itself is excluded)
- StartAddress data block 1, length data block 1, hash value of data block 1
- ...
- StartAddress data block m, length data block m, hash value of data block m

Note that there is one verification block table per every software part.

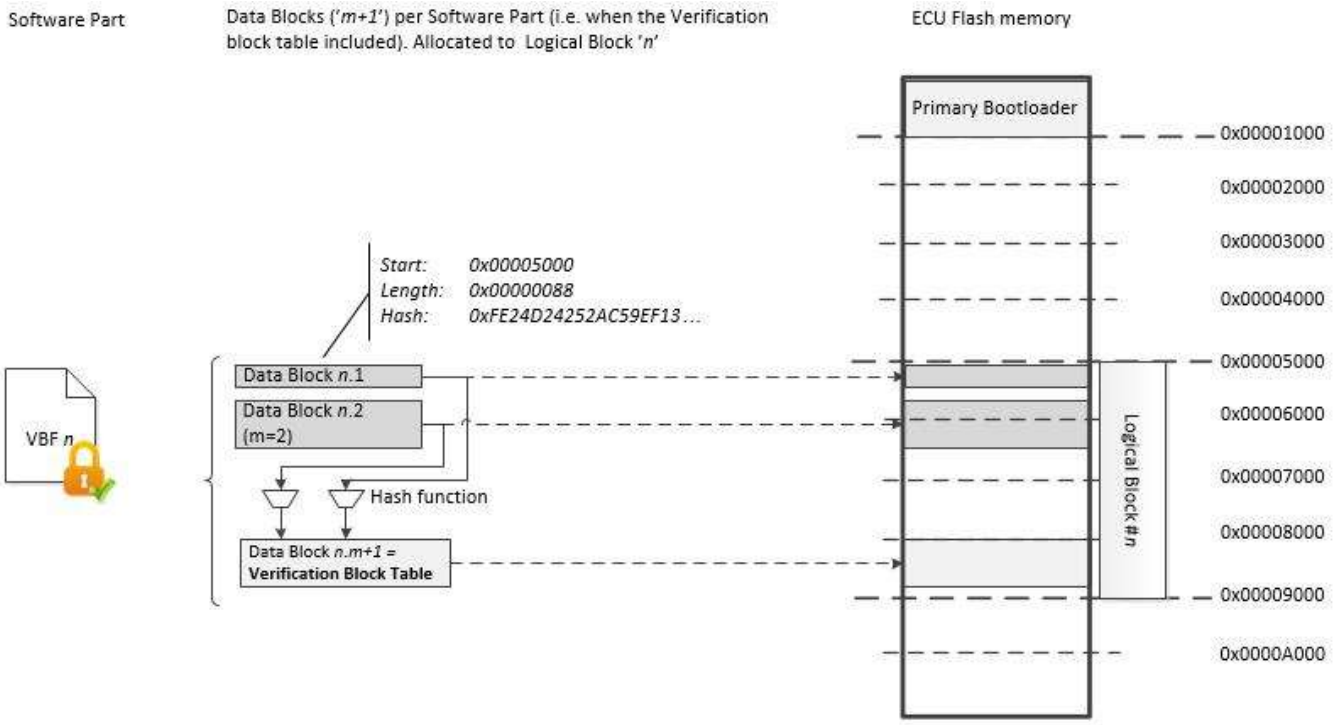


Figure - Verification Block Table overview

In *Figure - Verification Block Table overview* it is shown how the verification block table is constructed. The hash values are calculated on the data parts only, i.e. the start address and length is not used as input to the hash function. The placement of each data block is ensured as the start address and length are also defined in the table. The format of the table is defined in *[General Software Authentication].*

662802 v2 **Allocation of the Verification Block Table**

Version history:
(1):

Purpose: The position of the verification block table must be known by the bootloader in order to perform the verification. The verification block must not be part of any other data block, as the verification block table contains hash values of all other data blocks, i.e. it cannot contain the hash of itself.

Source:

Verification Method: Analysis AND Test

Legacy ID: 398372v2

A verification block table as defined Req ID 661295: *Format of the Verification Block* Table shall be stored in a separate data block. The start address shall be defined by the logical block definition, i.e. by the ECU Software Structure. The correct location of the verification block is ensured by including its address information (start and length) when it is being signed. The length (unprocessed) must be calculated by the ECU when the data is programmed.

Design consideration: When delta encoding is applied, the length of the Verification Block Table could be derived by using the start address and Number of data blocks.

662803 v1 **Verification Block Table - Reserve space at application build process**

Version history:
(1):

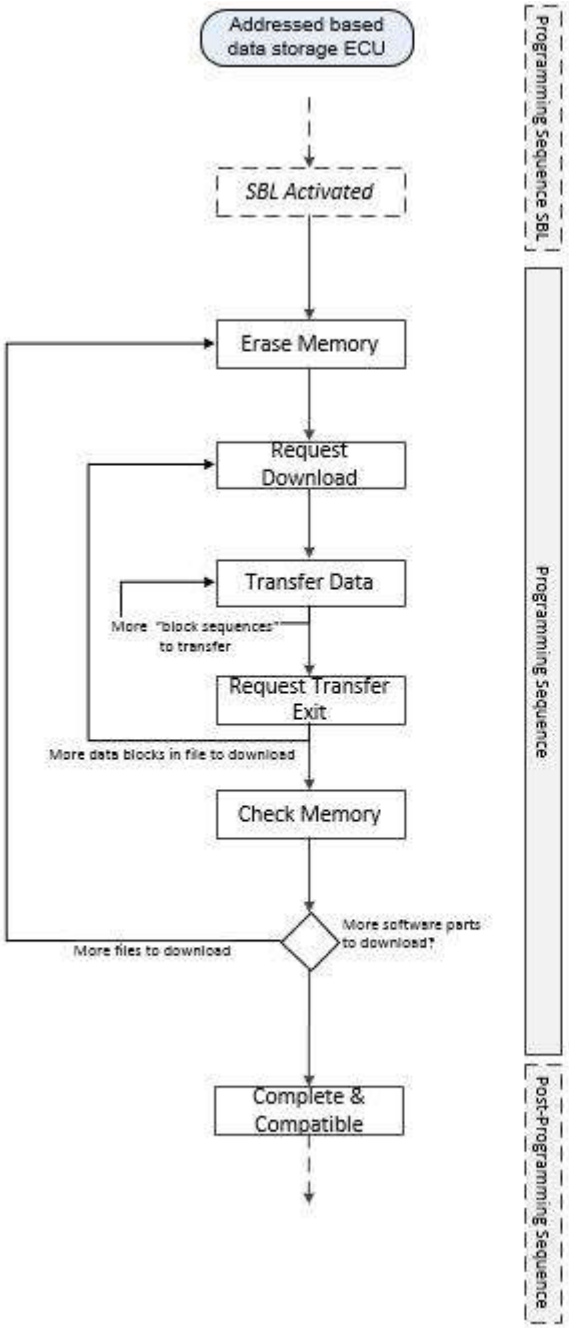
Purpose: The verification block table is part of the post processing sequence, meaning that it is generated after the actual software is build.
Source:
Verification Method: Analysis AND Test
Legacy ID: 402292v1

The application software build process must reserve memory space for the verification block table within a logical block. It is the software supplier that creates the block table and includes it in the software part.

4.11.1.3.1.4 Validation of ECU software(s) - Authenticity Verification

4.11.1.3.1.4.1 Authenticity Verification - Overview

An overview of the authenticity verification was previously described in the beginning of the section *Software Authentication - Introduction*. The programming sequence, highlighting the authenticity verification parts is shown in *Figure - Programming Sequence and authenticity verification*.



1. The addressed logical block shall be set to invalid, when the memory is to be erased. Then, start to erase the complete logical block if it is not already erased. The memory size in erase memory request) shall match the size of the logical block(s).
2. Data transfer to a single logical block is initiated. For every request download, always check that the logical block is invalid. Only one logical block and no restricted area shall be addressed by the download requests (abort otherwise).
3. The data blocks are programmed. Again, data must not be programmed to restricted areas. If a processing method (e.g. compression) is applied, the bootloader will not at RequestDownload know the final length of the data to be written
4. Verify the authenticity (signature verification) of the updated logical block). The start address and length of the verification block table is used as input to the verification function. Unused areas shall be blank/empty. If the check is failed, the addressed logical block shall be kept as invalid. A successful verification must be required to set the logical block to valid. The CheckMemory will contain the corresponding signature to the downloaded data. The verification result is stored (validity status) and used as input to Complete&Compatible check. The verification is done on data in the non-volatile memory. It is not allowed to pre-calculate and verify received data in the volatile memory.
5. If an application call-out is performed (e.g. for comaptibility), the call must be within a already verified (valid) range.

Note: *When the application is to be started, the application start address shall also be within a verified range.*

Figure - Programming Sequence and authenticity verification.

4.11.1.3.1.4.2 Validation of Logical Blocks

662805 v1 **Condition for a logical block to be valid**

Version history:
(1):

Purpose: A logical block must be verified successfully with respect to its authenticity every time it has been altered, in order to be considered as valid.

Source:

Verification Method: Analysis AND Test
Legacy ID: 400992v3

A logical block must only be set to valid if it has passed the authenticity verification as defined in *Figure - Authenticity Verification function*. The verification shall be done on data in the non-volatile memory (i.e. at the target location).

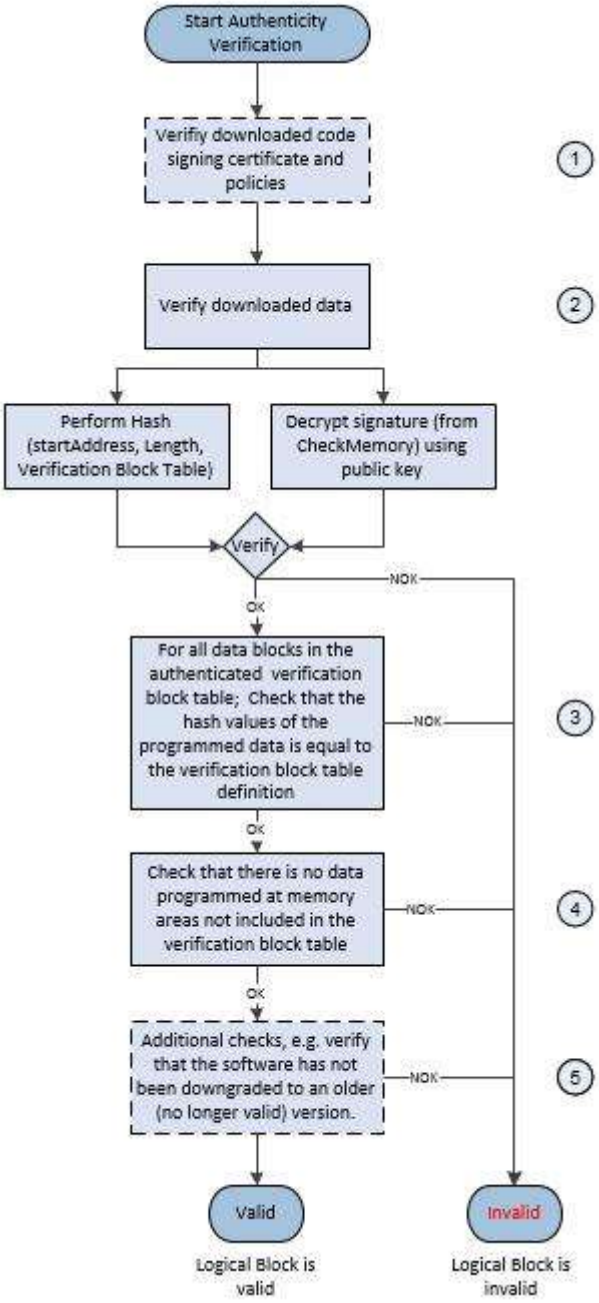


Figure - Authenticity Verification function

(1) - Applicable only if code signing certificates are applied, see *[Software Authentication for file based systems*] for additional requirements.
(2) - Signature verification. Note that the start address and length are used as input to ensure the placement, i.e. the length must for some use cases be calculated (delta encoding). The format and length of the verification block table shall be verified. Prior to the signature verification is performed, the integrity of the public key shall be ensured.

(3) - Verify that the hash value (per data block) is identical to the definition in the verification block table.
(4) - Verify that the non-programmed areas (not part of the authenticated verification block table) are erased. Previously programmed data that is no longer in use shall have been erased ("uninstalled").
(5) - Additional checks, e.g. if the ECU has support to prevent downgrade, meaning that an older (no longer valid) software part must not be installed as it might include some vulnerabilities. GRI Basetech Team to be consulted if such protection shall be applied (processes must be defined at off-board to support such a use case).

When the authenticity verification function is completed, it must be ensured that the verification status (validity status information) is stored correctly.
Mandatory for the ECU Software Structure: The Logical Blocks defined in the programmed ESS covers the complete memory, i.e. all memory are always to be verified.

662804 v1 **Conditions for a logical block to be invalid**

Version history:
(1):

Purpose: A logical block must be verified with respect to its authenticity every time it is modified. Any condition that triggers a change of memory (erase/write) of a logical block shall result in that the logical block is set to (or kept) invalid.
Source:
Verification Method: Analysis AND Test
Legacy ID: 400994v2

When the ECU internal conditions for altering (erase/write) a logical block is true, e.g. triggered by a physical/functional EraseMemory, RequestDownload etc., request, the ECU must ensure that the validity status of the addressed logical block is set to invalid prior to the actual memory erasure (or data write) operation is started. The logical block shall be invalid if the authenticity verification is failed. For every RequestDownload, the ECU shall always check that the logical block is 'invalid', otherwise the ECU shall:

- (i) abort the request, for ECUs where a EraseMemory request must be received prior to the RequestDownload *or*
- (ii) set the logical block to 'invalid' and continue with the requested service, where no EraseMemory is required

4.11.1.3.1.4.3 Validity Status Information

A software part is considered as complete (valid) when the authenticity verification of the belonging logical block is passed. Hence, besides the verificaiton at download the ECU must either verify the software at every boot (recommended) or store information in a protected non-volatile memory about the verification status of each logical block (e.g. valid or invalid), with respect to the authenticity verification at software download. This information is referred to as validity information. Every time a logical block is altered, e.g. erased or programmed, its status must be set to invalid until a successful verification has been completed.

For ECUs verifying the authenticity of the bootloader and application software(s) prior to every execution (e.g. at start), there is no need to store the validity status information in a non-volatile memory. Due to e.g. start-up timing requirements, the ECU possibilities to do the signature check (as in CheckMemory) at every start is limited. To be able to perform a verification at start, the solutions provided by the ECU hardware platform in terms of cryptographic support is to be used.

662806 v1 **Validity Status Information - Properties**

Version history:
(1):

Purpose: The validity information is the status of the authenticity verification. Only a trusted entity must be able to store (alter) the validity information. A minimum size is defined to avoid that a "bit-flip" result in that a logical block is set to 'valid'.

Source:
Verification Method: Analysis AND Test
Legacy ID: 400961v6

The validity status information for each logical block, i.e. if the block has passed- or failed the authenticity verification at software download, shall have a minimum length of 8 octets. One single value shall represent 'Valid' and all other values shall be interpreted as 'Invalid'. 'Valid' must not be represented by setting all bytes to the same value, e.g. 0xFF, or to the value of erased memory.
If the authenticity verification is not performed at every boot sequence, the validity information must be stored in a non-volatile protected (secure) memory. The best available solution provided by the ECU shall be used (hardware security module or equivalent, EEPROM, a pattern in flash etc.). The validity status must not be altered by other means than by the trustworthy verifying component, i.e. it must not be altered by the application or by a tester tool or any other unauthorized entity addressing that area. It must also not be possible to read, or upload, using a tester or debugger tool.

Note: *An ECU unique value for 'Valid' shall be applied.*

4.11.1.3.1.4.4 Public Key Storage

See *[General Software Authentication]* for requirements about public key programming during test and production.

662807 v1 **Public Key storage**

Version history:
(1):

Purpose: The trust anchor shall not be possible to manipulate, that could be used to bypass the protection.
Source:
Verification Method: Analysis AND Test
Legacy ID: 366329v2

The trust anchor, i.e. the public key, shall be stored in a protected (read-only) area, typically in a hardware secure module or a protected boot area. It shall be one-time-programmable.

662808 v1 **Re-programmable initial development key**

Version history:
(1):

Purpose: Define key management with respect to the test, development and release (production).
Source:
Verification Method: Analysis AND Test
Legacy ID: 429095v1

The ECU shall use an initial public key for development purposes. It is stored in the ECU by the ECU supplier (e.g. "virgin" ECU). When the initial development key is stored in the ECU, the public key for production shall always be possible to program. When a valid public key has been programmed, the initial development key must no longer be used by the ECU. The development key defined in *[General Software Authentication]* shall be applied unless else is explicitly specified.

4.11.1.3.1.5 Additional requirements for ECUs having multiple processors

The term main processor is in this context used to define the microcontroller having the interface to the network used for software download. The requirements in this section are valid for ECUs where all software to the secondary processor is not directly sent to the secondary processor but being addressed to the main processor, i.e.

it is considered as one single ECU from a tester point of view.

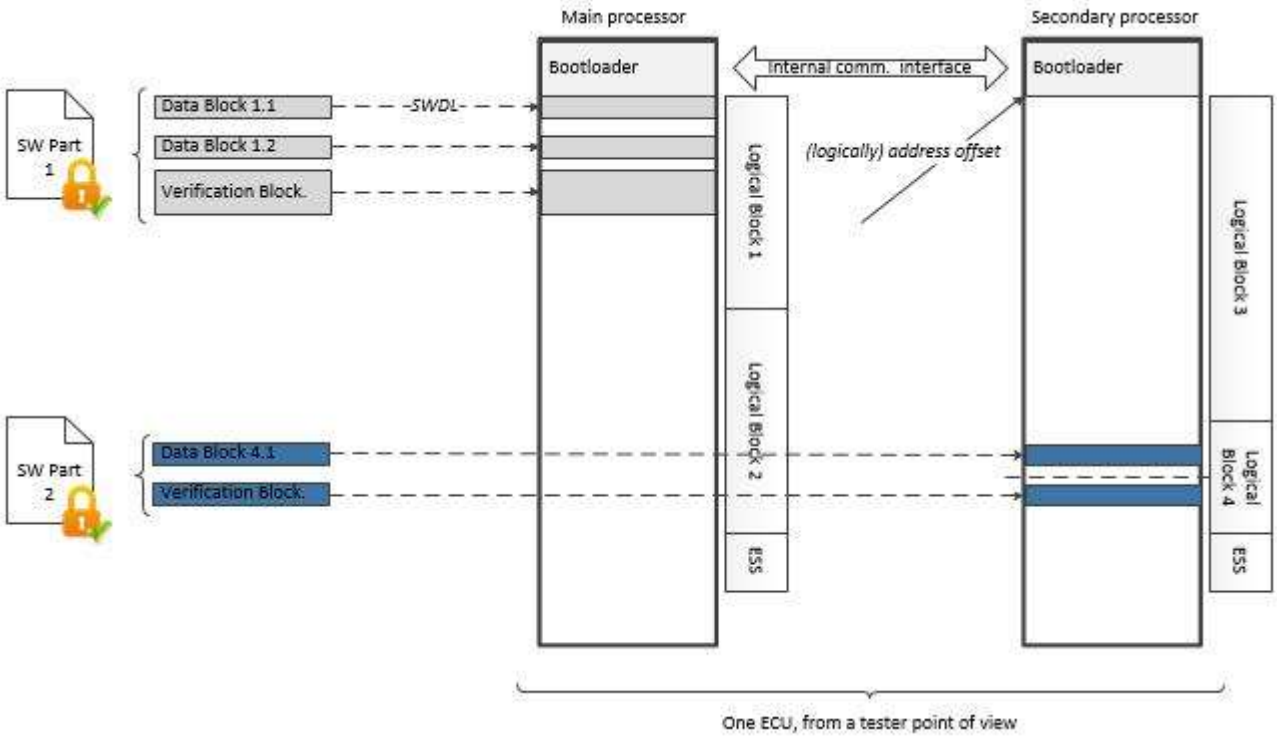


Figure - Additional processors - Overview

The bootloader of the main processor must be able to identify the software parts belonging to a memory device at the secondary processor, typically by using an address offset, and transfer it accordingly. It is recommended that the main processors keeps some information about the memory layout of also the secondary processor in order to do a first filtering before the data is transferred to the secondary processor. The principle to apply is that the logical blocks shall be verified by the entity where it is executed.

662817 v1 **Multiple processors - verification principle**

Version history:
(1):

Purpose: From a security point of view, the software is to be finally verified at the target processor. This will reduce the risk of having software altered when it is in transit.
Source:
Verification Method: Analysis AND Test
Legacy ID: 401206v1

The logical block shall be verified by the target processor respectively, i.e. either by the main- or the secondary processor.

Note: *The actual ECU implementation is highly HW dependent, e.g. capabilities of the processors and internal interfaces.*

4.11.1.3.1.5.1 Multiple processors - Secondary Bootloader

The Secondary Bootloader is a special scenario, where it is required to address two (main/secondary processor) different memory devices in a single software part. See *Figure - SBL verification for ECUs having additional processor*.

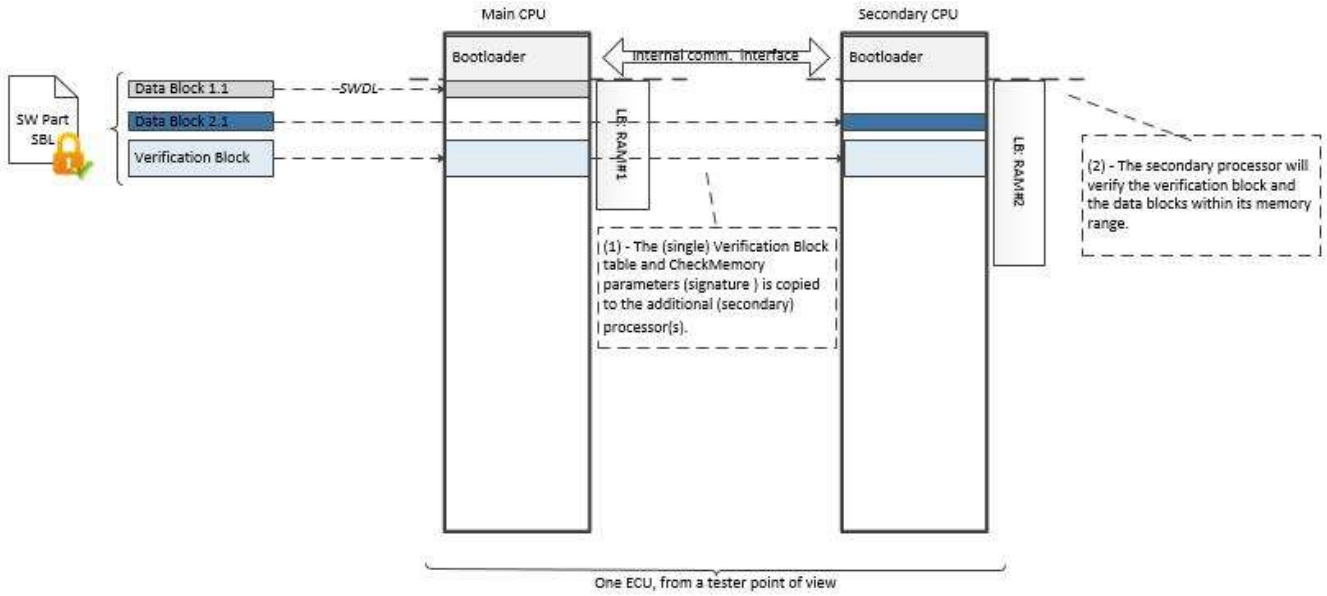


Figure - SBL verification for ECUs having additional processor

- **Responsibility of the main processor:**
- The main processor will verify the authenticity of the verification block table and verify and the data block within its memory range. The main processor will request the verification result from the secondary processor, i.e. for the data blocks addressed to the secondary processor. Hence, the main processor must forward a copy of the verification block, its signature and all data blocks belonging to the secondary processor.
- **Responsibility of the secondary processor:**
- The secondary processor will also verify the authenticity of the verification block table but not verify the number of data blocks part of the verification block. The actual number of downloaded data blocks, verification block excluded, shall be 1. The secondary processor shall verify that the start address, length and hash value of the downloaded data block is identical to the values defined in the verification block. It shall report the verification result to the main processor.

662818 v1 **Multiple processors - Main processor SBL verification**

Version history:
(1):

Purpose: Define the verification performed by the main processor with respect to the SBL.
Source:
Verification Method: Analysis AND Test
Legacy ID: 402299v1

The main processor shall (at least) verify the data blocks dedicated to memory space of the main processor itself. The main processor shall forward the data blocks belonging to the secondary processor together with a copy of the verification block table.

Note: *The verification is, besides this exception, to be performed as for any other logical block (e.g. including signature verification etc.*).

662819 v1 **Multiple processors - Secondary processor SBL verification**

Version history:
(1):

Purpose: Define the verification performed by the secondary processor with respect to the SBL.
Source:
Verification Method: Analysis AND Test
Legacy ID: 402300v2

The secondary processor shall first verify the verification block table and then the downloaded data blocks within its memory space. The secondary processor will not receive all data blocks as defined in the verification block table.

4.11.1.3.1.6 Additional requirements for ECUs implementing delta encoding

The delta file will contain information about the target version, as for any other processing method.

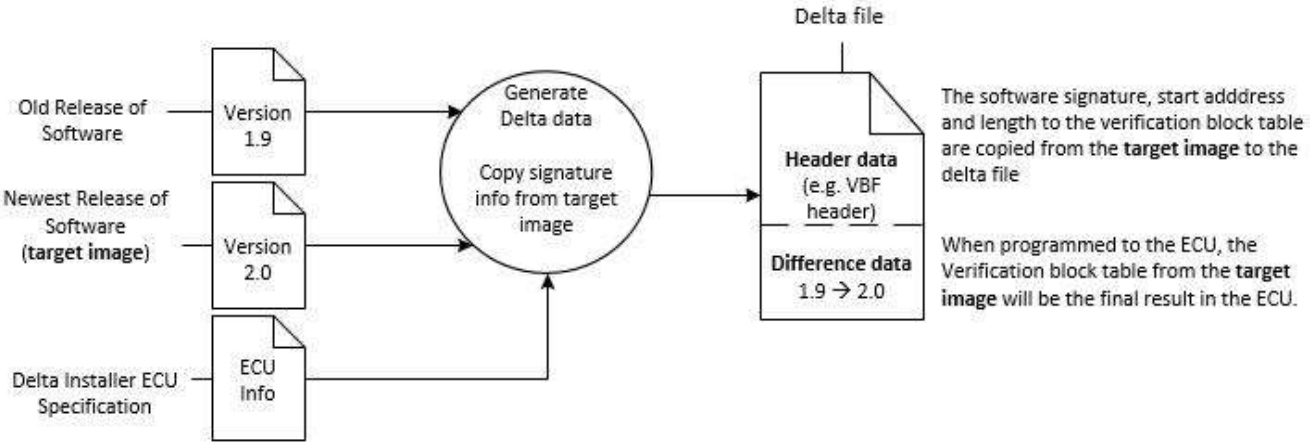


Figure - Content of a delta file

Figure - Content of a delta file shows that the required header information for authenticity verification is added to the delta file, i.e. the signature of the target software (version 2.0).

For ECUs implementing delta encoding, there are some conflicting requirements, e.g. that a complete logical block must always be erased but delta requirement to only erase sectors that are altered (no eraseMemory requests are required by the client). If the ECU is storing Validity Status Information in the flash memory within a logical block range, that flash sector will require some "special handling" to ensure that the altered logical block is always set to invalid.

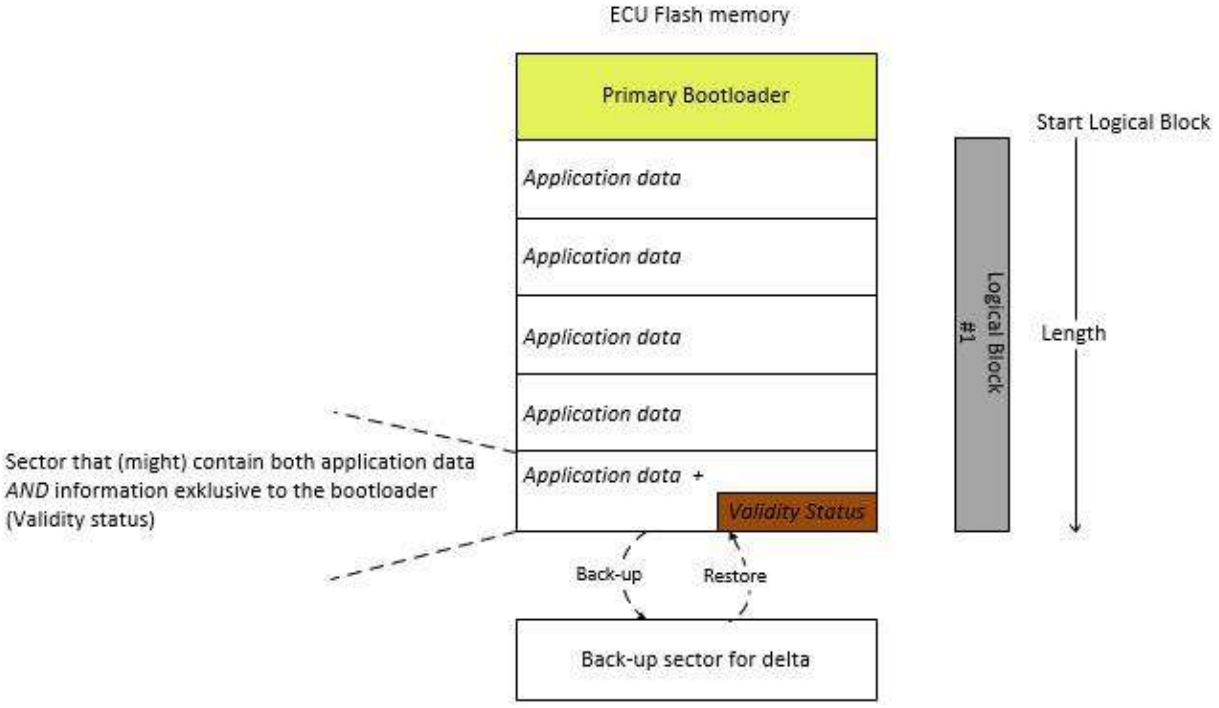


Figure - Delta programming and validity status

Figure - Delta programming and validity status shows a scenario where the ECU is storing the validity status in a flash sector, which also could contain application data (to not waste a single sector for validity status). The bootloader shall always handle that "shared" sector no matter if the application data part has been altered or not. See example:

Example.
(1) - Some part in the logical block is about to be altered.
(2) - The bootloader must always set the logical block to invalid, prior to the memory is altered, i.e. the Validity Status is set to invalid at the RequestDownload request (erase memory request is not applied for delta encoding)
(3) - When the Validity Status is erased, the shared sector is copied to the delta back-up sector.
(4) - The shared sector is erased
(5) - The data stored in the back-up sector is copied to the shared sector again, now with the validity status set to invalid.
(6) - Start the delta download
(7) - Verify the logical block

It is ECU specific how the Validity Status is stored, previous section is to illustrate an example only.

662820 v1 **Delta encoding - Verification**

Version history:
(1): 增加差分的需求。

Purpose: The verification of a delta installation shall fulfill the same level of authenticity verification as for any other processing method. Due to the nature of a delta

installation, some requirements are clarified by this dedicated requirement.

Source:
Verification Method: Test
Legacy ID: 416271v1

When a delta encoding processing method is used, following must (still apply):

- The final target shall be verified, i.e. the unprocessed data after the delta encoding has been applied.
- The verification block table and CheckMemory parameters shall be generated from the target software and copied to the delta file when it is generated.
- A single logical block shall be addressed, e.g. the "delta-library" used must not itself perform any read/erase operations (these must be requested from the bootloader)

4.11.1.3.1.7 Signing method

662821 v1 **Software authentication signing method and algorithms**

Version history:
(1):

Purpose: Define how to identify the signing methods to be used, e.g. the public-key algorithm, key sizes etc.
Source:
Verification Method: Analysis
Legacy ID: 420111v1

The ECU shall implement the default signing method as defined in *[General Software Authentication].*

4.11.1.3.1.8 Software Authentication - Generic security requirements

This section contains the generic Software Authentication security requirements. The actual implementation might differ, e.g. dependent of if code signing certificates or only the key-pair are used when signing/verifying the software, when the ECU could perform the authenticity verification etc.

662822 v1 **Allocation of the Authenticity Verification function**

Version history:
(1):

Purpose: Tamper resistance, to prevent to that the function is bypassed.
Source:
Verification Method: Analysis
Legacy ID: 366327v1

The authenticity verification function shall be stored in a protected area. If a hardware security module is deployed to the ECU this is required to be used for signature verification function.

662824 v1 **Authenticity verification at every start-up**

Version history:
(1):

Purpose: Define when the ECU authenticity verification shall be started (mandatory on condtion)
Source:

Verification Method: Analysis AND Test
Legacy ID: 419731v1

If the ECU is able to fulfill the application start-up conditions, the authenticity verification shall be started every time the (main) application is to be started (via bootloader), typically at every ECU Reset or Power-Up. Hence, the validity status information shall not be permanently stored in the ECU.

Note: *Design consideration - if the verification using digital signatures is not feasible every start, the ECU could after a successful signature verification at software download calculate a message authentication code (MAC) over the verified logical block. During start, that MAC is verified. ECU internal secret keys shall be applied in that case, both the keys and MAC shall be stored in protected memory. This shall be considered if the ECU has the capabilities to calculate and store the MAC and still fulfill the start-up timing* If the verification cannot be performed at start (boot), it must be proven by the ECU supplier by providing e.g. hardware- or performance figures why it is not possible and how the system is hardened against any updates except the defined software download sequence. If any software fails to be authenticated during boot, that sequence shall be aborted.

Design considerations: If the signature verification (as part of CheckMemory) could not be applied at start-up at any stage, the boot and application software(s) are to be signed (and verified) explicitly for verification at boot. These methods relies on solutions using either symmetric or asymmetric keys as provided by the ECU hardware. It relies on HW support to provide the initial root-of-trust (read only) code and a one-time-programmable verification key. Unless others are explicitly required, any others keys but the public key used at CheckMemory are supplier responsibility. Symmetric key solutions (e.g. AES-CMAC calculated over a logical block) requires that the confidential key is in a secured memory, protected from being altered and read as well as ensuring that a verification request origins from a trustworthy component in a secure environment.

Note 1: *Adaptation data that is changed when application is running is not part of start-up check.*
Example.

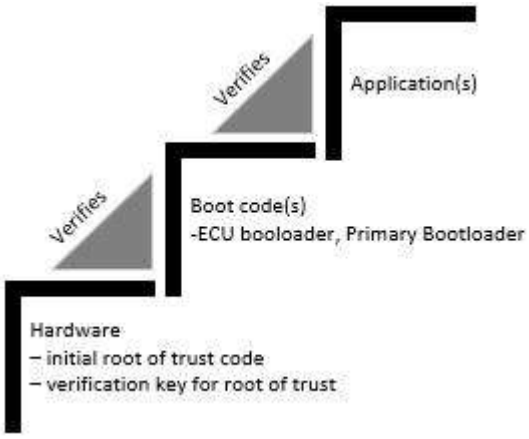


Figure - Chain-of-trust for authenticity verification at start

Note 2: *If the ECU calculates a cryptographic message authentication code (MAC, e.g. AES-CMAC) upon a valid signature verification at software download and verifies that MAC at every start-up, device unique keys must be used.*

Version history:
(2): Verification Method: Analysis AND Test change to Analysis.

Purpose: This requirement is optional. Authenticity checks could be performed at some time(s) when the main application is up and running. This run time check is to be performed by an additional trustworthy entity, a hardware secure module, and not the main application itself.
Source:
Verification Method: Analysis
Legacy ID: 419911v1

An optional check could be performed by an trustworthy entity, e.g. a hardware security module, when the main application is running. What to verify and how to react is ECU and function specific, but a minimum action is to store an event (security audit log) that is possible to read using a diagnostic identifier.

662823 v1 **Authenticity verification at software download (Program mode)**

Version history:
(1):

Purpose: Verifying code after a software download (at installation) is the minimum level of the concept.
Source:
Verification Method: Analysis AND Test
Legacy ID: 360452v1

The authenticity verification function shall always be performed at software download (installation).

662826 v1 **Debug tools and ports**

Version history:
(1):

Purpose: To avoid memory manipulation and e.g. deactivate the verification function or program software.
Source:
Verification Method: Analysis AND Test
Legacy ID: 419132v0

The ECU must be protected from all kind of memory manipulations (e.g. via diagnostic services or debuggers) except the diagnostic services supported and specified by in *Global Master Reference Database*.

•Software Download

4.11.1.3.2 Software Download

4.11.1.3.2.1 Programming Sequence Overview

This section describes the overall software programming sequence of ECU as a reference for rapid develpment of bootloader requirements, but it does not mean the sequence of some functions/routines are fixed, which should be noted, e.g. Check complete & compatible. The non implementation development of some funtions/routines shall be evaluated and approved by Geely Basetech Team.

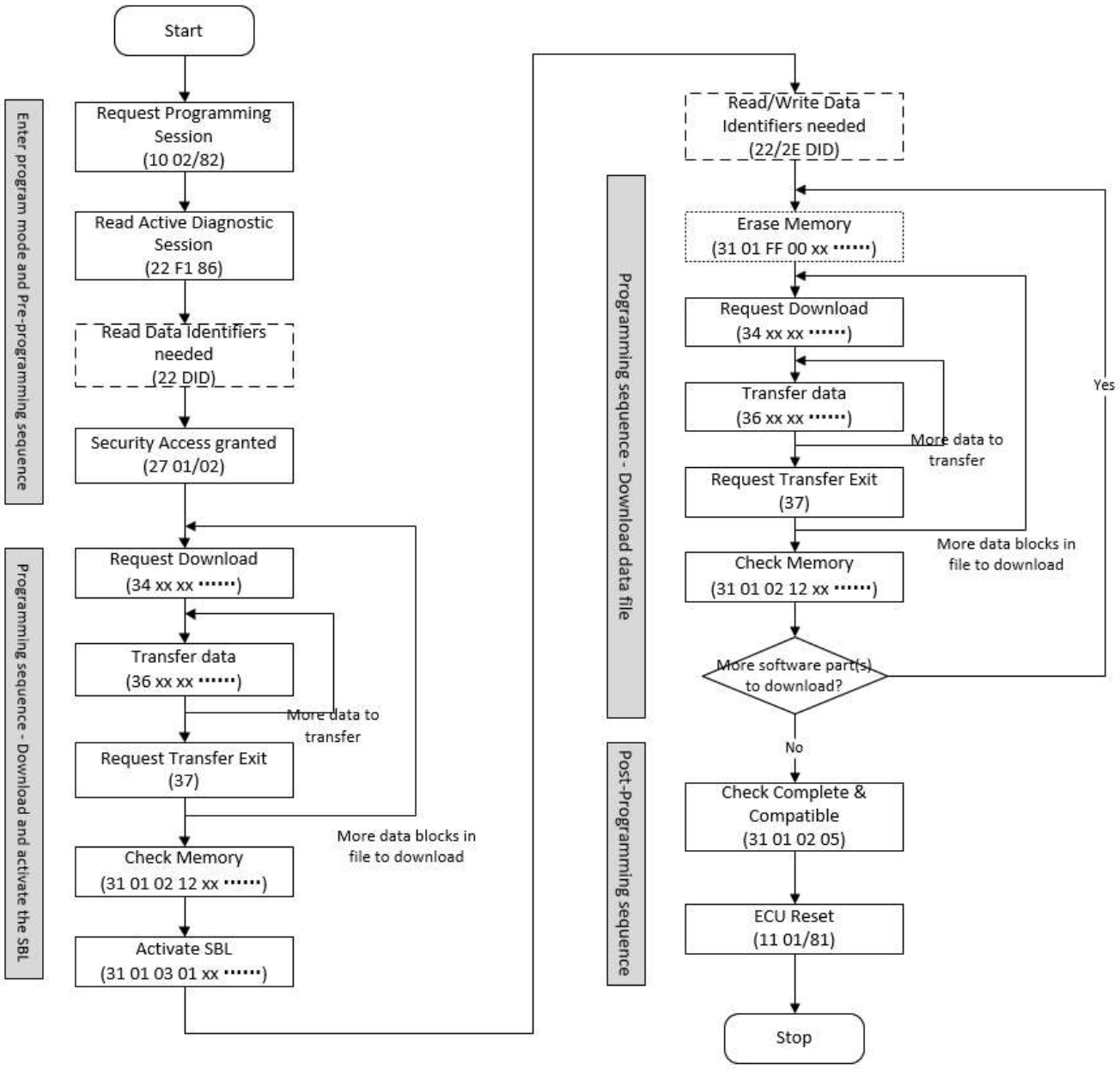


Figure - Programming sequence

4.11.1.3.2.2 Download and activate the SBL

It is important that the authenticity of the secondary bootloader (SBL) is verified prior to its activated. Otherwise, there is an increased risk of having the primary bootloader replaced or by other means bypass the provided verification mechanisms and start the application. The principle of verifying the SBL is the same as for any other logical blocks in the non-volatile memory, but for the SBL it is recommended to implement a single data block (but with addition of a *verification block table*). This might result in a simplified verification of the SBL. A single block SBL is however not suitable for ECU hardware architectures having additional microcontrollers, but only one SBL (software part) is used. Refer to requirements about secondary processors.

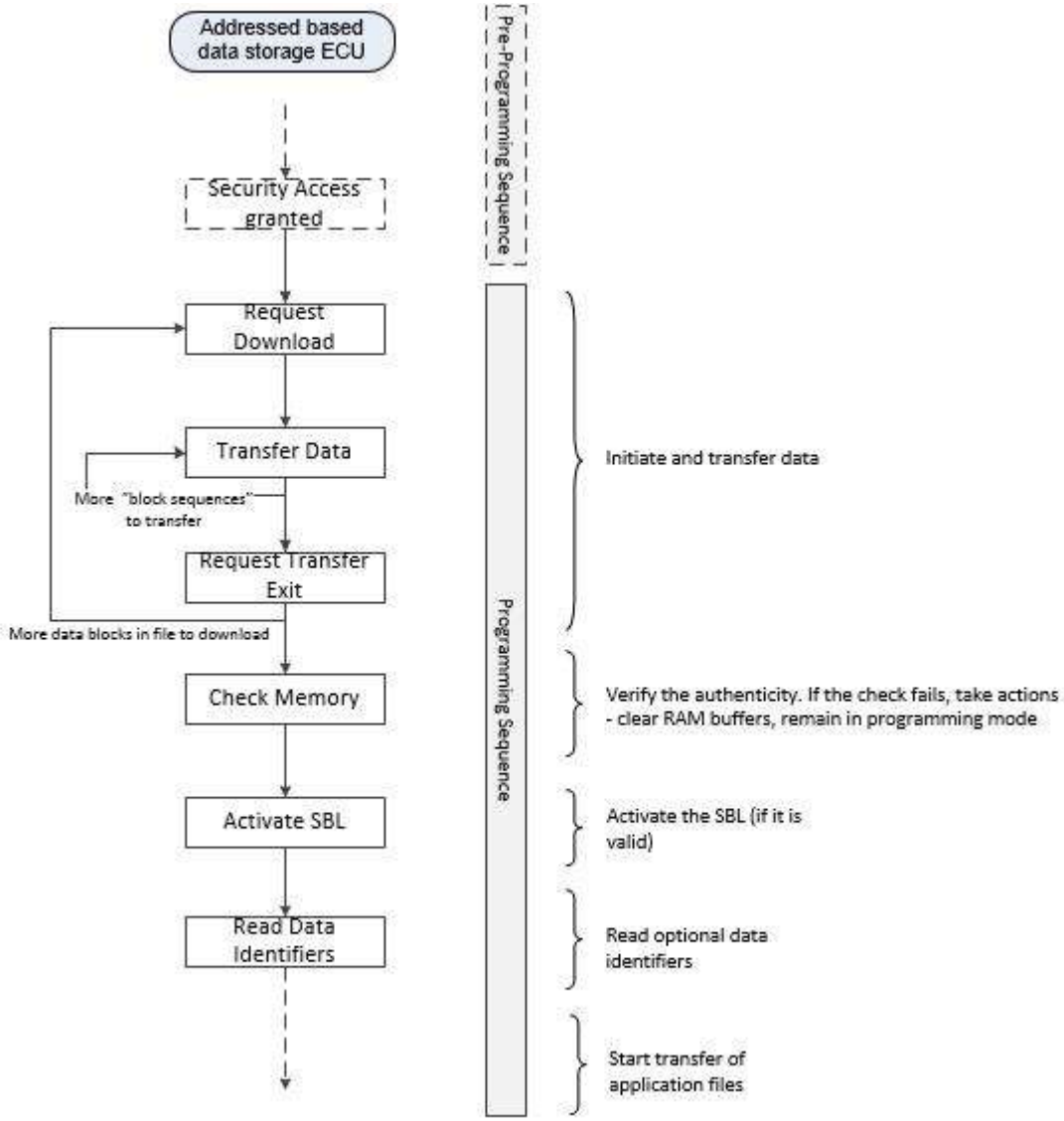


Figure - Overview - Programming Sequence SBL

The *Figure - Overview - Programming Sequence SBL* shows an overview of the programming sequence, a more detailed description of the Tester-ECU requests and responses are to be shown in next coming sections.

662814 v1 **Secondary Bootloader - Condition for activation**

Version history:
(1):

Purpose: The authenticity of the secondary bootloader must be verified. Otherwise, the verification mechanism could be bypassed and arbitrary application could be started.

Source:

Verification Method: Test

Legacy ID: 400704v3

The primary bootloader must ensure that the authenticity verification of the secondary bootloader is passed prior to it is activated. If the verification fails, the ECU shall remain in the primary bootloader and the volatile memory buffer shall be cleared. The used address parameter of the *ActivateSBL* routine control must be within the address range of the validated block(s).

662815 v1 **Secondary Bootloader Verification of unused data**

Version history:
(1):

Purpose: The authenticity verification shall check that there is no data programmed to memory not part of the authenticated verification block table. The SBL is stored in a volatile memory, RAM, and dependent of the actual RAM implemenation it might be faster for the bootloader to fill (write) the unused memory with known data (e.g. 0xFF) instead of checking (read) that no data has been programmed to memory not defined in the verification block.
Source:
Verification Method: Analysis
Legacy ID: 416451v1

The bootloader shall ensure that there is no SBL data programmed to memory that is not defined in the verification block table.

4.11.1.3.2.3 Memory Erasure

662809 v1 **Erase Identifier - restrictions**

Version history:
(1):

Purpose: To ensure that a complete logical block is erased is required for several purposes: (1) To ensure that all information about validity status is always erased (if part of the logical block). Otherwise, implicitly knowledge of the bootloader implementation is required to ensure that it is erased. (2) No unknown data is stored in the addressed logical block, i.e. previously installed data deleted ("uninstalled). (3) It will also enable simplified off-board review of the logical block definitions, by checking that the erase identifier is identical to the expected logical block.
Source:
Verification Method: Analysis AND Test
Legacy ID: 400878v3

The erase identifier range(s) of the software part header must match a valid logical block. The ECU shall at an *eraseMemory* request verify that the erase startAddress and Length matches a logical block range, otherwise the ECU shall abort the request. If the eraseMemory request is valid, the ECU must ensure that the complete memory for the addressed logical block is erased.

4.11.1.3.2.4 Request Download

662810 v1 **Request Download - restrictions on number of addressed logical blocks**

Version history:
(1):

Purpose: To verify that there is only one software part allocated to each logical block and to enable verification for the downloaded data range, i.e. each logical block is verified independently.
Source:
Verification Method: Analysis
Legacy ID: 400894v3

The ECU shall check that only one logical block is being updated for a single downloaded software part, i.e. one logical block is allowed to be addressed from a valid RequestDownload to a CheckMemory request is detected. If any memory outside that first logical block is addressed by a RequestDownload request(s), e.g. a second logical

block or some location not defined by the ESS at all, the ECU shall abort the RequestDownload and inform the client via a NRC.

As the processed length (compressed/encrypted) length is used as memorySize parameter of the RequestDownload request, there are scenarios when it is not possible to detect this at the RequestDownload stage but identified later when the data is uncompressed/decrypted and about to be written to the memory. Anyhow, the download shall be aborted when it is detected.

To be able to recover when download sequence is aborted, for example trying download the same software part once again or continue downloading the next software part (another logical block), the ECU shall upon a rejected request enter a state where no logical block is active and wait for next request to erase or program the memory.

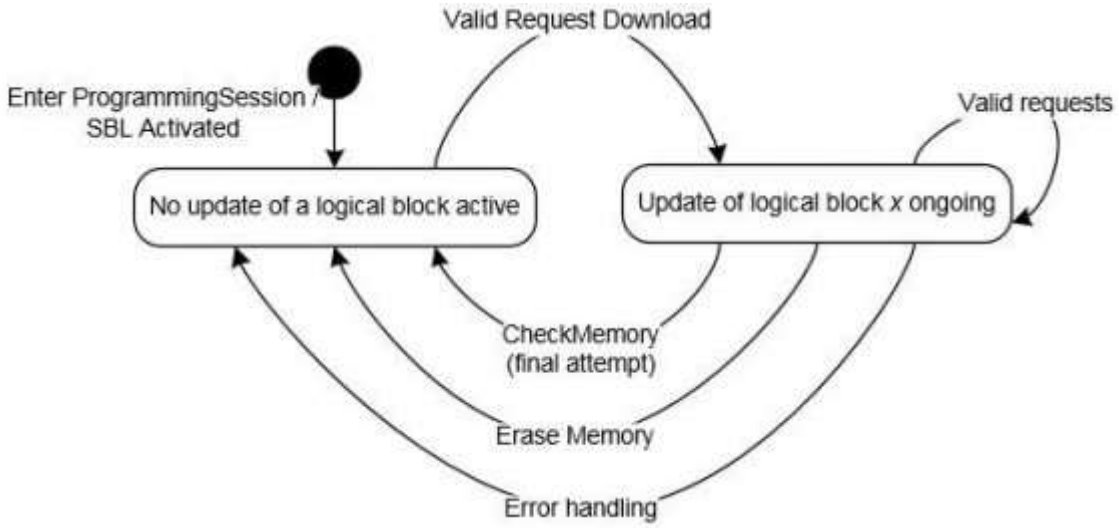


Figure - Simplified logical block active states overview

Notes:
- "Figure Simplified logical block active states overview" shows the principle only, details are omitted. 'Valid requests' are e.g. ReadDataByIdentifer requests, TransferData, RequestsTransferExit. "Error handling" includes the previously described scenario, where e.g. a RequestDownload addresses a logical block that is the 'active' block.
- The figure does not show the actual verification status of each logical block.

Example.
- the ECU is having a file based memory structure, not requiring an explicit erase memory request prior to request download. SBL is already verified and activated.

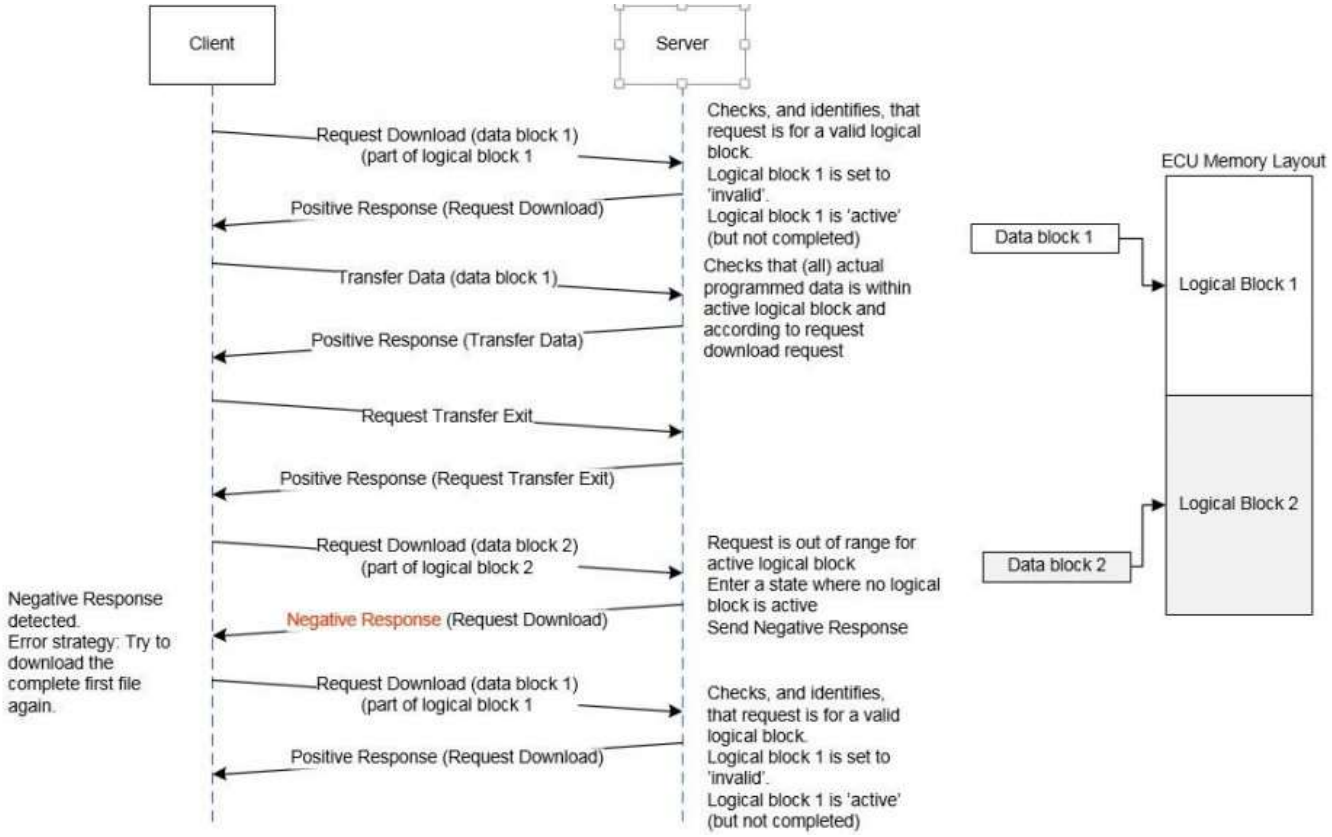


Figure - Example of ECU reaction after failed request (out of active logical block)

4.11.1.3.2.5 Check Memory

662811 v3 **Check Memory - Return values**

Version history:
(3): 对仅部分数据下载的情况也以0x06返回。

Purpose: Define the Check Memory response values.
Source:
Verification Method: Analysis AND Test
Legacy ID: 405471v2

The result values from the CheckMemory request shall be one byte long and represented according to *Table - CheckMemory return Values*. This means that the authenticity verification for a given logical block is passed when the routine returns 0 (CheckMemoryResultValue is 0x00)

Return values from the CheckMemory routine - CheckMemoryResponseValue

Value	Description
Others	*User defined*
0x0C	*Decryption of P12 file failed, i.e. PWD file (SWK2) and P12 (SWK1) file are not paired.*
0x0B	*Certificate verification failed, i.e. applicable for ECUs supporting code signing certificates.*
0x0A	*Error during storage of validity status information*

0x09	*An additional processor reported failed verification, i.e. only applicable for ECUs having additional processors.*
0x08	*ESS does not cover all programmable memory, i.e. only applicable when the logical block for the ESS is verified.*
0x07	*Read error during hash calculation over memory content.*
0x06	*No data downloaded at all or only partial data downloaded, nothing to verify.*
0x05	*The "blank" check failed, i.e. unexpected data at areas not included in the authenticated verification block table (e.g. data has not been "uninstalled").*
0x04	*The address or hash values of the downloaded data blocks does not match the expected values*
0x03	*Invalid format or length of the Verification Block Table*
0x02	*The public key ("root certificate") is not stored or is not complete (integrity check failed).*
0x01	*The signed data could not be authenticated, i.e. signature verification failed*
0x00	*The verification is passed*

Table - CheckMemory return values

Note: The certificate verification of extracted ECU certificates (i.e. SWK1 and SWK2) shall not be included in the CheckMemory process, but shall be performed under the Application mode and the results should be fed back through ReadDataIdentifier.

662812 v1 **Check Memory - verification of programmed data**

Version history:
(1):

Purpose: The CheckMemory is executed in the ECU via the routineIdentifier(Check Memory). This will inform the ECU that a software part is downloaded and that the authenticity verification of the stored data shall be started. For some scenarios, the software download tools will require two attempts (e.g. for development and production units)

Source:

Verification Method: Test

Legacy ID: 405174v2

The ECU shall start the authenticity verification of programmed data when the routineIdentifier CheckMemory is received. It shall be possible to make two consecutive CheckMemory attempts (i.e. one false attempt is possible). The verification result from the latest CheckMemory attempt shall be applied in the ECU. If the CheckMemory verification has failed and no more attempts are allowed, a new data transfer of the software part must be performed in order to verify the software at a CheckMemory request again. If a third consecutive CheckMemory request is sent, that request must be aborted.

Tester	Description	Data								Explanation
~~Tx~~	CheckMemory request (f,f)	11	04	31	01	02	12	xx	xx	
Rx	Flow control	30	00	00	00	00	00	00	00	
~~Tx~~	CheckMemory request (c,f)	21	xx	xx	xx	xx	xx	xx	xx	
...	...	...								
	CheckMemory request (c,f)	25	xx	xx	00	00	00	00	00	
Rx	Negative response	03	7F	31	78	00	00	00	00	Response pending (if needed)
Rx	Positive response	06	71	01	02	12	10	01	00	Verification **failed**
~~Tx~~	CheckMemory request (f,f)	11	04	31	01	02	12	~~yy~~	~~yy~~	A second consecutive CheckMemory attempt is ok, as no other request has been sent in between the CheckMemory requests.
Rx	Flow control	30	00	00	00	00	00	00	00	
~~Tx~~	CheckMemory request (c,f)	21	~~yy~~	~~yy~~	~~yy~~	~~yy~~	~~yy~~	~~yy~~	~~yy~~	
...	...	...								
	CheckMemory request (c,f)	25	~~yy~~	~~yy~~	00	00	00	00	00	
Rx	Negative response	03	7F	31	78	00	00	00	00	
Rx	Positive response	06	71	01	02	12	10	00	00	Verification **successful**

Table - CheckMemory requests

Table - CheckMemory requests shows when two CheckMemory attempts are possible, i.e. that no other request is sent in between. To save time, the ECU shall at the second CheckMemory attempt reuse the previously performed verifications, e.g. hash calculation.

4.11.1.3.2.6 Complete and Compatible - Validation of ECU software(s)

The section will define the relation between the complete check and authenticity verification, where a successful authenticity verification is required to pass the complete check. There are however use cases where exceptions might be required. Typical use cases are when e.g. an "APP" has failed the verification and the fault will be indicated elsewhere, i.e. the "APP" will not be started even though the main application is started. The responsibility of how to inform the user is left to the function to decide.

The bootloader shall at every start (initiation phase) and upon request from the diagnostic tester (routineIdentifier(Check Complete & Compatible), check if the ECU software(s) is complete and compatible before starting execution of the application.

The purpose of the "Complete & Compatible Check" is to ensure that the expected software parts are programmed successfully and are valid (complete), e.g. to detect if any part of the software is missing, broken (partly present/downloaded) or has been altered. This check shall also decide whether the combination of stored software parts are compatible to each other. The compatible check can be done in many different ways and will be different from case to case. It is either exclusively handled by the bootloader or by using a call-out to a function block that is part of the application. The compatibility check is only performed once the complete check has passed successfully.

The Complete & Compatible check shall be implemented in such a way that it must not be updated for each new or modified data file.

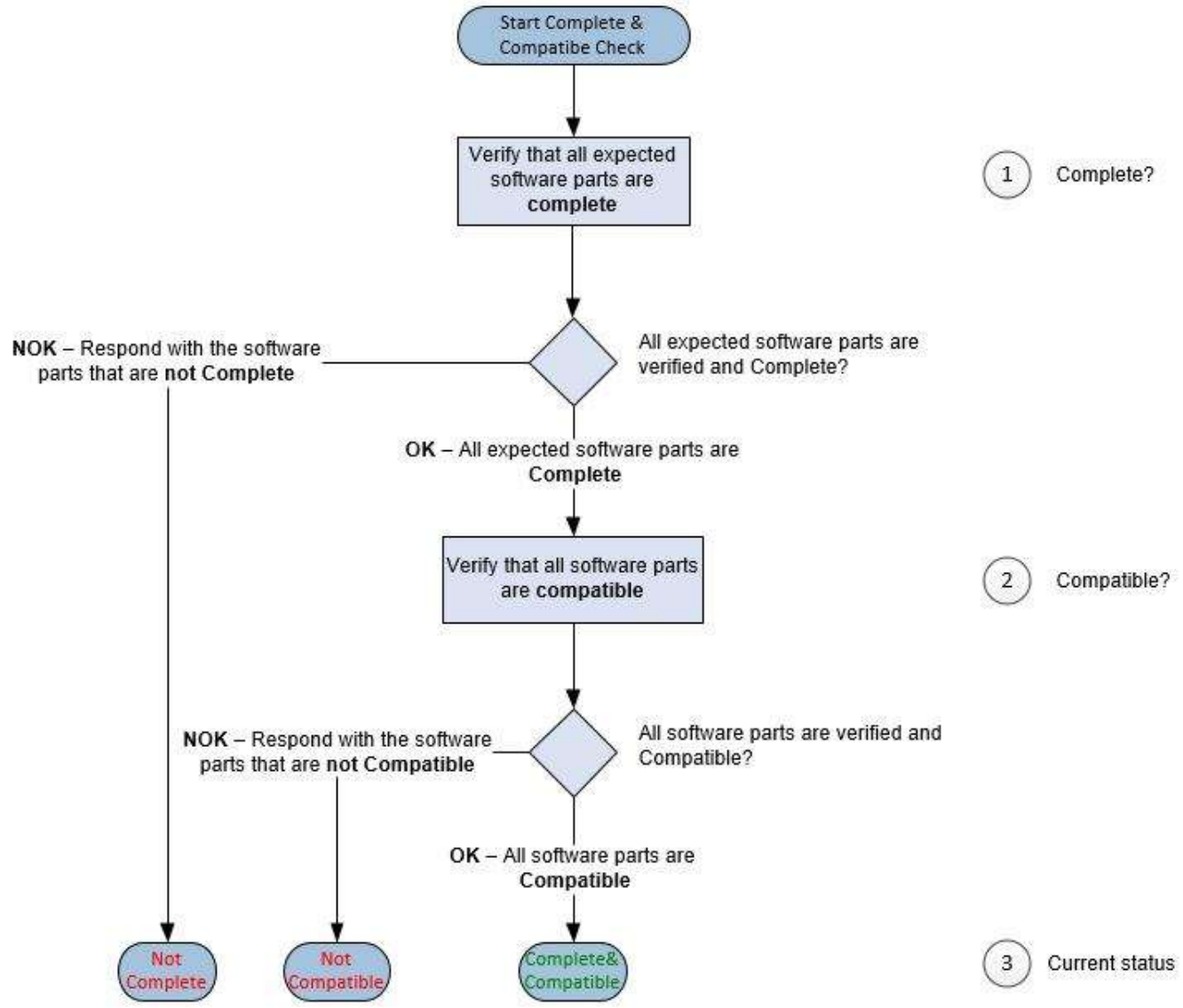


Figure - CompleteCompatibleFunction()
The check shown in *Figure - CompleteCompatibleFunction()* will be referred to as the **CompleteCompatibleFunction()**:

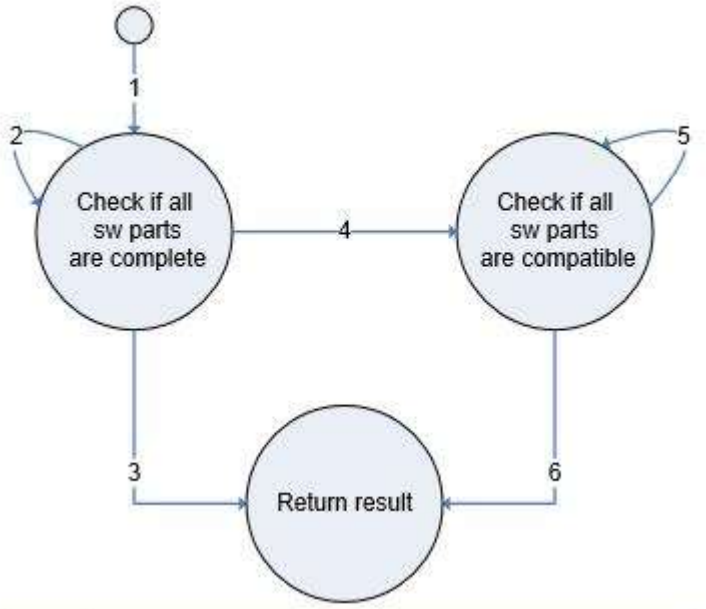
1. Check that software parts are complete.
2. Check that the software parts are compatible. The compatibility check shall only be performed if all expected software parts are complete.
3. The final result of the Complete & Compatible check, including the individual status per software part. The application must not be started unless both the Complete and Compatible check are passed successfully.

662828 v1 **Complete and Compatible sequence diagram**

Version history:
(1):

Purpose: Definition in which order the ECU shall validate the stored software(s).
Source:
Verification Method: Test
Legacy ID: 53959v4

The ECU shall first check that all software parts are complete prior to the compatibility check is performed. If the ECU contains incomplete software(s) it shall not check whether the software(s) is compatible. Hence, the ECU cannot indicate that both the complete and compatible check has failed. The software parts shall be checked in the order defined in *Figure - CompleteCompatibleFunction() sequence diagram.*



Step	Description
1	The bootloader starts the Complete & Compatible function
2	The Complete & Compatible function checks if stored SW parts(s) is **complete** in the following order: (1) ESS (2) EXE (3) Signal Configuration (4) Car Config (if applicable) (5..) Used defined SW parts(s)
3	Return result. If any SW part(s) is incomplete the Complete & Compatible function shall indicate that in the response
4	If all SW part(s) are complete
5	The Complete & Compatible function checks if stored SW parts(s) is **compatible,** using the same order as defined in step 2 but ESS is optional.
6	Return result

Figure - CompleteCompatibleFunction() sequence diagram

662827 v1 **Validation of ECU Software(s)**

Version history:
(1):

Purpose: To prevent start of incomplete or incompatible software(s).
Source:
Verification Method: Test
Legacy ID: 53935v1

The bootloader shall at every start check if the ECU software(s) is complete and compatible before starting execution of the application.

4.11.1.3.2.6.1 Complete and Compatible - Complete check

662830 v1 **Complete and Compatible - Complete function**

Version history:
(1):

Purpose: Define the responsible of performing the complete check. This verification must be performed by a trusted entity, i.e. no call-out to a non-verified area.
Source:
Verification Method: Analysis AND Test
Legacy ID: 401157v3

The complete check is always to be performed by the bootloader.

662829 v2 **Complete and Compatible - Definition of being Complete**

Version history:
(2): Verification Method: Demonstration change to Analysis ANS Test.

Purpose: The purpose with the complete validation is check that present softwares have been installed (programmed) as expected, i.e. an integrity check. This requirement defines the condition for the whole application to be considered as complete.
Source:
Verification Method: Analysis AND Test
Legacy ID: 403711v2

To be considered complete, all expected software parts must have passed the authenticity verification (using signatures). Hence, the CompleteCompatibleFunction() shall return that a software part is complete or incomplete according to that verification result. See *Figure - CompleteCompatibleFunction() Complete response*.

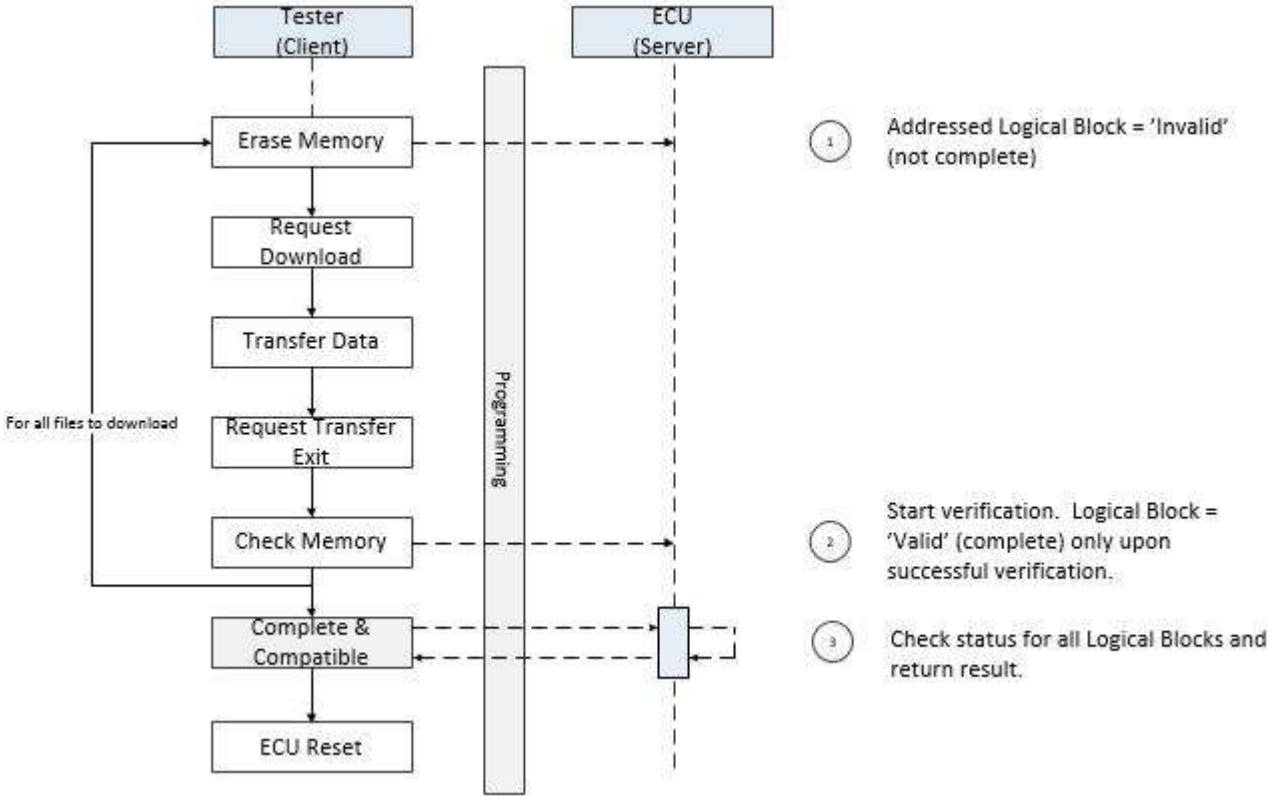


Figure - CompleteCompatibleFunction() Complete response

It might be a few exceptions where the main application might be started even though a software part has not passed the authenticity verification, e.g. an "APP" where the failed verification results in inhibiting the verified application from starting and this is indicated by the main application (function) itself. Every exception must be agreed with GRI Basetech Team.

The ECU is free to add additional conditions for being complete, e.g. some known patterns in the beginning and end of each software part. That is however left to the ECU designer to decide. Ideally, such patterns are checked prior to the software part is being signed and it will be implicitly verified as part of the authenticity verification.

4.11.1.3.2.6.2 Complete and Compatible - Compatible check

662832 v1 **Complete and Compatible - Compatible function**

Version history:
(1):

Purpose: Define alternatives of how to perform the compatibility check, to not limit the ECU designer to use a specific method.
Source:
Verification Method: Inspection
Legacy ID: 401156v2

The compatible check is either exclusively executed by the bootloader or via a call-out to a function placed in a downloaded software part (e.g. application). The bootloader must always verify that the compatible function applied is available and is complete, i.e. it has passed the complete check (resides in a verified memory location) and the existence is verified by e.g. some known start- and end-patterns prior to execution.

Note 1: *It is the responsibility of the bootloader and application developer to agree upon how to realize this check.*

Please pay attention to e.g. post-build (e.g. reconfiguration of the communication matrix by moving an existing signal) consistency check requirements, which might require information about the application build process.
Note 2: *See also "Appendix: Section Examples of different compatible functions".*

662831 v2 **Complete&Compatible - Definition of being Compatible**

Version history:
(2): Update Verification Method to Analysis AND Test.

Purpose: The purpose with the compatibility validation is to detect if present softwares are compatible in such way that the ECU is able to start and communicate. Hence, the ECU application could inform the user by e.g. setting a diagnostic trouble code.
Source:
Verification Method: Analysis AND Test
Legacy ID: 401140v3

The ECU is compatible if it is at least able to start the application, is able to respond to diagnostics and alert the user if a function is disabled or degraded. An ECU that is compatible shall always be able to enter Programming Session and be re-programmed.

Note: *The application must react accordingly, i.e. enter a defined fail-safe mode if it is started but a function is disabled or degraded. Such a behavior is however the responsibility of the application to decide.*

4.11.1.3.2.6.3 CompleteCompatibleFunction()

662836 v1 **CompleteCompatibleFunction - error handling**

Version history:
(1):

Purpose: Define a error handling strategy, where a restricted approach is applied meaning that the application must not be started unless it is proven to be valid.
Source:
Verification Method: Inspection
Legacy ID: 405051v1

In case of an unexpected fault during the CompleteCompatibleFunction() that prevents the function to be completed and the root cause cannot be derived to a specific software part, the ECU shall be considered as not complete (or not compatible respectively, dependent of the check that was not completed). This shall be indicated in the response to the routineIdentifier(Check Complete & Compatible) request. If the bootloader cannot identify which part that failed the verification, the bit for the Executable shall be set.

662833 v1 **CompleteCompatibleFunction()**

Version history:
(1):

Purpose: To define the functionality of the CompleteCompatibleFunction().
Source:
Verification Method: Analysis AND Test
Legacy ID: 53957v3

The CompleteCompatibleFunction() is invoked from the bootloader via either the routine (Check Complete & Compatible) or during the Init phase. The CompleteCompatibleFunction() shall be implemented in a way that allows new or modified software part without an updated application(EXE). The CompleteCompatibleFunction() shall detect if any of the software parts are not as expected, missing, broken (partly downloaded), altered etc. The

function is also responsible to decide whether the combination of stored software parts are compatible to each other.

662834 v1 **CompleteCompatibleFunction() return value**

Version history:
(1):

Purpose: Definition of the bitcoded CompleteCompatibleFunction() return value. When the complete check is part of the bootloader, the bootloader might have limited knowledge of the relation between a logical block and actual sw part type (DATA, EXE).
Source:
Verification Method: Analysis AND Test
Legacy ID: 53958v6

The return value from the CompleteCompatibleFunction() shall be 4 bytes long and bit coded and contain the type of failure and which software part(s) that has failed. If a failure has occurred the bit value is set to one (1), not effected bits are set to zero (0). This means the Complete and compatible function has “PASSED”, when complete and compatible function returns 0. The diagnostic database shall contain the actual return values for an ECU.

Example.
Bit (4) shall be set to "0" if the ECU doesn't supports a software part (e.g. the Car Configuration).

Return values from CompleteCompatibleFunction()	
Bit number	**Description**
Bit 6-31 (msb)	User defined software parts(s). One bit per SW part which is documented by each ECU respectively.
Bit 5	ECU Software Structure
Bit 4	Car Config, if applicable.
Bit 3	Signal Configuration
Bit 2	Executable
Bit 1	Compatible check
Bit 0 (lsb)	Complete check

Table - Bit coded return value from the CompleteCompatibleFunction()

662835 v2 **Maximum time to perform the Complete&Compatible validation**

Version history:
(2): CC校验的最大执行时间由20ms调整为200ms，匹配RID 0205的P4server_max时间。

Purpose: To define the maximum time to perform the Complete & Compatible validation of ECU software(s), to fulfill the ECU start-up timing requirement.
Source:
Verification Method: Test
Legacy ID: 53970v3

The maximum time to perform the total Complete & Compatible sequence described in *Figure - CompleteCompatibleFunction()* shall not exceed 200ms.

4.11.1.3.2.7 ECU Reset - Application start

662813 v1 **ECU Reset - Application Start**

Version history:
(1):

Purpose: To ensure that the application is executed from a trusted location.
Source:
Verification Method: Analysis AND Test
Legacy ID: 416011v0

When the application start up conditions are fulfilled, according to the Complete&Compatible function, the application start address must be within the range of a validated logical block.

4.11.1.3.2.8 Request Upload

662816 v2 **RequestUpload - Restricted use**

Version history:
(2): Update Verification Method to Analysis.

Purpose: Define restricted areas and recommendations if the RequestUpload service is supported.
Source:
Verification Method: Analysis
Legacy ID: 429415v1

If the RequestUpload service is supported by the ECU, the following areas shall be blocked and not possible to upload data from:
(i) Boot area and where the verification function is stored
(ii) Area where the public key is stored
(iii) Area where validity status information is stored
(iv) Area where any confidential key is stored

Note: For intellectual properties rights (e.g. when the software part is encrypted at download), the service shall be used with care.

•Appendix

4.11.1.3.3 Appendix

4.11.1.3.3.1 Examples of different scenarios for the Complete & Compatible function

All bits are equal to '0'	The ECU contains complete and compatible software parts. The ECU can start with full normal operational functionality
Bit 6 to 31: is set to '1'	Failure within user defined software parts(s).
Bit 5: is set to '1'	Failure within the ECU Software Structure
Bit 4: is set to '1'	Failure within the Car Configuration (not applicable for every ECUs)
Bit 3: is set to '1'	Failure within the Signal Configuration
Bit 2: is set to '1'	Failure within the ExecutableSoftware
Bit 1: is set to '1'	The ECU contains incompatible software part(s)
Bit 0: is set to '1'	The ECU contains incomplete software part(s)

- If the Signal Configuration data file is created with same fix and target file as the application but with a different net file the returned value from the CompleteCompatibleFunction() shall be compatible. (0b00000000000000000000000000000000)
- If the Signal Configuration data file is created with the same net and target file as the application but with a different fix file the returned value from the CompleteCompatibleFunction() shall be incompatible. (0b00000000000000000000000000001010)
- If a data file is modified but still is supported by the application the returned value from the

CompleteCompatibleFunction() shall be complete and compatible.
(0b00000000000000000000000000000000)
- If a data file is present (bit 6, user defined software) which makes the application incapable of responding on e.g. diagnostic requests the returned value from the CompleteCompatibleFunction() shall be incompatible
(0b00000000000000000000000001000010)
- If a data file (bit 6, user defined software) is missing or broken the returned value from the CompleteCompatibleFunction() shall be incomplete.
(0b00000000000000000000000001000001)
- If the EXE data file is broken or corrupt but the CompleteCompatibleFunction block is complete the returned value from the CompleteCompatibleFunction() shall be incomplete.
(0b00000000000000000000000000000101)
- If the Signal Configuration data file is missing or broken the returned value from the CompleteCompatibleFunction() shall be incomplete.
(0b00000000000000000000000000001001)
- If the bootloader doesn't find correct value or can't read the StartCompleteCompatibleSignature or EndCompleteCompatibleSignature the bootloader shall treat this as incomplete EXE file to further validations and return the value of
(0b00000000000000000000000000000101)

4.11.1.3.3.2 Examples of different Compatible functions

4.11.1.3.3.2.1 Bootloader Compatbile check

If the bootloader is performing the compatibility check itself, it could (for example) using some patterns where the locations are defined by the ESS (Compatibility Information).

Example.

- Each logical block contains compatibility Information in the "header/info" part, according to the TIER 1 defined format. This header part must support functional growth, i.e. that the software structure is changed from e.g. three to four logical blocks.
- The dependencies to the application is checked, for each software part.

4.11.1.3.3.2.2 Application Complete & Compatible block

To support use-cases where the compatibility check is highly dependent to the application build process, i.e. where the bootloader does not have access to all required information, a possible realization is put the compatibility check into the application (EXE) and let the bootloader to a call-out to that block. This might be required when the ECU shall support the AUTOSAR post-build loadable concept, where it shall be possible to reconfigure the communication matrix (typically move an existing signal from one frame to another, add a signal gateway, etc.)

Example.
A *Application Complete & Compatible block* is defined and part of the EXE software part. The structure and location of this block is known to the bootloader.

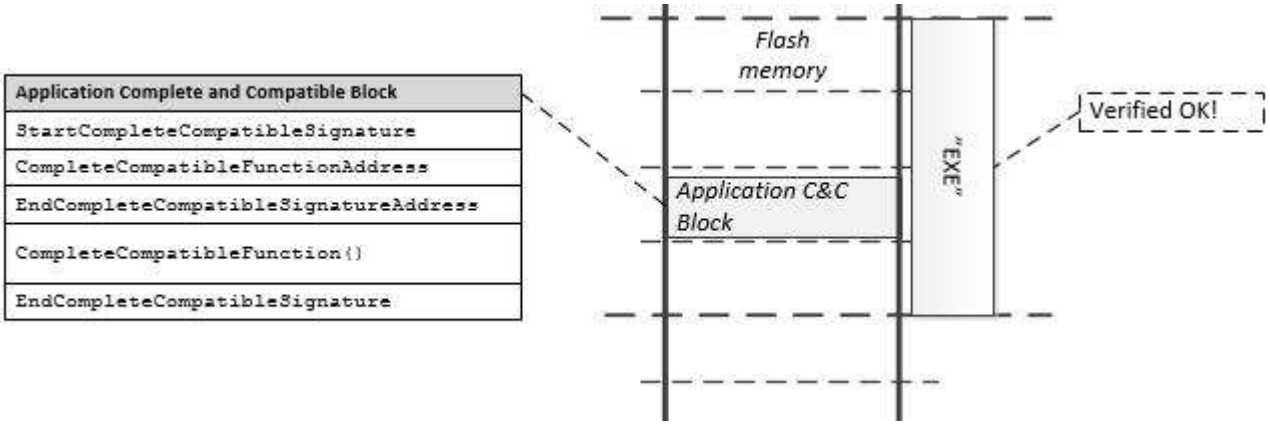
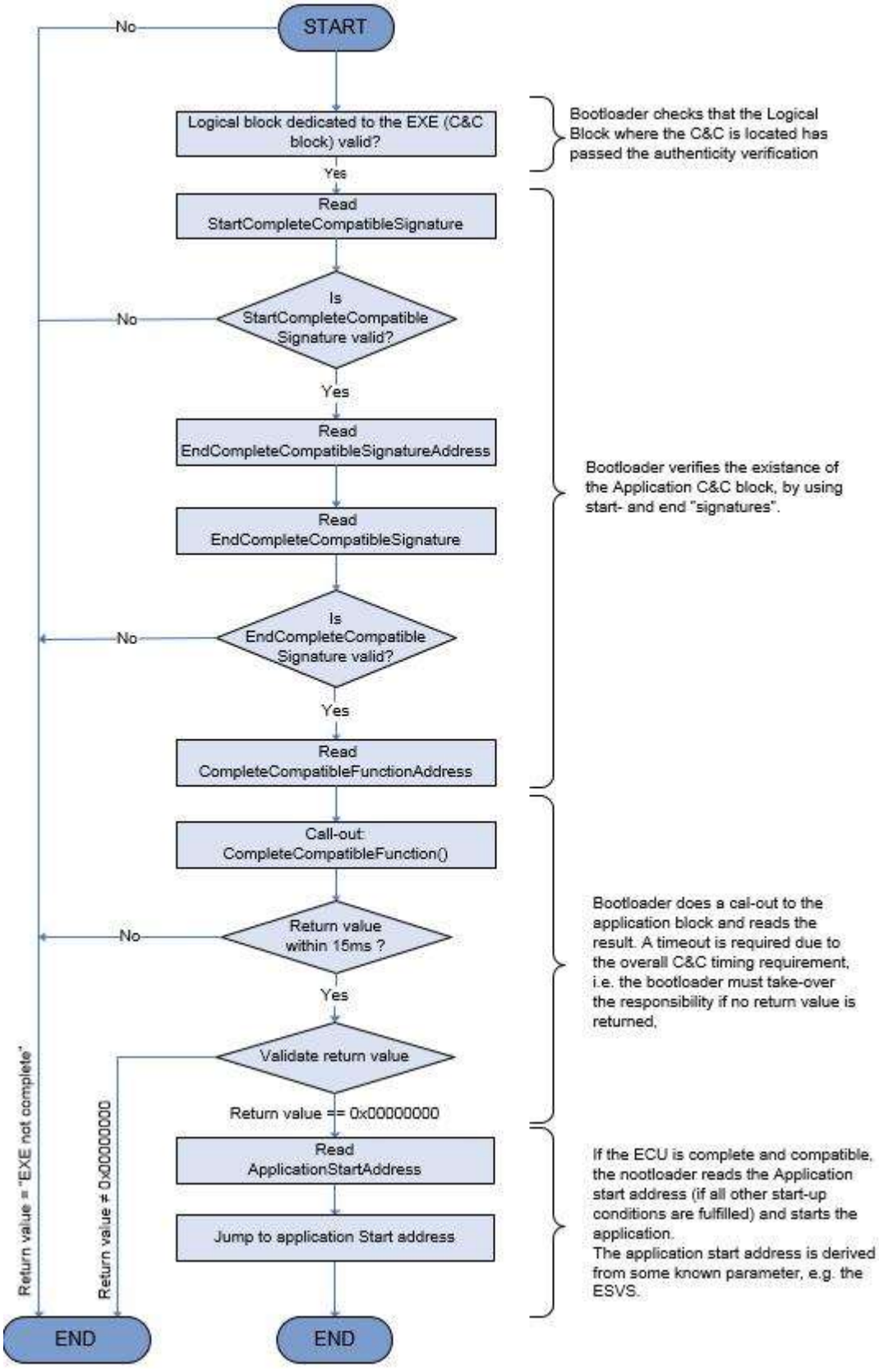


Figure - Application Complete & Compatible block

- StartCompleteCompatibleSignature: To verify the existence (start) of the Application Complete &

Compatible block

- CompleteCompatibleFunctionAddress: To define the size and location of the CompeteCompatibleFunction(), i.e. here it is only the compatibility check.
- EndCompleteCompatibleSignatureAdress: To be possible to identify where the EndCompleteCompatibleSignature is located
- Complete&Compatible(): The function invoked from the bootloader, using a call-out.
- EndCompleteCompatibleSignature: To verify the existence (end) of the Application Complete & Compatible block.



Example - Application (EXE) Complete and Compatible block

Note 1: *In case of no response value received, and the a timer has expired, the bootloader terminate the execution of the application block and treat the EXE as incomplete during further validations.*

Note 2: *The location and size of the Application Complete & Compatible block must be agreed between the bootloader and the application developer.*
Note 3: *No call-out is allowed unless it is proven that the call-out address is located in a valid (verified) location, as shown in "Figure - Application Complete & Compatible block".*

4.11.1.4 Bootloader requirements for SWDL with no check memory only for GEEA3.0

•Software Download

4.11.1.4.1 Software Download

4.11.1.4.1.1 Programming Sequence Overview

This section describes the overall software programming sequence of ECU as a reference for rapid develpment of bootloader requirements, but it does not mean the sequence of some functions/routines are fixed, which should be noted, e.g. Check complete & compatible. The non implementation development of some funtions/routines shall be evaluated and approved by Geely Basetech Team.

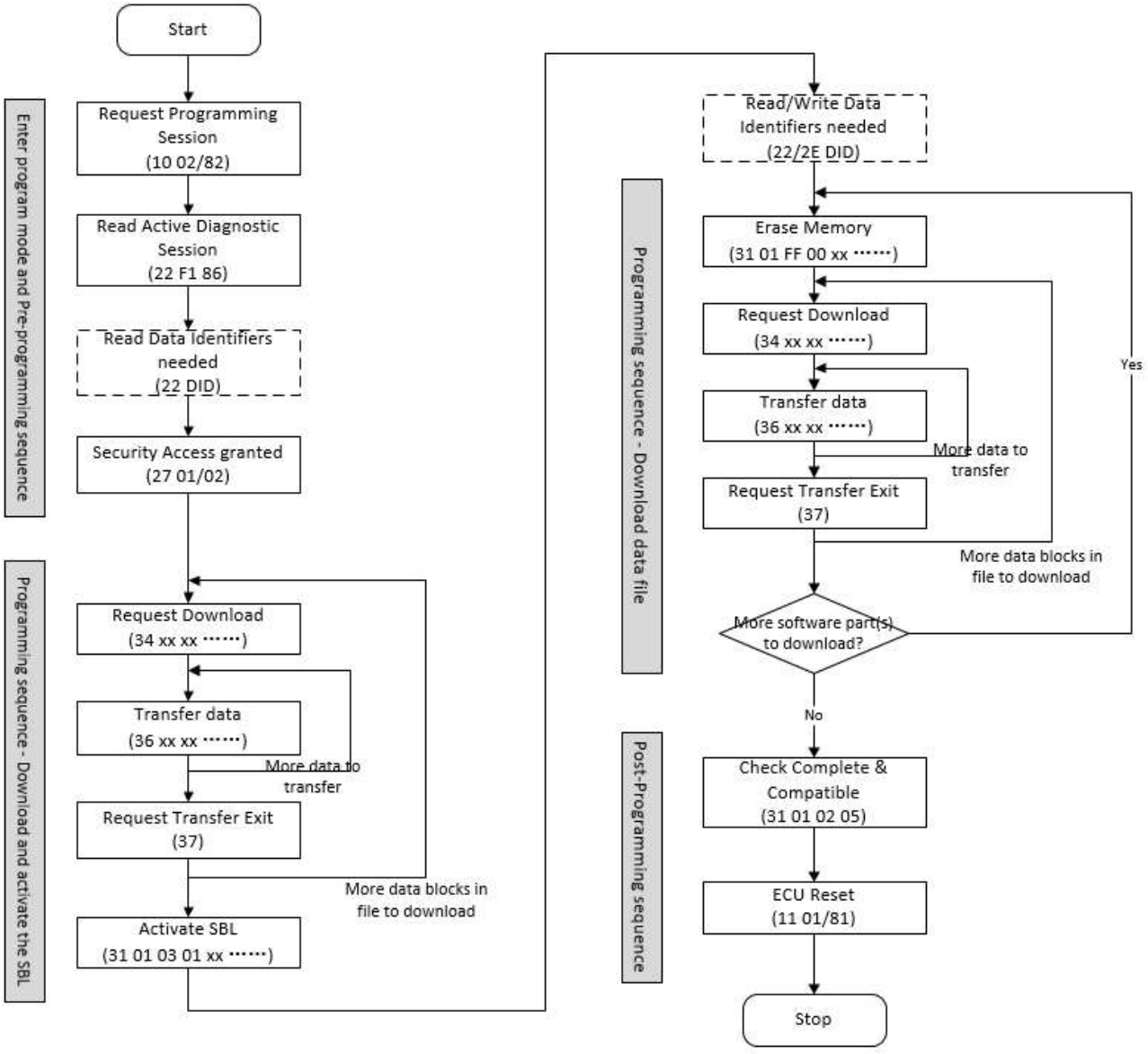


Figure - Programming sequence

•Validation of ECU software(s)

4.11.1.4.2 Validation of ECU software(s)

The bootloader shall at every start check if the ECU software(s) is complete and compatible before starting execution of the application. The Complete & Compatible function block, see Table " *Application Complete & Compatible block*" shall be a part of the application; it is each application developer's responsibility to implement this functionality.

The requirements on the CompleteCompatibleFunction() is to detect if any part of the software is missing or broken (partly present/downloaded), the function is also responsible to decide whether the combination of stored software parts are compatible to each other. This specification doesn't specify how the application shall detect if the ECU is complete and compatible, this can be done in many different ways and will be different from case to case.

The CompleteCompatibleFunction() is executed from the bootloader via either the routineIdentifier(Check Complete & Compatible) or during the Init phase via a call-out, the function shall return a value back to the bootloader if the ECU is capable to start with full operational functionality or not.

The CompleteCompatibleFunction() shall not be implemented in a way that it must be updated for each new or modified data file.

662827 v1 **Validation of ECU Software(s)**

Version history:
(1):

Purpose: To prevent start of incomplete or incompatible software(s).
Source:
Verification Method: Test
Legacy ID: 53935v1

The bootloader shall at every start check if the ECU software(s) is complete and compatible before starting execution of the application.

4.11.1.4.2.1 Application Complete & Compatible block

It is important that the block structure listed in Table " *Application Complete & Compatible block*" always is followed since the bootloader read the information at defined memory addresses.

The dependency between the bootloader and the application is the memory address where the Complete & Compatible block is started from. The bootloader must also know the value of StartCompleteCompatibleSignature and EndCompleteCompatibleSignature. If any of these are changed the bootloader must be updated.

In Table " *Application Complete & Compatible block*" the memory location and size of the addresses is a recommendation, this must be agreed between the bootloader and application developer.

Application Complete & Compatible Block			
Name	Memory location	Size	Notes
StartCompleteCompatibleSignature	Start address of the C&C block	8 bytes	
ApplicationStartAddress	+ 8 bytes from start	4 bytes	Recommended size
CompleteCompatibleFunctionAddress	+ 12 bytes from start	4 bytes	Recommended size
EndCompleteCompatibleSignatureAddress	+ 16 bytes from start	4 bytes	Recommended size
CompleteCompatibleFunction()	Memory address defined at + 12 bytes from start Note: *Does not need to start at +20 bytes from start*	Not defined	All data needed shall be placed between the StartCompleteCompatibleSignature and EndCompleteCompatibleSignature
EndCompleteCompatibleSignature	Memory address defined at + 16 bytes from start	8 bytes	Shall be placed in the memory after the CompleteCompatibleFunction() for validation purpose.

Table - Application Complete & Compatible block

662844 v1 **Complete & Compatible block location**

Version history:
(1):

Purpose: To define the location and of the Complete & Compatible block.
Source:
Verification Method: Analysis
Legacy ID: 53936v1

The Complete & Compatible function block, see Table " *Application Complete & Compatible block"* shall be a part of the application, it is each application developers responsibility to implement this functionality.

Note: The memory locations and size of the addresses is a recommendation, this must be agreed between the bootloader and application developer.

662845 v1 **Complete & Compatible block structure**

Version history:
(1):

Purpose: To avoid updates of the bootloader it is important that the Complete & Compatible block is strictly followed. If e.g. the start address of the C&C block is changed drives an updated bootloader.
Source:
Verification Method: Analysis
Legacy ID: 53952v1

The block structure listed in Table " *Application Complete & Compatible block*" shall always be followed since the bootloader read the information at defined memory addresses.

4.11.1.4.2.1.1 StartCompleteCompatibleSignature

The bootloader must know the memory address where the StartCompleteCompatibleSignature is located.

662846 v2 **StartCompleteCompatibleSignature**

Version history:
(2): Verification Method: Analysis AND Test change to Analysis.

Purpose: To define a start signature of the Complete & Compatible block which the primary bootloader uses to check the consistency of the Complete & Compatible block.
Source:
Verification Method: Analysis
Legacy ID: 53953v1

The StartCompleteCompatibleSignature shall be 8 bytes long. A value of 0xFFFFFFFFFFFFFFFF is not allowed.Recommended is an 8 bytes StartCompleteCompatibleSignature with a value of 0x5374617274536967 (equal to text string "StartSig" in ASCII-format).

Note: *The Endianness for the CPU used shall be the same for both the application and bootloader*.

4.11.1.4.2.1.2 ApplicationStartAddress

662847 v1 **ApplicationStartAdress**

Version history:
(1):

Purpose: To define the size and memory location where the application start address is placed in the Complete & Compatible block.
Source:
Verification Method: Analysis
Legacy ID: 53954v1

When the bootloader starts the application it shall jump to the address defined by ApplicationStartAddress.

The application start address shall be placed 8 bytes from start in the Complete & Compatible block with a recommended length of 4 bytes.

4.11.1.4.2.1.3 CompleteCompatibleFunctionAddress

662848 v1 **CompleteCompatibleFunctionAddress**

Version history:
(1):

Purpose: To define the size and memory location where the CompleteCompatibleFunction() starts.
Source:
Verification Method: Analysis
Legacy ID: 53955v1

When the bootloader starts the CompleteCompatibleFunction() it shall start execution of the function at the address defined by CompleteCompatibleFunctionAddress.

The CompleteCompatibleFunction() start address shall be placed 12 bytes from start in the Complete & Compatible block with a recommended length of 4 bytes.

4.11.1.4.2.1.4 EndCompleteCompatibleSignatureAddress

662849 v1 **EndCompleteCompatibleSignatureAddress**

Version history:
(1):

Purpose: To define the size and memory location where the EndCompleteCompatibleSignature is started from
Source:
Verification Method: Analysis
Legacy ID: 53956v1

When the bootloader verifies the consistency of the total Complete & compatible block it shall read at the address defined by EndCompleteCompatibleSignatureAddress.

The EndCompleteCompatibleSignature start address shall be placed 16 bytes from start in the Complete & Compatible block with a recommended length of 4 bytes.

Note: Since the CompleCompatileFunction() may have different length for different software(s) setup in the ECU, the EndCompleteCompatibleSignature can not be placed on a fixed memory address.

4.11.1.4.2.1.5 CompleteCompatibleFunction()

Section " *Examples of different scenarios for the Complete & Compatible function"* gives some use-case examples of the complete and compatible function.

662850 v1 **CompleteCompatibleFunction()**

Version history:
(1):

Purpose: To define the functionality of the CompleteCompatibleFunction().
Source:
Verification Method: Analysis AND Test
Legacy ID: 53957v2

The CompleteCompatibleFunction() is invoked from the bootloader via either the routine (Check Complete & Compatible) or during the Init phase via a call-out. The call-out memory address is defined in the parameter CompleteCompatibleFunctionAddress. The function shall return a bit coded value back to the bootloader if the ECU is capable to start with full operational functionality or not.

The CompleteCompatibleFunction() shall be implemented in a way that allows new or modified software part without an updated application(EXE).

The CompleteCompatibleFunction() shall detect if any of the software parts are missing or broken (partly downloaded), the function is also responsible to decide whether the combination of stored software parts are compatible to each other. The CompleteCompatibleFunction() returns back the information to the bootloader.

662851 v1 **CompleteCompatibleFunction() return value**

Version history:
(1):

Purpose: Definition of the bitcoded CompleteCompatibleFunction() return value
Source:
Verification Method: Test
Legacy ID: 53958v3

The return value from the CompleteCompatibleFunction() shall be 4 bytes long and bit coded according to Table " *Bit coded return value from the CompleteCompatibleFunction()"* , the return value shall contain type of failure and which software part(s) that has failed. If a failure has occurred the bit value is set to one (1), not effected bits are set to zero (0). This means the Complete and compatible function has “PASSED”, when complete and compatible function returns 0.

Bit (4) shall be set to "0" if the ECU dosen't supports the Car Configuration concept.

Return value from the CompleteCompatibleFunction()	
Bit number	Description
Bit 5 to 31 (msb)	User defined software part(s), One bit per SW which is documented by respectively ECU.
Bit 4	Car Configuration If applicable
Bit 3	Signal Configuration
Bit 2	Executable, which include the Complete & Compatible block
Bit 1	Compatible check
Bit 0 (lsb)	Complete check

Table - Bit coded return value from the CompleteCompatibleFunction()

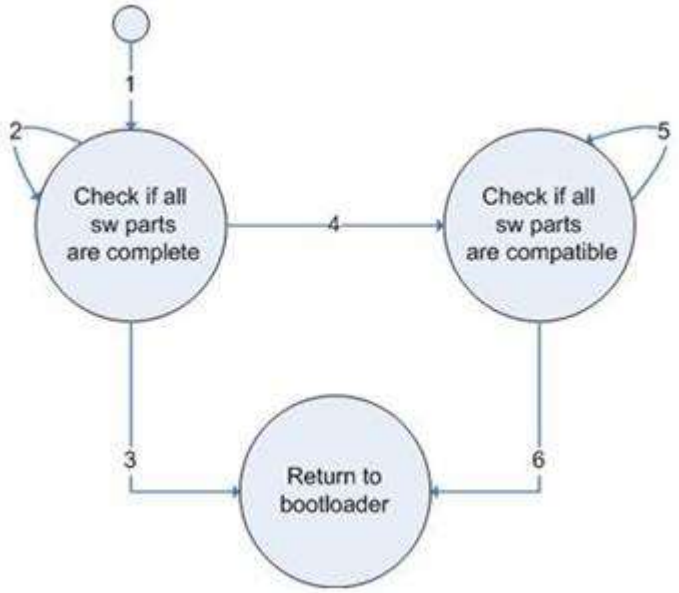
662852 v1 **CompleteCompatibleFunction() sequence diagram**

Version history:
(1):

Purpose: Definition in which order the ECU shall validate the stored software(s).
Source:
Verification Method: Test
Legacy ID: 53959v2

The ECU shall implement the CompleteCompatibleFunction() to validate the stored software

parts(s) in the following sequence. The CompleteCompatibleFunction() shall first detect if all software parts is complete. If the ECU contains incomplete software(s) it shall not check whether the software(s) is compatible, this means that complete and compatible can not be set at the same time.



Step	Description
1	The bootloader do a call-out to the Complete & Compatible function()
2	The Complete & Compatible function checks if stored SW part(s) is complete in the following order: (1) EXE (2) Signal Configuration (3) Car Config (if applicable) (4..) User defined SW(s)
3	If any SW part(s) is incomplete the Complete & Compatible function returns the result to the bootloader.
4	If all SW part(s) are complete
5	The Complete & Compatible function checks if stored SW part(s) is compatible in the following order: (..) Same order as in step 2
6	The Complete & Compatible function returns the result to the bootloader.

Figure - CompleteCompatibleFunction() sequence diagram

662853 v2 **Maximum time to execute the CompleteCompatibleFunction()**

Version history:
(2): Verification Method: Analysis AND Test change to Analysis.

Purpose: To prevent the ECU to get stuck in the application during validation of software(s).

Source:

Verification Method: Analysis
Legacy ID: 53961v1

If the CompleteCompatibleFunction doesn't return and produce a result within 15 ms the bootloader shall terminate the execution of the CompleteCompatibleFunction and treat this as incomplete EXE file during further validations.

Note: *The implementation of the timeout can be realized with a timer that aborts the CompleteCompatibleFunction() and a executing the CompleteCompatibleFunction within a sandbox, protected memory area or any similar technique.*

4.11.1.4.2.1.6 EndCompleteCompatibleSignature

662854 v2 **EndCompleteCompatibleSignature**

Version history:
(2): Verification Method: Analysis AND Test change to Analysis.

Purpose: To define the end signature in the Complete & Compatible block.
Source:
Verification Method: Analysis
Legacy ID: 53963v1

When the bootloader verifies the consistency of the total Complete & compatible block it shall read the EndCompleteCompatibleSignature. The EndCompleteCompatibleSignature memory address is defined by EndCompleteCompatibleSignatureAddress.

The EndCompleteCompatibleSignature shall be 8 bytes long. A value of 0xFFFFFFFFFFFFFFFF is not allowed.

Recommended is an 8 bytes EndCompleteCompatibleSignature with a value of 0x456e645369676e61 (equal to text string "EndSigna" in ASCII-format).

Note: *The Endianness for the CPU used shall be the same for both the application and bootloader.*

4.11.1.4.2.1.7 Complete & Compatible flow chart diagram

The Complete & Compatible sequence is initiated by either the routineIdentifier(Check Complete & Compatible) or during the Init phase in the bootloader.

The routineIdentifier(Check Complete & Compatible) is only possible to activate when the bootloader is in the programmingSession state, see Figure " *Simplified bootloader state diagram".*

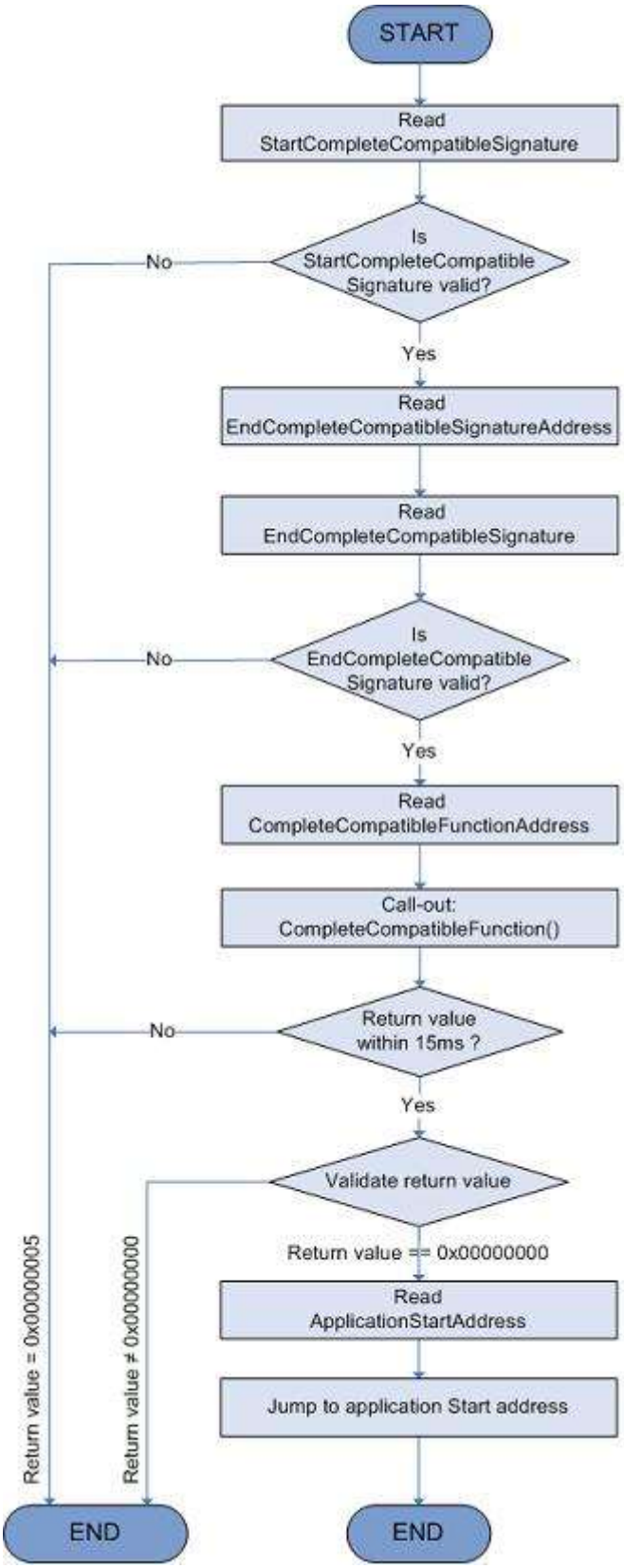


Figure - Complete & Compatible flow chart diagram

662855 v1 **Complete & Compatible flow chart diagram**

Version history:
(1):

Purpose: Define the sequence that shall be used by the bootloader when validate the ECUs software(s).
Source:
Verification Method: Analysis
Legacy ID: 53965v1

Before the bootloader does a call-out to the CompleteCompatibleFunction() it shall verify that the total Complete & Compatible block is present by reading the StartCompleteCompatibleSignature and EndCompleteCompatibleSignature. If the bootloader doesn't find the correct value of the StartCompleteCompatibleSignature or EndCompleteCompatibleSignature shall treat this as incomplete EXE file to further validation and set the return value to: (bit 0 and 2 = '1', remaining bits = '0')

662835 v2 **Maximum time to perform the Complete&Compatible validation**

Version history:
(2): CC校验的最大执行时间由20ms调整为200ms，匹配RID 0205的P4server_max时间。

Purpose: To define the maximum time to perform the Complete & Compatible validation of ECU software(s), to fulfill the ECU start-up timing requirement.
Source:
Verification Method: Test
Legacy ID: 53970v3

The maximum time to perform the total Complete & Compatible sequence described in *Figure - CompleteCompatibleFunction()* shall not exceed 200ms.

•Appendix

4.11.1.4.3 Appendix

4.11.1.4.3.1 Examples of different scenarios for the Complete & Compatible function

All bits are equal to '0'	The ECU contains complete and compatible software parts. The ECU can start with full normal operational functionality
Bit 6 to 31: is set to '1'	Failure within user defined software parts(s).
Bit 5: is set to '1'	Failure within the ECU Software Structure
Bit 4: is set to '1'	Failure within the Car Configuration (not applicable for every ECUs)
Bit 3: is set to '1'	Failure within the Signal Configuration
Bit 2: is set to '1'	Failure within the ExecutableSoftware
Bit 1: is set to '1'	The ECU contains incompatible software part(s)
Bit 0: is set to '1'	The ECU contains incomplete software part(s)

- If the Signal Configuration data file is created with same fix and target file as the application but with a different net file the returned value from the CompleteCompatibleFunction() shall be compatible.
(0b00000000000000000000000000000000)
- If the Signal Configuration data file is created with the same net and target file as the application but with a different fix file the returned value from the CompleteCompatibleFunction() shall be incompatible.
(0b00000000000000000000000000001010)
- If a data file is modified but still is supported by the application the returned value from the CompleteCompatibleFunction() shall be complete and compatible.
(0b00000000000000000000000000000000)
- If a data file is present (bit 6, user defined software) which makes the application incapable of responding on e.g. diagnostic requests the returned value from the CompleteCompatibleFunction() shall be incompatible
(0b00000000000000000000000001000010)
- If a data file (bit 6, user defined software) is missing or broken the returned value from the CompleteCompatibleFunction() shall be incomplete.
(0b00000000000000000000000001000001)
- If the EXE data file is broken or corrupt but the CompleteCompatibleFunction block is complete the returned value from the CompleteCompatibleFunction() shall be incomplete.
(0b00000000000000000000000000000101)
- If the Signal Configuration data file is missing or broken the returned value from the CompleteCompatibleFunction() shall be incomplete.
(0b00000000000000000000000000001001)
- If the bootloader doesn't find correct value or can't read the StartCompleteCompatibleSignature or EndCompleteCompatibleSignature the bootloader shall treat this as incomplete EXE file to further validations and return the value of
(0b00000000000000000000000000000101)

4.11.1.5 General Bootloader requirements for file system

•File Download and Installation Sequence Overview

4.11.1.5.1 File Download and Installation Sequence Overview

This section describes the overall File Download and Installation sequence of ECU as a reference for rapid develpment of upgrade requirements.
Note：There is no mandatory order for file download and digital envelope download, as long as the digital envelope matches the file.

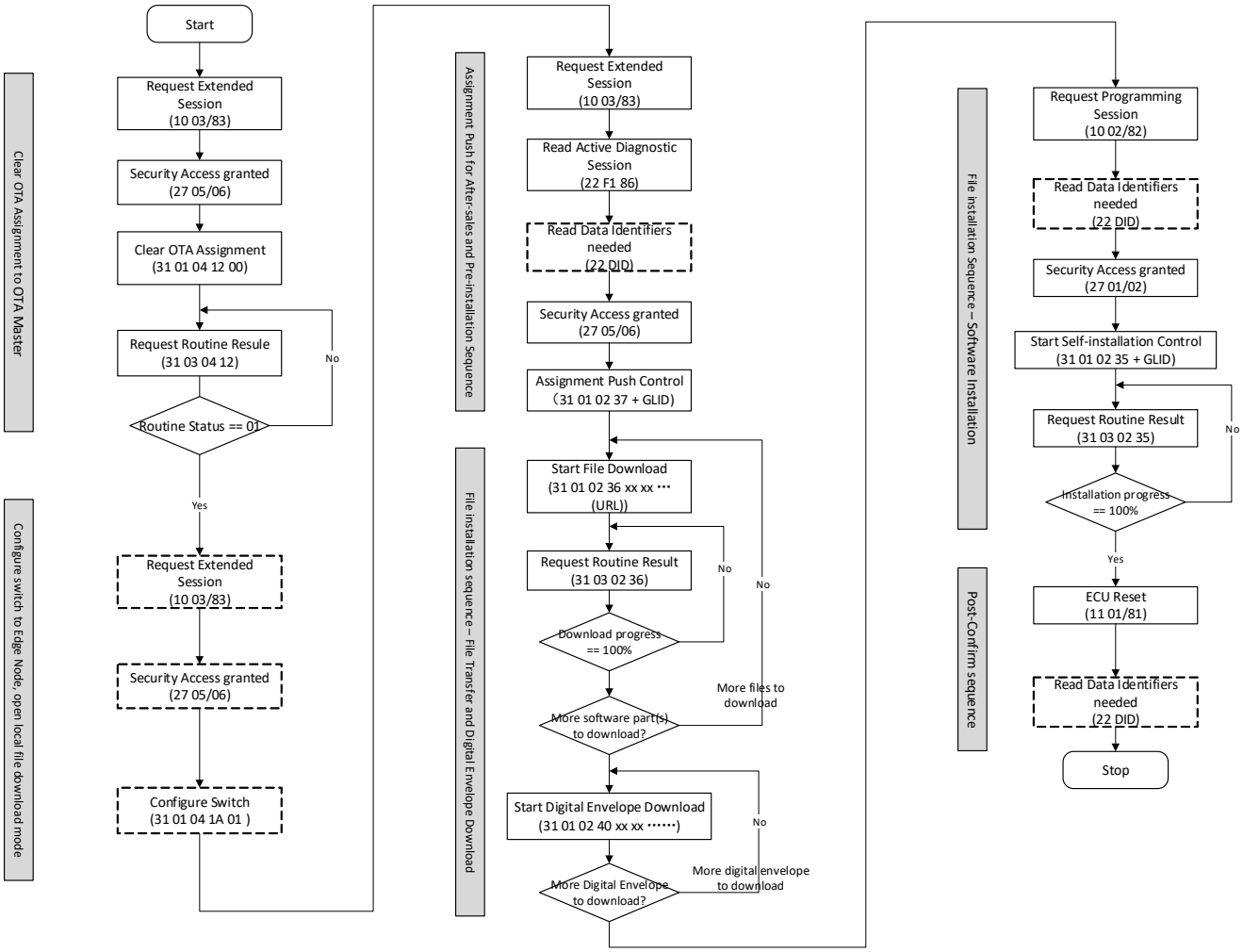


Figure - File Downlad and Installation Sequence Overview

•**Target**

4.11.1.5.2 Target

Allow any file system ECU to be installed at any assembly plant or dealership, via security data links.

The following requirement principles shall be met for software download to an ECU.

662859 v1	**Correct target software installation**

Version history:
(1):

Purpose: Ensure all target file could be installed accomplish.
Source:
Verification Method: Test
Legacy ID:

Once received an installation request, ECU shall install all target files in one installation operation.

666807 v3	**File transfer performance**

Version history:
(3):

Purpose: To decrease the whole update time, ECU shall focus its CPU resources on file transmission.
Source:
Verification Method: Analysis AND Test
Legacy ID:

To ensure a more efficient update process, ECU resources need to be allocated resonably. ECU shall focus its internal resources on software file transmission when after-sales assignment is activated.
The overall file transmission rate is depends on different types of telematic & connectivity modules. In terms of performence, it is expected to reach half of the hardware communication bandwidth.

662860 v1 **Target data file parsing method**

Version history:
(1):

Purpose: Define the parsing requirements for the software files.
Source:
Verification Method: Test
Legacy ID:

ECU shall parse VBF format file base on [SWDL_1]. It shall extract the content of “version section” “header section” “data section” so that installation processing implement correctly.

662858 v1 **The storage partition of software file**

Version history:
(1):

Purpose: Ensure all target file could be installed accomplish.
Source:
Verification Method: Analysis
Legacy ID:

All data files waiting to be installed shall be stored in a specific partition. This partition only store the data file which is released by OEM(i.e. VBF file).

•Data compression and encryption

4.11.1.5.3 Data compression and encryption

662861 v1 **The compression requirements for file system ECU**

Version history:
(1):

Purpose: To decrease the time to download data the ECU shall support decompression of the received compressed data.
Source:

Verification Method: Analysis
Legacy ID:

To decrease the file transmission time of an ECU, File compression shall be used (i.e. The input software file of VBF shall be compressed). Due to the higher computing power of ECU with operation system, the algorithm with higher compression should be considered.

Note: Software files are compressed before generating VBF files. Therefore, the software file cannot be compressed again during the VBF file generation.

•Assignment push Control

4.11.1.5.4 Assignment push Control

The section only describes the diagnostic control scheme and implementation of after-sales assignment push.

662863 v1 **After-sales assignment Identifier**

Version history:
(1):

Purpose: Define the after-sales assignment identifier.
Source:
Verification Method: Test
Legacy ID:

The GLID parameter shall be a specific string value to indicate an after-sales assignment. It is defined as the string ffffffff-1111-1111-1111-111111111111.

662862 v2 **After-sales assignment activation**

Version history:
(2):

Purpose: Define the activation method for after-sales assignment.
Source:
Verification Method: Analysis
Legacy ID:

After received the assignment push routine (0x0237, defined in ReqID 665400) with the GLID request parameter by file system ECUs, it shall attempt to activate after-sales assignment flow, and then ECUs could handle subsequent file transfer routine after the success.

662864 v1 **After-sales assignment handling**

Version history:
(1):

Purpose: Define how to handle the after-sales upgrade assignment.
Source:
Verification Method: Analysis AND Test
Legacy ID:

If condition is satisfying, the file ECUs shall always activate a new after-sales assignment whether or not an assignment already exists, i.e. OTA assignment, or the previous after-sales assignment.
When start a new after-sales assignment, the old one and its correspongding data files if have shall be aborted and deleted by file ECUs.

662867 v4 After-sales assignment lifetime

Version history:
(4): 大文件刷写一次任务的生命周期从35min调整为20min。

Purpose: Define a lifetime to prevent the file ECUs from staying in the after-sales assignment state for long time. It means that it is a temporary upgrade assignment in workshop.
Source:
Verification Method: Test
Legacy ID:

The lifetime of after-sales assignment shall be maximun 20 minutes in non-programming session, during every driving cycle, meaning that no considering this time in programming session.
The timer shall be restarted after processing Assignment push Control startRoutine or File Transfer Control startRoutine. If exceeded, the after-sales assignment and its corresponding data files shall be aborted and deleted by file ECUs.

662865 v1 After-sales assignment push - routine accomplish

Version history:
(1):

Purpose: Define the response after received the assignment push.
Source:
Verification Method: Test
Legacy ID:

After launching the after-sales assignment successfully, ECU shall response routine complete to external diagnostic client. And the one byte's AssignmentStatus response parameter shall present which assignment had existed ever or not.

Return value	Description
0x00	Empty assignment
0x01	OTA assignment (UUID) exist ever
0x02	After-sales assignment (GLID) exist ever

662866 v1 Integrity and authentication for data files

Version history:
(1):

Purpose: The external diagnostic tool cannot fully simulate the instruction of OTA Server. As an authorized device in workshop, it allows to simplify the process.
Source:
Verification Method: Analysis
Legacy ID:

File ECUs shall skip and ignore the integrity and authentication for data files getting from URL address during after-sales upgrade assignment.

662868 v1 **The end of After-sales assignment**

Version history:
(1):

Purpose: Define the end of after-sales assignment.
Source:
Verification Method: Test
Legacy ID:

ECU shall be allowed to end the after-sales assignment and clear its corresponding data information, i.e. file data, after resetting power supplier and re-entering application, or a successful upgrade.

•File transfer control

4.11.1.5.5 File transfer control

The section only describle the diagnostic control scheme of file transmission(target file download).

After the ECU obtains the URL address, it needs to access the backend server to get to software files by HTTPS protocol, which is same way as downloading software files during the OTA upgrade. This is out of the scope of this document, and please refer to OTA function requirement specification for details.

662869 v3 **File transfer**

Version history:
(3): 增加有线网络本地售后大文件的传输场景。

Purpose: Define the activation method for file transmission
Source:
Verification Method: Test
Legacy ID:

Only during after-sales assignment, file system ECUs shall receive and process the file transfer start routine (RID 0x0236), then it shall attempt to access target server address and get target file via the vehicle wireless network(Programming software stored in the background) or wired network(Programming software is stored on the local diagnostic tester,implement by changing the Switch configuration of edge nodes through diagnostic routine (RID: 041A)). In this process, ECU shall return the progress via diagnostic response until the whole routine process accomplish.

If ECU does not execute this routine due to some reasons, it should response routine aborted to external diagnostic client.

662870 v1 **File transfer - URL Information**

Version history:
(1):

Purpose: Define an identifier for each URL.
Source:
Verification Method: Test
Legacy ID:

A file transfer control routine carries only one URL address information and uses fileRecordNumber parameter to uniquely identify the file downloaded from this URL address.

Note: the fileRecordNumber parameter can correspond to files downloaded from different URL addresses, and a file downloaded from a URL only corresponds to one fileRecordNumber parameter.

662874 v1 **File transfer control - Failure handling**

Version history:
(1):

Purpose: Define the handling of data files in the case of exceptions.
Source:
Verification Method: Analysis AND Test
Legacy ID:

File system ECUs shall discard this unaccomplished data file if abort a routine during its execution.

662872 v2 **File transfer routine - Download abort**

Version history:
(2): Supplement the description.

Purpose: Define the response during file transfer.
Source:
Verification Method: Test
Legacy ID:

When file transfer routine is aborted, The one byte's RoutineStatusRecord shall present failure reason. The return value of download progress will fix at the routine abort point.

Note：The value of download progress is 0, If routine abort is earlier than download starting.

Return value	Description
0x00	No Error
0x01	Unknown Error
0x02	Bad Request, i.e. HTTP error code 400
0x03	Unauthorized, i.e. HTTP error code 401
0x04	Forbidden, i.e. HTTP error code 403
0x05	Resource Not Found, i.e. HTTP error code 404

0x06	Method Not Allowed, i.e. HTTP error code 405
0x07	Request Timeout, i.e. HTTP error code 408
0x08	Failed Dependncy, i.e. HTTP error code 424
0x09	Internal Server Error, i.e. HTTP error code 500
0x0A	Service Unavailable, i.e. HTTP error code 503
0x0B	Internal Storage Error, e.g. Insufficient storage space found
0x0C	A Valid Package Already Exists, e.g. Repeated download
0x0D	File Transfer Is Incomplete
0x0E	File Verification Error, i.e. Signature verification failed
0x0F	Routine max run timeout, i.e. Software transfer time timeout
0x10	No After-sales assignment
0x11~0xFF	User defined

662873 v2 **File transfer routine - Download progress**

Version history:
(2):

Purpose: Define the response during file transfer.
Source:
Verification Method: Test
Legacy ID:

ECU shall response download progress during file transfer. The data format are defined in Global Master Reference Database.

It is expected that ECU can sample the download progress every 1s or faster internally.

662871 v1 **File transfer routine- Download accomplish**

Version history:
(1):

Purpose: Define the response during file transfer.
Source:
Verification Method: Test
Legacy ID:

After all files are downloaded successfully from server, ECU shall response routine complete to external diagnostic client.

•Digital Envelope Transfer

4.11.1.5.6 Digital Envelope Transfer

662879 v1 **Clearing of Digital Envelope**

Version history:
(1):

Purpose: Define when to clear the Digital Envelope.
Source:
Verification Method: Analysis

Legacy ID:

At the end of the after-sales assignment or after software installation regardless of success or failure, the digital envelope information and the decrypted password shall be cleared.

662875 v2 Digital Envelope Transfer

Version history:
(2):

Purpose: Define how the digital envelope is transferred and how it matches the downloaded files.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Digital envelope transfer routine (0x0240, defined in ReqID 665401) provides the digital envelope information which matching the existing file record number through its parameter DEtoFileRecordeNumber. The data format is defined in Global Master Reference Database.

662877 v1 Digital Envelope Transfer routine - Transfer abort

Version history:
(1):

Purpose: Define the response during digital envelope transfer.
Source:
Verification Method: Test
Legacy ID:

When digital envelope transfer routine is aborted, the one byte's RoutineStatusRecord shall present failure reason.

Return value	Description
0x00	No Error
0x01	No corresponding FileRecordNumber was found.
0x02	Digital Envelope decryption failure.

662876 v1 Digital Envelope Transfer routine - Transfer accomplish

Version history:
(1):

Purpose: Define the response during digital envelope transfer.
Source:
Verification Method: Test
Legacy ID:

After digital envelope transferred successfully, ECU shall response routine complete to external diagnostic client.

662880 v2 Digital Envelope to FileRecordNumber

Version history:
(2):

Purpose: Define the handling of record number.
Source:
Verification Method: Analysis AND Test
Legacy ID:

There may be such a table in the memory of file ECUs to map the download file(s) and digital envelope through an agreed record number.

When the record number is 0xFF, it indicates that the digital envelope is applied to all download files during this after-sales assignment.

Note: The digital envelope with the latest record number shall be always used.

Example 1, different Digital Envelops match different FileURLs (VBF files) throuth DEtoFileRecordNumber and FileRecordNumber.

FileRecordNumber (hex)	FileURLAddress	DEtoFileRecordNumber (hex)	Digital Envelope
01	FileURL #1	01	Digital Envelope #1
02	FileURL #2	02	Digital Envelope #2
03	FileURL #3	03	Digital Envelope #3
04	FileURL #4	04	Digital Envelope #4
05	FileURL #5	05	Digital Envelope #5

Example 2, One Digital Envelop matches different FileURL (VBF file) throuth DEtoFileRecordNumber and FileRecordNumber.

FileRecordNumber (hex)	FileURLAddress	DEtoFileRecordNumber (hex)	Digital Envelope
01	FileURL #1	01	Digital Envelope #1
01	FileURL #2	01	Digital Envelope #1
01	FileURL #3	01	Digital Envelope #1
01	FileURL #4	01	Digital Envelope #1
01	FileURL #5	01	Digital Envelope #1

Example 3, One Digital Envelop matches different FileURL (VBF file) throuth DEtoFileRecordNumber (0xFF) and FileRecordNumber.

FileRecordNumber (hex)	FileURLAddress	DEtoFileRecordNumber (hex)	Digital Envelope
01	FileURL #1	FF	Digital Envelope #1
02	FileURL #2	FF	Digital Envelope #1
03	FileURL #3	FF	Digital Envelope #1
04	FileURL #4	FF	Digital Envelope #1
05	FileURL #5	FF	Digital Envelope #1

662878 v1 Use of Digital Envelope

Version history:
(1):

Purpose: Defines how to use digital envelopes.

Source:
Verification Method: Analysis
Legacy ID:

The file system ECU should decrypt the received digital envelope with the private key and extract the decryption password corresponding to the downloaded file. Then ECUs should decrypt the downloaded file with the decryption password during installation.

•ECU program mode

4.11.1.5.7 ECU program mode

Program mode means that the ECU executing file installation mode. This section describes the communicaton and diagnostic action in this mode.

662881 v1 Reliable installation capability

Version history:
(1):

Purpose: Improve the stability of software upgrades, and ensure that the ECU can be update continuously.
Source:
Verification Method: Test
Legacy ID:

The file transfer process is performed in application mode. To enable the ECU to retain its functionality after installation failed, the dual memory scheme shall be supported. if the installation process successed, the new program will be loaded in next startup. Otherwise, ECU still runs the last valid program.

•General file installation order

4.11.1.5.8 General file installation order

662882 v1 General file installation order

Version history:
(1):

Purpose: Define basic installation sequence of software files
Source:
Verification Method: Analysis
Legacy ID:

Each data file act as independent logical block, It shall support installation independent.
Installation process shall include three phase at least.
- software signature verification
- software complete and compatible
- software installation

After all target files passed software signature verification and complete&compatible. ECU could start software installation progress.

•Softeware authentication

4.11.1.5.9 Softeware authentication

4.11.1.5.9.1 Software authentication for file system ECU- introduction

The purpose of Software Authentication is described in *[General Software Authentication]*, i.e. to verify that the installed software is authentic (originates from a legitimate source and has not been altered). To perform the authenticity verification, using digital signatures, the ECU must (at least) know:
• The data that must be checked for authenticity. Verification Block Tables are used *[General Software Authentication]*. The verification block table (one per software part) contains the placement and hash value for the expected programmed data blocks.
• How to get the corresponding digital signature.
• Conditions for starting the verification and what is required for the verification to pass or fail respectively. If a complete verification is not performed at every application start, information about the verification status (validity status information) must be stored.
• How many software parts (and their location) that must be successfully verified in order to start the application: ECU Software Structure and Logical Blocks are introduced.

The ECU authenticity verification (for a single software part, i.e. a logical block) is performed according to *Figure - Authenticity verification principle*:
(1) - The data blocks of the software part is programmed to the ECU. The corresponding digital signature located in the VBF header section. It will initiate the verification when receive file installation order.
(2) - The signed verification block table is verified. The placement of the verification block is ensured as its start address and length parameters are part of the signature generation, i.e. the length (unprocessed) must be calculated by the ECU (the start address is known for the updated logical block).
(3) - The programmed data blocks, according to the authenticated verification block, are then verified. The placement of all ingoing data blocks are ensured as their start address and lengths together with the corresponding hash values are part of the verification block table.

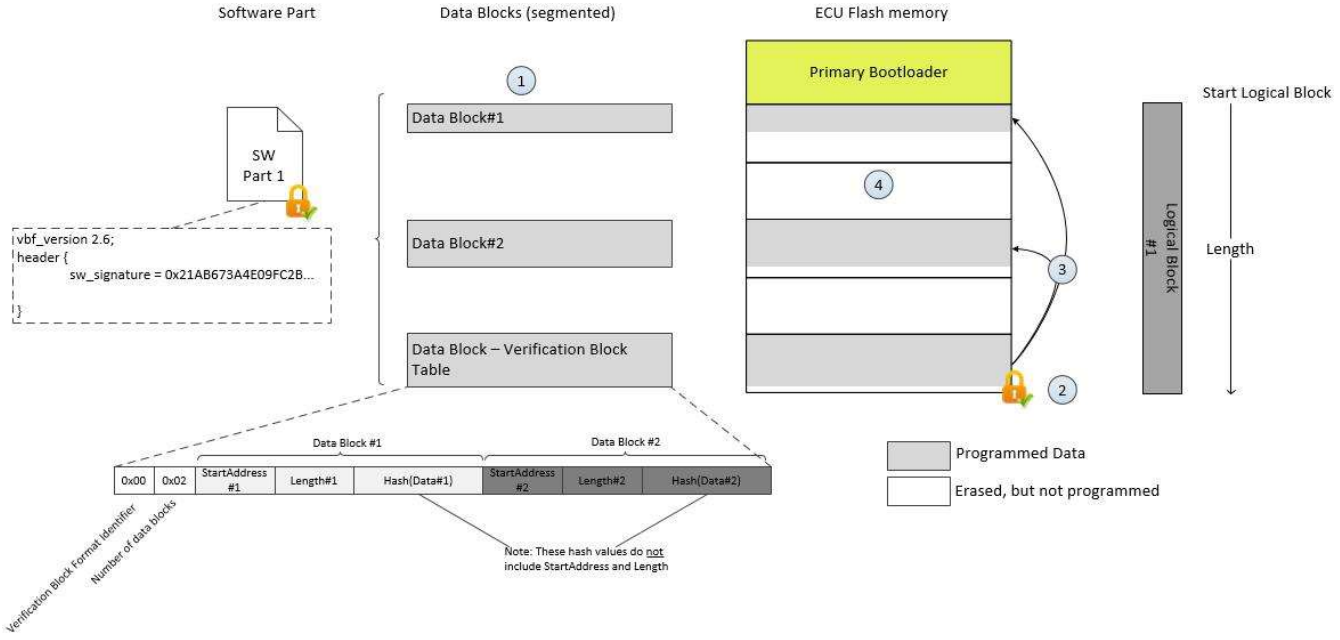


Figure - Authenticity verification principle

The *Figure - Authenticity verification principle* does not show all details of the software preparation (off-board) nor the signing. The purpose is to introduce the reader of how the verification is performed.

At software **installation**, the ECU must:

- Mark the uninstalled files as invalid (i.e. it has not passed the signature verification), prior to any data erase/compress/copy operations.
- Check at every uninstalled file that the logical block is marked as invalid.
- At file installation process, verify according to (1) to (3) in *Figure - Authenticity verification principle*.
- Only upon a successful authenticity verification, set the validity status for the verified logical block to valid.

4.11.1.5.9.2 Verification Block Table

A *Verification Block Table* is introduced, representing the expected segmented data blocks (including address, length and hash values per data block) of a complete software part. By that, the signature could be created on the verification block table. When that block is verified correctly at the ECU, the ECU could continue to verify also the other data blocks that has been programmed. Hence, the security principle of "chain-of-trust" is applied. Note that the verification block table is downloaded as any other data block, i.e. the verification block is a separate data block with defined properties.

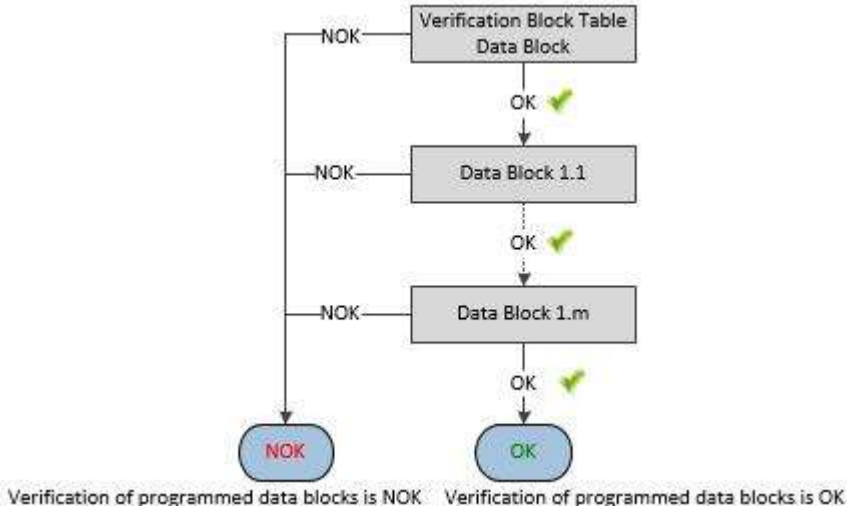


Figure - Verification of programmed data blocks

The verification block table must be put in a separate data block (i.e. downloaded as a separate data block to the ECU) and it shall contain:

- Verification Block Table format identifier, e.g. a version number if the format of the table is changed in the future
- Number of data blocks (the verification block table itself is excluded)
- StartAddress data block 1, length data block 1, hash value of data block 1
- ...
- StartAddress data block m, length data block m, hash value of data block m

Note that there is one verification block table per every software part.

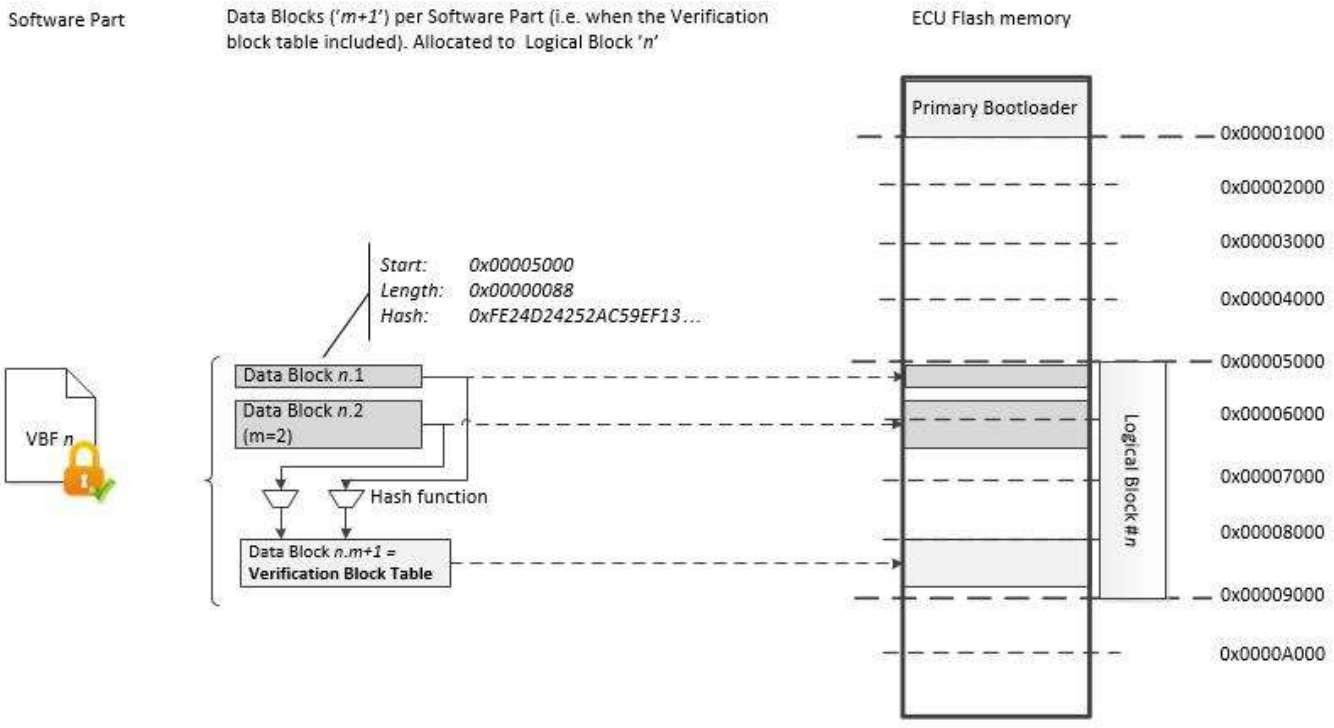


Figure - Verification Block Table overview

In *Figure - Verification Block Table overview* it is shown how the verification block table is constructed. The hash

values are calculated on the data parts only, i.e. the start address and length is not used as input to the hash function. The placement of each data block is ensured as the start address and length are also defined in the table. The format of the table is defined in *[General Software Authentication].*

662802 v1 **Allocation of the Verification Block Table**

Version history:
(1):

Purpose: The position of the verification block table must be known by the bootloader in order to perform the verification. The verification block must not be part of any other data block, as the verification block table contains hash values of all other data blocks, i.e. it cannot contain the hash of itself.
Source:
Verification Method: Analysis AND Test
Legacy ID: 398372v2

A verification block table as defined Req ID1600: *Format of the Verification Block* Table shall be stored in a separate data block. The start address shall be defined by the logical block definition, i.e. by the ECU Software Structure. The correct location of the verification block is ensured by including its address information (start and length) when it is being signed. The length (unprocessed) must be calculated by the ECU when the data is programmed.

Design consideration: When delta encoding is applied, the length of the Verification Block Table could be derived by using the start address and Number of data blocks.

662803 v1 **Verification Block Table - Reserve space at application build process**

Version history:
(1):

Purpose: The verification block table is part of the post processing sequence, meaning that it is generated after the actual software is build.
Source:
Verification Method: Analysis AND Test
Legacy ID: 402292v1

The application software build process must reserve memory space for the verification block table within a logical block. It is the software supplier that creates the block table and includes it in the software part.

4.11.1.5.9.3 Validation of ECU software(s) - Authenticity Verification

4.11.1.5.9.3.1 Validation of Logical Blocks

662883 v1 **Condition for a logical block to be valid**

Version history:
(1):

Purpose: A logical block must be verified successfully with respect to its authenticity every time it has been altered, in order to be considered as valid.
Source:
Verification Method: Test
Legacy ID:

If signature verification have passed, the corresponding file shall be marked “valid” status. The

detail step refer Figure-Authentication verification function for file system.

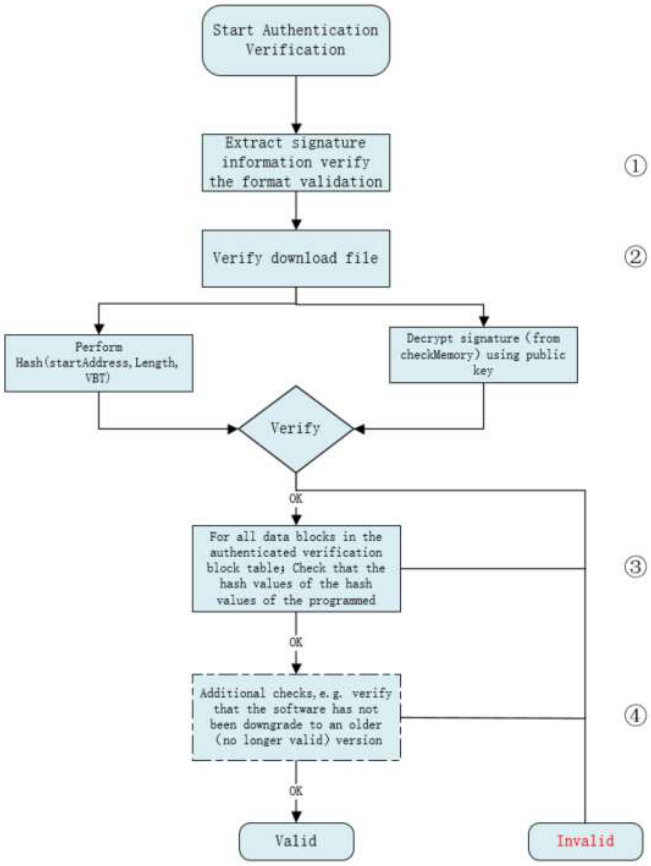


Figure - Authenticity Verification function for file system

1 - Applicable only if the signature are applied, The detail description about signature see Req 3499 and Req 3500.
1 - Signature verification. Note that the start address and length are used as input to ensure the placement, i.e. the length must for some use cases be calculated (delta encoding). The format and length of the verification block table shall be verified. Prior to the signature verification is performed, the integrity of the public key shall be ensured.
1 - Verify that the hash value (per data block) is identical to the definition in the verification block table.
1 - Additional checks, e.g. if the ECU has support to prevent downgrade, meaning that an older (no longer valid) software part must not be installed as it might include some vulnerabilities. GRI Basetech Team to be consulted if such protection shall be applied (processes must be defined at off-board to support such a use case).

When the authenticity verification function is completed, it must be ensured that the verification status (validity status information) is stored correctly. Ensure ECU could implement software installation.

662884 v1 **Conditions for a logical block to be invalid**

Version history:

(1):

Purpose: A logical block must be verified with respect to its authenticity every time it is modified. Any condition that triggers a change of memory (erase/write) of a logical block shall result in that the logical block is set to (or kept) invalid.
Source:
Verification Method: Analysis
Legacy ID:

When the ECU finish the download process. it will identify all logical block to “Not validation” status.

4.11.1.5.9.3.2 Public Key Storage

See *[General Software Authentication]* for requirements about public key programming during test and production.

662807 v1 **Public Key storage**

Version history:
(1):

Purpose: The trust anchor shall not be possible to manipulate, that could be used to bypass the protection.
Source:
Verification Method: Analysis AND Test
Legacy ID: 366329v2

The trust anchor, i.e. the public key, shall be stored in a protected (read-only) area, typically in a hardware secure module or a protected boot area. It shall be one-time-programmable.

662808 v1 **Re-programmable initial development key**

Version history:
(1):

Purpose: Define key management with respect to the test, development and release (production).
Source:
Verification Method: Analysis AND Test
Legacy ID: 429095v1

The ECU shall use an initial public key for development purposes. It is stored in the ECU by the ECU supplier (e.g. "virgin" ECU). When the initial development key is stored in the ECU, the public key for production shall always be possible to program. When a valid public key has been programmed, the initial development key must no longer be used by the ECU. The development key defined in *[General Software Authentication]* shall be applied unless else is explicitly specified.

4.11.1.5.9.3.3 Softeware Authentication for file system ECU

662885 v1 **Number of attempts to verify file signatures**

Version history:
(1):

Purpose: Define the number of signature verification attempt

Source:
Verification Method: Test
Legacy ID:

The ECU determines the number of signature verification attempts by extracting VBF file signature information. If ECU only search one signature in VBF header section (i.e. “sw_signature_dev”), it is only allowed to implement signature verification one time; if two signature is extracted in VBF header section (i.e.“sw_signature_dev” and “sw_signature”), it could implement signature verification twice in a row. (i.e. when signature verification is failed by “sw_signature_dev”, it could be attempted the second time by “sw_signature”). As soon as one of signature verification passed, the file status will be “valid” status.

•Validation of ECU software(s) - Complete and Compatible

4.11.1.5.10 Validation of ECU software(s) - Complete and Compatible

The purpose of the "Complete & Compatible Check" is to ensure that the expected software parts are installed successfully and are valid (complete), e.g. to detect if any part of the software is missing, broken (partly present/downloaded) or has been altered. This check shall also decide whether the combination of stored software parts are compatible to each other. The compatible check can be done in many different ways and will be different from case to case. It is either exclusively handled by the bootloader or by using a call-out to a function block that is part of the application. The compatibility check is only performed once the complete check has passed successfully.

The Complete & Compatible check shall be implemented in such a way that it must not be updated for each new or modified data file.

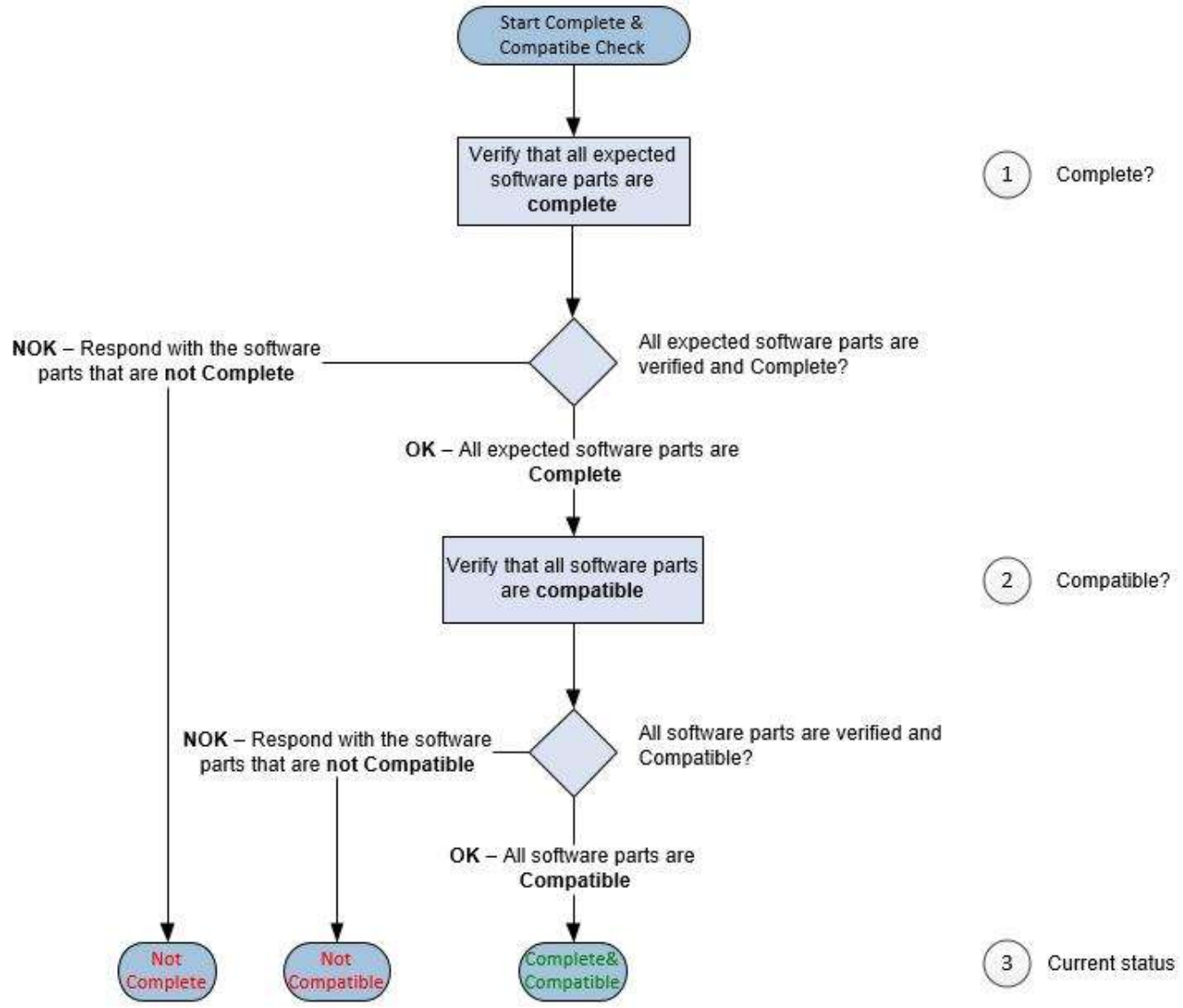


Figure - CompleteCompatibleFunction()

The check shown in *Figure - CompleteCompatibleFunction()* will be referred to as the **CompleteCompatibleFunction()**:

1. Check that software parts are complete.
2. Check that the software parts are compatible. The compatibility check shall only be performed if all expected software parts are complete.
3. The final result of the Complete & Compatible check, including the individual status per software part. The application must not be started unless both the Complete and Compatible check are passed successfully.

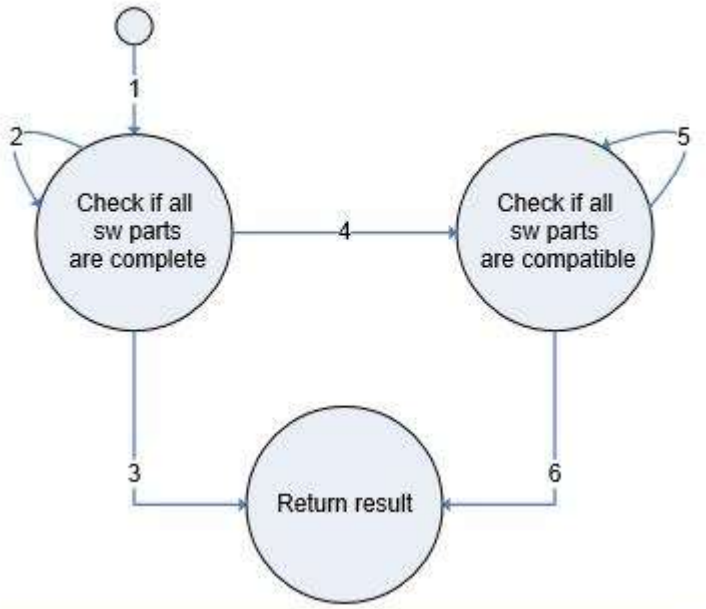
662828 v1 **Complete and Compatible sequence diagram**

Version history:
(1):

Purpose: Definition in which order the ECU shall validate the stored software(s).
Source:
Verification Method: Test

Legacy ID: 53959v4

The ECU shall first check that all software parts are complete prior to the compatibility check is performed. If the ECU contains incomplete software(s) it shall not check whether the software(s) is compatible. Hence, the ECU cannot indicate that both the complete and compatible check has failed. The software parts shall be checked in the order defined in *Figure - CompleteCompatibleFunction() sequence diagram.*



Step	Description
1	The bootloader starts the Complete & Compatible function
2	The Complete & Compatible function checks if stored SW parts(s) is **complete** in the following order: (1) ESS (2) EXE (3) Signal Configuration (4) Car Config (if applicable) (5...) Used defined SW parts(s)
3	Return result. If any SW part(s) is incomplete the Complete & Compatible function shall indicate that in the response
4	If all SW part(s) are complete
5	The Complete & Compatible function checks if stored SW parts(s) is **compatible,** using the same order as defined in step 2 but ESS is optional.
6	Return result

Figure - CompleteCompatibleFunction() sequence diagram

662827 v1 **Validation of ECU Software(s)**

Version history:
(1):

Purpose: To prevent start of incomplete or incompatible software(s).
Source:
Verification Method: Test
Legacy ID: 53935v1

The bootloader shall at every start check if the ECU software(s) is complete and compatible before starting execution of the application.

•File installation

4.11.1.5.11 File installation

662886 v1 **File installation**

Version history:
(1):

Purpose: Define the activation method for file installation.
Source:
Verification Method: Analysis AND Test
Legacy ID:

File system ECUs shall only perform software installation which matches the assignment ID, i.e. defined in "*Req ID 642642* After-sales assignment activation".

Because the operation system of each ECU is different, the installation scheme is determined by the developer.

662887 v1 **File installation routine- installation accomplish**

Version history:
(1):

Purpose: Define the response during file installation.
Source:
Verification Method: Test
Legacy ID:

After all files are installed successfully, ECU shall response routine complete to external diagnostic client.

662888 v1 **File installation-installation abort**

Version history:
(1):

Purpose: Define the response during file installation.
Source:
Verification Method: Test
Legacy ID:

When Installation routine process aborted, The one byte's RoutineStatusRecord would present failure reason. The return value of installation progress will fix at the routine abort point.
Note：The value of download progress is 0, If routine abort is earlier than download starting.

Return value	Description

0x00	No Error
0x01	Unknown Error
0x02	The signed data could not be authenticated, i.e. signature verification failed
0x03	The address or hash values of the downloaded data blocks (VBT) does not match the expected values, software integrity check failed.
0x04	Invalid format or length of the Verification Block Table
0x05	Cannot extract signature information in software package
0x06	VBF format error
0x07	File incomplete, cannot be decompressed
0x08	decompression process failure
0x09	The storage space is insufficient and cannot be decompressed.
0x0A	File incompatibility
0x0B	No authorization assignment
0x0C	File decryption failure
0x0D~0xFF	User defined

662889 v1 **File installation-installation progress**

Version history:
(1):

Purpose: Define the response during file installation.
Source:
Verification Method: Test
Legacy ID:

ECU shall response installation progress during software installation. The data format are defined in Global Master Reference Database.

•Download file from local tester

4.11.1.5.12 Download file from local tester

975940 v1 **Downloader requirement**

Version history:
(1):

Purpose: Define the downloader additional requirement in the process.
Source:
Verification Method: Analysis AND Test
Legacy ID:
Zeekr Requirement ID: 819787

After receiving after-sale GLID (ffffffff-2222-2222-2222-222222222222) and File transfer routine(URL address), downloader shall prepare the http client accordingly.

1. The http client shall get http server IP address and port number through URL address in File transfer routine, and then connent to server in the URL address to download file. DNS is not

required in the process.
2. The http client shall work in VLAN 201 and use its IP address in VLAN201. The http client port number shall be dynamically allocated.
3. The http client shall support dynamic ARP protocol and ICMP protocol .

975939 v2 **Edge node requirement**

Version history:
(2):

Purpose: Define the edge node additional requirement in the process.
Source:
Verification Method: Analysis AND Test
Legacy ID:
Zeekr Requirement ID: 819783

After receiving Config Switch Request (31 01 DC 00 01), edge node shall change switch configuration to add VLAN 201 route to downloaders.

Adding the route includes:
1. Add port which connect to tester to VLAN 201.
2. Switch shall support broadcast in VLAN 201.

After receiving Config Switch Request (31 01 DC 00 02) , edge node shall change switch configuration to delete VLAN 201 route to downloaders.

Deleting the route means:
1. Cancel all the rules added above, and let switch back to normal.

975937 v1 **Roles in the process of downloading file from tester**

Version history:
(1):

Purpose: To give a general introduction to roles in downloading file from tester.
Source:
Verification Method: Analysis
Legacy ID:
Zeekr Requirement ID: 819753

In the process of downloading file from tester, there are three roles.

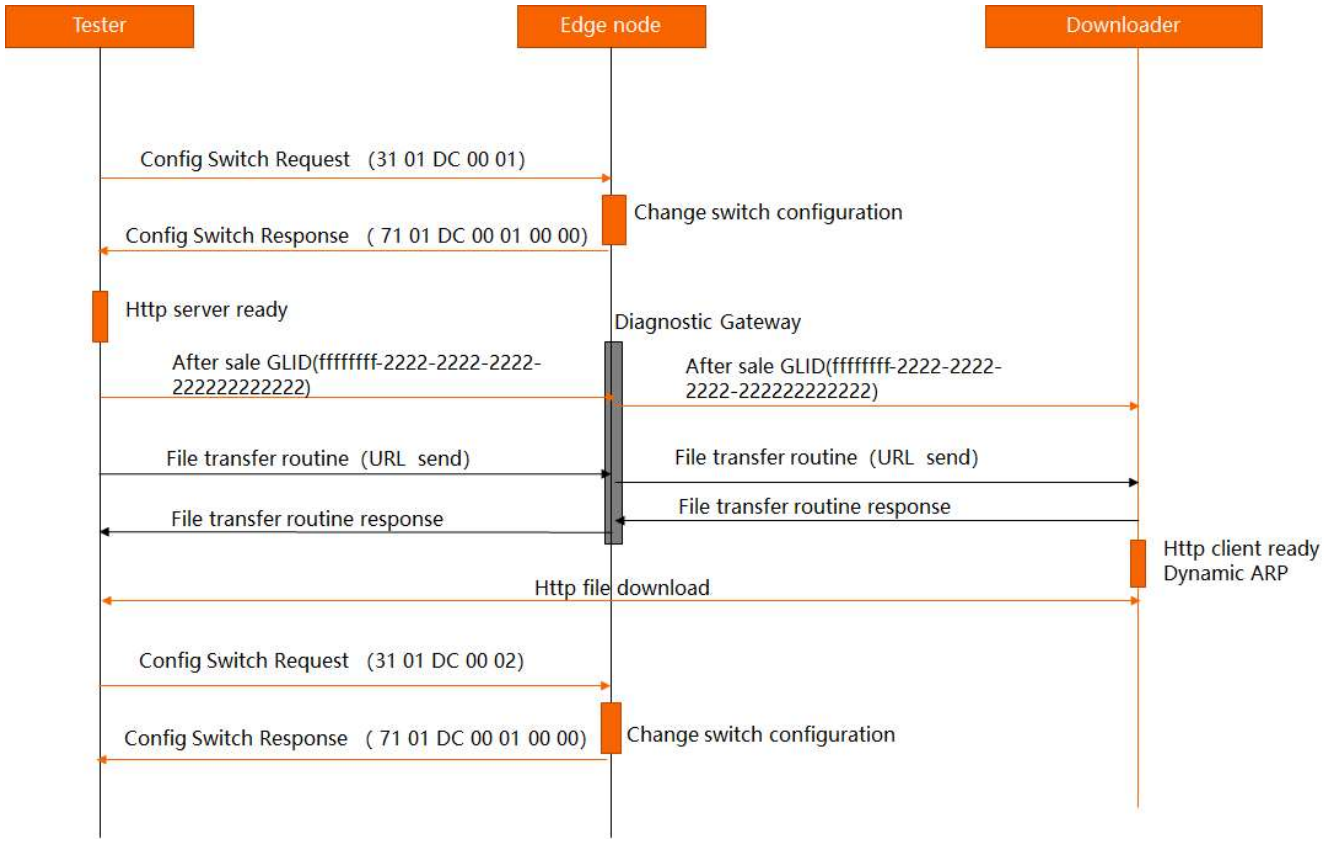
The first is tester,which is the file source.

The second is DoIP edge node, which receive diagnostic command and dynamically change

switch configuration to open or close a channel for downloading flie. In ZEEA3.0 platform, this edge node is CSCBCMCore. The edge node switch is VESA.

The last is downloader, which is the file sink. In ZEEA3.0 platform, this sink may be CSCACore, UXC or ADCU30.

The general additional interaction among the three roles in the process is shown below, more detailed example refer to the *Appendix* chapter.



975938 v1 **Tester role requirement**

Version history:
(1):

Purpose: Define the tester additional requirement in the process.
Source:
Verification Method: Analysis AND Test
Legacy ID:
Zeekr Requirement ID: 819777

Tester shall act as a http server for other ECUs to download file.

The http server shall be ready from power on.

The http server shall work in VLAN 201 and shall support send and receive tagged frame in VLAN 201. MAC address is not specified, which means MAC address is left to implement. The tester shall support dynamic ARP protocol and ICMP protocol .

Before downloading, tester shall send them to downloader in URL address through File transfer routine, such asThe http server IP address and Port number are not specified and are left to implement.
http://10.201.0.99:80/50d09bfe721a7e3a0ffd7a05a081aedd/8892679392A.vbf

•Appendix

4.11.1.5.13 Appendix

4.11.1.5.13.1 Examples of message flow

There is an example of message flow that simply describes the interaction process and considers that all request/response can be received and processed. The specific implementation details are not defined here.

Phase	Step	Role	Role	Data (hex)	Comments
Clear OTA assignment	(1)	Client	OTA Master	10 03	
		OTA Master	Client	50 03 00 32 01 F4	
	(2)	Client	OTA Master	27 05	
		OTA Master	Client	67 05 + Seed	
		Client	OTA Master	27 06 + Valid Key	
		OTA Master	Client	67 06	
	(3)	Client	OTA Master	31 01 04 12 00	Clear OTA assignment
		OTA Master	Client	71 01 04 12 20	
		Client	OTA Master	31 03 04 12	
		OTA Master	Client	71 03 04 12 20 01	Routine Status：Success
Configure switch, open local file download mode (Optional)	(1)	Client	Edge Node	10 03	For scenarios where programming software is stored on the local diagnostic tester and the file system ECU downloads software from the local diagnostic tester, the switch needs to be configured in advance. For other scenarios, this step needs to be skipped.
		Edge Node	Client	50 03 00 32 01 F4	
	(2)	Client	Edge Node	27 05	
		Edge Node	Client	67 05 + Seed	
		Client	Edge	27 06 + Valid Key	

			Node		
		Edge Node	Client	67 06	
	(3)	Client	Edge Node	31 01 04 1A 01	
		Edge Node	Client	71 01 04 1A 10 00 01	
Download process	(1)	Client	Server	10 03	
		Server	Client	50 03 00 32 01 F4	
	(2)	Client	Server	27 05	
		Server	Client	67 05 + Seed	
		Client	Server	27 06 + Valid Key	
		Server	Client	67 06	
	(3)	Client	Server	31 01 02 37 [66 66 66 66 66 66 66 66 2d 31 31 31 31 2d 31 31 31 31 2d 31 31 31 31 31 31 31 31 31 31 31]	Initial after-sales assignment with GLID.
		Server	Client	71 01 02 37 10 00	
	(4)	Client	Server	31 01 02 36 [03] [00 4D] [68 74 74 70 73 3a 2f 2f 6f 74 61 73 65 61 2d 63 64 6e 2e 67 65 65 6c 79 2e 63 6f 6d 2f 35 30 64 30 39 62 66 65 37 32 31 61 37 65 33 61 30 66 66 64 37 61 30 35 61 30 38 31 61 65 64 64 2f 38 38 39 32 36 37 39 33 39 32 41 2e 76 62 66]	fileRecordNumber: 0x03; URLLength: 0x004D; URLAddress: https://otasea-cdn.geely.com/50d09bfe721a7e3a0ffd7a05a081aedd/8892679392A.vbf
		Server	Client	71 01 02 36 22	
		Client	Server	31 03 02 36	
		Server	Client	71 03 02 36 22 [00] [xx]	AbortReason: 0x00 (No error); Download progress: phy=xx*1%<100%;
		Client	Server	31 03 02 36	
		Server	Client	71 03 02 36 20 [00] [64]	Download progress: phy=0x64*1% =100%
	(5)	Client	Server	31 01 02 40 [03] [01] [20] [yy (32 bytes)]	Digital Envelope with DEtoFileRecordNumber (0x03) is mapped to the file with fileRecordNumber (0x03) downloaded from step (4).
		Server	Client	71 01 02 40 10 [00]	
Installation process	(1)	Client	Server	10 82	
		Client	Server	10 02	
		Server	Client	50 02 00 19 01 F4	
	(2)	Client	Server	27 01	
		Server	Client	67 01 + Seed	
		Client	Server	27 01 + Valid Key	
		Server	Client	67 01	
	(3)	Client	Server	31 01 02 35 [66 66 66 66 66 66 66 66 2d 31 31 31 31 2d 31 31 31 31 2d 31 31 31 31 31 31 31 31 31 31 31]	Initial installation with GLID.
		Server	Client	71 01 02 35 22	
		Client	Server	31 03 02 35	

		Server	Client	71 03 02 35 22 [xx] [00] [zz (49 bytes)]	phy=xx*1%<100%; AbortReason: 0x00 (No error); zz = string text;
		Client	Server	31 03 02 35	
		Server	Client	71 03 02 35 20 [64] [00] [zz (49 bytes)]	Installation progress: phy=0x64*1%=100%;
	(4)	Client	Server	11 81	
Configure switch, close local file download mode	(1)	Client	Edge Node	10 03	For scenarios where programming software is stored on the local diagnostic tester and the file system ECU downloads software from the local diagnostic tester, the switch needs to be configured. For other scenarios, this step needs to be skipped.
		Edge Node	Client	50 03 00 32 01 F4	
	(2)	Client	Edge Node	27 05	
		Edge Node	Client	67 05 + Seed	
		Client	Edge Node	27 06 + Valid Key	
		Edge Node	Client	67 06	
	(3)	Client	Edge Node	31 01 04 1A 02	
		Edge Node	Client	71 01 04 1A 10 00 02	

4.11.1.5.13.2 Examples of different Compatible functions

4.11.1.5.13.2.1 Bootloader Compatbile check

If the bootloader is performing the compatibility check itself, it could (for example) using some patterns where the locations are defined by the ESS (Compatibility Information).

Example.

- Each logical block contains compatibility Information in the "header/info" part, according to the TIER 1 defined format. This header part must support functional growth, i.e. that the software structure is changed from e.g. three to four logical blocks.
- The dependencies to the application is checked, for each software part.

4.11.1.5.13.2.2 Application Complete & Compatible block

To support use-cases where the compatibility check is highly dependent to the application build process, i.e. where the bootloader does not have access to all required information, a possible realization is put the compatibility check into the application (EXE) and let the bootloader to a call-out to that block. This might be required when the ECU shall support the AUTOSAR post-build loadable concept, where it shall be possible to reconfigure the communication matrix (typically move an existing signal from one frame to another, add a signal gateway, etc.)

Example.
A *Application Complete & Compatible block* is defined and part of the EXE software part. The structure and location of this block is known to the bootloader.

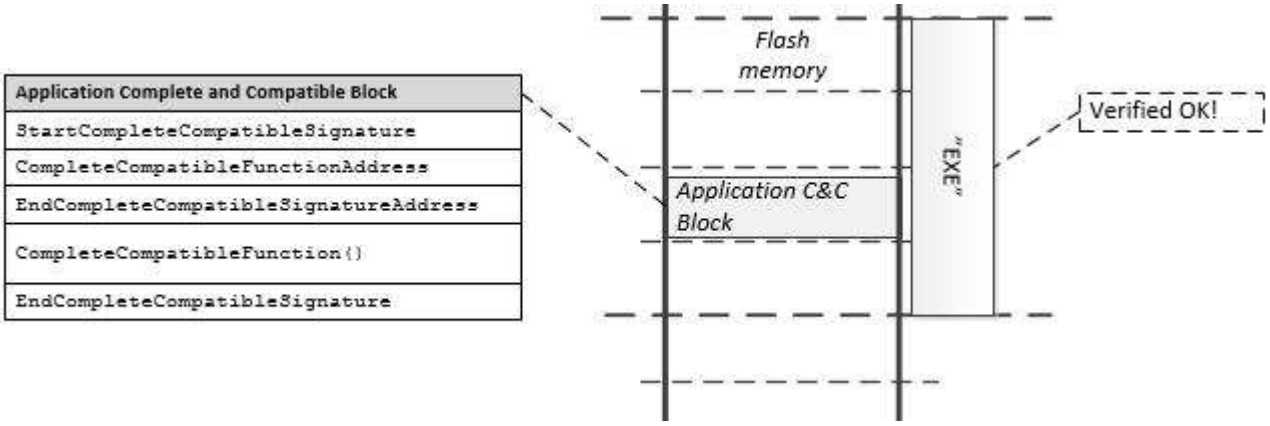
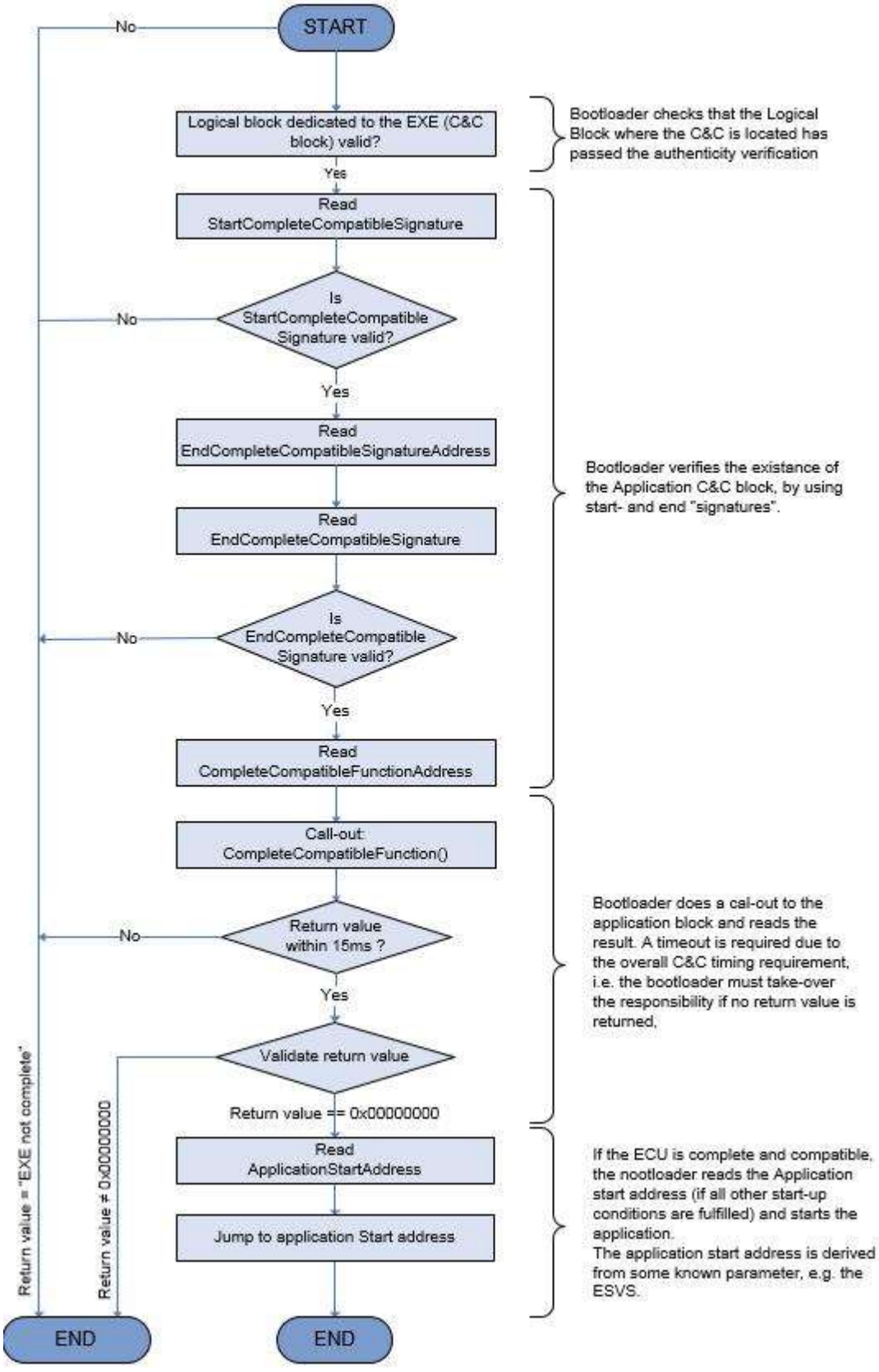


Figure - Application Complete & Compatible block

- StartCompleteCompatibleSignature: To verify the existence (start) of the Application Complete & Compatible block
- CompleteCompatibleFunctionAddress: To define the size and location of the CompeteCompatibleFunction(), i.e. here it is only the compatibility check.
- EndCompleteCompatibleSignatureAdress: To be possible to identify where the EndCompleteCompatibleSignature is located
- Complete&Compatible(): The function invoked from the bootloader, using a call-out.
- EndCompleteCompatibleSignature: To verify the existence (end) of the Application Complete & Compatible block.



Example - Application (EXE) Complete and Compatible block

Note 1: *In case of no response value received, and the a timer has expired, the bootloader terminate the execution of the application block and treat the EXE as incomplete during further validations.*

Note 2: *The location and size of the Application Complete & Compatible block must be agreed between the bootloader and the application developer.*
Note 3: *No call-out is allowed unless it is proven that the call-out address is located in a valid (verified) location, as shown in "Figure - Application Complete & Compatible block".*

4.11.2 Software Authentication

4.11.2.1 General Software Authentication

•Definitions and Abbreviations

4.11.2.1.1 Definitions and Abbreviations

Definition	Description
Authentication	Authentication is the process of assuring and verifying authenticity.
Authenticity	The property of being genuine and being able to be verified and trusted.
Digital Signature	Used to assure originator authenticity and data integrity (of a software)
Hash	Hash algorithms are also called digital fingerprinting algorithms. They are irreversible functions that provide a fixed-size hash based on various inputs. Irreversibility and collision resistance are necessary attributes for successful hash functions. Examples of hash functions are Secure Hash Algorithm 2
Integrity	The property that specified information remains unchanged between observations
Key-pair	The key-pair is used with a public key algorithm. The private key, which is secret, and the public key, which is publicly available. The private and public keys are cryptographically related. The private key cannot be derived from the public key. The private key can be used only by its owner, and the public key can be used by third parties to perform operations with the key owner.
Trust Anchor	The public key of trusted entity (root Certificate Authority certificate or the public key only)
Trust Centre	A trusted party of the public key infrastructure, including key management systems, with a central authority.

Table - Software Authentication Definitions

Abbreviation	Description
ECU	Electronic Control Unit
PKI	Public Key Infrastructure. See definition of Trust Centre
PKIX	Public Key Infrastructure X.509
SW	Software
SWDL	Software Download
VBF	Versatile Binary Format.
VBT	Verification Block Table.

Table - Software Authentication Abbreviations

•Requisite documents

4.11.2.1.2 Requisite documents

The following documents are either referenced by this specification, or contain information that is relevant to this specification.

Source	LC in Elektra

Internal	Versatile Binary Format (VBF)
Internal	General Bootloader requirements
Internal	Software Download and Upload sequences
Internal	Software Authentication for file based systems
Internal	Software Authentication*

* Added here for reference. This is applicable for legacy ECUs not supporting CheckMemory.

The Software Authentication concept includes a list of specifications that has a tight relation to each other, the *"Figure - Software Authentication specification component structure"*, shows the overview and their relation.

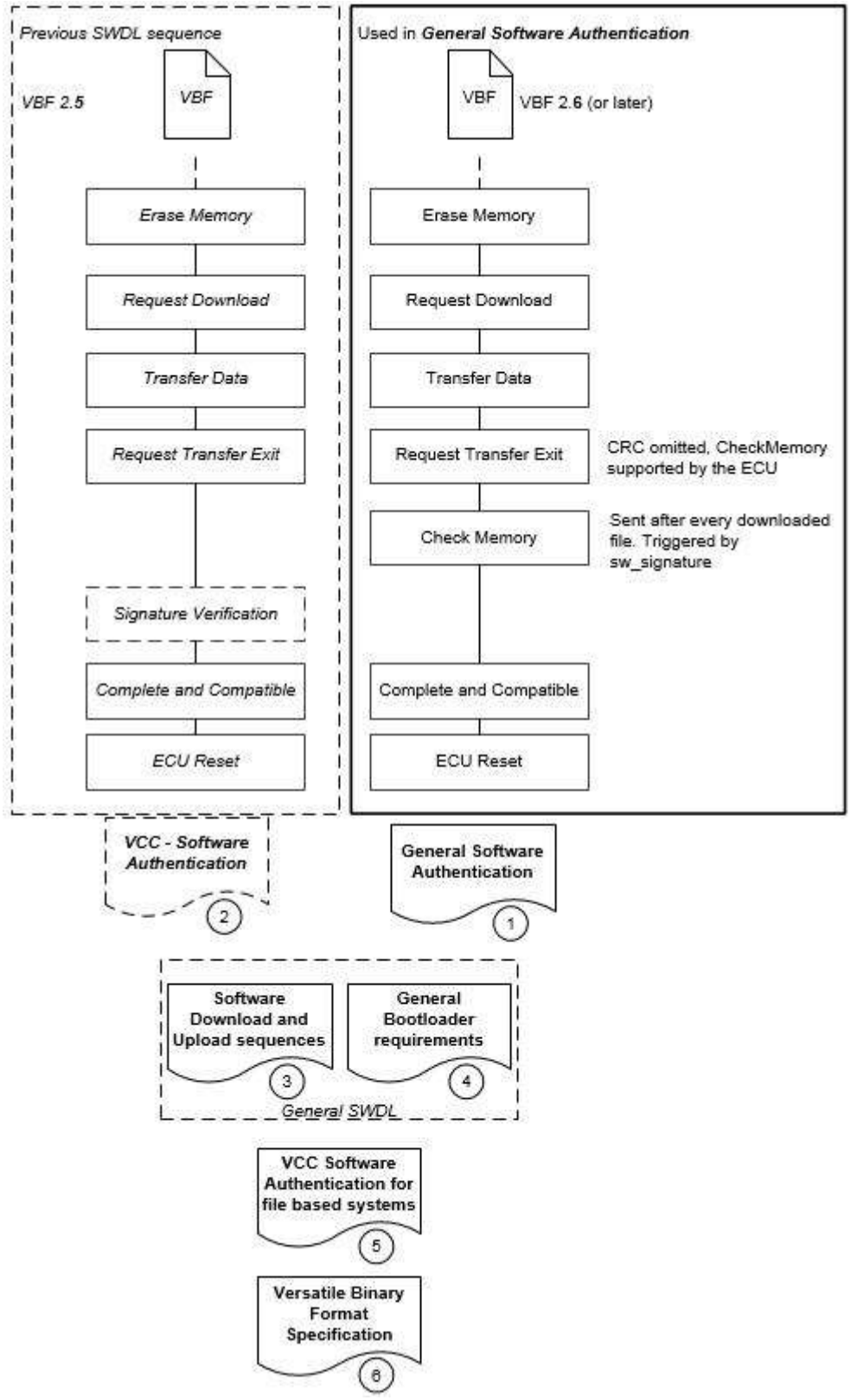


Figure - Software Authentication specification component structure

1 - The *"General Software Authentication"* is defining the software authentication purpose and system overview, together with the applicable signing methods and algorithms.

2 - The " *Software Authentication*" is defining a legacy signing method (for ECUs not supporting CheckMemory), where the complete set of software parts are signed and the single signature is transferred (downloaded) to the ECU in a dedicated software part. Added here for reference only.
3 and 4 - The *“Software Download and Upload sequences”* and *“General Bootloader requirements*” includes the default software authentication method requirements for the client and server respectively, where each software part is independently signed and the signature is transferred in a CheckMemory routine control. Introduced together with the data file format version VBF 2.6. Note: There is a dedicated " *General Bootloader requirements*" version for ECUs not supporting CheckMemory.
5 - The *"Software Authentication for file based systems"*, defines the requirements when code signing certificates are used in the ECU.
6 - The "*VBF (Versatile Binary Format*)" data file format. VBF 2.6 is required when signing methods using CheckMemory are applied.
•Introduction

4.11.2.1.3 Introduction

The purpose of Software Authentication is to provide the following security services in order to protect the software in the vehicle:

- Authentication (proof of origin): the recipient ECU of the downloaded software shall be confident that the software originates from a legitimate source.
- Integrity: the recipient ECU shall be able to verify that the original software has not been altered (e.g. manipulated by an unauthorized user) in transit.
- Copy Protection: it is used to prevent unauthorized feature activation, by e.g. copying a software from one ECU to another. This is not the same as encryption, for confidentiality. Copy protection is realized by key management.

The two first terms, Authentication (proof of origin) and Integrity, are commonly referred to as Authenticity. It is accomplished by the use of public-key algorithms (asymmetric algorithms), where the key (private key) used to encrypt the software is separated from the key used to decrypt it (public key). The private (secret) key is exclusively known to the originator of the software, thus this key is used for signing (generating a digital signature). The recipient(s) having access to the public key is able to verify the downloadable parts. The software together with the signature shall be sent to the recipient ECU. Standard code signing technology relies on asymmetric cryptography in the form of public key infrastructure (PKI) and provides a well proven quality history.

To summarize: Software Authentication concept is applied to ensure that the software being executed in the vehicle is authentic by:

- Secure download/flashing (prevent "malware" software by verifying the downloaded software part). Focus of this specification.
- Secure boot (prevent from booting modified and unauthorized software)

It does however NOT cover e.g. a secure delivery process of complete file from software supplier to the OEM but describes how to sign the data blocks of a software part once those criteria(s) for signing it are fulfilled. Hence, conditions to ensure that a software part is valid and trusted prior to it is stored in software archives, distributed to e.g. cloud etc. or for performing the actual signing of the data blocks are to be specified elsewhere.

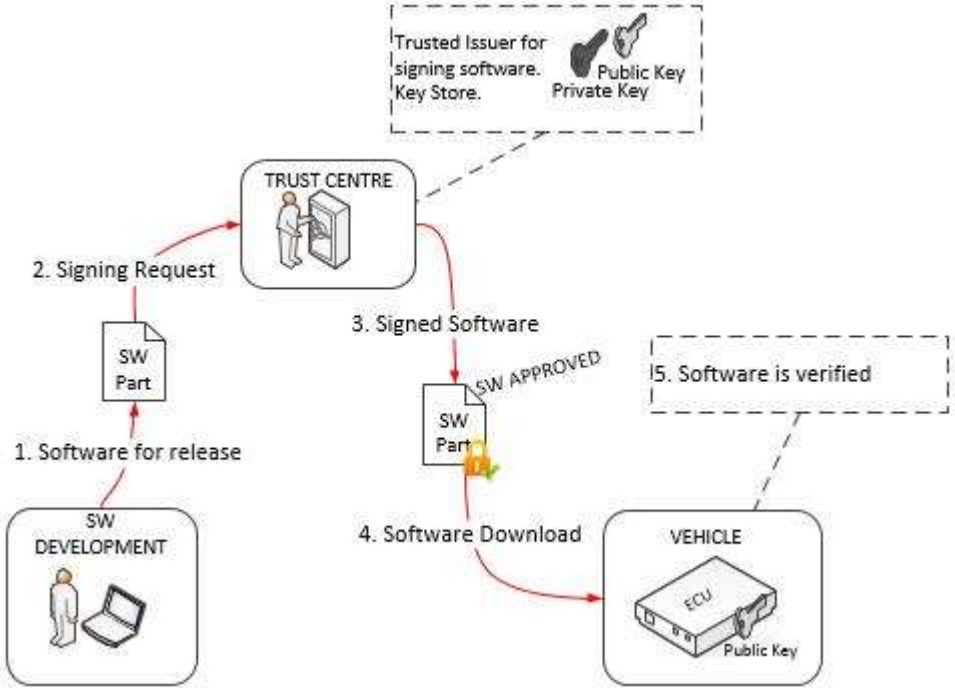


Figure - General Signing model
1. The software is developed and released.
2. The software is sent to the trust centre for signing.
3. The software is signed using a private key and by that a signature (signed software) is generated by the trust centre.
4. The software including signature is stored and transferred (software download) to the ECU.
5. The software stored in the ECU is being verified by using the public key. In order to perform SWDL, authentication of the tester tool is required using Security Access.

In a vehicle the ECU capabilities differs, i.e. some are "high-performance" ECUs but some are having less capabilities. Based on that, the Software Authentication concept has currently only support for a simplified key structure for signing/verifying a software part:

One layer of keys ("raw private/public key pairs") is applied, where only the raw public key data with minimum metadata is used for verification, i.e. no key chain verification as for a more standardized certificate verification of multiple levels (root, policy and issuer authorities). Still, certificates could be used for key management purposes (storage, distribution) and when signing.

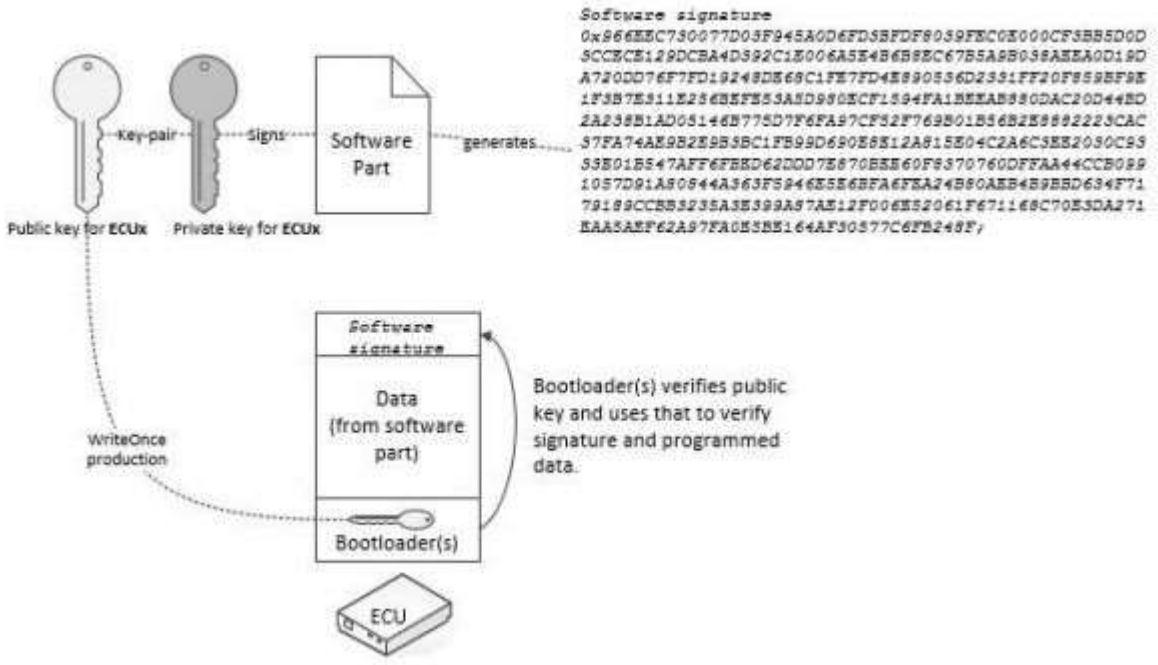


Figure - One-Layer key structure

The figure below gives a brief overview of concerned systems for the Software Authentication concept.

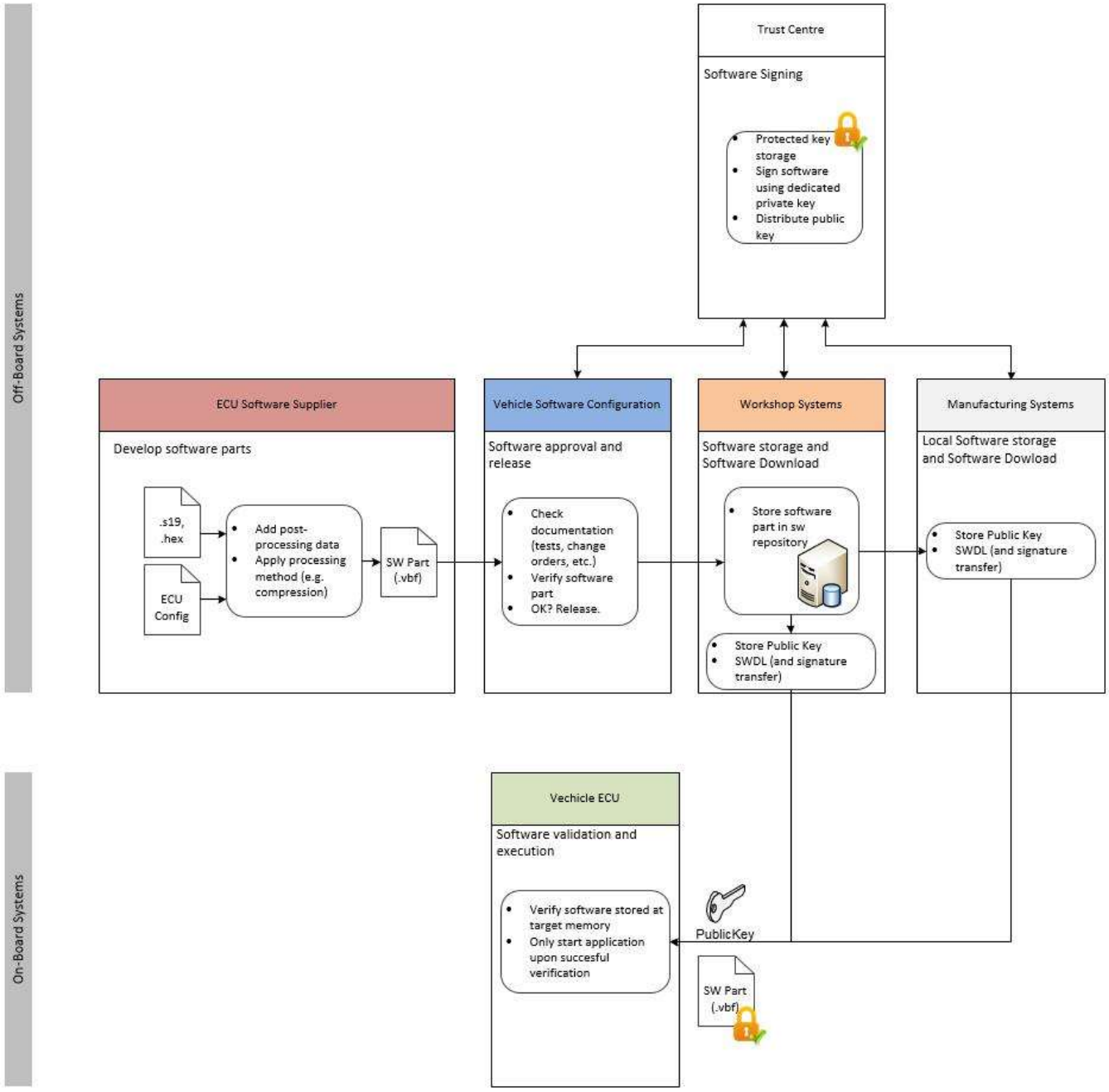


Figure - Software Authentication concerned systems

Figure - Software Authentication concerned systems is a simplified view, where details are omitted (e.g. temporarily software repositories at manufacturing). But, the figures does however illustrate that there are several systems involved which will result in the fact that several interfaces must be defined.

A typical software signing function is described in *Figure - Software signing function overview*. The data that is signed together with the methods (hash function and signing algorithm) must be known to both the signer (Trust Centre) and verifier (ECU). A hash value (digest) is calculated on the data and that hash is being encrypted with the signer's private (secret) key. This encrypted hash is called a signature. The software part together with the signature forms the signed data that is transferred to the ECU.

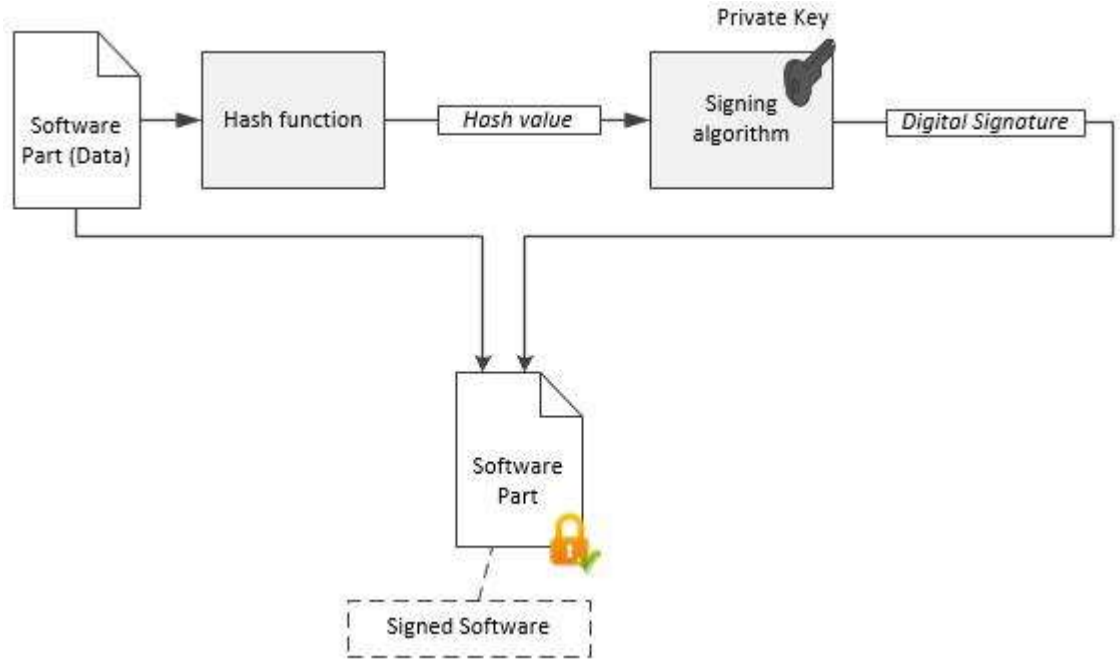


Figure - Software signing function overview

The secret private key is kept protected by the trust centre and must not be exposed, i.e. the data to be signed is an input to the signing function where the private key is stored as shown in *Figure - Trust Centre signing*.

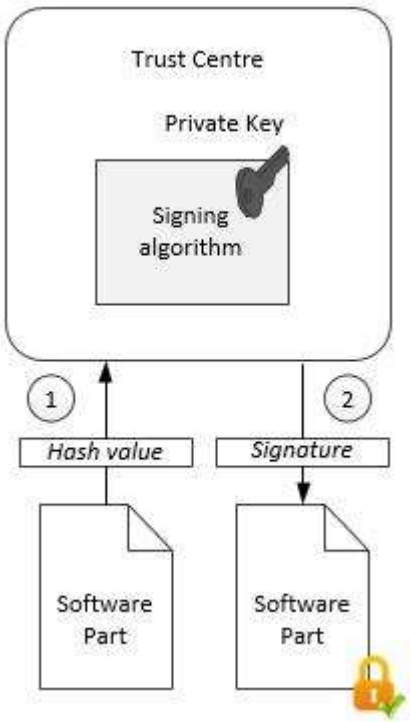


Figure - Trust Centre signing

The software authentication concept must support different signing methods, e.g. for future

growth (change of key lengths, algorithms, etc.) but also for legacy reasons. This variability must be handled in a controlled manner at the off-board systems and because of that the term **Signing Method** is introduced. See chapter *Signing Methods* for further details.

•System Description

4.11.2.1.4 System Description

To support legacy systems and enable future growth, the Software Authentication concept must be prepared to support several signing methods even though an ECU will only support one. The defined signing method will affect:

- Algorithms and key lengths chosen
- Sign and verify with code signing certificates or by only using the key-pair (no certificates)
- How to transfer the signature data
- How to initiate the verification in the ECU and inform the user of the result
- ...

Software Authentication Generation 2.0 - each software part is signed individually: Each software part is signed individually and each software part is carrying the digital signature, separately generated as previously shown in *Figure - Software signing function overview.*

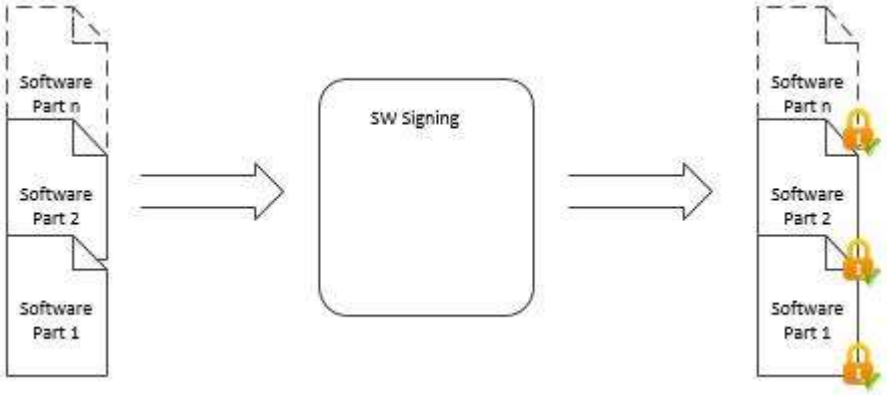


Figure - Software signing of each software part independently

For off-board tool reference purposes (to manage the variability), the legacy concept "Software Authentication Generation 1.0 (Legacy) - complete set of software parts are signed" can be found in *[Software Authentication]*.

This section will describe the following main parts, for the method where each software part is signed individually:

- Software part creation
  - Responsible: ECU Software Supplier
- Sign
  - Responsible: Trust Centre
- Software Download
- Verification

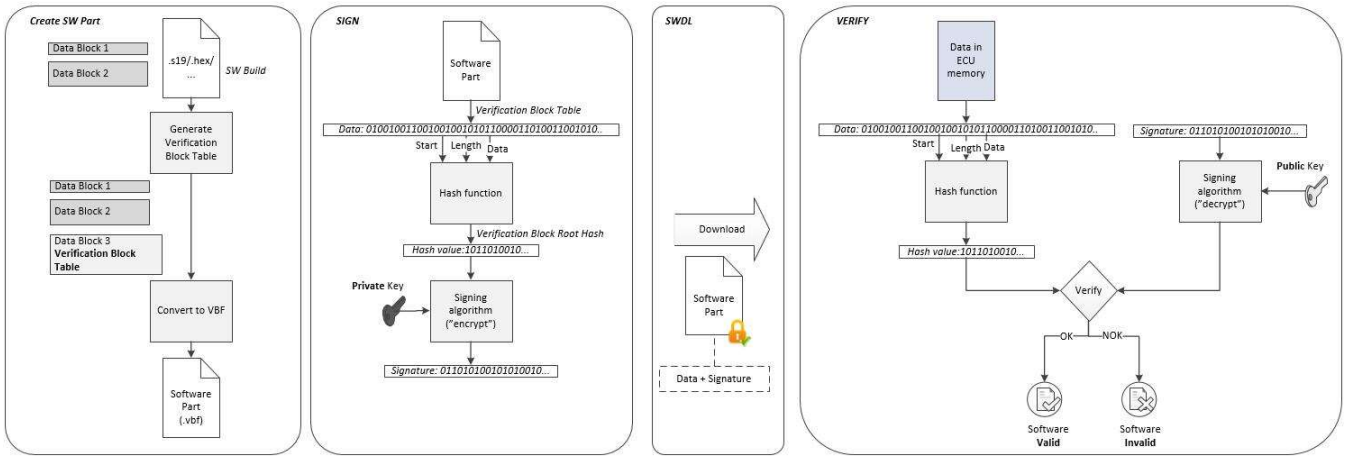


Figure - Create, Sign, Download and Verify a software part

4.11.2.1.4.1 Create Software Part

A software part could be segmented into several data blocks. To verify the data stored, the ECU must have knowledge of the data that must be verified and where it is located. The ECU software supplier will, once the application is build, generate a *Verification Block Table* and append it to the Software Part. This block table contains information of all other data blocks (location and hash values), i.e. it is equivalent to a manifest. The Verification Block Table will be digitally signed and when the ECU has verified it, the ECU could continue to verify all other expected data blocks. Hence, there will be one Verification Block Table per Software Part.

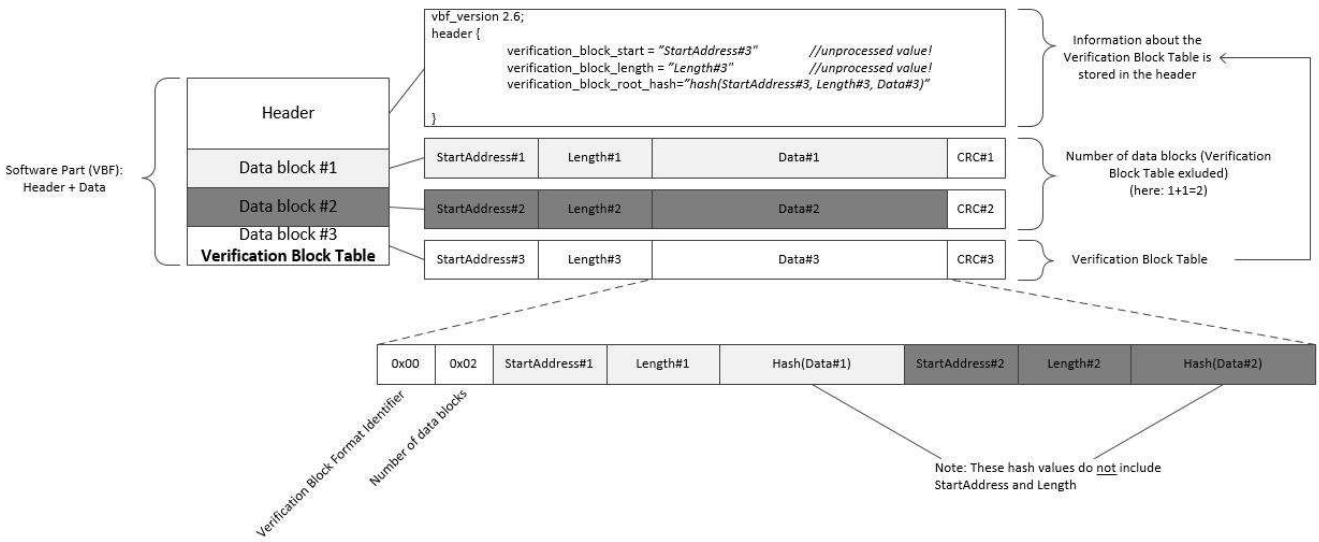


Figure - Verification Block Table format

Example.

The ECU software supplier has build an application having two data blocks.

Based on the two data blocks, the ECU software supplier generates and adds a Verification Block Table (a dedicated data block, stored at a known start address):

The start address, length and hash value of the Verification Block table itself are added to the

file header. The hash values of the data blocks stored in the Verification Block Table are calculated on the data only, but the hash value of the Verification Block Table (i.e. verification_block_root_hash in software part header) is calculated using also its start address and length. Note that the lengths defined in the VBT are the lengths of the data blocks prior to e.g. any compression is applied.

4.11.2.1.4.1.1 Verification Block Table

661295 v2 **Format of the Verification Block Table**

Version history:
(2): VBF软件文件升级后，更新VBT的StartAddress和Length的长度取决于ReqID 661296的定义。

Purpose: Define the format of the verification block table
Source:
Verification Method: Analysis
Legacy ID: 398360v4

The verification block table, encoded with big endian, shall have the following format:

- Verification Block Table Format Identifier. The verification block will start with most significant byte of this format identifier. It defines the version of the verification block table, mainly intended for tool parsing and format checks in the ECU, i.e. it shall not be used to configure e.g. the actual algorithm to be used by the bootloader. Size 16 bits.
- Number of data blocks. This shall be identical to the number of data blocks included in the software part. The data block containing the verification block table itself shall not be included. Size 16 bits.
- StartAddress [size depends on VBT Format Identifier as defined in ReqID 661296] , Length [size depends on VBT Format Identifier as defined in ReqID 661296], Hash value. Every data blocks shall be defined, except the block table itself. The size of the hash is dependent of the hash function used and is specified elsewhere. The Length is the unprocessed length (see other requirements for more details).
- The data blocks shall be sorted according to the order the data blocks are being programmed to the ECU.

Example.

- Verification Block Table Format Identifier uses 0x0000.
- SHA-256 hash function is used, i.e. each hash value is 256 bits.
- Number of data blocks are two, i.e. the total length of the verification block table, in octets, will be (16+16+2x(256+32+32))/8=84 octets.

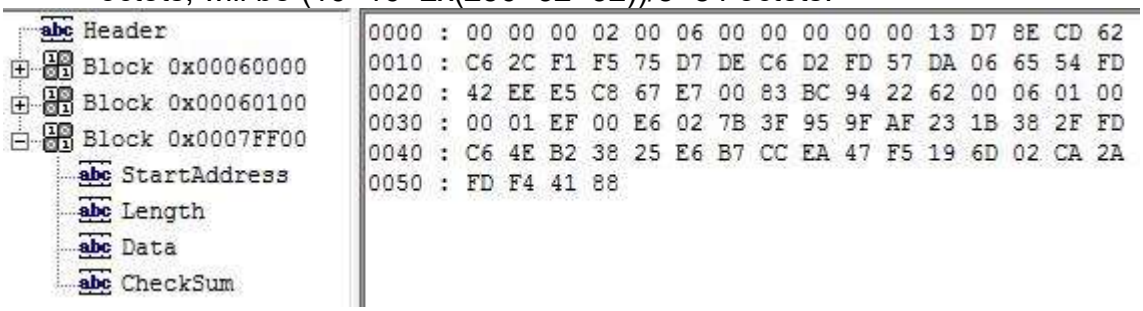


Figure - Example of a VBF file with a verification block table

Figure - Example of a VBF file with a verification block table shows a software part (VBF data

file). The data shown belongs to the verification block. The Verification block starts at address 0x0007FF00, i.e. it is the third data block in the figure:
0000
0002
00060000 00000013
D78ECD62C62CF1F575D7DEC6D2FD57DA066554FD42EEE5C867E70083BC942262
00060100 0001EF00
E6027B3F959FAF231B382FFDC64EB23825E6B7CCEA47F5196D02CA2AFDF44188

- The format identifier is 0x0000
- Number of data blocks are two (0x0002)
- Data block 1: Start address 0x00060000 and length 0x00000013. Hash 0xD78ECD...
- Data block 2: Start address 0x00060100 and length 0x0001EF00. Hash 0xE6027B...

661299 v2 **Verification Block Root Hash**

Version history:
(2): 引用需求号更新。

Purpose: Define how to calculate the verification block root hash, carrying the information of unprocessed data that is required to support use cases of having processing methods not necessarily known to the Trust Centre.

Source:

Verification Method: Analysis

Legacy ID: 404694v2

Req:
The verification_block_root_hash must be calculated on the "raw" unprocessed data in the verification block table, i.e. when no prior to any padding, encryption/compression/delta encoding methods have been applied to the VBT data block. The verification block root hash shall be calculated as Hash(startAddress||length||Verification Block Table data), where || represents the concatenation. The order shall be according to *Figure - Verification Block Root Hash calculation*. The start address (verification_block_start) and length (verification_block_length) are included to ensure the placement of the verification block and shall be 32 bits or 64 bits long each. The (root) hash function shall be the same as used when calculating the hash value of the data blocks defined in the verification block table (*"Req ID 661296: Verification Block Table Format Identifier - Definition"*).

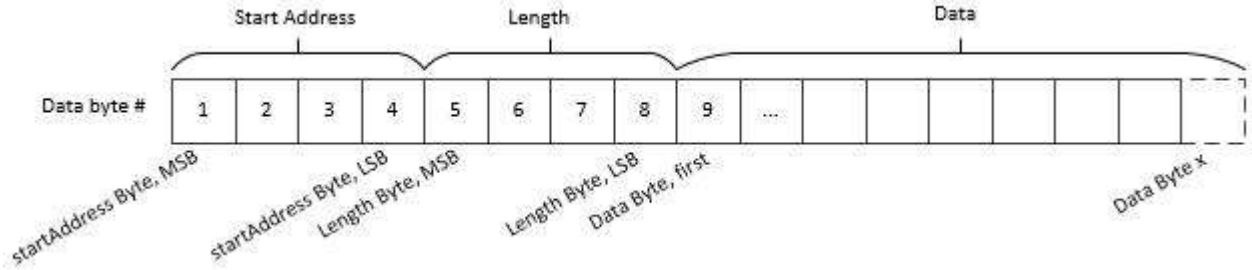


Figure - Verification Block Root Hash calculation (with 32bits long)
The root hash value shall be added by the software supplier to the header of the software part.

Example 1.
All values are in hexadecimal. SHA-256 is used as hash function.

Hash(startAddress||length||Verification Block Table data)=
Hash(*00 05 FF 00***00 00 00 54** 00 00 00 02 00 00 F0 00 00 00 00 88 B7 07 24 15 45 A3 46 26 5A AB 1F FB 32 FF 64 B5 5B F8 F8 DC 1B 56 A4 6E F3 3C E3 D1 5D B1 1D 33 00 00 F1 00 00 00 00 10 5A 2C FE 8A B9 35 91 85 25 D4 4F D6 FD 87 C7 0F C8 3B 4F 29 D1 A7 27 67 2E 1B 48 F3 80 47 3F C1) =
9B B4 E0 92 1C 4B 84 1C 31 C3 CB 3F DD 15 D0 B9 E8 ED 70 5E D3 C2 52 DC C9 2A D5 06 E0 5E 2B AE.

Start Address: *0005FF00*
Length: **00000054**
Data: 00 00 00 02 00 00 F0 00 00 00 00 88 B7 07 24 15 45 A3 46 26 5A AB 1F FB 32 FF 64 B5 5B F8 F8 DC 1B 56 A4 6E F3 3C E3 D1 5D B1 1D 33 00 00 F1 00 00 00 00 10 5A 2C FE 8A B9 35 91 85 25 D4 4F D6 FD 87 C7 0F C8 3B 4F 29 D1 A7 27 67 2E 1B 48 F3 80 47 3F C1

Example 2.
The length of the data block for a verification block table is 44 bytes long, prior to any padding or processing method is applied.
All data blocks shall be aligned to a multiple of 8 bytes. Hence, the VBT data block is padded to 48 bytes, but verification_block_length is still 44 (0x2C). It is this length that is used when calculating the verification_block_root_hash.

Note: Be aware of the differences of how to manage the padding of data and start/length information for data blocks that are defined in the verification block table versus the data block containing the VBT.

661297 v1 **Verification Block Table - Input order to Hash function per data block**

Version history:
(1):

Purpose: Define the input order to the hash function per data block. If the order is changed, the output from the hash function will be changed also.The start address and length of a data block are NOT input to the hash function. The placement of a data block is ensured by explicitly defining the start address and length per corresponding data block: start address, length, hash (data block).
Source:
Verification Method: Analysis
Legacy ID: 398373v1

The hash value for each data block in the verification block table shall be calculated as *Hash(Data)*, i.e. the start address and length are excluded. The data to be hashed starts with the data at the start address.

661296 v2 **Verification Block Table Format Identifier - Definition**

Version history:
(2): VBF软件文件升级后，增加VBT的Verification Block Format Identifier 0x0003。

Purpose: Define the applicable value of the Verification Block Table Format Identifier.

Source:
Verification Method: Analysis
Legacy ID: 404696v3

The Verification Block Table Format Identifier shall be according to *Table - Verification Block Format Identifier*.

Format of the Verification Block Table	Verification Block Format Identifier	Hash function	The Size of StartAddress	The Size of Length	Comment
As defined in *"Req ID 661295 Format of the Verification Block Table"*	0x0000	SHA-256	32 bits	32 bits	Hash(Data block)
As defined in *"Req ID 661295: Format of the Verification Block Table"*	0x0001	SHA-384	32 bits	32 bits	Hash(Data block)
As defined in *"Req ID 661295: Format of the Verification Block Table"*	0x0002	SHA-512	32 bits	32 bits	Hash(Data block)
As defined in *"Req ID 661295: Format of the Verification Block Table"*	0x0003	SHA-256	64 bits	64 bits	Hash(Data block)
-	0x0004-0xFFFF	/	/	/	Spare

Table - Verification Block Format Identifier

Example.
1. The signing method used is, as defined in *"Req ID 1598: Default Signing Method"*, e.g. SHA256_RSA2048. This means that Verification Block Table Format Identifier shall be 0x0000.

2. The large size dataBlock for HPC, e.g. exceeding 4GB, the Verification Block Table Format Identifier should be 0x0003. Other software parts belonging to this HPC can also use the same VBT Format Identifier, except SBL.

661298 v2 **Verification Block Table Header Identifiers**

Version history:
(2): 引用需求号更新。

Purpose: Define the required header identifiers required to support the applied signing method
Source:
Verification Method: Analysis
Legacy ID: 404693v3

The header section of the delivered software part must contain the following identifier with

respect to the verification block table, when a verification block table is constructed according to the format in *"Req ID 661295: Format of the Verification Block Table"*.
Alternative 1 - verification block header identifiers (default configuration):

- **verification_block_root_hash**: The hash value of the unprocessed verification block table.
- **verification_block_start**: The start address of the unprocessed verification block table.
- **verification_block_length**: The length of the unprocessed verification block table.

Upon OEM approval, the header section can exclude the verification_block_start and verification_block_length identifiers:
Alternative 2 - verification block header identifiers:
verification_block_root_hash

This might be applied for ECUs where existing installation and verification mechanisms, using e.g. ECU specific manifests, are to be reused in the ECU to ensure authenticity that is not possible to realize using the verification block table format.
Example.

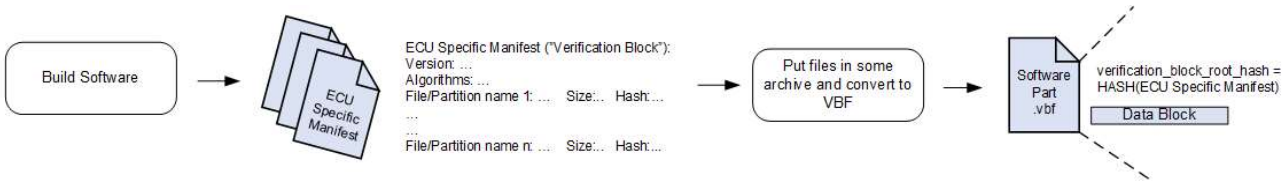


Figure - Example of ECU supplier specific verification block (manifest)

"Figure - Example of ECU supplier specific verification block (manifest)" shows a data file header where verification_block_root_hash is present only.

661302 v1 **Verification Block Table in a padded data block**

Version history:
(1):

Purpose: To clarify how to manage the verification block table when there is a need to align that data block to a specific length, e.g. to match the start and/or end of a data block to the boundary of a flash sector (for an addressed based ECU). This requirement is valid for the data block containing the VBT, i.e. for all other data blocks the padding will be treated as any other data when generating the VBT.

Source:
Verification Method: Test
Legacy ID: 457183v0

Req:

If there is a must to align and pad the data blocks to some specific size, the padded data must have the value of erased memory, i.e. it will be included in the ECU "blank" check.

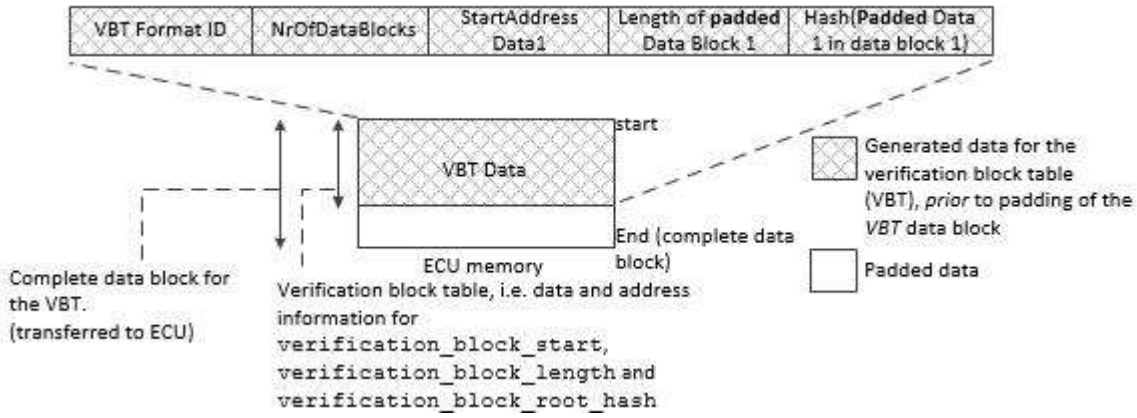


Figure - Data block with aligned verification block table data block

Note: The verification Block Table*_block_start, verification_block_length and verification_block_root_hash shall be* calculated on data (blocks)*generated using the "raw" unprocessed data in the verification block table prior to* some processing methods have been*any padding is applied*, i.e. prior to the any compression, encryption or delta encoding is applied *to this specific data block.*

661300 v1 **Verification Block Table on unprocessed data**

Version history:
(1):

Purpose: To enable that the target software is verified.
Source:
Verification Method: Analysis
Legacy ID: 404184v2

Req:
The data block definitions in the Verification Block Table shall be prior to processing methods have been applied, i.e. prior to the any compression, encryption or delta encoding is applied. If there is a need to align the data blocks to a specific size prior to a processing method is applied (e.g. to match a boundary of a flash sector and to be a multiple of the minimum programmable block size for the specific ECU), it is the padded data blocks that are to be used as inputs when defining the data blocks in the Verification Block Table.

Example.

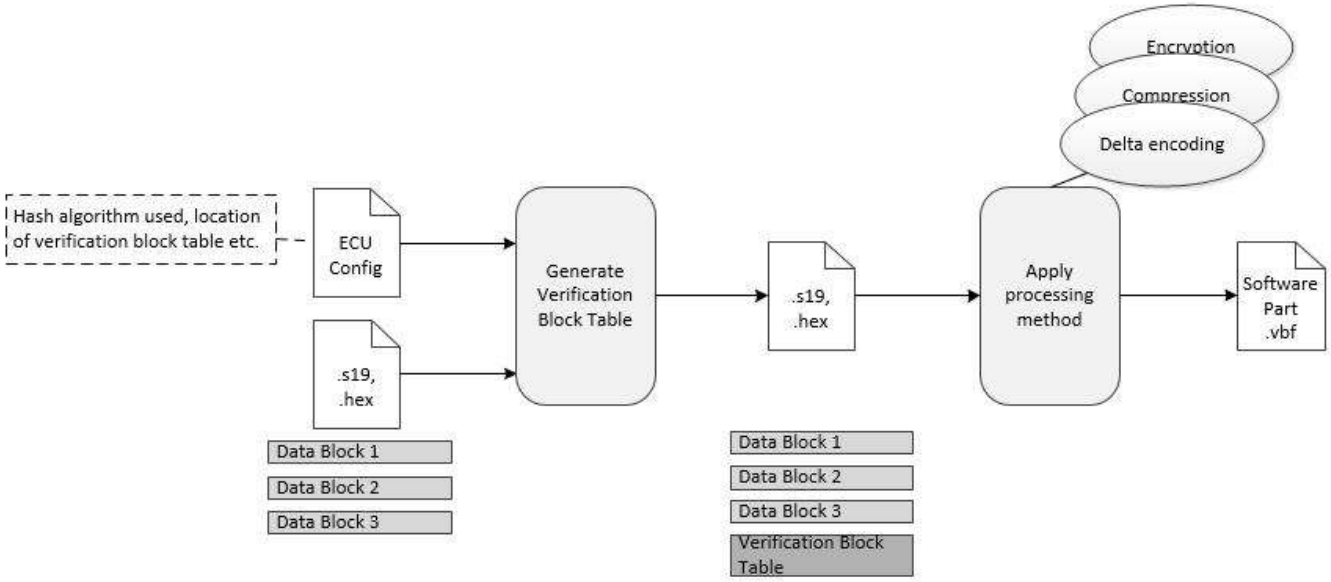


Figure - Generate verification block table on unprocessed data

661301 v1 **Verification Block Table per Software Part**

Version history:
(1):

Purpose: To enable that each software part is to be signed and verified individually.
Source:
Verification Method: Analysis
Legacy ID: 398371v1

Each single software part must contain a verification block table, i.e. it will be one verification block per logical block defined in the ECU.

4.11.2.1.4.2 Sign Software Part

The software is signed with the confidential private key. It is the Verification Block Table that is signed, as defined by the software part header identifier.

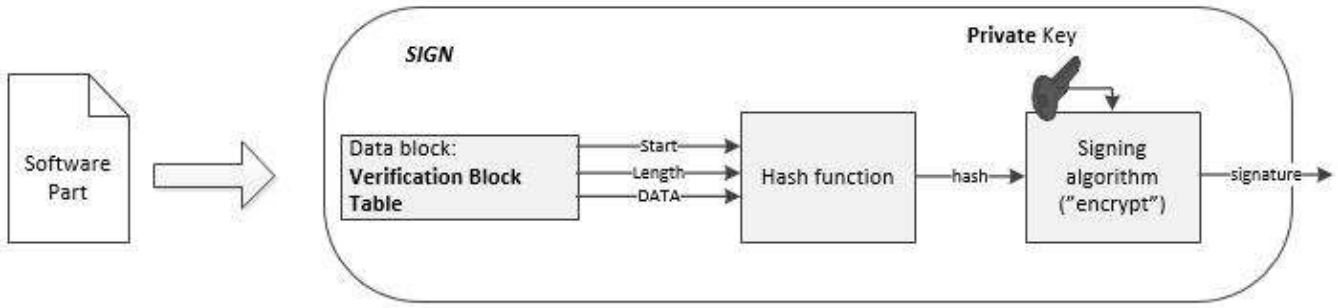


Figure - Software Signing Tool

There are many use-cases to be supported, including different diagnostic clients, in order to download (signed) software to a vehicle:
- key balancing, e.g. having global/common keys used for a period of time or ECU unique keys

- software download at manufacturing, workshops, over-the-air (including vehicle internal tester), test&development, fault-tracking (e.g. when a software download using workshop diagnostic client fails and R&D tools must be used), repair analysis etc.

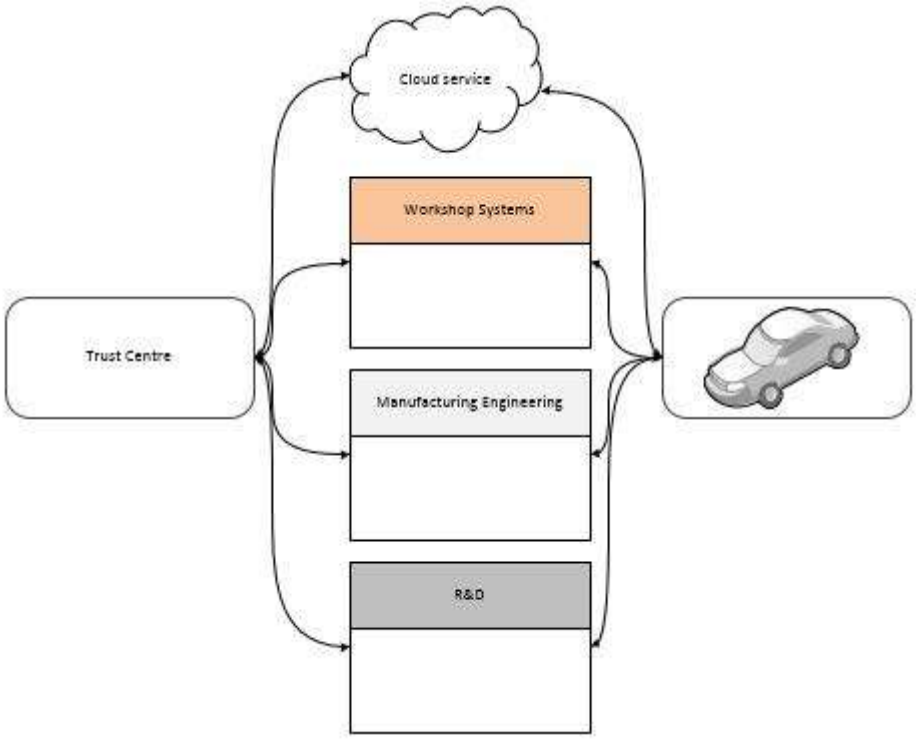


Figure - Example of entities downloading software to a vehicle

To support the variability as shown in *Figure - Example of entities downloading software to a vehicle* as well as balancing key usage, the signature data for production (e.g. customer vehicles) vehicle is kept detached (separate file).

Example.
Unique keys are used.
A software application *12345678AA.vbf* is to be signed using private key#1 for ECU#1 and private key#2 for ECU#2. The sw application part could be programmed in both ECUs but the belonging signature data must be transferred to the correct ECU respectively, otherwise the application will not start.

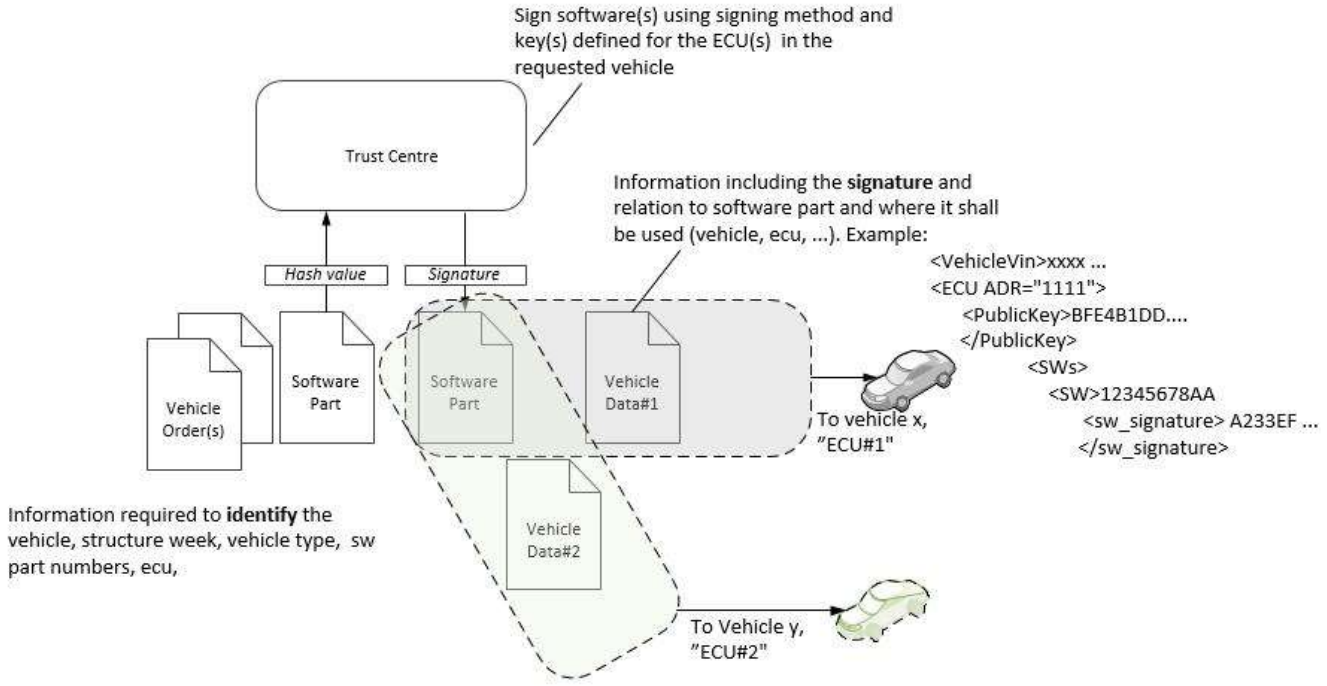


Figure - Example: Detached signature

The *Figure - Example: Detached signature* shows the principle of detached production signature.

Hence, the diagnostic client must have the capability to correlate belonging signature for a software part and transfer it to the vehicle.

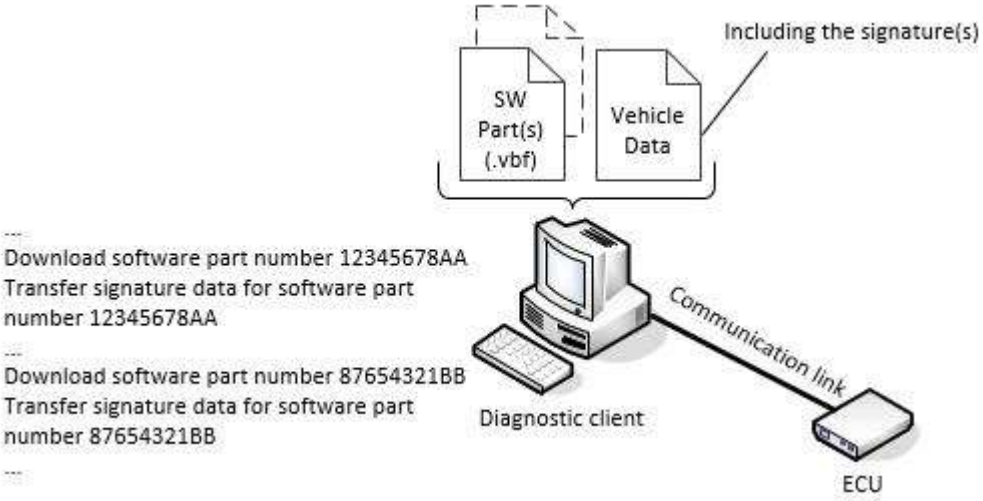


Figure - Download a software and transfer signature data

To support the test&development phases, a development key-pair will be used. To simplify the software part management, the software signature for development will be placed in the part header (the R&D diagnostic client will also have the capability to read the software signature for production from the software part header *[VBF (Versatile Binary Format)]*) .

Refer to section *Key Management for Test and Production ECUs* for further details.

661306 v1 **Append signature to software part**

Version history:
(1):

Purpose: Define how to append the signature to the software part. The signing tools must support the use-case of sign and re-sign software parts, typically during development but also to enable for e.g. key updates. To support the lifecycle of the part, i.e. the software is verified (development) prior to it is released to production, two signature types will be applicable for the final software part (to not have to re-sign the software when it is to be used in a test object versus a production vehicle).

Source:

Verification Method: Analysis

Legacy ID: 428666v2

The digital signature when the software part is signed with the intended private key for **development**, sw_signature_dev, shall be appended to the software part header by the software supplier when a part is delivered.. The resulting sw_signature signed by the trust centre at OEM with the intended key for **production** shall be stored in a separate file, using a standardized format separately (e.g. .xml). The signing tool must support both sign and re-sign functionality of a software part.

Example.
(1) A software part is delivered containing the sw_signature_dev. This software can only be used in an ECU equipped with the public key for development.
(2) The software part is, when successfully verified, validated and ready for production, signed with the private key for production. This means that two signatures are present (the sw_signature_dev is kept).
(3) This software part can now be used in both a development and production ECUs, but it must be ensured that the corresponding signature is transferred at SWDL.

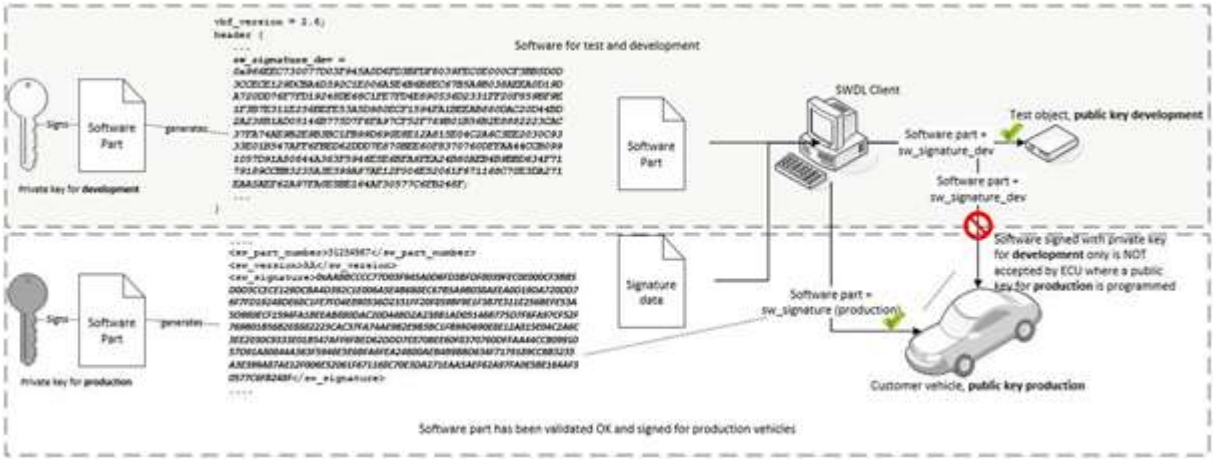


Figure - Signature for test and development and production

661304 v1 Sign Software Part - Private Key syntax

Version history:
(1):

Purpose: Define the key representation for key distribution at test&development and for development tool usage.
Source: PKCS #8: Private-Key Information Syntax Standard, RSA Laboratories Technical Note Version 1.2, Revised November 1, 1993. PKCS #1 v2.2: RSA Cryptography Standard, RSA Laboratories, October 27, 2012. RFC1421.
Verification Method: Analysis
Legacy ID: 428665v1

The Private key shall be represented in an unencrypted PKCS #8 PrivateKeyInfo format as defined in *PKCS #8: Private-Key Information Syntax Standard, RSA Laboratories Technical Note Version 1.2, Revised November 1, 1993*.
The PrivateKey part of of the PrivateKeyInfo shall be a represented as an ASN.1 type RSAPrivateKey as defined in *PKCS #1 v2.2: RSA Cryptography Standard, RSA Laboratories, October 27, 2012.*
The Private Key shall be stored using the PEM format (base64 encoded data) as defined in RFC1421.

Note: *This is the formats to (at least) be supported by signing development tool. The format used by the Trust Centre (production) is specified elsewhere.*

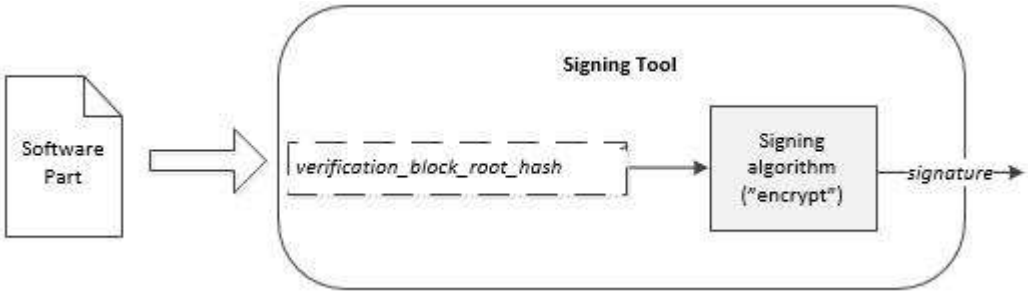
661305 v1 Signing function - Generate signature

Version history:
(1):

Purpose: Define the input data to the signing function.
Source:
Verification Method: Analysis
Legacy ID: 428667v3

The data to be signed is defined by the verification_block_start and verification_block_length identifiers. These start address, length and data of the addressed data block shall be used as inputs when generating the hash value. The signing entity has two possibilities:
(1) always calculate the hash value over the verification block table and ensure that the content of the table is correct. This is the preferred option.
(2) use the already calculated hash value (verification_block_root_hash).

Example.
The verification_block_root_hash might be used when the processing method is not known (e.g. delta compression method) as defined by the software part data format identifier but it might also be used to optimize off-board functions (e.g. computational time), alternatively when the verification_block_start and verification_block_length are not present in data file header.



Sign - Processing method unknown

661303 v1 **Signing function - Integrity Check**

Version history:
(1):

Purpose: To ensure that the software part is not altered when updated, i.e. validate the integrity.
Source:
Verification Method: Analysis
Legacy ID: 428664v1

The signing tool used must ensure the integrity of the software part. If the tool is updating some software part identifier, the file_checksum shall be checked before and after the update and the checksum shall be identical.

4.11.2.1.4.2.1 Sign and verify unprocessed data

At the ECU side (where the software part is verified) it is the target data to be executed that shall be verified, e.g. the data stored in the non-volatile memory (for an application file). Hence, it is the unprocessed data that must be signed, i.e. the data prior to any processing method (**delta encoding**, **compression**, **encryption**, etc.) has been applied.

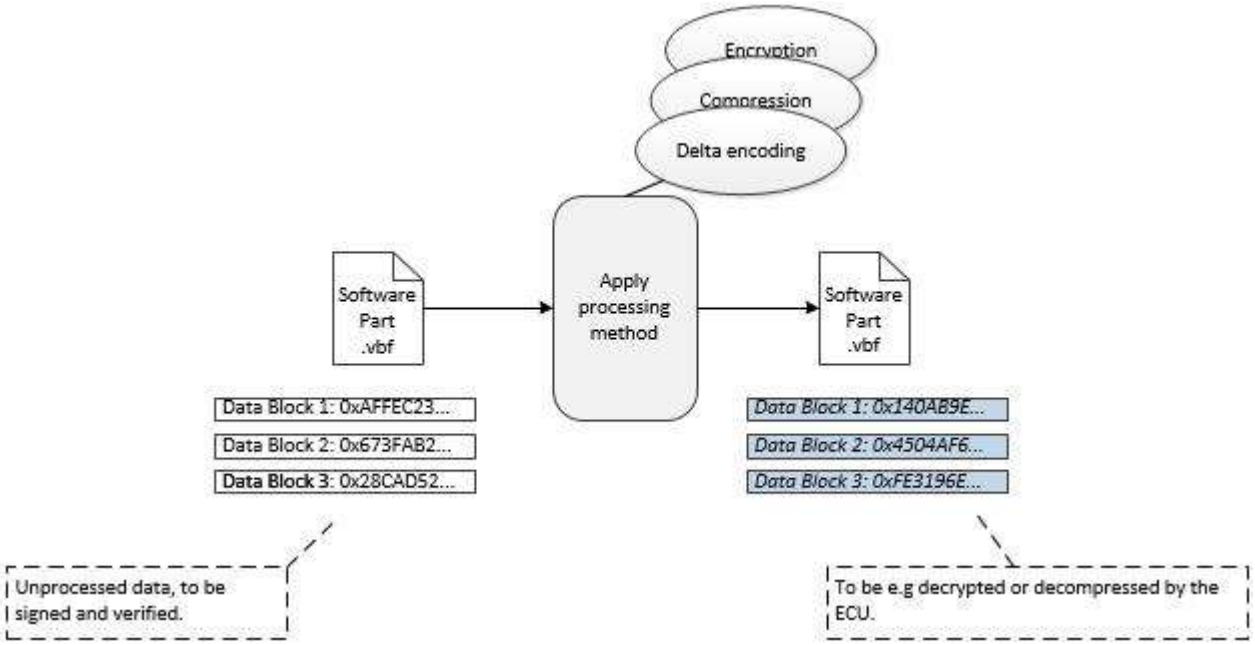


Figure - Signing and verification of unprocessed (target) data
From an ECU point of view, see *Figure - Verification of unprocessed data,* where the data stored in the non-volatile memory is verified,

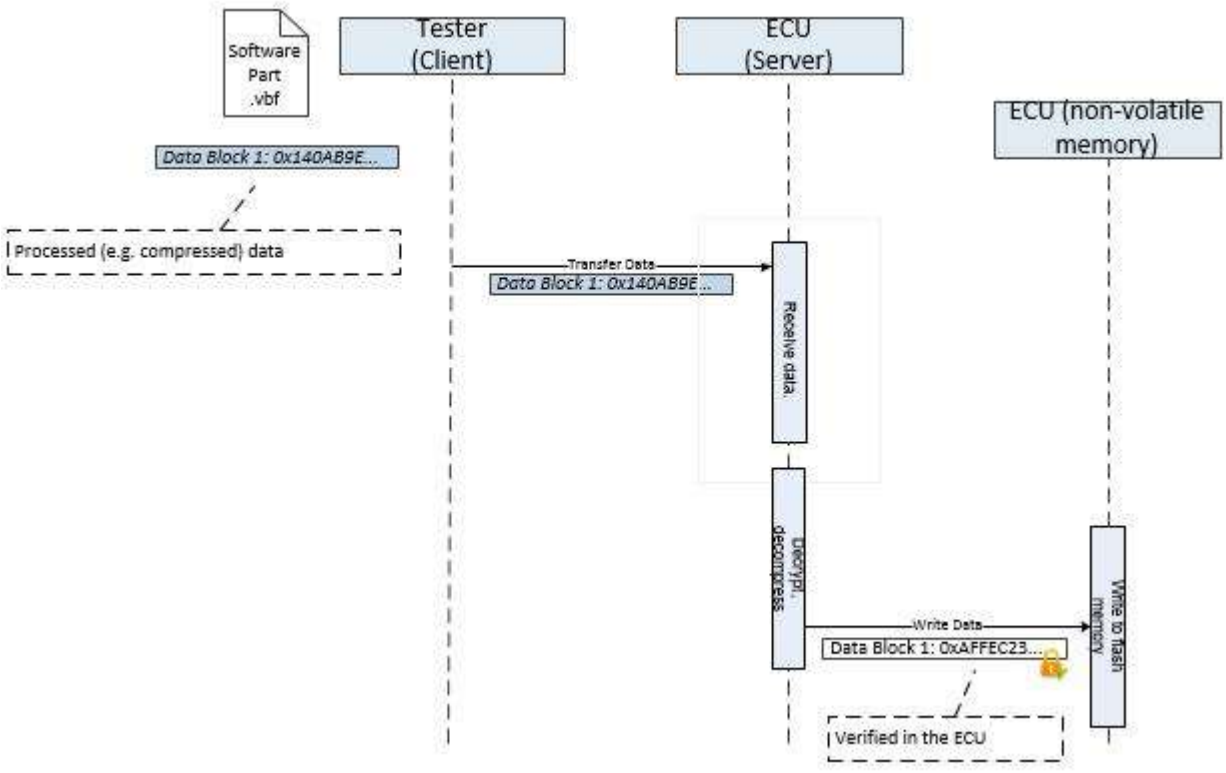


Figure - Verification of unprocessed data

661307 v1	Sign and verify unprocessed data

Version history:
(1):

Purpose: Define how to handle compressed, encrypted or delta encoded software.
Source:
Verification Method: Analysis
Legacy ID: 404594v0

The signing and verification shall be based on unprocessed data.

4.11.2.1.4.2.2 Sign a Delta Software Part

When delta encoding is used, the delta file shall contain information (hash values, signature, ...) identical to the final target software:

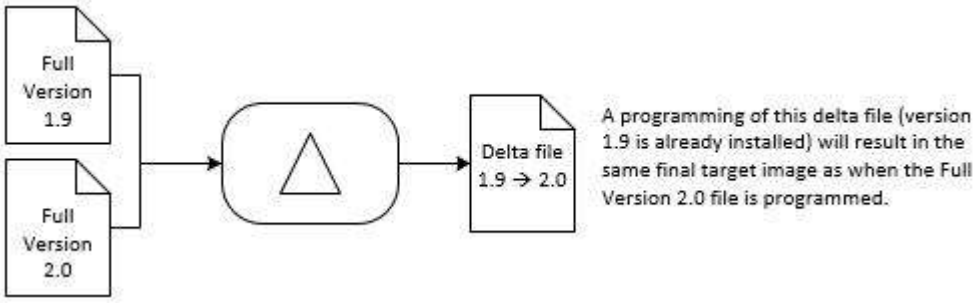


Figure - Signature for a delta file
When a delta file, as shown in *Figure - Signature for a delta file* is programmed, the ECU must finally verify the final data that has been installed in the non-volatile memory.

661308 v1 **Sign a delta file**

Version history:
(1): 增加差分的需求。

Purpose: Define how to sign a delta file. The programmed (installed) data shall be verified, i.e. not the transferred (streamed) data.
Source:
Verification Method: Analysis
Legacy ID: 429082v3

A delta software part shall contain the same header identifiers as the corresponding full version, with respect to following Software Authentication identifiers: verification_block_start, verification_block_length, verification_block_root_hash and sw_signature_dev.

4.11.2.1.4.3 Download a (signed) software part

This section describes how to download a signed software part and transfer the signature data, where the signature data is transferred after each software part is downloaded.

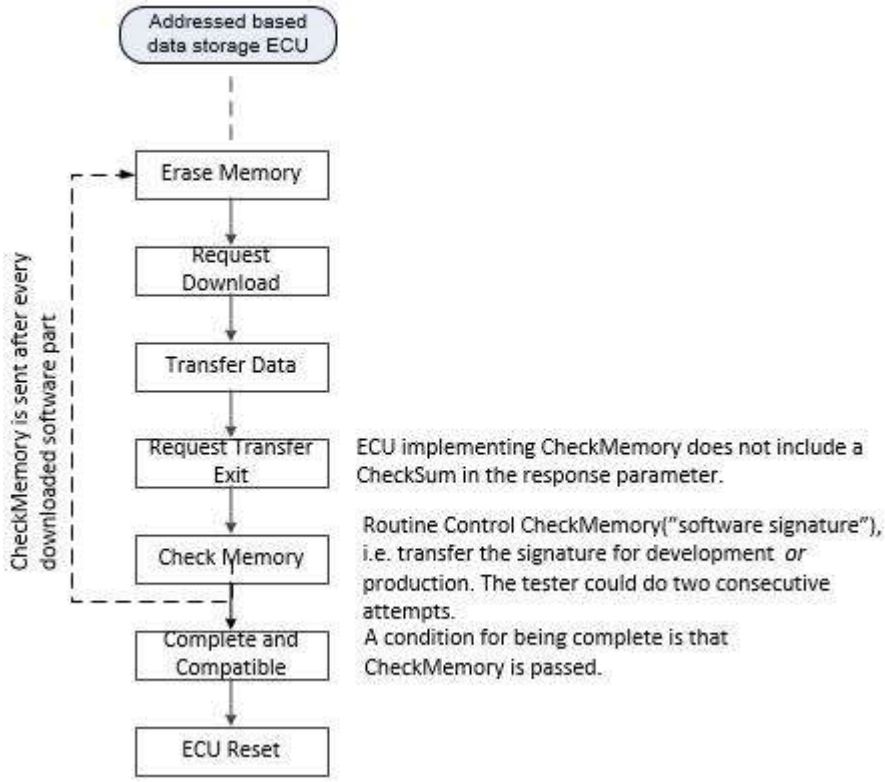


Figure - SWDL of a signed software part

The software signature is transferred when the complete data file has been downloaded using the CheckMemory routine control, which will also initiate the ECU verification. The result values provided in the CheckMemory response will inform the tester whether the verification has passed or not . Note that ECUs implementing CheckMemory shall not respond with a checksum value at the RequestTransferExit service. The CheckMemory return values are specified in *[General Bootloader requirements].* The result value is 0 (zero) when the verification is passed. If the verification is failed, the tester might do a second attempt.

Prior to the programming of a data file starts it must be ensured that the expected public key is stored in the ECU, e.g. either an initial development key is stored and to be kept (test and development only) or the dedicated public key shall be programmed (if it is not already programmed).

4.11.2.1.4.4 Verify a signed software part

This section describes and defines the general ECU verification requirements, relevant to both off- and on-board (ECU) systems. More detailed ECU requirements are however part of *[General Bootloader requirements]*.

An ECU is divided into several “Logical Blocks”, one logical block per software part and each logical block is signed/verified independently. A successful signature verification means that a logical block is complete (complete & compatible). The ECU must know the logical blocks that are mandatory for the whole application to be complete.

The logical block definition, ECU Software Structure, is to be re-programmable in order to support use-case when the software structure is changed from e.g. three to four files or when a software part growths beyond its dedicated range.

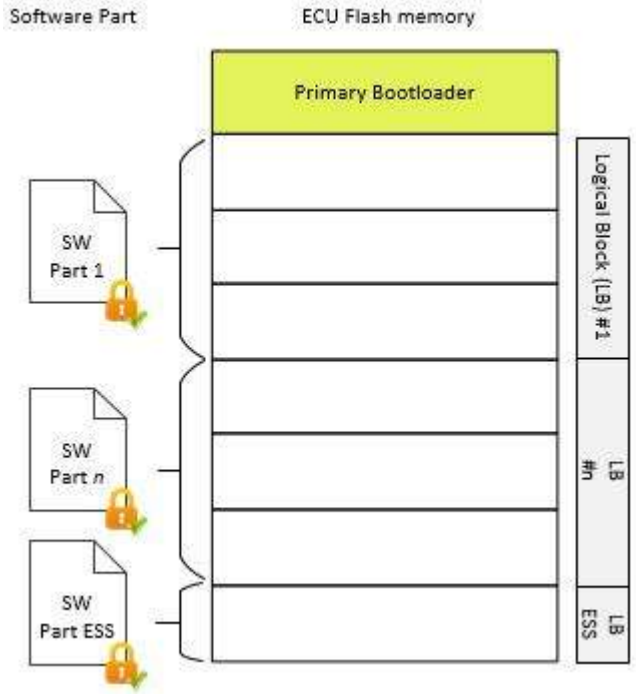


Figure - ECU Software Structure, ESS

The programmed public key is used to verify the software part, i.e. the logical block at:

- Installation, i.e. at Software Download (must).

But, the memory not changed (e.g. memory not used for adaptations, "user space", etc.) after e.g. software download can also be verified:

- At every start of the application (secure boot), starting from a root of trust and trusted read-only start-up code.
- At run-time, i.e. when the application is running.

A software part must have been verified OK for the application to be started in the ECU. The secondary bootloader, stored in volatile memory, is not shown in *Figure - ECU Software Structure, ESS.* The SBL must be successfully verified prior to it is activated.

4.11.2.1.4.5 Key Management for Test and production ECUs

To e.g. simplify the daily life for developers/testers etc. and to keep track of the keys used for signing and verification during test and development, a known key is used. This must not be used in production vehicles.

An empty (e.g. spare part) ECU shall by default contain the defined development key. The initial development key is used to verify software until the production key has been programmed to the ECU (WriteDataByIdentifier). Hence, a transition from that initial development key to a production key must be supported.

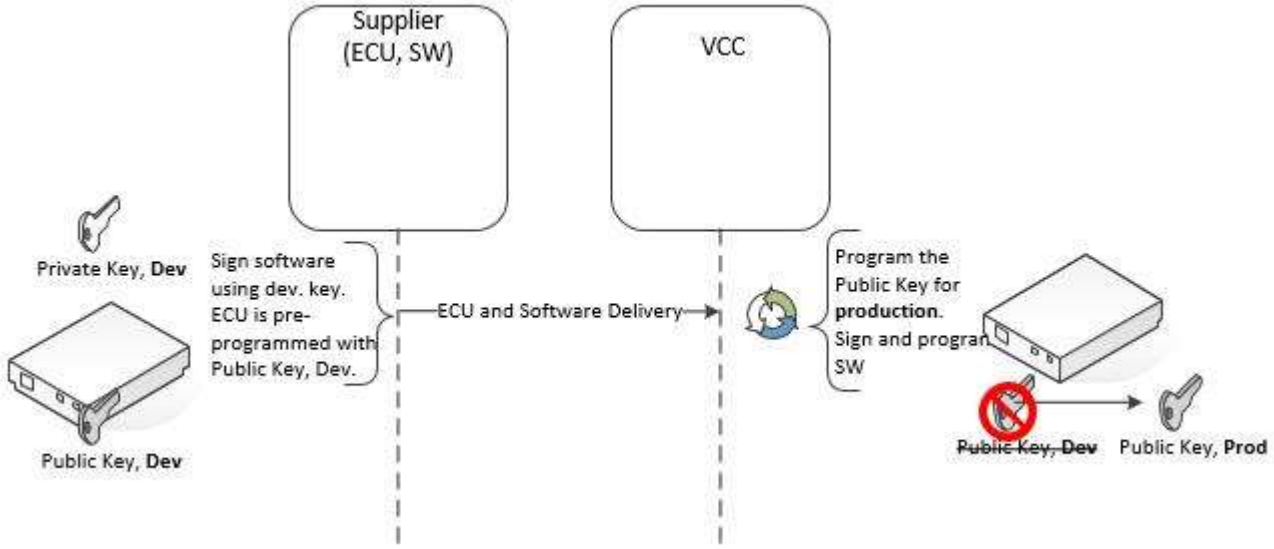


Figure - Transition from Development to Production

To simplify fault tracking, but also to ensure the integrity of the public key when it is programmed for production, a checksum of the public key will be appended when it is stored. That checksum is also possible to read once a public key has been successfully been programmed (i.e. it is not possible to read the checksum when an initial development key has been stored in the ECU by the ECU provider).

To prepare for future growth, where even more additional metadata will be appended to the public key to e.g. support a layered key structure, the term Public Key Type is used. That is used to e.g. inform the trust centre about the key structure supported by the ECU but could also be used by the diagnostic clients programming the public key (and metadata) if there is a need to store future public key types differently (e.g. using different diagnostic services).
Example - Public Key Types.

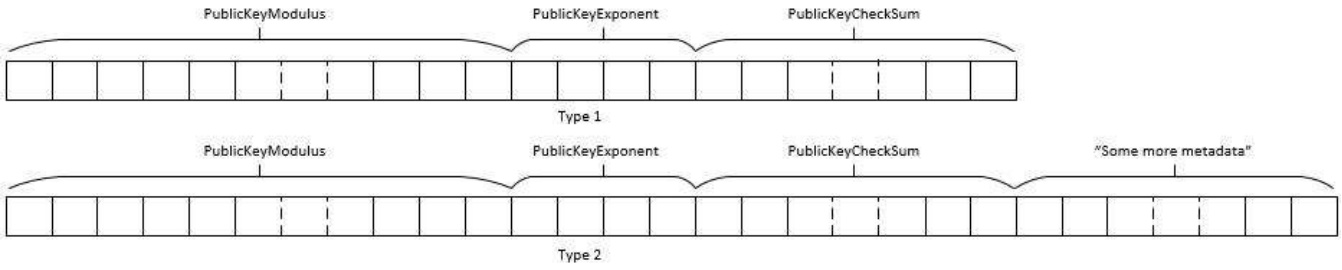


Figure - Public Key Types

The first type in *Figure - Public Key Types* is the format defined in the *Software Settings File* section by the swAuthPublicKeyType as RAW_KEY_WITH_CHECKSUM.

661309 v1 **Production Key programming**

Version history:
(1):

Purpose: To ensure that no production vehicle leaves manufacturing or workshops

equipped with a development key. A development key is to be considered as known, i.e. it adds no security value.

Source:
Verification Method: Analysis
Legacy ID: 435017v0

It must be ensured that no production vehicles leaves the plant or workshop equipped with the development key, i.e. a valid public key for production shall be programmed to the ECU. This is applicable also for the use-case when no software part at all is being programmed at End-of-Line (e.g. an ECU is pre-programmed with the software, replaced etc.).

This requirement is not applicable for the ECU supplier but for off-board systems where the vehicle is manufactured (or at workshops).

662082 v3 **Program and read of the public key**

Version history:
(3):

Purpose: To clarify the responses for write and read of public key depending whether or not the production public key is stored in an ECU
Source:
Verification Method: Test
Legacy ID: 419994

When the public key shall be stored in the ECU it shall be represented in hexadecimal format. To ensure the integrity of the public key, a checksum shall be appended. The ECU must successfully verify the checksum prior to it is stored in memory. The public key for production vehicles shall by default only be possible to write once. If it can be updated, a secure method must be agreed upon with OEM. The programmed public key shall be applied immediately, i.e. without requiring an ECU reset. There shall be different negative response codes to separate the cases:

- Trying to program a public key a second time, when a key is already stored
- The public key to be written has incorrect format, e.g. incorrect length or incorrect checksum.

The public key data (e.g. modulus part) shall not be possible to read, but the checksum of the public key shall be possible to read using the same DID as used when programming it. The public key checksum shall be calculated as Hash(PublicKeyModulus||PublicKeyExponent), where || represents concatenation. The algorithm used to calculate the checksum shall use the hash function SHA-256. Upon a request to read the DID, the ECU shall respond:

- Negative Response: Conditions Not Correct, when the initial development key is stored and no valid public key has been programmed.
- Positive Response: PublicKeyCheckSum, when a public key has been programmed.

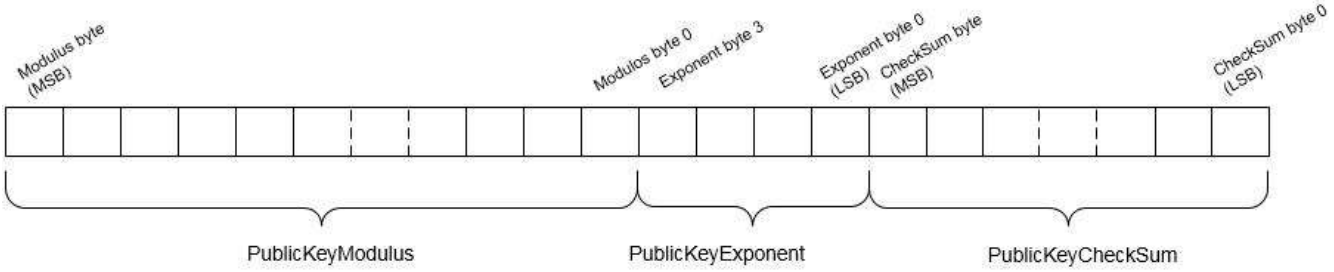


Figure - Public key DID representation

Example.

WriteDataByIDentifier_PublicKey(PublicKeyModulus || *PublicKeyExponent* || *PublicKeyCheckSum)*, where || represents concatentation.

If the initial development key is stored in the ECU and the same key is programmed again using the Public Key DID, the ECU shall respond with the PublicKeyCheckSum when reading the DID.

661313 v1 **Public Key Types**

Version history:
(1):

Purpose: To prepare the interfaces off-board of future public key types and to be able to control the diagnostic clients how to install the public key.
Source:
Verification Method: Analysis
Legacy ID: 442538v1

Information about the public key type must be provided by the entity managing the keys. That information is optionally used to control how the diagnostic client shall program the public key, e.g. what diagnostic service(s) that shall be applied.
The Public key type (PublicKeyModulus||PublicKeyExponent||PublicKeyChecksum) shall be referred to as "RAW_KEY_WITH_CHECKSUM".

This requirement is not applicable for the ECU supplier but for off-board systems.

661312 v1 **Software AUthentication Test and Development Key for RSA4096**

Version history:
(1):

Purpose: To keep track of the keys for RSA4096 used for signing and verification during test and development.
Source:
Verification Method: Analysis
Legacy ID:

For test and development only, a development key provided by the OEM shall be used. Unless otherwise is explicitly defined by the vehicle project, the following key pair shall be used:

```
-----BEGIN PUBLIC KEY-----
MIICIjANBgkqhkiG9w0BAQEFAAOCAg8AMIICCgKCAgEA1c3G1NC4Ec4sXzaEmHnY
TNrgAl0j6E8Jsr9+cJ4Xh1XTN7HmDfHhxIdkMTfNS+b79LtrZhwAAj4hgMDoSVO6
CPxkl88473mTbfKR4LPd9ChXj0g+5vOb21ToTGCRdKPyCBVCbNowxfYFd2NpKiPu
2m4hUGM8GE/zAnppz7RNB9JCHDPy2Du1k4kR+/8F3qMLF0N2VpgxhbGqCADhuLgF
QbMU9nSWo6WO3hKASQFG7OBpewIoX5q6Oyadp5Gqp/l9MWkI6B3mD9KKFti73/9F
8xiwTRiv/LeuXM0kEpmVtNeauEUdXo4EzdAuQXz0rZnA3a62Rs+uxcKNMvIQiW/l
GDD0Vs7vRVLR/XRT2BU+LGaajoth85pc6CdJaEszCfUeiVyiEoljr/etfRw8QJ+1
lEOeH8fFC4PzeJCVg2+chys9KpOYk5VnFU+2z4+S4yfeYWIM6akG91E1ktNs+d8F
ceLqx4uUxLXuKGRZKg+X87I35yDDysOqSKs3OnsTj+DENuYJsbjFLXScM4+UnKpv
1JD5eDccWeo/RJ7i62c4w5l8C17A23aD0KjgJow2wGwnS3FhnE3K/MQh48nxPwZg
0ydfccGqSvaq+Un332AIth2orbdUIuZbCQcHiAA8iiAARLSWYzbDMsFwWAVXd1Sd
2e/zcDChXVs9Uux4xMHX4z0CAwEAAQ==
-----END PUBLIC KEY-----

Private-Key: (4096 bit, 2 primes)
modulus:
    00:d5:cd:c6:d4:d0:b8:11:ce:2c:5f:36:84:98:79:
    d8:4c:da:e0:02:5d:23:e8:4f:09:b2:bf:7e:70:9e:
    17:87:55:d3:37:b1:e6:0d:f1:e1:c4:87:64:31:37:
    cd:4b:e6:fb:f4:bb:6b:66:1c:00:02:3e:21:80:c0:
    e8:49:53:ba:08:fc:64:97:cf:38:ef:79:93:6d:f2:
    91:e0:b3:dd:f4:28:57:8f:48:3e:e6:f3:9b:db:54:
    e8:4c:60:91:74:a3:f2:08:15:42:6c:da:30:c5:f6:
    05:77:63:69:2a:23:ee:da:6e:21:50:63:3c:18:4f:
    f3:02:7a:69:cf:b4:4d:07:d2:42:1c:33:f2:d8:3b:
    b5:93:89:11:fb:ff:05:de:a3:0b:17:43:76:56:98:
    31:85:b1:aa:08:00:e1:b8:b8:05:41:b3:14:f6:74:
    96:a3:a5:8e:de:12:80:49:01:46:ec:e0:69:7b:02:
    28:5f:9a:ba:3b:26:9d:a7:91:aa:a7:f9:7d:31:69:
    08:e8:1d:e6:0f:d2:8a:16:d8:bb:df:ff:45:f3:18:
    b0:4d:18:af:fc:b7:ae:5c:cd:24:12:99:95:b4:d7:
    9a:b8:45:1d:5e:8e:04:cd:d0:2e:41:7c:f4:ad:99:
    c0:dd:ae:b6:46:cf:ae:c5:c2:8d:32:f2:10:89:6f:
    e5:18:30:f4:56:ce:ef:45:52:d1:fd:74:53:d8:15:
    3e:2c:66:9a:8e:8b:61:f3:9a:5c:e8:27:49:68:4b:
    33:09:f5:1e:89:5c:a2:12:89:63:af:f7:ad:7d:1c:
    3c:40:9f:b5:94:43:9e:1f:c7:c5:0b:83:f3:78:90:
    95:83:6f:9c:87:2b:3d:2a:93:98:93:95:67:15:4f:
    b6:cf:8f:92:e3:27:de:61:62:0c:e9:a9:06:f7:51:
    35:92:d3:6c:f9:df:05:71:e2:ea:c7:8b:94:c4:b5:
    ee:28:64:59:2a:0f:97:f3:b2:37:e7:20:c3:ca:c3:
    aa:48:ab:37:3a:7b:13:8f:e0:c4:36:e6:09:b1:b8:
    c5:2d:74:9c:33:8f:94:9c:aa:6f:d4:90:f9:78:37:
    1c:59:ea:3f:44:9e:e2:eb:67:38:c3:99:7c:0b:5e:
    c0:db:76:83:d0:a8:e0:26:8c:36:c0:6c:27:4b:71:
    61:9c:4d:ca:fc:c4:21:e3:c9:f1:3f:06:60:d3:27:
    5f:71:c1:aa:4a:f6:aa:f9:49:f7:df:60:08:b6:1d:
    a8:ad:b7:54:22:e6:5b:09:07:07:88:00:3c:8a:20:
```

```
    00:44:b4:96:63:36:c3:32:c1:70:58:05:57:77:54:
    9d:d9:ef:f3:70:30:a1:5d:5b:3d:52:ec:78:c4:c1:
    d7:e3:3d
publicExponent: 65537 (0x10001)
privateExponent:
    1e:d0:7d:1f:30:03:cf:81:25:11:05:f4:ff:5a:d4:
    4b:63:62:54:c4:e2:c7:79:18:bf:3b:ed:37:c2:f9:
    46:26:9e:fe:10:74:45:3f:3e:d8:49:e1:2d:5c:cb:
    08:6c:fe:94:4b:ac:d3:77:26:a3:2c:1c:3d:07:d0:
    92:bc:ef:fb:9a:90:ce:f6:6d:50:ca:77:6e:a9:be:
    cf:a1:6c:d4:63:3a:17:6b:04:b1:d9:9b:75:75:9b:
    d3:62:39:a4:93:7a:6f:22:cd:43:8e:00:e8:8f:28:
    43:88:3e:e0:75:74:57:e3:f6:86:34:ff:34:cb:d5:
    0c:de:e2:c0:2e:4e:c9:d9:4a:70:f1:25:c7:77:77:
    98:8f:71:59:9d:ec:55:25:af:94:07:43:97:80:70:
    e6:c1:6f:ed:9e:5e:e9:f6:0f:bf:e3:d5:1a:1e:3c:
    95:d2:67:30:67:7f:38:c6:67:d7:fd:12:06:ca:0d:
    52:96:24:d0:20:a5:29:41:0b:d5:10:d6:b3:d7:dd:
    12:43:38:e4:bf:e5:ea:1c:0b:9b:e1:99:4d:cb:99:
    a1:4a:f5:16:59:cc:d6:10:34:ea:3f:5b:7d:c8:73:
    42:aa:a7:6c:78:d0:85:6f:43:33:6a:14:83:ae:bc:
    c6:d6:86:eb:d6:c3:ec:39:b9:8b:05:ab:4f:7b:a2:
    8b:62:16:4d:52:cd:59:2d:b7:e6:67:2c:2a:49:73:
    79:45:23:8e:2f:4c:5c:91:8a:f9:0f:48:41:d5:e7:
    d1:60:58:86:45:50:0a:0d:33:6c:3c:70:36:8d:65:
    6b:67:a8:51:fa:f6:cd:b4:17:fd:aa:ea:21:6e:5e:
    bf:89:ad:a9:01:18:fe:48:51:72:a3:ab:c3:99:d9:
    22:7e:38:92:66:23:89:4c:e2:d1:66:11:fd:09:7a:
    79:6c:e0:e6:a0:a1:b3:5f:7e:a3:16:e2:94:55:50:
    a4:9d:6b:69:ea:42:4a:5a:e3:e3:7e:23:cc:40:4e:
    32:20:6c:8b:b8:12:fd:88:f7:8d:38:28:2d:0e:fc:
    14:c1:50:91:98:12:ab:76:bf:8d:fd:89:63:58:59:
    6f:9e:8a:56:b7:f2:06:f3:90:c9:0a:fe:91:38:90:
    c1:a2:3b:d8:13:49:a4:a6:32:d2:2f:43:ac:6c:e4:
    65:63:33:99:b7:42:fc:27:c4:09:b4:ee:be:2b:9a:
    04:84:29:71:26:c3:e5:81:fb:2f:cd:b9:a9:a8:e7:
    43:34:19:22:4d:48:35:26:ce:f6:4e:65:61:aa:bf:
    b2:33:5a:87:5a:fa:56:bf:1d:c5:46:2c:4b:fc:b6:
    8b:fe:26:99:41:2b:82:42:62:20:55:06:0a:3c:81:
    1b:4b
prime1:
    00:eb:c2:bb:3d:1f:0e:a2:8b:84:22:d3:7a:d1:c5:
    6e:05:6d:49:df:9d:39:d0:ff:f4:8d:60:29:60:35:
    c3:94:d9:31:82:e3:3c:67:6e:b8:3d:56:76:74:c9:
    77:04:e1:ec:2a:57:fb:4b:ba:e9:15:56:d7:e2:cc:
    0c:a1:85:c7:ef:f3:37:00:e9:ad:a2:aa:a6:9f:44:
    b5:d5:56:cd:ea:b0:5f:ca:cf:37:99:79:17:65:de:
    c8:e5:2b:47:34:59:b2:2c:4a:61:5e:bf:cd:09:d1:
    72:c5:c5:08:83:44:e6:45:1b:bf:ef:2d:58:e4:a7:
    74:0c:7d:45:a6:d7:c2:ae:e3:af:a6:fc:5a:a9:f4:
    4c:18:bc:7a:e5:24:86:cb:bb:c2:70:8e:97:81:10:
    18:7e:f5:be:17:a8:3f:57:57:b1:ca:8c:02:15:a9:
```

```
    47:3a:55:bc:1e:1d:e5:7c:98:1b:08:e7:50:8f:cf:
    7c:9d:a2:88:54:eb:8c:c6:37:e6:a0:e8:a5:9b:15:
    58:60:be:cf:25:0d:fc:a7:ab:e8:a0:ba:e9:4e:e2:
    a6:27:9a:2f:7b:aa:9b:fb:5f:99:c7:39:91:89:2e:
    10:eb:49:71:30:16:e4:a5:3a:5a:21:d3:ef:82:98:
    5e:3f:59:bf:eb:2d:7a:61:96:e1:da:9c:e7:3c:2e:
    4d:8f
prime2:
    00:e8:28:80:e9:3e:13:58:3a:c3:1e:12:68:bd:0c:
    49:1d:b5:81:69:95:00:4f:8f:ce:aa:aa:67:94:d3:
    42:02:10:65:03:6f:4e:8b:4a:f7:7c:db:c6:a9:4a:
    ca:02:5d:fc:61:82:4a:f2:3e:50:5f:bc:e2:5c:d3:
    6d:47:c6:7a:19:49:d5:2e:a7:64:53:58:c8:07:df:
    84:de:74:b6:0c:4d:2d:17:cc:d8:58:b8:ae:44:8c:
    39:84:62:50:e7:e3:29:18:f8:0f:9f:1e:bc:4a:f6:
    6d:b6:c0:ec:49:4e:9a:bf:c7:d6:dc:2d:bf:db:57:
    0d:f3:ee:de:ed:3a:f5:59:4f:32:a0:23:2a:63:55:
    18:57:00:0b:42:04:56:48:36:dc:da:7e:d4:24:7c:
    00:5f:75:3c:42:3b:72:85:29:0d:42:d3:92:8b:58:
    7c:27:8a:b2:24:3c:80:d6:9c:13:0e:9c:8e:e6:ce:
    75:f0:b7:a4:34:f9:f1:85:67:ea:5f:85:90:5e:85:
    6c:90:e1:26:24:1c:ee:99:17:0d:26:71:d9:95:bc:
    da:31:ed:84:48:a5:d1:26:44:94:fa:12:dd:7b:96:
    21:53:f3:da:48:a6:52:a6:b1:45:a5:22:78:8c:34:
    1b:a8:b9:78:1e:d6:68:4c:0b:c2:85:f8:8f:69:16:
    34:73
exponent1:
    00:b3:00:5c:27:71:10:2c:8b:df:5c:8d:d2:ff:b8:
    75:eb:7a:ea:0e:f5:12:33:c4:52:8a:90:cb:79:77:
    f8:b2:68:21:b6:6e:0f:ab:bc:9d:f8:3c:50:9e:25:
    64:cf:79:35:55:85:0d:90:97:2e:67:7c:53:57:a8:
    73:a2:a3:af:85:58:d5:97:44:d5:07:df:a7:b8:df:
    73:6b:b2:87:8f:bc:30:4f:0c:fb:fa:f3:bc:b7:24:
    ea:c5:4d:11:b8:06:ae:48:00:85:63:6a:1f:24:68:
    39:4b:3d:69:54:39:98:d8:39:d4:85:a5:99:8b:aa:
    34:6b:53:25:f2:22:79:77:4d:fa:4c:bf:e7:9a:88:
    98:ca:63:38:b0:08:06:ac:63:37:a3:3c:30:7d:d6:
    99:b2:b3:a4:8b:d3:bb:9b:f6:42:50:35:f3:53:3c:
    fe:8e:a5:0f:ba:92:1a:6e:7a:12:74:b5:24:2e:bc:
    6b:b2:47:f8:58:f9:ab:1d:c8:cd:bb:b8:48:1c:b2:
    9d:32:7f:4d:49:a6:10:01:c2:7d:52:a3:f7:86:64:
    81:a1:29:e9:09:6d:25:17:3b:a3:c7:66:c7:40:2f:
    90:bb:04:3d:03:35:2e:8e:e0:fa:14:1b:15:0b:c3:
    e2:33:19:52:a5:1e:17:80:47:0d:de:6d:29:08:41:
    63:b3
exponent2:
    00:b2:1b:b8:e2:b8:7c:10:d9:6c:2e:1b:03:7d:dd:
    fa:0e:2f:dc:ec:0a:a3:25:66:f7:af:d9:72:04:2c:
    9b:5c:88:05:67:40:32:ab:0c:c5:98:65:e8:20:a3:
    3f:1d:d7:cb:33:67:c9:65:d5:c6:c5:dc:5c:71:c8:
    c6:f3:24:e3:31:99:82:52:fb:9b:2d:92:54:4c:2e:
    00:7a:c6:61:90:48:79:4a:e4:1e:2a:da:ec:df:ac:
```

```
    10:c8:6e:c1:11:56:26:86:80:71:43:ab:49:db:dc:
    f9:49:16:f3:70:3e:3c:dc:db:4d:0c:b8:90:93:8f:
    53:26:0c:e1:bc:70:da:20:3d:71:62:87:f8:12:23:
    0b:bc:5b:4d:47:c5:9a:08:67:06:c3:1e:40:55:09:
    49:23:f3:06:5a:ed:ce:ab:38:56:73:29:6c:29:39:
    a6:6e:93:aa:9d:c0:49:3b:dc:9b:86:3f:38:26:fc:
    c8:60:f8:12:a3:8e:09:ec:46:4f:fa:1e:b7:26:bd:
    08:10:92:9b:37:63:76:ed:be:50:99:69:08:fc:26:
    d9:e3:2a:0c:ad:cb:81:0c:f1:da:b5:8c:59:9d:62:
    0d:89:a1:f0:9a:81:f7:b0:e9:19:bc:5a:18:cf:bf:
    7e:64:d0:7d:c6:72:c9:ab:65:ee:43:e7:f0:e6:37:
    61:df
coefficient:
    00:ea:de:45:8c:02:8d:2a:c1:38:ac:8a:f8:3a:69:
    e5:9e:50:9e:9b:0d:7a:a5:38:42:1c:f6:f3:8e:ca:
    60:29:6f:f2:3d:f9:92:a3:94:7c:50:18:ed:6e:b3:
    b0:ef:1b:a0:4a:11:88:91:f8:37:22:fa:53:2d:3a:
    6a:95:e8:d2:00:fc:8a:c3:40:46:ef:75:66:11:f3:
    4f:2e:76:7d:92:50:b8:26:9d:f4:97:eb:08:e1:96:
    6c:3f:25:cd:06:52:97:5d:f3:4b:c3:fb:df:66:4b:
    0e:fa:c9:38:35:14:16:3d:87:64:47:5d:72:44:af:
    62:25:d0:9a:2b:1d:c3:72:78:45:68:81:b0:03:84:
    25:bd:d7:21:a3:46:8a:6e:ca:48:a0:32:0f:90:b3:
    93:a0:fa:fd:dd:8a:44:a2:d6:f4:07:87:73:fc:ef:
    65:09:b4:3a:19:46:9f:1d:50:dd:f3:b9:d0:46:26:
    c2:91:fd:5a:80:c2:e4:1d:07:42:37:7b:93:05:cb:
    2a:c7:5d:6d:11:aa:05:3a:1c:5e:99:75:4a:b2:4a:
    ec:88:04:e0:da:a4:03:c4:39:de:45:5c:42:25:e9:
    44:dd:d5:40:bf:49:23:01:ee:a8:ba:c8:bd:65:50:
    bd:ac:70:a9:0a:0e:70:97:02:33:b7:0e:9d:23:c6:
    f4:ed

-----BEGIN RSA PRIVATE KEY-----
MIIJKgIBAAKCAgEA1c3G1NC4Ec4sXzaEmHnYTNrgAl0j6E8Jsr9+cJ4Xh1XTN7Hm
DfHhxIdkMTfNS+b79LtrZhwAAj4hgMDoSVO6CPxkl88473mTbfKR4LPd9ChXj0g+
5vOb21ToTGCRdKPyCBVCbNowxfYFd2NpKiPu2m4hUGM8GE/zAnppz7RNB9JCHDPy
2Du1k4kR+/8F3qMLF0N2VpgxhbGqCADhuLgFQbMU9nSWo6WO3hKASQFG7OBpewIo
X5q6Oyadp5Gqp/l9MWkI6B3mD9KKFti73/9F8xiwTRiv/LeuXM0kEpmVtNeauEUd
Xo4EzdAuQXz0rZnA3a62Rs+uxcKNMvIQiW/lGDD0Vs7vRVLR/XRT2BU+LGaajoth
85pc6CdJaEszCfUeiVyiEoljr/etfRw8QJ+1lEOeH8fFC4PzeJCVg2+chys9KpOY
k5VnFU+2z4+S4yfeYWIM6akG91E1ktNs+d8FceLqx4uUxLXuKGRZKg+X87I35yDD
ysOqSKs3OnsTj+DENuYJsbjFLXScM4+UnKpv1JD5eDccWeo/RJ7i62c4w5l8C17A
23aD0KjgJow2wGwnS3FhnE3K/MQh48nxPwZg0ydfccGqSvaq+Un332AIth2orbdU
IuZbCQcHiAA8iiAARLSWYzbDMsFwWAVXd1Sd2e/zcDChXVs9Uux4xMHX4z0CAwEA
AQKCAgAe0H0fMAPPgSURBfT/WtRLY2JUxOLHeRi/O+03wvlGJp7+EHRFPz7YSeEt
XMsIbP6US6zTdyajLBw9B9CSvO/7mpDO9m1Qynduqb7PoWzUYzoXawSx2Zt1dZvT
Yjmkk3pvIs1DjgDojyhDiD7gdXRX4/aGNP80y9UM3uLALk7J2Upw8SXHd3eYj3FZ
nexVJa+UB0OXgHDmwW/tnl7p9g+/49UaHjyV0mcwZ384xmfX/RIGyg1SliTQIKUp
QQvVENaz190SQzjkv+XqHAub4ZlNy5mhSvUWWczWEDTqP1t9yHNCqqdseNCFb0Mz
ahSDrrzG1obr1sPsObmLBatPe6KLYhZNUs1ZLbfmZywqSXN5RSOOL0xckYr5D0hB
1efRYFiGRVAKDTNsPHA2jWVrZ6hR+vbNtBf9quohbl6/ia2pARj+SFFyo6vDmdki
fjiSZiOJTOLRZhH9CXp5bODmoKGzX36jFuKUVVCknWtp6kJKWuPjfiPMQE4yIGyL
```

```
uBL9iPeNOCgtDvwUwVCRmBKrdr+N/YljWFlvnopWt/IG85DJCv6ROJDBojvYE0mk
pjLSL0OsbORlYzOZt0L8J8QJtO6+K5oEhClxJsPlgfsvzbmpqOdDNBkiTUg1Js72
TmVhqr+yM1qHWvpWvx3FRixL/LaL/iaZQSuCQmIgVQYKPIEbSwKCAQEA68K7PR8O
oouEItN60cVuBW1J35050P/0jWApYDXDlNkxguM8Z264PVZ2dMl3BOHsKlf7S7rp
FVbX4swMoYXH7/M3AOmtoqqmn0S11VbN6rBfys83mXkXZd7I5StHNFmyLEphXr/N
CdFyxcUIg0TmRRu/7y1Y5Kd0DH1FptfCruOvpvxaqfRMGLx65SSGy7vCcI6XgRAY
fvW+F6g/V1exyowCFalHOlW8Hh3lfJgbCOdQj898naKIVOuMxjfmoOilmxVYYL7P
JQ38p6vooLrpTuKmJ5ove6qb+1+ZxzmRiS4Q60lxMBbkpTpaIdPvgpheP1m/6y16
YZbh2pznPC5NjwKCAQEA6CiA6T4TWDrDHhJovQxJHbWBaZUAT4/OqqpnlNNCAhBl
A29Oi0r3fNvGqUrKAl38YYJK8j5QX7ziXNNtR8Z6GUnVLqdkU1jIB9+E3nS2DE0t
F8zYWLiuRIw5hGJQ5+MpGPgPnx68SvZttsDsSU6av8fW3C2/21cN8+7e7Tr1WU8y
oCMqY1UYVwALQgRWSDbc2n7UJHwAX3U8QjtyhSkNQtOSi1h8J4qyJDyA1pwTDpyO
5s518LekNPnxhWfqX4WQXoVskOEmJBzumRcNJnHZlbzaMe2ESKXRJkSU+hLde5Yh
U/PaSKZSprFFpSJ4jDQbqLl4HtZoTAvChfiPaRY0cwKCAQEAswBcJ3EQLIvfXI3S
/7h163rqDvUSM8RSipDLeXf4smghtm4Pq7yd+DxQniVkz3k1VYUNkJcuZ3xTV6hz
oqOvhVjVl0TVB9+nuN9za7KHj7wwTwz7+vO8tyTqxU0RuAauSACFY2ofJGg5Sz1p
VDmY2DnUhaWZi6o0a1Ml8iJ5d036TL/nmoiYymM4sAgGrGM3ozwwfdaZsrOki9O7
m/ZCUDXzUzz+jqUPupIabnoSdLUkLrxrskf4WPmrHcjNu7hIHLKdMn9NSaYQAcJ9
UqP3hmSBoSnpCW0lFzujx2bHQC+QuwQ9AzUujuD6FBsVC8PiMxlSpR4XgEcN3m0p
CEFjswKCAQEAshu44rh8ENlsLhsDfd36Di/c7AqjJWb3r9lyBCybXIgFZ0AyqwzF
mGXoIKM/HdfLM2fJZdXGxdxcccjG8yTjMZmCUvubLZJUTC4AesZhkEh5SuQeKtrs
36wQyG7BEVYmhoBxQ6tJ29z5SRbzcD483NtNDLiQk49TJgzhvHDaID1xYof4EiML
vFtNR8WaCGcGwx5AVQlJI/MGWu3OqzhWcylsKTmmbpOqncBJO9ybhj84JvzIYPgS
o44J7EZP+h63Jr0IEJKbN2N27b5QmWkI/CbZ4yoMrcuBDPHatYxZnWINiaHwmoH3
sOkZvFoYz79+ZNB9xnLJq2XuQ+fw5jdh3wKCAQEA6t5FjAKNKsE4rIr4OmnlnlCe
mw16pThCHPbzjspgKW/yPfmSo5R8UBjtbrOw7xugShGIkfg3IvpTLTpqlejSAPyK
w0BG73VmEfNPLnZ9klC4Jp30l+sI4ZZsPyXNBlKXXfNLw/vfZksO+sk4NRQWPYdk
R11yRK9iJdCaKx3DcnhFaIGwA4Qlvdcho0aKbspIoDIPkLOToPr93YpEotb0B4dz
/O9lCbQ6GUafHVDd87nQRibCkf1agMLkHQdCN3uTBcsqx11tEaoFOhxemXVKskrs
iATg2qQDxDneRVxCJelE3dVAv0kjAe6ousi9ZVC9rHCpCg5wlwIztw6dI8b07Q==
-----END RSA PRIVATE KEY-----
```

661310 v1 **Software Authentication Test and Development Key**

Version history:
(1):

Purpose: To keep track of the keys used for signing and verification during test and development

Source:

Verification Method: Analysis

Legacy ID: 420232v2

For test and development only, a development key provided by the OEM shall be used. Unless otherwise is explicitly defined by the vehicle project, the following key pair shall be used:

```
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAt+l0S0X6piDTHDDpY4bp
zV+5k97KRcnWCJT3fbnuqdB4RXaUgJ33BSTXMOLADwRuYFMjvVADvyypu7RcxRFa
Hc4lfUIDT34cej4aaOiaABCNGCisJr1xrkrJuSMLm8EBZ0apAV5w8dm9f1ZLl2Fk
/8HZbpOrQGbVy/QC9fxTEVGpgFwHFqvLmCX+AvOJfleRemTMLHpx6IMzWQqpWSPP
```

```
SmvkJBr3jKkEXWW2dIcZQknjaQPdpMl1/qc8B8GRZ1RF/l/PRXL4vUeVuoGnVFBV
KZIvgYJxm0Mc6ycWyofiuoOgHoXvdeRjiC0LU3a2s9ZoGeJsK2dPCp3e/pNCQ86H
rQIDAQAB
-----END PUBLIC KEY-----


Private-Key: (2048 bit)
modulus:
    00:b7:e9:74:4b:45:fa:a6:20:d3:1c:30:e9:63:86:
    e9:cd:5f:b9:93:de:ca:45:c9:d6:08:94:f7:7d:b9:
    ee:a9:d0:78:45:76:94:80:9d:f7:05:24:d7:30:e2:
    c0:0f:04:6e:60:53:23:bd:50:03:bf:2c:a9:bb:b4:
    5c:c5:11:5a:1d:ce:25:7d:42:03:4f:7e:1c:7a:3e:
    1a:68:e8:9a:00:10:8d:18:28:ac:26:bd:71:ae:4a:
    c9:b9:23:0b:9b:c1:01:67:46:a9:01:5e:70:f1:d9:
    bd:7f:56:4b:97:61:64:ff:c1:d9:6e:93:ab:40:66:
    d5:cb:f4:02:f5:fc:53:11:51:a9:80:5c:07:16:ab:
    cb:98:25:fe:02:f3:89:7e:57:91:7a:64:cc:2c:7a:
    71:e8:83:33:59:0a:a9:59:23:cf:4a:6b:e4:24:1a:
    f7:8c:a9:04:5d:65:b6:74:87:19:42:49:e3:69:03:
    dd:a4:c9:75:fe:a7:3c:07:c1:91:67:54:45:fe:5f:
    cf:45:72:f8:bd:47:95:ba:81:a7:54:50:55:29:92:
    2f:81:82:71:9b:43:1c:eb:27:16:ca:87:e2:ba:83:
    a0:1e:85:ef:75:e4:63:88:2d:0b:53:76:b6:b3:d6:
    68:19:e2:6c:2b:67:4f:0a:9d:de:fe:93:42:43:ce:
    87:ad
publicExponent: 65537 (0x10001)
privateExponent:
    67:bf:c8:3e:2a:95:12:a0:d3:d7:44:74:75:14:07:
    d3:36:dc:2e:e1:f1:03:db:af:e5:99:7b:e0:ae:42:
    48:03:f5:c5:61:f6:b6:73:e6:85:3d:5a:34:16:c6:
    b7:f2:0c:fe:44:08:96:64:8c:28:8d:de:96:a8:51:
    e9:4e:37:a3:36:c7:09:59:73:1a:a6:0f:14:9a:f2:
    35:1a:7a:bd:ec:98:5b:f7:9d:de:20:e2:ff:aa:eb:
    0f:89:08:a4:6e:06:07:a7:e1:f1:86:c0:7a:7f:16:
    1a:be:a8:d8:16:36:6e:dd:81:76:92:d1:79:fc:49:
    41:cc:3e:db:5b:e3:d4:91:62:fa:26:04:1b:1f:5e:
    2b:d4:7c:b2:68:5c:ec:40:f2:c6:e2:83:7d:71:e0:
    f9:fa:41:24:8d:36:1c:e8:c4:05:38:ab:23:37:d1:
    c8:c6:ad:e5:de:98:d5:0a:7e:aa:ba:54:5a:63:d5:
    e1:49:be:84:0d:de:66:71:78:c8:df:5c:53:88:ea:
    00:c1:8d:af:81:9f:0a:c4:a0:6e:b0:f7:da:a1:22:
    72:46:65:68:3b:24:b7:89:d0:ce:de:3a:f3:d5:07:
    94:d7:17:9c:c7:90:77:7b:6c:7a:21:72:e5:17:25:
    8a:84:20:01:1b:9b:e5:96:5c:17:0b:de:85:38:cf:
    95
prime1:
    00:df:d1:18:63:6f:2c:1b:14:3b:95:3d:56:dd:5b:
    6f:01:6e:f9:4c:2b:cb:eb:dd:89:b0:8e:dd:f3:f0:
    0f:f7:71:75:d0:7e:b0:be:6b:51:3a:8c:cf:9b:81:
    fa:34:4f:fc:98:d6:65:b1:2f:82:55:a8:d1:b5:1a:
```

```
    af:7b:da:6f:02:43:14:7e:69:52:11:9e:a9:95:44:
    b1:19:7f:7b:7c:67:30:11:67:88:9d:b1:b5:6e:6a:
    69:17:64:57:dc:5e:fd:33:ae:84:dc:2a:73:54:8d:
    be:04:c7:6f:6e:7f:31:f1:04:b3:6b:f7:9f:62:15:
    b0:75:12:a8:4f:02:4a:cb:d7
prime2:
    00:d2:5b:6c:9d:a4:9b:a9:6e:29:67:3f:d1:c2:53:
    1a:50:5b:0a:7c:c1:64:01:ff:20:20:8c:68:32:b1:
    1a:fb:8e:73:54:29:3b:e7:fc:94:0f:63:06:a2:87:
    56:a4:e3:48:75:21:b7:10:97:fc:3a:1d:b3:e2:ab:
    e6:4b:04:8d:de:fb:ac:d3:e4:3d:71:20:da:04:aa:
    e2:98:f4:e3:6c:f4:fa:c3:a7:1f:73:30:4f:3c:fe:
    8a:3e:21:06:e7:02:b4:e5:9d:4e:37:64:67:15:f3:
    64:73:cf:30:2d:54:19:76:22:ee:44:b5:cb:8f:94:
    a0:f5:15:91:68:56:cc:38:1b
exponent1:
    7e:33:cf:06:b2:77:32:45:b4:5b:30:9d:3c:70:04:
    25:d0:c7:6d:b5:fc:64:61:24:f4:93:7a:7f:c4:4b:
    9c:81:33:a7:7e:e8:76:56:d9:14:a4:b5:a3:c0:24:
    af:3e:b2:f6:13:5e:80:0c:83:f7:7d:1b:d2:7c:db:
    9a:80:ce:bb:7d:cb:9e:84:10:ac:b2:c4:78:d0:a4:
    f3:f5:b8:51:ab:75:a5:3a:b6:04:05:62:82:82:2a:
    03:f0:a6:c2:32:25:9f:f0:b6:25:d7:21:f4:f9:7f:
    bd:fe:1e:cd:35:97:99:89:c7:0a:08:34:ad:00:01:
    e1:e1:c5:59:d7:b7:09:3d
exponent2:
    00:87:f8:a4:9a:b9:9e:0c:c4:b2:6a:94:ec:07:4a:
    24:46:30:b2:f4:b5:24:e9:cd:79:7c:d0:85:41:cf:
    0c:fb:f1:b6:46:7e:68:c4:a9:95:22:e5:05:92:e5:
    1c:72:74:9f:8f:66:fd:a7:f2:36:0d:72:c9:a6:09:
    2b:50:ee:5e:ad:f5:cc:5f:22:b7:3c:7a:d9:b2:0e:
    ab:6d:e7:4d:62:4e:70:11:2b:e3:be:57:49:c0:c9:
    5f:9e:8d:46:a2:e8:32:fa:00:d6:60:23:bc:26:8a:
    2f:32:54:88:75:a4:58:d8:ed:f7:49:de:a0:f7:ec:
    40:a6:6b:0c:94:7f:16:7e:65
coefficient:
    46:53:1a:19:21:2d:fe:36:73:85:3a:71:a2:84:fa:
    5c:35:76:db:10:f0:34:86:14:a9:70:1c:d8:73:df:
    2b:d0:43:d9:73:d2:26:1f:b2:ba:9d:82:57:9b:4c:
    cd:4a:bc:f2:1f:fa:f5:5b:2a:8d:6c:20:39:db:f8:
    55:19:11:15:b4:a9:a1:00:dc:93:31:53:c5:c6:17:
    cb:36:e9:00:d8:6e:c8:75:69:6a:85:4d:42:01:2a:
    86:36:59:fc:d7:0b:57:8a:76:7d:c0:bc:43:8a:4d:
    b1:14:07:4d:28:5a:ec:30:28:93:6e:68:14:1a:c0:
    17:6c:e1:7c:39:19:e6:d8
-----BEGIN RSA PRIVATE KEY-----
MIIEowIBAAKCAQEAt+l0S0X6piDTHDDpY4bpzV+5k97KRcnWCJT3fbnuqdB4RXaU
gJ33BSTXMOLADwRuYFMjvVADvyypu7RcxRFaHc4lfUIDT34cej4aaOiaABCNGCis
Jr1xrkrJuSMLm8EBZ0apAV5w8dm9f1ZLl2Fk/8HZbpOrQGbVy/QC9fxTEVGpgFwH
FqvLmCX+AvOJfleRemTMLHpx6IMzWQqpWSPPSmvkJBr3jKkEXWW2dIcZQknjaQPd
pMl1/qc8B8GRZ1RF/l/PRXL4vUeVuoGnVFBVKZIvgYJxm0Mc6ycWyofiuoOgHoXv
```

```
deRjiC0LU3a2s9ZoGeJsK2dPCp3e/pNCQ86HrQIDAQABAoIBAGe/yD4qlRKg09dE
dHUUB9M23C7h8QPbr+WZe+CuQkgD9cVh9rZz5oU9WjQWxrfyDP5ECJZkjCiN3pao
UelON6M2xwlZcxqmDxSa8jUaer3smFv3nd4g4v+q6w+JCKRuBgen4fGGwHp/Fhq+
qNgWNm7dgXaS0Xn8SUHMPttb49SRYvomBBsfXivUfLJoXOxA8sbig31x4Pn6QSSN
NhzoxAU4qyM30cjGreXemNUKfqq6VFpj1eFJvoQN3mZxeMjfXFOI6gDBja+BnwrE
oG6w99qhInJGZWg7JLeJ0M7eOvPVB5TXF5zHkHd7bHohcuUXJYqEIAEbm+WWXBcL
3oU4z5UCgYEA39EYY28sGxQ7lT1W3VtvAW75TCvL692JsI7d8/AP93F10H6wvmtR
OozPm4H6NE/8mNZlsS+CVajRtRqve9pvAkMUfmlSEZ6plUSxGX97fGcwEWeInbG1
bmppF2RX3F79M66E3CpzVI2+BMdvbn8x8QSza/efYhWwdRKoTwJKy9cCgYEA0lts
naSbqW4pZz/RwlMaUFsKfMFkAf8gIIxoMrEa+45zVCk75/yUD2MGoodWpONIdSG3
EJf8Oh2z4qvmSwSN3vus0+Q9cSDaBKrimPTjbPT6w6cfczBPPP6KPiEG5wK05Z1O
N2RnFfNkc88wLVQZdiLuRLXLj5Sg9RWRaFbMOBsCgYB+M88GsncyRbRbMJ08cAQl
0MdttfxkYST0k3p/xEucgTOnfuh2VtkUpLWjwCSvPrL2E16ADIP3fRvSfNuagM67
fcuehBCsssR40KTz9bhRq3WlOrYEBWKCgioD8KbCMiWf8LYl1yH0+X+9/h7NNZeZ
iccKCDStAAHh4cVZ17cJPQKBgQCH+KSauZ4MxLJqlOwHSiRGMLL0tSTpzXl80IVB
zwz78bZGfmjEqZUi5QWS5RxydJ+PZv2n8jYNcsmmCStQ7l6t9cxfIrc8etmyDqtt
501iTnARK+O+V0nAyV+ejUai6DL6ANZgI7wmii8yVIh1pFjY7fdJ3qD37ECmawyU
fxZ+ZQKBgEZTGhkhLf42c4U6caKE+lw1dtsQ8DSGFKlwHNhz3yvQQ9lz0iYfsrqd
glebTM1KvPIf+vVbKo1sIDnb+FUZERW0qaEA3JMxU8XGF8s26QDYbsh1aWqFTUIB
KoY2WfzXC1eKdn3AvEOKTbEUB00oWuwwKJNuaBQawBds4Xw5GebY
-----END RSA PRIVATE KEY-----
```

661311 v1 **Software Authentication Test and Development Key for RSA3072**

Version history:
(1):

Purpose: To keep track of the keys used for signing and verification during test and development.

Source:

Verification Method: Analysis

Legacy ID:

For test and development only, a development key provided by the OEM shall be used. Unless otherwise is explicitly defined by the vehicle project, the following key pair shall be used:

```
-----BEGIN PUBLIC KEY-----
MIIBojANBgkqhkiG9w0BAQEFAAOCAY8AMIIBigKCAYEAqGGC9HYwODLUE2I0VxY4
05Rv+Kn5DwdBmTgqWiaXdJzfIR+AdgGoqnqSHSfINfxdql0nMT0oeNO4B1OIZOaj
mvJJ9K7bSo7QF+bt+VWWtjlAWQN8uStnyakLG8MraZ30NX4WqJ1FQa2Hlu0R7L17
rjaN+Gvd8SokRt0gwGIqefa4wpBLV7Fp/Dy5qQPeRquP549BF0r7T0sNxpZoqLGg
SxcDrpJwaGKg3eqv0JIKfVdmFOKkWSy28CpN5VxJ5bjcf/rzyLVtVuXY+MmWQwp5
bVGrj1XcSjwkhj/6LgCeSptARo3Hv8tbcnJZNLTD/WO6hQuYeu0+O4A+MVZCKzo4
16LwWoMWgrAi8NOZlz/ayUQMaUZd2ICIPuY+jy/jG5Ekrot+N9BLePNFw+ASAuUv
0HQzNjynSd9pMjlzuzLlJKGsiCl+AUtVTMJE1PrwGwYJsZ8T7CmF1xKm3Re2GyQ6
WYriY5yjrvFl/OSTm240X4Ke1N7B+3R1tvXX8gASYtsTAgMBAAE=
-----END PUBLIC KEY-----

Private-Key: (3072 bit)
modulus:
    00:a8:61:82:f4:76:30:38:32:d4:13:62:34:57:16:
```

```
    38:d3:94:6f:f8:a9:f9:0f:07:41:99:38:2a:5a:26:
    97:74:9c:df:21:1f:80:76:01:a8:aa:7a:92:1d:27:
    c8:35:fc:5d:aa:5d:27:31:3d:28:78:d3:b8:07:53:
    88:64:e6:a3:9a:f2:49:f4:ae:db:4a:8e:d0:17:e6:
    ed:f9:55:96:b6:39:40:59:03:7c:b9:2b:67:c9:a9:
    0b:1b:c3:2b:69:9d:f4:35:7e:16:a8:9d:45:41:ad:
    87:96:ed:11:ec:bd:7b:ae:36:8d:f8:6b:dd:f1:2a:
    24:46:dd:20:c0:62:2a:79:f6:b8:c2:90:4b:57:b1:
    69:fc:3c:b9:a9:03:de:46:ab:8f:e7:8f:41:17:4a:
    fb:4f:4b:0d:c6:96:68:a8:b1:a0:4b:17:03:ae:92:
    70:68:62:a0:dd:ea:af:d0:92:0a:7d:57:66:14:e2:
    a4:59:2c:b6:f0:2a:4d:e5:5c:49:e5:b8:dc:7f:fa:
    f3:c8:b5:6d:56:e5:d8:f8:c9:96:43:0a:79:6d:51:
    ab:8f:55:dc:4a:3c:24:86:3f:fa:2e:00:9e:4a:9b:
    40:46:8d:c7:bf:cb:5b:72:72:59:34:b4:c3:fd:63:
    ba:85:0b:98:7a:ed:3e:3b:80:3e:31:56:42:2b:3a:
    38:d7:a2:f0:5a:83:16:82:b0:22:f0:d3:99:97:3f:
    da:c9:44:0c:69:46:5d:d8:80:88:3e:e6:3e:8f:2f:
    e3:1b:91:24:ae:8b:7e:37:d0:4b:78:f3:45:c3:e0:
    12:02:e5:2f:d0:74:33:36:3c:a7:49:df:69:32:39:
    73:bb:32:e5:24:a1:ac:88:29:7e:01:4b:55:4c:c2:
    44:d4:fa:f0:1b:06:09:b1:9f:13:ec:29:85:d7:12:
    a6:dd:17:b6:1b:24:3a:59:8a:e2:63:9c:a3:ae:f1:
    65:fc:e4:93:9b:6e:34:5f:82:9e:d4:de:c1:fb:74:
    75:b6:f5:d7:f2:00:12:62:db:13
publicExponent: 65537 (0x10001)
privateExponent:
    4d:14:7e:6f:4c:5f:5c:8a:a0:1c:d3:7a:e4:56:ee:
    e2:a4:73:f1:a0:55:bc:34:75:2a:1f:d4:fb:79:e9:
    21:5f:e2:a0:f6:1b:c7:69:90:fd:a1:a8:9b:a2:27:
    7b:d7:30:45:99:e9:24:18:3c:1b:bb:c7:e5:ca:ae:
    88:8a:30:60:f1:03:3b:f8:83:b8:88:d1:e7:ef:36:
    07:4a:cf:f8:51:13:0c:ec:94:be:eb:88:d2:7f:08:
    ef:c7:93:6b:00:fc:d7:34:06:49:5f:dd:61:68:7f:
    c6:6e:76:23:2a:c6:a0:84:a6:cf:0f:0a:05:a9:65:
    fb:ee:a4:5c:8d:53:a6:8b:74:1e:d5:11:38:6e:a5:
    f6:eb:11:8e:b6:8f:9d:48:52:53:0d:78:1d:a2:9c:
    92:ba:e7:ee:7d:d2:21:4f:56:f2:a8:f1:6d:8b:e9:
    db:4f:33:17:03:a5:42:54:68:6a:e2:84:7f:91:14:
    ed:21:94:09:33:44:7f:77:94:f8:8d:7b:dc:d5:fe:
    8c:bb:0d:6c:ff:e0:05:49:93:96:a9:59:28:9a:ca:
    ed:61:49:36:a2:7f:2e:92:76:04:b7:d5:11:35:9f:
    3b:c8:71:52:4e:d1:61:9b:14:9c:12:78:ed:db:fa:
    32:ee:a1:05:1d:b4:fe:9b:74:78:8d:2b:55:95:51:
    b1:a4:3f:03:a3:e4:19:dd:2c:ab:6e:ad:fb:e6:96:
    52:87:37:42:be:5a:f6:5e:b9:0a:f8:99:11:a7:12:
    80:95:90:71:f7:9a:e3:6c:bb:14:2f:91:cc:91:8c:
    2d:4c:ce:d0:05:60:fd:a0:04:e2:2c:e9:17:8a:9e:
    57:39:62:87:56:d0:b6:fd:47:b9:54:1e:49:59:bd:
    db:fc:8d:9e:58:33:ab:56:1b:97:6f:1e:3e:a1:fe:
    59:7c:5c:58:40:44:f8:3b:67:f9:97:c5:b1:f5:7e:
    d9:26:78:9d:18:52:f5:35:53:ee:70:e8:79:bd:ab:
```

```
    64:e3:a4:89:3e:58:b9:f0:91
prime1:
    00:db:65:43:5c:88:1c:c7:e2:5b:41:4a:c6:6c:c2:
    e5:1d:69:c8:29:81:43:50:9c:f5:8f:69:e2:c2:f6:
    b2:bb:ef:e3:52:9d:3a:3c:f0:90:3e:d5:78:15:2c:
    6f:32:32:c7:fe:48:cc:7a:4e:e4:5a:bb:fa:38:e6:
    99:c5:b6:17:3c:6f:6d:4c:13:c6:01:75:f9:e0:e2:
    fd:a9:55:b2:ab:01:2a:2b:1c:c7:75:2c:87:1d:2e:
    eb:2a:a2:17:a5:f9:6b:12:f5:97:5b:c9:d7:f1:8b:
    b8:94:7c:95:ea:4b:40:49:93:7f:e7:66:46:f3:97:
    21:b2:1b:27:8c:5d:a7:58:38:2b:00:42:91:17:f9:
    a4:47:e9:44:fd:38:46:16:28:c6:86:5b:4c:0c:5c:
    22:b8:31:59:52:91:1b:59:97:f3:80:92:e9:3e:bb:
    63:46:60:41:a2:72:da:36:6c:33:5f:48:87:f1:c6:
    65:40:8b:57:c9:ea:f0:13:c9:b9:06:fd:e9
prime2:
    00:c4:79:54:9e:2a:c0:e0:f4:35:4d:ce:f6:be:e6:
    4b:ab:b2:3c:88:53:e7:d4:65:03:e5:b6:a8:c2:90:
    41:fc:fa:90:25:d1:6e:8e:2d:44:b1:09:41:7c:4f:
    ca:8e:34:cf:f5:b5:87:45:58:97:ab:f1:79:12:51:
    e9:9d:e3:6f:3d:82:44:31:c3:30:21:eb:51:aa:76:
    3d:db:27:76:fa:cb:17:10:0f:c9:38:c2:6b:26:1f:
    4a:c5:b3:91:ba:48:7c:a4:a3:81:34:7b:66:a5:0d:
    22:ac:a6:9b:9e:54:d5:85:04:11:df:05:2f:2d:2e:
    2e:d6:0a:43:53:b4:e5:0c:70:f2:43:3c:95:26:ff:
    51:13:ef:10:66:35:25:9a:a1:1f:0f:a2:fc:0e:c7:
    79:fd:c7:42:9b:2f:4d:e9:25:04:35:6b:c8:30:a1:
    ce:50:bb:86:fd:65:ab:44:7d:29:34:3d:ef:c6:08:
    70:b5:7f:da:20:48:82:be:6b:4e:2f:c7:9b
exponent1:
    23:d1:0d:45:2e:e7:5b:ee:65:2d:6c:92:a6:30:cf:
    0c:95:37:5b:27:42:21:07:ee:c2:c1:eb:86:1e:96:
    21:3e:51:63:2c:4d:d4:a3:05:39:6c:ca:14:d2:d3:
    38:63:b7:e8:98:63:e1:57:bd:c9:de:44:44:43:c9:
    38:d0:b8:4c:62:78:7a:62:04:02:3b:0b:57:1e:57:
    80:ec:7f:a5:91:24:1c:33:af:90:3e:7b:b4:54:f3:
    89:5b:3a:f7:1a:97:d3:59:8a:a3:2e:79:75:72:74:
    e8:b0:ca:1c:9c:ba:5b:08:9c:69:df:75:97:d0:c5:
    6c:79:28:64:ae:e9:61:ea:91:82:92:5b:73:89:8a:
    bb:3e:a6:c7:9c:07:3d:18:dd:08:14:21:7f:27:0f:
    77:f8:7e:77:8f:3e:e2:ca:7e:b3:2f:c8:aa:38:17:
    45:e6:f2:f0:1d:45:e8:7a:93:8c:6b:9f:dd:e5:a0:
    a3:ff:db:8e:18:2f:a7:5b:58:3c:7e:f9
exponent2:
    59:33:e9:1a:42:b8:fd:a9:82:c6:07:f0:79:6d:df:
    ef:5d:66:24:53:4c:0f:5f:a9:ac:53:09:3e:04:f7:
    20:df:b6:30:a5:6c:b7:7a:90:14:7c:ea:51:81:7c:
    aa:a2:09:c5:3a:1c:e7:7c:38:a8:a5:62:dd:d1:78:
    ea:e9:86:e8:fb:4a:f0:f8:f8:d2:7d:9f:bf:70:1d:
    d7:71:8c:90:83:42:be:55:1c:7b:fb:b1:60:46:96:
    d1:3a:1d:30:e1:96:4d:bd:ba:7c:d4:86:1e:eb:de:
```

```
    d1:5a:77:8f:24:5f:29:1c:56:ec:d5:d2:bc:85:9a:
    58:2d:a2:b1:cf:90:aa:dc:60:f1:ec:00:a0:0b:0d:
    89:aa:61:dc:b5:c6:93:f5:11:40:2d:34:99:95:31:
    aa:ea:1c:30:8f:02:a5:96:db:d9:51:4d:3b:df:4a:
    e7:d6:ac:8c:41:eb:c7:95:b7:ee:73:2c:78:d1:fe:
    32:39:91:14:55:37:9f:32:e2:f5:c8:7f
coefficient:
    7b:cc:48:3f:3c:02:be:6c:96:d4:b1:ab:eb:97:a1:
    d9:16:0b:d8:bd:44:89:7d:1d:e6:7a:5e:23:87:cf:
    1b:45:18:24:c4:c7:49:79:40:98:07:a7:36:55:2b:
    a9:8e:b6:9a:c3:a0:4f:84:1e:12:4f:04:94:c5:a8:
    b9:dd:f7:0d:82:f6:b7:2c:b4:60:31:e0:f5:87:f7:
    7c:51:51:5d:c7:dc:36:3d:b9:6b:03:ab:94:2a:57:
    72:5c:4e:f2:8d:da:14:d7:31:2b:73:fd:8c:58:5f:
    53:8f:82:e3:7a:98:ef:8a:07:eb:70:15:28:2b:cb:
    dc:4d:56:3d:1b:86:63:49:1b:78:66:cf:22:79:03:
    cb:ac:5c:26:fa:ea:be:dd:95:d5:a1:90:8c:3a:1c:
    47:c3:28:1a:d7:ff:65:49:55:09:9d:ac:bd:09:c2:
    85:be:42:80:bb:c7:33:a4:96:92:d7:c4:90:9e:63:
    90:80:39:db:6a:d6:55:7d:39:77:84:2c
-----BEGIN RSA PRIVATE KEY-----
MIIG4gIBAAKCAYEAqGGC9HYwODLUE2I0VxY405Rv+Kn5DwdBmTgqWiaXdJzfIR+A
dgGoqnqSHSfINfxdql0nMT0oeNO4B1OIZOajmvJJ9K7bSo7QF+bt+VWWtjlAWQN8
uStnyakLG8MraZ30NX4WqJ1FQa2Hlu0R7L17rjaN+Gvd8SokRt0gwGIqefa4wpBL
V7Fp/Dy5qQPeRquP549BF0r7T0sNxpZoqLGgSxcDrpJwaGKg3eqv0JIKfVdmFOKk
WSy28CpN5VxJ5bjcf/rzyLVtVuXY+MmWQwp5bVGrj1XcSjwkhj/6LgCeSptARo3H
v8tbcnJZNLTD/WO6hQuYeu0+O4A+MVZCKzo416LwWoMWgrAi8NOZlz/ayUQMaUZd
2ICIPuY+jy/jG5Ekrot+N9BLePNFw+ASAuUv0HQzNjynSd9pMjlzuzLlJKGsiCl+
AUtVTMJE1PrwGwYJsZ8T7CmF1xKm3Re2GyQ6WYriY5yjrvFl/OSTm240X4Ke1N7B
+3R1tvXX8gASYtsTAgMBAAECggGATRR+b0xfXIqgHNN65Fbu4qRz8aBVvDR1Kh/U
+3npIV/ioPYbx2mQ/aGom6Ine9cwRZnpJBg8G7vH5cquiIowYPEDO/iDuIjR5+82
B0rP+FETDOyUvuuI0n8I78eTawD81zQGSV/dYWh/xm52IyrGoISmzw8KBall++6k
XI1Tpot0HtUROG6l9usRjraPnUhSUw14HaKckrrn7n3SIU9W8qjxbYvp208zFwOl
QlRoauKEf5EU7SGUCTNEf3eU+I173NX+jLsNbP/gBUmTlqlZKJrK7WFJNqJ/LpJ2
BLfVETWfO8hxUk7RYZsUnBJ47dv6Mu6hBR20/pt0eI0rVZVRsaQ/A6PkGd0sq26t
++aWUoc3Qr5a9l65CviZEacSgJWQcfea42y7FC+RzJGMLUzO0AVg/aAE4izpF4qe
Vzlih1bQtv1HuVQeSVm92/yNnlgzq1Ybl28ePqH+WXxcWEBE+Dtn+ZfFsfV+2SZ4
nRhS9TVT7nDoeb2rZOOkiT5YufCRAoHBANtlQ1yIHMfiW0FKxmzC5R1pyCmBQ1Cc
9Y9p4sL2srvv41KdOjzwkD7VeBUsbzIyx/5IzHpO5Fq7+jjmmcW2FzxvbUwTxgF1
+eDi/alVsqsBKiscx3Ushx0u6yqiF6X5axL1l1vJ1/GLuJR8lepLQEmTf+dmRvOX
IbIbJ4xdp1g4KwBCkRf5pEfpRP04RhYoxoZbTAxcIrgxWVKRG1mX84CS6T67Y0Zg
QaJy2jZsM19Ih/HGZUCLV8nq8BPJuQb96QKBwQDEeVSeKsDg9DVNzva+5kursjyI
U+fUZQPltqjCkEH8+pAl0W6OLUSxCUF8T8qONM/1tYdFWJer8XkSUemd4289gkQx
wzAh61Gqdj3bJ3b6yxcQD8k4wmsmH0rFs5G6SHyko4E0e2alDSKsppueVNWFBBHf
BS8tLi7WCkNTtOUMcPJDPJUm/1ET7xBmNSWaoR8PovwOx3n9x0KbL03pJQQ1a8gw
oc5Qu4b9ZatEfSk0Pe/GCHC1f9ogSIK+a04vx5sCgcAj0Q1FLudb7mUtbJKmMM8M
lTdbJ0IhB+7CweuGHpYhPlFjLE3UowU5bMoU0tM4Y7fomGPhV73J3kREQ8k40LhM
Ynh6YgQCOwtXHleA7H+lkSQcM6+QPnu0VPOJWzr3GpfTWYqjLnl1cnTosMocnLpb
CJxp33WX0MVseShkrulh6pGCkltziYq7PqbHnAc9GN0IFCF/Jw93+H53jz7iyn6z
L8iqOBdF5vLwHUXoepOMa5/d5aCj/9uOGC+nW1g8fvkCgcBZM+kaQrj9qYLGB/B5
bd/vXWYkU0wPX6msUwk+BPcg37YwpWy3epAUfOpRgXyqognFOhznfDiopWLd0Xjq
6Ybo+0rw+PjSfZ+/cB3XcYyQg0K+VRx7+7FgRpbROh0w4ZZNvbp81IYe697RWneP
```

```
JF8pHFbs1dK8hZpYLaKxz5Cq3GDx7ACgCw2JqmHctcaT9RFALTSZlTGq6hwwjwKl
ltvZUU0730rn1qyMQevHlbfucyx40f4yOZEUVTefMuL1yH8CgcB7zEg/PAK+bJbU
savrl6HZFgvYvUSJfR3mel4jh88bRRgkxMdJeUCYB6c2VSupjraaw6BPhB4STwSU
xai53fcNgva3LLRgMeD1h/d8UVFdx9w2PblrA6uUKldyXE7yjdoU1zErc/2MWF9T
j4LjepjvigfrcBUoK8vcTVY9G4ZjSRt4Zs8ieQPLrFwm+uq+3ZXVoZCMOhxHwyga
1/9lSVUJnay9CcKFvkKAu8czpJaS18SQnmOQgDnbatZVfTl3hCw=
-----END RSA PRIVATE KEY-----
```

•Signing Methods

4.11.2.1.5 Signing Methods

4.11.2.1.5.1 Signing Method - Variability control

Different off-board systems must know: If, when, how etc. software for an ECU shall be signed. This concept uses a dedicated software part, referred to as Software Settings File, to carry this information.

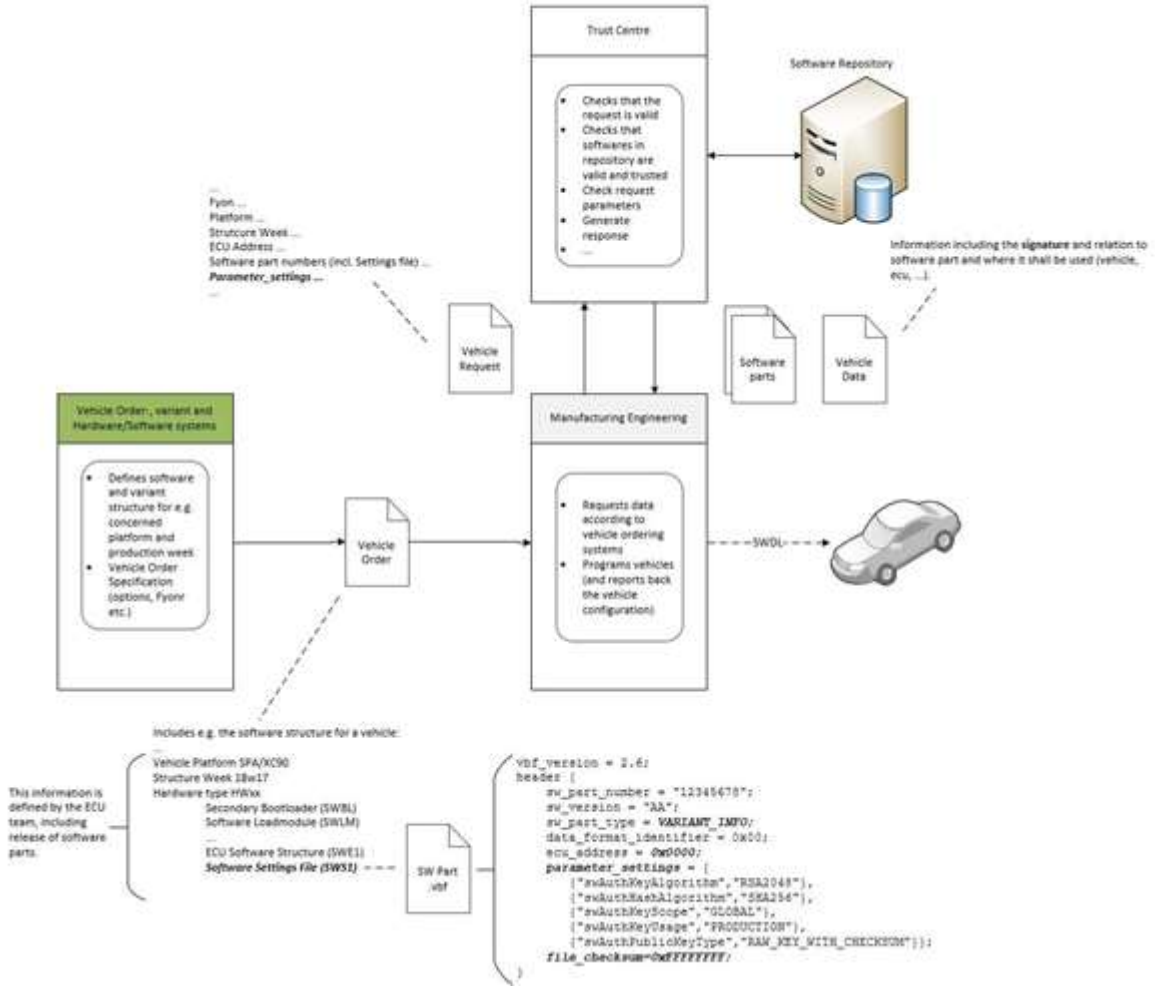


Figure - Software Authentication variability management

The *Figure - Software Authentication variability management* shows a simplified view (e.g. details about information flow is omitted) of how to define and distribute needed variant information.
(i) A separate software part, a Software Settings File, with a specific identifier parameter_settings is applied.
(ii) The Software Settings File is version handled and released as any other software part.
Refer to section "Software Settings File" for further information about that format.

661315 v1 **Default Signing Method**

Version history:
(1):

Purpose: Define the default signing method. The ECU shall use the default signing method, unless else is explicitly defined elsewhere, e.g. in the LC : Diagnostics and ECU Platform.
Source:
Verification Method: Analysis
Legacy ID: 410732v0

The default signing method shall be SHA256_RSA2048_sw_part_based.

661316 v2 **Signing Method #2**

Version history:
(2):

Purpose: Define the second signing method. If need more secure software authentication, the ECU shall use this signing method.
Source:
Verification Method: Analysis
Legacy ID:

The signing method #2 shall be SHA256_RSA3072_sw_part_based.

Note: The use of signing method #2 need to be approved by Base Technology Team and Cyber Security Team.

675313 v1 **Signing Method #3**

Version history:
(1):

Purpose: Define the third signing method. If need more secure software authentication, the ECU shall use this signing method.
Source:
Verification Method: Analysis
Legacy ID:

The signing method #3 shall be SHA256_RSA4096_sw_part_based.

Note: The use of signing method #3 need to be approved by Base Technology Team and Cyber Security Team.

661314 v1 **Signing Method - Variability control**

Version history:
(1):

Purpose: To control the variability at the back-end systems but also to inform the ECU of the signing method (at design time) that shall be applied. Note that a specific ECU does not have to support all methods, but only one.

Source:

Verification Method: Analysis

Legacy ID: 404595v3

To define and distribute the Signing Method, a Software Settings File (a software part) shall be used for methods where each software part is signed individually.

Note: The notation <HashAlgorithm_SigningAlgorithm_SigningObject>, e.g. SHA256_RSA2048_sw_part is used in nextcoming requirements to define algorithms to be applied. The "sw_part" is used to seperate this method from legacy concepts. See also section "Software Settings File" actual definitions.

4.11.2.1.5.2 Signing Method - Algorithm Definitions

661317 v2 **Signing Method SHA256_RSA2048**

Version history:
(2):

Purpose: Define the hash function, public key algorithm and the key size for the private- and public key.

Source: 1. FIPS PUB 180-4, Secure Hash Standard (SHS) FEDERAL INFORMATION PROCESSING STANDARDS PUBLICATION, March 2012.
2. PKCS #1 v2.2: RSA Cryptography Standard, RSA Laboratories, October 27, 2012.

Verification Method: Analysis

Legacy ID: 404598v0

The requirements of signing method SHA256_RSA2048 is defined.

Requirement	Description
Hash function	The hash function to be used is the SHA-256 (256 bits) as defined in *FIPS PUB 180-4, Secure Hash Standard (SHS), March 2012*.
Public Key Algorithm	The public key algorithm shall be: *PKCS #1 v2.2: RSA Cryptography Standard, RSA Laboratories, October 27, 2012*, where the RSASSA-PSS signature scheme shall be used. The signature scheme includes a salt when generating a signature. The salt length shall be set to the same length as the underlying hash value. This means that the resulting signature will be different every time a file is signed.
Key Size	The key size shall be 2048 bits. This is the size of the modulus part of the key representations.

661320 v2 **Signing Method SHA256_RSA3072**

Version history:
(2):

Purpose: Define the hash function, public key algorithm and the key size for the private- and public key.
Source: 1. FIPS PUB 180-4, Secure Hash Standard (SHS) FEDERAL INFORMATION PROCESSING STANDARDS PUBLICATION, March 2012.
2. PKCS #1 v2.2: RSA Cryptography Standard, RSA Laboratories, October 27, 2012.
Verification Method: Analysis
Legacy ID:

The requirements of signing method SHA256_RSA3072 is defined.

Requirement	Description
Hash function	The hash function to be used is the SHA-256 (256 bits) as defined in *FIPS PUB 180-4, Secure Hash Standard (SHS), March 2012*.
Public Key Algorithm	The public key algorithm shall be: *PKCS #1 v2.2: RSA Cryptography Standard, RSA Laboratories, October 27, 2012*, where the RSASSA-PSS signature scheme shall be used. The signature scheme includes a salt when generating a signature. The salt length shall be set to the same length as the underlying hash value. This means that the resulting signature will be different every time a file is signed.
Key Size	The key size shall be 3072 bits. This is the size of the modulus part of the key representations.

675314 v1 **Signing Method SHA256_RSA4096**

Version history:
(1):

Purpose: Define the hash function, public key algorithm and the key size for the private- and public key.
Source: 1. FIPS PUB 180-4, Secure Hash Standard (SHS) FEDERAL INFORMATION PROCESSING STANDARDS PUBLICATION, March 2012.
2. PKCS #1 v2.2: RSA Cryptography Standard, RSA Laboratories, October 27, 2012.
Verification Method: Analysis
Legacy ID:

The requirements of signing method SHA256_RSA4096 is defined.

Requirement	Description
Hash function	The hash function to be used is the SHA-256 (256 bits) as defined in *FIPS PUB 180-4, Secure Hash Standard (SHS), March 2012*.
Public Key Algorithm	The public key algorithm shall be: *PKCS #1 v2.2: RSA Cryptography Standard, RSA Laboratories, October 27, 2012*, where the RSASSA-PSS signature scheme shall be used. The signature scheme includes a salt when generating a signature. The salt length shall be set to the same length as the underlying hash value. This means that the resulting signature will be different every time a file is signed.
Key Size	The key size shall be 4096 bits. This is the size of the modulus part of the key representations.

4.11.2.1.5.3 Signing Method - Key balancing

The software authentication concept must support different strategies with respect to key

management and key balancing. Three main security classes with respect to software authenticity are defined in *Table - Software Authentication security classes*. After a threat and risk analysis the proper security class shall be defined and applied for ECUs in the vehicle platform(s). Based on that decision, the affected variant management systems shall be updated accordingly. Hence, both the signing method and key usage must be configurable. In case of an incident, or if a signing key for other reasons must be changed, the key must be able to change (when global keys are used).

661323 v1 **Software authentication security classes**

Version history:
(1):

Purpose: To align the terminology but also describe the expected possibilities of how to balance the key usage. This variability must be managed in the variant handling systems off-board.
Source:
Verification Method: Analysis
Legacy ID: 441876v1

The software authentication security classes as defined in the *Table - Software Authentication security classes* must be supported with respect to key management. This means that it shall be possible to configure the key usage in the off-board variant management systems:
1) Integrity check only, i.e. no cryptographic keys needed
2) ECU global keys used for a period of time. This shall be applied by default.
3) ECU unique keys

Security Class – Software Authentication	Meaning
SwAuth Class 1	Integrity check only, i.e. only using checksums/hash values, for error detection. This is not part of the General Software Authentication document, but added here for legacy reasons.
SwAuth Class 2	Authenticity (origin + integrity), using common/global keys. A dedicated signing key per ECU type shall anyhow be applied, e.g. one for all door modules and another for the climate control respectively (for segmentation) The key used shall be possible to change after a period of time (structure week) and (optional) vehicle type.
SwAuth Class 3	Authenticity + “copy between vehicles protection”. An ECU unique key shall be applied. This class is recommended for sellable features but also for risk reduction (e.g. a key is revealed). SwAuth Class 3 shall always be applied when there is device unique keys for diagnostic service securityAccess, for programming services.

Table - Software Authentication security classes

4.11.2.1.5.4 Software Settings File

The Software Settings File (SWS1) is used to provide different systems in a controlled manner with the information required to sign a software part. This file is version handled in the systems as any other application software part. See *Figure - Software Authentication variability management.*

661324 v1 **Software Settings - Definition**

Version history:
(1):

Purpose: To define the format of the software settings file, used to distribute variant information.

Source:
Verification Method: Analysis
Legacy ID: 456934v0

The software settings file shall be based on the *[LC : Versatile Binary Format Specification]* and the requirements specified there, and contain:
(i) The identifier sw_part_type shall be set to VARIANT_INFO
(ii) The identifier parameter_settings with valid definitions
(iii) The ecu_address shall be set to 0x0000
(iv) No data blocks. That means that the file_checksum will be 0xFFFFFFFF

The valid "name" / "value" definitions for paramater_settings are:
swAuthKeyAlgorithm : RSA2048, RSA3072, RSA4096
swAuthHashAlgorithm : SHA256, SHA384, SHA512
swAuthKeyScope : GLOBAL, NODE_UNIQUE
swAuthKeyUsage : TEST, PRODUCTION
swAuthPublicKeyType : RAW_KEY_WITH_CHECKSUM

Example - parameter_settings:
parameter_settings = { {"swAuthKeyAlgorithm","RSA2048"}, {"swAuthKeyScope","GLOBAL"}, "swAuthPublicKeyType","RAW_KEY_WITH_CHECKSUM"}, {"swAuthHashAlgorithm","SHA256"} };

Notes:
(i) The reason for using a general ecu_address is to enable using the same settings file partnumber for many different ECUs that will use identical parameter_settings definitions.
(ii) The swAuthPublicKeyType is shown in "Figure - Public Key Types", where the Type 1 is the RAW_KEY_WITH_CHECKSUM.
(iii) The vbf version to be used is project dependent and to be specified elsewhere.

661325 v1 Software Settings File - valid part numbers

Version history:
(1):

Purpose: Define software settings file that can be globally used, i.e. used by many ECUs. These are managed by the OEM.

Source:

Verification Method: Analysis

Legacy ID: 458432v0

The software settings files according to "*Table - SWS1 part numbers*" are to be used, unless explicitly defined elsewhere (e.g. due to specific project and ECU needs):

Part number	Settings
32218509	swAuthKeyAlgorithm : RSA2048 swAuthHashAlgorithm : SHA256 swAuthKeyScope : GLOBAL swAuthKeyUsage : TEST swAuthPublicKeyType : RAW_KEY_WITH_CHECKSUM
32218510	swAuthKeyAlgorithm : RSA2048 swAuthHashAlgorithm : SHA256 swAuthKeyScope : GLOBAL swAuthKeyUsage : PRODUCTION swAuthPublicKeyType : RAW_KEY_WITH_CHECKSUM
32218511	swAuthKeyAlgorithm : RSA2048 swAuthHashAlgorithm : SHA256 swAuthKeyScope : NODE_UNIQUE swAuthKeyUsage : TEST swAuthPublicKeyType : RAW_KEY_WITH_CHECKSUM
32218512	swAuthKeyAlgorithm : RSA2048 swAuthHashAlgorithm : SHA256 swAuthKeyScope : NODE_UNIQUE swAuthKeyUsage : PRODUCTION swAuthPublicKeyType : RAW_KEY_WITH_CHECKSUM
8895532104	swAuthKeyAlgorithm : RSA3072 swAuthHashAlgorithm : SHA256 swAuthKeyScope : GLOBAL swAuthKeyUsage : TEST swAuthPublicKeyType : RAW_KEY_WITH_CHECKSUM
8895532111	swAuthKeyAlgorithm : RSA3072 swAuthHashAlgorithm : SHA256 swAuthKeyScope : GLOBAL swAuthKeyUsage : PRODUCTION swAuthPublicKeyType : RAW_KEY_WITH_CHECKSUM
8895532113	swAuthKeyAlgorithm : RSA3072 swAuthHashAlgorithm : SHA256 swAuthKeyScope : NODE_UNIQUE swAuthKeyUsage : TEST swAuthPublicKeyType : RAW_KEY_WITH_CHECKSUM
8895532114	swAuthKeyAlgorithm : RSA3072 swAuthHashAlgorithm : SHA256 swAuthKeyScope : NODE_UNIQUE swAuthKeyUsage : PRODUCTION swAuthPublicKeyType : RAW_KEY_WITH_CHECKSUM

Table - SWS1 part numbers

•Appendix

4.11.2.1.6 Appendix

4.11.2.1.6.1 Example - Data format identifiers and signing (sw_part_based)

Example of a software part with identifiers used for software authentication, where each part is signed individually using the development key.

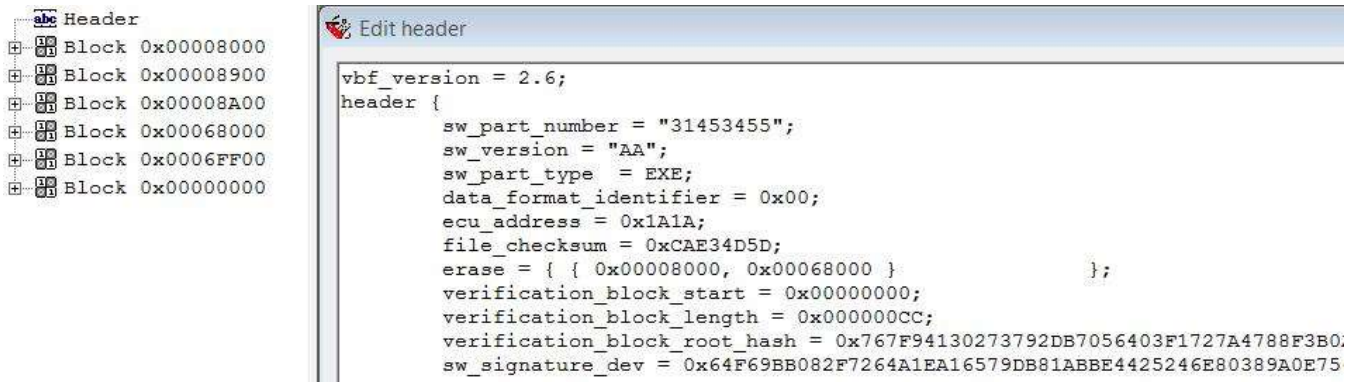


Figure - Example VBF version 2.6
Note that the verification block here starts at address 0x00000000 (verification_block_start), example only.

At the time of this release, the format used by the trust centre to distribute the sw_signature for production to diagnostic clients is TBD. But, the transfer to the ECU will still be by using the routine control CheckMemory.

4.11.2.1.6.2 Example - Software Part evolution

An overview of the evolution of a software part from ECU Software Supplier to Trust Centre is shown in *Figure - Software Part evolution*.

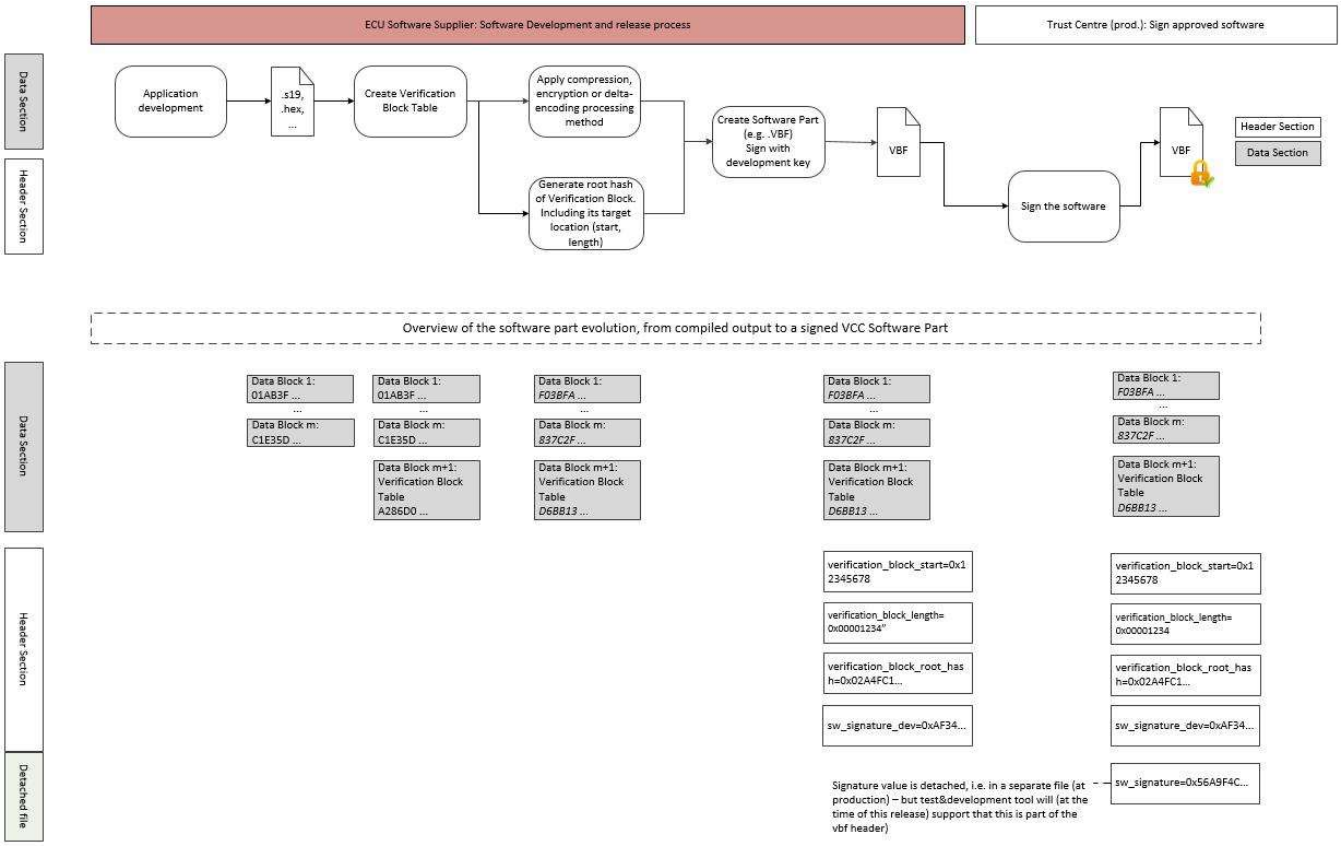


Figure - Software Part evolution

4.11.2.1.6.3 Example - Signing function when the processing method is unknown

Example.
The ECU supplier has used an encryption method where the secret key is not known to the Trust Centre.
The Trust Centre must rely on and sign the software part by using the Verification Block Root Hash value (that is calculated using unprocessed data as input). See *Figure - Signing using Verification Block Root Hash*.

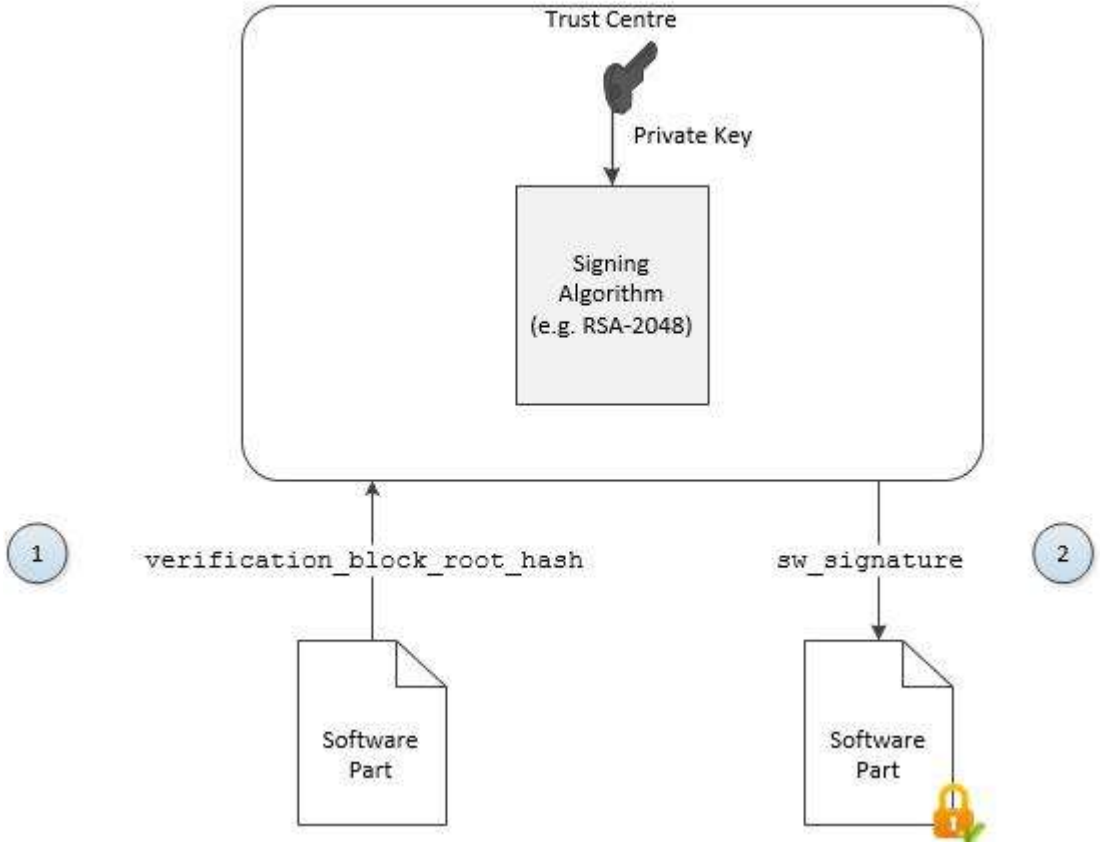


Figure - Signing using Verification Block Root Hash
(1) The hash value (verification_block_root_hash) is transferred to trust centre for signing. Note that the start address and length are included when calculating the hash value.
(2) The resulting signature (sw_signature) is appended to the software part.

The processing method is to be defined by the data_format_identifier *[Versatile Binary Format]*.

4.11.2.1.6.4 Example - PublicKeyCheckSum

The purpose of this example is to show how to calulate the PublicKeyCheckSum, when SHA-256 is used. For simplicity, the previously described initial development key is used as input data for hash calculation.

```
PublicKeyCheckSum = Hash( PublicKeyModulus||PublicKeyExponent) =
Hash( B7 E9 74 4B 45 FA A6 20 D3 1C 30 E9 63 86 E9 CD 5F B9 93 DE CA
45 C9 D6 08 94 F7 7D B9 EE A9 D0 78 45 76 94 80 9D F7 05 24 D7 30 E2
C0 0F 04 6E 60 53 23 BD 50 03 BF 2C A9 BB B4 5C C5 11 5A 1D CE 25 7D
42 03 4F 7E 1C 7A 3E 1A 68 E8 9A 00 10 8D 18 28 AC 26 BD 71 AE 4A C9
B9 23 0B 9B C1 01 67 46 A9 01 5E 70 F1 D9 BD 7F 56 4B 97 61 64 FF C1
D9 6E 93 AB 40 66 D5 CB F4 02 F5 FC 53 11 51 A9 80 5C 07 16 AB CB 98
25 FE 02 F3 89 7E 57 91 7A 64 CC 2C 7A 71 E8 83 33 59 0A A9 59 23 CF
4A 6B E4 24 1A F7 8C A9 04 5D 65 B6 74 87 19 42 49 E3 69 03 DD A4 C9
75 FE A7 3C 07 C1 91 67 54 45 FE 5F CF 45 72 F8 BD 47 95 BA 81 A7 54
50 55 29 92 2F 81 82 71 9B 43 1C EB 27 16 CA 87 E2 BA 83 A0 1E 85 EF
75 E4 63 88 2D 0B 53 76 B6 B3 D6 68 19 E2 6C 2B 67 4F 0A 9D DE FE 93
```

```
42 43 CE 87 AD 00 01 00 01) =
31 DD D9 EC 24 20 1A 6A 82 5D 84 02 18 60 9A AB 65 B0 4E FA 44 A0 4A
99 12 F4 BA E8 E6 91 BB 9C
```

Programming using the public key DID:
WriteDataByIDentifier_PublicKey(*PublicKeyModulus* || PublicKeyExponent || PublicKeyCheckSum) =
WriteDataByIDentifier_PublicKey(*B7 E9 74 4B 45 FA ... 00 01 00 01 31 DD D9 ... BB 9C*)

4.11.3 Data Compression and Encryption

4.11.3.1 Data Compression and Encryption

•Overview

4.11.3.1.1 Overview

Vehicle software is increasing in size. To be able to reduce the time to download software into a vehicle the data stream transfer must be reduced by means of a compression method. Two compression methods are approved by The OEM: method #1 LZSS. LZSSshall be used, unless it is not possible to implement LZSS in design.

This document states the requirements for data compression/decompression used in ECU software download.

4.11.3.1.1.1 Definitions and Abbreviations

Abbreviations	Stands for	Description
ECU	Electronic Control Unit.	In vehicle electronic control unit with a computer/microcontroller.
LZSS	Lempel Ziv Storer Szymanski	Compression method #1.
VBF	Versatile Binary Format	A data file format used at the OEM.

4.11.3.1.1.2 External Publications

[1] "The Data Compression Book" by Mark Nelson, ISBN 1-55851-434-1.

•Software Compression

4.11.3.1.2 Software Compression

4.11.3.1.2.1 Compression Method #1

Software compression is used to minimize the size of the actual data to transmit to in-vehicle ECUs during software download. The compression is performed outside the vehicle before the data is transmitted and the software decompression unpacks the received data in the vehicle ECUs during software download.

Note: Compression method #1 is the default algorithm for compression and decompression.

4.11.3.1.2.1.1 Data Compression

The compression method for compressing the data #1 shall be **Lempel-Ziv-Storer-Szymanski** (**LZSS**) which is a lossless data compression method, a derivative of LZ77 (Abraham Lempel and Jacob Ziv, 1977) that was created in 1982 by James Storer and Thomas Szymanski.

LZSS is a dictionary encoding technique. Unlike Huffman coding, which attempts to reduce the average amount of bits required to represent a symbol, LZSS attempts to replace a string of symbols with a reference to a dictionary location of the same string.

The dictionary in LZSS is a sliding window. Compression is achieved by replacing byte arrays found in previously read data with position/length-pairs pointing in to the sliding window. If the position/length-pair takes up more bits than the byte array that it tries to compress, the data is left uncompressed. This is indicated with a single bit flag, '1' indicates that data to follow is uncompressed and '0' indicates that data to follow is a position/length-pair. A flag bit and an uncompressed byte or a position/length-pair are in this document called a compressed data segment.

To speed up the compression algorithm a binary search tree is used to store previously compressed data.

Appendix: Compression Method #1 Examples, contains C-code examples implementing LZSS compression and decompression.

•Data Encryption

4.11.3.1.3 Data Encryption

Not applicable in this document issue.

•Software Component Requirements

4.11.3.1.4 Software Component Requirements

4.11.3.1.4.1 Compression Method Selection

4.11.3.1.4.1.1 Method Selection

662890 v1 **Data Compression Method Selection**

Version history:
(1):

Purpose: State the compression method selection
Source:
Verification Method: Analysis
Legacy ID: 467271v2

The compression method #1 LZSS shall be used as default method for data compression. Exception to this requirement is allowed only when approved by Geely Basetech Team.

4.11.3.1.4.2 Compression Method #1 Requirements

4.11.3.1.4.2.1 Data Compression

662892 v1 **Data compression Method 1 parameters**

Version history:

(1):

Purpose: State the compression method #1 parameters
Source: Data Compression
Verification Method: Analysis
Legacy ID: 52683v2

The size of the sliding window in the LZSS compression method shall be 1024 bytes.

662893 v1 **Data compression Method 1 parameters**

Version history:
(1):

Purpose: State the compression method #1 parameters
Source: Data Compression
Verification Method: Analysis
Legacy ID: 52687v2

The size of the look-ahead buffer in the LZSS compression method shall be 16 bytes.

662894 v1 **Data compression Method 1 parameters**

Version history:
(1):

Purpose: State the compression parameters
Source: Data Compression
Verification Method: Test
Legacy ID: 52688v1

First bit in a compressed data segment shall indicate if the following data in the segment is an uncompressed byte or a position/length-pair. ‘1’ indicates an uncompressed byte and ‘0’ a position/length-pair.

662891 v1 **Data compression method 1**

Version history:
(1):

Purpose: State the compression method
Source: Data Compression
Verification Method: Test
Legacy ID: 52677v2

The data compression method #1 shall be LZSS.

•Appendix

4.11.3.1.5 Appendix

4.11.3.1.5.1 Compression Method #1 Examples

4.11.3.1.5.1.1 LZSS Compression C-code Example

This example serves as a LZSS compression implementation. The example code follows the MISRA guidelines and uses size specific data types that have to be defined regarding the used hardware target.

The compression function LZSS_CompressData, read uncompressed data from an input buffer with byte access and writes compressed data to an output buffer with bit access with the following functions:

```
void InputByte(LZSS_OutBuffer_T, Uint8_T byte)
                  Reads one byte from a data input buffer.

void OutputBit(LZSS_OutBuffer_T buffer, Uint8_T bitVal)
                  Writes one bit with value bitValue to a data output buffer

void OutputBits(LZSS_InBuffer_T buffer, Uint32_T dataValue, Uint8_T numberOfBits)
                  Writes numberOfBits with value dataValue to a data output buffer.

/*
* IDENTIFICATION
*
* Product Name: General ECU
* Module Abbr : Compression
* File Name   : lzss_comp.c
* Date        : 2007-06-19
* Revision    : 1.1
*
*-------------------------------------------------------------------------------
* CONTENTS
*
*1      DESCRIPTION
*
*2      DEFINITIONS
*       2.1           Constants
*              2.2    Local functions (forward declarations)
*       2.3           Variables
*       2.4           Macros
*
*       3      LOCAL FUNCTIONS
*              3.1    LZSS_InitTree
*              3.2    LZSS_ContractNode
*              3.3    LZSS_ReplaceNode
*              3.4    LZSS_FindNextNode
*              3.5    LZSS_DeleteNode
*              3.6    LZSS_AddNode
*
*4      GLOBAL FUNCTIONS
*       4.1    LZSS_CompressData
*
*-------------------------------------------------------------------------------
*/
/*
*-------------------------------------------------------------------------------
*1      DESCRIPTION
```

```
*-----------------------------------------------------------------------------
*
* This file implements the LZSS compression used for the converter tool.
* LZSS is a dictionary compression method and uses a sliding window as
* dictionary. Compression is achieved by replacing byte arrays found in
* previously read data with position/length-pairs pointing in to the sliding
* window. If the position/length-pair takes up more bits than the byte array
* that it tries to compress, the data is left uncompressed. This is indicated
* with a single bit flag, '1' indicates that data to follow is uncompressed and
* '0' indicates that data to follow is a position/length-pair.
* To speed up the compression algorithm a binary search tree is used to store
* previously compressed data.
*
* Code in this module is based on the code examples in the book "The Data
* Compression Book" by Mark Nelson, ISBN 1-55851-434-1
*/
#include <bitio.h>    /* Data IO functions */
#include <aptypes.h>  /* MISRA type definitions */
#include <lzss_comp.h>

/*
*-----------------------------------------------------------------------------
*2      DEFINITIONS
*-----------------------------------------------------------------------------
*/

/*
*-----------------------------------------------------------------------------
*2.1 Constants
*-----------------------------------------------------------------------------
*/

/*
* Number of bits allocated to indices into the text window
*/
#define LZSS_INDEX_BIT_COUNT      (Uint8_T)10

/**
* @brief Number of bits allocated for the length of an encode phrase
**/
#define LZSS_LENGTH_BIT_COUNT     (Uint8_T)4

/**
* @brief Size of the sliding window
**/
#define LZSS_WINDOW_SIZE          (Uint16_T)( 1 << LZSS_INDEX_BIT_COUNT )

/*
* Number of bytes to encode a byte array.
* Used to calculate if compression should be done or not
*/
#define LZSS_BREAK_EVEN           (Uint8_T)( ( 1 + LZSS_INDEX_BIT_COUNT + \
                                               LZSS_LENGTH_BIT_COUNT ) / 9 )

/*
* End of stream indicator
*/
#define LZSS_END_OF_STREAM        (Uint8_T)0

/*
* Size of the look-ahead buffer
*/
#define LZSS_RAW_LOOK_AHEAD_SIZE  (Uint8_T)( 1 << LZSS_LENGTH_BIT_COUNT )
```

```
/*
* The real look-ahead size, e.g. the maximum number of bytes to match
*/
#define LZSS_LOOK_AHEAD_SIZE      (Uint8_T)( LZSS_RAW_LOOK_AHEAD_SIZE + \
                                             LZSS_BREAK_EVEN )

/*
* Value of the tree root
*/
#define LZSS_TREE_ROOT            LZSS_WINDOW_SIZE

/*
* Null index for the tree structure
*/
#define UNUSED                    (Uint8_T)0

/*
*-------------------------------------------------------------------------------
*    2.2 Local functions (forward declarations)
*-------------------------------------------------------------------------------
*/

void LZSS_InitTree( Uint16_T r );

void LZSS_ContractNode( Uint16_T old_node, Uint16_T new_node );

void LZSS_ReplaceNode( Uint16_T old_node, Uint16_T new_node );

Uint16_T LZSS_FindNextNode( Uint16_T node );

void LZSS_DeleteNode( Uint16_T p );

Uint8_T LZSS_AddNode( Uint16_T new_node, Uint16_T *match_position );


/*
*-------------------------------------------------------------------------------
*2.3    Variables
*-------------------------------------------------------------------------------
*/

/*
* The sliding window used by the decompression algorithm
*/
Uint8_T LZSS_window[ LZSS_WINDOW_SIZE ];

/*
* Binary tree of all of the byte arrays in the window.
*/
struct {
    Uint16_T parent;
    Uint16_T smallChild;
    Uint16_T largeChild;
} tree[ LZSS_WINDOW_SIZE + 1 ];


/*
*-------------------------------------------------------------------------------
*2.4    Macros
*-------------------------------------------------------------------------------
*/
```

```
/*
* Arithmetic modulo to get the correct index in the sliding window
*/
#define LZSS_MOD_WINDOW( a )           (Uint16_T)( ( a ) & \
                                                   ( LZSS_WINDOW_SIZE - 1 ) )


/*
*-------------------------------------------------------------------------------
*3      LOCAL FUNCTIONS
*-------------------------------------------------------------------------------
*/

/*
*-------------------------------------------------------------------------------
*   3.1 LZSS_InitTree
*-------------------------------------------------------------------------------
*
* LZSS Initialize tree
*
* Initialize the binary tree used by the compression to
* keep track of previously compressed byte arrays
*
* param rootChild     Root node to the tree
*
* return Void
*
*/
void LZSS_InitTree( Uint16_T rootChild )
{
/* Add a root child to initialize the tree */
    tree[ LZSS_TREE_ROOT ].parent = UNUSED;
    tree[ LZSS_TREE_ROOT ].smallChild = UNUSED;
    tree[ LZSS_TREE_ROOT ].largeChild = rootChild;
    /* Set child parent to the tree root */
    tree[ rootChild ].parent = LZSS_TREE_ROOT;
    tree[ rootChild ].largeChild = UNUSED;
    tree[ rootChild ].smallChild = UNUSED;
}

/*
*-------------------------------------------------------------------------------
*   3.2 LZSS_ContractNode
*-------------------------------------------------------------------------------
*
* LZSS contract node
*
* Replace a node with another node already in the tree. This function assumes
* that newNode is a child to the old node.
*
* param oldNode       Node to remove
* param newNode       Node to move to the oldNode position
*
* return Void
*
*/
void LZSS_ContractNode( Uint16_T oldNode, Uint16_T newNode )
{
/* Move the parent node to the new node */
    tree[ newNode ].parent = tree[ oldNode ].parent;
    /* Change the parent on the child node */
    if ( tree[ tree[ oldNode ].parent ].largeChild == oldNode )
    {
```

```
        tree[ tree[ oldNode ].parent ].largeChild = newNode;
    }
    else
    {
        tree[ tree[ oldNode ].parent ].smallChild = newNode;
    }
    /* Detach the old node */
    tree[ oldNode ].parent = UNUSED;
    tree[ oldNode ].smallChild = UNUSED;
    tree[ oldNode ].largeChild = UNUSED;
}

/*
*-------------------------------------------------------------------------------
*   3.3 LZSS_ReplaceNode
*-------------------------------------------------------------------------------
*
* LZSS replace node
*
* Replace a node with a new node that was not previously in the tree
*
* param oldNode       Node to remove
* param newNode       New node not previously in the tree
*
* return Void
*
*/
void LZSS_ReplaceNode( Uint16_T oldNode, Uint16_T newNode )
{
    /* If the old node is the smallChild*/
    if ( tree[ tree[ oldNode ].parent ].smallChild == oldNode )
    {
        /* Set the new node as smallChild*/
        tree[ tree[ oldNode ].parent ].smallChild = newNode;
    }
    else
    {
        /* Else set the new node as largeChild*/
        tree[ tree[ oldNode ].parent ].largeChild = newNode;
    }
    /* Insert the old node at the new node index in the tree array*/
    tree[ newNode ] = tree[ oldNode ];
    /* Set the small child parent to the new node*/
    tree[ tree[ newNode ].smallChild ].parent = newNode;
    /* Set the large child parent to the new node*/
    tree[ tree[ newNode ].largeChild ].parent = newNode;
    /* Detach the old node */
    tree[ oldNode ].parent = UNUSED;
    tree[ oldNode ].smallChild = UNUSED;
    tree[ oldNode ].largeChild = UNUSED;
}

/*
*-------------------------------------------------------------------------------
*   3.4 LZSS_FindNextNode
*-------------------------------------------------------------------------------
*
* LZSS find node
*
* Find the largest node under the argument node. Used when deleting
* a node. This routine assumes that the argument node has a smaller child.
*
* param node   Node to search from
```

```
*
* return largest node under the argument node
*
*/
Uint16_T LZSS_FindNextNode( Uint16_T node )
{
    Uint16_T next = UNUSED;

/* Start at the small child of the node*/
    next = tree[ node ].smallChild;
    /* And find the largest node under the small child*/
    while ( tree[ next ].largeChild != UNUSED )
    {
        next = tree[ next ].largeChild;
    }
    return( next );
}

/*
*-------------------------------------------------------------------------------
*   3.5 LZSS_DeleteNode
*-------------------------------------------------------------------------------
*
* LZSS delete node
*
* Deletes a node from the binary tree.
*
* param node   Node to delete
*
* return void
*
*/
void LZSS_DeleteNode(Uint16_T node)
{
Uint16_T replNode = 0;

/* If the large child is empty, just move up the small child*/
if(tree[node].largeChild == UNUSED)
{
        LZSS_ContractNode( node, tree[ node ].smallChild );
    }
    /* If the small child is empty, just move up the large child */
    elseif ( tree[ node ].smallChild == UNUSED )
    {
        LZSS_ContractNode( node, tree[ node ].largeChild );
    }
    /* Otherwise, replace the node to be deleted with the largest node on
     * the small child side of the tree */
    else
    {
        /* Find replacment node.
         * LZSS_FindNextNode will always get the largest node on the small side
          * and the found node will not have any larger child */
        replNode = LZSS_FindNextNode(node);

        /* Contract the replacement node with its small child */
        LZSS_ContractNode(replNode, tree[replNode].smallChild);

        /* Replace the node to delete with the found node */
        LZSS_ReplaceNode(node, replNode);
    }
}
```

```
/*
*-----------------------------------------------------------------------------
*   3.6 LZSS_AddNode
*-----------------------------------------------------------------------------
*
* LZSS add node
*
* In this routine most of the encoder work is done. The routine adds
* a new node to the binary tree. It finds the best match among all existing
* nodes already in the tree and returns the length and the position of the match
* If the new_node has a duplicate in the tree, the old_node is deleted, for
* reasons of efficiency.
*
* param newNode       Node to add to the binary tree (pointer to the byte array)
* param matchPos      Return pointer to the node position in the window
*
* return length of the match
*
*/
Uint8_T LZSS_AddNode( Uint16_T newNode, Uint16_T *matchPos )
{
    Uint8_T i = 0;
    Uint16_T testNode = 0;
    Int16_T delta = 0;
    Uint8_T matchLen = 0;
    Uint16_T *child;
    BOOL_T matchDone = FALSE;
    Uint8_T retVal = 0;

/* If the new byte is END_OF_STREAM, exit */
    if ( newNode == LZSS_END_OF_STREAM )
    {
        return( 0 );
    }
    /* Get the first node to test with */
    testNode = tree[ LZSS_TREE_ROOT ].largeChild;
    /* Set the length of the match to 0 */
    matchLen = 0;

    while ( !matchDone )
    {
        i = 0;
        delta = 0;
        /* Look for matches in the tree. Test current test node with the new
         * node until they differ */
        while((i < LZSS_LOOK_AHEAD_SIZE) && (delta == 0))
        {
            delta = LZSS_window[ LZSS_MOD_WINDOW( newNode + (Uint16_T)i ) ] -
                    LZSS_window[ LZSS_MOD_WINDOW( testNode + (Uint16_T)i ) ];

        i++;
        }
        /* If the nodes didn't match every LZSS_LOOK_AHEAD_SIZE bytes
         * decrease i to get the current match length */
        if(delta != 0)
        {
        i--;
        }
        /* If i larger than previous match in the tree */
        if ( i >= matchLen )
        {
        /* Set new match length to i */
            matchLen = i;
```

```
            /* Set match position to the test node position */
            *matchPos = testNode;
            /* If the byte array in the look-ahead buffer is an exact match of
             * the test node, replace the test node with the new node to remove
             * redundant data */
            if ( matchLen >= LZSS_LOOK_AHEAD_SIZE )
            {
                LZSS_ReplaceNode( testNode, newNode );
                /* Search finished */
                retVal = matchLen;
                matchDone = TRUE;
            }
        }
        if(!matchDone)
       {
        /* If the new node is larger than the test node */
        if ( delta >= 0 )
        {
              /* Continue to test with the larger child */
            child = &tree[ testNode ].largeChild;
        }
        /* If new node is smaller than or equal to the test node */
        else
        {
              /* Continue to test with smaller child */
            child = &tree[ testNode ].smallChild;
        }
        /* If the new test node is a leaf */
        if ( *child == UNUSED )
        {
              /* Add the new node to the tree */
            *child = newNode;
            tree[ newNode ].parent = testNode;
            tree[ newNode ].largeChild = UNUSED;
            tree[ newNode ].smallChild = UNUSED;
            /* Search finished */
            retVal = matchLen;
            matchDone = TRUE;
        }
        /* Continue to test the child*/
        testNode = *child;
       }
    }
    return retVal;
}

/*
*-----------------------------------------------------------------------------
*4      GLOBAL FUNCTIONS
*-----------------------------------------------------------------------------
*/

/*
*-----------------------------------------------------------------------------
*   4.1 LZSS_CompressData
*-----------------------------------------------------------------------------
*
* LZSS compress data
*
* This routine compress data from an in buffer using the LZSS algorithm. LZSS is
* a dictionary compression method where the dictionary is a sliding window. The
* routine uses a look-ahead buffer and finds matches in previously read bytes
* stored in a sliding window.
```

```
* If a match is found an index/length pair is written to the output buffer
* (index is the position in the sliding window). If no match is found the read
* byte is just bypassed to the out buffer. To indicate if the data is an
* index/length pair or a plain text byte a single bit is used as indicator flag
* and is written to the out buffer.
* To speed up the compression a binary tree is used for storing previously
* compressed data.

*
* param inBuf  Input data buffer with byte access
* param outBufoutput data buffer with bit access
*
* return void
*
*/
void LZSS_CompressData(LZSS_InputBuffer_T *inBuf, LZSS_OutputBuffer_T *outBuf )
{
/** Index var. for loop */
    Uint8_T    i             = 0;
    /** Byte read from input stream */
    Uint16_T   inByte               = 0;
    /** Number of bytes in the look head buffer */
    Uint8_T    aheadBytes    = 0;
    /** Current position in the window */
    Uint16_T   winPos               = 1;
    /** Number of bytes to replace in the window */
    Uint8_T    replCnt       = 0;
    /** Length of the data match found in the window */
    Uint8_T    matchLen      = 0;
    /** Position in the window of the data match */
    Uint16_T   matchPos      = 0;
    /** Indicator of End Of Stream reached */
    BOOL_T            eosReached    = FALSE;

    /* Start with filling up the look-ahead buffer */
        while( (aheadBytes< LZSS_LOOK_AHEAD_SIZE) && (!eosReached))
          {
             /* Get next input byte */
             inByte = InputByte( inBuf );
             /* If input stream is finished, exit */
             if ( inByte == LZSS_END_OF_INPUT_STREAM )
             {
                eosReached = TRUE;
             }
             else
             {
                /* Add byte to wondow */
                LZSS_window[ winPos + (Uint16_T)aheadBytes ] = (Uint8_T) inByte;
                /* Increase look-ahead bytes*/
                aheadBytes++;
             }

          }
    /* Initialize the tree */
    LZSS_InitTree( winPos );
    /* While there still are bytes in the look ahead buffer, loop */
    while ( aheadBytes > 0 )
    {
        /* If previously match length greater than look ahead bytes it's
         * not possible to code correctly */
        if ( matchLen > aheadBytes )
        {
        /* Set matched length to look-ahead buffer length */
```

```
            matchLen = aheadBytes;
        }
        /* If the match is smaller than the compressed data (position/length-
         * pair) there will be negative compression so just output the byte */
        if ( matchLen <= LZSS_BREAK_EVEN )
        {
            /* Set consumed bytes in input stream to 1 */
            replCnt = 1;
            /* Indicate that next byte in output stream is uncompressed
             * by output a '1' */
            OutputBit( outBuf, (Uint8_T)1 );
            /* Output uncompressed byte */
            OutputBits( outBuf,
                        (Uint32_T) LZSS_window[ winPos ], (Uint8_T)8 );
        }
        /* The match is larger than a position/length pair, compression can be
         * done */
        else
        {
        /* Indicate that the following bits are compressed data by output a
             * '0' */
            OutputBit( outBuf, (Uint8_T)0 );
            /* Output position in the window */
            OutputBits( outBuf,
                        (Uint32_T) matchPos, LZSS_INDEX_BIT_COUNT );
            /* Output the length of the match, (length is the number of bytes
             * that is greater than LZSS_BREAK_EVEN) */
            OutputBits( outBuf,
                        (Uint32_T) ( matchLen - ( LZSS_BREAK_EVEN + 1 ) ),
                        LZSS_LENGTH_BIT_COUNT );
            /* Set consumed bytes in input stream to the length of the match
*/
            replCnt = matchLen;
        }
        /* Delete consumed bytes and add new bytes in the window */
        for ( i = 0 ; i < replCnt ; i++ )
        {
        /* Remove consumed byte from the window */
            LZSS_DeleteNode( LZSS_MOD_WINDOW( winPos +
                                (Uint16_T)LZSS_LOOK_AHEAD_SIZE ) );
            /* Get next input byte */
            inByte = InputByte( inBuf );
            /* If input stream is finished */
            if ( inByte == LZSS_END_OF_INPUT_STREAM )
            {
               /* Decrease the look-ahead bytes */
                aheadBytes--;
            }
            else
            {
               /* Add the new byte from the input stream to the window */
                LZSS_window[ LZSS_MOD_WINDOW( winPos +
                           (Uint16_T)LZSS_LOOK_AHEAD_SIZE ) ]= (Uint8_T) inByte;
            }
            /* Increase the position in the window */
            winPos = LZSS_MOD_WINDOW( winPos + (Uint16_T)1 );
            /* If there still are bytes in the look-ahead buffer */
            if ( aheadBytes != 0 )
            {
               /* Add byte position to the tree and get the length of the
                 * match */
                matchLen = LZSS_AddNode( winPos, &matchPos );
            }
```

```
        }
    }
    /* Input stream finished, write end of stream to the output buffer
     * uncompressed */
    OutputBit( outBuf, (Uint8_T)0 );
    OutputBits( outBuf, (Uint32_T) LZSS_END_OF_STREAM, LZSS_INDEX_BIT_COUNT );
}


/*
*-----------------------------------------------------------------------------
* End of file
*-----------------------------------------------------------------------------
*/
```

4.11.3.1.5.1.2 LZSS Decompression C-code Example

This example serves as a LZSS decompression implementation. The example code follows the MISRA guidelines and uses size specific data types that have to be defined regarding the used hardware target.

The decompression function LZSS_ExpandData , read compressed data from an input buffer with bit access and writes decompressed data to an output buffer with byte access with the following functions:

```
Uint8_T InputBit(LZSS_InBuffer_T buffer)
                  Read one bit from a data input buffer.

Uint32_T InputBits(LZSS_InBuffer_T buffer, Uint8_T numberOfBits)
                  Read numberOfBits from a data input buffer.

void OutputByte(LZSS_OutBuffer_T, Uint8_T byte)
                  Writes one byte to a data output buffer.


/*
* IDENTIFICATION
*
* Product Name: General ECU
* Module Abbr : Bootloader Decompression
* File Name   : lzss_decomp.c
* Date        : 2007-06-14
* Revision    : 1.0
*
*-----------------------------------------------------------------------------
* CONTENTS
*
*1      DESCRIPTION
*
*2      DEFINITIONS
*       2.1           Constants
*       2.2           Variables
*       2.3           Macros
*
*3      GLOBAL FUNCTIONS
*       3.1 LZSS_ExpandData
*
*-----------------------------------------------------------------------------
*/

/*
```

```
*-----------------------------------------------------------------------------
*1      DESCRIPTION
*-----------------------------------------------------------------------------
*
* This file implements the LZSS decompression used in ECU bootloaders.
* LZSS is a dictionary compression method and uses a sliding window as
* dictionary. Compression is achieved by replacing byte arrays found in
* previously read data with position/length-pairs pointing in to the sliding
* window. If the position/length-pair takes up more bits than the byte array
* that it tries to compress, the data is left uncompressed. This is indicated
* with a single bit flag, '1' indicates that data to follow is uncompressed and
* '0' indicates that data to follow is a position/length-pair.
*
* Code in this module is based on the code examples in the book "The Data
* Compression Book" by Mark Nelson, ISBN 1-55851-434-1
*
*/
#include <bitio.h>           /* Data IO functions */
#include <aptypes.h>         /* MISRA type definitions */
#include <lzss_decomp.h>

/*
*-----------------------------------------------------------------------------
*2      DEFINITIONS
*-----------------------------------------------------------------------------
*/

/*
*-----------------------------------------------------------------------------
*2.1 Constants
*-----------------------------------------------------------------------------
*/

/*
* Number of bits allocated to indices into the text window
*/
#define LZSS_INDEX_BIT_COUNT      (Uint8_T)10

/*
* Number of bits allocated for the length of an encode phrase
*/
#define LZSS_LENGTH_BIT_COUNT     (Uint8_T)4

/*
* Size of the sliding window
*/
#define LZSS_WINDOW_SIZE          (Uint16_T)( 1 << LZSS_INDEX_BIT_COUNT )

/*
* Number of bytes to encode a byte array.
* Used to calculate if compression should be done or not
*/
#define LZSS_BREAK_EVEN           (Uint8_T)( ( 1 + LZSS_INDEX_BIT_COUNT + \
                                               LZSS_LENGTH_BIT_COUNT ) / 9 )

/*
* End of stream indicator
*/
#define LZSS_END_OF_STREAM        (Uint8_T)0

/*
*-----------------------------------------------------------------------------
*2.2 Variables
```

```
*-----------------------------------------------------------------------------
*/

/*
* The sliding window used by the decompression algorithm
*/
Uint8_T LZSS_window[ LZSS_WINDOW_SIZE ];

/*
*-----------------------------------------------------------------------------
*2.3 Macros
*-----------------------------------------------------------------------------
*/

/*
* Arithmetic modulo to get the correct index in the sliding window
*/
#define LZSS_MOD_WINDOW( a )           (Uint16_T)( ( a ) & \
                                                   ( LZSS_WINDOW_SIZE - 1 ) )

/*
*-----------------------------------------------------------------------------
*3      GLOBAL FUNCTIONS
*-----------------------------------------------------------------------------
*/


/*
*-----------------------------------------------------------------------------
*   3.1 LZSS_ExpandData
*-----------------------------------------------------------------------------
*
* LZSS expand data stream
*
* This routine expands data from an in buffer with the LZSS algorithm.
* The routine read in flag bits to decide whether to read in an uncompressed
* character or an index/length pair and expands the data stream to an out
* buffer.
*
* param inBuf     Input data buffer with bit access
* param outBuf    Output data buffer with byte access
*
* return Void
*
*/
void LZSS_ExpandData(LZSS_InputBuffer_T *inBuf, LZSS_OutputBuffer_T *outBuf)
{
    /** Index var. for loop */
    Uint8_T i = 0;
    /** Current window position */
    Uint16_T winPos = 1;
    /** Byte to write to output buffer */
    Uint8_T outByte = 0;
    /** Length of the data match found */
    Uint8_T matchLen = 0;
    /** Position in the window of the data match */
    Uint16_T matchPos = 0;
    /** Indicator of End Of Stream reached */
    BOOL_T     eosReached = FALSE;

    while ( !eosReached )
    {
        /* If next bit is 1, next byte is uncompressed*/
```

```
        if ( InputBit( inBuf ) == 1)
        {
            /* Get uncompressed byte */
            outByte = (Uint8_T) InputBits( inBuf, (Uint8_T)8 );
            /* Output byte*/
            OutputByte(outByte, outBuf);
            /* Add byte in window */
            LZSS_window[ winPos ] = outByte;
            /* Increase window position */
            winPos = LZSS_MOD_WINDOW( winPos + 1 );
        }
        /* If next bit is 0, compressed data follows */
        else
        {
        /* Get compressed data as window position of match*/
            matchPos = (Uint16_T) InputBits( inBuf, LZSS_INDEX_BIT_COUNT );
            /* If end of stream, exit */
            if ( matchPos == LZSS_END_OF_STREAM )
            {
                eosReached = TRUE;
            }
            else
            {
            /* Get length of string match */
            matchLen = (Uint8_T) InputBits( inBuf, LZSS_LENGTH_BIT_COUNT );
            /* Add break even + 1 to get the correct length. Length zero and
                 * the break even value are subtracted from the length during
                 * compression to save space. */
            matchLen = matchLen + (LZSS_BREAK_EVEN +1);
            /* For every byte in match */
            for ( i = 0 ; i < matchLen ; i++ )
            {
              /* Get matching byte from window */
                outByte = LZSS_window[ LZSS_MOD_WINDOW( matchPos + i ) ];
                /* Output byte */
                OutputByte(outByte, outBuf);
                /* Add matched byte to current window position */
                LZSS_window[ winPos ] = outByte;
                /* Increase window position */
                winPos = LZSS_MOD_WINDOW( winPos + 1 );
            }
            }
        }
    }
}


/*
*-----------------------------------------------------------------------------
* End of file
*-----------------------------------------------------------------------------
*/
```

4.11.3.1.5.1.3 Implementation Advice Regarding Decompression

The decompression algorithm shall be implemented by each ECU specified (elsewhere) to have this capability. The compressed data to be transferred to an ECU may be divided into several data blocks, each having a well defined start address and length (or stop address) and may further generate one or several transferData messages.

The decompression algorithm consumes data bit by bit via the InputBit and InputBits functions

(operating on LZSS_InBuffer_T), that in turn should by-pass received data (bits) to the decompression algorithm. Examples of these functions (and buffer LZSS_InBuffer_T) are not provided here, it is up to each supplier to implement these.

Implementations of both the decompression algorithm along with bit buffers have demonstrated a problem, worthwhile addressing for others to avoid, which occurs between two dataTransfer messages.
Figure: A bit stream, below illustrates two dataTransfer messages, and the containing bytes and bits.

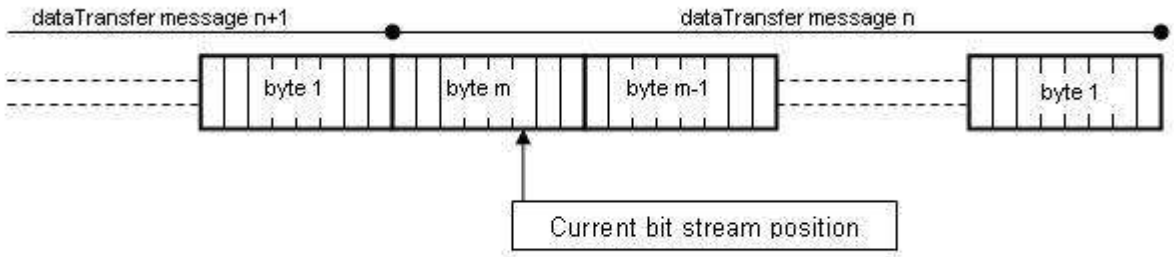


Figure: A bit stream
Suppose that each received dataTransfer message is stored in a buffer, on which the functions InputBit and InputBits have bit access to support the decompression algorithm.

If the capacity of the buffer is equal to the maximum size of a dataTransfer message, then a problem will occur when the current bit position is near the end of the buffer and the decompression algorithm requires more bits than currently stored – bits that are available in the next dataTransfer message.

Care must therefore be taken when consuming bits that may be part of two consecutive dataTransfer messages, illustrated by the Figure: The bit buffer problem illustrated, below. A solution to this problem is not presented here but this information is hopefully of use when designing the bit buffer.

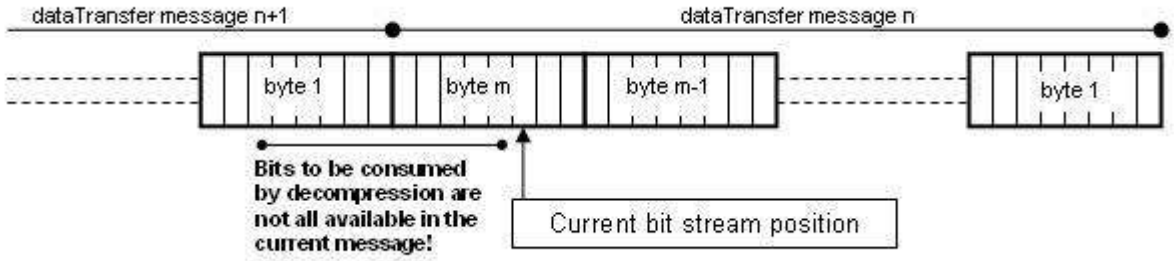


Figure: The bit buffer problem illustrated

4.11.4 Delta Encoding for ECU

4.11.4.1 Delta Encoding Introduction

•Definitions & Abbreviations

4.11.4.1.1 Definitions & Abbreviations

Definition	Description
On-board system	Systems and logical components located in the vehicle
Off-board system	Systems and logical components located outside the vehicle
Normal operational mode	The ECU is executing the application
Program mode	The ECU is executing the primary or secondary bootloader
Full version	The complete software product in file form or installed in an ECU
Target version	The target version is the newer version between two versions.
Source/current version	The source/current version is older then the target version.
Patch, patching	Producing a target version given a source version and a delta file (difference file)
Delta encoding	Delta encoding (or data differencing) consists of producing a difference given a source and a target, with patching producing a target given a source and a difference.

Abbreviation	Description
ECU	Electronic Control Unit. In vehicle electronic control unit with a computer/microcontroller.
SW	Software
OTA	Over The Air. Refers to a software distribution channel over Wi-Fi or mobile broadband
LIN	Local Interconnect Network. In Vehicle Network type.
CAN	Controller Area Network. In Vehicle Network type.
SBL	Secondary bootloader
RAM	Random Access Memory. Associated with volatile types of memory where stored information is lost if power is removed
UDS	Unified Diagnostic Services

•Requisite Documents

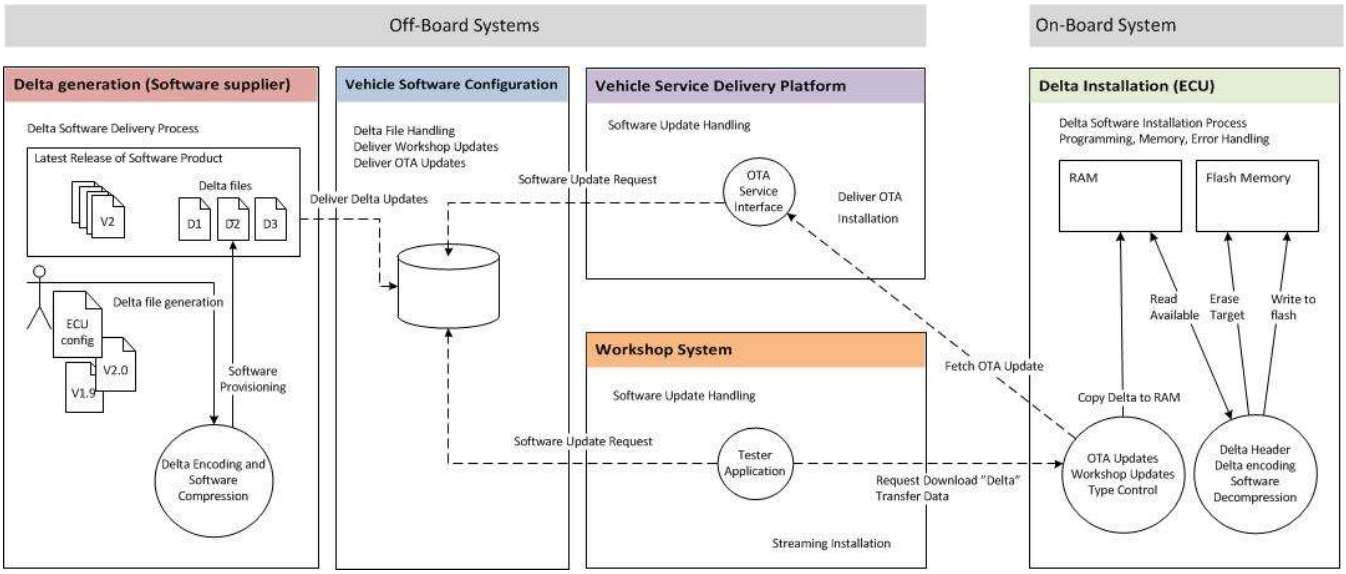
4.11.4.1.2 Requisite Documents

Reference	Source	Document
[Delta_1]	External	Kris Coppieters. A cross-platform binary diff. Dr. Dobb's Journal, May 1995
[Delta_2]	External	Brenda S. Baker, Udi Manber and Robert Muth. Compressing Differences of Executable Code, April 22, 1999
[Delta_3]	External	Nikolai Samteladze, Ken Christensen. DELTA: Delta Encoding for Less Traffic for Apps
[Delta_4]	External	Colin Percival, Naive Differences of Executable Code
[Delta_5]	Internal	Data Compression and Encryption
[Delta_6]	Internal	Versatile Binary Format Specification

•Logical System Components

4.11.4.1.3 Logical System Components

The image below gives a component and interface overview of the Delta Encoded Software updates system.



Subsystem and Interface Overview

4.11.4.2 Delta generation

•Requirements

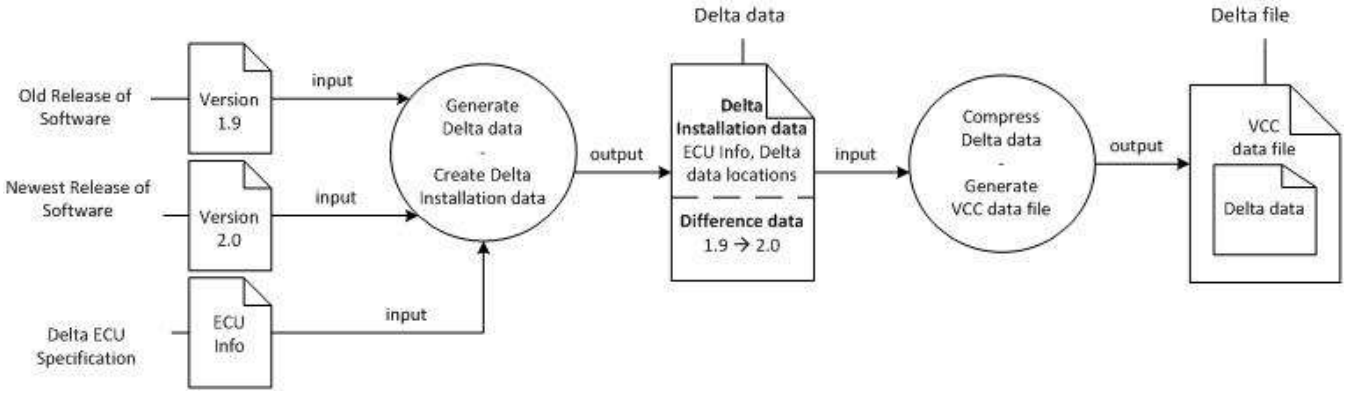
4.11.4.2.1 Requirements

Delta encoding – or more generally, data differencing – is an approach to storing and transferring data using calculated differences between software versions rather than the complete new version of the software. A delta encoding algorithm takes an old and a new version of a file as input and computes the difference between them. The difference data can then be used to construct the new version of the file by merging it with the old. The measures of interest for delta encoding implementations are the size of the delta and the amount of memory and the time required to construct a delta and to apply it.

In order to update an existing software in an ECU by means of delta software delivery, the ECU needs information on how the delta data shall be programmed, E.g. Memory addresses to locate the differences that shall be updated. Also if there are any specific conditions that must be fulfilled before the installation can start, E.g. Software version check, to ensure that the delta is compatible a version number must be provided. This type of “ECU Info” needs to be provided to the ECU somehow and should be included in the delta file during the delta generation process.

In addition to delta encoding the Delta Software Delivery Process can also add Software Compression in order to minimize the size of the data that is to be transmitted to the ECUs.

The generation of delta files can be described as in the image below.



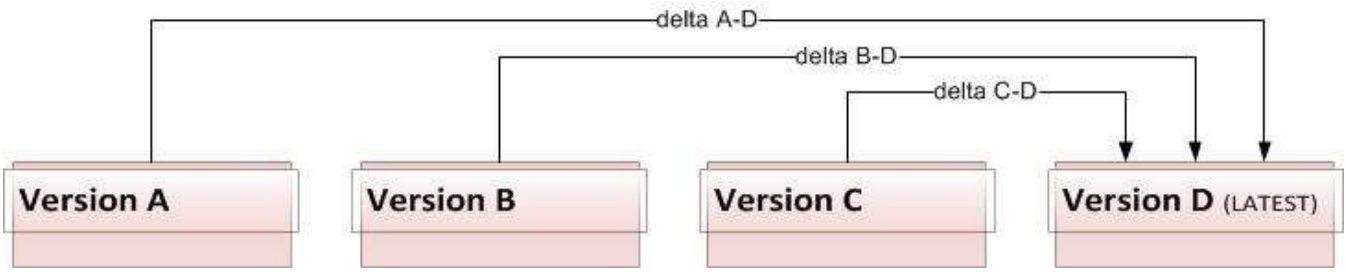
Delta generation process

6286 v4 **Compatibility between versions**

Version history:
(4): Verification Method has been changed from Test to Analysis

Purpose: To guarantee compatibility between versions
Source:
Verification Method: Analysis
Legacy ID: 400851v2

Together with the latest software version for every new software release there shall be delta files generated for three previous versions of the of the software.



Different originating software versions

6287 v2 **Compression**

Version history:
(2):

Purpose: To decrease the size of the delta encoded data
Source:
Verification Method: Inspection
Legacy ID: 400792v1

The delta encoded data shall be compressed.

See *Data Compression and Encryption Specification and Versatile Binary Format Specification*

for details.

6288 v2 **Data file type**

Version history:
(2):

Purpose: To ensure support for data file format
Source:
Verification Method: Inspection
Legacy ID: 400689v1

The delta files shall be provided within the data file format. *See [Versatile Binary Format Specification]*

6289 v1 **Delta file distribution**

Version history:
(1):

Purpose: To ensure consistent software deliveries
Source:
Verification Method: Inspection
Legacy ID: 409691v0

The delta files shall be provided to the Software Configuration System in the same software archive as the target version that they are intended to create.

6290 v4 **Delta file name**

Version history:
(4):

Purpose: To guarantee delta version traceability
Source:
Verification Method: Inspection
Legacy ID: 403939v3

The generated delta files shall have the source and target software part number and version included in the filename.

Example:

sw_part_number = "31808832"; // Target version part no
sw_version = "AB"; // Target version
sw_current_part_number = "31808831"; // Source/current version part no
sw_current_version = "A";

A VBF file that contains a delta and uses the header data above shall utilize the file name of:
<sw_current_part_number><sw_current_version>_<sw_part_number>< sw_version>.VBF

E.g: **31808831A _31808832AB.VBF**

6291 v2 **Delta file release information**

Version history:
(2):

Purpose: To guarantee installation efficiency
Source:
Verification Method: Inspection
Legacy ID: 400796v1

Together with the delta file delivery there shall be formal information about the software supplier's test results.
The figures shall be results from system tests conducted in a system testing environment.
The delta file installation time and delta file size shall be stated together with the install time and file size of a full target version.

6292 v3 **Delta file size**

Version history:
(3): Verification Method has been changed from Test to Analysis

Purpose: To guarantee installation efficiency
Source:
Verification Method: Analysis
Legacy ID: 400795v1

The generated compressed delta file shall be at least 50% smaller, compared to a complete compressed full version *See [Data Compression and Encryption]*.

Note: If the full version file is not compressed due to decreased install time as a result of the decompression routine, then the file shall be compressed using the compression method described in *[Data Compression and Encryption]* in order to perform the comparison. The file size shall be verified by selecting the file in Windows Explorer and select properties (Or equivalent for e.g Linux).

6293 v2 **Delta file structure**

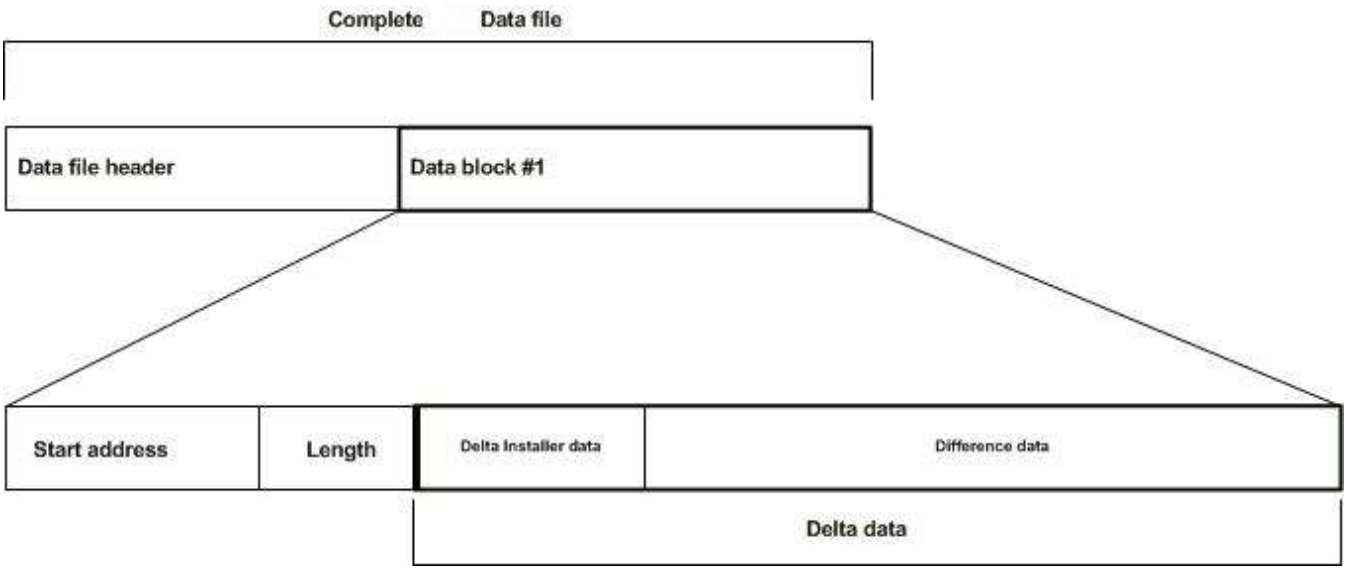
Version history:
(2):

Purpose: To ensure a consistent delta file structure
Source:
Verification Method: Inspection
Legacy ID: 407512v1

Delta files are Data files with one single data block that shall consist of delta installation parameters and the actual delta/difference data.

See also [Versatile Binary Format Specification]

The image below shows how the delta is located within a Data file



Delta file structure

6294 v4 **Delta file version info**

Version history:
(4): Verification Method has been changed from Test to Analysis AND Test

Purpose: To guarantee compatibility between ECU software versions.
Source:
Verification Method: Analysis AND Test
Legacy ID: 400819v2

The delta installation data shall include the Software product number as well as the Software version number for the source version (see below example), so that the ECU can read this data and decide to reject the installation before start if the installed ECU software version is not compatible.

E.g
sw_current_part_number = "31808831";
sw_current_version = "A";

See [Versatile Binary Format Specification] for details

6295 v2 **Delta installation RAM buffer size**

Version history:
(2): Verification Method has been changed from Test to Analysis

Purpose: To ensure that the ECU provides enough memory for the delta installation
Source:
Verification Method: Analysis
Legacy ID: 413074v0

The delta installation data shall include the size of the RAM needed for the delta installation, so that the ECU can read this data and decide to reject the installation before start if the memory amount can not be allocated.

6296 v2 **Delta installation time**

Version history:
(2):

Purpose: To guarantee installation efficiency
Source:
Verification Method: Inspection
Legacy ID: 400794v1

The Installation time of the generated delta shall be at least 50% shorter in average compared to a complete installation of the compressed full version.
(I.e. The time from the first RequestDownload request until the CheckMemory final response)

6297 v1 **New and obsolete data info**

Version history:
(1):

Purpose: To ensure that the bootloader can locate and apply the delta file changes as well as remove unused data.
Source:
Verification Method: Inspection
Legacy ID: 417656v0

The delta installation data shall contain instructions for the bootloader to locate where in memory the delta data shall be written.
The installation data shall also contain instructions in order to locate and erase any obsolete data from the old version that otherwise would remain unused in memory after the delta update.

4.11.4.3 Delta installation

•Requirements

4.11.4.3.1 Requirements

Delta encoded data is applied to the ECU software by the bootloader’s delta installation routine.

6298 v2 **Data decoding**

Version history:
(2):

Purpose: To decode the delta file
Source:
Verification Method: Test
Legacy ID: 400914v1

The ECU shall support decoding of a delta-encoded file

Note: The dataFormatIdentifier in the RequestDownload request indicates if the data is delta encoded

6299 v2 Data decompression

Version history:
(2):

Purpose: To decompress the delta file
Source:
Verification Method: Test
Legacy ID: 400913v1

The ECU shall support decompression of a compressed delta file

Note: The dataFormatIdentifier in the RequestDownload request indicates if the data is compressed

6304 v3 Delta SWDL in normal operational mode

Version history:
(3):

Purpose: Reduce installation time
Source:
Verification Method: Test
Legacy ID: 400877v2

If the ECU supports software installation in normal operational mode then delta download and installation shall also be supported.

6305 v2 Delta SWDL in program mode

Version history:
(2):

Purpose: Reduce programming time
Source:
Verification Method: Test
Legacy ID: 400876v1

The ECU shall support delta download and installation of software in program mode.

6302 v3 Delta installation RAM usage

Version history:
(3): Verification Method has been changed from Test to Analysis

Purpose: To ensure fast delta installation
Source:
Verification Method: Analysis
Legacy ID: 400890v1

The bootloader shall use volatile memory (RAM) when processing the delta data.
This refers to physically accessible RAM not occupied by the boot loader

6300 v3 Delta installation data

Version history:
(3): Verification Method has been changed from Test to Analysis

Purpose: To ensure that the ECU installation process receives all installation data provided in the delta file
Source:
Verification Method: Analysis
Legacy ID: 400883v1

When the bootloader receives a delta file, it shall read and validate the delta installation data provided prior start of the installation phase.
(E.g. Software version information shall be provided and used to decide if installation can start.)

The delta installation data shall be provided within the beginning of the streamed delta data, the complete delta file shall not be downloaded before this information is checked.

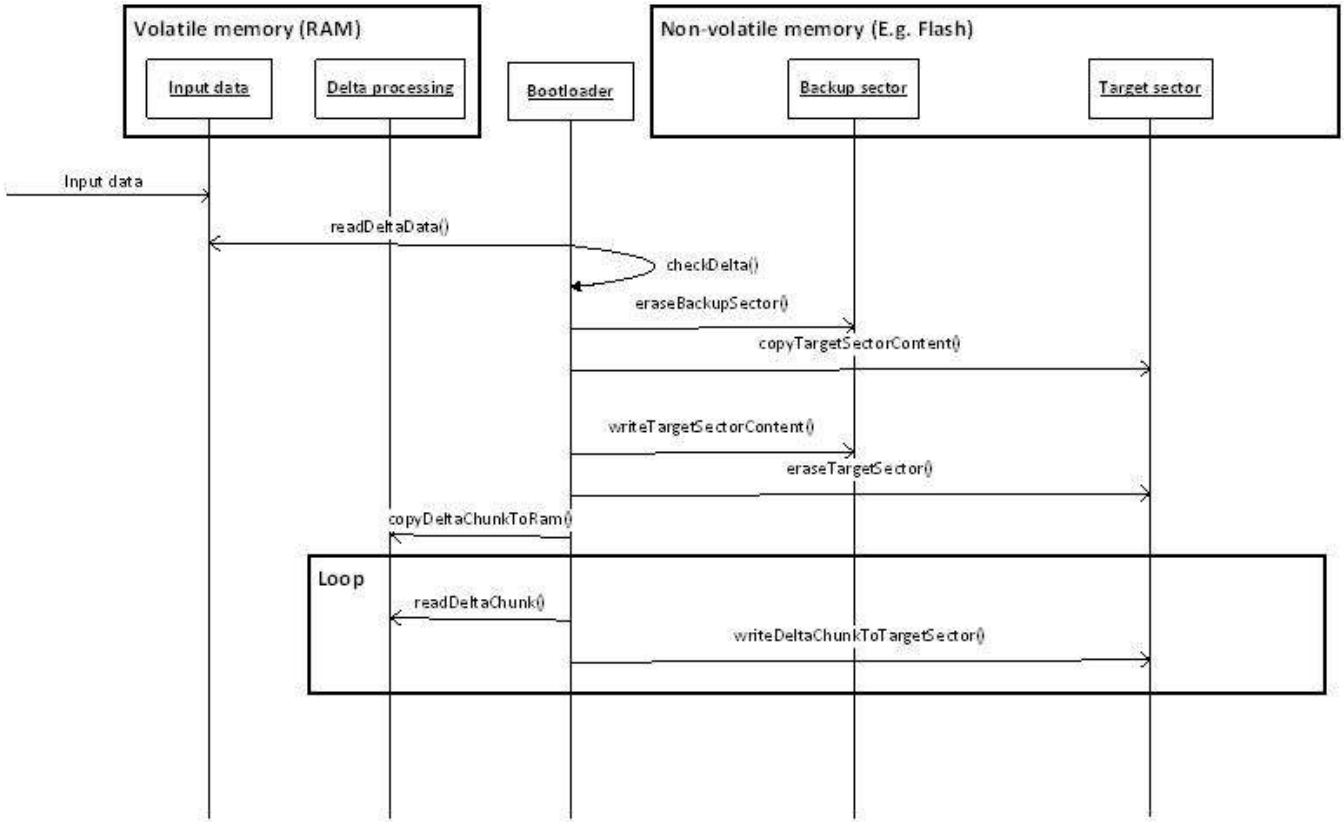
6301 v4 Delta installation memory segment

Version history:
(4): Verification Method has been changed from Test to Analysis

Purpose: To enable delta data manipulation
Source:
Verification Method: Analysis
Legacy ID: 400885v2

The ECU memory map must have memory segment (e.g. flash sector) dedicated for delta installation in order to read, modify and place back data.

Note: The image below illustrates an example. It shall not be considered a solution



Delta installation - Extra sector usage example

6303 v3 Delta memory segment size

Version history:
(3): Verification Method has been changed from Test to Analysis

Purpose: To enable delta data manipulation
Source:
Verification Method: Analysis
Legacy ID: 400889v1

The available non-volatile memory segment size shall be at least the same size compared to the largest possible segment size within the ECU. (E.g. largest flash sector)

6306 v4 Obsolete data removal

Version history:
(4): Verification Method has been changed from Test to Analysis

Purpose: To ensure that data that is located in memory that is no longer a part of the installed software product is erased.
Source:
Verification Method: Analysis
Legacy ID: 416292v2

The bootloader shall use instruction provided in the delta installation data to locate and erase

data that is no longer a part of the installed software product.

6307 v2 **Software consistency check**

Version history:
(2):

Purpose: To ensure that the target image is correct after installation
Source:
Verification Method: Test
Legacy ID: 400916v1

The bootloader shall verify that the applied delta resulted in the correct software image by using the method specified in *[Software Authentication Specification].*

6308 v3 **Streamed installation**

Version history:
(3): Changed the Verification method from Test to Analysis. Updated the Legacy Req. ID reference with the updated Req. ID

Purpose: To reduce programming time
Source:
Verification Method: Analysis
Legacy ID: 400893v1

The ECU shall support streamed installation using UDS, meaning that the bootloader shall start to decode and install data before the complete data file is received.
For more information please see *[General Software Download - Req ID 3188: Writing of received data]*

6309 v2 **Target sector erase check**

Version history:
(2): Verification Method has been changed from Test to Analysis AND Test

Purpose: To ensure that a fallback is possible on each sector during installation
Source:
Verification Method: Analysis AND Test
Legacy ID: 420311v0

Before a target sector is erased and replaced with the delta update, a backup of the sector shall be stored and verified to be exactly the same as the target sector.

4.11.4.3.1.1 Error Handling

6310 v1 **Consistency check failure**

Version history:
(1):

Purpose: To guarantee that the correct image is installed

Source:
Verification Method: Test
Legacy ID: 413452v0

The ECU shall reject the installation and respond with a negative response code if the ECU software is not verified as consistent using the method defined in *[Software Authentication Specification]*

6311 v2 Delta file corruption

Version history:
(2):

Purpose: To guarantee that corrupted data is not used as installation data
Source:
Verification Method: Test
Legacy ID: 400955v1

The ECU shall reject the installation and respond with a negative response code if the bootloader detects that the delta update package is corrupted.

6312 v3 Error code mapping

Version history:
(3):

Purpose: To ensure negative diagnostic response code feedback for error codes during the delta installation
Source:
Verification Method: Test
Legacy ID: 400952v2

Error codes used during the delta installation shall have corresponding negative diagnostic response codes.
See [ISO 14229-1] for NRC reference

The following table shows possible error code mappings:

Delta Installation Error codes	Description	Negative response code (NRC)
NOT_ENOUGH _MEMORY	Either the non-volatile memory or the RAM buffer provided in the init-routine is less than needed to apply the update	generalProgrammingFailure
FAILED_DECOMPRESSION	The bootloader failed to decompress the update package	generalProgrammingFailure
PACKAGE_CORRUPTED	The delta update package was identified as corrupted	generalProgrammingFailure

FAILED_CONSISTENCY_CHECK	The delta update was not identified as a valid software image	generalProgrammingFailure
WRONG_TARGET_VERSION	Source image version is not as expected in delta	uploadDownloadNotAccepted

6313 v2 **Full version install check**

Version history:
(2):

Purpose: To ensure that a full version installation can be installed after a failed delta installation
Source:
Verification Method: Test
Legacy ID: 400959v1

After a failed delta installation the bootloader shall not prevent or resume installation when a full version is to be installed.

Note: Full version is indicated by the dataFormatIdentifier in the RequestDownload request

See [Versatile Binary Format Specification] for reference

6314 v1 **Not enough memory**

Version history:
(1):

Purpose: To guarantee that there is enough memory to perform a delta installation
Source:
Verification Method: Test
Legacy ID: 412891v0

The ECU shall reject the installation and respond with a negative response code if the bootloader detects that either non-volatile memory or the RAM buffer provided in the init-routine is less than needed to apply the update

6315 v2 **Recover from interrupted Installation**

Version history:
(2):

Purpose: To ensure recovery after interrupted Installation
Source:
Verification Method: Test
Legacy ID: 400957v1

If the ECU has memory enough to store a copy of the source version before the delta installation starts, then it shall be possible to fall back to the source version.

If the ECU does not have enough memory available to fall back to the source version then it shall be possible to resume the update installation from the beginning or from the last successfully written code.

6316 v2 Subsequent reprogramming during recover

Version history:
(2):

Purpose: To avoid loss of data or broken ECUs
Source:
Verification Method: Test
Legacy ID: 400958v1

The ECU software shall be designed in such way that an aborted download or install, or removing ECU power at any time during the download and install operation, shall not prevent subsequent reprogramming of the same delta file.

Note: Subsequent reprogramming means that the diagnostic tester is restarting the download from the beginning.

6317 v2 Version mismatch

Version history:
(2):

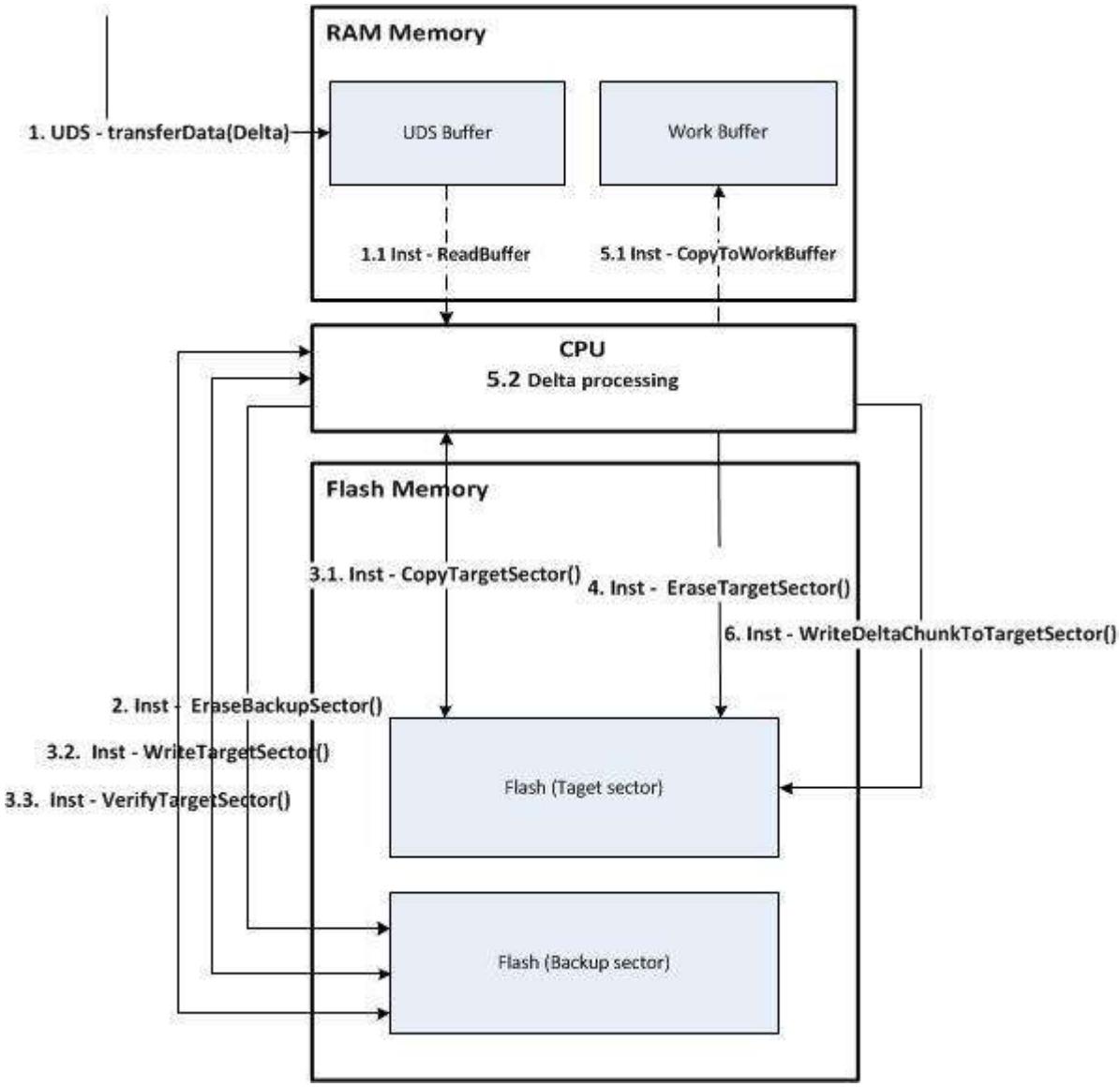
Purpose: To guarantee compatibility between versions
Source:
Verification Method: Test
Legacy ID: 400953v1

The ECU shall reject the installation and respond with a negative response code if the currently installed ECU software version does not match *sw_current_version* provided in the delta installation data.

4.11.4.4 Extended information

•Delta installation - Sequence and memory usage

4.11.4.4.1 Delta installation - Sequence and memory usage



Delta installation - Sequence and memory usage

4.11.5 Delta Encoding for System

4.11.5.1 Vehicle Software Configuration

•Requirements

4.11.5.1.1 Requirements

6318 v2	Delta update requests

Version history:
(2):

Purpose: To guarantee that delta versions are delivered on Workshop requests
Source:

Verification Method: Test
Legacy ID: 400860v1

If delta files exists for a software product, and the currently installed software version is compatible with one of the delta versions, then the Vehicle Software Configuration system shall deliver a delta version.

6319 v2 **Full version update requests**

Version history:
(2):

Purpose: To guarantee full version software updates when delta files exists.
Source:
Verification Method: Test
Legacy ID: 412831v1

It shall be possible to explicitly request a full version instead of a delta file.

6320 v2 **Version handling**

Version history:
(2):

Purpose: To guarantee delta version traceability
Source:
Verification Method: Test
Legacy ID: 400853v1

The Vehicle Software Configuration system shall store delta files with a reference to its source and target version.

4.11.5.2 Workshop System

•Requirements

4.11.5.2.1 Requirements

6321 v2 **Full version update requests**

Version history:
(2):

Purpose: To guarantee that a full version download is requested if a delta file failed to be installed
Source:
Verification Method: Test
Legacy ID: 400875v1

The Tester Application shall request a full version download from the Vehicle Software Configuration system if the delta file installation fails *(E.g if the failure is caused by data*

corruption or similar) after two retries.

6322 v1 **Update request parameter - sw_version**

Version history:
(1):

Purpose: To enable delta version selection in the Vehicle Software Configuration system
Source:
Verification Method: Test
Legacy ID: 412846v0

The Software Update Request to the Vehicle Software Configuration system shall contain the currently installed software version number.

4.11.5.3 Vehicle Service Delivery Platform

•Requirements

4.11.5.3.1 Requirements

6323 v3 **System availability notification**

Version history:
(3):

Purpose: To avoid unexpected downtime, loss of data or broken ECUs
Source:
Verification Method: Inspection
Legacy ID: 402773v2

Any change/update of the hardware or software in the production environment shall be documented and communicated with the respective responsible for the backend systems in good time before the change.

4.11.6 Versatile Binary Format Specification 2.7

4.11.6.1 Versatile Binary Format Generic reqs

•Software part format

4.11.6.1.1 Software part format

991947 v1 **Software part format**

Version history:
(1):

Purpose: To define the software part format for software update.
Source:
Verification Method: Test
Legacy ID:

The software part(s) shall be submitted in VBF format by supplier. The detail format is described in *[Versatile Binary Format Specification]*.

•VBF Additional Requirements

4.11.6.1.2 VBF Additional Requirements

991948 v1 **Maximim allowed VBF File size**

Version history:
(1):

Purpose: To restrict the size of the VBF file to an optimal value that can be handled by EoL and aftermarket tools.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Maximum allowed size of an uncompressed VBF file for an ECU per software part is 20 gigabytes (GB).

It is necessary for all ECUs to support data compression method(s) as defined in [Data Compressoon and Encryption] and the size of compressed VBF file should be at least 30-40% smaller in average when comared to the uncompressed(non-compressed) version.

991950 v1 **Maximum allowed number of data block**

Version history:
(1):

Purpose: To restrict the number of data block of VBF file to an optimal value that can be handled by EOL, aftermarket tools and over-the-air.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Maximum allowed number of data block in VBF file for an ECU per software part is 20.

991949 v1 **Maximum allowed number of erase ranges in VBF header**

Version history:
(1):

Purpose: To restrict the number of erase ranges in VBF header to an optimal value that can be handled by EOL, aftermarket tools and over-the-air.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Maximum allowed number of erase ranges in VBF header for an ECU per software part is 20.

4.11.6.2 Versatile Binary Format Specification

•Overview

4.11.6.2.1 Overview

4.11.6.2.1.1 Requisite document

Reference number	Source	Title
[VBF_1]	Internal	Data Compression and Encryption Specification
[VBF_2]	Internal	UDS Services
[VBF_3]	Internal	General Software Authentication

Note:
Internal references are typically available as separate chapters in ECU specific Base Technology SWRS.

4.11.6.2.1.2 Definitions and Abbreviations

Definition	Description
Domain ID	Part of the ECU Source Address (SA), states which Domain the ECU is physical connected to.
Network ID	Part of the ECU Source Address (SA), states which Network the ECU is physical connected to.
ECU ID	Part of the ECU Source Address (SA).
Tester	A system that controls functions such as test, inspection, monitoring, or diagnosis of an in-vehicle electronic control unit and may be dedicated to a specific type of operators (e.g. a scan tool dedicated to garage mechanics or a test tool dedicated to assembly plant agents). The tester is also referenced as the client.

Abbreviation	Description
CM	CoMments
ECU	Electronic Control Unit
PBL	Primary BootLoader
SBL	Secondary BootLoader
SSBL	Special Secondary BootLoader
VBF	Versatile Binary Format
WS	White Space

•Versatile Binary Format (VBF)

4.11.6.2.2 Versatile Binary Format (VBF)

4.11.6.2.2.1 VBF version

This document specifies VBF version 2.7 of the Versatile Binary Format.

4.11.6.2.2.2 Introduction

The VBF file contains three parts: VBF version section, a header section and a data section. VBF version section specifies the version number of the VBF specification that the file is compliant with. VBF version uses ASCII representation. The header section contains general information relating to file usage, setup, etc. The header section uses ASCII representation. The data section contains the actual data that is to be transferred to the target ECU. The data section uses binary representation.

Table: An Overview of the Versatile Binary Format

Example A
address_and_length_format_identifier = 0x88

Name	Description	Type	Data
Version Section	VBF Version Number	Text	vbf_version=2.7;
Header section	Header	Text	n bytes, ASCII
Data section	Block 1	Start address	8 bytes, binary

		Length	8 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary
	Block 2	Start address	8 bytes, binary
		Length	8 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary
	:	:	:
	Block n	Start address	8 bytes, binary
		Length	8 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary

Example B
address_and_length_format_identifier = 0x44

Name	Description	Type	Data
Version Section	VBF Version Number	Text	vbf_version=2.7;
Header section	Header	Text	n bytes, ASCII
Data section	Block 1	Start address	4 bytes, binary
		Length	4 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary
	Block 2	Start address	4 bytes, binary
		Length	4 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary
	:	:	:
	Block n	Start address	4 bytes, binary
		Length	4 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary

Note: For an ECU that comprises multiple software parts, it can simultaneously support both address_and_length_format_identifier = 0x44 and address_and_length_format_identifier = 0x88.
But it is recommended to use only one fomat.

4.11.6.2.2.3 File Naming

991959 v1 **File Naming**

Version history:
(1):

Purpose: To identify the release of the VBF file.
Source:
Verification Method: Analysis AND Test
Legacy ID:

All delivered or released VBF files shall be named the same as the corresponding released software part number which consist of (sw_part_number and sw_version) with an extension of ".VBF

Examples:
A VBF file with a header entry of
sw_part_number = "31808832";
sw_version = "AB";
shall utilize the file name of 31808832AB.VBF

A VBF file with a header entry of
sw_part_number = "31808832";
sw_version = "A";
shall utilize the file name of 31808832A.VBF

Delta files shall be released with a filename reference to its source and target version by including the software part number and software version of source and target in the delta filename.
Example:
sw_part_number = "31808832"; // Target version part no
sw_version = "AB"; // Target version
sw_current_part_number = "31808831"; // Source/current version part no
sw_current_version = "A";

The format of a delta filename shall be: <sw_current_part_number><sw_current_version><separator><sw_part_number>< sw_version>.VBF

A VBF file containing delta and the header data above shall have the name: **31808831A_31808832AB.VBF**

4.11.6.2.2.4 VBF file generation

991960 v1 **VBF file generation**

Version history:
(1):

Purpose: To generate the VBF file.
Source:
Verification Method: Analysis
Legacy ID:

The VBF Convert tool shall generate VBF files from Motorola S-record or Intel Hex or zip formatted files.

•Notation and lexical elements

4.11.6.2.3 Notation and lexical elements

The VBF file header section uses some basic lexical elements, which are described, in this section. Also, the notation for describing the syntax is given here.

4.11.6.2.3.1 Notation

Words in bold indicate identifiers, which are special tokens.

Italicized words indicate a lexical item, and are described in text format.

The grouping symbols [ ], brackets, indicate that the contents are optional.

The grouping symbols { }, braces, group a set of items.

The grouping symbols ( ), parentheses, contain a set of items separated by |, then one of the items must appear.

4.11.6.2.3.2 General structure

991972 v1 **Header expression termination**

Version history:
(1):

Purpose: To define the header structure.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The header consists of sequences of expressions. An expression is made up of identifiers, reserved words, constants, integers and is always terminated with a semicolon (;).

Expressions containing multiple values are grouped into nested sections using matching brace characters “{“ and “}”. Data shall be contained in braces only if there is more than one set of data with the exception of description [see section *Header section - description*] and erase [see section *Header section - erase*] identifiers. In these cases, a single set of data shall also be contained in braces. Refer to section *Notation and lexical elements - Identifiers, reserved words, and expressions* for general format requirements for expressions and to section *Overview of the VBF file* for detailed format requirements for each identifier.

4.11.6.2.3.3 Identifiers, reserved words and expressions

991974 v1 **Header Identifiers**

Version history:
(1):

Purpose: To define allowed header identifiers
Source:
Verification Method: Analysis AND Test
Legacy ID:

Identifier is the name of a variable and IdentifierValue is the data assigned to the variable. Allowed identifiers are **vbf_version**, **header**, **description**, **sw_part_number**, **sw_current_part_number**, **sw_version**, **sw_current_version**, **sw_part_type**, **data_format_identifier, address_and_length_format_identifier, ecu_address**, **erase**, **call**, **verification_block_start**, **verification_block_length**, **verification_block_root_hash**, **sw_signature_dev**, **sw_signature, parameter_settings** and **file_checksum**. Duplicates of identifiers are not allowed.

991975 v1 **IdentifierValue format**

Version history:
(1):

Purpose: To define the format of the IdentifierValue.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Each element within an IdentifierValue shall either be an integer, real, string embedded within quotes, or a reserved IdentifierValue (reserved word). Each identifier has a specific format allowed for its identifier values as defined in section 5 . The lexical rules for identifiers and reserved words are identical to those for “C” programming language with upper- and lower-case always distinguished.

Unless otherwise specified, a valid expression shall always consist of the following format (where WS/CM is defined as any combination of white space characters and complete comments).

[WS/CM]identifier[WS/CM]=[WS/CM]IdentifierValue[WS/CM];[WS/CM]

When the IdentifierValue is enclosed within matching brace characters "{" and "}" as allowed by this specification,

the format of the IdentifierValue (including braces) shall be as follows :

{[WS/CM]IdentifierValue[WS/CM][,[WS/CM]IdentifierValue[WS/CM]]} ;

where the optional field of [,[WS/CM]IdentifierValue[WS/CM]] may occur multiple times .

Example:
sw_part_type = CARCFG;

In this example, **sw_part_type** is an identifier, and CARCFG is the reserved word value of the identifier **sw_part_type** .

4.11.6.2.3.4 Integers

991978 v1 **Integers**

Version history:
(1):

Purpose: Clarification of the integer format definition used by the VBF file.
Source:
Verification Method: Analysis AND Test
Legacy ID:

An integer is a sequence of digits. It shall be represented in hexadecimal format, start with the prefix “0x” and the following characters must be chosen from the string “0123456789abcdefABCDEF”.

Examples:

Valid Format:
1. 0xFF //Hexadecimal
2 0x12345678 //Hexadecimal, 32 bits representation

Invalid Format:
1. 0b11111111 //Binary
2. 255 //Decimal
3. 0x12 0x34 0x56 0x78 //Each byte separated with prefix “0x”.

4.11.6.2.3.5 Comments

991979 v1 **Comments**

Version history:
(1):

Purpose: To define the usage and format of comments within the VBF file.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Comments are allowed in the header section only and may appear within an expression as described in section *Identifiers, reserved words, and expressions.* Valid comments shall always be ignored by a VBF parser. Comments can be grouped together by “/*” and “*/” and stretch over several lines (C-style), or they can start with “//” and stretch to the end of the line (C++ style). Neither bracket syntax nests. It is recommended that C++-style comments be used wherever possible, since it is much more difficult to accidentally comment out more than was intended (as compared to using the C-style syntax).

4.11.6.2.3.6 White Space Characters

991981 v1 **White Space Characters**

Version history:
(1):

Purpose: To define allowed white space characters.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Blank Space, Horizontal Tab (or Tab), Line Feed, Carriage Return, Form Feed, and Vertical Tab characters are defined as white space characters for the purpose of this specification. No other white space characters are allowed. A VBF parser shall always ignore valid white space characters unless otherwise specified in this document.

Table : ASCII Representation of White Space Characters

	Description	Symbol	Hex Value	Binary Value
1	Horizontal Tab (or Tab)	HT	09	00001001
2	Line Feed	LF	0A	00001010
3	Vertical Tab	VT	0B	00001011
4	Form Feed	FF	0C	00001100
5	Carriage Return	CR	0D	00001101
6	Blank Space	SPACE	20	00100000

4.11.6.2.3.7 Non-printing Characters

991982 v1 **Non-printing Characters**

Version history:
(1):

Purpose: To define the range of non-printing characters allowed.
Source:
Verification Method: Analysis AND Test
Legacy ID:

0x00 to 0x1F represents non-printing characters. All Non-printing characters, except white space characters, are not allowed in VBF file header. For white space non-printing (0x09 to 0x0D) character handling, refer to section *White Space Characters*

•Overview of the VBF file

4.11.6.2.4 Overview of the VBF file

4.11.6.2.4.1 Version Section

991984 v1 **Version section**

Version history:
(1): Added VBF version 2.7.

Purpose: To identify the version of the VBF file.
Source:

Verification Method: Analysis AND Test
Legacy ID:

The first line of the VBF file shall indicate the version of the VBF file. No comments or non-white space characters are allowed before or after the expression. No white space characters are allowed before the expression. Note that this means the first byte of the VBF file shall always be 0x76 (i.e., ASCII 'v'). This specification is for version 2.7 of the VBF file, and so the first line shall always read:

vbf_version = 2.7;

Examples:

Valid Format:
1. vbf_version=2.7;
2. vbf_version = 2.7 ;

Invalid Format:
1. vbf_version=2.7;
2. /*Comment*/vbf_version/* Comment*/=/*Comment*/2.7/*Comment*/;
3. text vbf_version text= text 2.7 text; text

vbf_version is immediately followed by the header identifier with the exception of white space as described above. The header section as explained below contains all information needed to identify the VBF file and also information for the tester to perform a software download operation.

4.11.6.2.4.2 Header Section

991985 v1 Header section

Version history:
(1):

Purpose: To define the start of the header section.
Source:
Verification Method: Analysis AND Test
Legacy ID:

All the header information is contained in braces { }. The information between the braces in the header section shall consist entirely of expressions as defined in follow section. No comments or non-white space characters are allowed before **header** identifier or in between the **header** identifier and beginning braces ({).
Examples:

Valid Format:
1. vbf_version = 2.7; header {
2. vbf_version = 2.7;
header{
3. vbf_version=2.7;
header
{
Invalid Format:
1. vbf_version=2.7;
/*Comment*/
header{
2. header /*Comment*/ {

991987 v1 Header section - Format

Version history:
(1): Add a new identifier 'address_and_1eng_fformat_identifier'.

Purpose: To define the format of the header section
Source:
Verification Method: Analysis AND Test
Legacy ID:

The format of the header section is defined as follows (the grouping symbols [ ] indicates optional identifiers, i.e. the brackets are not to be part of the identifier defintion):

header {

[**description** = { *"text string row 1"* ,
"text string row 2" ,
"text string row 3" ,
:
"text string row n"
} ;]

sw_part_number = "part number";

sw_version = “version suffix”;

sw_part_type = *type*;

[**sw_current_part_number =** "installed part number";]
[**sw_current_version =** "installed version suffix";]

data_format_identifier = *data compression and encryption method*;

[**address_and_length_format_identifier** = *data address and length method*;]

ecu_address = *ECU target address;*

[**erase** = { { *start address 1, length 1* },
{ *start address 2, length 2* },
:
{ *start address n, length n* }
} ;]

[**call** = *start address* **;**]

[**verification_block_root_hash** = *hash value of unprocessed data to be signed;*]
[**verification_block_start** = *start of the unprocessed verification block table;*]
[**verification_block_length** = *unprocessed length of the verification block table;*]
[**sw_signature_dev =** *software signature (test and development);*]
[**sw_signature** = *software signature;*]

[**parameter_settings** = { {"*parameter name 1", "value"*}, {"*parameter name n", "value"*} };]

file_checksum = *checksum*;
}

4.11.6.2.4.2.1 description

991993 v1 **description**

Version history:
(1):

Purpose: To define the format and use of the description identifier.
Source:
Verification Method: Test
Legacy ID:

The **description** identifier is optional and can be used for a short description of the file. The **description** IdentifierValue may contain a maximum of 16 rows, with each row containing a maximum of 80 characters (bytes). Each row is contained in quotes (") and is terminated by a comma (,) or by right brace (}) followed by semicolon (;) if the row is a last row. White space within quotes is not ignored. All of the description rows shall be contained within braces whether there is one row or multiple rows. No quotes (") are allowed within a row.

Examples:

Valid Format:

```
1.  description =   {"Created: 2002-03-14"};

2.  description =   { "SOP software for S60 AWD" ,
                      "Created: 2002-03-14"
                    };
```

Invalid Format:

```
1.  description =    " Created: 2002-03-14" ;                  // Missing braces

2.  description =    "SOP software for S60 AWD" ,               // Missing braces
                     "Created: 2002-03-14";

3.  description =   { SOP software for S60 AWD,                 // Missing quotes
                      Created: 2002-03-14
                    };
4.  description =   { "SOP software for "S60" AWD"};            // Extra quotes
```

4.11.6.2.4.2.2 sw_part_number

991994 v1 **sw_part_number**

Version history:
(1):

Purpose: To define the format and use of the sw_part_number identifier.
Source:
Verification Method: Test
Legacy ID:

The **sw_part_number** IdentifierValue contains the vehicle manufacturer part number for identification of the software component (VBF file). The **sw_part_number** IdentifierValue shall be contained in quotes and shall consist of a maximum of 20 characters. No white space characters or comments are allowed within these quotes .

Examples:

Valid Format:
1. sw_part_number = "31808832"; // Part number format

Invalid Format:
1. sw_part_number = 31808832; // Missing quotes

2. sw_part_number = "318088323322256789713"; // Maximum characters > 20

3. sw_part_number = " 31808832 "; // White Space within quotes

4. sw_part_number = "/*Comment*/31808832 /*Comment*/"; // Comments within quotes

4.11.6.2.4.2.3 sw_current_part_number

992019 v1 **sw_current_part_number**

Version history:
(1):

Purpose: To define the format and use of the sw_current_part_number identifier
Source:
Verification Method: Inspection
Legacy ID:

The **sw_current_part_number** IdentifierValue contains the vehicle manufacturer part number for identification of the software component (VBF file) that is expected to be installed before a **delta** software update occurs.
This field is mandatory only for software updates with compression type **delta.**
The **sw_current_part_number** IdentifierValue shall be contained in quotes and shall consist of a maximum of 20 characters. No white space characters or comments are allowed within these quotes.

Examples:

Valid Format:
1. sw_current_part_number = "31808832"; // Part number format

Invalid Format:
1. sw_current_part_number = 31808832; // Missing quotes

2. sw_current_part_number = "318088323322256789713"; // Maximum characters > 20

3. sw_current_part_number = " 31808832 "; // White Space within quotes

4. sw_current_part_number = "/*Comment*/31808832 /*Comment*/"; // Comments within quotes

4.11.6.2.4.2.4 sw_version

992024 v1 **sw_version**

Version history:
(1):

Purpose: To define the format and use of the sw_version identifier.
Source:
Verification Method: Test
Legacy ID:

The **sw_version** IdentifierValue contains the vehicle manufacturer version for identification of the software component (VBF file). The **sw_version** IdentifierValue shall be contained in quotes and shall consist of a maximum of 4 characters (bytes). No white space characters or comments are allowed within these quotes. **sw_version** IdentifierValue is case sensitive and shall only contain the uppercase characters A-Z.

Examples:

Valid Format:
1. sw_version = "ABC"; // Version suffix format

2. sw_version = “AA"; // Version suffix format

3. sw_version = “A”; // Version suffix format

Invalid Format:
1. sw_version = ABC; // Missing quotes

2. sw_part_number = "ABCDE"; // Maximum characters > 4

3. sw_part_number = " ABC "; // White Space within quotes

4. sw_part_number = "/*Comment*/ABC /*Comment*/"; // Comments within quotes

5. sw_version = aBC; // Lowercase character

6. sw_version = 1BC; // Numerical character

4.11.6.2.4.2.5 sw_current_version

992026 v1 **sw_current_version**

Version history:
(1):

Purpose: To define the format and use of the sw_current_version identifier
Source:
Verification Method: Inspection
Legacy ID:

The **sw_current_version** IdentifierValue contains the vehicle manufacturer version for identification of the software component (VBF file) that is expected to be installed before a **delta** software update occurs.
This field is mandatory only for software updates with compression type **delta**.
The **sw_current_version** IdentifierValue shall be contained in quotes and shall consist of a maximum of 4 characters (bytes). No white space characters or comments are allowed within these quotes.
The **sw_current_version** IdentifierValue is case sensitive and shall only contain the uppercase characters A-Z.

Examples:

Valid Format:
1. sw_current_version = "ABC"; // Version suffix format

2. sw_current_version = “AA"; // Version suffix format

3. sw_current_version = “A”; // Version suffix format

Invalid Format:
1. sw_current_version = ABC; // Missing quotes

2. sw_current_version = "ABCDE"; // Maximum characters > 4

3. sw_current_version = " ABC "; // White Space within quotes

4. sw_current_version = "/*Comment*/ABC /*Comment*/"; // Comments within quotes

5. sw_current_version = aBC; // Lowercase character

6. sw_current_version = 1BC; // Numerical character

4.11.6.2.4.2.6 sw_part_type

992028 v2 **sw_part_type**

Version history:
(2): Add new sw_part_type identifier value: DIFDLT_PKG.

Purpose: To define the format and use of the sw_part_type identifier.
Source:
Verification Method: Test
Legacy ID:

The **sw_part_type** identifier indicates different types of a software parts in an ECU. Only the reserved word IdentifierValues defined in the table below shall be considered valid for the **sw_part_type** identifier.

Table: Software Part Types and Allowed Reserved Words

sw_part_type IdentifierValue Reserved Words	Description
CARCFG	Car configuration
CUSTOM	Customer parameters
DATA	Data or parameters (i.e., calibrations)
EXE	Executable
SBL	Secondary Bootloader
SIGCFG	Signal configuration
PBL	Primary Bootloader
SSBL	Special SBL which has the possibility to replace the PBL
TEST	Test program, (i.e. production test, diagnostics)
FBL	Flashable Bootloader
CERT	Certificate
PWD	Password for certificate
APP	Application file
MAP	Map file used for navigation
ESS	ECU Software Structure
SIGNED	Digital Signature file
VARIANT_INFO	Settings file (for offboard tool settings)
DIFDLT_PKG	Differential-or-delta input package used by system/tool to generate differential/delta software part.

The identifier **sw_part_type** in a single VBF file can only be assigned one of the reserved word values in the table above.

A VBF file with **sw_part_type = SBL or SSBL** is not allowed to use the identifier **erase**. These software types are downloaded to RAM and no erase operation shall be performed in this case.

Examples:

Valid Format:
1. sw_part_type = SIGCFG ;

2. sw_part_type = EXE ; // or any other valid type but "SBL" or "SSBL"
erase = { { 0x00004200, 0x00003CBF },
{ 0x00008010, 0x00002FF3 }
};

Invalid Format:
1. sw_part_type = exe; // Invalid case

2. sw_part_type = SBL; // Invalid combination of sw_part_type and erase
erase = { { 0x00004200, 0x00003CBF },
{ 0x00008010, 0x00002FF3 }
} ;

3. sw_part_type = MYOWNTYPE; // Unknown IdentifierValue

4.11.6.2.4.2.7 data_format_identifier

992031 v1 **data_format_identifier**

Version history:
(1):

Purpose: To define the format and usage of the data_format_identifier.
Source:
Verification Method: Test
Legacy ID:

The data_format_identifier is optional and when is presented with a non-zero value in the header section, it represents that a data compression/encryption method is used. Valid values for the data_format_identifier are one byte long with a range of 0x00 to 0xFF with upper nibble representing "Data Compression Method" and lower nibble representing "Data Encryption Method". Refer to REQPROD 459337: RequestDownload - dataFormatIdentifier for valid values. If the identifier is not present in the header section, it represents that neither compression nor encryption is used.

Example of Valid Format:

1. data_format_identifier = 0x00; // represents no compression and no encryption

2. data_format_identifier = 0x10; // represents compression method #1 and no encryption

3. data_format_identifier = 0x20; // represents compression method #2 and no encryption

Example of invalid Format:

1. data_format_identifier = 0x123; // Invalid format (must be from 0x00 – 0xFF)

2. data_format_identifier = {0x10}; // Extra braces

3. data_format_identifier = 23; // Invalid format (integers shall be hexadecimal)

4.11.6.2.4.2.8 address_and_length_format_identifier

992348 v1 **address_and_length_format_identifier**

Version history:
(1): New filed, support value 0x44 and 0x88.

Purpose: To identify the address and length format.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The identifier "address_and_length_format_identifier" is one byte long with constant value 0x44 or 0x88, and represents the length information of block start address and block length, which directly affects the structure of the data sector.

Note1: It is crucial information for external tools to correctly parse VBF data sectors.
Note2: Only one format is allowed for the software parts of an ECU, except for SBL. If one ECU use both formats, it requires an approval from the Basetech Team.
Note3: A VBF file with sw_part_type = SBL or SSBL or TEST is only allowed to use the address_and_length_format_identifier value 0x44.

Example of Valid Format:

1. address_and_length_format_identifier = 0x44; // represents the length information of block start address(4 bytes) and block length(4 bytes)
2. address_and_length_format_identifier = 0x88; // represents the length information of block start address(8 bytes) and block length(8 bytes)

Example of Invalid Format:

1. address_and_length_format_identifier = 44; // Invalid format (must be 0x44)
2. address_and_length_format_identifier = {0x88}; // Extra braces

4.11.6.2.4.2.9 ecu_address

992032 v1 **ecu_address**

Version history:
(1):

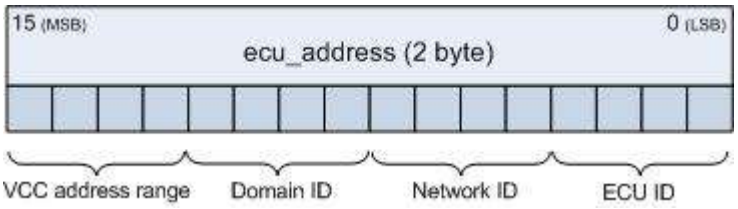
Purpose: To define the format and use of the ecu_address identifier.
Source:
Verification Method: Test
Legacy ID:

The **ecu_address** identifier indicates the physical target address of the ECU. The **ecu_address** is divided into three parts where the combination of Domain ID, Network ID and ECU ID is utilized. The **ecu_address** depends on where in the network topology the ECU is placed.

The ecu_address length is two bytes long .

The addressing format and valid address ranges for a physical target address (TA) is defined in *[VBF_2] UDS Services*. See the figure below for an overview.

Figure : ecu_address identifier format



Examples:
Valid Format:
1. ecu_address = 0x1321; // Domain ID 0x3, Network ID 0x2, ECU ID 0x1

Invalid Format:
1. ecu_address = {0x0321}; // Extra braces

4.11.6.2.4.2.10 erase

992033 v1 **erase**

Version history:
(1): Compared to VBF2.6, there is no need for an erase identifier when address_and_length_format_identifier is 0x88.

Purpose: To define the format and use of the erase identifier
Source:
Verification Method: Test
Legacy ID:

The **erase** identifier indicates the range(s) of memory that needs to be erased before a software download operation can take place. The erase start address and length need not to be matched to start address and length of a flash sector. The erase length can span over more than one flash sector, or can be within a single flash sector. The erase start address and length can automatically be derived from a Motorola S-record file or an Intel Hex file during generation of a VBF file, but there is no guarantee that this will align with acceptable flash erase segment boundaries for a given SBL. The **erase** identifier is mandatory for memory types (i.e., flash) that need to be erased before programming, expect that the ECU is having a file based memory structure, **not** requiring this identifier.

When choose address_and_length_format_identifier = 0x44, the erase start address and length are four bytes long with a range of 0x00000000 to 0xFFFFFFFF.

When choose address_and_length_format_identifier = 0x88. Then corresponding VBF header has no erase identifier.

The **erase** identifier values shall always be contained in two sets of braces whether there is one or more ranges present.

Note: If **sw_part_type = SBL or SSBL**, the VBF file shall not contain the **erase** identifier
Note: If delta encoding is used **(data_format_identifier = 0x80 or 0x90)** , the VBF file shall not contain the **erase** identifier

Example A:

address_and_length_format_identifier = 0x44

Valid Format:
1. erase = { { /*start address*/ 0x00004200, /*Length*/ 0x00003CBF }
};

2. erase = { { 0x00004200, 0x00003CBF },

```
        { 0x00008010, 0x00002FF3 }
    };                          // Two blocks: 0x00004200-0x00007EBE, 0x00008010-0x0000B002
```

Invalid Format:

```
1.  erase =   { 0x00004200, 0x00003CBF };                 // Missing outer braces

2.  erase =   { { 0xFFFFFFFE, 0x00000002 }                // Address Overflow
              };

3.  erase =   { 0x00004200, 0x00003CBF },                 // Missing outer braces
              { 0x00008010, 0x00002FF3 };

4.  erase =   { 0x00004200, 0x00003CBF,                   // Missing inner braces
                0x00008010, 0x00002FF3
              };
```

4.11.6.2.4.2.11 call

992035 v1 **call**

Version history:
(1):

Purpose: To define the format and use of the call identifier.
Source:
Verification Method: Test
Legacy ID:

The **call** entry is mandatory in VBF files with **sw_part_type** identifier values equal to SBL or SSBL and optional in VBF files with **sw_part_type** equal to TEST. For all other identifier values of **sw_part_type** , the **call** entry is not allowed. The **call** entry indicates the start address to a pre-defined executable function in ECU memory. The call identifier start address is four bytes long with a range of 0x00000000 to 0xFFFFFFFF

Examples A:

Valid Format:

```
1.    sw_part_type = SBL;
        call = 0x00000100;                  // Start address for SBL

2.    sw_part_type = SSBL;
        call = 0x00000100;                  // Start address for SSBL

3.    sw_part_type = TEST;
        call = 0x00000100;                  // Start address for TEST
```

Invalid Format:

1. sw_part_type = SBL;
   With no **call** identifier

2. sw_part_type = SSBL;
   With no **call** identifier

```
3.    sw_part_type = EXE:                   // or any other IdentifierValue but SBL, SSBL or TEST
   call = 0x00000100;
```

4.11.6.2.4.2.12 verification_block_start

992040 v1 **verification_block_start**

Version history:
(1): Apply different byte lengths based on the identifier address_and_length_format_identifier.

Purpose: To define the format and use of the verification_block_start identifier. The purpose of the identifier is to identify the start address of the data (unprocessed) that is signed.
Source:
Verification Method: Test
Legacy ID:

The **verification_block_start** represents the start address of the unprocessed data that is to be signed, according to the signing method *[General Software Authentication]*.

When **address_and_length_format_identifier** = 0x44, the **verification_block_start** address is four bytes long integer with a range of 0x00000000 to 0xFFFFFFFF.

When **address_and_length_format_identifier** = 0x88, the **verification_block_start** address is eight bytes long integer with a range of 0x0000000000000000 to 0xFFFFFFFFFFFFFFFF.

Example:

Valid Format:
1. address_and_length_format_identifier = 0x44
verification_block_start = 0x0002FF00;

2. address_and_length_format_identifier = 0x88
verification_block_start = 0x000000000002FF00;

Note: There are use cases where a data file is not signed, e.g. sw_part_type SIGNED or prior to a file has been released, approved and signed.

4.11.6.2.4.2.13 verification_block_length

992045 v1 **verification_block_length**

Version history:
(1): Apply different byte lengths based on the identifier address_and_length_format_identifier.

Purpose: To define the format and use of the verification_block_length identifier. The purpose of the identifier is to identify the length of the unprocessed data that is signed.
Source:
Verification Method: Test
Legacy ID:

The **verification_block_length** represents the unprocessed length (prior to compression/encryption is applied) of the data that is to be signed, according to the signing method *[General Software Authentication]*.

When **address_and_length_format_identifier** = 0x44, the **verification_block_length** is four bytes long integer with a range of 0x00000000 to 0xFFFFFFFF.

When **address_and_length_format_identifier** = 0x88, the **verification_block_length** is eight bytes long integer with a range of 0x0000000000000000 to 0xFFFFFFFFFFFFFFFF.

Example:

Valid Format:
1. address_and_length_format_identifier = 0x44
verification_block_length = 0x00000054;

2. address_and_length_format_identifier = 0x88
verification_block_length = 0x0000000000000054;

Note: There are use cases where a data file is not signed, e.g. sw_part_type SIGNED or prior to a file has been released, approved and signed.

4.11.6.2.4.2.14 verification_block_root_hash

992049 v1 **verification_block_root_hash**

Version history:
(1):

Purpose: To define the format and use of the verification_block_root_hash identifier. The purpose of the hash value is to provide the Trust Centre (signing entity) with required information of the unprocessed data (no compression/encryption method applied).
Source:
Verification Method: Test
Legacy ID:

The verification_block_root_hash represents the hash value of the unprocessed data that is to be signed, according to the signing method [General Software Authentication]. The verification_block_root_hash shall be represented as an integer and the length is dependent of the hashing method used.

Example:

Valid Format:
1. verification_block_root_hash =
0xF1F62AC8EDFECD610234DBDA84F7561998ADC47A0F0EDE6CA23F0FC22974F9F4;

Note: There are use cases when a data file is not signed, e.g. prior to a file has been released, approved and signed.

4.11.6.2.4.2.15 sw_signature

992051 v1 **sw_signature**

Version history:
(1):

Purpose: To define the format and use of the sw_signature identifier. The purpose of the digital signature is to validate the authenticity of the programmed data in the ECU.
Source:
Verification Method: Test
Legacy ID:

The sw_signature represents the digital signature according to the applied signature method [General Software Authentication]. The sw_signature shall be represented as an integer and the length is dependent of the signing method used.

Example:

Valid Format:
1. sw_signature =
0x514E3E55831E43E89D4118E2F29F4996C3A8ECBD11B1CDB8F4940AE4BF8BF866C28EA6F1B4B1458A743
4788F0460434368ACE9863FDFF7F61E90AF2A95421A6757E553115EC00E37274066678AEFD54DD963637203
FAEA496F44C79EC11E706C3911749374AED7BCB7C1768E2EB09400D6146D8771BC6A5C0B10D420E785BCA
D91ED553537A43CFE9EC0A48B66179416D9615A1108588005568F46331E2F5BD13F10005A7F0CE2B1AA820
CF34C7C297F6CFDAC7A436D6C0FA2B0551C032B63FAB04DD5A2E8F54B78BAD9CE609C0B022E93F716D85
7616D2FAAD2BBEEBDFB162A6330123FCB23FB52D56ECBD449DB89594B99768AD3B6CBDB222DB47359A9
D48F;

Note: There are use cases where a data file is not signed, e.g. prior to a file has been released, approved and signed.
Note: In production, e.g. at manufacturing and workshop systems, this information is to be transferred in a separate (non-vbf) file that is defined elsewhere.

4.11.6.2.4.2.16 sw_signature_dev

992054 v1 **sw_signature_dev**

Version history:
(1):

Purpose: To define the format and use of the sw_signature_dev identifier. The purpose of the digital signature is to validate the authenticity of the programmed data in the ECU for test & development objects.
Source:
Verification Method: Test
Legacy ID:

The sw_signature_dev represents the digital signature for test and development according to the applied signature method *[General Software Authentication]*. The sw_signature_dev shall be represented as an integer and the length is dependent of the signing method used.

Example:

The software release candidate is signed with a dedicated key for development and the resulting digital signature is stored at the sw_signature_dev. A software successfully verified and ready for release (production) is signed with the dedicated key for production (append sw_signature), but the sw_signature_dev is still kept. Refer to sw_signature identifier for format example.

4.11.6.2.4.2.17 file_checksum

992061 v1 **file_checksum**

Version history:
(1):

Purpose: To define the format and use of the file_checksum identifier.
Source:
Verification Method: Test
Legacy ID:

A four-byte file checksum shall be provided in the header section with a range of 0x00000000 to 0xFFFFFFFF. The purpose of this checksum is to validate the integrity of the file itself prior to attempting a download. The checksum shall be a CRC32 of all the data contained in the data section of the file, including start address, length and checksum for all data blocks. This means that if the data bytes to download are compressed (i.e., data_format_identifier contains a non-zero value), then the file checksum is performed using the compressed data.

The tester shall check this checksum against the file checksum it has calculated before the software download operation is performed with the VBF file.

The file checksum shall meet the following criteria:
· 32 bit CRC, polynomial = 0x04C11DB7
(x^32 + x^26 + x^23 + x^22 + x^16 + x^12 + x^11 + x^10 + x^8 + x^7 + x^5 + x^4 + x^2 + x^1 + 1)
· Initial value = 0xFFFFFFFFFFFFFFFF
· Bit reversed

For a fast CRC32 calculation a look-up table implementation is the preferred solution.
See Appendix B for "C-code" example.

Example:
file_checksum = 0x29058c73

Note: This is the same CRC32 algorithm as used by "WinZip" (among many others).

992062 v1 **file_checksum - no data blocks**

Version history:
(1):

Purpose: Define the file_checksum value for a data file with no data blocks. This is a special case when the data file is used as an information carrier to offboard tool-chain, i.e. no data downloaded to the ECU.

Source:

Verification Method: Test

Legacy ID:

The file_checksum for a data file having no data blocks shall be 0xFFFFFFFF.

4.11.6.2.4.2.18 parameter_settings

992063 v1 **parameter_settings**

Version history:
(1):

Purpose: The file_checksum for a data file having no data blocks shall be 0xFFFFFFFF.

Source:

Verification Method: Test

Legacy ID:

The optional **parameter_settings** identifier is used to define and carry configurations required by off-board tools. To be able to define multiple parameters using this identifier, a parameter is defined by a "name" and "value" pair. Each "name" and "value" pair must be contained in two sets of braces and every pair is to be separated by a comma. A single element of each pair, i.e. the "name" and "value" respectively, must be quoted and separated by a comma. The overall parameter_settings definitions must be embraced by two outer braces whether there is one or more "name" and "value" pairs defined.

Example 1:
Valid format:

1. parameter_settings = {{"swAuthKeyAlgorithm", "RSA4096"}};
2. parameter_settings = {{"nameOne", "VALUE1"}, {"nameTwo", "VALUE2"}, {"nameThree", "VALUE3"}};

Invalid format:
1. parameter_settings = {"swAuthKeyAlgorithm", "RSA4096"}; //Missing outer braces
2. parameter_settings = {{"swAuthKeyAlgorithm"}}; //Incomplete pair

Example 2:

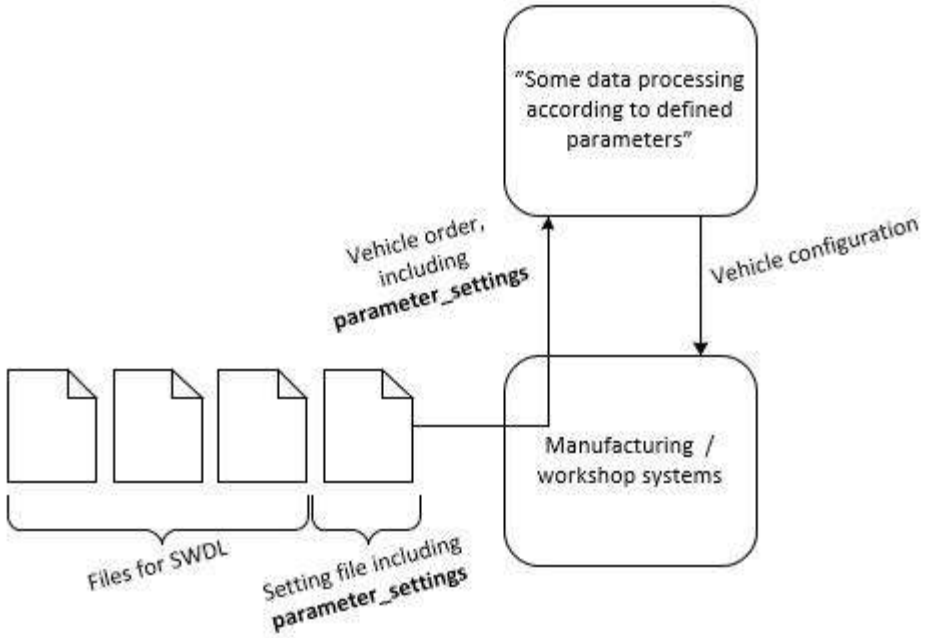


Figure - Usage of parameter_settings

The *Figure - Usage of parameter_settings* shows where off-board tools detects that some configuration is required and reacts according to the parameter_settings.
Notes: *This requirement does neither specify the applicable "name" and "value" pairs nor when the identifier is required. These are to be specified by the concept(s) requiring the parameter_settings identifier. For example, "nameOne", "VALUE1" shall only be considered as examples of the format and not valid element names. See e.g. "REQPROD 456934: Software Settings - Definition"*

4.11.6.2.4.2.19 Header section example #1

The first example is an VBF-file with **sw_part_type** = EXE.

Figure: Header section example #1:

```
vbf_version = 2.7;
header {

   description = { "Application SW DIM"};

  // Software part number
   sw_part_number = "1122334455";
```

```
   // Software version
   sw_version = "A";

   // Software type
   sw_part_type = EXE;

   // Compression and Encryption Method
   data_format_identifier = 0x10;


   // Address and Length Format
   address_and_length_format_identifier = 0x88;

   // ECU address
   ecu_address = 0x1321;


   //Verification block root hash value
   verification_block_root_hash =
0xFC10397BEABFBFF17519DCC6144EB4D86429CCC5682D8ED35AF8FBA984F26ED2;

   //Start address of the verification block
   verification_block_start = 0x000000000005FF00;

   //Length of the verification block
   verification_block_length = 0x0000000000000054;

   //Software signature for test and development purposes
   sw_signature_dev =
0x16956823884E89FBFB1AFB81055C43732E30B066DB24AD7B4D7BF8ABB4EA3C98F2839C3443FF267A39
37E49ED4E50E028C94FAD8B214930CEE37C18111D8548F6B3D706FE32E1CA5B4C245276A9396762233B1
7C9A4DE258EA7EA1DF1D8E1006E5B1FE1061441864A9718A2F696A89615ABDC3013F1F46682A03FAAB92
ADFD43858F6D159020B09632FF83AD1F29525A43455CF21B8D6800CC88D439D87651295552B3D4414DF5
5D764172595B59AF2F354D27F75661565798FABFEF6638E5BF92ABECEAC7F119E893C6BD238A464C1CC9
65E084690650D048AE51F43295857DF5B450F60C5C19D2699C385090651B94282C3FE51703F249DAB626
44EEDC07EF;

   //Software signature
   sw_signature =
0x514E3EB5831E43E89D5118E2F29F4996C3A8ECBD11B4CDB8F4940AE4BF8BC866C28EA6A1B4B1458A74
34788F0460434368ACE9863FDFF7F61E90AF2A95421A6757E553115EC00E37274066678AEFD54DD96363
7203FAEAA96F44C79EC11E706C3921F49374AED7BCB7C1768E2EB09400D6146D8771BC6A5C0B10D420E7
85BCAD91ED553537A43CFE9EC0A48B66179416D9615A1108588005568F46331E2F5BD13F10005A7F0CE2
B1AA820CF3467C297F6CFDAC7A136D6C0FA2B0551C038B63FAB04DD5A2E8F54B78BAD9CE609C0B022E93
F716D857616D2FAAD2BBEEBDFB162A6330123FCB23FB52D56ECBD449DB89594B99768AD3B6CBDB222DB4
7359A9D48F;

   // File checksum
   file_checksum = 0x1A47EDA3;
}
<binary data section follows>
```

4.11.6.2.4.2.20 Header section example #2

The first example is an VBF-file with **sw_part_type** = EXE.

Figure: Header section example #1:

```
vbf_version = 2.7;
header {

   description = { "Application SW DIM"};
```

```
  // Software part number
   sw_part_number = "1122334455";

   // Software version
   sw_version = "A";

   // Software type
   sw_part_type = EXE;

   // Compression and Encryption Method
   data_format_identifier = 0x10;


   // Address and Length Format
   address_and_length_format_identifier = 0x44;

   // ECU address
   ecu_address = 0x1321;

   // Erase information
   // start length
   erase = {0x000200000, 0x00040000};

   //Verification block root hash value
   verification_block_root_hash =
0xFC10397BEABFBFF17519DCC6144EB4D86429CCC5682D8ED35AF8FBA984F26ED2;

   //Start address of the verification block
   verification_block_start = 0x0005FF00;

   //Length of the verification block
   verification_block_length = 0x00000054;

   //Software signature for test and development purposes
   sw_signature_dev =
0x16956823884E89FBFB1AFB81055C43732E30B066DB24AD7B4D7BF8ABB4EA3C98F2839C3443FF267A39
37E49ED4E50E028C94FAD8B214930CEE37C18111D8548F6B3D706FE32E1CA5B4C245276A9396762233B1
7C9A4DE258EA7EA1DF1D8E1006E5B1FE1061441864A9718A2F696A89615ABDC3013F1F46682A03FAAB92
ADFD43858F6D159020B09632FF83AD1F29525A43455CF21B8D6800CC88D439D87651295552B3D4414DF5
5D764172595B59AF2F354D27F75661565798FABFEF6638E5BF92ABECEAC7F119E893C6BD238A464C1CC9
65E084690650D048AE51F43295857DF5B450F60C5C19D2699C385090651B94282C3FE51703F249DAB626
44EEDC07EF;

   //Software signature
   sw_signature =
0x514E3EB5831E43E89D5118E2F29F4996C3A8ECBD11B4CDB8F4940AE4BF8BC866C28EA6A1B4B1458A74
34788F0460434368ACE9863FDFF7F61E90AF2A95421A6757E553115EC00E37274066678AEFD54DD96363
7203FAEAA96F44C79EC11E706C3921F49374AED7BCB7C1768E2EB09400D6146D8771BC6A5C0B10D420E7
85BCAD91ED553537A43CFE9EC0A48B66179416D9615A1108588005568F46331E2F5BD13F10005A7F0CE2
B1AA820CF3467C297F6CFDAC7A136D6C0FA2B0551C038B63FAB04DD5A2E8F54B78BAD9CE609C0B022E93
F716D857616D2FAAD2BBEEBDFB162A6330123FCB23FB52D56ECBD449DB89594B99768AD3B6CBDB222DB4
7359A9D48F;

   // File checksum
   file_checksum = 0x1A47EDA3;
}
<binary data section follows>
```

4.11.6.2.4.2.21 Header section example #3

The second example is a VBF-file with **sw_part_type** = SBL.

Figure: Header section example #2:

```
vbf_version = 2.7;
header {

   //This is a comment
   /*This is also a comment */

   //Short description of the VBF file
      description= {"SBL for DIM",
      "Created 2015-10-10"
   };
   //Software part number
   sw_part_number= "8888326699";

   //Software version
   sw_version= "A";

   //Software type
   sw_part_type= SBL;

   //Compression and Encryption Method
   data_format_identifier= 0x10;

   // Address and Length Format
   address_and_length_format_identifier = 0x44;

   //ECU address
   ecu_address= 0x1123;

   //Start address for the SBL
   call= 0x00000100;

   //Verification block root hash value
   verification_block_root_hash =
   0xAA10397BEABFBFF17519DCC6144EB4D86429CCC5682D8ED35AF8FBA984F26ED2;

   //Start address of the verification block
   verification_block_start = 0x0001FF00;

   //Length of the verification block
   verification_block_length = 0x0000002C;

   //Software signature for test and development purposes
   sw_signature_dev =
   0x7EE874D0FFEBC4CA63AA3377AB3FACBC839690F903B96F30F4968D18101BDAD72EDC343E6A07644
69D47AAEFD92F416E7C44502CCD31274450DD5952E8F1C2B75BFFBC157C112AD1E7B4F46C99B89EED1BA
DB38D06C6B21036A55E0FF9B2EFFEC4DBB5856C7CD946F432AC2E3CD41CE049D05C728F8C93EEF0AAA67
03245ACD136BC3B33691781EFE67430AEBCA99B0FBCF50CD1D30A9CC7DC4D81B35208788375727E0DD01
06B46BBB80DB3887DFD2D69DF0B4ADCD7BD69AF24C72F55E357457CB1D389A4976344417EF8A730BBB5D
4D8C22BA8D1258CEBEE9B218AF1D5D54328C47BF4D0FA8567F1AB123512BEE45828DE1D4E3440DF182D1
637737BA8C77D;

   //Software signature
   sw_signature =
   0xAC4E3EB5831E43E89D5118E2F29F4996C3A8ECBD11B4CDB8F4940AE4BF8BC866C28EA6A1B4B1458
A7434788F0460434368ACE9863FDFF7F61E90AF2A95421A6757E553115EC00E37274066678AEFD54DD96
3637203FAEAA96F44C79EC11E706C3921F49374AED7BCB7C1768E2EB09400D6146D8771BC6A5C0B10D42
0E785BCAD91ED553537A43CFE9EC0A48B66179416D9615A1108588005568F46331E2F5BD13F10005A7F0
CE2B1AA820CF3467C297F6CFDAC7A136D6C0FA2B0551C038B63FAB04DD5A2E8F54B78BAD9CE609C0B022
E93F716D857616D2FAAD2BBEEBDFB162A6330123FCB23FB52D56ECBD449DB89594B99768AD3B6CBDB222
DB47359A9D48F;
```

```
   // File checksum
   file_checksum = 0xA1DC5A45;
}
<binary data section follows>
```

4.11.6.2.4.2.22 Header section example #4

A VBF-file containing delta encoded data, with **data_format_identifier = 0x80**. Note that the identifiers for Software Authentication, e.g. the sw_signature, are for the full target image (after delta enconding has been applied)

Figure: Header section example #3:

```
vbf_version = 2.7;
header {

   //Description
   description = {"SWLM (Delta) for SUM from Version 31429076AD to 31429077AA"
                  };
   //Software part number (New part number after this update)
   sw_part_number = "8866314291";

   //Software version (New software version after this update)
   sw_version = "A";

   //Current Software part number
      //(Expected part number, installed before this update)
   sw_current_part_number = "8866314290";

   //Current Software version
      //(Expected software version, installed before this update)
   sw_current_version = "B";

   //Software part type
   sw_part_type = DATA;

   //Data Format Identifier
   //Delta encoded data and no compression method
   data_format_identifier = 0x80;

   // Address and Length Format
   address_and_length_format_identifier = 0x88;

   //ECU address
   ecu_address= 0x1123;

   //Verification block root hash value
   verification_block_root_hash =
0xAA10397BEABFBFF17519DCC6144EB4D86429CCC5682D8ED35AF8FBA984F26ED2;

   //Start address of the verification block
   verification_block_start = 0x000000000001FF00;

   //Length of the verification block
   verification_block_length = 0x0000000000000054;

   //Software signature for test and development purposes
   sw_signature_dev =
0x073A43404527ABFC1C70F637FBD689A83B0C14172C6C588268F80F7021CD82DF9489C676452B7B1BB8
5D538B670F4E14FA71933AD1612E75F66574955B82755B0FF1A29D4D5A9421EED6D9E705098CCD5F4E2A
71C526D8DD90AF02212EB715937A3D49311B385E1C3B754C90F0EFE20BB22240582A8FD03BF6FF04B59C
114AA7B07D8BDCCBCEC77A8E00CCC9D1CE18EC446A8DA45756CF3CFD594134859E407C7421D50221B1D7
1A1976AA68E36D5642FA762B1DAFBB9804FFD8C479CA037307D8AC522AC4CB946248064E5A789797B1B5
5760EAC40FBC7E616F848C8A4B389899FB8DEF7DDCA4933E2F11C1638D05F66144AEC30E8DBAF2317827
```

```
DB27438560;

   //Software signature
   sw_signature =
0xAC4E3EB5831E43E89D5118E2F29F4996C3A8ECBD11B4CDB8F4940AE4BF8BC866C28EA6A1B4B1458A74
34788F0460434368ACE9863FDFF7F61E90AF2A95421A6757E553115EC00E37274066678AEFD54DD96363
7203FAEAA96F44C79EC11E706C3921F49374AED7BCB7C1768E2EB09400D6146D8771BC6A5C0B10D420E7
85BCAD91ED553537A43CFE9EC0A48B66179416D9615A1108588005568F46331E2F5BD13F10005A7F0CE2
B1AA820CF3467C297F6CFDAC7A136D6C0FA2B0551C038B63FAB04DD5A2E8F54B78BAD9CE609C0B022E93
F716D857616D2FAAD2BBEEBDFB162A6330123FCB23FB52D56ECBD449DB89594B99768AD3B6CBDB222DB4
7359A9D48F;

   // File checksum
   file_checksum = 0xA1DC5A45;
}
<binary data section follows>
```

4.11.6.2.4.3 Data Section

991992 v1 **Data section - start**

Version history:
(1):

Purpose: To define the start of the data section.
Source:
Verification Method: Analysis AND Test
Legacy ID:

After the header section ends, the data section immediately follows. The header section ends with a } character (ASCII 0x7D). The binary data section begins with the next byte immediately following this ASCII } character. The data section might be omitted, i.e. a data file could contain zero data blocks for some special cases.

992075 v1 **Data section - structure**

Version history:
(1):

Purpose: To define the structure of the data section.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The data section may contain more than one block of data, and every block includes four parts.

When address_and_length_format_identifier = 44:

1. 4-byte start address, physical address in ECU memory, with a range of 0x00000000 to 0xFFFFFFFF.
2. 4-byte length, number of data bytes in block (excluding Start address, Length and Checksum), with a range of 0x00000001 to 0xFFFFFFFF. If data compression is used, this length represents the compressed length.
3. Data
4. 2-byte checksum for the data block (excluding Start address and Length) with a range of 0x0000 to 0xFFFF.

The Start address, Length, and Checksum fields shall always be stored with the most significant byte (msb) first.

Table: Data Section

Version Section	VBF Version Number	Text	vbf_version=2.7;
Header section	Header	Text	n bytes, ASCII
Data section	Block 1	Start address	4 bytes, binary

		Length	4 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary
	Block 2	Start address	4 bytes, binary
		Length	4 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary
	:	:	:
	Block n	Start address	4 bytes, binary
		Length	4 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary

When address_and_length_format_identifier = 0x88:

1. 8-byte start address, physical address in ECU memory, with a range of 0x0000000000000000 to 0xFFFFFFFFFFFFFFFF.
2. 8-byte length, number of data bytes in block (excluding Start address, Length and Checksum), with a range of 0x0000000000000001 to 0xFFFFFFFFFFFFFFFF. If data compression is used, this length represents the compressed length.
3. Data
4. 2-byte checksum for the data block (excluding Start address and Length) with a range of 0x0000 to 0xFFFF.

The Start address, Length, and Checksum fields shall always be stored with the most significant byte (msb) first.

Table: Data Section

Version Section	VBF Version Number	Text	vbf_version=2.7;
Header section	Header	Text	n bytes, ASCII
Data section	Block 1	Start address	8 bytes, binary
		Length	8 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary
	Block 2	Start address	8 bytes, binary
		Length	8 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary
	:	:	:
	Block n	Start address	8 bytes, binary
		Length	8 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary

992074 v1 **Software part version in the data section**

Version history:
(1):

Purpose: To prevent resistant software part version stored in the ECU.
Source: NA
Verification Method: Analysis
Legacy ID:

The software part version containing sw_part_number and sw_version shall reside somewhere in the data section in the VBF file that is being programmed into the ECU and this downloaded part number shall be readable using service 0x22 (ReadDataByIdentifier).

4.11.6.2.4.3.1 Checksum calculation

992076 v1 **Data block checksum**

Version history:
(1):

Purpose: To define the format and use of the data block checksum.
Source:
Verification Method: Test
Legacy ID:

The VBF file shall include a 2-byte checksum for every data block. The 2-byte checksum shall be calculated including all data bytes (excluding Start address, Length, and Checksum) in the data block. If a data compression method is used, the data block checksum shall be calculated using unprocessed data (i.e. uncompressed and/or not encrypted). Block checksums should not be used to verify the integrity of the VBF file.

The checksum algorithm to be used shall be the CRC16-CITT:

- Polynomial = 0x1021 (x^16+x^12+x^5+1)
- Initial value = 0xFFFF

For a fast CRC16-CITT calculation a look-up table implementation is the preferred solution. See Appendix A for "C-code" example.

•Appendix A

4.11.6.2.5 Appendix A

4.11.6.2.5.1 CRC16-CITT C-code example

This example uses a look-up table with pre-calculated CRCs for fast calculation.

```
/* function declarations */
unsigned int CalcCRC(unsigned int size,unsigned char *data);

/* test data array */
unsigned char data[256] =
{  0x00,0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08,0x09,0x0a,0x0b,0x0c,0x0d,0x0e,0x0f,
   0x10,0x11,0x12,0x13,0x14,0x15,0x16,0x17,0x18,0x19,0x1a,0x1b,0x1c,0x1d,0x1e,0x1f,
   0x20,0x21,0x22,0x23,0x24,0x25,0x26,0x27,0x28,0x29,0x2a,0x2b,0x2c,0x2d,0x2e,0x2f,
   0x30,0x31,0x32,0x33,0x34,0x35,0x36,0x37,0x38,0x39,0x3a,0x3b,0x3c,0x3d,0x3e,0x3f,
   0x40,0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48,0x49,0x4a,0x4b,0x4c,0x4d,0x4e,0x4f,
   0x50,0x51,0x52,0x53,0x54,0x55,0x56,0x57,0x58,0x59,0x5a,0x5b,0x5c,0x5d,0x5e,0x5f,
   0x60,0x61,0x62,0x63,0x64,0x65,0x66,0x67,0x68,0x69,0x6a,0x6b,0x6c,0x6d,0x6e,0x6f,
   0x70,0x71,0x72,0x73,0x74,0x75,0x76,0x77,0x78,0x79,0x7a,0x7b,0x7c,0x7d,0x7e,0x7f,
   0x80,0x81,0x82,0x83,0x84,0x85,0x86,0x87,0x88,0x89,0x8a,0x8b,0x8c,0x8d,0x8e,0x8f,
   0x90,0x91,0x92,0x93,0x94,0x95,0x96,0x97,0x98,0x99,0x9a,0x9b,0x9c,0x9d,0x9e,0x9f,
   0xa0,0xa1,0xa2,0xa3,0xa4,0xa5,0xa6,0xa7,0xa8,0xa9,0xaa,0xab,0xac,0xad,0xae,0xaf,
   0xb0,0xb1,0xb2,0xb3,0xb4,0xb5,0xb6,0xb7,0xb8,0xb9,0xba,0xbb,0xbc,0xbd,0xbe,0xbf,
   0xc0,0xc1,0xc2,0xc3,0xc4,0xc5,0xc6,0xc7,0xc8,0xc9,0xca,0xcb,0xcc,0xcd,0xce,0xcf,
   0xd0,0xd1,0xd2,0xd3,0xd4,0xd5,0xd6,0xd7,0xd8,0xd9,0xda,0xdb,0xdc,0xdd,0xde,0xdf,
   0xe0,0xe1,0xe2,0xe3,0xe4,0xe5,0xe6,0xe7,0xe8,0xe9,0xea,0xeb,0xec,0xed,0xee,0xef,
   0xf0,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7,0xf8,0xf9,0xfa,0xfb,0xfc,0xfd,0xfe,0xff
};

unsigned int crctab[256] =
{
   0x0000, 0x1021, 0x2042, 0x3063, 0x4084, 0x50A5, 0x60C6, 0x70E7,
```

```
    0x8108, 0x9129, 0xA14A, 0xB16B, 0xC18C, 0xD1AD, 0xE1CE, 0xF1EF,
    0x1231, 0x0210, 0x3273, 0x2252, 0x52B5, 0x4294, 0x72F7, 0x62D6,
    0x9339, 0x8318, 0xB37B, 0xA35A, 0xD3BD, 0xC39C, 0xF3FF, 0xE3DE,
    0x2462, 0x3443, 0x0420, 0x1401, 0x64E6, 0x74C7, 0x44A4, 0x5485,
    0xA56A, 0xB54B, 0x8528, 0x9509, 0xE5EE, 0xF5CF, 0xC5AC, 0xD58D,
    0x3653, 0x2672, 0x1611, 0x0630, 0x76D7, 0x66F6, 0x5695, 0x46B4,
    0xB75B, 0xA77A, 0x9719, 0x8738, 0xF7DF, 0xE7FE, 0xD79D, 0xC7BC,
    0x48C4, 0x58E5, 0x6886, 0x78A7, 0x0840, 0x1861, 0x2802, 0x3823,
    0xC9CC, 0xD9ED, 0xE98E, 0xF9AF, 0x8948, 0x9969, 0xA90A, 0xB92B,
    0x5AF5, 0x4AD4, 0x7AB7, 0x6A96, 0x1A71, 0x0A50, 0x3A33, 0x2A12,
    0xDBFD, 0xCBDC, 0xFBBF, 0xEB9E, 0x9B79, 0x8B58, 0xBB3B, 0xAB1A,
    0x6CA6, 0x7C87, 0x4CE4, 0x5CC5, 0x2C22, 0x3C03, 0x0C60, 0x1C41,
    0xEDAE, 0xFD8F, 0xCDEC, 0xDDCD, 0xAD2A, 0xBD0B, 0x8D68, 0x9D49,
    0x7E97, 0x6EB6, 0x5ED5, 0x4EF4, 0x3E13, 0x2E32, 0x1E51, 0x0E70,
    0xFF9F, 0xEFBE, 0xDFDD, 0xCFFC, 0xBF1B, 0xAF3A, 0x9F59, 0x8F78,
    0x9188, 0x81A9, 0xB1CA, 0xA1EB, 0xD10C, 0xC12D, 0xF14E, 0xE16F,
    0x1080, 0x00A1, 0x30C2, 0x20E3, 0x5004, 0x4025, 0x7046, 0x6067,
    0x83B9, 0x9398, 0xA3FB, 0xB3DA, 0xC33D, 0xD31C, 0xE37F, 0xF35E,
    0x02B1, 0x1290, 0x22F3, 0x32D2, 0x4235, 0x5214, 0x6277, 0x7256,
    0xB5EA, 0xA5CB, 0x95A8, 0x8589, 0xF56E, 0xE54F, 0xD52C, 0xC50D,
    0x34E2, 0x24C3, 0x14A0, 0x0481, 0x7466, 0x6447, 0x5424, 0x4405,
    0xA7DB, 0xB7FA, 0x8799, 0x97B8, 0xE75F, 0xF77E, 0xC71D, 0xD73C,
    0x26D3, 0x36F2, 0x0691, 0x16B0, 0x6657, 0x7676, 0x4615, 0x5634,
    0xD94C, 0xC96D, 0xF90E, 0xE92F, 0x99C8, 0x89E9, 0xB98A, 0xA9AB,
    0x5844, 0x4865, 0x7806, 0x6827, 0x18C0, 0x08E1, 0x3882, 0x28A3,
    0xCB7D, 0xDB5C, 0xEB3F, 0xFB1E, 0x8BF9, 0x9BD8, 0xABBB, 0xBB9A,
    0x4A75, 0x5A54, 0x6A37, 0x7A16, 0x0AF1, 0x1AD0, 0x2AB3, 0x3A92,
    0xFD2E, 0xED0F, 0xDD6C, 0xCD4D, 0xBDAA, 0xAD8B, 0x9DE8, 0x8DC9,
    0x7C26, 0x6C07, 0x5C64, 0x4C45, 0x3CA2, 0x2C83, 0x1CE0, 0x0CC1,
    0xEF1F, 0xFF3E, 0xCF5D, 0xDF7C, 0xAF9B, 0xBFBA, 0x8FD9, 0x9FF8,
    0x6E17, 0x7E36, 0x4E55, 0x5E74, 0x2E93, 0x3EB2, 0x0ED1, 0x1EF0
};

/* driver */
void main(void)
{
   unsigned int crc;

   crc=CalcCRC(256,data);

   while(1); /* the result of the above calculation shall be: crc=0x3FBD */
}


/* Calculate CRC */
unsigned int CalcCRC(unsigned int size, unsigned char *data)
{
   unsigned int crc=0xffff; /* initial value */
   int tmp;
   int i;

   for(i=0;i<size;i++) {
      tmp=(crc>>8)^data[i];
      crc=(crc<<8)^crctab[tmp];
   }
   return crc;
}
```

•Appendix B

4.11.6.2.6 Appendix B

4.11.6.2.6.1 CRC32 C-code example

This example uses a look-up table with pre-calculated CRCs for fast calculation.

```
/* function declarations */
void generate_crc32_table(void);
unsigned long calc_crc32(unsigned long size, unsigned char *data);

/* variables */
unsigned long crc32;
static unsigned long crc32_table[256];

/* test data array */
unsigned char data[256] =
{  0x00,0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08,0x09,0x0a,0x0b,0x0c,0x0d,0x0e,0x0f,
   0x10,0x11,0x12,0x13,0x14,0x15,0x16,0x17,0x18,0x19,0x1a,0x1b,0x1c,0x1d,0x1e,0x1f,
   0x20,0x21,0x22,0x23,0x24,0x25,0x26,0x27,0x28,0x29,0x2a,0x2b,0x2c,0x2d,0x2e,0x2f,
   0x30,0x31,0x32,0x33,0x34,0x35,0x36,0x37,0x38,0x39,0x3a,0x3b,0x3c,0x3d,0x3e,0x3f,
   0x40,0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48,0x49,0x4a,0x4b,0x4c,0x4d,0x4e,0x4f,
   0x50,0x51,0x52,0x53,0x54,0x55,0x56,0x57,0x58,0x59,0x5a,0x5b,0x5c,0x5d,0x5e,0x5f,
   0x60,0x61,0x62,0x63,0x64,0x65,0x66,0x67,0x68,0x69,0x6a,0x6b,0x6c,0x6d,0x6e,0x6f,
   0x70,0x71,0x72,0x73,0x74,0x75,0x76,0x77,0x78,0x79,0x7a,0x7b,0x7c,0x7d,0x7e,0x7f,
   0x80,0x81,0x82,0x83,0x84,0x85,0x86,0x87,0x88,0x89,0x8a,0x8b,0x8c,0x8d,0x8e,0x8f,
   0x90,0x91,0x92,0x93,0x94,0x95,0x96,0x97,0x98,0x99,0x9a,0x9b,0x9c,0x9d,0x9e,0x9f,
   0xa0,0xa1,0xa2,0xa3,0xa4,0xa5,0xa6,0xa7,0xa8,0xa9,0xaa,0xab,0xac,0xad,0xae,0xaf,
   0xb0,0xb1,0xb2,0xb3,0xb4,0xb5,0xb6,0xb7,0xb8,0xb9,0xba,0xbb,0xbc,0xbd,0xbe,0xbf,
   0xc0,0xc1,0xc2,0xc3,0xc4,0xc5,0xc6,0xc7,0xc8,0xc9,0xca,0xcb,0xcc,0xcd,0xce,0xcf,
   0xd0,0xd1,0xd2,0xd3,0xd4,0xd5,0xd6,0xd7,0xd8,0xd9,0xda,0xdb,0xdc,0xdd,0xde,0xdf,
   0xe0,0xe1,0xe2,0xe3,0xe4,0xe5,0xe6,0xe7,0xe8,0xe9,0xea,0xeb,0xec,0xed,0xee,0xef,
   0xf0,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7,0xf8,0xf9,0xfa,0xfb,0xfc,0xfd,0xfe,0xff
};

/* driver */
void main(void)
{
   unsigned long crc32;

   generate_crc32_table();
   crc32=calc_crc32(256,data);

   while(1); /* the result of the above calculation shall be: crc32=0x29058c73 */
}

/* calculate the crc32 value */
unsigned long calc_crc32(unsigned long size, unsigned char *data)
{

   unsigned long i, crc;

   crc=0xffffffff;       // initial value

   for(i=0;i<size;i++) {
      crc = ((crc>>8)&0x00FFFFFF)^crc32_table[(int)(crc^data[i])&0xff];
   }

   return ~crc;
}

#define CRC32_POLY 0xedb88320 // 0x04c11db7 bit reversed
```

```
/* generate the lock-up table */
void generate_crc32_table(void)
{
   int i, j;
   unsigned long crc;

   for(i=0;i<256;i++) {
      crc=i;
      for(j=8;j>0;j--) {
         if(crc&1) {
            crc=(crc>>1)^CRC32_POLY;
         }
         else{
            crc>>=1;
         }
      }
      crc32_table[i]=crc;
   }
}

// As an alternative to the "generate_crc32_table()" function the following table
can be used

/*
static unsigned long crc32_tab[256] =
{  0x00000000L, 0x77073096L, 0xee0e612cL, 0x990951baL, 0x076dc419L, 0x706af48fL,
   0xe963a535L, 0x9e6495a3L, 0x0edb8832L, 0x79dcb8a4L, 0xe0d5e91eL, 0x97d2d988L,
   0x09b64c2bL, 0x7eb17cbdL, 0xe7b82d07L, 0x90bf1d91L, 0x1db71064L, 0x6ab020f2L,
   0xf3b97148L, 0x84be41deL, 0x1adad47dL, 0x6ddde4ebL, 0xf4d4b551L, 0x83d385c7L,
   0x136c9856L, 0x646ba8c0L, 0xfd62f97aL, 0x8a65c9ecL, 0x14015c4fL, 0x63066cd9L,
   0xfa0f3d63L, 0x8d080df5L, 0x3b6e20c8L, 0x4c69105eL, 0xd56041e4L, 0xa2677172L,
   0x3c03e4d1L, 0x4b04d447L, 0xd20d85fdL, 0xa50ab56bL, 0x35b5a8faL, 0x42b2986cL,
   0xdbbbc9d6L, 0xacbcf940L, 0x32d86ce3L, 0x45df5c75L, 0xdcd60dcfL, 0xabd13d59L,
   0x26d930acL, 0x51de003aL, 0xc8d75180L, 0xbfd06116L, 0x21b4f4b5L, 0x56b3c423L,
   0xcfba9599L, 0xb8bda50fL, 0x2802b89eL, 0x5f058808L, 0xc60cd9b2L, 0xb10be924L,
   0x2f6f7c87L, 0x58684c11L, 0xc1611dabL, 0xb6662d3dL, 0x76dc4190L, 0x01db7106L,
   0x98d220bcL, 0xefd5102aL, 0x71b18589L, 0x06b6b51fL, 0x9fbfe4a5L, 0xe8b8d433L,
   0x7807c9a2L, 0x0f00f934L, 0x9609a88eL, 0xe10e9818L, 0x7f6a0dbbL, 0x086d3d2dL,
   0x91646c97L, 0xe6635c01L, 0x6b6b51f4L, 0x1c6c6162L, 0x856530d8L, 0xf262004eL,
   0x6c0695edL, 0x1b01a57bL, 0x8208f4c1L, 0xf50fc457L, 0x65b0d9c6L, 0x12b7e950L,
   0x8bbeb8eaL, 0xfcb9887cL, 0x62dd1ddfL, 0x15da2d49L, 0x8cd37cf3L, 0xfbd44c65L,
   0x4db26158L, 0x3ab551ceL, 0xa3bc0074L, 0xd4bb30e2L, 0x4adfa541L, 0x3dd895d7L,
   0xa4d1c46dL, 0xd3d6f4fbL, 0x4369e96aL, 0x346ed9fcL, 0xad678846L, 0xda60b8d0L,
   0x44042d73L, 0x33031de5L, 0xaa0a4c5fL, 0xdd0d7cc9L, 0x5005713cL, 0x270241aaL,
   0xbe0b1010L, 0xc90c2086L, 0x5768b525L, 0x206f85b3L, 0xb966d409L, 0xce61e49fL,
   0x5edef90eL, 0x29d9c998L, 0xb0d09822L, 0xc7d7a8b4L, 0x59b33d17L, 0x2eb40d81L,
   0xb7bd5c3bL, 0xc0ba6cadL, 0xedb88320L, 0x9abfb3b6L, 0x03b6e20cL, 0x74b1d29aL,
   0xead54739L, 0x9dd277afL, 0x04db2615L, 0x73dc1683L, 0xe3630b12L, 0x94643b84L,
   0x0d6d6a3eL, 0x7a6a5aa8L, 0xe40ecf0bL, 0x9309ff9dL, 0x0a00ae27L, 0x7d079eb1L,
   0xf00f9344L, 0x8708a3d2L, 0x1e01f268L, 0x6906c2feL, 0xf762575dL, 0x806567cbL,
   0x196c3671L, 0x6e6b06e7L, 0xfed41b76L, 0x89d32be0L, 0x10da7a5aL, 0x67dd4accL,
   0xf9b9df6fL, 0x8ebeeff9L, 0x17b7be43L, 0x60b08ed5L, 0xd6d6a3e8L, 0xa1d1937eL,
   0x38d8c2c4L, 0x4fdff252L, 0xd1bb67f1L, 0xa6bc5767L, 0x3fb506ddL, 0x48b2364bL,
   0xd80d2bdaL, 0xaf0a1b4cL, 0x36034af6L, 0x41047a60L, 0xdf60efc3L, 0xa867df55L,
   0x316e8eefL, 0x4669be79L, 0xcb61b38cL, 0xbc66831aL, 0x256fd2a0L, 0x5268e236L,
   0xcc0c7795L, 0xbb0b4703L, 0x220216b9L, 0x5505262fL, 0xc5ba3bbeL, 0xb2bd0b28L,
   0x2bb45a92L, 0x5cb36a04L, 0xc2d7ffa7L, 0xb5d0cf31L, 0x2cd99e8bL, 0x5bdeae1dL,
   0x9b64c2b0L, 0xec63f226L, 0x756aa39cL, 0x026d930aL, 0x9c0906a9L, 0xeb0e363fL,
   0x72076785L, 0x05005713L, 0x95bf4a82L, 0xe2b87a14L, 0x7bb12baeL, 0x0cb61b38L,
   0x92d28e9bL, 0xe5d5be0dL, 0x7cdcefb7L, 0x0bdbdf21L, 0x86d3d2d4L, 0xf1d4e242L,
   0x68ddb3f8L, 0x1fda836eL, 0x81be16cdL, 0xf6b9265bL, 0x6fb077e1L, 0x18b74777L,
   0x88085ae6L, 0xff0f6a70L, 0x66063bcaL, 0x11010b5cL, 0x8f659effL, 0xf862ae69L,
   0x616bffd3L, 0x166ccf45L, 0xa00ae278L, 0xd70dd2eeL, 0x4e048354L, 0x3903b3c2L,
```

```
    0xa7672661L, 0xd06016f7L, 0x4969474dL, 0x3e6e77dbL, 0xaed16a4aL, 0xd9d65adcL,
    0x40df0b66L, 0x37d83bf0L, 0xa9bcae53L, 0xdebb9ec5L, 0x47b2cf7fL, 0x30b5ffe9L,
    0xbdbdf21cL, 0xcabac28aL, 0x53b39330L, 0x24b4a3a6L, 0xbad03605L, 0xcdd70693L,
    0x54de5729L, 0x23d967bfL, 0xb3667a2eL, 0xc4614ab8L, 0x5d681b02L, 0x2a6f2b94L,
    0xb40bbe37L, 0xc30c8ea1L, 0x5a05df1bL, 0x2d02ef8dL
};
*/
```

4.11.7 Versatile Binary Format Specification 2.6

4.11.7.1 Versatile Binary Format Generic reqs

•Software part format

4.11.7.1.1 Software part format

660570 v1 **Software part format**

Version history:
(1):

Purpose: To define the software part format for software update.
Source:
Verification Method: Test
Legacy ID:

The software part(s) shall be submitted in VBF format by supplier. The detail format is described in *[Versatile Binary Format Specification].*

•VBF Additional Requirements

4.11.7.1.2 VBF Additional Requirements

660568 v1 **Maximim allowed VBF File size**

Version history:
(1):

Purpose: To restrict the size of the VBF file to an optimal value that can be handled by EoL and aftermarket tools.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Maximum allowed size of an uncompressed VBF file for an ECU per software part is 4 gigabytes (GB).

It is necessary for all ECUs to support data compression method(s) as defined in [Data Compressoon and Encryption] and the size of compressed VBF file should be at least 30-40% smaller in average when comared to the uncompressed(non-compressed) version.

693576 v1 **Maximum allowed number of data block**

Version history:
(1):

Purpose: To restrict the number of data block of VBF file to an optimal value that can be handled by EoL, aftermarket tools and over-the-air.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Maximum allowed number of data block in VBF file for an ECU per software part is 20.

693571 v1 **Maximum allowed number of erase ranges in VBF header**

Version history:
(1):

Purpose: To restrict the number of erase ranges in VBF header to an optimal value that can be handled by EoL, aftermarket tools and over-the-air.
Source:
Verification Method: Analysis AND Test
Legacy ID:

Maximum allowed number of erase ranges in VBF header for an ECU per software part is 20.

4.11.7.2 Versatile Binary Format Specification

•Overview

4.11.7.2.1 Overview

4.11.7.2.1.1 Requisite documents

Reference number	Source	Title
[VBF_1]	Internal	Data Compression and Encryption Specification
[VBF_2]	Internal	UDS Services
[VBF_3]	Internal	General Software Authentication

Note:
Internal references are typically available as separate chapters in ECU specific Base Technology SWRS.

4.11.7.2.1.2 Definitions and Abbreviations

Definition	Description
Domain ID	Part of the ECU Source Address (SA), states which Domain the ECU is physical connected to.
Network ID	Part of the ECU Source Address (SA), states which Network the ECU is physical connected to.
ECU ID	Part of the ECU Source Address (SA).
Tester	A system that controls functions such as test, inspection, monitoring, or diagnosis of an in-vehicle electronic control unit and may be dedicated to a specific type of operators (e.g. a scan tool dedicated to garage mechanics or

	a test tool dedicated to assembly plant agents). The tester is also referenced as the client.

Abbreviation	Description
CM	CoMments
ECU	Electronic Control Unit
PBL	Primary BootLoader
SBL	Secondary BootLoader
SSBL	Special Secondary BootLoader
VBF	Versatile Binary Format
WS	White Space

•Versatile Binary Format (VBF)

4.11.7.2.2 Versatile Binary Format (VBF)

4.11.7.2.2.1 VBF version

This document specifies VBF version 2.6 of the Versatile Binary Format.

4.11.7.2.2.2 Introduction

The VBF file contains three parts: VBF version section, a header section and a data section. VBF version section specifies the version number of the VBF specification that the file is compliant with. VBF version uses ASCII representation. The header section contains general information relating to file usage, setup, etc. The header section uses ASCII representation. The data section contains the actual data that is to be transferred to the target ECU. The data section uses binary representation.

Table: An Overview of the Versatile Binary Format

Name	Description	Type	Data
Version Section	VBF Version Number	Text	vbf_version=2.6;
Header section	Header	Text	n bytes, ASCII
Data section	Block 1	Start address	4 bytes, binary
		Length	4 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary
	Block 2	Start address	4 bytes, binary
		Length	4 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary
	:	:	:
	Block n	Start address	4 bytes, binary
		Length	4 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary

4.11.7.2.2.3 File Naming

663041 v1 **File Naming**

Version history:
(1):

Purpose: To identify the release of the VBF file.
Source: N/A
Verification Method: Analysis AND Test
Legacy ID: 64686v2

All delivered or released VBF files shall be named the same as the corresponding released software part number which consist of (sw_part_number and sw_version) with an extension of ".VBF

Examples:
A VBF file with a header entry of
sw_part_number = "31808832";
sw_version = "AB";
shall utilize the file name of 31808832AB.VBF

A VBF file with a header entry of
sw_part_number = "31808832";
sw_version = "A";
shall utilize the file name of 31808832A.VBF

Delta files shall be released with a filename reference to its source and target version by including the software part number and software version of source and target in the delta filename.
Example:
sw_part_number = "31808832"; // Target version part no
sw_version = "AB"; // Target version
sw_current_part_number = "31808831"; // Source/current version part no
sw_current_version = "A";

The format of a delta filename shall be: <sw_current_part_number><sw_current_version><separator><sw_part_number><sw_version>.VBF

A VBF file containing delta and the header data above shall have the name:
31808831A _31808832AB.VBF

4.11.7.2.2.4 VBF file generation

663042 v1 **VBF file generation**

Version history:
(1):

Purpose: To generate the VBF file.
Source: N/A
Verification Method: Analysis
Legacy ID: 64690v0

The VBF Convert tool shall generate VBF files from Motorola S-record or Intel Hex formatted files.

•Notation and lexical elements

4.11.7.2.3 Notation and lexical elements

The VBF file header section uses some basic lexical elements, which are described, in this section. Also, the notation for describing the syntax is given here.

4.11.7.2.3.1 Notation

Words in bold indicate identifiers, which are special tokens.

Italicized words indicate a lexical item, and are described in text format.

The grouping symbols [ ], brackets, indicate that the contents are optional.

The grouping symbols { }, braces, group a set of items.

The grouping symbols ( ), parentheses, contain a set of items separated by |, then one of the items must appear.

4.11.7.2.3.2 General structure

663043 v1 **Header expression termination**

Version history:
(1):

Purpose:
Source: N/A
Verification Method: Analysis AND Test
Legacy ID: 64693v1

The header consists of sequences of expressions. An expression is made up of identifiers, reserved words, constants, integers and is always terminated with a semicolon (;).

Expressions containing multiple values are grouped into nested sections using matching brace characters “{“ and “}”. Data shall be contained in braces only if there is more than one set of data with the exception of description [see section *Header section - description*] and erase [see section *Header section - erase*] identifiers. In these cases, a single set of data shall also be

contained in braces. Refer to section *Notation and lexical elements - Identifiers, reserved words, and expressions* for general format requirements for expressions and to section *Overview of the VBF file* for detailed format requirements for each identifier.

4.11.7.2.3.3 Identifiers, reserved words and expressions

663044 v1 **Header Identifiers**

Version history:
(1):

Purpose: To define allowed header identifiers
Source: N/A
Verification Method: Analysis AND Test
Legacy ID: 64694v4

Identifier is the name of a variable and IdentifierValue is the data assigned to the variable. Allowed identifiers are **vbf_version**, **header**, **description**, **sw_part_number**, **sw_current_part_number**, **sw_version**, **sw_current_version**, **sw_part_type**, **data_format_identifier**, **ecu_address**, **erase**, **call**, **verification_block_start**, **verification_block_length**, **verification_block_root_hash**, **sw_signature_dev**, **sw_signature, parameter_settings** and **file_checksum**. Duplicates of identifiers are not allowed.

663045 v1 **IdentifierValue format**

Version history:
(1):

Purpose: To define the format of the IdentifierValue.
Source: N/A
Verification Method: Analysis AND Test
Legacy ID: 64695v0

Each element within an IdentifierValue shall either be an integer, real, string embedded within quotes, or a reserved IdentifierValue (reserved word). Each identifier has a specific format allowed for its identifier values as defined in section 5 . The lexical rules for identifiers and reserved words are identical to those for “C” programming language with upper- and lower-case always distinguished.

Unless otherwise specified, a valid expression shall always consist of the following format (where WS/CM is defined as any combination of white space characters and complete comments).

[WS/CM]identifier[WS/CM]=[WS/CM]IdentifierValue[WS/CM];[WS/CM]

When the IdentifierValue is enclosed within matching brace characters "{" and "}" as allowed by this specification, the format of the IdentifierValue (including braces) shall be as follows :

{[WS/CM]IdentifierValue[WS/CM][,[WS/CM]IdentifierValue[WS/CM]]} ;

where the optional field of [,[WS/CM]IdentifierValue[WS/CM]] may occur multiple times .

Example:
sw_part_type = CARCFG;

In this example, **sw_part_type** is an identifier, and CARCFG is the reserved word value of the identifier **sw_part_type** .

4.11.7.2.3.4 Integers

663046 v1 **Integers**

Version history:
(1):

Purpose: Clarification of the integer format definition used by the VBF file.
Source: N/A
Verification Method: Analysis AND Test
Legacy ID: 64696v1

An integer is a sequence of digits. It shall be represented in hexadecimal format, start with the prefix “0x” and the following characters must be chosen from the string “0123456789abcdefABCDEF”.

Examples:

Valid Format:
1. 0xFF //Hexadecimal
2 0x12345678 //Hexadecimal, 32 bits representation

Invalid Format:
1. 0b11111111 //Binary
2. 255 //Decimal
3. 0x12 0x34 0x56 0x78 //Each byte separated with prefix “0x”.

4.11.7.2.3.5 Comments

663047 v1 **Comments**

Version history:
(1):

Purpose: To define the usage and format of comments within the VBF file.
Source: N/A
Verification Method: Analysis AND Test
Legacy ID: 64698v0

Comments are allowed in the header section only and may appear within an expression as described in section *Identifiers, reserved words, and expressions.* Valid comments shall always be ignored by a VBF parser. Comments can be grouped together by “/*” and “*/” and stretch over several lines (C-style), or they can start with “//” and stretch to the end of the line (C++

style). Neither bracket syntax nests. It is recommended that C++-style comments be used wherever possible, since it is much more difficult to accidentally comment out more than was intended (as compared to using the C-style syntax).

4.11.7.2.3.6 White Space Characters

663048 v1 **White space characters**

Version history:
(1):

Purpose: To define allowed white space characters.
Source: N/A
Verification Method: Analysis AND Test
Legacy ID: 64699v0

Blank Space, Horizontal Tab (or Tab), Line Feed, Carriage Return, Form Feed, and Vertical Tab characters are defined as white space characters for the purpose of this specification. No other white space characters are allowed. A VBF parser shall always ignore valid white space characters unless otherwise specified in this document.

Table : ASCII Representation of White Space Characters

	Description	Symbol	Hex Value	Binary Value
1	Horizontal Tab (or Tab)	HT	09	00001001
2	Line Feed	LF	0A	00001010
3	Vertical Tab	VT	0B	00001011
4	Form Feed	FF	0C	00001100
5	Carriage Return	CR	0D	00001101
6	Blank Space	SPACE	20	00100000

4.11.7.2.3.7 Non-printing Characters

663049 v1 **Non-printing characters**

Version history:
(1):

Purpose: To define the range of non-printing characters allowed.
Source: N/A
Verification Method: Analysis AND Test
Legacy ID: 64701v0

0x00 to 0x1F represents non-printing characters. All Non-printing characters, except white space characters, are not allowed in VBF file header. For white space non-printing (0x09 to 0x0D) character handling, refer to section *White Space Characters*

•Overview of the VBF file

4.11.7.2.4 Overview of the VBF file

4.11.7.2.4.1 Version Section

663050 v1 **Version section**

Version history:
(1):

Purpose: To identify the version of the VBF file.
Source: N/A
Verification Method: Analysis AND Test
Legacy ID: 64702v1

The first line of the VBF file shall indicate the version of the VBF file. No comments or non-white space characters are allowed before or after the expression. No white space characters are allowed before the expression. Note that this means the first byte of the VBF file shall always be 0x76 (i.e., ASCII 'v'). This specification is for version 2.6 of the VBF file, and so the first line shall always read:

vbf_version = 2.6;

Examples:

Valid Format:
1. vbf_version=2.6;
2. vbf_version = 2.6 ;

Invalid Format:
1. vbf_version=2.6;
2. /*Comment*/vbf_version/* Comment*/=/*Comment*/2.6/*Comment*/;
3. text vbf_version text= text 2.6 text; text

vbf_version is immediately followed by the header identifier with the exception of white space as described above. The header section as explained below contains all information needed to identify the VBF file and also information for the tester to perform a software download operation.

4.11.7.2.4.2 Header Section

663051 v2 **Header section**

Version history:
(2): remove unfound content "section 4.3"

Purpose: To define the start of the header section.
Source: N/A
Verification Method: Analysis AND Test
Legacy ID: 64703v1

All the header information is contained in braces { }. The information between the braces in the header section shall consist entirely of expressions as defined in follow section. No comments or non-white space characters are allowed before **header** identifier or in between the **header** identifier and beginning braces ({).
Examples:

Valid Format:
1. vbf_version = 2.6; header {
2. vbf_version = 2.6;
header{
3. vbf_version=2.6;
header
{
Invalid Format:
1. vbf_version=2.6;
/*Comment*/
header{
2. header /*Comment*/ {

663052 v1 **Header section - Format**

Version history:
(1):

Purpose: To define the format of the header section
Source: N/A
Verification Method: Analysis AND Test
Legacy ID: 64705v5

The format of the header section is defined as follows (the grouping symbols [ ] indicates optional identifiers, i.e. the brackets are not to be part of the identifier defintion):

header {

[**description** = { *"text string row 1" ,*
"text string row 2" ,
"text string row 3" ,
:
"text string row n"
} ;]

sw_part_number = "part number";

sw_version = “version suffix”;

sw_part_type = *type*;

[**sw_current_part_number =** "installed part number";]
[**sw_current_version =** "installed version suffix";]

[**data_format_identifier** = *data compression and encryption method*;]

ecu_address = *ECU target address;*

[**erase** = { { *start address 1, length 1* },
{ *start address 2, length 2* },
:
{ *start address n, length n* }
} ;]

[**call** = *start address* **;**]

[**verification_block_root_hash** = *hash value of unprocessed data to be signed;*]
[**verification_block_start** = *start of the unprocessed verification block table;*]
[**verification_block_length** = *unprocessed length of the verification block table;*]
[**sw_signature_dev =** *software signature (test and development);*]
[**sw_signature** = *software signature;*]

[**parameter_settings** = { {"*parameter name 1", "value"*}, {"*parameter name n", "value"*} };]

file_checksum = *checksum*;
}

4.11.7.2.4.2.1 description

663053 v1 **description**

Version history:
(1):

Purpose: To define the format and use of the description identifier.
Source: N/A
Verification Method: Test
Legacy ID: 64706v0

The **description** identifier is optional and can be used for a short description of the file. The **description** IdentifierValue may contain a maximum of 16 rows, with each row containing a maximum of 80 characters (bytes). Each row is contained in quotes (") and is terminated by a comma (,) or by right brace (}) followed by semicolon (;) if the row is a last row. White space within quotes is not ignored. All of the description rows shall be contained within braces whether there is one row or multiple rows. No quotes (") are allowed within a row.

Examples:

Valid Format:
1. description = {"Created: 2002-03-14"};

2. description = { "SOP software for S60 AWD" ,
"Created: 2002-03-14"
};

Invalid Format:
1. description = " Created: 2002-03-14" ; // Missing braces

2. description = "SOP software for S60 AWD" , // Missing braces
"Created: 2002-03-14";

3. description = { SOP software for S60 AWD, // Missing quotes
Created: 2002-03-14
};
4. description = { "SOP software for "S60" AWD"}; // Extra quotes

4.11.7.2.4.2.2 sw_part_number

663054 v1 **sw_part_number**

Version history:
(1):

Purpose: To define the format and use of the sw_part_number identifier.
Source: N/A
Verification Method: Test
Legacy ID: 64707v2

The **sw_part_number** IdentifierValue contains the vehicle manufacturer part number for identification of the software component (VBF file). The **sw_part_number** IdentifierValue shall be contained in quotes and shall consist of a maximum of 20 characters. No white space characters or comments are allowed within these quotes .

Examples:

Valid Format:
1. sw_part_number = "31808832"; // Part number format

Invalid Format:
1. sw_part_number = 31808832; // Missing quotes

2. sw_part_number = "3180883233222567897 13"; // Maximum characters > 20

3. sw_part_number = " 31808832 "; // White Space

within quotes

4. sw_part_number = "/*Comment*/31808832 /*Comment*/"; // Comments within quotes

4.11.7.2.4.2.3 sw_current_part_number

663055 v1 **sw_current_part_number**

Version history:
(1):

Purpose: To define the format and use of the sw_current_part_number identifier
Source:
Verification Method: Inspection
Legacy ID: 408537v2

The **sw_current_part_number** IdentifierValue contains the vehicle manufacturer part number for identification of the software component (VBF file) that is expected to be installed before a **delta** software update occurs.
This field is mandatory only for software updates with compression type **delta.**
The **sw_current_part_number** IdentifierValue shall be contained in quotes and shall consist of a maximum of 20 characters. No white space characters or comments are allowed within these quotes.

Examples:

Valid Format:
1. sw_current_part_number = "31808832"; // Part number format

Invalid Format:
1. sw_current_part_number = 31808832; // Missing quotes

2. sw_current_part_number = "318088323322256789713"; // Maximum characters > 20

3. sw_current_part_number = " 31808832 "; // White Space within quotes

4. sw_current_part_number = "/*Comment*/31808832 /*Comment*/"; // Comments within quotes

4.11.7.2.4.2.4 sw_version

663056 v1 **sw_version**

Version history:
(1):

Purpose: To define the format and use of the sw_version identifier.
Source: N/A
Verification Method: Test
Legacy ID: 64708v0

The **sw_version** IdentifierValue contains the vehicle manufacturer version for identification of the software component (VBF file). The **sw_version** IdentifierValue shall be contained in quotes and shall consist of a maximum of 4 characters (bytes). No white space characters or comments are allowed within these quotes. **sw_version** IdentifierValue is case sensitive and shall only contain the uppercase characters A-Z.

Examples:

Valid Format:
1. sw_version = "ABC"; // Version suffix format

2. sw_version = “AA"; // Version suffix format

3. sw_version = “A”; // Version suffix format

Invalid Format:
1. sw_version = ABC; // Missing quotes

2. sw_part_number = "ABCDE"; // Maximum characters > 4

3. sw_part_number = " ABC "; // White Space within quotes

4. sw_part_number = "/*Comment*/ABC /*Comment*/"; // Comments within quotes

5. sw_version = aBC; // Lowercase character

6. sw_version = 1BC; // Numerical character

4.11.7.2.4.2.5 sw_current_version

663057 v1 **sw_current_version**

Version history:
(1):

Purpose: To define the format and use of the sw_current_version identifier

Source:
Verification Method: Inspection
Legacy ID: 408538v1

The **sw_current_version** IdentifierValue contains the vehicle manufacturer version for identification of the software component (VBF file) that is expected to be installed before a **delta** software update occurs.
This field is mandatory only for software updates with compression type **delta**.
The **sw_current_version** IdentifierValue shall be contained in quotes and shall consist of a maximum of 4 characters (bytes). No white space characters or comments are allowed within these quotes.
The **sw_current_version** IdentifierValue is case sensitive and shall only contain the uppercase characters A-Z.

Examples:

Valid Format:
1. sw_current_version = "ABC"; // Version suffix format

2. sw_current_version = “AA"; // Version suffix format

3. sw_current_version = “A”; // Version suffix format

Invalid Format:
1. sw_current_version = ABC; // Missing quotes

2. sw_current_version = "ABCDE"; // Maximum characters > 4

3. sw_current_version = " ABC "; // White Space within quotes

4. sw_current_version = "/*Comment*/ABC /*Comment*/"; // Comments within quotes

5. sw_current_version = aBC; // Lowercase character

6. sw_current_version = 1BC; // Numerical character

4.11.7.2.4.2.6 sw_part_type

663058 v1 **sw_part_type**

Version history:
(1):

Purpose: To define the format and use of the sw_part_type identifier.
Source: N/A
Verification Method: Test

Legacy ID: 64709v5

The **sw_part_type** identifier indicates different types of a software parts in an ECU. Only the reserved word IdentifierValues defined in the table below shall be considered valid for the **sw_part_type** identifier.

Table: Software Part Types and Allowed Reserved Words

sw_part_type IdentifierValue Reserved Words	Description
CARCFG	Car configuration
CUSTOM	Customer parameters
DATA	Data or parameters (i.e., calibrations)
EXE	Executable
SBL	Secondary Bootloader
SIGCFG	Signal configuration
PBL	Primary Bootloader
SSBL	Special SBL which has the possibility to replace the PBL
TEST	Test program, (i.e. production test, diagnostics)
FBL	Flashable Bootloader
CERT	Certificate
PWD	Password for certificate
APP	Application file
MAP	Map file used for navigation
ESS	ECU Software Structure
SIGNED	Digital Signature file
VARIANT_INFO	Settings file (for offboard tool settings)

The identifier **sw_part_type** in a single VBF file can only be assigned one of the reserved word values in the table above.

A VBF file with **sw_part_type = SBL or SSBL** is not allowed to use the identifier **erase**. These software types are downloaded to RAM and no erase operation shall be performed in this case.

Examples:

Valid Format:

```
1.      sw_part_type = SIGCFG ;

2.      sw_part_type = EXE ;                          // or any other valid type but "SBL" or
"SSBL"
     erase =  {   { 0x00004200, 0x00003CBF },
                  { 0x00008010, 0x00002FF3 }
              };
```

Invalid Format:

```
1.      sw_part_type = exe;                           //  Invalid case

2.      sw_part_type = SBL;                           // Invalid combination of sw_part_type and
```

```
erase
    erase =  {   { 0x00004200, 0x00003CBF },
                 { 0x00008010, 0x00002FF3 }
             } ;

3.      sw_part_type = MYOWNTYPE;                    // Unknown IdentifierValue
```

4.11.7.2.4.2.7 data_format_identifier

663059 v1 **data_format_identifier**

Version history:
(1):

Purpose: To define the format and usage of the data_format_identifier.
Source: N/A
Verification Method: Test
Legacy ID: 64712v3

The data_format_identifier is optional and when is presented with a non-zero value in the header section, it represents that a data compression/encryption method is used. Valid values for the data_format_identifier are one byte long with a range of 0x00 to 0xFF with upper nibble representing "Data Compression Method" and lower nibble representing "Data Encryption Method". Refer to REQPROD 459337: RequestDownload - dataFormatIdentifier for valid values. If the identifier is not present in the header section, it represents that neither compression nor encryption is used.

Example of Valid Format:

1. data_format_identifier = 0x00; // represents no compression and no encryption

2. data_format_identifier = 0x10; // represents compression method #1 and no encryption

3. data_format_identifier = 0x20; // represents compression method #2 and no encryption

Example of invalid Format:

1. data_format_identifier = 0x123; // Invalid format (must be from 0x00 – 0xFF)

2. data_format_identifier = {0x10}; // Extra braces

3. data_format_identifier = 23; // Invalid format (integers shall be hexadecimal)

4.11.7.2.4.2.8 ecu_address

663060 v1 **ecu_address**

Version history:
(1):

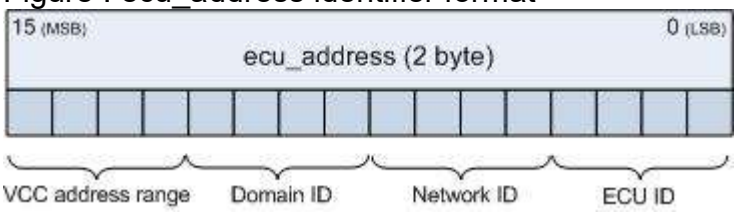
Purpose: To define the format and use of the ecu_address identifier.
Source: N/A
Verification Method: Test
Legacy ID: 64714v0

The **ecu_address** identifier indicates the physical target address of the ECU. The **ecu_address** is divided into three parts where the combination of Domain ID, Network ID and ECU ID is utilized. The **ecu_address** depends on where in the network topology the ECU is placed.

The ecu_address length is two bytes long .

The addressing format and valid address ranges for a physical target address (TA) is defined in *[VBF_2] UDS Services*. See the figure below for an overview.

Figure : ecu_address identifier format



Examples:

Valid Format:
1. ecu_address = 0x1321; // Domain ID 0x3, Network ID 0x2, ECU ID 0x1

Invalid Format:
1. ecu_address = {0x0321}; // Extra braces

4.11.7.2.4.2.9 erase

663061 v1 **erase**

Version history:
(1):

Purpose: To define the format and use of the erase identifier
Source: N/A
Verification Method: Test
Legacy ID: 64715v2

The **erase** identifier indicates the range(s) of memory that needs to be erased before a software download operation can take place. The erase start address and length need not to be matched to start address and length of a flash sector. The erase length can span over more than one flash sector, or can be within a single flash sector. The erase start address and length can automatically be derived from a Motorola S-record file or an Intel Hex file during generation of a VBF file, but there is no guarantee that this will align with acceptable flash erase segment boundaries for a given SBL. The **erase** identifier is mandatory for memory types (i.e., flash) that need to be erased before programming. The erase start address and length are four bytes long with a range of 0x00000000 to 0xFFFFFFFF. The **erase** identifier values shall always be contained in two sets of braces whether there is one or more ranges present.

Note: If **sw_part_type = SBL or SSBL**, the VBF file shall not contain the **erase** identifier
Note: If delta encoding is used **(data_format_identifier = 0x80 or 0x90)** , the VBF file shall not contain the **erase** identifier

Examples:

Valid Format:

```
1.  erase =   {  { /*start address*/ 0x00004200, /*Length*/ 0x00003CBF }
          };

2.  erase =   {  { 0x00004200, 0x00003CBF },
                 { 0x00008010, 0x00002FF3 }
              };                          // Two blocks: 0x00004200-0x00007EBE, 0x00008010-
0x0000B002
```

Invalid Format:

```
1.  erase =   { 0x00004200, 0x00003CBF };                    // Missing outer braces

2.  erase =   { { 0xFFFFFFFE, 0x00000002 }                   // Address Overflow
            };

3.  erase =   { 0x00004200, 0x00003CBF },                    // Missing outer braces
            { 0x00008010, 0x00002FF3 };

4.  erase =   { 0x00004200, 0x00003CBF,                      // Missing inner braces
              0x00008010, 0x00002FF3
            };
```

4.11.7.2.4.2.10 call

663062 v1 **call**

Version history:
(1):

Purpose: To define the format and use of the call identifier.
Source: N/A

Verification Method: Test
Legacy ID: 64716v0

The **call** entry is mandatory in VBF files with **sw_part_type** identifier values equal to SBL or SSBL and optional in VBF files with **sw_part_type** equal to TEST. For all other identifier values of **sw_part_type** , the **call** entry is not allowed. The **call** entry indicates the start address to a pre-defined executable function in ECU memory. The call identifier start address is four bytes long with a range of 0x00000000 to 0xFFFFFFFF.

Examples:

Valid Format:

```
1.    sw_part_type = SBL;
        call = 0x00000100;                    // Start address for SBL

2.    sw_part_type = SSBL;
        call = 0x00000100;                    // Start address for SSBL

3.    sw_part_type = TEST;
        call = 0x00000100;                    // Start address for TEST
```

Invalid Format:

```
1.    sw_part_type = SBL;
    With no call identifier

2.    sw_part_type = SSBL;
    With no call identifier

3.    sw_part_type = EXE:                     // or any other IdentifierValue but SBL,
SSBL or TEST
    call = 0x00000100;
```

4.11.7.2.4.2.11 verification_block_start

663063 v1 **verification_block_start**

Version history:
(1):

Purpose: To define the format and use of the verification_block_start identifier. The purpose of the identifier is to identify the start address of the data (unprocessed) that is signed.
Source:
Verification Method: Test
Legacy ID: 417752v0

The **verification_block_start** represents the start address of the unprocessed data that is to be signed, according to the signing method *[General Software Authentication]*. The **verification_block_start** address is four bytes long integer with a range of 0x00000000 to

0xFFFFFFFF.

Example:

Valid Format:
1. verification_block_start = 0x0002FF00;

Note: There are use cases where a data file is not signed, e.g. sw_part_type SIGNED or prior to a file has been released, approved and signed.

4.11.7.2.4.2.12 verification_block_length

663064 v1 **verification_block_length**

Version history:
(1):

Purpose: To define the format and use of the verification_block_length identifier. The purpose of the identifier is to identify the length of the unprocessed data that is signed.
Source:
Verification Method: Test
Legacy ID: 417753v0

The **verification_block_length** represents the unprocessed length (prior to compression/encryption is applied) of the data that is to be signed, according to the signing method *[General Software Authentication]*. The **verification_block_length** is four bytes long integer with a range of 0x00000000 to 0xFFFFFFFF.

Example:

Valid Format:
1. verification_block_length = 0x00000054;

Note: There are use cases where a data file is not signed, e.g. sw_part_type SIGNED or prior to a file has been released, approved and signed.

4.11.7.2.4.2.13 verification_block_root_hash

663065 v1 **verification_block_root_hash**

Version history:
(1):

Purpose: To define the format and use of the verification_block_root_hash identifier. The purpose of the hash value is to provide the Trust Centre (signing entity) with required information of the unprocessed data (no compression/encryption method applied).
Source:

Verification Method: Test
Legacy ID: 417751v1

The verification_block_root_hash represents the hash value of the unprocessed data that is to be signed, according to the signing method [General Software Authentication]. The verification_block_root_hash shall be represented as an integer and the length is dependent of the hashing method used.

Example:

Valid Format:
1. verification_block_root_hash = 0xF1F62AC8EDFECD610234DBDA84F7561998ADC47A0F0EDE6CA23F0FC22974F9F4;

Note: There are use cases when a data file is not signed, e.g. prior to a file has been released, approved and signed.

4.11.7.2.4.2.14 sw_signature

663066 v1 **sw_signature**

Version history:
(1):

Purpose: To define the format and use of the sw_signature identifier. The purpose of the digital signature is to validate the authenticity of the programmed data in the ECU.
Source:
Verification Method: Test
Legacy ID: 417716v2

The sw_signature represents the digital signature according to the applied signature method [General Software Authentication]. The sw_signature shall be represented as an integer and the length is dependent of the signing method used.

Example:

Valid Format:
1. sw_signature = 0x514E3E55831E43E89D4118E2F29F4996C3A8ECBD11B1CDB8F4940AE4BF8BF866C28EA6F1B4B1458A7434788F0460434368ACE9863FDFF7F61E90AF2A95421A6757E553115EC00E37274066678AEFD54DD963637203FAEA496F44C79EC11E706C3911749374AED7BCB7C1768E2EB09400D6146D8771BC6A5C0B10D420E785BCAD91ED553537A43CFE9EC0A48B66179416D9615A1108588005568F46331E2F5BD13F10005A7F0CE2B1AA820CF34C7C297F6CFDAC7A436D6C0FA2B0551C032B63FAB04DD5A2E8F54B78BAD9CE609C0B022E93F716D857616D2FAAD2BBEEBDFB162A6330123FCB23FB52D56ECBD449DB89594B99768AD3B6CBDB222DB47359A9D48F;

Note: There are use cases where a data file is not signed, e.g. prior to a file has been released,

approved and signed.
Note: In production, e.g. at manufacturing and workshop systems, this information is to be transferred in a separate (non-vbf) file that is defined elsewhere.

4.11.7.2.4.2.15 sw_signature_dev

663067 v1 **sw_signature_dev**

Version history:
(1):

Purpose: To define the format and use of the sw_signature_dev identifier. The purpose of the digital signature is to validate the authenticity of the programmed data in the ECU for test & development objects.
Source:
Verification Method: Test
Legacy ID: 435002v0

The sw_signature_dev represents the digital signature for test and development according to the applied signature method *[General Software Authentication]*. The sw_signature_dev shall be represented as an integer and the length is dependent of the signing method used.

Example:

The software release candidate is signed with a dedicated key for development and the resulting digital signature is stored at the sw_signature_dev. A software successfully verified and ready for release (production) is signed with the dedicated key for production (append sw_signature), but the sw_signature_dev is still kept. Refer to sw_signature identifier for format example.

4.11.7.2.4.2.16 file_checksum

663068 v1 **file_checksum**

Version history:
(1):

Purpose: To define the format and use of the file_checksum identifier.
Source: N/A
Verification Method: Test
Legacy ID: 64718v0

A four-byte file checksum shall be provided in the header section with a range of 0x00000000 to 0xFFFFFFFF. The purpose of this checksum is to validate the integrity of the file itself prior to attempting a download. The checksum shall be a CRC32 of all the data contained in the data section of the file, including start address, length and checksum for all data blocks. This means that if the data bytes to download are compressed (i.e., data_format_identifier contains a non-zero value), then the file checksum is performed using the compressed data. The tester shall check this checksum against the file checksum it has calculated before the software download operation is performed with the VBF file.

The file checksum shall meet the following criteria:
· 32 bit CRC, polynomial = 0x04C11DB7
(x^32 + x^26 + x^23 + x^22 + x^16 + x^12 + x^11 + x^10 + x^8 + x^7 + x^5 + x^4 + x^2 + x^1 + 1)
· Initial value = 0xFFFFFFFF
· Bit reversed

For a fast CRC32 calculation a look-up table implementation is the preferred solution.
See Appendix B for "C-code" example.

Example:
file_checksum = 0x29058c73

Note: This is the same CRC32 algorithm as used by "WinZip" (among many others).

663069 v1 **file_checksum - no data blocks**

Version history:
(1):

Purpose: Define the file_checksum value for a data file with no data blocks. This is a special case when the data file is used as an information carrier to offboard tool-chain, i.e. no data downloaded to the ECU.
Source:
Verification Method: Test
Legacy ID: 465519v0

The file_checksum for a data file having no data blocks shall be 0xFFFFFFFF.

4.11.7.2.4.2.17 parameter_settings

663070 v1 **parameter_settings**

Version history:
(1):

Purpose: The file_checksum for a data file having no data blocks shall be 0xFFFFFFFF.

Source:
Verification Method: Test
Legacy ID: 445413v2

The optional **parameter_settings** identifier is used to define and carry configurations required by off-board tools. To be able to define multiple parameters using this identifier, a parameter is defined by a "name" and "value" pair.
Each "name" and "value" pair must be contained in two sets of braces and every pair is to be separated by a comma. A single element of each pair, i.e. the "name" and "value" respectively, must be quoted and separated by a comma. The overall parameter_settings definitions must

be embraced by two outer braces whether there is one or more "name" and "value" pairs defined.

Example 1:
Valid format:

1. parameter_settings = {{"swAuthKeyAlgorithm", "RSA4096"}};
2. parameter_settings = {{"nameOne", "VALUE1"}, {"nameTwo", "VALUE2"}, {"nameThree", "VALUE3"}};

Invalid format:
1. parameter_settings = {"swAuthKeyAlgorithm", "RSA4096"}; //Missing outer braces
2. parameter_settings = {{"swAuthKeyAlgorithm"}}; //Incomplete pair

Example 2:

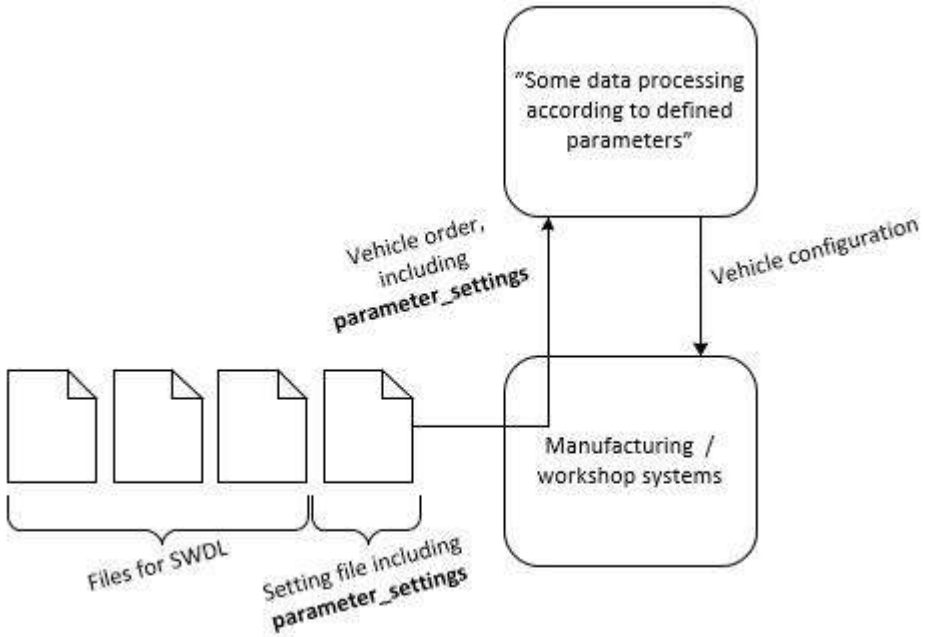


Figure - Usage of parameter_settings

The *Figure - Usage of parameter_settings* shows where off-board tools detects that some configuration is required and reacts according to the parameter_settings.
Notes: *This requirement does neither specify the applicable "name" and "value" pairs nor when the identifier is required. These are to be specified by the concept(s) requiring the parameter_settings identifier. For example, "nameOne", "VALUE1" shall only be considered as examples of the format and not valid element names. See e.g. "REQPROD 456934: Software Settings - Definition"*

4.11.7.2.4.2.18 Header section example #1

The first example is an VBF-file with **sw_part_type** = EXE.

Figure: Header section example #1:

```
vbf_version = 2.6;
header {

   description = { "Application SW DIM"};

  // Software part number
   sw_part_number = "11223344";

   // Software version
   sw_version = "AB";

   // Software type
   sw_part_type = EXE;

   // Compression and Encryption Method
   data_format_identifier = 0x10;

   // ECU address
   ecu_address = 0x1321;

   // Erase information
   // start length
   erase = {0x000200000, 0x00040000};

   //Verification block root hash value
   verification_block_root_hash =
0xFC10397BEABFBFF17519DCC6144EB4D86429CCC5682D8ED35AF8FBA984F26ED2;

   //Start address of the verification block
   verification_block_start = 0x0005FF00;

   //Length of the verification block
   verification_block_length = 0x00000054;

   //Software signature for test and development purposes
   sw_signature_dev =
0x16956823884E89FBFB1AFB81055C43732E30B066DB24AD7B4D7BF8ABB4EA3C98F283
9C3443FF267A3937E49ED4E50E028C94FAD8B214930CEE37C18111D8548F6B3D706FE3
2E1CA5B4C245276A9396762233B17C9A4DE258EA7EA1DF1D8E1006E5B1FE1061441864
A9718A2F696A89615ABDC3013F1F46682A03FAAB92ADFD43858F6D159020B09632FF83
AD1F29525A43455CF21B8D6800CC88D439D87651295552B3D4414DF55D764172595B59
AF2F354D27F75661565798FABFEF6638E5BF92ABECEAC7F119E893C6BD238A464C1CC9
```

```
65E084690650D048AE51F43295857DF5B450F60C5C19D2699C385090651B94282C3FE5
1703F249DAB62644EEDC07EF;

   //Software signature
   sw_signature =
0x514E3EB5831E43E89D5118E2F29F4996C3A8ECBD11B4CDB8F4940AE4BF8BC866C28E
A6A1B4B1458A7434788F0460434368ACE9863FDFF7F61E90AF2A95421A6757E553115E
C00E37274066678AEFD54DD963637203FAEAA96F44C79EC11E706C3921F49374AED7BC
B7C1768E2EB09400D6146D8771BC6A5C0B10D420E785BCAD91ED553537A43CFE9EC0A4
8B66179416D9615A1108588005568F46331E2F5BD13F10005A7F0CE2B1AA820CF3467C
297F6CFDAC7A136D6C0FA2B0551C038B63FAB04DD5A2E8F54B78BAD9CE609C0B022E93
F716D857616D2FAAD2BBEEBDFB162A6330123FCB23FB52D56ECBD449DB89594B99768A
D3B6CBDB222DB47359A9D48F;

   // File checksum
   file_checksum = 0x1A47EDA3;
}
<binary data section follows>
```

4.11.7.2.4.2.19 Header section example #2

The second example is a VBF-file with **sw_part_type** = SBL.

Figure: Header section example #2:

```
vbf_version = 2.6;
header {

   //This is a comment
   /*This is also a comment */

   //Short description of the VBF file
      description= {"SBL for DIM",
      "Created 2015-10-10"
   };
   //Software part number
   sw_part_number= "31888832";

   //Software version
   sw_version= "AB";

   //Software type
   sw_part_type= SBL;

   //Compression and Encryption Method
   data_format_identifier= 0x10;

   //ECU address
   ecu_address= 0x1123;

   //Start address for the SBL
   call= 0x00000100;
```

```
   //Verification block root hash value
   verification_block_root_hash =
   0xAA10397BEABFBFF17519DCC6144EB4D86429CCC5682D8ED35AF8FBA984F26ED2;


   //Start address of the verification block
   verification_block_start = 0x0001FF00;

   //Length of the verification block
   verification_block_length = 0x0000002C;

   //Software signature for test and development purposes
   sw_signature_dev =
   0x7EE874D0FFEBC4CA63AA3377AB3FACBC839690F903B96F30F4968D18101BDAD72
EDC343E6A0764469D47AAEFD92F416E7C44502CCD31274450DD5952E8F1C2B75BFFBC1
57C112AD1E7B4F46C99B89EED1BADB38D06C6B21036A55E0FF9B2EFFEC4DBB5856C7CD
946F432AC2E3CD41CE049D05C728F8C93EEF0AAA6703245ACD136BC3B33691781EFE67
430AEBCA99B0FBCF50CD1D30A9CC7DC4D81B35208788375727E0DD0106B46BBB80DB38
87DFD2D69DF0B4ADCD7BD69AF24C72F55E357457CB1D389A4976344417EF8A730BBB5D
4D8C22BA8D1258CEBEE9B218AF1D5D54328C47BF4D0FA8567F1AB123512BEE45828DE1
D4E3440DF182D1637737BA8C77D;

   //Software signature
   sw_signature =
   0xAC4E3EB5831E43E89D5118E2F29F4996C3A8ECBD11B4CDB8F4940AE4BF8BC866C
28EA6A1B4B1458A7434788F0460434368ACE9863FDFF7F61E90AF2A95421A6757E5531
15EC00E37274066678AEFD54DD963637203FAEAA96F44C79EC11E706C3921F49374AED
7BCB7C1768E2EB09400D6146D8771BC6A5C0B10D420E785BCAD91ED553537A43CFE9EC
0A48B66179416D9615A1108588005568F46331E2F5BD13F10005A7F0CE2B1AA820CF34
67C297F6CFDAC7A136D6C0FA2B0551C038B63FAB04DD5A2E8F54B78BAD9CE609C0B022
E93F716D857616D2FAAD2BBEEBDFB162A6330123FCB23FB52D56ECBD449DB89594B997
68AD3B6CBDB222DB47359A9D48F;

   // File checksum
   file_checksum = 0xA1DC5A45;
}
<binary data section follows>
```

4.11.7.2.4.2.20 Header section example #3

A VBF-file containing delta encoded data, with **data_format_identifier = 0x80**. Note that the identifiers for Software Authentication, e.g. the sw_signature, are for the full target image (after delta enconding has been applied)

Figure: Header section example #3:

```
vbf_version = 2.6;
header {

   //Description
   description = {"SWLM (Delta) for SUM from Version 31429076AD to
31429077AA"
```

```
                    };
   //Software part number (New part number after this update)
   sw_part_number = "31429077";

   //Software version (New software version after this update)
   sw_version = "AA";

   //Current Software part number
      //(Expected part number, installed before this update)
   sw_current_part_number = "31429076";

   //Current Software version
      //(Expected software version, installed before this update)
   sw_current_version = "AD";

   //Software part type
   sw_part_type = DATA;

   //Data Format Identifier
   //Delta encoded data and no compression method
   data_format_identifier = 0x80;

   //ECU address
   ecu_address= 0x1123;

   //Verification block root hash value
   verification_block_root_hash =
0xAA10397BEABFBFF17519DCC6144EB4D86429CCC5682D8ED35AF8FBA984F26ED2;

   //Start address of the verification block
   verification_block_start = 0x0001FF00;

   //Length of the verification block
   verification_block_length = 0x00000054;

   //Software signature for test and development purposes
   sw_signature_dev =
0x073A43404527ABFC1C70F637FBD689A83B0C14172C6C588268F80F7021CD82DF9489
C676452B7B1BB85D538B670F4E14FA71933AD1612E75F66574955B82755B0FF1A29D4D
5A9421EED6D9E705098CCD5F4E2A71C526D8DD90AF02212EB715937A3D49311B385E1C
3B754C90F0EFE20BB22240582A8FD03BF6FF04B59C114AA7B07D8BDCCBCEC77A8E00CC
C9D1CE18EC446A8DA45756CF3CFD594134859E407C7421D50221B1D71A1976AA68E36D
5642FA762B1DAFBB9804FFD8C479CA037307D8AC522AC4CB946248064E5A789797B1B5
5760EAC40FBC7E616F848C8A4B389899FB8DEF7DDCA4933E2F11C1638D05F66144AEC3
0E8DBAF2317827DB27438560;

   //Software signature
   sw_signature =
0xAC4E3EB5831E43E89D5118E2F29F4996C3A8ECBD11B4CDB8F4940AE4BF8BC866C28E
A6A1B4B1458A7434788F0460434368ACE9863FDFF7F61E90AF2A95421A6757E553115E
C00E37274066678AEFD54DD963637203FAEAA96F44C79EC11E706C3921F49374AED7BC
B7C1768E2EB09400D6146D8771BC6A5C0B10D420E785BCAD91ED553537A43CFE9EC0A4
```

```
8B66179416D9615A1108588005568F46331E2F5BD13F10005A7F0CE2B1AA820CF3467C
297F6CFDAC7A136D6C0FA2B0551C038B63FAB04DD5A2E8F54B78BAD9CE609C0B022E93
F716D857616D2FAAD2BBEEBDFB162A6330123FCB23FB52D56ECBD449DB89594B99768A
D3B6CBDB222DB47359A9D48F;

   // File checksum
   file_checksum = 0xA1DC5A45;
}
<binary data section follows>
```

4.11.7.2.4.2.21 Header section example #4

The fourth example is an VBF-file with sw_part_type = SIGNED, i.e. this data file itself is not signed but used to transfer a digital signature (legacy Software Authentication concept).

Figure: Header section example #4:

```
vbf_version = 2.6;
header {

   description = { "Digital Signature"};

   // Software part number
   sw_part_number = "22223344";

   // Software version
   sw_version = "AB";

   // Software type
   sw_part_type = SIGNED;

   // Compression and Encryption Method
   data_format_identifier = 0x10;

   // ECU address
   ecu_address = 0x1321;

   // Erase information
   // start length
   erase = {0x000100000, 0x00040000};

   // File checksum
   file_checksum = 0x2A47EDA3;
}
<binary data section follows>
```

Note that there are also other use cases where a data file is not signed, for example prior to it has been released, approved and signed by the Trust Centre.

4.11.7.2.4.3 Data Section

663071 v1	Data section - start

Version history:

(1):

Purpose: To define the start of the data section.
Source: N/A
Verification Method: Analysis AND Test
Legacy ID: 64725v1

After the header section ends, the data section immediately follows. The header section ends with a } character (ASCII 0x7D). The binary data section begins with the next byte immediately following this ASCII } character. The data section might be omitted, i.e. a data file could contain zero data blocks for some special cases.

663073 v1 Data section - structure

Version history:
(1):

Purpose: To define the structure of the data section.
Source: N/A
Verification Method: Analysis AND Test
Legacy ID: 64727v1

The data section may contain more than one block of data, and every block includes four parts.

1. 4-byte start address, physical address in ECU memory, with a range of 0x00000000 to 0xFFFFFFFF.
2. 4-byte length, number of data bytes in block (excluding Start address, Length and Checksum), with a range of 0x00000001 to 0xFFFFFFFF. If data compression is used, this length represents the compressed length.
3. Data
4. 2-byte checksum for the data block (excluding Start address and Length) with a range of 0x0000 to 0xFFFF.

The Start address, Length, and Checksum fields shall always be stored with the most significant byte (msb) first.

Table: Data Section

Version Section	VBF Version Number	Text	vbf_version=2.6;
Header section	Header	Text	n bytes, ASCII
Data section	Block 1	Start address	4 bytes, binary
		Length	4 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary
	Block 2	Start address	4 bytes, binary
		Length	4 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary
	:	:	:
	Block n	Start address	4 bytes, binary
		Length	4 bytes, binary

		Data	n bytes, binary
		Checksum	2 byte, binary

663072 v1 **Software part version in the data section**

Version history:
(1):

Purpose: To prevent resistant software part version stored in the ECU.
Source: N/A
Verification Method: Analysis
Legacy ID: 64726v0

The software part version containing sw_part_number and sw_version shall reside somewhere in the data section in the VBF file that is being programmed into the ECU and this downloaded part number shall be readable using service 0x22 (ReadDataByIdentifier).

4.11.7.2.4.3.1 Checksum calculation

663074 v1 **Data block checksum**

Version history:
(1):

Purpose: To define the format and use of the data block checksum.
Source: N/A
Verification Method: Test
Legacy ID: 64728v3

The VBF file shall include a 2-byte checksum for every data block. The 2-byte checksum shall be calculated including all data bytes (excluding Start address, Length, and Checksum) in the data block. If a data compression method is used, the data block checksum shall be calculated using unprocessed data (i.e. uncompressed and/or not encrypted). Block checksums should not be used to verify the integrity of the VBF file.

The checksum algorithm to be used shall be the CRC16-CITT:

- Polynomial = 0x1021 (x^16+x^12+x^5+1)
- Initial value = 0xFFFF

For a fast CRC16-CITT calculation a look-up table implementation is the preferred solution. See Appendix A for "C-code" example.

•Appendix A

4.11.7.2.5 Appendix A

4.11.7.2.5.1 CRC16-CITT C-code example

This example uses a look-up table with pre-calculated CRCs for fast calculation.

```
/* function declarations */
```

```
unsigned int CalcCRC(unsigned int size,unsigned char *data);

/* test data array */
unsigned char data[256] =
{  0x00,0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08,0x09,0x0a,0x0b,0x0c,0x0d,0x0e,0x0f,
   0x10,0x11,0x12,0x13,0x14,0x15,0x16,0x17,0x18,0x19,0x1a,0x1b,0x1c,0x1d,0x1e,0x1f,
   0x20,0x21,0x22,0x23,0x24,0x25,0x26,0x27,0x28,0x29,0x2a,0x2b,0x2c,0x2d,0x2e,0x2f,
   0x30,0x31,0x32,0x33,0x34,0x35,0x36,0x37,0x38,0x39,0x3a,0x3b,0x3c,0x3d,0x3e,0x3f,
   0x40,0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48,0x49,0x4a,0x4b,0x4c,0x4d,0x4e,0x4f,
   0x50,0x51,0x52,0x53,0x54,0x55,0x56,0x57,0x58,0x59,0x5a,0x5b,0x5c,0x5d,0x5e,0x5f,
   0x60,0x61,0x62,0x63,0x64,0x65,0x66,0x67,0x68,0x69,0x6a,0x6b,0x6c,0x6d,0x6e,0x6f,
   0x70,0x71,0x72,0x73,0x74,0x75,0x76,0x77,0x78,0x79,0x7a,0x7b,0x7c,0x7d,0x7e,0x7f,
   0x80,0x81,0x82,0x83,0x84,0x85,0x86,0x87,0x88,0x89,0x8a,0x8b,0x8c,0x8d,0x8e,0x8f,
   0x90,0x91,0x92,0x93,0x94,0x95,0x96,0x97,0x98,0x99,0x9a,0x9b,0x9c,0x9d,0x9e,0x9f,
   0xa0,0xa1,0xa2,0xa3,0xa4,0xa5,0xa6,0xa7,0xa8,0xa9,0xaa,0xab,0xac,0xad,0xae,0xaf,
   0xb0,0xb1,0xb2,0xb3,0xb4,0xb5,0xb6,0xb7,0xb8,0xb9,0xba,0xbb,0xbc,0xbd,0xbe,0xbf,
   0xc0,0xc1,0xc2,0xc3,0xc4,0xc5,0xc6,0xc7,0xc8,0xc9,0xca,0xcb,0xcc,0xcd,0xce,0xcf,
   0xd0,0xd1,0xd2,0xd3,0xd4,0xd5,0xd6,0xd7,0xd8,0xd9,0xda,0xdb,0xdc,0xdd,0xde,0xdf,
   0xe0,0xe1,0xe2,0xe3,0xe4,0xe5,0xe6,0xe7,0xe8,0xe9,0xea,0xeb,0xec,0xed,0xee,0xef,
   0xf0,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7,0xf8,0xf9,0xfa,0xfb,0xfc,0xfd,0xfe,0xff
};

unsigned int crctab[256] =
{
   0x0000, 0x1021, 0x2042, 0x3063, 0x4084, 0x50A5, 0x60C6, 0x70E7,
   0x8108, 0x9129, 0xA14A, 0xB16B, 0xC18C, 0xD1AD, 0xE1CE, 0xF1EF,
   0x1231, 0x0210, 0x3273, 0x2252, 0x52B5, 0x4294, 0x72F7, 0x62D6,
   0x9339, 0x8318, 0xB37B, 0xA35A, 0xD3BD, 0xC39C, 0xF3FF, 0xE3DE,
   0x2462, 0x3443, 0x0420, 0x1401, 0x64E6, 0x74C7, 0x44A4, 0x5485,
   0xA56A, 0xB54B, 0x8528, 0x9509, 0xE5EE, 0xF5CF, 0xC5AC, 0xD58D,
   0x3653, 0x2672, 0x1611, 0x0630, 0x76D7, 0x66F6, 0x5695, 0x46B4,
   0xB75B, 0xA77A, 0x9719, 0x8738, 0xF7DF, 0xE7FE, 0xD79D, 0xC7BC,
   0x48C4, 0x58E5, 0x6886, 0x78A7, 0x0840, 0x1861, 0x2802, 0x3823,
   0xC9CC, 0xD9ED, 0xE98E, 0xF9AF, 0x8948, 0x9969, 0xA90A, 0xB92B,
   0x5AF5, 0x4AD4, 0x7AB7, 0x6A96, 0x1A71, 0x0A50, 0x3A33, 0x2A12,
   0xDBFD, 0xCBDC, 0xFBBF, 0xEB9E, 0x9B79, 0x8B58, 0xBB3B, 0xAB1A,
   0x6CA6, 0x7C87, 0x4CE4, 0x5CC5, 0x2C22, 0x3C03, 0x0C60, 0x1C41,
   0xEDAE, 0xFD8F, 0xCDEC, 0xDDCD, 0xAD2A, 0xBD0B, 0x8D68, 0x9D49,
   0x7E97, 0x6EB6, 0x5ED5, 0x4EF4, 0x3E13, 0x2E32, 0x1E51, 0x0E70,
   0xFF9F, 0xEFBE, 0xDFDD, 0xCFFC, 0xBF1B, 0xAF3A, 0x9F59, 0x8F78,
   0x9188, 0x81A9, 0xB1CA, 0xA1EB, 0xD10C, 0xC12D, 0xF14E, 0xE16F,
   0x1080, 0x00A1, 0x30C2, 0x20E3, 0x5004, 0x4025, 0x7046, 0x6067,
   0x83B9, 0x9398, 0xA3FB, 0xB3DA, 0xC33D, 0xD31C, 0xE37F, 0xF35E,
   0x02B1, 0x1290, 0x22F3, 0x32D2, 0x4235, 0x5214, 0x6277, 0x7256,
   0xB5EA, 0xA5CB, 0x95A8, 0x8589, 0xF56E, 0xE54F, 0xD52C, 0xC50D,
   0x34E2, 0x24C3, 0x14A0, 0x0481, 0x7466, 0x6447, 0x5424, 0x4405,
   0xA7DB, 0xB7FA, 0x8799, 0x97B8, 0xE75F, 0xF77E, 0xC71D, 0xD73C,
   0x26D3, 0x36F2, 0x0691, 0x16B0, 0x6657, 0x7676, 0x4615, 0x5634,
   0xD94C, 0xC96D, 0xF90E, 0xE92F, 0x99C8, 0x89E9, 0xB98A, 0xA9AB,
   0x5844, 0x4865, 0x7806, 0x6827, 0x18C0, 0x08E1, 0x3882, 0x28A3,
   0xCB7D, 0xDB5C, 0xEB3F, 0xFB1E, 0x8BF9, 0x9BD8, 0xABBB, 0xBB9A,
   0x4A75, 0x5A54, 0x6A37, 0x7A16, 0x0AF1, 0x1AD0, 0x2AB3, 0x3A92,
   0xFD2E, 0xED0F, 0xDD6C, 0xCD4D, 0xBDAA, 0xAD8B, 0x9DE8, 0x8DC9,
   0x7C26, 0x6C07, 0x5C64, 0x4C45, 0x3CA2, 0x2C83, 0x1CE0, 0x0CC1,
   0xEF1F, 0xFF3E, 0xCF5D, 0xDF7C, 0xAF9B, 0xBFBA, 0x8FD9, 0x9FF8,
   0x6E17, 0x7E36, 0x4E55, 0x5E74, 0x2E93, 0x3EB2, 0x0ED1, 0x1EF0
};

/* driver */
void main(void)
{
   unsigned int crc;
```

```
   crc=CalcCRC(256,data);

   while(1); /* the result of the above calculation shall be: crc=0x3FBD */
}


/* Calculate CRC */
unsigned int CalcCRC(unsigned int size, unsigned char *data)
{
   unsigned int crc=0xffff; /* initial value */
   int tmp;
   int i;

   for(i=0;i<size;i++) {
      tmp=(crc>>8)^data[i];
      crc=(crc<<8)^crctab[tmp];
   }
   return crc;
}
```

•Appendix B

4.11.7.2.6 Appendix B

4.11.7.2.6.1 CRC32 C-code example

This example uses a look-up table with pre-calculated CRCs for fast calculation.

```
/* function declarations */
void generate_crc32_table(void);
unsigned long calc_crc32(unsigned long size, unsigned char *data);

/* variables */
unsigned long crc32;
static unsigned long crc32_table[256];

/* test data array */
unsigned char data[256] =
{  0x00,0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08,0x09,0x0a,0x0b,0x0c,0x0d,0x0e,0x0f,
   0x10,0x11,0x12,0x13,0x14,0x15,0x16,0x17,0x18,0x19,0x1a,0x1b,0x1c,0x1d,0x1e,0x1f,
   0x20,0x21,0x22,0x23,0x24,0x25,0x26,0x27,0x28,0x29,0x2a,0x2b,0x2c,0x2d,0x2e,0x2f,
   0x30,0x31,0x32,0x33,0x34,0x35,0x36,0x37,0x38,0x39,0x3a,0x3b,0x3c,0x3d,0x3e,0x3f,
   0x40,0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48,0x49,0x4a,0x4b,0x4c,0x4d,0x4e,0x4f,
   0x50,0x51,0x52,0x53,0x54,0x55,0x56,0x57,0x58,0x59,0x5a,0x5b,0x5c,0x5d,0x5e,0x5f,
   0x60,0x61,0x62,0x63,0x64,0x65,0x66,0x67,0x68,0x69,0x6a,0x6b,0x6c,0x6d,0x6e,0x6f,
   0x70,0x71,0x72,0x73,0x74,0x75,0x76,0x77,0x78,0x79,0x7a,0x7b,0x7c,0x7d,0x7e,0x7f,
   0x80,0x81,0x82,0x83,0x84,0x85,0x86,0x87,0x88,0x89,0x8a,0x8b,0x8c,0x8d,0x8e,0x8f,
   0x90,0x91,0x92,0x93,0x94,0x95,0x96,0x97,0x98,0x99,0x9a,0x9b,0x9c,0x9d,0x9e,0x9f,
   0xa0,0xa1,0xa2,0xa3,0xa4,0xa5,0xa6,0xa7,0xa8,0xa9,0xaa,0xab,0xac,0xad,0xae,0xaf,
   0xb0,0xb1,0xb2,0xb3,0xb4,0xb5,0xb6,0xb7,0xb8,0xb9,0xba,0xbb,0xbc,0xbd,0xbe,0xbf,
   0xc0,0xc1,0xc2,0xc3,0xc4,0xc5,0xc6,0xc7,0xc8,0xc9,0xca,0xcb,0xcc,0xcd,0xce,0xcf,
   0xd0,0xd1,0xd2,0xd3,0xd4,0xd5,0xd6,0xd7,0xd8,0xd9,0xda,0xdb,0xdc,0xdd,0xde,0xdf,
   0xe0,0xe1,0xe2,0xe3,0xe4,0xe5,0xe6,0xe7,0xe8,0xe9,0xea,0xeb,0xec,0xed,0xee,0xef,
   0xf0,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7,0xf8,0xf9,0xfa,0xfb,0xfc,0xfd,0xfe,0xff
};


/* driver */
void main(void)
{
   unsigned long crc32;

   generate_crc32_table();
```

```
   crc32=calc_crc32(256,data);

   while(1); /* the result of the above calculation shall be: crc32=0x29058c73 */
}


/* calculate the crc32 value */
unsigned long calc_crc32(unsigned long size, unsigned char *data)
{

   unsigned long i, crc;

   crc=0xffffffff;       // initial value

   for(i=0;i<size;i++) {
      crc = ((crc>>8)&0x00FFFFFF)^crc32_table[(int)(crc^data[i])&0xff];
   }

   return ~crc;
}

#define CRC32_POLY 0xedb88320 // 0x04c11db7 bit reversed

/* generate the lock-up table */
void generate_crc32_table(void)

{
   int i, j;
   unsigned long crc;

   for(i=0;i<256;i++) {
      crc=i;
      for(j=8;j>0;j--) {
         if(crc&1) {
            crc=(crc>>1)^CRC32_POLY;
         }
         else{
            crc>>=1;
         }
      }
      crc32_table[i]=crc;
   }
}

// As an alternative to the "generate_crc32_table()" function the following table
can be used

/*
static unsigned long crc32_tab[256] =
{  0x00000000L, 0x77073096L, 0xee0e612cL, 0x990951baL, 0x076dc419L, 0x706af48fL,
   0xe963a535L, 0x9e6495a3L, 0x0edb8832L, 0x79dcb8a4L, 0xe0d5e91eL, 0x97d2d988L,
   0x09b64c2bL, 0x7eb17cbdL, 0xe7b82d07L, 0x90bf1d91L, 0x1db71064L, 0x6ab020f2L,
   0xf3b97148L, 0x84be41deL, 0x1adad47dL, 0x6ddde4ebL, 0xf4d4b551L, 0x83d385c7L,
   0x136c9856L, 0x646ba8c0L, 0xfd62f97aL, 0x8a65c9ecL, 0x14015c4fL, 0x63066cd9L,
   0xfa0f3d63L, 0x8d080df5L, 0x3b6e20c8L, 0x4c69105eL, 0xd56041e4L, 0xa2677172L,
   0x3c03e4d1L, 0x4b04d447L, 0xd20d85fdL, 0xa50ab56bL, 0x35b5a8faL, 0x42b2986cL,
   0xdbbbc9d6L, 0xacbcf940L, 0x32d86ce3L, 0x45df5c75L, 0xdcd60dcfL, 0xabd13d59L,
   0x26d930acL, 0x51de003aL, 0xc8d75180L, 0xbfd06116L, 0x21b4f4b5L, 0x56b3c423L,
   0xcfba9599L, 0xb8bda50fL, 0x2802b89eL, 0x5f058808L, 0xc60cd9b2L, 0xb10be924L,
   0x2f6f7c87L, 0x58684c11L, 0xc1611dabL, 0xb6662d3dL, 0x76dc4190L, 0x01db7106L,
   0x98d220bcL, 0xefd5102aL, 0x71b18589L, 0x06b6b51fL, 0x9fbfe4a5L, 0xe8b8d433L,
   0x7807c9a2L, 0x0f00f934L, 0x9609a88eL, 0xe10e9818L, 0x7f6a0dbbL, 0x086d3d2dL,
   0x91646c97L, 0xe6635c01L, 0x6b6b51f4L, 0x1c6c6162L, 0x856530d8L, 0xf262004eL,
```

```
    0x6c0695edL, 0x1b01a57bL, 0x8208f4c1L, 0xf50fc457L, 0x65b0d9c6L, 0x12b7e950L,
    0x8bbeb8eaL, 0xfcb9887cL, 0x62dd1ddfL, 0x15da2d49L, 0x8cd37cf3L, 0xfbd44c65L,
    0x4db26158L, 0x3ab551ceL, 0xa3bc0074L, 0xd4bb30e2L, 0x4adfa541L, 0x3dd895d7L,
    0xa4d1c46dL, 0xd3d6f4fbL, 0x4369e96aL, 0x346ed9fcL, 0xad678846L, 0xda60b8d0L,
    0x44042d73L, 0x33031de5L, 0xaa0a4c5fL, 0xdd0d7cc9L, 0x5005713cL, 0x270241aaL,
    0xbe0b1010L, 0xc90c2086L, 0x5768b525L, 0x206f85b3L, 0xb966d409L, 0xce61e49fL,
    0x5edef90eL, 0x29d9c998L, 0xb0d09822L, 0xc7d7a8b4L, 0x59b33d17L, 0x2eb40d81L,
    0xb7bd5c3bL, 0xc0ba6cadL, 0xedb88320L, 0x9abfb3b6L, 0x03b6e20cL, 0x74b1d29aL,
    0xead54739L, 0x9dd277afL, 0x04db2615L, 0x73dc1683L, 0xe3630b12L, 0x94643b84L,
    0x0d6d6a3eL, 0x7a6a5aa8L, 0xe40ecf0bL, 0x9309ff9dL, 0x0a00ae27L, 0x7d079eb1L,
    0xf00f9344L, 0x8708a3d2L, 0x1e01f268L, 0x6906c2feL, 0xf762575dL, 0x806567cbL,
    0x196c3671L, 0x6e6b06e7L, 0xfed41b76L, 0x89d32be0L, 0x10da7a5aL, 0x67dd4accL,
    0xf9b9df6fL, 0x8ebeeff9L, 0x17b7be43L, 0x60b08ed5L, 0xd6d6a3e8L, 0xa1d1937eL,
    0x38d8c2c4L, 0x4fdff252L, 0xd1bb67f1L, 0xa6bc5767L, 0x3fb506ddL, 0x48b2364bL,
    0xd80d2bdaL, 0xaf0a1b4cL, 0x36034af6L, 0x41047a60L, 0xdf60efc3L, 0xa867df55L,
    0x316e8eefL, 0x4669be79L, 0xcb61b38cL, 0xbc66831aL, 0x256fd2a0L, 0x5268e236L,
    0xcc0c7795L, 0xbb0b4703L, 0x220216b9L, 0x5505262fL, 0xc5ba3bbeL, 0xb2bd0b28L,
    0x2bb45a92L, 0x5cb36a04L, 0xc2d7ffa7L, 0xb5d0cf31L, 0x2cd99e8bL, 0x5bdeae1dL,
    0x9b64c2b0L, 0xec63f226L, 0x756aa39cL, 0x026d930aL, 0x9c0906a9L, 0xeb0e363fL,
    0x72076785L, 0x05005713L, 0x95bf4a82L, 0xe2b87a14L, 0x7bb12baeL, 0x0cb61b38L,
    0x92d28e9bL, 0xe5d5be0dL, 0x7cdcefb7L, 0x0bdbdf21L, 0x86d3d2d4L, 0xf1d4e242L,
    0x68ddb3f8L, 0x1fda836eL, 0x81be16cdL, 0xf6b9265bL, 0x6fb077e1L, 0x18b74777L,
    0x88085ae6L, 0xff0f6a70L, 0x66063bcaL, 0x11010b5cL, 0x8f659effL, 0xf862ae69L,
    0x616bffd3L, 0x166ccf45L, 0xa00ae278L, 0xd70dd2eeL, 0x4e048354L, 0x3903b3c2L,
    0xa7672661L, 0xd06016f7L, 0x4969474dL, 0x3e6e77dbL, 0xaed16a4aL, 0xd9d65adcL,
    0x40df0b66L, 0x37d83bf0L, 0xa9bcae53L, 0xdebb9ec5L, 0x47b2cf7fL, 0x30b5ffe9L,
    0xbdbdf21cL, 0xcabac28aL, 0x53b39330L, 0x24b4a3a6L, 0xbad03605L, 0xcdd70693L,
    0x54de5729L, 0x23d967bfL, 0xb3667a2eL, 0xc4614ab8L, 0x5d681b02L, 0x2a6f2b94L,
    0xb40bbe37L, 0xc30c8ea1L, 0x5a05df1bL, 0x2d02ef8dL
};
*/
```

4.11.8 Bootloader - UDSonCAN

4.11.8.1 Bootloader - UDSonCAN

•Introduction

4.11.8.1.1 Introduction

This document specifies OEM specific bootloaders requirements for a CAN ECU. It intends to provide guidance and requirements to architects, system designers, application developers, and suppliers who are implementing the bootloader to an ECU connected to a CAN network.

Important to understand is that this document only specifies the CAN network specific requirements and not the complete bootloader. Figure - Document structure, shows the specifications that are related for a CAN ECU when implementing the bootloader.

The main aim with software download is to fulfill the needs of replace software's at repair shop and the assembly plant.

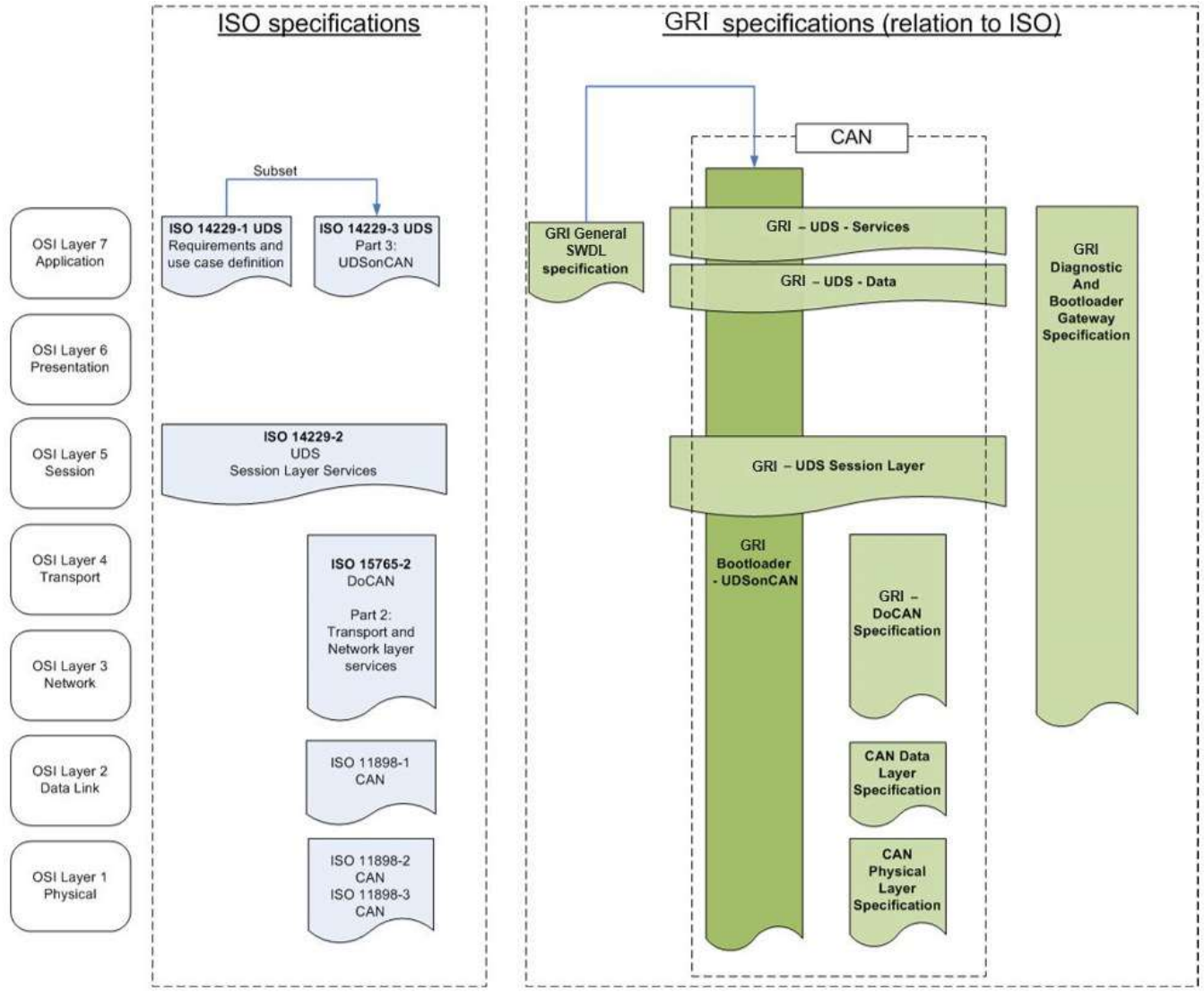


Figure - Document Structure

•Overview

4.11.8.1.2 Overview

This document specifies the CAN specific bootloader requirements that shall apply to a CAN based ECU. All diagnostic messages on CAN shall be sent and received according to this and referenced documents in section Requisite documents.

Additional requirements are added if the ECU shall act as a SWDL gateway to another network which is defined in *[BL_CAN_7] Diagnostic and Bootloader Gateway Specification*.

Note

This document covers both the older CAN standard, referred to as classic CAN, and the updated standard CAN Flexible Data-Rate, referred to as CAN FD. Unless explicitly specifying the standard of CAN in this document, any reference to CAN refers to both the classic CAN and CAN FD standards.

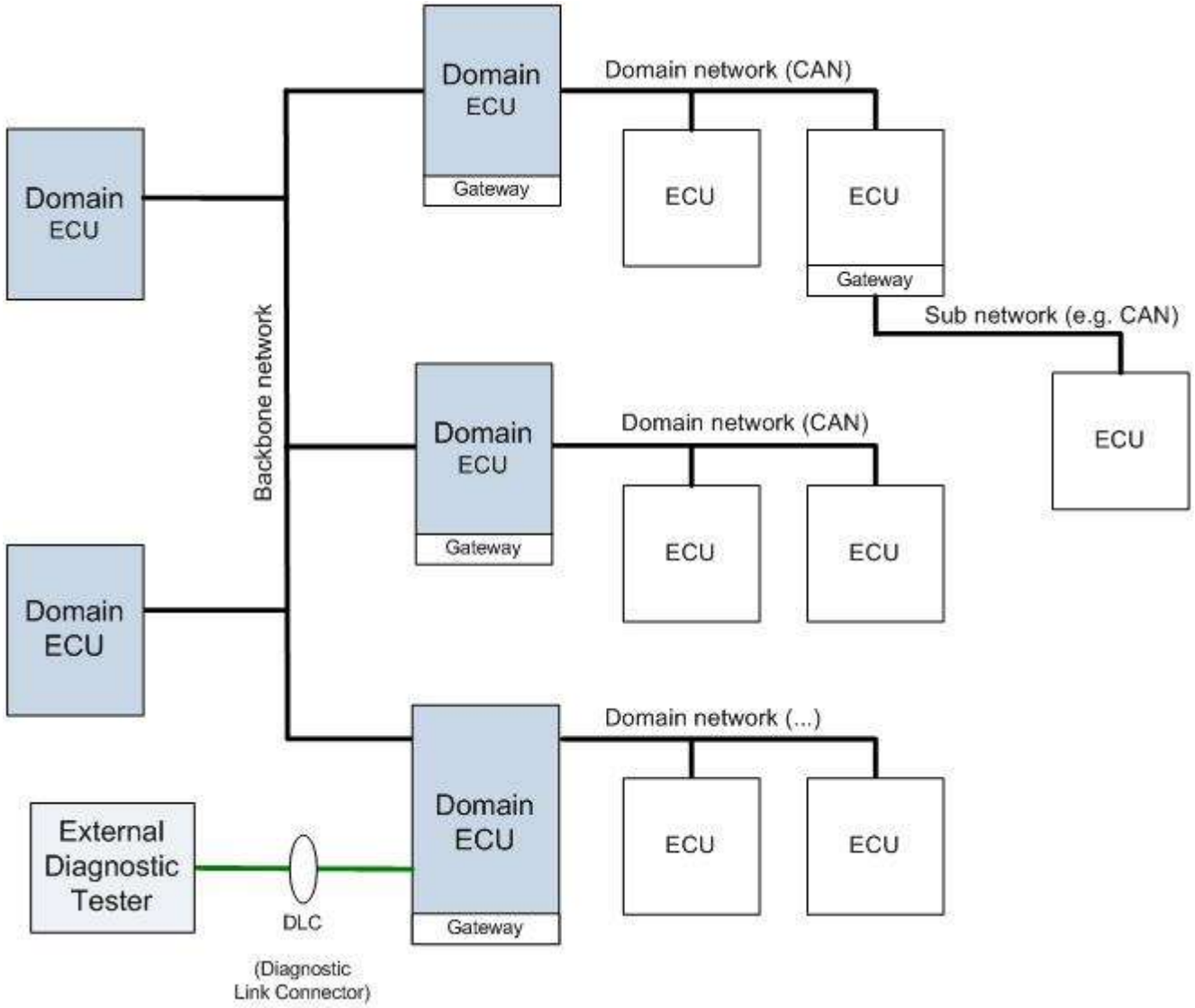


Figure - Example of the vehicle network topology.

•Requisite Documents

4.11.8.1.3 Requisite Documents

Reference number	Source	Document
[BL_CAN_1]	Internal	General Software Download
[BL_CAN_2]	Internal	Session generic reqs
[BL_CAN_3]	Internal	UDS Session server reqs
[BL_CAN_4]	Internal	UDS Services
[BL_CAN_5]	Internal	DoCAN
[BL_CAN_6]	Internal	CAN Datalink Layer
[BL_CAN_7]	Internal	Diagnostic and Bootloader Gateway Specification
[BL_CAN_8]	ISO	ISO 14229-3: Road vehicles - Unified diagnostic services (UDS) - Part 3: UDS on CAN implementation (UDSonCAN)

Note:
Internal references are typically available as separate chapters in ECU specific Base Technology SWRS.
Information obtaining ISO documents may be found on the internet: www.iso.org

•Definitions and Abbreviations

4.11.8.1.4 Definitions and Abbreviations

Definition	Description
Client	Software entity which uses services of a server
Client-Server Communication	A specific form of communication in a possibly distributed system in which software entities act as clients, servers or both, where 1...n clients are requesting services via a specific protocol from typically one server.
Cold Start	The start-up of the ECU from a powered-down or off state.
Gateway	An ECU that is connected to two or more independent communication networks that allows information to flow between the networks.
Program mode	Program mode means that the ECU executing the primary or secondary bootloader.
Sender-Receiver Communication	A communication pattern which offers asynchronous distribution of information where a sender communicates information to one or more receivers, or a receiver receives information from one or several senders.
Server	Software entity which provides services for clients
Warm start	The start-up of the ECU initiated by the application.

Abbreviation	Description
ECU	Electronic Control Unit
PBL	Primary Boot Loader
SBL	Secondary Boot Loader
UDS	Unified Diagnostics Services
USDT	Unified Segmented Data Transfer

•Unified Diagnostic Services Implementation

4.11.8.1.5 Unified Diagnostic Services Implementation

This section specifies the diagnostic services and data required for a CAN bootloader.

4.11.8.1.5.1 Diagnostic Services

The required diagnostic services for the bootloader are specified in *[BL_CAN_4] UDS Services* with the restrictions/additions as defined by the subsequent sub-sections.

4.11.8.1.5.1.1 RequestDownload (34 hex) Service

The required diagnostic services for the bootloader are specified in *[BL_CAN_2] UDS Services* with the restrictions/additions as defined by the subsequent sub-sections.

662900 v2	RequestDownload Service

Version history:
(2): 要求CAN节点刷写时最小的接收性能有2048字节扩大为3842字节。

Purpose: To define the diagnostic data used by the bootloader.
Source:
Verification Method: Test
Legacy ID: 52235v1

The maxNumberOfBlockLenght shall be dimensioned to be as big as possible, i.e. 0x0FFF. And minimum allowed maxNumberOfBlockLenght shall be 3842 (0x0F00 + 2, *2 byte will take the SID and block sequence counter*).

•ECU Program Mode

4.11.8.1.6 ECU Program Mode

Program mode means that the ECU executing the primary or secondary bootloader. This chapter describes state and flow chart diagrams unique to the CAN bootloader. Additional shall the CAN bootloader also implement the non network dependent bootloader requirements described in *[BL_CAN_1] General Software Download*.

662907 v1 **Bootloader CAN busoff handling**

Version history:
(1):

Purpose: To increase availablility of the system. Avoid that an ECU gets stuck in bus-off state, or exits programming session due to network errors.
Source:
Verification Method: Test
Legacy ID: 466900

If hardware support automatic CAN busoff recovery, that recovery mechanism shall be enabled.

If hardware doesn’t support automatic bus-off recovery the bootloader shall check all active CAN controllers for bus-off status at maximum 10 ms intervals and if bus-off is detected start the bus-off recovery sequence within 10ms.
This requirement applies to Primary Boot Loader (PBL) and Secondary Boot Loader (SBL).

Note: If a permanent busoff condition is detected that can not be recovered, and a complete and compatible application exist, the S3server timer shall still run and will finally elapse, causing and exit of the programming session.

662908 v1 **General bootloader requirements**

Version history:
(1):

Purpose: Non-network dependent bootloader requirements are defined in a separate document which is applicable for all bootloaders.
Source:
Verification Method: Analysis
Legacy ID: 52249v2

The CAN bootloader shall implement the non network dependent requirement defined in

[BL_CAN_1] General Software Download .

4.11.8.1.6.1 Bootloader state diagram

There are two bootloaders required, the primary bootloader and the secondary bootloader. These are described in more detail in *[BL_CAN_1] General Software Download.* The CAN bootloader state diagram , the figure below is a complement to the flow chart diagram described in section *Bootloader flow chart diagram - Overview.*

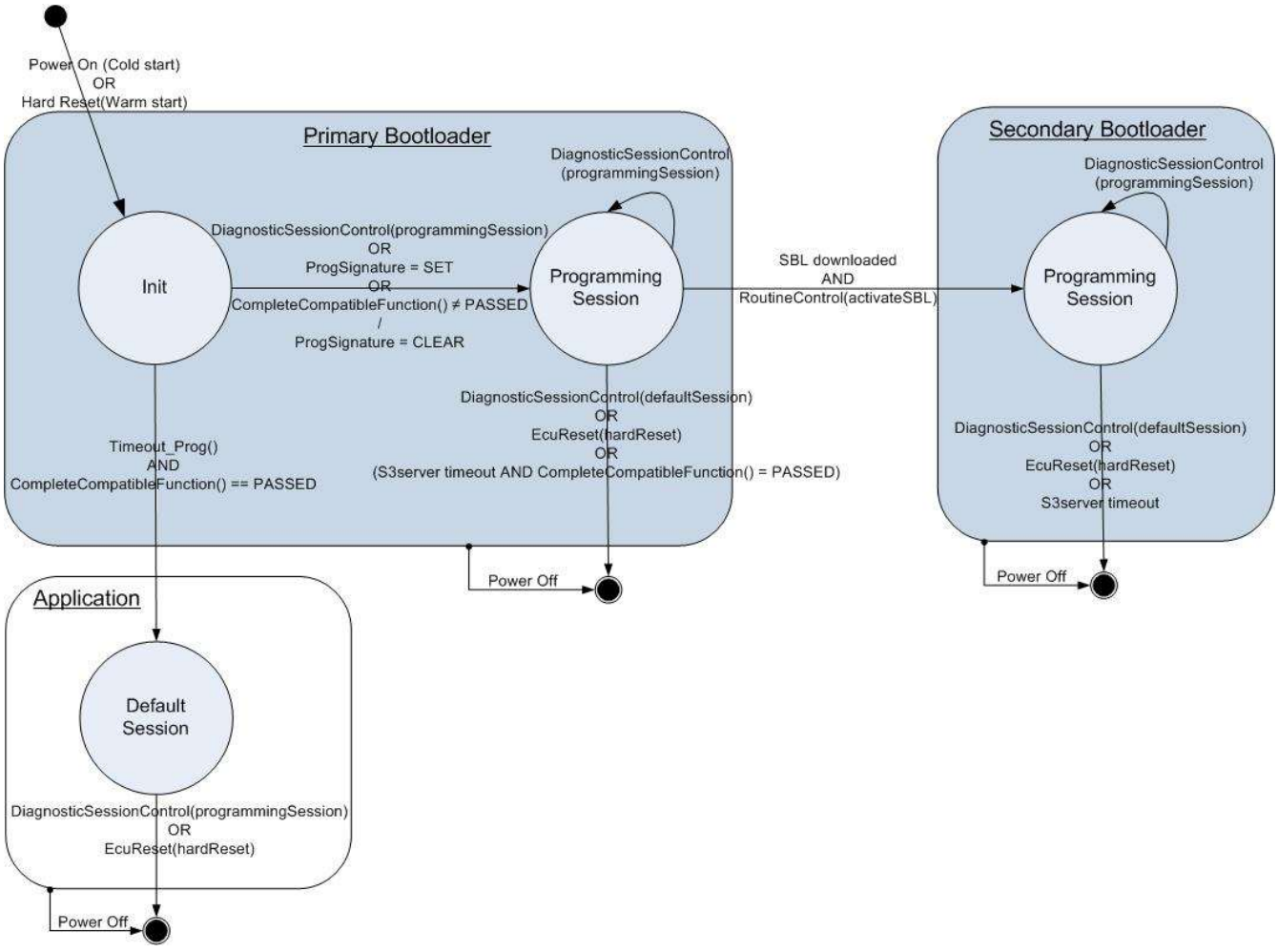


Figure – Bootloader State diagram

4.11.8.1.6.2 Bootloader flow chart diagram - Overview

The CAN bootloader flow chart diagram applies to all CAN based bootloader ECUs, requirements. The flow chart diagram is divided in states. Each state is further described in the following sections:

- Bootloader flow chart diagram (Init)
- Bootloader flow chart diagram (ProgSession)
- Flow chart diagram (Application)

Some figures contain yellow requirements tags. These requirement tags show where each requirement applies in the flow chart. To avoid problems with hardware initialization (i.e., write

once registers, PLL-setup, memory re-map, interrupt vector table re-map, etc.), the ECU shall make a complete reset when exiting the application.

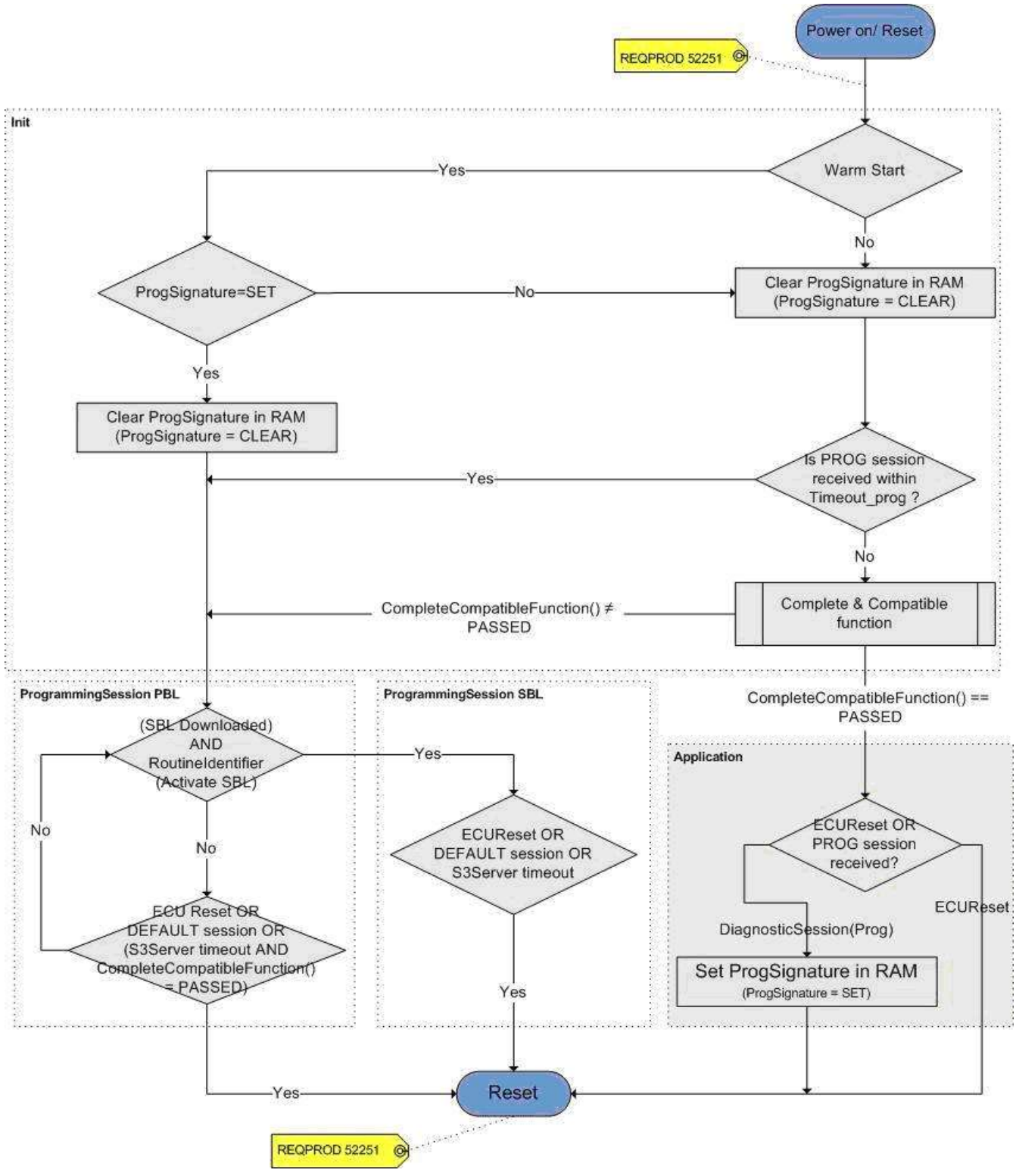


Figure - Bootloader flow chart diagram (Overview)

662909 v1 **Bootloader flow chart**

Version history:
(1):

Purpose: To define the bootloader flow chart. The flow chart diagram is applicable for all ECUs implementing CAN.
Source:
Verification Method: Analysis AND Test
Legacy ID: 52250v1

The CAN bootloader implementation shall follow flow chart in accordance to the figure *Bootloader flow chart diagram (Overview)* . The figure shows the order in which different sessions are activated.

662910 v1 **Complete hardware initiation when starting the ECU**

Version history:
(1):

Purpose: To avoid problems with hardware initialisation (i.e., write once registers, PLL-setup, memory re-map, interrupt vector table re-map, etc.)
Source:
Verification Method: Analysis
Legacy ID: 52251v1

The ECU shall always start from reset, regardless if the reset was initiated by a diagnostic request or by power up.

4.11.8.1.6.2.1 Bootloader flow chart diagram (Init)

The ECU shall be able to detect if start-up was caused by power-on (cold start) or initiated by the application (warm start).
Before the ECU starts the application, the ECU shall listen for the diagnostic request DiagnosticSessionControl(ProgrammingSession) during Timeout_prog. This time window may be used as a "backdoor" if the ECU is for some reason not able to communicate when the application is started. Duration of Timeout_prog is defined in *[BL_CAN_1] General Software Download*.
The Complete & Compatible function is responsible to decide whether ECUs software's are complete and compatible. The implementation of this step is specified in *[BL_CAN_1] General Software Download*.

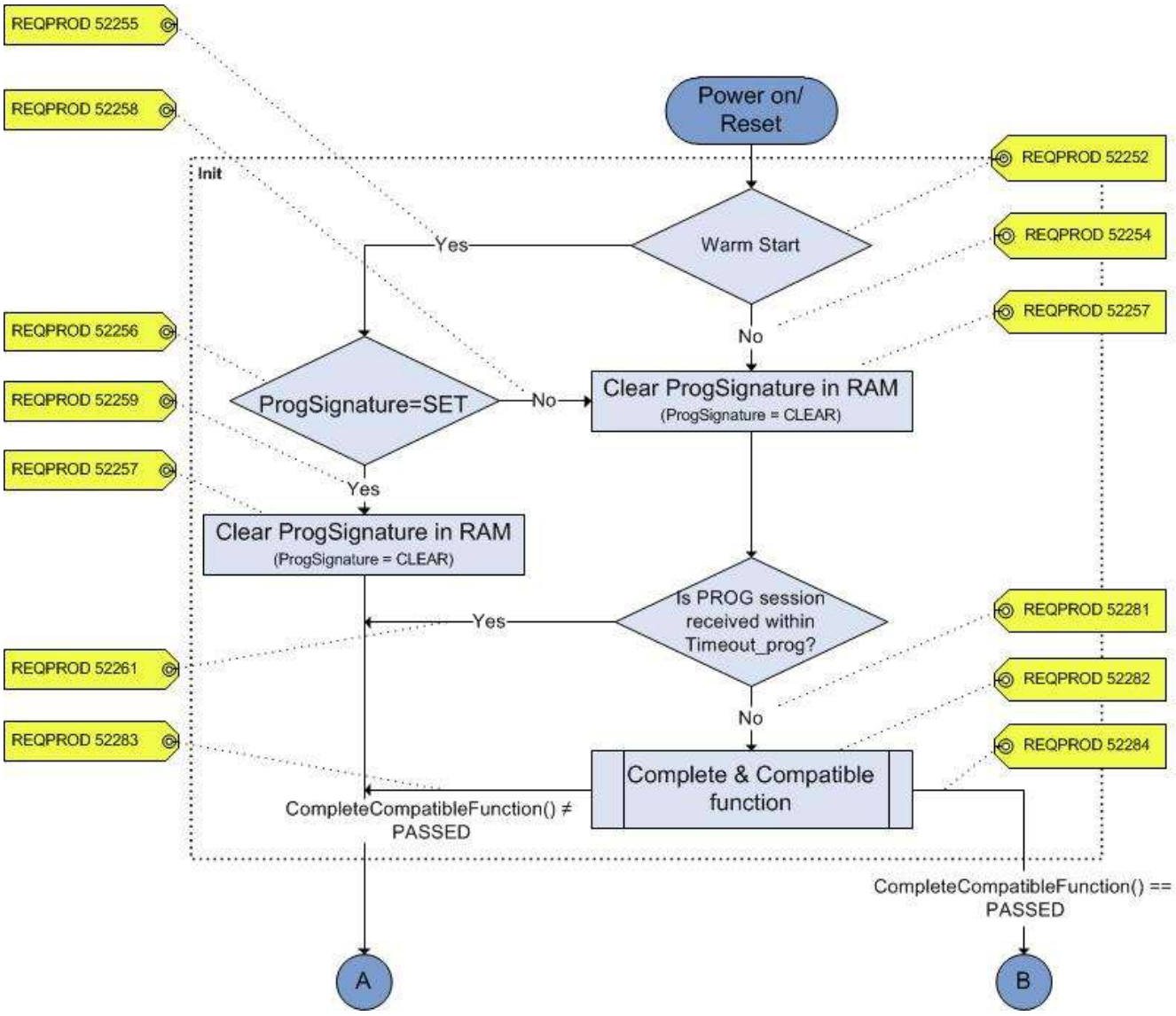


Figure - Bootloader flow chart diagram (Init)

662914 v1 **Check ProgSignature in RAM**

Version history:
(1):

Purpose: To define the "Check ProgSignature in RAM" state that is used to communicate status between the application and the bootloader.

Source:

Verification Method: Analysis

Legacy ID: 52256v1

The ECU shall after a warm start check whether the ProgSignature at fixed RAM address is SET or not.
Note:
The ProgSignature in RAM shall be written by the application when a DiagnosticSessionControl(programmingSession) received, the format and length of the

ProgSignature is described in *[BL_CAN_1] General Software Download*.

662915 v1 **Clear ProgSignature in RAM**

Version history:
(1):

Purpose: To define the "Clear ProgSignature in RAM" function that provides protection against unwanted bootloader activation.
Source:
Verification Method: Analysis
Legacy ID: 52257v1

Note: Th e format and length of the ProgSignature is described in *[BL_CAN_1] General Software Download*.

662911 v1 **Cold or warm start detected**

Version history:
(1):

Purpose: The warm start-up state checks the cause of start-up to avoid having the bootloader unintentionally entering programming mode during cold start
Source:
Verification Method: Analysis
Legacy ID: 52252v1

The ECU shall be able to detect the type of start (cold or warm) and use that information to decide the next transition.
Note:
This information can for example be found in a transceiver, in the processor or in a voltage regulator. Where it is found is dependent on the hardware.

662912 v1 **Cold start detected**

Version history:
(1):

Purpose: There is a difference in actions between warm and cold start-up. This requirement defines the next transition when cold start is detected
Source:
Verification Method: Analysis
Legacy ID: 52254v1

When cold start is detected next step is to clear the ProgSignature.

662920 v1 **Complete & Compatible function**

Version history:
(1):

Purpose: The ECU shall be able to detect if the current downloaded software is

compatible or not.

Source:
Verification Method: Analysis
Legacy ID: 52282v2

The ECU shall detect if the software is compatible or not, by using Complete & Compatible function specified in *[BL_CAN_1] General Software Download*.

662922 v1 **Complete and compatible software(s)**

Version history:
(1):

Purpose: To define the action when the software(s) is/are complete and compatible.
Source:
Verification Method: Analysis
Legacy ID: 52284v1

If the return value from the Complete & Compatible function is equal to PASSED, it is safe to start the application.

662919 v1 **No PROG received within Timeout_Prog**

Version history:
(1):

Purpose: To define next transition when no DiagnosticSessionConrol(ProgrammingSession) request is received within Timeout_Prog.
Source:
Verification Method: Analysis
Legacy ID: 52281v1

If no DiagnosticSessionConrol(ProgrammingSession) request is received within Timeout_Prog shall the ECU enter the Complete & Compatible function.

662921 v1 **Non complete or compatible software(s)**

Version history:
(1):

Purpose: To define the action when software(s) is/are neither complete nor compatible.
Source:
Verification Method: Analysis
Legacy ID: 52283v1

If the return value from the Complete & Compatible function is not equal to PASSED it means the ECU is either not complete or compatible, and therefore the ECU shall go to “programmingSession” state.

662918 v1 **PROG received within Timeout_Prog**

Version history:
(1):

Purpose: To be able to enter the programmingSession state when the DiagnosticSessionConrol(ProgrammingSession) request is received within Timeout_prog.
Source:
Verification Method: Test
Legacy ID: 52261v1

When the ECU receives the diagnostic request DiagnosticSessionConrol(ProgrammingSession) within Timeout_Prog shall the ECU enter the “programmingSession” state.

662917 v1 ProgSignature is SET

Version history:
(1):

Purpose: The purpose of setting the ProgSignature to a predefined value is to enter the programmingSession state
Source:
Verification Method: Analysis
Legacy ID: 52259v1

When the ProgSignature is SET the next step is to enter the “Clear ProgSignature in RAM” state.

662916 v1 ProgSignature is not SET

Version history:
(1):

Purpose: To define next transition when the ProgSignature not is set
Source:
Verification Method: Analysis
Legacy ID: 52258v1

When the ProgSignature is not SET the ECU shall enter the “Clear ProgSignature in RAM” state.

662913 v1 Warm start detected

Version history:
(1):

Purpose: There is a difference in actions between warm and cold start-up. This requirement defines the next transition when warm start is detected.
Source:
Verification Method: Analysis
Legacy ID: 52255v1

When warm start is detected next step is to check if the ProgSignature is SET.

4.11.8.1.6.2.2 Bootloader flow chart diagram (ProgSession)

ProgrammingSession state is entered when ProgSignature is set or when the software's is not Complete & Compatible or a ProgrammingSession was received before Timeout_Prog time ran out.
When the ECU has entered the programmingSession state the ECU shall be able to receive, process and respond to diagnostic requests supported by the bootloader.

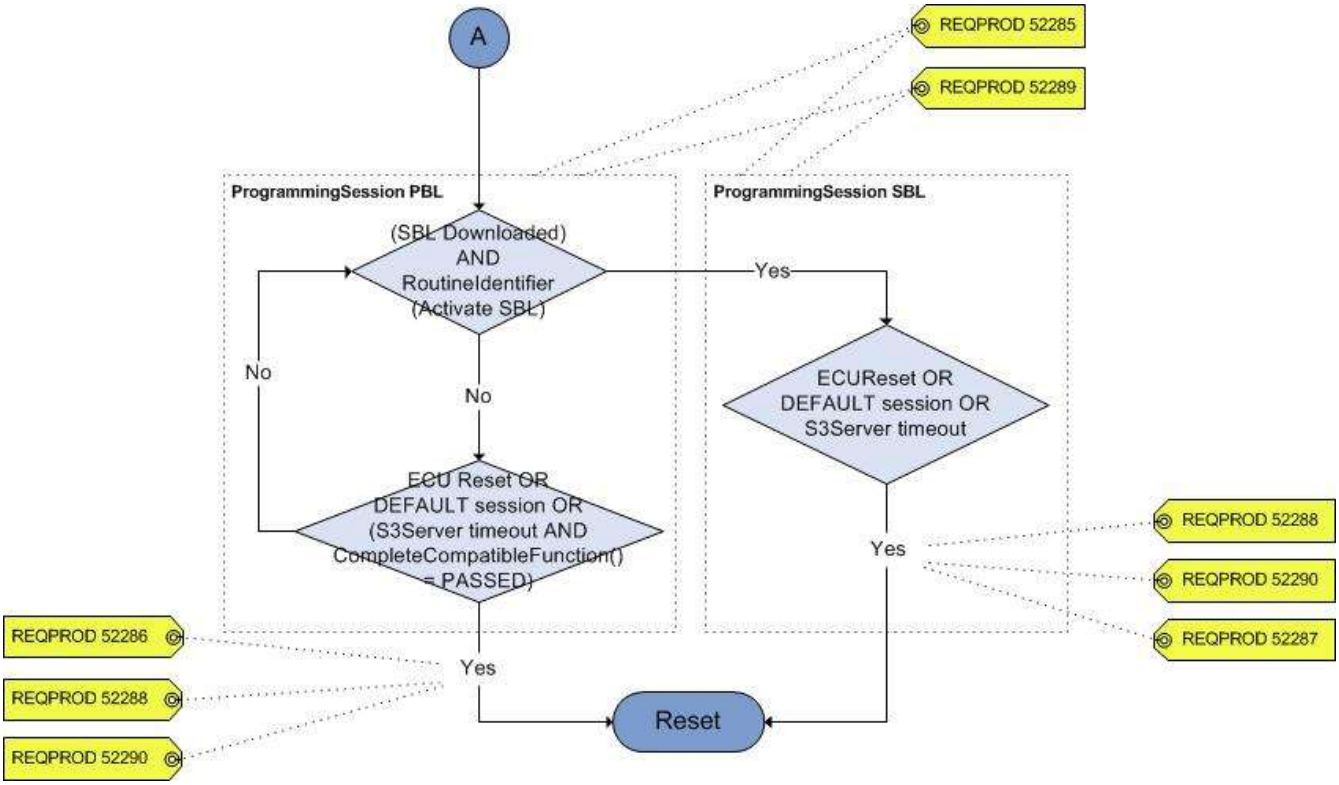


Figure - Bootloader flow chart diagram (ProgSession)

662923 v1 **Receive and process diagnostic programming session requests**

Version history:
(1):

Purpose: To define when the ECU shall be able to receive and process a diagnostic programming session request.
Source:
Verification Method: Test
Legacy ID: 52285v1

When the ECU is in programmingSession state, the ECU shall be able to receive and process diagnostic programming session requests.

662927 v1 **Reception of DiagSessionControl(programmingSession)**

Version history:

(1):

Purpose: To prevent a reset when the bootloader is in the programmingSession state and receive a diagnostic request DiagnosticSessionControl(programmingSession)
Source:
Verification Method: Test
Legacy ID: 52289v1

When the bootloader is in the programmingSession state (PBL or SBL) and receives a DiagnosticSessionControl(programmingSession) request, the ECU shall not reset i.e. the ECU shall stay in the programmingSession state.

662926 v1 Reception of DiagnosticSessionControl(defaultSession)

Version history:
(1):

Purpose: To define the behaviour when the ECU receives the DiagnosticSessionControl(defaultSession) request in the programmingSession state.
Source:
Verification Method: Test
Legacy ID: 52288v1

When the ECU receives a DiagnosticSessionControl(defaultSession) request, the ECU shall make a reset, thus restarting the ECU from Init state.

662928 v1 Reception of the ECUReset service

Version history:
(1):

Purpose: To define the behaviour when the ECU receives the ECUReset request in the programmingSession state.
Source:
Verification Method: Test
Legacy ID: 52290v1

When the ECU receives an ECUReset request, the ECU shall make a reset, thus restarting the ECU from Init state.

662925 v1 S3Server timer timeout in PBL

Version history:
(1):

Purpose: To define the behaviour when S3server times out in PBL.
Source:
Verification Method: Test
Legacy ID: 52286v2

If S3 $_{server}$ times out when running PBL, the ECU shall make a test on Complete and Compatible function and if Complete and Compatible function returns PASSED the ECU shall make a reset.
Note:
The S3 $_{server}$ is defined in [BL_CAN_2] UDS Session generic reqs.

662924 v1 **S3Server timer timeout in SBL**

Version history:
(1):

Purpose: To define the behaviour when the S3server times out in SBL
Source:
Verification Method: Test
Legacy ID: 52287v2

When the S3 $_{server}$ times out when running SBL, the ECU shall make a reset.

Note: *The S3 $_{server}$ is defined in [BL_CAN_2] UDS Session generic reqs*.

4.11.8.1.6.2.3 Flow chart diagram (Application)

This is the normal operation mode of the ECU. To make the ECU enter the programmingSession, a client needs to send a DiagnosticSessionControl(programmingSession) request to the ECU. When all criteria for entering the bootloader has been met, it shall be enough sending one DiagnosticSessionControl(programmingSession) request to make the ECU set the ProgSignature and execute the reset.

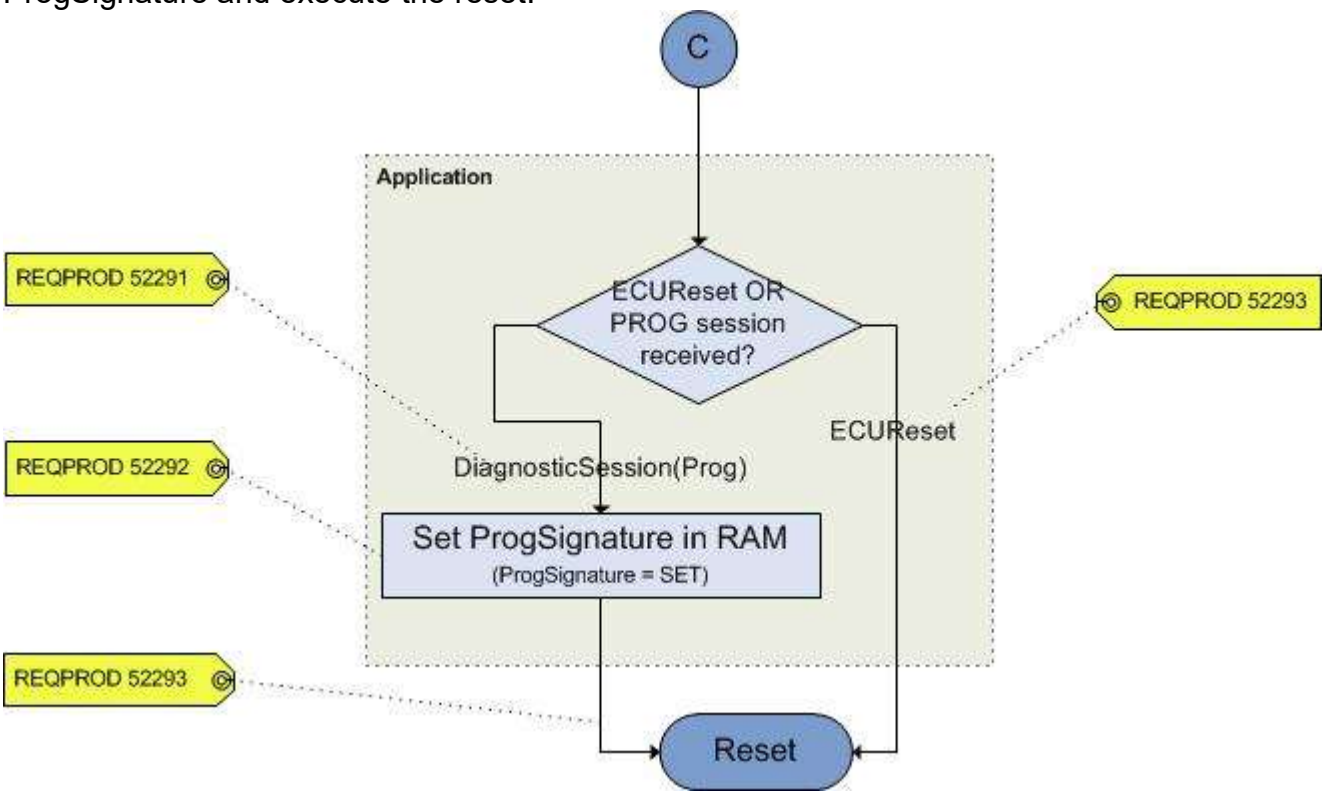


Figure - Flow chart diagram (Start Application)

662929 v1 **Enter bootloader from ECU application**

Version history:
(1):

Purpose: To define how to set the ECU to program mode when the ECU application is active.
Source:
Verification Method: Test
Legacy ID: 52291v1

When all criteria for entering the bootloader has been met, it must be possible to enter the bootloader from the application by receiving the DiagnosticSessionControl(programmingSession) service only once.

662931 v1 **Trig a reset in ECU application mode**

Version history:
(1):

Purpose: To enter the bootloader from the application
Source:
Verification Method: Test
Legacy ID: 52293v1

The ECU application shall trig a reset on reception of a DiagnosticSessionControl(programmingSession) or an ECUReset(hardReset)

662930 v1 **Write ProgSignature**

Version history:
(1):

Purpose: The ProgSignature is used after reset to determine whether the ECU shall stay in the bootloader or start the application.
Source:
Verification Method: Analysis
Legacy ID: 52292v2

W hen the ECU application receives a DiagnosticSessionControl(programmingSession) request, the ProgSignature shall be written to a fixed address in RAM.

Note: *The format and length of the ProgSignature is described in [BL_CAN_1] General Software Download*.

•Requirements

4.11.8.1.7 Requirements

667098 v2 **Zone Master Node - Transmission of Network Management PDU on CAN network**

Version history:
(2):

Purpose: To ensure that ECUs on CAN network will be ready to receive diagnostic requests even if the zone master ECU is not capable of activating communication on CAN ECUs by means of Autosar Network Management. A typical case is when the zone master ECU has no valid application programmed.

Source:

Verification Method: Test

Legacy ID:

When the ECU as the Zone Master is in programming session, the PBL/SBL shall periodically transmit a CAN frame with CAN identifier 0x53F with a period of 400ms(± 10%) on the missing frame master CAN network connected to the ECU. Frame content shall be 0x3F, 0x40, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF.

From the time that the ECU has entered programming session, the first CAN frame shall be transmitted after maximum 10ms.

If the ECU exits programming session, this CAN frame shall not be transmitted.

Note2: *The frame content is based on that NodeId is in the first byte, ControlBitVector is in the second byte with PNI bit set, and the remaining six bytes of NM User Data are available for PNC requests.*

4.11.9 Bootloader - UDSonLIN

4.11.9.1 Bootloader - UDSonLIN - Generic

•Introduction

4.11.9.1.1 Introduction

This document specifies LIN slave bootloader diagnostic requirements. It intends to provide guidance and requirements to architects, system designers, application developers and suppliers of how LIN bootloaders are to be implemented.
Important to understand is that this document only specifies the LIN network specific requirements and not the complete bootloader. Figure - Document structure, shows the specifications that are related for a LIN ECU when implementing the bootloder.

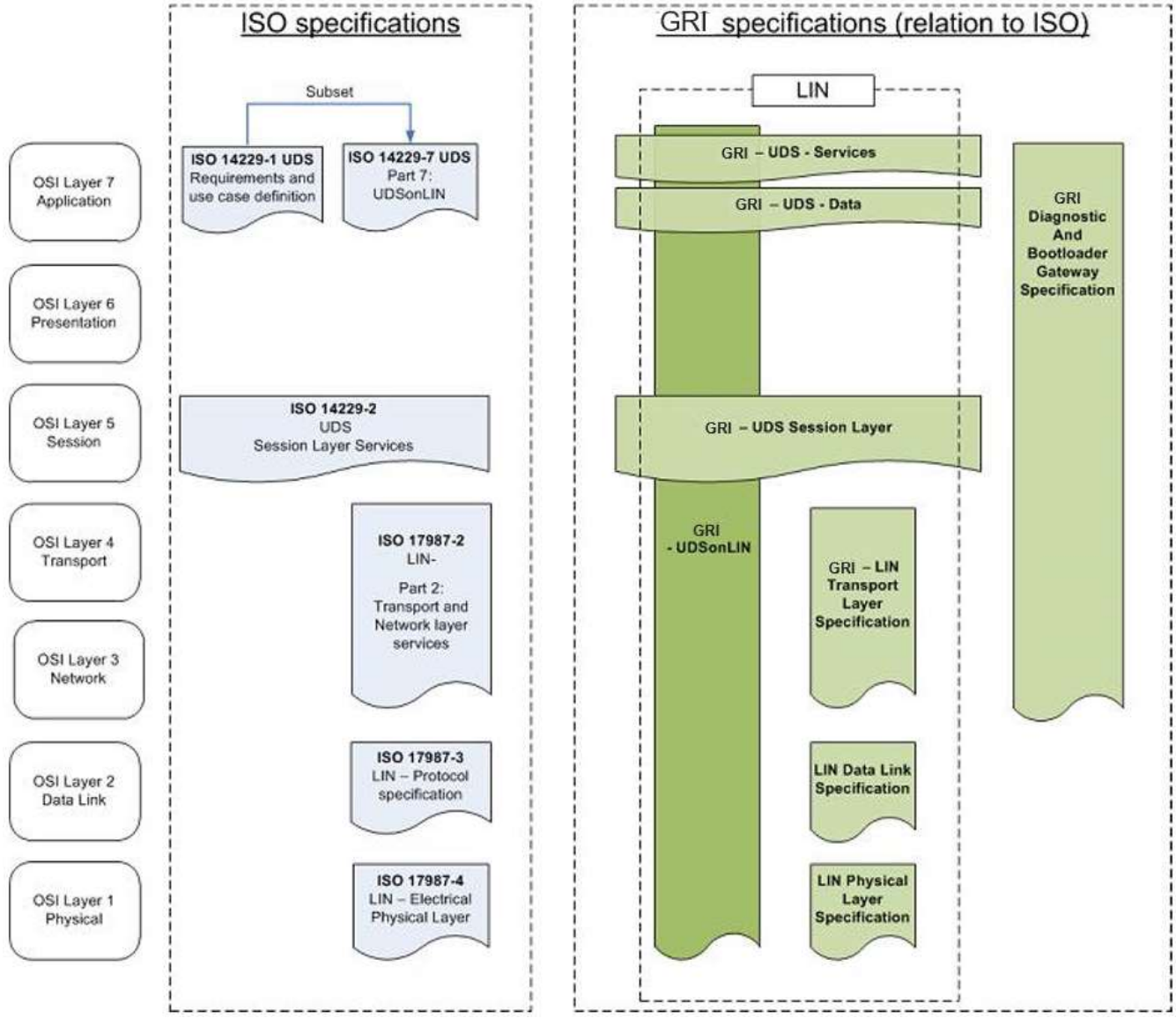


Figure - Document structure

•Overview

4.11.9.1.2 Overview

Depending on configuration, this document may consist in different combinations of the following sections:

· Generic. Applicable for all programmable ECUs and the LIN master if there are programmable LIN slaves on LIN network.

· LIN slave. Applicable for all programmable LIN slaves.

The requirements in this document are applicable for all ECUs with LIN, regardless of higher level network (i.e. network the LIN master is connected to).

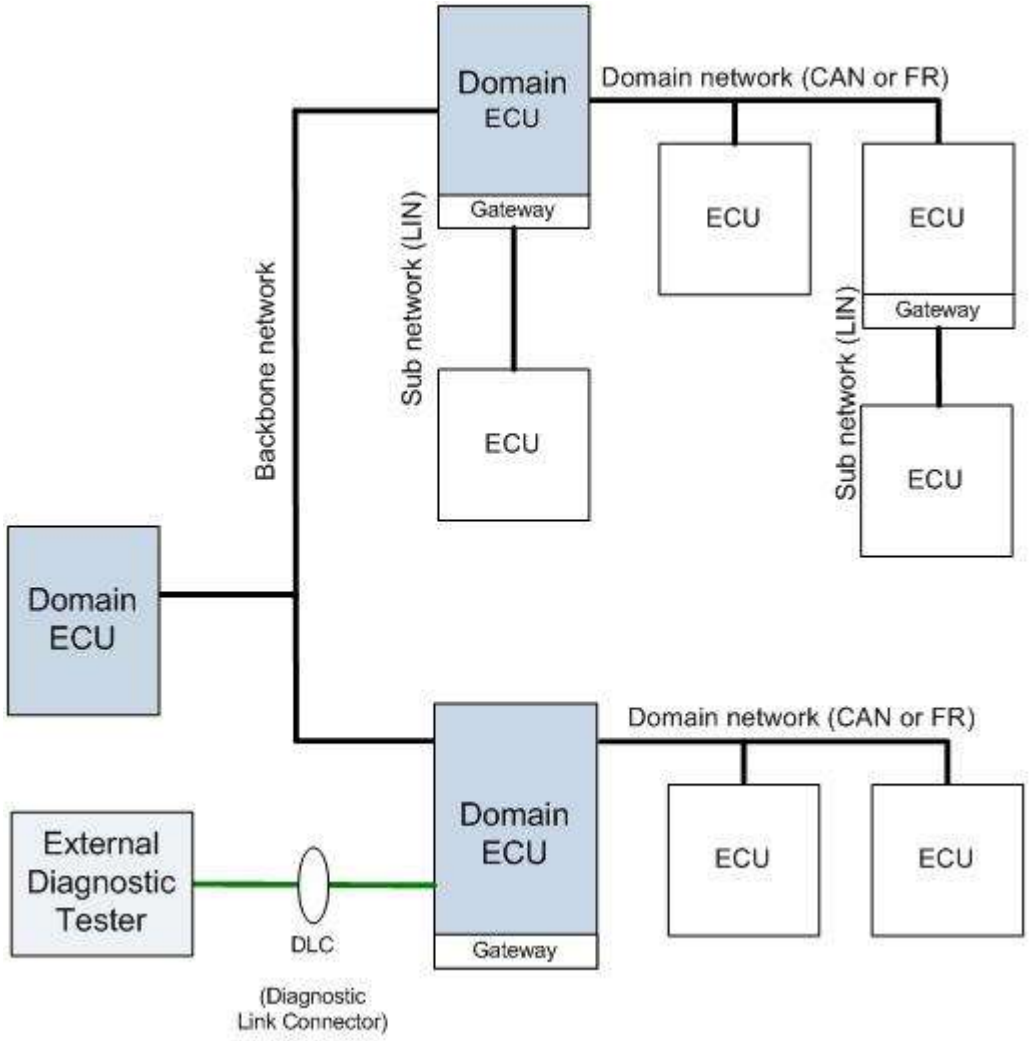


Figure - Example of the vehicle network topology

•Requisite documents

4.11.9.1.3 Requisite documents

Reference	Source	Document
[BL_LIN_1]	Internal	General Software Download
[BL_LIN_2]	Internal	UDS Services
[BL_LIN_3]	Internal	UDS Data
[BL_LIN_4]	Internal	UDS Session
[BL_LIN_5]	Internal	LIN Transport Layer
[BL_LIN_6]	Internal	LIN Data Link Layer
[BL_LIN_7]	Internal, Base Technology DPR	LIN Physical Layer
[BL_LIN_8]	ISO	ISO 14229-7 Road vehicles -- Unified diagnostic services (UDS) -- Part 7: Unified diagnostic services on Local Interconnect Network implementation (UDSonLIN). (2015-02-01)

Note:
Internal references are typically found as a seperate chapter in the ECU specific Base Technology SWRS.

•Definitions and Abbreviations

4.11.9.1.4 Definitions and Abbreviations

Definition	Description
Cold Start	The start-up of the ECU from a powered-down or off state.
ECU ID	Part of the ECU Address, unique identifier for each ECU on a network.
Diagnostics Only Mode	See reference [BL_LIN_8].
Frame identifier	See reference [BL_LIN_8].
Implementer	Person for the diagnostic implementation of the ECU. It may be the vehicle manufacturer or the ECU supplier diagnostic software designer.
LIN master	See reference [BL_LIN_8].
LIN slave	See reference [BL_LIN_8].
N_As	Network layer timing parameter As
Network ID	Part of the ECU Address, states which Network the ECU is physical connected to.
Program mode	Program mode means that the ECU executing the primary or secondary bootloader.
Warm start	The start-up of the ECU initiated by the application.
Schedule table	See reference [BL_LIN_8].
Tester	A system that controls functions such as test, inspection, monitoring, or diagnosis of an on-vehicle electronic control unit and may be dedicated to a specific type of operators (e.g. a scan tool dedicated to garage mechanics or a test tool dedicated to assembly plant agents). The tester is also referenced as the client.

Abbr.	Stands for
ECU	Electronic Control Unit
Fr/FR	FlexRay
LIN	Local Interconnect Network
NAD	Node Address
PBL	Primary Bootloader
SA	Source Address
SBL	Secondary Bootloader
SPRMIB	Suppress Positive Response Message Indication Bit
TA	Target Address
UDS	Unified Diagnostic Services

4.11.9.2 Bootloader - UDSonLIN - LIN Slave

•Unified diagnostic services implementation requirements

4.11.9.2.1 Unified diagnostic services implementation requirements

4.11.9.2.1.1 Diagnostic Services

663013 v1 **LIN slave supported NAD types**

Version history:
(1):

Purpose: The LIN slaves needs to support both physically and functional addressed NAD addresses.
Source:
Verification Method: Test
Legacy ID: 82641v0

Each reprogrammable LIN slave shall support at least two (2) USDT diagnostic NADs: functionally addressed requests and physically addressed requests and responses.

4.11.9.2.1.2 Diagnostic Data

663014 v1 **RequestDownload (34) maxNumberOfBlockLength**

Version history:
(1):

Purpose: It is preferable to have the RequestDownload parameter maxNumberOfBlockLength as high as possible to minimize download time (decreases the number of diagnostic responses the LIN slave will send). However due to the limited bandwidth on LIN maxNumberOfBlockLength shall not be limited to make other networks timeout the diagnostic LIN request.
Source:
Verification Method: Test
Legacy ID: 82485v1

The maxNumberOfBlockLength shall be at least 20% of the total physical RAM present on processor but not bigger than 2Kb.

•Session Layer requirements

4.11.9.2.2 Session Layer requirements

4.11.9.2.2.1 Queued requests

663015 v1 **No support of queued request in programming session**

Version history:
(1):

Purpose: Since the LIN protocol dosen't support the usage of queued request feature.
Source:
Verification Method: Analysis AND Test
Legacy ID: 372728v1

The LIN slave shall in programming session not implement support for queued request, according to *[BL_LIN_4] UDS Session,* requirement "*Server support of queued requests in programming session*".

Note: *The LIN network/transport layer standard is not allowing this feature.*

4.11.9.2.2.2 P2Server

702826 v1 **P2Server_max - ProgrammingSession**

Version history:
(1): Increase robustness to send diagnostic response during P2server_max. It will not affect the upgrade on the entire vehicle.

Purpose: P2Server_max is the maximum time for the server to start with the response message after the reception of a request message. Considering that the LIN scheduling cycle is 15ms, P2server should be extended to improve robustness and ensure server-side give the diagnostic response.
Source:
Verification Method: Test
Legacy ID:

The maximum time for $P2_{Server}$ in programmingSession is 50 ms.

Note: This requirement should replace general requirements.

•ECU program mode

4.11.9.2.3 ECU program mode

Program mode means that the ECU is executing the primary or secondary bootloader. This chapter describes state and flow chart diagrams unique to the LIN bootloader. Additionally the LIN bootloader shall also implement the non-network dependent bootloader requirements described in *[BL_LIN_1] General Software Download*.

Note that it is not a requirement to use the SBL SWDL strategy for LIN ECU's, since LIN ECUs often have less RAM and flash compared to other network ECUs.

663016 v1 **General bootloader requirements**

Version history:
(1):

Purpose: Non-network dependent bootloader requirements are defined in a separate document which is applicable for all bootloaders.
Source:
Verification Method: Analysis
Legacy ID: 77542v1

The LIN bootloader shall implement the non network dependent requirement defined in *[BL_LIN_1] General Software Download*.

4.11.9.2.3.1 Bootloader state diagram

The bootloader state diagram, the figure below is a complement to the flow chart diagram described in section
Bootloader flow chart diagram - Overview.

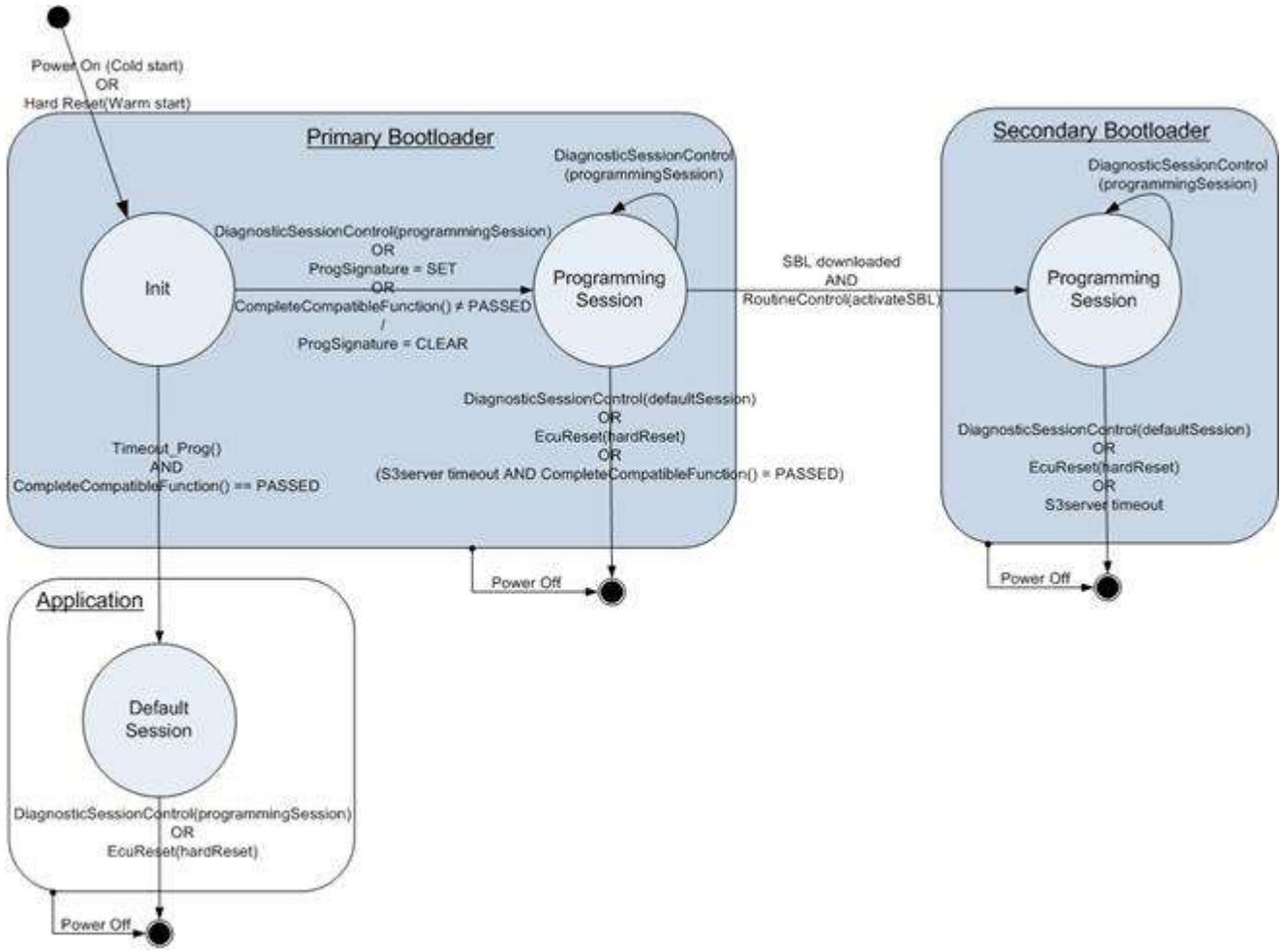


Figure – Bootloader state diagram

4.11.9.2.3.2 Bootloader flow chart diagram - Overview

The LIN bootloader flow chart diagram applies to all LIN based bootloader ECUs. The flow chart diagram is divided in three (3) parts. Each part is further described in the following chapters *Bootloader flow chart diagram (Init) , Bootloader flow chart diagram (ProgSession)* and *Flow chart diagram (Application) .* Some figures contain yellow requirements tags. These requirement tags show where each requirement is applicable in the flow chart.

To avoid problems with hardware initialization (i.e., write once registers, PLL-setup, memory re-map, interrupt vector table re-map, etc.), the ECU shall make a complete reset when exiting the application.

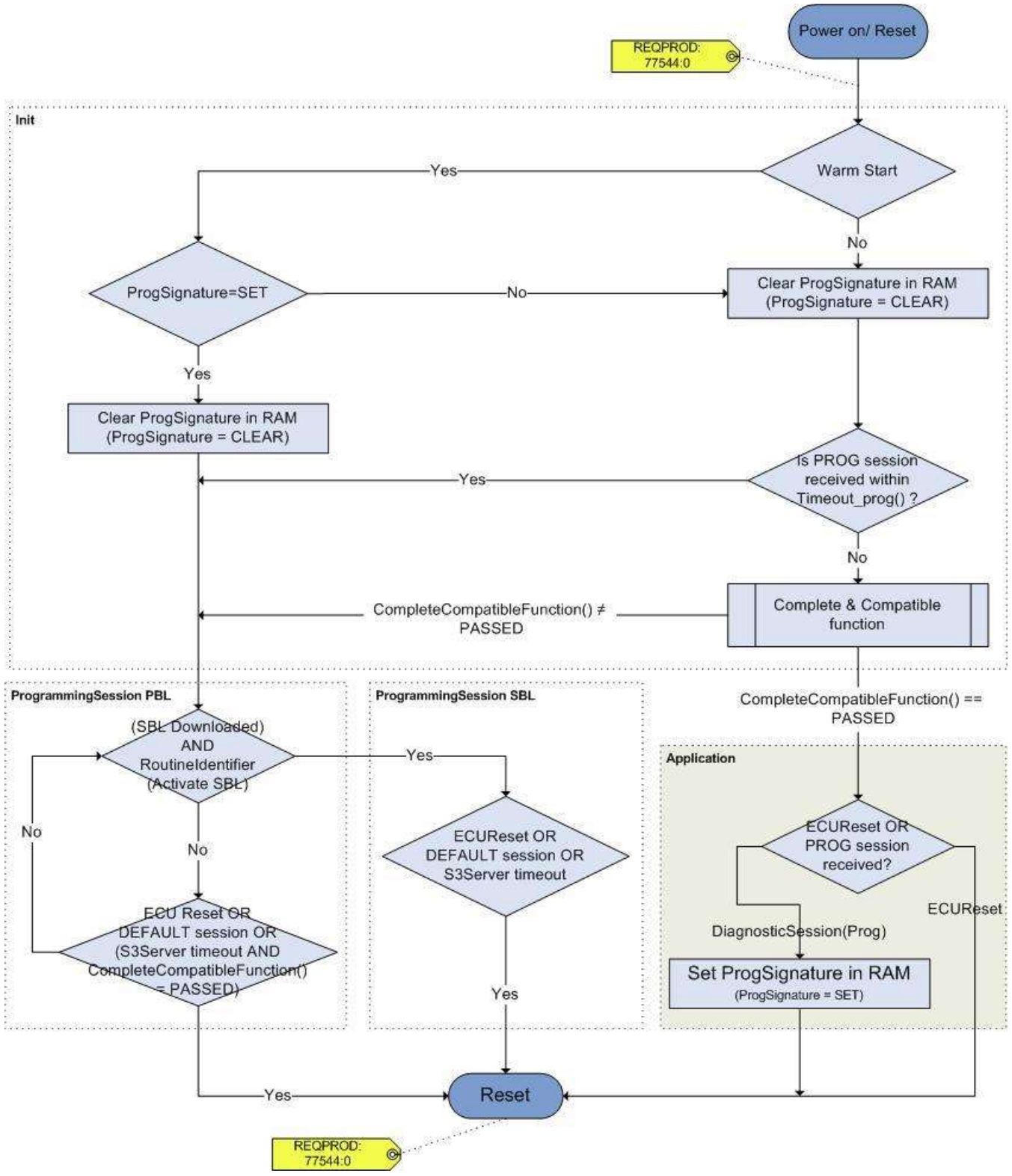


Figure - Bootloader flow chart diagram (Overview)

Version history:
(1):

Purpose: To define the bootloader flow chart. The flow chart diagram is applicable for all ECUs implementing LIN.
Source: Bootloader flow chart diagram - Overview
Verification Method: Analysis AND Test
Legacy ID: 77543v0

LIN bootloader implementation shall follow flow chart in accordance to the figure *Bootloader flow chart diagram (Overview)* . The figure shows the order in which different sessions are activated.

663018 v1 Complete hardware initiation when starting the ECU

Version history:
(1):

Purpose: To avoid problems with hardware initialisation (i.e., write once registers, PLL-setup, memory re-map, interrupt vector table re-map, etc.)
Source: Bootloader flow chart diagram - Overview
Verification Method: Analysis
Legacy ID: 77544v0

The ECU shall always start from reset, regardless if the reset was initiated by a diagnostic request or by power up.

4.11.9.2.3.3 Bootloader flow chart diagram (Init)

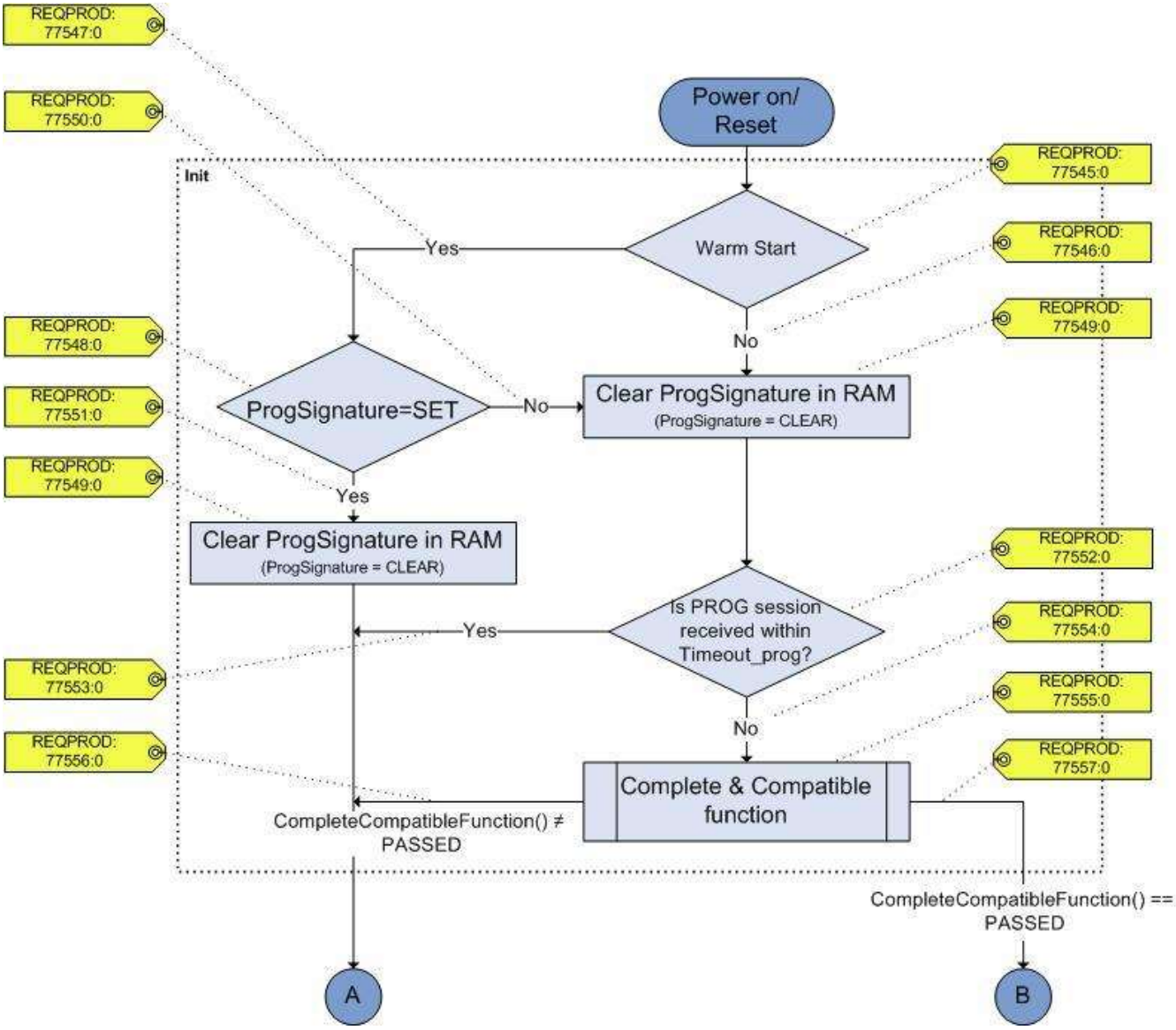


Figure - Bootloader flow chart diagram (Init)

663019 v1	Check ProgSignature in RAM

Version history:
(1):

Purpose: To define the "Check ProgSignature in RAM" state that is used to communicate status between the application and the bootloader.
Source: Bootloader flow chart diagram (Init)
Verification Method: Analysis
Legacy ID: 77548v1

The ECU shall after a warm start check whether the ProgSignature at fixed RAM address is SET or not.
Note:

The ProgSignature in RAM shall be written by the application when a DiagnosticSessionControl (programmingSession) received, the format and length of the ProgSignature is described in *[BL_LIN_1] General Software Download*.

663020 v1 **Clear ProgSignature in RAM**

Version history:
(1):

Purpose: To define the "Clear ProgSignature in RAM" function that provides protection against unwanted bootloader activation.
Source: Bootloader flow chart diagram (Init)
Verification Method: Analysis
Legacy ID: 77549v2

The ECU shall CLEAR the ProgSignature variable.

Note:
The format and length of the ProgSignature is described in *[BL_LIN_1] General Software Download*.

663021 v1 **Cold or warm start detected**

Version history:
(1):

Purpose: The warm start-up state checks the cause of start-up to avoid having the bootloader unintentionally entering programming mode during cold start
Source: Bootloader flow chart diagram (Init)
Verification Method: Analysis
Legacy ID: 77545v0

To get a complete hardware initiation when starting the ECU, the ECU shall be able to detect the type of start (cold or warm) and use that information to decide the next transition.
Note:
This information can for example be found in a transceiver, in the processor or in a voltage regulator. Where it is found is dependent on the hardware.

663022 v1 **Cold start detected**

Version history:
(1):

Purpose: There is a difference in actions between warm and cold start-up. This requirement defines the next transition when cold start is detected
Source: Bootloader flow chart diagram (Init)
Verification Method: Analysis
Legacy ID: 77546v0

When cold start is detected next step is to clear the ProgSignature.

663023 v1 Complete & Compatible function

Version history:
(1):

Purpose: The ECU shall be able to detect if the current downloaded software is compatible or not.
Source: Bootloader flow chart diagram (Init)
Verification Method: Analysis
Legacy ID: 77555v1

The ECU shall detect if the software is compatible or not, by using Complete & Compatible function specified in *[BL_LIN_1] General Software Download*.

663024 v1 Complete and compatible software(s)

Version history:
(1):

Purpose: To define the action when the software(s) is/are complete and compatible.
Source: Bootloader flow chart diagram (Init)
Verification Method: Analysis
Legacy ID: 77557v0

If the return value from the Complete & Compatible function is equal to PASSED, it is safe to start the application.

663025 v1 No PROG received within Timeout_Prog

Version history:
(1):

Purpose: To define next transition when no DiagnosticSessionControl (ProgrammingSession) request is received within Timeout_Prog.
Source: Bootloader flow chart diagram (Init)
Verification Method: Analysis
Legacy ID: 77554v0

If no DiagnosticSessionControl (ProgrammingSession) request is received within Timeout_Prog shall the ECU enter the Complete & Compatible function.

663026 v2 Non complete or compatible software(s)

Version history:
(2): Verification Method: Analysis change to Analysis AND Test.

Purpose: To define the action when software(s) is/are neither complete nor compatible.
Source: Bootloader flow chart diagram (Init)
Verification Method: Analysis AND Test
Legacy ID: 77556v0

If the return value from the Complete & Compatible function is not equal to PASSED it means

the ECU is either not complete or compatible, and therefore the ECU shall go to “programmingSession” state.

663027 v1 **PROG received within Timeout_Prog**

Version history:
(1):

Purpose: To be able to enter the programmingSession state when the DiagnosticSessionControl (ProgrammingSession) request is received within Timeout_Prog.
Source: Bootloader flow chart diagram (Init)
Verification Method: Analysis
Legacy ID: 77553v1

When the ECU receives the diagnostic request DiagnosticSessionControl (ProgrammingSession) within Timeout_Prog the ECU shall enter the “programmingSession” state.

663028 v1 **ProgSignature is not set**

Version history:
(1):

Purpose: To define next transition when the ProgSignature not is set
Source: Bootloader flow chart diagram (Init)
Verification Method: Analysis
Legacy ID: 77550v0

When the ProgSignature is not SET the ECU shall enter the “Clear ProgSignature in RAM” state.

663029 v1 **ProgSignature is set**

Version history:
(1):

Purpose: The purpose of setting the ProgSignature to a predefined value is to enter the programmingSession state
Source: Bootloader flow chart diagram (Init)
Verification Method: Analysis
Legacy ID: 77551v0

When the ProgSignature is SET the next step is to enter the “Clear ProgSignature in RAM” state.

663030 v1 **Timeout_Prog**

Version history:
(1):

Purpose: Before the ECU starts the application, the ECU shall listen for the diagnostic

request DiagnosticSessionControl (ProgrammingSession) during Timeout_Prog. This time window may be used as a "backdoor" if the ECU is for some reason not able to communicate when the application is started. Due to LIN being a slow network the Timeout_Prog timeout has to be extended compared to other networks.

Source: Bootloader flow chart diagram (Init)
Verification Method: Analysis
Legacy ID: 77552v1

The ECU shall during Timeout_Prog = 10 ms listen for DiagnosticSessionControl (ProgrammingSession) requests. If a LIN break field is detected during this 10ms time, the LIN ECU shall wait until the complete LIN frame response has been received and handled or another 10ms have passed.

663031 v1 Warm start detected

Version history:
(1):

Purpose: There is a difference in actions between warm and cold start-up. This requirement defines the next transition when warm start is detected.
Source: Bootloader flow chart diagram (Init)
Verification Method: Analysis
Legacy ID: 77547v0

When warm start is detected next step is to check if the ProgSignature is SET.

4.11.9.2.3.4 Bootloader flow chart diagram (ProgSession)

ProgrammingSession state is entered when ProgSignature is set or when the software's is not Complete & Compatible or a ProgrammingSession was received before Timeout_Prog time ran out.

When the ECU has entered the programmingSession state the ECU shall be able to receive process and respond to diagnostic requests supported by the bootloader.

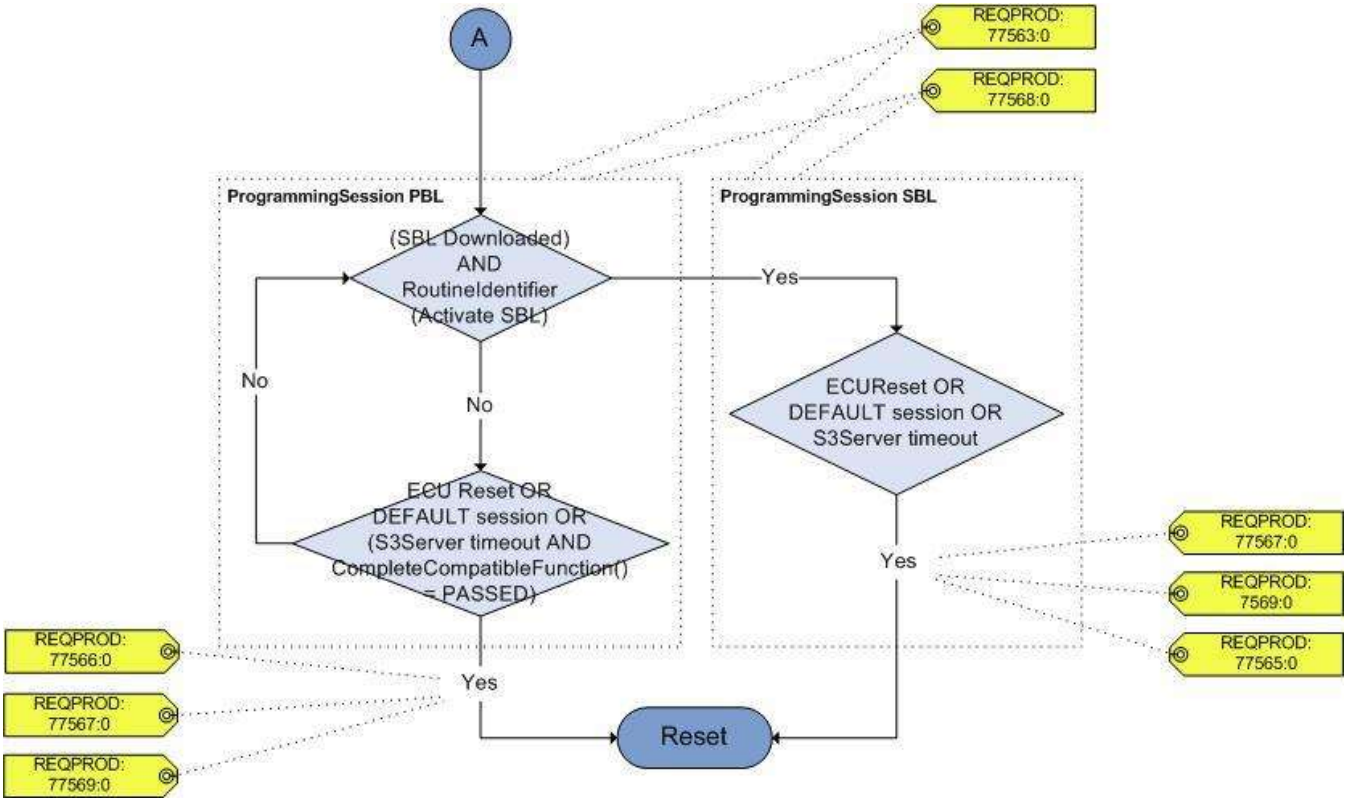


Figure - Bootloader flow chart diagram (ProgSession)

663032 v1 **Receive and process diagnostic programming session requests**

Version history:
(1):

Purpose: To define when the ECU shall be able to receive and process a diagnostic programming session request.
Source: Bootloader flow chart diagram (ProgSession)
Verification Method: Test
Legacy ID: 77563v0

When the ECU is in programmingSession state, the ECU shall be able to receive and process diagnostic programming session requests.

663033 v1 **Reception of DiagnosticSessionControl (programmingSession)**

Version history:
(1):

Purpose: To prevent a reset when the bootloader is in the programmingSession state and receive a diagnostic request DiagnosticSessionControl(programmingSession)
Source: Bootloader flow chart diagram (ProgSession)
Verification Method: Test
Legacy ID: 77568v0

When the bootloader is in the programmingSession state (PBL or SBL) and receives a

DiagnosticSessionControl (programmingSession) request, the ECU shall not reset i.e. the ECU shall stay in the programmingSession state.

663034 v1 **Reception of DiagnosticSessionControl(defaultSession)**

Version history:
(1):

Purpose: To define the behaviour when the ECU receives the DiagnosticSessionControl (defaultSession) request in the programmingSession state.
Source: Bootloader flow chart diagram (ProgSession)
Verification Method: Test
Legacy ID: 77567v0

When the ECU receives a DiagnosticSessionControl (defaultSession) request, the ECU shall make a reset, thus restarting the ECU from Init state.

663035 v1 **Reception of the ECUReset service**

Version history:
(1):

Purpose: To define the behaviour when the ECU receives the ECUReset request in the programmingSession state.
Source: Bootloader flow chart diagram (ProgSession)
Verification Method: Test
Legacy ID: 77569v0

When the ECU receives an ECUReset request, the ECU shall make a reset, thus restarting the ECU from Init state.

663036 v1 **S3Server timer timeout in PBL**

Version history:
(1):

Purpose: To define the behaviour when S3server times out in PBL.
Source: Bootloader flow chart diagram (ProgSession)
Verification Method: Test
Legacy ID: 77566v1

If $S3_{server}$ times out when running PBL, the ECU shall make a test on Complete and Compatible function and if Complete and Compatible function returns PASSED the ECU shall make a reset.
Note:
The $S3_{server}$ is defined in [BL_LIN_4] UDS Session.

663037 v1 **S3Server timer timeout in SBL**

Version history:
(1):

Purpose: To define the behaviour when the S3server times out in SBL
Source: Bootloader flow chart diagram (ProgSession)
Verification Method: Test
Legacy ID: 77565v1

When the S3 $_{server}$ times out when running SBL, the ECU shall make a reset.

Note: *The S3 $_{server}$ is defined in [BL_LIN_4] UDS Session*.

4.11.9.2.3.5 Bootloader flow chart diagram (Application)

This is the normal operation mode of the ECU. To make the ECU enter the programmingSession, a tester needs to send a DiagnosticSessionControl (programmingSession) request to the ECU. When all criteria for entering the bootloader have been met, it shall be enough sending one DiagnosticSessionControl (programmingSession) request to make the ECU set the ProgSignature and execute the reset.

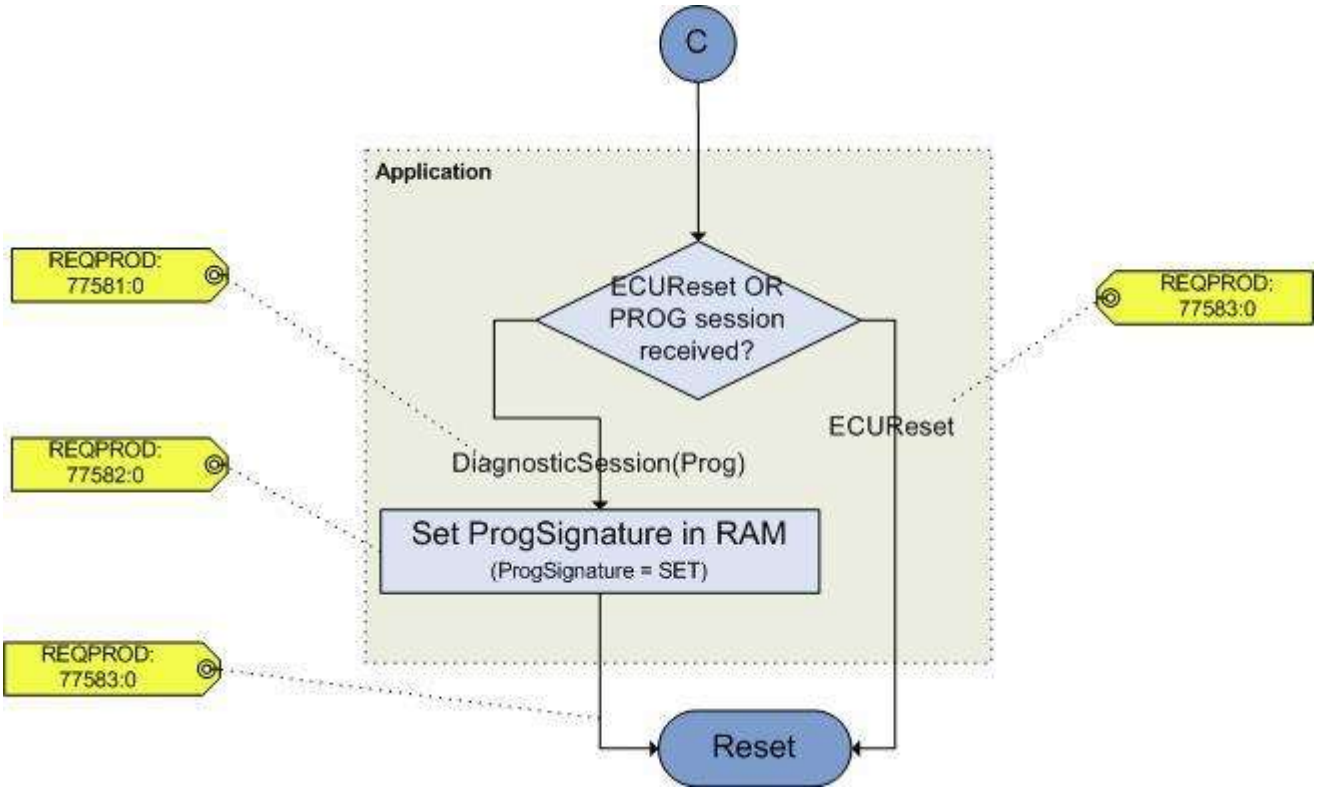


Figure - Flow chart diagram (Start Application)

663038 v1	Enter bootloader from ECU application

Version history:
(1):

Purpose: To define how to set the ECU to program mode when the ECU application is active. Criteria's can be defined, outside of bootloader scoop, which can prevent or delay the ECU to go to the bootloader. Such criteria can be vehicle

speed.
Source: Flow chart diagram (Application)
Verification Method: Test
Legacy ID: 77581v0

When all criteria for entering the bootloader have been met, it must be possible to enter the bootloader from the application by receiving the DiagnosticSessionControl (programmingSession) service only once.

663039 v1 **Trig a reset in ECU application mode**

Version history:
(1):

Purpose: To enter the bootloader from the application
Source: Flow chart diagram (Application)
Verification Method: Test
Legacy ID: 77583v0

The ECU application shall trig a reset on reception of a DiagnosticSessionControl (programmingSession) or an ECUReset (hardReset).

663040 v2 **Write ProgSignature**

Version history:
(2): Verification Method: Analysis AND Test change to Analysis.

Purpose: The ProgSignature is used after reset to determine whether the ECU shall stay in the bootloader or start the application.
Source: Flow chart diagram (Application)
Verification Method: Analysis
Legacy ID: 77582v1

When the ECU application receives a DiagnosticSessionControl (programmingSession) request, the ProgSignature shall be written to a fixed address in RAM.

Note: The format and length of the ProgSignature is described in *[BL_LIN_1] General Software Download*.

4.11.10 Bootloader - UDSonIP

4.11.10.1 BL - UDSonIP - Generic reqs

•Introduction

4.11.10.1.1 Introduction

This document specifies bootloader requirements for an ECU connected to an IP based network. It intends to provide guidance and requirements to architects, system designers, application developers, and suppliers who are implementing the bootloader to an ECU connected to an IP based network.
Important to understand is that this document only specifies the IP specific requirements and

not the complete bootloader. Figure: *Document structure overview* , shows the specifications that are related to the implementation specification for an ECU connected to IP based network . The main aim with software download is to fulfill the needs of replace software's at repair shop and the assembly plant.

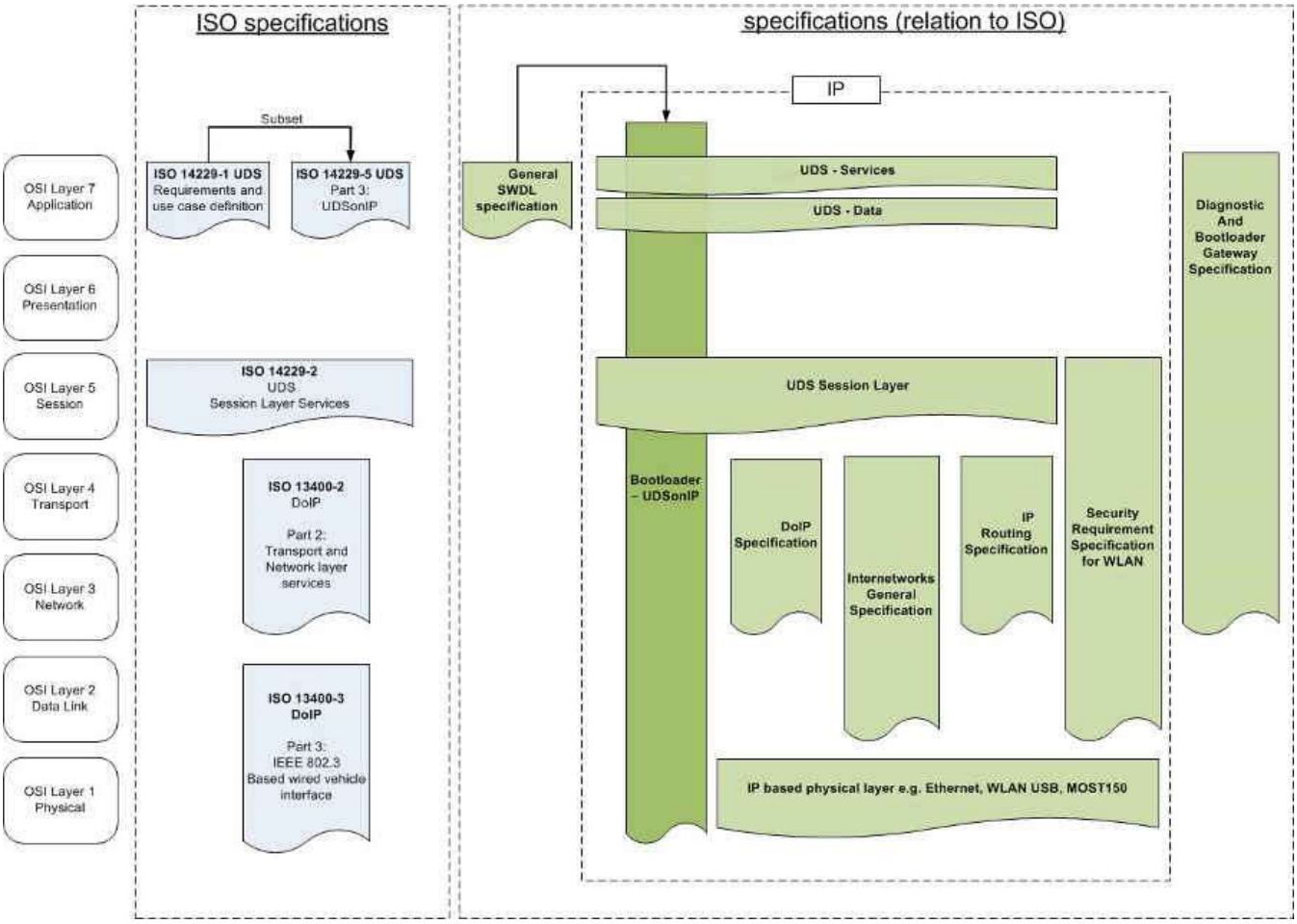


Figure Document structure overview

•Overview

4.11.10.1.2 Overview

This document defines implementation specific requirements that shall apply to an IP ECU and the Edge Node. All diagnostic messages on IP shall be sent and received according to this and referenced documents. The requirements in this document cover the startup sequences, adjustments of diagnostic services, data, timers etc. There are two types of ECUs described in this document. The Edge Node is the interface towards an external tester. The bootloader of the Edge Node is responsible of sending announcement messages in accordance to *DoIP Communication*, as well as respond to Vehicle Identification Request Message and other mandatory DoIP messages, See Requisite documents., sent from the external diagnostic tester. The ECU connected to the External Diagnostic Tester, Figure: Example of a vehicle network topology, contains the Edge Node functionality. The IP ECU is an in-vehicle ECU communicating on a TCP/IP based in-vehicle network.

Additional requirements are added for the bootloader if the ECU shall act as a gateway or a router to a domain network which is defined in the *Diagnostic And Bootloader Gateway*

Specification and *IP Routing and firewalling Specification.*

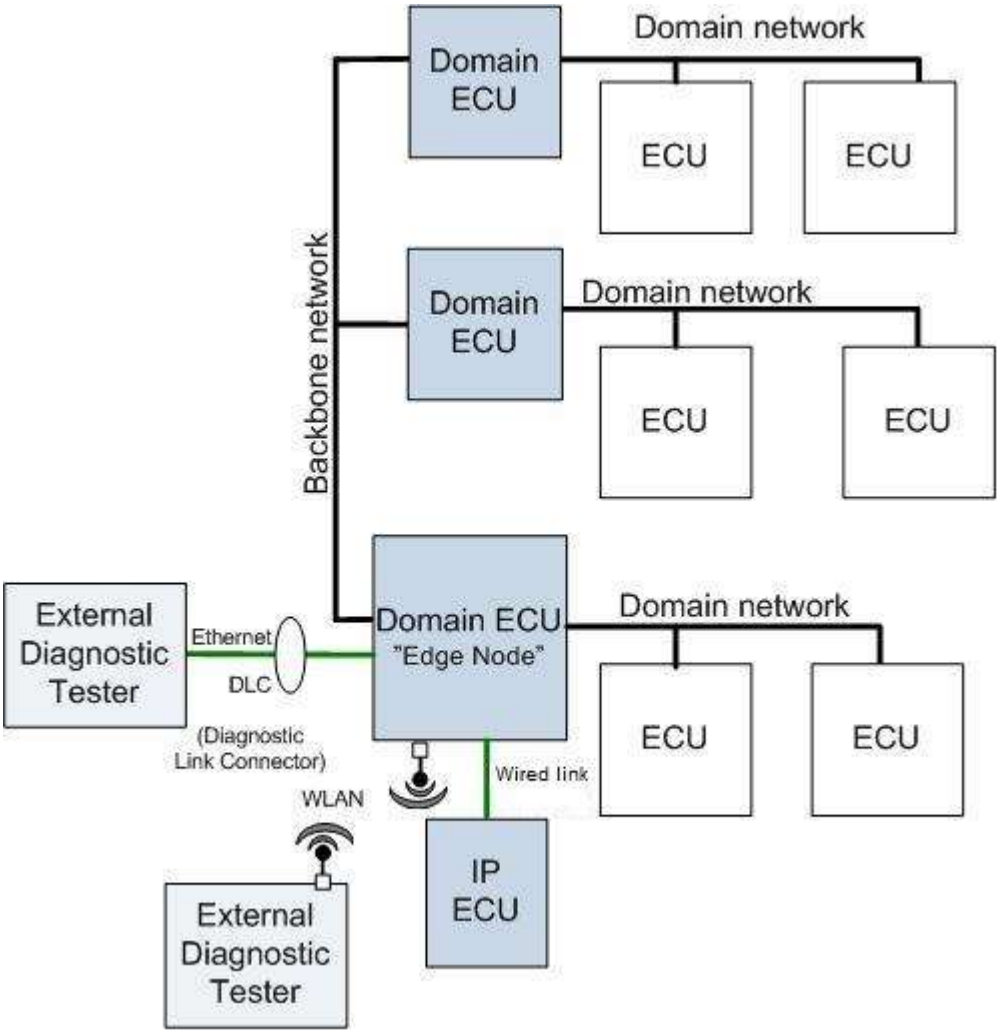


Figure Example of a vehicle network topology

•Requiste documents

4.11.10.1.3 Requiste documents

Reference	Source	Document
[BL_IP_1]	Internal	General Software Download
[BL_IP_2]	Internal	UDS Services
[BL_IP_3]	Internal	UDS Data
[BL_IP_4]	Internal	UDS Session
[BL_IP_5]	Internal	DoIP Communication
[BL_IP_6]	Internal	Internetworks General Specification
[BL_IP_7]	Internal	Diagnostic And Bootloader Gateway
[BL_IP_8]	Internal	WLAN Physical and Datalink Layer
[BL_IP_9]	Internal	WLAN Security
[BL_IP_10]	ISO	Road vehicles — Unified diagnostic services (UDS) — Part 5: Unified diagnostic services on Internet Protocol implementation

		(UDSonIP). (2012-03-28)
[BL_IP_10]	ISO	Road vehicles — Diagnostic communication over Internet Protocol (DoIP) — Part 3: IEEE802.3 based wired. (2012-01-11)

•Definitions and Abbreviations

4.11.10.1.4 Definitions and Abbreviations

Definition	Description
Cold Start	The start-up of the ECU from a powered-down or off state.
DoIP Session	In this document a DoIP session is when a TCP connection is established between the Edge Node and the Tester on port 13400.
Edge Node	An ECU (internal vehicle network node) which is the network node that is exposed towards the external network. The node implements a diagnostic over IP-protocol (DoIP - ISO13400) and provides diagnostic access to test equipment.
Gateway	An ECU that is connected to two or more independent communication networks that allows information to flow between the networks.
IP Bootloader	A bootloader supporting SWDL on IP based network.
IP ECU	This is an ECU communicating on a TCP/IP based in-vehicle network without interface to external tester.
PseudoProgramming Session	It is a state in the application only supporting the programming session requirements which are defined for the bootloader.
Router	An ECU that is connected to two or more independent IP based communication networks that allows information to flow between the networks.
Tester	A system that controls functions such as test, inspection, monitoring, or diagnosis of an on-vehicle electronic control unit and may be dedicated to a specific type of operators (e.g. a scan tool dedicated to garage mechanics or a test tool dedicated to assembly plant agents). The tester is also referenced as the client.
Warm start	The start-up of the ECU initiated by the application.

Abbreviation	Description
DoIP	Diagnostic communication over IP
DLC	Diagnostic Link Control
ECU	Electric Control Unit
IP	Internet Protocol
TCP	Transmission Control Protocol
SWDL	Software Download
WLAN	Wireless Local Area Network

•Introduction

4.11.10.1.5 Introduction

This document specifies bootloader requirements for an ECU connected to an IP based network. It intends to provide guidance and requirements to architects, system designers, application developers, and suppliers who are implementing the bootloader to an ECU connected to an IP based network.

Important to understand is that this document only specifies the IP specific requirements and not the complete bootloader. Figure: *Document structure overview* , shows the specifications

that are related to the implementation specification for an ECU connected to IP based network . The main aim with software download is to fulfill the needs of replace software's at repair shop and the assembly plant.

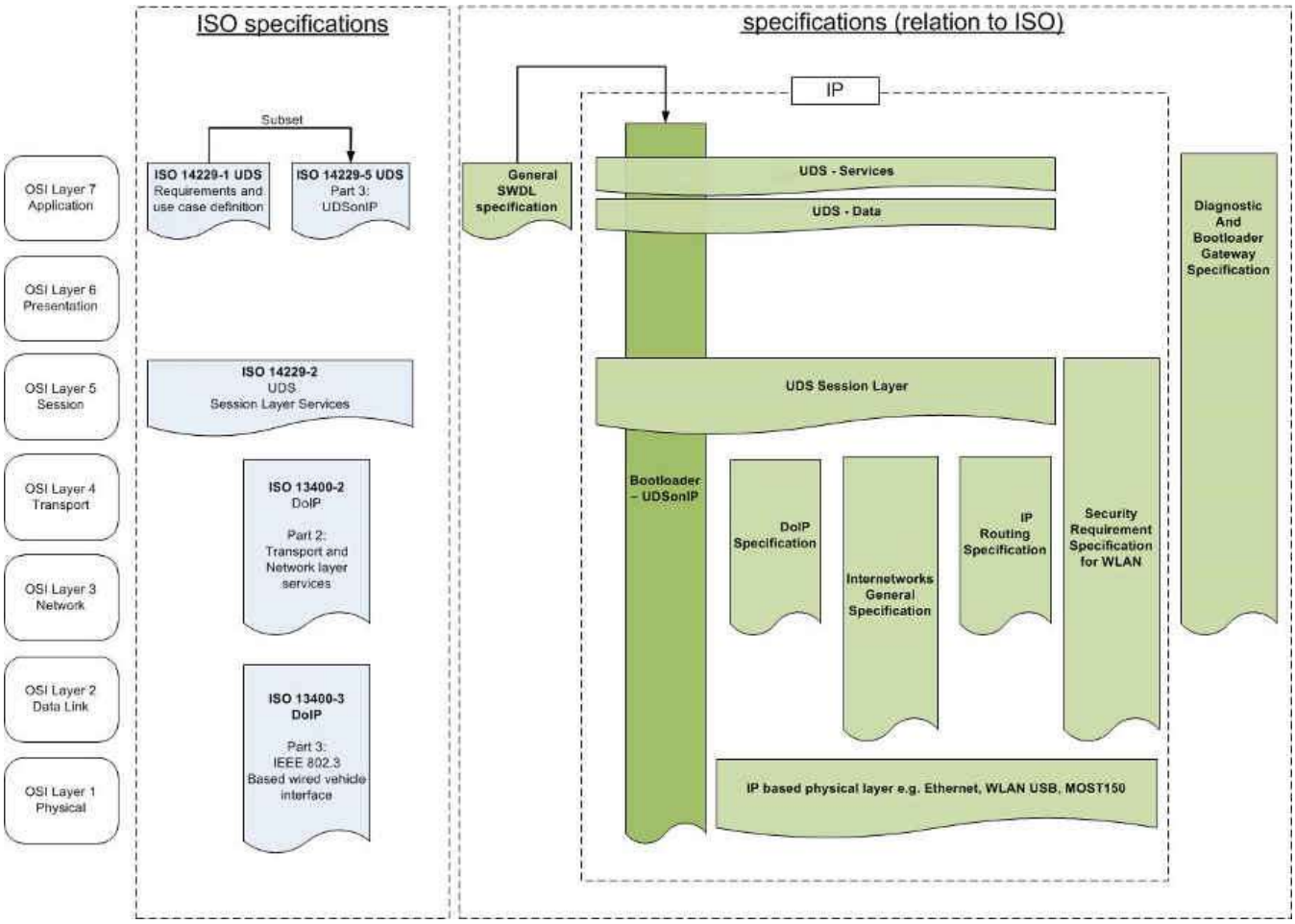


Figure Document structure overview

This document defines implementation specific requirements that shall apply to an IP ECU and the Edge Node. All diagnostic messages on IP shall be sent and received according to this and referenced documents. The requirements in this document cover the startup sequences, adjustments of diagnostic services, data, timers etc. There are two types of ECUs described in this document. The Edge Node is the interface towards an external tester. The bootloader of the Edge Node is responsible of sending announcement messages in accordance to *DoIP Communication*, as well as respond to Vehicle Identification Request Message and other mandatory DoIP messages, See Requisite documents., sent from the external diagnostic tester. The ECU connected to the External Diagnostic Tester, Figure: Example of a vehicle network topology, contains the Edge Node functionality. The IP ECU is an in-vehicle ECU communicating on a TCP/IP based in-vehicle network.
Additional requirements are added for the bootloader if the ECU shall act as a gateway or a router to a domain network which is defined in the *Diagnostic And Bootloader Gateway Specification* and *IP Routing and firewalling Specification.*

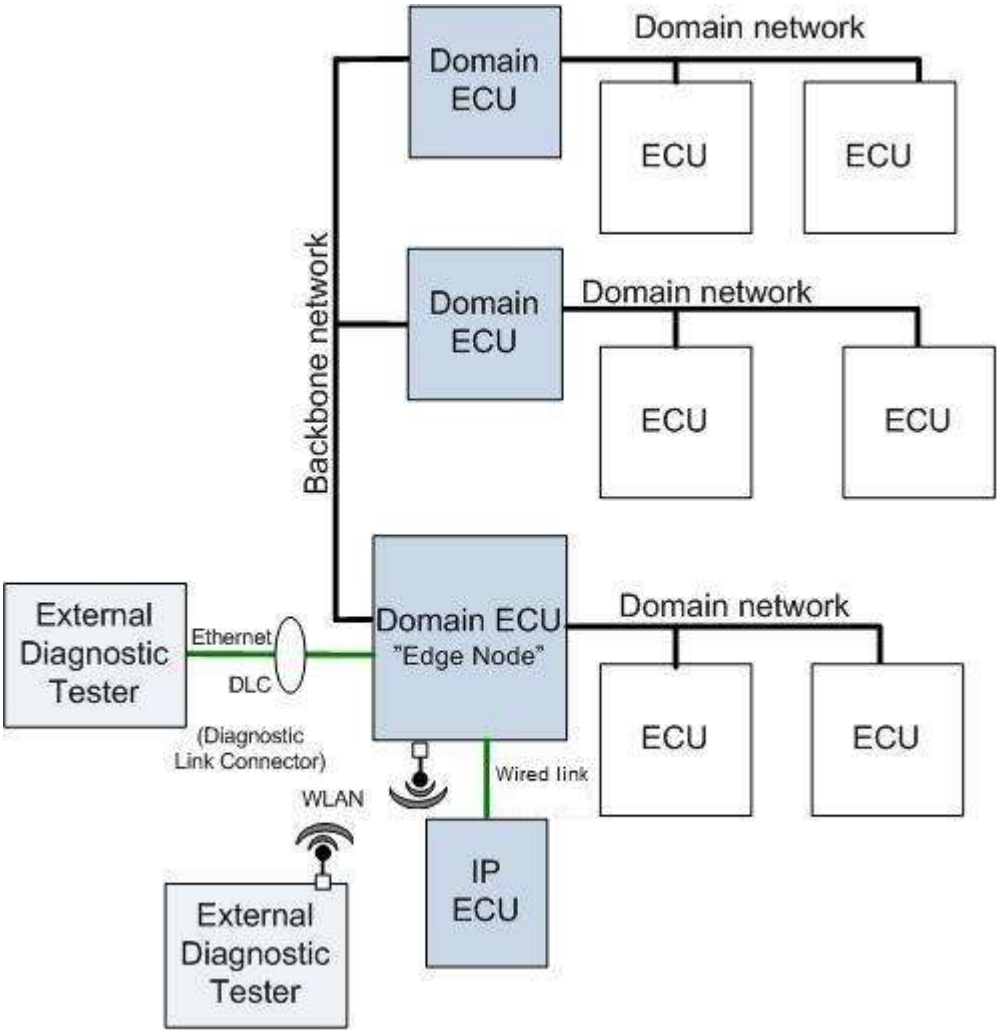


Figure Example of a vehicle network topology

4.11.10.1.5.1 Definitions and Abbreviations

Definition	Description
Cold Start	The start-up of the ECU from a powered-down or off state.
DoIP Session	In this document a DoIP session is when a TCP connection is established between the Edge Node and the Tester on port 13400.
Edge Node	An ECU (internal vehicle network node) which is the network node that is exposed towards the external network. The node implements a diagnostic over IP-protocol (DoIP - ISO13400) and provides diagnostic access to test equipment.
Gateway	An ECU that is connected to two or more independent communication networks that allows information to flow between the networks.
IP Bootloader	A bootloader supporting SWDL on IP based network.
IP ECU	This is an ECU communicating on a TCP/IP based in-vehicle network without interface to external tester.
PseudoProgramming	It is a state in the application only supporting the programming session

Session	requirements which are defined for the bootloader.
Router	An ECU that is connected to two or more independent IP based communication networks that allows information to flow between the networks.
Tester	A system that controls functions such as test, inspection, monitoring, or diagnosis of an on-vehicle electronic control unit and may be dedicated to a specific type of operators (e.g. a scan tool dedicated to garage mechanics or a test tool dedicated to assembly plant agents). The tester is also referenced as the client.
Warm start	The start-up of the ECU initiated by the application.

Abbreviation	Description
DoIP	Diagnostic communication over IP
DLC	Diagnostic Link Control
ECU	Electric Control Unit
IP	Internet Protocol
TCP	Transmission Control Protocol
SWDL	Software Download
WLAN	Wireless Local Area Network

4.11.10.1.5.2 External publications

Source	Title	Version or Date	Document number
ISO	Road vehicles — Unified diagnostic services (UDS) — Part 5: Unified diagnostic services on Internet Protocol implementation (UDSonIP)	2012-03-28	ISO/DIS 14229-5
ISO	Road vehicles — Diagnostic communication over Internet Protocol (DoIP) — Part 3: IEEE802.3 based wired	2012-01-11	ISO 13400-3

4.11.10.1.5.3 Requiste documents

Reference	Source	Document
[BL_IP_1]	Internal	General Software Download
[BL_IP_2]	Internal	UDS Services
[BL_IP_3]	Internal	UDS Data
[BL_IP_4]	Internal	UDS Session
[BL_IP_5]	Internal	DoIP Communication
[BL_IP_6]	Internal	Internetworks General Specification
[BL_IP_7]	Internal	Diagnostic And Bootloader Gateway
[BL_IP_8]	Internal	WLAN Physical and Datalink Layer
[BL_IP_9]	Internal	WLAN Security
[BL_IP_10]	ISO	Road vehicles — Unified diagnostic services (UDS) — Part 5: Unified diagnostic services on Internet Protocol implementation (UDSonIP). (2012-03-28)
[BL_IP_10]	ISO	Road vehicles — Diagnostic communication over Internet Protocol (DoIP) — Part 3: IEEE802.3 based wired. (2012-01-11)

•Requirements

4.11.10.1.6 Requirements

662932 v1 **Application layer**

Version history:
(1):

Purpose: To define the Application layer used by the bootloader
Source: Chapter Application layer; ISO 14229-5 Road vehicles - Unified diagnostic services (UDS) - Part 5: Unified diagnostic services on Internet Protocol implementation (UDSonIP)
Verification Method: Analysis
Legacy ID: 55170v1

The IP bootloader application layer shall be compliant with *ISO 14229-5 Road vehicles — Unified diagnostic services (UDS) — Part 5: Unified diagnostic services on Internet Protocol implementation (UDSonIP).*

662940 v1 **Bootloader General SWDL requirements**

Version history:
(1):

Purpose: Non-network dependent bootloader requirements are defined in a separate document which is applicable for all bootloaders.
Source: General Software Download Specification; Chapter The Edge Node - PseudoProgrammingSession; Chapter ECU PROGRAM MODE
Verification Method: Analysis
Legacy ID: 74003v1

The Edge Node bootloader, PseudoProgrammingSession state and the IP ECU bootloader shall implement the non network dependent requirement defined in *General Software Download Specification.*

662941 v1 **Check ProgSignature in RAM**

Version history:
(1):

Purpose: To define the "Check ProgSignature in RAM" state that is used to communicate status between the application and the bootloader.
Source: Chapter IP ECU flow chart - Init; Chapter The Edge Node's flow chart - Init
Verification Method: Analysis
Legacy ID: 55460v1

The ECU shall after a warm start check if the ProgSignature at fixed RAM address is SET.

Note: The ProgSignature in RAM shall be written by the application when a DiagnosticSessionControl(programmingSession) received, the format and length of the ProgSignature is described in *General Software Download Specification.*

662942 v1 Clear ProgSignature

Version history:
(1):

Purpose: To define the "Clear ProgSignature in RAM" function that provides protection against unwanted bootloader activation.
Source: Chapter IP ECU flow chart - Init; Chapter The Edge Node's flow chart - Init
Verification Method: Analysis
Legacy ID: 55467v1

The bootloader shall CLEAR the ProgSignature variable according to *Figure Edge Node flow chart - Init* or *Figure IP ECU flow chart - Init.*

Note: The format and length of the ProgSignature is described in *General Software Download Specification.*

662943 v1 Cold or warm start detected

Version history:
(1):

Purpose: The warm startup state checks the cause of startup to avoid having the bootloader unintentionally entering programming mode during cold start.
Source: Chapter The Edge Node's flow chart - Init; Chapter IP ECU flow chart - Init
Verification Method: Analysis
Legacy ID: 55456v1

The ECU shall detect the type of start (cold or warm) and use that information to decide the next transition.

Note: This information can for example be found in a transceiver, in the processor or in a voltage regulator. Where it is found is dependent on the hardware

662944 v1 Cold start detected

Version history:
(1):

Purpose: There is a difference in actions between warm and cold start-up. This requirement defines the next transition when cold start is detected
Source: Chapter IP ECU flow chart - Init; Chapter The Edge Node's flow chart - Init
Verification Method: Analysis
Legacy ID: 55457v1

When cold start is detected next step is to clear ProgSignature.

662945 v1 Complete & compatible function

Version history:
(1):

Purpose: The ECU shall be able to detect if the current downloaded software is compatible or not.
Source: General Software Download Specification; Chapter IP ECU flow chart - Init; Chapter The Edge Node's flow chart - Init
Verification Method: Analysis
Legacy ID: 55469v1

The ECU shall detect if the software is compatible or not, by using Complete & Compatible function specified in *GeneralSoftware Download Spec*ification.

662946 v1 **Complete and compatible software(s)**

Version history:
(1):

Purpose: To define the action when software is complete and compatible
Source: General Software Download Specification; Chapter IP ECU flow chart - Init; Chapter The Edge Node's flow chart - Init
Verification Method: Analysis
Legacy ID: 55472v1

If the return value from the Complete & Compatible function is equal to PASSED, it is safe to start the application.

662947 v1 **Complete hardware initiation when starting the ECU**

Version history:
(1):

Purpose: To avoid problems with hardware initialisation (i.e., write once registers, PLL-setup, memory re-map, interrupt vector table re-map, etc.)
Source: Chapter Edge Node's bootloader flow chart - Overview
Verification Method: Analysis
Legacy ID: 55453v1

The ECU shall always start from reset, regardless if the reset was initiated by a diagnostic request or by power up.

662936 v1 **Diagnostic gateway**

Version history:
(1):

Purpose: To be able to gateway diagnostic messages in the bootloader.
Source: Chapter Overview; Diagnostic And Bootloader Gateway Specification
Verification Method: Analysis
Legacy ID: 55157v1

ECUs which shall gateway diagnostic messages shall implement the applicable bootloader requirements specified in *Diagnostic And Bootloader Gateway Specification.*

662937 v1 **Diagnostic router**

Version history:
(1):

Purpose: To define how to route signals from one IP based network to another.
Source: IP routing and firewalling
Verification Method: Analysis
Legacy ID: 55159v2

The IP bootloader which shall route diagnostic messages shall support the routing requirements applicable for a bootloader defined *IP Routing and Firewalling Specification.*

662948 v1 **Enter the bootloader from ECU application mode**

Version history:
(1):

Purpose: To define how to set the ECU to program mode when the ECU application is active
Source: Chapter: The Edge Node's flow chart - Application; Chapter: IP ECU flow chart - Application
Verification Method: Test
Legacy ID: 55524v3

It must be possible to enter the bootloader from the application when the vehicle and the diagnostic client are connected over an Ethernet point-to-point link, by receiving the DiagnosticSessionControl(programmingSession) service only once and all the criteria, defined by application, have been met.

662949 v1 **ProgSignature is not set**

Version history:
(1):

Purpose: To define next transition when the ProgSignature not is SET
Source: Chapter IP ECU flow chart - Init; Chapter The Edge Node's flow chart - Init
Verification Method: Analysis
Legacy ID: 55464v1

When the ProgSignature is not SET the ECU shall enter the “Clear ProgSignature in RAM” state.

662950 v1 **ProgSignature is set**

Version history:
(1):

Purpose: The purpose of setting the ProgSignature to a predefined value is to enter the "programmingSession" state
Source: Chapter IP ECU flow chart - Init; Chapter The Edge Node's flow chart - Init

Verification Method: Analysis
Legacy ID: 55465v1

When the ProgSignature is SET the next step is to enter the “Clear ProgSignature in RAM” state

662951 v1 **Receive and process diagnostic programming session request**

Version history:
(1):

Purpose: To define when the ECU shall be able to receive and process a diagnostic programming session request
Source: Chapter The Edge Node's flow chart - ProgrammingSession and Undefined state; Chapter The Edge Node's PseudoProgrammingSession flow chart; Chapter IP ECU flow chart - ProgrammingSession
Verification Method: Test
Legacy ID: 55476v1

When the ECU is in programmingSession state or Undefined state, the ECU shall be able to receive and process diagnostic programming session requests.

662938 v1 **RequestDownload Service**

Version history:
(1):

Purpose: To decrease the total software download time
Source: UDS Services
Verification Method: Test
Legacy ID: 55165v3

Minimum allowed maxNumberOfBlockLength shall be 16386 bytes.

Note: 16kB data + 2 byte for SID and block sequence counter.

662935 v1 **Session layer**

Version history:
(1):

Purpose: To define the the Session layer used by the bootloader.
Source: UDS Session; Chapter Session layer; Chapter The Edge Node's PseudoProgrammingSession flow chart
Verification Method: Analysis
Legacy ID: 55173v2

The IP bootloader shall implement the session layer defined in *UDS Session.*

662933 v1 **TCP for internal communication**

Version history:
(1):

Purpose: To get a robust solution and future safe communication, diagnostic messages shall be sent on TCP. The implementation of IP ECUs will be less complex if only TCP is used and no apparant advantage in having both protocols concludes in only having TCP.
Source: Internetworks General Specification; Chapter IP ECUs Network/transport layer; Chapter Edge Node's Network/transport layer
Verification Method: Analysis
Legacy ID: 55177v1

TCP shall be used for interna diagnostic communication on IP based network, in accordance to *Internetworks General Specification*

662934 v1 TCP-IP layer

Version history:
(1):

Purpose: To communicate over IP the TCP, IP and UDP standards are used. This requirement defines the aspects of a TCP/IP stack that shall be fulfilled from the OEMs point of view.
Source: Internetworks General Specification; Chapter The Edge Node's PseudoProgrammingSession flow chart; Chapter Edge Node's Network/transport layer; Chapter IP ECUs Network/transport layer
Verification Method: Analysis
Legacy ID: 55176v1

The IP Bootloader shall implement the *Internetworks General Specification.*

662952 v1 The reception of DiagnosticSessionControl(defaultSession)

Version history:
(1):

Purpose: To define the behaviour when the ECU receives the DiagnosticSessionControl(defaultSession) request in the "programmingSession" state.
Source: Chapter The Edge Node's PseudoProgrammingSession flow chart; Chapter The Edge Node's flow chart - ProgrammingSession and Undefined state
Verification Method: Test
Legacy ID: 55485v1

When the ECU received a diagnostic service DiagnosticSessionControl(defaultSession) request, the ECU shall make a reset, thus restarting the ECU from Init state.

662953 v1 The reception of DiagnosticSessionControl(programmingSession)

Version history:
(1):

Purpose: To prevent a reset when the bootloader is in the programmingSession state and receive a diagnostic request DiagnosticSessionControl(programmingSession)
Source: Chapter: The Edge Node's flow chart - ProgrammingSession and Undefined state; Chapter: The Edge Node's PseudoProgrammingSession flow chart
Verification Method: Test
Legacy ID: 55487v1

When the bootloader is in the programmingSession state and receives a DiagnosticSessionControl(programmingSession) request, the request shall not make the ECU to reset. The bootloader shall stay in the programmingSession state.

662954 v1 The reception of ECU Reset service

Version history:
(1):

Purpose: To define the behaviour when the ECU receives the ECUReset request
Source: Chapter The Edge Node's PseudoProgrammingSession flow chart; Chapter IP ECU flow chart - ProgrammingSession; Chapter IP ECU flow chart - Application; Chapter The Edge Node's flow chart - ProgrammingSession and Undefined state; Chapter The Edge Node's - Application
Verification Method: Test
Legacy ID: 55484v1

When the ECU receives an ECUReset request, the ECU shall make a reset, thus restarting the ECU from Init state.

662939 v3 TransferData Service

Version history:
(3):

Purpose: There is an requirement to support the complete vehicle to be re-programmed via a Ethernet interface. Since the resource is limited the diagnostic tester needs to reduce the message length if an ECU(server) is responding with a maxNumberOfBlockLenght larger than the sending data capacity of client.
Source:
Verification Method: Test
Legacy ID: 341553v0

The TransferData message length of 8194 bytes (2002 hex) shall be supported indenpendent if the parameter maxNumberOfBlockLenght returned in the RequestDownload service is larger.

Note 1: *The Diagnostic tester is normally using an interface towards the vehicle supporting message length larger than 4 kB, therefore it is required to support as large block lengths as possible to reduce the software download time.*

Note 2: *The 8194 bytes is calculated from the formula 8kB data + 2 bytes, where the 2 byte will take the SID and block sequence counter.*

662955 v1 Trig a reset in ECU application mode

Version history:

(1):

Purpose: To enter the bootloader from application.
Source: Chapter: The Edge Node's flow chart - Application; Chapter: IP ECU flow chart - Application
Verification Method: Test
Legacy ID: 55530v2

The ECU application shall trig a reset on reception of a DiagnosticSessionControl(programmingSession) on a wired connection.

662956 v1 **Warm start detected**

Version history:
(1):

Purpose: There is a difference in actions between warm and cold start-up. This requirement defines the next transition when warm start is detected.
Source: Chapter IP ECU flow chart - Init; Chapter The Edge Node's flow chart - Init
Verification Method: Analysis
Legacy ID: 55459v1

When warm start is detected next step is to check if ProgSignature is Set.

662957 v1 **Write ProgSignature**

Version history:
(1):

Purpose: The ProgSignature is used after reset to determine whether the ECU shall stay in the bootloader or start the application.
Source: Chapter: The Edge Node's flow chart - Application; Chapter: IP ECU flow chart - Application
Verification Method: Analysis
Legacy ID: 55526v2

When the ECU application receives a DiagnosticSessionControl(programmingSession) request on a wired connection the ProgSignature shall be written to a fixed address in RAM.
Note: The format and length of the ProgSignature is described in *General Software Download Specification.*

4.11.10.2 BL - UDSonIP - IP ECU reqs

•Introduction

4.11.10.2.1 Introduction

Program mode means that the ECU is executing the primary or secondary bootloader. This chapter describes state and flow charts unique to IP ECU Bootloader. Additional shall this bootloader also implement the non network dependent bootloader requirements described in *General Software Download Specification.*

The IP ECUs shall implement a TCP/IP for in-vehicle communication. The requirements on

TCP/IP stack is defined in *Internetworks General Specification*. TCP will be used for communication to get a robust solution and future safe communication. This means that it is guaranteed that all messages will reach the target without development of an extra transport layer which would have been the case with UDP. IP ECUs bootloaders shall only support the parts of DoIP such as header and Diagnostic Message, see *DoIP Communication.*

4.11.10.2.1.1 IP ECU bootloader state diagram

The IP ECU requires two bootloaders, the primary bootloader and the secondary bootloader. These are described in more detail in the *General Software Download Specification*.

The IP ECU state diagram **,** Figure: *IP ECU State diagram* is a complement to the flow chart described in chapters IP ECU flow chart - Init, IP ECU flow chart – ProgrammingSession and IP ECU flow chart – Application.

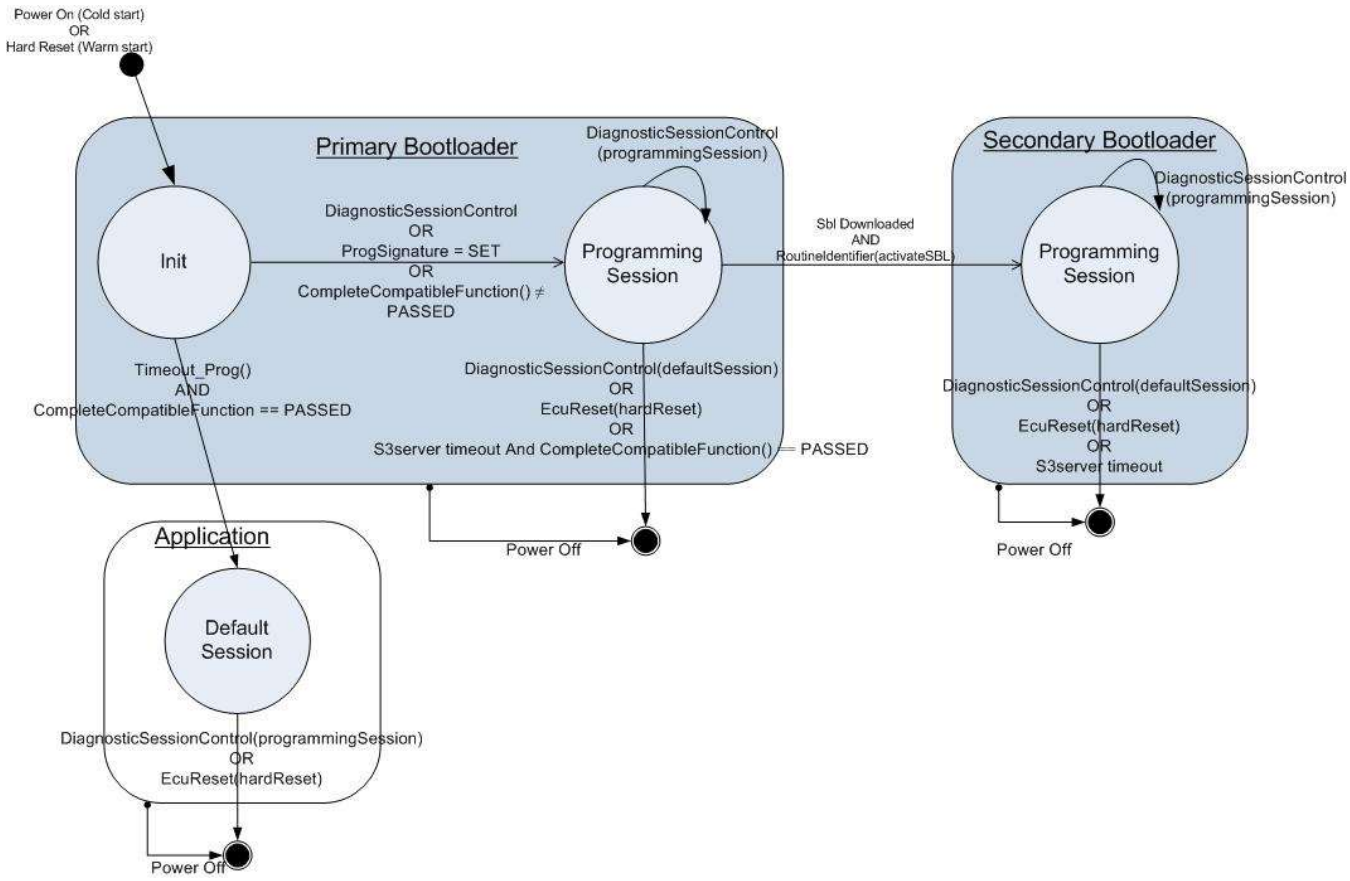
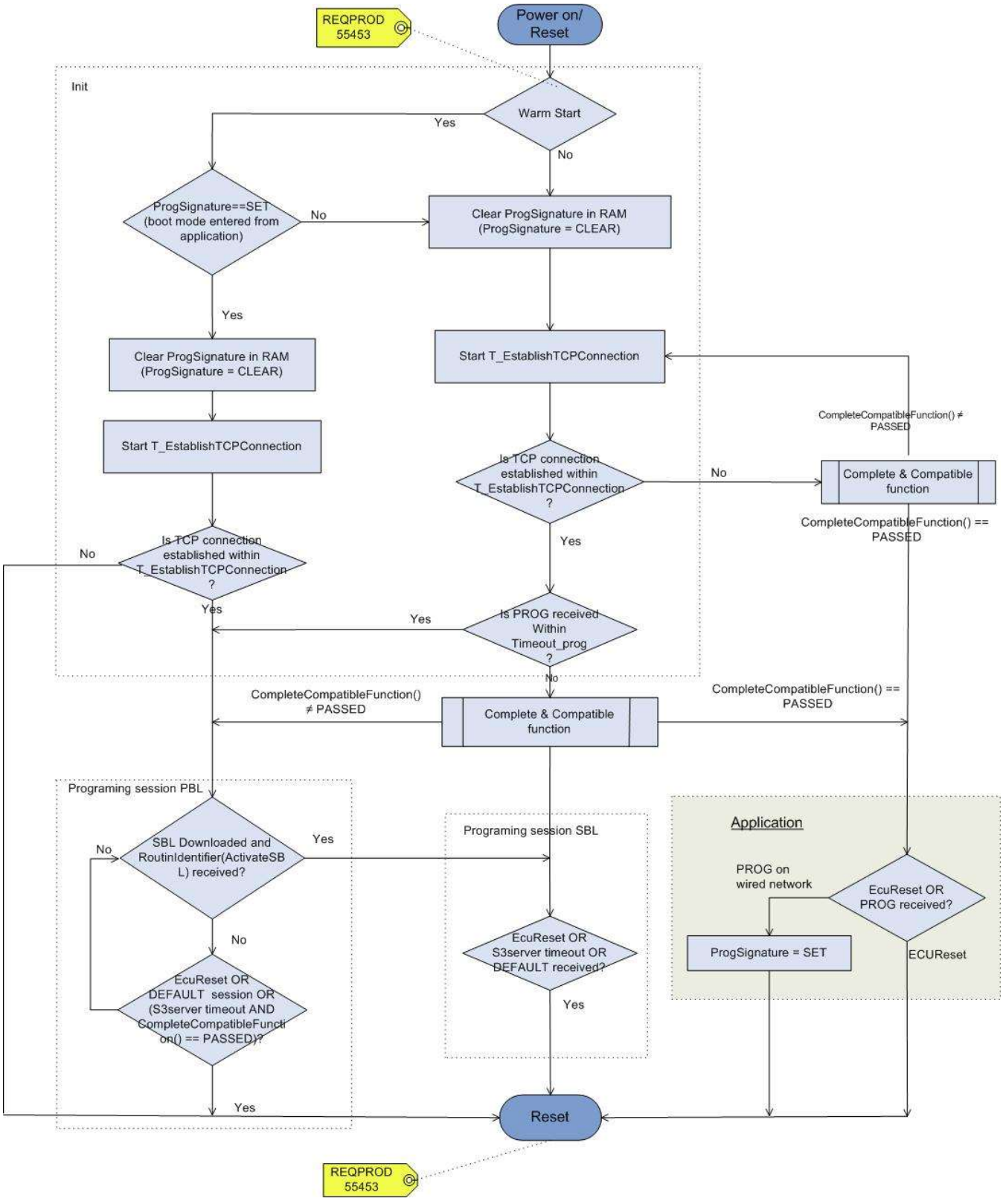


Figure IP ECU State diagram

4.11.10.2.1.2 IP ECU bootloader flow chart - Overview

The flow chart shown in Figure: *IP ECU bootloader flow chart - Overview* is applicable for the IP ECU. The flow chart is divided in states.
Each state is further described in the following chapters. Some figures contain yellow requirements tags. These requirement tags show where each requirement applies in the flow chart.
To avoid problems with hardware initialization (i.e., write once registers, PLL-setup, memory re-

map, interrupt vector table re-map, etc.), when the bootloader is entered, the ECU shall make a complete reset when exiting the application.



4.11.10.2.1.2.1 IP ECU flow chart - Application

This is the normal operation mode of the ECU. To make the ECU enter the programmingSession, a client needs to send a DiagnosticSessionControl(programmingSession) request to the ECU. When all criteria, defined by application, for entering the bootloader has been met, it shall be enough sending one DiagnosticSessionControl(programmingSession) request to make the ECU set the ProgSignature and execute the reset

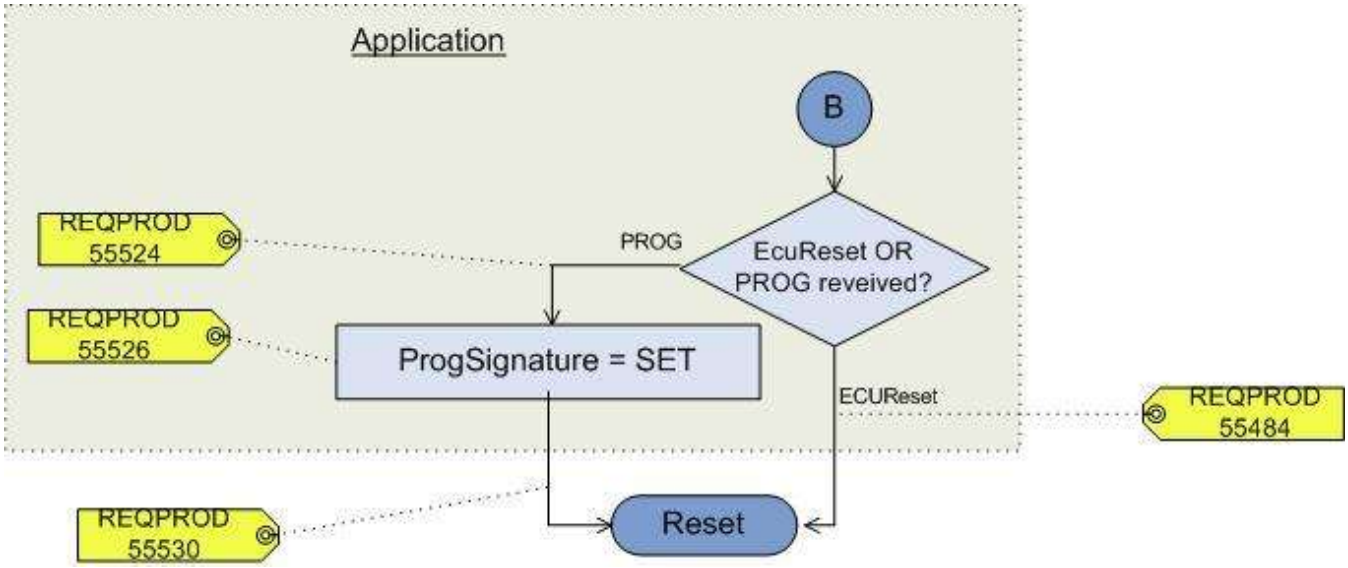


Figure IP ECU flow chart – Application

4.11.10.2.1.2.2 IP ECU flow chart - Init

The ECU shall be able to detect if startup was cause by power – on(cold start) or initiated by application(warm start).
Before the ECU starts the application, the ECU shall listen for the diagnostic request DiagnosticSessionConrol(ProgrammingSession) during Timeout_prog, defined in *General Software Download Specification.* This time window may be used as a "backdoor" for reprogramming the ECU if the ECU is for some reason not able to communicate when the application is started.

The Complete & Compatible function is responsible to decide whether ECUs software's are complete and compatible. The implementation of this step is specified in *General Software Download Specification.*

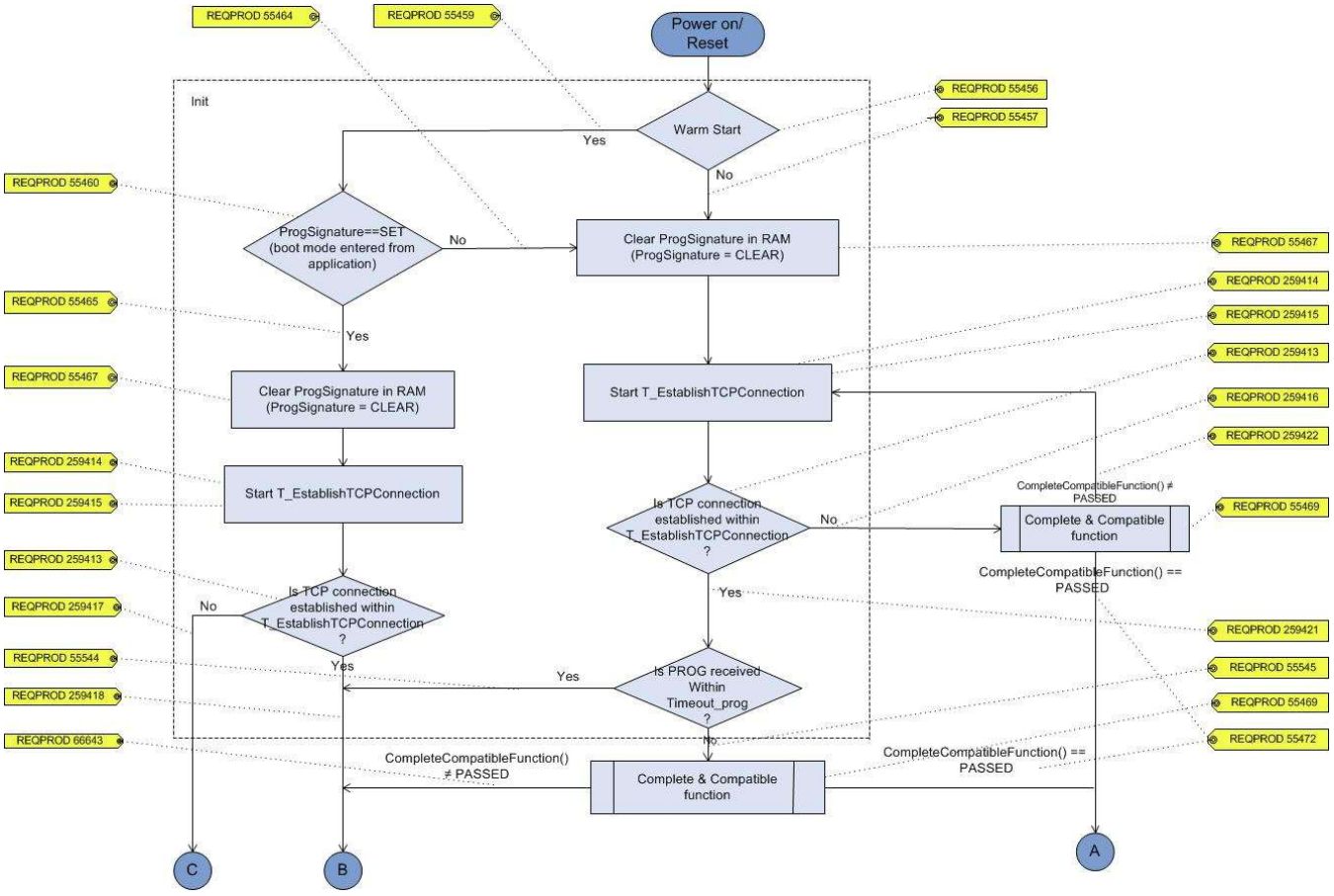


Figure IP ECU flow chart – Init

4.11.10.2.1.2.3 IP ECU flow chart - ProgrammingSession

ProgrammingSession state is entered when ProgSignature is set or when the software's is not Complete & Compatible or a ProgrammingSession was received before Timeout_Prog time ran out.

When the ECU has entered the programmingSession state the ECU shall be able to receive, process and respond to diagnostic requests supported by the bootloader.

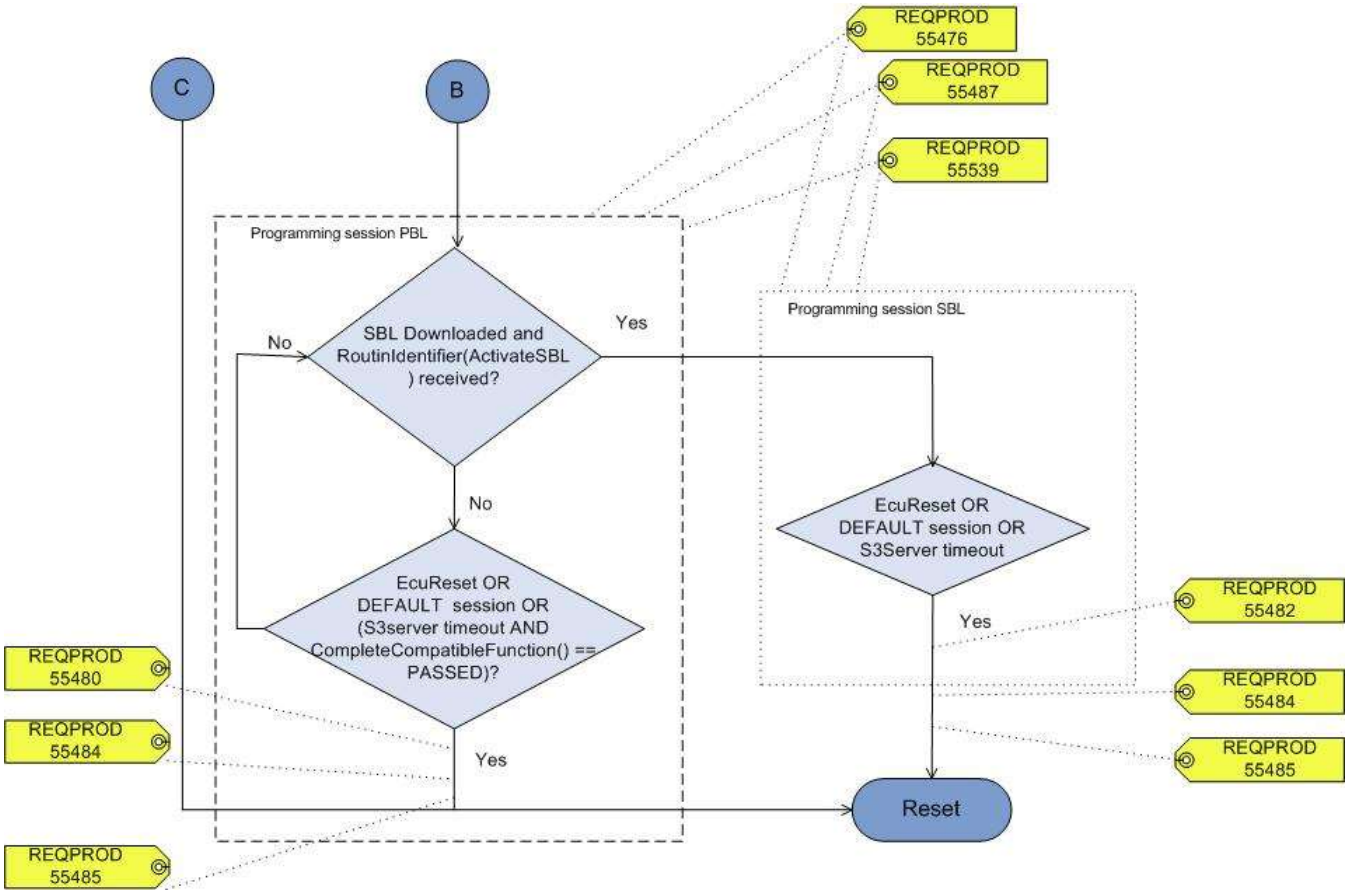


Figure IP ECU bootloader flow chart - ProgrammingSession

•Requirement

4.11.10.2.2 Requirement

662959 v1 **IP ECU - Non complete or compatible software(s)**

Version history:
(1):

Purpose: To define the actions when software(s) is(are) not complete or not compatible.
Source: General Software Download Specification; Chapter IP ECU flow chart - Init
Verification Method: Analysis
Legacy ID: 66643v1

If the return value from the Complete & Compatible function is not equal to PASSED it means the ECU is either not complete or compatible, and therefore the ECU shall go to Programming Session state.

662960 v1 **IP ECU - Bootloader startup sequence**

Version history:
(1):

Purpose: To define the different states of the IP ECU bootloader.

Source: Chapter: IP ECU bootloader flow chart - Overview
Verification Method: Analysis
Legacy ID: 55542v1

The IP ECU shall implement a bootloader sequence in accordance to Figure: *IP ECU bootloader flow chart - Overview* . The figure shows the order in which events shall take place.

662961 v1 **IP ECU - Define T_EstablishedTCPconnection timeout**

Version history:
(1):

Purpose: The T_EstablishedTCPConnection timeout is running a limited time when evaluating if TCP connection is established to Edge Node during startup of bootloader.
Source:
Verification Method: Analysis
Legacy ID: 259414v2

- If ProgSignature is set then T_EstablishedTCPConnection shall be set to 30 000 ms.
- If ProgSignature is not set then T_EstablishedTCPConnection shall be set to 1000 ms.

662958 v1 **IP ECU - DoIP Layer**

Version history:
(1):

Purpose: From a gateway/router point of view it is convenient to support parts of DoIP specification such as Diagnostic Message and the frame format.
Source: DoIP Communication
Verification Method: Analysis
Legacy ID: 55178v2

IP ECU shall support those parts of DoIP that are defined for in-vehicle DoIP Communication in *DoIP Communication.*

662962 v1 **IP ECU - Establish TCP connection**

Version history:
(1):

Purpose: The ECU shall not end up in an infint loop if no TCP communication is established with the Edge node. Therefore the ECU shall evaluate if a connection is established to Edge node within a specific time, T_EstablishedTCPConnection.
Source:
Verification Method: Analysis
Legacy ID: 259413v1

The IP ECU shall evaluate if TCP connection is established within T_EstablishedTCPConnection timeout.

662963 v1 **IP ECU - Incomplete or incompatible software**

Version history:
(1):

Purpose: If there is no completely downloaded data or if software parts are not compatible to each other, it shall be possible to check for TCP connection again.
Source:
Verification Method: Analysis
Legacy ID: 259422v2

If the data is not completely downloaded or if any software parts are not compatible to each other, enter T_EstablishedTCPConnection.

662964 v1 **IP ECU - No PROG received within Timeout_Prog**

Version history:
(1):

Purpose: To define next transition when no DiagnosticSessionConrol(ProgrammingSession) request is received within Timeout_Prog
Source: Chapter: IP ECU flow chart - Init
Verification Method: Analysis
Legacy ID: 55545v1

If no DiagnosticSessionConrol(ProgrammingSession) request on Ethernet is received within Timeout_Prog the IP ECU shall enter the Complete & Compatible function

662965 v1 **IP ECU - PROG received within Timeout_Prog**

Version history:
(1):

Purpose: To be able to enter the "programmingSession" state when the DiagnosticSessionConrol(ProgrammingSession) request is received within Timeout_prog.
Source: Chapter IP ECU flow chart - Init
Verification Method: Analysis
Legacy ID: 55544v1

When the IP ECU receives the diagnostic request DiagnosticSessionConrol(ProgrammingSession) on Ethernet within Timeout_Prog the IP ECU shall enter the “programmingSession” state.

662966 v1 **IP ECU - Restart T_EstablishedTCPconnection**

Version history:
(1):

Purpose: To define when T_EstablishedTCPconnection shall be started and restarted.
Source:
Verification Method: Analysis
Legacy ID: 259415v0

The T_EstablishedTCPConnection shall be restarted when ProgSignature is cleared or application is not complete or compatible.

662967 v1 **IP ECU - S3Server timer times out in the PBL**

Version history:
(1):

Purpose: To define the behaviour when S3server times out in PBL.
Source: Chapter IP ECU flow chart - ProgrammingSession
Verification Method: Test
Legacy ID: 55480v2

If S3server times out when running PBL, the ECU shall make a test on Complete and Compatible function and if Complete and Compatible function returns PASSED the ECU shall make a reset.

Note: The S3server is defined in *UDS Session.*

662968 v1 **IP ECU - S3Server timer times out in the SBL**

Version history:
(1):

Purpose: To define the behaviour when the S3server times out in SBL
Source: Chapter IP ECU flow chart - ProgrammingSession
Verification Method: Test
Legacy ID: 55482v2

When the S3server times out when running SBL, the ECU shall make a reset.

Note: The S3server is defined in *UDS Session.*

686983 v1 **IP ECU - Start S3Server Timer**

Version history:
(1):

Purpose: To define when the S3Server timer shall be started, in order to prevent the peer from not completing initialization and causing their own diagnostic session to exit.
Source:
Verification Method: Test
Legacy ID:

The S3server shall be started after a DoIP connection is established between server and client.

662969 v1 **IP ECU - TCPconnection is established after cold start.**

Version history:
(1):

Purpose: To define next step when TCP connection is established after cold start.
Source:
Verification Method: Analysis
Legacy ID: 259421v1

When TCP connection is established after cold start, the IP ECU shall start listen to DiagnosticSessionControl, see Flow chart *IP ECU bootloader flow chart - Init.*

662970 v1 **IP ECU - TCPconnection is established after warm start.**

Version history:
(1):

Purpose: To define next step when TCP connection is established after warm start
Source:
Verification Method: Analysis
Legacy ID: 259418v1

The IP ECU shall enter programmingSession, when TCP connection is established after warm start, see Flow charts *IP ECU bootloader flow chart – Init* and *IP ECU bootloader flow chart - ProgrammingSession .*

662971 v1 **IP ECU - TCPconnection is not established after cold start.**

Version history:
(1):

Purpose: The ECU shall be able to handle the senario where no TCP connection is established in bootloader.
Source:
Verification Method: Analysis
Legacy ID: 259416v0

When TCP connection is not established after cold start the IP ECU shall enter the Complete & Compatible function.

662972 v1 **IP ECU - TCPconnection is not established after warm start.**

Version history:
(1):

Purpose: The ECU shall be able to handle the scenario where no TCP connection is established in bootloader.
Source:
Verification Method: Analysis
Legacy ID: 259417v1

When TCP connection is not established after warm start the IP ECU shall do a reset, see flow charts *IP ECU bootloader flow chart - Init* and *IP ECU bootloader flow chart -* ProgrammingSession .

662973 v2 **IP ECU - Transmission of Network Management PDU on CAN network**

Version history:
(2):

Purpose: To ensure that ECUs on CAN network will be ready to receive diagnostic requests even if the domain master ECU is not capable of activating communication on CAN ECUs by means of Autosar Network Management. A typical case is when the domain master has no valid application programmed.
Source: Same as Req ID:3327, it is applicable for the IP ECU.
Verification Method: Test
Legacy ID:

When the IP ECU as the zone master is in programming session, the PBL/SBL or Pesudo PBL shall periodically transmit a CAN frame with CAN identifier 0x53F with a period of 400ms(± 10%) on the missing frame master CAN network connected to the ECU. Frame content shall be 0x3F, 0x40, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF.

From the time that the IP ECU has entered programming session, the first CAN frame shall be transmitted after maximum 10ms.

Note: *The frame content is based on that NodeId is in the first byte, ControlBitVector is in the second byte with PNI bit set, and the remaining six bytes of NM User Data are available for PNC requests.*

4.11.10.3 BL - UDSonIP - Edge node reqs

•Introduction

4.11.10.3.1 Introduction

Program mode means that the ECU executing the primary or secondary bootloader. This chapter describes state and flow charts unique to Edge Node Bootloader. Additional shall this bootloader also implement the non network dependent bootloader requirements described in *General Software Download Specification.*

The Edge Node shall implement a TCP/IP stack including UDP for communication with external tester on Ethernet and WLAN. A TCP/IP stack shall also be used for in-vehicle communication with IP ECUs. The requirements on TCP/IP stack is defined in *Internetworks General Specification*. The Edge Node shall also support the DoIP protocol defined in *DoIP Communication*. The communication defined in this specification with the exception of the parts defined for in-vehicle DoIP Communication shall be used for communication with the external tester. DoIP messages and header defined for in-vehicle DoIP Communication in *DoIP Communication* shall be used for communication with in-vehicle ECUs

4.11.10.3.1.1 Edge Node's bootloader state diagram

The Edge Node will only have one bootloader, a PBL including the SBL functionality. The reason is to decrease the number of times the TCP connection to external tester is lost and needs to be reestablished. If the Edge Node should support both PBL and SBL, the TCP connection will be lost every time the SBL is activated. *The Edge Node's bootloader state diagram*, is a complement to the flow chart described is section The Edge Node's flow chart -

Init, The Edge Node's flow chart - ProgrammingSession and The Edge Node's flow chart - Application.

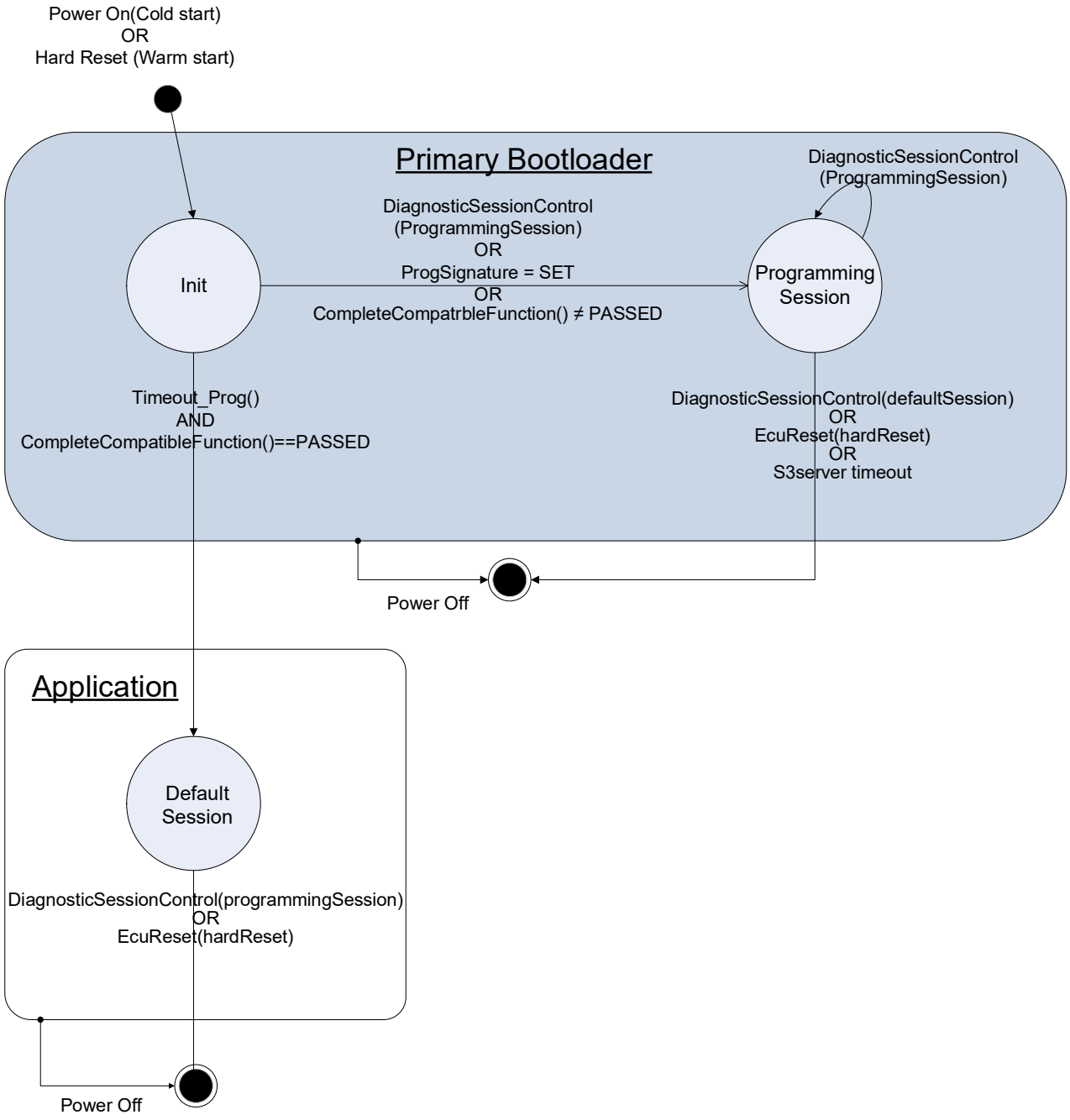


Figure: The Edge Node's bootloader state diagram

4.11.10.3.1.2 Edge Node's bootloader flow chart Overview

The flow chart shown in the figure below is applicable for the Edge Node. The flow chart is divided in states. Each state is further described in the following chapters. The flow chart showing the physical link connection is described in chapter Edge Node connection flow chart diagram. Some figures contain yellow requirements tags. These requirement tags show where

each requirement applies in the flow chart.
To avoid problems with hardware initialization (i.e., write once registers, PLL-setup, memory re-map, interrupt vector table re-map, etc.), when the bootloader is entered, the ECU shall make a complete reset when exiting the application.

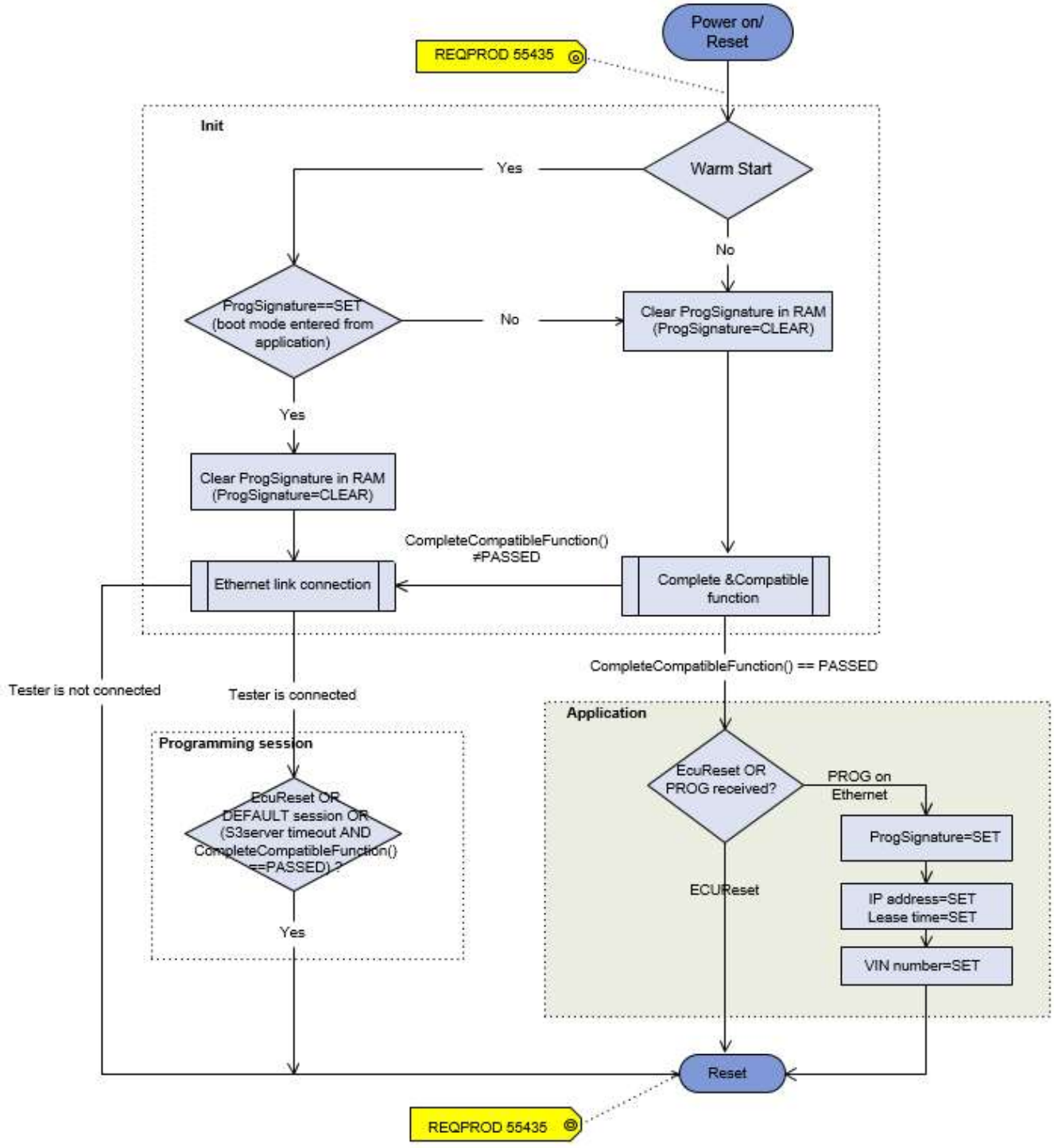


Figure Edge Node bootloader flow chart - Overview

4.11.10.3.1.2.1 Edge Node connection flow chart diagram

The ECU containing the Edge Node functionality is connected to the external tester either via the DLC or via WLAN. The Edge Node is the vehicle´s interface to external tester. The PBL shall support the methods in DoIP, see *DoIP Communication*, enabling the tester to establish a connection with the vehicle. It shall be able to detect an active Ethernet link connection in the bootloader. Only the Ethernet link shall be supported by the bootloader.
The Ethernet active link is described in accordance to *Road vehicles — Diagnostic communication over Internet Protocol (DoIP) — Part 3: IEEE802.3 based wired.*

The Edge Node's PBL shall handle the IP address assignment needed when communication with external tester and to perform the vehicle announcement procedure in accordance to *DoIP Communication*. When IP address is saved in RAM by the ECU application it will be used by the PBL after the reset(initiated by DiagnosticSessionControl(programmingSession)), see chapter Edge Node's bootloader flow chart – Overview and The Edge Node's flow chart - Application. The flow chart describing this scenario is shown in Figure: *Edge Node Link connection and DoIP connection flow chart*, and shall be supported by the PBL

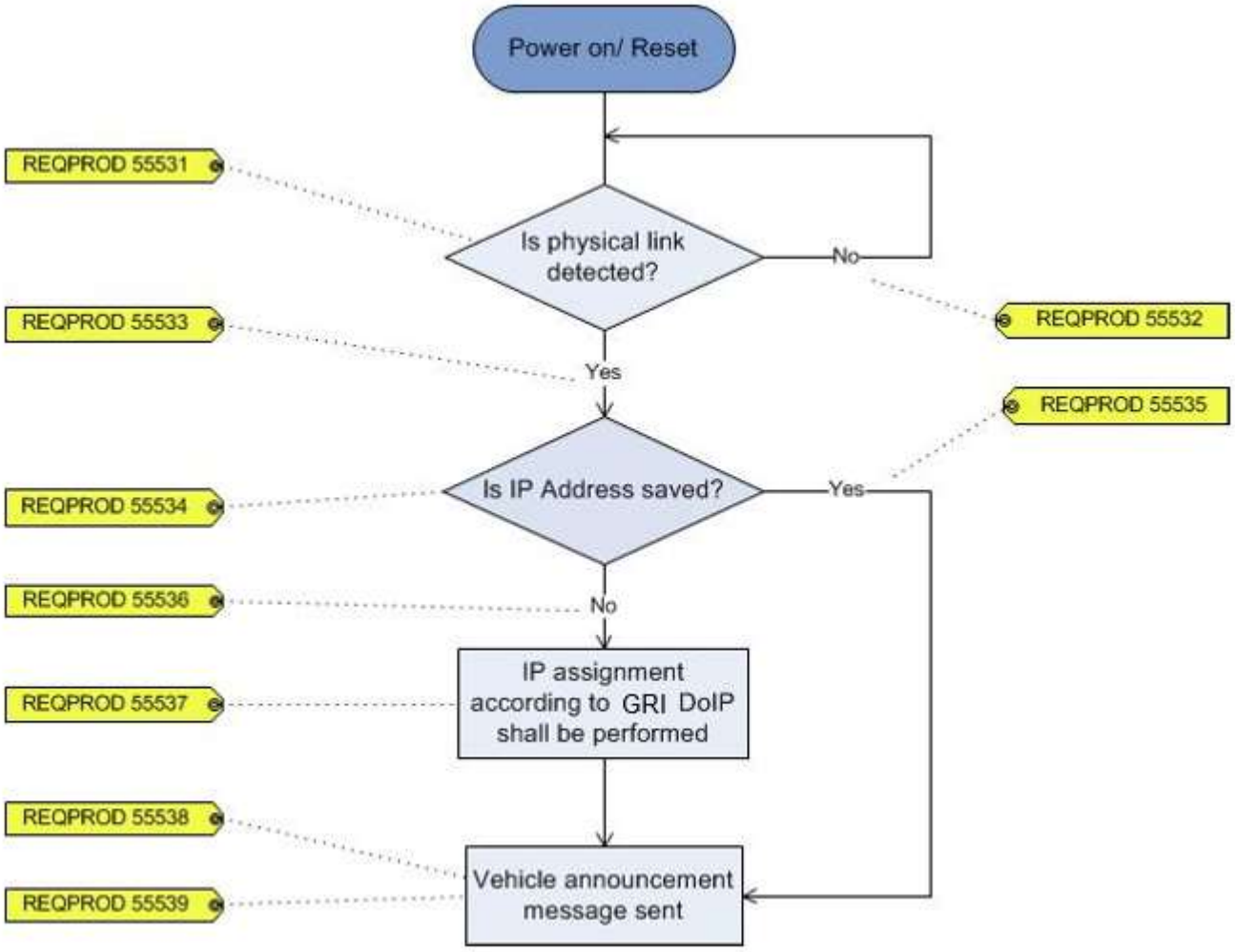


Figure: Edge Node flow chart - Link connection and DoIP connection flow chart

4.11.10.3.1.2.2 The Edge Node's flow chart - Application

To enter the bootloader a client needs to send a DiagnosticSessionControl(programmingSession) request on Ethernet to the ECU. When all criteria, defined by application, for entering the bootloader has been met, it shall be enough sending one DiagnosticSessionControl(programmingSession) request to make the ECU set the ProgSignature and execute the reset.

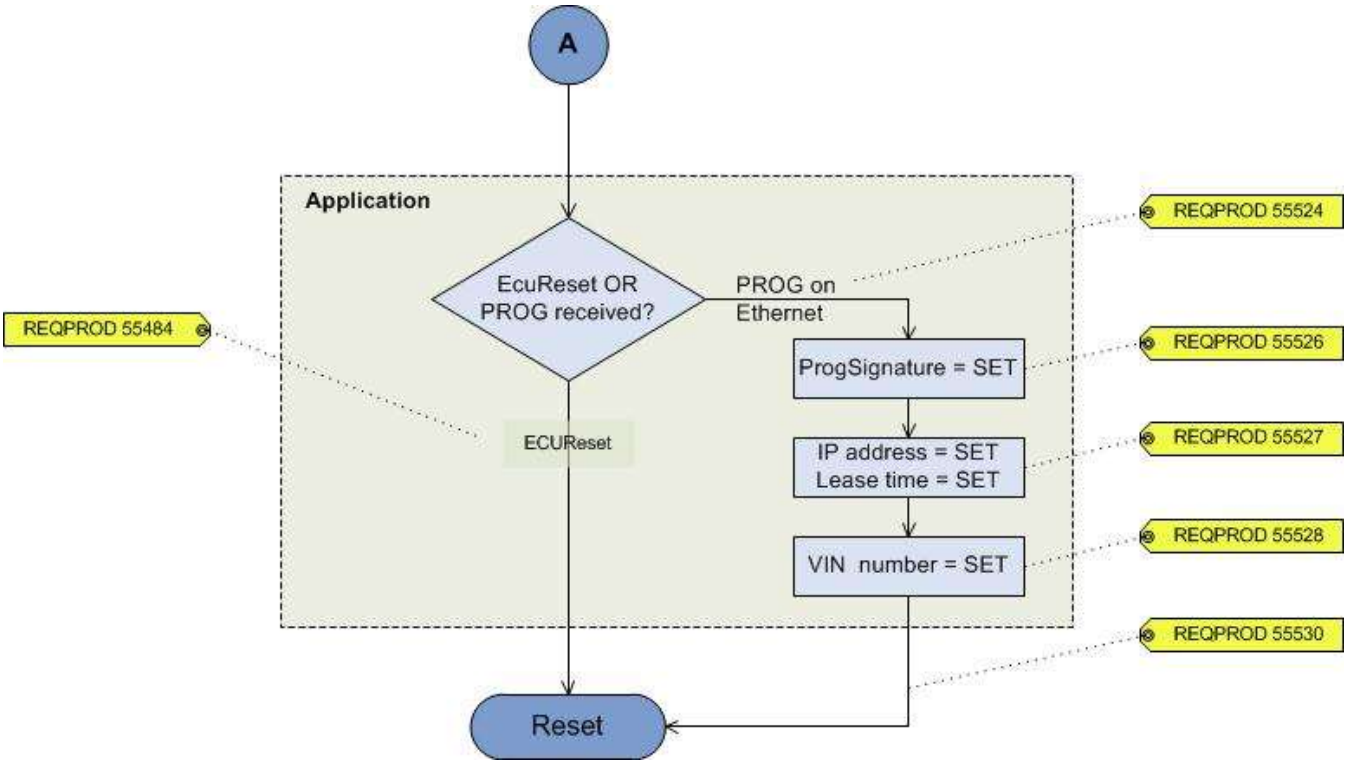


Figure Edge Node flow chart - Application

4.11.10.3.1.2.3 The Edge Node's flow chart - Init

The ECU shall be able to detect if startup was caused by power – on (cold start) or initiated by application(warm start).
The Complete & Compatible function is responsible to decide whether the total ECU's software's are complete and compatible. This means that all necessary software components are present and is compatible to each other to start the application. The implementation of this step is specified in *GeneralSoftware Download Specification.*
To prevent the in-vehicle ECUs' S3timer to time out due to start-up time and communication establishment, the Edge Node is responsible of sending TesterPresent until communication is established with external tester. Once TesterPresent is received from external tester the responsibility is handed over to tester. If no testerPresent is received from external tester within 30s from PowerOn or reset, the Edge Node stops sending and gatewaying the request.

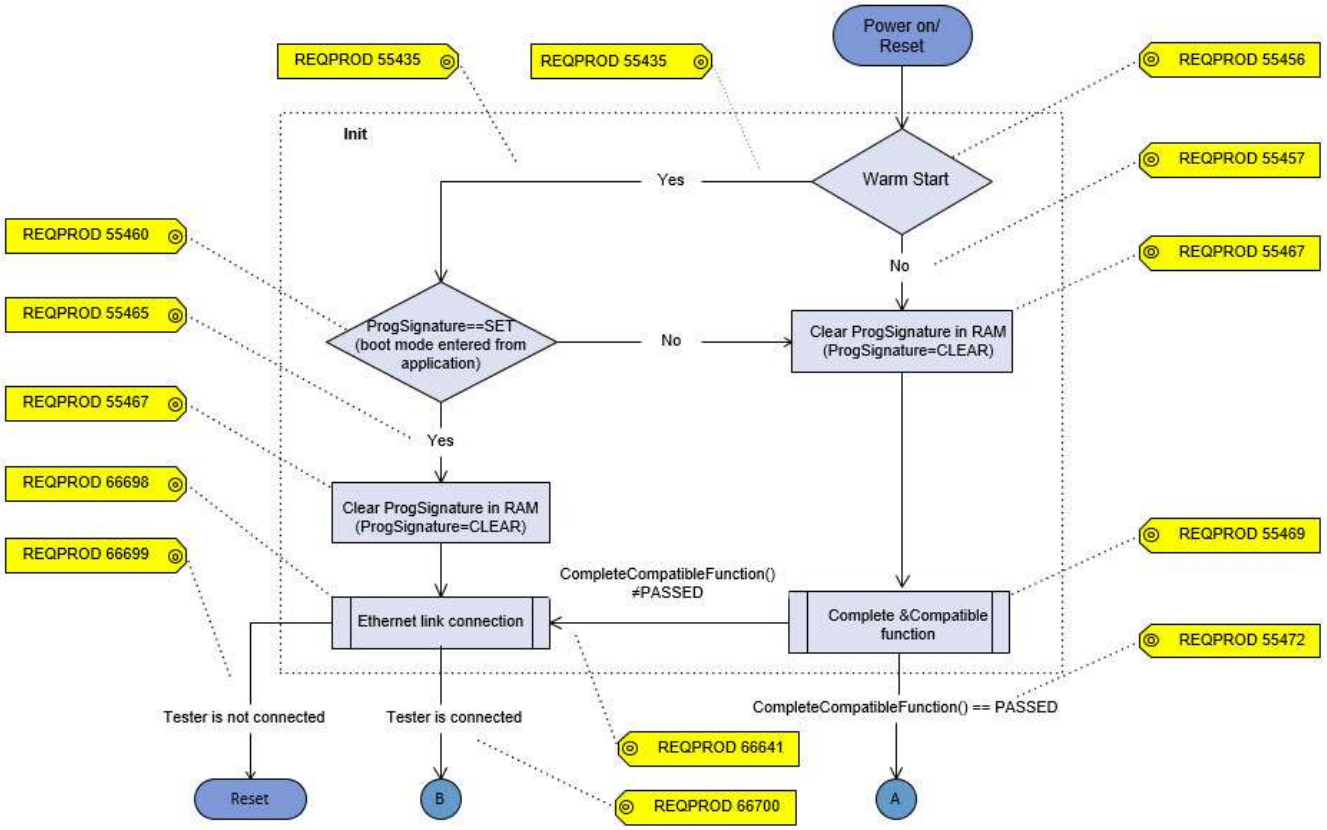


Figure Edge Node flow chart - Init

4.11.10.3.1.2.4 The Edge Node's flow chart - ProgrammingSession and Undefined state

Programming Session is entered when ProgSignature is set, also when the software is not Complete & Compatible.
When the ECU has entered the programmingSession state, the ECU shall be able to receive, process and respond to diagnostic requests in the bootloader.

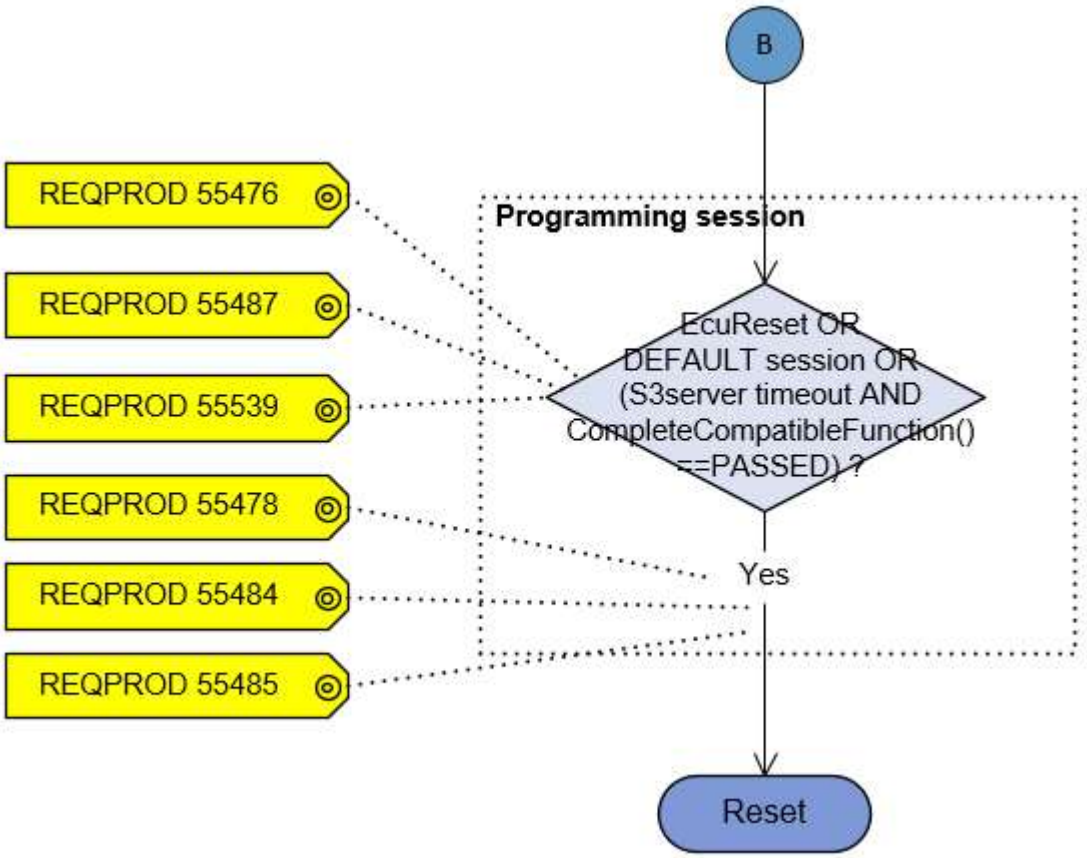


Figure Edge Node flow chart - Programming session state

•Requirement

4.11.10.3.2 Requirement

662980 v1 **Edge Node - Physical link connection and DoIP connection**

Version history:
(1):

Purpose: To describe how to handle the use of IP address and sending of vehicle announcements during start-up of the bootloader.
Source: DoIP Communication; Chapter:Edge Node connection flow chart diagram
Verification Method: Analysis
Legacy ID: 55540v2

The Edge Node shall implement flow chart Figure: *Edge Node flow chart - Link connection and DoIP connection* .

663008 v1 **Edge Node - Activate Secondary Bootloader not supported**

Version history:
(1):

Purpose: The routine identifier Activate Secondary Bootloader shall not be supported by

the Edge Node since it does not support a SBL.

Source: UDS Data; Chapter: RoutineIdentifier 0x0301 Activate Secondary Bootloader; Chapter: The Edge Node's PseudoProgrammingSession flow chart

Verification Method: Test

Legacy ID: 55169v2

The Edge Node shall not support the routine identifier 0x0301 Activate Secondary Bootloader. The Edge Node shall respond with NRC 0x31 requestOutOfRange if the RoutineControl service with routine identifier 0x0301 is received.

662981 v1 Edge Node - Bootloader flow chart

Version history:
(1):

Purpose: To define the Edge Node bootloader startup sequence

Source: Chapter Edge Node's bootloader flow chart - Overview; Figure: Edge Node bootloader flow chart - Overview

Verification Method: Analysis

Legacy ID: 55454v1

The Edge Node bootloader implementation shall follow flow chart in accordance to Figure: *Edge Node bootloader flow chart - Overview*. The figure shows the order in which different sessions are activated.

662982 v1 Edge Node - Check external Ethernet link

Version history:
(1):

Purpose: To be able to communicate and receive diagnostic services from an external tester over Ethernet, an Ethernet link shall exis

Source: Chapter Edge Node connection flow chart diagram; Road vehicles - Diagnostic communication over Internet Protocol (DoIP) - Part 3: IEEE802.3 based wired

Verification Method: Test

Legacy ID: 55531v1

The ECU shall detect an external Ethernet link by using the OBD activation line, see *Road vehicles — Diagnostic communication over Internet Protocol (DoIP) — Part 3: IEEE802.3 based wired.*

662983 v1 Edge Node - Check if IP address is set in RAM

Version history:
(1):

Purpose: When a TCP connection between external tester and vehicle exists, the application saves IP address when DiagnosticSessionControl(programmingSession) is received. The IP address is used during startup when new connection with tester is established. The startup time will be decreased since no IP assignment is needed

Source: Chapter Edge Node connection - flow chart diagram
Verification Method: Analysis
Legacy ID: 55534v1

During startup sequence the bootloader shall check if IP address flag is SET (equal to 0x381169287f549257) at fixed RAM address or not

662974 v1 **Edge Node - DoIP layer**

Version history:
(1):

Purpose: To enable communication over IP with a tester the DoIP specification shall be supported by the Edge Node's bootloader.
Source: DoIP Communication; Chapter: Network/transport layer; Chapter: The Edge Node's PseudoProgrammingSession flow chart
Verification Method: Analysis
Legacy ID: 55175v2

The bootloader of the Edge Node shall implement the *DoIP Communication.*

662975 v1 **Edge Node - DoIP messages used for internal communication**

Version history:
(1):

Purpose: To define which DoIP messages shall be used when Edge Node are communicating with in-vehicle ECUs
Source: DoIP Communication
Verification Method: Analysis
Legacy ID: 66714v2

For in-vehicle communication, Edge Node shall use the DoIP messages defined for in-vehicle DoIP Communication in *DoIP Communication.*

662987 v1 **Edge Node - Edge Nodes PBL will contain SBL functionality**

Version history:
(1):

Purpose: The Edge Node shall only support one bootloder, the PBL. The PBL shall include SBL functionality. The reason is to be able to keep the TCP connection established in the PBL through out the SWDL procedure. The TCP connecion would be lost if activating the SBL. New connection between external tester and Edge Node must be established.
Source: Chapter Edge Node's bootloader state diagram; General Software Download Specification
Verification Method: Analysis
Legacy ID: 55449v1

The Edge Node's PBL shall contain both PBL and SBL functionality defined in *General Software Download Specification*

662988 v1 **Edge Node - Ethernet link connection**

Version history:
(1):

Purpose: To verify that connection with external diagnostic tester exists before IP address request and sending of vehicle announcement.
Source: Chapter: The Edge Node's flow chart - Init; Chapter Edge Node connection flow chart diagram
Verification Method: Analysis
Legacy ID: 66698v1

Before entering programmingSession state or undefined state it shall be confirmed that there is an Ethernet link connected to vehicle, the Edge Node shall have requested an IP address and vehicle announcement shall have sent in accordance to Edge Node flow chart - Link connection and DoIP connection figure: *Edge Node flow chart - Link connection and DoIP connection* chapter: Edge Node connection flow chart diagram.

662990 v1 **Edge Node - Ethernet link exists**

Version history:
(1):

Purpose: To define the next step when Ethernet link is established and ProgSignature=SET has been detected in earlier steps in flow chart.
Source: Chapter: The Edge Node's flow chart - Init
Verification Method: Analysis
Legacy ID: 66700v1

When external diagnostic tester has established a connection enter Programming Session.

662991 v1 **Edge Node - Ethernet link is detected**

Version history:
(1):

Purpose: When Ethernet link is detected, IP address assignment is done or saved IP address is used to set up TCP connection with tester as well as sending vehicle announcement.
Source: Chapter Edge Node connection flow chart diagram; Road vehicles - Diagnostic communication over Internet Protocol (DoIP) - Part 3: IEEE802.3 based wired
Verification Method: Analysis
Legacy ID: 55533v1

If external Ethernet link is detected, next step is to enter " Is IP Address SET?".

662992 v1 **Edge Node - Ethernet link not detected**

Version history:
(1):

Purpose: If there is no Ethernet link detected it is not possible to communicate with external tester. It is no use to get an IP address, send vehicle announcement and try to set up connection with tester unless link exists.
Source: Chapter Edge Node connection flow chart diagram; Road vehicles - Diagnostic communication over Internet Protocol (DoIP) - Part 3: IEEE802.3 based wired
Verification Method: Analysis
Legacy ID: 55532v1

If external Ethernet link is not detected, retry to detect the link.

662993 v1 **Edge Node - Ethernet link timeout**

Version history:
(1):

Purpose: To define the next step if diagnostic tester has not established a connection to the vehicle.
Source: Chapter: The Edge Node's flow chart - Init
Verification Method: Analysis
Legacy ID: 66699v1

If the external tester has not established a TCP connection and DoIP session within 10 seconds the Edge Node shall do a reset.

662994 v1 **Edge Node - IP address assignment during startup sequence**

Version history:
(1):

Purpose: To be able to communicate with external tester an IP address is required
Source: Chapter Edge Node connection - flow chart diagram; DoIP Communication
Verification Method: Analysis
Legacy ID: 55537v2

During the startup sequence the bootloader of the Edge Node shall perform an IP address assignment in accordance to *DoIP Communication.*

662996 v1 **Edge Node - IP address is SET**

Version history:
(1):

Purpose: Define next step when TCP/IP connection flag is set
Source: Chapter Edge Node connection - flow chart diagram
Verification Method: Analysis
Legacy ID: 55535v1

When IP address Flag is SET get the IP address and its Lease time from fixed RAM address and enter Vehicle announcement sent state. Clear the flag by setting it to 0x0000000000000000.

662995 v1 **Edge Node - IP address is not set in RAM**

Version history:
(1):

Purpose: Define next step when IP address flag is not set
Source: Chapter Edge Node connection - flow chart diagram
Verification Method: Analysis
Legacy ID: 55536v1

When IP address flag is not set next step is to enter the IP assignment state.

662997 v1 **Edge Node - Non complete or compatible software(s)**

Version history:
(1):

Purpose: To define the actions when software(s) is(are) not complete or not compatible.
Source: General Software Download Specification; Chapter The Edge Node's flow chart - Init
Verification Method: Analysis
Legacy ID: 66641v1

If the return value from the Complete & Compatible function is not equal to PASSED it means the ECU is either not complete or compatible, and therefore the ECU shall go to Ethernet link state .

662999 v1 **Edge Node - Receiving DoIP messages**

Version history:
(1):

Purpose: ISO standard is followed and therefore messages in this standard shall be processed by Edge Node.
Source: DoIP Communication; Chapter: Edge Node connection flow chart diagram; Chapter: The Edge Node's PseudoProgrammingSession flow chart
Verification Method: Test
Legacy ID: 55539v2

The Edge Node shall be able to receive, process and respond to DoIP messages defined in *DoIP Communication.*

663001 v1 **Edge Node - S3Server timer times out in the PBL**

Version history:
(1):

Purpose: To define the behaviour when S3server times out in PBL
Source: Chapter: The Edge Node's flow chart - ProgrammingSession and Undefined state
Verification Method: Test

Legacy ID: 55478v2

When S3server times out when running PBL, the Edge Node shall make a reset equal to hard reset, meaning that the ECU restarts and return to the Init state.

Note:The S3server is defined in *UDS Session.*

663002 v1 **Edge Node - Set IP address**

Version history:
(1):

Purpose: The IP address used by the Edge Node's application shall be used by the bootloader during startup. Instead of looking for a new address at startup the address used in application mode shall be used. The flag shall indicate whether an IP address is saved or not.
Source: Chapter The Edge Node's flow chart - Application
Verification Method: Analysis
Legacy ID: 55527v1

The IP address used by Edge Node application as well as its Lease time shall be set at fixed RAM address and a 4 byte flag shall be set at fixed RAM address when DiagnosticSessionControl(programmingSession) is received in the ECU application. An 8 byte RAM area shall be reserved for the IP address this purpose. A 4 byte RAM area for lease time shall be reserved.

Note: The developer of application and bootloader shall agree upon which FIXED RAM address to use.

663003 v1 **Edge Node - Set VIN number**

Version history:
(1):

Purpose: The vehicle announcement contains a VIN number parameter. It shall be set to the vehicle's VIN number when known and to default(defined by DoIP) when not known
Source: DoIP Communication
Verification Method: Analysis
Legacy ID: 55528v2

The 17 byte VIN number shall be written to a fix RAM address (irrespective of it is the correct VIN or default defined by *DoIP Communication)*. A 4 byte flag shall also be written to a fixed RAM address indicating that the VIN number is set.

Note: The developer of application and bootloader shall agree upon which FIXED RAM address to use.

663011 v1 **Edge Node - Start S3Server Timer**

Version history:
(1):

Purpose: To define when the S3Server timer shall be started
Source:
Verification Method: Test
Legacy ID: 471767v0

The S3Server shall be started after a DoIP connection is established.

663004 v2 **Edge Node - Start sending TesterPresent**

Version history:
(2): For implementations that do not disconnect DoIP sessions, this requirement can be omitted.

Purpose: To define when TesterPresent shall be sent.
Source: Chapter ECU program mode; UDS Services, Bootloader - Implementation on MOST150 based networks
Verification Method: Test
Legacy ID: 76863v4

The Edge Node shall start sending TesterPresent within 2000ms from change to programming session, for Ethernet the TesterPresent must be started as soon as a client connects.

Note: If the transition from application session to programming session does not require actively disconnecting the DoIP session with the client, then TesterPresent at this time shall be still the responsibility of Client. In addition, it will influence the requirement of *ReqID 663005* and *ReqID 663006*.
Comment: The AB partition design of PBL is one possible approach to achieve seamless transition.

663005 v1 **Edge Node - The periodicity of TesterPresent**

Version history:
(1):

Purpose: The Edge Node is responsible of distributing TesterPresent to in-vehicle ECUs from start-up untill TesterPresent is received from tester. To prevent the Edge node from keeping the vehicle in the current session forever if the connection to the tester is not established, the autonomous TesterPresent transmission must be limited. When the tester is sending the TesterPresent, the Edge Node shall qateway this request and all other requests to in-vehicle ECUs
Source: Chapter ECU Program mode
Verification Method: Test
Legacy ID: 76868v2

The Edge Node shall at change to programming session send TesterPresent until it receives and accepts a RoutingActivation command from a diagnostic tester or if no RoutingActivation is received send TesterPresent for 30000 ms (+/-500ms) from first TesterPresent sent.

663006 v1 **Edge Node - The sending of TesterPresent**

Version history:
(1):

Purpose: To prevent in-vehicle ECUs to reset due to missing TesterPresent from tester, the Edge Node will be responsible of sending TesterPresent to in-vehicle ECUs for a limited amount of time. until external diagnostic tester is connected.

Source: Chapter ECU Program mode;
Verification Method: Test
Legacy ID: 76822v2

The Edge Node shall send a functional addressed TesterPresent request with SPRMIB=TRUE with a periodicy of 1000ms(+/-10%) to all in-vehicle ECUs including itself.
Note: this means that the Edge Node is not allowed to change session due to S3 timeout while it is generating tester present messages.

663012 v3 Edge Node - Transmission of Network Management PDU on CAN network

Version history:
(3): 增加备注，针对支持伪编程会话的ECU，可以接受使用应用层的NM报文ID，且需PNC29置1。

Purpose: To ensure that ECUs on CAN network will be ready to receive diagnostic requests even if the domain master ECU is not capable of activating communication on CAN ECUs by means of Autosar Network Management. A typical case is when the domain master has no valid application programmed.
Source: Same as Req ID:3327, it is applicable for the edge node.
Verification Method: Test
Legacy ID:

When the IP ECU as the master is in programming session, the PBL/SBL or Pesudo PBL shall periodically transmit a CAN frame with CAN identifier 0x53F with a period of 400ms(± 10%) on the missing frame master CAN network connected to the ECU Frame content shall be 0x3F, 0x40, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF.

From the time that the ECU has entered programming session, the first CAN frame shall be transmitted after maximum 10ms.

Note1: *The frame content is based on that NodeId is in the first byte, ControlBitVector is in the second byte with PNI bit set, and the remaining six bytes of NM User Data are available for PNC requests.*
Note2: *If the ECU supports pseudo programming session, that transmitting standard application Network Management message with at least PNC29 set can be used to replace the CAN frame with CAN identifier 0x53F, is acceptable.*

987287 v1 Edge Node - Transmission of TesterPresent

Version history:
(1): 新增需求，在TCP延迟率高的时候，通过边缘节点主动发送功能寻址3E 80，增加本地DoIP诊断和刷写的鲁棒性。

Purpose: When execute SWDL to a vehicle using a local area network, TCP retransmission delays the transmission of diagnostic request sent by the tool to the edge nodes. In order to ensure the robustness of the diagnostics, the edge nodes need to maintain the programming session of the vehicle in specific scenarios.

Source:

Verification Method: Analysis AND Test

Legacy ID:

Different from the definition of requirements ReqID 663005 and ReqID 663006. Edge Node shall actively send once vehicle functional addressed TesterPresent request with SPRMIB=TRUE to all in-vehicle ECUs including itself when it is detected that the vehicle or zone functional addressed request has not been received for more than 3 second during DoIP communication with external tools, i.e. through OBD P2P connection or wireless connection.

Note: The maximum duration for sending this diagnostic request is until the TCP socket is closed or Network Management VFC release, e.g. TCP KeepAlive mechanism exhausted, or TCP socket waved to close normally.

663007 v1 **Edge Node - Vehicle announcement messages during startup sequence**

Version history:
(1):

Purpose: To enable communication with external tester, the Edge Node has to indicate that the vehicle exists. This is done by announcement messages

Source: Chapter Edge Node connection flow chart diagram; DoIP Communication

Verification Method: Test

Legacy ID: 55538v2

he bootloader of the Edge Node shall send vehicle announcement messages during startup sequence in accordance to *DoIP Communication.*

4.11.11 Dual Bank Software Update

4.11.11.1 General A/B Swap Requirements

•General A/B Swap Requirements

4.11.11.1.1 General A/B Swap Requirements

975731 v3 **Bootloader addressing schema**

Version history:
(3):

Purpose:

Source:

Verification Method: Test

Legacy ID:

Zeekr Requirement ID: 643364

The ECU shall use Flexible Runtime Address Mapping according to AUTOSAR nomenclature

975732 v3 **Bootloader timeout**

Version history:
(3):

Purpose: To clarify what to do as there is no active partition to exit to from the programming session which is non-default session
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 643353

If bootloader is in programming session and the $S3_{Server}$ timer expires and no active partition has been possible to boot the ECU shall go to low power mode.

975733 v2 Routing of PDUs in programming session

Version history:
(2):

Purpose: To ensure UCM Master/External tester can communicate with other ECUs even if the ECU is locked in bootloader
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 646429

The ECU shall route any incoming diagnostic request to any connected networks, if the ECU is in bootloader.

975734 v2 Software synchronization

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 762484

The ECU should be able to synchronize the partition A （active partition）software to partition B （inactive partition）after receiving the synchronization command (0x0255+00) and the command will be send after flash process is sucessfull.
The synchronization process should be able to be performed under the application and the ECU function can't be affected.
There will be a DID (D0A1) to read the Synchronization status.
Note1: The sync software may go through several operation cycles, and the ECU should sleep when the sleep conditions are met.
Note2: If there is an exception in the synchronization process, the ECU must be able to retry.
Note3: Operation cycle refer to REQ: 2361.

Exception handling for the synchronization process includes:
1 If received the synchronization command repeatedly during the flash process,the ECU should reply to the negtive response.
2 If received the synchronization command during the synchronization process, the ECU should
reply to the positive response and continue the synchronization.
3 If receiveed the software synchronization command after A/B bank switch failed, the ECU should performs software synchronization.
4 When a programming session request is received, the synchronization process can be aborted.
5 Synchronization failure is automatically retried three times.
6 The software synchronization process should supports resynchronization of breakpoints.

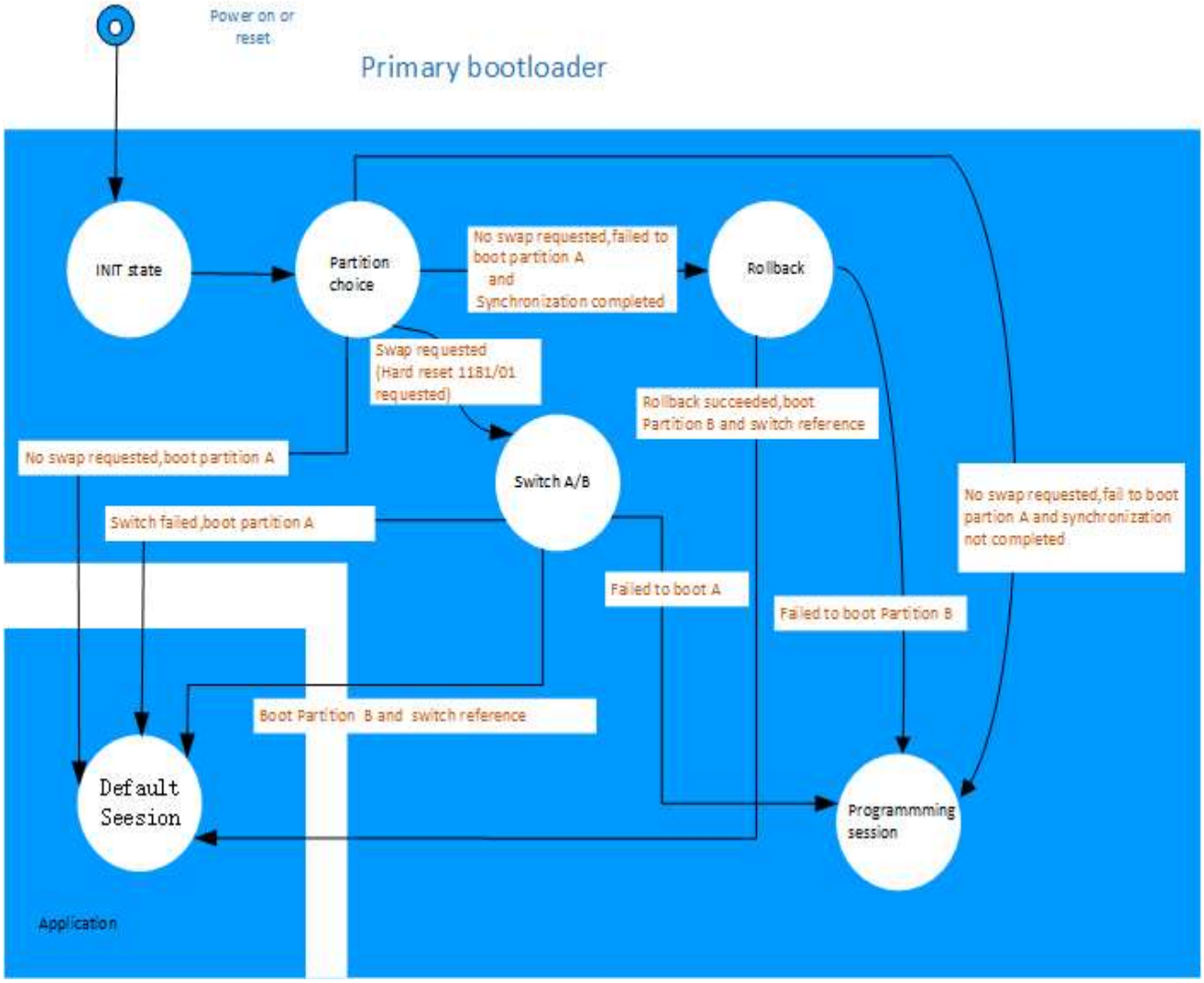
Note：
The default value of D0A1 is "02".
Once the ECU begins to perform the flash program, DID D0A1 returns 00 until the software flash is completed. During synchronization process, ECU returns 01.

4.11.11.2 A/B Swap flowchart

•overview of the boot loader flowchart

4.11.11.2.1 overview of the boot loader flowchart

This is an overview of the boot loader flowchart



•INIT State

4.11.11.2.2 INIT State

975703 v2	**Bootloader startup**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 712217

The ECU shall always enter Bootloader in INIT state when the ECU starts irrespectably on the reason for the start.

•Partition choice

4.11.11.2.3 Partition choice

975705 v2 **Bootloader state variable**

Version history:
(2):

Purpose:
Source:
Verification Method: Analysis
Legacy ID:
Zeekr Requirement ID: 712229

The bootloader shall read from a memory area which state the bootloader is in. The variable shall be stored a non-volatile area. The following states shall be possible

- NORMAL
- SWITCH
- SWITCH_FAILED
- ROLLBACK
- FAILURE

975704 v2 **Transistion to Partition choice**

Version history:
(2):

Purpose:
Source:
Verification Method: Analysis
Legacy ID:
Zeekr Requirement ID: 712227

After bootloader has entered INIT state it shall transisition to "Partition choice" once HW and SW is iniated.

•Rollback

4.11.11.2.4 Rollback

975706 v3 **Failure to boot NORMAL partition**

Version history:
(3): 如果一直处于A分区，某一次无法启动（不包括第一次升级成功切换的那次），且已经同步完成，则应回滚，从B分区启动。如果仅实现AB，无同步能力，则不再选择从B启动而是进入编程会话。

Purpose:
Source:
Verification Method: Analysis

Legacy ID:
Zeekr Requirement ID: 712236

If the bootloader can not for any reason complete the boot of the software in partition A and software synchronization completed, when the bootloader state is NORMAL the bootloader shall enter state ROLLBACK and attempt to boot the old saved software in partition B.

975707 v2 **Rollback requested from application**

Version history:
(2):

Purpose:
Source:
Verification Method: Analysis
Legacy ID:
Zeekr Requirement ID: 712250

If the application has requested ROLLBACK the bootloader shall boot partition B, this is now the primary partition and for reference this is now Partition A.

If the boot of partition B is successful, then change bootloader state to NORMAL.

•Switch A/B

4.11.11.2.5 Switch A/B

975708 v2 **SWITCH requested from application**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 712243

If the bootloader is in state SWITCH which was requested from application the bootloader shall switch to and boot partition B and perform compatibility checks.

•Default session

4.11.11.2.6 Default session

975711 v2 **Failure to perform SWITCH**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 712246

If any checks in the bootloader fails during bootloader state SWITCH operation, the ECU shall boot partition A again and set state SWITCH_FAILED.

975709 v2 Normal Partition selection

Version history:
(2):

Purpose:
Source:
Verification Method: Analysis
Legacy ID:
Zeekr Requirement ID: 712234

If the bootloader state is set to NORMAL then the ECU shall boot the partition A.

975712 v2 Rollback sucessful

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713826

If the boot of partition B is successful, this is now the primary partition and for reference this is now Partition A,then change bootloader state to NORMAL.

975710 v2 Succesfull SWITCH performed

Version history:
(2):

Purpose:
Source:

Verification Method: Analysis
Legacy ID:
Zeekr Requirement ID: 712245

If the bootloader is in state SWITCH and successfully boot Partition B, this is now the primary partition and for reference this is now partition A.

If the boot of partition B is successful, then change bootloader state to NORMAL.

•Programming session

4.11.11.2.7 Programming session

975713 v2 **FAILURE state**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 712239

If the bootloader state is set to FAILURE the ECU failed to boot any partition last power cycle.

The bootloader shall try to boot partition A first. If Partition A fails, the bootloader shall try to boot partition B. If this also fails it shall stay in programming session and keep state as FAILURE

975716 v2 **Maximum time to boot**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 712255

If the bootloader takes more than a configurable time to boot any partition the bootloader state shall be set to FAILURE and jump to Programming session.

The timeout shall be default set to 180 seconds. The value shall be choosen as 2 times the worst case scenario calculated by supplier.

975715 v3 **Retry to boot**

Version history:
(3):

Purpose:
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 712252

In the case bootloader is in programming session and both partition has failed to boot the bootloader shall be in state FAILURE.

If either of the following events occurs:
- Power on
- UDS Service 11 is received with sub service 01
- UDS Service 10 is received with sub service 01

The ECU shall revert to INIT state and attempt to start partition A first and then B before ending up in programming session again if both partition again fail to boot.

975714 v2 **Rollback failed**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 712247

If the ROLLBACK is incomplete and the bootloader can not boot partition B, the ECU shall transition to Programming session and set the bootloader state to FAILURE.

4.11.11.3 A/B Swap Requirements

•Induction

4.11.11.3.1 Induction

It offer the greatest advantages if the software updates are downloaded to the target ECUs while driving. Downloading the updates in application context can reduce the overall down time of the vehicle during a software update to a minimum. A multi-partition memory concept must be supported in the target ECU. The download is performed in an inactive memory area in application context. Once the download is completed, the new software is activated in the flash

bootloader context.

•ECU reprogram in application mode

4.11.11.3.2 ECU reprogram in application mode

975717 v1 **Session for reprogram in appliaction mode**

Version history:
(1):

Purpose: To define the session for reprogram in application mode
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 712262

If the ECU supports AB swap and software download, it shall implement reprogram diagnostic session $40(OTA session) in application mode, the format of the request and response for this subservice is defined as follows.
Request:

Data Byte	Parameter Name	Cvt	Byte value
#1	DiagnosticSessionControl Request SID	M	0x10
#2	sub-function = [diagnosticSessionType]	M	0x40

Positive Response:

Data Byte	Parameter Name	Cvt	Byte value
#1	DiagnosticSessionControl Response SID	M	0x50
#2	sub-function = [diagnosticSessionType]	M	0x40
#3 #4 #5 #6	sessionParameterRecord[] = [P2Server_max (high byte) P2Server_max (low byte) P2*Server_max (high byte) P2*Server_max (low byte)]	 M M M M	 0x00 – 0xFF 0x00 – 0xFF 0x00 – 0xFF 0x00 – 0xFF

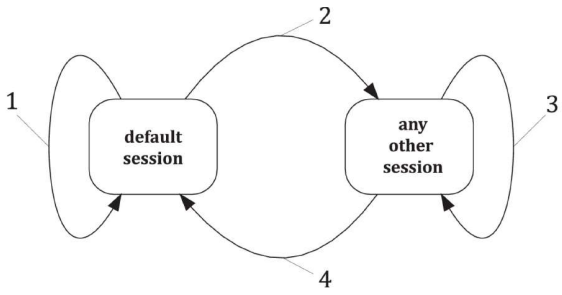
•session transition

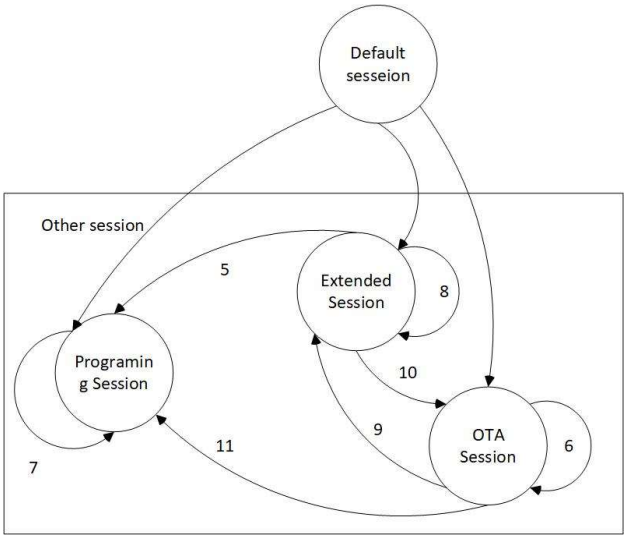
4.11.11.3.3 session transition

975718 v1 **session transition description**

Version history:
(1):

Purpose: To describe diagnostic session transition
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 712266





Diagnostic sessions that the ECU might use:

- Default session
- Programming session
- Extend the session
- OTA sessions

State transitions 1 from default session to default session: Re-entering the default session, the ECU performs full initialization, resets all events, Settings and controls activated in the default session mode, but does not include operations that have been programmed to a non-volatile storage location.

State transitions 2 from default session to non-default session: The configuration does not need to be reset when the configuration is switched from default session to non-default session.

State transitions 3 from non-default session to non-default session: When switching from non-default session to non-default session, the security state switches to the locked state, but the state of communication control service and control DTC setting service is not affected.

State transitions 4 from non-default session to default session: From non-default session to default session, the ECU performs full initialization, resets all events activated in non-default mode, Settings and controls, etc., but does not include operations that have been programmed to a non-volatile storage location.

State transitions 5,6,7,8,9, 10 and 11 are state transitions within state transitions 3, and ECU behavior is the same as transition 3.

•requirements for ECU wich support AB swap

4.11.11.3.4 requirements for ECU wich support AB swap

975725 v2 **AB partition additional requirements for ECUs having multiple processors**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 712355

The term main processor is in this context used to define the microcontroller having the interface to the network used for software download. The requirements in this section are valid for ECUs where all software to the secondary processor is not directly sent to the secondary processor but being addressed to the main processor, i.e. it is considered as one single ECU from a tester point of view.
When the primary processor runs in partition A, the secondary processor should also run in partition A. When the partition is switched, the primary and secondary processors should also be switched simultaneously.

975730 v2 **Additional requirement for switch partitions（reprogram in OTA session）**

Version history:
(2):

Purpose: In order to distinguish the partition switching action in the two reprogramming scenarios, bootloader and OTA
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 714441

For ECU that implements the AB partition, ECU shall support switch partition if the conditions of ECU switching to inactive partition are met, if reprogram in OTA session, reset or restart (ignition on) should not result in partition switching before ECU receiving a routine request to switch partitions.
If switch partition fails, the ECU will stay in the previous partition.

975729 v2 **Additional requirement for switch partitions（reprogram via flash bootloader）**

Version history:
(2):

Purpose: In order to distinguish the partition switching action in the two reprogramming scenarios, bootloader and OTA

Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713850

For ECU that implements the AB partition, ECU shall support switch partition,if reprogram via flash bootloader,reset should result in partition switching.
If switch partition fails, the ECU will stay in the previous partition.

Note:
After the software integrity and compatibility check passes, the partition switch flag is automatically set and the partition switch is performed after the ECU reset.
If a hard reset command is not received during the programming session, the ECU needs not to perform partition switching on the next power-on cycle.

975722 v1 **Additional requirements for software downloads**

Version history:
(1):

Purpose: To define the partitions for software downloads

Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 712342

The software download is performed in an inactive memory area either while driving down the road in the application context or when the vehicle is at rest in the bootloader context.
In both variants, the software update is always activated when the vehicle is in a safe state, typically in the flash bootloader context, during software update activation, the vehicle is not operational.

975723 v1 **Address remapping strategy**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 712350

Divide the flash area into two areas, one is A bank, the other one is B bank, A/B can be switched. By mapping the address in area A/B, the address of the running area is always the

same, the address of the reprogramming area is also fixed.

975727 v1 **Automatic partition switching behavior must be avoided**

Version history:
(1):

Purpose: To avoid an unexpected ECU restart
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 712364

In order to support flexible vehicle flash strategy, the ECU that supports AB partition must ensure that the current running partition application is still valid and reliable after the program is downloaded. The automatic partition switching behavior must be avoided. Partition switch can only be allowed after diagnostic request is received.

975719 v2 **Partitioning**

Version history:
(2):

Purpose: Ensure flash is used efficiently

Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 712523

ECU Shall use Flexible Runtime Address Mapping, below is the minimum amount of partitions

- Partition A
- Partition B
- Calibration data

Note: Partition A and Partition B are logical partitions. Other partitions can be shared and therefore use fixed memory allocation.

975726 v2 **Software activation requirements**

Version history:
(2):

Purpose: To define software activation methods for different ECUs
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 712356

The ECU that supports AB swap needs to wait for the master computer to send commend to activate the new software(switch partition).
ECU that does not support AB partition restarts and activates itself according to the upgrade process.

975720 v1 **Software is downloaded to the inactive partition**

Version history:
(1):

Purpose: To define the partitions for software downloads
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 712293

For ECU that implements the AB partition, ECU shall be re-programmed in application mode, and the software will be downloaded to the partition that the ECU has not activated.

975728 v2 **Support for write while read**

Version history:
(2):

Purpose: To ensure hardware is supporting features required for SWDL

Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 712520

The ECU shall support "Read-While-Write" functionality in the memory stack, it is not allowed to use preemption. This shall be supported by hardware and software.

975724 v2 **Switch between active and inactive partitions**

Version history:
(2):

Purpose: To define the method of switching
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 712353

The ECU that implements the AB partition should support the run partition switch process, which is used to switch the run partition from the current run partition to another partition.

For example, if the ECU is currently running on partition A, after this switching process, the ECU should run on partition B after resetting, and vice versa. If the switchover fails, the ECU should keep running in the runnable partition before the switchover. That is, if the switchover from partition A to partition B fails, the final running partition is partition A software version.

975721 v2 **Upgrade performed in the background of application context**

Version history:
(2):

Purpose: To define the impact of OTA behavior on driving
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 712334

The ECU shall support AB dual partition update, and support the updating of inactive partition when another partition is running. During the ECU reprogramming process, the system should operate normally, its function and performance shall not be affected by the reprogramming. After the inactive partition is updated successfully and receive diagnostic command 0255+01, the ECU shall mark the partition switching flag and switch to the updated partition when perform hard reset.

4.11.11.4 Dual Bank SWDL Based On MCU

•Introduction

4.11.11.4.1 Introduction

Described the requirements for dual bank SWDL based on MCU. In case of conflicts with [int_1] General Software Download, this chapter shall prevail.

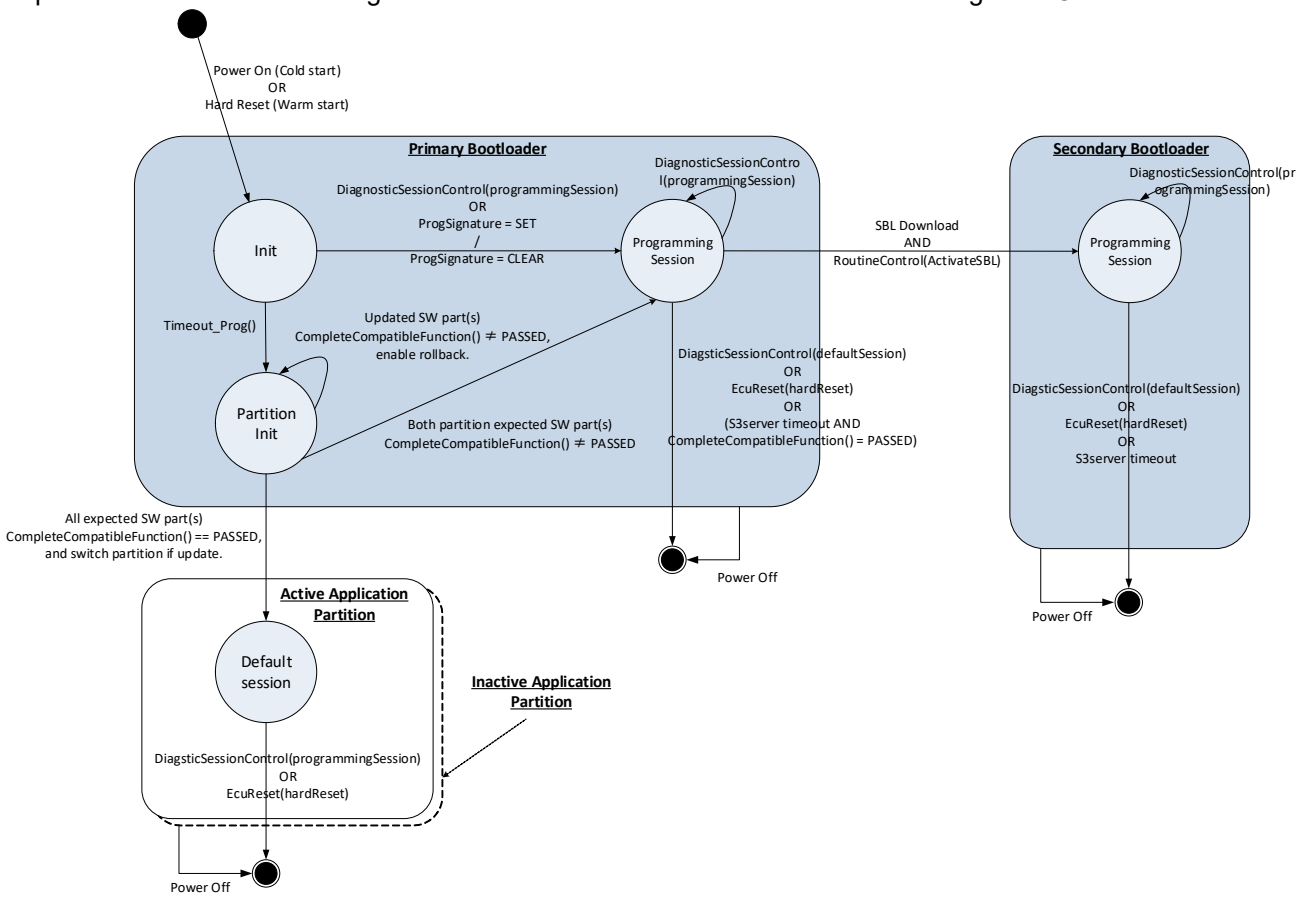
•Scheme #1

4.11.11.4.2 Scheme #1

This section describes that closer to a dual bank scheme based on extended memory.

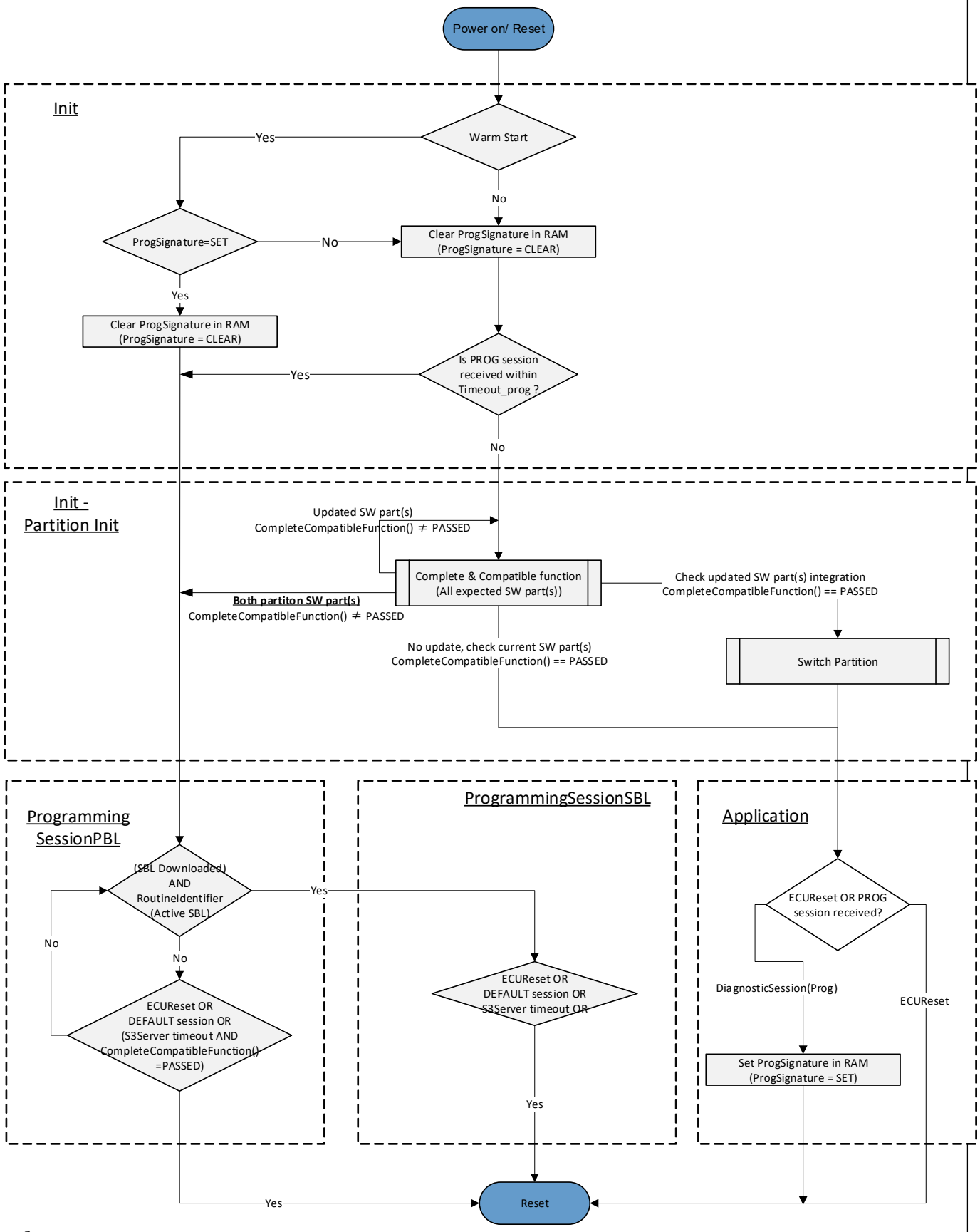
4.11.11.4.2.1 State diagram

There are two bootloaders required, the primary bootloader and the secondary bootloader. These are described in more detail in [Int_1] General Software Download. The dual bank bootloader state diagram, the figure below is a complement to the flow chart diagram described in section Bootloader flow chart diagram - Overview.



4.11.11.4.2.2 Flow chart diagram

The flow chart shown in Figure: Dual bank ECU bootloader flow chart - Overview is applicable and supplementary for the dual memory ECU. The flow chart is divided in states.
The focus of this diagram is on “Init - Partition Init”, and other state requirements may conflict with Bootloader – UDSonXX requirments, the Bootloader - UDSonXX requirements shall prevail.
Init - Partition Init state is further described in the following chapters.


Power on/ Reset
Init
Warm Start
Yes
No
ProgSignature=SET
No
Clear ProgSignature in RAM
(ProgSignature = CLEAR)
Yes
Clear ProgSignature in RAM
(ProgSignature = CLEAR)
Is PROG session received within Timeout_prog ?
Yes
No
Init - Partition Init
Updated SW part(s)
CompleteCompatibleFunction() ≠ PASSED
Complete & Compatible function
(All expected SW part(s))
Check updated SW part(s) integration
CompleteCompatibleFunction() == PASSED
Both partiton SW part(s)
CompleteCompatibleFunction() ≠ PASSED
No update, check current SW part(s)
CompleteCompatibleFunction() == PASSED
Switch Partition
Programming SessionPBL
ProgrammingSessionSBL
Application
(SBL Downloaded) AND RoutineIdentifier (Active SBL)
Yes
No
No
ECUReset OR DEFAULT session OR (S3Server timeout AND CompleteCompatibleFunction() =PASSED)
ECUReset OR DEFAULT session OR S3Server timeout OR
Yes
ECUReset OR PROG session received?
DiagnosticSession(Prog)
ECUReset
Set ProgSignature in RAM
(ProgSignature = SET)
Yes
Reset

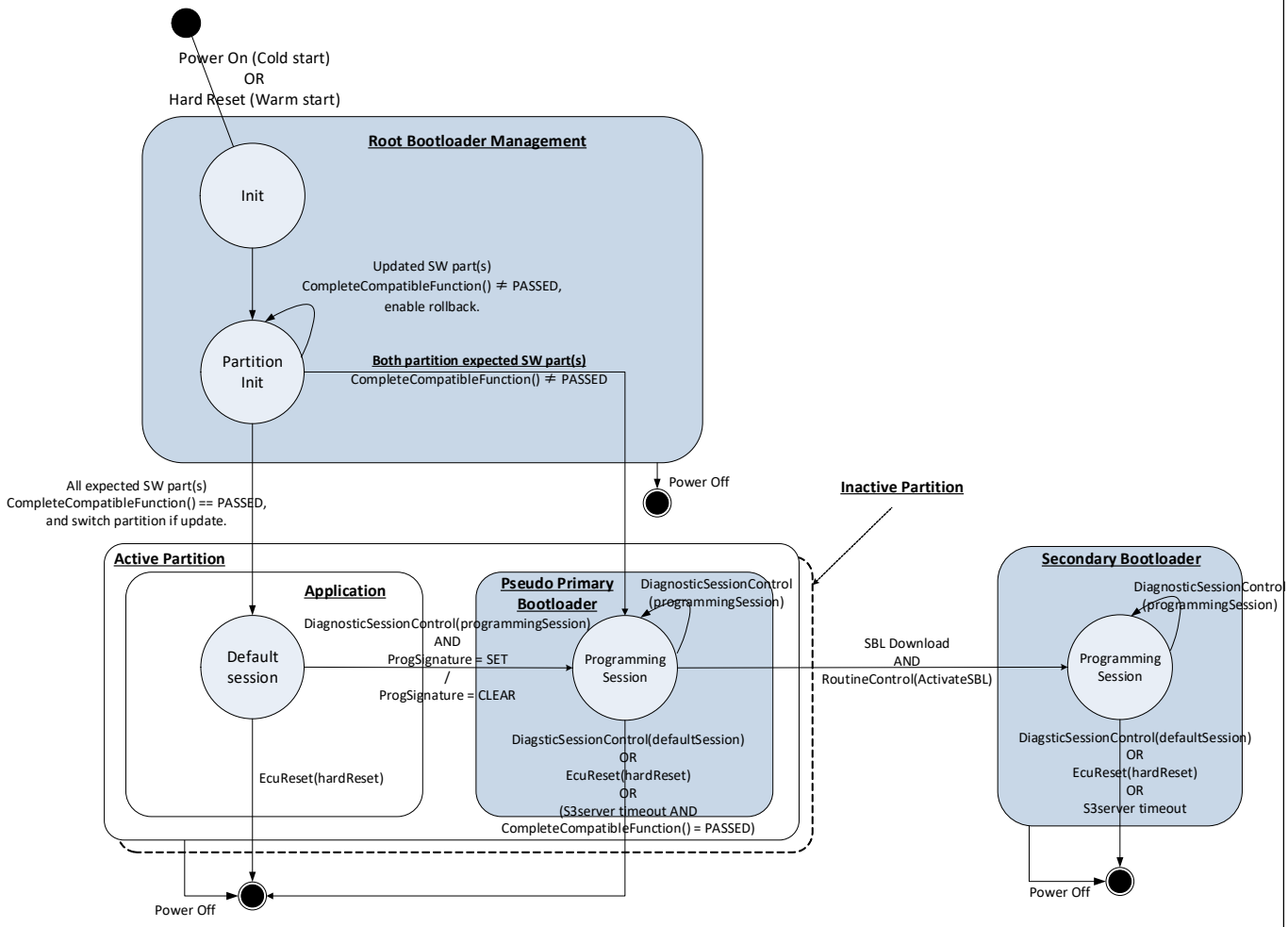
•Scheme #2

4.11.11.4.3 Scheme #2

This section describes that closer to a hardware based dual bank schemes with dual bootloader. It is possible that no need to reset ECU when programming session is entered.

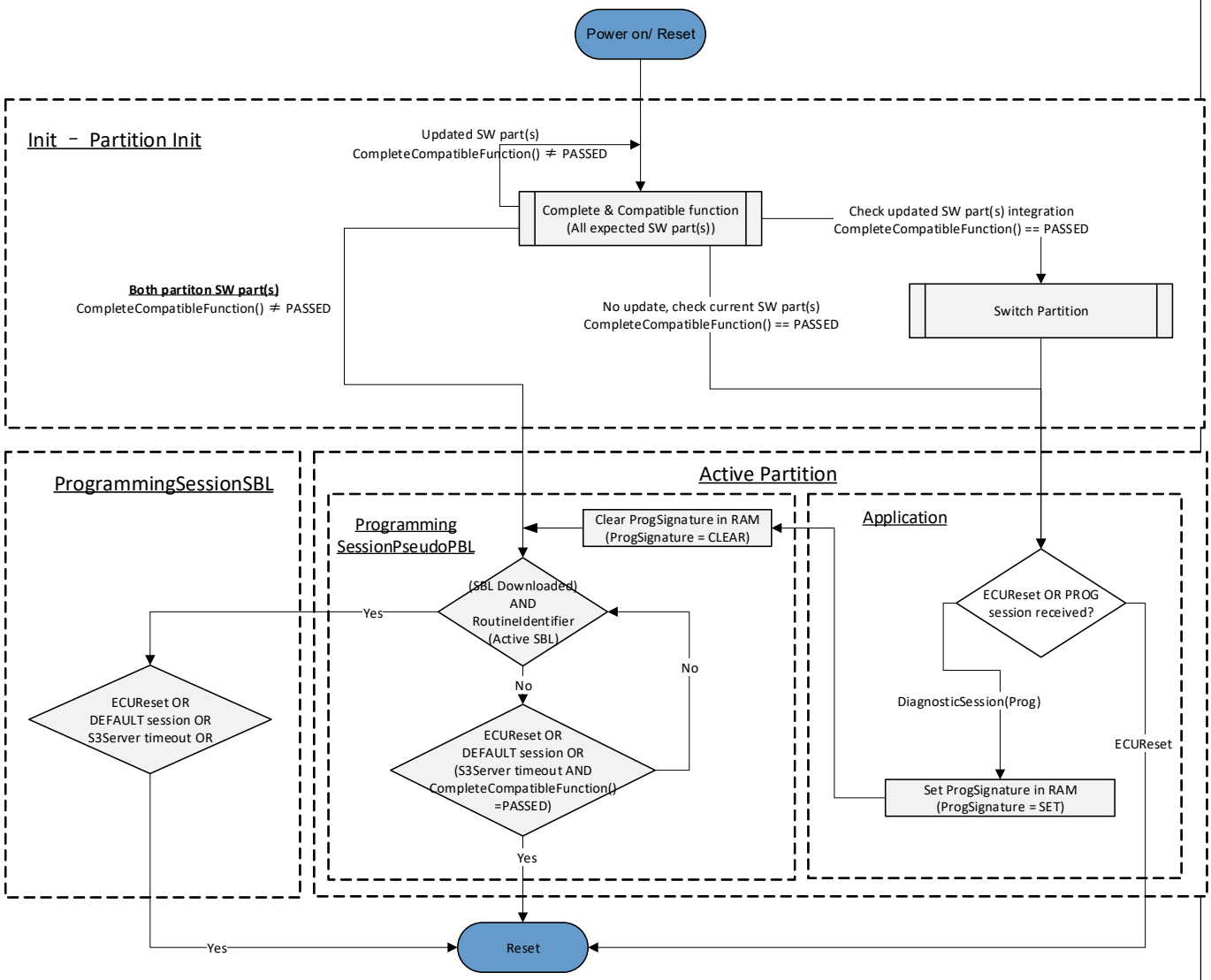
4.11.11.4.3.1 State diagram

There are three bootloaders required, the root bootloader management, the primary bootloader and the secondary bootloader. These are described in more detail in [Int_1] General Software Download. The dual bank bootloader state diagram, the figure below is a complement to the flow chart diagram described in section Bootloader flow chart diagram - Overview.



4.11.11.4.3.2 Flow chart diagram

The flow chart shown in Figure: Dual bank ECU bootloader flow chart - Overview is applicable and supplementary for the dual memory ECU. The flow chart is divided in states.
The focus of this diagram is on “Init - Partition Init”, and other state requirements may conflict with Bootloader – UDSonXX requirments, the Bootloader - UDSonXX requirements shall prevail.
Init - Partition Init state is further described in the following chapters.



•**Requirement**

4.11.11.4.4 Requirement

692237 v1 **Dual Bank Scheme**

Version history:
(1):

Purpose: To consider the possibility and convenience of Update Module updates.
Source:
Verification Method: Analysis
Legacy ID:

Based on the description in the above section, it is recommended to use scheme #2.

Note: Suppliers need feedback on which scheme to support when shaking hands on this requirement.

702988 v1 **Pseudo Primary Bootloader - Programming Session**

Version history:
(1):

Purpose: Different schemes will have different performances. Clear requirements have been made for specific scheme here.

Source:

Verification Method: Test

Legacy ID:

In scheme #2, the application shall transit to “PseudoPrimaryBootloader - ProgrammingSession” state when programming session is received in application, meaning still running in the same partition as the application program but perform bootloader function. Within the “PseudoPrimaryBootloader - ProgrammingSession” state the ECU shall support the following:

- All the programmingSession requirements, including re-programming itself.
- SWDL gateway requirement if need.
- SWDL general specification requirements.
- Maintain power supply without causing a restart during session transition.
- Ethernet connection may not be affected, i.e. TCP connection.
- Disable non diagnostic communication e.g. network management messages between etc. Additionally, the ECU shall ensure the monitoring and logging of all diagnostic trouble codes (DTCs) is disabled.
- Disable normal application functionality.

4.11.11.5 Dual Bank SWDL On POXIS System

•Introduction

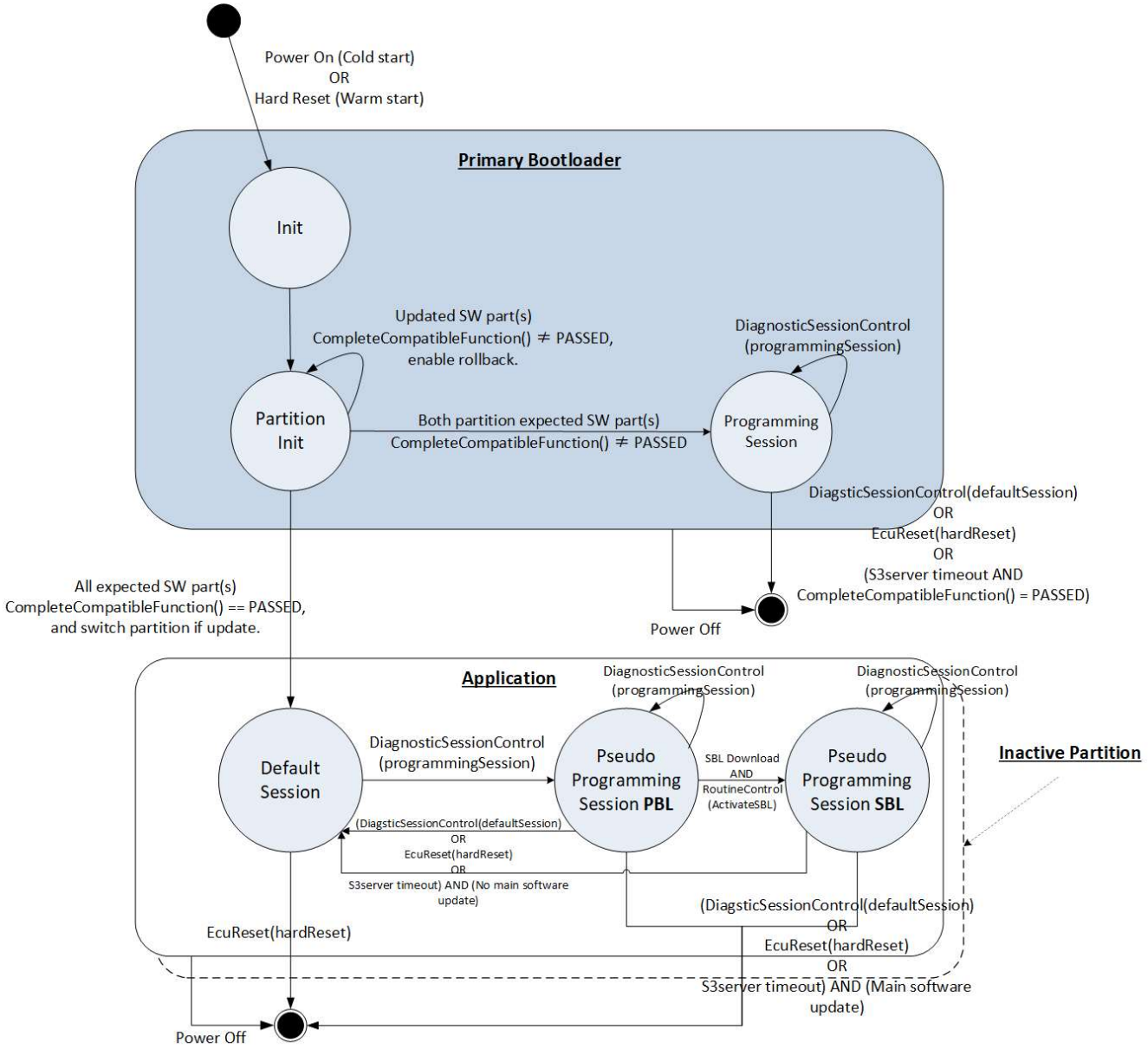
4.11.11.5.1 Introduction

Described the requirements for dual bank SWDL based on SoC. In case of conflicts with [int_1] General Software Download, this chapter shall prevail.

•State diagram

4.11.11.5.2 State diagram

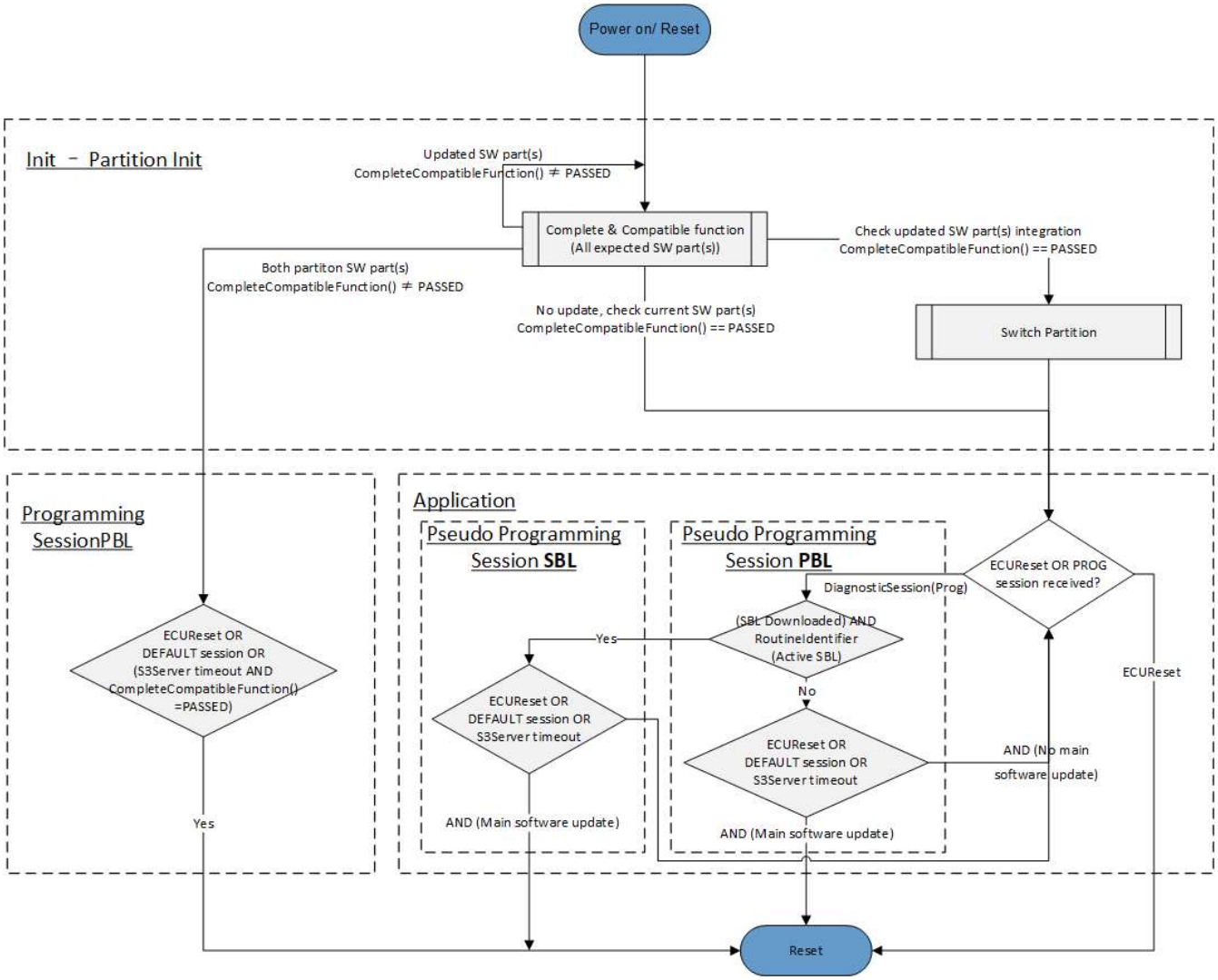
There are one bootloaders required, the primary bootloader. These are described in more detail in [Int_1] General Software Download. The dual bank bootloader state diagram, the figure below is a complement to the flow chart diagram described in section Bootloader flow chart diagram - Overview.



•Flow chart diagram

4.11.11.5.3 Flow chart diagram

The flow chart shown in Figure: Dual bank ECU bootloader flow chart - Overview is applicable and supplementary for the dual memory ECU. The flow chart is divided in states.
The focus of this diagram is on “Init - Partition Init” and "Application", and other state requirements may conflict with Bootloader – UDSonXX requirments, the Bootloader - UDSonXX requirements shall prevail.
Init - Partition Init state and Application state is further described in the following chapters.



•Requirement

4.11.11.5.4 Requirement

703100 v2 **Diagnostic session for PseudoProgrammingSession**

Version history:
(2): Verification Method: Test change to Analysis AND Test.

Purpose: When the ECU is in PseudoProgrammingSession state it shall act as it is in programmingSession state.

Source:

Verification Method: Analysis AND Test

Legacy ID:

The application shall enter a “PseudoProgrammingSession” state, meaning still running the application but simulate the diagnostic session change to programmingSession i.e. bootloader. Within the “PseudoProgrammingSession” state the ECU shall support the following:

- All the programmingSession requirements, including re-programming itself.
- SWDL gateway requirement if need
- SWDL general specification requirements

- Maintain the functionality active that is needed to communicate via WLAN i.e. WLAN security, TCP connection, DoIP and other software's supporting these functionalities.
- Routing of IP traffic between IP networks.
- Disable non diagnostic communication e.g. network management messages between etc. Additionally, the ECU shall ensure the monitoring and logging of all diagnostic trouble codes (DTCs) is disabled.

699068 v1 **Software Update on inactive partition**

Version history:
(1):

Purpose: Fully utilize the advantages of dual bank in the POXIS system to upgrade inactive partitions while certain applications are still available.
Source:
Verification Method: Analysis AND Test
Legacy ID:

ECU with POXIS system has the capability of dual bank, and it shall update another "inactive partition" in a pseudo programming session during download and installation.

This means that after receiving the programming session, the ECU will not enter the so-called Recovery Mode and shall still implement software data reception and final installation under the application mode of the current "active partition".

4.11.11.6 For Dual Bank ECU but Without Seamless Update

•Additional Requirements for Complete and Compatible

4.11.11.6.1 Additional Requirements for Complete and Compatible

This section describes the newly added requirements of Routine-CompleteAndCompatibleCheck implementation for dual bank ECU. And the Complete and Compatible - Validation of ECU software(s) are still described in more detail in *[Bootoalder requirements for SWDL with check memory]* .

682019 v3 **Common Software Part(s) Handle**

Version history:
(3):

Purpose: Configuration file, user data are possible to be one shared memory sector. When update these, no need to synchronize main software part(s) data (e.g.support dual bank) to inactive partition, avoiding increasing unexpected SWDL time.
Source:
Verification Method: Test
Legacy ID:

Every SWDL process, only the software part(s) in common shared memory within dual-bank ECU is updated (i.e. certificate data or configuration data) or no update, synchronization and switch will not be performed and the return value from the CompleteCompatibleFunction() shall be set to zero (0) indicating "PASSED" when handle Routine - Check CompleteAndCompatible diagnostic request.

682024 v3 **Expected Software Part(s) CompleteAndCompatible Check and Switch**

Version history:
(3):

Purpose: For dual-bank ECU, step Routine - CompleteAndCompatibleCheck is a crucial step which also includes functional requirements for software synchronization and set startup identifier.
In addition, It is necessary to identify and execute different strategies based on the upgraded software part(s) during SWDL.

Source:

Verification Method: Analysis AND Test

Legacy ID:

If software part(s) that supports dual bank has been updated or synchronization successfully, ECU will check complete and compatible for expected SW part(s) of inactive partition. Complete and Compatible - Validation of ECU software(s) is defined in *[Bootoalder requirements for SWDL with check memory]*
After the result of CompleteAndCompatibleCheck is passed, Update Module should set the "SWITCH" identifier for the next startup in this step. I.e. the target slot (B slot) is switched to a pre-activated active slot.

Note: A negative response should be returned during the process of erasing memory and transmitting data again once synchronization and CompleteAndCompatible check are completed.

682017 v3 Main Software Part(s) Update and Synchronization

Version history:
(3):

Purpose: For dual-bank ECU, to decrease the total programming time it shall be possible to replace only the specific data file that shall be replaced and not being forced to replace all software(s) that already is present in the ECU. In terms of implementation, it will synchronize the software data from the active partition to the inactive partition instead of downloading the software data again.

Source:

Verification Method: Test

Legacy ID:

Due to chip design decisions, it is difficult to independently and flexibly switch each software partition. If so, it is necessary to shake hands with GRI Basetech Team for recognition.

It is applicable for switching only the AB slot memory as a whole. After intended updating some software part(s), Update Module shall support synchronizing the non-updated software part(s) from the active partition to the inactive partitions memory sector(s).

The prerequisite for performing synchronization is that at least one but not all software part(s) that supports dual bank have been updated successfully. The sequence is as follows:
1) Successfully update any inactive partition of SW part(s) independently, such as SW part 2 B bank.

2) After receiving Routine - CheckCompleteAndCompatible diagnostic service, it shall be necessary to identify which software part(s) in the B slot have not been updated and then synchronize these corresponding program data of the A slot to the non-updated software part(s) of B slot. For example, In below figure SW part 1 in A slot is synchronized to SW part 1 in B slot and SW part 3 in A slot is synchronized to SW part 3 in B slot.

Comment: Partition data synchronization of non-updated software part(s) implicitly indicates that memory erasure needs to be performed internally by Update Module before synchronization, if necessary.

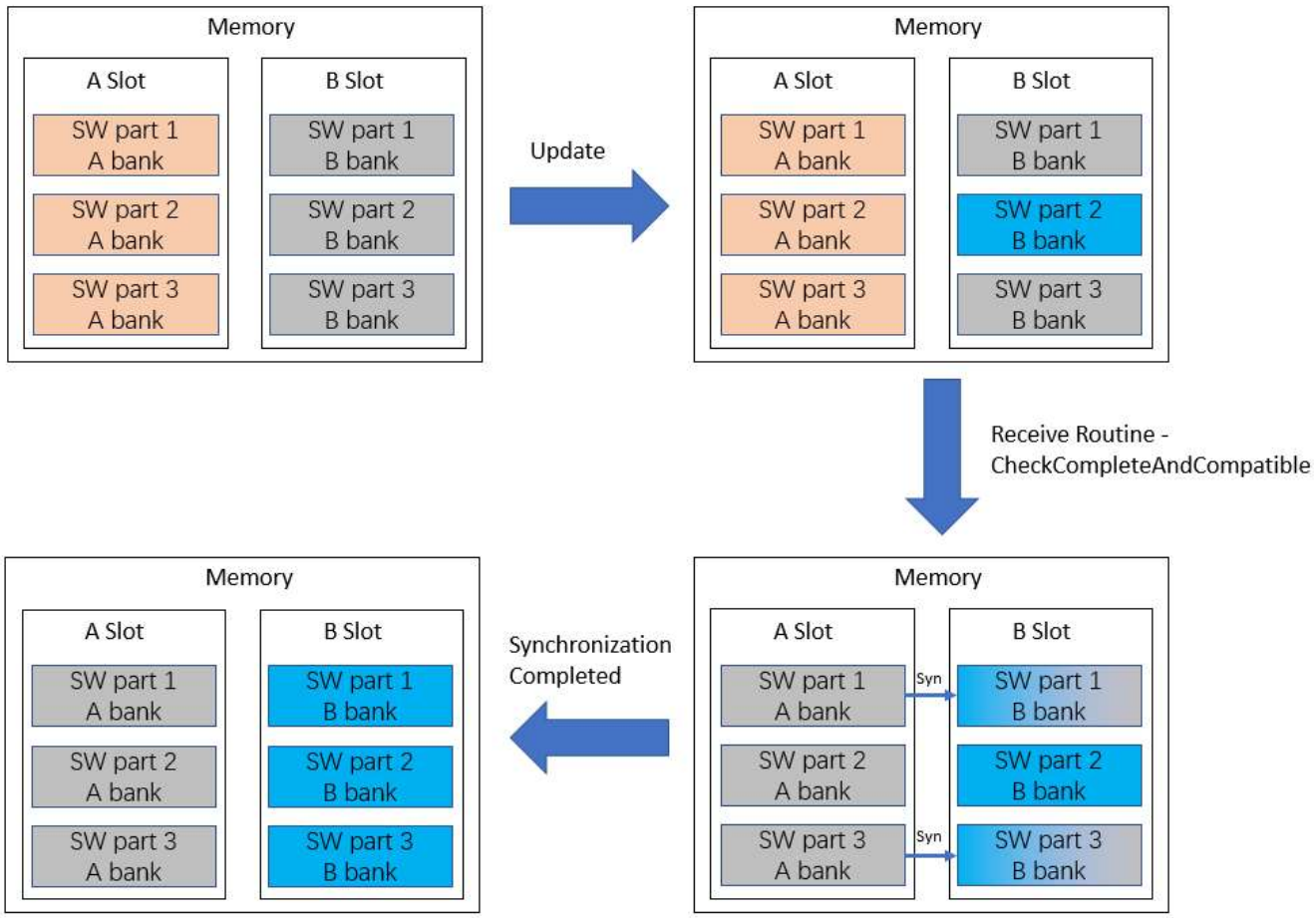


Figure: The transition of partition after receiving Routine - CheckCompleteAndCompatible

Note: Due to synchronization this routine may consume more time, which is negotiable.

4.11.11.7 External Flash A/B Requirements

•General Flash Requirements

4.11.11.7.1 General Flash Requirements

975935 v1	Rollback Requirements

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 767181

The EMMC A/B ECU needs to support the rollback through the RID0296, and automatic rollback is not allowed (After the EMMC software copyed to the internal flash, the integrity and compatibility check needs to be supported).

The rollback process needs to be supported under bootloader mode and support 27 01/02 level unlock.

975934 v1 **Software Flash Requirements**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 767176

If the ECU does not support A/B swap, an external EMMC should be used to implement the software backup and rollback.

Note:
External flash ECU needs to be flashed in boot mode,following the standard SWDL flash process.

975936 v2 **Software Synchronization Requirements**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 767182

The External Flash ECU should support the execution of software synchronization through the diagnostic command RID0255+00 and following the requirements in REQ:762484.

The software synchronization commands need to be supported under the application mode and the ECU function can't be affected.

Note:
For external flash ECU, software synchronization means copying software from the internal flash(active partition) to the external EMMC flash.

4.11.11.8 Diagnostic data for dual-bank ECU

•DID

4.11.11.8.1 DID

692287 v1 **DID - D088**

Version history:
(1):

Purpose: To enable readout the A/B partition information.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D088
Name	A/B Partition Information
Description	Read out the every partition activation status and software status in every chip. The activation Status indicates which partition is currently active, and the software status indicates whether the partition software is available/valid to switch or rollback.
Content	**SubParameter Name** — **Length, Data Type** Total number of chips supported A/B bank — [1 Byte, Unsigned] Chip#1 "A" Partition Activation Status — [1 Byte, Unsigned] Chip#1 "A" Partition Software Status — [1 Byte, Unsigned] Chip#1 "B" Partition Activation Status — [1 Byte, Unsigned] Chip#1 "B" Partition Software Status — [1 Byte, Unsigned] ..... Chip#N "A" Partition Activation Status — [1 Byte, Unsigned] Chip#N "A" Partition Software Status — [1 Byte, Unsigned] Chip#N "B" Partition Activation Status — [1 Byte, Unsigned] Chip#N "B" Partition Software Status — [1 Byte, Unsigned]
Default	/
Security access required	/

Other Session		Programming Session	
Read	Write	Read	Write
M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

4.11.11.9 Non-sensible Programming For OTA

•Introduction

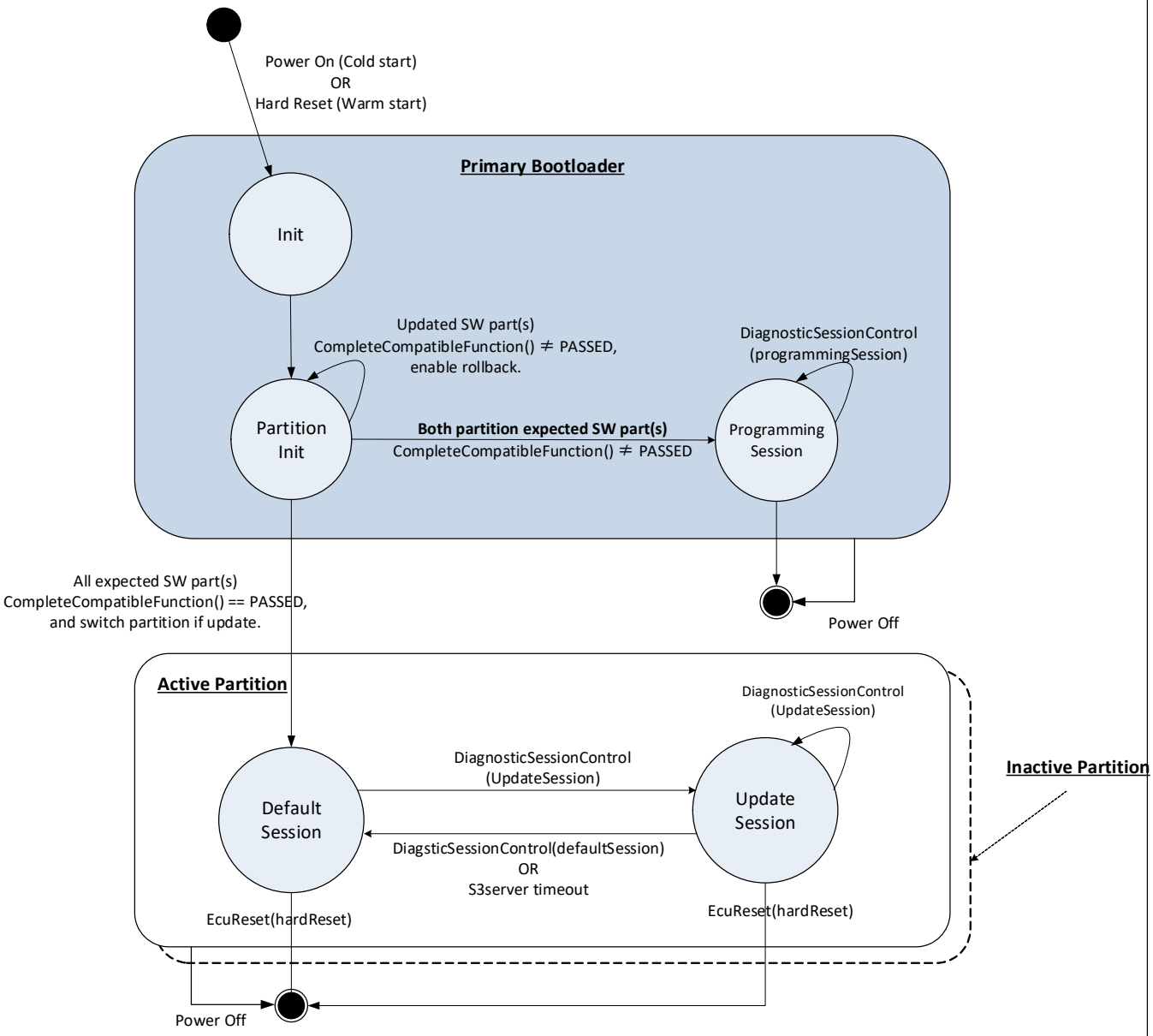
4.11.11.9.1 Introduction

The Seamless Update approach shall introduce a generic mechanism to update ECU software during runtime. While the current ECU software is executed and fully available from functional point of view (e.g. during driving) a new ECU software shall be programmed in the background (installation phase). At the time of the installation, which can be interrupted and continued over several driving cycles, the authenticity and integrity of the new SW shall be verified. In case of positive verification results, the ECU shall be able to activate the new SW. The activation of the SW shall always require a special ECU mode (e.g. boot), hence the activation of new SW must not be started or even executed while driving. The activation shall be done in a vehicle safe-state, e.g. stand- still, engine off and applied parking brake. In case of detected anomalies or errors after or during activation of the new SW, the ECU shall be able to realize an ECU internal rollback to previous SW. ECU internal rollback implies the approach, that the previous SW is still present on the ECU and can be re-activated.

•State diagram

4.11.11.9.2 State diagram

There are one bootloaders required, the primary bootloader. The focus of this figure is on the actions executed during Update Session.



•General Requirements

4.11.11.9.3 General Requirements

682075 v2 **Buffer handling during Seamless Update**

Version history:
(2):

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

Usually, data blocks are transmitted by the Client using the 36 service, and from the perspective of the ECU, its DCM module will receive these data blocks from the communication protocol stack.

Because Update process shall run and use the UDS protocol with the lowest priority in Dcm, and reference its own data buffer within the Dcm module.

The maximum data byte received by ETH ECU shall support 16kB+2 (SID).

The maximum data byte received by CAN ECU shall support 4kB (including SID)

975911 v1 **Delta update and total update**

Version history:
(1):

Purpose: To define the upgrade modes supported by the ECU
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713026

ECU should support delta update and total update. Before update，ECU should determine whether the update package is a delta update package，if it is，ECU should do the delta reduction，and then execute update；if it is not delta update package，ECU execute update directly.

Note1: The delta update in OTA process will base on the software of active partition.
Note2: More delta information detail in In-Place Reconstruction of Delta Files(Req 6411).

985756 v3 **Diagnostic services supported**

Version history:
(3):

Purpose: Define the diagnostic services to be supported in OTA session mode
Source:

Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713022

The following services shall be supported in the application in OTA session
●DiagnosticSessionControl (10)
●ECUReset (11)
●ReadDataByIdentifier (22)
●SecurityAccess (27)
●Authentication (29)
●TesterPresent(3E)
●RoutineControl (31)
●RequestDownload(34)
●TransferData (36)
●RequestTransferExit (37)
●RequestFileTransfer (38)
●ClearDiagnosticInformation (14)
●CommunicationControl (28)
●ControlDTCSetting (85)

Note 1: The services $34, $36, $37, $38 are not applicable to ECU that supports distributed downloads.

975910 v1 **Ensure non diagnostic communication**

Version history:
(1):

Purpose: To both have diagnostic and application communication between tester and ECU when executing the reprogramming in OTA session.
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713024

Whenever an ECU is executing reprogramming in OTA session, it shall ensure that non diagnostic communication e.g. network management messages between ECUs are normal.

975909 v1 **Monitoring and logging of DTCs**

Version history:
(1):

Purpose: To logging and monitoring of DTCs when the reprogramming in OTA session executing.
Source:
Verification Method: Test
Legacy ID:

Zeekr Requirement ID: 713023

The ECU shall ensure that the monitoring and logging of all diagnostic trouble codes (DTCs) is enabled when executing the reprogramming (OTA Session).

682066 v2 Normal Operation during Update Session

Version history:
(2):

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

During the execution period of the Seamless Update Process, i.e. Update Session, the ECU shall operate normally in all functions, including functional logic, wake-up sleep strategy, communication, and fault detection.

682079 v2 Processing the data buffer

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID:

To write data to inactive program partitions robustly, Dcm needs to receive a complete blockSequenceCounter from the TransferData service before programming the data into memory. Otherwise, the received data should be discarded.

During the processing of data returning a positive response, due to UDS protocol preemption, the fully received data block can still be programmed into memory in the background, even if there is a possibility of not being able to return a positive response for transferData request, as the client will start Seamless Update process again to read the last successfully programmed address.

975912 v1 Requirement for delta update tool

Version history:
(1):

Purpose: To define the requirements for the tool
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713027

If ECU support delta update Supplier should provide a delta update tool that can generate delta update package based on The last two committed software versions.

682085 v1 **Services - RequestDownload**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID:

To continue transmitting software data from the last successfully programmed memory address, the ECU shall support receiving and processing download requests for successfully programmed memory addresses and effective memory lengths.

975913 v3 **action retry**

Version history:
(3):

Purpose: ECU support action retry
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713707

If a request from OTA Master failed the ECU shall support action retry.

•OTA SWDL Sequence

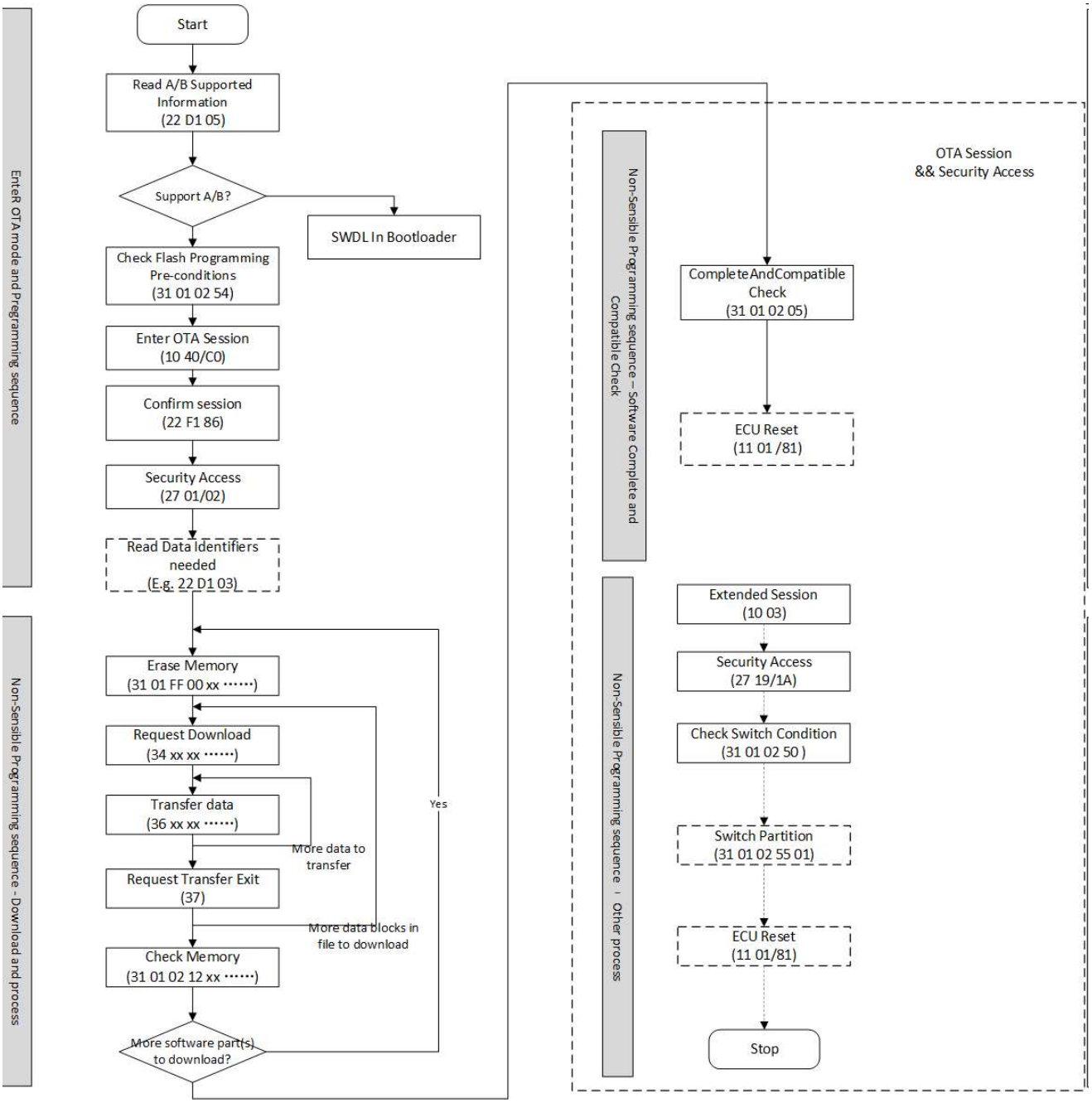
4.11.11.9.4 OTA SWDL Sequence

985754 v5 **OTA SWDL Sequence**

Version history:
(5):

Purpose: Define the OTA SWDL Sequence
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713007

The SWDL sequence shall be perform as below during the OTA process



4.11.11.9.4.1 Enter OTA Session

975917 v2 **Condition Check**

Version history:
(2):

Purpose: To check whether the vehicle meets the programming conditions in application mode
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement 713020

ID:

OTA Master must check whether the ECU meets the programming conditions before enter OTA session, the check programming pre-conditions routine with routine identifier 0x0254 shall be implemented as defined in Global Master Reference Database.

985757 v2 **Confirm session**

Version history:
(2):

Purpose: Confirm the session
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713710

OTA Master will confirm the session after the action of enter OTA session to ensure all target ECU in a correct session.

ECU shall support to response current diagnostic session when receive ReadDataByIdentifier service for 0xF186 (ActiveDiagnosticSessionDataIdentifier), e.g. OTA session (0x40).

985755 v2 **OTA Session**

Version history:
(2):

Purpose: Define a session mode for non-sensible programming
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713709

The ECU shall support a special session mode for non-sensible programming and the service is 10 40.

Note: This mode allows the A/B Swap ECU to perform software updates for the inactive partition while the application software is running in the active partition.

975916 v3 **Read AB Information**

Version history:
(3):

Purpose: Read out which ECUs support flashing during driving
Source:

Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713016

The AB partition information can be read through DID $D105 (22 service) and the response shall include as below
1 ECU support non-sensible Programming
2 ECU not supported non-sensible Programming

A data record with identifier 0xD105 shall be implemented exactly as defined in Global Master Reference Database.

4.11.11.9.4.2 Security Access

975919 v1 **Security access in OTA programming mode**

Version history:
(1):

Purpose: To avoid illegal or unauthorized programming
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713021

ECU reprogramming shall be protected by a security check to prevent from inadvertent and unauthorized programming. ECU unlock process follows seed and key algorithm and service $27 from UDS. The unlocking process in OTA mode is shown in Security Access algorithm defined by Base Tech.

4.11.11.9.4.3 Data Transfer

975920 v2 **Data Transfer**

Version history:
(2):

Purpose: To define memory address based download sequence
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713025

OTA master will read the resume information (DID 0xD103) and erase form the breakpoint with Routine control 0x FF00.and then transfer data through service 34,36,37.

Then ECU can be reprogramming again using request download, transfer data and transfer exit service. After each download files, its authenticity shall be checked using the routine check memory ($31 01 02 12).

After all the data is being downloaded ECU shall start application validation process.This application validation shall be done using routine control check complete and compatible ($31 01 02 05).

4.11.11.9.4.3.1 Transfer Resume

985758 v3 **Breakpoint record**

Version history:
(3):

Purpose: Define the way of resume transfer data
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713741

The ECU shall record the breakpoint and the breakpoint only can be generate in service 36. the breakpoint information shall be store in non-volatile memory and can be read out through service 22 D1 03.

Note:
The response to this D103 is 6 bytes, the first byte refers to 14229-1 Appendix H, the second byte is reserved, and the last four directly fill in the specific breakpoint address.
Example:
1 The response is “62 D1 03 15 00 22 33 44 FF”.
2 The ECU provides the information the ECU has memory size of 256bytes and total size of 1024Gb.
3 The last data which was written ended at address 0x00223344FF.

If the ECU supports resume from breakpoint (in OTA session), the block address information needs to be arranged in ascending order.

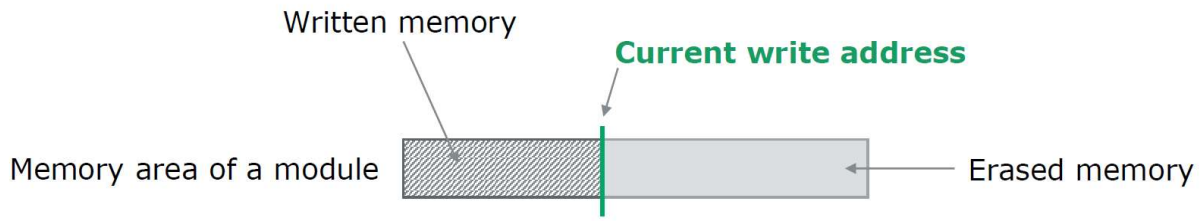
975923 v1 **Intended Shutdown –Suspend & Resume of a Download**

Version history:
(1):

Purpose: Define the way of resume
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713028

Downloading a large software update may require several driving cycles to complete the transfer.

Suspending & resuming the download is required. During a transfer, if a shutdown of software download stack is triggered the Current Write Address is stored in non-volatile memory. On the next start the download can resume at the stored write address.

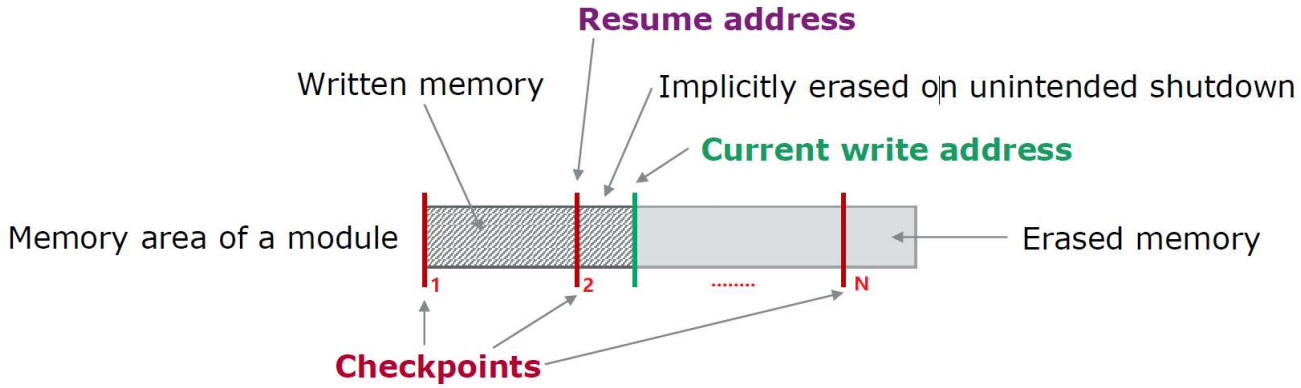


975922 v2 **Unintended Shutdown –Resume at the latest Checkpoint**

Version history:
(2):

Purpose: Define the way of data resume
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713029

The download transfer may be interrupted at any time –there is not enough time to store the Current Write address in non-volatile memory. The download can continue at the Resume Address which is the address of the last reached Checkpoint.e.g checkpoint 2. and The written data after the Resume Address is implicitly erased.The number of check points is based on the size of the ECU, and it is recommended to design the check points at a ratio of 10%.



Note1:For software that has been flashed in the inactive area, it cannot be lost due to reset or hibernation, or session switching or power loss.
Note2:If the breakpoint is just between two VBFs or ECU have not be flash, maybe there is no breakpoint and When the master reads the breakpoint, the ECU can reply to the default value of 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF .

4.11.11.9.4.3.2 Erase

975924 v2 **Erase form breakpoint**

Version history:
(2):

Purpose: Define the erasure capability of the ECU
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713746

If there has a breakpoint OTA master needs to read the resume information (DID 0xD103) and erase form the breakpoint with Routine control 0x FF00. the ECU shall respond with the resume address and support erasing form the breakpoint.

4.11.11.9.4.4 AB Partition Switch

975927 v2 **Automatic partition switching behavior**

Version history:
(2):

Purpose: To avoid an unexpected ECU restart
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713034

In order to support flexible vehicle flash strategy, the ECU that supports AB partition must ensure that the current running partition application is still valid and reliable after programming. The automatic partition switching behavior must be avoided. Partition switch can only be allowed after diagnostic request (RID:0255+01) is received.

975925 v2 **Check switch conditions**

Version history:
(2):

Purpose: To define the conditions for the ECU switch partition
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713041

When all ECUs have completed data download and check, OTA master will check whether the conditions of ECU switching to inactive partition are met.

975926 v6 **Switch partition**

Version history:
(6):

Purpose: To define A/B Swap Solution
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713038

If the conditions of all ECUs switching to inactive partition are met, OTA master will send switch requests (RID：0255+01) to each target ECU . After receiving a routine request switch partition, the ECU shall set a switch flag (inactive partition) and switch when received 11 81 or 11 01in APP or bootloader.
Reprogram in OTA session, restart should not result in partition switching before ECU receiving a routine request to switch partitions.

Note1: If switch failed, the flag should be cleared and keep to run the current activate partition.
Note2:If switch sucessed, the flag also should be cleared.
Note3:The flag should not be lost when the vehicle power-down.
Note4:ECU can switch when Abnormal power lost and restart.
Note5:If the hard reset command is not received in the Non-sensible programming process, the ECU needs to perform a partition switch on the next power-on cycle.
Note6：This flag can be set separately from the non-sensible Programming.

4.11.11.9.4.5 Clear DTC

975928 v1 **Clear DTC**

Version history:
(1):

Purpose: Clear DTCs that generate in programming
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 713718

After all ECUs Partition switched, OTA master will clear DTC with service 14.

•CompleteAndCompatible Check

4.11.11.9.5 CompleteAndCompatible Check

682129 v3 **CompleteAndCompatible Check**

Version history:
(3):

Purpose: For dual-bank ECU, to decrease the total programming time it shall be possible to replace only the specific data file that shall be replaced and not being forced to replace all software(s) that already is present in the ECU. In terms of implementation, it will synchronize the software data from the active partition to the inactive partition instead of downloading the software data again.

Source:

Verification Method: Test

Legacy ID:

CompleteAndCompatible Check mainly performs complete and compatible Check, in addition there are other operation defined as follows.

As chip design decisions, it is difficult to independently and flexibly switch each software partition. If so, it is necessary to shake hands with GRI Basetech Team for recognition.

It is applicable for switching only the AB slot memory as a whole. After intended updating some software part(s), Update Module shall support synchronizing the non-updated software part(s) in the active partition to the inactive partitions memory sector(s).

The prerequisite for performing synchronization is that at least one but not all software part(s) that supports dual bank have been updated successfully. The sequence is as follows:
Sequence:
1) Successfully update any inactive partition of SW part(s) independently, such as SW part 2 B bank.

2) After receiving Routine - CheckCompleteAndCompatible diagnostic service, it shall be necessary to identify which software part(s) in the B slot have not been updated and then synchronize these corresponding program data of the A slot to the non-updated software part(s) of B slot. For example, In below figure SW part 1 in A slot is synchronized to SW part 1 in B slot and SW part 3 in A slot is synchronized to SW part 3 in B slot.

Comment: Partition data synchronization of non-updated software part(s) implicitly indicates that memory erasure needs to be performed internally by Update Module before synchronization, if necessary.

3) Check CompleteAndCompatible for expected SW part(s) of inactive partition. Complete and Compatible - Validation of ECU software(s) is defined in *[Bootoalder requirements for SWDL with check memory]*

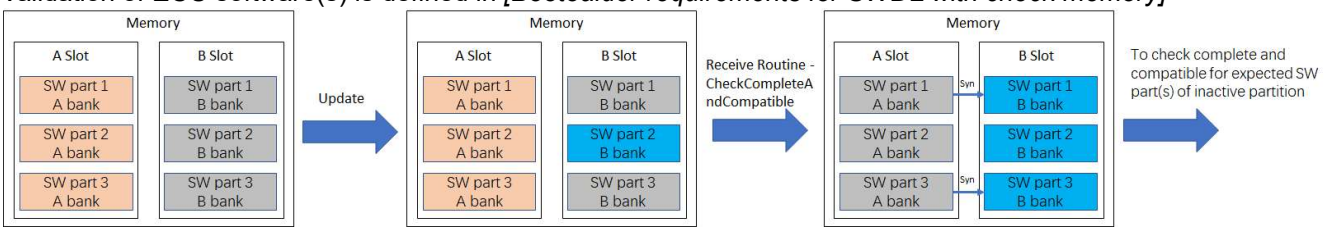


Figure: The transition of partition after receiving Routine - CheckCompleteAndCompatible

Note1: Due to synchronization, this routine may consume more time, which is negotiable.

•Erase & write Inactive program partition

4.11.11.9.6 Erase & write Inactive program partition

682067 v2 **Access and erase of inactive program partitions**

Version history:
(2):

Purpose: No longer designing the secondary bootloader, cancel the process of downloading and activating the secondary bootloader.
Source:
Verification Method: Test
Legacy ID:

Update process needs to obtain read and write access to inactive program partitions at runtime, and supports erasing and rewriting of inactive program partitions.

•Software download and process for distributed method

4.11.11.9.7 Software download and process for distributed method

682069 v1 **Software download and process for distributed method**

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

Distributed method means that the software part(s) transfer does not use UDS protocol, may be other protocols used. It is important that process the software part(s) downloaded and write data to inactive partition (i.e. installation).
The entire process will last for multiple operation cycles, so the ECU needs to know and record where it is installed each time.

•Update process status

4.11.11.9.8 Update process status

682680 v3 **Seamless Update process status**

Version history:
(3):

Purpose: To describe the states during entire seamless update process as a prerequisite for specific operations.
Source:
Verification Method: Test
Legacy ID:

The status of Seamless Update process shall be stored in non-volatile memory.
It is possible to read out the status of whole Seamless Update process for ECU in default, extended and Update Session.

Value (hex)	Status	Description

0x00	Idle	No software downloads or installations have been conducted.
0x01	Processing	Software processing
0x02	Processed	Software processing is completed and verified.
0x03	Processing failed	Software processing is failed
0x04	CompleteAndCompatible Checking	Checking complete and compatible of expected software part(s).
0x05	Ready	The complete and compatible check of expected software part(s) in the target partition has been done.
0x06	Ready failed	The complete and compatible check of expected software part(s) in the target partition is not Passed.
0x07	Activating	Software Flag Activation has been trigger.
0x08	Verifying (Option)	Optional, if external memory is used, it should be copied to the internal program Flash during the activation process and subjected to secondary verification.
0x09	Activating succeeded	Switch partition identifier is set successfully.
0x0A	Activating failed	NA
0x0B	Client - Rolling back	Actively switching partition flags when process is in Idle
0x0C	Rolling back succeeded	Switch partition identifier is set successfully.
0x0D	Rolling up failed	NA
0x0E	Cleaning up	The software data of inactive partition shall be invalidated and cleared. Thise is a intermediate state which may not be read.
0x0F	Synchronizing	Software synchronization is executing.
0x10	Synchronizing succeeded	Software synchronization is successful.
0x11	Synchronizing failed	Software synchronization is failed.

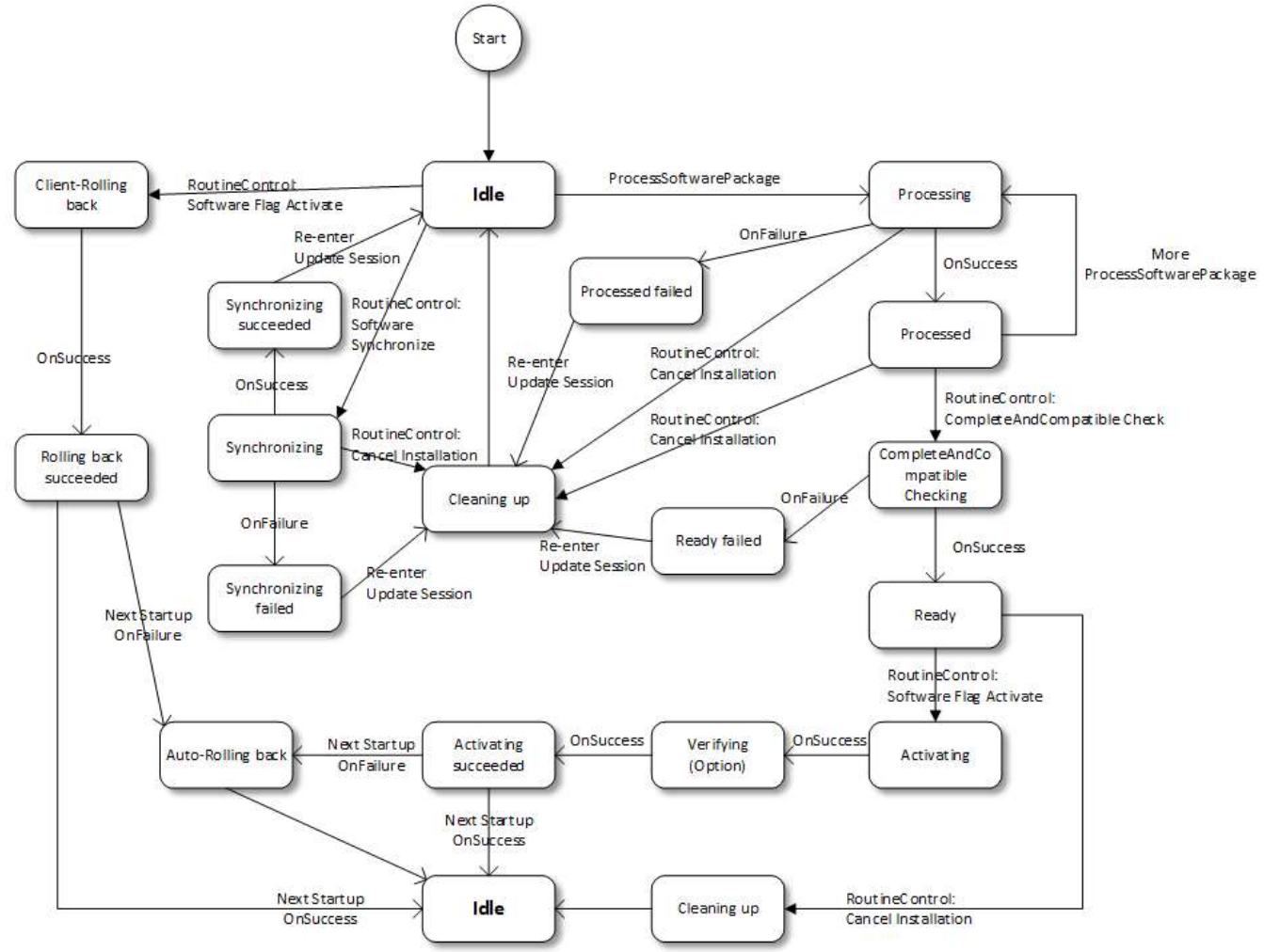


Figure - The transition of process status

682668 v3 Software process status

Version history:
(3):

Purpose: To describe the states during software processing.
Source:
Verification Method: Test
Legacy ID:

It is possible to read out the status of software download and process for ECU in default, extended and Update Session.

Status	Description
Init	Initialization status, indicating no software processing.
Wait	Software processing resting state.
Processing	During software processing, including download (transmission), installation and verification.
Processed	Software processing is completed and verified.

4.11.11.10 Non-sensible Programming Requirement For Private ECU

•General Requirements

4.11.11.10.1 General Requirements

975933 v1 **Exception Handling After Failure**

Version history:
(1):

Purpose: Define the Public ECU Exception handling.
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 807453

If the private ECU programming fails, the Public ECU Master should not switch partition. the startup partition flag of the Public ECU Master is automatically cleared and Public ECU Master shall stay in current partition.

If the Public ECU Master fails to switch partition, the startup partition flag of the Public ECU Master is automatically cleared and Public ECU Master will stay in current partition.

Note:Version Information(DID 0xF1AE) stored in the Public ECU Master should remain the current partition.

975931 v1 **Flash Way of Private ECU Not Support AB Swap**

Version history:
(1):

Purpose: Define the way of private ECU of not support AB swap update
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 807451

For the Public ECU Master supports AB Swap and the private ECU does not support AB Swap, FOTA Master will trigger the Public ECU Master to upgrade the private ECU through the UDS service $31(RID 0x0294+01) in bootloader after 27 service(01/02) unlock, and the upgrade result of the private ECU needs to be feed back to the Public ECU Master. The FOTA Master can check the update result of the private ECU through the $03 subservice of $31 service.

Note1: If the private node is not updated, the Public ECU Master also needs to respond positively to RID:0294+01, and the installation progress will be directly fed back to 100%.
Note2: If only flash the private node, the Public ECU Master needs to comply with the standard non-sensible flash process, but this flag should not be set.

975930 v1 Private ECU Support AB Swap

Version history:
(1):

Purpose: Define the way of private ECU of support AB swap update .

Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 807450

If the Public ECU Master supports AB Swap and the private ECU also supports AB Swap, the private ECU must support no-sensitive programming. For details about the programming process can reference the BT requirements or defined by suppliers.

Note:The switch between the AB partition of a private node must follow the Public ECU Master including flash and rollback.

975932 v1 Rollback of Private ECU

Version history:
(1):

Purpose: Define the way of rollback private ECU.
Source:
Verification Method: Test
Legacy ID:
Zeekr Requirement ID: 807452

To facilitate the rollback of private ECU(s), the Public ECU Master must store the private ECU's programming software.
FOTA Master will trigger the Public ECU Master to rollback the private ECU through the UDS service $31(RID 0x0294+00) in bootloader after 27 service(01/02) unlock.

Note: If the private node is not rollback, the Public ECU Master also needs to respond positively to RID:0294+00, and the installation progress will be directly fed back to 100%.
Note2: If only rollback the private node, the Public ECU Master needs to comply with the standard non-sensible flash process, but this flag should not be set.

975929 v1 Way of Update Private ECU

Version history:
(1):

Purpose: Define the way of all private ECU update.

Source:
Verification Method: Analysis
Legacy ID:
Zeekr Requirement ID: 807449

Private ECU not addressable from a client/tester point of view, so tester cannot directly upgrade private ECU. If this private ECU have programmable memory areas that supports software download, the programming file of the private ECU needs to be packaged into the VBF file of the Public ECU Master.After receiving the complete VBF file, the Public ECU Master needs to extract the data of the private ECU in the VBF.

4.11.11.11 Diagnostic data related to Non-sensible Programming

•DID

4.11.11.11.1 DID

692300 v1 **DID - D084**

Version history:
(1):

Purpose: To trace software process status during seamless update.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D084
Name	Software Process Status during Seamless Update Process
Description	Refer to ReqID 682668.
Content	**SubParameter Name** — **Length, Data Type** Software Management Stauts — [1 Byte, Unsigned]
Default	/
Security access required	/

Other Session		Programming Session	
Read	Write	Read	Write
M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

692299 v1 DID - D086

Version history:
(1):

Purpose: To trace the entire flow of seamless update process.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D086		
Name	Seamless Update Process Status		
Description	Refer to ReqID 682680.		
Content	**SubParameter Name** Update Process Stauts	**Length, Data Type** [1 Byte, Unsigned]	
Default	/		
Security access required	/		
Other Session		Programming Session	
Read	Write	Read	Write
M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

703409 v3 DID - D0A1

Version history:
(3): Requirements ID in description correction, and add the default value.

Purpose:
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D0A1
Name	ECU AB Sync Status
Description	Refer to ReqID 975734.

Content	SubParameter Name	Length, Data Type
	ECU AB Sync Status	[1 Byte, Unsigned]
Default	0x02	
Security access required	/	

Other Session		Programming Session	
Read	Write	Read	Write
M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

692459 v2 **DID - D103**

Version history:
(2): 更新描述，明确该DID仅适用于非分布式ECU。

Purpose: To get last successfully written memory address information.
Source:
Verification Method: Test
Legacy ID:

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

DID [Hex]	D103	
Name	Request Resume Information	
Description	Refer to ReqID 682081. Note: It is applicable for non-distributed ECU.	
Content	**SubParameter Name**	**Length, Data Type**
	Format And Length Identifier	[1 Byte, Unsigned]
	Last Memory Address Written	[Max. 5 Byte, Hex]
Default	/	
Security access required	/	

Other Session		Programming Session	
Read	Write	Read	Write
M	N	N	N
Periodic Read	I/O Control	Periodic Read	I/O Control
N	N	N	N

•RID

4.11.11.11.2 RID

682545 v4 **RID - 0250 Installation Condition Check**

Version history:
(4): Delete the security access required.

Purpose: All ECUs must support routines defined for seamless update.
Source:
Verification Method: Test
Legacy ID:

ECU shall support Routine - Installation Condition Check. The following data identifiers should be taken into account, and exactly as defined in *Global Master Reference Database.*

RID [Hex]	0250		
Name	Installation Condition Check		
Description	Refer to ReqID 682142. Note: It is applicable for non-distributed ECU. The request for distributed ECU shall refer to ECU SWRS - FOTA chapter.		
Type	1		
$P4_{Server_Max}$	$P2_{Server_Max}$ = 50ms		
Security access	/		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	N
Response parameters	**Start [0x01] RoutineStatusRecords**	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	
	Condition fullfilled to execute activation	[1 Byte, Unsigned]	

Available in application	Available in bootloader
M	N

703410 v2 **RID - 0254 Check Flash Programming Pre-conditions**

Version history:
(2): 更新描述，明确该RID仅适用于非分布式ECU。

Purpose: All ECUs must support routines defined for seamless update.
Source:
Verification Method: Test
Legacy ID:

ECU shall support Routine - Check Flash Programming Pre-conditions. The following data identifiers should be taken into account, and exactly as defined in *Global Master Reference Database.*

RID [Hex]	0254		
Name	Check Flash Programming Pre-conditions Note: It is applicable for non-distributed ECU. The request for distributed ECU shall refer to ECU SWRS - FOTA chapter.		
Description	Refer to ReqID 700051.		
Type	1		
$P4_{Server_Max}$	$P2_{Server_Max}$ = 50ms		
Security access			
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	N
Response parameters	**Start [0x01] RoutineStatusRecords**	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	
Available in application		Available in bootloader	
M		N	

682546 v4 **RID - 0255 Software Synchronization and Flag Activate**

Version history:
(4): 更新描述，明确该RID仅适用于非分布式ECU。

Purpose: All ECUs must support routines defined for seamless update.
Source:
Verification Method: Test
Legacy ID:

ECU shall support Routine - Software Synchronization and Flag Activate. The following data identifiers should be taken into account, and exactly as defined in *Global Master Reference Database.*

RID [Hex]	0255		
Name	Software Synchronization and Flag Activate		
Description	Refer to ReqID 682143 and 699903. Note: It is applicable for non-distributed ECU. The request for distributed ECU shall refer to ECU SWRS - FOTA chapter.		
Type	1		
$P4_{Server_Max}$	$P2_{Server_Max}$ = 50ms		
Security access	Level 01/02		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	N

Response parameters	Start [0x01] RoutineStatusRecords	Length, Data Type
	Routine Type	[4 bits, Unsigned]
	Routine Status	[4 bits, Unsigned]
	Activate Indication	[1 Byte, Unsigned]

Available in application	Available in bootloader
$40 - Update Session	N

1040185 v1 **RID - 0294 Install Private Module**

Version history:
(1):

Purpose: Define the routine ID to trigger the private node update or rollback.
Source:
Verification Method: Test
Legacy ID:

ECU shall support Routine - Install Private Module. The following data identifiers should be taken into account, and exactly as defined in *Global Master Reference Database.*

RID [Hex]	0294		
Name	Install Private Module		
Description	Refer to ReqID 975931 and 975932. Note: It is applicable for the public ECU master supports AB swap and the private ECU not supports AB swap.		
Type	2		
$P4_{Server_Max}$	$P2_{Server_Max}$ = 200 ms		
Security access	Level 01/02		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	M

Response parameters	**Start [0x01] RoutineStatusRecords**	**Length, Data Type**
	Routine Type	[4 bits, Unsigned]
	Routine Status	[4 bits, Unsigned]
	Install type	[1 Byte, Unsigned]
	Result [0x03] RoutineStatusRecords	**Length, Data Type**
	Routine Type	[4 bits, Unsigned]
	Routine Status	[4 bits, Unsigned]
	Installation progress	[1 byte, Unsigned]
	Abort reason	[1 byte, Hex]
	Reason statement	[49 bytes, Hex]

Available in application	Available in bootloader
N	MPS

1040186 v1 **RID - 0296 Rollback External Flash Module**

Version history:
(1):

Purpose: Define the routine ID for triggering rollback.
Source:
Verification Method: Test
Legacy ID:

ECU shall support Routine - Rollback External Flash Module. The following data identifiers should be taken into account, and exactly as defined in *Global Master Reference Database*.

RID [Hex]	0296		
Name	Rollback External Flash Module		
Description	Refer to ReqID 975935. Note: It is applicable for the the ECU not support A/B swap but support an external backup flash.		
Type	2		
$P4_{Server_Max}$	$P2_{Server_Max}$ = 200 ms		
Security access	Level 01/02		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	M
Response parameters	**Start [0x01] RoutineStatusRecords** — **Length, Data Type** Routine Type — [4 bits, Unsigned] Routine Status — [4 bits, Unsigned] **Result [0x03] RoutineStatusRecords** — **Length, Data Type** Routine Type — [4 bits, Unsigned] Routine Status — [4 bits, Unsigned] Abort reason — [1 byte, Hex]		
Available in application		Available in bootloader	
N		MPS	

4.12 Service-Oriented Vehicle Diagnostics (SOVD)

4.12.1 Service-Oriented Vehicle Diagnostics (SOVD)

4.12.1.1 General Requirements

•Overview

4.12.1.1.1 Overview

Service-Oriented Vehicle Diagnostics concept provides an API which standardizes the methods for diagnosing SOC and classical ECUs, the retrieval of the diagnostic capabilities, and the discovery of the SOVD methods in a vehicle.

•Requisite documents

4.12.1.1.2 Requisite documents

The following documents are either referenced by this specification, or contain information that is relevant to this specification.

Reference	Source	Document
[SOVD_1]	External	ASAM SOVD API Specification / ISO DIS 17978-3-2025 Road vehicles - Service-oriented vehicle diagnostics (SOVD) - Part 3:Application programming interface(API)
[SOVD_2]	External	R. T. Fielding; “REST: Architectural Styles and the Design of Network-based Software Architectures,” PhD dissertation, University of California, Irvine, 2000
[SOVD_3]	External	IETF RFC 8259
[SOVD_4]	External	IETF RFC 7540
[SOVD_5]	External	OpenAPI Specification v3.1.0
[SOVD_6]	External	IETF RFC 8693
[SOVD_7]	External	IETF RFC 6749
[SOVD_8]	External	IETF RFC 6750
[SOVD_9]	External	ASAM Data Types
[SOVD_10]	External	RFC 6762
[SOVD_11]	External	RFC 6763
[SOVD_12]	External	ISO 17978-3 Road vehicles — Service-oriented vehicle diagnostics (SOVD) — Part 3: Application programming interface (API)
[SOVD_13]	External	RFC-2131: Dynamic Host Configuration Protocol
[SOVD_14]	External	RFC-2132: DHCP Options and BOOTP Vendor Extensions (October 1993)
[SOVD_15]	External	RFC-4361: Node-specific Client Identifiers for Dynamic Host Configuration Protocol Version Four (February 2006)
[SOVD_16]	External	RFC 3927: Dynamic Configuration of IPv4 Link-Local Addresses

Note:
Internal references is typically found as a separate chapter in the ECU specific Base Technology SWRS.
Information obtaining ISO documents may be found on the internet: www.iso.org
•Definitions and Abbreviations

4.12.1.1.3 Definitions and Abbreviations

Definition	Description
SOVD	Service-Oriented Vehicle Diagnostics
VUCM	Vehicle Update and Configuration Module, manage the vehicle package download and vehicle installation.

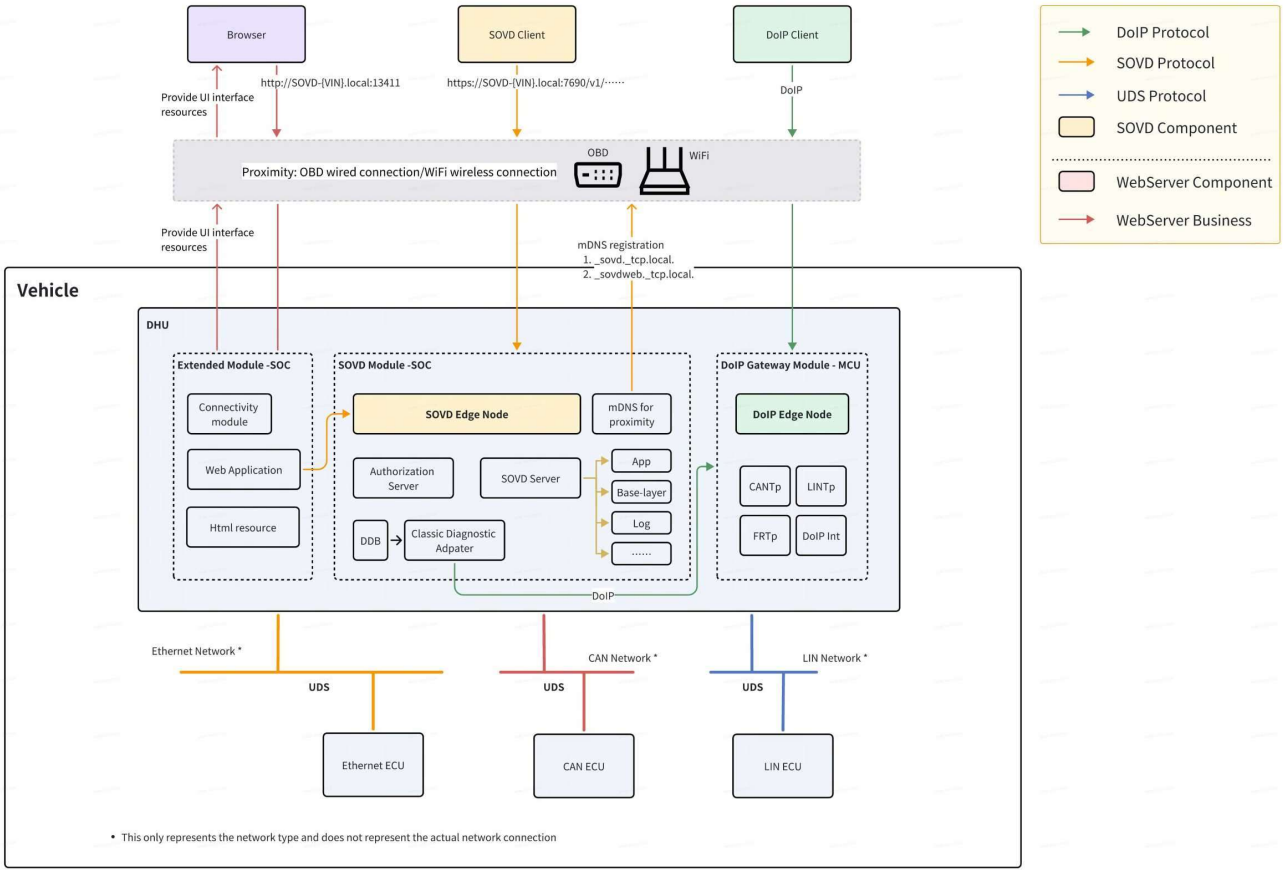
Abbreviation	Description
SOVD	Service-Oriented Vehicle Diagnostics
CVT	Convention
CDA	Classic Diagnostic Adapter

4.12.1.2 SOVD Architecture

•SOVD Architecture

4.12.1.2.1 SOVD Architecture

SOVD technology has been introduced into the EEA architecture of distributed systems, and some infrastructure components are required for SOVD edge nodes, SOVD ECU, and SOVD Client to meet the technical requirements. To achieve this a reference architecture that clusters functions into serveral blocks has been introduced. This architecture is displayed in following figure.



Main components of this reference architecture are:

SOVD Edge Node,

- SOVD Gateway
- Diagnostic Management with SOVD handling
- SOVD to UDS Adapter
- Web server providing visual interface and input

SOVD ECUs,

- Diagnostic Management with SOVD handling
- SOVD to UDS Adapter (When there are sub networks connected)

SOVD Client

The following chapters will introduce the components

4.12.1.3 SOVD - Edge Node (Gateway)

•SVOD Vehicle Gateway

4.12.1.3.1 SVOD Vehicle Gateway

SOVD Vehicle Gateway is the core node in the SOVD architecture, located at the vehicle communication boundary (usually deployed on the vehicle communication gateway or central computing platform), serving as a unified entry

point for external diagnostic clients to access in vehicle resources. Upon request from a SOVD client the SOVD Vehicle Gateway acts as SOVD Edge node and routes the requests to respective internal SOVD endpoints. The routing takes place based on the entity part in the URI of the SOVD request. The SOVD Vehicle Gateway must extract this part of the URI and route towards the corresponding endpoint. The setup of potential internal SOVD endpoints can be statically configured or dynamically discovered using mDNS. The forwarding itself takes place on application layer, i.e. the SOVD gateway acts as HTTP reverse proxy.

In summary, SOVD vehicle gateway is responsible for receiving external requests, performing security checks, path resolution, and forwarding requests to the target entity. SOVD Gateway can integrate or connect multiple SOVD private servers to achieve unified management of different diagnostic domains in the vehicle.

To support the performance of SOVD technology, SOVD Edge node will be deployed in Adaptive Platform.

1043291 v2 **As the server of mTLS connection**

Version history:
(2): Add the SOVD port number.

Purpose: mTLS connection is necessary for SOVD communication.
Source:
Verification Method: Test
Legacy ID:

In the scenario of SOVD communication with external clients, the vehicle shall act as the server-side and shall have server related TLS conection capabilites. The public SOVD Server shall listen on port number 7690.

The SOVD client will actively initiate an mTLS connection to the server and successfully create a secure link.

1042962 v1 **General requirements**

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis
Legacy ID:

The main responsibilities of SOVD Vehicle Gateway include:

- Receive SOVD API requests from external diagnostic client or invehicle webserver;
- Perform client identity authentication and permission verification for external diagnostic client;
- Resolve the diagnostic domain to which the target entity of the request belongs;
- Forward the request to the corresponding SOVD private server or underlying UDS entity;
- Manage client connections and session status;
- Accurately return the response to the original requester.

1042951 v1 **Performance requirement**

Version history:
(1):

Purpose:
Source:
Verification Method: Test

Legacy ID:

To meet the real-time requirements of the in vehicle diagnostic system, when the client accesses private server resources or classical UDS entities through SOVD Vehicle Gateway, the performance requirements for request processing and routing are shown in Table:

Table: SOVD routing performance requirement

Items	Description	Performance
Authentication and authorization	Verify client tokens (such as OAuth2/Bearer Token)	≤ 10 ms
Entity permission analysis	Determine whether the current client has access to the target entity	≤ 5 ms
Routing - SOVD private server	Analyze the request target, determine which private server to forward to, and execute the forwarding	≤ 5 ms
Routing - UDS entity	Convert SOVD API method into underlying UDS requests through the SOVD to UDS module and send them to the corresponding ECU	≤ 10 ms

Note: It is recommended to control the overall SOVD Gateway forwarding delay within 20ms to meet real-time diagnostic requirements.

•SOVD Server Discovery

4.12.1.3.2 SOVD Server Discovery

SOVD server discovery shall be implemented via Multicast DNS (mDNS) and DNS Service Discovery (DNS-SD). The purpose is to automatically discover and locate SOVD servers on the local network and obtain their connection information. Service discovery requirements for remote networks are not defined in this document.

908753 v1 **AutoIP protocol support requirement**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID:

The SOVD Edge node shall support AutoIP protocol.

849663 v1 **DHCP protocol support requirement**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID:

The SOVD Edge node shall support DHCP protocol.

1042926 v1 **Debug interface**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID:

When the ECU has not injected the certificate and private key, it has to use the HTTP protocol to communicate with external clients, using port number 7692. After injecting the certificate and private key, terminate the use of the port number for external communication or allow it only after reliable protection.

Note: When using debug interface, base_uri is the same as using HTTPs, and all SOVD API methods allow access.

908760 v2 External IP allocation

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID:

When the DoIP activation line is activated and Ethernet link can be established on the physical and data link layer throught OBD connector, dynamic IP address allocation shall be initiated based on ReqID 849663 and ReqID 908753 requirements, and sent to the external tester.

Note: There shall be a VLAN between the DLC and the EdgeNode.

1042798 v1 Hostname Announcement

Version history:
(1):

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

After the valid VIN has been wroten successfully, the SOVD Edge Node shall immediately stop responding to MAC based hostname and switch to VIN based hostname broadcasting. The old hostname shall naturally exit the cache after TTL expires.

1031152 v1 Hostname definition

Version history:
(1): 新增需求，统一Hostname的定义。

Purpose: The hostname is a unique identifier for each edge node within the local area network.
Source:
Verification Method: Test
Legacy ID:

If VIN is available and valid, the Hostname shall be SOVD-{VIN}. Otherwise, the Hostname shall be SOVD-{MAC}.

849661 v3 mDNS Announcement/Response

Version history:
(3):

Purpose: Clarify the characteristics of mDNS Announcement.
Source:
Verification Method: Test
Legacy ID:

The SOVD Edge Node shall send mDNS Announcement according to the following requirement when the funciton is triggered through by hardward or software method.

Parameter	Definition
Service registration stage	When an edge node first joins the network or registers for a new service e.g., ECU initialization, ip change, hostname change, it shall send three consecutive multicasts with interval timing 1 second to ensure reliable transmission of multicat packets and avoid confilcts.
Maintenance phase	Edge node shall regularly send one service announcement to maintain local caching. The default interval is 60 seconds with a random jitter of 0~10%. The TTL (Time to Live) for service records shall set to 120.
Query response mechanism	When receiving a service query, the vehicle shall respond mDNS response within random delays time 500ms to 1000ms.

908320 v3 mDNS paramenter for Web

Version history:
(3):

Purpose: Use Multicast DNS to obtain the IP address and DNS Service to identify Web servers.
Source:
Verification Method: Test
Legacy ID:

The Web server or public SOVD server shall implement mDNS host according to [SOVD_10]. The following parameter shall be used.

Parameter	Definition
Service Type	Service type is used to identify a specific service, rather than a specific device. The protocol "_sovdweb._tcp.local." of the service shall be used.
Service Instance	The service instance is a specific implementation of a service tpye, e.g. {Hostname}._sovdweb._tcp.local.
Protocol	TCP
Port	Communication based on HTTPs, using port number 13411.

849660 v4 mDNS parameter for SOVD

Version history:
(4): TXT parameter update.

Purpose: Use Multicast DNS to obtain the IP address and DNS Service to identify SOVD servers.
Source:
Verification Method: Test

Legacy ID:

The public SOVD gatewy or public SOVD server shall implement mDNS host according to [SOVD_10]. The following parameter shall be used.

Parameter	Definition
Service Type	Service type is used to identify a specific service, rather than a specific device. The protocol "_sovd._tcp.local." of the service shall be used.
Service Instance	The service instance is a specific implementation of a service tpye, e.g. {Hostname}._sovd._tcp.local.
Protocol	TCP
Port	Communication based on HTTPs, using port number 7690.
TXT	accessurl=https://{hostname}.local:7690. SOVD Client can use this accessurl to get the SOVD version through */version-info* API.

•Concurrency and Connection Management

4.12.1.3.3 Concurrency and Connection Management

The SOVD Gateway shall manage client connections and internal request forwarding according to defined rules. This section specifies the off-board access strategy for SOVD and the in-vehicle request queue mechanism for SOVD.

4.12.1.3.3.1 Client Connection Management

As the unified access point for off-board diagnostic requests, the SOVD Gateway shall provide connection limit control and admission rejection capabilities.

908756 v3 **Concurrent SOVD External Client Connection**

Version history:
(3): Improve requirement description.

Purpose: Restrict the number of external clients accessing simultaneously to avoid excessive external client access and occupation.
Source:
Verification Method: Analysis AND Test
Legacy ID:

A single SOVD Gateway shall allow a maximum of 5 concurrent authenticated client connections. This limit covers all external clients that have passed authentication. If there are more external clients accessing, Gateway shall reject the connection, e.g. TCP RST.

Rationale:
In mTLS connection, Gateway shall use a combination of source IP of TCP socket four-tuple and the Client ID as identification for external clients to determine the number of clients.

Note: The Client ID uses the CN filed value in the client certificate from the mTLS connection

1042955 v2 **Handling when internal connection resources are exhausted**

Version history:
(2): Improve requirement description.

Purpose: To indicate appropriate response code in specific scenarios.
Source:
Verification Method: Test

Legacy ID:

When the number of existing connections in in-vehicle reaches the limit and there are no reusable connections, new request from internal and external clients shall be rejected after waiting for a timeout, and the HTTP status code 503 Service Unavailable shall be returned, indicating that the client should retry after a specified time.

1075934 v1 **SOVD Client Connection Limit**

Version history:

Purpose: Requirements for the client.
Source:
Verification Method: Analysis
Legacy ID:

For each SOVD client, the number of established and active TCP connections should be limited to 30. If a request is made to establish the 31st TCP, the first TCP request connection should be disconnected.

Note: This requirement is only applicable to the SOVD Client.

1047368 v2 **The number of connection to SOVD Client**

Version history:
(2): Improve requirement description.

Purpose: Specify the connection between the gateway and the client.
Source:
Verification Method: Test
Legacy ID:

In theory, there is no limit to the number of TCP connections established between SOVD Gateway and the same authentication SOVD Client. But Gateway shall Ensured that resources shall be released promptly after connections are closed or timeout.

Note: Connection between Client and Gateway counting is based on “established and active” TCP/TLS connections.

4.12.1.3.3.2 Invehicle Connection Management

SOVD Gateway shall act as a reverse proxy, and maintains a connection pool to the backend server, which includes a certain number of pre-established or on-demand, reusable TCP connections. (Usually HTTP/1.1 Keep Alive or HTTP/2.0)

1075871 v1 **Binding of request response**

Version history:

Purpose: The SOVD Gateway shall validate the validity and uniqueness of Client id, then use it for client identification, routing distribution, and response forwarding.
Source:
Verification Method: Analysis AND Test
Legacy ID:

For each successfully established client connection, SOVD Gateway shall use the Client ID and TCP socket four-tuple as the unique session identifier and shall remain valid during the session.
When forwarding requests, the SOVD Gateway shall include X-Client-Id: {Client Id} in the HTTP request header, to ensure that responses can be accurately returned to the initiator. In addition, each request in the queue shall be

bound to the Client ID and priority information for subsequent response correlation and scheduling optimization.

Note: The Client ID uses the CN filed value in the client certificate from the mTLS connection.

1075998 v1 **Connection pool**

Version history:

Purpose: To define how the gateway allocates TCP connections.
Source:
Verification Method: Analysis AND Test
Legacy ID:

SOVD Gateway shall manage the connection pool and check for available, healthy, and protocol compatible connections.

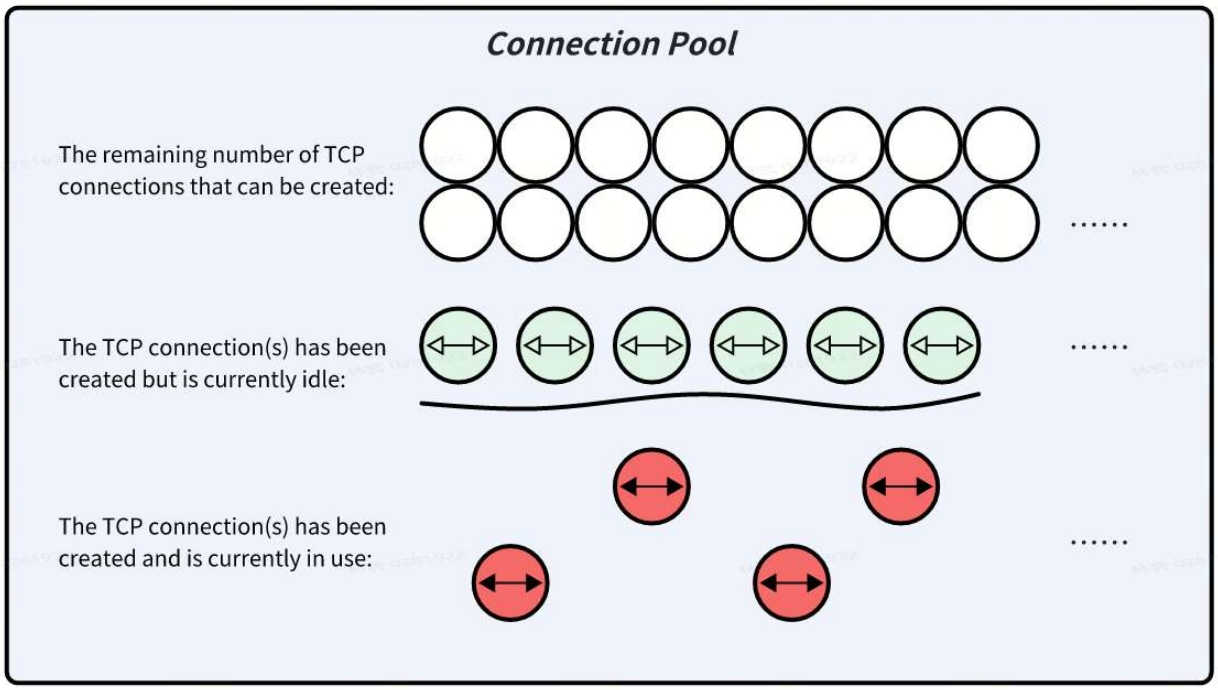
- If there are available connections in the pool, take them out directly for use, e.g. pre-established connection.
- If there are no available connections in the pool but the maximum connection limit has not been reached, create a new TCP connection and add it to the connection pool.

a) The maximum allowed waiting time during the TCP connection with the target entity is 30 seconds. If the TCP connection is not established after the timeout, the underlying transmission connection will be actively closed and the socket will be released.

b) When establishing a TCP connection with downstream entities, the SOVD gateway shall implement an application layer retry mechanism. It shall be to retry three times and adopt an exponential backoff strategy (the retry interval is suggested to be 1 second, 2 seconds, or 4 seconds), only retrying recoverable errors such as connection rejection, timeout, or unreachable host. After all retries fail, the Gateway should return the HTTP status code 502 Bad Gateway and clearly indicate the reason for the "connection failure" (such as no response from the ECU or unreachable network) in the response body. It shall be to include information such as the number of retries, total time consumption, and suggested operations.

- If the pool is full, Gateway shall wait until a useful connection is released before retrieving it for use. If the wait time exceeds, an HTTP status code 503 will be returned. The timeout period is set to 30 seconds.

Figure - The TCP connection(s) in Connection Pool



1042963 v2 **Internal request forwarding queue mechanism**

Version history:
(2): Delete the Client ID requirements.

Purpose: To execute forwarding in an orderly manner.
Source:
Verification Method: Test
Legacy ID:

To support orderly processing of concurrent requests from multiple clients and avoid overloading of target entities, SOVD Gateway shall implement a request queue management mechanism based on network segment isolation internally.

1043323 v2 **Pre-established connection as the in-vehicle client**

Version history:
(2): Requirement Refactoring.

Purpose: To ensure timely routing and forwarding of SOVD requests by the gateway.
Source:
Verification Method: Test
Legacy ID:

After ECU startup, SOVD Gateway shall attempt to establish at least one persistent TCP connection (with a dynamic source port and a destination port of 7690) with the SOVD server in advance.

If the establishment with the target server fails, such as the server not starting, the server rejecting, etc., the gateway shall try again within 500ms.

Note: the specific SOVD server depends on the project.

1076320 v1 **Return and reuse mechanism for connections**

Version history:

Purpose: To define how the gateway manages TCP connections.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The connections in the connection pool managed by SOVD Gateway shall meet the following operational requirements:
1. After the internal request/response is completed, the connection will be marked as idle and put back into the connection pool, waiting for the next reuse. (within KeepAlive timeout)
2. If the idle time of the connection exceeds KeepAlive timeout or is considered unhealthy, it shall be closed.
3. If the TCP connection with the client has been released or the TCP connection KeepAlive has timed out, the TCP resources in the connection pool shall be released by SOVD Gateway.
4. If the server does not return the response after 90 seconds, SOVD Gateway shall return the HTTP status code 504.

1047374 v2 **The number of connections in the connection pool**

Version history:
(2): Requirement Refactoring

Purpose: To define the minimum requirement for the number of resource connections requested.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The connection pool shall be able to have at least 240 pairs of TCP connections simultaneously, with information such as the idle/occupied status of the connections.

4.12.1.4 SOVD API Methods

•SOVD API Methods

4.12.1.4.1 SOVD API Methods

1043538 v2 **General API Method Requirement**

Version history:

Purpose:
Source:
Verification Method: Analysis
Legacy ID:

The following general API mehtod requirements shall be supported:

- The ECU shall be compliant to *[12] ISO 17978-3*: Road vehicles — Service-oriented vehicle diagnostics (SOVD) — Part 3: Application programming interface (API).

•Access of supported SOVD server versions

4.12.1.4.2 Access of supported SOVD server versions

This chapter provides a detailed explanation of the API method for obtaining the supported versions of SOVD servers. The core is to obtain the SOVD standard version information supported by the server through specific GET requests, providing a basic path for subsequent API requests.

1043626 v1 Capability Descripiton-Related APIs

Version history:

Purpose:
Source:
Verification Method:
Legacy ID:

The methods used for the Capability Description-related APIs are as follows:

No.	API Method Description	HTTP Method	URI	Support	Comment
1	Read all supported versions of the SOVD server	GET	https://{sovd-server-host}/version-info	M	

Note: the {exve-manufacturer-specific} of URI does not require support.

•Access to capability description content

4.12.1.4.3 Access to capability description content

This chapter defines obtaining all the information required to use a specific SOVD method in an independent manner, without relying on a complete offline capability description. Diagnostic tools can request such online capability descriptions on their various entities and resources, providing diagnostic clients with corresponding online documents and OpenAPI specifications to obtain all relevant information required for interaction with the operating resource.

1043504 v1 Retrieve online capability description

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

This API method shall be used to request an Online Capability Description for a specified entity, resource, or collection of resources. The returned content is a self-contained and valid OpenAPI specification that describes all relevant information required to interact with the resource, including:
a) Supported HTTP methods
b) Request and response structures
c) Defined HTTP status codes
d) SOVD specific extended information (such as supported capabilities, translation identifiers, etc.)

Method					
GET /{Any path}/docs					
Path parameters and Query parameters					
Param Type	**Param Name**	**Type**	**Support**	**CVT**	**Description**
Path params	any-path	string	M	M	To obtain the resource path for capability

					description, e.g../components/Engine/data/fuelLevel/docs

Request Header
No use specific header parameters
Request Body
Not applicable
Response Format and Parameters
Response Status Code: 200 (The request was successful) The online capability description of the underlying resource in form of an OpenAPI specification in JSON document format.
Response Status Code: • 403, Access denied due to insufficient authorization level • 404, The requested resource does not exist or does not support documentation

Attribute	Parent	Type	Support	CVT	Description
-	-	GenericError	M	M	An error response includes a more detailed description of the error.

•Discovery of entities and resources

4.12.1.4.4 Discovery of entities and resources

The main content of this chapter is entity discovery, querying entity sub entities, and querying related entity APIs (such as components of a certain region or applications of a certain component), which allows clients to traverse the topology structure to determine which methods are available for which resources.

1043619 v1 **Entity-Related APIs**

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

The methods used for the Entity-related APIs are as follows:

No.	API Method Description	HTTP Method	URI	Support	Comment
1	Discover contained Entities	GET	/{entity-collection}	M	
2	Query Sub-entities of an Entity	GET	• /areas/{area-id}/subareas • /components/{component-id}/subcomponents	M	
3	Query Related Entities of an Entity	GET	• /areas/{area-id}/contains • /components/{component-id}/hosts • /components/{component-id}/depends-on • /apps/{app-id}/depends-on	M	
4	Query Entity Capabilities	GET	• / • /{entity-path}	M	

•Tagged Entities and resources (Enhancement)

4.12.1.4.5 Tagged Entities and resources (Enhancement)

This chapter mainly describes adding predefined tags to tagged resources for querying and identifying them through the SOVD API. This type of label can add relevant context to resources and can be applied in various scenarios, such as labeling all resources that belong to legally required diagnostics, or labeling specific system resources distributed across multiple entities and resource collections.

1043606 v1 Tag-Related APIs

Version history:

Purpose:
Source:
Verification Method:
Legacy ID:

The methods used for the Tag-related APIs are as follows:

No.	API Method Description	HTTP Method	URI	Support	Comment
1	Query for tagged Entities and resources	GET	• /{entity-collection} • /{entity-path}/{resource-collection}	M	
2	Retrieve Tags from an Entity or a Resource	GET	• / • /{entity-path} • /{entity-path}/{resource-collection}/{resource-id}	M	
3	Query for Tagged Capabilities of an Entity	GET	• / • /{entity-path}	M	

•Fault handling

4.12.1.4.6 Fault handling

This chapter defines the SOVD API interface for reading and managing fault information. This interface provides a standardized way for vehicle diagnostic systems to access, filter, and manipulate fault information from different entities. Through this API method, diagnostic tools can obtain detailed fault data and perform deletion operations if supported。

1043511 v1 Definition and requirements of faults

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

SOVD faults are used to abstract various fault events identified in vehicle systems and support standardized expression of fault states and additional information. If it is a traditional ECU, each UDS fault code shall correspond to an SOVD fault.

SOVD faults shall include a set of general attributes used to describe the basic information of the fault. These attributes include:

- code: The corresponding DTC code value

- scope: Identify the storage area to which the fault belongs
- display_code: DTC code used for display
- fault_name: Brief description of the malfunction
- severity: Fault severity level
- status: indicates the current status of the fault, and its field structure shall follow the DTC status mask defined in ISO 14229-1.
- symptom: Description of fault symptoms

Table below specifies Fault type

Attribute	Parent	Type	Support	CVT	Description
code	-	string	M	M	Fault code in the native representation of the Entity.
scope	-	string	O	O	Defines the scope (e.g., user-defined fault memories) for fault entries. The capability description defines which scopes are supported.
display_code	-	string	M	O	Display representation for the fault code.
fault_name	-	string	M	M	Name / description of the fault
fault_translation_id	-	string	O	O	Optional identifier for translating the fault name
severity	-	integer	O	O	Severity defines the impact on the availability of the vehicle. Lower number means higher severity. For a classic ECU this resembles the level from DDB for HPCs the SOVD API recommends 1 = FATAL, 2 = ERROR, 3 = WARN, and 4 = INFO.
status	-	object	M	C_1	Detailed status information for the fault as key-value pairs. For a classic ECU this resembles the status byte of a DTC. The content of the keys and values is OEM specific. The keys are defined in the capability description.
symptom	-	string	O	C_1	Detailed symptom / failure mode information for the fault. For a classic ECU this resembles the symptom of a DTC. The content of the attribute is OEM specific.
symtom_translation_id	-	string	O	C_1	Translation identifier for the symptom name.

For each configured SOVD fault, the system/ECU shall support the collection and storage of environmental data (environment_data) related to it. This data shall include extended data records (extended_data_decords) and freeze frame snapshots (snapshots).
When deleting SOVD faults, the underlying behavior shall simulate the clearing mechanism of service 0x14 ClearDiagnosticInformation in ISO 14229-1 to ensure consistency and standardization of fault lifecycle management.

1043513 v1 **Definition of Scope**

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

The Scope parameter mentioned specifies the storage scope of the fault entry, and the valid values include:

- Default: Default scope, including general fault storage
- Temporary: Temporary fault storage, which may be cleared after system restart

- Permanent: permanent fault storage, retained even after restart
- Security: Fault entries related to security
- Diagnostics: Diagnostic specific fault storage
- Calibration: Calibration related fault entries
- Operation: Malfunctions generated during the operation process
- Safety: Safety critical system failure
- UserDefined [1-9]: User defined scope (1-9), used for fault storage in specific scenarios
- Manufacturer Specific: Manufacturer specific scope, defined by Geely.

All scope values shall be consistent with the scope list defined in the capability description document, unsupported scope values shall result in a 400 Bad Request response.

1043514 v1 Definition of Status

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

The status parameter mentioned is a set of key value pairs.
For classic ECUs, the following parameters are included:

- TestFailed: "0" or "1", indicating whether the current test has failed
- TestFailedThisOperationCycle: "0" or "1", indicating whether the test failed during this operation cycle
- PendingDTC: "0" or "1", indicating whether it is a pending DTC
- ConfirmedDTC: "0" or "1", indicating whether it is a confirmed DTC
- TestNotCompletedSinceLastClear: "0" or "1", indicating whether the test has not been completed since the last clearing
- TestFailed SinceLastClear: "0" or "1", indicating whether the test has failed since the last clearing
- TestNotCompletedThisOperationCycle: "0" or "1", indicating whether the test was not completed during this operation cycle
- WarningIndicatorRequested: "0" or "1", indicating whether to request a warning indicator.
- AggregatedStatus: "active", "pending", "stored" or "cleared", indicating the summary status.
- mask: the DTC status byte in hex.

For HPC, the following parameters are included:

- aggregatedStatus:

1) Active: indicate a faults that are currently occurring or have not been resolved.
2) Pending: indicate a fault that has been detected but not yet fully confirmed or processed.
3) Stored: indicate a faults that have occurred in the past but may have been resolved currently.
4) Cleared: indicate a faults that have occurred in the past and have been repaired or manually cleared.

1043604 v1 Fault APIs

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

The methods used for the fault APIs are as follows:

No.	API Method Description	HTTP Method	URI	Support	Comment
1	Query for faults from an Entity	GET	/{entity-path}/faults	M	
2	Read a fault resource	GET	/{entity-path}/faults/{fault-code}	M	
3	Delete all faults of an Entity	DELETE	/{entity-path}/faults	M	
4	Delete single fault of an Entity	DELETE	/{entity-path}/faults/{fault-code}	M	

1043516 v1 **The environment data for classic ECU**

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

The environment_data for classic ECU in response of "Read a Fault Resource" API method is defined in following table:

Attribute	Parent	Type	Support	CVT	Description
extended_data_records	-	ExtendedDataRecordsInformation	M	M	Fault code in the native representation of the Entity.
Operation_cycle_counter_1	ExtendedDataRecordsInformation	string	M	M	
Operation_cycle_counter_2	ExtendedDataRecordsInformation	string	M	M	
Operation_cycle_counter_3	ExtendedDataRecordsInformation	string	M	M	
Operation_cycle_counter_4	ExtendedDataRecordsInformation	string	M	M	
Operation_cycle_counter_5	ExtendedDataRecordsInformation	string	MC	M	
Operation_cycle_counter_6	ExtendedDataRecordsInformation	string	M	M	
Operation_cycle_counter_7	ExtendedDataRecordsInformation	string	O	M	
DTC_fault_detection_counter	ExtendedDataRecordsInformation	string	M	M	
Maximum_DTC_fault_detection_counter_since_last_clear	ExtendedDataRecordsInformation	string	M	M	
DTC_Time_Stamp_22	ExtendedDataRecordsIn	string	M	M	

	formation				
DTC_Time_Stamp_23	ExtendedDataRecordsInformation	string			
DTC_Status_indicator_30	ExtendedDataRecordsInformation	string			
snapshots	-	SnapshotsInformation[]	O	O	Defines the scope (e.g., user-defined fault memories) for fault entries. The capability description defines which scopes are supported.
Snapshot_Record_20	SnapshotsInformation[]	object	M	M	Display representation for the fault code.
Snapshot_Record_21	SnapshotsInformation[]	object	O	M	Name / description of the fault

•Data Resource read/write access

4.12.1.4.7 Data Resource read/write access

This chapter mainly describes how the SOVD API allows SOVD clients to perform read and write operations on various data values of entities. SOVD itself does not distinguish between identification, measurement values, and other data values. To facilitate diagnostic tool differentiation, categories defined by SOVD (such as those used to distinguish measurement values) can be used. The SOVD client can define a temporary data list for the need to read multiple values in batches multiple times. By reading this data list, the client can retrieve all defined values in one response.

1043560 v1 **Data Resource APIs**

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

The methods used for the data resource APIs are as follows:

No.	API Method Description	HTTP Method	URI	Support	Comment
1	Retrieve categories supported by a data resource collection	GET	/{entity-path}/data-categories	O	
2	Retrieve groups supported by a data resource collection	GET	/{entity-path}/data-groups	O	
3	Query for data from an Entity	GET	/{entity-path}/data	M	
4	Read a data resource	GET	/{entity-path}/data/{data-id}	M	
5	Read multiple data values from an Entity				
5.1	Query for data-lists from an Entity	GET	/{entity-path}/data-lists	M	
5.2	Creating a data-list for reading multiple data values at once from an Entity	POST	/{entity-path}/data-lists	M	
5.3	Read a data-list resource	GET	/{entity-path}/data-lists/{data-list-id}	M	
5.4	Delete an existing data-list	DELETE	/{entity-path}/data-lists/{data-list-id}	M	
6	Write a data resource	PUT	/<entity-path>/data/{data-id}	M	

1043535 v1 **Definition and Requirements of Data**

Version history:

Purpose:
Source:
Verification Method: Analysis
Legacy ID:

Each SOVD data shall represent a readable or writable data item provided by a diagnostic entity (such as ECU or HPC).
SOVD data shall include the following standardized attributes:

- id: Unique identifier;
- name: Data name;
- category: Data classification;

SOVD data supports organization by group, and all data within the same group should have the same category type. Data category is used to identify the type and access characteristics of data, and standard classifications include:

- identData, represents the fixed identification parameter of an entity (such as VIN), which is usually read-only and may only support writing during the manufacturing phase;
- currentData, represents dynamically changing measurement values (such as battery voltage) and only supports reading;
- storedData, represents configuration parameters that can be read and written at runtime;
- sysInfo, represents system operating status information (such as CPU load) and only supports reading.

The SOVD data list is used to support unified access and batch operations for multiple diagnostic data items. Diagnostic entities should support the creation, reading, and deletion of data lists, and ensure the consistency and maintainability of their contents. Each data list should consist of a set of defined SOVD data, which can be used to read multiple data values at once to improve diagnostic efficiency.

All read and write operations of SOVD data should simulate the ReadDataByIdentifier (0x22) and WriteDataByIdentifier (0x2E) service behaviors defined in ISO 14229-1 to ensure consistency and standardization of the underlying data access mechanism. The content query method of the data list should be consistent with the reading of individual data, ensuring standardized return data format for easy parsing and use.

•Cyclic-subscriptions

4.12.1.4.8 Cyclic-subscriptions

This chapter describes how an SOVD client is allowed to issue cyclic-subscriptions of SOVD resources, e.g., data or data-list. Subscriptions are modeled as individual resources and allow decoupling the subscription management from the information delivery.

1043658 v1 **Cyclic-Subscription APIs**

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

The methods used for the Cyclic-Subscription APIs are as follows:

No.	API Method Description	HTTP	URI	Support	Comment

		Method			
1	Query for cyclic-subscriptions from an Entity	GET	/{entity-path}/cyclic-subscriptions	O	
2	Issue a new cyclic-subscription	POST	/{entity-path}/cyclic-subscriptions	O	
3	Retrieve details of a cyclic-subscription	GET	/{entity-path}/cyclic-subscriptions/{id}	O	
4	Update a cyclic-subscription	PUT	/{entity-path}/cyclic-subscriptions/{id}	O	
5	Delete a cyclic-subscription	DELETE	/{entity-path}/cyclic-subscriptions/{id}	O	

•Triggers

4.12.1.4.9 Triggers

This chapter mainly describes how by using triggers, clients can receive notifications when specific events occur, which can avoid continuous polling and reduce resource consumption; The client can also use triggers to write tags to the SOVD server logs when specific events occur, which helps to identify relevant records by analyzing log files during debugging.
Note: Examples of events include measurement values entering critical ranges, malfunctions, or changes in script execution status. Each trigger references a resource to be observed and contains conditions that define when it triggers. Observable resource types include data, faults, operation execution, script execution, updates, and locks.

1043663 v2 **Triggers APIs**

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

The methods used for the Triggers APIs are as follows:

No.	API Method Description	HTTP Method	URI	Support	Comment
1	Query for triggers from an Entity	GET	/{entity-path}/triggers	O	
2	Issue a new trigger	POST	/{entity-path}/triggers	O	
3	Retrieve details of a trigger	GET	/{entity-path}/triggers/{id}	O	
4	Update a trigger	PUT	/{entity-path}/triggers/{id}	O	
5	Delete a trigger	DELETE	/{entity-path}/triggers/{id}	O	

•Configuration

4.12.1.4.10 Configuration

This chapter mainly describes the read and write methods of SOVD API supporting configuration data, such as components and applications that need to be configured according to specific environments, such as vehicle equipment, country/region, customer requirements, variant coding, etc,
Note: Configuration resources can be represented as a set of parameters or byte streams to support multiple types of configuration data (such as manifest files (ASCII format) or binary files, etc.). And the SOVD API also provides a way to reset specific configuration resources to default values, or reset all configurations of an entity to default values.

1043684 v1 **Configuration APIs**

Version history:

Purpose:
Source:

Verification Method: Analysis AND Test
Legacy ID:

The methods used for the Configuration APIs are as follows:

No.	API Method Description	HTTP Method	URI	Support	Comment
1	Query for configurations from an Entity	GET	/{entity-path}/configurations	O	
2	Read a configuration resource	GET	/{entity-path}/configurations/{configuration-id}	C	
3	Write a configuration resource	PUT	/{entity-path}/configurations/{configuration-id}	C	
4	Reset all configurations to default value	DELETE	/{entity-path}/configurations	O	
5	Reset configuration to default value	DELETE	/{entity-path}/configurations/{configuration-id}	O	

•Clearing data

4.12.1.4.11 Clearing data

This chapter mainly describes that diagnostic tools can define resources for cached data, learning data, and client based on actual needs, query data types to be cleared, and perform corresponding data clearing and data clearing status queries according to actual needs.

1043685 v1 **Clearing Data-Releated APIs**

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

The methods used for the Clearing Data-Releated APIs are as follows:

No.	API Method Description	HTTP Method	URI	Support	Comment
1	Retrieve list of supported clear-data types	GET	/{entity-path}/clear-data	O	
2	Start clearing of data	PUT	• /{entity-path}/clear-data/cached-data • /{entity-path}/clear-data/learned-data • /{entity-path}/clear-data/client-defined-resources	O	
3	Retrieve status of clearing data	GET	/{entity-path}/clear-data/status	O	

•Control of operations

4.12.1.4.12 Control of operations

The chapter mainly describes the operational resources of executable diagnostic objects, such as I/O control, program, and software functions. The diagnostic tool operates the corresponding SOVD API in two ways: synchronous execution (the API method that executes the operation returns after the operation is completed, and the response contains the operation result) and asynchronous execution (the API method that executes the

operation returns immediately, and the client needs to query the execution status provided as a temporary resource); To achieve control over corresponding resources.

1043686 v1 **Control of operation APIs**

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

The methods used for the Control of operation APIs are as follows:

No.	API Method Description	HTTP Method	URI	Support	Comment
1	Query for operations from an Entity	GET	/{entity-path}/operations	M	
2	Query for executions of an operation	GET	/{entity-path}/operations/{operation-id}/executions	M	
3	Retrieve details of an operation	GET	/{entity-path}/operations/{operation-id}	M	
4	Retrieve details of an operation	GET	/{entity-path}/operations/{operation-id}	M	
5	Start execution of an operation	POST	/{entity-path}/operations/{operation-id}/executions	M	
6	Read the status of an operation execution	GET	/{entity-path}/operations/{operation-id}/executions/{execution-id}	M	
7	Terminate the execution of an operation	DELETE	/{entity-path}/operations/{operation-id}/executions/{execution-id}	M	
8	Support for execute / freeze / reset and custom capabilities	PUT	/{entity-path}/operations/{operation-id}/executions/{execution-id}	M	

1043765 v1 **proximity proof required**

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

The parameter proximity_proof_required shall be set to false.

•Script Execution

4.12.1.4.13 Script Execution

The chapter mainly describes to handle executable scripts on a Manufacture. It provides methods to manage the available scripts (List / Upload / Remove) and to control the execution of scripts (Start, Terminate, Get Status) available on the SOVD Server.

1043687 v2 **Script execution APIs**

Version history:

Purpose: To define the API method for script function.

Source:
Verification Method: Analysis AND Test
Legacy ID:

The methods used for the Script execution APIs are as follows:

No.	API Method Description	HTTP Method	URI	Support	Comment
1	Upload a script	POST	/{entity-path}/scripts	M	
2	Query for scripts from an Entity	GET	/{entity-path}/scripts	M	
3	Retrieve details of a script	GET	/{entity-path}/scripts/{script-id}	M	
4	Remove a script	DELETE	/{entity-path}/scripts/{script-id}	M	
5	Start execution of a script	POST	/{entity-path}/scripts/{script-id}/executions	M	
6	Query for executions of a Script	GET	/{entity-path}/scripts/{script-id}/executions	M	
7	Read the status of a script execution	GET	/{entity-path}/scripts/{script-id}/executions/{execution-id}	M	
8	Terminate the script execution	PUT	/{entity-path}/scripts/{script-id}/executions/{execution-id}	M	
9	Remove the execution of a script	DELETE	/{entity-path}/scripts/{script-id}/executions/{execution-id}	M	
10	Upload a script compression file	POST	/{entity-path}/scripts/batch	M	Geely specific API

1074516 v1 Scripts runtime

Version history:

Purpose: To have the ability to execute multiple scripts simultaneously.
Source:
Verification Method: Analysis AND Test
Legacy ID:

ECU shall be able to execute more different main scripts simultaneously, and the execution of each main script shall be independently managed and controlled.

Note: The content of the script should avoid conflicts that may cause execution issues.

1073566 v1 The scrpit handling when upload

Version history:

Purpose: To define the management of scripts.
Source:
Verification Method: Test
Legacy ID:

If a new script is uploaded, it will be stored in a dedicated directory for managing scripts and shall not be lost due to SWDL.
If the script is not registered for execution, then when uploading a script with the same file name and suffix again, the newly uploaded script shall be used to overwrite the old script.
If the script has already been registered for execution, uploading a script with the same file name and suffix again shall report a 409 conflict error and return a detailed reason in parameter "GenericError".

Generic Error example:

```
{
  "error_code": "precondition-not-fulfilled",
  "message": "The script has been registered in execution",
  "parameter": {
    "items": [
      {
        "script_id": "DrivingComputer",
        "execution_id": [
          "fd34f39d-06e7-494b-af2d-8928e1458fb0",  // {execution-id}
          "cb73a10d-89a3-342d-eb5d-3527a3963ac1"   // Another {execution-id}
        ]
      }
    ]
  }
}
```

4.12.1.4.13.1 Multi script compression upload

1073567 v1 **API - Multi script compression upload**

Version history:

Purpose: To define an API method to support bulk-data script upload.
Source:
Verification Method: Test
Legacy ID:

The following method is to upload compressed scripts.

Method
POST {entity-path}/scripts/batch

Request Header
Content-Type: Depends on Request Body
Content-Encoding: gzip
Content-Length：Size of the http-content
Content-Disposition: form-data; name="{file name}"; filename="{file name with file extension}"

Table1: Request Body

Attribute	Parent	Type	Length	Convention	Description
[FileContent]	-	FileContent		M	

Table2: Response Status Code

Status Code	Response Body	Description
200	Table 3	The request was successful.

Table3: Response Body

Attribute	Parent	Type	Length	Convention	Description
ids	-	Array[string]		M	Unique identifier list for scripts.

1073594 v1 **Processing after script decompression**

Version history:

Purpose: To describe the processing of compressed file.
Source:
Verification Method: Test
Legacy ID:

After successful decompression of the script compressed file, the original compressed file shall be deleted to avoid memory occupation.

The script file obtained by decompressing the script compression package shall be overwritten according to the rule *ReqID 1073566*.

1073591 v1 Script compression method

Version history:

Purpose: To define the compression method when upload bulk-data script.
Source:
Verification Method: Test
Legacy ID:

Batch processing of scripts allows for the use of compressed packaging. The compression method shosll support zip format.

•Support of target modes

4.12.1.4.14 Support of target modes

This chapter defines the information used to query the current and possible states of the target entity, as well as the operations required to trigger state transitions, in order for the entity to perform specific interactions (such as executing operations or changing entity data/configuration) to ensure that the entity is in a specific state, so that diagnostic tools can further effectively process related requests. The API specifies the general provision and control basis for patterns, which ExVe manufacturers can use to publicly and interactively specify patterns (such as controlling input sources in HPC environments), and supports two ways of specifying patterns: explicit (defining activation patterns before interaction) and prompt based (including activation patterns during interaction).

1043842 v1 Explicit control for security access

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

The API method - explicit control of entity states for security access shall add new attribute "x-security-pincode" to participate in security access algorithm caluculations. The request body shall be as follows:

Attribute	Type	Convention	Description
value	string	M	The enumeration value to be set for the specified mode.
mode_expiration	string	O	Defines after how many seconds the mode expires and should therefore be automatically reset to the mode’s default value.
x-security-pincode	string	M	security constant, used as an input for security access computation.

1043690 v1 Target Mode APIs

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

The methods used for the Target Modes APIs are as follows:

No.	API Method Description	HTTP Method	URI	Support	Comment
1	Query for modes of an Entity	GET	/{entity-path}/modes	M	
2	Retrieve details of a mode	GET	/{entity-path}/modes/{mode-id}	M	
3	Explicit control of Entity states via their defined modes	PUT	/{entity-path}/modes/{mode-id}	M	

1043966 v1 **The parameter mode_expiration**

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

The mode_expiration shall define after how many seconds the mode expires and therefore be automatically reset to the mode's default value. If this parameter is ingnored or the value is 0, it defaults to no time limit for the mode.

•Locking

4.12.1.4.15 Locking

This chapter mainly defines that in the SOVD system, the execution of certain entities' methods requires a locking mechanism to avoid conflicts between different clients or operations. Therefore, the system supports client locking of resources to ensure exclusive access during critical operations. Locks can apply to specific scopes or to the entire entity. A client can hold locks for multiple scopes simultaneously, achieving flexible resource control.

1043691 v1 **Locking APIs**

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

The methods used for the Target Modes APIs are as follows:

No.	API Method Description	HTTP Method	URI	Support	Comment
1	Acquire a lock on an Entity	POST	/{entity-path}/locks	O	
2	Query for locks of an Entity	GET	/{entity-path}/locks	O	
3	Retrieve details of a lock	GET	/{entity-path}/locks/{lock-id}	O	
4	Modify the expiration time of a lock on an Entity	PUT	/{entity-path}/locks/{lock-id}	O	
5	Release an acquired lock on an Entity	DELETE	/{entity-path}/locks/{lock-id}	O	

•Restarting an Entity

4.12.1.4.16 Restarting an Entity

This chapter mainly describes the content related to restarting entities, which is the process of restarting/restarting running entities (components, applications, functions). For example, when an entity feels slow or unresponsive, performing a restart can restore it to normal operation. Forced restarts are supported and can cause interruptions or running processes.
Note:

- Region restart is not supported
- Before restarting the entity, it is recommended to identify the entities that may be affected in advance, which can be confirmed by querying sub entities or associated entities
- Restarting an entity may result in errors being detected by other entities
- Restarting will delete temporary resources

1043703 v2 **Restarting an Entity APIs**

Version history:

Purpose: To define the API method for restarting.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The methods used for the Restarting an Entity APIs are as follows:

No.	API Method Description	HTTP Method	URI	Support	Comment
1	Read the status of an Entity	GET	/{entity-path}/status	O	
2	Start an Entity	PUT	/{entity-path}/status/start	O	
3	Restart an Entity	PUT	/{entity-path}/status/restart	M	
4	Forced Restart an Entity	PUT	/{entity-path/status/restart/force-restart	O	
5	Shutdown an Entity	PUT	/{entity-path}/status/shutdown	O	
6	Forced Shutdown an Entity	PUT	/{entity-path/status/force-shutdown	O	
7	Read the OEM-specific status of an Entity	GET	/{entity-path}/x-api-status	M	Geely specific API

•Handling of bulk-data

4.12.1.4.17 Handling of bulk-data

The chapter describes to enable the SOVD client to manage batch data (such as kernel dump files, temporarily generated log files, etc.), including downloading, uploading (including source proof and other security artifacts), and deleting batch data; This feature aims to provide a unified abstraction for different underlying file systems and implement flexible data transmission mechanisms through standardized protocols, meeting the needs of vehicle diagnostic systems in remote diagnosis, fault analysis, OTA upgrades, and other aspects.

1043705 v1 **Bulk-data APIs**

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

The methods used for the Bulk-data APIs are as follows:

No.	API Method Description	HTTP Method	URI	Support	Comment
1	Retrieve bulk-data categories of an Entity	GET	/{entity-path}/bulk-data	M	
2	Retrieve details of bulk-data resource	GET	/{entity-path}/bulk-data/{category}	M	
3	Download bulk-data	GET	/{entity-path}/bulk-data/{category}/{bulk-data-id}	M	
4	Upload bulk-data	POST	/{entity-path}/bulk-data/{category}	M	
5	Delete all bulk-data defined by category	DELETE	/{entity-path}/bulk-data/{category}	M	
6	Delete specific bulk-data resource	DELETE	/{entity-path}/bulk-data/{category}/{bulk-data-id}	M	

1043982 v1 **Data Category and Logical Organization**

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

Bulk data is logically grouped by category (BulkDataCategory) and defined in diagnostic database.

Each category can contain multiple specific data resources and can be filtered through attributes such as creation time. This structured organization makes it easier for clients to access specific types of data on demand.

Bulk data resources can be associated with raw data sources (such as log files) to achieve mapping from raw data to diagnostic resources.

1043983 v1 **Metadata management**

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

ECU entities shall support querying balk data metadata to assist clients in determining whether to perform further operations. Metadata should include at least:

- File size;
- Creation time;
- Data type (MIME type);
- Storage path or unique identifier;
- Access permissions;
- Signature information.

1075488 v1 **The bulk-data identifier**

Version history:

Purpose: To define the unique bulk-data identifier for indexing.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The bulk-data identifier, e.g. log file, execution result file, is designed based on file name (excluding extension).

Attention: If the file name is the same but the extension is different, it will cause bulk-data identifer to match the wrong file. Both the client and server should avoid this scenario.

1073973 v1 The overwrite of same bulk data

Version history:

Purpose: To define the processing method for uploading the same bulk data.
Source:
Verification Method: Test
Legacy ID:

For bulk data uploads, for files with the same file name and extension, the new one will overwrite the old one.

•Logging

4.12.1.4.18 Logging

This chapter mainly describes how the SOVD logging method provides access to entity log information. Diagnostic tools can access log information through a collection of entry resources. The source of each entry can come from different contexts. These log information are obtained through a collection of entry resources, and each destination source may come from a different context (representing a specific log format, such as RFC5424 or AUTOSAR-DLT). SOVD maps specific elements of these log formats to SOVD standardized values.

1043706 v1 Logging APIs

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

The methods used for the Logging APIs are as follows:

No.	API Method Description	HTTP Method	URI	Support	Comment
1	Retrieve resources of an SOVD log	GET	/{entity-path}/logs	M	
2	Read log entries from an SOVD log	GET	/{entity-path}/logs/entries	M	
3	Configure SOVD log	PUT	/{entity-path}/bulk-data/{category}/{bulk-data-id}	M	
4	Retrieve the current SOVD log configuration from an Entity	GET	/{entity-path}/logs/config	M	
5	Reset SOVD logging configuration to default	DELETE	/{entity-path}/logs/config	M	

•Communication-logs

4.12.1.4.19 Communication-logs

This chapter defines the method for SOVD servers to provide recording of vehicle communication (bus tracking), such as on CAN and automotive Ethernet. The SOVD API supports two different mechanisms (local storage, dynamic storage) to retrieve communication logs from the SOVD server for further analysis of issues in the vehicle

Note:
Local storage: Communication logs are stored locally by the SOVD server and can be retrieved after stopping the logs.
Dynamic storage: Communication logs are sent to the SOVD client when they are recorded, rather than being stored by the SOVD server.

1043708 v1 Communication-Logs APIs

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

The methods used for the Communication-Logs APIs are as follows:

No.	API Method Description	HTTP Method	URI	Support	Comment
1	Query for communication-logs from an Entity	GET	/{entity-path}/communication-logs	O	
2	Configure communication-logs for an Entity	POST	/{entity-path}/communication-logs	O	
3	Read the status of a communication-logs	GET	/{entity-path}/communication-logs/{communication-log-id}	O	
4	Control a communication-log	PUT	/{entity-path}/communication-logs/{communication-log-id}	O	
5	Delete communication-logs	DELETE	/{entity-path}/communication-logs/{communication-log-id}	O	

4.12.1.5 SOVD Authentication

•General Requirements

4.12.1.5.1 General Requirements

SOVD身份验证的目的至少：一是验证外部客户端的身份凭证和访问权限，二是授权外部客户端访问令牌，以此保护车辆中可访问实体的资源，防止在访问SOVD服务器时滥用功能或攻击破坏。
本章节仅设计客户端和服务器之间的验证和授权过程，而端到端的安全，如编码和编程数据的签名，不在本节中规定，而是分布在各个SOVD API章节中。

The purpose of SOVD authentication at least: firstly, to verify the identity credentials and access permissions of external clients; secondly, to authorize external clients to access tokens, in order to protect the resources of accessible Entities in the vehicle and prevent the abuse of functions or attacks when accessing SOVD servers.

This chapter only focuses on the validation and authorization process between the client and server, while end-to-end security, e.g., signatures for coding and programming data, is not specified in this sector but is distributed across various SOVD API chapters.

1039363 v1 Request with token

Version history:

Purpose:
Source:
Verification Method: Analysis
Legacy ID:

Before executing requests on the SOVD server, the SOVD client should always have a valid token in the/token URI when sending SOVD API request.

1039362 v1 Resource Access of SOVD Server

Version history:

Purpose:
Source:
Verification Method: Analysis
Legacy ID:

The SOVD servers protect all resources against unauthorized access. The Vehicle Authorization Server does not directly prevent the SOVD server from being accessed by the SOVD client, but it does provide information, with what kind of access an SOVD client is authenticated. Only if the SOVD client has the appropriate access rights, the SOVD server grants the access to the requested resources and methods.

For SOVD methods, which enumerate resources (e.g., data-lists, subcomponents, ...) or event sources, that send contents of resources (e.g. cyclic subscriptions, triggers) the server ensures that the client does not receive information about resources which are not accessible by the client based on its current authorization. In this case the server may filter out the inaccessible resources.

Therefore, a receiving SOVD server may only process further requests with a valid token. If no token or an invalid token is provided with a request, the SOVD server needs to deny the request or redirect the SOVD client to the Vehicle Authorization Server. Therefore, a token provides the legitimacy to the SOVD client to communicate with the SOVD server.

With each request, the SOVD client sends a valid token to the receiving SOVD server. Only if the token validation was successful, the SOVD server may execute the request.

1039370 v1 The ability of Token Verification

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

The in-vehicle authorization server or Private SOVD Server should require its own token verification capability.

1039359 v1 Vehicle authorization server

Version history:

Purpose:

Source:
Verification Method: Analysis
Legacy ID:

The vehicle shall have a vehicle authorization server that provides identity verification, token issuance, verification of issued tokens, and permission confirmation. This enables the SOVD server to decide how to handle requests from SOVD clients.

•Offline authentication and authorization

4.12.1.5.2 Offline authentication and authorization

1039404 v1 **Access token**

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

Token shall uses JWT format and needs to meet the standard declaration specified in CSRS requirements.

The Audience declaration in the payload should include at least the host name information of the gateway. Additionally, it should include at least a private declaration: client role (from client certificate), requested access permissions.

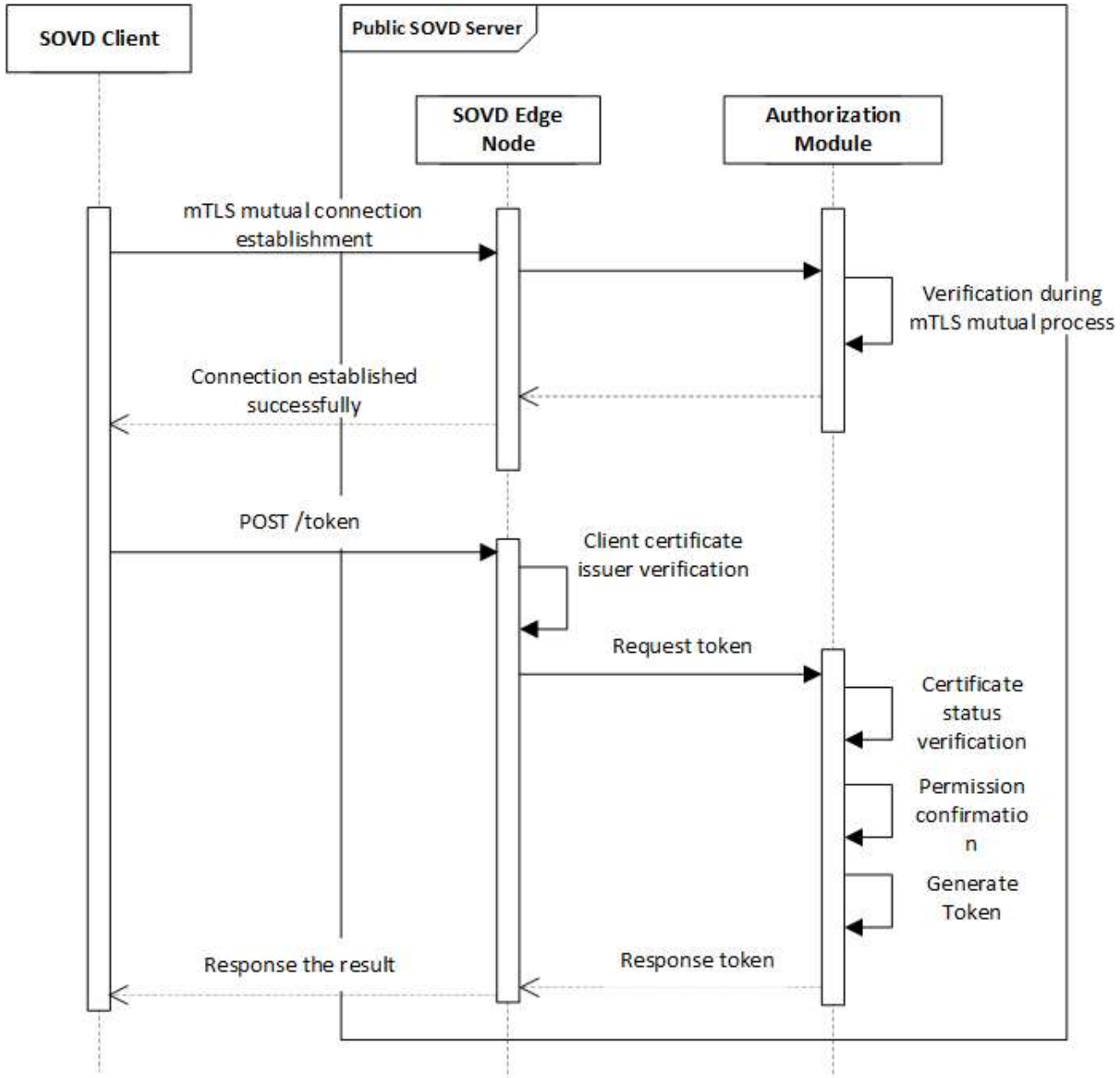
1039374 v1 **Authentication and authorization process**

Version history:

Purpose:
Source:
Verification Method: Analysis
Legacy ID:

The verification and authorization process in offline mode is as follows:

- Establish mTLS mutual authentication between SOVD client and Public SOVD server;
- Based on mTLS connection, SOVD client requests a token.



1039394 v1 **Client certificate identity verification**

Version history:

Purpose: To avoid connections from clients holding non diagnostic tool certificates, SOVD Gatewy will verify the identity of the client certificate.

Source:

Verification Method: Analysis AND Test

Legacy ID:

When the SOVD Edge Node receives a POST/token from the SOVD client, it shall verify whether the client certificate in the mTLS process is a diagnostic device specific certificate, for example, by comparing whether the certificate issuer CN field contains the Diagnostic Issuing character.
If the verification fails, it is considered to be an unauthorized diagnostic device client connection, and a 400 response code shall be returned, and the TCP connection shall be closed

1039376 v2 Request a token

Version history:

Purpose: The SOVD client can request a token via this method. The Authorization Server accepts requests to create tokens from authenticated SOVD clients only. Therefore, an SOVD client authenticates against the Authorization Server or have some other credentials to proof its identity prior a token is created and returned via the SOVD API. An SOVD client, which cannot be authorized is not be able to request a token through the SOVD API.

Source:

Verification Method: Test

Legacy ID:

The following method is the step for requesting a token after https with mTLS established.

Parameters	Requirements
Method	POST /token
Path Parameters	No support
Query Parameters	No support
Request Headers	Content-Type: application/json
Request Body	See Table1: Request Body for requesting a token
Response Status Codes	See Table2: Response Status Code for requesting a token
Response Body	See Table3: Response Body for requesting a token

Table1: Request Body for requesting a token

Attribute	Parent	Type	Length	Convention	Description
client_ocsp_response	-	string	-	C1	OCSP response data (i.e., DER encoded Base64) for certificate, used to verify whether the certificate has been revoked.
scopes	-	Array object		C2	List of permission scope applied for. The final returned permission is the intersection permission of the certificate and copies. If omitted, all permissions specified in the role in the certificate will be used.
C[1]: This parameter is associated with pre-sales and non pre-sales mode. When ECU is in pre-sales mode, there is no need to carry this parameter. When ECU is in non pre-sales mode, it is necessary to carry the parameter "client_ocsp_response". C[2]: Used when scope permission constraints are required.					

Table2: Response Status Code for requesting a token

Status Code	Response Body	Description
200	Table 3	The request was successful.
400	**GenericError**	If there is a verification error or something else is missing or is invalid. (parameter missing, no diagnostic certificate, etc)

Table3: Response Body for requesting a token

Attribute	Parent	Type	Length	Convention	Description
access_token	-	string	-	M	JWT format access token, containing information such as permissions, validity period, vehicle information, etc.
token_type	-	string	-	M	Default is Bearer
expires_in	-	string	-	M	The expiration time of the token, in seconds, defaults to 21600 seconds.

1039393 v1 **mTLS connection**

Version history:

Purpose: mTLS mutual authentication is the foundation of SOVD diagnostic communication.
Source:
Verification Method: Test
Legacy ID:

The SOVD client and Public SOVD Server shall establish an mTLS mutual authentication connection.

The SOVD client shall use a dedicated diagnostic tool certificate as the client certificate.

4.12.1.6 SOVD Vehicle Update

•General Requirements

4.12.1.6.1 General Requirements

In the classic approaches, the diagnostic tester has control over the swdl process through UDS protocol and the vehicle simply executes the prescribed steps. The classic approaches requires high time accuracy and transmission stability. In contrast to the classic approach, the SOVD update API puts the control into the Vehicle Update Module and leverages the OTA capabilities of a connected vehicle to retrieve, download and install the available vehicle updates.

The follow chapter describe the new update procedure - stepwise vehicle update. With the stepwise vehicle update the SOVD client can control which update is installed and when the update is prepared and executed. The following figure illustrates the brief process of stepwise vehicle update.

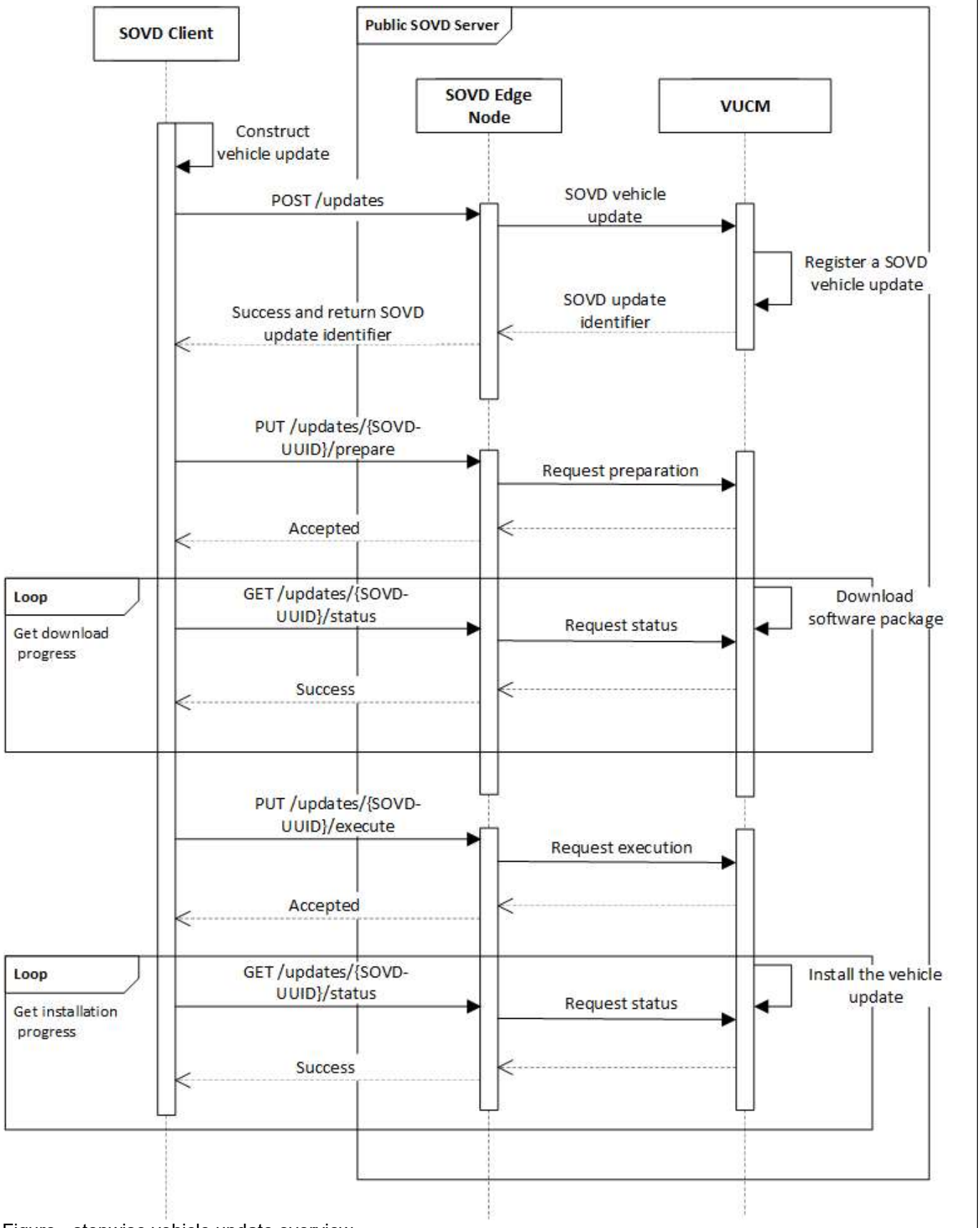


Figure - stepwise vehicle update overview.

1041390 v1 Cloud interaction restrictions during SOVD vehicle update

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

During the SOVD vehicle update, the communication link between VUCM and the remote OTA cloud backend does not require any communication interaction. VUCM does not need to send any requests to the cloud backend.

1041382 v1 Definition of Vehicle update and configuration module

Version history:

Purpose:
Source:
Verification Method: Analysis
Legacy ID:

A software module called Vehicle Update and Configuration Module (VUCM) has been defined on the vehicle. It serves as an update module for the vehicle, managing the business of vehicle updates and configurations.
The SOVD Edge Node shall receive an API method {base_uri}/updates [/{xxx}] based on vehicle software update and direct it to this software module.

1041388 v1 Identification of the vehicle update

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

VUCM shall be able to distinguish between SOVD vehicle update from the proximity and OTA vehicle update from the remote.

1041524 v1 The handle of OTA update

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

For OTA update identifier returned through the SOVD API method - Query for the update from vehicle, it shall,

- support to use the SOVD API method - Delete a vehicle update to delete the resource.
- support to use the SOVD API method - Retrieve details of a vehicle update to return detailed information for a given update identifier.
- not support to prepare or execute the OTA update, otherwise return a response status code with an error.

•Detailed Update Information Requirements

4.12.1.6.2 Detailed Update Information Requirements

1041396 v1 **API Mehotd - Query for the update from vehicle**

Version history:

Purpose: To returns the available update for the vehicle.
Source:
Verification Method:
Legacy ID:

The following method is to query for the update from vehicle

Parameters	Requirements
Method	GET /updates
Path Parameters	No support
Query Parameters	See Table1: Query parameters
Request Headers	Content-Type: application/json
Request Body	No support
Response Status Codes	See Table2: Response Status Code
Response Body	See Table3: Response Body

Table1: Query parameters

Attribute	Parent	Type	Length	Convention	Description
origin	-	UpdateOrigins	-	O	Defines from where the update is requested, e.g., inside a workshop or by an OTA update request. Default: remote

Table2: Response Status Code

Status Code	Response Body	Description
200	Table 3	The request was successful.

Table3: Response Body

Attribute	Parent	Type	Length	Convention	Description
items	-	string[]		M	Update assignment identifier. E.g., SOVD update identifer, OTA update identifier.

1041466 v1 **API method - Retrieve details of a vehicle update**

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

The following method is to retrieve deails of a vehicle update.

Parameters	Requirements
Method	GET /updates/{Update-UUID}
Path Parameters	Update-UUID
Query Parameters	See Table1: Query parameters
Request Headers	Content-Type: application/json
Request Body	No support
Response Status Codes	See Table2: Response Status Code
Response Body	See Table3: Response Body

Table1: Query parameters

Attribute	Parent	Type	Length	Convention	Description
include-schema	-	boolean	-	O	Specifies whether the response should include schema information or not. The default is false.

Table2: Response Status Code

Status Code	Response Body	Description
200	Table 3	The request was successful.

Table3: Response Body

Attribute	Parent	Type	Length	Convention	Description
id	-	string	36 bytes	M	Identifier for the update. (equal to the installationOrderId of available assignment)
update_name	-	string	-	M	Name of the update. (equal to the name of available assignment)
automated	-	boolean	-	O	Specifies whether the update package can be installed automatedly or not. If missing, the attribute defaults to false.
origin	-	UpdateOrigins[]	-	M	List of origins for which the update is applicable. There will be at least one value in the attribute.
notes	-	ReleaseNoteList[]	-	O	Release notes for the updates, may also be a URI. (equal to the releaseNoteList of available assignment)
preconditions	-	Preconditions[]	-	O	Preconditions which shall be fulfilled for installing the update (e.g., doors locked, parking, ...). In general, not being able to fulfill the preconditions would block the preparation of the update. (equal to the preconditions of available assignment)
duration	-	integer	-	M	Time for which the vehicle is not available when the update is installed and activated. The time is defined in seconds. (equal to the totalintallationTime of available assignment)
size	-	integer	-	M	The download size of the update is defined in kilo

					bytes. (equal to the downloadsize of available assignment)
[Other parameter]	-	OtherParameter[]	-	O	Other parameters defined in the available assignment.

1041397 v2 **Update origins definition**

Version history:

Purpose: To determine the update origins to specify in which environment or by whom the software update is triggered by the client.
Source:
Verification Method: Test
Legacy ID:

The update origins are divided into the following:

Value	Trigger source	Environment	Description
remote	OTA Cloud	URL based download	The update is performed over-the-air.
proximity	SOVD Client	URL based download	The update is performed in a manufactory or a workshop, and the delivery of the vehicle update package is actively downloaded through a URL link by VUCM.
x-proximity-pushing	SOVD Client	Files based transfer	The update is performed in a manufactory or a workshop, and the delivery of the vehicle update package is passively receiving uploads from SOVD Client.
x-remote-rescue	OTA Cloud	URL based download	The update is performed over-the-air.

If VUCM obtains the update origin is x-proximity-pushing, it will ignore the url from available assignment and wait for data pushing from the SOVD client.

1041394 v1 **Vehicle update information**

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

The SOVD client will request the query method of the SOVD Update API, and VUCM shall return the currently stored and valid update identifier, i.e., vehicle update created in SOVD method or remote OTA vehicle update, or none.

•Update Requirements

4.12.1.6.3 Update Requirements

1044347 v1 **API - Method - Delete vehicle update**

Version history:

Purpose:
Source:
Verification Method: Test

Legacy ID:

SOVD Client shall request to delete one vehicle update through SOVD API method.

The following method is to delete a vehicle update.

Parameters	Requirements
Method	DELETE /updates/{Update-UUID}
Path Parameters	{Update-UUID}
Query Parameters	No support
Request Headers	NA
Request Body	No support
Response Status Codes	See Table1: Response Status Code
Response Body	No support

Table1: Response Status Code

Status Code	Response Body	Description
204	-	The request was successful.
405	-	Deleting is not foreseen for an update-package is in phase execute.

1040209 v1 API Method - Register a vechicle update

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

SOVD Client shall request parameters and json files related to the vehicle update, with the file part implemented according to the OTA function, when creating the vehicle update through SOVD Update API method.

The following method is to register a vehicle update.

Parameters	Requirements
Method	POST /updates
Path Parameters	No support
Query Parameters	No support
Request Headers	Content-Type: multipart/form-data Content-Length: size = the content Content-Disposition: form-data; name="{file name}"; filename="{file name with file extension}"
Request Body	See Table1: Request Body for registering a vehicle update
Response Status Codes	See Table2: Response Status Code
Response Body	Depend on the corresponding response status code.

Table1: Request Body for registering a vehicle update

Attribute	Parent	Type	Length	Convention	Description
--bondary					
sovdResourceDuration	-	integer	-	M	Minutes (Min). The retention time of SOVD upgrade

					resources on the VUCM side. After the expiration of the time limit, the resources of this SOVD-UUID will be automatically deleted.
origins	-	UpdateOrigins	-	M	It is an enumeration. Distinguish whether the data source is actively obtained by the Updater Module through the URL or pushed by the client through the stream data. **UpdateOrigins, is defined in ReqID 1041397.**
signature	-	string	-	O	Signature of transmitted files
--bondary name="AvailableAssignment"; filename="AvailableAssignment.json";					
[File1 Content][1]	-	raw data		M	If origins is set to proximity, VUCM ignores the field specificedurls and urls defined in the SOVDUpdate file. At this point, software downloads are no longer initiated by VUCM by accessing URLs, but instead SOVDClient uses a new interface to proactively push to VUCM.
--bondary name="AssignmentFile"; filename="AssignmentFile.json";					
[File2 Content]2	-	raw data		C	If origins is set to proximity, VUCM ignores the field specificedurls and urls defined in the SOVDUpdate file. At this point, software downloads are no longer initiated by VUCM by accessing URLs, but instead SOVDClient uses a new interface to proactively push to VUCM.
--bondary					
Note1: Follows the logic of OTA subsystem requirements Note2: Follows the logic of OTA subsystem requirements and be applicable for Geely 3.0 Platform.					

Table2: Response Status Code for registering a vehicle update

Status Code	Response Body	Description
201	Table 3	The request was successful.
409	**GenericError**	If the update assignment conflicted, i.e. Only one vehicle upgrade is allowed at a time.

Table3: Response Body for registering a vehicle update

Attribute	Parent	Type	Length	Convention	Description
id	-	string		M	Identifier for the update. I.e., SOVD update identifer.

1041300 v1 **API Method - Trigger a vehicle package download**

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

The following method is to trigger a vehicle package download.

Parameters	Requirements
Method	PUT /updates/{SOVD-UUID}/prepare
Path Parameters	{SOVD-UUID}
Query Parameters	No support
Request Headers	Content-Type: application/json, if need.
Request Body	No suppport
Response Status Codes	See Table1: Response Status Code
Response Headers	Redirect the SOVD client for requesting the status: Location: {base_uri}/updates/{SOVD-UUID}/status
Response Body	Depend on the corresponding response status code.

Table1: Response Status Code

Status Code	Response Body	Description
202	-	No error is detected.
409	**Table2**	An update is already in preparation and not yet done or aborted.

Table2: Error response

Attribute	Parent	Type	Length	Convention	Description
error_code	GenericError	string		M	SOVD standardized error code. I.e., • **update-preparation-in-progress**: when the update is already in download phase and not yet done or aborted. • **update-execution-in-progress**: when the update is already in installation phase and not yet done or aborted.
message	GenericError	string	-	M	Message describing the problem in more detail
parameters	GenericError	string	-	M	Key-valie-list which provides additional information on the error.
id	parameters	string	-	M	Identifier for the update, e.g., SOVD update identifer, OTA update identifier.

1041301 v1 API Method - Trigger a vehicle update installation

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

The following method is to trigger a vehicle update installation.

Parameters	Requirements
Method	PUT /updates/{SOVD-UUID}/execute
Path Parameters	{SOVD-UUID}
Query Parameters	No support
Request Headers	Content-Type: application/json
Request Body	See Table1:
Response Status Codes	See Table2: Response Status Code

Response Headers	Redirect the SOVD client for requesting the status: Location: {base_uri}/updates/{SOVD-UUID}/status
Response Body	Depend on the corresponding response status code.

Table1: Request Body

Attribute	Parent	Type	Length	Convention	Description
availableFlashTime	-	float	4 bytes	M	Minutes (Min). The installation time that the vehicle condition can support. Compare with the update time in the available assignment, and if it is less than the update time, VUCM will report installation delay.
signaturesequence	-	boolean	-	O	Verification sequence of signature_dev and signature_prod. True means to validate signature_dev first, while false means to validate signature_prod first. If the attribute is omitted, it defaults to false.
ischeckcompletecompatible	-	boolean	-	O	The request of checkCompleteAndCompatible shall be sent or not. True, need to send. False, no need to send. If the attribute is ignored, it defaults to false.
ischeckmemory	-	boolean	-	O	The request of checkMemory shall be sent or not. True, need to send. False, no need to send. If the attribute is omitted, it defaults to false.
time76to36	-	integer	-	O	The interval time from the previous UDS $76 response to the next UDS $36 request, in millisecond. If the attribute is omitted, the client will use best performance.
waitNRC78	-	integer	-	O	Client counters updated after receiving UDS resposne NRC 78, in millisecond. If the attribute is omitted, the client shall use the default value 5000ms.
P6client	-	integer	-	O	Timeout for the client to wait after the successful transmission of a request message for the complete reception of the corresponding response message, in millisecond. If the attribute is omitted, P6client is set to 6000 ms.
P4servermax	-	integer	-	O	The time between the reception of a request and the start of the transmission of the final response at the server side, in millisecond. If the attribute is omitted, P4servermax is set to 1000000 ms.

Table2: Response Status Code

Status Code	Response Body	Description
202	-	No error is detected.
409	**Table3**	An update is already in preparation/execution and not yet done or aborted.

Table3: Error response

Attribute	Parent	Type	Length	Convention	Description
error_code	GenericError	string		M	SOVD standardized error code. I.e., • **update-preparation-in-progress**: when the update is already in download phase and not yet done or aborted. • **update-execution-in-progress**: when the update is already in installation phase and not yet done or aborted.
message	GenericError	string	-	M	Message describing the problem in more detail

parameters	GenericError	string	-	M	Key-valie-list which provides additional information on the error.
id	parameters	string	-	M	Identifier for the update, e.g., SOVD update identifer, OTA update identifier.

1041319 v1 **SOVD update Identifier**

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

The SOVD update identifier is included in the issued available assignment file, and is represented by {SOVD-UUID}

1041657 v1 **Software package download**

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

The SOVD client will trigger VUCM to initial vehicle package download through the SOVD API mehtod */prepare*, and VUCM shall skip download authorization to start.
The conditions for software package download and HMI display can refer to the download in one car OTA update.

To avoid triggering installation by manually clicking the "Schedule update" or "Update Immediately" button on the HMI screen after successful download through the SOVD API method. VUCM shall pop up a window indicating that it is currently in SOVD vehicle update.

1041637 v1 **The processing of registering a vehicle update**

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

After registering a SOVD vehicle update, VUCM shall return whether the SOVD vehicle update was created successfully or not. It means that if there are currently no SOVD or OTA update, VUCM shall always accept and store the requested parameters and files, then returen the vehicle update identifier. If the SOVD update registeration is unsuccessful, the requested parameters and files will not be stored and need to be cleared.

1041625 v1 **The sovdResourceDuration definition and handle**

Version history:

Purpose:
Source:

Verification Method: Analysis AND Test
Legacy ID:

The sovdResourceDuration indicates the retention time of SOVD update resources on the VUCM side. After the expiration of the time limit, the resources of this SOVD Update identifier will be automatically invalidated.

- When in the pending state of preparing or executing, if the duration of the SOVD update resource times out, the resource will be directly invalidated.
- When in the process of preparing or executing, that is, when the status is inProgress, the duration of SOVD update resources shall pause timing.
- When in the failed or completed state of preparing or executing, the duration of SOVD update resources times out, and the resource will be directly invalidated.

1041646 v1 Update conflict handling

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

There are some conflicting scenarios in the vehicle update:

- If the vehicle already has an OTA update and a new SOVD vehicle update is registered, a response status code 409 conflicted shall be returned with an error code indicating the current OTA update status and identifier.
- If the vehicle already has an OTA update, when requesting update preparation or execution, a response status code 409 conflicted shall be returned with an error code indicating the current OTA update status and identifier.
- If the vehicle already has an SOVD vehicle update, registering a new SOVD update again, a response status code 409 conflicted shall be returned with an error code indicating the current SOVD update status and identifier.

1041634 v2 Update origin request

Version history:

Purpose: Create an update with an origin.
Source:
Verification Method: Test
Legacy ID:

When registering a vehicle update through the SOVD API method, the SOVD client shall inform VUCM of the current update origin for the SOVD vehicle update.

1041666 v1 Vehicle update and resource handling

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

In general, registering a new vehicle update or actively deleting/revoking a vehicle update will delete the old update information and resource data. In addition, when the SOVD vehicle update resource time expires, VUCM shall actively set the SOVD vehicle update as invalid.

1041658 v1 **Vehicle update installation**

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

The SOVD client will trigger VUCM to initial vehicle update installation through the SOVD API mehtod */execute*, and VUCM shall skip installation authorization to start.
The conditions for vehicle update installation and HMI display can refer to the installation in one car OTA update. In addition, there are special agreements for some steps in the installation process defined in *LC - InstallationMasterBOOT*:

- The controller for deploying VUCM shall be installed using distributed sensing and does not require the first installation to be completed first.
- Do not perform the pre-check.
- Post verification only retains installation summary and exception report output, and notification of HMI installation results.
- And other requirements that meet the need of the factory and after-sales update.

•Update Status Requirements

4.12.1.6.4 Update Status Requirements

1041733 v1 **Processing of delayed installation**

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

When SOVD client receives installation updates postpone from VUCM during installation execution, it allow SOVD clients to request SOVD API method */execute* to install again.

1041675 v1 **Read the status of the vehicle update**

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

SOVD Client shall request the update status through following method:

Parameters	Requirements
Method	GET /updates/{SOVD-UUID}/status
Path Parameters	{SOVD-UUID}
Query Parameters	No support
Request Headers	Content-Type: application/json

Request Body	No support
Response Status Codes	See Table1: Response Status Code
Response Body	See Table2: Response Body

Table1: Response Status Code

Status Code	Response Body	Description
200	Table 3	The request was successful.

Table2: Response Body

Attribute	Parent	Type	Length	Convention	Description
phase	-	**Phase**	-	M	Current phase of the update process. **Phase** includes: • prepare, Update preparation • execute, Update execution
status	-	**Status**	-	M	Status of the preparation or execution. **Status** is defined in Table3.
progress	-	integer	-	M	Progress of the preparation or execution in percent.
subprogress	-	Progress[]	-	O	Progress of the updated ECUs/Entities.
- entity	Progress[]	string:uri-reference	-	M	URI of the ECU/Entity.
- status	Progress[]	**Status**	-	M	Status of the preparation or execution. **Status** is defined in Table3.
- progress	Progress[]	integer	-	O	Progress of the preparation or execution in percent for ECU/Entity.
- error	Progress[]	GenericError	-	O	Only set if status is failed
error	-	GenericError	-	O	Only set if status is failed

Table3: Status type

Status	Description
phase: prepare	
pending	The preparation of the update has not yet started.
inProgress	The preparation of the update has started.
failed	The preparation of the update has failed. Details on the reason are specified in the error attribute.
completed	The preparation of the update has been completed successfully, but the execution of the update has not yet been requested.
downloadPause	The preparation of the update has been suspended.
phase: execute	
pending	The preparation of the update has been completed, but the update itself has not yet been started.
inProgress	The installation of the update has started.
failed	The installation of the update has failed. Details on the reason are specified in the error attribute.
completed	The installation of the update has been completed successfully.
installationDelay	The installation of the update has been postponed.

•Update Report Requirements

4.12.1.6.5 Update Report Requirements

1041385 v2 **Data acquisition for vehicle update**

Version history:

Purpose: To define the entity and category of bulk-data API method.
Source:
Verification Method: Test
Legacy ID:

The SOVD client shall specifie entity Identifier apps/VUCM to retrieve necessary data resources and logs during the vehicle update process.
Among them, the bulk-data category is set as "ota_assignment_data". In "ota_assignment_data", the Installation Summary identifier and Error Report identifier are used to identify the installation summary and exception report of one vehicle update, respectively.
Note: The batch acquisition method for data resources and logs refers to the bulk-data API method of the SOVD API.

1041734 v2 **Data storage for vehicle update**

Version history:

Purpose: Store the necessary log data for analysis.
Source:
Verification Method: Analysis AND Test
Legacy ID:

VUCM shall be able to store installation summaries and error reports throughout the whole vehicle update. It is recommend to refer to the naming convention defined in OTA vehicle updates.

4.12.1.7 SOVD Private Server

•Requirements

4.12.1.7.1 Requirements

1044319 v1 **As the in-vehicle server**

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

The server shall listen for TCP connections with destination port 7690. The source port of the connection initiated by the client is different, so there would be multiple connections at the same time.

1045995 v1 **The number of connection for SOVD diagnsotic**

Version history:

Purpose:
Source:

Verification Method: Test
Legacy ID:

SOVD private servers shall be able to support at least 10 TCP connections simultaneously.

4.12.1.8 HTTP Protocol Requirement

•Requirement

4.12.1.8.1 Requirement

SOVD supports HTTP/1.1 or HTTP/2.0 protocols, with main differences in concurrency, resource overhead, and transmission efficiency. When implementing SOVD diagnostics using HTTP/2.0, only the performance improvements brought by its multiplexing, header compression, and binary framing can be utilized, without enabling any HTTP/2.0 specific control features such as server push, stream priority, and dependent streams.

1044365 v1 **In-vehicle SOVD communication vlan allocation**

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

The SOVD protocol communication in vehicle uses HTTP protocol and the same VLAN tag as conventional DoIP diagnostics.

4.12.1.8.1.1 HTTP/1.1

1043035 v1 **HTTP/1.1 Communication**

Version history:

Purpose:
Source:
Verification Method:
Legacy ID:

When a client initiates multiple parallel requests to the same target entity, since HTTP/1.1 does not support true multiplexing, it is usually necessary to establish independent TCP connections (i.e. occupy independent socket resources) for each request. For example, if the client sends three parallel requests to the target entity simultaneously, it must establish three independent TCP connections separately, occupying three socket resources. When the socket resources reach their limit, new requests will be queued and wait for existing connections to be released.

4.12.1.8.1.2 HTTP/2.0

1043287 v1 **HTTP/2.0 Communication**

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

HTTP/2.0 introduced multiplexing mechanism, allowing multiple requests and responses to be transmitted in parallel over the same TCP connection. Only one TCP connection needs to be established between the client and the same target entity to process multiple requests concurrently. All requests and responses are logically isolated and managed through a unique Stream ID.

- The stream initiated by the client uses odd IDs (such as 1, 3, 5...);
- The response stream returned by the server uses even IDs (such as 2, 4, 6, etc.);
- Stream IDs are dynamically allocated by both communication parties as needed, without the need for prior negotiation.

1043289 v1 **HTTP/2.0 additional requirement**

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

HTTP/2.0 encapsulates and transmits all data in the form of binary frames, with a recommended maximum length of 16384 bytes (16KB) per frame. If a request or response exceeds this limit, the protocol stack shall automatically split it into multiple DATA frames. The receiving end reassembles frames belonging to the same data stream based on the Stream ID to ensure the integrity and orderliness of the original message.

4.12.1.9 SOVD - Classic Diagnostic Adapter

•Requirements

4.12.1.9.1 Requirements

1074702 v1 **Connect Vehicle Implementation Rules**

Version history:

Purpose: Define the implementation rules after establishing the vehicle DoIP connection.
Source:
Verification Method: Test
Legacy ID:

Under the current implementation strategy, the CDA module shall initiate vehicle functional addressed TesterPresent request with SPRMIN=TRUE after connecting vehicle successfully.

After starting the connection, if the OBD connection status is unplugged or the wireless is turned off and SOVD2UDS data stream is not received for 60 seconds, the connected vehicle will be automatically released. To avoid accidentally releasing the vehicle connection without data interaction, the SOVD client can use a GET method to get vehicle connection defined in *ReqID 1074058* item 1 with a cycle of 50 seconds as a heartbeat.

SOVD client shall use a DELETE method defined in *ReqID 1074058* item 3 to reset the configuration to release vehicle connection.

1044054 v1 **Convert SOVD to/from UDS**

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

As a module for converting SOVD protocol to UDS protocol, CDA should have a mapping list of SOVD entities and UDS ECU. This list shall have been derived from the conversion and association between the SOVD OPENAPI database and the UDS ECU diagnostic database.

1043515 v1 Restart an UDS ECU

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

To map the subfunction of UDS ECUReset ($11) service, the parameters resetType and suppressResponse shall be used.

Attribute	Parent	Type	Support	CVT	Description
parameters	-	Value	M	M	Parameters for executing the restart.
resetType	Value	string	M	M	The subfunction paremeter of UDS ECUReset ($11) service.
suppressResponse	Value	boolean	M	M	Indicates if a positive response message shall be supported by the ECU. False, do not suppress a positive response message. True, suppress response message. The default is false.

1047398 v2 Specific responce Status Code 422 Supported

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

When converting to UDS diagnostic request and receiving negative response code (except NRC 0x78), the CDA module shall return with response status code 422. It is applicable to negative responses returned through physical addressing requests.

Response Status Code: 422

Attribute	Parent	Type	CVT	Description
error_code	-	sting	M	SOVD standardized error code. If a UDS negative response is received, value "error-response" shall be used in this scenario.
message	-	string	M	Message describing the problem in more detail. If a UDS negative response is received, value "UDS negative response received from ECU." shall be used in this scenario.
parameters	-	Parameters	M	Key-value-list which provides additional information on the

				error.
service	Parameters	string	C	Describe the UDS service ID that returns a negative response code
nrc	Parameters	string	C	Describe the NRC that returns a negative response code
descripiton	Parameters	string	C	Detailed description of NRC
C: Only applicable for ECU returns the negative response.				

Note: In the scenario where a SOVD API can send multiple UDS requests, the 422 response code is only allowed to be returned when all are abnormal. Otherwise, the response body must still be replied with the 200 response code defined in the API interface, where errors are indicated. For example, in the following scenario:

- Simultaneously read a snapshot of a DTC and the extended data interface;
- Read the interface for data lists. (200 response code)

1043510 v2 Specific responce Status Code 502 Supported

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

When converting to a UDS diagnostic request and receiving a response, but unable to correctly parse the response data through the built-in diagnostic database, the CDA module shall return with response status code 502. It is applicable for UDS data returned through physical addressing requests that cannot be parsed.

Response Status Code: 502

Attribute	Parent	Type	CVT	Description
error_code	-	sting	M	SOVD standardized error code. If the received UDS data cannot be parsed, value "invalid-response-content" shall be used in this scenario.
message	-	string	M	Message describing the problem in more detail. IfIf the received UDS data cannot be parsed, value "Unanalyzable or invalid response received from ECU." shall be used in this scenario.
parameters	-	Parameters	M	Key-value-list which provides additional information on the error.
UDSRawData	Parameters	string	C	The most primitive UDS response data received.

1043505 v1 Specific responce Status Code 504 Supported

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

When converted to a UDS diagnostic request and no final response is received within the defined time, the CDA module shall return with response status code 504. It is applicable to no responses returned through physical addressing requests, as well as no response returned by any ECU through functional addressing.

Response Status Code: 504

Attribute	Parent	Type	CVT	Description

-	-	GenericData	M	A more detailed description of the error .

1047424 v1 **The handling of DoIP errors**

Version history:

Purpose:
Source:
Verification Method: Analysis AND Test
Legacy ID:

When CDA encountered some errors while processing DoIP protocol, the CDA module shall return a response status code with value 500.

1043531 v1 **The handling of multiple requests for same entity**

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

When CDA processes multiple requests for the same SOVD entity (without limiting which SOVD client), if there is an ongoing diagnostic request, CDA needs to wait until response is received or time (P6client or P6*client) out before next diagnostic is sent to the same entity.

Note: At this point, the client may consider the HTTP timeout, but this is necessary to ensure the robustness of UDS access.

1043529 v1 **The handling of parallel requests**

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

CDA shall allocate independent channels for each SOVD entity. When multiple different SOVD entities are requested in parallel, CDA shall receive and process them simultaneously, allocate different independent channels for different target entities (UDS ECU), and send them one by one in DoIP protocol.
In theory, the principle of receiving first, processing first, and sending first shall be followed.

1043509 v1 **The lock mechanism after UDS service sent**

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

The locking mechanism means that only one client is allowed to access UDS requests involving ECU operation

control at the same time.
This is a very complex mechanism, which means that the UDS diagnostic services need to be classified.

When executing the following UDS diagnostic service, a lock operation is required and another client access request should be denied.

UDS diagnostic service (hex)	Lock required
10	Required
11	Required
27	Required
28	Required
85	Required
22	Not required
23	Not required
2E	Required
14	Required
19	Not required
2F	Required
31	Required

1043512 v1 **The map UDS-based content to SOVD**

Version history:

Purpose:
Source:
Verification Method: Test
Legacy ID:

The following table provide an overview on how to map UDS-based content to SOVD.

UDS service (hex)	HTTP method	REST resource
10	PUT	/{entity-path}/modes/session
11	PUT	/{entity-path}/status/restart
27	PUT	/{entity-path}/modes/security
28	PUT	/{entity-path}/modes/commctrl
85	PUT	/{entity-path}/modes/dtcsetting
22	GET	/{entity-path}/data/ /{entity-path}/configurations/
2E	PUT	/{entity-path}/data/ /{entity-path}/configurations/
14 FF FF FF	DELETE	/{entity-path}/faults
14 specific DTC	DELETE	/{entity-path}/faults/{fault-code}
19 02	GET	/{entity-path}/faults?status[mask]=xx /{entity-path}/faults?status[pendingDTC]=1%status[confirmedDTC]=1 ……
19 04	GET	/{entity-path}/faults/{fault-code}?x-sovd2uds-faultdatetype=snapshot
19 06	GET	/{entity-path}/faults/{fault-code}?x-sovd2uds-faultdatetype=extendeddata
19 0A	GET	/{entity-path}/faults
2F DID 03 [1]	POST	/{entity-path}/operations/{operation-id}/executions

2F DID 03 [2]	PUT	/{entity-path}/operations/{operation-id}/executions/{execution-id} Note: "capability": "execute"
2F DID 00 [3]	DELETE	/{entity-path}/operations/{operation-id}/executions/{execution-id}
2F DID 01	PUT	/{entity-path}/operations/{operation-id}/executions/{execution-id} Note: "capability": "reset"
2F DID 02	PUT	/{entity-path}/operations/{operation-id}/executions/{execution-id} Note: "capability": "freeze"
22 IO-DID [4]	GET	/{entity-path}/operations/{operation-id}/executions/{execution-id}
31 01 RID [5]	POST	/{entity-path}/operations/{operation-id}/executions
31 01 RID	PUT	/{entity-path}/operations/{operation-id}/executions/{execution-id} Note: "capability": "execute"
31 02 RID	PUT	/{entity-path}/operations/{operation-id}/executions/{execution-id} Note: "capability": "stop"
31 02 RID [6]	DELETE	/{entity-path}/operations/{operation-id}/executions/{execution-id}
31 03 RID	GET	/{entity-path}/operations/{operation-id}/executions/{execution-id}

1. For IO control access, it is necessary to first create temporary operation resources through the POST method, and the operations are always asynchronous.
2. After creating temporary operation resources, the parameters that need to be controlled shall be adjusted through the PUT method.
3. For returning ECU control in IO access, directly use DELETE to delete resources without using the stop capability carried in the PUT method.
4. Due to the asynchronous operation of IO control, the SOVD client shall use the GET method (matching ReadDataIdentifier service ($22) in UDS) to obtain IO control results.
5. For routine control access, it is necessary to first create temporary operation resources through the POST method, and the operation can be synchronous or asynchronous.
6. If the DELETE method is used to stop, it means that the operation resource is completely deleted, and the PUT and GET executed by the routine will no longer be valid.

1076475 v1 The number of connection from Gateway

Version history:

Purpose: Multiple clients will access multiple UDS ECU node of the vehicle simultaneously, so it is necessary to ensure sufficient connection resources.
Source:
Verification Method: Analysis AND Test
Legacy ID:

The Classic Diagnostic Adatper shall support at least 100 TCP connections from SOVD Gateway simultaneously.

1076484 v1 The usage of diagnostic database

Version history:

Purpose: The CDA module will uses diagnostic database to support the translation of SOVD request/response and UDS request/response.
Source:
Verification Method: Analysis AND Test
Legacy ID:

General diagnostic database shall be pre-set in the Adapter module, and when there is no ECU-specific diagnostic database, general diagnostic database shall be used for translation.

When there are ECU-specific diagnostic databases, the Adapter module shall first confirm whether there is a corresponding match in the ECU-specific diagnostic database upon receiving the SOVD request. If there is, it shall be used for parsing. Otherwise, continue to use general diagnostic database. If a corresponding match cannot be found in the general diagnostic database, a HTTP 404 response code will be returned.

Generic Error example:

```
{
  "error_code": "sovd-server-failure",
  "message": "The component/ECU is not defined."
}
```

Note:
1. The update method of ECU-specific diagnostic database is defined in LC: SOVDtoUDSAdapter.

1074703 v1 **Vehicle connection management (Connect to UDS)**

Version history:

Purpose: Define the strategy of client management, and remember which SOVD client created the vehicle connection to prevent the active disconnection action from being disconnected by other SOVD clients.

Source:

Verification Method: Test

Legacy ID:

The CDA module shall identify and manage accessed clients through their Client Id from SOVD Gateway.

CDA shall store and bind the Client ID for each vehicle connection. When receiving the API method to release the vehicle connection, it shall be determined whether the client identifier is consistent with the client identifier that created the connection. If they are consistent, deletion and release are allowed, otherwise an error response code is returned.

Note: If a client creates a PUT method to connect vehicle, the connected channel can be used by other clients.

4.13 Appendix for GEEA3.0&EEA4.0

4.13.1 Gateway

4.13.1.1 Zone Network Gateway - CAN - CAN without TP

•Gateway allocation of communication channels

4.13.1.1.1 Gateway allocation of communication channels

4.13.1.1.1.1 Backbone network

705195 v1 **Receive(Rx) CCs on Backbone Network**

Version history:
(1):

Purpose: Ensure that no diagnostic requests are lost for routing of PDUs between Communication Interface modules with same period or rate.

Source:

Verification Method: Test

Legacy ID:

For each network behind the GW: The Zone Network GW shall be able to open at least three(3) receive (Rx) communication channels (CCs) for receiving diagnostic request from the Backbone Network (CANFD) in addition to one(1) Rx CC for request to the GW and one(1) Rx CC for functional request (testerPresent).

Note: the receive communication channels indicates different target addresses.

The ECU supplier shall confirm the exact number of communication channels for the specific project, e.g. for a GW with many ECUs behind the GW this number may need to be increased due to performance requirements.

Example:
One network behind the gateway: 3 + 1 + 1(testerPresent) = 5 Rx Communication Channels on the Backbone.
Two networks behind the gateway: 2 x 3 + 1 + 1(testerPresent) = 8 Rx Communication Channels on the Backbone.

705196 v1 **Transmit(Tx) CCs on Backbone Network**

Version history:
(1):

Purpose: It should be possible to send responses so that timing requirement for reading information from ECUs can be fulfilled, e.g. DRO requirement.
Source:
Verification Method: Test
Legacy ID:

For each network behind the GW: The Zone Network GW shall be able to open three(3) and not more than three(3) transmit (Tx) communication channels (CCs) for sending diagnostic responses on the Backbone Network (CANFD) to the Vehicle GW (Edge Node) in addition to one(1) Tx CC for response from the GW.

The ECU supplier shall confirm the exact number of communication channels for the specific project, e.g. for a GW with many ECUs behind the GW this number may need to be increased due to performance requirements.

Rationale:
If more responses shall be sent than available Tx CCs, responses need to be queued until a Tx CC is released and available for next response etc.

Note:
There is a limitation in how many parallel responses the Vehicle GW can receive, hence limitation of parallel Tx CCs is needed in GW for responses on a functional request.

Example:
One network behind the gateway: 1 + 3 = 4 Tx Communication Channels on the Backbone.
Two networks behind the gateway: 1+ 2 x 3 = 7 Tx Communication Channels on the Backbone.

4.13.1.1.1.2 Zone Network

704495 v1 **Receive(Rx) CCs on Zone network**

Version history:
(1):

Purpose: Ensure that no diagnostic response are lost for routing of PDUs between Communication Interface modules with same period or rate.
Source:
Verification Method: Test
Legacy ID:

The Zone Network GW shall be able to open as many receive (Rx) communication channels (CCs) for receiving diagnostic responses as the total number of transmit (Tx) communication channels (CCs) that can be open by all ECUs on the Zone network.
The ECU supplier shall confirm with OEM the exact number of communication channels for the specific project.

Comment:
The number of Rx CCs are project specific.
The number of Rx CCs will be the number of ECUs on the Zone network, since the requirement on Tx communication channels on a ECU is one (1).

705197 v1 **Transmit(Tx) CCs on Zone network**

Version history:
(1):

Purpose: To match the Rx communication channel requirement on the Backbone network (CANFD), where three(3) different addressed request could be received, Zone Network GW should be able to forward the requests as soon as possible.

Source:

Verification Method: Test

Legacy ID:

For each network behind the GW: The Zone Network GW shall be able to open three (3) transmit (Tx) communication channels (CCs) for sending diagnostic physical requests to the ECUs on the Zone network.
For each network behind the GW: The Zone Network GW shall be able to open one additional transmit (Tx) communication channel (CC) for functional requests.

The ECU supplier shall confirm with OEM the exact number of communication channels for the specific project.

•Gateway latency requirements

4.13.1.1.2 Gateway latency requirements

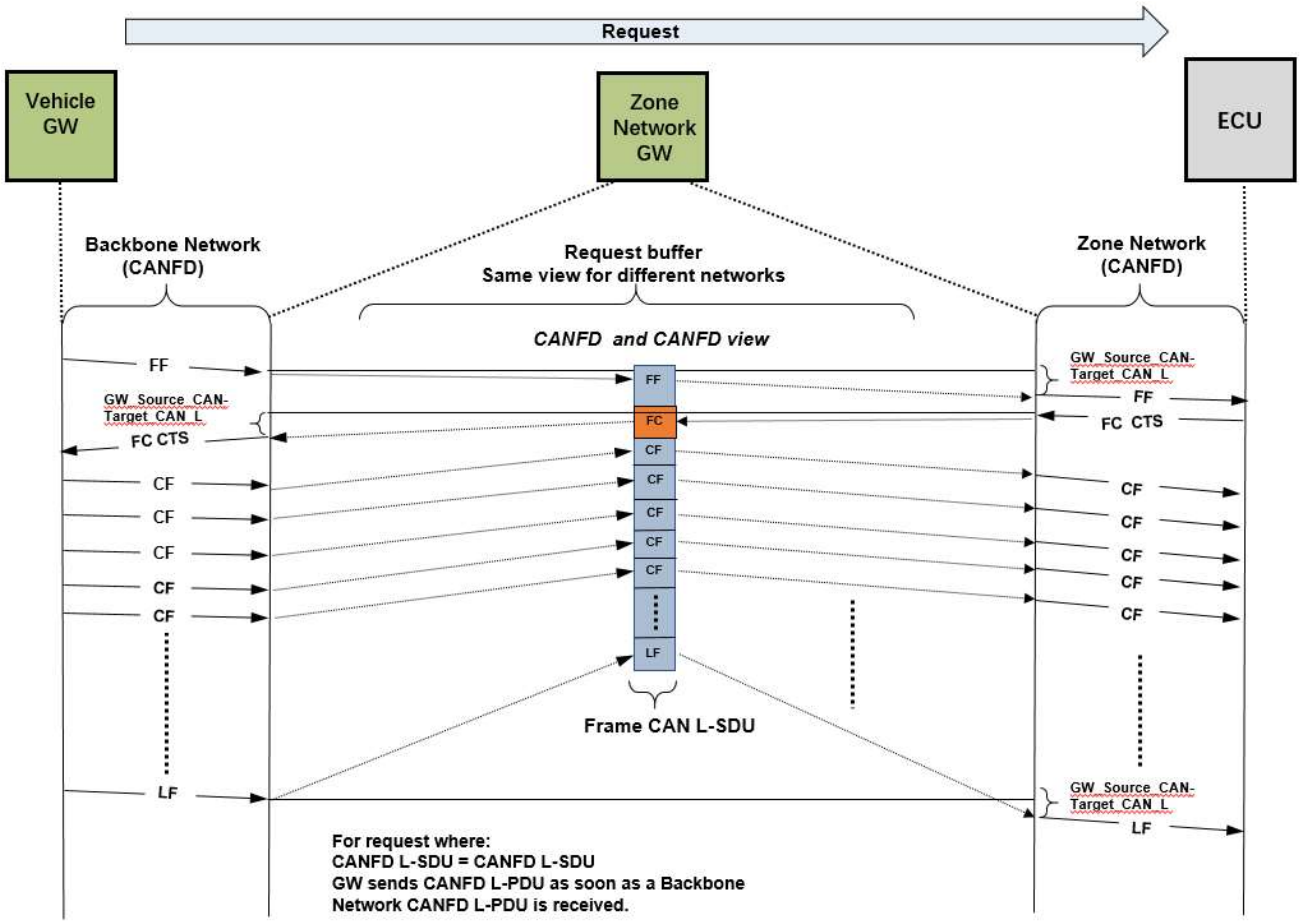
4.13.1.1.2.1 Diagnostic requests/responses CAN to CAN

This section will describe and give grounds for the following requirements:

- REQPROD 704491 Latency Latency CAN to CAN, GW_Source_CAN-Target_CAN_L

Since the Backbone Network CAN L-SDU have the same size as Zone Network CAN L-SDU, the Zone Network CAN L-SDU should be sent out as soon as the Backbone Network CAN L-SDU is received, and vice versa. See example in Figure: "Zone Network GW: CAN to CAN".

For request:



For resposne:

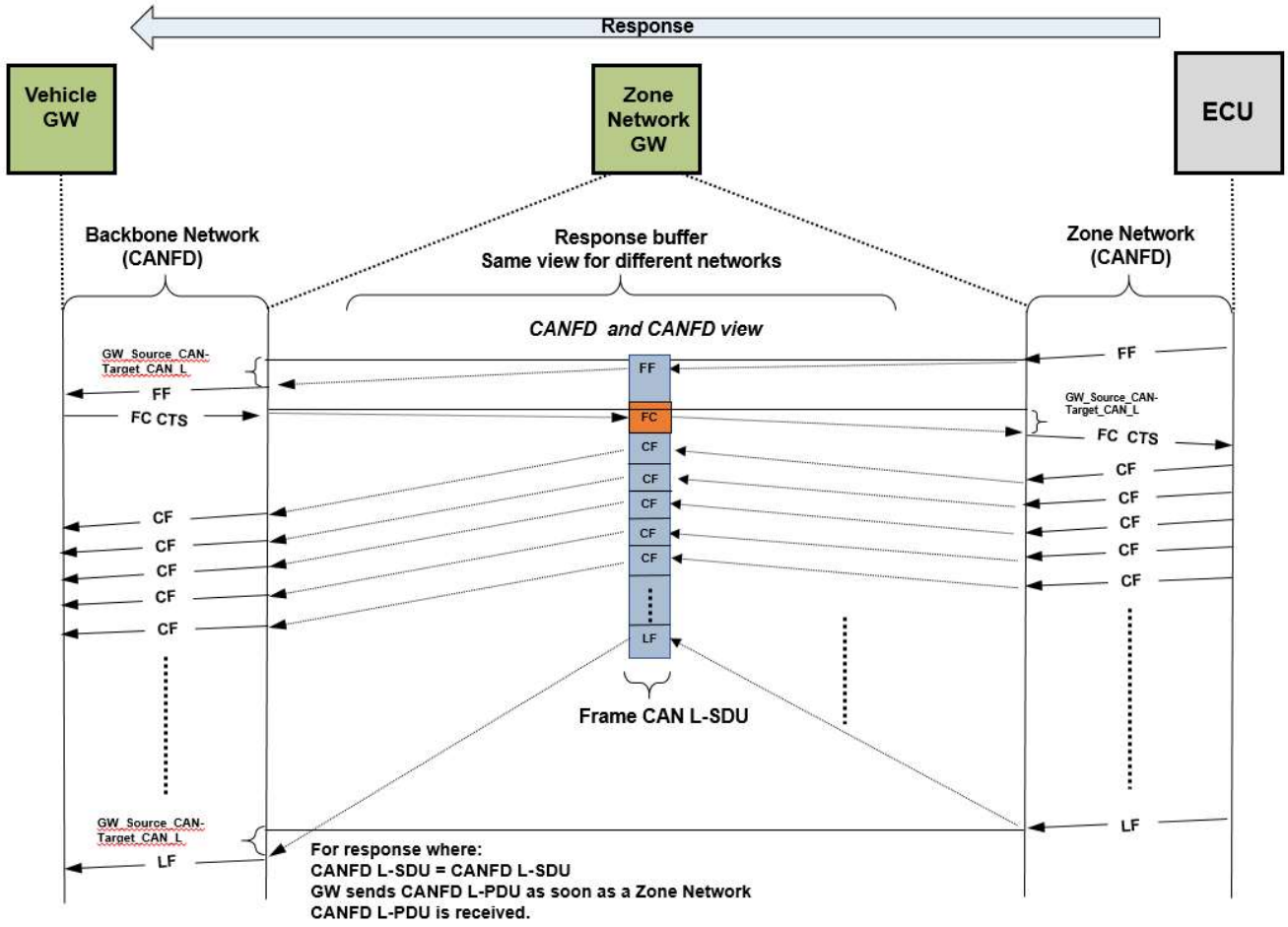


Figure: Zone Network GW: CAN to CAN

Definition of latency timing parameters specified in Table Zone Network GW: CAN to CAN:

GW_Source_CAN-Target_CAN_L: Time between receiving the every frame in request/response from CAN(FD) network until every frame is sent out on same type CAN(FD) network

Timing Parameter	SWDL	DIAG
	Max Value	Max Value
GW_Source_CAN-Target_CAN_L	2 ms	2 ms

Table: Zone Network GW: CAN to CAN request/response timing parameter

704491 v1 **Latency CAN to CAN, GW_Source_CAN-Target_CAN_L**

Version history:
(1):

Purpose: Clarify and define a maximum latency time in order to meet the performance requirements when the receiving performance of multiple frames by the peer node is the highest (e.g. STmin=0) during SWDL while data-buffering may be necessary.

Source:

Verification Method: Test

Legacy ID:

If a communication channel can be allocated, time between receiving the every frame in request/response from

CAN(FD) Source network until every frame is sent out on same type CAN(FD) Target network shall be max 2ms, and no messages shall be dropped.

Note: If the destination network is under high busload, e.g. at least 80% busload, the actual latency may be higher than the requirement, this requires approval from the GRI Basetech Team, but no message shall be dropped.

Rationale:
1) If the Backbone network is CAN network there is immediately for sending a CAN frame on the Zone network. The same goes for the opposite direction. The L-SDU on the Zone network need to be limited to the same size on the Backbone network.
2) If the Backbone network is CANFD network there is immediately for sending a CANFD frame on the Zone network. The same goes for the opposite direction. The L-SDU on the Backbone network need to be the same size on the Zone network.
3) If the Backbone network is CANFD network there is not allowed to directly send a CAN frame on the Zone network due to not suitable L-SDU. This type can only use CAN Transport Protocol per routing path.

5 DETAILED CHANGE INFO

660805 v5 **ECU Addresses**

(4) => (5):
Description modified.

Current:ECU Addresses(5)

CAN
ECUs with CAN interface
All CAN ECUs shall have a CAN ID according to ECU SWRS Requirements "ECU Addresses in GEEA3.0 - Superset". The CAN ID is a static address calculated from the ECU address and is used to address CAN ECUs for SWDL and Diagnostics. The CAN ID is included in the ECU extract.

LIN
LIN slave ECUs, using SWDL functionality (bootloader)
All LIN ECUs using a bootloader shall use a complete ECU address according to ECU SWRS Requirements "ECU Addresses in GEEA3.0 - Superset". The ECU address is a static address that is used to address LIN ECUs for SWDL and Diagnostics. The ECU address is included in the LDF file.

LIN slave ECUs without SWDL, but using LIN diagnostic frames
A LIN slave ECU (not using a bootloader) shall use a NAD address according to ECU SWRS Requirements "ECU Addresses in GEEA3.0 - Superset". Otherwise the LIN master will not be able to address diagnostic requests to the LIN slave. The NAD address is the last byte from the 2 byte ECU address and is used for diagnostics. The NAD address is included in the LDF file.

LIN slave ECUs without SWDL and without LIN diagnostic frames
If there is no need for a LIN master to address diagnostic requests to a LIN slave ECU, then a NAD address is NOT needed. In this case the LIN slave will be assigned a node address default value. The default value will be 0x7D. In case of multiple LIN Slaves on the same network the addresses will be set in decreasing order starting from 0x7D. The LIN address to be used by the LIN slave is included in the LDF file.

LIN masters
LIN masters shall use the NAD according to ECU SWRS "ECU Addresses in GEEA3.0 - Superset" in order to address diagnostic requests to LIN slaves.

Other Bus
Slave ECUs, i.e. with GMSL interface
All Slave ECUs shall use a complete ECU address according to ECU SWRS Requirements "ECU Addresses in GEEA3.0 - Superset". The ECU address is a static address that is used to address ECUs for SWDL and Diagnostics.

IP

ECUs with IP-based interfaces

The internal MAC addresses and internal IP addresses of IP-ECUs shall be assigned as according to ECU SWRS Requirements "ECU Addresses in GEEA3.0 - Superset". The IP address must be implemented into the IP ECU by supplier. The following IP address space has been chosen: 198.18.0.0/16 for the vehicle network. From this address space, the vehicle IP network is using one subnet according to below:

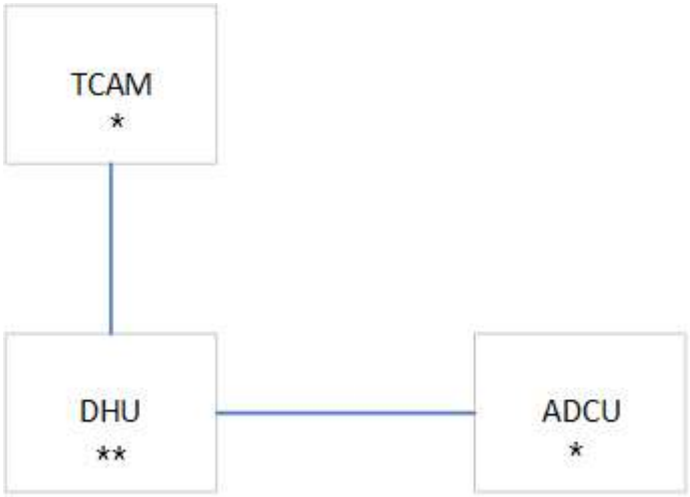


Figure:IP subnets of the GEEA 3.0 network

NOTE 1: In *Figure IP subnets of the vehicle network* and in the *table "ECU Addresses"*, some IP addresses are marked with a * or a **.

* == Internal ECU-IP address used for Diagnostic (ISO13400) TCP clients (Diagnostic Server) to connect to the Edge Node (DHU).

** == Internal ECU-IP address used by Edge Node (DHU) acting as TCP Server (Diagnostic Client) offering Diagnostic Service

Overview of IP-address space :

The vehicle IP network shall use one subnet according to table below.

Network Friendly Name:	**VEHICLE IP NETWORK**
Subnet ID:	198.18.0.0 / 16
Usable IP addresses:	198.18.0.1 – 198.18.255.254
Netmask:	255.255.255.0
~~**Broadcast address:**~~	~~198.18.255.255~~
Default GW address:	DHU – Ethernet interface according to, ***Table:*** *ECU addresses – Showing IP-based ECU's*

IP address space allocation depending on physical network

The IP address range of 198.18.0.0 / 16 has been divided into different address ranges to be able to identify which physical communication carrier is used for a specific ECU. The tables below are used in conjunction with *Req. ID* 660814.

Example: ECU's which are connected to Ethernet will have an IP address range of: 198.18.32.1-198.18.47.255.

Network Friendly Name:	**Ethernet**

Network ID:	198.18. **32**.0
IP addresses:	198.18.32.1 – 198.18.47.255

Relationship between ECU addresses and IP addresses
The ECU address consist of 16 bits and are divided into 4 nibbles as follows: [Address range].[Network id].[Zone id].[Ecu id]

NOTE: The addressing format for diagnostic request and response, source address (SA) and target address (TA), using Zone, Network ID and ECU ID is defined in [*UDS Services*].

The IP address v4 consist of 32 bits and are divided into 4 bytes. It is required for diagnostic purposes, that it shall be possible to calculate the ECU address from the IP address of a specific ECU. The IP addresses have been divided as follows:

198.18.{ssssnnnn+VLAN ID}.{zzzzeeee}.

ssss = Consists of 4 bits, defines the start of the Network ID.
nnnn = Consists of 4 bits, defines the Network ID
zzzz = Consists of 4 bits, defines the Zone ID
eeee = Consists of 4 bits, defines the ECU ID

The ECU addresses are always represented in hexadecimal form and the IP addresses are written in decimal form. This means when calculating the ECU address from the IP address there is a need for conversion from hexadecimal to decimal.

Example:
ECU Address 0x1101, would mean IP address 198.18.44.17.

198.18.[S] [N].[Z][E]

[S] = 0x2 (Network Friendly Name = "**Ethernet**", Network ID: 198.18. **32**.0)
[N] = 0x0
[Z] = 0x1
[E] = 0x1

Table: IP Address calculation

IP address calculation				
	198	**18**	**{ssssnnnn+VLANID}**	**{zzzzeeee}**
Binary	1100 0110	0001 0010	0010 0000+0000 1100	0001 0001
Hex	0xC6	0x12	0x20+0xC	0x11
Decimal	198	18	44	17

Previous: ECU Addresses(4)

CAN
ECUs with CAN interface
All CAN ECUs shall have a CAN ID according to ECU SWRS Requirements "ECU Addresses in GEEA3.0 - Superset". The CAN ID is a static address calculated from the ECU address and is used to address CAN ECUs for SWDL and Diagnostics. The CAN ID is included in the ECU extract.

LIN
LIN slave ECUs, using SWDL functionality (bootloader)
All LIN ECUs using a bootloader shall use a complete ECU address according to ECU SWRS Requirements "ECU Addresses in GEEA3.0 - Superset". The ECU address is a static address that is used to address LIN ECUs for SWDL and Diagnostics. The ECU address is included in the LDF file.

LIN slave ECUs without SWDL, but using LIN diagnostic frames

A LIN slave ECU (not using a bootloader) shall use a NAD address according to ECU SWRS Requirements "ECU Addresses in GEEA3.0 - Superset". Otherwise the LIN master will not be able to address diagnostic requests to the LIN slave. The NAD address is the last byte from the 2 byte ECU address and is used for diagnostics. The NAD address is included in the LDF file.

LIN slave ECUs without SWDL and without LIN diagnostic frames
If there is no need for a LIN master to address diagnostic requests to a LIN slave ECU, then a NAD address is NOT needed. In this case the LIN slave will be assigned a node address default value. The default value will be 0x7D. In case of multiple LIN Slaves on the same network the addresses will be set in decreasing order starting from 0x7D. The LIN address to be used by the LIN slave is included in the LDF file.

LIN masters
LIN masters shall use the NAD according to ECU SWRS "ECU Addresses in GEEA3.0 - Superset" in order to address diagnostic requests to LIN slaves.

Other Bus
Slave ECUs, i.e. with GMSL interface
All Slave ECUs shall use a complete ECU address according to ECU SWRS Requirements "ECU Addresses in GEEA3.0 - Superset". The ECU address is a static address that is used to address ECUs for SWDL and Diagnostics.

IP
ECUs with IP-based interfaces
The internal MAC addresses and internal IP addresses of IP-ECUs shall be assigned as according to ECU SWRS Requirements "ECU Addresses in GEEA3.0 - Superset". The IP address must be implemented into the IP ECU by supplier. The following IP address space has been chosen: 198.18.0.0/16 for the vehicle network. From this address space, the vehicle IP network is using one subnet according to below:

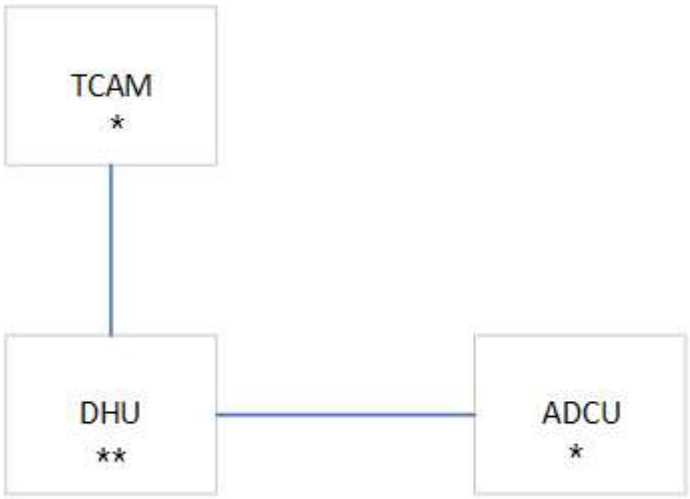


Figure:IP subnets of the GEEA 3.0 network

NOTE 1: In *Figure IP subnets of the vehicle network* and in the *table "ECU Addresses"*, some IP addresses are marked with a * or a **.

* == Internal ECU-IP address used for Diagnostic (ISO13400) TCP clients (Diagnostic Server) to connect to the Edge Node (DHU).

** == Internal ECU-IP address used by Edge Node (DHU) acting as TCP Server (Diagnostic Client) offering Diagnostic Service

Overview of IP-address space :

The vehicle IP network shall use one subnet according to table below.

Network Friendly Name:	**VEHICLE IP NETWORK**
Subnet ID:	198.18.0.0 / 16
Usable IP addresses:	198.18.0.1 – 198.18.255.254
Netmask:	255.255.0.0
Broadcast address:	198.18.255.255
Default GW address:	DHU – Ethernet interface according to, ***Table:*** *ECU addresses – Showing IP-based ECU's*

IP address space allocation depending on physical network

The IP address range of 198.18.0.0 / 16 has been divided into different address ranges to be able to identify which physical communication carrier is used for a specific ECU. The tables below are used in conjunction with *Req. ID* 660814.

Example: ECU's which are connected to Ethernet will have an IP address range of: 198.18.32.1-198.18.47.255.

Network Friendly Name:	**Ethernet**
Network ID:	198.18. **32**.0
IP addresses:	198.18.32.1 – 198.18.47.255

Relationship between ECU addresses and IP addresses
The ECU address consist of 16 bits and are divided into 4 nibbles as follows: [Address range].[Network id].[Zone id].[Ecu id]

NOTE: The addressing format for diagnostic request and response, source address (SA) and target address (TA), using Zone, Network ID and ECU ID is defined in [*UDS Services*].

The IP address v4 consist of 32 bits and are divided into 4 bytes. It is required for diagnostic purposes, that it shall be possible to calculate the ECU address from the IP address of a specific ECU. The IP addresses have been divided as follows:

198.18.{ssssnnnn+VLAN ID}.{zzzzeeee}.

ssss = Consists of 4 bits, defines the start of the Network ID.
nnnn = Consists of 4 bits, defines the Network ID
zzzz = Consists of 4 bits, defines the Zone ID
eeee = Consists of 4 bits, defines the ECU ID

The ECU addresses are always represented in hexadecimal form and the IP addresses are written in decimal form. This means when calculating the ECU address from the IP address there is a need for conversion from hexadecimal to decimal.

Example:
ECU Address 0x1101, would mean IP address 198.18.44.17.

198.18.[S] [N].[Z][E]

[S] = 0x2 (Network Friendly Name = "**Ethernet**", Network ID: 198.18. **32**.0)
[N] = 0x0
[Z] = 0x1
[E] = 0x1

Table: IP Address calculation

IP address calculation				
	198	**18**	**{ssssnnnn+VLANID}**	**{zzzzeeee}**

Binary	1100 0110	0001 0010	0010 0000+0000 1100	0001 0001
Hex	0xC6	0x12	0x20+0xC	0x11
Decimal	198	18	44	17

660807 v3 **Missing frame master - CAN**

(2) => (3):
Description modified.

Current:Missing frame master - CAN(3)

The ECU routing diagnostic requests to and from a CAN network shall be missing frame master for that network. The missing frame master ECU's are DHU/IHU,ZCUD,ZCUP,ADCU/ASDM,XCU/XCM/XCUP/PCMU/ZCUT, and the monitoring slave ECU will be defined in the subsystem design from Basetech.

Previous: Missing frame master - CAN(2)

The ECU routing diagnostic requests to and from a CAN network shall be missing frame master for that network. The missing frame master ECU's are DHU,ZCUD,ZCUP,ADCU/ASDM, and the monitoring slave ECU will be defined in the subsystem design from Basetech.

660818 v3 **Eth Configuration parameters Update**

(2) => (3):
Attribute 'Purpose' modified: No preview available.
Description modified.

Current:Eth Configuration parameters Update(3)

IP ECUs shall support updating Ethernet configuration parameters via software download (OTA, UDS, USB flash disk and other approaches). Typical configuration examples for each layer are listed below:

1.Physical Layer: PHY master/slave attribute and related parameters.
2.Data Link Layer: VLAN configuration and related parameters.
3.Network Layer: IP address and related parameters.
4.Transport Layer: UDP port number and related parameters.
5.Application Layer: SOME/IP parameters, DoIP data update configurations, etc.

Refer to BT documents and Ethernet Subsystem Requirements Specification-[ETH REF 24] for complete and detailed configuration parameters.

Previous: Configuration Update(2)

ECU with Ethernet shall support the update of its network configuration parameters via OTA or USB flash drives, etc.

涉及以太网的ECU需支持通过OTA或U盘等方式对其网络配置参数进行更新。

660861 v3 **CAN FD bit register settings for 2 Mbps**

(2) => (3):
Description modified.

Current:CAN FD bit register settings for 2 Mbps(3)

CANFD sample point setting

Parameter	Min	Nominal	Max
Sample point(Arbitration part)	-	80%	-

Sample point(Data part)	-	70%	-
Sample frequency	-	1	-

CANFD sample point setting table

Bit per second	Clock	Nq	Tq	Prop_Seg	Phase_Seg1	Phase_Seg2	SJW	SP	SSP offset	TDC enable
Arbitraion part(500kbps)	40MHz	80	25ns	47	16	16	16	80%	/	/
Data part(2Mbps)	40MHz	20	25ns	7	6	6	6	70%	65%	Yes(Auto TDC)
Arbitraion part(500kbps)	80MHz	160	12.5ns	95	32	32	32	80%	/	/
Data part(2Mbps)	80MHz	40	12.5ns	15	12	12	12	70%	67.5%	Yes(Auto TDC)

Note 1:Other settings may only be used after agreement with the Geely Base Technology responsible.以上图示仅为示例，如果有不同的配置，需要和Geely Basetech团队确认后使用。

Note 2:The unit of SSP offset is the minimum time quantum (mTq) before BRP((Baud Rate Prescaler)) frequency division. It is recommended to configure it one mTq ahead of SP, and BRP should be configured as 1 or 2. Meanwhile, it is recommended that the same Tq length is used in the nominal bit time and the in the data bit time。SSP offset的单位是未经BRP(Baud Rate Prescaler)分频前的最小时间量子（mTq），推荐配置为比SP提前一个mTq，且BRP应配置为1或2。同时，建议数据场和仲裁场的Tq length相同。

Previous: CAN FD bit register settings for 2 Mbps(2)

CANFD sample point setting

Parameter	Min	Nominal	Max
Sample point(Arbitration part)	-	80%	-
Sample point(Data part)	-	70%	-
Sample frequency	-	1	-

CANFD sample point setting table

Bit per second	Clock	Nq	Tq	Prop_Seg	Phase_Seg1	Phase_Seg2	SJW	SP	SSP offset	TDC enable
Arbitraion part(500kbps)	40MHz	80	25ns	47	16	16	16	80%	/	/
Data part(2Mbps)	40MHz	20	25ns	7	6	6	6	70%	65%	Yes(Auto TDC)
Arbitraion part(500kbps)	80MHz	160	12.5ns	95	32	32	32	80%	/	/
Data part(2Mbps)	80MHz	40	12.5ns	15	12	12	12	70%	67.5%	Yes(Auto TDC)

Other settings may only be used after agreement with the OEM project Base Technology responsible.

660868 v2 **Register read out using Diagnostics**

(1) => (2):
Description modified.

Current:Register read out using Diagnostics(2)

The ECU shall be possible to read some registers of the PHY and Switch through UDS diagnostic and local log function. The contents of the registers read by the UDS diagnostic must follow *[ETH REF 41]*, and the contents of the registers read by the local log function must follow *[ETH REF 42]*.

Previous: Register read out using Diagnostics(1)

It shall be possible to read all registers using diagnostics according to [ETH REF 22]. It shall be possible to read one register at a time.

660869 v2 **Register write using Diagnostics**

(1) => (2):
Description modified.

Current:Register write using Diagnostics(2)

The ECU shall be possible to write some registers of the PHY and Switch through UDS diagnostic. The contents of the registers write by the UDS diagnostic must follow *[ETH REF 41]*.

Previous: Register write using Diagnostics(1)

All registers that can be modified in a chip set, shall be possible to set using diagnostics according to *[ETH REF 22]*.

660907 v4 **Configuration Update**

(3) => (4):
Attribute 'Source' modified: No preview available.
Description modified.

Current:Configuration Update(4)

In the requirements within this document, there are parameters required to be "configurable". Configurability of these parameters shall be implemented according to *[REQID 660818]*.

Previous: Configuration Update(3)

In the requirements within this document, there are parameters required to be "configurable". Configurability of these parameters shall be implemented according to *[REQPROD 436145]*.

660953 v3 **Separation time (STmin) non-programming session client side**

(2) => (3):
Description modified.

Current:Separation time (STmin) non-programming session client side(3)

When in non-programming session the ECU shall be able to receive CF N_PDU with minimum 1ms separation time (i.e. STmin set to 1ms).

Previous: Separation time (STmin) non-programming session client side(2)

When in non-programming session the ECU shall be able to receive CF N_PDU with minimum 1ms separation time (i.e. STmin set to 1ms).

Note: It is allowed to support shorter separation time than this requirement specifies.

661023 v2 Local Configurations

(1) => (2):
Attribute 'Source' modified: No preview available.
Description modified.

Current:Local Configurations(2)

In the requirements within this LC, there are parameters required to be "configurable". Configurability of these parameters shall be implemented according to *[REQID 660818]*.

Previous: Local Configurations(1)

In the requirements within this LC, there are parameters required to be "configurable". Configurability of these parameters shall be implemented according to [REQPROD 436145].

661024 v3 ICMP support

(2) => (3):
Description modified.

Current:ICMP support(3)

ICMP shall be implemented as specified by *[RFC 792]* and *[RFC 1122]*. The implementation shall include support for ICMP messages of the type stated in the table below.

Message Type	Description
Echo	Used for testing and troubleshooting
Destination Unreachable	Destination host is not reachable
Time Exceeded	The TTL field has reached zero

For ECUs that don't use three-layer routers, it is not a must to support ICMP Message where Message Type is Time Exceed.

Previous: ICMP support(2)

ICMP shall be implemented as specified by *[RFC 792]* and *[RFC 1122]*. The implementation shall include support for ICMP messages of the type stated in the table below.

Message Type	Description
Echo	Used for testing and troubleshooting
Destination Unreachable	Destination host is not reachable
Time Exceeded	The TTL field has reached zero

661037 v2 Graceful TCP Connection Shutdown on ECU Reset

(1) => (2):
Description modified.

Current:Graceful TCP Connection Shutdown on ECU Reset(2)

During a reset procedure, an IP-ECU shall close all open sockets and ensure that TCP's connection termination procedure runs to completion before doing the actual reset.

To shorten TCP re-connection time, it is OK to use RST message to quickly close TCP connection.

Previous: Graceful TCP Connection Shutdown on ECU Reset(1)

During a reset procedure, an IP-ECU shall close all open sockets and ensure that TCP's connection termination procedure runs to completion before doing the actual reset.

661186 v3 **Local configurations**

(2) => (3):
Description modified.

Current:Local configurations(3)

Configurations are done according to *[REQID 660818].*

Previous: Local configurations(2)

Configurations are done according to *[REQPROD 436145].*

661197 v3 **SOME IP Header Validation**

(2) => (3):
Description modified.

Current:SOME IP Header Validation(3)

The SOME/IP Header shall be validated in the order indicated by the return codes in the following table:

Return code	Code Name	Description
0x07	SOMEIPXF_E_WRONG_PROTOCOL_VERSION	Version of SOMEIP protocol not supported
~~0x02~~	~~SOMEIPXF_E_UNKNOWN_SERVICE~~	~~Requested Service ID is unknown~~
0x08	SOMEIPXF_E_WRONG_INTERFACE_VERSION	The interface version of the requested service mismatch
~~0x03~~	~~SOMEIPXF_E_UNKNOWN_METHOD~~	~~The requested Method ID is unknown~~
0x0A	SOMEIPXF_E_WRONG_MESSAGE_TYPE	An unexpected message type was received (e.g. REQUEST_NO_RETURN for a method defined as REQUEST)
0x04	SOMEIPXF_E_NOT_READY	The application providing the method is not running
0x09	SOMEIPXF_E_MALFORMED_MESSAGE	Deserialization error so that payload cannot be deserialized
0x01	SOMEIPXF_E_NOT_OK	An unspecified error occurred

Previous: SOME IP Header Validation(2)

The SOME/IP Header shall be validated in the order indicated by the return codes in the following table:

Return code	Code Name	Description

0x07	SOMEIPXF_E_WRONG_PROTOCOL_VERSION	Version of SOMEIP protocol not supported
0x02	SOMEIPXF_E_UNKNOWN_SERVICE	Requested Service ID is unknown
0x08	SOMEIPXF_E_WRONG_INTERFACE_VERSION	The interface version of the requested service mismatch
0x03	SOMEIPXF_E_UNKNOWN_METHOD	The requested Method ID is unknown
0x0A	SOMEIPXF_E_WRONG_MESSAGE_TYPE	An unexpected message type was received (e.g. REQUEST_NO_RETURN for a method defined as REQUEST)
0x04	SOMEIPXF_E_NOT_READY	The application providing the method is not running
0x09	SOMEIPXF_E_MALFORMED_MESSAGE	Deserialization error so that payload cannot be deserialized
0x01	SOMEIPXF_E_NOT_OK	An unspecified error occurred

661222 v5 **Return Codes in the Response Messages**

(4) => (5):
Description modified.

Current:Return Codes in the Response Messages(5)

Return Codes in the Response Messages of methods shall be used to transport ~~application errors and~~ the response data of a method from the provider to the caller of a method. Response Message (Message Type=0x80) shall be used.

Application error shall be transport according to *[REQID 661224]*.

Previous: Return Codes in the Response Messages(4)

Return Codes in the Response Messages of methods shall be used to transport application errors and the response data of a method from the provider to the caller of a method. Response Message (Message Type=0x80) shall be used.

661274 v3 **Mapping between PCP and Traffic Classes**

(2) => V9(3):
Description modified.

Current:Mapping between PCP and Traffic Classes(3)

The mapping of PCP values to traffic classes and vice versa shall correspond to the recommendation in Table 34-1 of *[ETH REF 8]*:

PCP0 shall be mapped to traffic class 1.

PCP1 shall be mapped to traffic class 0.

PCP2 shall be mapped to traffic class 2.

PCP3 shall be mapped to traffic class 3.

PCP4 shall be mapped to traffic class 0.

PCP5 shall be mapped to traffic class 5.

PCP6 shall be mapped to traffic class 6.

PCP7 shall be mapped to traffic class 7.

The value of the PCP field shall be assigned to the use cases according to the assignment stated in *[ETH REF 24]*.

If another requirement that defines PCP mapping differs from this one, it shall override this requirement.

Previous: Mapping between PCP and Traffic Classes(2)

The mapping of PCP values to traffic classes and vice versa shall correspond to the recommendation in Table 34-1 of *[ETH REF 8]*:

PCP0 shall be mapped to traffic class 1.

PCP1 shall be mapped to traffic class 0.

PCP2 shall be mapped to traffic class 2.

PCP3 shall be mapped to traffic class 3.

PCP4 shall be mapped to traffic class 4.

PCP5 shall be mapped to traffic class 5.

PCP6 shall be mapped to traffic class 6.

PCP7 shall be mapped to traffic class 7.

The value of the PCP field shall be assigned to the use cases according to the assignment stated in *[ETH REF 21]*.

661293 v3 **Security Level usage**

(2) => (3):
Description modified.

Current:Security Level usage(3)

The ECU shall use the security access levels defined in *Table - Security Access Levels.* Other levels are however also allowed, but requires an analysis to identify the need of separating information or services further.

Security Access Level	Purpose	Comment
01	Used for software download	A vehicle unique security access constant shall be generated in the manufacturing process.
03	Car mode transition	A common security access constant shall be applied.
05	General for diagnostic services **(Reserved)**	A common security access constant shall be applied. If the ECU has a specific need, it shall be discussed and be aligned with the Basetech Team.
11	Immo and ITP	A vehicle unique security access constant shall be generated in the manufacturing process.
19	**Security diagnostic services**	A vehicle unique security access constant shall be generated in the manufacturing process. This is the protected area that shall be applied by the ECU.

1D	Test and development only	Uses the algorithm supported by OEM tool-chain (but not used at manufacturing or workshops)
1F	Legal requirements for ERA-GLONASS	-
31	Vehicle Speed Limitation	A common security access constant shall be applied.
33	Engine Oil Maintenance & EEPROM Removal	Used for engine-related ECU nodes. Only applicable in aftersale scene.
5F	Pyrotechnic end-of-life activation	ISO 26021 requirement

Table - Security Access Levels

Previous: Security Level usage(2)

The ECU shall use the security access levels defined in *Table - Security Access Levels.* Other levels are however also allowed, but requires an analysis to identify the need of separating information or services further.

Security Access Level	Purpose	Comment
01	Used for software download	A vehicle unique security access constant shall be generated in the manufacturing process.
03	Car mode transition	A common security access constant shall be applied.
05	General for diagnostic services	This is the default area that shall be applied by the ECU, unless there is a specific need to separate the specific information from others.
11	Immo and ITP	A vehicle unique security access constant shall be generated in the manufacturing process.
19	Security Audit Log & File transport	A vehicle unique security access constant shall be generated in the manufacturing process.
1D	Test and development only	Uses the algorithm supported by OEM tool-chain (but not used at manufacturing or workshops)
1F	Legal requirements for ERA-GLONASS	-
31	Vehicle Speed Limitation	A common security access constant shall be applied.
5F	Pyrotechnic end-of-life activation	ISO 26021 requirement

Table - Security Access Levels

661429 v2 **Internal IP format to Sub IP format in request**

(1) => (2):
Attribute 'Purpose' modified: No preview available.
Description modified.

Current:Internal IP format to Sub IP format in request(2)

If a request is received from an internal IP network that should be transported to a sub IP network, the InVehicle GW shall translate the DoIP target address (DoIP TA) to the Sub IP target address (IP TA) as specified by *[GW_14] Internetworks General Specification* and use the InVehicle GW Sub IP address as the Sub IP target address (IP SA), format on frames as defined in DoIP *[GW_26] ISO 13400-2*.

Comment:
1. See example in Figure: " *Diagnostic request from InVehicle IP to Sub IP*".
2. The allocation of subnetwork IP addresses are defined in ECU SWRS requirements "ECU Addresses in GEEA3.0&EEA4.0 - Superset".

Previous: Domain IP format to internal IP format in request(1)

If a request is received from an domain IP network that should be transported to an internal IP network, the Domain GW shall translate the DoIP target address (DoIP TA) to an internal IP target address (IP TA) as specified by *[GW_14] Internetworks General Specification* and use the Domain GW internal IP address as the internal IP target address (IP SA), format on frames as defined in DoIP *[GW_26] ISO 13400-2*.

Comment:
See example in Figure: " *Diagnostic request from Domain IP to internal IP"*.

661431 v4 Large buffers for response

(3) => (4):
Description modified.

Current:Large buffers for response(4)

For each network behind the GW: In application mode, the Zone GW shall be able to allocate up to 4 kB large buffers for response.

Note: for one specifical network behind the GW: 8 kB for 1 UDS response read out of EDR data.

Previous: Large buffers for response(3)

For each network behind the GW: In application mode, the Zone GW shall be able to allocate up to 8 kB large buffers for response.

Note: 8 kB for 1 UDS response read out of EDR data.

662756 v4 Power Consumption

(3) => (4):
Description modified.

Current:Power Consumption(4)

When an ECU executing either the primary or secondary bootloader, all of the ECU's I/O and controls shall be set into a safe and minimize power consumption where components cannot be damaged and the area is safe for people working on the vehicle.
Note: "safe" means that ECU shall ensure all controls (e.g. actuators, sensors, etc) are in a safe state before and after entering and exiting the primary or secondary bootloader.

Minimize power consumption means that the ECU only shall supply the necessary components to perform software download. The requirements for ECU(s) are as follows

Item	ECU Type	Power consumption
Type 1	General ECU	≤ 150 mA
Type 2	Zone ECU Integrated multiple ECU capabilites	≤ 200 mA
Type 3	ECU with auxiliary processors	≤ 250 mA
Type 4	ECU with POSIX system	≤ 300 mA
Type 5	ECU with POSIX system and OTA-distributed capability	≤ 2000 mA

The implementation of minimizing power consumption, for example:

- Control the LIN Slave Node to enter sleep mode.
- Output shutdown signal for controlled business.

- Disable any SWDL unrelated actuators, sensors, chips, and so on.

Note1: It shall be allowed with a deviation through clarifying the composition of power consumption to OTA Team.
Note2: The supplier shall provide a complete and detailed report explaining the current consumption of the ECU during the programming session.

Previous: Power Consumption(3)

When an ECU executing either the primary or secondary bootloader, all of the ECU's I/O and controls shall be set into a safe and minimize power consumption where components cannot be damaged and the area is safe for people working on the vehicle.
Note: "safe" means that ECU shall ensure all controls (e.g. actuators, sensors, etc) are in a safe state before and after entering and exiting the primary or secondary bootloader.

Minimize power consumption means that the ECU only shall supply the necessary components to perform software download. The power consumption of the vast majority of ECUs shall be within the range of 100mA, and the power consumption of Zone ECUs shall not exceeded 150mA. The implementation of minimizing power consumption, for example:

- Control the LIN Slave Node to enter sleep mode.
- Output shutdown signal for controlled business.
- Disable any SWDL unrelated actuators, sensors, chips, and so on.

Note: It may be allowed with a deviation through clarifying the composition of power consumption to OTA Team.

662802 v2 **Allocation of the Verification Block Table**

(1) => (2):
Description modified.

Current:Allocation of the Verification Block Table(2)

A verification block table as defined Req ID 661295: *Format of the Verification Block* Table shall be stored in a separate data block. The start address shall be defined by the logical block definition, i.e. by the ECU Software Structure. The correct location of the verification block is ensured by including its address information (start and length) when it is being signed. The length (unprocessed) must be calculated by the ECU when the data is programmed.

Design consideration: When delta encoding is applied, the length of the Verification Block Table could be derived by using the start address and Number of data blocks.

Previous: Allocation of the Verification Block Table(2)

A verification block table as defined Req ID 661295: *Format of the Verification Block* Table shall be stored in a separate data block. The start address shall be defined by the logical block definition, i.e. by the ECU Software Structure. The correct location of the verification block is ensured by including its address information (start and length) when it is being signed. The length (unprocessed) must be calculated by the ECU when the data is programmed.

Design consideration: When delta encoding is applied, the length of the Verification Block Table could be derived by using the start address and Number of data blocks.

Current:Allocation of the Verification Block Table(2)

A verification block table as defined Req ID 661295: *Format of the Verification Block* Table shall be stored in a separate data block. The start address shall be defined by the logical block definition, i.e. by the ECU Software Structure. The correct location of the verification block is ensured by including its address information (start and length) when it is being signed. The length (unprocessed) must be calculated by the ECU when the data is

programmed.

Design consideration: When delta encoding is applied, the length of the Verification Block Table could be derived by using the start address and Number of data blocks.

Previous: Allocation of the Verification Block Table(1)

A verification block table as defined Req ID1600: *Format of the Verification Block* Table shall be stored in a separate data block. The start address shall be defined by the logical block definition, i.e. by the ECU Software Structure. The correct location of the verification block is ensured by including its address information (start and length) when it is being signed. The length (unprocessed) must be calculated by the ECU when the data is programmed.

Design consideration: When delta encoding is applied, the length of the Verification Block Table could be derived by using the start address and Number of data blocks.

662802 v1 Allocation of the Verification Block Table

Current:Allocation of the Verification Block Table(1)

A verification block table as defined Req ID1600: *Format of the Verification Block* Table shall be stored in a separate data block. The start address shall be defined by the logical block definition, i.e. by the ECU Software Structure. The correct location of the verification block is ensured by including its address information (start and length) when it is being signed. The length (unprocessed) must be calculated by the ECU when the data is programmed.

Design consideration: When delta encoding is applied, the length of the Verification Block Table could be derived by using the start address and Number of data blocks.

Previous: Allocation of the Verification Block Table(2)

A verification block table as defined Req ID 661295: *Format of the Verification Block* Table shall be stored in a separate data block. The start address shall be defined by the logical block definition, i.e. by the ECU Software Structure. The correct location of the verification block is ensured by including its address information (start and length) when it is being signed. The length (unprocessed) must be calculated by the ECU when the data is programmed.

Design consideration: When delta encoding is applied, the length of the Verification Block Table could be derived by using the start address and Number of data blocks.

Current:Allocation of the Verification Block Table(1)

A verification block table as defined Req ID1600: *Format of the Verification Block* Table shall be stored in a separate data block. The start address shall be defined by the logical block definition, i.e. by the ECU Software Structure. The correct location of the verification block is ensured by including its address information (start and length) when it is being signed. The length (unprocessed) must be calculated by the ECU when the data is programmed.

Design consideration: When delta encoding is applied, the length of the Verification Block Table could be derived by using the start address and Number of data blocks.

Previous: Allocation of the Verification Block Table(1)

A verification block table as defined Req ID1600: *Format of the Verification Block* Table shall be stored in a

separate data block. The start address shall be defined by the logical block definition, i.e. by the ECU Software Structure. The correct location of the verification block is ensured by including its address information (start and length) when it is being signed. The length (unprocessed) must be calculated by the ECU when the data is programmed.

Design consideration: When delta encoding is applied, the length of the Verification Block Table could be derived by using the start address and Number of data blocks.

663093 v5 **Autosar configuration - required configuration values**

(4) => (5):
Description modified.

Current:Autosar configuration - required configuration values(5)

These are mandatory values for non-programming session. Using an Autosar version 4.0.3 or higher means that the communication stack must support Partial Network Cluster (PNC).

Parameter	Value
CanControllerFdBaudrateConfig	ECU with "Classic CAN": N/A ECU with CAN FD support: Configure according to requirements in the *[AR_Conf_4] CAN Data Link Layer* and/or the provided ECU Extract.
CanSetBaudrateApi (4.1.1-)	FALSE
CanIfRxPduReadData	FALSE
CanIfTxPduType	STATIC
CanIfSetBaudrateApi (4.1.1-)	FALSE
CanNmActiveWakeupBitEnabled (4.1.1-)	TRUE
CanNmAllNmMessagesKeepAwake (4.0.3-)	FALSE
CanNmBusLoadReductionActive	FALSE
CanNmBusLoadReductionEnabled	FALSE
CanNmBusSynchronizationEnabled	FALSE
CanNmCarWakeUpRxEnabled (4.0.3-)	FALSE
CanNmComUserDataSupport (4.0.2)	FALSE
CanNmComUserDataSupport (4.0.3-)	TRUE
CanNmImmediateNmCycleTime	0.02
CanNmImmediateNmTransmissions	20
CanNmImmediateRestartEnabled	FALSE
CanNmMainFunctionPeriod	0.005
CanNmMsgCycleOffset	0.1
CanNmMsgCycleTime	1.0
CanNmMsgTimeoutTime	0.5
CanNmNodeDetectionEnabled	TRUE
CanNmNodeIdEnabled (4.1.1-)	TRUE
CanNmPassiveModeEnabled	FALSE
CanNmPduCbvPosition	1
CanNmPduNidPosition	0
CanNmMsgReducedTime	NA
CanNmRepeatMessageTime	1.0
CanNmPnEiraCalcEnabled (4.0.3-)	TRUE
CanNmPnEnabled (4.0.3-)	TRUE
CanNmPnEraCalcEnabled (4.0.3-)	For a PNC gateway: TRUE.

	For a non-PNC-gateway: FALSE.
CanNmPnHandleMultipleNetworkRequests (4.0.3-)	TRUE
CanNmPnInfoLength (4.0.3-)	4
CanNmPnInfoOffset (4.0.3-)	2
CanNmPnResetTime (4.0.3-)	2.95
CanNmRetryFirstMessageRequest (4.2.1-)	TRUE
CanNmTimeoutTime	4.0
CanNmUserDataEnabled (4.0.3-)	TRUE
CanNmUserDataLength (4.0.3-4.1.3)	6
CanNmWaitBusSleepTime	4.0
CanSMBorCounterL1ToL2	For GEEA3.0: 32 For EEA4.0: 10
CanSMBorCounterL2Err (4.0.2)	For GEEA3.0: 31 For EEA4.0: 9
CanSMBorTimeL1	For GEEA3.0: 0.01 For EEA4.0: 0.1
CanSMBorTimeL2	For GEEA3.0: 60.0 For EEA4.0: 1.0
CanSMBorTimeTxEnsured	1.0
CanSMEnableBusOffDelay (4.1.3-)	FALSE
CanSMIcomSupport (4.2.1-)	FALSE
CanSMMainFunctionPeriod	0.005
CanSMPncSupport (4.1.2-)	TRUE
CanSMSetBaudrateApi (4.1.1-)	FALSE
CanTpChannelMode	CANTP_MODE_HALF_DUPLEX
CanTpFlexibleDataRateSupport (4.2.1-)	FALSE
CanTpMainFunctionPeriod	0.005
CanTpNar	1.0
CanTpNas	1.0
CanTpNbr	0.02
CanTpNbs	1.0
CanTpNcr	1.0
CanTpNcs	0.02
CanTpRxAddressingFormat	Standard
CanTpRxPaddingActivation	CANTP_ON
CanTpRxWftMax	255
ComFilterAlgorithm	ALWAYS
ComMMainFunctionPeriod	0.005
ComMNmPassiveModeEnable (4.1.1-)	FALSE
ComMNmVariant (4.0.3-)	FULL For managed channel :LIGHT
ComMPncNmRequest (4.0.3-)	TRUE
ComMPncEnabled (4.0.3-)	TRUE
ComMPncGatewayEnabled (4.0.3-)	Domain master ECU: TRUE Other than domain master ECU: FALSE
ComMPncPrepareSleepTimer (4.0.3-)	0.025
ComMPncSupport (4.0.3-)	TRUE
ComM0PncVectorAvoidance (4.3.1-)	FALSE Note: 1.This parameter is for PNC GW and ComMPncGatewayEnabled =TRUE.

	2.Tier2 Vector 4.2.2 BSW already integrate this parameter.
NmBusSynchronizationEnabled	FALSE
NmComUserDataSupport (4.0.2)	FALSE
NmComUserDataSupport (4.0.3-)	TRUE
NmCoordinatorSupportEnabled	FALSE
NmCoordinatorSyncSupport (4.1.1-)	FALSE
NmNodeDetectionEnabled	TRUE
NmNodeIdEnabled	TRUE
NmPassiveModeEnabled	FALSE
NmUserDataEnabled (4.0.3-)	TRUE

Note:
When a configuration parameter value is mandatory for all Autosar versions, no Autosar version is denoted after the parameter name.
When different parameter values are required depending on Autosar version or the parameter exist only for one Autosar version, the applicable version is denoted, e.g. "(4.0.3)".
4.0.3- with a dash after means it is applicable for 4.0.3 and later versions.

Previous: Autosar configuration - required configuration values(4)

These are mandatory values for non-programming session. Using an Autosar version 4.0.3 or higher means that the communication stack must support Partial Network Cluster (PNC).

Parameter	Value
CanControllerFdBaudrateConfig	ECU with "Classic CAN": N/A ECU with CAN FD support: Configure according to requirements in the *[AR_Conf_4] CAN Data Link Layer* and/or the provided ECU Extract.
CanSetBaudrateApi (4.1.1-)	FALSE
CanIfRxPduReadData	FALSE
CanIfTxPduType	STATIC
CanIfSetBaudrateApi (4.1.1-)	FALSE
CanNmActiveWakeupBitEnabled (4.1.1-)	TRUE
CanNmAllNmMessagesKeepAwake (4.0.3-)	FALSE
CanNmBusLoadReductionActive	FALSE
CanNmBusLoadReductionEnabled	FALSE
CanNmBusSynchronizationEnabled	FALSE
CanNmCarWakeUpRxEnabled (4.0.3-)	FALSE
CanNmComUserDataSupport (4.0.2)	FALSE
CanNmComUserDataSupport (4.0.3-)	TRUE
CanNmImmediateNmCycleTime	0.02
CanNmImmediateNmTransmissions	20
CanNmImmediateRestartEnabled	FALSE
CanNmMainFunctionPeriod	0.005
CanNmMsgCycleOffset	0.1
CanNmMsgCycleTime	1.0
CanNmMsgTimeoutTime	0.5
CanNmNodeDetectionEnabled	TRUE
CanNmNodeIdEnabled (4.1.1-)	TRUE
CanNmPassiveModeEnabled	FALSE
CanNmPduCbvPosition	1
CanNmPduNidPosition	0
CanNmMsgReducedTime	NA

CanNmRepeatMessageTime	1.0
CanNmPnEiraCalcEnabled (4.0.3-)	TRUE
CanNmPnEnabled (4.0.3-)	TRUE
CanNmPnEraCalcEnabled (4.0.3-)	For a PNC gateway: TRUE. For a non-PNC-gateway: FALSE.
CanNmPnHandleMultipleNetworkRequests (4.0.3-)	TRUE
CanNmPnInfoLength (4.0.3-)	4
CanNmPnInfoOffset (4.0.3-)	2
CanNmPnResetTime (4.0.3-)	2.95
CanNmRetryFirstMessageRequest (4.2.1-)	TRUE
CanNmTimeoutTime	4.0
CanNmUserDataEnabled (4.0.3-)	TRUE
CanNmUserDataLength (4.0.3-4.1.3)	6
CanNmWaitBusSleepTime	4.0
CanSMBorCounterL1ToL2	For GEEA3.0: 32 For EEA3.5: 10
CanSMBorCounterL2Err (4.0.2)	For GEEA3.0: 31 For EEA3.5: 9
CanSMBorTimeL1	For GEEA3.0: 0.01 For EEA3.5: 0.1
CanSMBorTimeL2	For GEEA3.0: 60.0 For EEA3.5: 1.0
CanSMBorTimeTxEnsured	1.0
CanSMEnableBusOffDelay (4.1.3-)	FALSE
CanSMIcomSupport (4.2.1-)	FALSE
CanSMMainFunctionPeriod	0.005
CanSMPncSupport (4.1.2-)	TRUE
CanSMSetBaudrateApi (4.1.1-)	FALSE
CanTpChannelMode	CANTP_MODE_HALF_DUPLEX
CanTpFlexibleDataRateSupport (4.2.1-)	FALSE
CanTpMainFunctionPeriod	0.005
CanTpNar	1.0
CanTpNas	1.0
CanTpNbr	0.9
CanTpNbs	1.0
CanTpNcr	1.0
CanTpNcs	0.02
CanTpRxAddressingFormat	Standard
CanTpRxPaddingActivation	CANTP_ON
CanTpRxWftMax	255
ComFilterAlgorithm	ALWAYS
ComMMainFunctionPeriod	0.005
ComMNmPassiveModeEnable (4.1.1-)	FALSE
ComMNmVariant (4.0.3-)	FULL For managed channel :LIGHT
ComMPncNmRequest (4.0.3-)	TRUE
ComMPncEnabled (4.0.3-)	TRUE
ComMPncGatewayEnabled (4.0.3-)	Domain master ECU: TRUE Other than domain master ECU: FALSE
ComMPncPrepareSleepTimer (4.0.3-)	0.025

ComMPncSupport (4.0.3-)	TRUE
ComM0PncVectorAvoidance (4.3.1-)	FALSE Note: 1.This parameter is for PNC GW and ComMPncGatewayEnabled =TRUE. 2.Tier2 Vector 4.2.2 BSW already integrate this parameter.
NmBusSynchronizationEnabled	FALSE
NmComUserDataSupport (4.0.2)	FALSE
NmComUserDataSupport (4.0.3-)	TRUE
NmCoordinatorSupportEnabled	FALSE
NmCoordinatorSyncSupport (4.1.1-)	FALSE
NmNodeDetectionEnabled	TRUE
NmNodeIdEnabled	TRUE
NmPassiveModeEnabled	FALSE
NmUserDataEnabled (4.0.3-)	TRUE

Note:
When a configuration parameter value is mandatory for all Autosar versions, no Autosar version is denoted after the parameter name.
When different parameter values are required depending on Autosar version or the parameter exist only for one Autosar version, the applicable version is denoted, e.g. "(4.0.3)".
4.0.3- with a dash after means it is applicable for 4.0.3 and later versions.

663137 v4 **Spare memory**

(3) => (4):
Description modified.

Current:Spare memory(4)

- **The CPU load assessment regulations for the MCU system are as follows:**

The average load of a single core is less than 70% of the total computing power of a single core, the peak load of a single core is less than 90% of the total computing power of a single core, and the load of all cores in any actual scenario is less than 85% of the total computing power of all cores.

- **The CPU load assessment regulations for the SOC system are as follows:**

The average load of a single core should be less than 80% of the total computing power of a single core, the peak load of a single core should be less than 90% of the total computing power of a single core, and the load of all cores in any actual scenario should be less than 80% of the total computing power of all cores.

Previous: Spare memory(3)

- **The CPU load assessment regulations for the MCU system are as follows:**

The average load of a single core is less than 70% of the total computing power of a single core, the peak load of a single core is less than 90% of the total computing power of a single core, and the load of all cores in any actual scenario is less than 85% of the total computing power of all cores.

- **The CPU load assessment regulations for the SOC system are as follows:**

The peak load of a single core should be less than 90% of the total computing power of a single core, and the load of all cores in any actual scenario should be less than 80% of the total computing power of all cores.

665395 v2 **RID - B001**

(1) => (2):
Description modified.

Current:RID - B001(2)

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	B001		
Name	Start New Operation Cycle		
Description	The routine shall be able to: 1. End the current operation cycle: the ECU shall perform all actions it performs when ending an operation cycle (saving data to non-volatile memory, etc). 2. Start a new operation cycle: the ECU shall perform all actions it performs when starting an operation cycle. Note:It shall be possible to execute it in the extended diagnostic session only.		
Type	1		
$P4_{Server_Max}$	5000ms		
Security access	Either **Level 05/06** or **Level 19/1A**		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	N
Request parameters	**Start [0x01] Request parameter**	**Length, Data Type**	
	DTC number	[3 bytes, Hex]	
	Set DTC prepassed/prefailed	[1 byte, Hex]	
	Repeat Counter	[1 byte, Unsigned]	
Response parameters	**Start [0x01]RoutineStatusRecords**	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	
Available in application		Available in bootloader	
O		N	

Previous: RID - B001(1)

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	B001
Name	Start New Operation Cycle
Description	The routine shall be able to: 1. End the current operation cycle: the ECU shall perform all actions it performs when ending an operation cycle (saving data to non-volatile memory, etc). 2. Start a new operation cycle: the ECU shall perform all actions it performs when starting an operation cycle. Note:It shall be possible to execute it in the extended diagnostic session only.
Type	1
$P4_{Server_Max}$	5000ms
Security access	Level 05/06

Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	N

Request parameters	**Start [0x01] Request parameter**	**Length, Data Type**
	DTC number	[3 bytes, Hex]
	Set DTC prepassed/prefailed	[1 byte, Hex]
	Repeat Counter	[1 byte, Unsigned]

Response parameters	**Start [0x01]RoutineStatusRecords**	**Length, Data Type**
	Routine Type	[4 bits, Unsigned]
	Routine Status	[4 bits, Unsigned]

Available in application	Available in bootloader
O	N

665403 v2 **RID - A000**

(1) => (2):
Description modified.

Current:RID - A000(2)

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	A000
Name	Substitue Value In Ethernet Register
Description	Routine for manipulating register values. The value of the register shall only temporarily be substituted by the routine. After a restart, register values shall return to the previous value stored in long term memory. Note： Ethernet node can be used based on internal requirements.
Type	2
$P4_{Server_Max}$	200ms
Security access	Either **Level 05/06** or **Level 19/1A**

Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	M	M

Request parameters	**Start [0x01] Request parameter**	**Length, Data Type**
	Register	[1 byte, Unsigned]
	Page	[1 byte, Unsigned]
	Value	[2 bytes, Unsigned]

Response parameters	**Start [0x01] RoutineStatusRecords**	**Length, Data Type**
	Routine Type	[4 bits, Unsigned]
	Routine Status	[4 bits, Unsigned]
	Additional Result Info	[1 byte, Unsigned]
	Stop [0x02] RoutineStatusRecords	**Length, Data Type**
	Routine Type	[4 bits, Unsigned]
	Routine Status	[4 bits, Unsigned]
	Reserved	[1 byte, Unsigned]
	Result [0x03] RoutineStatusRecords	**Length, Data Type**
	Routine Type	[4 bits, Unsigned]
	Routine Status	[4 bits, Unsigned]
	Additional Result Info	[1 byte, Unsigned]

Available in application	Available in bootloader
O	N

Previous: RID - A000(1)

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	A000		
Name	Substitue Value In Ethernet Register		
Description	Routine for manipulating register values. The value of the register shall only temporarily be substituted by the routine. After a restart, register values shall return to the previous value stored in long term memory. Note：Ethernet node can be used based on internal requirements.		
Type	2		
$P4_{Server_Max}$	200ms		
Security access	Level 05/06		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	M	M
Request parameters	**Start [0x01] Request parameter**	**Length, Data Type**	
	Register	[1 byte, Unsigned]	
	Page	[1 byte, Unsigned]	
	Value	[2 bytes, Unsigned]	

Response parameters	**Start [0x01] RoutineStatusRecords**	**Length, Data Type**
	Routine Type	[4 bits, Unsigned]
	Routine Status	[4 bits, Unsigned]
	Additional Result Info	[1 byte, Unsigned]
	Stop [0x02] RoutineStatusRecords	**Length, Data Type**
	Routine Type	[4 bits, Unsigned]
	Routine Status	[4 bits, Unsigned]
	Reserved	[1 byte, Unsigned]
	Result [0x03] RoutineStatusRecords	**Length, Data Type**
	Routine Type	[4 bits, Unsigned]
	Routine Status	[4 bits, Unsigned]
	Additional Result Info	[1 byte, Unsigned]

Available in application	Available in bootloader
O	N

665404 v2 **RID - A001**

(1) => (2):
Description modified.

Current:RID - A001(2)

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database*.

RID [Hex]	A001		
Name	Ethernet Test Mode 1		
Description	Transmitter droop test mode. Valid only for BroadR-Reach/100BASE-T1. Note1：This is mandatory for Ethernet node. Note2：The test mode routine will disable all diagnostic communication on the Ethernet link, when started it is not possible to stop the routine using service RoutineControlType - stopRoutine. The routine shall stop either when the routine reached the time limit given in the test duration parameter in the request or when the ECU is resetted.		
Type	2		
$P4_{Server_Max}$	200ms		
Security access	Either **Level 05/06** or **Level 19/1A**		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	M
Request parameters	**Start [0x01] Request parameter**	**Length, Data Type**	
	Test Duration Setting	[2 bytes, Unsigned]	
	Test Parameter N	[2 bytes, Unsigned]	

Response parameters	**Start [0x01] RoutineStatusRecords**	**Length, Data Type**
	Routine Type	[4 bits, Unsigned]
	Routine Status	[4 bits, Unsigned]
	Additional Result Info	[1 byte, Unsigned]
	Result [0x03] RoutineStatusRecords	**Length, Data Type**
	Routine Type	[4 bits, Unsigned]
	Routine Status	[4 bits, Unsigned]
	Additional Result Info	[1 byte, Unsigned]

Available in application	Available in bootloader
MC	N

Previous: RID - A001(1)

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	A001		
Name	Ethernet Test Mode 1		
Description	Transmitter droop test mode. Valid only for BroadR-Reach/100BASE-T1. Note1：This is mandatory for Ethernet node. Note2：The test mode routine will disable all diagnostic communication on the Ethernet link, when started it is not possible to stop the routine using service RoutineControlType - stopRoutine. The routine shall stop either when the routine reached the time limit given in the test duration parameter in the request or when the ECU is resetted.		
Type	2		
$P4_{Server_Max}$	200ms		
Security access	Level 05/06		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	M
Request parameters	**Start [0x01] Request parameter**	**Length, Data Type**	
	Test Duration Setting	[2 bytes, Unsigned]	
	Test Parameter N	[2 bytes, Unsigned]	
Response parameters	**Start [0x01] RoutineStatusRecords**	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	
	Additional Result Info	[1 byte, Unsigned]	
	Result [0x03] RoutineStatusRecords	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	
	Additional Result Info	[1 byte, Unsigned]	

Available in application	Available in bootloader
MC	N

665405 v2 **RID - A002**

(1) => (2):
Description modified.

Current:RID - A002(2)

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	A002		
Name	Ethernet Test Mode 2		
Description	Transmitter jitter test in MASTER mode. Note1： This is mandatory for Ethernet node. Note2： The test mode routine will disable all diagnostic communication on the Ethernet link, when started it is not possible to stop the routine using service RoutineControlType - stopRoutine. The routine shall stop either when the routine reached the time limit given in the test duration parameter in the request or when the ECU is resetted.		
Type	2		
$P4_{Server_Max}$	200ms		
Security access	Either **Level 05/06** or **Level 19/1A**		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	M
Request parameters	**Start [0x01] Request parameter** Test Duration Setting	**Length, Data Type** [2 bytes, Unsigned]	
Response parameters	**Start [0x01] RoutineStatusRecords** Routine Type Routine Status Additional Result Info **Result [0x03] RoutineStatusRecords** Routine Type Routine Status Additional Result Info	**Length, Data Type** [4 bits, Unsigned] [4 bits, Unsigned] [1 byte, Unsigned] **Length, Data Type** [4 bits, Unsigned] [4 bits, Unsigned] [1 byte, Unsigned]	
Available in application		Available in bootloader	
MC		N	

Previous: RID - A002(1)

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	A002
Name	Ethernet Test Mode 2

Description	Transmitter jitter test in MASTER mode. Note1：This is mandatory for Ethernet node. Note2：The test mode routine will disable all diagnostic communication on the Ethernet link, when started it is not possible to stop the routine using service RoutineControlType - stopRoutine. The routine shall stop either when the routine reached the time limit given in the test duration parameter in the request or when the ECU is resetted.		
Type	2		
$P4_{Server_Max}$	200ms		
Security access	Level 05/06		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	M
Request parameters	**Start [0x01] Request parameter**	**Length, Data Type**	
	Test Duration Setting	[2 bytes, Unsigned]	
Response parameters	**Start [0x01] RoutineStatusRecords**	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	
	Additional Result Info	[1 byte, Unsigned]	
	Result [0x03] RoutineStatusRecords	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	
	Additional Result Info	[1 byte, Unsigned]	

Available in application	Available in bootloader
MC	N

665406 v2 **RID - A003**

(1) => (2):
Description modified.

Current:RID - A003(2)

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	A003
Name	Ethernet Test Mode 3
Description	Transmit jitter test in SLAVE mode. Note1：This is mandatory for Ethernet node. Note2：The test mode routine will disable all diagnostic communication on the Ethernet link, when started it is not possible to stop the routine using service RoutineControlType - stopRoutine. The routine shall stop either when the routine reached the time limit given in the test duration parameter in the request or when the ECU is resetted.
Type	2
$P4_{Server_Max}$	200ms

Security access	Either **Level 05/06** or **Level 19/1A**		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	M
Request parameters	**Start [0x01] Request parameter** Test Duration Setting		**Length, Data Type** [2 bytes, Unsigned]
Response parameters	**Start [0x01] RoutineStatusRecords** Routine Type Routine Status Additional Result Info **Result [0x03] RoutineStatusRecords** Routine Type Routine Status Additional Result Info		**Length, Data Type** [4 bits, Unsigned] [4 bits, Unsigned] [1 byte, Unsigned] **Length, Data Type** [4 bits, Unsigned] [4 bits, Unsigned] [1 byte, Unsigned]

Available in application	Available in bootloader
MC	N

Previous: RID - A003(1)

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	A003		
Name	Ethernet Test Mode 3		
Description	Transmit jitter test in SLAVE mode. Note1：This is mandatory for Ethernet node. Note2：The test mode routine will disable all diagnostic communication on the Ethernet link, when started it is not possible to stop the routine using service RoutineControlType - stopRoutine. The routine shall stop either when the routine reached the time limit given in the test duration parameter in the request or when the ECU is resetted.		
Type	2		
$P4_{Server_Max}$	200ms		
Security access	Level 05/06		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	M
Request parameters	**Start [0x01] Request parameter** Test Duration Setting		**Length, Data Type** [2 bytes, Unsigned]

Response parameters	**Start [0x01] RoutineStatusRecords**	**Length, Data Type**
	Routine Type	[4 bits, Unsigned]
	Routine Status	[4 bits, Unsigned]
	Additional Result Info	[1 byte, Unsigned]
	Result [0x03] RoutineStatusRecords	**Length, Data Type**
	Routine Type	[4 bits, Unsigned]
	Routine Status	[4 bits, Unsigned]
	Additional Result Info	[1 byte, Unsigned]

Available in application	Available in bootloader
MC	N

665407 v2 **RID - A004**

(1) => (2):
Description modified.

Current:RID - A004(2)

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	A004		
Name	Ethernet Test Mode 4		
Description	Transmitter distortion test. Note1：This is mandatory for Ethernet node. Note2：The test mode routine will disable all diagnostic communication on the Ethernet link, when started it is not possible to stop the routine using service RoutineControlType - stopRoutine. The routine shall stop either when the routine reached the time limit given in the test duration parameter in the request or when the ECU is resetted.		
Type	2		
$P4_{Server_Max}$	200ms		
Security access	Either **Level 05/06** or **Level 19/1A**		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	M
Request parameters	**Start [0x01] Request parameter**	**Length, Data Type**	
	Test Duration Setting	[2 bytes, Unsigned]	
Response parameters	**Start [0x01] RoutineStatusRecords**	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	
	Additional Result Info	[1 byte, Unsigned]	
	Result [0x03] RoutineStatusRecords	**Length, Data Type**	
	Routine Type	[4 bits, Unsigned]	
	Routine Status	[4 bits, Unsigned]	
	Additional Result Info	[1 byte, Unsigned]	

Available in application	Available in bootloader
MC	N

Previous: RID - A004(1)

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	A004		
Name	Ethernet Test Mode 4		
Description	Transmitter distortion test. Note1：This is mandatory for Ethernet node. Note2：The test mode routine will disable all diagnostic communication on the Ethernet link, when started it is not possible to stop the routine using service RoutineControlType - stopRoutine. The routine shall stop either when the routine reached the time limit given in the test duration parameter in the request or when the ECU is resetted.		
Type	2		
$P4_{Server_Max}$	200ms		
Security access	Level 05/06		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	M
Request parameters	**Start [0x01] Request parameter** Test Duration Setting	**Length, Data Type** [2 bytes, Unsigned]	
Response parameters	**Start [0x01] RoutineStatusRecords** Routine Type Routine Status Additional Result Info **Result [0x03] RoutineStatusRecords** Routine Type Routine Status Additional Result Info	**Length, Data Type** [4 bits, Unsigned] [4 bits, Unsigned] [1 byte, Unsigned] **Length, Data Type** [4 bits, Unsigned] [4 bits, Unsigned] [1 byte, Unsigned]	

Available in application	Available in bootloader
MC	N

665408 v2 **RID - A005**

(1) => (2):
Description modified.

Current:RID - A005(2)

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database.*

RID [Hex]	A005		
Name	Ethernet Test Mode 5		
Description	Normal operation at full power (for the PSD mask). Note1：This is mandatory for Ethernet node. Note2：The test mode routine will disable all diagnostic communication on the Ethernet link, when started it is not possible to stop the routine using service RoutineControlType - stopRoutine. The routine shall stop either when the routine reached the time limit given in the test duration parameter in the request or when the ECU is resetted.		
Type	2		
$P4_{Server_Max}$	200ms		
Security access	Either **Level 05/06** or **Level 19/1A**		
Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	M
Request parameters	**Start [0x01] Request parameter** Test Duration Setting	**Length, Data Type** [2 bytes, Unsigned]	
Response parameters	**Start [0x01] RoutineStatusRecords** Routine Type Routine Status Additional Result Info **Result [0x03] RoutineStatusRecords** Routine Type Routine Status Additional Result Info	**Length, Data Type** [4 bits, Unsigned] [4 bits, Unsigned] [1 byte, Unsigned] **Length, Data Type** [4 bits, Unsigned] [4 bits, Unsigned] [1 byte, Unsigned]	
Available in application		Available in bootloader	
MC		N	

Previous: RID - A005(1)

The following data identifiers should be taken into account,and exactly as defined in *Global Master Reference Database*.

RID [Hex]	A005
Name	Ethernet Test Mode 5
Description	Normal operation at full power (for the PSD mask). Note1：This is mandatory for Ethernet node. Note2：The test mode routine will disable all diagnostic communication on the Ethernet link, when started it is not possible to stop the routine using service RoutineControlType - stopRoutine. The routine shall stop either when the routine reached the time limit given in the test duration parameter in the request or when the ECU is resetted.
Type	2
$P4_{Server_Max}$	200ms
Security access	Level 05/06

Supported Sub-functions	Start [0x01]	Stop [0x02]	Result [0x03]
	M	N	M

Request parameters	**Start [0x01] Request parameter** Test Duration Setting	**Length, Data Type** [2 bytes, Unsigned]
Response parameters	**Start [0x01] RoutineStatusRecords** Routine Type Routine Status Additional Result Info **Result [0x03] RoutineStatusRecords** Routine Type Routine Status Additional Result Info	**Length, Data Type** [4 bits, Unsigned] [4 bits, Unsigned] [1 byte, Unsigned] **Length, Data Type** [4 bits, Unsigned] [4 bits, Unsigned] [1 byte, Unsigned]

Available in application	Available in bootloader
MC	N

666478 v5 **DTC - Control module communication bus off**

(4) => (5):
Description modified.

Current:DTC - Control module communication bus off(5)

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database*.

DTC Code (High Byte and Middle Byte)	GMRDB	Failure type (Low Byte)	0x00 (the ECU is emission) 0x88 (the ECU is not emission related)
Description	Control module communication bus off Note: Mandatory that all Public CAN ECUs on the network shall be able to detect and set bus-off DTCs when the CAN controller has gotten into the state bus-off.		
Test run criteria (TRC)	**TRC1:** The TRC to perform the bus-off check shall be according to the General Test Run Criteria specified in "*Requirements from section Detection of faults*". However, the inhibit criteria "during 5 seconds from any transition from one usage mode to another" and "During 5 seconds from the car mode of factory, transport and crash transmition to dyno,normal" according to the Generic Test Run Criteria is not applicable for this DTC. **TRC2**: The detection shall be within specific voltage range defined in *ReqID 660853 Operating voltage range hysteresis.*		

Detection Criteria	**For all Autosar versions:** **For GEEA3.0 Architecture:** There shall be **32** detections of CAN bus off in CANSM until a DTC for CAN bus off is set to testFailed. **For EEA4.0 Architecture:** There shall be **10** detections of CAN bus off in CANSM until a DTC for CAN bus off is set to testFailed. Then a diagnostic trouble code (DTC) for CAN bus off shall be set within 100 ms. Depending on Autosar version, a fault detection counter is implemented in CANSM or DEM. The number of CAN bus off events required until a DTC is set to testFailed shall be same regardless of Autosar version. **Specific for Autosar 4.0.2:** If the bus off counter in CANSM exceeds CanSMBorCounterL2Err, and is defined in ReqID 663093, a diagnostic trouble code (DTC) shall be set to testFailed (FDC=+127). **Specific for Autosar 4.0.3 and later:** where no bus off counter exist in CANSM but a fault detection counter FDC10 in DEM shall acts as a prescaler until DTC is set: The fault detection counter FDC10 in DEM shall be increased by (127/ (CanSMBorCounterL2Err+1)) at every test call from CANSM to DEM with EventStatus := DEM_EVENT_STATUS_PRE_FAILED. This results in that 32 consecutive calls are required to make the FDC10 counter reach +127 and to set DTC to testFailed.

DTC CALIBRATION for GEEA3.0 Architecture

comfirmedDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
Note 1	255	OCC2 or OCC5	Yes or No	10
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	4	Yes	4	128

DTC CALIBRATION for EEA4.0 Architecture

comfirmedDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
Note 1	255	OCC2 or OCC5	Yes or No	100
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	13	Yes	13	128

Notes in the table above:
1. Parameter confirmedDTCLimit of DTC Calibration is usually configured as 1. The Emission ECU can configure parameter confirmedDTCLimit according to legislation regulation, e.g. may be different from 1.

Previous: DTC - Control module communication bus off(4)

The following diagnostic trouble code should be taken into account,and exactly as defined in *Global Master Reference Database.*

DTC Code (High Byte and Middle Byte)	GMRDB	Failure type (Low Byte)	0x00 (the ECU is emission) 0x88 (the ECU is not emission related)
Description	Control module communication bus off Note: Mandatory that all Public CAN ECUs on the network shall be able to detect and set bus-off DTCs when the CAN controller has gotten into the state bus-off.		

Test run criteria (TRC)	**TRC1:** The TRC to perform the bus-off check shall be according to the General Test Run Criteria specified in "*Requirements from section Detection of faults*". However, the inhibit criteria "during 5 seconds from any transition from one usage mode to another" and "During 5 seconds from the car mode of factory, transport and crash transmition to dyno,normal" according to the Generic Test Run Criteria is not applicable for this DTC. **TRC2**: The detection shall be within specific voltage range defined in *ReqID 660853 Operating voltage range hysteresis.*
Detection Criteria	**For all Autosar versions:** **For GEEA3.0 Architecture:** There shall be **32** detections of CAN bus off in CANSM until a DTC for CAN bus off is set to testFailed. **For EEA3.5 Architecture:** There shall be **10** detections of CAN bus off in CANSM until a DTC for CAN bus off is set to testFailed. Then a diagnostic trouble code (DTC) for CAN bus off shall be set within 100 ms. Depending on Autosar version, a fault detection counter is implemented in CANSM or DEM. The number of CAN bus off events required until a DTC is set to testFailed shall be same regardless of Autosar version. **Specific for Autosar 4.0.2:** If the bus off counter in CANSM exceeds CanSMBorCounterL2Err, and is defined in ReqID 663093, a diagnostic trouble code (DTC) shall be set to testFailed (FDC=+127). **Specific for Autosar 4.0.3 and later:** where no bus off counter exist in CANSM but a fault detection counter FDC10 in DEM shall acts as a prescaler until DTC is set: The fault detection counter FDC10 in DEM shall be increased by (127/ (CanSMBorCounterL2Err+1)) at every test call from CANSM to DEM with EventStatus := DEM_EVENT_STATUS_PRE_FAILED. This results in that 32 consecutive calls are required to make the FDC10 counter reach +127 and to set DTC to testFailed.

DTC CALIBRATION for GEEA3.0 Architecture

comfirmedDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
Note 1	255	OCC2 or OCC5	Yes or No	10
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	4	Yes	4	128

DTC CALIBRATION for EEA3.5 Architecture

comfirmedDTCLimit	agedDTCLimit	agingCounter	Warning Indicator	Test Peroid (ms)
Note 1	255	OCC2 or OCC5	Yes or No	100
FDC10 Max value	unconfirmedDTCLimit	Jump down	Step up	Step down
127	13	Yes	13	128

Notes in the table above:
1. Parameter confirmedDTCLimit of DTC Calibration is usually configured as 1. The Emission ECU can configure parameter confirmedDTCLimit according to legislation regulation, e.g. may be different from 1.

675384 v3 **Return Code**

(2) => (3):
Attribute 'Purpose' added: No preview available.
Description modified.

Current:Return Code(3)

Currently defined Return Codes are shown in below table:

ID	CodeName	Description
0x00	E_OK	No error occurred.
0x01	E_NOT_OK	An unspecified error occurred.
~~0x02~~	~~E_UNKNOWN_SERVICE~~	~~The requested Service ID is unknown.~~
~~0x03~~	~~E_UNKNOWN_METHOD~~	~~The requested Method ID is unknown. Service ID is known.~~
0x04	E_NOT_READY	Service ID and Method ID ARE known. Application not running.
0x05	E_NOT_REACHABLE	System running the service is not reachable (internal error code only).
0x06	E_TIMEOUT	A timeout occurred (internal error code only).
0x07	E_WRONG_PROTOCOL_VERSION	Version of SOME/IP protocol not supported.
0x08	E_WRONG_INTERFACE_VERSION	The interface version of the requested service mismatch.
0x09	E_MALFORMED_MESSAGE	Deserialization error so that payload cannot be deserialized.
0x0a	E_WRONG_MESSAGE_TYPE	An unexpected message type was received (e.g. REQUEST_NO_RETURN for a method defined as REQUEST).
0x0b	E_E2E_REPEATED	Repeated E2E calculation error.
0x0c	E_E2E_WRONG_SEQUENCE	Wrong E2E sequence error.
0x0d	E_E2E	Not further specified E2E error.
0x0e	E_E2E_NOT_AVAILABLE	E2E not available.
0x0f	E_E2E_NO_NEW_DATA	No new data for E2E calculation present.
0x10 - 0x1f	RESERVED	Reserved for generic SOME/IP errors. These errors will be specified in future versions of this document.
0x20 - x5e	RESERVED	Reserved for specific errors of services and methods. These errors are specified by the interface specification.

Previous: Return Code(2)

Currently defined Return Codes are shown in below table:

ID	CodeName	Description
0x00	E_OK	No error occurred.
0x01	E_NOT_OK	An unspecified error occurred.
0x02	E_UNKNOWN_SERVICE	The requested Service ID is unknown.
0x03	E_UNKNOWN_METHOD	The requested Method ID is unknown. Service ID is known.
0x04	E_NOT_READY	Service ID and Method ID ARE known. Application not running.
0x05	E_NOT_REACHABLE	System running the service is not reachable (internal error code only).
0x06	E_TIMEOUT	A timeout occurred (internal error code only).
0x07	E_WRONG_PROTOCOL_VERSION	Version of SOME/IP protocol not supported.
0x08	E_WRONG_INTERFACE_VERSION	The interface version of the requested service mismatch.
0x09	E_MALFORMED_MESSAGE	Deserialization error so that payload cannot be deserialized.
0x0a	E_WRONG_MESSAGE_TYPE	An unexpected message type was received (e.g. REQUEST_NO_RETURN for a method defined as REQUEST).
0x0b	E_E2E_REPEATED	Repeated E2E calculation error.
0x0c	E_E2E_WRONG_SEQUENCE	Wrong E2E sequence error.
0x0d	E_E2E	Not further specified E2E error.
0x0e	E_E2E_NOT_AVAILABLE	E2E not available.
0x0f	E_E2E_NO_NEW_DATA	No new data for E2E calculation present.
0x10 - 0x1f	RESERVED	Reserved for generic SOME/IP errors. These errors will be specified in future versions of this document.
0x20 - x5e	RESERVED	Reserved for specific errors of services and methods. These errors are specified by the interface specification.

849660 v4 mDNS parameter for SOVD

(3) => (4):
Description modified.

Current:mDNS parameter for SOVD(4)

The public SOVD gatewy or public SOVD server shall implement mDNS host according to [SOVD_10]. The following parameter shall be used.

Parameter	Definition
Service Type	Service type is used to identify a specific service, rather than a specific device. The protocol "_sovd._tcp.local." of the service shall be used.
Service Instance	The service instance is a specific implementation of a service tpye, e.g. {Hostname}._sovd._tcp.local.
Protocol	TCP
Port	Communication based on HTTPs, using port number 7690.
TXT	accessurl=https://{hostname}.local:7690. SOVD Client can use this accessurl to get the SOVD version through */version-info* API.

Previous: mDNS parameter for SOVD(3)

The public SOVD gatewy or public SOVD server shall implement mDNS host according to [SOVD_10]. The following parameter shall be used.

Parameter	Definition
Service Type	Service type is used to identify a specific service, rather than a specific device. The protocol "_sovd._tcp.local." of the service shall be used.
Service Instance	The service instance is a specific implementation of a service tpye, e.g. {Hostname}._sovd._tcp.local.
Protocol	TCP
Port	Communication based on HTTPs, using port number 7690.
TXT	path=https://{hostname}.local:7690. SOVD Client can use this accessurl to get the SOVD version through */version-info* API.

849661 v3 mDNS Announcement/Response

(2) => (3):
Description modified.

Current:mDNS Announcement/Response(3)

The SOVD Edge Node shall send mDNS Announcement according to the following requirement when the funciton is triggered through by hardward or software method.

Parameter	Definition
Service registration stage	When an edge node first joins the network or registers for a new service e.g., ECU initialization, ip change, hostname change, it shall send three consecutive multicasts with interval timing 1 second to ensure reliable transmission of multicat packets and avoid confilcts.
Maintenance phase	Edge node shall regularly send one service announcement to maintain local caching. The default interval is 60 seconds with a random jitter of 0~10%. The TTL (Time to Live) for service records shall set to 120.

Query response mechanism	When receiving a service query, the vehicle shall respond mDNS response within random delays time 500ms to 1000ms.

Previous: mDNS Announcement/Response(2)

The SOVD Edge Node shall send mDNS Announcement according to the following requirement when the funciton is triggered through by hardward or software method.

Parameter	Definition
Service registration stage	When an edge node first joins the network or registers for a new service e.g., ECU initialization, ip change, hostname change, it shall send three consecutive multicasts with interval timing 1 second to ensure reliable transmission of multicat packets and avoid confilcts.
Maintenance phase	Edge node shall regularly send one service announcement to maintain local caching. The default interval is 60 seconds with a random jitter of 0~10%. The TTL (Time to Live) for service records shall set to 2 minutes.
Query response mechanism	When receiving a service query, the vehicle shall respond mDNS response within random delays time 500ms to 1000ms.

853765 v4 **General negative response code**

(3) => (4):
Description modified.

Current:General negative response code(4)

The general supported negative response code shall be within the following list:

Diagnostic Services	Allowed Negative Response Code
DiagnosticSessionControl (0x10)	0x12, 0x13, 0x22, 0xF0~0xFE
ECUReset (0x11)	0x12, 0x13, 0x22, 0xF0~0xFE
SecurityAccess (0x27)	0x12, 0x13, 0x24, 0x31, 0x35, 0x36, 0x37, 0x7F/0x7E
CommunicationControl (0x28)	0x12, 0x13, 0x22, 0x31, 0x7F/0x7E, 0xF0~0xFE
Authentication (0x29)	0x12, 0x13, 0x22, 0x24, 0x50, 0x51, 0x52, 0x53, 0x54, 0x55, 0x56, 0x57, 0x58, 0x5D
TesterPresent (0x3E)	0x12, 0x13
ControlDTCSetting (0x85)	0x12, 0x13, 0x22, 0x31, 0x7F/0x7E, 0xF0~0xFE
ReadDataByIdentifier (0x22)	0x13, 0x14, 0x22, 0x31, 0x33, 0x7F/0x7E
ReadMemoryByAddress (0x23)	0x13, 0x14, 0x31, 0x33, 0x7F/0x7E
ReadDataByPeriodicIdentifier (0x2A)	0x13, 0x31
DynamicallyDefineDataIdentifier (0x2C)	0x12, 0x13, 0x31, 0x33, 0x7F/0x7E
WriteDataByIdentifier (0x2E)	0x13, 0x22, 0x31, 0x33, 0x72, 0x7F/0x7E
WriteMemoryByAddress (0x3D)	0x13, 0x22, 0x31, 0x33, 0x72, 0x7F/0x7E
ClearDiagnosticInformation (0x14)	0x13, 0x22, 0x31, 0x72
ReadDTCInformation (0x19)	0x12, 0x13, 0x31
InputOutputControlByIdentifier (0x2F)	0x13, 0x22, 0x31, 0x33, 0x7F/0x7E
RoutineControl (0x31)	0x12, 0x13, 0x22, 0x24, 0x31, 0x33, 0x72
RequestDownload (0x34)	0x13, 0x22, 0x31, 0x33, 0x70
RequestUpload (0x35)	0x13, 0x22, 0x31, 0x33, 0x70
TransferData (0x36)	0x13, 0x24, 0x31, 0x71, 0x72, 0x73, (0x92), (0x93)
RequestTransferExit (0x37)	0x13, 0x24, 0x31, 0x72
RequestFileTransfer (0x38)	0x13, 0x22, 0x24, 0x31, 0x33, 0x70

For the negative response code NRC 0x22 which conditions not correct, it is not possible to identify which condition is not met. Therefore, utilize negative response code - VehicleManufactureSpecificConditionNotCorrect range to match the conditions with each negative response code one by one.

Negateive Response Code (NRC) for VehicleManufactureSpecificConditionNotCorrect	Descrpition
0xF0	Condition "Vehicle speed < 3km/h and VehSpdLgtQf is equal to 2 or 3" is not met.
0xF1	Condition "Main propulsion system is not active which means that the signal UsgModSts is not equal to "Driving"" is not met.
0xF2	Condition "The power supply voltage of ECU for SWDL" is not met.
0xF3	Condition "Main propulsion system is not active which means that the signal HvSysRlySts is equal to as "HvActvnSts_Open"" is not met.
0xF4	Condition "High voltage is less than 60V (The high voltage will be detected by MGM/IEM/IGM itself)" is not met.
0xF5	Condition "Shifter is in "P"or "N" position" is not met.
0xF6	Condition "PowerPackStatus is under flashable status which means that the signal EngSt1WdSts is equal to as any one of "EngSt1_Ini", "EngSt1_Awake" and "EngSt1_Rdy" is not met.
0xF7 ~ 0xFE	Undefined

Note1: NRC 0x78 is in general supported by each diagnostic service, as not otherwise stated in the data link specific implementation document, therefore it is not listed in the list of applicable response codes of the diagnostic services.
Note2: NRC 0x34 is in general supported by each diagnostic service for ECU supporting Authentication (0x29) service, therefore it is not listed in the list of applicable response codes of the diagnostic services.
Note3: Regarding to negative response code for VehicleManufactureSpecificConditionNotCorrect, the smaller the value, the higher the priority.
Note4: Which specific negateive response code to use shall follow specific requirements.

Previous: General negative response code(3)

The general supported negative response code shall be within the following list:

Diagnostic Services	Allowed Negative Response Code
DiagnosticSessionControl (0x10)	0x12, 0x13, 0x22, 0xF0~0xFE
ECUReset (0x11)	0x12, 0x13, 0x22, 0xF0~0xFE
SecurityAccess (0x27)	0x12, 0x13, 0x24, 0x31, 0x35, 0x36, 0x37, 0x7F/0x7E
CommunicationControl (0x28)	0x12, 0x13, 0x22, 0x31, 0x7F/0x7E, 0xF0~0xFE
Authentication (0x29)	0x12, 0x13, 0x22, 0x24, 0x50, 0x51, 0x52, 0x53, 0x54, 0x55, 0x56, 0x57, 0x58, 0x5D
TesterPresent (0x3E)	0x12, 0x13
ControlDTCSetting (0x85)	0x12, 0x13, 0x22, 0x31, 0x7F/0x7E, 0xF0~0xFE
ReadDataByIdentifier (0x22)	0x13, 0x14, 0x22, 0x31, 0x33, 0x7F/0x7E
ReadMemoryByAddress (0x23)	0x13, 0x14, 0x31, 0x33, 0x7F/0x7E
ReadDataByPeriodicIdentifier (0x2A)	0x13, 0x31
DynamicallyDefineDataIdentifier (0x2C)	0x12, 0x13, 0x31, 0x33, 0x7F/0x7E
WriteDataByIdentifier (0x2E)	0x13, 0x22, 0x31, 0x33, 0x72, 0x7F/0x7E
WriteMemoryByAddress (0x3D)	0x13, 0x22, 0x31, 0x33, 0x72, 0x7F/0x7E
ClearDiagnosticInformation (0x14)	0x13, 0x22, 0x31, 0x72
ReadDTCInformation (0x19)	0x12, 0x13, 0x31
InputOutputControlByIdentifier (0x2F)	0x13, 0x22, 0x31, 0x33, 0x7F/0x7E
RoutineControl (0x31)	0x12, 0x13, 0x22, 0x24, 0x31, 0x33, 0x72
RequestDownload (0x34)	0x13, 0x22, 0x31, 0x33, 0x70

RequestUpload (0x35)	0x13, 0x22, 0x31, 0x33, 0x70
TransferData (0x36)	0x13, 0x24, 0x31, 0x71, 0x72, 0x73, (0x92), (0x93)
RequestTransferExit (0x37)	0x13, 0x24, 0x31, 0x72
RequestFileTransfer (0x38)	0x13, 0x22, 0x24, 0x31, 0x33, 0x70

For the negative response code NRC 0x22 which conditions not correct, it is not possible to identify which condition is not met. Therefore, utilize negative response code - VehicleManufactureSpecificConditionNotCorrect range to match the conditions with each negative response code one by one.

Negateive Response Code (NRC) for VehicleManufactureSpecificConditionNotCorrect	Descrpition
0xF0	Condition "Vehicle speed < 3km/h and VehSpdLgtQf is equal to 2 or 3" is not met.
0xF1	Condition "Main propulsion system is not active which means that the signal UsgModSts is not equal to "Driving"" is not met.
0xF2	Condition "The power supply voltage of ECU for SWDL" is not met.
0xF3	Condition "Main propulsion system is not active which means that the signal HvSysRlySts is equal to as "HvActvnSts_Open"" is not met.
0xF4	Condition "High voltage is less than 60V (The high voltage will be detected by MGM/IEM/IGM itself)" is not met.
0xF5	Condition "Shifter is in "P"or "N" position, where TrsmActrPosn TrsmActrPosnSafe corresponds shifter position" is not met.
0xF6	Condition "PowerPackStatus is under flashable status which means that the signal EngSt1WdSts is equal to as any one of "EngSt1_Ini", "EngSt1_Awake" and "EngSt1_Rdy" is not met.
0xF7 ~ 0xFE	Undefined

Note1: NRC 0x78 is in general supported by each diagnostic service, as not otherwise stated in the data link specific implementation document, therefore it is not listed in the list of applicable response codes of the diagnostic services.
Note2: NRC 0x34 is in general supported by each diagnostic service for ECU supporting Authentication (0x29) service, therefore it is not listed in the list of applicable response codes of the diagnostic services.
Note3: Regarding to negative response code for VehicleManufactureSpecificConditionNotCorrect, the smaller the value, the higher the priority.
Note4: Which specific negateive response code to use shall follow specific requirements.

857424 v3 **Memory Requirements**

(2) => (3):
Attribute 'Purpose' modified: No preview available.
Description modified.

Current:Memory Requirements(3)

ECU shall allocate reasonable and dedicated memory for each software part during update. These memory shall be dedicated to software part storage to avoid overlapping use with other data partitions. If the software part needs to share the storage partition with other data due to memory size limitations, it needs to be approved by the Basetech team.

Comment:
Based address memory ECU, receives software part data in RAM, and after processing (decompression/decryption), it is directly written into the program flash partition.

Based file memory ECU, requires software part storage memory and another storage memory for the original installation files obtained by decoding/refactoring software parts (in most cases), which means there will be at least two storage memory. Also additonal buffer memory.
If these two storage memory share storage areas with other temporay data, the temporary data shall be cleared

and the storage memory for update shall be organized before enabling software part transfer. The clearing action shall be performed asynchronously to avoid diagnostic response timeout.

For ECU supporting OTA distributed update, it should have independent and sufficient memory to store and update software during SWDL process, rather than being shared with other upgrade methods such as OTA distributed upgrades.

Previous: Memory Requirements(2)

For ECU supporting OTA distributed update, it should have independent and sufficient memory to store and update software during SWDL process, rather than being shared with other upgrade methods such as OTA distributed upgrades.

871742 v2 **NM Parameter Configuration**

(1) => (2):
Attribute 'Verification Method' modified: Test => Analysis AND Test.

Current:NM Parameter Configuration(2)

Configuration Parameter	Value
UdpNmActiveWakeupBitEnabled	TRUE
UdpNmAllNmMessagesKeepAwake	FALSE
UdpNmBusSynchronizationEnabled	FALSE
UdpNmCarWakeUpRxEnabled	FALSE
UdpNmNodeDetectionEnabled	TRUE
UdpNmNodeIdEnabled	TRUE
UdpNmPassiveModeEnabled	FALSE
UdpNmPduCbvPosition	1
UdpNmPduNidPosition	0
UdpNmComUserDataSupport	TRUE
UdpNmImmediateNmCycleTime	0.02
UdpNmImmediateNmTransmissions	20
UdpNmImmediateRestartEnabled	FALSE
UdpNmMainFunctionPeriod	0.005
UdpNmMsgCycleOffset	0
UdpNmMsgCycleTime	1.0
UdpNmRepeatMessageTime	1.0
UdpNmRepeatMsgIndEnabled	FALSE
UdpNmRetryFirstMessageRequest	TRUE
UdpNmPnEnabled	TRUE
UdpNmPnEiraCalcEnabled	TRUE
UdpNmPnEraCalcEnabled	TRUE For PNC Gateway FALSE For Other
UdpNmPnHandleMultipleNetworkRequests	TRUE
UdpNmPnInfoLength	6
UdpNmPnInfoOffset	2
UdpNmPnResetTime	2.95
UdpNmTimeoutTime	3
UdpNmWaitBusSleepTime	4.0

Previous: NM Parameter Configuration(1)

Configuration Parameter	Value
UdpNmActiveWakeupBitEnabled	TRUE
UdpNmAllNmMessagesKeepAwake	FALSE
UdpNmBusSynchronizationEnabled	FALSE

UdpNmCarWakeUpRxEnabled	FALSE
UdpNmNodeDetectionEnabled	TRUE
UdpNmNodeIdEnabled	TRUE
UdpNmPassiveModeEnabled	FALSE
UdpNmPduCbvPosition	1
UdpNmPduNidPosition	0
UdpNmComUserDataSupport	TRUE
UdpNmImmediateNmCycleTime	0.02
UdpNmImmediateNmTransmissions	20
UdpNmImmediateRestartEnabled	FALSE
UdpNmMainFunctionPeriod	0.005
UdpNmMsgCycleOffset	0
UdpNmMsgCycleTime	1.0
UdpNmRepeatMessageTime	1.0
UdpNmRepeatMsgIndEnabled	FALSE
UdpNmRetryFirstMessageRequest	TRUE
UdpNmPnEnabled	TRUE
UdpNmPnEiraCalcEnabled	TRUE
UdpNmPnEraCalcEnabled	TRUE For PNC Gateway FALSE For Other
UdpNmPnHandleMultipleNetworkRequests	TRUE
UdpNmPnInfoLength	6
UdpNmPnInfoOffset	2
UdpNmPnResetTime	2.95
UdpNmTimeoutTime	3
UdpNmWaitBusSleepTime	4.0

908756 v3 **Concurrent SOVD External Client Connection**

(2) => (3):
Attribute 'Purpose' added: No preview available.
Attribute 'Verification Method' modified: Test => Analysis AND Test.
Description modified.

Current:Concurrent SOVD External Client Connection(3)

A single SOVD Gateway shall allow a maximum of 5 concurrent authenticated client connections. This limit covers all external clients that have passed authentication. If there are more external clients accessing, Gateway shall reject the connection, e.g. TCP RST.

Rationale:
In mTLS connection, Gateway shall use a combination of source IP of TCP socket four-tuple and the Client ID as identification for external clients to determine the number of clients.

Note: The Client ID uses the CN filed value in the client certificate from the mTLS connection

Previous: Concurrent SOVD Client Connection(2)

The number of authenticated SOVD Client connections simultaneously maintained by SOVD Edge Node for SOVD Diagnostic is up to 5.

992028 v2 **sw_part_type**

(1) => (2):
Description modified.

Current:sw_part_type(2)

The **sw_part_type** identifier indicates different types of a software parts in an ECU. Only the reserved word

IdentifierValues defined in the table below shall be considered valid for the **sw_part_type** identifier.

Table: Software Part Types and Allowed Reserved Words

sw_part_type IdentifierValue Reserved Words	Description
CARCFG	Car configuration
CUSTOM	Customer parameters
DATA	Data or parameters (i.e., calibrations)
EXE	Executable
SBL	Secondary Bootloader
SIGCFG	Signal configuration
PBL	Primary Bootloader
SSBL	Special SBL which has the possibility to replace the PBL
TEST	Test program, (i.e. production test, diagnostics)
FBL	Flashable Bootloader
CERT	Certificate
PWD	Password for certificate
APP	Application file
MAP	Map file used for navigation
ESS	ECU Software Structure
SIGNED	Digital Signature file
VARIANT_INFO	Settings file (for offboard tool settings)
DIFDLT_PKG	Differential-or-delta input package used by system/tool to generate differential/delta software part.

The identifier **sw_part_type** in a single VBF file can only be assigned one of the reserved word values in the table above.

A VBF file with **sw_part_type = SBL or SSBL** is not allowed to use the identifier **erase**. These software types are downloaded to RAM and no erase operation shall be performed in this case.

Examples:

Valid Format:

```
1.      sw_part_type = SIGCFG ;

2.      sw_part_type = EXE ;                        // or any other valid type but "SBL" or "SSBL"
    erase =  {   { 0x00004200, 0x00003CBF },
                 { 0x00008010, 0x00002FF3 }
             };
```

Invalid Format:

```
1.      sw_part_type = exe;                         //  Invalid case

2.      sw_part_type = SBL;                         // Invalid combination of sw_part_type and erase
    erase =  {   { 0x00004200, 0x00003CBF },
                 { 0x00008010, 0x00002FF3 }
             } ;

3.      sw_part_type = MYOWNTYPE;                   // Unknown IdentifierValue
```

Previous: sw_part_type(1)

The **sw_part_type** identifier indicates different types of a software parts in an ECU. Only the reserved word IdentifierValues defined in the table below shall be considered valid for the **sw_part_type** identifier.

Table: Software Part Types and Allowed Reserved Words

sw_part_type IdentifierValue	Description

Reserved Words	
CARCFG	Car configuration
CUSTOM	Customer parameters
DATA	Data or parameters (i.e., calibrations)
EXE	Executable
SBL	Secondary Bootloader
SIGCFG	Signal configuration
PBL	Primary Bootloader
SSBL	Special SBL which has the possibility to replace the PBL
TEST	Test program, (i.e. production test, diagnostics)
FBL	Flashable Bootloader
CERT	Certificate
PWD	Password for certificate
APP	Application file
MAP	Map file used for navigation
ESS	ECU Software Structure
SIGNED	Digital Signature file
VARIANT_INFO	Settings file (for offboard tool settings)

The identifier **sw_part_type** in a single VBF file can only be assigned one of the reserved word values in the table above.

A VBF file with **sw_part_type = SBL or SSBL** is not allowed to use the identifier **erase**. These software types are downloaded to RAM and no erase operation shall be performed in this case.

Examples:

Valid Format:

```
1.    sw_part_type = SIGCFG ;

2.    sw_part_type = EXE ;                    // or any other valid type but "SBL" or "SSBL"
    erase =  {   { 0x00004200, 0x00003CBF },
                 { 0x00008010, 0x00002FF3 }
             };
```

Invalid Format:

```
1.    sw_part_type = exe;                     //  Invalid case

2.    sw_part_type = SBL;                     // Invalid combination of sw_part_type and erase
    erase =  {   { 0x00004200, 0x00003CBF },
                 { 0x00008010, 0x00002FF3 }
             } ;

3.    sw_part_type = MYOWNTYPE;               // Unknown IdentifierValue
```

1029361 v2 **OBDonUDS address assignment**

(1) => (2):
Description modified.

Current:OBDonUDS address assignment(2)

The OBD diagnostic addresses and function addressing of the ECU are defined in ISO 15765-4;
The specific address allocation for the ECU is as follows:

- GEEA3.0 & EEA4.0 ECU SWRS Req 667016 url:swap://10.24.24.227:27000/x04000000209E3DE1
- GEEA2.0 ECU SWRS Req 267539 url:swap://10.24.24.227:27000/x04000000197AB54D

If cannot find the ECU from the above requirements, please contact the Basetech team .

Previous: OBDonUDS address assignment(1)

The OBD diagnostic addresses and function addressing of the ECU are defined in ISO 15765-4;
The specific address allocation for the ECU is as follows:

- GEEA3.0 & EEA3.5 ECU SWRS Req 667016 url:swap://10.24.24.227:27000/x04000000209E3DE1
- GEEA2.0 ECU SWRS Req 267539 url:swap://10.24.24.227:27000/x04000000197AB54D

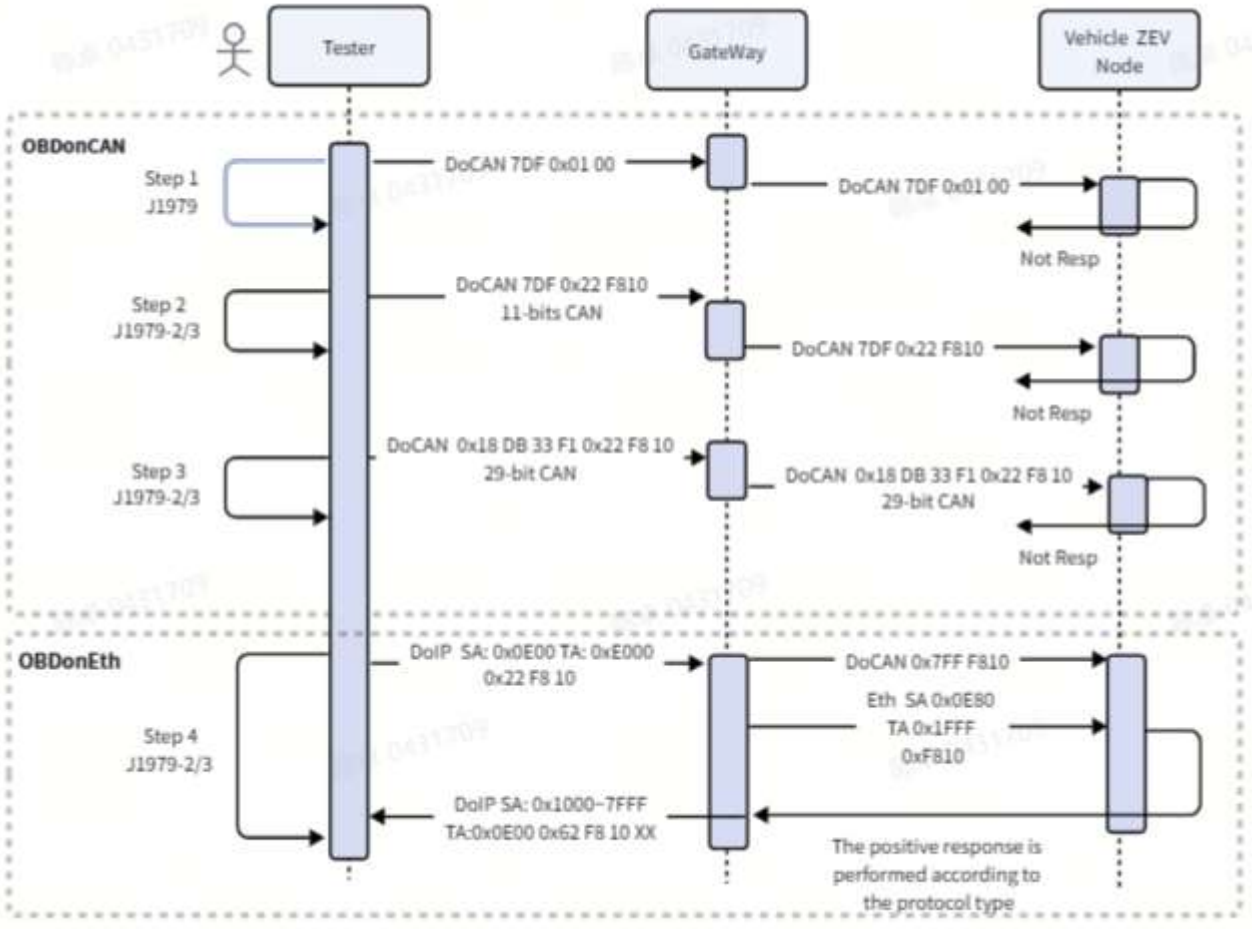
If cannot find the ECU from the above requirements, please contact the Basetech team .

1030394 v2 **Regulation link identification process requirment-ECU**

(1) => (2):
Description modified.

Current:Regulation link identification process requirment-ECU(2)

According to the 1979-3 regulations, external devices need to identify whether the vehicle's diagnostic link is CAN or Eth. The specific process is roughly as follows: the external device will broadcast different protocol addresses within the vehicle to determine the specific link. Currently, the Geely Research Institute specifies that the ZEV link should use Ethernet. The process is illustrated in the following diagram.



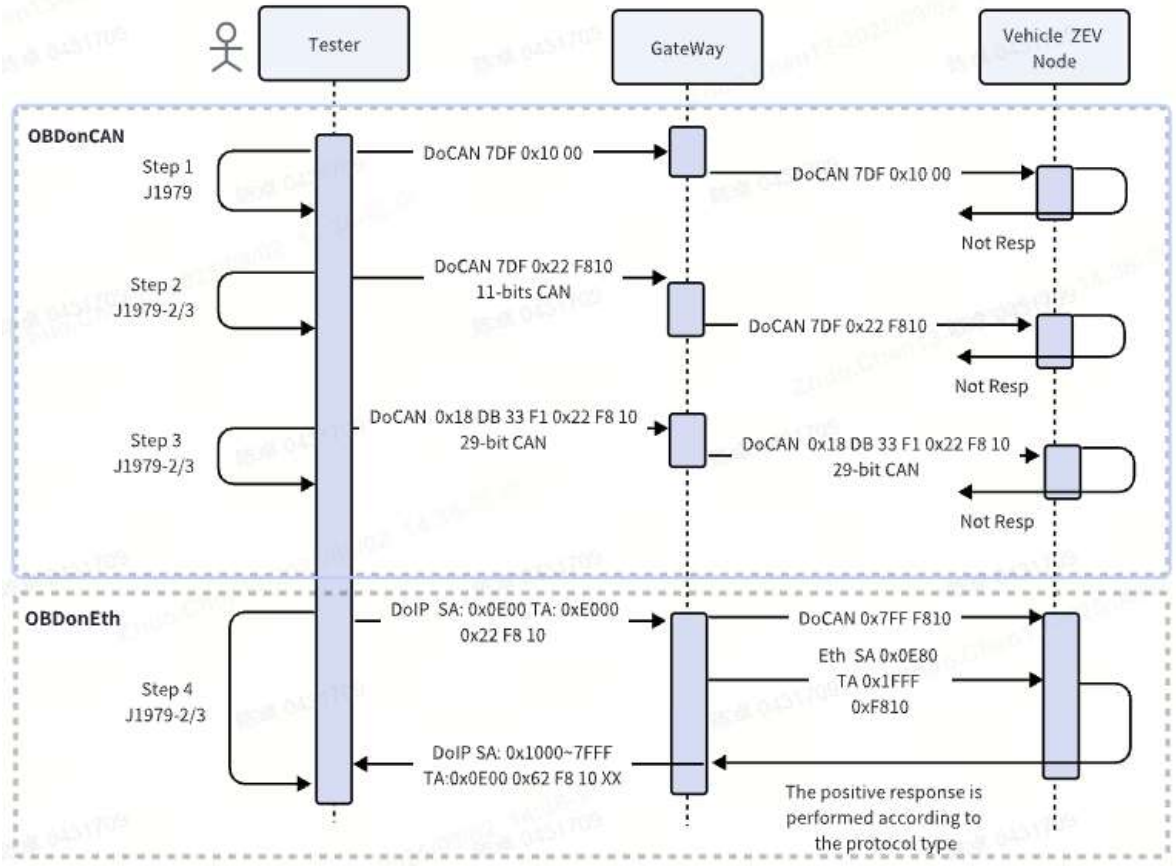
The diagnostic link mentioned above only supports one scenario on a vehicle that will not coexist.
The ZEV nodes involved cannot provide any positive or negative feedback in steps 1 to 3.

	Request message	Except ZEV ECU responds
Step1	0x7DF 0x01 00	No Respond
Step2	0x7DF 0xF8 10	No Respond
Step3	0x18 DB 33 F1 0x22 F8 10	No Respond
step4	CAN:0x7FF 0x22 F810	Positive Respond

	FR&Eth：SA 0x1FFF TA :0x0E80 0x22 F810	

Previous: Regulation link identification process requirment-ECU(1)

According to the 1979-3 regulations, external devices need to identify whether the vehicle's diagnostic link is CAN or Eth. The specific process is roughly as follows: the external device will broadcast different protocol addresses within the vehicle to determine the specific link. Currently, the Geely Research Institute specifies that the ZEV link should use Ethernet. The process is illustrated in the following diagram.



The diagnostic link mentioned above only supports one scenario on a vehicle that will not coexist.
The ZEV nodes involved cannot provide any positive or negative feedback in steps 1 to 3.

	Request message	Except ZEV ECU responds
Step1	0x7DF 0x10 00	No Respond
Step2	0x7DF 0xF8 10	No Respond
Step3	0x18 DB 33 F1 0x22 F8 10	No Respond
step4	CAN:0x7FF 0x22 F810 FR&Eth：SA 0x1FFF TA :0x0E80 0x22 F810	Positive Respond

1032778 v2 **Test Report Requirements**

(1) => (2):
Attribute 'Purpose' added: No preview available.
Description modified.

Current:Test Report Requirements(2)

It shall be possible to enable debug mode on WLAN transceiver to enable them to connect to WLAN testers to perform RF parameter testing.

ECUs with integrated Wi-Fi modules shall provide module-level Radio Frequency (RF) test reports and vehicle-level wireless performance assessment reports, and shall confirm the test results with the GRI Basetech team.

Previous: Test Report Requirements(1)

It shall be possible to enable debug mode on WLAN transceiver to enable them to connect to WLAN testers to perform RF parameter testing.

The ECU integrating the Wi-Fi module shall provide RF test reports at both component and whole-vehicle levels, and shall confirm the test results with the GRI Basetech team.

1032790 v2 WLAN Parameter Monitor

(1) => (2):
Attribute 'Verification Method' modified: Analysis AND Test => Analysis.

Current:WLAN Parameter Monitor(2)

At minimum, it shall be possible to monitor (online analysis):

- Used Frequency
- Used channel
- SSID (if connected)
- Data rate (Bitrate)
- MAC address of BSSID
- SNR
- PER
- Link Level (RSSI)
- Link Noise
- Link Quality for a specific BSS when connected
- Tx-power
- Missed beacon
- Discarded packets
- Security Credentials

The ECU that integrates the Wi-Fi module shall support logging the above parameters in a log file and shall confirm the log-reading method with the GRI Basetech team.

集成Wi-Fi模组的ECU需支持在日志文件中记录上述参数，并与GRI Basetech团队确认日志读取方式。

Previous: WLAN Parameter Monitor(1)

At minimum, it shall be possible to monitor (online analysis):

- Used Frequency
- Used channel
- SSID (if connected)

- Data rate (Bitrate)
- MAC address of BSSID
- SNR
- PER
- Link Level (RSSI)
- Link Noise
- Link Quality for a specific BSS when connected
- Tx-power
- Missed beacon
- Discarded packets
- Security Credentials

The ECU that integrates the Wi-Fi module shall support logging the above parameters in a log file and shall confirm the log-reading method with the GRI Basetech team.

集成Wi-Fi模组的ECU需支持在日志文件中记录上述参数，并与GRI Basetech团队确认日志读取方式。

1042955 v2 **Handling when internal connection resources are exhausted**

(1) => (2):
Attribute 'Verification Method' added: Test.
Attribute 'Purpose' added: No preview available.
Description modified.

Current:Handling when internal connection resources are exhausted(2)

When the number of existing connections in in-vehicle reaches the limit and there are no reusable connections, new request from internal and external clients shall be rejected after waiting for a timeout, and the HTTP status code 503 Service Unavailable shall be returned, indicating that the client should retry after a specified time.

Previous: Client Connection Management(1)

As a unified access point for external diagnostic client requests, SOVD vehicle gateway have the ability to control the number of connections and reject access. The specific requirements are as follows:

- The SOVD vehicle gateway shall maintain up to 5 authenticated client connections simultaneously. This restriction covers all external clients that have been authenticated.
- When the number of existing connections reaches the limit, new access requests shall be rejected and an HTTP status code 503 Service Unavailable shall be returned, prompting the client to try connecting again after the specified time.
- After successfully establishing a connection, SOVD vehicle gateway will use the Client ID carried in the Token as the unique identifier and is used for client identification, routing distribution, and response feedback in subsequent requests.
- All diagnostic requests shall carry the Client ID field in the HTTP request header. Requests that are not carried or have an invalid Client ID should be rejected (returning 401 Unauthorized or 403 Forbidden).

Note: The statistics of connection numbers are based on TCP/TLS connections that have been established and are active. Resources should be released promptly after connection closure or timeout.

1042963 v2 **Internal request forwarding queue mechanism**

(1) => (2):

Attribute 'Purpose' added: No preview available.
Description modified.

Current:Internal request forwarding queue mechanism(2)

To support orderly processing of concurrent requests from multiple clients and avoid overloading of target entities, SOVD Gateway shall implement a request queue management mechanism based on network segment isolation internally.

Previous: Internal request forwarding queue mechanism(1)

To support orderly processing of concurrent requests from multiple clients and avoid overloading of target entities, SOVD Gateway shall implement a request queue management mechanism based on network segment isolation internally.
SOVD Gateway shall retain the original Client ID when forwarding requests to ensure that the response can be accurately returned to the initiator. Each request in the queue needs to be bound with a Client ID and priority information for subsequent response association and scheduling optimization.

1043291 v2 As the server of mTLS connection

(1) => (2):
Attribute 'Purpose' added: No preview available.
Description modified.

Current:As the server of mTLS connection(2)

In the scenario of SOVD communication with external clients, the vehicle shall act as the server-side and shall have server related TLS conection capabilites. The public SOVD Server shall listen on port number 7690.

The SOVD client will actively initiate an mTLS connection to the server and successfully create a secure link.

Previous: As the server of mTLS connection(1)

In the scenario of SOVD communication with external clients, the vehicle shall act as the server-side and shall have server related TLS conection capabilites. The SOVD client will actively initiate an mTLS connection to the server and successfully create a secure link.

1043323 v2 Pre-established connection as the in-vehicle client

(1) => (2):
Attribute 'Purpose' added: No preview available.
Description modified.

Current:Pre-established connection as the in-vehicle client(2)

After ECU startup, SOVD Gateway shall attempt to establish at least one persistent TCP connection (with a dynamic source port and a destination port of 7690) with the SOVD server in advance.

If the establishment with the target server fails, such as the server not starting, the server rejecting, etc., the gateway shall try again within 500ms.

Note: the specific SOVD server depends on the project.

Previous: As the in-vehicle client(1)

When establishing a connection with an internal SOVD private service, the Public SOVD server shall create a TCP connection with a dynamic source port and a destination port of 7690.

1047368 v2 The number of connection to SOVD Client

(1) => (2):

Attribute 'Purpose' added: No preview available.
Description modified.

Current:The number of connection to SOVD Client(2)

In theory, there is no limit to the number of TCP connections established between SOVD Gateway and the same authentication SOVD Client. But Gateway shall Ensured that resources shall be released promptly after connections are closed or timeout.

Note: Connection between Client and Gateway counting is based on “established and active” TCP/TLS connections.

Previous: The number of connection to SOVD Client(1)

In theory, there is no limit to the number of TCP connections for the same authentication SOVD Client.

1047374 v2 The number of connections in the connection pool

(1) => (2):
Attribute 'Purpose' added: No preview available.
Attribute 'Verification Method' modified: Test => Analysis AND Test.
Description modified.

Current:The number of connections in the connection pool(2)

The connection pool shall be able to have at least 240 pairs of TCP connections simultaneously, with information such as the idle/occupied status of the connections.

Previous: The number of connection to in-vehicle SOVD server(1)

The SOVD vehicle gateway shall be able to simultaneously support initiating 10 TCP connections to the same SOVD private server.